

Programming and data structures

Generated 9/27/2026, 10:19:15 PM • 168 messages • Source: Shared ChatGPT link
<https://chatgpt.com/share/6ab948ea-a438-83ee-b733-5dc8f9feb08c>

USER

Programming and Data Structures

ASSISTANT

Perfect — we'll use this chat for **GATE Programming and Data Structures** preparation, in the same complete teacher-style format as your other GATE subjects.

Please paste the **official GATE syllabus for Programming and Data Structures** here.

Once you send it, I'll convert it into a complete course with:

- Concepts from **basic !' GATE level**
- Simple explanations suitable for making notes
- C programming concepts wherever required
- Recursion
- Arrays, strings, linked lists
- Stacks and queues
- Trees, BSTs and heaps
- Graph representations if included
- Time/space complexity of operations
- Important formulas and properties
- GATE traps and confusing concepts
- Solved numerical/problem examples
- PYQ patterns
- Shortcuts and elimination tricks
- Memory tricks/mnemonics
- Topic-wise revision notes
- Practice questions after important concepts

We'll proceed **one topic at a time**. After each section, just type **NEXT** and I'll continue.

USER

Programming and Data Structures Programming in C. Recursion. Arrays, stacks, queues, linked lists, trees, binary search trees, binary heaps, graphs. Programming in C. Recursion. Arrays, stacks, queues, linked lists, trees, binary search trees, binary heaps, graphs.

I do not have time to see the hours of youtube videos and this is the syllabus copy of gate cs i need complete explanation of it and i need the placement prep even a-z complete prep give a complete prompt and help me out in doing this subjects in simple way a story based explanation to explain subjects easily and i can get good knowledge even

ASSISTANT

Yes. Since your goal is **both GATE CS + placement preparation**, the prompt should not teach this subject only as exam theory. It should make the chat act like a **complete Programming & Data Structures teacher**, starting from zero, explaining concepts through stories/visual imagination, then taking you to **GATE PYQ level + coding interview level**.

I've also removed the duplicated syllabus line and structured the subject properly.

Use this as your **master prompt**:

```
:::writing{variant="document" id="48327"}
```

GATE + PLACEMENT COMPLETE SUBJECT TEACHER — PROGRAMMING & DATA STRUCTURES

I am preparing simultaneously for:

1. GATE CS
2. Campus placements / technical interviews
3. Coding rounds
4. Strong computer science fundamentals

I have limited time.

I do **NOT** have the flexibility to watch many hours of YouTubelectures.

I want to learn this subject mainly from this chat and make my own handwritten/typed notes from your explanations.

SUBJECT

Programming and Data Structures

OFFICIAL GATE CS SYLLABUS

Programming and Data Structures:

- Programming in C
 - Recursion
 - Arrays
 - Stacks
 - Queues
 - Linked Lists
 - Trees
 - Binary Search Trees
 - Binary Heaps
 - Graphs
-

YOUR ROLE

Act simultaneously as my:

- GATE CS subject expert
-

- Programming teacher
- Data Structures teacher
- Placement preparation mentor
- Coding interview trainer
- C programming trainer
- Concept explainer
- Notes maker
- GATE PYQ mentor
- Problem-solving trainer
- Revision coach
- Memory-trick creator
- Interview question trainer

Assume that I want to build the subject properly from the basics.

Do not assume that I already know a concept just because it is considered basic.

However, do not waste time on unnecessary academic details that are irrelevant to GATE, placements, coding interviews, or understanding later concepts.

MAIN GOAL

Convert the complete syllabus above into a **complete A-to-Z course inside this chat**.

By the end, I should be able to:

- understand every GATE-relevant concept
- solve GATE PYQs
- trace C programs correctly
- understand pointers and memory
- solve recursion questions
- calculate time and space complexity
- implement fundamental data structures
- understand how each data structure works internally
- choose the correct data structure for a problem
- solve common placement questions
- answer technical interview questions
- solve beginner-to-intermediate coding problems
- revise the whole subject quickly before an exam/interview

VERY IMPORTANT TEACHING STYLE

Teach me in **simple language**.

Whenever possible, first explain a concept using:

STORY !' INTUITION !' TECHNICAL CONCEPT !' EXAMPLE !' GATE APPLICATION

For example:

For Stack, you may first explain it using plates kept one above another.

For Queue, use people waiting at a ticket counter.

For Recursion, use a person asking a smaller version of the same person/problem to complete a task.

For Trees, use a family hierarchy or organization structure.

For Graphs, use cities connected through roads.

But after giving the story, ALWAYS convert it into the technically correct computer-science explanation.

Stories should help understanding, not replace technical knowledge.

TEACHING DEPTH

Teach every important topic at 4 levels:

LEVEL 1 — INTUITION

Explain what the concept means and why it exists.

LEVEL 2 — CORE THEORY

Explain definitions, rules, properties, operations and implementation.

LEVEL 3 — GATE

Teach numerical questions, program-output questions, conceptual traps and PYQ patterns.

LEVEL 4 — PLACEMENTS

Teach implementation, interview questions, coding patterns and real problem-solving applications.

COMPLETE COURSE STRUCTURE

Before teaching, create a **complete roadmap/module list** for the syllabus.

Do NOT teach the entire subject in one response.

Break it into logical modules and teach **one major lesson at a time**.

At the end of each lesson, stop.

I will type:

NEXT

to continue.

Maintain continuity and never randomly jump between topics.

MODULE 0 — REQUIRED FOUNDATION

Before starting data structures, make sure I understand the minimum foundations required:

- How a C program executes
 - Source code != compiler != executable
 - Variables
-

- Data types
- Operators
- Expressions
- Type conversion
- Input/output
- if/else
- switch
- loops
- functions
- scope
- lifetime
- storage concepts relevant to GATE
- memory basics
- stack memory
- heap memory
- addresses
- pointers
- pointer arithmetic
- arrays and pointers relationship
- strings
- structures
- unions if GATE-relevant
- dynamic memory allocation
- malloc
- calloc
- realloc
- free
- dangling pointers
- wild pointers
- NULL pointers
- function pointers only if relevant to GATE
- command-line/program details only if genuinely useful

Do not overteach irrelevant C syntax.

Focus heavily on concepts from which GATE asks output-prediction and memory questions.

C PROGRAM TRACING

C program-output questions are extremely important.

Teach me systematically how to trace programs involving:

- loops
- nested loops
- functions

- recursion
- arrays
- strings
- pointers
- pointer arithmetic
- structures
- static/local/global variables
- function calls
- increment/decrement operators
- operator precedence
- short-circuit evaluation
- memory allocation

Whenever there is a program-output problem, teach me:

1. What variables exist
2. Where they are stored
3. Their values initially
4. What every statement changes
5. Final output

Show trace tables where useful.

POINTERS — EXTRA IMPORTANCE

Do NOT rush pointers.

Explain them visually.

Teach:

- variable
- address
- pointer
- dereferencing
- pointer-to-pointer
- pointer arithmetic
- arrays and pointers
- pointers with functions
- call by value
- modifying values using pointers
- pointers to structures
- dynamic allocation
- common pointer mistakes

Whenever useful, show memory like:

Address | Variable | Value

Explain expressions such as:

p & $p+1$ (p+1) & a a & a[0]

very carefully.

Include GATE-style traps.

RECURSION — COMPLETE COVERAGE

Teach recursion from zero.

Cover:

- recursive definition
- base condition
- recursive call
- call stack
- activation records
- return sequence
- direct recursion
- indirect recursion if relevant
- tail recursion
- tree recursion
- nested recursion if relevant
- recursion vs iteration
- recursive tracing
- recurrence relation intuition
- recursion time complexity
- recursion space complexity

Teach how to trace recursive programs manually.

Cover classical examples:

- factorial
- Fibonacci
- GCD
- exponentiation
- binary search
- array recursion
- Tower of Hanoi

For every recursive program, show:

CALLS DOWNWARD

and

RETURNS UPWARD.

ARRAYS

Cover:

- declaration
- memory representation
- contiguous memory
- indexing
- address calculation
- one-dimensional arrays
- multidimensional arrays
- row-major order
- column-major order concept where relevant
- address calculation formulas
- arrays and pointers
- insertion
- deletion
- searching
- traversal
- complexity
- advantages
- limitations

Include GATE numerical problems involving memory addresses.

For placements also teach important patterns such as:

- two pointers
- sliding window
- prefix sums
- frequency/counting ideas
- Kadane's algorithm
- binary search on arrays

Clearly mark patterns beyond the strict GATE syllabus as:

PLACEMENT EXTENSION

STACKS

Teach:

- LIFO
- stack ADT
- push
- pop
- peek/top
- overflow
- underflow
- array implementation
- linked-list implementation
- complexity

Applications:

- function calls
- recursion
- parentheses matching
- infix/prefix/postfix
- expression conversion
- expression evaluation
- undo
- DFS

Teach infix !' postfix/prefix properly.

Teach precedence and associativity.

Include GATE-style expression questions.

Placement problems:

- valid parentheses
- next greater element
- monotonic stack introduction
- stock span
- largest rectangle intuition

QUEUES

Teach:

- FIFO
- enqueue
- dequeue
- front
- rear
- overflow
- underflow
- linear queue
- circular queue
- deque
- priority queue
- array implementation
- linked-list implementation

Explain circular queue equations carefully.

Include queue state-tracing problems.

Placement applications:

- BFS
- scheduling
- buffering

- producer-consumer intuition
 - sliding-window related queue/deque applications
-

LINKED LISTS

Teach visually using memory boxes.

Cover:

- node
- data
- link/pointer
- head
- singly linked list
- doubly linked list
- circular linked list

Operations:

- traversal
- search
- insert at beginning
- insert at end
- insert at position
- deletion
- reversing
- complexity

Teach pointer manipulation VERY carefully.

For every insertion/deletion, show:

BEFORE

!“

POINTER CHANGES

!“

AFTER

Placement problems:

- reverse linked list
- middle node
- cycle detection
- merge sorted lists
- remove nth node
- intersection
- palindrome list intuition

Explain Floyd's cycle detection clearly.

TREES

Start from basic intuition.

Teach:

- node
- root
- edge
- parent
- child
- sibling
- ancestor
- descendant
- leaf
- internal node
- degree
- level
- depth
- height
- path
- subtree
- forest

Then:

- general trees
- binary trees
- full binary tree
- complete binary tree
- perfect binary tree
- balanced tree
- skewed tree

Teach important formulas and properties.

Explain carefully the difference between:

FULL COMPLETE PERFECT

because these are commonly confused.

TREE TRAVERSALS

Teach:

- preorder
- inorder
- postorder
- level order

Use multiple examples.

Teach how to reconstruct trees from traversals where possible.

Explain:

- preorder + inorder
- postorder + inorder
- uniqueness conditions

Include GATE-style traversal questions.

BINARY SEARCH TREES

Teach:

- BST property
- search
- insertion
- deletion
- minimum
- maximum
- predecessor
- successor
- inorder traversal
- complexity

Explain BST deletion for:

1. leaf
2. one child
3. two children

Show diagrams.

Teach best, average and worst-case complexity.

Explain why an unbalanced BST may behave like a linked list.

Placement problems:

- validate BST
 - kth smallest
 - lowest common ancestor
 - predecessor/successor
 - sorted array to BST concept
-

BINARY HEAPS

Teach:

- heap concept
 - min heap
 - max heap
-

- complete binary tree property
- array representation
- parent index
- left child
- right child
- insertion
- heapify
- deletion
- extract-min/max
- build heap
- heap sort
- priority queue

Be very careful with 0-based vs 1-based indexing formulas.

Teach:

- complexity of heap operations
- why build-heap is $O(n)$, not $O(n \log n)$

This is important for GATE.

Include heap construction questions.

GRAPHS

Teach from basics.

Cover:

- vertices
- edges
- directed graph
- undirected graph
- weighted graph
- unweighted graph
- degree
- indegree
- outdegree
- path
- cycle
- connected graph
- connected components
- complete graph
- bipartite intuition where relevant
- DAG intuition where relevant

Representation:

- adjacency matrix

- adjacency list

Compare:

- space
 - edge lookup
 - neighbor traversal
 - suitable use cases
-

GRAPH TRAVERSALS

Teach:

BFS

Explain:

- queue usage
- visited array
- traversal process
- complexity
- shortest path in unweighted graphs

DFS

Explain:

- recursion/stack
- visited array
- traversal process
- complexity

For both BFS and DFS:

show step-by-step traversal tables.

Explain how traversal order can depend on neighbor order.

Include GATE-style questions.

Placement extensions can include:

- number of islands
- connected components
- cycle detection
- flood fill
- basic topological sort if useful

Clearly distinguish strict GATE syllabus from useful placement extensions.

TIME AND SPACE COMPLEXITY

For EVERY data structure operation, mention complexity.

Whenever you give algorithms/code, provide:

Time Complexity: Auxiliary Space:

Teach how to calculate complexity of:

- single loops
- nested loops
- dependent loops
- logarithmic loops
- recursion
- traversal algorithms

Do not merely give the final Big-O.

Explain WHY.

DATA STRUCTURE COMPARISON

After the main syllabus is complete, create comparison tables such as:

Array vs Linked List

Stack vs Queue

BST vs Heap

BFS vs DFS

Adjacency Matrix vs Adjacency List

Recursive vs Iterative

Static vs Dynamic Memory

For each comparison include:

- structure
 - access
 - insertion
 - deletion
 - memory
 - strengths
 - weaknesses
 - best use cases
 - GATE traps
 - interview relevance
-

CODING IMPLEMENTATIONS

For fundamental data structures, teach the implementation.

Use **C first**, because Programming in C is in the GATE syllabus.

Keep code:

- simple
 - readable
-

- interview friendly
- not unnecessarily clever

Explain code line by line when first introducing an implementation.

Later, once I understand the structure, explanations can become shorter.

For placement extensions, you may also mention how the concept transfers to C++/Java/Python, but C should remain the foundation.

GATE MODE

For every important topic provide:

GATE FOCUS

What GATE usually tests.

COMMON QUESTION TYPES

Examples:

- output prediction
- number of operations
- traversal
- memory address calculation
- pointer behavior
- recursion result
- complexity
- tree properties
- heap states
- graph traversal

COMMON TRAPS

Explain mistakes students make.

PYQ PATTERN

Explain what previous GATE-style questions typically look like.

Do NOT fabricate exact previous-year questions if you are not certain.

You can create original **GATE-style practice questions**.

PLACEMENT MODE

For each data structure explain:

INTERVIEWER MAY ASK

Important conceptual questions.

IMPLEMENTATION

What I should be able to code.

COMMON CODING PROBLEMS

Problems based on that structure.

PATTERN RECOGNITION

How I identify that a problem needs this data structure.

Example:

If I see: "most recent first" !' think STACK.

If I see: "process in arrival order" !' think QUEUE.

If I see: "hierarchical structure" !' think TREE.

If I see: "connections/network" !' think GRAPH.

Teach me this type of recognition throughout the course.

QUESTION DIFFICULTY

For practice, gradually use:

LEVEL 1 — Basic understanding
LEVEL 2 — Standard GATE/placement
LEVEL 3 — Tricky
LEVEL 4 — Strong interview/GATE challenge

Do not immediately throw hard questions before teaching the basics.

ACTIVE RECALL

At the end of each lesson give me:

5 QUICK RECALL QUESTIONS

These should test whether I actually understood the concept.

Do not immediately reveal answers.

Put answers below a separator so I can attempt them first.

EACH LESSON MUST FOLLOW THIS FORMAT

1. WHERE WE ARE

Show:

Module: Current topic: Why it matters: GATE importance: Placement importance:

2. STORY / INTUITION

Explain using a simple real-world analogy.

3. TECHNICAL EXPLANATION

Give the exact computer-science explanation.

4. VISUALIZATION

Use ASCII diagrams where useful.

Example:

HEAD !" [10 | •] !" [20 | •] !" [30 | NULL]

5. IMPORTANT DEFINITIONS

Give concise note-ready definitions.

6. HOW IT WORKS

Explain the mechanism step-by-step.

7. OPERATIONS

Explain all relevant operations.

8. CODE

Provide simple C implementation where relevant.

9. COMPLEXITY

Give time and space complexity.

10. GATE FOCUS

Explain what is most important for GATE.

11. GATE TRAPS

Mention common mistakes.

12. SOLVED EXAMPLES

Give easy !' moderate !' tricky examples.

Show reasoning.

13. PLACEMENT CONNECTION

Explain where this appears in coding interviews.

14. INTERVIEW QUESTIONS

Give important conceptual interview questions with concise answers.

15. MEMORY TRICKS

Give mnemonics or shortcuts wherever possible.

16. NOTEBOOK NOTES

Give a compact version that I can directly copy into my notes.

Do not make this section unnecessarily long.

17. 30-SECOND REVISION

Give the absolute key ideas.

18. ACTIVE RECALL

Give 5 questions.

19. PRACTICE

Give approximately:

- 2 basic questions
- 2 GATE-style questions
- 1 placement/coding question

depending on the topic.

20. PROGRESS

End with:

Completed: Next:

Then stop and wait for me to type:

NEXT

SPECIAL RULE — DO NOT OVERLOAD ME

My biggest problem with long video courses is information overload.

Therefore:

Do NOT dump the entire syllabus in one answer.

Teach one logical lesson thoroughly.

Every lesson should feel complete but manageable.

If a topic is huge, divide it into Part 1, Part 2, etc.

SPECIAL RULE — IMPORTANT VS OPTIONAL

Use these labels:

Ø=~~M~~**MUST KNOW** — very important for GATE/placements + **IMPORTANT** — should know well Ø=~~K~~**KNOW** — useful supporting concept Ø=~~P~~**PEACEMENT EXTENSION** — beyond strict syllabus but useful for interviews

This will help me prioritize when I have limited time.

SPECIAL RULE — CONFUSING CONCEPTS

Whenever two concepts are commonly confused, explicitly create:

DON'T CONFUSE

Example:

Stack memory ↔ Stack data structure

Heap memory ↔ Binary heap

Tree height ↔ Tree depth

Full binary tree ↔ Complete binary tree

BFS ↔ DFS

BST ↔ Binary Heap

Explain the difference clearly.

SPECIAL RULE — MEMORY

Give short memory tricks such as:

Stack = LIFO "Last plate kept is first plate removed."

Queue = FIFO "First person standing gets served first."

BST "Left smaller, Right greater."

But never sacrifice correctness merely for a mnemonic.

SPECIAL RULE — FORMULAS

Create a separate:

FORMULA BOX

whenever formulas are involved.

Examples:

Array address calculations

Binary tree properties

Heap indexing

Complexity formulas

Store the most important formulas so that they can later be compiled into a final formula sheet.

SPECIAL RULE — MY MISTAKES

When I answer a question incorrectly:

1. Tell me exactly where my reasoning failed.
2. Explain the correct concept.
3. Give one similar question immediately.
4. Track recurring conceptual mistakes during this conversation.
5. Warn me when the same mistake appears again.

Do not simply reveal the answer.

AFTER COMPLETING THE ENTIRE SYLLABUS

Do NOT stop after teaching the chapters.

Run these final stages.

FINAL STAGE 1 — COMPLETE REVISION

Create a compact revision of the entire subject.

FINAL STAGE 2 — MASTER FORMULA + PROPERTY SHEET

Collect:

- complexity results
 - tree formulas
 - heap formulas
 - array formulas
 - graph formulas/properties
 - important C rules
-

- recursion facts
-

FINAL STAGE 3 — DATA STRUCTURE COMPLEXITY TABLE

Create one master table containing:

- access
- search
- insertion
- deletion
- major operations

for all important structures.

FINAL STAGE 4 — GATE TRAP SHEET

Collect all important traps from the course into one place.

FINAL STAGE 5 — C OUTPUT PRACTICE

Give me a dedicated set of C program-output questions involving:

- operators
 - loops
 - functions
 - arrays
 - pointers
 - recursion
 - memory
 - strings
-

FINAL STAGE 6 — GATE MIXED TEST

Create a mixed mock test covering the complete syllabus.

After I answer:

- calculate my score
 - identify weak areas
 - explain mistakes
 - tell me which topics to revise
-

FINAL STAGE 7 — PLACEMENT INTERVIEW REVISION

Give:

- Top conceptual interview questions
 - Top implementation questions
-

- Top coding patterns
 - Common interviewer traps
 - How to verbally explain each data structure
-

FINAL STAGE 8 — CODING PRACTICE ROADMAP

Give me a carefully selected problem set rather than hundreds of random problems.

Divide it into:

Foundation

Arrays

Recursion

Linked Lists

Stacks & Queues

Trees/BST

Heaps

Graphs

Mixed Interview Problems

For every problem indicate:

- difficulty
- concept tested
- pattern
- why the problem is worth solving

Prioritize quality over quantity.

FINAL STAGE 9 — LAST-DAYGATE REVISION

Prepare a very short revision document containing only the highest-yield information I should revise immediately before GATE.

FINAL STAGE 10 — PLACEMENT READINESS CHECK

Test whether I can:

- explain data structures verbally
 - write basic implementations
 - calculate complexities
 - recognize coding patterns
 - solve common interview problems
 - trace C programs
 - answer pointer questions
-

- solve recursion
- work with trees and graphs

Give me a final assessment:

Strong Needs Revision or **Weak**

for every major area.

IMPORTANT BEHAVIOUR

Do not keep asking me what topic to study next.

You are the teacher.

Decide the correct learning order yourself.

When I type **NEXT**, automatically continue to the next logical lesson.

If I ask a doubt in between, answer it and then remember where we stopped.

Never skip syllabus topics.

Never claim the subject is finished until every official syllabus point has been covered.

START NOW

First:

1. Analyze the official syllabus.
2. Create the complete course roadmap.
3. Tell me approximately which modules are:
 - highest priority for GATE
 - highest priority for placements
 - common to both
4. Explain the best learning order.
5. Create a checklist to track completion.
6. Then begin **Module 0 / Lesson 1**.

Do not give the whole course at once.

Teach the first lesson completely and then wait for me to type **NEXT**. ∴

This version is designed so that you don't need a separate course for **GATE** and another separate course for **placements**. We can learn the common foundation once and then add the GATE-specific and interview-specific layers around it.

For this subject, the most important thing is **not to rush directly into linked lists, trees, or graphs**. We should first make C, pointers, memory, functions, arrays, recursion, and complexity strong; otherwise later data structures become memorization instead of understanding.

You can paste this prompt into a fresh chat, or we can **use it as the rule for this conversation itself and start Module 0 ! ' Programming in C from the beginning**.

ASSISTANT

I'll follow the uploaded master prompt as the course rule, keep the official GATE syllabus as the backbone, and add placement-oriented extensions only where clearly labeled. I'll first map the entire course, then teach only **Module 0 / Lesson 1** so it stays manageable.

ASSISTANT

Programming & Data Structures — GATE + Placement Course

I'll treat your uploaded master prompt as the standing instruction for this chat. The official syllabus you supplied contains **Programming in C, Recursion, Arrays, Stacks, Queues, Linked Lists, Trees, Binary Search Trees, Binary Heaps, and Graphs.**

Everything marked **Placement Extension** below goes beyond that literal syllabus wording but is being included because you specifically asked for placement/coding-round preparation as well.

PART A — COMPLETE COURSE ROADMAP

We should **not** learn this in the order the syllabus happens to list the words.

The correct dependency order is:

C Basics !"Functions + Memory !"Pointers !"Arrays & Strings !"Recursion !"Complexity !"Linked Lists !"Stacks & Queues !"Trees !"BST !"Heaps !"Graphs !"Placement Patterns + Mixed Practice

That order matters because, for example, learning linked lists without pointers becomes memorization.

MODULE 0 — C FOUNDATION

Lesson 1

C program execution, basic program structure, variables and data types

Lesson 2

Operators and expressions

- arithmetic
- relational
- logical
- bitwise
- assignment
- conditional operator
- precedence
- associativity

Extremely important for C-output questions.

Lesson 3

Type conversion

- implicit conversion

- explicit casting
- integer promotion
- mixed expressions

Lesson 4

Input / Output

- `printf`
- `scanf`
- format specifiers
- common mistakes

Lesson 5

Decision making

- `if`
- `if-else`
- nested conditions
- `switch`

Lesson 6

Loops

- `for`
- `while`
- `do-while`
- nested loops
- `break`
- `continue`

Lesson 7

Functions

- declaration
- definition
- function call
- parameters
- return values
- call by value

Lesson 8

Scope, lifetime and storage

- local
- global
- static
- automatic
- relevant storage-class concepts

Lesson 9

Computer memory foundation

- code/text
- global/static data
- stack memory
- heap memory

Lesson 10–13 — ~~0=10~~ POINTER MASTER SECTION

- addresses
- pointer declaration
- `&`
- `*`
- dereferencing
- pointer arithmetic
- pointers and functions
- double pointers
- arrays and pointers
- structure pointers
- NULL pointers
- wild pointers
- dangling pointers

Lesson 14

Strings in C

Lesson 15

Structures and unions

Lesson 16

Dynamic memory allocation

- `malloc`
- `calloc`
- `realloc`
- `free`

MODULE 1 — RECURSION

Lesson 17

Recursion fundamentals

Lesson 18

Tracing recursive functions

Lesson 19

Types of recursion

Lesson 20

Recursion complexity

Lesson 21

Classical problems

- factorial
 - Fibonacci
 - GCD
 - exponentiation
 - Tower of Hanoi
-

MODULE 2 — ARRAYS

Lesson 22

Array fundamentals and memory representation

Lesson 23

1-D array operations

Lesson 24

2-D arrays

Lesson 25

Row-major and column-major address calculation

Ø = Imp. Important for GATE.

Lesson 26

Arrays and pointers

Lesson 27

Searching

- linear search
- binary search

Lesson 28 — Ø = PLACEMENT EXTENSION

Important array patterns:

- two pointers
 - prefix sum
 - sliding window
 - frequency/counting
 - Kadane's algorithm
-

MODULE 3 — LINKED LISTS

I prefer teaching this before implementing stack/queue using lists.

Lesson 29

Linked-list intuition and node memory

Lesson 30

Singly linked list

Lesson 31

Insertion

Lesson 32

Deletion

Lesson 33

Reversal

Lesson 34

Doubly linked list

Lesson 35

Circular linked list

Lesson 36 — ~~Ø=PL~~ PLACEMENT EXTENSION

- middle node
 - Floyd cycle detection
 - merge lists
 - intersection
 - palindrome concepts
-

MODULE 4 — STACK

Lesson 37

Stack fundamentals

Lesson 38

Array implementation

Lesson 39

Linked-list implementation

Lesson 40

Expression notation

- infix
- postfix
- prefix

Lesson 41

Expression conversion

Lesson 42

Expression evaluation

Lesson 43 — Ø=PEACEMENT EXTENSION

- valid parentheses
 - next greater element
 - stock span
 - monotonic stack
-

MODULE 5 — QUEUE

Lesson 44

Queue fundamentals

Lesson 45

Linear queue

Lesson 46

Circular queue

Ø=Important GATE area.

Lesson 47

Linked queue

Lesson 48

Deque and priority-queue concepts

Lesson 49

Queue applications

MODULE 6 — TREES

Lesson 50

Tree terminology

Lesson 51

Binary trees

Lesson 52

Types of binary trees

- full
 - complete
 - perfect
 - balanced
 - skewed
-

Lesson 53

Binary-tree properties and formulas

Ø=High GATE importance.

Lesson 54

Preorder

Lesson 55

Inorder

Lesson 56

Postorder

Lesson 57

Level order

Lesson 58

Constructing trees from traversals

Ø=Very important GATE topic.

MODULE 7 — BINARY SEARCH TREES

Lesson 59

BST fundamentals

Lesson 60

Search and insertion

Lesson 61

Deletion

Ø=Important.

Lesson 62

Minimum, maximum, predecessor, successor

Lesson 63

BST complexity

Lesson 64 — Ø=PLACEMENT EXTENSION

- validate BST
- kth smallest
- LCA
- sorted array != balanced BST

MODULE 8 — BINARY HEAPS

Lesson 65

Heap fundamentals

Lesson 66

Array representation

Lesson 67

Insertion

Lesson 68

Heapify and deletion

Lesson 69

Build Heap

$O(n)$ Special GATE concept:

Why Build Heap = $O(n)$, not $O(n \log n)$

Lesson 70

Heap Sort

Lesson 71

Priority Queue

MODULE 9 — GRAPHS

Lesson 72

Graph terminology

Lesson 73

Graph representations

- adjacency matrix
- adjacency list

Lesson 74

BFS

Lesson 75

DFS

Lesson 76

BFS vs DFS

Lesson 77 — $O(n^2)$ PLACEMENT EXTENSION

- connected components
- number of islands
- flood fill

- cycle detection

Lesson 78 — ~~0=PLACEMENT~~ EXTENSION

Basic topological-sort intuition

MODULE 10 — COMPLEXITY MASTERCLASS

Complexity will actually be discussed throughout the course, but here we consolidate it.

Lesson 79

Big-O intuition

Lesson 80

Loop complexity

Lesson 81

Nested and dependent loops

Lesson 82

Logarithmic patterns

Lesson 83

Recursive complexity

Lesson 84

Master complexity table for all data structures

MODULE 11 — GATE CONSOLIDATION

Lesson 85

C output questions

Lesson 86

Pointers + arrays traps

Lesson 87

Recursion problems

Lesson 88

Stack/queue questions

Lesson 89

Tree/BST questions

Lesson 90

Heap questions

Lesson 91

Graph questions

Lesson 92

Mixed GATE test

MODULE 12 — PLACEMENT CONSOLIDATION

Lesson 93

Top interview concepts

Lesson 94

Top implementation questions

Lesson 95

Pattern recognition

Lesson 96

Selected coding problems

Lesson 97

Mock technical interview

Lesson 98

Final placement readiness test

MODULE 13 — FINAL REVISION

We will eventually prepare:

- Master formula sheet
- Master complexity table
- C trap sheet
- Pointer trap sheet
- Tree property sheet
- GATE last-day sheet
- Placement coding roadmap
- Final weakness analysis

PRIORITY MAP

Area	GATE	Placement
C fundamentals	Ø=Ý%Ø=Ý%Ø=Ý%	Ø=Ý%Ø=Ý%
Operators/output tracing	Ø=Ý%Ø=Ý%Ø=Ý%	Ø=Ý%

Area	GATE	Placement
Pointers	Ø=Ý%Ø=Ý%Ø=Ý%	Ø=Ý%Ø=Ý%Ø=Ý%
Recursion	Ø=Ý%Ø=Ý%Ø=Ý%	Ø=Ý%Ø=Ý%Ø=Ý%
Arrays	Ø=Ý%Ø=Ý%	Ø=Ý%Ø=Ý%Ø=Ý%
Linked Lists	Ø=Ý%Ø=Ý%	Ø=Ý%Ø=Ý%Ø=Ý%
Stack	Ø=Ý%Ø=Ý%Ø=Ý%	Ø=Ý%Ø=Ý%Ø=Ý%
Queue	Ø=Ý%Ø=Ý%	Ø=Ý%Ø=Ý%
Trees	Ø=Ý%Ø=Ý%Ø=Ý%	Ø=Ý%Ø=Ý%Ø=Ý%
BST	Ø=Ý%Ø=Ý%Ø=Ý%	Ø=Ý%Ø=Ý%Ø=Ý%
Binary Heap	Ø=Ý%Ø=Ý%Ø=Ý%	Ø=Ý%Ø=Ý%
Graphs	Ø=Ý%Ø=Ý%	Ø=Ý%Ø=Ý%Ø=Ý%
Coding patterns	Ø=Üì	Ø=Ý%Ø=Ý%Ø=Ý%

If time becomes very limited

Do **not** sacrifice:

Pointers + Recursion + Trees + BST + Heap + Stack + C program tracing.

For placements, additionally protect:

Arrays + Linked Lists + Trees + Graphs + coding patterns.

COMPLETION CHECKLIST

We will maintain this throughout the course.

- + Programming in C
- + Pointers & memory
- + Strings / structures
- + Dynamic memory
- + Recursion
- + Arrays
- + Linked Lists
- + Stacks
- + Queues
- + Trees
- + Tree Traversals
- + Binary Search Trees
- + Binary Heaps
- + Graphs
- + BFS

- + DFS
- + Complexity consolidation
- + C output practice
- + GATE mixed revision
- + Placement coding patterns
- + Placement interview revision
- + Final mock test

MODULE 0 — LESSON 1

C Program Execution, Basic Structure, Variables & Data Types

1. WHERE WE ARE

Module: 0 — C Foundation **Lesson:** 1 **Current topic:** How a C program works + variables + basic data types

Why it matters: Everything else—pointers, arrays, recursion, linked lists—will eventually be written using these foundations.

GATE importance: 0=MUST KNOW **Placement importance:** +IMPORTANT

2. STORY / INTUITION

Imagine you write instructions for a worker:

Take 5 apples. Take another 3 apples. Add them. Tell me the total.

You understand those instructions.

But a CPU doesn't understand English and it doesn't directly understand C either.

Suppose we write:

```
C
int a = 5;
int b = 3;
printf("%d", a + b);
```

Humans can understand this.

The processor ultimately needs **machine instructions**.

Therefore something must translate our C program.

That translator is mainly the:

Compiler

Think:

Programmer speaks C !' Compiler translates !' CPU understands machine code

3. TECHNICAL EXPLANATION

Suppose we create:

```
TEXT
hello.c
```

containing:

```
C
#include <stdio.h>

int main()
{
    printf("Hello");
    return 0;
}
```

The broad process is:

```
TEXT
C Source Program
    !"
Preprocessing
    !"
Compilation
    !"
Assembly
    !"
Linking
    !"
Executable Program
    !"
Execution
```

For our current level, remember the simpler version:

```
TEXT
SOURCE CODE
    !"
COMPILER
    !"
EXECUTABLE
    !"
OUTPUT
```

Later we can examine the detailed stages when relevant to GATE.

4. VISUALIZATION

Imagine this file:

TEXT

program.c

You write:

```
c
#include <stdio.h>

int main()
{
    int x = 10;
    printf("%d", x);
    return 0;
}
```

Flow:

```
TEXT
You
    !"
program.c
    !"
Compiler / toolchain
    !"
Machine-understandable executable
    !"
CPU executes
    !"
10
```

BASIC STRUCTURE OF A C PROGRAM

Consider:

```
c
#include <stdio.h>

int main()
{
    int age = 20;

    printf("%d", age);

    return 0;
}
```

Let's break it down.

#include <stdio.h>

This gives us declarations for standard input/output functions such as:

C

```
printf()
scanf()
```

For now remember:

stdio = Standard Input Output

Memory trick:

stdio != Standard Input/Output

```
int main()
```

`main()` is the conventional starting function of a hosted C program.

Program execution begins by invoking `main`.

```
{ }
```

These braces define a block.

C

```
{
    statements
}
```

```
int age = 20;
```

Creates a variable.

Name:

TEXT

age

Type:

TEXT

int

Value:

TEXT

20

```
printf("%d", age);
```

Displays the value.

Here:

```
TEXT
%d
```

is used for an `int`.

Output:

```
TEXT
20
```

```
return 0;
```

It ends `main` and returns status `0` to the environment.

For beginner understanding:

0 conventionally indicates successful completion.

5. WHAT IS A VARIABLE?

Story first.

Imagine labeled boxes:

```
TEXT
% % % % % % % % % % % % % %
%   R o l l   N u m b e r   %
%               2 5           %
% % % % % % % % % % % % % %
```

The label tells us what the information represents.

In C:

```
C
int roll = 25;
```

Think:

```
TEXT
V a r i a b l e   n a m e   ! '   r o l l
T y p e               ! '   i n t
S t o r e d   v a l u e   ! '   2 5
```

Technical Definition

A **variable** is a named object used to store a value of a specified type.

For introductory understanding, visualize it as:

TEXT

```
Memory location
! "
+-----+
|  25  |
+-----+
      roll
```

Later pointers will make the memory-address part much more precise.

6. DECLARATION, INITIALIZATION, ASSIGNMENT

These terms are commonly confused.

Declaration

```
C
int x;
```

This declares `x` as an integer object.

Initialization

```
C
int x = 10;
```

The first value is provided when the object is created.

Assignment

```
C
x = 20;
```

The existing object receives a new value.

DON'T CONFUSE

```
C
int x = 10;
```

Here `=` participates in initialization.

But:

```
C
x = 20;
```

is assignment.

For beginner problem solving, both visually look like "put a value into x", but technically the situations differ.

Example

```
C
int x = 10;
x = 30;
```

Trace:

Step	x
initialization	10
x = 30	30

Final value:

```
TEXT
30
```

7. DATATYPES

Why do we need types?

Suppose memory contains:

```
TEXT
01000001
```

Should that represent:

- a number?
- a character?
- part of another value?

A programming language needs information about how values should be interpreted and manipulated.

That is one major job of a **data type**.

Basic C Types

The most important types initially are:

TEXT

```
char
int
float
double
```

int

Used for integer values.

```
C
int marks = 95;
int temperature = -5;
```

Examples:

TEXT

```
0
15
-30
1000
```

char

Used for character-sized integer data; commonly used to store characters.

```
C
char grade = 'A';
```

Notice:

```
C
'A'
```

uses **single quotes**.

float

Floating-point type.

```
C
float price = 19.5f;
```

double

Higher-precision floating-point type than `float` in typical C implementations.

```
C
double pi = 3.1415926535;
```

Simple Table

Type	Typical purpose	Example
char	character/small integer data	'A'
int	integers	25
float	floating-point number	3.5f
double	higher-precision floating-point	3.14159

IMPORTANT GATE WARNING — SIZE

Do **not** blindly memorize:

```
TEXT
int = 4 bytes
```

as a universal rule.

The C language standard does not require every machine to use exactly the same size for all these types.

For practical problems, the question may explicitly give sizes or use an implementation where common sizes apply.

We will later learn `sizeof`.

8. VARIABLE NAMING BASICS

Valid examples:

```
C
age
studentAge
marks1
_total
```

Invalid:

```
TEXT
2marks
student age
```

And reserved language keywords cannot be used as identifiers.

For example:


```
C
int int = 5;
```

is invalid because `int` is a keyword.

9. CODE EXAMPLE

```
C
#include <stdio.h>

int main()
{
    int age = 20;
    char grade = 'A';
    float marks = 91.5f;

    printf("%d\n", age);
    printf("%c\n", grade);
    printf("%f\n", marks);

    return 0;
}
```

Conceptually:

```
TEXT
a g e      !'   2 0
g r a d e !'   A
m a r k s !'   9 1 . 5
```

FORMAT SPECIFIERS — INITIAL SET

Data	printf format
integer	%d
character	%c
floating-point	%f
string	%s

Strings will come later.

10. MEMORY VISUALIZATION

Consider:

```
C
int a = 10;
int b = 20;
```

For conceptual understanding:

TEXT

Memory

Some address

```
+-----+
|  10  |
+-----+
      a
```

Another address

```
+-----+
|  20  |
+-----+
      b
```

Later we will literally access those addresses using pointers.

That future connection is:

TEXT

VARIABLE

!"

has a value

!"

exists somewhere in memory

!"

has an address

!"

POINTER can hold that address

This is why today's basic lesson matters.

11. HOW VALUES CHANGE

Program:

C

```
int x = 5;
x = 8;
x = x + 2;
```

Trace carefully.

Initially:

TEXT

x = 5

Then:

C

x = 8

Then:

C

```
x = x + 2
```

Important:

The right side is evaluated using the **old/current value**.

TEXT

```
x + 2  
= 8 + 2  
= 10
```

Then **10** is assigned to **x**.

Final:

TEXT

```
x = 10
```

SOLVED EXAMPLE 1 — EASY

C

```
int a = 5;  
a = 10;  
  
printf("%d", a);
```

Output:

TEXT

```
10
```

Why?

The old value **5** was replaced by **10**.

SOLVED EXAMPLE 2 — MODERATE

C

```
int a = 5;  
int b = a;  
a = 10;  
  
printf("%d %d", a, b);
```

Trace:

Initially:

TEXT

a = 5

Then:

c

b = a

So:

TEXT

b = 5

Then:

TEXT

a = 10

Now:

TEXT

a = 10

b = 5

Output:

TEXT

10 5

Important idea:

Changing **a** afterward does **not automatically change** **b**.

SOLVED EXAMPLE 3 — GATE-STYLE FOUNDATION

c

```
int x = 4;
```

```
int y = x;
```

```
x = x + 3;
```

```
y = y + x;
```

Trace:

Statement	x	y
x = 4	4	—
y = x	4	4
x = x + 3	7	4

Statement	x	y
$y = y + x$	7	11

Final:

TEXT

```
x = 7
y = 11
```

This trace-table habit will become extremely useful when we reach pointers and recursion.

12. GATE FOCUS

For this introductory section, GATE is unlikely to ask something merely like:

What is a variable?

Instead, basic C concepts become ingredients in harder questions.

Expect questions eventually involving combinations such as:

TEXT

```
variable values
+
operators
+
scope
+
functions
+
pointers
+
arrays
```

So our goal is not memorizing syntax.

Our goal is:

Learn how C actually evaluates programs.

13. GATE TRAPS

Trap 1 — Assignment is not mathematical equality

Mathematically:

TEXT

```
x = x + 1
```

looks impossible.

Programming interpretation:

TEXT

```
take old x
add 1
store result back in x
```

Example:

TEXT

```
x = 5
x = x + 1
```

becomes:

TEXT

```
x = 6
```

Trap 2 — 'A' and "A" are not the same

C

```
'A'
```

character constant.

C

```
"A"
```

string literal.

Remember this.

Trap 3 — Variable values don't magically stay connected

C

```
int a = 10;
int b = a;

a = 20;
```

b remains **10**.

We'll later see pointers, where we **can** create address-based relationships.

Trap 4 — Don't assume type sizes unnecessarily

If a question depends on exact memory size, inspect the information given.

14. PLACEMENT CONNECTION

An interviewer normally won't hire you because you know:

```
int stores integers.
```

But these foundations determine whether you can understand:

```
C
int x = 10;
int *p = &x;
```

Then later:

```
C
struct Node {
    int data;
    struct Node *next;
};
```

And suddenly we have a linked list.

So the journey is:

```
TEXT
Variable
    !"
Memory
    !"
Address
    !"
Pointer
    !"
Node
    !"
Linked List
    !"
Trees / Graphs
```

That's why we're building from the bottom.

15. INTERVIEW QUESTIONS

Q1. What is a variable?

A named object that stores a value of a specified type.

Q2. What is a data type?

It describes the kind of data an object can hold and helps determine how operations on that data are interpreted.

Q3. Difference between declaration and initialization?

```
C
int x;
```

Declaration.

```
c
int x = 5;
```

Declaration with initialization.

Q4. Where does execution start in a normal C program?

At `main()` in the usual hosted C environment.

Q5. What does `return 0;` from `main` conventionally indicate?

Successful program termination.

16. MEMORY TRICKS

C program

Remember:

Write !' Translate !' Run

```
TEXT
C source
! "
Compiler/toolchain
! "
Executable
! "
Run
```

Variable

Think:

Variable = Named Box

Four basic types

Remember:

C I F D

Char Int Float Double

17. DON'T CONFUSE

Stack memory " Stack data structure

We haven't studied either deeply yet.

But remember this from Day 1.

Stack memory

! region/mechanism associated with function calls and automatic local objects.

Stack data structure

! LIFO abstract data structure.

Same word.

Different concept.

Ø=7% causes confusion later.

18. NOTEBOOK NOTES

You can copy this compact section into your notes.

C Program Basics

A C source program is translated into executable machine instructions before execution.

Basic flow:

TEXT

Source code ! Translation / toolchain ! Executable ! Execution

Basic program:

```
C
#include <stdio.h>

int main()
{
    printf("Hello");
    return 0;
}
```

Variable

A variable is a named object used to store a value.

```
C
int x = 10;
```

x ! variable int ! type 10 ! initial value

Basic types

TEXT

```
char
int
float
double
```

Important

```
c
int x;          // declaration
int x = 10;     // initialization
x = 20;         // assignment
```

Common `printf` formats:

```
TEXT
% d !'  i n t
% c !'  c h a r
% f !'  f l o a t i n g   p o i n t   d i s p l a y
% s !'  s t r i n g
```

Do not universally assume exact data-type sizes unless established by the question/implementation.

19. 30-SECOND REVISION

Remember only these six things:

1. C source code must be translated before CPU execution.
2. `main()` is the program's main entry function in normal hosted C programs.
3. Variable = named object holding a value.
4. Type tells C how data is interpreted/used.
5. Basic types: `char`, `int`, `float`, `double`.
6. Assignment changes an existing value.

And most importantly:

```
c
x = x + 1;
```

means:

Take current `x`, add 1, store the result back into `x`.

20. ACTIVE RECALL

Try these **without looking at the answers**.

Q1

What is the broad purpose of a compiler?

Q2

What is the difference between:

```
c
int x;
```

and

```
C
int x = 5;
```

Q3

What is the final value?

```
C
int x = 3;
x = x + 4;
```

Q4

Are these the same?

```
C
'A'
"A"
```

Q5

```
C
int a = 5;
int b = a;
a = 8;
```

What is `b`?

ANSWERS — CHECK AFTER ATTEMPTING

A1

It translates C source code as part of producing executable machine-understandable code.

A2

```
C
int x;
```

declares `x`.

```
C
int x = 5;
```

declares and initializes `x` with `5`.

A3

```
TEXT
7
```

A4

No.

```
C
'A'
```

is a character constant.

```
C
"A"
```

is a string literal.

A5

```
TEXT
5
```

PRACTICE

Q1

Find final `x`:

```
C
int x = 10;
x = 20;
x = x + 5;
```

Q2

Find `a` and `b`:

```
C
int a = 2;
int b = a;
b = 7;
```

Q3 — GATE FOUNDATION

Trace:

```
C
int a = 3;
int b = 4;

a = a + b;
b = a - b;
```

Find final `a` and `b`.

Q4 — GATE FOUNDATION

```
C
int x = 5;
int y = x;

x = x + y;
y = x + y;
```

Find final `x` and `y`.

Q5 — PLACEMENT THINKING

Suppose someone asks:

Why doesn't `b` change when `a` changes here?

```
C
int a = 10;
int b = a;
a = 20;
```

Try explaining it verbally in **one sentence**, as if answering an interviewer.

PROGRESS

Completed '

Module 0 / Lesson 1

- C program execution intuition
- basic C structure
- variables
- declaration
- initialization
- assignment
- basic data types
- introductory program tracing
- first GATE traps

Next

Module 0 / Lesson 2 — Operators & Expressions

This will be a very important lesson because we will cover:

Arithmetic !' relational !' logical !' increment/decrement !' precedence !' associativity !' short-circuit evaluation

Type **NEXT**.

NEXT

MODULE 0 — LESSON 2

We continue exactly from the course structure in your uploaded master prompt: simple explanation first, then technical detail, GATE traps, tracing, placement connection, notes, recall, and practice.

- Arithmetic operators
- Relational operators
- Logical operators
- Assignment operators
- Increment/decrement
- Bitwise operators — introduction
- Conditional operator
- Operator precedence
- Associativity
- Short-circuit evaluation
- Common C traps

You give it:

TEXT

10 + 5

Here:

- 10 !' operand
- 5 !' operand
- + !' operator

The calculator performs an operation on the operands.

Similarly in C:

C

```
int x = 10 + 5;
```

+ is an operator.

But C contains many operators.

Think of operators as **commands represented by symbols**.

Examples:

TEXT

+	!'	a d d
-	!'	s u b t r a c t
*	!'	m u l t i p l y
/	!'	d i v i d e
>	!'	c o m p a r e
& &	!'	l o g i c a l A N D
+	!'	i n c r e a s e
&	!'	b i t w i s e A N D / a d d r e s s o p e r a t o r d e p e n d i n g o n c o n t e x t

The real difficulty isn't knowing that + means addition.

The difficulty is understanding expressions such as:

C

```
a + b * c
```

or:

C

```
x++ + ++y
```

or:

C

```
a && b || c
```

That is where GATE questions become interesting.

3. TECHNICAL EXPLANATION

What is an operator?

An **operator** is a symbol that instructs C to perform some operation on one or more operands.

Example:

```
c
a + b
```

Here:

```
TEXT
a      !'  o p e r a n d
+      !'  o p e r a t o r
b      !'  o p e r a n d
```

What is an expression?

An **expression** is a combination of operands and operators that produces a value or otherwise performs computation according to C's rules.

Examples:

```
c
5 + 3
```

```
c
a * b
```

```
c
x > 10
```

```
c
a + b * c
```

4. ARITHMETIC OPERATORS

The main arithmetic operators are:

Operator	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Remainder

Addition

```
C
int a = 5;
int b = 3;
int c = a + b;
```

Result:

```
TEXT
c = 8
```

Subtraction

```
C
int x = 10;
int y = 4;

int z = x - y;
```

Result:

```
TEXT
6
```

Multiplication

```
C
int x = 4;
int y = 5;

int z = x * y;
```

Result:

```
TEXT
20
```

5. INTEGER DIVISION — 0 = MUST KNOW

Consider:

```
C
int x = 5 / 2;
```

A beginner may say:

TEXT

2.5

But both operands are integers.

Therefore integer division occurs.

Result:

TEXT

2

The fractional part is discarded toward zero.

So:

C

```
printf("%d", 5 / 2);
```

prints:

TEXT

2

Compare

C

5 / 2

Result:

TEXT

2

But:

C

5.0 / 2

contains floating-point arithmetic.

Result:

TEXT

2.5

Similarly:

C

5 / 2.0

produces floating-point division.

0=1 MEMORY TRICK

Integer / Integer != Integer division

Think:

INT ÷ INT = decimal gets cut off

Example:

TEXT

9 / 4 = 2

not 2.25 .

GATE TRAP

C

float x = 5 / 2;

What is stored?

You might think:

TEXT

2.5

Wrong.

First:

TEXT

5 / 2

is evaluated.

Both operands are integers.

Therefore:

TEXT

5 / 2 = 2

Then 2 is converted to float .

So:

TEXT

```
x = 2.0
```

0=Very common conceptual trap.

Correct way

C

```
float x = 5.0 / 2;
```

Result:

TEXT

2.5

6. MODULUS / REMAINDER OPERATOR %

Example:

C

```
10 % 3
```

Division:

TEXT

```
10 = 3 × 3 + 1
```

Remainder:

TEXT

1

Therefore:

C

```
10 % 3
```

is:

TEXT

1

Common uses

Check even number

```
C
n % 2 == 0
```

Check odd number

```
C
n % 2 != 0
```

Get last digit

```
C
n % 10
```

Example:

```
TEXT
347 % 10 = 7
```

Remove last digit

```
C
n / 10
```

Example:

```
TEXT
347 / 10 = 34
```

because integer division occurs.

This pair is extremely useful in coding:

```
TEXT
n % 10 !' last digit
n / 10 !' remove last digit
```

7. RELATIONAL OPERATORS

Relational operators compare values.

Operator	Meaning
<	less than
>	greater than
<=	less than or equal

Operator	Meaning
>=	greater than or equal
==	equal to
!=	not equal

Example

```
C
5 > 3
```

This condition is true.

In C, relational expressions produce integer values:

```
TEXT
true    != 1
false   != 0
```

So:

```
C
printf("%d", 5 > 3);
```

prints:

```
TEXT
1
```

And:

```
C
printf("%d", 5 < 3);
```

prints:

```
TEXT
0
```

~~0=1~~ DON'T CONFUSE

```
C
=
```

means:

assignment

But:

```
C
==
```

means:

comparison for equality

Example:

```
C
x = 5;
```

Put 5 into x.

But:

```
C
x == 5
```

Ask:

Is x equal to 5?

8. GATE CLASSIC TRAP — = vs ==

Consider:

```
C
int x = 5;

if (x = 10)
    printf("Yes");
```

Notice:

```
C
x = 10
```

not:

```
C
x == 10
```

The assignment puts 10 into x.

The value of assignment expression is 10.

In a condition:

TEXT

```
0 !' f a l s e
n o n z e r o !' t r u e
```

Therefore condition is true.

Output:

TEXT

Yes

And now:

TEXT

x = 10

Very important.

9. LOGICAL OPERATORS

Logical operators combine or negate conditions.

Operator	Meaning		
&&	logical AND		
,		,	logical OR
!	logical NOT		

Logical AND &&

Both conditions must be true.

TEXT

A && B

Truth table:

A	B	A && B
0	0	0
0	1	0
1	0	0
1	1	1

Memory trick:

AND = everyone must agree.

Logical OR ||

At least one must be true.

A	B	A B
0	0	0
0	1	1
1	0	1
1	1	1

Memory trick:

OR = one approval is enough.

Logical NOT !

Reverses truth.

TEXT

```
! 0 !' 1
! 1 !' 0
```

More generally:

TEXT

```
! nonzero !' 0
! 0 !' 1
```

Example:

```
c
printf("%d", !5);
```

5 is nonzero !' true.

NOT true !' false.

Output:

TEXT

```
0
```

10. C TRUTH RULE

Remember this:

TEXT

```
0 = false
nonzero = true
```

Examples:

```
C
if (0)
```

false.

```
C
if (1)
```

true.

```
C
if (-5)
```

also true.

```
C
if (100)
```

true.

C does not require a condition to literally equal `1`.

Any nonzero scalar value is treated as true in this basic context.

11. SOLVED EXAMPLE

```
C
int a = 5;
int b = 10;

printf("%d", a < b && b > 7);
```

Evaluate:

```
TEXT
a < b
5 < 10
true != 1
```

Then:

TEXT

```
b > 7
10 > 7
true != 1
```

Therefore:

TEXT

```
1 && 1
= 1
```

Output:

TEXT

```
1
```

12. SHORT-CIRCUIT EVALUATION

This is one of the most important concepts today.

Suppose:

C

```
A && B
```

If **A** is false, do we need to evaluate **B**?

No.

Because:

TEXT

```
false AND anything = false
```

Therefore C may skip evaluation of **B**.

This is called:

Short-circuit evaluation

&& Rule

TEXT

```
A && B
```

If **A** is false:

TEXT

```
B is not evaluated.
```

|| Rule

TEXT

A || B

If A is true:

TEXT

B is not evaluated.

Because:

TEXT

true OR anything = true

GATE EXAMPLE

C

```
int x = 0;
int y = 5;
```

```
if (x != 0 && y / x > 2)
    printf("Yes");
```

Look carefully.

First:

C

```
x != 0
```

Since `x = 0`:

TEXT

false

With `&&`, once first condition is false, the second side:

C

```
y / x
```

is not evaluated.

Therefore division by zero is avoided here.

`0 != 0` is a practical use of short-circuiting.

Very Important Example

```
C
int x = 0;
int y = 5;

if (x && ++y)
    printf("A");

printf("%d", y);
```

First operand:

```
TEXT
x = 0
```

false.

Because `&&` short-circuits:

```
C
++y
```

is **not executed**.

Therefore:

```
TEXT
y remains 5
```

Output:

```
TEXT
5
```

This is a classic output-prediction pattern.

13. ASSIGNMENT OPERATORS

Basic assignment:

```
C
x = 10;
```

Compound assignment:

```
C
x += 5;
```

means conceptually:

```
C
x = x + 5;
```

Similarly:

Operator	Similar basic idea
<code>+=</code>	add then assign
<code>-=</code>	subtract then assign
<code>*=</code>	multiply then assign
<code>/=</code>	divide then assign
<code>%=</code>	remainder then assign

Example:

```
C
int x = 10;
x += 3;
```

Result:

```
TEXT
13
```

Then:

```
C
x *= 2;
```

Result:

```
TEXT
26
```

14. INCREMENT AND DECREMENT $\emptyset = \acute{Y}\% \emptyset = \acute{Y}\% \emptyset = \acute{Y}\%$

Increment:

```
C
++
```

means increase by 1.

Decrement:

```
C
--
```

means decrease by 1.

Four Forms

```
C
++x
x++
--x
x--
```

The difficult part:

Prefix vs Postfix

Prefix Increment

```
C
++x
```

Conceptually:

1. increment `x`
2. yield the incremented value

Example:

```
C
int x = 5;
int y = ++x;
```

Trace:

```
TEXT
x starts 5
++x !' x becomes 6
y receives 6
```

Final:

```
TEXT
x = 6
y = 6
```

Postfix Increment

```
C
x++
```

Conceptually:

1. the expression yields the old value
2. `x` is incremented as a side effect

Example:

```
C
int x = 5;
int y = x++;
```

Trace:

```
TEXT
y receives old x !' 5
x becomes 6
```

Final:

```
TEXT
x = 6
y = 5
```

MEMORY TRICK

Prefix

```
TEXT
++x
```

`++` comes **before** `x`.

Change first, use new value.

Postfix

```
TEXT
x++
```

`++` comes **after** `x`.

Use old value, then change.

Compare

```
C
int x = 5;
printf("%d", ++x);
```

Output:

TEXT

6

x = 6.

C

```
int x = 5;
printf("%d", x++);
```

Output:

TEXT

5

Afterward:

TEXT

x = 6

15. PREFIX/POSTFIX TRACE EXAMPLE

C

```
int x = 4;
int y = x++;
int z = ++x;
```

Start:

TEXT

x = 4

Statement:

C

y = x++;

y receives old x.

TEXT

y = 4
x = 5

Then:

C

z = ++x;

First increment:

TEXT

```
x = 6
```

Then assign:

TEXT

```
z = 6
```

Final:

Variable	Value
x	6
y	4
z	6

16. VERY IMPORTANT C WARNING — UNDEFINED BEHAVIOR

You may encounter expressions such as:

C

```
i = i++ + ++i;
```

or similar questions online.

Do **not** blindly try to trace every such expression as though C guarantees one answer.

Certain expressions that modify the same object multiple times without the required sequencing can have **undefined behavior**.

That means the C language does not specify one reliable result.

0-6% C-quality reasoning should distinguish:

TEXT

well-defined expression

from:

TEXT

undefined behavior

We will revisit sequencing carefully when we reach advanced C output questions.

For now:

Do not assume every scary increment expression has one valid output.

17. CONDITIONAL / TERNARY OPERATOR

Syntax:

```
C
condition ? value_if_true : value_if_false
```

Example:

```
C
int a = 10;
int b = 20;

int max = (a > b) ? a : b;
```

Condition:

```
TEXT
a > b
10 > 20
false
```

Therefore:

```
TEXT
max = b = 20
```

Story

Think:

Ask a question !' choose one of two doors.

```
TEXT
condition?
  /  \
yes   no
 /    \
A      B
```

Example

```
C
int n = 7;

printf("%s", (n % 2 == 0) ? "Even" : "Odd");
```

Since:

TEXT

7 % 2 = 1

condition false.

Output:

TEXT

Odd

18. BITWISE OPERATORS — INTRODUCTION

We will study these more carefully when needed.

C provides:

Operator	Meaning	
&	bitwise AND	
\	`	bitwise OR
^	bitwise XOR	
~	bitwise NOT	
<<	left shift	
>>	right shift	

Do **not confuse** them with logical operators.

DON'T CONFUSE

Logical AND:

C

&&

Bitwise AND:

C

&

Logical OR:

C

||

Bitwise OR:

```
c
|
```

These are different.

Example: Bitwise AND

Take:

```
TEXT
5 = 0101
3 = 0011
```

Bitwise AND:

```
TEXT
0101
0011
----
0001
```

Therefore:

```
c
5 & 3
```

equals:

```
TEXT
1
```

19. XOR — USEFUL IDEA

XOR truth pattern:

```
TEXT
s a m e   b i t s           ! '   0
d i f f e r e n t   b i t s ! '   1
```

Examples:

```
TEXT
0 ^ 0 = 0
1 ^ 1 = 0
0 ^ 1 = 1
1 ^ 0 = 1
```

Useful properties:

TEXT

```
x ^ x = 0
x ^ 0 = x
```

PEACEMENT EXTENSION: These properties are useful in problems such as finding the unique number when every other number appears twice.

We will revisit them properly.

20. SHIFT OPERATORS

Example:

```
C
5 << 1
```

Binary:

```
TEXT
5 = 0101
```

Left shift one position:

```
TEXT
1010
```

which is **10** in this simple positive-value example.

Often:

```
TEXT
x << 1
```

acts like multiplication by 2 when the operation is defined and representable for the values/type involved.

Similarly:

```
TEXT
x >> 1
```

often resembles division by 2 for nonnegative integers.

But exact rules and edge cases matter, so don't turn this into a universal shortcut yet.

21. OPERATOR PRECEDENCE Ø=Ý%Ø=Ý%Ø=Ý%

Consider:

```
C
2 + 3 * 4
```

Two possible interpretations:

TEXT

$(2 + 3) * 4 = 20$

or:

TEXT

$2 + (3 * 4) = 14$

C chooses based on **operator precedence**.

Multiplication has higher precedence than addition.

Therefore:

TEXT

```
2 + 3 * 4
= 2 + 12
= 14
```

Story

Think of precedence as **VIP priority**.

Some operators get evaluated before others when the grammar determines grouping.

Example:

TEXT

* gets higher priority than +

Important Precedence Order

You do not need to memorize the entire C standard table immediately.

Start with this practical high-yield order:

TEXT

```
()                highest among this simplified set
postfix ++ --
unary ++ -- ! etc.
* / %
+ -
< <= > >=
== !=
&
^
|
&&
||
?:
assignment
```

This is a useful working hierarchy.

We will add details gradually.

MEMORY CHAIN

For the most common arithmetic part:

```
TEXT
Brackets
!"
Unary/increment context
!"
Multiply Divide Modulus
!"
Add Subtract
!"
Comparison
!"
Logical
!"
Assignment
```

22. PARENTHESES ARE YOUR FRIEND

Instead of:

```
c
a + b * c
```

you can make intent explicit:

```
c
a + (b * c)
```

or:

```
c
(a + b) * c
```

This improves clarity.

For GATE, however, you must understand precedence because they may intentionally omit parentheses.

23. ASSOCIATIVITY

What if two operators have the same precedence?

Then **associativity** helps determine grouping.

Example:

C

$10 - 5 - 2$

- is left associative.

So:

TEXT

$(10 - 5) - 2$
 $= 5 - 2$
 $= 3$

Not:

TEXT

$10 - (5 - 2)$
 $= 7$

Another Example

Assignment is right associative.

C

$a = b = 5;$

groups like:

C

$a = (b = 5);$

First:

TEXT

$b = 5$

Then the assignment expression yields the assigned value, so:

TEXT

$a = 5$

Final:

TEXT

$a = 5$
 $b = 5$

DON'T CONFUSE

Precedence

Determines which operators bind more tightly.

Associativity

Helps determine grouping when operators are at the same precedence level.

24. SOLVED EXAMPLE — PRECEDENCE

C

```
int x = 2 + 3 * 4;
```

* first:

TEXT

$$3 * 4 = 12$$

Then:

TEXT

$$2 + 12 = 14$$

So:

TEXT

$$x = 14$$

25. SOLVED EXAMPLE — PARENTHESES

C

```
int x = (2 + 3) * 4;
```

Parentheses first:

TEXT

$$2 + 3 = 5$$

Then:

TEXT

$$5 * 4 = 20$$

So:

TEXT

$$x = 20$$

26. SOLVED EXAMPLE — RELATIONAL + LOGICAL

```
C
int a = 5;
int b = 10;
int c = 15;

int x = a < b && b < c;
```

Evaluate:

```
TEXT
5   <   10   !'   1
10  <   15   !'   1
```

Then:

```
TEXT
1   &&   1   !'   1
```

Therefore:

```
TEXT
x = 1
```

27. SOLVED EXAMPLE — SHORT CIRCUIT

```
C
int x = 0;
int y = 10;

int z = x && ++y;
```

First operand:

```
TEXT
x = 0
```

Since false, && stops.

++y does not execute.

Therefore:

```
TEXT
y = 10
z = 0
```

28. SOLVED EXAMPLE — OR SHORT CIRCUIT

```
C
int x = 5;
int y = 10;

int z = x || ++y;
```

`x = 5` is nonzero != true.

For OR:

```
TEXT
true || anything = true
```

Second operand is skipped.

Therefore:

```
TEXT
y = 10
z = 1
```

29. GATE TRAP — CHAINED COMPARISON

Suppose you write:

```
C
1 < x < 10
```

Many students interpret this mathematically as:

```
TEXT
x is between 1 and 10
```

But C does not interpret chained comparisons like mathematics.

It groups according to operator rules.

Conceptually:

```
C
(1 < x) < 10
```

The first comparison produces:

```
TEXT
0 or 1
```

Then that result is compared with `10`.

So this does **not** express the desired range test.

Correct:

```
C
x > 1 && x < 10
```

Excellent C/GATE trap.

30. GATE TRAP — % AND DIVISION

Consider:

```
C
int x = 17;

printf("%d %d", x / 10, x % 10);
```

Calculate:

```
TEXT
17 / 10 = 1
17 % 10 = 7
```

Output:

```
TEXT
1 7
```

This idea appears repeatedly in digit-processing programs.

31. GATE TRAP — INTEGER EXPRESSION FIRST

```
C
double x = 7 / 2;
```

7 / 2:

```
TEXT
3
```

Then convert:

```
TEXT
3.0
```

Not:

```
TEXT
3.5
```

Correct:

```
C
double x = 7.0 / 2;
```

32. GATE TRAP — LOGICAL RESULT

```
C
int x = 5 && 10;
```

Both are nonzero.

Therefore:

```
TEXT
x = 1
```

Not:

```
TEXT
50
```

Logical operators produce logical result values, not arithmetic multiplication.

33. BITWISE VS LOGICAL — CRUCIAL EXAMPLE

Consider:

```
C
5 && 3
```

Both operands nonzero.

Result:

```
TEXT
1
```

But:

```
C
5 & 3
```

Binary:

```
TEXT
0101
0011
----
0001
```

Result:

TEXT

1

Coincidentally same result here.

But consider:

C

6 && 3

Both nonzero:

TEXT

1

Now bitwise:

TEXT

6 = 0110

3 = 0011

TEXT

0110

0011

0010

Therefore:

TEXT

6 & 3 = 2

So:

TEXT

6 && 3 = 1

6 & 3 = 2

Completely different operations.

34. HOW TO TRACE EXPRESSIONS

Whenever you encounter a scary C expression, follow this order:

STEP 1

Write the current values of all variables.

STEP 2

Identify parentheses.

STEP 3

Check operators and grouping.

STEP 4

Check prefix/postfix effects.

STEP 5

Check short-circuiting.

STEP 6

Perform arithmetic/comparison.

STEP 7

Perform assignment.

STEP 8

Update the trace table.

This method will prevent many mistakes.

35. TRACE EXAMPLE

```
c
int a = 3;
int b = 4;

int c = a++ + b;
```

Start:

Variable	Value
a	3
b	4

a++ yields old value:

```
TEXT
3
```

Then a becomes:

```
TEXT
4
```

Expression value:

```
TEXT
3 + 4 = 7
```


Therefore:

```
TEXT
c = 7
```

Final:

Variable	Value
a	4
b	4
c	7

36. PLACEMENT CONNECTION

Operators look basic, but coding interviews constantly use them.

Examples:

Check even/odd

```
C
n % 2
```

Extract digits

```
C
n % 10
```

Remove digit

```
C
n / 10
```

Range checking

```
C
x >= low && x <= high
```

Guard dangerous operation

```
C
ptr != NULL && ptr->next != NULL
```

Short-circuit evaluation helps protect the second condition.

Bit manipulation

```
C
x & 1
```

is often used to inspect the lowest bit.

Placement problems use bitwise operators frequently.

37. INTERVIEW QUESTIONS

Q1. Difference between `=` and `==` ?

`=` performs assignment.

`==` checks equality.

Q2. What is integer division?

When integer operands are divided, the fractional part is discarded toward zero.

Example:

```
TEXT
5 / 2 = 2
```

Q3. Difference between `x++` and `++x` ?

`++x` increments first and yields the incremented value.

`x++` yields the old value and increments afterward as a side effect.

Q4. What is short-circuit evaluation?

In logical expressions, C may skip evaluating the second operand when the first operand already determines the result.

Q5. What values represent true and false in C conditions?

```
TEXT
0 !' false
nonzero !' true
```

Q6. Difference between `&&` and `&` ?

`&&` is logical AND and short-circuits.

`&` is bitwise AND for integer operands and also has another role as the address-of operator in unary context.

Pointers will make that second meaning clear.

38. MEMORY TRICKS

Arithmetic

Remember:

A S M D R

- Add
- Subtract
- Multiply
- Divide
- Remainder

Comparison

TEXT

```
=      !'   g i v e  
==     !'   a s k
```

= gives a value.

== asks whether values are equal.

Logical

TEXT

```
& &  !'   A L L  
| |   !'   A N Y  
!     !'   O P P O S I T E
```

Increment

TEXT

```
+ + x  !'   c h a n g e !'   u s e  
x + +  !'   u s e !'   c h a n g e
```

Short Circuit

TEXT

```
F A L S E   & &   a n y t h i n g   !'   S T O P  
T R U E    | |   a n y t h i n g   !'   S T O P
```

39. FORMULA / RULE BOX

Copy these.

TEXT

I n t e g e r / I n t e g e r ! ' i n t e g e r d i v i s i o n

TEXT

n % 1 0 ! ' l a s t d e c i m a l d i g i t f o r n o n n e g a t i v e n

TEXT

n / 1 0 ! ' r e m o v e l a s t d e c i m a l d i g i t f o r n o n n e g a t i v e n

TEXT

0 ! ' f a l s e
n o n z e r o ! ' t r u e

TEXT

A && B:
i f A f a l s e ! ' B s k i p p e d

TEXT

A || B:
i f A t r u e ! ' B s k i p p e d

TEXT

+ + x ! ' i n c r e m e n t f i r s t
x + + ! ' o l d v a l u e u s e d f i r s t

40. NOTEBOOK NOTES

Operators

An operator performs an operation on operands.

Arithmetic

TEXT

+ - * / %

Relational

TEXT

< > <= >= == !=

Relational result:

TEXT

```
true = 1
false = 0
```

Logical

TEXT

```
&& || !
```

In conditions:

TEXT

```
0 = false
nonzero = true
```

Assignment

TEXT

```
=
+=
-=
*=
/=
%=
```

Increment / Decrement

TEXT

```
++ x !' increment first, use new value
x ++ !' use old value, then increment
```

Integer division

TEXT

```
5 / 2 = 2
```

TEXT

```
5.0 / 2 = 2.5
```

Short circuit

TEXT

```
A && B
```

If A false !' B not evaluated.

TEXT

```
A || B
```

If A true !' B not evaluated.

Precedence

Higher-precedence operations bind before lower-precedence operations.

Example:

TEXT

```
2 + 3 * 4 = 14
```

Associativity

Used to determine grouping between operators of equal precedence.

Example:

TEXT

```
10 - 5 - 2  
= (10 - 5) - 2  
= 3
```

41. 30-SECOND REVISION

Before moving on, remember:

1. `5 / 2 = 2`
2. `5.0 / 2 = 2.5`
3. `%` gives remainder.
4. `=` assignment; `==` comparison.
5. `0` false; nonzero true.
6. `&&` requires both conditions.
7. `||` requires at least one.
8. `++x` = change first.
9. `x++` = use old value first.
10. `&&` and `||` short-circuit.
11. `*` / `%` bind tighter than `+` - .
12. Be careful with expressions that modify the same variable multiple times; some can have undefined behavior.

42. ACTIVE RECALL

Try these first.

Q1

What is:

C

```
7 / 2
```

Q2

What is:

```
C
7 % 2
```

Q3

Final values?

```
C
int x = 5;
int y = x++;
```

Q4

Final values?

```
C
int x = 5;
int y = ++x;
```

Q5

What happens to `y` ?

```
C
int x = 0;
int y = 5;

x && ++y;
```

ANSWERS

A1

```
TEXT
3
```

A2

```
TEXT
1
```

A3

TEXT

```
x = 6  
y = 5
```

A4

TEXT

```
x = 6  
y = 6
```

A5

Since `x = 0`, `&&` short-circuits.

`++y` does not execute.

TEXT

```
y = 5
```

43. PRACTICE

Q1

Find the output:

```
C  
#include <stdio.h>  
  
int main()  
{  
    int x = 17;  
  
    printf("%d %d", x / 10, x % 10);  
  
    return 0;  
}
```

Q2

Find final values:

```
C  
int x = 8;  
  
x += 4;  
x *= 2;
```

What is `x`?

Q3 — GATE STYLE

Find the output:

```
C
int x = 5;
int y = 10;

printf("%d", x < y && y > 7);
```

Q4 — GATE STYLE

Trace carefully:

```
C
int x = 4;
int y = x++;
int z = ++x;

printf("%d %d %d", x, y, z);
```

Q5 — GATE TRAP

Find `x` and `y`:

```
C
int x = 0;
int y = 10;

int z = x && ++y;
```

Also find `z`.

Q6 — SHORT CIRCUIT

```
C
int x = 5;
int y = 10;

int z = x || ++y;
```

Find:

```
TEXT
x
y
z
```

Q7 — PRECEDENCE

Find:

```
C
int x = 2 + 4 * 3 - 6 / 2;
```

Do not calculate randomly.

Use precedence.

~~Ø=Q8~~ — THINK LIKE GATE

What is wrong with using:

C

$1 < x < 10$

to check whether `x` lies between `1` and `10`?

What should we write instead?

Ø=DO — PLACEMENT THINKING

Without converting an integer to a string, how could you:

1. obtain its last decimal digit?
2. remove its last decimal digit?

Example:

TEXT
n = 583

What operations would produce:

TEXT

3

and:

TEXT

58

?

PROGRESS

Completed '

Module 0 / Lesson 1

- C program basics
- variables

- data types

Module 0 / Lesson 2

- arithmetic operators
- integer division
- remainder
- relational operators
- logical operators
- truth values
- assignment operators
- prefix/postfix increment
- conditional operator
- bitwise introduction
- precedence
- associativity
- short-circuit evaluation
- major GATE traps

Next

Module 0 / Lesson 3 — Type Conversion & Type Casting

Next we will understand one of the reasons C output questions become confusing:

```
C
int + float
char + int
5 / 2
5 / 2.0
(float)5 / 2
```

We will cover:

implicit conversion !' explicit casting !' integer promotion !' mixed expressions !' common GATE traps.

Type **NEXT**.

USER

NEXT

ASSISTANT

MODULE 0 — LESSON 3

Type Conversion & Type Casting in C

We continue with the structure from your uploaded master prompt: intuition !' technical concept !' GATE traps !' solved examples !' placement connection !' notes !' recall !' practice.

1. WHERE WE ARE

Module: 0 — C Foundation **Lesson:** 3 **Current topic:** Type Conversion and Type Casting

Why it matters: C often combines values of different types in the same expression. If you don't know how those types interact, output questions become confusing very quickly.

GATE importance: 0=Ý%04V%0K%0 **Placement importance:** +P+IMPORTANT

Today we will cover:

- Implicit type conversion
- Explicit type casting
- Integer promotion
- Mixed-type expressions
- Division traps
- `char` in arithmetic
- Assignment conversion
- Common GATE traps

2. STORY / INTUITION

Imagine two people speaking different languages.

One speaks:

integers

The other speaks:

floating-point numbers

Suppose C sees:

```
c
5 + 2.5
```

It cannot perform the operation meaningfully by keeping one operand as an integer and the other as floating-point without rules.

So C converts them to compatible types.

Conceptually:

```
TEXT
5
!"
5.0
```

Then:

TEXT

```
5.0 + 2.5 = 7.5
```

This automatic conversion is called:

Implicit Conversion

But sometimes **you** explicitly tell C:

Treat this value as another type.

Example:

```
C
(float)5
```

That is:

Explicit Type Casting

3. TECHNICAL EXPLANATION

Type Conversion

Type conversion means changing a value from one data type to another.

Two major categories:

TEXT

```
TYPE  CONVERSION
```

```
|
```

```
+---- Implicit Conversion
```

```
|
```

```
+---- Explicit Conversion / Casting
```

4. IMPLICIT TYPE CONVERSION

C performs some conversions automatically.

Example:

```
C
int a = 5;
float b = 2.5f;

float c = a + b;
```

In the expression:

```
c
a + b
```

we have:

```
TEXT
a !' i n t
b !' f l o a t
```

Conceptually, the integer value gets converted so that the arithmetic can be performed in a common floating type.

So:

```
TEXT
5 !' 5 . 0
```

Then:

```
TEXT
5.0 + 2.5 = 7.5
```

Therefore:

```
TEXT
c = 7.5
```

5. WHY CONVERSION EXISTS

Suppose C did not have conversion rules.

What would this mean?

```
c
5 + 2.5
```

Should the result be:

```
TEXT
7?
```

or:

```
TEXT
7.5?
```

or something else?

C has precise rules to decide how operands are converted before operations.

6. SIMPLE MIXED-TYPE EXAMPLE

```
C
int x = 10;
double y = 2.5;

double z = x + y;
```

Conceptually:

```
TEXT
10
!"
10.0
```

Then:

```
TEXT
10.0 + 2.5 = 12.5
```

So:

```
TEXT
z = 12.5
```

7. EXPLICIT TYPE CASTING

Sometimes you manually request conversion.

Syntax:

```
C
(type) expression
```

Example:

```
C
(float)5
```

This tells C:

Convert the value 5 to float .

Example

```
C
int a = 5;
int b = 2;
```

```
float result = (float)a / b;
```

Without the cast:

```
C
a / b
```

would be:

```
TEXT
5 / 2
= 2
```

because both operands are integers.

But with:

```
C
(float)a
```

we conceptually have:

```
TEXT
5.0 / 2
```

Therefore:

```
TEXT
2.5
```

8. ~~0=1~~ THE BIGGEST DIVISION TRAP

Consider:

```
C
float x = 5 / 2;
```

Many students think:

```
TEXT
x = 2.5
```

Wrong.

The assignment type does **not** force the operands to be floating-point before division.

First evaluate:

```
C
5 / 2
```


Both operands are integers.

So:

TEXT

```
5 / 2 = 2
```

Then convert the result to `float`.

Therefore:

TEXT

```
x = 2.0
```

Correct versions

C

```
float x = 5.0 / 2;
```

or:

C

```
float x = (float)5 / 2;
```

or:

C

```
int a = 5;
int b = 2;

float x = (float)a / b;
```

All give:

TEXT

```
2.5
```

MEMORY TRICK

Conversion happens where the expression happens.

Not merely where the result is stored.

Compare:

C

```
float x = 5 / 2;
```

VS:

C

```
float x = (float)5 / 2;
```

First:

TEXT

```
5 / 2
!"
integer division
!"
2
!"
stored as 2.0
```

Second:

TEXT

```
(float)5 / 2
!"
floating-point division
!"
2.5
```

9. SOLVED EXAMPLE

C

```
int a = 7;
int b = 2;

float x = a / b;
```

Step 1:

TEXT

```
a / b
= 7 / 2
= 3
```

Step 2:

TEXT

```
3 converted to float
```

Therefore:

TEXT

```
x = 3.0
```

Not 3.5.

10. SOLVED EXAMPLE WITH CAST

```
C
int a = 7;
int b = 2;

float x = (float)a / b;
```

Step 1:

```
TEXT
(float)a
!' 7 . 0
```

Step 2:

```
TEXT
7.0 / 2
= 3.5
```

Therefore:

```
TEXT
x = 3.5
```

11. ASSIGNMENT CONVERSION

Suppose:

```
C
int x = 3.9;
```

A floating-point value is being stored into an integer object.

Conceptually:

```
TEXT
3.9
!"
converted to int
!"
3
```

The fractional part is discarded toward zero.

So:

```
TEXT
x = 3
```

Important

This is not rounding.

```
C
int x = 9.99;
```

does **not** become:

```
TEXT
10
```

It becomes:

```
TEXT
9
```

Negative Example

```
C
int x = -3.9;
```

conversion toward zero gives:

```
TEXT
-3
```

Not -4.

12. DON'T CONFUSE

Conversion to int

```
TEXT
3 . 9 !' 3
- 3 . 9 !' - 3
```

This is truncation toward zero.

It is not normal mathematical rounding.

13. INTEGER PROMOTION

This is a very important C concept.

Small integer types such as:

TEXT

```
char
short
```

are often promoted to `int` before arithmetic operations.

Consider:

```
C
char a = 10;
char b = 20;

int c = a + b;
```

You might imagine:

TEXT

```
char + char != char
```

But generally, before the arithmetic operation, the values are promoted to an integer type, typically `int` in ordinary situations.

So conceptually:

TEXT

```
a !' int 10
b !' int 20
```

Then:

TEXT

```
10 + 20 = 30
```

14. WHY INTEGER PROMOTION MATTERS

It explains why arithmetic on `char` values does not simply remain character arithmetic.

Example:

```
C
char c = 'A';

printf("%d", c + 1);
```

A character constant/value can participate in integer arithmetic.

If `'A'` has the character code 65 in the execution character set—common on ASCII systems—then:

TEXT

```
'A' + 1
!' 66
```

For GATE-style questions, character code assumptions are often explicit or implicitly ASCII-like.

Do not blindly assume ASCII if a question makes exact character-code values material without context.

15. CHARACTER AS INTEGER VALUE

In C:

```
C
char c = 'A';
```

`char` is an integer type.

That means arithmetic is possible:

```
C
c + 1
```

Example under ASCII:

```
TEXT
'A' = 65
'B' = 66
```

So:

```
C
printf("%C", 'A' + 1);
```

commonly prints:

```
TEXT
B
```

Again, this depends on the character encoding relationship assumed by the environment/question.

16. MIXED EXPRESSIONS

Consider:

```
C
int a = 5;
float b = 2.0f;
double c = 3.0;
```

Expression:

```
C
a + b + c
```

C applies conversion rules so operands are brought to compatible types.

Broad intuition:

TEXT

```
int
!"
float/double as required
```

Eventually the expression is evaluated using a suitable common type.

For GATE foundation, understand this hierarchy conceptually:

TEXT

```
smaller integer types
!"
int
!"
floating types when mixed
```

Later, when necessary, we can discuss the exact usual arithmetic conversions in more depth.

17. USUAL HIGH-YIELD CONVERSION IDEA

For common exam problems:

TEXT

```
char/short
!" promotion
int
!"
float
!"
double
```

This is a useful intuition.

But ~~it is~~ important:

Do not treat this simple diagram as a full substitute for all C standard conversion rules involving signedness and exact ranks.

For most introductory GATE expressions, it is enough to reason correctly.

18. SOLVED EXAMPLE — INT + FLOAT

C

```
int a = 5;
float b = 2.5f;

float c = a + b;
```

Convert `a` conceptually:

TEXT

```
5 !" 5 . 0
```

Then:

TEXT

```
5.0 + 2.5  
= 7.5
```

Therefore:

TEXT

```
c = 7.5
```

19. SOLVED EXAMPLE — INT + DOUBLE

C

```
int a = 4;  
double b = 1.25;  
  
double c = a + b;
```

Conceptually:

TEXT

```
4 !' 4 . 0
```

Then:

TEXT

```
4.0 + 1.25 = 5.25
```

20. SOLVED EXAMPLE — MULTI-STEP EXPRESSION

C

```
int a = 5;  
int b = 2;  
  
double x = a / b + 1.5;
```

Be careful.

First:

TEXT

```
a / b  
= 5 / 2  
= 2
```

because both operands are integers.

Then:

TEXT

$$2 + 1.5 \\ = 3.5$$

Therefore:

TEXT

$$x = 3.5$$

Many students incorrectly calculate:

TEXT

$$5 / 2 = 2.5$$

and then get 4.0.

Wrong.

21. CAST POSITION MATTERS

Compare:

C

$$(\text{float})(5 / 2)$$

and:

C

$$(\text{float})5 / 2$$

These are not equivalent.

Case 1

C

$$(\text{float})(5 / 2)$$

First:

TEXT

$$5 / 2 = 2$$

Then cast:

TEXT

$$(\text{float})2 = 2.0$$

Result:

TEXT

2.0

Case 2

C

```
(float)5 / 2
```

First convert:

TEXT

5 !' 5 . 0

Then:

TEXT

5.0 / 2 = 2.5

Result:

TEXT

2.5

Ø=Excellent GATE trap.

22. GATE TRAP — CAST AFTER DAMAGE

Suppose:

C

```
double x = (double)(7 / 2);
```

Many students think:

I used double, so answer is 3.5.

Wrong.

Inside parentheses:

TEXT

7 / 2 = 3

Then:

TEXT

(double)3 = 3.0

So:

```
TEXT
x = 3.0
```

To get 3.5:

```
C
double x = (double)7 / 2;
```

23. INTEGER PROMOTION EXAMPLE

Consider:

```
C
char a = 10;
char b = 20;

printf("%d", a + b);
```

The `char` values participate in arithmetic after integer promotions.

So:

```
TEXT
10 + 20 = 30
```

Output:

```
TEXT
30
```

24. GATE TRAP — CHARACTER ADDITION

Assume ASCII:

```
C
char c = 'A';

printf("%c", c + 2);
```

ASCII:

```
TEXT
'A' = 65
```

Then:

TEXT

65 + 2 = 67

ASCII 67:

TEXT

'C'

Output:

TEXT

C

25. GATE TRAP — char vs "char"

Remember from Lesson 1:

C

'A'

is a character constant.

But:

C

"A"

is a string literal.

You can perform integer-style character arithmetic with:

C

'A' + 1

But "A" is not the same kind of object/value.

We'll understand strings and pointers later.

26. FLOATING-POINT LITERAL TYPES

A useful C detail:

C

3.14

is a double literal by default.

C

3.14f

is a `float` literal.

Example:

```
C
float x = 3.14f;
```

`f` explicitly marks the literal as `float`.

For most beginner calculations this may not change the visible result, but it matters for type rules and precision.

27. EXPLICIT CASTING SYNTAX

General form:

```
C
(target_type) expression
```

Examples:

```
C
(int)3.9
```

Result:

```
TEXT
3
```

```
C
(float)5
```

Result conceptually:

```
TEXT
5.0
```

```
C
(double)7
```

Result conceptually:

```
TEXT
7.0
```

28. SOLVED EXAMPLE

C

```
float x = (int)3.9 + 0.5f;
```

First:

TEXT

```
(int)3.9  
= 3
```

Then:

TEXT

```
3 + 0.5
```

integer is converted for mixed arithmetic:

TEXT

```
3.0 + 0.5  
= 3.5
```

Therefore:

TEXT

```
x = 3.5
```

29. SOLVED EXAMPLE — NESTED CAST

C

```
int x = (int)(5.8 + 2.4);
```

First:

TEXT

```
5.8 + 2.4  
= 8.2
```

Then:

TEXT

```
(int)8.2  
= 8
```

Therefore:

TEXT

```
x = 8
```

30. SOLVED EXAMPLE — DIFFERENT CAST POSITION

Compare:

```
C
(int)5.8 + 2.4
```

First:

```
TEXT
(int)5.8
= 5
```

Then:

```
TEXT
5 + 2.4
= 7.4
```

So:

```
TEXT
7.4
```

But:

```
C
(int)(5.8 + 2.4)
```

becomes:

```
TEXT
8
```

Cast position changes the result.

31. GATE OUTPUT TRACING METHOD

Whenever you see mixed types:

Step 1

Identify the type of every operand.

Example:

```
C
5 / 2.0
```

TEXT

```
5      !'   i n t
2 . 0  !'   d o u b l e
```

Step 2

Determine conversions.

TEXT

```
5      !'   d o u b l e
```

Step 3

Perform operation.

TEXT

```
5.0 / 2.0 = 2.5
```

Step 4

Check destination type.

Example:

C

```
int x = 5 / 2.0;
```

Expression gives:

TEXT

```
2.5
```

Then assignment to `int`:

TEXT

```
2
```

Therefore:

TEXT

```
x = 2
```

32. VERY IMPORTANT EXAMPLE

C

```
int x = 5 / 2.0;
```

Students often stop at:

TEXT

2.5

But `x` is an `int`.

So:

TEXT

```
5 / 2.0
!' 2.5
!' assign to int
!' 2
```

Final:

TEXT

`x = 2`

33. ANOTHER IMPORTANT EXAMPLE

C

```
float x = 5 / 2 * 2.0f;
```

Evaluate left to right within multiplication/division precedence.

First:

TEXT

```
5 / 2
= 2
```

Then:

TEXT

```
2 * 2.0
= 4.0
```

Final:

TEXT

`x = 4.0`

Not `5.0`.

34. CHANGE ONE OPERAND

Now:

```
C
float x = 5.0f / 2 * 2;
```

First:

```
TEXT
5.0 / 2
= 2.5
```

Then:

```
TEXT
2.5 * 2
= 5.0
```

So:

```
TEXT
x = 5.0
```

One tiny type change completely changes the result.

35. GATE TRAP — LEFT-TO-RIGHT GROUPING

Consider:

```
C
int x = 10 / 4 * 4;
```

`/` and `*` have the same precedence and associate left-to-right.

So:

```
TEXT
10 / 4
= 2
```

Then:

```
TEXT
2 * 4
= 8
```

Not:

```
TEXT
10 / 16
```

and not `10`.

Result:

TEXT

8

36. FLOAT VERSION

C

```
float x = 10.0f / 4 * 4;
```

First:

TEXT

```
10.0 / 4  
= 2.5
```

Then:

TEXT

```
2.5 * 4  
= 10.0
```

So:

TEXT

```
x = 10.0
```

37. DON'T CONFUSE — CASTING VS CHANGING ORIGINAL VARIABLE

Suppose:

C

```
int x = 5;  
  
float y = (float)x;
```

This does **not** permanently change `x` into a float.

Afterward:

TEXT

```
x !'  s t i l l  i n t  
y !'  f l o a t
```

The cast produces a converted value for that expression.

Memory trick:

Casting changes the view/value used in the expression, not necessarily the declared type of the original variable.

38. EXAMPLE

```
C
int x = 5;

printf("%f", (double)x);
```

x itself remains:

```
TEXT
int
```

But the expression:

```
C
(double)x
```

produces a `double` value.

39. ASSIGNMENT FROM FLOAT TO INT — DATA LOSS

Example:

```
C
double salary = 12345.89;
int x = salary;
```

x becomes:

```
TEXT
12345
```

Fractional information is lost.

This is one reason narrowing conversions can be dangerous.

40. PLACEMENT CONNECTION

Type conversion matters heavily in coding interviews when computing:

Average

Bad:

```
C
int sum = 7;
int count = 2;

float avg = sum / count;
```

Result:

```
TEXT
3.0
```

not 3.5.

Correct:

```
C
float avg = (float)sum / count;
```

Result:

```
TEXT
3.5
```

41. PLACEMENT EXAMPLE — PERCENTAGE

Bad:

```
C
int obtained = 450;
int total = 500;

float percentage = obtained / total * 100;
```

First:

```
TEXT
450 / 500
= 0
```

integer division.

Then:

```
TEXT
0 * 100
= 0
```

Wrong result.

Correct:

```
C
float percentage = (float)obtained / total * 100;
```

Now:

```
TEXT
450.0 / 500
= 0.9
```

Then:

TEXT

```
0.9 * 100
= 90.0
```

42. INTERVIEW QUESTIONS

Q1. What is implicit type conversion?

Automatic conversion performed by the language when required by an expression or assignment.

Q2. What is explicit type conversion?

A conversion explicitly requested using a cast.

Example:

```
C
(float)x
```

Q3. Why does this give 2.0?

```
C
float x = 5 / 2;
```

Because `5 / 2` is evaluated as integer division first, yielding `2`, which is then converted to floating-point.

Q4. How do you force floating-point division?

Make at least one operand floating-point before division.

Example:

```
C
(float)a / b
```

Q5. What happens when 3.9 is converted to int?

It becomes `3` by discarding the fractional part toward zero.

Q6. Does `(float)x` permanently change the type of `x`?

No. It creates a converted value for that expression.

43. GATE TRAPS SUMMARY

Trap 1

```
C
float x = 5 / 2;
```

Result:

TEXT

2.0

Trap 2

C

```
(float)(5 / 2)
```

Result:

TEXT

2.0

Trap 3

C

```
(float)5 / 2
```

Result:

TEXT

2.5

Trap 4

C

```
int x = 5 / 2.0;
```

Expression:

TEXT

2.5

but final stored value:

TEXT

2

Trap 5

C

```
float x = 10 / 4 * 4;
```

Result:

TEXT

8.0

because:

TEXT

10 / 4 = 2

first.

44. MEMORY TRICKS

Integer division

TEXT

INT / INT

!' i n t e g e r d i v i s i o n

Floating division

TEXT

If one operand is floating BEFORE division

!' f l o a t i n g - p o i n t d i v i s i o n

Cast

Think:

CAST BEFORE the operation if you want the operation changed.

Bad:

C

(float)(5 / 2)

Damage already happened.

Good:

C

(float)5 / 2

Float !' Int

Think:

Decimal tail gets chopped.

TEXT

```
8 . 9 !' 8
```

45. FORMULA / RULE BOX

TEXT

```
int / int !' integer division
```

TEXT

```
(float)a / b !' floating-point division
```

TEXT

```
(float)(a / b)  
!' integer division happens first if a, b are int
```

TEXT

```
floating !' int  
!' fractional part discarded toward zero
```

TEXT

```
char/short arithmetic  
!' integer promotions generally occur first
```

TEXT

Cast does not change the declared type of the original variable.

46. NOTEBOOK NOTES

Type Conversion

Changing a value from one data type to another.

Implicit

Done automatically.

Example:

```
c  
int a = 5;  
float b = 2.5f;  
  
float c = a + b;
```

a is converted appropriately for the mixed expression.

Explicit Cast

Syntax:

```
C
(type)expression
```

Example:

```
C
(float)a
```

Integer Division

```
C
5 / 2
```

gives:

```
TEXT
2
```

Floating Division

```
C
5.0 / 2
```

gives:

```
TEXT
2.5
```

Important

```
C
float x = 5 / 2;
```

gives:

```
TEXT
2.0
```

because division happens before assignment.

Cast Position

```
c
(float)(5 / 2)
```

!' 2.0

```
c
(float)5 / 2
```

!' 2.5

Float to Int

```
c
(int)3.9
```

!' 3

Fractional part discarded toward zero.

Integer Promotion

Small integer types such as `char` and `short` are commonly promoted before arithmetic operations.

47. 30-SECOND REVISION

Remember these:

1. C can convert types automatically.
2. You can explicitly cast using `(type)`.
3. `5 / 2 = 2`.
4. `5.0 / 2 = 2.5`.
5. `float x = 5 / 2` gives `2.0`.
6. `(float)(5 / 2)` gives `2.0`.
7. `(float)5 / 2` gives `2.5`.
8. Converting `3.9` to `int` gives `3`.
9. Cast position matters.
10. `char` participates in integer arithmetic through integer promotions.

48. ACTIVE RECALL

Try before checking.

Q1

What is stored in `x`?

```
C
float x = 9 / 4;
```

Q2

What is stored?

```
C
float x = 9.0 / 4;
```

Q3

What is the value?

```
C
(float)(9 / 4)
```

Q4

What is the value?

```
C
(float)9 / 4
```

Q5

What is stored in `x`?

```
C
int x = 7.8;
```

ANSWERS

A1

```
TEXT
2.0
```

because:

```
TEXT
9 / 4 = 2
```

first.

A2

```
TEXT
2.25
```

A3

TEXT

2.0

A4

TEXT

2.25

A5

TEXT

7

49. PRACTICE

~~Q1~~ Q1

Find x :

C

```
double x = 8 / 3;
```

~~Q2~~ Q2

Find x :

C

```
double x = 8.0 / 3;
```

~~Q3~~ Q3 — GATE STYLE

Find x :

C

```
double x = (double)(8 / 3);
```

~~Q4~~ Q4 — GATE STYLE

Find x :

C

```
double x = (double)8 / 3;
```

Q=Q5

Find final value:

```
C
int x = 9.8 + 2.4;
```

Q=Q6

Trace:

```
C
int a = 5;
int b = 2;

double x = a / b + 0.5;
```

Find x.

Q=Q7 — CAST POSITION

Compare:

```
C
double a = (int)4.8 + 2.5;
```

and:

```
C
double b = (int)(4.8 + 2.5);
```

Find both.

Q=Q8 — GATE TRAP

What is stored?

```
C
float x = 5 / 2 * 2.0f;
```

Explain each step.

Q=Q9 — PLACEMENT THINKING

You have:

C

```
int sum = 17;  
int count = 4;
```

Write an expression that produces the accurate average:

TEXT

4.25

instead of:

TEXT

4

PROGRESS

Completed '

Module 0 / Lesson 1

- C program basics
- variables
- data types

Module 0 / Lesson 2

- operators
- precedence
- increment/decrement
- short-circuiting

Module 0 / Lesson 3

- implicit conversion
- explicit casting
- integer promotion
- mixed expressions
- assignment conversion
- division traps
- cast-position traps

Next

Module 0 / Lesson 4 — Input & Output in C

Next we will cover:

- `printf()`
- `scanf()`
- `%d`, `%f`, `%lf`, `%c`, `%s`
- why `scanf` usually needs `&`

- formatted output
- character input problems
- common GATE/output traps
- one of the first real connections to **memory addresses and pointers**

Type **NEXT**.

USER

NEXT

ASSISTANT

MODULE 0 — LESSON 4

Input & Output in C — printf() and scanf()

We continue according to your uploaded course rules: intuition !' technical concept !' tracing !' GATE traps !' placement use !' notebook notes !' recall !' practice.

1. WHERE WE ARE

Module: 0 — C Foundation **Lesson:** 4 **Current topic:** Input and Output in C

GATE importance: $\phi = \gamma\% \text{ NO-ST\% KNOW}$ Placement importance: $\phi = \gamma\% \text{ NO-ST\% KNOW}$

Today you should understand:

- `printf()`
- `scanf()`
- Format specifiers
- Why `scanf()` usually uses `&`
- `%d`, `%c`, `%f`, `%lf`, `%s`
- Formatting output
- Reading multiple values
- Character-input traps
- Return values of `printf/scanf` conceptually
- First connection to **addresses and pointers**

2. STORY / INTUITION

Imagine your C program is a person sitting inside a room.

It has two doors:

TEXT



If the program wants information from you:

```
C
scanf(...)
```

If the program wants to show information to you:

```
C
printf(...)
```

Simple memory trick:

scanf = take information in printf = send information out

3. TECHNICAL EXPLANATION

C provides formatted input/output functions through:

```
C
#include <stdio.h>
```

The two important functions today are:

```
C
printf()
scanf()
```

4. printf() — OUTPUT

Consider:

```
C
#include <stdio.h>

int main()
{
    printf("Hello");
    return 0;
}
```

```
}
```

Output:

```
TEXT
Hello
```

`printf()` means:

print formatted

It can print both ordinary text and values.

5. PRINTING A VARIABLE

```
C
int age = 20;

printf("%d", age);
```

Output:

```
TEXT
20
```

Here:

```
TEXT
%d
```

is a **format specifier**.

It tells `printf` how the supplied value should be interpreted/formatted.

6. BASIC FORMAT SPECIFIERS

MUST KNOW

Data	printf()
int	%d
char	%c
float / double value passed to printf	%f
string	%s

Example:

```
C
int age = 20;
char grade = 'A';
float marks = 92.5f;

printf("%d\n", age);
printf("%c\n", grade);
printf("%f\n", marks);
```

Output approximately:

```
TEXT
20
A
92.500000
```

7. WHY DOES `%f` PRINT MANY DECIMAL PLACES?

Consider:

```
C
float x = 3.5f;

printf("%f", x);
```

Typical output:

```
TEXT
3.500000
```

By default, `%f` displays six digits after the decimal point.

But you can control that.

8. FORMATTED DECIMAL OUTPUT

```
C
float x = 3.14159f;

printf("%.2f", x);
```

Output:

```
TEXT
3.14
```

Here:

```
TEXT
.2
```

means:

Print 2 digits after the decimal point.

Example

```
c
printf("%.3f", 5.67891);
```

Output:

```
TEXT
5.679
```

Notice that formatting performs rounding for display.

DON'T CONFUSE

Earlier:

```
c
(int)5.9
```

produces:

```
TEXT
5
```

because conversion to integer discards the fractional portion.

But:

```
c
printf("%.1f", 5.98);
```

may display:

```
TEXT
6.0
```

because formatted output rounds to the requested displayed precision.

These are different concepts.

9. PRINTING MULTIPLE VALUES

```
C
int a = 10;
int b = 20;

printf("%d %d", a, b);
```

Output:

```
TEXT
10 20
```

The correspondence is positional:

```
TEXT
F i r s t   % d   ! '   a
S e c o n d   % d   ! '   b
```

Example

```
C
int age = 20;
char grade = 'A';

printf("Age = %d, Grade = %c", age, grade);
```

Output:

```
TEXT
Age = 20, Grade = A
```

10. ESCAPE SEQUENCES

You will frequently see:

```
TEXT
\n
\t
```

Newline

```
C
printf("Hello\nWorld");
```

Output:

TEXT

Hello
World

Tab

C

```
printf("A\tB");
```

Produces tab-separated output.

Conceptually:

TEXT

A B

Other useful ones

Escape	Meaning
<code>\n</code>	newline
<code>\t</code>	horizontal tab
<code>\\</code>	backslash
<code>\"</code>	double quote
<code>\'</code>	single quote

11. PRINTING QUOTATION MARKS

Suppose you want:

TEXT

Hello "C"

Use:

C

```
printf("Hello \"C\"");
```

Why?

Because ordinary:

TEXT

"

already marks string boundaries.

So:

TEXT

\"

means:

Treat this quote as a character inside the string.

12. `scanf()` — INPUT

Now suppose we want the user to enter an integer.

```
C
int age;

scanf("%d", &age);
```

If user enters:

TEXT

20

then:

TEXT

age = 20

13. STORY — WHY DOES `scanf` NEED `&`?

This is your first major connection to pointers.

Imagine:

```
C
int age;
```

creates a box somewhere in memory.

Conceptually:

TEXT

```
Address 1000
+-----+
|   age   |
```

```
|   ?   |  
+-----+
```

If `scanf` has to place a value into that box, it needs to know:

Where exactly is the box?

So we give:

```
C  
&age
```

`&age` means:

Address of `age`

14. VERY IMPORTANT VISUALIZATION

Suppose:

```
C  
int age;
```

is located at address `1000`.

Then conceptually:

Expression	Meaning
<code>age</code>	value stored in <code>age</code>
<code>&age</code>	address of <code>age</code>

Initially:

```
TEXT  
Address 1000  
+-----+  
|   ?   |  
+-----+  
    age
```

Program:

```
C  
scanf("%d", &age);
```

User types:

TEXT

25

`scanf` knows where to put it:

TEXT

Address 1000

+-----+

| 25 |

+-----+

age

0-1111 prepares you for pointers.

15. WHY THIS WOULD BE WRONG

C

```
scanf("%d", age);
```

For `%d`, `scanf` expects a pointer to an appropriate integer object.

But `age` represents the value of the integer, not its address.

Correct:

C

```
scanf("%d", &age);
```

MEMORY TRICK

`printf`

Usually needs the **value**:

C

```
printf("%d", age);
```

`scanf`

Usually needs the **address**:

C

```
scanf("%d", &age);
```

Think:

TEXT

P R I N T ! ' t e l l m e t h e v a l u e .

SCAN !' tell me WHERE to store the value .

16. COMPLETE INPUT PROGRAM

```
C
#include <stdio.h>

int main()
{
    int age;

    printf("Enter age: ");
    scanf("%d", &age);

    printf("Age = %d", age);

    return 0;
}
```

Possible interaction:

```
TEXT
Enter age: 20
Age = 20
```

17. READING TWO INTEGERS

```
C
int a, b;

scanf("%d %d", &a, &b);
```

User can enter:

```
TEXT
10 20
```

Then:

```
TEXT
a = 10
b = 20
```

Full example

```
C
#include <stdio.h>
```

```
int main()
{
    int a, b;

    scanf("%d %d", &a, &b);

    printf("%d", a + b);

    return 0;
}
```

Input:

```
TEXT
5 7
```

Output:

```
TEXT
12
```

18. `scanf` FORMAT SPECIFIERS

This is where something important changes.

Variable type	<code>scanf()</code> format
int	%d
char	%c
float	%f
double	%lf
string/char array	%s

⚠️ Notice:

```
TEXT
float    !'   % f
double  !'   % l f
```

for `scanf`.

19. IMPORTANT `%f` / `%lf` DIFFERENCE

For `scanf`:

```
C
float x;
scanf("%f", &x);
```

Correct.

For:

```
c
double x;
scanf("%lf", &x);
```

Correct.

Do not interchange them casually.

20. BUT `printf` IS DIFFERENT

For ordinary `printf` floating output:

```
c
double x = 3.5;
printf("%f", x);
```

is the standard form.

Why this apparent difference?

Because when calling a variadic function like `printf`, `float` arguments undergo default argument promotions and are passed as `double`.

So for our practical notes:

`printf`

TEXT

```
f l o a t / d o u b l e ! ' % f
```

`scanf`

TEXT

```
f l o a t ! ' % f
d o u b l e ! ' % l f
```

Memorize this distinction.

21. FORMAT RULE BOX

`printf`

TEXT

```
i n t ! ' % d
c h a r ! ' % c
f l o a t ! ' % f
d o u b l e ! ' % f
s t r i n g ! ' % s
```

scanf

TEXT

```
i n t      !' % d
c h a r    !' % c
f l o a t  !' % f
d o u b l e !' % l f
s t r i n g !' % s
```

22. INPUTTING A FLOAT

C

```
float price;

scanf("%f", &price);

printf("%.2f", price);
```

Input:

TEXT

12.5

Output:

TEXT

12.50

23. INPUTTING A DOUBLE

C

```
double pi;

scanf("%lf", &pi);

printf("%f", pi);
```

Input:

TEXT

3.14159

Possible output:

TEXT

3.141590

24. INPUTTING A CHARACTER

```
C
char ch;

scanf("%c", &ch);
```

Input:

```
TEXT
A
```

Then:

```
TEXT
ch = 'A'
```

25. CHARACTER INPUT TRAP Ø=Ý%Ø=Ý%

Consider:

```
C
int x;
char ch;

scanf("%d", &x);
scanf("%c", &ch);
```

User enters:

```
TEXT
10
A
```

You might expect:

```
TEXT
x = 10
ch = A
```

But there can be a problem.

After reading the integer, the newline generated when the user presses Enter may remain in the input stream.

Then:

```
C
scanf("%c", &ch);
```

can read that newline.

So `ch` may receive:

TEXT

```
'\n'
```

instead of `'A'`.

26. COMMON FIX

Use:

C

```
scanf(" %c", &ch);
```

Notice the space before `%c`.

TEXT

```
" %c"  
^
```

Whitespace in the format causes `scanf` to skip whitespace characters before reading the character.

Example:

C

```
int x;  
char ch;  
  
scanf("%d", &x);  
scanf(" %c", &ch);
```

Now the leftover newline is skipped.

Very useful practical C knowledge.

27. DON'T CONFUSE `%c` WITH `%s`

Character:

C

```
char c;  
scanf(" %c", &c);
```

String:

C

```
char name[20];  
scanf("%19s", name);
```

We will study strings properly later.

Notice that with a character array used for `%s`, you normally don't write:

```
C
&name
```

in the same way.

Why?

Because array expressions often decay to pointers to their first element in function-call contexts.

Do not memorize this yet without understanding.

We'll explain it fully in:

Arrays + Pointers

28. `printf()` OUTPUT TRACING

Consider:

```
C
int a = 10;
int b = 20;

printf("%d %d", b, a);
```

Output:

```
TEXT
20 10
```

Why?

Format specifiers match arguments by their order.

```
TEXT
f i r s t   % d   ! '   b
s e c o n d   % d   ! '   a
```

29. SOLVED EXAMPLE — ARITHMETIC INSIDE `printf`

```
C
int a = 5;
int b = 3;

printf("%d", a + b * 2);
```

Precedence first:

```
TEXT
b * 2
= 3 * 2
= 6
```


Then:

```
TEXT
a + 6
= 11
```

Output:

```
TEXT
11
```

So `printf` doesn't magically change expression rules.

The expression is evaluated first.

30. SOLVED EXAMPLE — INTEGER DIVISION

```
C
printf("%d", 7 / 2);
```

Output:

```
TEXT
3
```

because integer division occurs.

31. SOLVED EXAMPLE — FLOATING OUTPUT

```
C
printf("%.2f", 7.0 / 2);
```

Calculate:

```
TEXT
7.0 / 2
= 3.5
```

Formatted to two decimal places:

```
TEXT
3.50
```

Output:

```
TEXT
3.50
```

32. `printf()` CAN RETURN A VALUE

This is useful for tricky C questions.

`printf()` returns the number of characters successfully written, if successful.

Example:

```
C
int x = printf("Hello");
```

`Hello` contains:

```
TEXT
5 characters
```

So ordinarily:

```
TEXT
x = 5
```

GATE-STYLE EXAMPLE

```
C
printf("%d", printf("Hi"));
```

Evaluate inner function:

```
C
printf("Hi")
```

prints:

```
TEXT
Hi
```

and returns:

```
TEXT
2
```

Then outer:

```
C
printf("%d", 2);
```

prints:

TEXT

2

Final output:

TEXT

Hi2

0-6% Good C-output question.

33. ANOTHER EXAMPLE

C

```
int x = printf("ABC\n");
```

The output includes:

TEXT

A

B

C

newline

Typically the count returned is:

TEXT

4

because `\n` corresponds to one character written at the C stream level.

34. `scanf()` ALSO RETURNS A VALUE

`scanf()` returns the number of input items successfully matched and assigned, or `E0F` if input failure occurs before the first conversion.

For our beginner GATE understanding:

C

```
int a, b;
```

```
int x = scanf("%d %d", &a, &b);
```

If the user successfully provides:

TEXT

10 20

then:

TEXT

```
x = 2
```

because two items were successfully assigned.

This can appear in C questions.

35. SOLVED EXAMPLE

C

```
int a;  
  
int x = scanf("%d", &a);
```

If user types a valid integer:

TEXT

```
50
```

then generally:

TEXT

```
a = 50  
x = 1
```

because one assignment succeeded.

36. GATE TRAP — FORMAT MISMATCH

Suppose:

C

```
int x = 10;  
  
printf("%f", x);
```

This is not a valid way to convert and print an integer as floating point.

`printf` expects an argument matching the format requirements.

Using incompatible format specifiers can produce **undefined behavior**.

Rule:

Format specifier must match the argument type expected by that conversion.

Correct:

C

```
printf("%d", x);
```

If you want a floating representation:

```
C
printf("%f", (double)x);
```

37. GATE TRAP — `scanf` FORMAT MISMATCH

Wrong:

```
C
double x;
scanf("%f", &x);
```

Correct:

```
C
double x;
scanf("%lf", &x);
```

Why?

Because `%f` in `scanf` expects:

```
TEXT
float *
```

while `%lf` expects:

```
TEXT
double *
```

These pointer types are different.

38. FIRST POINTER CONNECTION

Look at:

```
C
scanf("%d", &x);
```

We already know:

```
TEXT
x
```

means roughly:

value stored in x.

And:

TEXT

&x

means:

address of x.

Soon we'll create a separate variable that can store that address:

C

```
int *p = &x;
```

Then:

TEXT

p

will contain the address of `x`.

This is the beginning of pointers.

Don't rush it yet.

39. MEMORY VISUALIZATION

Suppose:

C

```
int x = 10;
```

and imagine `x` is stored at address `2000`.

TEXT

Address	Value
---------	-------

2000	10
------	----

	x
--	---

!'

Then:

TEXT

x	!'	1	0
& x	!'	2	0 0 0

Later:

C

```
int *p = &x;
```

means conceptually:

TEXT

p !' 2 0 0 0

And:

C

*p

will access:

TEXT

10

Keep this mental picture.

40. OUTPUT FORMATTING — WIDTH

You may encounter:

C

```
printf("%5d", 25);
```

5 indicates a minimum field width.

Conceptually output:

TEXT

25

There are spaces before 25 so the field occupies at least 5 characters.

Example

C

```
printf("%05d", 25);
```

Output:

TEXT

00025

0 requests zero-padding.

Useful to recognize, but not our highest-priority topic.

41. OUTPUT PRECISION

For floating point:

```
C
printf("%.2f", 3.14159);
```

Output:

```
TEXT
3.14
```

Another

```
C
printf("%.0f", 3.7);
```

Displays zero digits after decimal, with rounding according to formatting:

```
TEXT
4
```

42. READING INPUT WITH SPACES

Suppose:

```
C
char name[20];

scanf("%19s", name);
```

and user enters:

```
TEXT
Siri Chandana
```

`%s` typically stops at whitespace.

So only:

```
TEXT
Siri
```

would be read by that conversion.

~~0=16~~ This is important when processing strings.

Later we'll study safer line input approaches such as `fgets()`.

43. COMMON BEGINNER MISTAKE


```
c
int a, b;

scanf("%d %d", a, b);
```

Wrong.

Correct:

```
c
scanf("%d %d", &a, &b);
```

because `scanf` needs addresses for those conversions.

44. COMMON BEGINNER MISTAKE

Wrong:

```
c
printf("%d", &x);
```

if your goal is to print the integer's value.

`&x` is the address.

Correct value:

```
c
printf("%d", x);
```

Printing addresses correctly is another topic and commonly uses `%p`.

45. PRINTING AN ADDRESS

If you want to display an address, C provides:

```
c
%p
```

Example:

```
c
int x = 10;

printf("%p", (void *)&x);
```

It may display something implementation-specific such as:

```
TEXT
0x7ffd...
```

Do not memorize the address.

Addresses can vary between executions/environments.

`0= %p` is the appropriate conversion for a `void *` pointer.

46. SOLVED EXAMPLE — COMPLETE TRACE

```
c
#include <stdio.h>

int main()
{
    int a;
    int b;

    scanf("%d %d", &a, &b);

    int c = a + b;

    printf("%d", c);

    return 0;
}
```

Input:

```
TEXT
7 8
```

Trace:

Step	a	b	c
declaration	?	?	—
scanf	7	8	—
c = a+b	7	8	15

Output:

```
TEXT
15
```

47. SOLVED EXAMPLE — AVERAGE

```
c
int a, b;

scanf("%d %d", &a, &b);

float avg = (float)(a + b) / 2;
```

```
printf("%.2f", avg);
```

Input:

```
TEXT
5 8
```

Calculate:

```
TEXT
a + b = 13
```

Cast:

```
TEXT
1 3 !' 1 3 . 0
```

Then:

```
TEXT
13.0 / 2 = 6.5
```

Output:

```
TEXT
6.50
```

48. GATE FOCUS

For input/output, focus on:

Ø=Format specifier matching Ø=scanf() requiring addresses Ø=%f vs %lf in scanf Ø=C character/newline behavior
Ø=Expression evaluation inside printf +fprintf return value +fscanf return value +F format-width/precision
questions

49. GATE TRAPS

Trap 1

```
C
float x;
scanf("%lf", &x);
```

Wrong format.

Use:

```
C
scanf("%f", &x);
```

Trap 2

```
C
double x;
scanf("%f", &x);
```

Wrong.

Use:

```
C
scanf("%lf", &x);
```

Trap 3

```
C
scanf("%d", x);
```

Wrong for an ordinary `int x`.

Use:

```
C
scanf("%d", &x);
```

Trap 4

```
C
printf("%d", 5 / 2);
```

prints:

```
TEXT
2
```

Trap 5

```
C
printf("%.1f", 5 / 2);
```

This is **not** simply a valid way to get `2.0`, because `5/2` has type `int` while `%f` requires a floating argument of the appropriate promoted type.

Correct approach:

```
C
printf("%.1f", (double)(5 / 2));
```

!' 2.0

or:

```
C
printf("%.1f", 5.0 / 2);
```

!' 2.5

Ø=Not matching the format matters.

50. PLACEMENT CONNECTION

In coding rounds, input/output may not be the hard part, but incorrect input handling can make a correct algorithm fail.

You should comfortably know:

```
C
scanf("%d", &n);
```

```
C
scanf("%d %d", &a, &b);
```

```
C
scanf("%f", &f);
```

```
C
scanf("%lf", &d);
```

and later:

```
C
fgets(...)
```

for strings/lines.

But placement success will increasingly depend on the algorithm, not complicated I/O tricks.

51. INTERVIEW QUESTIONS

Q1. What does `printf()` do?

Performs formatted output.

Q2. What does `scanf()` do?

Performs formatted input.

Q3. Why do we use `&x` with `scanf("%d", &x)` ?

Because `scanf` needs the address of the object where the input value should be stored.

Q4. Difference between `%f` and `%lf` in `scanf` ?

`%f` expects `float *`.

`%lf` expects `double *`.

Q5. Why doesn't `printf` normally use `&x` to print `x` ?

Because it needs the value being printed, not the address where `x` is stored.

Q6. What does `%c` read?

One character. Unlike many numeric conversions, it does not automatically skip leading whitespace unless whitespace is included in the format.

52. MEMORY TRICKS

I/O

TEXT

```
scanf  = IN
printf = OUT
```

Address

TEXT

`&` = address of

So:

C

`&x`

means:

TEXT

address of `x`

Scanf

Remember:

scanf needs the house address to deliver the value.

The variable is the house.

`&variable` is its address.

`%f / %lf`

Remember for **scanf**:

TEXT

```
f l o a t    !'   f
d o u b l e !'   l f
```

53. NOTEBOOK NOTES

Standard I/O

Use:

```
c
#include <stdio.h>
```

Output

```
c
printf()
```

Example:

```
c
int x = 10;
printf("%d", x);
```

Input

```
c
scanf()
```

Example:

```
c
int x;
scanf("%d", &x);
```

`&x` gives the address of `x`.

Common Formats

printf

TEXT

```
% d !' i n t
% c !' c h a r
% f !' f l o a t i n g - p o i n t o u t p u t
% s !' s t r i n g
% p !' p o i n t e r / a d d r e s s
```

scanf

TEXT

```
% d !' i n t *
% c !' c h a r *
% f !' f l o a t *
% l f !' d o u b l e *
% s !' c h a r a c t e r a r r a y
```

Character Input Trap

After numeric input:

```
C
scanf(" %c", &ch);
```

is commonly used so leading whitespace/newline is skipped.

Precision

```
C
printf("%.2f", x);
```

prints 2 digits after decimal.

Important

TEXT

```
p r i n t f !' g e n e r a l l y r e c e i v e s v a l u e s
s c a n f !' g e n e r a l l y r e c e i v e s a d d r e s s e s
```

54. 30-SECOND REVISION

Remember:

1. `printf` = output.
2. `scanf` = input.
3. `scanf("%d", &x)` needs `&x`.
4. `&x` means address of `x`.
5. `printf("%d", x)` prints the integer value.

6. `scanf: float, double. !'` `%lf`
7. `printf`: ordinary floating output uses `%f`.
8. `" %c"` can skip leftover whitespace.
9. `%.2f` displays two digits after the decimal.
10. `printf()` itself returns the number of characters written on success.

55. ACTIVE RECALL

Try first.

Q1

What does `&x` mean?

Q2

Which is correct for:

```
C
float x;
```

?

```
C
scanf("%f", &x);
```

or

```
C
scanf("%lf", &x);
```

Q3

Which is correct for:

```
C
double x;
```

?

Q4

What does this typically print?

```
C
printf("%.2f", 3.14159);
```

Q5

What is the output?

```
C
printf("%d", printf("Hi"));
```

ANSWERS

A1

Address of `x`.

A2

```
C
scanf("%f", &x);
```

A3

```
C
scanf("%lf", &x);
```

A4

```
TEXT
3.14
```

A5

```
TEXT
Hi2
```

The inner call prints `Hi` and returns `2`.

56. PRACTICE

~~Q1~~ Q1

Input:

```
TEXT
10 30
```

Program:

```
C
int a, b;

scanf("%d %d", &a, &b);

printf("%d", a + b);
```

What is the output?

~~Q=Q2~~

What does this print?

```
C
double x = 5.6789;

printf("%.2f", x);
```

~~Q=Q3~~ — GATE FOUNDATION

Find the output:

```
C
printf("%d", 9 / 2);
```

~~Q=Q4~~ — GATE OUTPUT

Find the output:

```
C
printf("%d", printf("GATE"));
```

~~Q=Q5~~ — INPUT TRAP

Why can this behave unexpectedly?

```
C
int x;
char ch;

scanf("%d", &x);
scanf("%c", &ch);
```

How would you usually modify the second line?

~~Q=Q6~~ — FORMAT QUESTION

Which format specifier should be used?

```
C
double value;
scanf(____, &value);
```

Q7 — MEMORY THINKING

Suppose:

```
C
int x = 50;
```

and imagine it is stored at address 1000 .

What would you conceptually say are:

```
TEXT
x
&x
```

?

Q8 — GATE STYLE

What is the output?

```
C
#include <stdio.h>

int main()
{
    int x = printf("AB");
    printf("%d", x);

    return 0;
}
```

Q9 — PLACEMENT FOUNDATION

Write the essential C statements to:

1. read two integers `a` and `b` ,
2. calculate their accurate floating-point average,
3. print the average to 2 decimal places.

Expected behavior for:

```
TEXT
5 8
```

should display:

```
TEXT
6.50
```

PROGRESS

Completed '

Module 0 / Lesson 1

- C execution basics
- variables
- basic data types

Module 0 / Lesson 2

- operators
- precedence
- logical operations
- increment/decrement
- short circuiting

Module 0 / Lesson 3

- type conversion
- casting
- integer promotions
- mixed expressions

Module 0 / Lesson 4

- `printf`
- `scanf`
- format specifiers
- formatted output
- addresses with `scanf`
- character-input trap
- `printf/scanf` return-value foundations
- first pointer connection

Next

Module 0 / Lesson 5 — Decision Making in C

Next we move from individual expressions to **controlling program flow**:

TEXT

```
if
if-else
else-if ladder
nested if
switch
```

We will also cover several classic GATE traps:

C

```
if (x = 5)
```

VS

- `if`
- `if-else`
- `else-if` ladder
- nested `if`
- truth values in C
- `=` vs `==`
- dangling `else`
- short-circuiting inside conditions
- `switch`
- `case`
- `break`
- `default`
- fall-through
- important GATE traps

Imagine a security guard standing at a gate.

He checks:

Do you have an ID card?

If yes:

TEXT

Allow entry

If no:

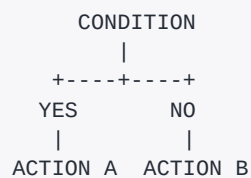
TEXT

Reject entry

That's exactly what an **if-else** does.

Conceptually:

TEXT



Example:

```
c
if (age >= 18)
    printf("Adult");
else
    printf("Minor");
```

The program asks a question and selects a path.

3. **if** STATEMENT

Basic syntax:

```
c
if (condition)
{
    statements;
}
```

Example:

```
c
int marks = 80;

if (marks >= 40)
```

```
{  
    printf("Pass");  
}
```

Since:

```
TEXT  
80 >= 40
```

is true, output is:

```
TEXT  
Pass
```

4. C CONDITION RULE Ø=Ý%

Remember from Operators:

```
TEXT  
Ø          !'   f a l s e  
n o n z e r o !'   t r u e
```

So all of these are true:

```
C  
if (1)
```

```
C  
if (5)
```

```
C  
if (-10)
```

But this is false:

```
C  
if (0)
```

5. EXAMPLE

```
C  
int x = -5;  
  
if (x)  
{  
    printf("True");  
}
```


Since `x` is nonzero:

TEXT

```
- 5 != true
```

Output:

TEXT

True

NOTE may test this.

6. if-else

Syntax:

```
C
if (condition)
{
    statements_if_true;
}
else
{
    statements_if_false;
}
```

Example:

```
C
int n = 7;

if (n % 2 == 0)
{
    printf("Even");
}
else
{
    printf("Odd");
}
```

Calculate:

TEXT

```
7 % 2 = 1
```

Condition:

TEXT

```
1 == 0 != false
```

Therefore output:

TEXT

Odd

7. STORY — TWO DOORS

Think:

TEXT

```
      n % 2 == 0 ?
     /         \
    YES         NO
    /           \
  "Even"       "Odd"
```

Only one branch executes.

8. **else-if** LADDER

Suppose marks are:

TEXT

```
90 + 'A'
75 + 'B'
60 + 'C'
otherwise 'D'
```

Use:

```
C
if (marks >= 90)
{
    printf("A");
}
else if (marks >= 75)
{
    printf("B");
}
else if (marks >= 60)
{
    printf("C");
}
else
{
    printf("D");
}
```

9. HOW **else-if** WORKS

Suppose:

TEXT

```
marks = 82
```

First:

TEXT

```
82 >= 90 !' false
```

Next:

TEXT

```
82 >= 75 !' true
```

So:

TEXT

```
B
```

is printed.

The remaining branches are skipped.

Important:

In an if / else if / else chain, once one condition succeeds, later branches are not tested.

10. COMMON BEGINNER MISTAKE

Suppose you write independent if s:

C

```
if (marks >= 60)
    printf("C");
```

```
if (marks >= 75)
    printf("B");
```

```
if (marks >= 90)
    printf("A");
```

If:

TEXT

```
marks = 95
```

all three conditions are true.

Output:

TEXT

```
CBA
```

That may not be what you intended.

DON'T CONFUSE

Separate `if`s

Multiple conditions may execute.

```
C
if (...)
if (...)
if (...)
```

`if / else if / else`

Only one branch of the chain executes.

```
C
if (...)
else if (...)
else if (...)
else
```

11. NESTED `if`

An `if` can exist inside another `if`.

Example:

```
C
int age = 20;
int hasID = 1;

if (age >= 18)
{
    if (hasID)
    {
        printf("Allowed");
    }
}
```

First:

```
TEXT
age >= 18
```

must be true.

Then:

TEXT
hasID

must be true.

Only then output:

TEXT
Allowed

12. VISUALIZATION

TEXT

```
age >= 18 ?
|
YES
|
hasID ?
|
YES
|
Allowed
```

This is a nested decision.

13. SAME IDEA WITH LOGICAL AND

Instead of:

```
c
if (age >= 18)
{
    if (hasID)
    {
        printf("Allowed");
    }
}
```

you can often write:

```
c
if (age >= 18 && hasID)
{
    printf("Allowed");
}
```

Same logical requirement.

14. GATE CLASSIC — `=` VS `==` `Ø=Ý%Ø=Ý%Ø=Ý%`

Consider:

```
c
int x = 5;

if (x == 10)
{
    printf("Yes");
}
```

`==` means comparison.

Since:

```
TEXT
5 == 10 !' false
```

Nothing prints.

Now:

```
c
int x = 5;

if (x = 10)
{
    printf("Yes");
}
```

This is completely different.

```
c
x = 10
```

assigns `10` to `x`.

The assignment expression evaluates to `10`.

Since:

```
TEXT
10 !' nonzero !' true
```

the condition succeeds.

Output:

```
TEXT
Yes
```

And now:

```
TEXT
x = 10
```

~~One~~ One of the most famous C traps.

MEMORY TRICK

TEXT

```
=    !'   G I V E
= =  !'   A S K
```

C

```
x = 5;
```

Give 5 to x.

C

```
x == 5
```

Ask whether x equals 5.

15. EVEN TRICKIER VERSION

C

```
int x = 5;

if (x = 0)
{
    printf("A");
}
else
{
    printf("B");
}
```

Assignment:

TEXT

```
x = 0
```

The condition evaluates to:

TEXT

```
0    !'   f a l s e
```

Therefore:

TEXT

```
B
```

prints.

Final:

TEXT

```
x = 0
```

16. EMPTY STATEMENT

Consider:

```
C
int x = 5;

if (x > 0);
{
    printf("Hello");
}
```

Notice:

```
C
if (x > 0);
```

There is a semicolon immediately after `if`.

That semicolon represents an empty statement.

So the `if` controls only:

```
TEXT
;
```

The following block:

```
C
{
    printf("Hello");
}
```

executes independently.

Therefore `Hello` prints regardless of the condition.

Example

```
C
int x = -5;

if (x > 0);
{
    printf("Hello");
}
```

Output:

TEXT

Hello

Very common exam trap.

MEMORY TRICK

Never casually write:

```
C
if (condition);
```

unless you genuinely intend an empty body.

17. BRACES MATTER

Consider:

```
C
if (x > 0)
    printf("A");
    printf("B");
```

Indentation does **not** control C syntax.

Without braces, only the next statement belongs to `if`.

This is interpreted as:

```
C
if (x > 0)
    printf("A");

printf("B");
```

So `B` always prints.

Example

If:

```
TEXT
x = -1
```

then:

```
TEXT
A !'  s k i p p e d
B !'  p r i n t e d
```

Output:

```
TEXT
B
```

Do not use braces, not indentation.

18. BEST PRACTICE

Prefer:

```
c
if (x > 0)
{
    printf("A");
    printf("B");
}
```

especially for placement coding and interview code.

It prevents mistakes.

19. DANGLING `else`

This is an important GATE concept.

Consider:

```
c
if (a)
    if (b)
        printf("X");
    else
        printf("Y");
```

Which `if` does the `else` belong to?

Rule:

An `else` is associated with the nearest unmatched `if`.

So it behaves like:

```
c
if (a)
{
    if (b)
        printf("X");
    else
        printf("Y");
}
```

The `else` belongs to:

```
c
if (b)
```

not:

```
c
if (a)
```

20. DANGLING `else` EXAMPLE

```
c
int a = 1;
int b = 0;

if (a)
    if (b)
        printf("X");
    else
        printf("Y");
```

Trace:

```
TEXT
a  =  1  !'  t r u e
```

Then:

```
TEXT
b  =  0  !'  f a l s e
```

So inner `else` executes.

Output:

```
TEXT
Y
```

21. BRACES REMOVE AMBIGUITY

If you want the `else` to belong to the outer `if`, use braces:

```
c
if (a)
{
    if (b)
        printf("X");
}
else
{
    printf("Y");
}
```

Now structure is clear.

22. SHORT-CIRCUIT INSIDE `if`

Consider:

```
C
int x = 0;
int y = 10;

if (x != 0 && y / x > 2)
{
    printf("Yes");
}
```

First condition:

```
TEXT
x != 0
0 != 0 !' false
```

Because of `&&`, C skips:

```
C
y / x > 2
```

Therefore division by zero is avoided.

23. SIDE-EFFECT EXAMPLE

```
C
int x = 0;
int y = 5;

if (x && ++y)
{
    printf("A");
}

printf("%d", y);
```

Since:

```
TEXT
x = 0 !' false
```

`++y` is not evaluated.

Therefore:

```
TEXT
y = 5
```

Output:

```
TEXT
5
```

24. OR SHORT-CIRCUIT

```
C
int x = 1;
int y = 5;

if (x || ++y)
{
    printf("%d", y);
}
```

`x` is true.

For:

```
TEXT
true || anything
```

the second operand is skipped.

So:

```
TEXT
y = 5
```

Output:

```
TEXT
5
```

25. RANGE CHECKING

Wrong mathematical-style thinking:

```
C
if (1 < x < 10)
```

This does not mean:

```
TEXT
1 < x AND x < 10
```

Correct:

```
C
if (x > 1 && x < 10)
```

~~0=100~~ Keep using logical AND.

26. EXAMPLE — LARGEST OF TWO NUMBERS

```
C
int a = 10;
int b = 20;

if (a > b)
{
    printf("%d", a);
}
else
{
    printf("%d", b);
}
```

Output:

```
TEXT
20
```

27. EXAMPLE — LARGEST OF THREE

```
C
int a = 10;
int b = 40;
int c = 25;

if (a >= b && a >= c)
{
    printf("%d", a);
}
else if (b >= a && b >= c)
{
    printf("%d", b);
}
else
{
    printf("%d", c);
}
```

Output:

```
TEXT
40
```

~~0=100~~ This type of conditional reasoning appears constantly in placements.

28. TERNARY CONNECTION

From Lesson 2:

```
C
condition ? true_value : false_value
```

Example:

```
C
int max = (a > b) ? a : b;
```

This can replace a simple:

```
C
if (a > b)
    max = a;
else
    max = b;
```

But for complex logic, `if-else` is usually clearer.

29. NOW — `switch`

Suppose you are building a menu:

```
TEXT
1  !'   A d d
2  !'   D e l e t e
3  !'   S e a r c h
4  !'   E x i t
```

Instead of many equality checks:

```
C
if (choice == 1)
...
else if (choice == 2)
...
else if (choice == 3)
...
...
```

you can use:

```
C
switch (choice)
```

30. BASIC `switch` SYNTAX

```
C
switch (expression)
{
    case value1:
        statements;
        break;

    case value2:
        statements;
        break;

    default:
        statements;
}
```

31. EXAMPLE

```
C
int choice = 2;

switch (choice)
{
    case 1:
        printf("Add");
        break;

    case 2:
        printf("Delete");
        break;

    case 3:
        printf("Search");
        break;

    default:
        printf("Invalid");
}
```

Since:

```
TEXT
choice = 2
```

C jumps to:

```
C
case 2:
```

Output:

```
TEXT
Delete
```


32. STORY / INTUITION FOR `switch`

Imagine a lift.

You press:

TEXT

3

The lift selects:

TEXT

Floor 3

Similarly:

TEXT

`switch(choice)`

checks the value and jumps to a matching `case` .

33. ROLE OF `break`

Consider:

```
c
switch (x)
{
    case 1:
        printf("A");
        break;

    case 2:
        printf("B");
        break;
}
```

`break` exits the switch.

Without `break` , execution may continue into subsequent cases.

This is called:

Fall-through

34. SWITCH FALL-THROUGH

Example:

```
c
int x = 1;

switch (x)
```

```
{
    case 1:
        printf("A");

    case 2:
        printf("B");

    case 3:
        printf("C");
}
```

`x = 1.`

Execution begins at:

```
C
case 1:
```

prints:

```
TEXT
A
```

No `break`.

Continue into:

```
C
case 2:
```

prints:

```
TEXT
B
```

No `break`.

Continue into:

```
C
case 3:
```

prints:

```
TEXT
C
```

Final output:

```
TEXT
ABC
```

0-766 famous GATE trap.

35. FALL-THROUGH WITH BREAK LATER

```
C
int x = 2;

switch (x)
{
    case 1:
        printf("A");

    case 2:
        printf("B");

    case 3:
        printf("C");
        break;

    case 4:
        printf("D");
}
```

Start at case 2:

```
TEXT
B
```

fall through:

```
TEXT
C
```

then:

```
C
break;
```

Final:

```
TEXT
BC
```

36. default

`default` executes if no case matches.

Example:

```
C
int x = 9;

switch (x)
{
    case 1:
        printf("One");
        break;
```

```
    case 2:
        printf("Two");
        break;

    default:
        printf("Other");
}
```

Output:

```
TEXT
Other
```

37. IS `default` MANDATORY?

No.

This is valid:

```
c
switch (x)
{
    case 1:
        printf("One");
        break;

    case 2:
        printf("Two");
        break;
}
```

If nothing matches, nothing inside executes.

38. DOES `default` HAVE TO BE LAST?

No.

For example:

```
c
switch (x)
{
    default:
        printf("D");
        break;

    case 1:
        printf("A");
        break;
}
```

is syntactically valid.

However, putting `default` last is usually clearer.

39. IMPORTANT SWITCH RULES

The controlling expression must be of integer, character, or enumeration type after the relevant promotions.

Common examples:

```
C
int
char
enum
```

Not floating-point cases like:

```
C
switch (3.5)
```

for standard C.

40. CASE LABELS

Case labels must be integer constant expressions.

Good:

```
C
case 1:
```

```
C
case 'A':
```

Bad conceptually:

```
C
case x:
```

where `x` is an ordinary runtime variable.

Case values must be compile-time constant integer expressions.

41. DUPLICATE CASE VALUES — INVALID

This is not valid:

```
C
switch (x)
{
    case 1:
        printf("A");
        break;

    case 1:
```

```
        printf("B");  
        break;  
    }  
}
```

Duplicate case values are not allowed.

42. CHARACTER SWITCH

```
c  
char grade = 'A';  
  
switch (grade)  
{  
    case 'A':  
        printf("Excellent");  
        break;  
  
    case 'B':  
        printf("Good");  
        break;  
  
    default:  
        printf("Other");  
}
```

Output:

```
TEXT  
Excellent
```

43. FALL-THROUGH CAN BE INTENTIONAL

Example:

```
c  
char ch = 'a';  
  
switch (ch)  
{  
    case 'a':  
    case 'e':  
    case 'i':  
    case 'o':  
    case 'u':  
        printf("Vowel");  
        break;  
  
    default:  
        printf("Not vowel");  
}
```

Here several cases intentionally fall into the same code.

So fall-through is not always a mistake.

44. switch VS if-else

Use `switch` when you compare one expression against multiple discrete constant values.

Example:

```
TEXT
choice == 1
choice == 2
choice == 3
```

Use `if` when conditions involve:

```
TEXT
ranges
multiple variables
logical combinations
```

Example:

```
C
if (marks >= 90)
```

or:

```
C
if (age >= 18 && hasID)
```

COMPARISON

Feature	if-else	switch
ranges	'	'L direct range cases
logical conditions	'	'L
equality to constants	'	'
many menu choices	okay	often cleaner
fall-through	no	yes

45. SOLVED EXAMPLE — if

```
C
int x = 0;

if (x)
    printf("A");
else
    printf("B");
```

`x = 0.`

False.

Output:

TEXT

B

46. SOLVED EXAMPLE — NONZERO

```
C
int x = -5;

if (x)
    printf("A");
else
    printf("B");
```

`-5` is nonzero.

True.

Output:

TEXT

A

47. SOLVED EXAMPLE — ASSIGNMENT TRAP

```
C
int x = 5;

if (x = 2)
    printf("%d", x);
```

Assignment:

TEXT

`x = 2`

Expression value:

TEXT

`2 != true`

Output:

TEXT

2

48. SOLVED EXAMPLE — ZERO ASSIGNMENT

```
C
int x = 5;

if (x = 0)
    printf("A");
else
    printf("B");

printf("%d", x);
```

Assignment:

```
TEXT
x = 0
```

Condition false.

So:

```
TEXT
B
```

Then:

```
TEXT
x = 0
```

Final combined output:

```
TEXT
B0
```

49. SOLVED EXAMPLE — DANGLING ELSE

```
C
int a = 1;
int b = 0;

if (a)
    if (b)
        printf("X");
    else
        printf("Y");
```

Outer:

```
TEXT
a = 1 !' t r u e
```

Inner:

TEXT

b = 0 !' f a l s e

else belongs to inner if.

Output:

TEXT

Y

50. SOLVED EXAMPLE — SWITCH

```
c
int x = 2;

switch (x)
{
    case 1:
        printf("A");
        break;

    case 2:
        printf("B");

    case 3:
        printf("C");
        break;

    default:
        printf("D");
}
```

Start:

TEXT

case 2

Print:

TEXT

B

No break.

Fall through:

TEXT

C

Then break.

Output:

TEXT

BC

51. GATE FOCUS

Most high-yield topics:

- `0` vs nonzero truth values
- `=` vs `==`
- semicolon after `if`
- braces and statement association
- dangling `else`
- short-circuit evaluation
- switch fall-through
- missing `break`
- matching `case`
- `default`
- nested condition tracing

These are classic program-output areas.

52. GATE TRAP SHEET

Trap 1

```
C
if (x = 5)
```

Assignment.

Not equality comparison.

Trap 2

```
C
if (x);
```

The body is an empty statement.

Trap 3

```
C
if (x)
    A;
```

```
B;
```

Only A belongs to `if`.

Trap 4

TEXT

```
else !' nearest unmatched if
```

Trap 5

Missing `break` in `switch`:

TEXT

fall-through

Trap 6

C

```
if (-5)
```

is true.

Trap 7

C

```
if (0)
```

is false.

53. PLACEMENT CONNECTION

Decision making appears in essentially every coding problem.

Examples:

Maximum

C

```
if (a > b)
```

Search validation

```
c
if (arr[i] == target)
```

Array bounds

```
c
if (index >= 0 && index < n)
```

Pointer safety

Later:

```
c
if (p != NULL && p->next != NULL)
```

Graph traversal

Later:

```
c
if (!visited[v])
```

Binary Search

Later:

```
c
if (target == arr[mid])
else if (target < arr[mid])
else
```

So decision statements are foundational for DSA.

54. INTERVIEW QUESTIONS

Q1. What values are treated as false in C?

0 is false; nonzero scalar values are treated as true in conditions.

Q2. Difference between = and ==?

= assigns.

== compares equality.

Q3. What is a dangling else?

An else without explicit braces associates with the nearest unmatched if.

Q4. Why is break used in switch?

To exit the switch and prevent unwanted fall-through.

Q5. Is default compulsory?

No.

Q6. Can `switch` directly use floating-point case selection?

Not in standard C; the controlling expression is based on integer/enumeration types.

Q7. Can multiple cases share one body?

Yes, by intentional fall-through.

55. MEMORY TRICKS

Truth

TEXT

```
0 = OFF
nonzero = ON
```

Assignment / Comparison

TEXT

```
=    !'   G I V E
= =  !'   A S K
```

Dangling Else

Remember:

Else loves the nearest lonely if.

Meaning:

TEXT

```
e l s e   ! '   n e a r e s t   u n m a t c h e d   i f
```

Switch

Think:

TEXT

```
M A T C H   ! '   E N T E R   ! '   K E E P   G O I N G   ! '   B R E A K
```

Without `break`:

TEXT

```
KEEP GOING
```

56. NOTEBOOK NOTES

if

```
C
if (condition)
{
    statements;
}
```

In C:

```
TEXT
0 = false
nonzero = true
```

if-else

```
C
if (condition)
{
    ...
}
else
{
    ...
}
```

else-if

```
C
if (c1)
{
}
else if (c2)
{
}
else
{
}
```

Only the first successful branch executes.

Important traps

```
C
if (x = 5)
```

assignment.

```
c
if (x == 5)
```

comparison.

```
c
if (x);
```

empty `if` body.

Without braces, only the next statement belongs to `if`.

Dangling `else` attaches to nearest unmatched `if`.

switch

```
c
switch (x)
{
    case 1:
        ...
        break;

    case 2:
        ...
        break;

    default:
        ...
}
```

Missing `break` may cause fall-through.

Case labels must be integer constant expressions.

57. 30-SECOND REVISION

Remember these 10:

1. `0` = false.
2. Nonzero = true.
3. `=` assigns.
4. `==` compares.
5. `if(condition);` has an empty body.
6. Without braces, only the next statement belongs to `if`.
7. `else` binds to the nearest unmatched `if`.
8. `&&` and `||` short-circuit.
9. Missing `break` in `switch` causes fall-through.
10. `switch` is best for discrete constant choices.

58. ACTIVE RECALL

Try before checking answers.

Q1

What happens?

```
c
int x = 0;

if (x)
    printf("A");
else
    printf("B");
```

Q2

What happens?

```
c
int x = 5;

if (x = 0)
    printf("A");
else
    printf("B");
```

Q3

Which `if` gets this `else`?

```
c
if (a)
    if (b)
        A;
    else
        B;
```

Q4

What happens without `break` inside `switch`?

Q5

What is wrong here?

```
c
if (x > 0);
{
    printf("Positive");
}
```

ANSWERS

A1

`x = 0` !' false.

Output:

TEXT

B

A2

`x = 0` is an assignment.

Condition becomes false.

Output:

TEXT

B

Final `x = 0`.

A3

The `else` belongs to:

C

`if (b)`

the nearest unmatched `if`.

A4

Execution can continue into subsequent cases: fall-through.

A5

The semicolon ends the `if` body as an empty statement, so the following block executes independently.

59. PRACTICE

Q1

Find output:

C

```
int x = 10;
```

```
if (x > 5)
    printf("Yes");
```

```
else
    printf("No");
```

Q2

Find output:

```
c
int x = -2;

if (x)
    printf("A");
else
    printf("B");
```

Q3 — GATE STYLE

```
c
int x = 5;

if (x = 3)
    printf("%d", x);
else
    printf("No");
```

Find output and final `x`.

Q4 — BRACES TRAP

```
c
int x = 0;

if (x)
    printf("A");
    printf("B");
```

Find output.

Q5 — SEMICOLON TRAP

```
c
int x = 0;

if (x);
    printf("A");
```

What prints?

Q6 — DANGLING ELSE

```

c
int a = 1;
int b = 0;

if (a)
    if (b)
        printf("X");
    else
        printf("Y");

```

Find output.

Q7 — SHORT CIRCUIT

```

c
int x = 0;
int y = 10;

if (x && ++y)
    printf("A");

printf("%d", y);

```

Find output.

Q8 — SWITCH

```

c
int x = 2;

switch (x)
{
    case 1:
        printf("A");

    case 2:
        printf("B");

    case 3:
        printf("C");
        break;

    default:
        printf("D");
}

```

Find output.

Q9 — SWITCH START POINT

```

c
int x = 3;

switch (x)

```

```
{
    case 1:
        printf("A");
        break;

    case 2:
        printf("B");

    case 3:
        printf("C");

    default:
        printf("D");
}
```

TEXT

Positive

Negative

Zero

TEXT

Positive

Negative

Zero

- C program basics

- operators and expressions

- type conversion and casting

- input/output

- `if`
- `if-else`
- `else-if`
- nested `if`
- truth values
- assignment-vs-comparison trap
- dangling `else`

- semicolon/braces traps
- `switch`
- `case`
- `break`
- `default`
- fall-through

Next

MODULE 0 / LESSON 6 — Loops in C

Next we enter another **very high-yield GATE + placement area**:

```
C
for
while
do-while
break
continue
```

Then we will learn how to calculate program behavior and complexity for:

```
C
for (i = 1; i <= n; i++)
```

```
C
for (i = 1; i <= n; i *= 2)
```

```
C
for (i = n; i > 0; i /= 2)
```

```
C
for (i = 0; i < n; i++)
    for (j = 0; j < n; j++)
```

This is where **C programming starts connecting directly with DSA time complexity**.

Type **NEXT**.

USER

NEXT

USER

NEXT

We continue exactly from the course structure in your uploaded master prompt.

Instead you tell the computer:

Repeat this action 100 times.

That is a **loop**.

Think:

```
TEXT
START
  ! "
Check condition
  ! "
Do work
  ! "
Update
  ! "
Go back
```

3. WHY LOOPS EXIST

Suppose we want:

```
TEXT
1
2
3
4
5
```

Without a loop:

```
C
printf("1\n");
printf("2\n");
printf("3\n");
printf("4\n");
printf("5\n");
```

With a loop:

```
C
for (int i = 1; i <= 5; i++)
{
    printf("%d\n", i);
}
```

Much better.

4. for LOOP

General syntax:


```
c
for (initialization; condition; update)
{
    statements;
}
```

Example:

```
c
for (int i = 1; i <= 5; i++)
{
    printf("%d ", i);
}
```

Output:

```
TEXT
1 2 3 4 5
```

5. HOW `for` EXECUTES

Take:

```
c
for (int i = 1; i <= 3; i++)
{
    printf("%d ", i);
}
```

Execution order:

```
TEXT
1. initialization
2. condition
3. body
4. update
5. condition
6. body
7. update
...
```

Visual:

```
TEXT
i = 1
! "
i <= 3 ?
! "   y e s
print i
! "
i++
! "
repeat
```

6. TRACE TABLE

```
C
for (int i = 1; i <= 3; i++)
{
    printf("%d ", i);
}
```

Step	i	Condition i <= 3	Action
start	1	true	print 1
update	2	true	print 2
update	3	true	print 3
update	4	false	stop

Output:

```
TEXT
1 2 3
```

7. VERY IMPORTANT — CONDITION CHECK COUNT

Suppose loop runs:

```
C
for (i = 1; i <= n; i++)
```

Body executes:

```
TEXT
n times
```

But condition:

```
C
i <= n
```

is checked:

```
TEXT
n + 1 times
```

Why?

Because after the last iteration, C checks once more and discovers the condition is false.

Q-6% TE may ask number of comparisons.

8. SIMPLE LOOP COUNT

```
C
for (i = 0; i < n; i++)
```

Values of `i`:

```
TEXT
0, 1, 2, ..., n-1
```

Number of iterations:

```
TEXT
n
```

9. ANOTHER LOOP

```
C
for (i = 1; i <= n; i++)
```

Values:

```
TEXT
1, 2, 3, ..., n
```

Iterations:

```
TEXT
n
```

10. STEP SIZE MATTERS

```
C
for (i = 0; i < n; i += 2)
```

Values:

```
TEXT
0, 2, 4, 6, ...
```

Approximately:

```
TEXT
n / 2
```

iterations.

Complexity?

Still:

TEXT

$O(n)$

because Big-O ignores constant factors.

$O=1$ is an important DSA connection.

11. MEMORY TRICK

If loop variable changes like:

TEXT

+1, +2, +5, +100

then usually iterations are proportional to n .

Think:

Additive growth !' often linear.

12. while LOOP

Syntax:

```
c
while (condition)
{
    statements;
}
```

Example:

```
c
int i = 1;

while (i <= 5)
{
    printf("%d ", i);
    i++;
}
```

Output:

TEXT

1 2 3 4 5

13. STORY FOR `while`

Imagine:

Keep walking while the traffic light is green.

You don't necessarily know in advance how many times you will repeat.

TEXT

```
condition true?  
    !"  
    do work  
    !"  
check again
```

14. `for` VS `while`

These are often logically interchangeable.

```
c  
for (i = 1; i <= 5; i++)  
{  
    printf("%d", i);  
}
```

can be written:

```
c  
i = 1;  
  
while (i <= 5)  
{  
    printf("%d", i);  
    i++;  
}
```

15. WHEN TO USE WHICH?

Typical convention:

`for`

Useful when iteration count/update pattern is clear.

```
c  
for (i = 0; i < n; i++)
```

`while`

Useful when repetition depends on a changing condition.

Example:

```
C
while (n > 0)
{
    ...
}
```

But technically many problems can use either.

16. do-while

Syntax:

```
C
do
{
    statements;
}
while (condition);
```

Notice:

```
C
while (condition);
```

ends with a semicolon.

17. SPECIAL PROPERTY OF do-while

A `do-while` executes the body:

At least once

because the condition is checked **after** the body.

Example

```
C
int x = 100;

do
{
    printf("Hello");
}
while (x < 10);
```

Condition:

```
TEXT
100 < 10
```

is false.

But body executes first.

Output:

```
TEXT
Hello
```

Exactly once.

18. COMPARE WITH `while`

```
C
int x = 100;

while (x < 10)
{
    printf("Hello");
}
```

Condition checked first.

False immediately.

Output:

```
TEXT
nothing
```

DON'T CONFUSE

`while`

```
TEXT
C H E C K  ! '   E X E C U T E
```

May execute zero times.

`do-while`

```
TEXT
E X E C U T E  ! '   C H E C K
```

Executes at least once.

Memory trick:

DO first, ask later.

19. GATE TRAP — SEMICOLON IN `do-while`

Correct:

```
C
do
{
    ...
}
while (condition);
```

The semicolon is required here.

But:

```
C
while (condition);
```

alone can create an empty loop.

Different situation.

20. EMPTY `while` LOOP

Consider:

```
C
int i = 0;

while (i < 5);
{
    i++;
}
```

The semicolon means:

```
C
while (i < 5)
;
```

The loop body is empty.

But `i` never changes inside that empty loop.

So:

```
TEXT
i = 0 forever
```

Condition:

```
TEXT
0 < 5
```

always true.

Result:

Infinite loop

Very important trap.

21. INFINITE LOOP

Common forms:

```
c
while (1)
{
}
```

or:

```
c
for (;;)
{
}
```

These are intentionally infinite unless something like `break`, `return`, or other control transfer exits.

22. break

`break` immediately exits the nearest enclosing loop or `switch`.

Example:

```
c
for (int i = 1; i <= 10; i++)
{
    if (i == 5)
        break;

    printf("%d ", i);
}
```

Trace:

```
TEXT
1  !'  p r i n t
2  !'  p r i n t
3  !'  p r i n t
4  !'  p r i n t
5  !'  b r e a k
```

Output:

```
TEXT
1 2 3 4
```

23. STORY FOR `break`

Imagine you are searching rooms for your phone.

TEXT

```
R o o m   1   ! '   n o t   f o u n d
R o o m   2   ! '   n o t   f o u n d
R o o m   3   ! '   F O U N D
```

Once found:

Stop searching.

That is `break`.

24. `continue`

`continue` does **not** terminate the loop.

It skips the rest of the current iteration and proceeds to the next iteration.

Example:

```
c
for (int i = 1; i <= 5; i++)
{
    if (i == 3)
        continue;

    printf("%d ", i);
}
```

Output:

TEXT

```
1 2 4 5
```

When:

TEXT

```
i = 3
```

printing is skipped.

But loop continues.

25. BREAK VS CONTINUE

Keyword	Meaning
break	leave loop completely

Keyword	Meaning
continue	skip current iteration

Memory trick:

TEXT

```
B R E A K      !'   G E T   O U T
C O N T I N U E !'   S K I P   A N D   G O   O N
```

26. continue IN A for LOOP

Consider:

```
c
for (i = 1; i <= 5; i++)
{
    if (i == 3)
        continue;

    printf("%d ", i);
}
```

When `continue` executes, control goes to the loop update:

```
c
i++
```

then condition is checked again.

27. continue IN A while LOOP — TRAP

Consider:

```
c
int i = 0;

while (i < 5)
{
    if (i == 2)
        continue;

    printf("%d ", i);
    i++;
}
```

Trace:

TEXT

```
i  =  0  !'   p r i n t   !'   i = 1
i  =  1  !'   p r i n t   !'   i = 2
i  =  2  !'   c o n t i n u e
```

Now `i++` is skipped.

So `i` stays:

```
TEXT
2
```

Again:

```
TEXT
i == 2
```

continue.

Forever.

Infinite loop

Excellent placement/GATE trap.

28. CORRECT VERSION

```
C
int i = 0;

while (i < 5)
{
    if (i == 2)
    {
        i++;
        continue;
    }

    printf("%d ", i);
    i++;
}
```

Now it progresses.

29. NESTED LOOPS

A loop inside another loop.

Example:

```
C
for (i = 1; i <= 3; i++)
{
    for (j = 1; j <= 2; j++)
    {
        printf("*");
    }

    printf("\n");
}
```

Output:

TEXT

```
**  
**  
**
```

30. STORY FOR NESTED LOOPS

Imagine a school:

TEXT

```
3 classrooms  
2 students checked per classroom
```

Outer loop:

TEXT

```
classrooms
```

Inner loop:

TEXT

```
students inside each classroom
```

For each outer iteration, inner loop runs fully.

31. TRACE NESTED LOOP

```
c  
for (i = 1; i <= 3; i++)  
{  
    for (j = 1; j <= 2; j++)  
    {  
        printf("%d%d ", i, j);  
    }  
}
```

Iterations:

TEXT

```
i=1:  
    j = 1  !'  1 1  
    j = 2  !'  1 2  
  
i=2:  
    j = 1  !'  2 1  
    j = 2  !'  2 2  
  
i=3:  
    j = 1  !'  3 1  
    j = 2  !'  3 2
```

Output:

TEXT

11 12 21 22 31 32

32. COMPLEXITY CONNECTION $\Theta = \Theta = \Theta$

Suppose:

```
c
for (i = 0; i < n; i++)
{
    printf("A");
}
```

The body runs:

TEXT

n times

So:

TEXT

Time Complexity = $O(n)$

33. TWO NESTED LINEAR LOOPS

```
c
for (i = 0; i < n; i++)
{
    for (j = 0; j < n; j++)
    {
        printf("A");
    }
}
```

Outer:

TEXT

n times

For every outer iteration, inner:

TEXT

n times

Total:

TEXT

$$n \times n = n^2$$

Therefore:

TEXT

$$\text{Time Complexity} = O(n^2)$$

34. THREE NESTED LOOPS

```
c
for (i = 0; i < n; i++)
{
    for (j = 0; j < n; j++)
    {
        for (k = 0; k < n; k++)
        {
            work;
        }
    }
}
```

Total:

TEXT

$$n \times n \times n \\ = n^3$$

So:

TEXT

$$O(n^3)$$

35. BUT DON'T BLINDLY COUNT LOOPS

This is crucial.

Two loops do not always mean:

TEXT

$$O(n^2)$$

Example:

```
c
for (i = 0; i < n; i++)
{
    work;
}

for (j = 0; j < n; j++)
```

```
{  
  work;  
}
```

They are sequential.

Total:

```
TEXT  
n + n  
= 2n
```

Big-O:

```
TEXT  
O(n)
```

~~O=V~~ Very important.

MEMORY TRICK

Nested loops

Usually:

```
TEXT  
MULTIPLY
```

Sequential loops

Usually:

```
TEXT  
ADD
```

Then simplify Big-O.

36. EXAMPLE

```
C  
for (i = 0; i < n; i++)  
  work;  
  
for (j = 0; j < n * n; j++)  
  work;
```

Total:

```
TEXT  
n + n2
```


Dominant term:

TEXT
 n^2

Therefore:

TEXT
 $O(n^2)$

37. LOGARITHMIC LOOP $O(\log n)$

Consider:

```
c
for (i = 1; i <= n; i = i * 2)
{
    work;
}
```

Values:

TEXT
1
2
4
8
16
32
...

Suppose after k iterations:

TEXT
 $i = 2^k$

Loop stops around:

TEXT
 $2^k > n$

So:

TEXT
 $k = \log_2 n$

Therefore:

$O(\log n)$

38. STORY FOR LOGARITHMIC LOOP

Imagine doubling your money:

TEXT

```
1 1
1 2
1 4
1 8
1 16
1 32
```

You reach a large number very quickly.

That is why multiplying the loop variable by a constant usually produces logarithmic iterations.

MEMORY TRICK

TEXT

```
i = i + constant
!' l i n e a r

i = i * constant
!' l o g a r i t h m i c
```

For common monotonic loop patterns.

39. ANOTHER LOG LOOP

C

```
for (i = n; i > 1; i = i / 2)
{
    work;
}
```

Values:

TEXT

```
n
n/2
n/4
n/8
...
```

Number of iterations:

TEXT

```
O(log n)
```

40. SOLVED EXAMPLE

Suppose:

```
TEXT
n = 16
```

Loop:

```
C
for (i = 1; i <= n; i *= 2)
```

Values entering body:

```
TEXT
1
2
4
8
16
```

Iterations:

```
TEXT
5
```

Broad complexity:

```
TEXT
O(log n)
```

Technically counts often involve floor/ceiling plus constants, but Big-O ignores those constants.

41. GATE LOOP TRAP

```
C
for (i = 1; i < n; i = i * 2)
```

If:

```
TEXT
i starts at 1
```

good.

But:

```
C
for (i = 0; i < n; i = i * 2)
```

is a problem.

Why?

Start:

```
TEXT
i = 0
```

Update:

```
TEXT
i = i * 2
= 0
```

Forever.

If `n > 0`, condition stays true.

Infinite loop

~~0 = 1~~ Excellent trap.

42. DECREASING LOOP

```
C
for (i = n; i > 0; i--)
{
    work;
}
```

Iterations:

```
TEXT
n
```

Complexity:

```
TEXT
O(n)
```

Direction does not matter.

What matters is how many iterations execute.

43. STEP BY 2

```
C
for (i = n; i > 0; i -= 2)
```

About:

TEXT

$n/2$

iterations.

Big-O:

TEXT

$O(n)$

44. DEPENDENT INNER LOOP

Consider:

```
c
for (i = 1; i <= n; i++)
{
    for (j = 1; j <= i; j++)
    {
        work;
    }
}
```

Inner iterations:

TEXT

```
w h e n   i = 1   !'   1
w h e n   i = 2   !'   2
w h e n   i = 3   !'   3
...
w h e n   i = n   !'   n
```

Total:

TEXT

$1 + 2 + 3 + \dots + n$

Formula:

$n(n+1)/2$

Dominant term:

TEXT

$n^2/2$

So:

TEXT

$O(n^2)$

Very important GATE pattern.

FORMULA BOX

TEXT

$$1 + 2 + 3 + \dots + n$$
$$= n(n+1)/2$$
$$= O(n^2)$$

45. ANOTHER DEPENDENT LOOP

```
c
for (i = 1; i <= n; i++)
{
    for (j = i; j <= n; j++)
    {
        work;
    }
}
```

Counts:

TEXT

n
n-1
n-2
...
1

Total again:

TEXT

$$n + (n-1) + \dots + 1$$

So:

TEXT

$$O(n^2)$$

46. MIXED LINEAR + LOG

```
c
for (i = 0; i < n; i++)
{
    for (j = 1; j <= n; j *= 2)
    {
        work;
    }
}
```

Outer:

TEXT

n

Inner:

TEXT

log n

Total:

TEXT

n log n

Therefore:

$O(n \log n)$

This complexity appears constantly in algorithms.

47. FIRST IMPORTANT COMPLEXITY TABLE

Loop pattern	Complexity
i++ to n	$O(n)$
i += 2	$O(n)$
i *= 2	$O(\log n)$
i /= 2	$O(\log n)$
two nested n loops	$O(n^2)$
nested n and log n	$O(n \log n)$
3 nested n loops	$O(n^3)$

Copy this.

48. GATE TRAP — MODIFYING LOOP VARIABLE INSIDE

```
c
for (i = 0; i < 10; i++)
{
    i++;
}
```

Don't simply say:

TEXT

10 iterations

There are two increments:

1. inside body
2. in `for` update

Trace:

TEXT

```
start 0
body increment !' 1
for increment !' 2

body at 2
!' 3
for update !' 4

...
```

Body executes for:

TEXT

0, 2, 4, 6, 8

So:

TEXT

5 iterations

~~0-10~~ ways trace actual variable changes.

49. SOLVED GATE-STYLE EXAMPLE

C

```
int i;

for (i = 1; i < 10; i = i * 2)
{
    printf("%d ", i);
}
```

Trace:

TEXT

```
i = 1 !' p r i n t 1
i = 2 !' p r i n t 2
i = 4 !' p r i n t 4
i = 8 !' p r i n t 8
i = 16 !' s t o p
```

Output:

TEXT

1 2 4 8

50. SOLVED EXAMPLE — `continue`

```
C
for (int i = 1; i <= 5; i++)
{
    if (i % 2 == 0)
        continue;

    printf("%d ", i);
}
```

Even values:

TEXT

2, 4

are skipped.

Output:

TEXT

1 3 5

51. SOLVED EXAMPLE — `break`

```
C
int i;

for (i = 1; i <= 10; i++)
{
    if (i * i > 20)
        break;
}

printf("%d", i);
```

Trace:

TEXT

i = 1	!	1	> 20	false	
i = 2	!	4	> 20	false	
i = 3	!	9	> 20	false	
i = 4	!	16	> 20	false	
i = 5	!	25	> 20	true	break

Output:

TEXT

5

52. NESTED `break` TRAP

Consider:

```
c
for (i = 0; i < 3; i++)
{
    for (j = 0; j < 3; j++)
    {
        if (j == 1)
            break;

        printf("*");
    }
}
```

`break` exits only the **nearest enclosing loop**.

For each outer iteration:

TEXT

```
j = 0 !' p r i n t *
j = 1 !' b r e a k i n n e r
```

Outer loop continues.

Output:

TEXT

Not just one `*`.

Ø=important.

53. NESTED `continue`

```
c
for (i = 1; i <= 2; i++)
{
    for (j = 1; j <= 3; j++)
    {
        if (j == 2)
            continue;

        printf("%d%d ", i, j);
    }
}
```

For each row, `j=2` is skipped.

Output:

```
TEXT
11 13 21 23
```

54. COMMON PLACEMENT LOOP PATTERNS

Traversing an array

```
C
for (i = 0; i < n; i++)
{
    process(arr[i]);
}
```

Reverse traversal

```
C
for (i = n - 1; i >= 0; i--)
{
    process(arr[i]);
}
```

Digit processing

```
C
while (n > 0)
{
    digit = n % 10;
    n = n / 10;
}
```

Binary search family

Often shrinks search space by half:

```
TEXT
O(log n)
```

Matrix traversal

Often:

```
TEXT
O(rows × columns)
```

55. DIGIT-PROCESSING STORY

Suppose:

TEXT

n = 583

Loop:

```
c
while (n > 0)
{
    int digit = n % 10;
    n = n / 10;
}
```

Trace:

n	n % 10	new n
583	3	58
58	8	5
5	5	0

Digits encountered:

TEXT

3, 8, 5

This is useful for:

- reversing numbers
- digit sum
- palindrome number
- counting digits

0-Placement foundation.

56. COMPLEXITY OF DIGIT LOOP

```
c
while (n > 0)
{
    n /= 10;
}
```

Each iteration divides **n** by 10.

So number of iterations is roughly:

TEXT

$\log \cdot \epsilon \quad n$

Therefore:

TEXT

$O(\log n)$

where log base is irrelevant in Big-O.

~~$O = \log$~~ No connection between C loops and complexity.

57. GATE TRAP — `i++` VS `++i` IN LOOP UPDATE

For:

C

```
for (i = 0; i < n; i++)
```

and:

C

```
for (i = 0; i < n; ++i)
```

in this context, both increment `i` once after each iteration.

So they usually produce the same loop behavior.

The difference between postfix/prefix matters when the resulting expression value is itself used.

58. GATE TRAP — LOOP VARIABLE AFTER EXIT

C

```
int i;

for (i = 1; i <= 5; i++)
{
}

printf("%d", i);
```

Trace:

TEXT

```
1,2,3,4,5 execute
t h e n   i + +   ! '   6
condition 6 <=5 false
```

So output:

TEXT

6

Very common output pattern.

59. ANOTHER EXAMPLE

```
C
int i;

for (i = 0; i < 5; i++)
{
}

printf("%d", i);
```

After loop:

TEXT

i = 5

60. BUT SCOPE CAN MATTER

```
C
for (int i = 0; i < 5; i++)
{
}
```

Here `i` is declared within the `for` initialization.

Its scope is limited to the loop/block context.

So you cannot simply use that same `i` afterward outside its scope.

We will study scope properly soon.

61. GATE FOCUS

MUST KNOW:

- exact loop iteration count
- final loop variable value
- nested loop tracing
- linear vs logarithmic loops
- dependent loops
- `break`
- `continue`
- semicolon traps
- infinite loops

- `while` vs `do-while`
 - nested loop complexity
-

62. GATE TRAP SHEET

Trap 1

```
C
while (condition);
```

may create an empty loop.

Trap 2

```
C
for (i = 0; i < n; i *= 2)
```

If `i = 0`, it remains 0.

Potential infinite loop.

Trap 3

```
break
```

only exits nearest loop.

Trap 4

```
continue
```

can skip an important update in `while`.

Potential infinite loop.

Trap 5

Nested loops do not automatically imply $O(n^2)$.

Check bounds carefully.

Trap 6

Sequential loops:

TEXT

ADD complexities

Nested loops:

TEXT

often MULTIPLY

63. INTERVIEW QUESTIONS

Q1. Difference between `while` and `do-while` ?

`while` checks before executing.

`do-while` executes first, then checks, so the body runs at least once.

Q2. Difference between `break` and `continue` ?

`break` exits the loop.

`continue` skips the remaining current iteration.

Q3. Complexity of:

C

```
for (i = 0; i < n; i++)
```

$O(n)$.

Q4. Complexity of:

C

```
for (i = 1; i <= n; i *= 2)
```

$O(\log n)$.

Q5. Complexity of two nested `n` loops?

Usually $O(n^2)$.

Q6. Why can a `continue` cause an infinite `while` loop?

Because it can skip the statement responsible for changing the loop-control variable.

64. MEMORY TRICKS

`for`

Think:

TEXT

S T A R T ! ' C H E C K ! ' W O R K ! ' C H A N G E

while

Think:

WHILE true, keep going.

do-while

Think:

DO first, ask later.

break

Break out.

continue

Skip this round.

Complexity

TEXT

ADD variable !' linear
MULTIPLY variable !' logarithmic

65. FORMULA BOX

TEXT

$1 + 2 + \dots + n$
 $= n(n+1)/2$
 $= O(n^2)$

TEXT

$1 + 2 + 4 + \dots$ up to n
!' $O(\log n)$ terms

TEXT

$n \times n$ nested
!' $O(n^2)$

TEXT

$n \times \log n$
!' $O(n \log n)$

66. NOTEBOOK NOTES

for

```
c
for (initialization; condition; update)
{
    statements;
}
```

Order:

```
TEXT
initialize
!' c o n d i t i o n
!' b o d y
!' u p d a t e
!' c o n d i t i o n . . .
```

while

```
c
while (condition)
{
    statements;
}
```

Condition checked before body.

May execute zero times.

do-while

```
c
do
{
    statements;
}
while (condition);
```

Body executes at least once.

break

Terminates nearest loop/switch.

continue

Skips rest of current iteration.

Complexity patterns

TEXT

```
i + +          !' 0 ( n )
i + = k       !' 0 ( n )
i * = 2       !' 0 ( l o g   n )
i / = 2       !' 0 ( l o g   n )
```

Two nested `n` loops:

TEXT

$O(n^2)$

Sequential `O(n)` loops:

TEXT

$O(n)$

67. 30-SECOND REVISION

Remember:

1. `for` = init !' condition !' body !' update.
2. `while` may execute zero times.
3. `do-while` executes at least once.
4. `break` exits the nearest loop.
5. `continue` skips current iteration.
6. `i++` up to `n` !' $O(n)$.
7. `i *= 2` !' $O(\log n)$.
8. nested `n × n` !' $O(n^2)$.
9. sequential loops add, nested loops usually multiply.
10. Semicolon after `while(condition)` can create an empty/infinite loop.

68. ACTIVE RECALL

Try first.

Q1

How many times does this body run?

C

```
for (i = 0; i < 10; i++)
```

Q2

Complexity?

C

```
for (i = 1; i <= n; i *= 2)
```

Q3

What is the special property of `do-while` ?

Q4

Difference between `break` and `continue` ?

Q5

What is complexity?

```
c
for (i = 0; i < n; i++)
    for (j = 0; j < n; j++)
        work;
```

ANSWERS

A1

```
TEXT
10 iterations
```

A2

```
TEXT
O(log n)
```

A3

Its body executes at least once.

A4

`break` exits the loop.

`continue` skips only the remainder of the current iteration.

A5

```
TEXT
O(n2)
```

69. PRACTICE

Q1

Find output:

```
c
for (int i = 1; i <= 5; i++)
{
```

```
    printf("%d ", i);  
}
```

Q2

Find output:

```
C  
int i = 3;  
  
while (i > 0)  
{  
    printf("%d ", i);  
    i--;  
}
```

Q3 — DO-WHILE

```
C  
int x = 10;  
  
do  
{  
    printf("A");  
}  
while (x < 5);
```

How many times is A printed?

Q4 — BREAK

```
C  
for (int i = 1; i <= 10; i++)  
{  
    if (i == 4)  
        break;  
  
    printf("%d ", i);  
}
```

Find output.

Q5 — CONTINUE

```
C  
for (int i = 1; i <= 5; i++)  
{  
    if (i == 2 || i == 4)  
        continue;
```

```
    printf("%d ", i);  
}
```

Find output.

~~Q6~~ — GATE LOOP COUNT

How many times does `work` execute?

```
C  
for (i = 1; i <= n; i += 2)  
{  
    work;  
}
```

Give approximate count and Big-O.

~~Q7~~ — LOG LOOP

For:

```
C  
for (i = 1; i <= 64; i *= 2)  
{  
    printf("%d ", i);  
}
```

List all printed values.

~~Q8~~ — NESTED COMPLEXITY

Find complexity:

```
C  
for (i = 0; i < n; i++)  
{  
    for (j = 1; j <= n; j *= 2)  
    {  
        work;  
    }  
}
```

~~Q9~~ — DEPENDENT LOOP

Find Big-O:

```
C  
for (i = 1; i <= n; i++)  
{  
    for (j = 1; j <= i; j++)
```

```
{
    work;
}
```

Q10 — INFINITE LOOP TRAP

What is wrong here?

```
c
for (i = 0; i < n; i *= 2)
{
    printf("%d", i);
}
```

Assume `n > 0`.

Q11 — continue TRAP

What happens?

```
c
int i = 0;

while (i < 5)
{
    if (i == 2)
        continue;

    i++;
}
```

Explain exactly.

Q12 — PLACEMENT FOUNDATION

Using a loop, write the logic to compute the **sum of digits** of:

```
TEXT
583
```

Expected:

```
TEXT
5 + 8 + 3 = 16
```

Use only the ideas already learned:

```
TEXT
%
/
```

```
while
```

PROGRESS

Completed '

Lesson 1: C program basics **Lesson 2:** Operators and expressions **Lesson 3:** Type conversion and casting **Lesson 4:** Input/output **Lesson 5:** Decision making **Lesson 6:** Loops

You now have the core control-flow foundation needed for real C programs.

Next

MODULE 0 / LESSON 7 — Functions in C

Next we will learn:

- why functions exist
- declaration / prototype
- definition
- function call
- parameters
- arguments
- return values
- `void`
- call by value
- local variables
- function execution tracing
- stack-frame intuition
- why changing a parameter does **not** automatically change the original variable
- first major bridge toward **pointers and recursion**

Example:

```
c
void change(int x)
{
    x = 100;
}

int a = 10;
change(a);
```

Why does `a` usually remain `10`?

That concept becomes extremely important for pointers, recursion, linked lists, trees, and interviews.

Type **NEXT**.

ASSISTANT

MODULE 0 — LESSON 7

Functions in C — Declaration, Definition, Parameters, Return Values, Call by Value

We continue according to your uploaded master prompt and the learning order already established.

1. WHERE WE ARE

Module: 0 — C Foundation **Lesson:** 7 **Current topic:** Functions in C

Why it matters: Functions are the bridge between basic C and major topics like recursion, stack frames, pointers, linked lists, trees, and modular programming.

GATE importance: $\emptyset = \gamma\%$ **Placement importance:** $\emptyset = \gamma\%$

Today we cover:

- Today we cover:
- why functions exist
 - function declaration / prototype
 - function definition
 - function call
 - arguments and parameters
 - return values
 - `void`
 - local variables
 - call by value
 - function tracing
 - nested calls
 - stack-frame intuition
 - common GATE traps

2. STORY / INTUITION

Imagine a restaurant.

You don't personally go into the kitchen and perform every cooking step.

You simply request:

Make one dosa.

The kitchen performs the task and returns the result.

Programming functions work similarly.

```
TEXT
MAIN PROGRAM
  |
  | request
  v
FUNCTION
  |
  | result
  v
MAIN PROGRAM
```

Example:

```
C
int square(int x)
{
    return x * x;
}
```

Now instead of repeatedly writing:

```
C
5 * 5
```

you can write:

```
C
square(5)
```

and receive:

```
TEXT
25
```

3. WHY FUNCTIONS EXIST

Suppose you need to calculate squares many times.

Without a function:

```
C
a = 5 * 5;
b = 7 * 7;
c = 9 * 9;
```

With a function:

```
C
int square(int x)
{
```

```
    return x * x;
}
```

Then:

```
c
a = square(5);
b = square(7);
c = square(9);
```

Functions give us:

- code reuse
- modularity
- readability
- easier debugging
- structured problem solving

4. BASIC FUNCTION

```
c
int add(int a, int b)
{
    return a + b;
}
```

Break it into pieces:

```
TEXT
int           !'  return  type
add           !'  function name
a,  b        !'  parameters
{  . . .  }   !'  function body
return       !'  value  returned
```

5. CALLING A FUNCTION

```
c
int result = add(3, 4);
```

Here:

```
TEXT
3 and 4
```

are sent to the function.

Inside:

```
c
int add(int a, int b)
```

the parameters receive:

```
TEXT
a = 3
b = 4
```

Then:

```
TEXT
a + b = 7
```

Function returns:

```
TEXT
7
```

So:

```
TEXT
result = 7
```

6. VISUALIZATION

```
TEXT
main()

add(3,4)
  |
  v
+-----+
| add    |
| a = 3  |
| b = 4  |
| return a + b |
+-----+
  |
  v
  7
  |
  v
result = 7
```

7. THREE IMPORTANT TERMS

You must distinguish:

Function Declaration / Prototype

```
C
int add(int, int);
```

Tells the compiler:

A function named `add` exists, takes two `int` arguments, and returns `int`.

Function Definition

```
C
int add(int a, int b)
{
    return a + b;
}
```

Contains actual implementation.

Function Call

```
C
add(3, 4);
```

Invokes the function.

DON'T CONFUSE

```
TEXT
D e c l a r a t i o n   ! '   t e l l s
D e f i n i t i o n     ! '   i m p l e m e n t s
C a l l                 ! '   e x e c u t e s
```

Memory trick:

Tell !' Build !' Use

8. COMPLETE EXAMPLE

```
C
#include <stdio.h>

int add(int, int);

int main()
{
    int result;
```

```
    result = add(5, 7);

    printf("%d", result);

    return 0;
}

int add(int a, int b)
{
    return a + b;
}
```

Output:

```
TEXT
12
```

9. WHY DECLARATION MAY BE NEEDED

Suppose `main()` calls:

```
C
add(5, 7);
```

before the full definition of `add` appears later in the file.

Then the compiler needs information about the function before that call.

So we provide:

```
C
int add(int, int);
```

before `main`.

10. ALTERNATIVE

If function definition appears before `main`:

```
C
int add(int a, int b)
{
    return a + b;
}

int main()
{
    printf("%d", add(3, 4));
}
```

then a separate earlier prototype is not needed for that call, because the compiler already saw the definition.

11. PARAMETERS VS ARGUMENTS

Example:

```
C
int add(int a, int b)
```

`a` and `b` are:

Parameters

Now:

```
C
add(10, 20);
```

`10` and `20` are:

Arguments

Memory trick:

```
TEXT
Function defines PARAMETERS
Caller supplies ARGUMENTS
```

12. REAL-LIFE ANALOGY

Function:

Make tea with (sugar, milk)

Parameters:

```
TEXT
sugar
milk
```

Actual call:

```
TEXT
makeTea(2, 100)
```

Arguments:

```
TEXT
2
100
```

13. RETURN VALUE

Function:

```
c
int square(int x)
{
    return x * x;
}
```

Call:

```
c
int y = square(5);
```

Function returns:

```
TEXT
25
```

So:

```
TEXT
y = 25
```

14. RETURN ENDS FUNCTION EXECUTION

Consider:

```
c
int test()
{
    return 5;

    printf("Hello");
}
```

Once:

```
c
return 5;
```

executes, control leaves the function.

The later statement is not executed.

15. FUNCTION RETURN TYPE

Example:


```
c
int add(int a, int b)
```

returns an `int`.

Another:

```
c
float average(int a, int b)
{
    return (float)(a + b) / 2;
}
```

returns `float`.

16. `void` FUNCTION

Sometimes a function does a task but returns no value.

Example:

```
c
void greet()
{
    printf("Hello");
}
```

Call:

```
c
greet();
```

Output:

```
TEXT
Hello
```

`void` means:

No value is returned.

17. FUNCTION WITH NO PARAMETERS

Example:

```
c
void hello(void)
{
    printf("Hello");
}
```

Call:

```
c
hello();
```

For modern C, writing:

```
c
void hello(void)
```

explicitly indicates no parameters.

18. FOUR COMMON FUNCTION CATEGORIES

Think of functions based on:

- arguments?
- return value?

1. No arguments, no return value

```
c
void greet(void)
{
    printf("Hello");
}
```

2. Arguments, no return value

```
c
void printSum(int a, int b)
{
    printf("%d", a + b);
}
```

3. No arguments, return value

```
c
int getFive(void)
{
    return 5;
}
```

4. Arguments and return value

```
c
int add(int a, int b)
{
    return a + b;
}
```

This fourth form is extremely common.

19. FUNCTION TRACING

Consider:

```
C
int addOne(int x)
{
    x = x + 1;
    return x;
}

int main()
{
    int a = 5;
    int b = addOne(a);

    printf("%d %d", a, b);
}
```

Let's trace.

Initially:

```
TEXT
main:
a = 5
```

Call:

```
C
addOne(a)
```

Parameter gets:

```
TEXT
x = 5
```

Inside:

```
C
x = x + 1;
```

So:

```
TEXT
x = 6
```

Return:

```
TEXT
6
```

Thus:

```
TEXT
b = 6
```

But what about `a`?

It remains:

```
TEXT
5
```

Output:

```
TEXT
5 6
```

Why?

Because C uses:

Call by Value

20. 0-6% CALL BY VALUE — MUST KNOW

When you call:

```
C
change(a);
```

the function receives a **copy of the value of `a`**.

Not the original variable itself.

Example:

```
C
void change(int x)
{
    x = 100;
}
```

Then:

```
C
int a = 10;

change(a);
```

Inside function:

TEXT

x = 10

Then:

TEXT

x becomes 100

But original:

TEXT

a remains 10

21. STORY FOR CALL BY VALUE

Imagine you have an original document.

You photocopy it and give the copy to your friend.

Your friend writes:

TEXT

100

on the photocopy.

Did your original document change?

No.

That is call by value.

TEXT

ORIGINAL a = 10

|
| copy value

v
x = 10

|
v
x = 100

Original a remains 10

22. MEMORY VISUALIZATION

Suppose:

C

```
int a = 10;  
change(a);
```

Conceptually:

TEXT

main stack frame

a = 10

Function call creates another activation:

TEXT

change stack frame

x = 10

They are separate objects.

Then:

TEXT

x = 100

changes only:

TEXT

change's x

not:

TEXT

main's a

23. FIRST STACK-FRAME INTUITION

When a function is called, it typically gets its own activation record / stack frame containing things such as:

- parameters
- local variables
- bookkeeping information
- return information

Conceptually:

TEXT

STACK

```
+-----+
| change() |
| x = 10   |
+-----+
| main()   |
| a = 10   |
+-----+
```

When `change()` returns:

TEXT

```
+-----+
| main() |
| a = 10  |
+-----+
```

This intuition becomes essential for recursion.

24. IMPORTANT

Do not over-literalize the simplified stack diagram as the exact physical implementation on every architecture/compiler.

For GATE learning:

Function calls create separate invocation contexts containing their own parameters/local variables.

That is the core idea.

25. SOLVED EXAMPLE — CALL BY VALUE

```
c
void change(int x)
{
    x = 50;
}

int main()
{
    int a = 10;

    change(a);

    printf("%d", a);
}
```

Output:

TEXT

10

26. WHY DOES **a** NOT CHANGE?

Because:

TEXT

```
a's value 10
!"  c o p i e d   i n t o
x
```

Then only the copy changes.

27. HOW DO WE CHANGE THE ORIGINAL?

This is exactly where **pointers** enter.

Later we will write:

```
c
void change(int *p)
{
    *p = 50;
}
```

Then:

```
c
change(&a);
```

Now the function receives the address of `a`.

It can access the original object.

Result:

```
TEXT
a = 50
```

~~Don't~~ This is one of the biggest reasons pointers exist.

Don't rush it yet—we will teach pointers deeply.

28. SWAP — CLASSIC INTERVIEW/GATE CONCEPT

Many beginners write:

```
c
void swap(int a, int b)
{
    int temp = a;
    a = b;
    b = temp;
}
```

Then:

```
c
int x = 10;
int y = 20;

swap(x, y);
```

They expect:

```
TEXT
x = 20
y = 10
```


But that does not happen.

Why?

Because:

TEXT

swap's a and b

are copies.

Only copies are exchanged.

Original `x` and `y` remain unchanged.

☐ This will become our first major pointer application.

29. LOCAL VARIABLES

Variables declared inside a function/block are usually local to that scope.

Example:

```
c
void test()
{
    int x = 5;
}
```

`x` belongs to that local scope.

Outside:

```
c
int main()
{
    printf("%d", x);
}
```

you cannot simply refer to that local `x`.

30. SAME VARIABLE NAME IN DIFFERENT FUNCTIONS

This is valid:

```
c
void f()
{
    int x = 10;
}

void g()
{
    int x = 20;
}
```

These are different local variables.

Conceptually:

TEXT

f ' s x " ` g ' s x

31. FUNCTION PARAMETER IS LOCAL

Example:

```
c
int square(int x)
{
    return x * x;
}
```

Parameter:

TEXT

x

belongs to that function invocation.

If `main` also has:

```
c
int x = 5;
```

these are separate variables.

32. SOLVED EXAMPLE — SAME NAME

```
c
int test(int x)
{
    x = x + 10;
    return x;
}

int main()
{
    int x = 5;

    int y = test(x);

    printf("%d %d", x, y);
}
```

Trace:

`main`:

TEXT

```
x = 5
```

test gets copy:

TEXT

```
x = 5
```

Inside:

TEXT

```
x = 15
```

Return:

TEXT

```
15
```

So:

TEXT

```
main x = 5  
y = 15
```

Output:

TEXT

```
5 15
```

33. NESTED FUNCTION CALLS

Suppose:

```
c  
int doubleValue(int x)  
{  
    return x * 2;  
}  
  
int addOne(int x)  
{  
    return x + 1;  
}
```

Expression:

```
c  
int y = addOne(doubleValue(5));
```

Evaluate inner call:

TEXT

```
doubleValue(5)
= 10
```

Then:

TEXT

```
addOne(10)
= 11
```

So:

TEXT

```
y = 11
```

34. VISUALIZE CALL CHAIN

TEXT

```
main
|
| doubleValue(5)
v
doubleValue
|
| returns 10
v
main
|
| addOne(10)
v
addOne
|
| returns 11
v
main
```

Soon recursion will create many levels of these calls.

35. FUNCTION CALL INSIDE `printf`

C

```
printf("%d", square(4));
```

First:

TEXT

```
square(4)
! ' 1 6
```

Then:

```
C
printf("%d", 16);
```

Output:

```
TEXT
16
```

36. FUNCTION CALL INSIDE EXPRESSION

```
C
int x = square(3) + square(4);
```

Values:

```
TEXT
square(3) = 9
square(4) = 16
```

Then:

```
TEXT
x = 25
```

37. GATE TRAP — RETURN TYPE MATTERS

Consider:

```
C
int divide(int a, int b)
{
    return a / b;
}
```

Call:

```
C
double x = divide(5, 2);
```

Inside function:

```
TEXT
5 / 2
= 2
```

because integer division.

Function returns integer:

TEXT

2

Then:

TEXT

x = 2.0

Not 2.5.

38. CORRECT FLOATING VERSION

C

```
double divide(int a, int b)
{
    return (double)a / b;
}
```

Then:

TEXT

5.0 / 2
= 2.5

Return:

TEXT

2.5

39. FUNCTION PROTOTYPE TYPE CHECKING

Suppose:

C

```
int add(int, int);
```

This tells the compiler what kinds of arguments the function expects.

Calling functions through correct declarations helps the compiler perform type checking/conversions.

Modern C code should use proper prototypes.

40. WRONG NUMBER OF ARGUMENTS

Suppose:

C

```
int add(int a, int b);
```

Then:

```
c
add(5);
```

does not match the declared function interface.

Likewise:

```
c
add(1, 2, 3);
```

does not match.

41. GATE TRAP — SEMICOLON

Declaration:

```
c
int add(int, int);
```

needs a semicolon.

Definition:

```
c
int add(int a, int b)
{
    return a + b;
}
```

does **not** end with a semicolon after the closing brace.

42. RETURN VALUE CAN BE USED DIRECTLY

Example:

```
c
int max(int a, int b)
{
    if (a > b)
        return a;

    return b;
}
```

Call:

```
c
printf("%d", max(10, 20));
```

Output:

TEXT

20

43. MULTIPLE returns

A function can contain multiple `return` statements.

Example:

```
C
int sign(int x)
{
    if (x > 0)
        return 1;

    if (x < 0)
        return -1;

    return 0;
}
```

Only the return reached during execution runs.

44. EARLY RETURN

Example:

```
C
int absolute(int x)
{
    if (x >= 0)
        return x;

    return -x;
}
```

For:

TEXT

x = 5

first return executes.

For:

TEXT

x = -5

second return executes.

45. GATE STYLE TRACE


```

c
int f(int x)
{
    x = x + 2;
    return x * 2;
}

int main()
{
    int x = 3;
    int y = f(x);

    printf("%d %d", x, y);
}

```

Trace:

main:

```

TEXT
x = 3

```

Function:

```

TEXT
parameter x = 3

```

Then:

```

TEXT
x = 5

```

Return:

```

TEXT
5 * 2 = 10

```

So:

```

TEXT
main x = 3
y = 10

```

Output:

```

TEXT
3 10

```

46. GATE STYLE — TWO CALLS

```

c
int f(int x)
{
    return x + 1;
}

int main()
{
    int a = 5;

    a = f(a);
    a = f(a);

    printf("%d", a);
}

```

Trace:

First:

```

TEXT
f(5) = 6
a = 6

```

Second:

```

TEXT
f(6) = 7
a = 7

```

Output:

```

TEXT
7

```

47. FUNCTION CALL SEQUENCE

Consider:

```

c
int f(int x)
{
    printf("F");
    return x + 1;
}

```

Then:

```

c
int y = f(5);

```

Function executes first:

TEXT

```
prints F
returns 6
```

Then:

TEXT

```
y = 6
```

This becomes relevant in output tracing when functions themselves print.

48. SIDE EFFECTS IN FUNCTIONS

A function can affect the external world by:

- printing
- modifying global/static state
- modifying objects through pointers
- performing I/O

But changing an ordinary by-value parameter alone does not change the caller's variable.

This distinction is important.

49. STACK-FRAME STORY

Imagine a stack of trays.

`main()` starts:

TEXT

```
+-----+
| main   |
| a = 10 |
+-----+
```

`f(a)` is called:

TEXT

```
+-----+
| f      |
| x = 10 |
+-----+
| main   |
| a = 10 |
+-----+
```

`f` returns:

TEXT

```
+-----+
| main   |
| a = 10 |
+-----+
```

```
+-----+
```

This is why function execution naturally leads to:

Recursion

because recursion means the same function creates another invocation before the previous one finishes.

50. DON'T CONFUSE

Stack memory

Used conceptually for function invocation data.

Stack data structure

A LIFO abstract data structure.

Later recursion connects them conceptually, but they are not the same topic.

51. PLACEMENT CONNECTION

Functions appear everywhere in DSA.

Soon you'll write:

```
c
int binarySearch(...)
```

```
c
void push(...)
```

```
c
int pop(...)
```

```
c
struct Node *insert(...)
```

```
c
void inorder(...)
```

```
c
void dfs(...)
```

Good placement code is usually divided into clear functions.

52. INTERVIEW QUESTION — CALL BY VALUE

Q. Does C support true call by reference?

C passes function arguments **by value**.

We can simulate reference-like modification by passing pointer values.

Example later:

```
C
change(&x);
```

The pointer itself is also passed by value, but it points to the caller's object.

Ø=1% This wording is technically important.

53. INTERVIEW QUESTIONS

Q1. What is a function?

A reusable block of code designed to perform a particular task.

Q2. What is a function prototype?

A declaration that specifies the function's name, return type, and parameter types.

Q3. Difference between parameter and argument?

Parameter = variable in function declaration/definition.

Argument = value/expression supplied at the call.

Q4. What is call by value?

The called function receives copies of argument values.

Q5. Why doesn't modifying a parameter modify the original caller variable?

Because the parameter is a separate local object initialized from the argument value.

Q6. What does `void` return type mean?

The function returns no value.

Q7. How can a function modify a caller's variable?

Pass its address using a pointer and modify the pointed object.

54. GATE TRAPS

Trap 1

```
C
void change(int x)
{
    x = 100;
```

```
}
```

does not modify caller variable directly.

Trap 2

```
C
double x = divide(5, 2);
```

If `divide` performs integer division internally, assigning to `double` afterward doesn't recover the lost fraction.

Trap 3

Local variables with same name in different functions are separate.

Trap 4

Return immediately exits the function.

Statements after an unconditional executed `return` won't execute.

Trap 5

A proper prototype matters for correct function calls in modern C.

55. MEMORY TRICKS

Function

Think:

INPUT !' WORK !' OUTPUT

```
TEXT
Arguments
    !"
Function
    !"
Return value
```

Declaration / Definition / Call

```
TEXT
T E L L !'   B U I L D !'   U S E
```

Call by value

56. NOTEBOOK NOTES

Function

Reusable block of code performing a task.

Basic form:

```
c
return_type function_name(parameters)
{
    statements;
    return value;
}
```

Example:

```
c
int add(int a, int b)
{
    return a + b;
}
```

Call:

```
c
int x = add(2, 3);
```

Prototype

```
c
int add(int, int);
```

Declares function interface.

Parameters vs Arguments

```
c
int add(int a, int b)
```

`a`, `b` !' parameters.

```
c
add(10, 20)
```

`10`, `20` !' arguments.

Call by Value

C passes argument values by value.

```
c
void change(int x)
{
    x = 100;
}
```

```
c
int a = 10;
change(a);
```

a remains 10.

void

```
c
void greet(void)
```

returns no value.

Stack intuition

Each active function call has its own invocation context containing parameters/local variables.

This is important for recursion.

57. 30-SECOND REVISION

Remember:

1. Function = reusable block of code.
 2. Prototype tells function interface.
 3. Definition contains implementation.
 4. Call invokes function.
 5. Parameters belong to the function.
 6. Arguments are supplied by caller.
 7. `return` sends a value back and exits.
 8. `void` means no return value.
 9. C passes arguments by value.
 10. To modify the original caller object, we'll later pass its address using pointers.
-

58. ACTIVE RECALL

Try first.

Q1

What's the difference between:

```
c
int add(int, int);
```

and:

```
c
int add(int a, int b)
{
    return a + b;
}
```

Q2

What are parameters here?

```
c
int multiply(int x, int y)
```

Q3

What are the arguments here?

```
c
multiply(4, 5);
```

Q4

What is printed?

```
c
void change(int x)
{
    x = 20;
}

int main()
{
    int a = 5;
    change(a);
    printf("%d", a);
}
```

Q5

Why doesn't `a` change?

ANSWERS

A1

First is a declaration/prototype.

Second is the definition.

A2

TEXT

x and y

A3

TEXT

4 and 5

A4

TEXT

5

A5

Because `change` receives a copy of the value of `a`.

59. PRACTICE

Q1

Find output:

```
c
int square(int x)
{
    return x * x;
}

int main()
{
    printf("%d", square(5));
}
```

Q2

Find `a` and `b`:

```
c
int inc(int x)
{
    x++;
    return x;
}

int a = 10;
int b = inc(a);
```

Q3 — CALL BY VALUE

```
C
void test(int x)
{
    x = x + 10;
}

int main()
{
    int x = 5;

    test(x);

    printf("%d", x);
}
```

Find output and explain why.

Q4 — RETURN VALUE

```
C
int f(int x)
{
    return x + 2;
}

int main()
{
    int a = f(3);
    int b = f(a);

    printf("%d %d", a, b);
}
```

Find output.

Q5 — INTEGER DIVISION INSIDE FUNCTION

```
C
int divide(int a, int b)
{
    return a / b;
}

int main()
{
    double x = divide(7, 2);

    printf("%f", x);
}
```

What value is conceptually stored in `x`?

Q6 — LOCAL VARIABLE

```
c
int f(int x)
{
    x = 20;
    return x;
}

int main()
{
    int x = 10;

    printf("%d %d", x, f(x));
}
```

Ignoring argument-evaluation-order complications here because `f(x)` does not modify the caller's `x`, what values should be printed?

Q7 — FUNCTION CHAIN

```
c
int doubleValue(int x)
{
    return x * 2;
}

int addOne(int x)
{
    return x + 1;
}

int main()
{
    int y = addOne(doubleValue(4));

    printf("%d", y);
}
```

Find output.

Q8 — SWAP THINKING

Why does this fail to swap caller variables?

```
c
void swap(int a, int b)
{
    int temp = a;
    a = b;
    b = temp;
}
```

Explain in one sentence.

0=09 — PLACEMENT FOUNDATION

Write a function:

```
c
int max(int a, int b)
```

that returns the larger of two integers.

PROGRESS

Completed '

Lesson 1: C program basics **Lesson 2:** Operators and expressions **Lesson 3:** Type conversion and casting **Lesson 4:** Input/output **Lesson 5:** Decision making **Lesson 6:** Loops **Lesson 7:** Functions

You now have the foundation needed to understand **scope, lifetime, stack frames, and eventually recursion.**

Next

MODULE 0 / LESSON 8 — Scope, Lifetime & Storage Concepts

Next we will study:

- local variables
- global variables
- scope
- lifetime
- automatic variables
- `static` local variables
- `extern` concept
- block scope
- function scope where relevant
- shadowing
- initialization behavior
- GATE output questions involving repeated function calls

A classic example we will fully trace:

```
c
void f()
{
    static int x = 0;
    x++;
    printf("%d ", x);
}

f();
f();
f();
```

Why does this print:

1 2 3

1 1 1

NEXT

2. STORY / INTUITION

Imagine a university.

There are two kinds of notices.

Notice inside one classroom

Only students in that classroom can see it.

That is like a:

Local Variable

Notice on the university main board

Many departments can access it.

That is like a:

Global Variable

Now imagine another question:

How long does the notice stay?

A classroom notice might be removed when the class ends.

A permanent university notice may remain throughout the day.

That is the difference between:

Scope

and

Lifetime

3. TWO DIFFERENT QUESTIONS

Whenever you see a variable, ask:

Question 1

Where can I use its name?

That is:

Scope

Question 2

How long does the object exist?

That relates to:

Storage duration / lifetime

Do not confuse them.

4. SCOPE — TECHNICAL IDEA

Scope determines the region of source code where an identifier can be referred to by its name.

Example:

```
c
void test(void)
{
    int x = 10;

    printf("%d", x);
}
```

Here `x` is declared inside `test`.

Its name is visible only in its applicable block scope.

You cannot simply use:

```
c
printf("%d", x);
```

inside another unrelated function.

5. LOCAL VARIABLE

Example:

```
c
void test(void)
{
    int x = 10;
}
```

`x` is local to that block.

Conceptually:

```
TEXT
test()
+-----+
| x = 10 |
+-----+
```

When execution leaves the function/block, an ordinary automatic local object's lifetime ends.

6. GLOBAL VARIABLE

A variable declared outside functions at file scope:

```
C
int x = 10;

int main(void)
{
    printf("%d", x);
}
```

Here:

```
C
int x = 10;
```

has file scope.

It can be referred to from functions in that translation unit after its declaration, subject to linkage/declaration rules.

For GATE intuition:

Global variable = declared outside functions and generally available to functions in that file after declaration.

7. SIMPLE GLOBAL EXAMPLE

```
C
#include <stdio.h>

int x = 100;

void f(void)
{
    printf("%d ", x);
}

int main(void)
{
    printf("%d ", x);
    f();

    return 0;
}
```

Output:

```
TEXT
100 100
```

Both functions refer to the same global object `x`.

8. LOCAL VS GLOBAL

```
c
int x = 10;

void f(void)
{
    int y = 20;
}
```

Here:

```
TEXT
x !'  global / file - scope  variable
y !'  local / block - scope  variable
```

9. STORY

Think:

```
TEXT
GLOBAL
= common notice board

LOCAL
= personal notebook
```

10. SHADOWING Ø=Ý%

Suppose:

```
c
int x = 10;

int main(void)
{
    int x = 20;

    printf("%d", x);
}
```

Which `x` is used?

Inside `main`, the local `x` hides the outer `x`.

Output:

```
TEXT
20
```

This is called:

Shadowing

11. VISUALIZATION

TEXT

```
Global:
x = 10

main():
    Local x = 20
```

Inside `main`:

TEXT

```
name x
!"
nearest visible declaration
!"
local x = 20
```

The global object still exists, but its name is hidden in that scope by the local declaration.

12. GATE TRAP — SAME NAME

```
c
int x = 5;

void f(void)
{
    int x = 10;

    printf("%d ", x);
}

int main(void)
{
    printf("%d ", x);
    f();

    return 0;
}
```

`main` prints global:

TEXT

5

`f` prints local:

TEXT

10

Output:

TEXT

5 10

13. BLOCK SCOPE

A block is enclosed by:

```
C
{
}
```

Example:

```
C
int main(void)
{
    int x = 10;

    {
        int y = 20;
        printf("%d", y);
    }

    return 0;
}
```

`y` exists only within the inner block's scope.

You cannot access it afterward by name.

14. BLOCK SHADOWING

```
C
int main(void)
{
    int x = 10;

    {
        int x = 20;
        printf("%d ", x);
    }

    printf("%d", x);
}
```

Inside inner block:

TEXT

x = 20

After block:

TEXT

```
x = 10
```

Output:

TEXT

```
20 10
```

15. VISUALIZATION

TEXT

Outer block:

```
x = 10
```

Inner block:

```
    x = 20    !•  h i d e s   o u t e r   x
```

Inner block ends

Outer x visible again:

```
x = 10
```

16. LIFETIME / STORAGE DURATION

Now the second question:

How long does the object exist?

An ordinary local variable such as:

```
c
void f(void)
{
    int x = 5;
}
```

normally has:

Automatic storage duration

A new `x` is created each time execution enters its block, and its lifetime ends when execution leaves that block.

17. AUTOMATIC LOCAL VARIABLE

Example:

```
c
void f(void)
{
```

```

int x = 0;
x++;

printf("%d ", x);
}

```

Calls:

```

c
f();
f();
f();

```

What happens?

Each call gets a fresh `x`.

Trace:

```

TEXT
Call 1:
x = 0
x + + !' 1
print 1

Call 2:
new x = 0
x + + !' 1
print 1

Call 3:
new x = 0
x + + !' 1
print 1

```

Output:

```

TEXT
1 1 1

```

18. STACK-FRAME CONNECTION

Each call has its own local invocation context.

Conceptually:

```

TEXT
f() call 1
+-----+
| x = 1 |
+-----+

```

returns.

That local `x` is gone.

Then another call:

TEXT

```
f() call 2
+-----+
| x = 0 |
+-----+
```

fresh object again.

This prepares us for recursion.

19. NOW STATIC LOCAL VARIABLE

Change:

```
c
int x = 0;
```

to:

```
c
static int x = 0;
```

Example:

```
c
void f(void)
{
    static int x = 0;

    x++;

    printf("%d ", x);
}
```

Calls:

```
c
f();
f();
f();
```

Output:

```
TEXT
1 2 3
```

Why?

Because a static local variable:

- has local/block scope
- but static storage duration

Its value survives between function calls.

20. STORY FOR STATICLOCAL

Automatic local variable is like:

A whiteboard erased after every class.

Static local variable is like:

A notebook stored in the classroom cupboard.

Each new class opens the same notebook and continues from the previous value.

21. TRACE STATICLOCAL

```
c
void f(void)
{
    static int x = 0;

    x++;

    printf("%d ", x);
}
```

Call 1

Initialization occurs:

```
TEXT
x = 0
```

Then:

```
TEXT
x + + !' 1
```

print:

```
TEXT
1
```

Call 2

It does **not** create a fresh `x = 0`.

Existing value:

```
TEXT
x = 1
```


Then:

TEXT

```
x++ !' 2
```

print:

TEXT

```
2
```

Call 3

TEXT

```
x = 2  
!"  
x++ = 3
```

print:

TEXT

```
3
```

Output:

TEXT

```
1 2 3
```

22. ~~0=0~~ DON'T CONFUSE

Automatic local

C

```
int x = 0;
```

inside a function:

TEXT

```
scope !' local  
lifetime !' each block execution / function invocation
```

Static local

C

```
static int x = 0;
```

inside function:

TEXT

```
scope != local
lifetime / storage duration != entire program execution
```

This distinction is extremely important.

23. SCOPE "LIFETIME

This is a major concept.

Static local:

```
C
void f(void)
{
    static int x;
}
```

`x` exists for the program's entire execution.

But can `main()` access it directly by writing:

```
C
x = 5;
```

No.

Because its **scope** is still local to `f`'s block.

So:

TEXT

```
Lifetime = long
Scope    = limited
```

Excellent exam concept.

24. FORMULA-LIKE MEMORY RULE

Think:

TEXT

```
SCOPE = WHERE NAME IS VISIBLE
```

```
LIFETIME = HOW LONG OBJECT EXISTS
```

25. INITIALIZATION OF AUTOMATIC LOCAL VARIABLE

Consider:

```
c
void f(void)
{
    int x;

    printf("%d", x);
}
```

`x` is an uninitialized automatic local variable.

Its value is **indeterminate**.

Using it in ways that require its value may lead to undefined behavior depending on circumstances.

For GATE beginner rule:

Do not assume uninitialized local variables become zero.

Very important.

26. COMMON BEGINNER MISTAKE

Wrong assumption:

```
c
int x;
```

inside function means:

```
TEXT
x = 0
```

No.

For an ordinary uninitialized automatic local variable:

```
TEXT
value = indeterminate
```

27. GLOBAL VARIABLES DEFAULT INITIALIZATION

Now:

```
c
int x;

int main(void)
{
    printf("%d", x);
}
```

A file-scope object with static storage duration that is not explicitly initialized is zero-initialized.

So:

```
TEXT
x = 0
```

Output:

```
TEXT
0
```

28. STATICLOCAL DEFAULTINITIALIZATION

Similarly:

```
C
void f(void)
{
    static int x;

    printf("%d", x);
}
```

If no explicit initializer is supplied, it is zero-initialized.

So first call prints:

```
TEXT
0
```

29. IMPORTANT INITIALIZATION TABLE

Variable category	If not explicitly initialized
automatic local	indeterminate
global/file-scope	zero-initialized
static local	zero-initialized

0=00 Copy this.

30. EXAMPLE — STATICWITHOUT INITIALIZER

```
C
void f(void)
{
    static int x;

    x++;
}
```

```

    printf("%d ", x);
}

int main(void)
{
    f();
    f();

    return 0;
}

```

Initially:

```

TEXT
x = 0

```

First call:

```

TEXT
1

```

Second:

```

TEXT
2

```

Output:

```

TEXT
1 2

```

31. GLOBAL VARIABLE CHANGES PERSIST

```

c
int x = 0;

void f(void)
{
    x++;
}

int main(void)
{
    f();
    f();

    printf("%d", x);

    return 0;
}

```

Trace:

TEXT

```
initial global x = 0
f ( ) !' 1
f ( ) !' 2
```

Output:

TEXT

2

Because both functions access the same global object.

32. GLOBAL VS STATICLOCAL

Both can retain values throughout program execution.

But their visibility differs.

Global/file-scope

Can potentially be accessed across functions/files depending on declarations/linkage.

Static local

Only accessible by name inside its block.

33. GATE STYLE EXAMPLE

```
c
int x = 10;

void f(void)
{
    x = x + 5;
}

int main(void)
{
    f();
    f();

    printf("%d", x);

    return 0;
}
```

Trace:

TEXT

```
x = 10

f ( ) !' x = 1 5
f ( ) !' x = 2 0
```

Output:

TEXT

20

34. LOCAL SHADOWS GLOBAL

```
C
int x = 10;

void f(void)
{
    int x = 5;

    x++;

    printf("%d ", x);
}

int main(void)
{
    f();

    printf("%d", x);

    return 0;
}
```

Inside `f`:

TEXT

```
local x = 5
! ' 6
```

prints:

TEXT

6

Global stays:

TEXT

10

Output:

TEXT

6 10

35. STATICLOCAL + PARAMETER

```

c
int f(int x)
{
    static int y = 0;

    y = y + x;

    return y;
}

```

Calls:

```

c
printf("%d ", f(2));
printf("%d ", f(3));
printf("%d ", f(4));

```

Trace:

Start:

```

TEXT
y = 0

```

First:

```

TEXT
y = 0 + 2 = 2
return 2

```

Second:

```

TEXT
y = 2 + 3 = 5
return 5

```

Third:

```

TEXT
y = 5 + 4 = 9
return 9

```

Output:

```

TEXT
2 5 9

```

0-5% Strong GATE-style pattern.

36. static AT FILE SCOPE

Now consider:


```
c
static int x = 10;
```

outside functions.

This still gives the object static storage duration, but importantly at file scope `static` also gives it **internal linkage**.

Simple intuition:

That name/object is restricted to that translation unit rather than being externally linkable from another source file.

For our current course, do not overcomplicate this.

Remember:

```
TEXT
static inside function
!' retain value + local scope

static at file scope
!' file - scope object with internal linkage
```

37. WHY THIS MATTERS

Suppose a large C program contains:

```
TEXT
file1.c
file2.c
```

A file-scope:

```
c
static int count;
```

in `file1.c` is intended to remain private to that translation unit.

Useful concept.

38. `extern` — SIMPLE INTUITION

Suppose another source file contains the actual global definition:

```
c
int count = 10;
```

Another file may use a declaration:

```
c
extern int count;
```

Conceptually:

```
extern says: "This object is defined elsewhere; I am declaring it here so I can refer to it."
```

39. DON'T MEMORIZE `extern` AS "CREATES GLOBAL"

That is wrong.

Example:

```
c
extern int x;
```

is generally a declaration, not necessarily a definition that allocates a new object there.

For GATE foundation:

```
TEXT
extern !' refer to externally defined object
```

40. STORAGE-CLASS KEYWORDS — HIGH-LEVEL VIEW

Important historical C storage-class specifiers include:

```
TEXT
auto
register
static
extern
```

For modern GATE/placement preparation, focus mainly on:

Ø= `static` Ø= `extern` concept Ø= `auto` Ø= `register`

41. `auto`

An ordinary block variable such as:

```
c
int x;
```

already has automatic storage duration unless another storage-class specifier changes it.

You can explicitly write:

```
c
auto int x;
```

but almost nobody does in normal code.

So:

TEXT

auto

! ' g e n e r a l l y o r d i n a r y a u t o m a t i c l o c a l s t o r a g e b e h a v i o r

Ø=Low practical importance.

42. register

Historically:

C

```
register int i;
```

was a hint to the compiler that frequent access might benefit from register storage.

Modern optimizing compilers generally decide such things themselves.

For GATE:

Know the concept, but do not spend much time on it.

Ø=KNOW.

43. IMPORTANT register DETAIL

Traditionally, you cannot apply the unary address-of operator `&` to an object declared with `register`.

Example:

C

```
register int x;
```

Then:

C

```
&x
```

is not permitted by the language rules.

This can appear as a conceptual question.

44. STORAGE DURATION SUMMARY

For our current level:

Automatic

Typical local variables:

C

```
void f(void)
{
```

```
int x;  
}
```

Lifetime tied to block execution.

Static

Global/file-scope variables and variables declared `static` have static storage duration:

```
TEXT  
exist for whole program execution
```

Dynamic

Later, with:

```
c  
malloc()
```

objects live until:

```
c  
free()
```

or program termination.

We have not studied this yet.

45. MEMORY CONNECTION

Broad intuition:

```
TEXT  
Program Memory  
  
+-----+  
| Code/Text |  
+-----+  
| Global/Static Data |  
+-----+  
| Heap |  
|           !" / g r o w s           |  
+-----+  
| |  
+-----+  
| Stack |  
| automatic locals |  
+-----+
```

This is conceptual.

Exact layout is implementation/system dependent.

We will study memory properly in Lesson 9.

46. GATE TRAP — STATIC INITIALIZED ONCE

Consider:

```
c
void f(void)
{
    static int x = 5;

    printf("%d ", x);

    x++;
}
```

Calls:

```
c
f();
f();
f();
```

Trace:

First:

```
TEXT
x = 5
print 5
x = 6
```

Second:

```
TEXT
print 6
x = 7
```

Third:

```
TEXT
print 7
```

Output:

```
TEXT
5 6 7
```

Initializer is not re-applied each time the function is called.

47. AUTOMATIC VERSION

```

c
void f(void)
{
    int x = 5;

    printf("%d ", x);

    x++;
}

```

Calls:

```

c
f();
f();
f();

```

Every call starts:

```

TEXT
x = 5

```

Output:

```

TEXT
5 5 5

```

Compare carefully.

48. SIDE-BY-SIDE

Automatic

```

c
int x = 5;

```

```

TEXT
C a l l   1   !'   5
C a l l   2   !'   5
C a l l   3   !'   5

```

Static

```

c
static int x = 5;

```

```

TEXT
C a l l   1   !'   5
C a l l   2   !'   6
C a l l   3   !'   7

```

49. GATE OUTPUT EXAMPLE

```
c
#include <stdio.h>

void fun(void)
{
    static int x = 1;

    printf("%d ", x);

    x = x * 2;
}

int main(void)
{
    fun();
    fun();
    fun();

    return 0;
}
```

Trace:

```
TEXT
x = 1
print 1
x = 2

print 2
x = 4

print 4
x = 8
```

Output:

```
TEXT
1 2 4
```

50. ANOTHER GATE OUTPUT EXAMPLE

```
c
int x = 1;

void f(void)
{
    int x = 2;

    {
        int x = 3;
        printf("%d ", x);
    }

    printf("%d ", x);
}
```

```

}

int main(void)
{
    f();
    printf("%d", x);
}

```

Trace:

Inner block:

```

TEXT
x = 3

```

Then function-local:

```

TEXT
x = 2

```

Then global:

```

TEXT
x = 1

```

Output:

```

TEXT
3 2 1

```

Great shadowing example.

51. HOW TO TRACE SCOPE QUESTIONS

Whenever same variable name appears multiple times:

Step 1

Mark every declaration.

Step 2

Draw scopes.

Example:

```

TEXT
global x

f:
    local x

    inner block:
        another x

```


Step 3

For every use, select:

nearest visible declaration.

Step 4

Then consider lifetime/static behavior separately.

This prevents confusion.

52. HOW TO TRACE STATIC QUESTIONS

Create a persistent box.

Example:

```
C
static int x = 0;
```

Write:

```
TEXT
STATIC STORAGE

x = 0
```

After every call, keep its latest value.

Do **not erase it** when function returns.

53. GATE FOCUS

Ø=NOT KNOWN:

- local vs global
- block scope
- shadowing
- automatic vs static local
- static survives calls
- global/static default zero initialization
- automatic local indeterminate if uninitialized
- scope vs lifetime
- `extern` basic meaning
- `static` file-scope/internal-linkage intuition

54. GATE TRAP SHEET

Trap 1

```
c
int x;
```

inside function is **not automatically zero**.

Trap 2

```
c
static int x;
```

is zero-initialized.

Trap 3

Global:

```
c
int x;
```

is zero-initialized.

Trap 4

Static local retains its value between calls.

Trap 5

Static local has long lifetime but still local scope.

Trap 6

A local declaration can shadow a global declaration.

Trap 7

`extern` does not simply mean "create a global variable."

55. PLACEMENT CONNECTION

In interviews, you may be asked:

Why should globals often be avoided?

Common reasons:

- hidden dependencies
- harder debugging
- accidental modification
- poorer modularity
- harder testing

Static local variables can be useful when a function needs to remember state between calls.

But overusing them can also create hidden state.

56. INTERVIEW QUESTIONS

Q1. What is scope?

The region of source code where an identifier is visible by name.

Q2. What is lifetime/storage duration?

How long the object exists during program execution.

Q3. Difference between local and global variables?

Local variables are declared within blocks/functions and have limited scope; globals are declared at file scope.

Q4. What does a static local variable do?

It has local scope but static storage duration, so its value persists between calls.

Q5. Is an uninitialized local `int` automatically zero?

No.

Q6. Is an uninitialized global `int` zero?

Yes, objects with static storage duration are zero-initialized if no explicit initializer is provided.

Q7. What is shadowing?

An inner declaration hides an outer declaration with the same name in that scope.

57. MEMORY TRICKS

Scope

Scope = SEE

Where can I see/use the name?

Lifetime

Lifetime = LIVE

How long does the object live?

Static local

Remember:

STATICSTAYS

TEXT

function ends
but value stays

Automatic local

Remember:

AUTO = AGAIN

Every function/block execution gets a fresh automatic local object.

58. FORMULA / RULE BOX

TEXT

Local automatic uninitialized
!' i n d e t e r m i n a t e

TEXT

Global uninitialized
!' z e r o

TEXT

Static local uninitialized
!' z e r o

TEXT

static local
!' l o c a l s c o p e + p r o g r a m - l o n g s t o r a g e d u r a t i o n

TEXT

nearest visible declaration
!' w i n s d u r i n g s h a d o w i n g

59. NOTEBOOK NOTES

Scope

Region where an identifier is visible.

Lifetime / Storage Duration

How long an object exists.

Automatic Local

```
c
void f(void)
{
    int x = 0;
}
```

Fresh variable each call.

Static Local

```
c
void f(void)
{
    static int x = 0;
}
```

Scope is local, but value persists across calls.

Global

```
c
int x = 0;
```

Declared outside functions at file scope.

Initialization

```
TEXT
a u t o m a t i c   l o c a l   u n i n i t i a l i z e d   ! '   i n d e t e r m i n a t e
g l o b a l   u n i n i t i a l i z e d   ! '   0
s t a t i c   l o c a l   u n i n i t i a l i z e d   ! '   0
```

Shadowing

Inner declaration hides outer declaration with same name.

`extern`

Declares an object that may be defined elsewhere.

60. 30-SECOND REVISION

Remember:

1. Scope = where name is visible.
 2. Lifetime = how long object exists.
 3. Ordinary local variable is typically automatic.
 4. Automatic local gets recreated on each entry.
 5. Static local retains value between calls.
 6. Static local still has local scope.
 7. Global/static objects are zero-initialized by default.
 8. Uninitialized automatic locals are not automatically zero.
 9. Inner variables can shadow outer ones.
 10. `extern` generally refers to an externally defined object.
-

61. ACTIVE RECALL

Try first.

Q1

What is the difference between scope and lifetime?

Q2

What prints?

```
c
void f(void)
{
    int x = 0;
    x++;

    printf("%d ", x);
}

f();
f();
f();
```

Q3

What prints if `x` becomes static?

```
c
static int x = 0;
```

Q4

What is the default value of an uninitialized global `int` ?

Q5

What happens when a local variable has the same name as a global variable?

ANSWERS

A1

Scope tells where the identifier can be used by name; lifetime tells how long the object exists.

A2

TEXT

1 1 1

A3

TEXT

1 2 3

A4

TEXT

0

A5

The local declaration shadows the global declaration within that scope.

62. PRACTICE

Q1

Find output:

```
c
int x = 5;

int main(void)
{
    printf("%d", x);
}
```

Q2

Find output:

```
c
int x = 5;

int main(void)
{
    int x = 10;

    printf("%d", x);
}
```

Q3 — AUTOMATIC

```
c
void f(void)
{
    int x = 5;

    x++;

    printf("%d ", x);
}

int main(void)
{
    f();
    f();
}
```

Find output.

Q4 — STATIC

```
c
void f(void)
{
    static int x = 5;

    x++;

    printf("%d ", x);
}

int main(void)
{
    f();
    f();
}
```

Find output.

Q5 — STATIC MULTIPLICATION

```
c
void f(void)
{
    static int x = 1;

    printf("%d ", x);

    x *= 3;
}

int main(void)
{
    f();
}
```



```

    f();
    f();
}

```

Find output.

Q6 — SHADOWING

```

c
int x = 1;

void f(void)
{
    int x = 2;

    {
        int x = 3;
        printf("%d ", x);
    }

    printf("%d ", x);
}

int main(void)
{
    f();
    printf("%d", x);
}

```

Find output.

Q7 — STATIC+ PARAMETER

```

c
int f(int n)
{
    static int sum = 0;

    sum += n;

    return sum;
}

int main(void)
{
    printf("%d ", f(2));
    printf("%d ", f(5));
    printf("%d", f(3));
}

```

Find output.

Q8 — INITIALIZATION TRAP

Which variable can safely be assumed to start at 0 ?

```

c
int a;

int main(void)
{
    int b;
    static int c;
}

```

Classify:

TEXT

a
b
c

0=00 — PLACEMENT THINKING

Suppose a function must count how many times it has been called.

Which is more suitable inside the function?

```

c
int count = 0;

```

or

```

c
static int count = 0;

```

Explain why in one sentence.

PROGRESS

Completed '

Lesson 1: C program basics **Lesson 2:** Operators & expressions **Lesson 3:** Type conversion **Lesson 4:** Input/output
Lesson 5: Decision making **Lesson 6:** Loops **Lesson 7:** Functions **Lesson 8:** Scope, lifetime & storage concepts

Next

MODULE 0 / LESSON 9 — Memory Basics: Stack, Heap, Global/Static Memory & Addresses

This is a **critical bridge lesson** before pointers.

We will finally build the complete mental model:

TEXT

PROGRAM MEMORY

%
% % % C o d e / T e x t

%%% Global / Static Data
%%% Stack
%%% Heap

Then we'll answer:

- Where do local variables usually live?
- What happens during a function call?
- What exactly is an address?
- Why does every object need memory?
- What is stack memory?
- What is heap memory?
- Why **heap memory** "binary heap"?
- How does `&x` actually fit into this picture?
- Why do pointers become necessary?

After that, we start the **Pointer Master Section**, one of the most important parts of this entire subject.

Type **NEXT**.

USER

NEXT

ASSISTANT

MODULE 0 — LESSON 9

Memory Basics — Stack, Heap, Global/Static Memory & Addresses

We continue from your uploaded master prompt and the exact course sequence we established.

This lesson is extremely important because it creates the mental model needed for **pointers, recursion, linked lists, trees, dynamic memory, and many GATE output questions**.

1. WHERE WE ARE

Module: 0 — C Foundation **Lesson:** 9 **Current topic:** Memory organization and addresses

Why it matters: A variable is not just a name like `x`. At runtime, program data exists somewhere in memory. Pointers work by referring to those locations.

GATE importance: 0=Ý%0MÚ%0KÍ0W **Placement importance:** 0=Ý%0MÚ%0KÍ0W

Today we cover:

- What memory means
- Memory addresses
- Conceptual program-memory layout
- Code/Text region

- Global/Static storage
- Stack memory
- Heap memory
- Function-call stack intuition
- Stack overflow intuition
- Heap allocation intuition
- Stack memory vs Stack data structure
- Heap memory vs Binary Heap
- `&x`
- Why pointers are necessary

2. STORY / INTUITION — MEMORY AS A HUGE APARTMENT

Imagine computer memory as a giant apartment building.

Every apartment has:

1. an **address**
2. something stored inside

Example:

TEXT

```
A p a r t m e n t   1 0 0 0   !'   2 5
A p a r t m e n t   1 0 0 4   !'   5 0
A p a r t m e n t   1 0 0 8   !'   9 0
```

Computer memory is conceptually similar.

You may have:

C

```
int x = 25;
```

Imagine `x` is stored somewhere such as:

TEXT

```
Address 1000
+-----+
|  25  |
+-----+
      x
```

So there are two different ideas:

TEXT

```
x      !'   v a l u e   s t o r e d   t h e r e
& x    !'   l o c a t i o n / a d d r e s s   o f   x
```

This is the foundation of pointers.

3. WHAT IS A MEMORY ADDRESS?

A **memory address** identifies a location in memory.

Suppose:

```
C
int x = 10;
```

Conceptually:

```
TEXT
Address      Value
-----
1000         10
              !
              x
```

Then:

```
TEXT
x  = 10
&x = address of x
```

For our imaginary example:

```
TEXT
&x = 1000
```

Do not expect real addresses to look exactly like simple decimal numbers.

You may see something like:

```
TEXT
0x7ffd8ab...
```

4. & — ADDRESS-OF OPERATOR

Consider:

```
C
int x = 10;
```

Then:

```
C
&x
```

means:

Address of x

This is why we earlier wrote:

```
C
scanf("%d", &x);
```

`scanf` needs to know:

Where should I place the value?

So we give it the address of `x`.

5. VALUE VS ADDRESS

Ø=~~NOT~~ **MUST KNOW**

Consider:

```
C
int x = 25;
```

Think:

```
TEXT
x
!"
VALUE
!"
25
```

But:

```
TEXT
&x
!"
ADDRESS
!"
where x lives
```

Do not confuse these.

6. VISUALIZATION

Suppose:

```
C
int x = 25;
```

and imagine:

TEXT

x is stored at address 5000

Then:

TEXT

```
Address
5000      !"

+-----+
|  25  |
+-----+
      x
```

So:

TEXT

```
x      !'    2 5
& x    !'    5 0 0 0
```

Soon we'll create:

C

```
int *p = &x;
```

Then:

TEXT

```
p      !'    5 0 0 0
```

and:

TEXT

```
* p    !'    2 5
```

That is the next major section.

7. PROGRAM MEMORY — BIG PICTURE

A running program is often explained conceptually using regions such as:

TEXT

```
+-----+
| Code / Text          |
+-----+
| Global / Static Data |
+-----+
| Heap                 |
|                       |
|      ...             |
|                       |
+-----+
| Stack                |
+-----+
```

A more visual textbook-style picture:

TEXT

Higher addresses

```
+-----+
|          STACK          |
| function calls          |
| automatic locals        |
| parameters              |
+-----+
|          free area      |
|                         |
+-----+
|          HEAP           |
| dynamic allocations      |
+-----+
| Global / Static Data    |
+-----+
| Code / Text              |
+-----+
Lower addresses
```

Ø=important:

This is a **conceptual model**.

Exact layout depends on the operating system, architecture, compiler, executable format, and runtime environment.

For GATE, the model is very useful.

8. REGION 1 — CODE / TEXT

Suppose your program contains:

```
c
int add(int a, int b)
{
    return a + b;
}
```

The machine instructions corresponding to program code are typically placed in a code/text region.

Think:

TEXT

CODE/TEXT

```
-----
instructions for main()
instructions for add()
instructions for other functions
```

For our course:

Code/Text = program instructions.

Ø=KNOW.

9. REGION 2 — GLOBAL / STATIC DATA

Consider:

```
C
int globalValue = 10;

static int count = 5;
```

Objects with static storage duration are associated conceptually with global/static data areas.

Remember Lesson 8:

```
TEXT
Global variable
!' exists for whole program execution

Static local
!' exists for whole program execution
```

Even though a static local has local scope, its storage duration is long.

10. EXAMPLE

```
C
int x = 10;

void f(void)
{
    static int y = 20;
    int z = 30;
}
```

Conceptually:

```
TEXT
GLOBAL / STATIC STORAGE
+-----+
| x = 10      |
| y = 20      |
+-----+
```

while ordinary local `z` is typically associated with the function's stack frame:

```
TEXT
STACK
+-----+
| f()        |
| z = 30     |
+-----+
```

11. REGION 3 — STACK MEMORY

This is one of the most important ideas.

When functions are called, their active invocation data is commonly maintained using a runtime stack.

Example:

```
c
void f(void)
{
    int x = 10;
}
```

When `f()` is active:

```
TEXT
STACK

+-----+
| f()    |
| x = 10 |
+-----+
```

When `f()` returns, that invocation is removed.

12. STORY — STACK OF PLATES

Imagine plates:

```
TEXT

    Plate C
    Plate B
    Plate A
```

The last plate placed on top is the first removed.

This is:

LIFO

Last In, First Out

Function calls naturally follow a similar nesting pattern.

13. FUNCTION CALL STACK

Suppose:

```
c
main()
```

calls:

```
c
f()
```

and `f()` calls:

```
c
g()
```

Then conceptually:

```
TEXT
STACK

+-----+
| g ( ) |      | !•  t o p
+-----+
| f ( ) |
+-----+
| main() |
+-----+
```

Who finishes first?

```
TEXT
g()
```

Then stack becomes:

```
TEXT

+-----+
| f ( ) |
+-----+
| main() |
+-----+
```

Then `f()` returns:

```
TEXT

+-----+
| main() |
+-----+
```

This is exactly why recursion works.

14. CALL ORDER VS RETURN ORDER

Suppose:

```
TEXT
main !'  A !'  B !'  C
```

Call order:

```
TEXT
main
A
B
```

```
C
```

Return order:

TEXT

```
C
B
A
main
```

Memory trick:

Calls go DOWN, returns come UP.

This will become our main recursion-tracing method.

15. STACK FRAME / ACTIVATION RECORD

Each active function invocation typically has an activation record or stack frame containing information such as:

- parameters
- local automatic variables
- return address/information
- saved machine state
- temporary values depending on implementation

For GATE intuition:

TEXT

```
Function Call
    !"
Activation Record
    !"
stored while function is active
```

16. EXAMPLE

```
C
int add(int a, int b)
{
    int sum = a + b;
    return sum;
}

int main(void)
{
    int x = 2;
    int y = add(5, 7);
}
```

While `add()` runs:

TEXT

STACK

```
+-----+
| add()   |
| a = 5   |
| b = 7   |
| sum = 12|
+-----+
| main()  |
| x = 2   |
| y waiting for value |
+-----+
```

When `add()` returns `12`, its frame is removed.

17. CALL BY VALUE CONNECTION

Remember:

```
c
void change(int x)
{
    x = 100;
}

int a = 10;
change(a);
```

Conceptually:

TEXT

STACK

```
change()
+-----+
| x = 100 | | !• c o p y
+-----+

main()
+-----+
| a = 10  | | !• o r i g i n a l
+-----+
```

Changing:

TEXT

`x = 100`

does not modify:

TEXT

`a`

because they are separate objects.

This connects Lesson 7 directly to memory.

18. STACK MEMORY — TYPICAL FEATURES

Conceptually:

- managed automatically
- function calls use it
- automatic local variables often live there
- allocations/deallocations are very fast
- limited size
- lifetime tied to function/block execution for automatic objects

But exact compiler behavior can optimize some variables into registers or elsewhere.

For GATE:

Think of automatic locals + function activation records as stack-related.

19. STACK OVERFLOW — INTUITION

Suppose a function keeps calling itself without stopping:

```
C
void f(void)
{
    f();
}
```

Call sequence:

```
TEXT
f
!"
f
!"
f
!"
f
!"
...
```

Each active invocation may require another stack frame.

Eventually the runtime stack can run out of available space.

This is called:

Stack Overflow

20. STORY

Imagine stacking plates endlessly.

TEXT

```
Plate
Plate
Plate
Plate
...
```

Eventually the stack becomes too large and collapses.

Similarly:

TEXT

```
too many active function calls
!'  s t a c k   s p a c e   e x h a u s t e d
```

This is why recursion requires a correct base condition.

21. REGION 4 — HEAP MEMORY

Now the other major memory area:

Heap

The heap is commonly used for **dynamic memory allocation**.

Example later:

C

```
int *p = malloc(sizeof(int));
```

This asks for memory during runtime.

Conceptually:

TEXT

HEAP

```
+-----+
| dynamically |
| allocated int |
+-----+
```

Unlike ordinary automatic local variables, this memory is not automatically tied to one function call's return.

22. STORY — HOTEL ROOMS

Think of heap memory like a hotel.

Yousay:

Give me one room.

The hotel allocates one.

You use it.

When finished, you must release it.

In C:

```
TEXT
malloc
!' request memory

free
!' release memory
```

We will study both in detail later.

23. STACK VS HEAP — FIRST COMPARISON

Feature	Stack	Heap
Typical use	function calls/local automatic data	dynamic allocation
Management	mostly automatic	programmer-controlled in C
Allocation	fast/simple	more flexible
Lifetime	tied to call/block for automatic locals	until explicitly freed
Size	usually more limited	usually larger
Example	int x; local	malloc(...)

24. IMPORTANT

This does **not** mean:

```
TEXT
Every local variable must physically be on stack
```

Compilers can optimize.

For exam mental models, however:

```
TEXT
automatic locals !' stack intuition
dynamic allocation !' heap intuition
```

is useful.

25. ~~0=00~~ DON'T CONFUSE — STACK MEMORY VS STACK DATA STRUCTURE

This distinction is essential.

Stack memory

Runtime memory mechanism used for things such as function calls.

Stack data structure

A LIFO abstract data structure that we intentionally implement.

Example operations:

TEXT

```
push
pop
peek
```

Same word.

Different concept.

26. ~~0=100~~ DON'T CONFUSE — HEAP MEMORY VS BINARY HEAP

Even more important.

Heap memory

Dynamic memory allocation region.

Used with:

C

```
malloc()
calloc()
realloc()
free()
```

Binary Heap

A tree-based data structure used for:

- priority queues
- heap sort
- min/max retrieval

Example:

TEXT

```
  90
 /  \
70   80
```

They are completely different concepts.

Memory trick:

Heap memory = storage

27. WHY HEAP MEMORY EXISTS

Suppose you want an array whose size is known only at runtime.

User enters:

```
TEXT
n = 1000
```

You may dynamically request memory based on `n`.

Later:

```
C
int *arr = malloc(n * sizeof(int));
```

This allows memory to be allocated while the program is running.

This becomes essential for:

- linked lists
- dynamic arrays
- trees
- graphs
- variable-sized data structures

28. LINKED LIST CONNECTION

A linked-list node may be created dynamically:

```
C
struct Node *newNode = malloc(sizeof(struct Node));
```

Conceptually:

```
TEXT
STACK

newNode
|
|
v

HEAP
+-----+
| data | next |
+-----+
```

The pointer lives in one place and points to an object elsewhere.

✓ This is why pointers + heap memory are fundamental to data structures.

29. TREES CONNECTION

Likewise:

```
TEXT
root
|
v

  HEAP
  10
 /  \
5    20
```

Nodes can be allocated dynamically and linked together using pointers.

So our learning dependency is:

```
TEXT
Memory
!"
Pointers
!"
Dynamic Allocation
!"
Linked Lists
!"
Trees
!"
Graphs
```

30. WHAT EXACTLY DOES A POINTER STORE?

Very important preview:

A pointer stores:

An address

Example:

```
C
int x = 10;
int *p = &x;
```

Suppose:

```
TEXT
&x = 5000
```

Then:

TEXT

p = 5000

Visualization:

TEXT

```
Address 3000          Address 5000
+-----+          +-----+
|  5000  |  -----> |   10   |
+-----+          +-----+
      p                  x
```

This is the central idea of pointers.

31. VARIABLE ITSELF ALSO HAS AN ADDRESS

Notice:

TEXT

p has a value

The value happens to be:

TEXT

address of x

But `p` itself also occupies memory.

So:

TEXT

```
p      !'  address stored inside p
&p     !'  address of pointer variable p
```

$\text{p} = \text{p}$ will become very important.

32. PREVIEW OF POINTER EXPRESSIONS

Suppose:

C

```
int x = 10;
int *p = &x;
```

Then conceptually:

Expression	Meaning
x	value of x
&x	address of x

Expression	Meaning
p	address stored in p
&p	address of p itself
*p	value stored at address p

If:

TEXT
p = &x

then:

TEXT
*p = x

So:

TEXT
*p = 10

We will go very slowly through this in the Pointer Master Section.

33. ADDRESS EXAMPLE

```
c
#include <stdio.h>

int main(void)
{
    int x = 10;

    printf("%d\n", x);
    printf("%p\n", (void *)&x);

    return 0;
}
```

Possible conceptual output:

TEXT
10
0x7ff...

The exact address can change between:

- runs
- machines
- compilers
- environments

So never memorize addresses.

34. WHY ADDRESSES CAN CHANGE

Modern operating systems may use mechanisms such as address-space layout randomization.

Even without going deep into OS details, remember:

An address seen in one run is not a permanent identity you should depend on.

35. VARIABLE SIZE AND ADDRESSES

Suppose:

```
c
int x;
```

If an `int` occupies 4 bytes on a particular system, it occupies multiple adjacent byte addresses.

Conceptually:

```
TEXT
5000
5001
5002
5003
```

may collectively represent one `int`.

This is why pointer arithmetic depends on the pointed type.

Later:

```
c
p + 1
```

does not necessarily mean:

```
TEXT
address + 1 byte
```

It means:

move to the next object of the pointed type.

Pointer arithmetic preview.

36. ARRAY CONNECTION

Consider:

```
C
int a[3] = {10, 20, 30};
```

Arrays are stored contiguously.

Conceptually:

```
TEXT
a[0]    a[1]    a[2]
 10      20      30
+-----+ +-----+ +-----+
```

If each int occupies 4 bytes:

```
TEXT
A d d r e s s   1 0 0 0   !'   a [ 0 ]
A d d r e s s   1 0 0 4   !'   a [ 1 ]
A d d r e s s   1 0 0 8   !'   a [ 2 ]
```

This explains why pointer arithmetic works naturally with arrays.

37. ADDRESS FORMULA PREVIEW

For an array:

```
TEXT
Address(a[i])
=
Base Address
+
i × size_of_element
```

We will formally derive this in the Arrays module.

Example:

```
TEXT
base = 1000
sizeof(int) = 4
```

Then:

```
TEXT
a[2]
address
=
1000 + 2×4
=
1008
```

0-6% TE loves address calculations.

38. STACK EXAMPLE WITH FUNCTION CALLS

Consider:

```
c
void B(void)
{
    int z = 30;
}

void A(void)
{
    int y = 20;
    B();
}

int main(void)
{
    int x = 10;
    A();
}
```

When `B()` is active:

TEXT

STACK

```
+-----+
| B()    |
| z = 30 |
+-----+
| A()    |
| y = 20 |
+-----+
| main() |
| x = 10 |
+-----+
```

Then `B` returns.

TEXT

```
+-----+
| A()    |
| y = 20 |
+-----+
| main() |
| x = 10 |
+-----+
```

Then `A` returns.

TEXT

```
+-----+
| main() |
| x = 10 |
+-----+
```

This is exactly the model we will use for recursion.

39. STATICLOCAL IS DIFFERENT

Consider:

```
c
void f(void)
{
    static int count = 0;
    int x = 10;
}
```

Conceptually:

```
TEXT
GLOBAL / STATIC STORAGE
+-----+
| count = 0 |
+-----+

STACK DURING f()
+-----+
| x = 10 |
+-----+
```

When `f()` returns:

```
TEXT
x
```

automatic lifetime ends.

But:

```
TEXT
count
```

continues to exist.

This explains Lesson 8.

40. HEAP OBJECT OUTLIVING FUNCTION CALL

Later:

```
c
int *create(void)
{
    int *p = malloc(sizeof(int));

    *p = 50;

    return p;
}
```

The pointer variable `p` itself is local.

But the dynamically allocated object can continue to exist after the function returns.

Conceptually during call:

```
TEXT
STACK
p % % % % % % % % % % % % % %
    |
    v
HEAP  +-----+
      | 50    |
      +-----+
```

After function returns, local `p` disappears, but if its address was returned and stored elsewhere, the allocated heap object can remain accessible.

This is a crucial dynamic-memory concept.

41. CONTRAST WITH RETURNING ADDRESS OF LOCAL

Suppose someone does:

```
c
int *bad(void)
{
    int x = 10;

    return &x;
}
```

`x` has automatic storage duration.

Its lifetime ends when `bad()` returns.

So returning its address creates a pointer that no longer refers to a live `x`.

This leads toward:

Dangling Pointer

We will study this carefully later.

Very important interview concept.

42. STORY — LOCAL ADDRESS

Imagine you receive:

Room number 205 in a temporary hotel.

Then the hotel closes.

You still have:

```
TEXT
205
```

written on paper.

But room 205 no longer represents your valid allocated room.

Similarly:

TEXT

```
pointer retains an address  
but object lifetime ended
```

!' dangling pointer.

43. STACK VS HEAP LIFETIME

Example automatic local:

```
C  
void f(void)  
{  
    int x = 10;  
}
```

Lifetime:

TEXT

```
enter f  
!"  
x exists  
!"  
leave f  
!"  
x lifetime ends
```

Dynamic object:

```
C  
p = malloc(...)
```

Lifetime broadly:

TEXT

```
malloc  
!"  
object exists  
!"  
continues across function calls  
!"  
free  
!"  
lifetime ends
```

44. MEMORY LEAK PREVIEW

Suppose:

```
c
p = malloc(...);
```

allocates memory.

But you lose every pointer to it and never call:

```
c
free(...)
```

Then the program can no longer usefully access that allocation, yet it remains allocated.

This is called:

Memory Leak

We'll study it in Dynamic Memory Allocation.

45. NULL POINTER PREVIEW

Sometimes we want a pointer to explicitly point to no valid object.

We use:

```
c
int *p = NULL;
```

Conceptually:

```
TEXT
p
!"
points to nothing
```

This is different from an uninitialized pointer.

We will study:

- NULL pointer
- wild pointer
- dangling pointer

in the pointer section.

46. STACK OVERFLOW VS HEAP EXHAUSTION

Do not confuse:

Stack overflow

Often caused by:

- excessively deep recursion

- huge stack allocations
- too many nested calls

Heap exhaustion/allocation failure

Dynamic allocator cannot satisfy a request.

Then functions like:

```
C
malloc()
```

may return:

```
TEXT
NULL
```

Different problems.

47. GATE FOCUS

Ø=Most nest-yield concepts:

- address of variable
- `&x`
- function-call stack
- automatic local lifetime
- static storage
- stack vs heap
- pointer stores address
- array contiguous memory
- dynamic memory lifetime
- dangling-pointer intuition
- recursion stack intuition

48. GATE TRAP — STACK "HEAP

Suppose:

```
C
void f(void)
{
    int x = 10;
}
```

Typical conceptual model:

TEXT

x !' s t a c k - r e l a t e d a u t o m a t i c s t o r a g e

But:

C

```
int *p = malloc(sizeof(int));
```

allocated object:

TEXT

* p o b j e c t !' h e a p

Pointer variable **p**, if local:

TEXT

p !' t y p i c a l l y s t a c k

while the object it points to:

TEXT

* p !' h e a p

Very important.

49. VISUALIZATION

C

```
void f(void)
{
    int *p = malloc(sizeof(int));
    *p = 25;
}
```

Conceptually:

TEXT

STACK

HEAP

+-----+		+-----+
p	----->	25
+-----+		+-----+

So asking:

Is p on stack or heap?

and:

Is the object pointed to by p on stack or heap?

are two different questions.

50. GATE / INTERVIEW TRAP

```
C
int *p = malloc(sizeof(int));
```

Do not say:

"p is allocated on heap."

More precise:

- local pointer variable `p` is an automatic local object
- `malloc` returns an address of dynamically allocated storage
- `p` stores that address

0-7% This level of precision is valuable in interviews.

51. COMPLEXITY CONNECTION

Stack push/pop for function-call bookkeeping is conceptually very fast.

But recursion depth matters for:

Auxiliary Space Complexity

Example:

```
C
factorial(n)
```

may have:

```
TEXT
n active calls
```

so recursion stack space:

```
TEXT
O(n)
```

Even if you don't explicitly allocate an array.

This is why space complexity includes call-stack usage.

52. EXAMPLE — RECURSION SPACE PREVIEW

Factorial:

```
C
fact(4)
```

calls:

```
TEXT
fact(4)
fact(3)
fact(2)
fact(1)
```

Before returning:

```
TEXT
STACK

fact(1)
fact(2)
fact(3)
fact(4)
main
```

Approximately:

```
TEXT
4 recursive frames
```

For `fact(n)`:

```
TEXT
O(n) stack space
```

We'll do this properly in the Recursion module.

53. INTERVIEW QUESTIONS

Q1. What is a memory address?

A value identifying a location in memory.

Q2. What does `&x` mean?

Address of `x`.

Q3. What does a pointer store?

An address.

Q4. What is stack memory commonly used for?

Function-call activation records and automatic local data.

Q5. What is heap memory commonly used for?

Dynamic memory allocation.

Q6. Difference between stack and heap?

Stack storage is typically managed automatically around function calls; heap storage is dynamically allocated and manually managed in C.

Q7. Is heap memory the same as a binary heap?

No. They are completely different concepts.

Q8. Why can excessive recursion cause stack overflow?

Because each active recursive call may consume stack space.

54. MEMORY TRICKS

Address

TEXT

& = WHERE?

C

&x

asks:

Where is x?

Pointer

Pointer points to a place.

Stack

Calls come in, returns go out.

LIFO.

Heap memory

Ask !' use !' free

TEXT

m a l l o c ! ' u s e ! ' f r e e

55. DON'T CONFUSE MASTER BOX

Stack Memory

Function-call/runtime storage.

Stack Data Structure

LIFO collection.

Heap Memory

Dynamic allocation area.

Binary Heap

Complete-tree-based priority structure.

Pointer Variable

Stores an address.

Pointed Object

Lives somewhere else and has its own value/lifetime.

56. NOTEBOOK NOTES

Memory Address

Each object exists at some memory location.

```
C
int x = 10;
```

TEXT

```
x    !'   v a l u e   =   1 0
& x  !'   a d d r e s s   o f   x
```

Conceptual Program Memory

TEXT

```
Code/Text
Global/Static
Heap
Stack
```

Stack

Typically associated with:

- function calls
- activation records
- automatic locals

Automatic local lifetime usually ends when its block/function exits.

Heap

Used for dynamic allocation:

```
C
malloc()
calloc()
realloc()
free()
```

Allocated object typically remains until explicitly released with `free`.

Pointer Preview

```
C
int x = 10;
int *p = &x;
```

Then:

```
TEXT
p    !'  a d d r e s s   o f   x
* p !'  v a l u e   o f   x
```

Important

```
TEXT
S t a c k   m e m o r y   " `   S t a c k   d a t a   s t r u c t u r e
H e a p   m e m o r y   " `   B i n a r y   h e a p
```

57. 30-SECOND REVISION

Remember these 10 ideas:

1. Every object exists somewhere in memory.
2. `x` gives the value of `x`.
3. `&x` gives the address of `x`.
4. A pointer stores an address.
5. Function calls are commonly managed using a stack.
6. Automatic locals typically belong to each active call.

7. Dynamic memory comes from heap storage.
 8. `malloc` requests heap memory; `free` releases it.
 9. Stack memory is not the stack data structure.
 10. Heap memory is not a binary heap.
-

58. ACTIVE RECALL

Try first.

Q1

If:

```
C
int x = 10;
```

what is the difference between:

```
TEXT
x
&x
```

?

Q2

Which memory concept is mainly associated with function calls?

Q3

Which memory concept is associated with `malloc()` ?

Q4

Are heap memory and binary heap the same?

Q5

Suppose:

```
C
int *p = malloc(sizeof(int));
```

If `p` is a local variable, what are the two separate objects involved?

ANSWERS

A1

```
TEXT
x    !'  v a l u e   s t o r e d   i n   x
& x !'  a d d r e s s   o f   x
```

A2

Stack.

A3

Heap.

A4

No.

A5

1. Pointer variable `p`
2. Dynamically allocated object whose address `p` stores

59. PRACTICE

Q1

Suppose conceptually:

TEXT

`x` is stored at address 1000
`x` contains 25

What are:

TEXT

`x`
`&x`

?

Q2

Which is most closely associated with each?

TEXT

- A. function activation record
- B. malloc allocation
- C. static global variable

Choose from:

TEXT

Stack
Heap
Global/Static storage

Q3 — FUNCTION STACK

Suppose:

```
TEXT
main calls A
A calls B
B calls C
```

When `C` is executing, write the conceptual stack from **top to bottom**.

~~Q4~~ — RETURN ORDER

For:

```
TEXT
main !'  A !'  B !'  C
```

what is the return order?

~~Q5~~ — STACK/HEAP THINKING

```
C
void f(void)
{
    int *p = malloc(sizeof(int));
}
```

Conceptually, where are:

1. local pointer variable `p`
2. dynamically allocated integer

typically associated?

~~Q6~~ — DANGLING POINTER INTUITION

Why is this dangerous?

```
C
int *f(void)
{
    int x = 10;
    return &x;
}
```

Explain using lifetime.

~~Q7~~ — ARRAY MEMORY

Assume:

TEXT

```
Base address = 1000  
sizeof(int) = 4
```

For:

C

```
int a[5];
```

what is the conceptual address of:

TEXT

```
a[3]
```

?

Use:

TEXT

```
Base + index × element size
```

Q8 — RECURSION SPACE

Suppose a recursive function creates exactly one new call with `n-1` until reaching `1`.

Approximately how many active recursive calls can exist for input `n`?

What auxiliary stack-space complexity should you expect?

Q9 — PLACEMENT THINKING

Why might a linked-list node normally be dynamically allocated rather than declared as an ordinary local variable inside a helper function?

Think about:

TEXT

```
lifetime
```

and whether the node must survive after that helper function returns.

PROGRESS

Completed '

Lesson 1: C program basics **Lesson 2:** Operators **Lesson 3:** Type conversion **Lesson 4:** Input/output **Lesson 5:** Decision making **Lesson 6:** Loops **Lesson 7:** Functions **Lesson 8:** Scope & lifetime **Lesson 9:** Memory basics

We have now built enough foundation to enter one of the most important sections in the entire Programming & Data Structures syllabus.

NEXT — POINTER MASTER SECTION

MODULE 0 / LESSON 10 — Pointers Part 1: Address, Pointer Declaration & Dereferencing

We will go extremely slowly and visually.

We will start from:

```
C
int x = 10;
int *p = &x;
```

and completely understand:

TEXT

x
&x
p
&p
*p

using memory diagrams like:

TEXT

Address 1000		Address 5000
+-----+		+-----+
5000	----->	10
+-----+		+-----+
p		x

Then we will answer the question that confuses many students:

If p contains an address, what exactly does *p do?

This is the foundation for arrays, linked lists, trees, dynamic memory, function modification, recursion, and many GATE questions.

Type NEXT.

USER

NEXT

ASSISTANT

MODULE 0 — LESSON 10

Pointers Part 1 — Address, Pointer Declaration & Dereferencing

We are now entering the **Pointer Master Section**, exactly as planned in your uploaded course prompt. This is one of

the most important areas for both GATE and placements, so we will go slowly and build the mental model correctly.

1. WHERE WE ARE

Module: 0 — C Foundation **Lesson:** 10 **Current topic:** Pointer basics

Why it matters: Pointers are required to properly understand:

- arrays
- strings
- dynamic memory
- linked lists
- trees
- graphs
- modifying variables through functions
- structures
- recursion-related memory
- many GATE C output questions

GATE importance: 0=Ý%0MÚ%0KÍ0W **Placement importance:** 0=Ý%0MÚ%0KÍ0W

Today we focus only on:

- memory address recap
- pointer meaning
- pointer declaration
- `&`
- `*`
- dereferencing
- `x`
- `&x`
- `p`
- `&p`
- `*p`
- modifying a variable through a pointer
- common pointer traps

We will **not rush into pointer arithmetic yet**.

2. STORY / INTUITION

Imagine your friend lives in a house.

Inside the house is:

TEXT

`1 1 0 , 0 0 0`

The house has an address:

TEXT

House No. 5000

Now suppose you write the house address on a piece of paper.

The paper contains:

TEXT

5000

That piece of paper is like a:

Pointer

The pointer doesn't contain the `110,000` directly.

It contains the **address of the place where `110,000` is stored.**

3. SIMPLE MEMORY MODEL

Suppose:

C

```
int x = 10;
```

Imagine `x` exists at address:

TEXT

5000

Memory:

TEXT

Address 5000

```
+-----+
|  10  |
+-----+
      x
```

Then:

TEXT

```
x = 10
```

and:

TEXT

```
&x = 5000
```

Remember:

TEXT

```
x    !'   W H A T   i s   i n s i d e ?
& x !'   W H E R E   i s   i t ?
```

4. NOW CREATE A POINTER

C

```
int *p = &x;
```

This means:

Create a pointer variable named `p` which can store the address of an `int`, and initialize it with the address of `x`.

Suppose:

TEXT

```
&x = 5000
```

Then:

TEXT

```
p = 5000
```

5. FULL VISUALIZATION

Suppose `p` itself is stored at address `1000`.

TEXT

Address 1000		Address 5000
+-----+		+-----+
5000	----->	10
+-----+		+-----+
p		x

Interpretation:

TEXT

```
p contains 5000
```

```
5000 is address of x
```

```
x contains 10
```

So:

TEXT

```
p = &x
```

6. WHAT IS A POINTER?

Technical definition:

A pointer is an object whose value is an address of another object or function.

For our beginner understanding:

Pointer = variable that stores an address.

Example:

```
C
int *p;
```

means:

p is a pointer to int .

7. POINTER DECLARATION

Syntax:

```
C
data_type *pointer_name;
```

Examples:

```
C
int *p;
```

Pointer to integer.

```
C
char *p;
```

Pointer to character.

```
C
float *p;
```

Pointer to float.

```
C
double *p;
```

Pointer to double.

8. WHY DOES POINTER HAVE A TYPE?

Suppose:

```
C
int *p;
```

This tells C:

The address stored in `p` is intended to point to an `int`.

The type matters for:

- dereferencing
- pointer arithmetic
- type checking
- interpreting memory

Later:

```
C
p + 1
```

depends on what type `p` points to.

9. THE TWO MOST IMPORTANT OPERATORS

Pointers begin with two operators:

&

Address-of operator.

Dereference / indirection operator when used with a pointer expression.

10. & — ADDRESS OF

Consider:

```
C
int x = 10;
```

Then:

```
C
&x
```

means:

Give me the address of `x` .

If we imagine:

TEXT

`x` lives at 5000

then:

TEXT

`&x = 5000`

11. `*` — DEREFERENCE

Suppose:

C

```
int *p = &x;
```

and:

TEXT

`p = 5000`

What does:

C

`*p`

mean?

It means:

Go to the address stored in `p` and access the object/value there.

Since `p = 5000` , go to memory location 5000 .

At 5000 :

TEXT

10

Therefore:

TEXT

`*p = 10`

12. STORY FOR DEREFERENCING

Suppose `p` contains:

TEXT

House No. 5000

Then:

`c`

`*p`

means:

Go to House 5000 and look inside.

If House 5000 contains:

TEXT

10

then:

TEXT

`*p = 10`

Memory trick:

`p` = address `*p` = value at that address

13. MASTER POINTER EXAMPLE

`c`

```
int x = 10;
int *p = &x;
```

Assume:

TEXT

x stored at 5000
p stored at 1000

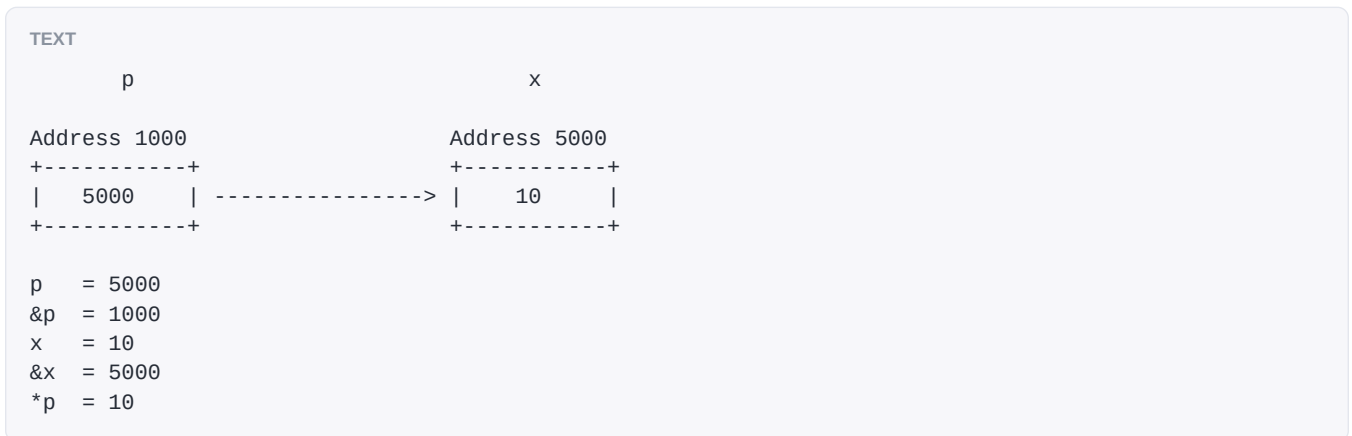
Then:

Expression	Meaning	Conceptual value
<code>x</code>	value of <code>x</code>	10
<code>&x</code>	address of <code>x</code>	5000

Expression	Meaning	Conceptual value
p	address stored in p	5000
&p	address of pointer p	1000
*p	value at address p	10

0-Memorize through understanding, not rote learning.

14. VISUAL MASTER DIAGRAM



This diagram is the foundation for almost everything that comes later.

15. IMPORTANT RELATIONSHIP

If:

```

c
p = &x;

```

then:

```

TEXT
p == &x

```

because both represent the address of `x`.

And:

```

TEXT
*p == x

```

because dereferencing the pointer accesses `x`.

So:

TEXT

```
p = &x
*p = x's value
```

16. SIMPLE CODE

C

```
#include <stdio.h>

int main(void)
{
    int x = 10;
    int *p = &x;

    printf("%d\n", x);
    printf("%d\n", *p);

    return 0;
}
```

Output:

TEXT

```
10
10
```

Why?

Because `p` points to `x`.

17. PRINTING ADDRESSES

To print addresses:

C

```
printf("%p", (void *)&x);
```

Similarly:

C

```
printf("%p", (void *)p);
```

Since:

TEXT

```
p = &x
```

these should represent the same address.

Exact displayed address is implementation-dependent.

18. POINTER VARIABLE ALSO HAS MEMORY

A pointer is itself a variable.

So:

```
C
int *p = &x;
```

means `p` also occupies memory somewhere.

Therefore:

```
TEXT
p
```

and:

```
TEXT
&p
```

are different.

19. DON'T CONFUSE `p` AND `&p`

Suppose:

```
TEXT
p stored at 1000
p contains 5000
```

Then:

```
TEXT
p  = 5000
&p = 1000
```

`p` gives the address stored inside the pointer.

`&p` gives the location of the pointer variable itself.

20. VERY IMPORTANT STORY

Think of `p` as a piece of paper.

The paper itself is stored in locker:

```
TEXT
1000
```

On the paper is written:

TEXT

5000

So:

TEXT

&p = where the paper is stored

p = what address is written on paper

That distinction will matter when we study pointer-to-pointer.

21. MODIFYING `x` THROUGH POINTER

This is where pointers become powerful.

Consider:

C

```
int x = 10;  
int *p = &x;
```

Now:

C

```
*p = 50;
```

What happens?

`p` points to `x`.

So `*p` refers to `x`.

Therefore:

TEXT

x becomes 50

22. VISUALIZATION

Before:

TEXT

```
p -----> x  
                10
```

Execute:

C

```
*p = 50;
```

After:

TEXT

p -----> x
50

So:

TEXT

x = 50
*p = 50

23. CODE EXAMPLE

C

```
#include <stdio.h>

int main(void)
{
    int x = 10;
    int *p = &x;

    *p = 50;

    printf("%d", x);

    return 0;
}
```

Output:

TEXT

50

0-10 is one of the central powers of pointers.

24. WHY DID `x` CHANGE?

Because:

C

*p

is not a copy of `x`.

It accesses the object located at the address stored in `p`.

Since that object is `x`, assigning through `*p` modifies `x`.

25. CONNECTION TO CALL BY VALUE

Remember:

```
C
void change(int x)
{
    x = 100;
}
```

does not modify caller's variable.

But now:

```
C
void change(int *p)
{
    *p = 100;
}
```

Then:

```
C
int x = 10;

change(&x);
```

will modify `x`.

Why?

Because:

```
TEXT
&x
```

is passed.

The function receives the address.

Then:

```
C
*p = 100
```

changes the object at that address.

26. VERY IMPORTANT PRECISION

Even here, C is still:

Call by Value

What is copied into the function?

The pointer value / address.

Example:

TEXT

```
&x = 5000
```

Call:

C

```
change(&x);
```

Function receives a copy:

TEXT

```
p = 5000
```

But both the copied pointer and original address refer to the same object `x`.

Therefore modification reaches the caller's object.

0-16 Interview-level wording.

27. FUNCTION VISUALIZATION

C

```
void change(int *p)
{
    *p = 100;
}
```

Caller:

C

```
int x = 10;
change(&x);
```

Conceptually:

TEXT

main:

```
x at 5000
+-----+
|  10  |
+-----+
```

change:

```
p contains 5000
  |
  v
+-----+
|  10  |
+-----+
      x
```

Then:

```
C
*p = 100;
```

becomes:

```
TEXT
x = 100
```

28. POINTER DECLARATION TRAP

Consider:

```
C
int *p;
```

The `*` tells us:

```
TEXT
p is pointer to int
```

But in:

```
C
*p = 10;
```

the `*` is used as dereference operator.

Same symbol.

Different role depending on context.

DON'T CONFUSE

Declaration

```
C
int *p;
```

means:

Declare `p` as pointer to int.

Expression

```
C
*p
```

means:

Access the object pointed to by `p`.

29. MULTIPLE DECLARATIONS TRAP Ø=Ý%

Consider:

```
C
int *p, q;
```

Many beginners think both are pointers.

Wrong.

This declares:

```
TEXT
p !'  p o i n t e r   t o   i n t
q !'  o r d i n a r y   i n t
```

Why?

The `*` belongs syntactically to the declarator `p`.

If both should be pointers:

```
C
int *p, *q;
```

MEMORY TRICK

Avoid:

```
C
int* p, q;
```

if it confuses you.

Prefer:

```
C
int *p;
int *q;
```

Very clear.

30. UNINITIALIZED POINTER — DANGER

Consider:


```
c
int *p;
```

What does `p` point to?

Nothing valid has been assigned.

Its value is indeterminate.

Using:

```
c
*p
```

is dangerous/undefined.

This kind of pointer is often informally called a:

Wild Pointer

We will study pointer errors in detail later.

31. BAD EXAMPLE

```
c
int *p;

*p = 10;
```

Problem:

`p` does not contain a valid address of an `int` object.

So dereferencing it is undefined behavior.

~~Never~~ Never dereference a pointer before it points to a valid object.

32. SAFE VERSION

```
c
int x;
int *p = &x;

*p = 10;
```

Now:

```
TEXT
p !'  v a l i d   a d d r e s s   o f   x
```

So:

```
c
*p = 10;
```

is safe.

33. NULL POINTER PREVIEW

You can explicitly say:

```
c
int *p = NULL;
```

This means:

p currently points to no valid object.

This is safer than leaving it uninitialized conceptually.

But:

```
c
*p
```

when `p == NULL` is still invalid.

Never dereference `NULL`.

34. NULL VS UNINITIALIZED

Uninitialized pointer

```
c
int *p;
```

Value is indeterminate.

NULL pointer

```
c
int *p = NULL;
```

Explicitly represents no valid target.

These are different.

35. POINTER SIZE — IMPORTANT CONCEPT

A pointer stores an address.

Its size depends mainly on the platform/address model, not on the size of the pointed object.

For example, on many 64-bit systems:

```
TEXT
sizeof(pointer)
"H    8    b y t e s
```

but do not assume universally unless specified.

Interesting:

```
C
int *p;
char *q;
double *r;
```

Their pointed types differ, but pointer object sizes are often equal on a given platform.

However, C does not require all object pointer representations to always be identical in every conceivable implementation.

For GATE, follow given assumptions.

36. POINTER TYPE MATTERS EVEN IF SIZE MAY MATCH

Consider:

```
C
int *p;
char *q;
```

Even if both pointers occupy the same number of bytes on a machine:

```
TEXT
p + 1
```

and:

```
TEXT
q + 1
```

behave differently.

Why?

Because pointer arithmetic uses the pointed type.

Later:

```
TEXT
int *p
p + 1 != next int
```

while:

TEXT

```
char *q
q + 1 != next char
```

We'll study this next.

37. SOLVED EXAMPLE 1

C

```
int x = 20;
int *p = &x;

printf("%d %d", x, *p);
```

Since:

TEXT

p points to x

Output:

TEXT

20 20

38. SOLVED EXAMPLE 2

C

```
int x = 20;
int *p = &x;

*p = 40;

printf("%d %d", x, *p);
```

After:

C

```
*p = 40;
```

x becomes 40.

Output:

TEXT

40 40

39. SOLVED EXAMPLE 3

```
c
int x = 5;
int *p = &x;

x = 10;

printf("%d", *p);
```

Does `p` remember old value 5?

No.

`p` stores the **address**, not a copy of the old value.

At that address, `x` is now:

```
TEXT
10
```

Therefore:

```
TEXT
*p = 10
```

Output:

```
TEXT
10
```

Very important.

40. STORY — ADDRESS STAYS, CONTENT CHANGES

`p` says:

Go to house 5000.

Initially house contains:

```
TEXT
5
```

Later house contains:

```
TEXT
10
```

The address did not need to change.

So:

TEXT

```
p !' same house
* p !' current content of house
```

41. SOLVED EXAMPLE 4

C

```
int x = 5;
int y = 10;

int *p = &x;

p = &y;

printf("%d", *p);
```

Trace:

Initially:

TEXT

```
p !' x
```

Then:

C

```
p = &y;
```

Now:

TEXT

```
p !' y
```

Therefore:

TEXT

```
*p = 10
```

Output:

TEXT

```
10
```

42. IMPORTANT — POINTER CAN CHANGE TARGET

A pointer variable can store different valid addresses at different times.

TEXT

```
p !' x
```

then:

TEXT

p !' y

Its target can change.

43. VISUALIZATION

Before:

TEXT

p -----> x = 5
y = 10

After:

C

p = &y;

TEXT

x = 5
p -----> y = 10

44. SOLVED EXAMPLE 5

C

```
int x = 5;  
int y = 10;  
  
int *p = &x;  
  
*p = y;
```

What changes?

*p refers to x .

y has value 10 .

So:

TEXT

x = 10

But p still points to x .

After:

TEXT

```
x = 10
y = 10
p = &x
```

Don't confuse:

```
c
p = &y;
```

with:

```
c
*p = y;
```

45. DON'T CONFUSE — CRITICAL

Case 1

```
c
p = &y;
```

Changes the **pointer**.

Now `p` points to `y`.

Case 2

```
c
*p = y;
```

Changes the **value of the object pointed to by p**.

The pointer's address stays unchanged.

Extremely important.

46. MEMORY DIAGRAM

Start:

```
TEXT
p -----> x = 5
          y = 10
```

Execute:


```
c
*p = y;
```

Result:

```
TEXT
p -----> x = 10
          y = 10
```

p still points to x.

47. GATE STYLE — WHAT CHANGES?

```
c
int a = 10;
int b = 20;

int *p = &a;

p = &b;
*p = 30;
```

Trace:

Initially:

```
TEXT
p !' a
```

Then:

```
TEXT
p !' b
```

Then:

```
TEXT
*p = 30
```

means:

```
TEXT
b = 30
```

Final:

```
TEXT
a = 10
b = 30
```

48. ADDRESS EQUALITY

If:

```
C
int *p = &x;
```

then:

```
C
p == &x
```

is true.

But:

```
C
&p == &x
```

is not the same question.

`&p` is location of pointer variable.

`&x` is location of integer variable.

49. POINTER TO SAME VARIABLE

```
C
int x = 10;

int *p = &x;
int *q = &x;
```

Now:

```
TEXT
p  !'  x
q  !'  x
```

So:

```
TEXT
*p = 10
*q = 10
```

If:

```
C
*p = 50;
```

then `x` becomes 50.

Therefore:

TEXT

*q = 50

because both point to the same object.

50. STORY — TWO MAPS, SAME HOUSE

p and q are two pieces of paper.

Both have written:

TEXT

House 5000

If something inside House 5000 changes, both directions still lead to the same updated house.

51. SOLVED EXAMPLE

C

```
int x = 10;

int *p = &x;
int *q = &x;

*q = 25;

printf("%d %d %d", x, *p, *q);
```

Output:

TEXT

25 25 25

52. POINTER AND TYPE SAFETY

Suppose:

C

```
int x = 10;
double *p = &x;
```

This is an incompatible pointer assignment.

An `int *` and `double *` are different pointer types.

A compiler should diagnose incompatible pointer use.

Do not casually mix pointer types.

53. VOID POINTER — ONLY A PREVIEW

C provides:

```
C
void *p;
```

A `void *` can hold addresses of objects of various types.

But you cannot directly dereference a `void *` as though C knows what object type is there.

We'll study this later only if useful.

Ø=KINOW, not today's focus.

54. GATE FOCUS

For Pointer Part 1, GATE loves questions involving:

- `&`
- `*`
- changing values through pointers
- pointer reassignment
- aliasing
- function modification
- local copies vs addresses
- pointer variable vs pointed variable
- multiple pointers to same object

55. ALIASING — IMPORTANT WORD

Suppose:

```
C
int x = 10;

int *p = &x;
int *q = &x;
```

Both `p` and `q` provide ways to access the same object.

This is an example of:

Aliasing

Multiple expressions/references can access the same memory object.

This matters in optimization and program reasoning.

For now:

p and q both point to x.

56. GATE TRAP SHEET

Trap 1

```
c
p
```

is not the same as:

```
c
*p
```

Trap 2

```
c
&p
```

is not the same as:

```
c
p
```

Trap 3

```
c
p = &y;
```

changes pointer target.

Trap 4

```
c
*p = y;
```

changes pointed value.

Trap 5

```
c
int *p, q;
```

only `p` is a pointer.

Trap 6

An uninitialized pointer must not be dereferenced.

Trap 7

`NULL` pointer also must not be dereferenced.

57. PLACEMENT CONNECTION

Pointers appear everywhere in C-based DSA.

Later you'll see:

```
C
struct Node
{
    int data;
    struct Node *next;
};
```

Here:

```
C
next
```

is a pointer.

It stores the address of another node.

So linked list:

```
TEXT
[10 | •] ----> [20 | •] ----> [30 | NULL]
```

is really:

```
TEXT
node contains data
+
address of next node
```

Without pointer understanding, linked lists look mysterious.

With pointers, they become natural.

58. TREE CONNECTION

Binary tree node:

```

c
struct Node
{
    int data;
    struct Node *left;
    struct Node *right;
};

```

Meaning:

TEXT

```

l e f t   ! '   a d d r e s s   o f   l e f t   c h i l d
r i g h t   ! '   a d d r e s s   o f   r i g h t   c h i l d

```

So trees are fundamentally linked using pointers.

59. INTERVIEW QUESTIONS

Q1. What is a pointer?

A variable/object that stores an address.

Q2. What does `&x` mean?

Address of `x`.

Q3. What does `*p` mean?

Access the object/value at the address stored in `p`.

Q4. If `p = &x`, what is `*p`?

The value of `x`.

Q5. Does `p = &y` change `y`?

No. It changes the pointer to point to `y`.

Q6. Does `*p = 10` change `p`?

No. It changes the object pointed to by `p`.

Q7. Why are pointer types needed?

For correct dereferencing, pointer arithmetic, and type checking.

60. MEMORY TRICKS

`&`

Think:

`&` = WHERE

```
c
&x
```

Where is x?

```
*
```

Think:

```
*\ = GO THERE**
```

```
c
*p
```

Go to the address stored in `p`.

Master relation

```
TEXT
p = &x
```

means:

```
TEXT
p points to x
```

and:

```
TEXT
*p = x
```

in terms of current value.

61. POINTER TRACE METHOD

Whenever you see pointer code:

STEP 1

Write all ordinary variables.

Example:

```
TEXT
x = 10
y = 20
```

STEP 2

Write addresses symbolically if exact addresses aren't needed.

Example:

TEXT

p !' x

instead of inventing numbers.

STEP 3

For:

C

p = &y;

move the arrow.

STEP 4

For:

C

*p = 30;

change the box at the arrow's destination.

This is the easiest way to solve pointer questions.

62. EXAMPLE USING ARROWS

C

int x = 10;

int y = 20;

int *p = &x;

*p = 15;

p = &y;

*p = 25;

Start:

TEXT

p !' x

x = 10

y = 20

After:

C

*p = 15;

TEXT

```
x = 15
```

Then:

C

```
p = &y;
```

TEXT

```
p != y
```

Then:

C

```
*p = 25;
```

TEXT

```
y = 25
```

Final:

TEXT

```
x = 15
```

```
y = 25
```

63. NOTEBOOK NOTES

Pointer

A pointer stores an address.

C

```
int x = 10;  
int *p = &x;
```

Then:

TEXT

```
x      !'  1 0  
&x     !'  a d d r e s s   o f   x  
p      !'  a d d r e s s   o f   x  
&p     !'  a d d r e s s   o f   p  
*p     !'  1 0
```

Address-of

```
C
&x
```

gives address of `x`.

Dereference

```
C
*p
```

accesses the object pointed to by `p`.

Modify through pointer

```
C
*p = 50;
```

changes the pointed object.

Important Difference

```
C
p = &y;
```

changes pointer target.

```
C
*p = y;
```

changes pointed value.

Call by value + pointer

C still passes pointers by value.

Passing an address allows a function to modify the original caller object.

64. 30-SECOND REVISION

Remember:

1. Pointer stores an address.
2. `&x` = address of `x`.
3. `p = &x` means `p` points to `x`.
4. `*p` = value/object at address stored in `p`.

5. `&p` = address of pointer variable itself.
 6. `*p = 50` changes the pointed object.
 7. `p = &y` changes the target.
 8. Two pointers can point to the same object.
 9. Never dereference an uninitialized or NULL pointer.
 10. Pointers make linked lists, trees, dynamic memory, and caller modification possible.
-

65. ACTIVE RECALL

Try without looking at answers.

Q1

If:

```
C
int x = 10;
int *p = &x;
```

what are:

```
TEXT
x
&x
p
*p
```

conceptually?

Q2

What does:

```
C
*p = 50;
```

do?

Q3

Difference between:

```
C
p = &y;
```

and:

```
C
*p = y;
```

?

Q4

In:

```
c
int *p, q;
```

which variables are pointers?

Q5

Why is this dangerous?

```
c
int *p;
printf("%d", *p);
```

ANSWERS

A1

TEXT

```
x      !'   v a l u e   1 0
& x    !'   a d d r e s s   o f   x
p      !'   a d d r e s s   o f   x
* p    !'   v a l u e   1 0
```

A2

Changes the object pointed to by `p` to `50`.

A3

TEXT

```
p = &y
!'   p   n o w   p o i n t s   t o   y
```

TEXT

```
*p = y
!'   c o p y   y ' s   v a l u e   i n t o   o b j e c t   c u r r e n t l y   p o i n t e d   t o   b y   p
```

A4

Only:

TEXT

```
p
```

is a pointer.

`q` is an ordinary `int`.

A5

`p` has not been initialized to a valid address, so dereferencing it is undefined behavior.

66. PRACTICE

Q1

Find final value of `x` :

```
C
int x = 10;
int *p = &x;

*p = 30;
```

Q2

Find output:

```
C
int x = 15;
int *p = &x;

printf("%d %d", x, *p);
```

Q3 — POINTER REASSIGNMENT

```
C
int x = 5;
int y = 10;

int *p = &x;

p = &y;

printf("%d", *p);
```

Find output.

Q4 — POINTED VALUE

```
C
int x = 5;
int y = 20;

int *p = &x;

*p = y;
```

Find final:

TEXT

x
y

and tell me what `p` still points to.

Q5 — TWO POINTERS

```
C
int x = 10;

int *p = &x;
int *q = &x;

*p = 25;

printf("%d %d %d", x, *p, *q);
```

Find output.

Q6 — TRACE

```
C
int a = 10;
int b = 20;

int *p = &a;

*p = 15;
p = &b;
*p = 30;
```

Find final:

TEXT

a
b

Q7 — FUNCTION + POINTER

```
C
void change(int *p)
{
    *p = 100;
}

int main(void)
{
    int x = 10;

    change(&x);
}
```

```
    printf("%d", x);  
}
```

Find output and explain why this differs from:

```
C  
void change(int x)  
{  
    x = 100;  
}
```

Q4 — GATE THINKING

Assume conceptually:

```
TEXT  
x lives at address 5000  
p lives at address 1000
```

and:

```
C  
int x = 20;  
int *p = &x;
```

Find:

```
TEXT  
x  
&x  
p  
&p  
*p
```

Q9 — PLACEMENT THINKING

Explain in one sentence why a linked-list node needs a pointer member such as:

```
C  
struct Node *next;
```

What information is stored inside `next` ?

PROGRESS

Completed '

Lesson 1: C basics **Lesson 2:** Operators **Lesson 3:** Type conversion **Lesson 4:** Input/output **Lesson 5:** Decision making **Lesson 6:** Loops **Lesson 7:** Functions **Lesson 8:** Scope & lifetime **Lesson 9:** Memory basics **Lesson 10:** Pointer basics — `&`, `*`, declaration and dereferencing

Next

MODULE 0 / LESSON 11 — Pointers Part 2: Pointer Arithmetic + Arrays

Next we will deeply understand:

```
C
p + 1
p - 1
p++
p--
```

and why:

```
C
int *p;
```

may move by an entire `int`, while:

```
C
char *p;
```

moves by a character-sized element.

Then we will connect:

```
C
int a[3] = {10, 20, 30};
```

with:

```
TEXT
a
&a[0]
p
*(p+1)
a[1]
```

This is one of the most important **GATE pointer + array** areas.

Type **NEXT**.

USER

NEXT

ASSISTANT

MODULE 0 — LESSON 11

Pointers Part 2 — Pointer Arithmetic + Arrays

We continue the Pointer Master Section from your uploaded course plan. This lesson is one of the most important bridges between **C pointers** and **arrays/data structures**.

1. WHERE WE ARE

Module: 0 — C Foundation **Lesson:** 11 **Current topic:** Pointer arithmetic and array-pointer relationship

Why it matters: Many GATE questions combine:

- pointers
- arrays
- memory addresses
- `p + 1`
- `*(p + i)`
- `a[i]`
- `&a[i]`

This also becomes essential for placements.

GATE importance: MUST KNOW **Placement importance:** MUST KNOW

Today we cover:

- `p + 1`
- `p - 1`
- `p++`
- `p--`
- why pointer arithmetic depends on type
- arrays in contiguous memory
- `a`
- `&a[0]`
- `a + 1`
- `*(a + 1)`
- relationship between `a[i]` and `*(a+i)`
- pointer subtraction basics
- valid and invalid pointer arithmetic
- major GATE traps

2. STORY / INTUITION

Imagine a row of houses.

Each house belongs to one family and occupies the same amount of space.

TEXT

House 1	House 2	House 3
10	20	30

Suppose the first house begins at address:

TEXT

1000

and every house occupies:

TEXT

4 units

Then:

TEXT

```
H o u s e   1   !'   1 0 0 0
H o u s e   2   !'   1 0 0 4
H o u s e   3   !'   1 0 0 8
```

Now suppose a pointer points to House 1.

If you say:

C

p + 1

you do **not** usually mean:

Move one raw byte.

You mean:

Move to the next object of the pointed type.

That is the key idea of pointer arithmetic.

3. POINTER ARITHMETIC — BASIC IDEA

Suppose:

C

```
int x[3] = {10, 20, 30};

int *p = &x[0];
```

Assume:

TEXT

sizeof(int) = 4

and:

TEXT

&x[0] = 1000

Then:

TEXT

```
p      = 1000
p + 1  = 1004
p + 2  = 1008
```

because `p` is an:

C

```
int *
```

So moving by 1 means:

Move by one `int` .

4. MASTER RULE

If:

C

```
T *p;
```

then conceptually:

TEXT

```
p + k
```

points `k` elements forward, where each element has type `T` .

Address movement:

TEXT

```
k × sizeof(T)
```

~~0 = 1000~~ Important formula.

FORMULA BOX

For conceptual address calculation:

TEXT

```
New Address
=
Old Address + k × sizeof(pointed type)
```

Example:

TEXT

```
int *p
sizeof(int) = 4
p = 1000
```

Then:

TEXT

```
p + 3
= 1000 + 3×4
= 1012
```

5. WHY TYPE MATTERS

Compare:

C

```
int *p;
char *q;
```

Suppose both conceptually contain:

TEXT

1000

If:

TEXT

```
sizeof(int) = 4
sizeof(char) = 1
```

then:

TEXT

```
p + 1 !' 1 0 0 4
q + 1 !' 1 0 0 1
```

So pointer arithmetic depends on the pointed type.

6. MEMORY TRICK

Think:

Pointer arithmetic counts objects, not raw bytes.

TEXT

```
int * !' jump one int
char * !' jump one char
```

7. p++

Suppose:

```
c
int *p = &a[0];
```

Then:

```
c
p++;
```

means:

Move pointer to next `int` .

So:

```
TEXT
Before:
p !' a [ 0 ]

After:
p !' a [ 1 ]
```

8. p--

Similarly:

```
c
p--;
```

moves back by one element of the pointed type.

Example:

```
TEXT
p !' a [ 2 ]
```

then:

```
c
p--;
```

gives:

```
TEXT
p !' a [ 1 ]
```

9. ARRAY MEMORY

Consider:

```
c
int a[4] = {10, 20, 30, 40};
```

Array elements are stored contiguously.

Suppose:

```
TEXT
Base address = 1000
sizeof(int) = 4
```

Then:

```
TEXT
a [ 0 ] !'  a d d r e s s  1 0 0 0 !'  v a l u e  1 0
a [ 1 ] !'  a d d r e s s  1 0 0 4 !'  v a l u e  2 0
a [ 2 ] !'  a d d r e s s  1 0 0 8 !'  v a l u e  3 0
a [ 3 ] !'  a d d r e s s  1 0 1 2 !'  v a l u e  4 0
```

Visualization:

```
TEXT
1000      1004      1008      1012
+-----+ +-----+ +-----+ +-----+
| 10 |    | 20 |    | 30 |    | 40 |
+-----+ +-----+ +-----+ +-----+
a[0]      a[1]      a[2]      a[3]
```

10. ARRAY NAME **a**

For most expressions, an array name such as:

```
c
a
```

is converted ("decays") to a pointer to its first element.

So in many expressions:

```
TEXT
a
```

behaves like:

```
TEXT
&a[0]
```

Thus:

```
TEXT
a !' address of first element
```

But this is not true in every context. We will cover exceptions.

11. SIMPLE RELATION

For:

```
C
int a[3];
```

in ordinary expression contexts:

```
TEXT
a == &a[0]
```

in terms of address value.

So:

```
C
int *p = a;
```

is effectively like:

```
C
int *p = &a[0];
```

12. CODE EXAMPLE

```
C
int a[3] = {10, 20, 30};

int *p = a;

printf("%d", *p);
```

Since:

```
TEXT
p !' a [ 0 ]
```

then:

```
TEXT
*p = 10
```


Output:

```
TEXT
10
```

13. NOW `a + 1`

Since `a` behaves like pointer to first element in this context:

```
TEXT
a !' & a [ 0 ]
```

Then:

```
TEXT
a + 1 !' & a [ 1 ]
```

Similarly:

```
TEXT
a + 2 !' & a [ 2 ]
```

14. DEREFERENCE

If:

```
TEXT
a + 1 !' a d d r e s s o f a [ 1 ]
```

then:

```
C
*(a + 1)
```

means:

Go to `a[1]` and get its value.

So:

```
TEXT
*(a + 1) = a[1]
```

15. MASTER ARRAY-POINTER RELATION

Remember this with understanding:

TEXT

```
a[i] == *(a + i)
```

And:

TEXT

```
&a[i] == a + i
```

These are fundamental.

16. EXAMPLE

C

```
int a[4] = {10, 20, 30, 40};
```

Then:

TEXT

```
a[0]      = 10
*(a + 0)   = 10
```

```
a[1]      = 20
*(a + 1)   = 20
```

```
a[2]      = 30
*(a + 2)   = 30
```

17. VISUALIZATION

TEXT

```
a
|
v
+---+---+---+---+
| 10 | 20 | 30 | 40 |
+---+---+---+---+
  0   1   2   3
```

Then:

TEXT

```
a + 0 !' f i r s t   b o x
a + 1 !' s e c o n d b o x
a + 2 !' t h i r d   b o x
```

And:

TEXT

```
* ( a + 0 ) !'   1 0
* ( a + 1 ) !'   2 0
* ( a + 2 ) !'   3 0
```

18. `p[i]` ALSO WORKS

Suppose:

```
C
int *p = a;
```

Then:

```
C
p[2]
```

means:

```
C
*(p + 2)
```

So if:

```
TEXT
a = {10, 20, 30}
```

then:

```
TEXT
p[2] = 30
```

19. IMPORTANT IDENTITY

In C:

```
TEXT
p[i] == *(p + i)
```

for valid pointer/array indexing.

This is why arrays and pointers are deeply connected.

20. STRANGE BUT VALID C FACT

Because:

```
TEXT
a[i]
```

is defined through pointer arithmetic as:

TEXT

```
*(a + i)
```

and addition is commutative:

TEXT

```
a + i == i + a
```

this means:

C

```
i[a]
```

can also mean:

C

```
*(i + a)
```

So:

C

```
2[a]
```

is equivalent to:

C

```
a[2]
```

0=Weird but real GATE-style fact.

Do **not** write code like this in real programs.

21. EXAMPLE

C

```
int a[3] = {10, 20, 30};  
  
printf("%d", 2[a]);
```

Equivalent to:

C

```
a[2]
```

Output:

TEXT

```
30
```

This may appear as a tricky question.

22. `a` VS `&a[0]`

In many expressions, these produce the same starting address value.

But their types originate differently:

TEXT

`a`

after decay:

TEXT

`int *`

while:

TEXT

`&a[0]`

is directly:

TEXT

`int *`

So they are very closely related in ordinary element access.

23. NOW IMPORTANT — `&a`

Suppose:

C

`int a[5];`

What is:

C

`&a`

?

It means:

Address of the whole array object.

Type:

TEXT

pointer to array of 5 int

Conceptually:

```
C
int (*)[5]
```

This is different from:

```
TEXT
a
```

which usually decays to:

```
C
int *
```

0-Major GATE trap.

24. SAME STARTING ADDRESS, DIFFERENT TYPE

Suppose array begins at:

```
TEXT
1000
```

Then numerically:

```
TEXT
a
&a[0]
&a
```

may all display the same starting address.

But their types differ.

This causes different pointer arithmetic.

25. EXAMPLE — `a + 1`

Assume:

```
C
int a[5];
sizeof(int) = 4;
```

Array size:

```
TEXT
5 × 4 = 20 bytes
```

If base address is:

TEXT

1000

then:

TEXT

a + 1

means next integer:

TEXT

1004

26. EXAMPLE — `&a + 1`

Now:

TEXT

&a

points to the **whole array**.

So:

TEXT

&a + 1

means:

Move by one whole array.

Array size = 20 bytes.

Thus:

TEXT

1000 + 20 = 1020

Extremely important.

27. DON'T CONFUSE

For:

C

int a[5];

Assume base = 1000, `sizeof(int)=4`.

TEXT

```
a + 1
!' 1 0 0 4
```

but:

TEXT

```
&a + 1
!' 1 0 2 0
```

Even though:

TEXT

```
a
```

and:

TEXT

```
&a
```

start at the same address.

Their **types** are different.

28. GATE MASTER TABLE

For:

```
c
int a[5];
```

Expression	Meaning
a	pointer to first element in most expressions
&a[0]	address of first element
a+1	address of second element
*(a+1)	second element
&a	address of whole array
&a+1	address just after whole array

Copy this.

29. ARRAY NAME IS NOT AN ORDINARY POINTER VARIABLE

This is crucial.

Suppose:


```
c
int a[5];
int *p = a;
```

Can we do:

```
c
p++;
```

Yes.

`p` is a pointer variable.

But can we do:

```
c
a++;
```

No.

Why?

Because `a` is an array object/name, not a modifiable pointer variable.

~~0-Mis~~ Major misconception.

30. STORY

Think:

`p` is a movable arrow.

```
TEXT
p !' a [ 0 ]
```

You can change it:

```
TEXT
p !' a [ 1 ]
```

But `a` identifies the actual array.

You cannot make the array name suddenly refer to somewhere else.

31. CODE

Valid:

```
c
int a[3];
int *p = a;

p++;
```

Invalid:

```
c
a++;
```

32. POINTER TRAVERSAL OF ARRAY

```
c
int a[4] = {10, 20, 30, 40};

int *p = a;

for (int i = 0; i < 4; i++)
{
    printf("%d ", *p);
    p++;
}
```

Trace:

TEXT

```
p !'  a [ 0 ] !'  p r i n t   1 0
p !'  a [ 1 ] !'  p r i n t   2 0
p !'  a [ 2 ] !'  p r i n t   3 0
p !'  a [ 3 ] !'  p r i n t   4 0
```

Output:

TEXT

```
10 20 30 40
```

33. BETTER VERSION WITHOUT MOVING `p`

```
c
for (int i = 0; i < 4; i++)
{
    printf("%d ", *(p + i));
}
```

Output:

TEXT

```
10 20 30 40
```

34. POINTER DIFFERENCE

Suppose:

```
C
int a[10];

int *p = &a[2];
int *q = &a[7];
```

Then:

```
C
q - p
```

gives the number of **elements** between them:

```
TEXT
7 - 2 = 5
```

not number of bytes.

So:

```
TEXT
q - p = 5
```

when both point into the same array object in a valid way.

35. IMPORTANT RULE

Pointer subtraction is meaningful/defined when pointers refer into the same array object (or one past it), according to C's rules.

Don't randomly subtract unrelated addresses.

For GATE:

pointer difference = difference in element positions, not raw bytes.

36. EXAMPLE

```
C
int a[5];

int *p = &a[1];
int *q = &a[4];

printf("%td", q - p);
```

Conceptually:

```
TEXT
4 - 1 = 3
```

So result is:

```
TEXT
3
```

The actual integer type of pointer difference is `ptrdiff_t`.

Ø=Øbod technical detail; concept matters more.

37. CAN WE ADD TWO POINTERS?

Example:

```
C
p + q
```

No meaningful standard pointer arithmetic operation exists for adding two pointers.

Allowed common operations include:

```
TEXT
pointer + integer
pointer - integer
pointer - pointer (within same array context)
comparison in valid contexts
```

But:

```
TEXT
pointer + pointer
```

is not valid pointer arithmetic.

38. VALID POINTER ARITHMETIC

Generally useful operations:

```
C
p + n
p - n
p++
p--
q - p
```

where the resulting pointer stays within valid array-related bounds or one past the array.

39. ONE-PAST-THE-ENDPOINTER

For:

```
C
int a[5];
```

these element addresses exist:

```
TEXT
&a[0]
&a[1]
&a[2]
&a[3]
&a[4]
```

C also permits forming a pointer:

```
C
a + 5
```

which is one past the last element.

But:

You must not dereference it.

So:

```
C
int *p = a + 5;
```

can be valid as a sentinel pointer.

But:

```
C
*p
```

there is invalid.

40. STORY

Imagine a fence:

```
TEXT
[0][1][2][3][4] | boundary
```

You are allowed to point to:

```
TEXT
boundary
```

to say:

Stop here.

But you cannot enter a non-existent element beyond the boundary.

41. ARRAY TRAVERSAL USING END POINTER

```
c
int a[4] = {10, 20, 30, 40};

int *p = a;
int *end = a + 4;

while (p < end)
{
    printf("%d ", *p);
    p++;
}
```

Output:

```
TEXT
10 20 30 40
```

This is a natural pointer traversal pattern.

42. GATE TRAP — PRECEDENCE WITH `*p++`

This expression:

```
c
*p++
```

can look scary.

Postfix `++` has higher precedence than unary `*`.

So it groups as:

```
c
*(p++)
```

Meaning:

1. use current pointer to access value
2. then increment pointer

Not:

```
c
(*p)++
```

which means increment the pointed value.

Ø=Very important.

43. COMPARE THREE EXPRESSIONS

Suppose:

```
C
int a[3] = {10, 20, 30};
int *p = a;
```

Case 1

```
C
*p++
```

means:

```
C
*(p++)
```

Value used:

```
TEXT
10
```

Then:

```
TEXT
p != a[1]
```

Case 2

```
C
(*p)++
```

Means:

Increment value pointed to by p .

So:

```
TEXT
a[0] : 10 != 11
```

Pointer stays at a[0] .

Case 3

Means:

1. increment pointer first
2. dereference new location

So:

```
TEXT
p  !'  a [ 1 ]
*   p  !'  2 0
```

```
TEXT
p  !'  a [ 1 ]
*   p  !'  2 0
```

44. MASTER COMPARISON

Expression	Meaning
*p++	use current value, then move pointer
(*p)++	increment pointed value
*++p	move pointer first, then access value
++*p	increment pointed value first

$\emptyset = Y\% \oplus H\% \oplus G\% \oplus U\%$ GATE table.

45. EXAMPLE — *p++

```
c
int a[3] = {10, 20, 30};
int *p = a;

printf("%d ", *p++);
printf("%d", *p);
```

First:

Then pointer moves:

Second prints:

TEXT

20

Output:

TEXT

10 20

46. EXAMPLE — (*p)++

C

```
int a[3] = {10, 20, 30};  
int *p = a;  
  
printf("%d ", (*p)++);  
printf("%d", *p);
```

First expression yields old value:

TEXT

10

and increments:

TEXT

a[0] = 11

Pointer unchanged.

Second prints:

TEXT

11

Output:

TEXT

10 11

47. EXAMPLE — *++p

C

```
int a[3] = {10, 20, 30};  
int *p = a;  
  
printf("%d", *++p);
```

Increment pointer:

TEXT

p !' a [1]

Dereference:

TEXT

20

Output:

TEXT

20

48. EXAMPLE — ++*p

```
C
int a[3] = {10, 20, 30};
int *p = a;

printf("%d", ++*p);
```

Dereference p !' a[0] .

Increment value first:

TEXT

1 0 !' 1 1

Output:

TEXT

11

49. GATE TRAP — sizeof(a) VS sizeof(p)

Suppose:

```
C
int a[10];
int *p = a;
```

If:

TEXT

sizeof(int)=4
pointer size=8

then:

```
C
sizeof(a)
```

is:

```
TEXT
10 × 4 = 40
```

because `a` is an actual array object here.

But:

```
C
sizeof(p)
```

is:

```
TEXT
8
```

on this assumed machine.

~~0=Very~~ important.

50. ARRAY DOES NOT DECAY IN `sizeof`

Earlier we said:

array name usually decays to pointer to first element.

One major exception:

```
C
sizeof(a)
```

Here the array does **not** decay.

It gives size of whole array.

Another important exception involves unary `&`:

```
C
&a
```

which takes address of whole array.

51. ARRAY LENGTH TRICK

If:

```
c
int a[10];
```

then in the same scope where `a` is an actual array:

```
c
sizeof(a) / sizeof(a[0])
```

gives:

```
TEXT
10
```

because:

```
TEXT
40 / 4 = 10
```

Useful practical C pattern.

52. BUT THIS FAILS AFTER ARRAY DECAYS TO FUNCTION PARAMETER

Suppose:

```
c
void f(int a[])
{
    printf("%zu", sizeof(a));
}
```

Inside a function parameter declaration like this, `a` is adjusted to a pointer parameter.

So:

```
c
sizeof(a)
```

gives pointer size, not entire original array size.

Important placement/interview trap.

We will revisit functions + arrays.

53. ARRAYS PASSED TO FUNCTIONS

Example:

```
c
void change(int a[])
{
    a[0] = 100;
}
```

Then:

```
C
int x[3] = {10, 20, 30};

change(x);
```

☒ decays to address of first element.

Function can access original array elements.

So after call:

```
TEXT
x[0] = 100
```

This looks unlike ordinary call-by-value behavior.

But technically C still passes a pointer value by value.

Ø=Same pointer principle.

54. VISUALIZATION

```
TEXT
Caller:

x
|
v
[10][20][30]
```

Call:

```
C
change(x);
```

Function gets:

```
TEXT
a !'  s a m e  f i r s t  e l e m e n t
```

Then:

```
C
a[0] = 100;
```

modifies original array:

```
TEXT
[100][20][30]
```

55. INTERVIEW PRECISION

If interviewer asks:

Are arrays passed by reference in C?

Best answer:

C passes arguments by value. When an array is used as a function argument, it usually decays to a pointer to its first element, and that pointer value is passed by value, allowing the function to modify the original array elements.

Wrong answer.

56. `a[i]` ADDRESS FORMULA

Suppose:

TEXT

Base address = B
Element size = W

Then:

TEXT

Address(`a[i]`)
=
 $B + iW$

For a zero-based 1D array.

FORMULA BOX

TEXT

$LOC(a[i]) = Base(a) + i \times w$

where:

TEXT

w = size of one element

57. EXAMPLE

Given:

TEXT

```
Base(a) = 2000  
sizeof(int) = 4
```

Find address of:

TEXT

```
a[6]
```

Calculation:

TEXT

```
2000 + 6×4  
= 2000 + 24  
= 2024
```

Answer:

TEXT

```
2024
```

0-6% TE basic numerical.

58. ARRAY INDEX WITH LOWER BOUND — PREVIEW

Some theoretical questions may use arrays with lower bound `L`.

Then:

TEXT

```
Address(A[i])  
=  
Base + (i-L) × w
```

For normal C arrays:

TEXT

```
L = 0
```

So:

TEXT

```
Base + i×w
```

We'll cover this properly in Arrays.

59. POINTER COMPARISON

Within the same array, pointers can be compared meaningfully for traversal.

Example:

```
c
while (p < end)
```

This is common.

But avoid making assumptions comparing arbitrary unrelated pointers.

For GATE:

Pointer ordering is meaningfully defined within the same array-related object context.

60. GATE FOCUS

0-Min
Most-yield concepts:

- pointer arithmetic scales by type size
- `a` behaves like `&a[0]` in many expressions
- `a[i] == *(a+i)`
- `&a[i] == a+i`
- `a+1` vs `&a+1`
- array name cannot be incremented
- pointer can be incremented
- `*p++` vs `(*p)++`
- `sizeof(array)` vs `sizeof(pointer)`
- arrays passed to functions decay to pointers
- pointer subtraction gives element difference

61. GATE TRAP SHEET

Trap 1

```
c
p + 1
```

does not necessarily mean address + 1 byte.

Trap 2

```
c
a + 1
```

and:


```
C
&a + 1
```

can jump different distances.

Trap 3

```
C
a++
```

invalid.

Trap 4

```
C
*p++
```

means:

```
C
*(p++)
```

Trap 5

```
C
(*p)++
```

increments pointed value.

Trap 6

```
C
sizeof(a)
```

on actual array "a" pointer size.

Trap 7

Inside:

```
C
void f(int a[])
```

a acts as a pointer parameter.

62. PLACEMENT CONNECTION

Arrays form the basis of many placement patterns:

- two pointers
- sliding window
- prefix sums
- binary search
- sorting
- dynamic arrays
- matrix problems

Even if you code in Java/Python later, understanding contiguous memory in C makes DSA concepts much clearer.

63. INTERVIEW QUESTIONS

Q1. What does `p + 1` mean?

Move to the next object of the pointed type.

Q2. Is `a` the same as a pointer variable?

No. An array name often decays to a pointer to its first element, but it is not a modifiable pointer variable.

Q3. What is `a[i]` equivalent to?

```
C
*(a + i)
```

Q4. Difference between `a+1` and `&a+1`?

`a+1` moves one element; `&a+1` moves one whole array.

Q5. Can `a++` be used?

No, because the array name is not a modifiable pointer variable.

Q6. Difference between `*p++` and `(*p)++`?

`*p++` advances the pointer after using current pointed value.

`(*p)++` increments the pointed value.

64. MEMORY TRICKS

Pointer arithmetic

Pointer jumps by objects.

Array relation

TEXT

INDEX = MOVE + DEREFERENCE

C

a[i]

means:

C

*(a+i)

a vs &a

Think:

TEXT

a

! ' f i r s t r o o m

&a

! ' w h o l e b u i l d i n g

Same starting location.

Different step size.

65. NOTEBOOK NOTES

Pointer arithmetic

For:

C

int *p;

TEXT

p + 1

moves by one `int`.

General:

TEXT

p + k

! ' k e l e m e n t s f o r w a r d

Array-pointer relation

C

```
int a[5];
```

In most expressions:

TEXT

```
a !' & a [ 0 ]
```

Important:

TEXT

```
a[i]    = *(a+i)
&a[i]   = a+i
```

a vs &a

For:

C

```
int a[5];
```

TEXT

```
a+1
!'  n e x t   i n t
```

TEXT

```
&a+1
!'  a f t e r   e n t i r e   a r r a y
```

Precedence

TEXT

```
*p++  = *(p++)
(*p)++ = increment pointed value
*++p  = increment pointer first, then dereference
++*p  = increment pointed value first
```

sizeof

Actual array:

C

```
sizeof(a)
```

!' size of whole array.

Pointer:

```
c
sizeof(p)
```

!' pointer size.

66. 30-SECOND REVISION

Remember:

1. Pointer arithmetic scales by pointed type.
2. `p+1` means next element.
3. Arrays are contiguous.
4. `a` usually decays to `&a[0]`.
5. `a[i] = *(a+i)`.
6. `&a[i] = a+i`.
7. `a+1` `` `&a+1` in step size.
8. `a++` is invalid.
9. `*p++` `` `(*p)++`.
10. `sizeof(array)` differs from `sizeof(pointer)`.

67. ACTIVE RECALL

Try first.

Q1

If:

```
TEXT
p = 1000
sizeof(int)=4
```

what conceptual address is:

```
TEXT
p + 3
```

?

Q2

What is:

```
c
a[2]
```

equivalent to using pointer notation?

Q3

Difference between:

```
c
a + 1
```

and:

```
c
&a + 1
```

for `int a[5]` ?

Q4

What does:

```
c
(*p)++;
```

change?

Q5

What does:

```
c
*p++;
```

do?

ANSWERS

A1

```
TEXT
1000 + 3×4 = 1012
```

A2

```
c
*(a + 2)
```

A3

`a+1` moves one `int` .

`&a+1` moves one whole array of 5 `int` .

A4

The pointed value.

A5

Equivalent to:

```
C
*(p++)
```

Uses current pointed value, then moves pointer forward.

68. PRACTICE

~~Q1~~ **Q1**

Given:

```
C
int a[4] = {10, 20, 30, 40};
```

Find:

```
C
*(a + 2)
```

~~Q2~~ **Q2**

If:

```
C
int *p = &a[1];
```

what does:

```
C
p + 2
```

point to?

~~Q3~~ **Q3 — ADDRESS**

Assume:

```
TEXT
&a[0] = 1000
sizeof(int) = 4
```

Find:

TEXT

&a[3]

Q4 — POINTER TRACING

```
C
int a[3] = {5, 10, 15};
int *p = a;

p++;

printf("%d", *p);
```

Find output.

Q5 — GATE TRAP

Assume:

```
C
int a[5];
```

Base address:

TEXT

1000

and:

TEXT

sizeof(int)=4

Find conceptual values of:

TEXT

a + 1
&a + 1

Q6 — PRECEDENCE

```
C
int a[3] = {10, 20, 30};
int *p = a;

printf("%d ", *p++);
printf("%d", *p);
```

Find output.

Q7 — VALUE INCREMENT

```
C
int a[3] = {10, 20, 30};
int *p = a;

printf("%d ", (*p)++);
printf("%d", *p);
```

Find output.

Q8 — WEIRD C

```
C
int a[4] = {10, 20, 30, 40};

printf("%d", 3[a]);
```

What prints and why?

Q9 — sizeof

Assume:

```
TEXT
sizeof(int)=4
pointer size=8
```

For:

```
C
int a[10];
int *p = a;
```

find:

```
TEXT
sizeof(a)
sizeof(p)
```

Q10 — PLACEMENT THINKING

Why can this function modify the original array?

```
C
void change(int a[])
{
    a[0] = 100;
```

```
}
```

Explain without saying simply:

Arrays are passed by reference.

Use correct C terminology.

PROGRESS

Completed '

Lesson 10: Pointer basics **Lesson 11:** Pointer arithmetic + arrays

You now understand the central relation:

TEXT

```
ARRAY INDEXING
=
POINTER ARITHMETIC
+
DEREFERENCING
```

Next

MODULE 0 / LESSON 12 — Pointers Part 3: Pointers with Functions + Swap + Pointer-to-Pointer

Next we will completely understand:

C

```
void swap(int *a, int *b)
```

and finally solve the classic question:

Why does ordinary `swap(a, b)` fail, but pointer-based swap works?

Then we move to:

C

```
int **pp;
```

and understand:

TEXT

```
x
&x
p
&p
*p
pp
&pp
```

using a three-level memory diagram.

This becomes very important for linked lists and functions that must modify pointers themselves.

Type **NEXT**.

ASSISTANT

MODULE 0 — LESSON 12

Pointers Part 3 — Functions, Swap & Pointer-to-Pointer

We continue the Pointer Master Section exactly from your course plan. This lesson connects **call by value + pointers + function modification + double pointers**, which later becomes crucial for linked lists, trees, dynamic memory, and interview questions.

1. WHERE WE ARE

Module: 0 — C Foundation **Lesson:** 12 **Current topic:** Pointers with functions and pointer-to-pointer

GATE importance: 0=Ý%0450K0W **Placement importance:** 0=Ý%0450K0W

Today we cover:

- why normal `swap()` fails
- pointer-based `swap()`
- passing addresses to functions
- C is still call by value
- function parameters that are pointers
- changing original variables
- changing the pointer itself
- pointer-to-pointer
- `int **pp`
- `x`, `&x`, `p`, `&p`, `*p`, `pp`, `*pp`, `**pp`
- when double pointers are needed
- GATE traps

2. STORY — WHY NORMAL SWAP FAILS

Suppose you have two original cards:

TEXT

```
C a r d   A   ! '   1 0
C a r d   B   ! '   2 0
```

You photocopy them and give the copies to your friend.

Your friend swaps the values on the copies.

TEXT

```
C o p y   A   ! '   2 0
C o p y   B   ! '   1 0
```

Do your original cards change?

No.

That is exactly what happens with ordinary function parameters in C.

3. NORMAL SWAP

Consider:

```
C
void swap(int a, int b)
{
    int temp = a;
    a = b;
    b = temp;
}
```

Call:

```
C
int x = 10;
int y = 20;

swap(x, y);
```

You might expect:

TEXT

```
x = 20
y = 10
```

But actually:

TEXT

```
x = 10
y = 20
```

Why?

Because C passes argument values by value.

4. MEMORY VISUALIZATION

Caller:

```
TEXT
main()

x = 10
y = 20
```

Call:

```
C
swap(x, y);
```

Function receives copies:

```
TEXT
swap()

a = 10
b = 20
```

Then inside:

```
TEXT
a = 20
b = 10
```

But originals are still:

```
TEXT
main()

x = 10
y = 20
```

When `swap()` returns, its local parameters disappear.

5. CALL BY VALUE RECAP

YOU MUST KNOW

C uses:

Call by Value

Meaning:

```
TEXT
argument value
!"  c o p i e d   i n t o
parameter
```

Therefore:

```
C
void change(int x)
{
    x = 100;
}
```

does not change caller's original variable.

6. HOW CAN WE CHANGE ORIGINAL VARIABLES?

Instead of passing the values:

```
C
swap(x, y);
```

pass their **addresses**:

```
C
swap(&x, &y);
```

Then function parameters can be pointers.

7. POINTER-BASED SWAP

```
C
void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}
```

Call:

```
C
int x = 10;
int y = 20;

swap(&x, &y);
```

Now:

```
TEXT
x = 20
y = 10
```

8. STORY

Normal swap gets:

```
TEXT
copies of values
```

Pointer swap gets:

```
TEXT
addresses of originals
```

So the function can go directly to the original memory locations.

9. VISUALIZATION

Suppose:

```
TEXT
x address = 5000
y address = 6000
```

Caller:

```
TEXT
5000      6000
+-----+ +-----+
| 10 |    | 20 |
+-----+ +-----+
  x        y
```

Call:

```
C
swap(&x, &y);
```

Function receives:

```
TEXT
a = 5000
b = 6000
```

So:

```
TEXT
a -----> x
b -----> y
```

10. TRACE SWAP STEP-BY-STEP

Function:

```
c
void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}
```

Initially:

```
TEXT
*a = 10
*b = 20
```

Step 1:

```
c
int temp = *a;
```

So:

```
TEXT
temp = 10
```

Step 2:

```
c
*a = *b;
```

So object pointed to by `a` becomes 20.

Therefore:

```
TEXT
x = 20
```

Step 3:

```
c
*b = temp;
```

So:

```
TEXT
y = 10
```

Final:

```
TEXT
x = 20
y = 10
```


11. COMPLETE PROGRAM

```
c
#include <stdio.h>

void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main(void)
{
    int x = 10;
    int y = 20;

    swap(&x, &y);

    printf("%d %d", x, y);

    return 0;
}
```

Output:

```
TEXT
20 10
```

12. IMPORTANT INTERVIEW PRECISION

Does pointer-based swap mean C suddenly became "call by reference"?

No.

~~C~~ C still passes by value.

What gets copied?

```
TEXT
address values
```

Suppose:

```
TEXT
&x = 5000
```

Then function gets:

```
TEXT
a = 5000
```

That pointer value itself is copied.

But both the caller and function can access the same original object through that address.

13. CORRECT INTERVIEW ANSWER

If asked:

Does C support pass by reference?

Say:

C passes arguments by value. Reference-like modification is achieved by passing pointer values that contain addresses of caller objects.

Excellent answer.

14. FUNCTION TAKING POINTER

General form:

```
C
void change(int *p)
{
    *p = 100;
}
```

Caller:

```
C
int x = 10;

change(&x);
```

Afterward:

```
TEXT
x = 100
```

15. HOW THE FUNCTION SEES IT

```
TEXT
main()

x = 10
address = 5000
```

Function:

```
TEXT
change()
```

```
p = 5000
```

Then:

```
c
*p = 100;
```

means:

Go to 5000 and store 100.

That's `x`.

16. SOLVED EXAMPLE

```
c
void addFive(int *p)
{
    *p = *p + 5;
}

int main(void)
{
    int x = 10;

    addFive(&x);

    printf("%d", x);
}
```

Inside:

```
TEXT
*p = 10
```

Then:

```
TEXT
10 + 5 = 15
```

So:

```
TEXT
x = 15
```

Output:

```
TEXT
15
```

17. FUNCTION CAN READ THROUGH POINTER

Pointers aren't only for modifying.

Example:

```
c
int getValue(int *p)
{
    return *p;
}
```

Caller:

```
c
int x = 50;

printf("%d", getValue(&x));
```

Output:

```
TEXT
50
```

18. POINTER PARAMETER IS STILL LOCAL

Important:

```
c
void f(int *p)
```

The pointer variable `p` is a local parameter.

So if inside function:

```
c
p = NULL;
```

does that automatically change the caller's pointer variable?

No.

Because `p` itself is passed by value.

19. VERY IMPORTANT EXAMPLE

```
c
void reset(int *p)
{
    p = NULL;
}

int main(void)
```

```
{
    int x = 10;
    int *ptr = &x;

    reset(ptr);
}
```

After the function:

TEXT
ptr still points to x

Why?

Function received a copy of ptr's value.

Only local parameter p changed.

20. VISUALIZATION

Caller:

TEXT
ptr = 5000

Call:

C
reset(ptr);

Function:

TEXT
p = 5000

Then:

C
p = NULL;

Only:

TEXT
local p

changes.

Caller still:

TEXT
ptr = 5000

0-10 This creates the need for pointer-to-pointer.

21. QUESTION

What if we actually want the function to modify the caller's pointer itself?

Example:

TEXT

```
p t r   o r i g i n a l l y   ! '   x
```

We want function to make:

TEXT

```
p t r   ! '   N U L L
```

or:

TEXT

```
p t r   ! '   a n o t h e r   o b j e c t
```

Then we need to give the function:

Address of the pointer

That means:

TEXT

```
pointer-to-pointer
```

22. POINTER-TO-POINTER

Suppose:

C

```
int x = 10;  
int *p = &x;
```

`p` is:

TEXT

```
pointer to int
```

Now:

C

```
int **pp = &p;
```

`pp` is:

Pointer to pointer to int

23. STORY

Imagine:

TEXT

```
x = treasure
```

`p` is a paper containing the treasure's address.

Now `pp` is another paper containing the address of the first paper.

TEXT

```
p p !' p !' x
```

Three levels.

24. MASTER DIAGRAM

Suppose:

TEXT

```
x lives at 5000
p lives at 3000
pp lives at 1000
```

Then:

TEXT

Address 1000		Address 3000		Address 5000
+-----+		+-----+		+-----+
3000	---->	5000	---->	10
+-----+		+-----+		+-----+
pp		p		x

This is the core double-pointer diagram.

25. WHAT DOES EACH EXPRESSION MEAN?

For:

C

```
int x = 10;
int *p = &x;
int **pp = &p;
```

Assume:

TEXT

&x = 5000
&p = 3000
&pp = 1000

Then:

Expression	Meaning	Conceptual value
x	value of x	10
&x	address of x	5000
p	address of x	5000
*p	value of x	10
&p	address of p	3000
pp	address of p	3000
*pp	value stored in p	5000
**pp	value of x	10
&pp	address of pp	1000

0-1% is an extremely important table.

26. FOLLOW THE ARROWS

TEXT

pp
!"
address of p

*pp
!"
p

**pp
!"
*p
!"
x

Memory trick:

Every * follows one arrow.

27. ONE STAR


```
c
*p
```

Follow one pointer level.

28. TWO STARS

```
c
**pp
```

Follow two levels.

```
TEXT
p p !' p !' x
```

Thus:

```
TEXT
**pp = x
```

in value terms.

29. MODIFYING X THROUGH DOUBLE POINTER

```
c
int x = 10;
int *p = &x;
int **pp = &p;

**pp = 50;
```

What changes?

`pp` points to `p`.

`p` points to `x`.

Therefore:

```
TEXT
**pp
```

accesses `x`.

So:

```
TEXT
x = 50
```

30. VISUALIZATION

Before:

TEXT

p p !' p !' x = 1 0

Execute:

C

```
**pp = 50;
```

After:

TEXT

p p !' p !' x = 5 0

31. MODIFYING POINTER THROUGH DOUBLE POINTER

Now:

C

```
int x = 10;
int y = 20;

int *p = &x;
int **pp = &p;
```

Initially:

TEXT

p !' x

Execute:

C

```
*pp = &y;
```

What is `*pp` ?

`*pp` refers to `p` .

So:

C

```
*pp = &y;
```

means:

TEXT

p = &y

Now:

TEXT

p !' y

0-1% is why double pointers are useful.

32. DON'T CONFUSE

```
**pp = 30
```

Changes:

TEXT

pointed integer value

```
*pp = &y
```

Changes:

TEXT

pointer p itself

Excellent GATE distinction.

33. FUNCTION THAT CHANGES CALLER'S POINTER

Suppose:

```
c
void reset(int **pp)
{
    *pp = NULL;
}
```

Caller:

```
c
int x = 10;
int *p = &x;

reset(&p);
```

What happens?

Function gets:

TEXT

address of p

Then:

```
c
*pp = NULL;
```

changes `p`.

Afterward:

```
TEXT
p = NULL
```

34. FULL TRACE

Caller:

```
TEXT
p !' x
```

Call:

```
c
reset(&p);
```

Function parameter:

```
TEXT
p p !' p
```

Then:

```
c
*pp = NULL;
```

Since `*pp` is `p`:

```
TEXT
p = NULL
```

Final:

```
TEXT
p !' n o t h i n g
```

35. WHY THIS MATTERS IN LINKED LISTS

Suppose:

TEXT

```
h e a d ! '   f i r s t   n o d e
```

You want an insertion function to change `head` itself.

For example inserting at beginning:

TEXT

old:

```
h e a d ! '   [ 1 0 ] ! '   [ 2 0 ]
```

After insertion:

TEXT

```
h e a d ! '   [ 5 ] ! '   [ 1 0 ] ! '   [ 2 0 ]
```

`head` must now store a new address.

In C, one way is:

C

```
void insertFront(struct Node **head, int value)
```

because the function may need to modify the caller's `head` pointer.

~~0-7%~~ This is a major linked-list application.

36. ALTERNATIVE DESIGN

Instead of double pointer, a function can often return the new pointer:

C

```
struct Node *insertFront(struct Node *head, int value)
```

Then caller does:

C

```
head = insertFront(head, value);
```

Both designs are common.

We'll compare them during linked lists.

37. DOUBLE POINTER + DYNAMIC MEMORY

Later you'll see patterns such as:

C

```
void allocate(int **p)
{
    *p = malloc(sizeof(int));
}
```

```
}
```

Caller:

```
C
int *x = NULL;

allocate(&x);
```

Function can assign the newly allocated address into caller's pointer `x`.

This is another reason double pointers matter.

38. SOLVED EXAMPLE

```
C
void change(int **pp)
{
    **pp = 100;
}

int main(void)
{
    int x = 10;
    int *p = &x;

    change(&p);

    printf("%d", x);
}
```

Trace:

```
TEXT
p p !' p !' x
```

Then:

```
C
**pp = 100;
```

Therefore:

```
TEXT
x = 100
```

Output:

```
TEXT
100
```

39. SOLVED EXAMPLE — POINTER TARGET CHANGE

```
c
void changePointer(int **pp, int *q)
{
    *pp = q;
}

int main(void)
{
    int x = 10;
    int y = 20;

    int *p = &x;

    changePointer(&p, &y);

    printf("%d", *p);
}
```

Initially:

```
TEXT
p !' x
```

Function:

```
TEXT
p p !' p
q !' y
```

Then:

```
c
*pp = q;
```

means:

```
TEXT
p = address of y
```

Now:

```
TEXT
p !' y
```

Output:

```
TEXT
20
```

40. GATE TRACE EXAMPLE

```

c
int x = 5;
int *p = &x;
int **q = &p;

**q = 10;

printf("%d %d %d", x, *p, **q);

```

All three access the same integer.

So output:

```

TEXT
10 10 10

```

41. ANOTHER GATE TRACE

```

c
int x = 5;
int y = 8;

int *p = &x;
int **q = &p;

*q = &y;

printf("%d", *p);

```

Trace:

Initially:

```

TEXT
p  !'  x
q  !'  p

```

Then:

```

c
*q = &y;

```

Since:

```

TEXT
*q = p

```

this makes:

```

TEXT
p  !'  y

```

Therefore:

TEXT

```
*p = 8
```

Output:

TEXT

```
8
```

42. VERY IMPORTANT * COUNTING METHOD

Suppose:

C

```
int ***r;
```

That means:

TEXT

```
pointer to  
pointer to  
pointer to  
int
```

Each * adds one level of indirection.

You don't need triple pointers much for GATE basics, but understand the pattern.

43. LEVEL TABLE

TEXT

```
int x;
```

Level 0:

TEXT

```
x !' i n t
```

C

```
int *p;
```

Level 1:

TEXT

```
p !' p o i n t e r t o i n t
```

C

```
int **pp;
```

Level 2:

TEXT

p p ! ' p o i n t e r t o p o i n t e r t o i n t

C

```
int ***ppp;
```

Level 3.

44. TYPE MATCHING

For:

C

```
int x;
```

Address:

C

```
&x
```

has type:

TEXT

```
int *
```

Therefore:

C

```
int *p = &x;
```

makes sense.

Now:

C

```
&p
```

has type:

TEXT

```
int **
```

Therefore:

C

```
int **pp = &p;
```

makes sense.

Great way to understand pointer levels.

45. FORMULA-LIKE TYPE RULE

If:

TEXT
x has type T

then:

TEXT
&x has type T*

If:

TEXT
p has type T*

then:

TEXT
*p has type T

This is extremely useful.

46. EXAMPLE

If:

C
double x;

then:

TEXT
x !' d o u b l e
& x !' d o u b l e *

If:

C
double *p;

then:

TEXT
p !' d o u b l e *
* p !' d o u b l e
& p !' d o u b l e * *

47. GATE TRAP — CHANGE POINTEE VS CHANGE POINTER

Suppose:

```
C
void f(int *p)
{
    p = NULL;
}
```

This does **not** make caller's pointer NULL.

But:

```
C
void f(int **p)
{
    *p = NULL;
}
```

called with:

```
C
f(&ptr);
```

can make caller's pointer NULL.

~~Ø=We~~ Must know.

48. GATE TRAP — DOUBLE POINTER DEREFERENCE

For:

```
C
int x = 10;
int *p = &x;
int **q = &p;
```

Don't say:

```
TEXT
*q = x
```

Wrong.

Actually:

```
TEXT
*q = p
```

because `q` points to `p`.

And:

TEXT

```
**q = x
```

in terms of the accessed value.

49. MASTER ARROW DIAGRAM

TEXT

```
q !' p !' x
```

Therefore:

TEXT

```
q
= address of p

*q
= p
= address of x

**q
= x's value
```

Use this every time.

50. SWAP USING POINTERS — INTERVIEW VERSION

The standard interview explanation:

Because C passes arguments by value, swapping two ordinary parameters swaps only local copies. Passing the addresses of the original variables allows the function to dereference those pointers and modify the caller's objects.

That is the answer you should give verbally.

51. PLACEMENT CONNECTION

Double pointers become useful in:

- linked-list head modification
- allocating memory inside helper functions
- arrays of strings
- dynamic 2D structures
- command-line arguments conceptually (`char **argv`)
- tree root modification
- APIs returning allocated objects through output parameters

52. char **argv PREVIEW

You may eventually see:

```
C
int main(int argc, char **argv)
```

Here:

```
TEXT
argv
```

is a pointer-to-pointer to character.

It represents command-line argument strings.

Not important right now; just recognize the type.

53. GATE FOCUS

Most frequent areas:

- call by value
- pointer parameter
- swap
- `&x` passed into function
- changing pointee vs changing pointer
- double pointer
- `*pp` vs `**pp`
- type levels
- linked-list head modification intuition

54. GATE TRAP SHEET

Trap 1

```
C
void f(int x)
```

changes only local copy.

Trap 2

```
C
void f(int *p)
{
    *p = 10;
}
```

can modify caller's object.

Trap 3

```
C
void f(int *p)
{
    p = NULL;
}
```

does not modify caller's pointer variable.

Trap 4

To modify caller's pointer itself, use something like:

```
C
void f(int **pp)
```

and pass:

```
C
&p
```

Trap 5

```
TEXT
*pp = p-like value
**pp = underlying int-like value
```

depending on declarations.

55. MEMORY TRICKS

Call by value

Copy the value.

Pointer parameter

Copy the address.

Double pointer

Address of an address-holder.

Stars

****Each ` follows one arrow.****

TEXT

p p !' p !' x

One star:

TEXT

* p p !' p

Two stars:

TEXT

* * p p !' x

56. NOTEBOOK NOTES

Swap Without Pointer

C

```
void swap(int a, int b)
```

swaps only local copies.

Swap With Pointer

C

```
void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}
```

Call:

C

```
swap(&x, &y);
```

modifies originals.

Pointer-to-Pointer

```
C
int x = 10;
int *p = &x;
int **pp = &p;
```

Then:

```
TEXT
p      !'  a d d r e s s   o f   x
* p    !'  x   v a l u e
p p    !'  a d d r e s s   o f   p
* p p  !'  p
* * p p !'  x   v a l u e
```

Modify pointer itself

```
C
void reset(int **pp)
{
    *pp = NULL;
}
```

Call:

```
C
reset(&p);
```

57. 30-SECOND REVISION

Remember:

1. C always passes arguments by value.
2. Ordinary swap fails because parameters are copies.
3. Passing `&x` gives function address of original.
4. `*p` can modify caller's object.
5. Passing a pointer still copies the pointer value.
6. Changing local pointer parameter doesn't change caller's pointer.
7. To modify pointer itself, pass its address.
8. `int **pp` = pointer to pointer to int.
9. `*pp` accesses the pointer.
10. `**pp` accesses the final integer.

58. ACTIVE RECALL

Try first.

Q1

Why does this fail to modify caller's `x` ?

```
c
void f(int x)
{
    x = 50;
}
```

Q2

Why does this work?

```
c
void f(int *p)
{
    *p = 50;
}
```

when called with:

```
c
f(&x);
```

Q3

For:

```
c
int x = 10;
int *p = &x;
int **pp = &p;
```

what is:

```
TEXT
*pp
```

?

Q4

What is:

```
TEXT
**pp
```

?

Q5

Why might we need `int **pp` instead of `int *p` ?

ANSWERS

A1

Because `x` is a local copy.

A2

Because `p` contains the address of caller's `x`, and dereferencing accesses the original object.

A3

`p` — the pointer value/address of `x`.

A4

The integer value stored in `x`.

A5

To allow a function to modify a pointer variable itself, such as changing what it points to.

59. PRACTICE

Q1

Find output:

```
c
void change(int *p)
{
    *p = 25;
}

int main(void)
{
    int x = 5;
    change(&x);
    printf("%d", x);
}
```

Q2

Why doesn't this modify `p` in the caller?

```
c
void reset(int *p)
{
    p = NULL;
}
```

Q3 — SWAP

Trace:

```
c
int x = 10;
int y = 20;

swap(&x, &y);
```

where:

```
c
void swap(int *a, int *b)
{
    int t = *a;
    *a = *b;
    *b = t;
}
```

Find final `x` and `y`.

Q4 — DOUBLE POINTER

```
c
int x = 10;
int *p = &x;
int **q = &p;

**q = 30;
```

Find final `x`.

Q5 — CHANGE TARGET

```
c
int x = 10;
int y = 20;

int *p = &x;
int **q = &p;

*q = &y;
```

Where does `p` point now?

What is `*p`?

Q6 — FUNCTION CHANGES POINTER

```
c
void setNull(int **p)
{
    *p = NULL;
}
```

```
int main(void)
{
    int x = 10;
    int *ptr = &x;

    setNull(&ptr);
}
```

What is `ptr` after the call?

Q7 — ADDRESS LEVELS

Assume:

TEXT

```
x address = 5000
p address = 3000
pp address = 1000
```

and:

C

```
int x = 25;
int *p = &x;
int **pp = &p;
```

Find conceptually:

TEXT

```
x
&x
p
&p
*p
pp
&pp
*pp
**pp
```

Q8 — GATE THINKING

C

```
void f(int **pp, int *q)
{
    *pp = q;
}

int main(void)
{
    int a = 10;
    int b = 20;

    int *p = &a;

    f(&p, &b);
}
```

```
    printf("%d", *p);  
}
```

Find output.

Q9 — PLACEMENT THINKING

Why might this linked-list function require:

```
C  
void insertFront(struct Node **head, int x)
```

instead of merely:

```
C  
void insertFront(struct Node *head, int x)
```

Answer in terms of modifying the caller's `head` pointer.

PROGRESS

Completed '

Lesson 10: Pointer basics **Lesson 11:** Pointer arithmetic + arrays **Lesson 12:** Pointer functions, swap & double pointers

You now understand the full chain:

```
TEXT  
VALUE  
!"  
ADDRESS  
!"  
POINTER  
!"  
POINTER TO POINTER
```

Next

MODULE 0 / LESSON 13 — Pointers Part 4: Pointer Traps, NULL, Wild, Dangling, `const` & Advanced Pointer Reasoning

Next we will consolidate the dangerous pointer cases:

```
C  
int *p;
```

VS

```
C
int *p = NULL;
```

VS

```
C
int *p = &x;
```

and learn:

- wild pointer
- NULL pointer
- dangling pointer
- returning address of local variable
- use-after-free intuition
- `const int *p`
- `int *const p`
- `const int *const p`
- common GATE pointer traps
- how to tell what can change: the value, pointer, or neither

Type **NEXT**.

USER

NEXT

ASSISTANT

MODULE 0 — LESSON 13

Pointers Part 4 — NULL, Wild, Dangling, `const` & Advanced Pointer Traps

We continue the Pointer Master Section from your uploaded course plan. This lesson is mainly about **pointer safety + GATE traps + interview reasoning**.

1. WHERE WE ARE

Module: 0 — C Foundation **Lesson:** 13 **Current topic:** Dangerous pointer states and `const` pointers

GATE importance: Ø=Ý%ØMÚ%ØKÍW **Placement importance:** Ø=Ý%ØMÚ%ØKÍW

Today we cover:

- valid pointer
- uninitialized / wild pointer
- `NULL` pointer
- dangling pointer

- returning address of local variable
 - pointer after `free()`
 - use-after-free
 - `const int *p`
 - `int *const p`
 - `const int *const p`
 - what can change: pointer or value?
 - common GATE traps
 - pointer-safety checklist
-

2. STORY / INTUITION

Think of a pointer as a paper containing a house address.

There are several situations.

Situation 1 — Valid address

TEXT

Paper says: House 5000
House 5000 exists

Good pointer.

Situation 2 — Random address

TEXT

Paper contains some unknown/random number

Dangerous.

This is like an uninitialized pointer.

Situation 3 — Explicitly says "NO ADDRESS"

TEXT

Paper says: NULL

You know it currently points nowhere valid.

Situation 4 — Address of a demolished house

TEXT

Paper says: House 5000
But House 5000 is no longer yours / object no longer exists

That is a:

Dangling Pointer

3. VALID POINTER

Example:

```
c
int x = 10;
int *p = &x;
```

Here:

```
TEXT
p  !'  x
```

and `x` is alive.

Therefore:

```
c
*p
```

is valid.

4. UNINITIALIZED POINTER

Consider:

```
c
int *p;
```

You declared `p`, but did not initialize it.

Its value is indeterminate.

You do **not** know where it points.

So:

```
c
*p = 10;
```

is invalid.

5. WILD POINTER

The term **wild pointer** is commonly used informally for a pointer containing an indeterminate/invalid address because it was never properly initialized.

Example:

```
c
int *p;

printf("%d", *p);
```

⚠️ Dangerous.

`p` does not point to a known valid `int`.

Dereferencing it causes undefined behavior.

6. STORY FOR WILD POINTER

Imagine someone hands you a paper containing:

```
TEXT
7348921
```

but nobody knows whether that address belongs to you or even corresponds to valid accessible memory.

Going there and modifying it would be dangerous.

That is the intuition behind a wild pointer.

7. SAFER INITIALIZATION

If a pointer should not point anywhere yet:

```
c
int *p = NULL;
```

This is much clearer than leaving it uninitialized.

Now you explicitly know:

```
TEXT
p points to no valid object
```

8. NULL POINTER

A null pointer represents:

No valid object/function target.

Example:

```
c
int *p = NULL;
```

Now:

TEXT

```
p == NULL
```

is true.

9. IMPORTANT

A NULL pointer is **not safe to dereference**.

This is invalid:

```
C
int *p = NULL;

printf("%d", *p);
```

Because there is no pointed object.

Ø=Rule:

NULL can be checked, but not dereferenced.

10. SAFE NULL CHECK

```
C
if (p != NULL)
{
    printf("%d", *p);
}
```

This ensures dereferencing only occurs when `p` is not null.

11. SHORT-CIRCUIT CONNECTION

Remember Lesson 2:

```
C
if (p != NULL && *p > 0)
```

C evaluates left side first.

If:

TEXT

```
p == NULL
```

then:

```
C
*p > 0
```

is skipped because of `&&`.

This helps avoid invalid dereferencing.

Excellent real-world short-circuit use.

12. DON'T CONFUSE

Uninitialized pointer

```
C
int *p;
```

Value:

```
TEXT
indeterminate
```

NULL pointer

```
C
int *p = NULL;
```

Value intentionally represents:

```
TEXT
no valid target
```

These are not the same.

13. DANGLING POINTER

A pointer becomes **dangling** when it still contains an address, but the object it used to point to is no longer alive.

Example:

```
C
int *p;
```

Suppose `p` points to an object.

Later that object's lifetime ends.

`p` may still contain the old address.

Now `p` is dangling.

14. CLASSIC DANGLING POINTER — LOCAL VARIABLE

Consider:

```
C
int *bad(void)
{
    int x = 10;

    return &x;
}
```

Problem:

```
TEXT
x
```

has automatic storage duration.

When `bad()` returns:

```
TEXT
x's lifetime ends
```

But the returned pointer contains:

```
TEXT
old address of x
```

So that pointer is dangling.

Very important GATE + interview concept.

15. VISUALIZATION

During function:

```
TEXT
STACK

bad()
+-----+
| x = 10 |
+-----+
```

Return:

```
C
return &x;
```

Suppose address was:

TEXT

5000

Caller receives:

TEXT

p = 5000

But after `bad()` ends:

TEXT

x no longer exists

So:

TEXT

p !' i n v a l i d f o r m e r o b j e c t

Dangling pointer.

16. STORY

You are given:

TEXT

Room 205

in a temporary hotel.

Then hotel closes.

You still have paper:

TEXT

205

But room is no longer valid for you.

Same idea.

17. WRONG CODE

C

```
int *f(void)
{
    int x = 10;
    return &x;
}
```

```
int main(void)
{
```

```

int *p = f();

printf("%d", *p);
}

```

Dereferencing `p` is invalid because the pointed automatic object no longer exists.

Undefined behavior.

18. STATICLOCAL CHANGES THE LIFETIME

Consider:

```

c
int *f(void)
{
    static int x = 10;

    return &x;
}

```

Now `x` has static storage duration.

It survives after `f()` returns.

So returning:

```

c
&x

```

is valid with respect to lifetime.

Important contrast.

19. COMPARE

Bad

```

c
int *f(void)
{
    int x = 10;
    return &x;
}

```

`x` dies on return.

Lifetime-valid

```
c
int *f(void)
{
    static int x = 10;
    return &x;
}
```

`x` exists for whole program execution.

20. DANGLING POINTER AFTER `free()`

Later we'll study:

```
c
int *p = malloc(sizeof(int));
```

Suppose valid dynamic storage is allocated.

Then:

```
c
free(p);
```

releases that allocation.

But what is still stored inside `p`?

Typically the old address value remains unless you change `p`.

So:

```
TEXT
p still contains old address
but object lifetime has ended
```

Therefore `p` is dangling.

21. USE-AFTER-FREE

This is dangerous:

```
c
int *p = malloc(sizeof(int));

*p = 10;

free(p);

printf("%d", *p);
```

After:


```
C
free(p);
```

the allocation is no longer yours to access.

Dereferencing:

```
C
*p
```

after that is:

Use-After-Free

Undefined behavior.

Ø=Major systems/security concept.

22. GOOD PRACTICE

After:

```
C
free(p);
```

you may often write:

```
C
p = NULL;
```

So:

```
TEXT
old dangling address
```

is replaced by:

```
TEXT
NULL
```

This doesn't magically fix all aliases, but it prevents this particular pointer from retaining the stale address.

23. IMPORTANT ALIASING TRAP

Suppose:

```
C
int *p = malloc(sizeof(int));
int *q = p;
```

Now:

TEXT

```
p !'  s a m e  a l l o c a t i o n
q !'  s a m e  a l l o c a t i o n
```

Then:

C

```
free(p);
p = NULL;
```

Does that make `q = NULL`?

No.

`q` still contains the old address.

Therefore:

TEXT

q

is dangling.

Very important.

24. VISUALIZATION

Before:

TEXT

```
p % % %
    % % % >  H E A P  O B J E C T
q % % %
```

After:

C

```
free(p);
p = NULL;
```

TEXT

```
p !'  N U L L

q !'  o l d   f r e e d   l o c a t i o n   !•   d a n g l i n g
```

Setting one pointer to NULL does not modify copies of that pointer value elsewhere.

25. THREE IMPORTANT BAD POINTER STATES

State	Example	Meaning
Wild/uninitialized	<code>int *p;</code>	unknown/indeterminate address
NULL	<code>int *p = NULL;</code>	intentionally no valid target
Dangling	old address after lifetime ends	target no longer alive

🗑️ Copy this table.

26. MEMORY TRICK

Wild

Never knew the house.

NULL

Know there is no house.

Dangling

Knew the house, but house is gone.

27. NOW `const` WITH POINTERS

This topic looks confusing because:

```
c
const int *p;
```

and:

```
c
int *const p;
```

mean different things.

We will solve it with one simple question:

What is constant — the pointed value or the pointer itself?

28. POINTER TO CONST

```
c
const int *p;
```

Read it as:

p is a pointer to const int.

Or:

Through p, you must not modify the pointed integer.

Example:

```
c
int x = 10;
const int *p = &x;
```

Allowed:

```
c
printf("%d", *p);
```

Not allowed through p:

```
c
*p = 20;
```

because the pointed value is treated as read-only through p.

29. CAN p ITSELF CHANGE?

Yes.

Example:

```
c
int x = 10;
int y = 20;

const int *p = &x;

p = &y;
```

This is allowed.

So:

```
TEXT
pointed value through p !' cannot modify
pointer p !' can move
```

30. VISUALIZATION

```
C
const int *p;
```

Think:

```
TEXT
p !' [ C O N S T   V A L U E ]
!'
pointer can move
```

Memory trick:

const near data type !' data protected through pointer

31. POINTER CONSTNESS VS OBJECT CONSTNESS — SUBTLE POINT

If:

```
C
int x = 10;
const int *p = &x;
```

you cannot do:

```
C
*p = 20;
```

through `p`.

But `x` itself is not necessarily a const object.

So this is still allowed:

```
C
x = 20;
```

Then:

```
TEXT
*p
```

will observe `20`.

The restriction is on modification **through that pointer expression**.

32. CONST POINTER

Now:

```
C
int *const p = &x;
```

Read:

p is a constant pointer to int.

This means pointer value cannot change.

So:

```
C
p = &y;
```

is not allowed.

But:

```
C
*p = 20;
```

is allowed because pointed integer is not const through this declaration.

33. VISUALIZATION

```
C
int *const p = &x;
```

Think:

```
TEXT
CONST POINTER
p % % % % % % % % !' x
      value may change

p cannot move elsewhere
```

Memory trick:

****const after ``!' pointer is fixed*****

34. POINTER TO CONST VS CONST POINTER

Pointer to const

```
C
const int *p;
```

Can:

TEXT

```
move pointer '  
modify through pointer 'L
```

Const pointer

C

```
int *const p = &x;
```

Can:

TEXT

```
move pointer 'L  
modify pointed value '
```

Ø=Must know.

35. CONST POINTER TO CONST DATA

C

```
const int *const p = &x;
```

Now both are restricted.

Through p:

TEXT

```
change pointer 'L  
change pointed value 'L
```

So:

C

```
p = &y;
```

not allowed.

And:

C

```
*p = 20;
```

not allowed.

36. MASTER TABLE

Declaration	Pointer can change target?	Value modifiable through pointer?
int *p	'	'
const int *p	'	'L
int *const p	'L	'
const int *const p	'L	'L

0=Y%0=9%0=5% exactly.

37. HOW TO READ COMPLICATED POINTER DECLARATIONS

A useful beginner trick:

Start from variable name and move outward.

Example:

```
c
int *const p;
```

Start at:

```
TEXT
p
```

Immediately next:

```
TEXT
const
```

So:

p is constant.

Then:

```
TEXT
*
```

pointer.

Then:

```
TEXT
int
```

So:

p is a constant pointer to int.

38. EXAMPLE

```
c
const int *p;
```

Start from `p`.

`*`:

p is a pointer.

Then:

```
TEXT
const int
```

So:

pointer to const int.

39. `int const *p`

This:

```
c
int const *p;
```

means the same as:

```
c
const int *p;
```

Both mean:

pointer to const int.

So:

```
TEXT
const int
```

and:

```
TEXT
int const
```

are equivalent type spelling here.

40. SOLVED EXAMPLE — POINTER TO CONST

```
C
int x = 10;
int y = 20;

const int *p = &x;

p = &y;
```

Valid.

Now:

```
TEXT
*p = 20
```

as observed value, but not because we modified through `p`; we changed its target.

Trying:

```
C
*p = 50;
```

is not permitted through this pointer type.

41. SOLVED EXAMPLE — CONST POINTER

```
C
int x = 10;
int y = 20;

int *const p = &x;
```

Allowed:

```
C
*p = 50;
```

Now:

```
TEXT
x = 50
```

Not allowed:

```
C
p = &y;
```

because `p` itself is constant.

42. SOLVED EXAMPLE — BOTH CONST

```
c
int x = 10;

const int *const p = &x;
```

Neither:

```
c
*p = 20;
```

nor:

```
c
p = &y;
```

is allowed through `p`.

43. WHY `const` MATTERS IN FUNCTIONS

Suppose function only needs to read an integer:

```
c
void printValue(const int *p)
{
    printf("%d", *p);
}
```

The interface communicates:

This function promises not to modify the pointed integer through `p`.

This is good API design.

44. ARRAY EXAMPLE

Suppose function should inspect array but not modify it:

```
c
void printArray(const int a[], int n)
{
    for (int i = 0; i < n; i++)
    {
        printf("%d ", a[i]);
    }
}
```

Here `const` indicates array elements should not be modified through this parameter.

Ø=Good placement coding habit.

45. POINTER SAFETY — CHECK BEFORE DEREFERENCE

Before using:

```
c
*p
```

mentally ask:

1.

Has `p` been initialized?

2.

Does `p` currently point to a live object?

3.

Is `p != NULL` ?

4.

Is the pointed type appropriate?

5.

Is the pointer within valid array bounds?

This checklist avoids many bugs.

46. ARRAY OUT-OF-BOUNDS POINTER

Suppose:

```
c
int a[3] = {10, 20, 30};
```

Valid elements:

```
TEXT
a[0]
a[1]
a[2]
```

You may form:

```
c
a + 3
```

as one-past pointer.

But:

```
c
*(a + 3)
```

is invalid.

There is no `a[3]` element.

~~0=100%~~ Important.

47. POINTER ARITHMETIC TRAP

This is valid traversal:

```
c
for (int *p = a; p < a + 3; p++)
{
    printf("%d", *p);
}
```

Because dereference occurs only while:

```
TEXT
p < a + 3
```

Once:

```
TEXT
p == a + 3
```

loop stops.

48. ADDRESS OF LOCAL VS STATIC— GATE SUMMARY

Automatic local

```
c
int *f(void)
{
    int x = 5;
    return &x;
}
```

'L'pointer becomes dangling after return.

Static local

```
c
int *f(void)
{
    static int x = 5;
    return &x;
}
```

```
}
```

' x survives for whole program execution.

49. POINTER TO FREED MEMORY — GATE SUMMARY

```
c
int *p = malloc(sizeof(int));

free(p);
```

After `free` :

```
TEXT
p is potentially dangling
```

Do not:

```
c
*p
```

until `p` is assigned a new valid target.

50. DOUBLE FREE PREVIEW

This is dangerous:

```
c
free(p);
free(p);
```

if `p` still contains the same already-freed allocation address.

This is called:

Double Free

Undefined behavior.

Later in dynamic memory we'll cover this carefully.

51. SETTING TO NULL HELPS

A common pattern:

```
c
free(p);
p = NULL;
```

Then another careless:

```
c
free(p);
```

is okay because standard `free(NULL)` does nothing.

Useful C fact.

But remember: other aliases may still dangle.

52. `free(NULL)` RULE

This is allowed:

```
c
free(NULL);
```

It has no effect.

So code like:

```
c
if (p != NULL)
    free(p);
```

is not required solely because `free(NULL)` itself is safe.

Still, null checks may be useful for program logic.

53. GATE TRAP — CHECK AFTER FREE

Wrong thinking:

```
c
free(p);

if (p != NULL)
    printf("%d", *p);
```

Why wrong?

Because `free(p)` does not automatically set `p` to `NULL`.

`p` may still contain a non-`NULL` stale address.

So:

```
TEXT
p != NULL
```

does **not** prove object is alive.

Important.

54. POINTER VALIDITY IS MORE THAN NON-NULL

A pointer can be:

```
TEXT
non-NULL
```

and still invalid/dangling.

Example:

```
TEXT
p = old freed address
```

So:

Non-NULL does not automatically mean safe to dereference.

55. GATE-STYLE TRACE — ALIASING AFTER FREE

Suppose conceptually:

```
C
int *p = malloc(sizeof(int));
int *q = p;

free(p);
p = NULL;
```

Then:

```
TEXT
p !' N U L L
q !' d a n g l i n g
```

This is exactly why pointer aliases need careful ownership management.

0=10 important systems-programming concept.

56. SOLVED GATE EXAMPLE — CONST

Given:

```
C
int x = 10;
int y = 20;

const int *p = &x;
```

Which is valid?


```
c
p = &y;
```

' Valid.

```
c
*p = 30;
```

'L Not allowed through `p`.

57. SOLVED GATE EXAMPLE — CONST POINTER

```
c
int x = 10;
int y = 20;

int *const p = &x;
```

Which is valid?

```
c
*p = 30;
```

' Valid.

```
c
p = &y;
```

'L Not allowed.

58. MEMORY TRICK FOR `const`

This trick is usually enough:

`const` before `*`

```
c
const int *p
```

DATA locked

`const` after `*`

```
c
int *const p
```

POINTER locked

Both

```
C
const int *const p
```

BOTH locked

59. GATE FOCUS

0-MUST KNOW:

- uninitialized pointer
- NULL pointer
- dangling pointer
- lifetime of local variable
- pointer after `free`
- aliasing after `free`
- non-NULL "" necessarily valid
- `const int *`
- `int *const`
- `const int *const`
- one-past pointer
- pointer safety

60. GATE TRAP SHEET

Trap 1

```
C
int *p;
```

then:

```
C
*p
```

!' invalid/uninitialized pointer dereference.

Trap 2

```
c
int *p = NULL;
```

then:

```
c
*p
```

!' invalid.

Trap 3

Returning:

```
c
&local_variable
```

!' dangling after function returns.

Trap 4

```
c
free(p);
```

does not automatically perform:

```
c
p = NULL;
```

Trap 5

Non-NULL pointer may still be dangling.

Trap 6

```
c
const int *p
```

pointer can move; pointed value cannot be modified through `p`.

Trap 7

```
C
int *const p
```

pointer cannot move; pointed value can change.

61. PLACEMENT CONNECTION

Pointer bugs are common causes of:

- crashes
- corrupted memory
- security vulnerabilities
- hard-to-debug behavior

Common bugs include:

TEXT

```
NULL dereference
use-after-free
double free
dangling pointer
out-of-bounds access
uninitialized pointer
```

Even if your placement coding language becomes Java or Python, interviewers may ask these if C/C++ appears on your resume.

62. INTERVIEW QUESTIONS

Q1. What is a NULL pointer?

A pointer value that represents no valid object/function target.

Q2. What is a wild pointer?

Informal term for an uninitialized pointer containing an indeterminate address.

Q3. What is a dangling pointer?

A pointer that refers to an object whose lifetime has ended.

Q4. Why is returning the address of an ordinary local variable dangerous?

Because the local object's lifetime ends when the function returns.

Q5. Does `free(p)` set `p` to NULL?

No.

Q6. Can a non-NULL pointer still be invalid?

Yes, for example a dangling pointer.

Q7. Difference between `const int *p` and `int *const p`?

First protects pointed data through `p`; second fixes the pointer itself.

63. MEMORY TRICKS

Wild

Unknown address

NULL

Known no-address

Dangling

Old dead address

Pointer to const

Move pointer, don't modify data through it

Const pointer

Modify data, don't move pointer

64. NOTEBOOK NOTES

Valid pointer

```
C
int x = 10;
int *p = &x;
```

Wild / uninitialized

```
C
int *p;
```

Do not dereference before initialization.

NULL

```
C
int *p = NULL;
```

Represents no valid target.

Never dereference NULL.

Dangling

Pointer to object whose lifetime ended.

Example:

```
c
int *f(void)
{
    int x;
    return &x;
}
```

After free

```
c
free(p);
p = NULL;
```

is common practice.

But aliases may still dangle.

const

TEXT

```
const int *p
!' pointer movable
!' value not modifiable through p
```

TEXT

```
int *const p
!' pointer fixed
!' value modifiable
```

TEXT

```
const int *const p
!' neither modifiable through p nor movable
```

65. 30-SECOND REVISION

Remember these 10:

1. `int *p;` is uninitialized.
2. Never dereference an uninitialized pointer.

3. `NULL` means no valid target.
 4. Never dereference `NULL`.
 5. Dangling pointer points to an object whose lifetime ended.
 6. Returning address of automatic local creates a dangling pointer.
 7. `free(p)` does not set `p = NULL`.
 8. Non-`NULL` does not guarantee validity.
 9. `const int *p` protects pointed data through `p`.
 10. `int *const p` fixes the pointer itself.
-

66. ACTIVE RECALL

Try first.

Q1

What's the difference between:

```
c
int *p;
```

and:

```
c
int *p = NULL;
```

Q2

Why is this bad?

```
c
int *f(void)
{
    int x = 10;
    return &x;
}
```

Q3

After:

```
c
free(p);
```

is `p` automatically `NULL`?

Q4

For:

```
c
const int *p;
```

can `p` point somewhere else later?

Q5

For:

```
c
int *const p = &x;
```

can `*p` be modified?

ANSWERS

A1

First has an indeterminate pointer value; second explicitly represents no valid target.

A2

`x`'s lifetime ends when the function returns, so the returned pointer dangles.

A3

No.

A4

Yes.

A5

Yes, assuming underlying object itself is modifiable.

67. PRACTICE

Q1

Classify:

```
c
int *p;
```

as:

```
TEXT
valid initialized
NULL
uninitialized/wild
dangling
```

Q2

Classify:


```
C
int *p = NULL;
```

~~Q3~~ — Dangling

Why is this unsafe?

```
C
int *f(void)
{
    int a = 5;
    return &a;
}
```

~~Q4~~ — Static

Is this lifetime-safe?

```
C
int *f(void)
{
    static int a = 5;
    return &a;
}
```

Explain.

~~Q5~~ — Const

Given:

```
C
int x = 10;
int y = 20;

const int *p = &x;
```

Which of these are allowed?

```
C
p = &y;
```

```
C
*p = 30;
```

~~Q6~~ — Const Pointer

Given:

```
c
int x = 10;
int y = 20;

int *const p = &x;
```

Which are allowed?

```
c
*p = 30;
```

```
c
p = &y;
```

Q7 — ALIASING AFTER FREE

```
c
int *p = malloc(sizeof(int));
int *q = p;

free(p);
p = NULL;
```

Classify:

```
TEXT
p
q
```

afterward.

Q8 — SAFETY THINKING

Why is this check insufficient after a pointer becomes dangling?

```
c
if (p != NULL)
{
    printf("%d", *p);
}
```

Explain in one sentence.

Q9 — INTERVIEW

Give a concise difference between:

```
C
const int *p;
```

```
C
int *const p;
```

```
C
const int *const p;
```

PROGRESS

Pointer Master Section Completed So Far '

Lesson 10: Addresses, declaration, dereferencing **Lesson 11:** Pointer arithmetic + arrays **Lesson 12:** Function pointers? No — **pointers with functions + double pointers** **Lesson 13:** NULL, wild, dangling, `const`, safety traps

We have now built a strong pointer foundation.

Next

MODULE 0 / LESSON 14 — Strings in C

Next we move to another extremely important C area:

```
C
char s[] = "GATE";
```

We will understand:

- character arrays
- null terminator `'\0'`
- `"GATE"` in memory
- `strlen`
- `sizeof`
- string input/output
- `%s`
- `fgets`
- `strcmp`
- `strcpy`
- `strcat`
- string literals
- `char s[] = "abc"` vs `char *s = "abc"`
- pointer + string relationship
- classic GATE traps such as:

```
C
char s[] = "abc";

sizeof(s)
strlen(s)
```

Why are the answers different?

That lesson directly prepares you for both **GATE C-output questions** and **placement string problems**.

Type **NEXT**.

USER

NEXT

ASSISTANT

MODULE 0 — LESSON 14

Strings in C — Character Arrays, '\0', strlen, sizeof, Input/Output & GATE Traps

We continue from the Pointer Master Section into **Strings**, exactly according to the course flow from your uploaded master prompt.

1. WHERE WE ARE

Module: 0 — C Foundation **Lesson:** 14 **Current topic:** Strings in C

GATE importance: 0=Ý%04Ú%0K%0W **Placement importance:** 0=Ý%04Ú%0K%0W

Today we cover:

- what a string is in C
- character arrays
- null terminator '\0'
- string memory layout
- `char s[] = "GATE"`
- `strlen()`
- `sizeof()`
- `%s`
- `scanf("%s", ...)`
- `fgets()`
- basic string library functions
- `strcmp()`
- `strcpy()`
- `strcat()`

- string literals
- `char s[] = "abc"` vs `char *s = "abc"`
- common GATE traps

2. STORY / INTUITION

Suppose you want to store:

TEXT
GATE

C does not have a built-in high-level `String` type like Java.

Instead, C stores a string as:

An array of characters ending with a special character called the null character.

So `"GATE"` is represented conceptually as:

TEXT
G A T E \0

That final:

C
'\0'

tells C:

The string ends here.

☞ This one character is the foundation of C strings.

3. WHAT IS A STRING IN C?

A C string is:

A sequence of characters terminated by the null character `'\0'`.

Example:

C
`char s[] = "GATE";`

Memory:

TEXT
Index 0 1 2 3 4
+-----+-----+-----+-----+-----+

```
s      | G | A | T | E | \0 |
+----+----+----+----+----+
```

So although "GATE" has:

```
TEXT
4 visible characters
```

the array contains:

```
TEXT
5 characters
```

because of '\0'.

4. IMPORTANT DIFFERENCE

```
TEXT
"GATE"
```

has 4 visible letters.

But storage requires:

```
TEXT
G A T E \0
```

So:

```
TEXT
4 + 1 = 5
```

~~0=7%~~ This directly creates the famous:

```
TEXT
strlen vs sizeof
```

difference.

5. DECLARING A STRING

Method 1

```
C
char s[] = "GATE";
```

Compiler determines required array size.

Equivalent array size:

```
C
char s[5] = "GATE";
```

because we need:

```
TEXT
G A T E \0
```

6. METHOD 2 — CHARACTER LIST

You can write:

```
C
char s[] = {'G', 'A', 'T', 'E', '\0'};
```

This is a proper C string because it ends in:

```
C
'\0'
```

7. IMPORTANT TRAP

Consider:

```
C
char s[] = {'G', 'A', 'T', 'E'};
```

This creates an array of four characters.

But it is **not guaranteed to be a valid C string**, because there is no null terminator.

So passing it to functions expecting a null-terminated string, such as:

```
C
printf("%s", s);
```

causes undefined behavior because they keep searching for `'\0'` beyond the array.

~~Very~~ ^{Very} important.

8. NULL CHARACTER

The null character is:

```
C
'\0'
```

Its numeric value is zero.

Do not confuse it with:

```
c
'0'
```

the character digit zero.

DON'T CONFUSE

Null character

```
c
'\0'
```

numeric value:

```
TEXT
0
```

Used to terminate strings.

Character '0'

```
c
'0'
```

is the digit character.

In ASCII:

```
TEXT
'0' = 48
```

These are completely different.

9. MEMORY TRICK

```
TEXT
'\0'
= NOTHING / END

'0'
= DIGIT ZERO
```

10. STRING OUTPUT — %s

Example:


```
C
char s[] = "GATE";

printf("%s", s);
```

Output:

```
TEXT
GATE
```

How does `printf` know when to stop?

It reads characters beginning at `s` until it encounters:

```
C
'\0'
```

11. POINTER CONNECTION

Remember:

```
C
char s[] = "GATE";
```

In most expressions:

```
TEXT
s
```

decays to:

```
TEXT
&s[0]
```

So `%s` receives a pointer to the first character.

Conceptually:

```
TEXT
s
|
v
G ! '   A ! '   T ! '   E ! '   \ 0
```

`printf("%s", s)` keeps reading until `'\0'`.

12. CHARACTER ACCESS

Because a string is a character array:

```
C
char s[] = "GATE";
```

you can access:

```
TEXT
s[0] = 'G'
s[1] = 'A'
s[2] = 'T'
s[3] = 'E'
s[4] = '\0'
```

13. MODIFYING A CHARACTER ARRAY STRING

```
C
char s[] = "GATE";

s[0] = 'L';

printf("%s", s);
```

Now memory becomes:

```
TEXT
L A T E \0
```

Output:

```
TEXT
LATE
```

This is valid because `s` is an array with modifiable elements.

14. `strlen()`

String functions are declared in:

```
C
#include <string.h>
```

`strlen()` gives the number of characters **before** `'\0'`.

Example:

```
C
char s[] = "GATE";

printf("%zu", strlen(s));
```

Result:

TEXT

4

'\0' is not counted.

15. sizeof() ON STRING ARRAY

Now:

C

```
char s[] = "GATE";
```

Array contains:

TEXT

G A T E \0

Each char has size exactly:

TEXT

1 byte

So:

C

```
sizeof(s)
```

is:

TEXT

5

16. ~~0~~ strlen() VS sizeof()

For:

C

```
char s[] = "GATE";
```

TEXT

```
strlen(s) = 4
```

```
sizeof(s) = 5
```

Why?

strlen

Counts characters until '\0'.

sizeof

Returns storage size of the entire array.

0 = famous GATE trap.

17. MASTER RULE

For:

```
C
char s[] = "abc";
```

Memory:

```
TEXT
a b c \0
```

Therefore:

```
TEXT
strlen(s) = 3
sizeof(s) = 4
```

18. GATE EXAMPLE

```
C
char s[] = "Hello";
```

How many elements?

```
TEXT
H e l l o \0
```

So:

```
TEXT
strlen(s) = 5
sizeof(s) = 6
```

19. EXPLICIT ARRAY SIZE

```
C
char s[10] = "abc";
```

What happens?

Array has 10 elements.

Initial memory:

```
TEXT
a b c \0 0 0 0 0 0 0
```

because remaining elements are zero-initialized.

Then:

```
TEXT
strlen(s) = 3
```

but:

```
TEXT
sizeof(s) = 10
```

Important.

20. EXAMPLE

```
C
char s[20] = "GATE";
```

Then:

```
TEXT
strlen(s) = 4
sizeof(s) = 20
```

Do not assume `sizeof` equals `strlen + 1` when the array was declared larger than necessary.

21. STRING INPUT USING `scanf`

Example:

```
C
char name[20];

scanf("%19s", name);
```

Notice:

```
TEXT
no &
```

Why?

Because `name` is an array.

In this expression context:

TEXT

```
n a m e ! ' & n a m e [ 0 ]
```

So it already provides a pointer to writable character storage.

22. WHY %19s INSTEAD OF %s ?

Suppose:

```
c
char name[20];
```

Using:

```
c
scanf("%s", name);
```

can overflow the array if the input is too long.

Safer:

```
c
scanf("%19s", name);
```

This limits the conversion to at most 19 input characters, leaving space for:

```
c
'\0'
```

0=19 important real-world habit.

23. scanf("%s") STOPS AT WHITESPACE

Suppose:

```
c
char name[30];

scanf("%29s", name);
```

Input:

```
TEXT
Siri Chandana
```

Only:

```
TEXT
Siri
```

is read.

Why?

Because `%s` skips leading whitespace and then reads until the next whitespace.

24. WHEN YOU NEED A FULL LINE

Use:

```
C
fgets()
```

Example:

```
C
char name[30];

fgets(name, sizeof(name), stdin);
```

Now input like:

```
TEXT
Siri Chandana
```

can be read including the space.

25. `fgets()` BASIC FORM

```
C
fgets(array, size, stdin);
```

Example:

```
C
char s[100];

fgets(s, sizeof(s), stdin);
```

It reads at most:

```
TEXT
size - 1
```

characters, leaving room for `'\0'`.

0-65535. Generally safer than unbounded input.

26. NEWLINE BEHAVIOR OF `fgets`

If space is available and a newline is read, `fgets` usually stores that newline too.

Example input:

```
TEXT
GATE<Enter>
```

May produce:

```
TEXT
G A T E \n \0
```

Then:

```
C
strlen(s)
```

may be:

```
TEXT
5
```

not 4.

0=important practical trap.

27. REMOVING `fgets` NEWLINE

A common pattern:

```
C
s[strcspn(s, "\n")] = '\0';
```

using:

```
C
#include <string.h>
```

But for GATE foundation, the main concept is simply:

`fgets` can retain the newline.

We don't need to memorize advanced cleanup yet.

28. `strlen()` IMPLEMENTATION IDEA

How would `strlen` work internally?

Conceptually:


```

c
int length = 0;

while (s[length] != '\0')
{
    length++;
}

```

For:

```

TEXT
GATE

```

Trace:

```

TEXT
G !' c o u n t
A !' c o u n t
T !' c o u n t
E !' c o u n t
\ 0 !' s t o p

```

Result:

```

TEXT
4

```

29. COMPLEXITY OF `strlen`

If string length is `n`, `strlen` must scan characters until `'\0'`.

Therefore:

```

TEXT
Time Complexity = O(n)

```

Auxiliary space:

```

TEXT
O(1)

```

0-1 placement connection.

30. POINTER VERSION OF STRING TRAVERSAL

```

c
char s[] = "GATE";
char *p = s;

while (*p != '\0')
{

```

```
printf("%c", *p);  
p++;  
}
```

Output:

```
TEXT  
GATE
```

This connects pointers and strings directly.

31. VISUALIZATION

```
TEXT  
p  
|  
v  
G A T E \0
```

After:

```
C  
p++;
```

```
TEXT  
p  
|  
v  
G A T E \0
```

and so on.

32. BASIC STRING LIBRARY FUNCTIONS

From:

```
C  
#include <string.h>
```

Important functions:

```
TEXT  
strlen  
strcpy  
strcat  
strcmp
```

You should understand what they do.

33. strcpy()

Syntax:

```
C
strcpy(destination, source);
```

Example:

```
C
char a[20];
char b[] = "GATE";

strcpy(a, b);
```

Afterward:

```
TEXT
a = "GATE"
```

It copies characters including the terminating `'\0'`.

34. IMPORTANT — DESTINATION MUST HAVE SPACE

Bad:

```
C
char a[3];

strcpy(a, "GATE");
```

"GATE" requires:

```
TEXT
5 bytes
```

including `'\0'`.

But array has only 3.

This causes buffer overflow / undefined behavior.

Ø=Important.

35. strcat()

Concatenates strings.

Example:

```
C
char a[20] = "Gate";
char b[] = "CS";
```

```
strcat(a, b);
```

Result:

```
TEXT
a = "GateCS"
```

Again, destination needs enough available space.

36. MEMORY BEFORE `strcat`

```
TEXT
a:
G a t e \0 _ _ _ ...
```

`strcat` finds the first `'\0'` and begins adding characters there.

After:

```
TEXT
G a t e C S \0
```

37. `strcmp()`

This is extremely important.

Suppose:

```
C
strcmp(s1, s2)
```

It compares strings lexicographically.

Broad rule:

```
TEXT
result == 0
!' s t r i n g s   e q u a l

result < 0
!' f i r s t   s t r i n g   c o m e s   b e f o r e   s e c o n d

result > 0
!' f i r s t   s t r i n g   c o m e s   a f t e r   s e c o n d
```

~~Do not~~ Do not memorize exact nonzero return values like `-1` or `1`.

The standard only guarantees negative / zero / positive.

38. GATE TRAP — strcmp

Wrong assumption:

TEXT

```
strcmp("abc", "abd") always returns -1
```

Not guaranteed.

Correct statement:

TEXT

```
returns a negative value
```

because first differing character makes "abc" less than "abd".

39. STRING COMPARISON WITH == — TRAP Ø=Ý%

In C, you generally should **not compare string contents** using:

C

```
s1 == s2
```

For arrays/pointers, that would involve addresses after decay, not lexicographical content equality.

Use:

C

```
strcmp(s1, s2) == 0
```

for string-content equality.

40. STORY

Think:

TEXT

```
== with pointers
```

```
!' Are these addresses the same?
```

```
strcmp
```

```
!' Do these strings contain the same sequence?
```

Very important distinction.

41. EXAMPLE

C

```
char a[] = "GATE";  
char b[] = "GATE";
```

They are separate arrays.

Their contents are equal.

So:

```
c
strcmp(a, b)
```

returns:

```
TEXT
0
```

But you should not reason that:

```
c
a == b
```

means content equality.

42. NOW STRING LITERAL VS CHARACTER ARRAY

This is one of the biggest GATE/interview areas.

Compare:

```
c
char s[] = "abc";
```

and:

```
c
char *p = "abc";
```

They look similar.

But they have important differences.

43. `char s[] = "abc"`

This creates an array containing a copy of the characters:

```
TEXT
a b c \0
```

The array itself is modifiable.

So:

```
C
s[0] = 'A';
```

is valid.

Result:

```
TEXT
Abc
```

44. `char *p = "abc"`

Here:

```
TEXT
"abc"
```

is a string literal.

`p` points to that literal.

Attempting to modify a string literal:

```
C
p[0] = 'A';
```

has undefined behavior.

~~0=1~~ We must know.

45. BEST PRACTICE

When pointing to a string literal, prefer:

```
C
const char *p = "abc";
```

This accurately communicates:

Do not modify the literal through `p`.

46. VISUALIZATION

Character array

```
C
char s[] = "abc";
```

TEXT

```
s:
[a][b][c][\0]
```

You own a modifiable array copy.

Pointer to literal

C

```
const char *p = "abc";
```

TEXT

```
p % % % % % % ! ' s t r i n g   l i t e r a l   " a b c "
```

Pointer can potentially move, but literal contents must not be modified.

47. sizeof DIFFERENCE Ø=Ý%

Assume pointer size is 8.

C

```
char s[] = "abc";
const char *p = "abc";
```

Then:

TEXT

```
sizeof(s) = 4
```

because actual array:

TEXT

```
a b c \0
```

But:

TEXT

```
sizeof(p) = 8
```

under the given assumption, because `p` is a pointer.

Ø=Ý% famous GATE trap.

48. strlen OF BOTH

For:


```
C
char s[] = "abc";
const char *p = "abc";
```

both:

```
C
strlen(s)
```

and:

```
C
strlen(p)
```

return:

```
TEXT
3
```

because both point to a null-terminated sequence of 3 visible characters.

So:

```
TEXT
strlen !' string content length
sizeof !' object storage size
```

49. MASTER TABLE

Assume 64-bit pointer = 8 bytes:

```
C
char s[] = "abc";
const char *p = "abc";
```

Expression	Result
strlen(s)	3
sizeof(s)	4
strlen(p)	3
sizeof(p)	8

Copy this.

50. STRING LITERAL SIZE

What is:

```
C
sizeof("abc")
```

?

The string literal itself is an array:

```
TEXT
a b c \0
```

So:

```
TEXT
sizeof("abc") = 4
```

Ø=important.

51. BUT `strlen("abc")`

Returns:

```
TEXT
3
```

Again:

```
TEXT
sizeof includes \0 storage
strlen doesn't count \0
```

52. ESCAPE CHARACTERS AFFECT LENGTH

Consider:

```
C
char s[] = "A\nB";
```

Characters are:

```
TEXT
'A'
'\n'
'B'
'\0'
```

`\n` is one character.

So:

TEXT

```
strlen(s) = 3  
sizeof(s) = 4
```

Do not count `\` and `n` as two separate characters.

53. ANOTHER EXAMPLE

C

```
char s[] = "A\tB";
```

Characters:

TEXT

```
A  
tab  
B  
\0
```

Therefore:

TEXT

```
strlen = 3  
sizeof = 4
```

54. EMBEDDED NULL — GATE TRAP

Consider:

C

```
char s[] = "ABC\0DEF";
```

Actual array contains:

TEXT

```
A B C \0 D E F \0
```

So:

TEXT

```
sizeof(s) = 8
```

But:

C

```
strlen(s)
```

stops at the **first** `'\0'`.

Therefore:

TEXT

```
strlen(s) = 3
```

0-1% Excellent GATE question.

55. VISUALIZATION

TEXT

Index:

0 1 2 3 4 5 6 7

A B C \0 D E F \0

!'

strlen stops here

So %s also prints only:

TEXT

ABC

56. CHARACTER ARRAY WITHOUT TERMINATOR

C

```
char s[3] = {'a', 'b', 'c'};
```

Here:

TEXT

```
sizeof(s) = 3
```

But calling:

C

```
strlen(s)
```

is invalid because there may be no '\0' within the array.

strlen continues reading beyond it looking for one.

Undefined behavior.

0-1% Be careful.

57. EXPLICIT SIZE EXACTLY TOO SMALL?

Consider:

```
C
char s[3] = "abc";
```

C permits initialization of a character array by a string literal even when there is room for the three characters but not the terminating null.

So the resulting array contains:

```
TEXT
a b c
```

with no `'\0'`.

Thus it is not a null-terminated C string.

~~0-40%~~ Advanced GATE trap.

58. COMPARE

```
C
char a[4] = "abc";
```

Memory:

```
TEXT
a b c \0
```

Valid string.

But:

```
C
char b[3] = "abc";
```

Memory:

```
TEXT
a b c
```

No terminator.

Do not use as a normal C string.

59. STRING LOOP

Common traversal:

```
C
for (int i = 0; s[i] != '\0'; i++)
{
    printf("%c", s[i]);
}
```

```
}
```

For "GATE":

```
TEXT
G
A
T
E
then stop at \0
```

60. COUNTING VOWELS — PLACEMENT FOUNDATION

```
C
int count = 0;

for (int i = 0; s[i] != '\0'; i++)
{
    if (s[i] == 'a' || s[i] == 'e' ||
        s[i] == 'i' || s[i] == 'o' ||
        s[i] == 'u')
    {
        count++;
    }
}
```

Time complexity:

```
TEXT
O(n)
```

Auxiliary space:

```
TEXT
O(1)
```

61. REVERSING A STRING — PLACEMENT IDEA

For length `n`, two-pointer approach:

```
TEXT
left = 0
right = n - 1
```

Then repeatedly swap:

```
TEXT
s[left]
s[right]
```

and move inward.

This gives:

TEXT

```
Time = O(n)
Space = O(1)
```

We'll practice more placement string problems later.

62. GATE FOCUS

0-MUST KNOW:

- C string = char array ending `'\0'`
- `'\0'` vs `'0'`
- `strlen`
- `sizeof`
- `%s`
- array decay
- `char s[]` vs `char *p`
- string literal modification
- embedded `'\0'`
- character array lacking terminator
- `strcmp`
- `%s` stops at whitespace in `scanf`

63. GATE TRAP SHEET

Trap 1

C

```
char s[] = "abc";
```

contains 4 elements.

Trap 2

TEXT

```
strlen("abc") = 3
sizeof("abc") = 4
```

Trap 3

```
C
char s[10] = "abc";
```

```
TEXT
strlen(s)=3
sizeof(s)=10
```

Trap 4

```
C
char s[] = "ABC\0DEF";
```

```
TEXT
strlen(s)=3
```

Trap 5

```
C
char s[] = {'a', 'b', 'c'};
```

not guaranteed to be a valid C string.

Trap 6

```
C
char *p = "abc";
```

modifying literal through `p` !' undefined behavior.

Trap 7

```
C
s1 == s2
```

does not perform string-content comparison.

Use `strcmp`.

64. INTERVIEW QUESTIONS

Q1. What is a string in C?

A null-terminated sequence of characters.

Q2. What is `'\0'` ?

The null character with numeric value zero used to terminate C strings.

Q3. Difference between `strlen` and `sizeof` ?

`strlen` counts characters before the first null terminator; `sizeof` gives object storage size in bytes.

Q4. Why doesn't `scanf("%s", name)` use `&name` ?

The array name decays to a pointer to its first element.

Q5. Difference between `char s[] = "abc"` and `char *p = "abc"` ?

First creates a modifiable character array; second points to a string literal, whose contents must not be modified.

Q6. How do we compare string contents?

Using `strcmp`.

Q7. Complexity of `strlen` ?

$O(n)$.

65. MEMORY TRICKS

String

Characters + STOP sign

STOP sign:

```
c
'\0'
```

`strlen`

Visible journey until STOP

Doesn't count stop sign.

`sizeof`

How much storage was reserved

Array vs Literal Pointer

TEXT

```
char s[] !' OWN COPY  
const char *p !' POINT TO LITERAL
```

66. NOTEBOOK NOTES

C String

Null-terminated character sequence.

C

```
char s[] = "GATE";
```

Memory:

TEXT

```
G A T E \0
```

Null Terminator

C

```
'\0'
```

Not the same as:

C

```
'0'
```

Length

C

```
strlen(s)
```

Counts characters before first `'\0'`.

Time:

TEXT

```
O(n)
```

Size

For:

```
C
char s[] = "abc";
```

```
TEXT
strlen(s)=3
sizeof(s)=4
```

Input

```
C
char s[20];
scanf("%19s", s);
```

stops at whitespace.

For full lines:

```
C
fgets(s, sizeof(s), stdin);
```

Important Functions

```
TEXT
strlen !' length
strcpy !' copy
strcat !' concatenate
strcmp !' compare
```

Array vs Literal

```
C
char s[] = "abc";
```

modifiable array.

```
C
const char *p = "abc";
```

pointer to string literal.

67. 30-SECOND REVISION

Remember:

1. C strings end with `'\0'`.
2. `'\0'` is not `'0'`.

3. `"abc"` needs 4 characters of storage.
 4. `strlen("abc") = 3`.
 5. `sizeof("abc") = 4`.
 6. `%s` prints until `'\0'`.
 7. `scanf("%s")` stops at whitespace.
 8. `fgets` can read spaces.
 9. `strcmp == 0` means equal strings.
 10. Never modify a string literal.
-

68. ACTIVE RECALL

Try first.

Q1

For:

```
C
char s[] = "GATE";
```

what are:

```
TEXT
strlen(s)
sizeof(s)
```

Q2

What is the difference between:

```
C
'\0'
```

and:

```
C
'\0'
```

?

Q3

What does:

```
C
strcmp(a, b) == 0
```

mean?

Q4

Why does `%s` not usually need `&array`?

Q5

Is this safe?

C

```
char *p = "abc";  
p[0] = 'A';
```

ANSWERS

A1

TEXT

```
strlen = 4  
sizeof = 5
```

A2

'\0' is the null character with value zero; '0' is the digit character.

A3

The two strings have equal contents.

A4

Because the array name usually decays to a pointer to its first element.

A5

No. Modifying a string literal is undefined behavior.

69. PRACTICE

Q1

Find:

C

```
char s[] = "abc";
```

TEXT

```
strlen(s)  
sizeof(s)
```

Q2

For:

```
C
char s[10] = "abc";
```

find:

```
TEXT
strlen(s)
sizeof(s)
```

Q3 — NULL CHARACTER

For:

```
C
char s[] = {'A', 'B', '\0', 'C'};
```

What does:

```
C
printf("%s", s);
```

print?

Q4 — EMBEDDED NULL

For:

```
C
char s[] = "ABC\0DEF";
```

find:

```
TEXT
strlen(s)
sizeof(s)
```

Q5 — POINTER VS ARRAY

Assume pointer size = 8:

```
C
char s[] = "hello";
const char *p = "hello";
```

Find:

TEXT

```
sizeof(s)
sizeof(p)
strlen(s)
strlen(p)
```

Q6 — COMPARISON

Why is this not the correct method for testing string contents?

```
c
if (a == b)
```

What should you normally use instead?

Q7 — VALID STRING?

Is this a valid null-terminated C string?

```
c
char s[3] = {'a', 'b', 'c'};
```

Can you safely call:

```
c
strlen(s);
```

?

Q8 — GATE TRAP

```
c
char s[3] = "abc";
```

How many array elements are present?

Does the array contain `'\0'` ?

Is it safe to treat it as a normal C string?

Q9 — PLACEMENT

Write the basic logic to count the number of characters in a string **without using** `strlen()`.

Use only:

```
TEXT
loop
index
```

```
'\0'
```

Also state time and auxiliary-space complexity.

PROGRESS

Completed '

Lesson 10: Pointer basics **Lesson 11:** Pointer arithmetic + arrays **Lesson 12:** Pointers with functions + double pointers **Lesson 13:** Pointer safety + const **Lesson 14:** Strings in C

Next

MODULE 0 / LESSON 15 — Structures & Unions in C

Next we will learn how C groups different types into one object:

```
C
struct Student
{
    int id;
    char grade;
    float marks;
};
```

We will cover:

- `struct`
- declaration and initialization
- accessing members with `.`
- structure pointers
- `->`
- array of structures
- nested structures
- structure assignment
- padding/alignment intuition
- `union`
- `struct` vs `union`
- `sizeof` GATE traps
- the critical expressions:

```
TEXT
p->data
(*p).data
```

and why they are equivalent.

This directly prepares us for the future:


```
c
struct Node
{
    int data;
    struct Node *next;
};
```

which is the foundation of linked lists, trees, and graph adjacency structures.

Type **NEXT**.

Type **NEXT**.

ASSISTANT

MODULE 0 — LESSON 15

Structures & Unions in C

We continue exactly from your uploaded course plan. This lesson is very important because **linked lists, trees, graphs, records, and real C programs all depend heavily on structures.**

1. WHERE WE ARE

Module: 0 — C Foundation **Lesson:** 15 **Current topic:** Structures and Unions

GATE importance: 0=1% MOST KNOW **Placement importance:** 0=1% NOT IMPORTANT

Today we cover:

- why structures exist
- defining a `struct`
- declaring structure variables
- initialization
- member access using `.`
- structure pointers
- member access using `->`
- `p->x` vs `(*p).x`
- arrays of structures
- nested structures
- structure assignment
- passing structures to functions
- self-referential structures
- padding/alignment intuition
- unions

- structure vs union
- common GATE traps

2. STORY / INTUITION

Suppose you want to store information about a student:

TEXT

Roll Number
Name
Marks
Grade

These have different data types:

TEXT

```
R o l l   N u m b e r   ! '   i n t  
N a m e               ! '   c h a r   a r r a y  
M a r k s             ! '   f l o a t  
G r a d e             ! '   c h a r
```

You could create separate variables:

C

```
int roll;  
char name[20];  
float marks;  
char grade;
```

But what if you have:

TEXT

100 students?

Managing separate variables becomes messy.

We want to say:

All these values together represent one Student.

That is exactly what a:

struct

does.

3. WHAT IS A STRUCTURE?

A structure is a user-defined type that groups related variables, possibly of different types, into a single object.

Example:

```
c
struct Student
{
    int roll;
    float marks;
    char grade;
};
```

Think:

```
TEXT
Student
+-----+
| roll   |
| marks  |
| grade  |
+-----+
```

These individual parts are called:

Members

4. DEFINING A STRUCTURE

Syntax:

```
c
struct StructureName
{
    data_type member1;
    data_type member2;
    ...
};
```

Example:

```
c
struct Student
{
    int id;
    float marks;
    char grade;
};
```

Notice the semicolon after the closing brace:

```
c
};
```

It is required.

5. DOES THIS CREATE A STUDENT VARIABLE?

No.

This:

```
c
struct Student
{
    int id;
    float marks;
};
```

defines a structure type/tag.

To create an actual object:

```
c
struct Student s1;
```

Now `s1` contains:

```
TEXT
id
marks
```

6. VISUALIZATION

```
c
struct Student s1;
```

Conceptually:

```
TEXT
s1

+-----+
| id     |
+-----+
| marks  |
+-----+
| grade  |
+-----+
```

One object contains multiple members.

7. ACCESSING MEMBERS — . OPERATOR

Suppose:

```
c
struct Student s1;
```

Access members using:

TEXT

.

Example:

```
C
s1.id = 101;
s1.marks = 92.5f;
s1.grade = 'A';
```

Then:

```
C
printf("%d", s1.id);
```

prints:

TEXT
101

8. MEMORY TRICK

For an ordinary structure variable:

Object DOT member

```
C
s1.id
s1.marks
s1.grade
```

9. COMPLETE EXAMPLE

```
C
#include <stdio.h>

struct Student
{
    int id;
    float marks;
    char grade;
};

int main(void)
{
    struct Student s1;

    s1.id = 101;
    s1.marks = 95.5f;
    s1.grade = 'A';
```

```

    printf("%d %.1f %c",
           s1.id,
           s1.marks,
           s1.grade);

    return 0;
}

```

Output:

```

TEXT
101 95.5 A

```

10. STRUCTURE INITIALIZATION

You can initialize when declaring:

```

C
struct Student s1 = {101, 95.5f, 'A'};

```

Values correspond to member order:

```

TEXT
id      = 101
marks   = 95.5
grade   = A

```

11. DESIGNATED INITIALIZERS

C also allows clearer initialization:

```

C
struct Student s1 =
{
    .id = 101,
    .marks = 95.5f,
    .grade = 'A'
};

```

Ø=Useful but not the highest-priority GATE topic.

12. STRUCTURES CAN MIX DATATYPES

Example:

```

C
struct Employee
{
    int id;
    char name[20];
}

```

```
double salary;  
};
```

This combines:

```
TEXT  
int  
char array  
double
```

inside one object.

That is the main advantage over a normal array.

13. ARRAY VS STRUCTURE

An ordinary array usually stores elements of the same type:

```
C  
int a[5];
```

A structure can group different types:

```
C  
struct Data  
{  
    int x;  
    float y;  
    char z;  
};
```

DON'T CONFUSE

Array

```
TEXT  
same element type
```

Structure

```
TEXT  
members may have different types
```

14. COPYING STRUCTURES

C allows structure assignment between compatible structure objects.

Example:

```
c
struct Student a = {1, 90.0f, 'A'};
struct Student b;

b = a;
```

This copies the members.

Afterward:

```
TEXT
b.id    = 1
b.marks = 90.0
b.grade = A
```

15. IMPORTANT

This is valid:

```
c
b = a;
```

for compatible structures.

This is different from arrays.

You cannot normally write:

```
c
array2 = array1;
```

for ordinary C arrays.

Ø=No GATE/interview distinction.

16. STRUCTURE POINTER

Now connect pointers.

Suppose:

```
c
struct Student s1;
```

We can create:

```
c
struct Student *p = &s1;
```

Now:

TEXT

p !' s 1

17. VISUALIZATION

TEXT

```
p
|
v
+-----+
| id      |
| marks   |
| grade   |
+-----+
          s1
```

p stores the address of the entire structure object.

18. HOW DO WE ACCESS A MEMBER THROUGH POINTER?

One way:

```
c
(*p).id
```

Why?

First:

```
c
*p
```

gives the structure object.

Then:

```
c
(*p).id
```

selects its member.

19. PARENTHESES ARE IMPORTANT

You must write:

```
c
(*p).id
```

not:

```
c
*p.id
```

because `.` has higher precedence than unary `*`.

Without parentheses:

```
c
*p.id
```

would group like:

```
c
*(p.id)
```

which is not what we mean when `p` is a pointer.

20. `->` OPERATOR

Because:

```
c
(*p).id
```

is common, C provides shorthand:

```
c
p->id
```

So:

MASTER EQUIVALENCE

```
TEXT
p->id == (*p).id
```

~~YOU MUST KNOW.~~

21. EXAMPLE

```
c
struct Student s1 = {101, 90.0f, 'A'};

struct Student *p = &s1;

printf("%d", p->id);
```

Output:

TEXT

101

Equivalent:

```
C
printf("%d", (*p).id);
```

22. MEMORY TRICK

Ordinary structure variable

Use:

TEXT

.

Example:

```
C
s.id
```

Pointer to structure

Use:

TEXT

->

Example:

```
C
p->id
```

Think:

DOT for object, ARROW for pointer

23. MODIFYING THROUGH STRUCTURE POINTER

```
C
struct Student s = {1, 50.0f, 'C'};
struct Student *p = &s;

p->marks = 95.0f;
```

Now:

TEXT

```
s.marks = 95.0
```

because `p` points to `s`.

24. COMPLETE POINTER EXAMPLE

```
C
#include <stdio.h>

struct Student
{
    int id;
    float marks;
};

int main(void)
{
    struct Student s = {1, 75.0f};

    struct Student *p = &s;

    p->marks = 90.0f;

    printf("%d %.1f", s.id, s.marks);

    return 0;
}
```

Output:

TEXT

```
1 90.0
```

25. GATE TRAP — `.` VS `->`

Suppose:

```
C
struct Student s;
struct Student *p = &s;
```

Use:

```
C
s.id
```

for object.

Use:

```
c
p->id
```

for pointer.

Do not write:

```
c
p.id
```

because `p` itself is not a structure object.

26. ARRAY OF STRUCTURES

Suppose we have 3 students:

```
c
struct Student s[3];
```

Now:

```
TEXT
s [ 0 ] !'  f i r s t   s t u d e n t
s [ 1 ] !'   s e c o n d
s [ 2 ] !'   t h i r d
```

Access:

```
c
s[0].id = 1;
s[1].id = 2;
s[2].id = 3;
```

27. VISUALIZATION

```
TEXT
s[0]          s[1]          s[2]
+-----+    +-----+    +-----+
| id   |    | id   |    | id   |
| marks|    | marks|    | marks|
+-----+    +-----+    +-----+
```

Very useful for storing records.

28. LOOP THROUGH ARRAY OF STRUCTURES

```
c
for (int i = 0; i < 3; i++)
{
```

```

    printf("%d %.1f\n",
           s[i].id,
           s[i].marks);
}

```

Time for scanning `n` structures:

TEXT

$O(n)$

29. POINTER TO STRUCTURE ARRAY

Suppose:

C

```

struct Student s[3];
struct Student *p = s;

```

Then:

TEXT

```

p      !'   s [ 0 ]
p + 1  !'   s [ 1 ]
p + 2  !'   s [ 2 ]

```

Exactly like normal pointer arithmetic, but step size is:

TEXT

`sizeof(struct Student)`

30. EXAMPLE

C

`(p + 1)->id`

means:

Move to second structure, then access `id`.

Equivalent to:

C

`s[1].id`

if `p = s`.

31. STRUCTURES INSIDE STRUCTURES

A structure member can itself be another structure.

Example:

```
c
struct Date
{
    int day;
    int month;
    int year;
};

struct Student
{
    int id;
    struct Date dob;
};
```

Now:

```
c
struct Student s;
```

Access:

```
c
s.dob.day
s.dob.month
s.dob.year
```

32. VISUALIZATION

TEXT

Student s

```
+-----+
| id      |
+-----+
| dob     |
|  day    |
|  month  |
|  year   |
+-----+
```

This is called a:

Nested Structure

33. PASSING STRUCTURE TO FUNCTION — BY VALUE

Consider:

```
c
void change(struct Student s)
{
    s.id = 100;
}
```

Call:

```
c
struct Student x = {1, 80.0f};

change(x);
```

Will `x.id` become 100?

No.

Because the structure is passed **by value**.

A copy is made.

Same call-by-value rule.

34. STORY

Earlier:

```
c
change(int x)
```

received a copy.

Similarly:

```
c
change(struct Student s)
```

receives a copy of the structure.

Changing the copy does not modify the caller's original object.

35. MODIFY ORIGINAL STRUCTURE USING POINTER

Use:

```
c
void change(struct Student *p)
{
    p->id = 100;
}
```

Call:


```
c
change(&x);
```

Now original structure changes.

Same principle as pointer-based swap.

36. COMPLETE EXAMPLE

```
c
struct Student
{
    int id;
};

void change(struct Student *p)
{
    p->id = 100;
}

int main(void)
{
    struct Student s = {1};

    change(&s);

    printf("%d", s.id);
}
```

Output:

```
TEXT
100
```

37. STRUCTURE RETURN VALUE

Functions can return structures.

Example:

```
c
struct Point
{
    int x;
    int y;
};

struct Point makePoint(void)
{
    struct Point p = {10, 20};
    return p;
}
```

Then:

```
C
struct Point a = makePoint();
```

is valid.

Ø=Useful C feature.

38. SELF-REFERENTIAL STRUCTURE Ø=Ý%Ø=Ý%Ø=Ý%

Now we reach something extremely important for data structures.

Can a structure contain a member of its **own exact type directly**?

Bad idea:

```
C
struct Node
{
    int data;
    struct Node next;
};
```

Why impossible?

Because then `next` contains another full `Node`, which contains another full `Node`, forever.

Size could never be determined.

39. STORY

Imagine:

```
TEXT
Box contains another identical Box
which contains another identical Box
which contains another...
```

Infinite size.

Not possible.

40. CORRECT SELF-REFERENCE USING POINTER

Instead:

```
C
struct Node
{
    int data;
    struct Node *next;
};
```

Why does this work?

Because `next` stores only:

```
TEXT
address of another Node
```

not another full node object inside itself.

Pointer size is finite.

$\emptyset = \text{NULL}$ is the foundation of linked lists.

41. VISUALIZATION

```
TEXT
Node 1                                Node 2

+-----+                            +-----+
| data=10 |                            | data=20 |
+-----+                            +-----+
| next ---|----->| next          |
+-----+                            +-----+
```

That is literally how linked structures are built.

42. LINKED LIST PREVIEW

```
C
struct Node
{
    int data;
    struct Node *next;
};
```

A linked list:

```
TEXT
head
|
v
+---+-----+   +---+-----+   +---+-----+
| 10 | next | -> | 20 | next | -> | 30 | NULL |
+---+-----+   +---+-----+   +---+-----+
```

This is why today's lesson is critical.

43. BINARY TREE PREVIEW

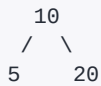
A tree node:

```
C
struct Node
{
```

```
int data;
struct Node *left;
struct Node *right;
};
```

Visualization:

TEXT



`left` and `right` store addresses.

Again:

Structure + pointers = dynamic data structures.

44. `sizeof(struct)` — IMPORTANT GATE AREA

Suppose:

```
c
struct A
{
    int x;
    char c;
};
```

You might think:

TEXT

```
sizeof(int) + sizeof(char)
= 4 + 1
= 5
```

But often:

TEXT

```
sizeof(struct A)
```

may be larger.

Why?

Because of:

Padding / Alignment

45. STORY FOR ALIGNMENT

Imagine parking spaces.

A car might occupy:

```
TEXT
4-unit aligned slots
```

Even if the next item needs only one unit, the system may add empty space so later objects align efficiently.

That empty space is:

Padding

46. SIMPLE PADDING EXAMPLE

Assume:

```
TEXT
sizeof(int) = 4
sizeof(char) = 1
int alignment = 4
```

Structure:

```
C
struct A
{
    char c;
    int x;
};
```

You might expect:

```
TEXT
1 + 4 = 5
```

But a common layout is:

```
TEXT
Offset 0    char c      1 byte
Offset 1-3  padding    3 bytes
Offset 4-7  int x       4 bytes
```

Total:

```
TEXT
8 bytes
```

0=16 important.

47. VISUALIZATION

TEXT

Byte:

0	1	2	3	4	5	6	7
c	P	P	P			x	

P = padding.

48. WHY PADDING EXISTS

Processors often prefer certain data types to start at appropriately aligned addresses.

Alignment can improve:

- access efficiency
- hardware compatibility

Exact rules depend on implementation/platform.

Therefore:

Do not calculate structure size by blindly summing member sizes unless alignment assumptions are explicitly given.

49. TAIL PADDING

Padding can also appear at the end of a structure so that elements in an array of structures remain properly aligned.

Example:

TEXT

struct size may be rounded up

to a multiple of its required alignment.

Important GATE concept.

50. MEMBER ORDER CAN CHANGE STRUCTURE SIZE

Consider:

```
c
struct A
{
    char a;
    int b;
    char c;
};
```

versus:

```
c
struct B
{
    int b;
    char a;
    char c;
};
```

Depending on platform alignment rules, these may have different sizes.

Grouping larger-alignment members appropriately can reduce padding.

Useful systems-programming insight.

51. IMPORTANT CAUTION

Never assume a specific size like:

```
TEXT
struct A = 12 bytes
```

unless:

- member sizes
- alignment rules
- architecture/compiler assumptions

are known or given.

For GATE, questions usually provide enough assumptions or use standard/common alignment models.

52. NOW — UNION

A union looks similar to a structure:

```
c
union Data
{
    int i;
    float f;
    char c;
};
```

But its memory behavior is different.

53. STRUCTURE STORY

In a structure, each member gets its own storage.

```
TEXT
STRUCT
+-----+
```

```
| int |
+-----+
| float |
+-----+
| char |
+-----+
```

All members can hold values at the same time.

54. UNION STORY

In a union, members share the same storage.

TEXT

UNION

```
+-----+
| shared memory |
+-----+
```

int / float / char
all use this same area

Usually only the currently stored member representation should be meaningfully interpreted according to C's rules and intended use.

55. STRUCT VS UNION

Consider:

```
c
struct S
{
    int i;
    float f;
};
```

Structure conceptually:

TEXT

```
+-----+
| i |
+-----+
| f |
+-----+
```

Both have separate storage.

Union:

```
c
union U
{
```



```
int i;
float f;
};
```

Conceptually:

```
TEXT
+-----+
| i or f |
| same memory |
+-----+
```

56. UNION SIZE

Broad rule:

```
TEXT
sizeof(union)
```

must be large enough for its largest member, plus any alignment-related padding.

Example:

```
c
union Data
{
    char c;
    int x;
    double d;
};
```

The size will be at least enough to store the largest member, subject to alignment.

It is **not** normally the sum of all member sizes.

Ø=0% favorite.

57. STRUCTURE SIZE VS UNION SIZE

Suppose hypothetically:

```
TEXT
int    = 4
double = 8
char   = 1
```

Ignoring/assuming straightforward alignment for illustration:

Structure roughly needs storage for all:

```
TEXT
4 + 8 + 1 + possible padding
```

Union needs storage for largest:

TEXT

8 + possible alignment considerations

58. MODIFYING UNION MEMBER

Example:

```
c
union Data
{
    int i;
    float f;
};

union Data d;

d.i = 10;
```

Now storage represents an `int`.

Then:

```
c
d.f = 2.5f;
```

the same memory is reused for `f`.

The previous `i` representation is overwritten.

59. STORY

Imagine one chair shared by three people.

At one time:

TEXT

int sits there

then later:

TEXT

float sits there

They don't get separate chairs.

That's union memory.

60. WHEN ARE UNIONS USED?

Possible uses:

- memory-efficient variant data
- hardware/register interpretation
- embedded systems
- protocol/message representations
- tagged unions

But structures are far more common in everyday DSA.

For GATE, mainly know memory-sharing behavior and size implications.

61. STRUCT VS UNION TABLE

Feature	struct	union
Storage	separate for members	shared
Members simultaneously meaningful	yes	generally one active interpretation at a time
Size	roughly all members + padding	largest member + alignment
Member addresses	may differ	begin at same union storage
Common use	records/nodes	memory sharing/variant data

Copy this.

62. GATE TRAP — MEMBER ADDRESSES IN UNION

Because union members share the same storage, they begin at the same address.

Conceptually:

TEXT

```
&u.i
&u.f
&u.c
```

have the same starting location, though their pointer types differ.

Useful conceptual question.

63. typedef — USEFUL EXTENSION

You may see:

```
c
typedef struct
{
    int x;
    int y;
} Point;
```

Now instead of:

```
C
struct Point p;
```

you can write:

```
C
Point p;
```

`typedef` creates a type alias.

Very common in real C and DSA code.

64. COMMON NODE STYLE

Instead of:

```
C
struct Node
{
    int data;
    struct Node *next;
};
```

you may see:

```
C
typedef struct Node
{
    int data;
    struct Node *next;
} Node;
```

Then:

```
C
Node *head;
```

instead of:

```
C
struct Node *head;
```

We'll use whichever keeps implementations clearest.

65. GATE FOCUS

~~0=14~~ MUST KNOW:

- structure definition
- member access `.`
- pointer member access `->`

- `p->x == (*p).x`
 - structure assignment
 - array of structures
 - passing struct by value
 - structure pointer to modify original
 - self-referential structure
 - padding/alignment
 - union storage sharing
 - struct vs union size concept
-

66. GATE TRAP SHEET

Trap 1

```
C
struct A
{
    int x;
};
```

needs:

```
TEXT
;
```

after the closing brace.

Trap 2

For object:

```
C
s.x
```

For pointer:

```
C
p->x
```

Trap 3

```
C
p->x
```

equals:

```
c
(*p).x
```

not:

```
c
*p.x
```

Trap 4

Passing structure normally copies it.

Trap 5

Self-reference must use a pointer:

```
c
struct Node *next;
```

not a full direct `struct Node next;`.

Trap 6

`sizeof(struct)` may include padding.

Trap 7

Union members share memory.

Trap 8

`sizeof(union)` is based mainly on its largest member plus alignment—not sum of all members.

67. PLACEMENT CONNECTION

Structures are the foundation of C implementations of:

Linked list

```
c
struct Node
{
    int data;
    struct Node *next;
};
```

Binary tree

```
C
struct Node
{
    int data;
    struct Node *left;
    struct Node *right;
};
```

Graph adjacency list

Nodes/edges can use structures and pointers.

Custom records

```
C
struct Employee
struct Product
struct Student
```

Understanding this lesson makes later DSA code much easier.

68. INTERVIEW QUESTIONS

Q1. What is a structure?

A user-defined aggregate type that groups related members, possibly of different types.

Q2. Difference between `.` and `->`?

`.` accesses a member through a structure object.

`->` accesses a member through a pointer to structure.

Q3. What is `p->x` equivalent to?

```
C
(*p).x
```

Q4. Can structures be assigned directly?

Yes, compatible structure objects can be assigned.

Q5. Why can't a structure directly contain itself?

Its size would become infinitely recursive/undeterminable.

Q6. How can it reference another object of its own type?

Using a pointer to its own structure type.

Q7. Difference between struct and union?

A struct allocates separate storage for members; a union shares storage among members.

69. MEMORY TRICKS

Structure

Different things packed into one record.

Dot

I have the object.

```
c  
s.data
```

Arrow

I have the address.

```
c  
p->data
```

Self-reference

Don't put the whole next house inside the house—store its address.

Union

One room, many possible occupants.

70. NOTEBOOK NOTES

Structure

```
c  
struct Student  
{  
    int id;  
    float marks;  
};
```


Object:

```
c
struct Student s;
```

Access:

```
c
s.id
```

Structure Pointer

```
c
struct Student *p = &s;
```

Access:

```
c
p->id
```

Equivalent:

```
c
(*p).id
```

Structure Assignment

```
c
b = a;
```

valid for compatible structures.

Self-Referential Structure

```
c
struct Node
{
    int data;
    struct Node *next;
};
```

Used in linked lists.

Padding

Structure size may exceed sum of member sizes because of alignment/padding.

Union

```
c
union Data
{
    int i;
    float f;
};
```

Members share the same storage.

71. 30-SECOND REVISION

Remember:

1. `struct` groups different members.
 2. `.` accesses member of object.
 3. `->` accesses member through pointer.
 4. `p->x = (*p).x`.
 5. Structures can be copied by assignment.
 6. Passing a structure normally passes a copy.
 7. Use a pointer parameter to modify original structure.
 8. Self-referential structures use pointers.
 9. Structure size may include padding.
 10. Union members share memory.
-

72. ACTIVE RECALL

Try first.

Q1

Which operator accesses `id` from:

```
c
struct Student s;
```

?

Q2

Which operator accesses `id` from:

```
c
struct Student *p;
```

?

Q3

What is:

```
c
p->id
```

equivalent to?

Q4

Why is this invalid as a recursive member?

```
c
struct Node
{
    struct Node next;
};
```

Q5

Main difference between structure and union?

ANSWERS

A1

```
c
s.id
```

A2

```
c
p->id
```

A3

```
c
(*p).id
```

A4

It would require a full `Node` inside another `Node` indefinitely, so the size cannot be finite.

A5

Structure members have separate storage; union members share storage.

73. PRACTICE

Q1

Given:

```
c
struct Point
{
    int x;
    int y;
};

struct Point p = {10, 20};
```

What are:

TEXT
p.x
p.y

?

~~Q2~~

Given:

```
c
struct Point p = {10, 20};
struct Point *q = &p;
```

What does:

c
q->x

produce?

~~Q3~~ — POINTER MODIFICATION

```
c
struct A
{
    int x;
};

struct A a = {10};
struct A *p = &a;

p->x = 50;
```

Find final:

TEXT
a.x

Q4 — STRUCTURE COPY

```
C
struct A
{
    int x;
};

struct A a = {10};
struct A b;

b = a;
b.x = 20;
```

Find:

TEXT
a.x
b.x

Q5 — PASS BY VALUE

```
C
struct A
{
    int x;
};

void change(struct A a)
{
    a.x = 100;
}

int main(void)
{
    struct A a = {10};

    change(a);

    printf("%d", a.x);
}
```

Find output and explain why.

Q6 — POINTER PARAMETER

Rewrite the previous `change()` so that it modifies the original structure's `x` to `100`.

Q7 — ARRAY OF STRUCTURES

Given:

```

C
struct A
{
    int x;
};

struct A a[3] = {{10}, {20}, {30}};

```

What does:

```

C
(a + 2)->x

```

produce?

Q8 — SELF-REFERENCE

Which is correct for a linked-list node?

A

```

C
struct Node
{
    int data;
    struct Node next;
};

```

B

```

C
struct Node
{
    int data;
    struct Node *next;
};

```

Explain why.

Q9 — PADDING THINKING

Assume:

```

TEXT
sizeof(char) = 1
sizeof(int) = 4
int must start at a multiple-of-4 address

```

For:

```

C
struct A
{

```

```
char c;
int x;
};
```

How many padding bytes are likely inserted between `c` and `x`?

What likely total size do you get under these assumptions?

0=10 — PLACEMENT CONNECTION

Explain what the `next` member stores here:

```
c
struct Node
{
    int data;
    struct Node *next;
};
```

Does `next` store the next node itself, or something else?

PROGRESS

Completed '

Lesson 10: Pointer basics **Lesson 11:** Pointer arithmetic + arrays **Lesson 12:** Pointers with functions + double pointers **Lesson 13:** Pointer safety + `const` **Lesson 14:** Strings **Lesson 15:** Structures & Unions

We now have nearly all the C foundation required for actual dynamic data structures.

Next

MODULE 0 / LESSON 16 — Dynamic Memory Allocation

This is the **final major C-foundation lesson** before we begin the full Recursion module.

We will study:

```
c
malloc()
calloc()
realloc()
free()
```

and deeply understand:

- why dynamic memory is required
- stack vs heap again
- `sizeof`
- allocating one object
- allocating arrays
- `malloc` return value
- checking `NULL`

- `malloc` vs `calloc`
- resizing with `realloc`
- freeing memory
- memory leaks
- dangling pointers
- use-after-free
- double free
- ownership intuition
- dynamic linked-list node creation

Example:

```
c
int *p = malloc(5 * sizeof *p);
```

We will understand exactly what happens in memory and why this line is one of the foundations of linked lists, trees, and graphs.

Type **NEXT**.

USER

NEXT

ASSISTANT

MODULE 0 — LESSON 16

Dynamic Memory Allocation in C — malloc, calloc, realloc, free

We now finish the major C-foundation section before moving into **Recursion**. This lesson is critical because linked lists, trees, dynamic arrays, and many graph structures depend on dynamic memory.

1. WHERE WE ARE

Module: 0 — C Foundation **Lesson:** 16 **Current topic:** Dynamic Memory Allocation

GATE importance: Ø=Ý%ØMÚSØKÍÓW **Placement importance:** Ø=Ý%ØMÚSØKÍÓW

Today we cover:

- why dynamic memory is needed
- stack vs heap recap
- `malloc()`
- `calloc()`
- `realloc()`
- `free()`
- `sizeof`

- allocating one object
 - allocating arrays
 - checking allocation failure
 - `malloc` vs `calloc`
 - memory leak
 - dangling pointer
 - use-after-free
 - double free
 - `realloc` traps
 - dynamic structure/node creation
 - complexity implications
-

2. STORY / INTUITION

Imagine you're organizing an event.

At compile time, you don't know whether:

TEXT

```
10 students  
100 students  
1000 students
```

will attend.

If you permanently reserve space for 10,000 students, you may waste memory.

If you reserve space for only 10, you may run out.

Instead you say:

Tell me the number of students at runtime, and I will request exactly the required space.

That is:

Dynamic Memory Allocation

3. STATIC/FIXEDARRAY VS DYNAMIC MEMORY

Suppose:

```
c  
int a[100];
```

Here the array size is fixed by this declaration.

But suppose the user enters:

TEXT

```
n = 500
```

and we want storage for `n` integers.

We can dynamically request memory:

C

```
int *a = malloc(n * sizeof *a);
```

Now the amount of allocated storage depends on `n` at runtime.

4. MEMORY RECAP

Conceptually:

TEXT

PROGRAM MEMORY

```
+-----+
| Code / Text      |
+-----+
| Global / Static  |
+-----+
| Heap            |
| dynamic allocations |
+-----+
|      ...      |
+-----+
| Stack          |
| function calls  |
| automatic locals |
+-----+
```

Dynamic allocation normally comes from:

Heap Storage

5. STACK VS HEAP

Stack-related automatic storage

C

```
void f(void)
{
    int x = 10;
}
```

`x` typically exists only while its block/function execution is active.

Heap allocation

```
C
int *p = malloc(sizeof *p);
```

The dynamically allocated object can continue to exist even after the current function returns, until it is released.

6. REQUIRED HEADER

Dynamic-memory functions are declared in:

```
C
#include <stdlib.h>
```

Important functions:

```
TEXT
malloc
calloc
realloc
free
```

Ø=Must know.

7. malloc()

`malloc` means roughly:

Memory Allocation

Basic form:

```
C
malloc(number_of_bytes);
```

Example:

```
C
int *p = malloc(sizeof(int));
```

This requests enough storage for one `int`.

8. BETTER STYLE

Prefer:

```
C
int *p = malloc(sizeof *p);
```

Why?

Because if `p`'s pointed type changes later, the allocation expression automatically follows it.

For example:

```
C
double *p = malloc(sizeof *p);
```

No need to manually replace `sizeof(int)` with `sizeof(double)`.

Good C style.

9. WHAT DOES `malloc()` RETURN?

`malloc()` returns a pointer to the allocated storage if successful.

Its return type is:

```
TEXT
void *
```

A `void *` can be converted to an object pointer type in C.

So this is valid C:

```
C
int *p = malloc(sizeof *p);
```

No cast is required in C.

10. IMPORTANT C VS C++ NOTE

In C:

```
C
int *p = malloc(sizeof *p);
```

is correct.

You may see:

```
C
int *p = (int *)malloc(sizeof(int));
```

but the cast is unnecessary in C.

For this course, we'll generally prefer:

```
C
int *p = malloc(sizeof *p);
```

Cleaner C.

11. MEMORY VISUALIZATION

```
c
int *p = malloc(sizeof *p);
```

Conceptually:

```
TEXT
STACK                HEAP

p
+-----+           +-----+
| address | -----> |       |
+-----+           +-----+
                        allocated int
```

Then:

```
c
*p = 50;
```

becomes:

```
TEXT
STACK                HEAP

p -----> +-----+
           |   50   |
           +-----+
```

12. VERY IMPORTANT

The pointer variable `p` and the dynamically allocated object are different things.

```
c
int *p = malloc(sizeof *p);
```

Conceptually:

```
TEXT

p
!'  l o c a l   p o i n t e r   v a r i a b l e

*p target
!'  d y n a m i c a l l y   a l l o c a t e d   o b j e c t
```

Don't say both are "on the heap."

13. ALLOCATION FAILURE

Dynamic allocation can fail.

If `malloc()` cannot satisfy the request, it returns:

```
C
NULL
```

Therefore:

```
C
int *p = malloc(sizeof *p);

if (p == NULL)
{
    /* allocation failed */
}
```

~~0=1%~~ Always know this rule.

14. SAFE BASIC PATTERN

```
C
int *p = malloc(sizeof *p);

if (p == NULL)
{
    return 1;
}

*p = 10;

printf("%d", *p);

free(p);
```

This is the basic lifecycle:

```
TEXT
ALLOCATE
!"
CHECK
!"
USE
!"
FREE
```

15. MEMORY TRICK

A-C-U-F

```
TEXT
Allocate
Check
Use
Free
```

16. ALLOCATING AN ARRAY

Suppose:

```
TEXT
n = 5
```

We need five integers.

Use:

```
C
int *a = malloc(n * sizeof *a);
```

If:

```
TEXT
sizeof(int) = 4
```

then requested size is:

```
TEXT
5 × 4 = 20 bytes
```

17. VISUALIZATION

```
C
int *a = malloc(5 * sizeof *a);
```

Conceptually:

```
TEXT
a
|
v
+---+---+---+---+---+
| ? | ? | ? | ? | ? |
+---+---+---+---+---+
  0   1   2   3   4
```

Because allocated storage behaves like contiguous space for the requested objects.

You can use:

```
C
a[0]
a[1]
...
a[4]
```

18. POINTER/ARRAY CONNECTION

Remember:

TEXT

```
a[i] == *(a + i)
```

So dynamic array:

C

```
int *a = malloc(n * sizeof *a);
```

can be accessed just like:

C

```
a[i]
```

because `a` points to the first allocated `int`.

19. COMPLETE DYNAMIC ARRAY EXAMPLE

C

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int n = 5;

    int *a = malloc(n * sizeof *a);

    if (a == NULL)
    {
        return 1;
    }

    for (int i = 0; i < n; i++)
    {
        a[i] = i + 1;
    }

    for (int i = 0; i < n; i++)
    {
        printf("%d ", a[i]);
    }

    free(a);

    return 0;
}
```

Output:

TEXT

```
1 2 3 4 5
```

20. COMPLEXITY

Allocation of `n` elements involves requesting memory proportional to `n`, though allocator implementation details vary.

For algorithm analysis, using the array afterward:

```
C
for (i = 0; i < n; i++)
```

is:

```
TEXT
O(n)
```

Auxiliary allocated space:

```
TEXT
O(n)
```

21. DOES `malloc()` INITIALIZE MEMORY?

No.

This is extremely important.

```
C
int *a = malloc(5 * sizeof *a);
```

The contents are **indeterminate**.

Conceptually:

```
TEXT
+---+---+---+---+---+
| ? | ? | ? | ? | ? |
+---+---+---+---+---+
```

Do not assume zeros.

0 = C++ favorite.

22. `calloc()`

`calloc` means roughly:

Contiguous Allocation

Syntax:

```
C
calloc(number_of_elements, size_of_each_element);
```

Example:

```
C
int *a = calloc(5, sizeof *a);
```

This allocates space for:

```
TEXT
5 integers
```

and initializes all bits of the allocated storage to zero.

For ordinary integer types, this yields integer zero values.

23. VISUALIZATION

```
C
int *a = calloc(5, sizeof *a);
```

Conceptually for ints:

```
TEXT
+---+---+---+---+---+
| 0 | 0 | 0 | 0 | 0 |
+---+---+---+---+---+
```

24. malloc() VS calloc()

Feature	malloc	calloc
Arguments	total bytes	count + element size
Initialization	indeterminate	zeroed storage
Example	malloc(n*sizeof *p)	calloc(n,sizeof *p)

Ø=I don't know.

25. MEMORY TRICK

```
TEXT
malloc
!' m e m o r y   o n l y

calloc
```

```
!'  c l e a r e d   a l l o c a t i o n
```

Think:

C in calloc = Clear

Not the official meaning, but good memory trick.

26. SOLVED EXAMPLE

```
C
int *p = calloc(4, sizeof *p);
```

For integer elements, immediately after successful allocation:

```
TEXT
p[0] = 0
p[1] = 0
p[2] = 0
p[3] = 0
```

27. free()

When dynamic storage is no longer needed:

```
C
free(p);
```

This releases the allocation associated with `p`.

Without releasing unused allocations, your program may leak memory.

28. STORY

Imagine renting a hotel room.

```
TEXT
malloc/calloc
!'  t a k e   r o o m

use
!'  s t a y   i n   r o o m

free
!'  r e t u r n   r o o m
```

If you leave without returning the room:

TEXT

memory leak

29. MEMORY LEAK

Example:

```
C
void f(void)
{
    int *p = malloc(100 * sizeof *p);

    /* use p */

    return;
}
```

If the allocation is never freed and no pointer to it remains accessible after `f` returns, the allocated storage remains occupied for the program's lifetime but cannot be usefully reached.

This is a:

Memory Leak

30. VISUALIZATION

During function:

TEXT

p -----> HEAP BLOCK

Function returns.

Local `p` disappears:

TEXT

```

    HEAP BLOCK
    still allocated
!
no pointer left
```

That memory is leaked.

31. GOOD VERSION

```
C
void f(void)
{
    int *p = malloc(100 * sizeof *p);
```

```

    if (p == NULL)
        return;

    /* use memory */

    free(p);
}

```

Now allocation is released.

32. DANGLING POINTER AFTER free

Consider:

```

c
int *p = malloc(sizeof *p);

*p = 10;

free(p);

```

After `free(p)`:

```

TEXT
allocated object's lifetime ended

```

but `p` may still contain the old address.

So:

```

TEXT
p  !'  d a n g l i n g

```

33. BAD

```

c
free(p);

printf("%d", *p);

```

This is:

Use-After-Free

Undefined behavior.

Ø=We don't know.

34. BETTER PRACTICE

```
c
free(p);
p = NULL;
```

Now `p` clearly indicates no valid target.

But remember from Lesson 13:

Other aliases may still be dangling.

35. ALIASING TRAP

```
c
int *p = malloc(sizeof *p);
int *q = p;

free(p);
p = NULL;
```

Then:

```
TEXT
p  !'  N U L L
q  !'  d a n g l i n g
```

`q` still contains the freed address.

36. DOUBLE FREE

Bad:

```
c
free(p);
free(p);
```

if `p` still refers to the same already-freed allocation.

This causes undefined behavior.

Known as:

Double Free

37. WHY `p = NULL` HELPS

```
c
free(p);
p = NULL;
```

If later:

```
c
free(p);
```

standard C guarantees:

```
c
free(NULL);
```

does nothing.

So this avoids one common double-free pattern through the same pointer variable.

Again, aliases still require care.

38. `realloc()`

Now suppose you allocated space for:

```
TEXT
5 integers
```

but later need:

```
TEXT
10 integers
```

Instead of manually allocating and copying, C provides:

```
c
realloc()
```

39. BASIC FORM

```
c
p = realloc(p, new_size_in_bytes);
```

Example:

```
c
p = realloc(p, 10 * sizeof *p);
```

This requests the allocation associated with `p` to be resized.

40. WHAT CAN `realloc()` DO?

It may:

Case 1

Extend the same block in place.

TEXT

```
o l d   l o c a t i o n   ! '   l a r g e r   b l o c k
```

Case 2

Allocate a new block elsewhere, copy preserved data, and release the old block.

TEXT

```
old location  
!"  c o p y  
new location
```

Therefore:

The address may change.

Very important.

41. VISUALIZATION

Before:

TEXT

```
p  
|  
v  
[10][20][30]
```

After larger `realloc`:

Could become:

TEXT

```
p  
|  
v  
[10][20][30][?][?]
```

or move somewhere completely different.

42. DATAPRESERVATION

When `realloc` succeeds, contents are preserved up to the minimum of:

TEXT

```
old size  
new size
```

Newly added bytes are not automatically initialized.

Ø=important.

43. GROWING MEMORY

```
C
int *p = malloc(3 * sizeof *p);
```

Suppose:

```
TEXT
p = [10,20,30]
```

Then:

```
C
p = realloc(p, 5 * sizeof *p);
```

After successful reallocation:

```
TEXT
first three values preserved:
10 20 30
```

new positions:

```
TEXT
p[3], p[4]
```

have indeterminate contents until initialized.

44. SHRINKING MEMORY

Suppose:

```
TEXT
[10][20][30][40][50]
```

Then:

```
C
p = realloc(p, 3 * sizeof *p);
```

successful result preserves:

```
TEXT
[10][20][30]
```

Data beyond new size is discarded.

45. ~~0=Big~~ realloc TRAP

This is risky:

```
C
p = realloc(p, new_size);
```

Why?

If `realloc` fails, it returns:

```
TEXT
NULL
```

and the original allocation remains valid.

But by assigning directly:

```
C
p = NULL
```

you may lose the only pointer to the original allocation.

That creates a memory leak.

46. SAFER realloc PATTERN

Use a temporary pointer:

```
C
int *temp = realloc(p, new_size);

if (temp != NULL)
{
    p = temp;
}
else
{
    /* original p is still valid */
}
```

~~0=Ex~~cellent interview/placement pattern.

47. STORY

Suppose your old hotel room exists.

You ask:

Move me to a larger room.

If the request fails, the old room is still yours.

But if you immediately throw away your old room number before checking the new one, you've lost access to it.

That's the direct- `realloc` assignment problem.

48. `realloc(NULL, size)`

Useful fact:

```
c
realloc(NULL, size)
```

behaves like an allocation request similar to:

```
c
malloc(size)
```

for nonzero size.

Ø=Øbod to know.

49. `realloc(p, 0)` — CAUTION

Handling a zero new size has historically had subtle/standard-version-dependent behavior.

For GATE and safe programming:

Avoid relying on `realloc(p, 0)` tricks unless a question explicitly specifies the C standard/behavior.

Use:

```
c
free(p);
p = NULL;
```

when your intent is simply to release memory.

50. ALLOCATING A STRUCTURE

Suppose:

```
c
struct Student
{
    int id;
    float marks;
};
```

Dynamic object:

```
c
struct Student *p = malloc(sizeof *p);
```

Now:

TEXT

```
p !' dynamically allocated student
```

Use:

C

```
p->id = 101;
p->marks = 90.0f;
```

51. VISUALIZATION

TEXT

STACK

HEAP

```
p -----> +-----+
              | id = 101   |
              | marks = 90  |
              +-----+
```

This directly prepares us for linked lists.

52. DYNAMIC LINKED-LIST NODE

C

```
struct Node
{
    int data;
    struct Node *next;
};
```

Create one node:

C

```
struct Node *node = malloc(sizeof *node);
```

Check:

C

```
if (node == NULL)
{
    /* allocation failed */
}
```

Initialize:

C

```
node->data = 10;
node->next = NULL;
```

53. MEMORY VISUALIZATION

```
TEXT
node
|
v
HEAP
+-----+
| data = 10      |
| next = NULL    |
+-----+
```

This exact pattern will appear constantly in linked lists and trees.

Ø=Very important.

54. WHY NOT ORDINARY LOCAL NODE?

Suppose:

```
c
struct Node *createNode(void)
{
    struct Node x;
    x.data = 10;

    return &x;
}
```

Bad.

Because `x` is automatic and dies when function returns.

55. CORRECT DYNAMIC VERSION

```
c
struct Node *createNode(void)
{
    struct Node *p = malloc(sizeof *p);

    if (p == NULL)
        return NULL;

    p->data = 10;
    p->next = NULL;

    return p;
}
```

The dynamically allocated node survives after the helper function returns.

Caller later frees it when appropriate.

Ø=This is fundamental DSA C code.

56. STACK VS HEAP EXAMPLE

```
c
struct Node *create(void)
{
    struct Node *p = malloc(sizeof *p);

    return p;
}
```

Inside function:

```
TEXT
p
!'  l o c a l   p o i n t e r   v a r i a b l e
!'  a u t o m a t i c   s t o r a g e
```

Pointed node:

```
TEXT
*p
!'  d y n a m i c   a l l o c a t i o n
!'  h e a p   s t o r a g e
```

When function returns:

```
TEXT
local p disappears
```

but returned address is copied to caller, so heap node remains accessible.

57. sizeof AND ALLOCATION

Why use:

```
c
malloc(n * sizeof *p)
```

instead of:

```
c
malloc(n * 4)
```

?

Because:

```
TEXT
sizeof(int)
```

is not universally 4 bytes.

And different pointed types have different sizes.

Portable code uses `sizeof`.

58. INTEGER OVERFLOW CAUTION

For huge values of `n`, conceptually:

```
c
n * sizeof *p
```

can overflow the size type before `malloc` sees it.

This matters in secure systems programming.

For GATE foundation:

Know that allocation size calculations themselves must be valid.

We won't go deep yet.

59. `calloc` CAN HELP WITH MULTIPLICATION API SHAPE

```
c
calloc(n, sizeof *p)
```

takes count and size separately.

However, you still need to handle allocation failure and should not assume it solves every oversized-allocation problem automatically across all environments.

Systems-level awareness.

60. GATE TRAP — `malloc` CONTENT

```
c
int *p = malloc(sizeof *p);
```

What is `*p` immediately after successful allocation?

Not guaranteed to be:

```
TEXT
0
```

It is indeterminate until initialized.

61. GATE TRAP — `calloc` CONTENT

```
C
int *p = calloc(1, sizeof *p);
```

For ordinary integer type:

```
TEXT
*p = 0
```

after successful allocation.

62. GATE TRAP — LEAK

```
C
int *p = malloc(sizeof *p);

p = malloc(sizeof *p);
```

What happened?

The first allocated address stored in `p` was overwritten by the second one.

If no other pointer preserved the first block, it is now leaked.

0-6% Classic memory-leak pattern.

63. VISUALIZATION

First:

```
TEXT
p !' B l o c k A
```

Then:

```
C
p = malloc(...);
```

Now:

```
TEXT
p !' B l o c k B

B l o c k A !' s t i l l a l l o c a t e d
    but lost
```

Memory leak.

64. CORRECT VERSION

If old block isn't needed:


```
c
free(p);
p = malloc(sizeof *p);
```

or if resizing existing allocation:

```
c
realloc(...)
```

depending on intent.

65. GATE TRAP — FREEING STACK MEMORY

This is invalid:

```
c
int x = 10;

free(&x);
```

`x` was not allocated by `malloc` / `calloc` / `realloc`.

`free()` should be applied only to pointers suitable for freeing dynamic allocations obtained from the allocation family.

66. INVALID FREE

Also dangerous:

```
c
int *p = malloc(5 * sizeof *p);

free(p + 1);
```

You must pass the appropriate original allocation pointer returned by the allocator, not an interior pointer.

Very important.

67. OWNERSHIP INTUITION

Dynamic allocation creates a responsibility:

Someone must eventually release this allocation.

Think:

```
TEXT
create memory
!"
who owns it?
```

```
!“  
who frees it?
```

This idea is called:

Ownership

C does not automatically enforce ownership.

The programmer must design it.

68. LINKED-LIST OWNERSHIP PREVIEW

If list contains:

```
TEXT  
h e a d ! '   n o d e 1 ! '   n o d e 2 ! '   n o d e 3
```

eventually you need to free all nodes.

Conceptually:

```
TEXT  
free node1  
free node2  
free node3
```

But before freeing one node, you often need to save the address of the next node.

We'll implement this carefully later.

69. DYNAMIC MEMORY COMPLEXITY

Suppose we allocate:

```
C  
int *a = malloc(n * sizeof *a);
```

Storage complexity:

```
TEXT  
O(n)
```

If we allocate one node per element:

```
TEXT  
n nodes
```

space:

TEXT

$O(n)$

The pointer structure affects additional overhead.

For example, singly linked-list node:

TEXT

data + one pointer

per element.

70. MEMORY LEAK VS DANGLING POINTER

Do not confuse.

Memory Leak

Memory still allocated, but you've lost access to it.

TEXT

memory exists
pointer lost

Dangling Pointer

Pointer exists, but object is gone.

TEXT

pointer exists
object gone

Memory trick:

TEXT

LEAK:
memory alive, pointer lost

DANGLING:
pointer alive, object dead

71. USE-AFTER-FREE VS DOUBLE FREE

Use-after-free

C

```
free(p);  
*p = 10;
```

Accessing released memory.

Double free

```
c
free(p);
free(p);
```

Attempting to free same allocation twice through stale pointer.

Both:

```
TEXT
undefined behavior
```

72. MASTER DYNAMIC-MEMORY TABLE

Function	Purpose
malloc(bytes)	allocate bytes, uninitialized
calloc(n,size)	allocate n elements, zeroed storage
realloc(p,newsize)	resize existing allocation
free(p)	release allocation

Memorize this.

73. GATE FOCUS

High-yield topics:

- heap vs stack
- malloc returns pointer
- check for NULL
- malloc uninitialized
- calloc zeroed
- realloc may move memory
- safe realloc with temporary pointer
- free
- dangling pointer
- memory leak
- use-after-free
- double free
- dynamic node creation

74. GATE TRAP SHEET

Trap 1

```
c
malloc(...)
```

does not zero memory.

Trap 2

```
c
calloc(...)
```

zeroes allocated storage.

Trap 3

```
c
free(p);
```

does not automatically set `p = NULL`.

Trap 4

```
c
p = realloc(p, size);
```

can lose original allocation if `realloc` fails.

Trap 5

After successful `realloc`, the address may change.

Trap 6

Never:

```
c
free(&local);
```

for ordinary local memory.

Trap 7

Never free an interior pointer such as:

```
C
free(p + 1);
```

Trap 8

Lost allocation:

```
TEXT
memory leak
```

Dead object with surviving pointer:

```
TEXT
dangling pointer
```

75. PLACEMENT CONNECTION

Dynamic memory is essential for implementing:

- linked lists
- trees
- adjacency lists
- queues using nodes
- stacks using nodes
- dynamically sized arrays
- custom data structures

Example:

```
C
struct Node *newNode = malloc(sizeof *newNode);
```

You will see this repeatedly from now on.

76. INTERVIEW QUESTIONS

Q1. Difference between `malloc` and `calloc`?

`malloc` allocates requested bytes without initialization; `calloc` allocates count × size and zeroes the storage.

Q2. What does `malloc` return on failure?

`NULL`.

Q3. Why check `malloc` result?

Dereferencing a failed allocation result would be invalid.

Q4. What is a memory leak?

Allocated memory that is no longer reachable and therefore cannot be freed/used properly.

Q5. What is a dangling pointer?

A pointer referring to an object whose lifetime has ended.

Q6. Can `realloc` change the address?

Yes.

Q7. Why use a temporary pointer with `realloc`?

So the original pointer remains available if reallocation fails.

77. MEMORY TRICKS

`malloc`

Make memory

`calloc`

Clear memory

`realloc`

Resize memory

`free`

Return memory

Leak vs Dangling

TEXT

LEAK:

Memory exists
Pointer missing

DANGLING:

Pointer exists
Memory/object gone

78. NOTEBOOK NOTES

Dynamic Allocation Header

```
C
#include <stdlib.h>
```

malloc

```
C
int *p = malloc(n * sizeof *p);
```

- allocates dynamic storage
- contents indeterminate
- returns `NULL` on failure

calloc

```
C
int *p = calloc(n, sizeof *p);
```

- allocates `n` elements
- zeroes allocated storage

realloc

```
C
int *temp = realloc(p, new_n * sizeof *p);

if (temp != NULL)
{
    p = temp;
}
```

- may keep same address
- may move block
- preserves existing contents up to smaller of old/new sizes

free

```
C
free(p);
p = NULL;
```

releases dynamic allocation.

Errors

TEXT

memory leak
dangling pointer
use-after-free
double free

79. 30-SECOND REVISION

Remember:

1. Dynamic memory comes from heap storage.
 2. Use `<stdlib.h>`.
 3. `malloc` allocates but doesn't initialize.
 4. `calloc` allocates and zeroes.
 5. Always consider allocation failure.
 6. `realloc` can move the allocation.
 7. Use a temporary pointer for safe `realloc`.
 8. `free` releases dynamic memory.
 9. After `free`, pointer may dangle.
 10. Lost allocated block = memory leak.
-

80. ACTIVE RECALL

Try first.

Q1

What does this allocate?

```
c
int *p = malloc(5 * sizeof *p);
```

Q2

Are the five integers initialized to zero?

Q3

What is the main difference if we use:

```
c
calloc(5, sizeof *p);
```

instead?

Q4

Why is this risky?

```
c
p = realloc(p, new_size);
```

Q5

What's the difference between memory leak and dangling pointer?

ANSWERS

A1

Enough dynamic storage for 5 objects of the type pointed to by `p`, here 5 integers.

A2

No. Their values are indeterminate until initialized.

A3

The allocated storage is zeroed; for integers the elements begin as zero.

A4

If `realloc` fails, it returns `NULL`, and direct assignment may lose the only pointer to the original allocation.

A5

Leak = allocation remains but access is lost. Dangling pointer = pointer remains but pointed object's lifetime ended.

81. PRACTICE

Q1

Which function should you use to allocate storage for 10 integers without automatic initialization?

Q2

Which function allocates storage for 10 integers with zeroed initial storage?

Q3 — MEMORY

For:

```
c
int *p = malloc(4 * sizeof *p);
```

Assuming allocation succeeds, where are these conceptually?

1. pointer variable `p`
2. four allocated integers

Q4 — LEAK

What's wrong?

```
c
int *p = malloc(sizeof *p);

p = malloc(sizeof *p);
```

Assume both calls succeed and no other pointer stores the first address.

Q5 — DANGLING

What is wrong?

```
c
int *p = malloc(sizeof *p);

*p = 10;

free(p);

printf("%d", *p);
```

Q6 — calloc

After:

```
c
int *p = calloc(5, sizeof *p);
```

assuming success, what should:

```
c
p[0]
p[1]
p[2]
```

initially contain for ordinary `int`?

Q7 — REALLOC TRAP

Rewrite this safely:

```
c
p = realloc(p, 100 * sizeof *p);
```

so that the old allocation is not lost if resizing fails.

0=08 — INVALID FREE

Why are these wrong?

```
c
int x = 10;
free(&x);
```

and:

```
c
int *p = malloc(10 * sizeof *p);
free(p + 2);
```

0=09 — LINKED LIST FOUNDATION

Given:

```
c
struct Node
{
    int data;
    struct Node *next;
};
```

Write the essential statements to:

1. dynamically allocate one node,
2. check allocation success,
3. set `data = 50`,
4. set `next = NULL`.

PROGRESS

C FOUNDATION COMPLETED '

We have now completed the planned major C foundation:

1. C program basics
2. Operators
3. Type conversion
4. Input/output
5. Decision making
6. Loops
7. Functions
8. Scope/lifetime/storage
9. Memory basics
10. Pointer basics
11. Pointer arithmetic + arrays

12. Pointers with functions + double pointers

13. Pointer safety + `const`

14. Strings

15. Structures & unions

16. Dynamic memory allocation

This gives you the foundation needed for nearly every later topic.

NEXT — MODULE 1: RECURSION

Lesson 17 — Recursion Fundamentals: Base Case, Recursive Call & Call Stack

We will start with the simplest story:

```
C
void fun(int n)
{
    if (n == 0)
        return;

    printf("%d ", n);

    fun(n - 1);
}
```

and trace:

```
TEXT
fun(3)
!"
fun(2)
!"
fun(1)
!"
fun(0)
```

Then we'll separately study the return journey:

```
TEXT
fun(0)
!'
fun(1)
!'
fun(2)
!'
fun(3)
```

You will learn:

- what recursion actually is
- base case
- recursive case
- why recursion stops
- activation records
- stack growth

- downward calls vs upward returns
- infinite recursion
- recursion vs loops
- first GATE output questions
- first recursion space-complexity analysis

Type **NEXT**.

USER

NEXT

ASSISTANT

MODULE 1 — LESSON 17

Recursion Fundamentals — Base Case, Recursive Call & Call Stack

We now begin the **Recursion module**, continuing directly from the course sequence already established.

1. WHERE WE ARE

Module: 1 — Recursion **Lesson:** 17 **Current topic:** Recursion fundamentals

GATE importance: Ø=Ý%ØMÚ%ØKÍØ **Placement importance:** Ø=Ý%ØMÚ%ØKÍØ

Today we cover:

- what recursion is
- recursive function
- base case
- recursive case
- call stack
- downward calls
- upward returns
- infinite recursion
- recursion vs loop
- first recursion output questions
- time and space intuition

2. STORY / INTUITION

Imagine standing in front of 5 locked doors.

Each door says:

To open me, first open the door with the next smaller number.

So:

TEXT

```
Door 5
!"
Door 4
!"
Door 3
!"
Door 2
!"
Door 1
```

Door 1 says:

Stop. No smaller door exists.

That stopping condition is the:

Base Case

Then the doors finish in reverse order:

TEXT

```
Door 1
!'
Door 2
!'
Door 3
!'
Door 4
!'
Door 5
```

That is recursion.

3. WHAT IS RECURSION?

A function is recursive when it calls itself directly or indirectly.

Simple example:

```
c
void fun(int n)
{
    if (n == 0)
        return;

    printf("%d ", n);

    fun(n - 1);
}
```

Here:

```
C
fun(n - 1);
```

calls the same function again.

So `fun()` is recursive.

4. TWO PARTS OF RECURSION

A correct recursive function usually needs:

1. Base Case

The stopping condition.

```
C
if (n == 0)
    return;
```

2. Recursive Case

The part that reduces the problem and calls itself.

```
C
fun(n - 1);
```

Both are essential.

5. MEMORY TRICK

Think:

```
TEXT
B A S E ! '   S T O P
R E C U R S I O N ! '   C O N T I N U E   S M A L L E R
```

A good recursive function asks:

When do I stop?

and

How do I move closer to stopping?

6. FIRST TRACE

Consider:

```
c
void fun(int n)
{
    if (n == 0)
        return;

    printf("%d ", n);

    fun(n - 1);
}
```

Call:

```
c
fun(3);
```

Trace downward:

```
TEXT
fun(3)
print 3
call fun(2)

fun(2)
print 2
call fun(1)

fun(1)
print 1
call fun(0)

fun(0)
return
```

Output:

```
TEXT
3 2 1
```

7. DOWNWARD CALLS

This phase is very important.

```
TEXT
fun(3)
!"
fun(2)
!"
fun(1)
!"
fun(0)
```

Each call waits for the next recursive call to finish.

This creates multiple active function invocations.

8. CALL STACK VISUALIZATION

When `fun(3)` calls `fun(2)`:

```
TEXT
STACK

+-----+
| fun(2) |
+-----+
| fun(3) |
+-----+
```

Then `fun(2)` calls `fun(1)`:

```
TEXT

+-----+
| fun(1) |
+-----+
| fun(2) |
+-----+
| fun(3) |
+-----+
```

Then `fun(1)` calls `fun(0)`:

```
TEXT

+-----+
| fun(0) |
+-----+
| fun(1) |
+-----+
| fun(2) |
+-----+
| fun(3) |
+-----+
```

This is why recursion consumes stack space.

9. BASE CASE REACHED

At:

```
C
fun(0);
```

condition:

```
C
if (n == 0)
```

is true.

So:

```
c
return;
```

executes.

No more recursive calls are created.

Now the stack starts unwinding.

10. UPWARD RETURNS

After `fun(0)` returns:

```
TEXT
fun(0)
! '
fun(1)
! '
fun(2)
! '
fun(3)
```

This is the second half of recursion.

~~Ø=Ý00~~ You must always think in two directions:

```
TEXT
DOWNWARD CALLS
UPWARD RETURNS
```

11. MASTER RECURSION RULE

Whenever tracing recursion, write:

```
TEXT
C A L L   P H A S E   ! "
R E T U R N   P H A S E   ! '
```

This one habit will solve many GATE questions.

12. WHAT IF CODE EXISTS AFTER RECURSIVE CALL?

Now consider:

```
c
void fun(int n)
{
    if (n == 0)
        return;

    printf("%d ", n);
```

```
    fun(n - 1);  
    printf("%d ", n);  
}
```

Call:

```
c  
fun(3);
```

This is where recursion becomes interesting.

13. TRACE CAREFULLY

Downward phase

`fun(3) :`

```
TEXT  
print 3  
call fun(2)
```

`fun(2) :`

```
TEXT  
print 2  
call fun(1)
```

`fun(1) :`

```
TEXT  
print 1  
call fun(0)
```

`fun(0) :`

```
TEXT  
return
```

So far output:

```
TEXT  
3 2 1
```

14. NOW RETURN PHASE

After `fun(0)` returns, execution resumes in `fun(1)` **after**:

```
C
fun(n - 1);
```

So:

```
TEXT
fun(1) prints 1
```

Then `fun(1)` returns to `fun(2)`.

`fun(2)` prints:

```
TEXT
2
```

Then `fun(3)` prints:

```
TEXT
3
```

Final output:

```
TEXT
3 2 1 1 2 3
```

0-Classic recursion pattern.

15. VISUALIZATION

Code:

```
C
printf("%d ", n);
fun(n - 1);
printf("%d ", n);
```

Think:

```
TEXT
BEFORE recursive call
!"
prints on downward journey

AFTER recursive call
!'
prints on upward journey
```

16. MEMORY TRICK

Code before recursion

Runs while going:

```
TEXT
D O W N !"
```

Code after recursion

Runs while coming:

```
TEXT
U P !'
```

Extremely useful.

17. GATE-STYLE EXAMPLE

```
C
void f(int n)
{
    if (n == 0)
        return;

    printf("%d ", n);
    f(n - 1);
}
```

Call:

```
C
f(4);
```

Output:

```
TEXT
4 3 2 1
```

18. ANOTHER VERSION

```
C
void f(int n)
{
    if (n == 0)
        return;

    f(n - 1);

    printf("%d ", n);
}
```

Call:

```
C
f(4);
```

Downward:

```
TEXT
f(4)
f(3)
f(2)
f(1)
f(0)
```

No printing yet.

Then upward:

```
TEXT
1
2
3
4
```

Output:

```
TEXT
1 2 3 4
```

Ø=I don't know.

19. DON'T CONFUSE

Version A

```
C
printf(...);
f(n - 1);
```

Output tends downward:

```
TEXT
n ... 1
```

Version B

```
C
f(n - 1);
printf(...);
```

Output tends upward:

TEXT

1 ... n

20. WHY DOES SECOND VERSION REVERSE THE ORDER?

Because every function call waits.

For:

C

f(3);

f(3) cannot print until:

TEXT

f(2)

finishes.

f(2) cannot print until:

TEXT

f(1)

finishes.

So smallest case completes first.

Then stack unwinds.

21. ACTIVATION RECORD

Each function invocation has its own:

- parameter values
- local variables
- return information
- execution state

Example:

C

```
void f(int n)
```

During:

TEXT

f(3)

f(2)

f(1)

there are three different `n` objects conceptually:

TEXT

```
f(3): n = 3
f(2): n = 2
f(1): n = 1
```

Do not think there is only one shared `n`.

22. VISUALIZATION

TEXT

STACK

```
+-----+
| f(1)   |
| n = 1  |
+-----+
| f(2)   |
| n = 2  |
+-----+
| f(3)   |
| n = 3  |
+-----+
```

Each call keeps its own state.

23. CONNECTION TO CALL BY VALUE

Why are the values separate?

Because each recursive call:

C

```
f(n - 1);
```

passes a value to the next invocation.

C is still call by value.

So:

TEXT

```
f(3) has n=3
f(2) has n=2
f(1) has n=1
```

separately.

24. INFINITE RECURSION

Consider:

```
c
void f(int n)
{
    printf("%d", n);

    f(n - 1);
}
```

There is no base case.

Call:

```
c
f(3);
```

gives:

```
TEXT
f(3)
f(2)
f(1)
f(0)
f(-1)
f(-2)
...
```

until stack space is exhausted or another failure occurs.

This is effectively:

Infinite recursion

25. BASE CASE ALONE IS NOT ENOUGH

Consider:

```
c
void f(int n)
{
    if (n == 0)
        return;

    f(n + 1);
}
```

Call:

```
c
f(5);
```

There is technically a base case:

TEXT

```
n == 0
```

But recursive step moves:

TEXT

```
5 !' 6 !' 7 !' . . .
```

away from zero.

So base case is never reached.

0 = 1/0 important reasoning.

26. RECURSION MUST MAKE PROGRESS

Correct recursion requires:

1. a base case
2. recursive calls that move toward that base case

Memory trick:

STOP + SHRINK

For many problems:

TEXT

```
stop condition  
+  
smaller problem
```

27. EXAMPLE

Correct:

C

```
f(n - 1);
```

with:

C

```
if (n == 0)
```

because:

TEXT

```
5 !' 4 !' 3 !' 2 !' 1 !' 0
```

Incorrect:

```
c
f(n + 1);
```

because:

```
TEXT
5 !' 6 !' 7 !' . . .
```

28. RECURSION VS LOOP

Suppose we want to print:

```
TEXT
5 4 3 2 1
```

Loop:

```
c
for (int i = 5; i >= 1; i--)
{
    printf("%d ", i);
}
```

Recursion:

```
c
void print(int n)
{
    if (n == 0)
        return;

    printf("%d ", n);
    print(n - 1);
}
```

Both can solve the problem.

29. DIFFERENCE

Loop

Uses explicit looping mechanism.

```
TEXT
for / while
```

Recursion

Uses repeated function calls.

TEXT

function calls itself

Recursion naturally uses the call stack.

30. SPACE DIFFERENCE

Loop version:

```
C
for (...)
```

typically uses:

TEXT
 $O(1)$

auxiliary space for this simple example.

Recursive version:

```
TEXT
f(n)
!'  f ( n - 1 )
!'  . . .
```

creates approximately n active calls.

So:

TEXT
Auxiliary stack space = $O(n)$

~~$O=V$~~ Very important.

31. TIME COMPLEXITY — SIMPLE RECURSION

Consider:

```
C
void f(int n)
{
    if (n == 0)
        return;

    f(n - 1);
}
```

Number of calls:

TEXT

$n+1$ approximately

Each call does constant extra work.

So:

TEXT

Time = $O(n)$

Stack depth:

TEXT

$O(n)$

Therefore:

TEXT

Auxiliary Space = $O(n)$

32. FORMULA BOX

For:

TEXT

$T(n) = T(n-1) + O(1)$

we get:

TEXT

$T(n) = O(n)$

We'll formally study recurrence relations later.

For now, just understand:

one recursive call for every decrement from n to 0 !' linear time.

33. SOLVED EXAMPLE — SUM

```
c
int sum(int n)
{
    if (n == 0)
        return 0;

    return n + sum(n - 1);
}
```

What is:

```
c
sum(3);
```

?

34. DOWNWARD CALLS

TEXT

```
sum(3)
= 3 + sum(2)
```

```
sum(2)
= 2 + sum(1)
```

```
sum(1)
= 1 + sum(0)
```

```
sum(0)
= 0
```

35. UPWARD RETURNS

Now substitute upward:

TEXT

```
sum(0) = 0
```

```
sum(1)
= 1 + 0
= 1
```

```
sum(2)
= 2 + 1
= 3
```

```
sum(3)
= 3 + 3
= 6
```

Answer:

TEXT

```
6
```

0-716 This is the exact method for tracing recursive return values.

36. STORY FOR RETURN VALUES

Think of each call saying:

I know part of the answer, but I need the smaller problem first.

Example:

TEXT

```
sum(3):  
I know answer = 3 + something.  
Let sum(2) tell me the something.
```

Then `sum(2)` waits for `sum(1)`.

37. FACTORIAL PREVIEW

Definition:

TEXT

$$n! = n \times (n-1) \times \dots \times 1$$

Recursive relation:

TEXT

$$n! = n \times (n-1)!$$

Base:

TEXT

$$0! = 1$$

So:

C

```
int fact(int n)  
{  
    if (n == 0)  
        return 1;  
  
    return n * fact(n - 1);  
}
```

38. TRACE `fact(4)`

Downward:

TEXT

```
fact(4)  
= 4 × fact(3)
```

```
fact(3)  
= 3 × fact(2)
```

```
fact(2)
```



```
= 2 × fact(1)

fact(1)
= 1 × fact(0)

fact(0)
= 1
```

Upward:

```
TEXT

fact(1) = 1
fact(2) = 2
fact(3) = 6
fact(4) = 24
```

Answer:

```
TEXT

24
```

39. TIME & SPACE OF FACTORIAL

For:

```
C

fact(n)
```

there are roughly:

```
TEXT

n+1 calls
```

So:

```
TEXT

Time = O(n)
```

Maximum stack depth:

```
TEXT

O(n)
```

So:

```
TEXT

Auxiliary Space = O(n)
```

40. WHY TIME IS NOT O(1)

Some beginners see one multiplication:

```
C
return n * fact(n - 1);
```

and think:

```
TEXT
0(1)
```

But the recursive call itself triggers many more calls.

You must count the whole recursion chain.

$O = 1/n$ important.

41. GATE OUTPUT EXAMPLE

```
C
void f(int n)
{
    if (n <= 0)
        return;

    printf("%d ", n);
    f(n - 2);
}
```

Call:

```
C
f(7);
```

Trace:

```
TEXT
7
5
3
1
- 1 !' s t o p
```

Output:

```
TEXT
7 5 3 1
```

Calls:

```
TEXT
approximately n/2
```

Asymptotically:

$O(n)$

because constant factor $1/2$ is ignored.

42. ANOTHER GATE EXAMPLE

```
c
void f(int n)
{
    if (n <= 0)
        return;

    f(n - 2);
    printf("%d ", n);
}
```

Call:

c

f(7);

Down:

TEXT

7 !' 5 !' 3 !' 1 !' - 1

Prints during return:

TEXT

1 3 5 7

Δ gain: before call = downward, after call = upward.

43. RECURSION WITH $n / 2$

Consider:

```
C
void f(int n)
{
    if (n <= 1)
        return;

    f(n / 2);
}
```

Values:

TEXT

```
n
n/2
n/4
n/8
...
1
```

Number of calls:

TEXT

$O(\log n)$

So:

TEXT

```
Time =  $O(\log n)$ 
Stack Space =  $O(\log n)$ 
```

~~O = Very~~ important connection to logarithmic loops.

44. EXAMPLE

For:

TEXT

$n = 16$

calls:

TEXT

```
f(16)
f(8)
f(4)
f(2)
f(1)
```

About:

TEXT

```
log2 16 + 1
= 5 calls
```

Asymptotically:

TEXT

$O(\log n)$

45. DON'T ASSUME ALL RECURSION IS $O(n)$

Recursive complexity depends on:

- number of recursive calls
- how input size changes
- work inside each call

Examples:

TEXT

```
f ( n - 1 ) !'   o f t e n   0 ( n )

f ( n / 2 ) !'   o f t e n   0 ( l o g   n )

t w o   c a l l s   f ( n - 1 ) !'   m a y   e x p l o d e   e x p o n e n t i a l l y
```

We'll study recursion complexity separately.

46. TWO RECURSIVE CALLS — PREVIEW

Consider:

C

```
void f(int n)
{
    if (n == 0)
        return;

    f(n - 1);
    f(n - 1);
}
```

Each call creates:

TEXT

2 more calls

This creates a recursion tree.

For example:

TEXT

```
      f(3)
     /  \
    f(2)  f(2)
   / \  / \
  ...
```

This is very different from one recursive call.

We'll study it deeply later.

47. DIRECT RECURSION

Function calls itself directly:

```
c
void f(void)
{
    f();
}
```

This is:

Direct Recursion

48. INDIRECT RECURSION — PREVIEW

Suppose:

```
c
void A(void)
{
    B();
}

void B(void)
{
    A();
}
```

A does not directly call itself.

But:

```
TEXT
A  !'  B  !'  A
```

This is:

Indirect Recursion

We'll cover recursion types later.

49. BASE CASE DESIGN

A good base case should be:

- simple
- guaranteed reachable
- solve the smallest problem directly

Examples:

Factorial:

```
c
if (n == 0)
    return 1;
```

Sum:

```
c
if (n == 0)
    return 0;
```

Array traversal:

```
c
if (index == n)
    return;
```

Binary search:

```
c
if (low > high)
    return NOT_FOUND;
```

50. RECURSIVE CASE DESIGN

Ask:

Can I express this problem in terms of a smaller version of itself?

Examples:

Factorial

```
TEXT
fact(n)
=
n × fact(n-1)
```

Sum

```
TEXT
sum(n)
=
n + sum(n-1)
```

Power

```
TEXT
a^n
=
a × a^(n-1)
```

This is the recursive relation.

51. GATE TRAP — BASE RETURN VALUE

Consider factorial:

```
C
int fact(int n)
{
    if (n == 0)
        return 0;

    return n * fact(n - 1);
}
```

This is wrong.

Why?

Eventually:

```
TEXT
fact(0)=0
```

Then:

```
TEXT
fact(1)=1×0=0
fact(2)=2×0=0
...
```

Everything becomes zero.

Correct:

```
C
return 1;
```

because:

```
TEXT
0! = 1
```

Base return value affects whole recursion.

52. SUM BASE VALUE

For:

```
TEXT
sum(n)=n+sum(n-1)
```

base should be:

```
C
sum(0) = 0;
```

because zero is additive identity.

Memory trick:

TEXT

```
A d d i t i o n   b a s e   ! '   0
M u l t i p l i c a t i o n   b a s e   ! '   1
```

Useful in many recursive definitions.

53. RECURSIVE PRINTING — GATE MASTER PATTERNS

Pattern A

C

```
print(n);
f(n-1);
```

Output:

TEXT

```
n ... 1
```

Pattern B

C

```
f(n-1);
print(n);
```

Output:

TEXT

```
1 ... n
```

Pattern C

C

```
print(n);
f(n-1);
print(n);
```

Output:

TEXT

```
n ... 1 1 ... n
```

Copy this.

54. GATE TRAP — n - - IN RECURSIVE ARGUMENT

Consider:

```
c
f(n--);
```

This can be very tricky.

Post-decrement passes the **old value** of `n`, then decrements the current invocation's local `n`.

So the recursive call may receive the same value instead of a smaller value.

Example:

```
c
void f(int n)
{
    if (n == 0)
        return;

    f(n--);
}
```

For positive `n`, the recursive call receives the old `n` again.

That can lead to non-terminating recursion.

Very important C + recursion trap.

55. COMPARE `n-1`, `--n`, `n--`

Safe/common:

```
c
f(n - 1);
```

passes smaller value without altering current `n`.

Prefix:

```
c
f(--n);
```

decrements current `n`, then passes decreased value.

Postfix:

```
c
f(n--);
```

passes old value first.

Potentially disastrous for recursion termination.

56. EXAMPLE

```
c
void f(int n)
{
    if (n == 0)
        return;

    printf("%d ", n);
    f(n - 1);
    printf("%d ", n);
}
```

Call:

```
c
f(3);
```

Remember each invocation has separate `n`.

So even after `f(2)` returns, outer:

```
TEXT
f(3)
```

still has:

```
TEXT
n = 3
```

Final:

```
TEXT
3 2 1 1 2 3
```

57. WHY OUTER `n` DOESN'T BECOME 2

Because:

```
c
f(n - 1);
```

computes value:

```
TEXT
2
```

and passes that value into a new invocation.

It does not modify the outer `n`.

Same call-by-value principle.

58. STACK SPACE "TOTAL CALLS"

Important distinction.

Suppose a recursive algorithm creates many calls overall.

Space complexity depends on:

maximum number of simultaneously active calls.

Not total calls.

Example recursion trees may have exponentially many total calls but only $O(n)$ depth.

$O(1)$ critical for later Fibonacci analysis.

59. RECURSION DEPTH

The number of active recursive levels at the deepest point is called:

Recursion Depth

Example:

TEXT

```
f(4)
f(3)
f(2)
f(1)
f(0)
```

Depth roughly:

TEXT

5

Asymptotically:

TEXT

$O(n)$

60. STACK OVERFLOW

If recursion depth becomes too large:

TEXT

```
too many active frames
!"
stack exhausted
!"
stack overflow
```

Reasons:

- missing base case
- base case never reached
- extremely large valid recursion depth

61. PLACEMENT CONNECTION

Recursion is fundamental for:

- binary search
- tree traversals
- DFS
- merge sort
- quicksort
- backtracking
- permutations
- subsets
- divide-and-conquer
- dynamic programming foundations

So recursion is not just a GATE theory topic.

62. WHEN RECURSION IS NATURAL

Recursion is especially natural when the structure itself is recursive.

Example tree:

TEXT

```
      Node
     /   \
  Tree   Tree
```

A binary tree is naturally:

one node + left subtree + right subtree.

So recursive algorithms fit trees extremely well.

63. RECURSION VS ITERATION

Neither is universally "better."

Recursion

Pros:

- elegant
- natural for recursive structures
- easier for divide-and-conquer/backtracking/tree traversal

Cons:

- call overhead
- stack usage
- stack overflow risk

Iteration

Pros:

- often less auxiliary space
- avoids recursive call overhead

Cons:

- some algorithms become harder to express manually
-

64. GATE FOCUS

Ø= MUST KNOW:

- base case
 - recursive case
 - call stack
 - separate activation records
 - downward vs upward execution
 - output before/after recursive call
 - recursion depth
 - stack space
 - termination condition
 - `n--` trap
 - simple recurrence intuition
-

65. GATE TRAP SHEET

Trap 1

Base case missing.

!' infinite recursion.

Trap 2

Base exists but recursive step moves away from it.

!' still infinite recursion.

Trap 3

Code after recursive call executes on return journey.

Trap 4

Every invocation has separate local variables/parameters.

Trap 5

```
c
f(n--);
```

may pass same old `n`.

Trap 6

Recursive time and stack space are not always the same.

Trap 7

Space depends on maximum active recursion depth, not total number of calls.

66. INTERVIEW QUESTIONS

Q1. What is recursion?

A technique where a function solves a problem by invoking itself on smaller subproblems.

Q2. What is a base case?

A condition that stops further recursive calls.

Q3. Why is a base case required?

Without a reachable stopping condition, recursive calls continue until stack failure.

Q4. Where are recursive calls tracked?

Typically through the runtime call stack.

Q5. Time complexity of `f(n-1)` with constant work per call?

`O(n)`.

Q6. Auxiliary space complexity?

`O(n)` due to recursion depth.

Q7. Why can recursion cause stack overflow?

Too many simultaneously active function calls consume stack space.

67. MEMORY TRICKS

Recursion

Solve smaller copy of same problem.

Base case

Emergency brake.

Call stack

Go DOWN, come UP.

Code placement

TEXT

```
Before recursive call !' DOWN
After recursive call !' UP
```

68. NOTEBOOK NOTES

Basic Recursive Form

```
C
function(n)
{
    if (base_condition)
        return base_value;

    return something + function(smaller_problem);
}
```

Example

```
C
int sum(int n)
{
    if (n == 0)
        return 0;

    return n + sum(n - 1);
}
```

Trace

TEXT

```
sum(3)
!' 3 + sum(2)
!' 3 + 2 + sum(1)
!' 3 + 2 + 1 + sum(0)
```


Then returns upward.

Complexity

For one call reducing by 1:

TEXT

```
Time  = O(n)
Space = O(n)
```

For one call reducing by half:

TEXT

```
Time  = O(log n)
Space = O(log n)
```

69. 30-SECOND REVISION

Remember:

1. Recursion = function calling itself.
2. Base case stops recursion.
3. Recursive case moves toward base.
4. Every call has its own parameters and locals.
5. Calls go downward.
6. Returns come upward.
7. Code before recursive call runs downward.
8. Code after recursive call runs upward.
9. `f(n-1)` usually gives linear depth.
10. Recursion uses call-stack space.

70. ACTIVE RECALL

Try first.

Q1

What are the two essential parts of recursion?

Q2

What is output?

```
c
void f(int n)
{
    if (n == 0)
        return;

    printf("%d ", n);
}
```

```
f(n - 1);  
}  
  
f(3);
```

Q3

What if `printf` is placed after the recursive call?

Q4

Why does recursion use extra memory?

Q5

What is wrong with:

```
c  
f(n--);
```

when you're trying to recursively reduce `n`?

ANSWERS

A1

Base case and recursive case.

A2

```
TEXT  
3 2 1
```

A3

For `f(3)`:

```
TEXT  
1 2 3
```

A4

Each active invocation needs its own call/activation record on the stack.

A5

Post-decrement passes the old value, so the recursive call may receive the same `n`, preventing progress.

71. PRACTICE

Q1

Find output:

```

c
void f(int n)
{
    if (n == 0)
        return;

    printf("%d ", n);
    f(n - 1);
}

f(5);

```

Q2

Find output:

```

c
void f(int n)
{
    if (n == 0)
        return;

    f(n - 1);
    printf("%d ", n);
}

f(4);

```

Q3

Find output:

```

c
void f(int n)
{
    if (n == 0)
        return;

    printf("%d ", n);
    f(n - 1);
    printf("%d ", n);
}

f(3);

```

Q4 — RETURN VALUE

Evaluate:

```

c
int sum(int n)
{
    if (n == 0)
        return 0;
}

```

```
    return n + sum(n - 1);  
}
```

Find:

TEXT
sum(5)

~~Q5~~ — FACTORIAL

Trace:

```
C  
int fact(int n)  
{  
    if (n == 0)  
        return 1;  
  
    return n * fact(n - 1);  
}
```

Find:

TEXT
fact(5)

~~Q6~~ — COMPLEXITY

Find time and auxiliary-space complexity:

```
C  
void f(int n)  
{  
    if (n <= 0)  
        return;  
  
    f(n - 1);  
}
```

~~Q7~~ — LOG RECURSION

Find time and stack-space complexity:

```
C  
void f(int n)  
{  
    if (n <= 1)  
        return;  
  
    f(n / 2);  
}
```

```
}
```

Q8 — TERMINATION TRAP

What happens?

```
c
void f(int n)
{
    if (n == 0)
        return;

    f(n + 1);
}
```

Call:

```
c
f(5);
```

Explain why the presence of a base case does not save it.

Q9 — POST-DECREMENT TRAP

What is problematic here?

```
c
void f(int n)
{
    if (n == 0)
        return;

    f(n--);
}
```

Assume positive `n`.

Q10 — PLACEMENT THINKING

Write a recursive function that prints:

```
TEXT
1 2 3 ... n
```

using the idea:

```
TEXT
recursive call first
printing after return
```

State time and auxiliary-space complexity.

PROGRESS

C Foundation '

Completed Lessons 1–16.

Recursion

' Lesson 17 — Fundamentals

Next

MODULE 1 / LESSON 18 — Recursion Tracing Masterclass

Next we will focus almost entirely on **GATE-style tracing**, including:

```
C
f(n - 1);
printf(...);
f(n - 1);
```

and questions such as:

```
C
void f(int n)
{
    if (n == 0)
        return;

    printf("%d ", n);
    f(n - 1);
    printf("%d ", n);
}
```

We will learn:

- recursion tree drawing
- exact call order
- exact output order
- return order
- number of calls
- number of prints
- nested recursive calls
- two recursive calls
- tricky `++n`, `n++`, `--n`, `n--`
- local/static variables inside recursion
- GATE output-question solving method

Type **NEXT**.

ASSISTANT

MODULE 1 — LESSON 18

Recursion Tracing Masterclass — Call Order, Return Order, Trees & GATE Output Questions

We now focus on the part of recursion that causes the most mistakes in GATE:

Tracing exactly what happens

The main goal today is to make you able to look at recursive C code and systematically determine:

- exact function-call order
- exact return order
- output
- number of calls
- number of prints
- recursion depth
- tricky `n++`, `++n`, `n--`, `--n`
- static variables inside recursion
- two-recursive-call trees

GATE importance: Ø=Ý%ØÁÚ\$ØÊÍØW **Placement importance:** Ø=Ý%ØÁÚ\$ØÊÍØW

1. THE GOLDEN RULE OF RECURSION TRACING

Whenever you see:

```
c
void f(int n)
{
    ...
    f(smaller_problem);
    ...
}
```

do **not** try to solve everything mentally.

Use this method:

```
TEXT
STEP 1 !' Write current call
STEP 2 !' Execute everything before recursive call
STEP 3 !' Go DOWN into recursive call
STEP 4 !' Reach base case
```

```
STEP 5 !' Come UP
STEP 6 !' Execute statements after recursive call
```

Memory trick:

DOWN first !' UP later

2. BASIC PATTERN1 — PRINT BEFORE CALL

Consider:

```
C
void f(int n)
{
    if (n == 0)
        return;

    printf("%d ", n);

    f(n - 1);
}
```

Call:

```
C
f(3);
```

Trace:

```
TEXT
f(3)
print 3
!"
f(2)
print 2
!"
f(1)
print 1
!"
f(0)
return
```

Output:

```
TEXT
3 2 1
```

3. BASIC PATTERN2 — PRINT AFTER CALL


```

c
void f(int n)
{
    if (n == 0)
        return;

    f(n - 1);

    printf("%d ", n);
}

```

Call:

```

c
f(3);

```

Downward phase:

```

TEXT
f(3)
!"
f(2)
!"
f(1)
!"
f(0)

```

No printing yet.

Return phase:

```

TEXT
f(0) returns

f ( 1 ) !' p r i n t   1
!'
f ( 2 ) !' p r i n t   2
!'
f ( 3 ) !' p r i n t   3

```

Output:

```

TEXT
1 2 3

```

4. MASTER MEMORY TRICK

```

TEXT
PRINT BEFORE recursive call
!' d e s c e n d i n g   o r d e r

PRINT AFTER recursive call
!' a s c e n d i n g   o r d e r

```

for the standard `n-1` pattern.

5. BASIC PATTERN3 — PRINT BOTH SIDES

```
c
void f(int n)
{
    if (n == 0)
        return;

    printf("%d ", n);

    f(n - 1);

    printf("%d ", n);
}
```

Call:

```
c
f(3);
```

Down:

```
TEXT
3 2 1
```

Up:

```
TEXT
1 2 3
```

Final:

```
TEXT
3 2 1 1 2 3
```

0-6% Classic GATE pattern.

6. DRAW IT AS A FRAME

For `f(3)`:

```
TEXT
f(3):
    print 3
    wait for f(2)
    print 3
```

For `f(2)`:

TEXT

```
f(2):
  print 2
  wait for f(1)
  print 2
```

For `f(1)`:

TEXT

```
f(1):
  print 1
  wait for f(0)
  print 1
```

This makes output obvious:

TEXT

```
3 2 1 1 2 3
```

7. VERY IMPORTANT — EVERY CALL HAS ITS OWN `n`

Consider:

C

```
void f(int n)
{
    if (n == 0)
        return;

    f(n - 1);

    printf("%d ", n);
}
```

For:

C

```
f(3);
```

you have separate invocation states:

TEXT

```
f(3): n = 3
f(2): n = 2
f(1): n = 1
f(0): n = 0
```

When `f(1)` finishes, outer `f(2)` still remembers:

TEXT

```
n = 2
```

because its activation record remained on the stack.

0=1% is the reason return-phase output works.

8. STACK VISUALIZATION

At deepest point:

TEXT

TOP

```
+-----+
| f(0): n = 0 |
+-----+
| f(1): n = 1 |
+-----+
| f(2): n = 2 |
+-----+
| f(3): n = 3 |
+-----+
```

BOTTOM

Then frames are removed:

TEXT

```
f(0) returns
f(1) resumes
f(1) returns
f(2) resumes
...
```

9. EXACT NUMBER OF CALLS

Consider:

```
c
void f(int n)
{
    if (n == 0)
        return;

    f(n - 1);
}
```

Call:

```
c
f(5);
```

Calls are:

TEXT

```
f(5)
f(4)
f(3)
```

```
f(2)
f(1)
f(0)
```

Total calls:

```
TEXT
6
```

General:

```
TEXT
n + 1 calls
```

if base call `f(0)` is included.

~~0-6%~~ NOTE may distinguish:

- number of recursive calls
- total function invocations

10. FUNCTION INVOCATIONS VS RECURSIVE CALL STATEMENTS

For `f(5)`:

Total invocations:

```
TEXT
6
```

But how many times does the recursive statement:

```
c
f(n - 1);
```

execute?

For:

```
TEXT
n = 5, 4, 3, 2, 1
```

So:

```
TEXT
5 times
```

Therefore:

```
TEXT
Total invocations = n + 1
Recursive-call executions = n
```

for this pattern.

~~0-6%~~ Good exact-count distinction.

11. NUMBER OF PRINTS

Example:

```
c
void f(int n)
{
    if (n == 0)
        return;

    printf("*");

    f(n - 1);
}
```

For `f(n)`:

```
TEXT
prints = n
```

because base call prints nothing.

12. TWO PRINTS PER NON-BASE CALL

```
c
void f(int n)
{
    if (n == 0)
        return;

    printf("*");

    f(n - 1);

    printf("*");
}
```

Each non-base call prints twice.

Number of non-base calls:

```
TEXT
n
```

So total prints:

```
TEXT
2n
```

Time:

TEXT

$O(n)$

13. CHANGE IN STEP SIZE

```
c
void f(int n)
{
    if (n <= 0)
        return;

    printf("%d ", n);

    f(n - 2);
}
```

Call:

```
c
f(8);
```

Values:

```
TEXT
8
6
4
2
0 !' s t o p
```

Output:

```
TEXT
8 6 4 2
```

Number of printing calls:

```
TEXT
4
```

Approximately:

```
TEXT
n/2
```

Asymptotically:

```
TEXT
 $O(n)$ 
```

14. ODD INPUT

Call:

```
c
f(7);
```

Trace:

```
TEXT
7
5
3
1
- 1 !' s t o p
```

Output:

```
TEXT
7 5 3 1
```

The base condition:

```
c
if (n <= 0)
```

handles both zero and negative values.

15. WHY `n == 0` COULD FAIL HERE

Suppose:

```
c
void f(int n)
{
    if (n == 0)
        return;

    f(n - 2);
}
```

Call:

```
c
f(5);
```

Values:

```
TEXT
5
3
1
-1
```



```
-3
...
```

It never hits:

```
TEXT
0
```

So recursion does not terminate.

~~0=Very~~ Very important base-case design trap.

Better:

```
C
if (n <= 0)
    return;
```

16. GATE TRACING RULE

Before solving recursion, check:

Will the sequence actually hit the base condition?

Do not blindly assume recursion terminates.

17. `n - 1` VS `--n`

Consider:

```
C
void f(int n)
{
    if (n == 0)
        return;

    f(--n);
}
```

For:

```
TEXT
n = 3
```

Prefix decrement:

```
TEXT
--n
```

first changes current call's `n`:

TEXT

3 !' 2

then passes:

TEXT

2

So calls:

TEXT

f(3)
f(2)
f(1)
f(0)

It terminates.

18. BUT CURRENT FRAME ALSO CHANGED

This matters if code executes afterward.

Example:

```
c
void f(int n)
{
    if (n == 0)
        return;

    f(--n);

    printf("%d ", n);
}
```

Call:

```
c
f(3);
```

Trace carefully.

19. TRACE f(--n)

Outer f(3):

TEXT

n = 3
- - n !' 2
calls f(2)

outer frame now stores n = 2

Inside `f(2)`:

TEXT

```
n = 2
- - n !' 1
calls f(1)

this frame now stores n = 1
```

Inside `f(1)`:

TEXT

```
n = 1
- - n !' 0
calls f(0)

this frame now stores n = 0
```

Then return.

Print on upward journey:

TEXT

```
0 1 2
```

Not:

TEXT

```
1 2 3
```

0=Important.

20. COMPARE WITH `f(n - 1)`

C

```
void f(int n)
{
    if (n == 0)
        return;

    f(n - 1);

    printf("%d ", n);
}
```

Call `f(3)` prints:

TEXT

```
1 2 3
```

because each outer frame's `n` remains unchanged.

DON'T CONFUSE

```
c
f(n - 1);
```

passes smaller value but doesn't modify current `n`.

```
c
f(--n);
```

modifies current `n` and passes new value.

~~0=Very~~ Very different when current `n` is later used.

21. THE `n--` DISASTER

Consider:

```
c
void f(int n)
{
    if (n == 0)
        return;

    f(n--);
}
```

Suppose current:

```
TEXT
n = 3
```

Post-decrement:

```
TEXT
n--
```

produces old value:

```
TEXT
3
```

for the function argument.

So recursive call becomes:

```
c
f(3);
```

Again.

The current frame's local `n` becomes 2, but it is suspended while the new `f(3)` executes.

The new call again starts with:

$$n = 3$$
$$\begin{array}{l} f(3) \\ !' \quad f(3) \\ !' \quad f(3) \\ !' \quad . \end{array}$$
$$\emptyset = \dot{Y} \% \emptyset = \dot{y} \% \emptyset = \dot{u} \% \emptyset \text{ trap.}$$

```
n - 1 !' pass smaller copy
- - n !' change first, pass smaller
n - - !' pass old value, change later locally
```

n - - can be dangerous because the child may receive the unchanged old value.

```
if (n == 0)
```

```
f(++n);
```

$$3 \quad !' \quad 4 \quad !' \quad 5 \quad !' \quad . \quad . \quad .$$

Likewise:

```
c
f(n++);
```

passes old value and can recursively call the same `n`.

~~0 = 10~~ Always inspect progress toward base.

24. RETURN VALUE TRACE

Consider:

```
c
int f(int n)
{
    if (n == 0)
        return 0;

    return n + f(n - 1);
}
```

Find:

```
c
f(4)
```

Write expansion:

```
TEXT
f(4)
= 4 + f(3)
= 4 + 3 + f(2)
= 4 + 3 + 2 + f(1)
= 4 + 3 + 2 + 1 + f(0)
= 4 + 3 + 2 + 1 + 0
= 10
```

25. RETURN-TABLE METHOD

Another method:

Call	Return
f(0)	0
f(1)	1 + 0 = 1
f(2)	2 + 1 = 3
f(3)	3 + 3 = 6
f(4)	4 + 6 = 10

This method is excellent for GATE numericals.

26. ANOTHER RETURN FUNCTION

```
c
int f(int n)
{
    if (n == 0)
        return 1;

    return 2 * f(n - 1);
}
```

Find:

```
c
f(4)
```

Trace:

```
TEXT
f(0) = 1

f(1) = 2 × 1 = 2
f(2) = 2 × 2 = 4
f(3) = 2 × 4 = 8
f(4) = 2 × 8 = 16
```

So:

```
TEXT
f(n) = 2^n
```

for nonnegative `n`.

27. IDENTIFYING THE MATHEMATICAL PATTERN

Recursive functions often encode mathematics.

Example:

```
c
return n + f(n-1);
```

!' sum.

```
c
return n * f(n-1);
```

!' factorial.

```
c
return 2 * f(n-1);
```

!' powers of 2.

GATE may ask final returned value rather than calling it by a familiar name.

28. TWO RECURSIVE CALLS

Now:

```
c
void f(int n)
{
    if (n == 0)
        return;

    f(n - 1);
    f(n - 1);
}
```

This creates a:

Recursion Tree

For:

```
c
f(2);
```

tree:

TEXT

```
      f(2)
     /  \
    f(1)  f(1)
   /  \  /  \
  f(0) f(0) f(0) f(0)
```

29. EXACT NUMBER OF CALLS

For `f(2)`:

TEXT

```
Level 0: 1 call
Level 1: 2 calls
Level 2: 4 calls
```

Total:

TEXT

$$1 + 2 + 4 = 7$$

For general n :

TEXT

$$1 + 2 + 4 + \dots + 2^n$$

Formula:

TEXT

$$2^{(n+1)} - 1$$

So:

TEXT

$$\text{Time} = O(2^n)$$

~~O=Very~~ Very important.

FORMULA BOX

TEXT

$$\begin{aligned} &1 + 2 + 4 + \dots + 2^n \\ &= 2^{(n+1)} - 1 \\ &= O(2^n) \end{aligned}$$

30. BUT WHAT IS STACK SPACE?

Even though total calls are exponential:

TEXT

$$O(2^n)$$

only one path is active deeply at a time.

Maximum path:

TEXT

$$\begin{aligned} &f(n) \\ &!' \quad f(n - 1) \\ &!' \quad \dots \\ &!' \quad f(0) \end{aligned}$$

Depth:

TEXT

$$O(n)$$

Therefore:

TEXT

Time = $O(2^n)$
Space = $O(n)$

$O(2^n)$ of this estimation is essential.

31. TOTAL CALLS "STACK DEPTH"

Think:

TEXT

Recursion TREE size
! ' t i m e

while:

TEXT

Longest root-to-leaf path
! ' r e c u r s i v e s t a c k d e p t h

Very important.

32. TWO CALLS WITH PRINT BETWEEN THEM

Consider:

```
c
void f(int n)
{
    if (n == 0)
        return;

    f(n - 1);

    printf("%d ", n);

    f(n - 1);
}
```

Call:

```
c
f(2);
```

Trace carefully.

33. TRACE `f(2)`

We have:

TEXT

```
f(2)
% % %   f ( 1 )
%       % % %   f ( 0 )
%       % % %   p r i n t   1
%       % % %   f ( 0 )
%
% % %   p r i n t   2
%
% % %   f ( 1 )
%       % % %   f ( 0 )
%       % % %   p r i n t   1
%       % % %   f ( 0 )
```

Output:

TEXT

```
1 2 1
```

0-Recursion-tree tracing.

34. FOR f(3)

Same pattern:

TEXT

```
f(3)
  f(2)
    f(1)
    print 2
    f(1)
  print 3
  f(2)
    f(1)
    print 2
    f(1)
```

Output:

TEXT

```
1 2 1 3 1 2 1
```

Notice symmetry.

35. NUMBER OF PRINTS IN THIS FUNCTION

C

```
void f(int n)
{
    if (n == 0)
        return;

    f(n - 1);
    printf("*");
}
```

```
f(n - 1);  
}
```

Let number of prints be $P(n)$.

Then:

```
TEXT  
P(n)  
=  
P(n-1) + 1 + P(n-1)
```

So:

```
TEXT  
P(n)  
=  
2P(n-1) + 1
```

with:

```
TEXT  
P(0)=0
```

Result:

```
TEXT  
P(n)=2^n - 1
```

Useful exact counting.

36. EXAMPLE

For:

```
TEXT  
n=3
```

prints:

```
TEXT  
2^3 - 1  
= 7
```

Indeed output contains 7 numbers:

```
TEXT  
1 2 1 3 1 2 1
```

37. DIFFERENT BRANCH SIZES

Consider:

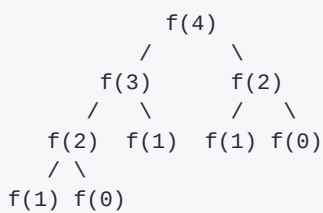
```
c
int f(int n)
{
    if (n <= 1)
        return 1;

    return f(n - 1) + f(n - 2);
}
```

This is Fibonacci-like recursion.

Tree for `f(4)`:

TEXT



Many repeated calculations appear.

38. IMPORTANT PLACEMENT INSIGHT

Notice:

TEXT

`f(2)`

may be calculated multiple times.

That is why naive Fibonacci recursion is inefficient.

Rough time:

TEXT

exponential

Often upper-bounded as:

TEXT

$O(2^n)$

More precisely it grows like:

TEXT

$O(\phi^n)$

where ϕ is the golden ratio, but $O(2^n)$ is a common simple upper bound.

Auxiliary stack:

```
TEXT
0(n)
```

After this becomes the motivation for dynamic programming.

39. STATIC VARIABLE INSIDE RECURSION

Now a major GATE-style concept.

Consider:

```
c
void f(int n)
{
    static int x = 0;

    if (n == 0)
        return;

    x++;

    printf("%d ", x);

    f(n - 1);
}
```

Call:

```
c
f(3);
```

Remember:

`x` is **shared across all recursive invocations** because it is static.

40. TRACE STATIC VARIABLE

Initially:

```
TEXT
x = 0
```

`f(3):`

```
TEXT
x = 1
print 1
```

`f(2):`

TEXT

```
same x
x = 2
print 2
```

f(1):

TEXT

```
same x
x = 3
print 3
```

Output:

TEXT

```
1 2 3
```

41. STATIC VS AUTOMATIC LOCAL

Compare:

C

```
void f(int n)
{
    int x = 0;

    if (n == 0)
        return;

    x++;

    printf("%d ", x);

    f(n - 1);
}
```

Every invocation gets its own fresh:

TEXT

```
x = 0
```

Then increments to 1.

Output:

TEXT

```
1 1 1
```

Ø=Important.

42. STATIC VARIABLE + RETURN PHASE

Consider:

```

c
void f(int n)
{
    static int x = 0;

    if (n == 0)
        return;

    x++;

    f(n - 1);

    printf("%d ", x);
}

```

Call:

```

c
f(3);

```

Trace downward:

```

TEXT
f(3): x=1
f(2): x=2
f(1): x=3
f(0): return

```

At this point shared static:

```

TEXT
x = 3

```

Now all returning calls print the same current static value:

```

TEXT
3 3 3

```

Excellent GATE trap.

43. WHY NOT 1 2 3 ?

Because unlike automatic local variables, there is only **one static `x`**.

All recursion frames share it.

When recursion reaches deepest point:

```

TEXT
x = 3

```

That value remains during all upward returns.

44. AUTOMATIC LOCAL + RETURN PHASE

Now:

```
C
void f(int n)
{
    int x = n;

    if (n == 0)
        return;

    f(n - 1);

    printf("%d ", x);
}
```

Each call has its own `x`.

For `f(3)`:

```
TEXT
f(3): x=3
f(2): x=2
f(1): x=1
```

Return prints:

```
TEXT
1 2 3
```

This contrasts strongly with static storage.

45. GLOBAL VARIABLE IN RECURSION

A global variable behaves similarly to static storage in the sense that recursive calls share the same global object.

Example:

```
C
int x = 0;

void f(int n)
{
    if (n == 0)
        return;

    x++;
    f(n - 1);
}
```

After:

```
C
f(5);
```

global:

TEXT

x = 5

assuming no other changes.

46. RECURSION WITH RETURN + SIDE EFFECT

Consider:

```
c
int f(int n)
{
    if (n == 0)
        return 0;

    printf("%d ", n);

    return f(n - 1);
}
```

Output for `f(3)`:

TEXT

3 2 1

Final returned value:

TEXT

0

Output and return value are separate questions.

47. ALWAYS TRACK TWO THINGS

For recursive questions containing both printing and return values, maintain:

TEXT

1. OUTPUT
2. RETURN VALUE

Do not mix them.

48. EXAMPLE

```
c
int f(int n)
{
    if (n == 0)
        return 1;
}
```

```
printf("%d ", n);

return n * f(n - 1);
}
```

Call:

```
c
int x = f(3);
```

Output:

```
TEXT
3 2 1
```

Return:

```
TEXT
3 × 2 × 1 × 1 = 6
```

So:

```
TEXT
x = 6
```

49. CALL COUNT WITH $n/2$

```
c
void f(int n)
{
    if (n <= 1)
        return;

    f(n / 2);
}
```

For $n=16$:

```
TEXT
f(16)
f(8)
f(4)
f(2)
f(1)
```

Total invocations:

```
TEXT
5
```

Depth:

TEXT

5

Asymptotically:

TEXT

$O(\log n)$

50. EXACT COUNT IDEA

For powers of 2:

TEXT

$n = 2^k$

Repeated division:

TEXT

2^k

$2^{(k-1)}$

...

2

1

Number of values:

TEXT

$k + 1$

Since:

TEXT

$k = \log_2 n$

calls:

TEXT

$\log_2 n + 1$

for that exact pattern.

51. GATE TRACING METHOD — MASTER TEMPLATE

When you get a recursion-output question, do this:

Step 1 — Identify base condition

TEXT

When exactly does recursion stop?

Step 2 — Identify recursive arguments

TEXT

n-1?
n-2?
n/2?
two recursive calls?

Step 3 — Check termination

Does it move toward base?

Step 4 — Mark statements

TEXT

Before recursive call = DOWN
After recursive call = UP

Step 5 — Draw call chain/tree

Don't mentally guess.

Step 6 — Track locals separately

Each ordinary local belongs to its own call.

Step 7 — Track static/global once

Shared across calls.

Step 8 — Separately compute output and returned value.

Use this every time.

52. GATE TRAP — MULTIPLE RECURSIVE CALLS

If you see:

C

```
f(n - 1);  
f(n - 1);
```

do not trace it as:

TEXT

n !' n - 1 !' n - 2

only once.

The second call starts another entire recursive subtree.

Think:

TEXT

```
      f(n)
     /  \
f(n-1)  f(n-1)
```

53. GATE TRAP — STATEMENTS BETWEEN CALLS

C

```
f(n - 1);
printf("%d", n);
f(n - 1);
```

The `printf` executes:

after the entire first subtree finishes, but before the second subtree starts.

This is why tree drawing matters.

54. SOLVED TRICKY EXAMPLE

C

```
void f(int n)
{
    if (n <= 0)
        return;

    printf("%d ", n);

    f(n - 2);

    printf("%d ", n);
}
```

Call:

C

```
f(5);
```

Down:

TEXT

```
5 3 1
```

Base:

TEXT

```
-1
```

Up:

TEXT

1 3 5

Output:

TEXT

5 3 1 1 3 5

55. SOLVED TWO-BRANCH EXAMPLE

```
c
void f(int n)
{
    if (n == 0)
        return;

    printf("%d ", n);

    f(n - 1);
    f(n - 1);
}
```

Call:

c

f(2);

Trace:

TEXT

f(2): print 2

first f(1):
 print 1
 f(0)
 f(0)

second f(1):
 print 1
 f(0)
 f(0)

Output:

TEXT

2 1 1

56. HOW MANY PRINTS?

For:

```

c
void f(int n)
{
    if (n == 0)
        return;

    printf("*");
    f(n-1);
    f(n-1);
}

```

Number of non-base calls:

TEXT
 $2^n - 1$

So number of stars:

TEXT
 $2^n - 1$

57. NUMBER OF LEAF CALLS

For full binary recursion:

```

c
f(n-1);
f(n-1);

```

with base at $n=0$:

Number of base calls:

TEXT
 2^n

Example $n=3$:

TEXT
 8 base calls

Internal non-base calls:

TEXT
 $2^n - 1 = 7$

Total:

TEXT
 $2^{(n+1)} - 1 = 15$

Useful exact formula.

FORMULA BOX — FULL BINARY RECURSION

TEXT

Base calls = 2^n

Non-base calls = $2^n - 1$

Total calls = $2^{(n+1)} - 1$

for:

C

```
if (n == 0) return;  
f(n-1);  
f(n-1);
```

58. RECURSION TREE COMPLEXITY

A recursion tree has:

TEXT

```
L e v e l    0   !'   1  
L e v e l    1   !'   2  
L e v e l    2   !'   4  
...  
L e v e l    n   !'    $2^n$ 
```

Total nodes:

TEXT

$O(2^n)$

Height:

TEXT

$O(n)$

Thus:

TEXT

Time = $O(2^n)$
Space = $O(n)$

59. DON'T CONFUSE TREE HEIGHT WITH NUMBER OF NODES

Example binary tree:

TEXT



Nodes may be many.

Height may be much smaller.

Same idea in recursion trees.

60. PLACEMENT CONNECTION

This reasoning appears later in:

- naive Fibonacci
- backtracking
- subset generation
- permutation generation
- tree traversal
- divide-and-conquer
- recursion-based search

If you can draw recursion trees, many placement complexity questions become easier.

61. INTERVIEW QUESTIONS

Q1. How do you trace recursive code?

Follow calls downward until base case, then resume suspended calls upward.

Q2. Where do statements after recursive calls execute?

During stack unwinding / return phase.

Q3. Why does each recursive call remember its own `n`?

Each invocation has its own activation record.

Q4. Why can two recursive calls cause exponential time?

Each call branches into multiple subcalls, creating a growing recursion tree.

Q5. Does exponential number of calls imply exponential stack space?

No. Stack space depends on maximum active depth.

Q6. Why is `f(n--)` dangerous?

Post-decrement passes the old value, so recursive input may not decrease.

62. MEMORY TRICKS

One recursive call

TEXT
CHAIN

Example:

TEXT
f(4)
!"
f(3)
!"
f(2)

Multiple recursive calls

TEXT
TREE

Example:

TEXT
f(3)
/ \
f(2) f(2)

Locals

One copy per frame

Static/global

One shared copy

63. NOTEBOOK NOTES

Recursion tracing

TEXT
C a l l s !"
Base
R e t u r n s !'

Before recursive call

Runs downward.

After recursive call

Runs upward.

One-call recurrence

TEXT

```
T(n)=T(n-1)+O(1)
! ' O ( n )
```

Divide-by-two recurrence

TEXT

```
T(n)=T(n/2)+O(1)
! ' O ( l o g n )
```

Two identical recursive calls

TEXT

```
T(n)=2T(n-1)+O(1)
! ' O ( 2 ^ n )
```

Stack space still:

TEXT

```
O(n)
```

64. GATE TRAP SHEET

Remember:

1. Base condition may never be reached.
2. `n--` passes old value.
3. `--n` modifies current frame too.
4. Code after recursion waits.
5. Every ordinary local is per-call.
6. Static/global is shared.
7. Two recursive calls create separate subtrees.
8. Total calls "`` maximum active calls.
9. Output and return value must be tracked separately.
10. Exact call counts may include base invocations.

65. 30-SECOND REVISION

For any recursive question:

TEXT

1. Find base
2. Check progress

3. Draw calls
4. Mark DOWN statements
5. Reach base
6. Mark UP statements
7. Track locals separately
8. Track static/global once
9. Draw tree if >1 recursive call
10. Compute time from total calls, space from depth

66. ACTIVE RECALL

Try first.

Q1

Find output:

```
c
void f(int n)
{
    if (n == 0)
        return;

    f(n - 1);
    printf("%d ", n);
}

f(3);
```

Q2

Find output:

```
c
void f(int n)
{
    if (n == 0)
        return;

    printf("%d ", n);
    f(n - 1);
    printf("%d ", n);
}

f(2);
```

Q3

For:

```
c
f(n - 1);
f(n - 1);
```

what broad structure should you draw?

Q4

If total calls are $O(2^n)$ but recursion depth is n , what is stack space?

Q5

Why can:

```
c
f(n--);
```

fail to terminate?

ANSWERS

A1

```
TEXT
1 2 3
```

A2

```
TEXT
2 1 1 2
```

A3

A binary recursion tree.

A4

```
TEXT
O(n)
```

A5

Because postfix decrement passes the old n to the child call, so the child can receive the same input.

67. PRACTICE

Q1

Find output:

```
c
void f(int n)
{
    if (n <= 0)
        return;

    printf("%d ", n);
    f(n - 2);
}
```

```
f(6);
```

~~Q2~~

Find output:

```
c
void f(int n)
{
    if (n <= 0)
        return;

    f(n - 2);
    printf("%d ", n);
}

f(6);
```

~~Q3~~

Find output:

```
c
void f(int n)
{
    if (n == 0)
        return;

    printf("%d ", n);
    f(n - 1);
    printf("%d ", n);
}

f(4);
```

~~Q4~~ — EXACT CALLS

For:

```
c
void f(int n)
{
    if (n == 0)
        return;

    f(n - 1);
}
```

with:

```
c
f(7);
```

find:

- total function invocations
- recursive-call statement executions

Q5 — PREFIX DECREMENT

Find output:

```
c
void f(int n)
{
    if (n == 0)
        return;

    f(--n);

    printf("%d ", n);
}

f(3);
```

Q6 — STATIC VARIABLE

Find output:

```
c
void f(int n)
{
    static int x = 0;

    if (n == 0)
        return;

    x++;

    f(n - 1);

    printf("%d ", x);
}

f(3);
```

Q7 — TWO RECURSIVE CALLS

Find output:

```
c
void f(int n)
{
    if (n == 0)
```



```

        return;

    f(n - 1);

    printf("%d ", n);

    f(n - 1);
}

f(3);

```

Q4 — EXACT TREE COUNT

For:

```

c
void f(int n)
{
    if (n == 0)
        return;

    f(n - 1);
    f(n - 1);
}

```

For input `n = 4`, find:

- number of base calls
- number of non-base calls
- total calls
- asymptotic time
- asymptotic stack space

Q9 — PLACEMENT THINKING

Consider naive Fibonacci:

```

c
int fib(int n)
{
    if (n <= 1)
        return n;

    return fib(n - 1) + fib(n - 2);
}

```

Explain in simple words:

1. Why many subproblems repeat.
2. Why time becomes exponential.
3. Why stack space is still only linear.

PROGRESS

Recursion Module

' Lesson 17 — Recursion Fundamentals ' Lesson 18 — Recursion Tracing Masterclass

You should now be able to separate:

```
TEXT
CALL ORDER
RETURN ORDER
OUTPUT
TOTAL CALLS
STACK DEPTH
```

which is the foundation for all harder recursion questions.

NEXT — MODULE 1 / LESSON 19

Types of Recursion

Next we cover:

- direct recursion
- indirect recursion
- head recursion
- tail recursion
- tree recursion
- nested recursion
- linear recursion
- mutual recursion
- tail-recursion optimization intuition
- which recursive forms can easily become loops
- GATE identification questions

Examples:

```
C
f(n - 1);
printf(...);
```

VS

```
C
printf(...);
f(n - 1);
```

and:

```
C
return f(f(n - 1));
```

We will learn exactly what type each represents and which distinctions actually matter for GATE.

Type **NEXT**.

USER

NEXT

ASSISTANT

MODULE 1 — LESSON 19

Types of Recursion — Direct, Indirect, Tail, Head, Tree, Nested & Mutual Recursion

We now move from **tracing recursion** to identifying **different recursion forms**.

For GATE, the important thing is not just remembering names. You should be able to look at code and say:

What type of recursion is this, what happens to the call stack, and how would I trace it?

GATE importance: ~~0=0%~~ **IMPORTANT** Placement importance: **+R** ~~IMP~~ **IMPORTANT**

1. BIG PICTURE

We will study:

- Direct recursion
- Indirect recursion
- Mutual recursion
- Linear recursion
- Tail recursion
- Head recursion
- Tree recursion
- Nested recursion

Some of these classifications overlap.

For example, a function can be:

TEXT

Direct + Linear + Tail recursive

at the same time.

~~0=0%~~ **Do not** assume every function belongs to exactly one category.

2. DIRECT

USER
NEXT

ASSISTANT

MODULE 1 — LESSON 19

Types of Recursion — Direct, Indirect, Tail, Head, Tree & Nested Recursion

We continue exactly from where Lesson 19 stopped.

2. DIRECT RECURSION

A function directly calls itself.

Example:

```
c
void f(int n)
{
    if (n == 0)
        return;

    f(n - 1);
}
```

Here:

TEXT
f !' f

So this is:

Direct Recursion

3. STORY / INTUITION

Imagine you ask yourself:

Solve problem 5.

Then you tell yourself:

First solve problem 4.

Then:

First solve problem 3.

You are directly asking **yourself again**.

That is direct recursion.

4. EXAMPLE

```
C
int fact(int n)
{
    if (n == 0)
        return 1;

    return n * fact(n - 1);
}
```

`fact()` directly calls:

```
C
fact()
```

Therefore:

```
TEXT
Direct Recursion
```

5. INDIRECT RECURSION

Now imagine function `A()` calls function `B()`, and `B()` eventually calls `A()` again.

Example:

```
C
void A(int n);
void B(int n);

void A(int n)
{
    if (n <= 0)
        return;

    B(n - 1);
}

void B(int n)
{
    if (n <= 0)
        return;

    A(n - 1);
}
```

Call relationship:

TEXT

A
!"
B
!"
A
!"
B
...

This is:

Indirect Recursion

6. MEMORY TRICK

Direct

TEXT

A !' A

Indirect

TEXT

A !' B !' A

Think:

Direct = I call myself.

Indirect = My friend calls me back.

7. MUTUAL RECURSION

When two or more functions recursively call one another, this is commonly called:

Mutual Recursion

Example:

TEXT

A !' B !' A

So mutual recursion is essentially an important form of indirect recursion.

Example:

```

c
void even(int n);
void odd(int n);

void even(int n)
{
    if (n == 0)
    {
        printf("Even");
        return;
    }

    odd(n - 1);
}

void odd(int n)
{
    if (n == 0)
    {
        printf("Odd");
        return;
    }

    even(n - 1);
}

```

8. TRACE MUTUAL RECURSION

Call:

```

c
even(4);

```

Trace:

```

TEXT
even(4)
!"
odd(3)
!"
even(2)
!"
odd(1)
!"
even(0)

```

At:

```

TEXT
even(0)

```

prints:

```

TEXT
Even

```

This is mutual/indirect recursion.

9. IMPORTANT GATE POINT

Don't classify based only on function name.

Look at the call chain.

If:

TEXT

A !' B !' C !' A

then recursion still exists even though no function directly calls itself.

10. LINEAR RECURSION

A recursive function that makes at most one recursive call per invocation is often called:

Linear Recursion

Example:

```
C
void f(int n)
{
    if (n == 0)
        return;

    f(n - 1);
}
```

Each invocation creates only one child.

Call structure:

TEXT

```
f(4)
|
f(3)
|
f(2)
|
f(1)
|
f(0)
```

It forms a chain rather than a branching tree.

11. COMPLEXITY OF SIMPLE LINEAR RECURSION

For:


```
c
f(n - 1);
```

with constant work:

```
TEXT
Time = O(n)
```

Maximum recursion depth:

```
TEXT
O(n)
```

Auxiliary stack:

```
TEXT
O(n)
```

12. TAIL RECURSION Ø=Ý%

Tail recursion is extremely important.

A function is tail recursive when:

The recursive call is effectively the last operation the function needs to perform before returning.

Example:

```
c
void f(int n)
{
    if (n == 0)
        return;

    printf("%d ", n);

    f(n - 1);
}
```

After:

```
c
f(n - 1);
```

there is no remaining computation in this invocation.

So this is tail recursive.

13. STORY

Think:

I finish all my work first, then hand the remaining job to the next call.

Example:

```
TEXT
print n
!"
call smaller problem
!"
nothing left to do
```

That is the main intuition behind tail recursion.

14. TAIL POSITION

Consider:

```
C
return f(n - 1);
```

The returned result is simply the result of the recursive call.

There is no:

```
TEXT
+ something
× something
print afterward
```

remaining.

So recursive call is in:

Tail Position

15. TAIL RECURSION EXAMPLE

```
C
int f(int n)
{
    if (n == 0)
        return 0;

    return f(n - 1);
}
```

Recursive call is the last computation.

Tail recursive.

16. NOT TAIL RECURSION

Consider factorial:

```
C
int fact(int n)
{
    if (n == 0)
        return 1;

    return n * fact(n - 1);
}
```

Is this tail recursive?

No.

Why?

After:

```
C
fact(n - 1)
```

returns, this invocation must still perform:

```
TEXT
n × returned_value
```

There is pending work.

Therefore:

```
TEXT
Not tail recursive
```

0-0% Common exam question.

17. VISUALIZATION

For:

```
C
return n * fact(n - 1);
```

`fact(4)` waits:

```
TEXT
4 × ?
```

`fact(3)` waits:

TEXT

3 × ?

fact(2) waits:

TEXT

2 × ?

So previous calls still have unfinished work.

That's why it isn't tail recursion.

18. ANOTHER NON-TAIL EXAMPLE

```
c
void f(int n)
{
    if (n == 0)
        return;

    f(n - 1);

    printf("%d ", n);
}
```

Not tail recursive.

Why?

Because after recursive call returns:

```
c
printf(...)
```

still needs to execute.

19. DON'T CONFUSE

Tail recursion

```
c
printf("%d", n);
f(n - 1);
```

Recursive call last.

Not tail recursion

```
c
f(n - 1);
printf("%d", n);
```

Work remains after recursive call.

20. WHY TAIL RECURSION MATTERS

Tail recursion can often be converted very naturally into a loop.

Example:

```
c
void f(int n)
{
    if (n == 0)
        return;

    printf("%d ", n);

    f(n - 1);
}
```

Equivalent iterative idea:

```
c
void f(int n)
{
    while (n != 0)
    {
        printf("%d ", n);
        n--;
    }
}
```

21. TAIL-CALL OPTIMIZATION

Some compilers/languages may optimize tail calls so that a new stack frame is not required for every recursive call.

This is called:

Tail Call Optimization / Tail Recursion Optimization

But very important:

Standard C does not guarantee that the compiler will perform this optimization.

So for ordinary GATE complexity analysis, unless optimization is explicitly specified, treat recursion as using stack frames.

0=1% important precision.

22. HEAD RECURSION

Head recursion is commonly used to describe a pattern where:

The recursive call occurs before the main processing of the current invocation.

Example:

```
c
void f(int n)
{
    if (n == 0)
        return;

    f(n - 1);

    printf("%d ", n);
}
```

Here recursion happens first.

Then processing happens while returning.

23. STORY

Head recursion says:

First let the smaller problem finish completely. Then I'll do my work.

So:

```
TEXT
CALL FIRST
WORK LATER
```

24. HEAD RECURSION OUTPUT

For:

```
c
void f(int n)
{
    if (n == 0)
        return;

    f(n - 1);

    printf("%d ", n);
}
```

Call:

```
c
f(4);
```

Output:

TEXT

1 2 3 4

because printing happens during stack unwinding.

25. TAIL VS HEAD

Tail-like pattern

```
C
printf("%d ", n);
f(n - 1);
```

Output:

TEXT

4 3 2 1

Head-like pattern

```
C
f(n - 1);
printf("%d ", n);
```

Output:

TEXT

1 2 3 4

26. MASTER TABLE

Type	Basic idea
Direct	function calls itself
Indirect	function returns to itself through other functions
Linear	at most one recursive call per invocation
Tail	recursive call is final required operation
Head	recursive call happens before current processing
Tree	more than one recursive branch
Nested	recursive call used as argument to same recursive function

27. IMPORTANT NOTE ON TERMINOLOGY

Terms like **head recursion** are commonly taught in data-structures courses, but the most technically important distinction for compiler optimization is usually:

TEXT

tail recursive
vs
non-tail recursive

For GATE, recognize the standard textbook classifications without overcomplicating them.

28. TREE RECURSION Ø=Ý%

Now consider:

```
C
void f(int n)
{
    if (n == 0)
        return;

    f(n - 1);
    f(n - 1);
}
```

Each call produces two more recursive calls.

This creates:

Tree Recursion

29. VISUALIZATION

For:

```
C
f(3);
```

TEXT

```

      f(3)
     /  \
    f(2)  f(2)
   /  \  /  \
  f(1) f(1) f(1) f(1)
 /  \ /  \ /  \ /  \
f(0) ... f(0) ... f(0) ... f(0)
```

This branches like a tree.

30. WHY CALLED TREE RECURSION?

Linear recursion looks like:

TEXT

```
A
|
B
|
C
|
D
```

Tree recursion looks like:

TEXT

```
  A
 / \
B   C
/ \ / \
... 
```

So the name comes directly from the call structure.

31. TREE RECURSION COMPLEXITY

For:

C

```
f(n - 1);
f(n - 1);
```

recurrence:

TEXT

$$T(n) = 2T(n-1) + O(1)$$

So:

TEXT

$$\text{Time} = O(2^n)$$

But maximum recursion depth:

TEXT

$$O(n)$$

Therefore stack space:

TEXT

$$O(n)$$

Remember:

Tree width affects total time.

Longest path affects stack space.

32. FIBONACCI — TREE RECURSION

```
c
int fib(int n)
{
    if (n <= 1)
        return n;

    return fib(n - 1) + fib(n - 2);
}
```

Each non-base call produces two recursive calls.

Therefore this is:

```
TEXT
Direct recursion
+
Tree recursion
+
Non-tail recursion
```

Multiple labels can apply simultaneously.

33. CLASSIFY FIBONACCI

For:

```
c
return fib(n - 1) + fib(n - 2);
```

Direct?

Yes. `fib()` calls `fib()`.

Linear?

No. There are two recursive calls.

Tree?

Yes.

Tail?

No.

Why?

After calls return, addition must be performed.

34. NESTED RECURSION

Now the unusual one.

Nested recursion occurs when:

A recursive call is used as an argument to another call of the same recursive function.

Classic form:

```
c
return f(f(n - 1));
```

Example:

```
c
int f(int n)
{
    if (n > 100)
        return n - 10;

    return f(f(n + 11));
}
```

This is a famous nested-recursion form similar to McCarthy 91.

You do not need to memorize that function.

Just recognize:

```
c
f(f(...))
```

as nested recursion.

35. SIMPLE NESTED SHAPE

```
c
int f(int n)
{
    if (n <= 0)
        return 0;

    return f(f(n - 1));
}
```

Notice:

```
TEXT
outer f(
    inner f(n-1)
)
```

Before outer call can proceed, inner recursive call must finish.

That's why it is called nested recursion.

36. MEMORY TRICK

Tree recursion:

```
TEXT
f(...)
f(...)
```

Two sibling recursive calls.

Nested recursion:

```
TEXT
f(f(...))
```

One recursive call is inside another.

37. TREE VS NESTED

Tree

```
C
f(n - 1);
f(n - 1);
```

```
TEXT
  f
 / \
f   f
```

Nested

```
C
f(f(n - 1));
```

```
TEXT
outer f
    !'
result of inner f
```

Do not confuse them.

38. MULTIPLE RECURSION

Some books use **multiple recursion** for functions making more than one recursive call.

Example:

```
C
f(n - 1);
f(n - 2);
```

Tree recursion is a typical form of multiple recursion.

~~Don't~~ know the term, but tree recursion is usually the more intuitive name.

39. RECURSION TYPE CLASSIFICATION EXAMPLE 1

```
C
void f(int n)
{
    if (n == 0)
        return;

    printf("%d ", n);
    f(n - 1);
}
```

Classification:

```
TEXT
D i r e c t '
L i n e a r '
T a i l '
T r e e 'L
N e s t e d 'L
```

40. EXAMPLE 2

```
C
void f(int n)
{
    if (n == 0)
        return;

    f(n - 1);
    printf("%d ", n);
}
```

Classification:

```
TEXT
D i r e c t '
L i n e a r '
T a i l 'L
H e a d - s t y l e '
T r e e 'L
```

41. EXAMPLE 3

```
c
int f(int n)
{
    if (n == 0)
        return 1;

    return n * f(n - 1);
}
```

Classification:

```
TEXT
D i r e c t '
L i n e a r '
T a i l 'L
```

Why not tail?

Because multiplication remains pending.

42. EXAMPLE 4

```
c
void f(int n)
{
    if (n == 0)
        return;

    f(n - 1);
    f(n - 1);
}
```

Classification:

```
TEXT
D i r e c t '
L i n e a r 'L
T r e e '
T a i l 'L
```

Even though the second recursive call is the final statement, the function as a whole is not normally classified as simple tail recursion because the first recursive call must return before the second can begin.

43. EXAMPLE 5

```
c
return f(f(n - 1));
```

Classification:

TEXT

```
D i r e c t '
N e s t e d '
```

44. EXAMPLE 6

TEXT

```
A !'   B !'   A
```

Classification:

TEXT

```
I n d i r e c t '
M u t u a l '
```

45. TAIL-RECURSIVE FACTORIAL

Can factorial be rewritten as tail recursion?

Yes, using an accumulator.

Ordinary factorial:

```
c
int fact(int n)
{
    if (n == 0)
        return 1;

    return n * fact(n - 1);
}
```

Pending multiplication makes it non-tail.

46. ACCUMULATOR IDEA

Instead, carry the answer so far.

```
c
int factTail(int n, int acc)
{
    if (n == 0)
        return acc;

    return factTail(n - 1, n * acc);
}
```

Call:

```
C
factTail(4, 1);
```

47. TRACE

```
TEXT
factTail(4,1)
!"
factTail(3,4)
!"
factTail(2,12)
!"
factTail(1,24)
!"
factTail(0,24)
```

Answer:

```
TEXT
24
```

Nothing remains to calculate after each recursive call.

Therefore:

```
TEXT
Tail Recursive
```

48. MEMORY TRICK — ACCUMULATOR

Instead of saying:

I'll multiply after the smaller call returns.

Tail recursion says:

I'll calculate now and carry the answer forward.

So:

```
TEXT
pending work
```

becomes:

```
TEXT
accumulator
```


49. CONVERTING TAIL RECURSION TO LOOP

Tail factorial:

```
c
int factTail(int n, int acc)
{
    if (n == 0)
        return acc;

    return factTail(n - 1, n * acc);
}
```

can naturally become:

```
c
int fact(int n)
{
    int acc = 1;

    while (n != 0)
    {
        acc = acc * n;
        n--;
    }

    return acc;
}
```

This is why tail recursion and iteration are closely related.

50. HEAD RECURSION TO ITERATION

Head recursion often requires thinking about reversed execution order.

Example:

```
c
void f(int n)
{
    if (n == 0)
        return;

    f(n - 1);

    printf("%d ", n);
}
```

prints:

```
TEXT
1 2 3 ... n
```

A loop can simply do:

```
c
for (int i = 1; i <= n; i++)
{
    printf("%d ", i);
}
```

So many recursive forms can still be converted to iteration, but tail recursion is especially direct.

51. GATE TRAP — LAST LINE DOES NOT AUTOMATICALLY MEAN TAIL RECURSION

Consider:

```
c
return 1 + f(n - 1);
```

The recursive call visually appears near the end.

But it's not tail recursive.

Why?

After:

```
c
f(n - 1)
```

returns, we still perform:

```
TEXT
+ 1
```

So:

Pending work exists.

52. CORRECT TEST FOR TAIL RECURSION

Ask:

Once the recursive call returns, does this invocation still need to do anything?

If:

```
TEXT
NO
```

! tail recursion.

If:

TEXT

YES

!' not tail recursion.

0=100% test.

53. EXAMPLES — TAIL OR NOT?

A

```
c
return f(n - 1);
```

Tail'

B

```
c
return 2 * f(n - 1);
```

Tail'L

C

```
c
f(n - 1);
return;
```

Tail'

D

```
c
f(n - 1);
printf("A");
```

Tail'L

E

```
c
return f(n - 1) + f(n - 2);
```

Tail'L

54. RECURSION CLASSIFICATION DOES NOT DIRECTLY TELL COMPLEXITY

Important.

Tail recursive function:

```
C
f(n - 1)
```

could be:

```
TEXT
O(n)
```

Another tail recursive function:

```
C
f(n / 2)
```

could be:

```
TEXT
O(log n)
```

So:

Tail/head/direct are structural classifications, not complexity classes.

Don't confuse.

55. DIRECT RECURSION CAN BE EXPONENTIAL

Example Fibonacci:

```
C
fib(n - 1) + fib(n - 2)
```

is direct recursion but exponential.

So:

```
TEXT
Direct recursion "O(n)"
```

56. INDIRECT RECURSION CAN ALSO TERMINATE NORMALLY

Example:

```
TEXT
A(4)
! ' B ( 3 )
! ' A ( 2 )
! ' B ( 1 )
! ' A ( 0 )
```

As long as arguments move toward a valid base case.

The termination rule remains the same:

STOP + PROGRESS

57. GATE STYLE — IDENTIFY TYPE

Consider:

```
c
int f(int n)
{
    if (n <= 1)
        return n;

    return f(n - 1) + f(n - 2);
}
```

Answer:

```
TEXT
Direct recursion
Tree / multiple recursion
Non-tail recursion
```

58. ANOTHER

```
c
void f(int n)
{
    if (n == 0)
        return;

    printf("%d", n);

    f(n - 1);
}
```

Answer:

```
TEXT
Direct
Linear
Tail recursive
```

59. ANOTHER

```
c
void f(int n)
{
    if (n == 0)
```

```
        return;

    f(n - 1);

    printf("%d", n);
}
```

Answer:

TEXT

Direct
Linear
Head-style
Non-tail

60. ANOTHER

```
C
int f(int n)
{
    if (n <= 0)
        return 0;

    return f(f(n - 1));
}
```

Answer:

TEXT

Direct
Nested recursion

61. ANOTHER

```
C
void A(int n)
{
    if (n == 0)
        return;

    B(n - 1);
}

void B(int n)
{
    if (n == 0)
        return;

    A(n - 1);
}
```

Answer:

TEXT

Indirect recursion
Mutual recursion

62. RECURSION TYPE SUMMARY VISUAL

TEXT

RECURSION

```
%  
% % %   D i r e c t  
%           f ! '   f  
%  
% % %   I n d i r e c t   /   M u t u a l  
%           A ! '   B ! '   A  
%  
% % %   L i n e a r  
%           o n e   r e c u r s i v e   b r a n c h  
%  
% % %   T r e e   /   M u l t i p l e  
%           m u l t i p l e   r e c u r s i v e   b r a n c h e s  
%  
% % %   T a i l  
%           n o   p e n d i n g   w o r k   a f t e r   r e c u r s i v e   c a l l  
%  
% % %   H e a d - s t y l e  
%           r e c u r s i o n   b e f o r e   c u r r e n t   p r o c e s s i n g  
%  
% % %   N e s t e d  
%           f ( f ( . . . ) )
```

63. GATE FOCUS

Ø=MUST KNOW:

- Direct recursion
- Indirect recursion
- Tail recursion
- Non-tail recursion
- Tree recursion

+IMPORANT:

- Linear recursion
- Head recursion
- Mutual recursion

Ø=KNOW:

- Nested recursion

For GATE, **tail recursion identification** and **tree-recursion complexity** are particularly useful.

64. GATE TRAP SHEET

Trap 1

```
C
return n * f(n - 1);
```

is **not** tail recursion.

Trap 2

Recursive call appearing on last source-code line does not automatically make it tail recursive.

Check pending operations.

Trap 3

```
C
f(n - 1);
f(n - 1);
```

is tree/multiple recursion, not linear recursion.

Trap 4

Indirect recursion does not require a function to explicitly call its own name.

Trap 5

A recursive function can have multiple classifications simultaneously.

Trap 6

Tail recursion does not automatically mean $O(1)$ space in C.

Tail-call optimization is not guaranteed.

65. PLACEMENT CONNECTION

These forms appear naturally in real problems.

Tail / linear

Simple accumulations and repeated reduction.

Tree recursion

- Fibonacci
- subset generation
- backtracking

- divide-and-conquer

Indirect recursion

Less common in interview coding but may appear in:

- state transitions
- parsers
- mutually dependent definitions

Recursive trees

Tree traversal itself naturally creates branching recursion.

66. INTERVIEW QUESTIONS

Q1. What is direct recursion?

A function directly calls itself.

Q2. What is indirect recursion?

A function calls itself through another function or sequence of functions.

Q3. What is tail recursion?

A recursion where the recursive call is the final required operation of the current invocation.

Q4. Is factorial's normal recursive implementation tail recursive?

No, because multiplication remains after recursive return.

Q5. What is tree recursion?

A recursion in which calls branch into multiple recursive calls.

Q6. Why can tree recursion be expensive?

The number of total calls can grow rapidly, often exponentially.

67. MEMORY TRICKS

Direct

Me !' Me

Indirect

Me !' Friend !' Me

Tail

Call and I'm DONE

Head

Call first, work later

Tree

One becomes many

Nested

Call inside call

TEXT

```
f(f(...))
```

68. NOTEBOOK NOTES

Direct

C

```
f(n - 1);
```

inside `f`.

Indirect

TEXT

```
A !' B !' A
```

Tail

C

```
return f(n - 1);
```

No pending work.

Non-tail

C

```
return n * f(n - 1);
```

Pending multiplication.

Tree

```
C
f(n - 1);
f(n - 1);
```

Multiple branches.

Nested

```
C
f(f(n - 1));
```

69. 30-SECOND REVISION

Remember:

1. `A != direct`.
 2. `A != B = indirect/mutual`.
 3. One recursive child = linear recursion.
 4. Multiple children = tree recursion.
 5. Tail recursion has no work after recursive call.
 6. `n * f(n-1)` is not tail recursion.
 7. Head recursion performs current work after returning.
 8. `f(f(...))` = nested recursion.
 9. One function may have multiple recursion labels.
 10. Tail-call optimization is not guaranteed in C.
-

70. ACTIVE RECALL

Q1

Classify:

```
C
return f(n - 1);
```

inside `f()`.

Q2

Is this tail recursive?

```
C
return n + f(n - 1);
```

Q3

What type is:

```
c
f(n - 1);
f(n - 2);
```

?

Q4

What type is:

```
c
return f(f(n - 1));
```

?

Q5

What type is:

```
TEXT
A !' B !' A
```

?

ANSWERS

A1

Direct, linear, tail recursion.

A2

No. Addition remains after the recursive call returns.

A3

Tree / multiple recursion.

A4

Nested recursion.

A5

Indirect / mutual recursion.

71. PRACTICE

Q1

Classify:

```

c
void f(int n)
{
    if (n == 0)
        return;

    printf("%d ", n);

    f(n - 1);
}

```

Q2

Classify:

```

c
void f(int n)
{
    if (n == 0)
        return;

    f(n - 1);

    printf("%d ", n);
}

```

Q3

Is this tail recursive?

```

c
int f(int n)
{
    if (n == 0)
        return 0;

    return 1 + f(n - 1);
}

```

Explain.

Q4

Classify:

```

c
int f(int n)
{
    if (n <= 1)
        return n;

    return f(n - 1) + f(n - 2);
}

```

Q=Q5

Classify:

```
C
int f(int n)
{
    if (n <= 0)
        return 0;

    return f(f(n - 1));
}
```

Q=Q6

Given:

```
TEXT
A  !'  B  !'  C  !'  A
```

what broad recursion type is this?

Q=Q7 — TAIL TEST

Which is tail recursive?

A

```
C
return n * f(n - 1);
```

B

```
C
return f(n - 1);
```

C

```
C
f(n - 1);
printf("%d", n);
return;
```

Q=Q8 — COMPLEXITY CONNECTION

For:

```

C
void f(int n)
{
    if (n == 0)
        return;

    f(n - 1);
    f(n - 1);
}

```

state:

- recursion type
- time complexity
- stack-space complexity

0=09 — PLACEMENT THINKING

Why can this ordinary factorial:

```

C
return n * fact(n - 1);

```

not immediately discard the current stack frame before the child returns?

What pending information/work must it remember?

PROGRESS

Recursion Module

' Lesson 17 — Fundamentals ' Lesson 18 — Tracing Masterclass ' Lesson 19 — Types of Recursion

NEXT — MODULE 1 / LESSON 20

Recursion Complexity — Recurrence Relations, Call Counts & Stack Space

Next we move into one of the highest-value GATE areas:

```

TEXT
T(n) = T(n-1) + 1
T(n) = T(n/2) + 1
T(n) = 2T(n-1) + 1
T(n) = 2T(n/2) + n

```

We will learn to derive complexity directly from recursive C code and distinguish:

```

TEXT
TIME = total work across all calls

SPACE = maximum active recursion depth

```

Then we'll solve GATE-style cases involving:

- $f(n-1)$
- $f(n-2)$
- $f(n/2)$
- two recursive calls
- unequal recursive branches
- work before/after calls
- loops inside recursion
- recursion trees
- exact call counts
- auxiliary stack complexity

Type **NEXT**.

USER

NEXT

ASSISTANT

MODULE 1 — LESSON 20

Recursion Complexity — Recurrences, Call Counts, Time & Stack Space

This is one of the most important recursion lessons for GATE.

The main idea today is:

Time complexity = total work done by all recursive calls. Space complexity = maximum number of active recursive calls at one time.

GATE importance: $\emptyset = Y\% \text{ Ours} \leq \text{Know}$ **Placement importance:** $\emptyset = Y\% \text{ Ours} \leq \text{Know}$

1. WHY RECURSION COMPLEXITY FEELS DIFFICULT

With loops, you often count iterations.

Example:

```
c
for (int i = 0; i < n; i++)
{
    printf("");
}
```

Clearly:

TEXT

```
n iterations
!' 0 ( n )
```

But recursion hides the repetition inside function calls.

Example:

```
c
void f(int n)
{
    if (n == 0)
        return;

    f(n - 1);
}
```

You need to understand:

TEXT

```
f(n)
!' f ( n - 1 )
!' f ( n - 2 )
!' . . .
!' f ( 0 )
```

That call structure determines complexity.

2. FIRST GOLDEN RULE

Suppose:

```
c
void f(int n)
{
    if (n == 0)
        return;

    f(n - 1);
}
```

Each invocation performs:

TEXT

```
constant work
+
one recursive call on n-1
```

So we write:

Recurrence

TEXT

$$T(n) = T(n-1) + O(1)$$

Usually simplified as:

TEXT

$$T(n) = T(n-1) + 1$$

for intuition.

3. WHAT IS A RECURRENCE?

A recurrence expresses the running time of a problem using the running time of a smaller problem.

For:

c

f(n - 1);

we say:

TEXT

current work
+
work for input n-1

So:

TEXT

$$T(n) = T(n-1) + c$$

where c is constant work.

4. EXPAND THE RECURRENCE

Start:

TEXT

$$T(n) = T(n-1) + c$$

Now:

TEXT

$$T(n-1) = T(n-2) + c$$

Therefore:

TEXT

$$\begin{aligned} T(n) \\ &= T(n-2) + 2c \end{aligned}$$

Continue:

```
TEXT
T(n)
= T(n-3) + 3c
```

Eventually:

```
TEXT
T(n)
= T(0) + nc
```

So:

```
TEXT
T(n) = O(n)
```

~~O=Vn~~ This expansion method is extremely useful.

5. STACK SPACE FOR $f(n-1)$

Call chain:

```
TEXT
f(n)
!"
f(n-1)
!"
f(n-2)
!"
...
f(0)
```

All these calls are simultaneously active before the base case returns.

Maximum depth:

```
TEXT
n+1
```

Asymptotically:

```
TEXT
O(n)
```

Therefore:

```
TEXT
Time  = O(n)
Space = O(n)
```

6. FORMULA BOX

For:

TEXT

$$T(n) = T(n-1) + O(1)$$

Result:

TEXT

$$\text{Time} = O(n)$$

If recursion depth also decreases one by one:

TEXT

$$\text{Stack Space} = O(n)$$

7. CASE 2 — $f(n-2)$

Consider:

```
C
void f(int n)
{
    if (n <= 0)
        return;

    f(n - 2);
}
```

Calls:

TEXT

```
n
n-2
n-4
n-6
...
```

How many calls?

Approximately:

TEXT

$$n/2$$

So:

TEXT

$$T(n) = T(n-2) + O(1)$$

Expansion gives:

TEXT

$O(n/2)$

Ignoring constants:

TEXT

$O(n)$

Stack depth:

TEXT

$O(n)$

asymptotically.

8. IMPORTANT

Do not write:

TEXT

$O(n/2)$

as final Big-O.

Because constant factors are ignored:

TEXT

$O(n/2) = O(n)$

~~$O=O$~~ Common beginner mistake.

9. CASE 3 — $f(n/2)$

Now:

```
c
void f(int n)
{
    if (n <= 1)
        return;

    f(n / 2);
}
```

Call values:

TEXT

n
n/2
n/4
n/8

...
1

How many times can you divide n by 2?

Approximately:

TEXT

$\log_2 n$

So:

TEXT

$T(n) = T(n/2) + O(1)$

Therefore:

TEXT

Time = $O(\log n)$

Recursion depth:

TEXT

$O(\log n)$

Thus:

TEXT

Space = $O(\log n)$

10. WHY LOG?

Suppose:

TEXT

$n = 32$

Calls:

TEXT

32
16
8
4
2
1

Only 6 levels.

And:

TEXT

```
l o g , 3 2 = 5
```

plus the base-level detail.

Asymptotically:

TEXT

```
O(log n)
```

11. MEMORY TRICK

TEXT

Subtract constant repeatedly

!' usually linear

Divide repeatedly

!' usually logarithmic

Examples:

TEXT

```
n !' n - 1 !' O ( n )
```

```
n !' n - 5 !' O ( n )
```

```
n !' n / 2 !' O ( l o g n )
```

```
n !' n / 3 !' O ( l o g n )
```

The logarithm base does not matter in Big-O.

12. CASE 4 — WORK INSIDE EACH CALL

Consider:

C

```
void f(int n)
{
    if (n == 0)
        return;

    for (int i = 0; i < n; i++)
    {
        printf("");
    }

    f(n - 1);
}
```

Many students immediately say:

TEXT

$O(n^2)$

Let's prove it.

13. WORK PER LEVEL

At $f(n)$:

TEXT

n iterations

At $f(n-1)$:

TEXT

$n-1$ iterations

At $f(n-2)$:

TEXT

$n-2$ iterations

...

At $f(1)$:

TEXT

1 iteration

Total work:

TEXT

$n + (n-1) + (n-2) + \dots + 1$

Formula:

TEXT

$n(n+1)/2$

Therefore:

TEXT

$O(n^2)$

14. RECURRENCE

TEXT

$$T(n) = T(n-1) + O(n)$$

Expansion:

TEXT

$$T(n) = n + (n-1) + \dots + 1$$

Hence:

TEXT

$$T(n) = O(n^2)$$

Stack depth still:

TEXT

$$O(n)$$

So:

TEXT

$$\begin{aligned} \text{Time} &= O(n^2) \\ \text{Space} &= O(n) \end{aligned}$$

$O \neq \Theta$ Time and space are different.

15. FORMULA BOX

For:

TEXT

$$T(n) = T(n-1) + n$$

Result:

TEXT

$$T(n) = O(n^2)$$

because:

TEXT

$$\begin{aligned} 1 + 2 + \dots + n \\ = n(n+1)/2 \end{aligned}$$

16. CASE 5 — CONSTANT LOOP + `n-1`

```

c
void f(int n)
{
    if (n == 0)
        return;

    for (int i = 0; i < 100; i++)
    {
        printf("*");
    }

    f(n - 1);
}

```

The loop executes exactly:

TEXT

100

times per recursive call.

100 is constant.

Therefore:

TEXT

$$T(n) = T(n-1) + O(1)$$

So:

TEXT

$$\text{Time} = O(n)$$

~~$O(n^2)$~~ Don't see a loop and automatically multiply by n .

17. CASE 6 — LOOP + $n/2$

```

c
void f(int n)
{
    if (n <= 1)
        return;

    for (int i = 0; i < n; i++)
    {
        printf("*");
    }

    f(n / 2);
}

```

Recurrence:

TEXT

$$T(n) = T(n/2) + n$$

Let's expand.

18. EXPANSION

TEXT

$$T(n)$$

$$= n + T(n/2)$$

$$= n + n/2 + T(n/4)$$

$$= n + n/2 + n/4 + \dots$$

This is a geometric series:

TEXT

$$n + n/2 + n/4 + n/8 + \dots$$

Sum is less than:

TEXT

$$2n$$

Therefore:

TEXT

$$T(n) = O(n)$$

Very important GATE pattern.

FORMULA BOX

TEXT

$$n + n/2 + n/4 + \dots$$

$$< 2n$$

Therefore:

TEXT

$$T(n) = T(n/2) + n$$

$$\neq O(n)$$

Stack depth:

TEXT

$$O(\log n)$$

So:

TEXT

Time = $O(n)$
Space = $O(\log n)$

19. DON'T CONFUSE

Because there are:

TEXT

$\log n$ recursion levels

and each first level has n work, some students say:

TEXT

$O(n \log n)$

Wrong here.

Why?

Each level's work is decreasing:

TEXT

n
 $n/2$
 $n/4$
...

not n at every level.

Total is only:

TEXT

$O(n)$

20. CASE 7 — TWO HALF-SIZED CALLS

Consider:

C

```
void f(int n)
{
    if (n <= 1)
        return;

    f(n / 2);
    f(n / 2);
}
```

Recurrence:

TEXT

$$T(n) = 2T(n/2) + O(1)$$

Let's use a recursion tree.

21. RECURSION TREE

Level 0:

TEXT

1 call

Level 1:

TEXT

2 calls

Level 2:

TEXT

4 calls

Level 3:

TEXT

8 calls

Number of levels:

TEXT

$\log_2 n$

At bottom:

TEXT

approximately n leaves

Total number of nodes:

TEXT

$O(n)$

Therefore:

TEXT

Time = $O(n)$

22. WHY NOT $O(2^n)$?

Important.

Earlier:

```
TEXT
2T(n-1)
```

gave exponential time.

But here:

```
TEXT
2T(n/2)
```

does not.

Because the problem size is halved each level.

Compare:

```
TEXT
T(n)=2T(n-1)
! '  O ( 2 ^ n )
```

versus:

```
TEXT
T(n)=2T(n/2)
! '  O ( n )
```

~~$O=$~~ HUGE GATE distinction.

23. STACK SPACE FOR TWO HALF CALLS

Even though both recursive calls exist:

```
C
f(n / 2);
f(n / 2);
```

they execute sequentially.

First entire branch finishes before second branch executes.

Maximum active chain:

```
TEXT
n
n/2
n/4
...
1
```

Depth:

TEXT

$O(\log n)$

Therefore:

TEXT

Space = $O(\log n)$

So:

TEXT

Time = $O(n)$
Space = $O(\log n)$

24. CASE 8 — TWO HALF CALLS + LINEAR WORK

Now:

C

```
void f(int n)
{
    if (n <= 1)
        return;

    for (int i = 0; i < n; i++)
        printf("*");

    f(n / 2);
    f(n / 2);
}
```

Recurrence:

TEXT

$T(n) = 2T(n/2) + O(n)$

~~$O(n)$~~ Very important.

25. RECURSION TREE METHOD

Level 0:

TEXT

1 problem × n work
= n

Level 1:

Two problems of size $n/2$:

TEXT

$$2 \times n/2 \\ = n$$

Level 2:

Four problems of size $n/4$:

TEXT

$$4 \times n/4 \\ = n$$

Every level performs:

TEXT

$$O(n)$$

Number of levels:

TEXT

$$O(\log n)$$

Therefore:

TEXT

$$O(n) \times O(\log n) \\ = \\ O(n \log n)$$

FORMULA BOX

TEXT

$$T(n) = 2T(n/2) + n \\ \therefore O(n \log n)$$

This recurrence appears in:

Merge Sort

~~0-MUST~~ KNOW.

26. STACK SPACE HERE

Maximum recursion depth:

TEXT

$$O(\log n)$$

If the function itself does not allocate an extra $O(n)$ local array at each level, recursive stack itself is:

TEXT

$O(\log n)$

Do not say:

TEXT

$O(n \log n)$

space merely because time is $O(n \log n)$.

27. CASE 9 — TWO $n-1$ CALLS

C

```
void f(int n)
{
    if (n == 0)
        return;

    f(n - 1);
    f(n - 1);
}
```

Recurrence:

TEXT

$T(n) = 2T(n-1) + O(1)$

Tree:

TEXT

```
L e v e l    0    !'    1
L e v e l    1    !'    2
L e v e l    2    !'    4
...
L e v e l    n    !'    2 ^ n
```

Total:

TEXT

$2^{n+1} - 1$

Therefore:

TEXT

Time = $O(2^n)$

Longest path:

TEXT

n

So:

TEXT

Space = $O(n)$

28. EXACT CALL COUNT

For:

```
C
if (n == 0)
    return;

f(n-1);
f(n-1);
```

Let total function invocations be $C(n)$.

Then:

TEXT
 $C(0)=1$

and:

TEXT
 $C(n)=1+2C(n-1)$

Solution:

TEXT
 $C(n)=2^{(n+1)}-1$

Useful exact GATE formula.

29. CASE 10 — THREE RECURSIVE CALLS

```
C
void f(int n)
{
    if (n == 0)
        return;

    f(n - 1);
    f(n - 1);
    f(n - 1);
}
```

Recurrence:

TEXT
 $T(n)=3T(n-1)+O(1)$

Calls grow like:

```
TEXT
1
3
9
27
...
```

Therefore:

```
TEXT
Time =  $O(3^n)$ 
```

Stack depth:

```
TEXT
 $O(n)$ 
```

30. GENERAL PATTERN

If:

```
TEXT
 $T(n) = aT(n-1) + O(1)$ 
```

for constant:

```
TEXT
 $a > 1$ 
```

then growth is exponential:

```
TEXT
 $O(a^n)$ 
```

Examples:

```
TEXT
 $2^{T(n-1)}$  !'  $O(2^n)$ 
 $3^{T(n-1)}$  !'  $O(3^n)$ 
```

31. CASE 11 — UNEQUAL BRANCHES

Consider Fibonacci-style:

```
C
int f(int n)
{
    if (n <= 1)
```

```

        return n;

    return f(n - 1) + f(n - 2);
}

```

Recurrence:

```

TEXT
T(n)
=
T(n-1)
+
T(n-2)
+
O(1)

```

This is exponential.

A common upper bound:

```

TEXT
O(2^n)

```

because both branches are no larger than $T(n-1)$:

```

TEXT
T(n)
<
2T(n-1)+O(1)

```

More precise growth is approximately:

```

TEXT
~ ( \phi ^ n )

```

where:

```

TEXT
\phi = 1.618

```

For ordinary GATE/placement reasoning:

```

TEXT
Exponential

```

is the important point.

32. FIBONACCI STACK SPACE

Although total calls are exponential, maximum active path is:

TEXT

```
fib(n)
!'  f i b ( n - 1 )
!'  f i b ( n - 2 )
!'  . . .
```

Maximum depth:

TEXT

$O(n)$

Therefore:

TEXT

Time = exponential
Space = $O(n)$

~~0-0%~~ Classic interview question.

33. TIME = TREE SIZE

Think of the full recursion tree.

Every call performs some work.

So broadly:

TEXT

TIME

"H total work of all recursion-tree nodes

34. SPACE = TREE HEIGHT

Only calls along the current active path need to remain on the call stack.

So broadly:

TEXT

RECURSIVE STACK SPACE

"H maximum root-to-leaf path

Memory trick:

TIME = TREE

SPACE = HEIGHT

~~0-0%~~ Memorize this.

35. CASE 12 — LOOP AFTER RECURSION

Consider:

```
c
void f(int n)
{
    if (n == 0)
        return;

    f(n - 1);

    for (int i = 0; i < n; i++)
        printf("*");
}
```

Does putting the loop after recursive call change time complexity?

No.

Work is still:

```
TEXT
1 + 2 + ... + n
```

So:

```
TEXT
O(n2)
```

It changes **execution order**, not total asymptotic work.

36. IMPORTANT

For complexity:

```
c
loop();
f(...);
```

versus:

```
c
f(...);
loop();
```

may have the same time complexity.

But for output/tracing, the order may completely change.

~~O = Separate:~~

```
TEXT
COMPLEXITY
```

from:

TEXT

OUTPUT ORDER

37. CASE 13 — TWO SEQUENTIAL RECURSIVE CALLS WITH DIFFERENT INPUTS

```
c
void f(int n)
{
    if (n <= 1)
        return;

    f(n / 2);
    f(n / 2);
}
```

Both branches count toward time.

But only one branch at a time contributes to stack depth.

This is the general principle for ordinary single-threaded recursion.

38. COMMON SPACE MISTAKE

Suppose:

TEXT

there are n leaves

Students may say:

TEXT

space = $O(n)$

Wrong unless all those calls are simultaneously active.

Usually they are not.

For recursive stack:

TEXT

maximum simultaneous calls

matters.

39. AUXILIARY SPACE VS OUTPUT/RESULT STORAGE

If recursion creates an output array of size n , you may need to distinguish:

TEXT

output space

from:

TEXT

auxiliary space

For example, if a function returns an output array of n values but only uses $O(\log n)$ recursion stack:

TEXT

Output space = $O(n)$

Auxiliary recursive stack = $O(\log n)$

$O=0\%$ and placement precision.

40. CASE 14 — LOCAL ARRAY IN EACH RECURSIVE CALL

Consider:

```
c
void f(int n)
{
    if (n == 0)
        return;

    int a[100];

    f(n - 1);
}
```

Each active frame contains a fixed-size array of 100 integers.

Per frame:

TEXT

$O(1)$

because 100 is constant.

Depth:

TEXT

$O(n)$

Total active stack:

TEXT

$O(n)$

41. WHAT IF LOCAL STORAGE DEPENDS ON n ?

Conceptually suppose each recursion level stores $O(n)$ extra data and calls $f(n-1)$.

Then maximum simultaneous storage could be:

TEXT

$$\begin{aligned} & n + (n-1) + \dots + 1 \\ &= \\ & O(n^2) \end{aligned}$$

So do not blindly say recursive space equals depth.

More precisely:

Sum the memory of simultaneously active frames.

For most standard simple recursion, each frame is $O(1)$, so space equals depth.

Important technical refinement.

42. MASTER COMPLEXITY TABLE

Recursive pattern	Time	Stack space
$T(n)=T(n-1)+1$	$O(n)$	$O(n)$
$T(n)=T(n-2)+1$	$O(n)$	$O(n)$
$T(n)=T(n/2)+1$	$O(\log n)$	$O(\log n)$
$T(n)=T(n-1)+n$	$O(n^2)$	$O(n)$
$T(n)=T(n/2)+n$	$O(n)$	$O(\log n)$
$T(n)=2T(n-1)+1$	$O(2^n)$	$O(n)$
$T(n)=2T(n/2)+1$	$O(n)$	$O(\log n)$
$T(n)=2T(n/2)+n$	$O(n \log n)$	$O(\log n)^*$
Fibonacci-like	exponential	$O(n)$

* assuming no additional large per-call storage.

Copy this table.

43. GATE METHOD — DERIVE RECURRENCE FROM CODE

Suppose:

```
c
void f(int n)
{
    if (n <= 1)
        return;
```

```

    for (int i = 0; i < n; i++)
        work();

    f(n / 2);
    f(n / 2);
}

```

Step 1 — Recursive calls

Two calls of size $n/2$:

TEXT
 $2T(n/2)$

Step 2 — Non-recursive work

Loop:

TEXT
 $O(n)$

Step 3 — Combine

TEXT
 $T(n) = 2T(n/2) + O(n)$

Step 4 — Solve

TEXT
 $O(n \log n)$

That's the entire workflow.

44. ANOTHER DERIVATION

```

c
void f(int n)
{
    if (n <= 1)
        return;

    for (int i = 0; i < n * n; i++)
        work();

    f(n / 2);
}

```

One recursive call:

TEXT
 $T(n/2)$

Loop:

TEXT

$$O(n^2)$$

So:

TEXT

$$T(n) = T(n/2) + O(n^2)$$

Expansion:

TEXT

$$\begin{aligned} &n^2 \\ &+ \\ &(n/2)^2 \\ &+ \\ &(n/4)^2 \\ &+ \dots \end{aligned}$$

which is:

TEXT

$$n^2 + n^2/4 + n^2/16 + \dots$$

Geometric sum:

TEXT

$$O(n^2)$$

Therefore:

TEXT

$$\text{Time} = O(n^2)$$

45. PATTERN— DOMINANT FIRST LEVEL

When:

TEXT

$$T(n) = T(n/2) + n^k$$

for positive constant k , the work forms a decreasing geometric series:

TEXT

$$\begin{aligned} &n^k \\ &+ \\ &(n/2)^k \\ &+ \\ &(n/4)^k \\ &+ \dots \end{aligned}$$

So usually:

TEXT

$O(n^k)$

Examples:

TEXT

$T(n) = T(n/2) + n \quad !' \quad O(n)$

$T(n) = T(n/2) + n^2 \quad !' \quad O(n^2)$

46. CASE 15 — WORK $\log n$ AT EACH $n-1$ CALL

Suppose:

TEXT

$T(n)$
=
 $T(n-1)$
+
 $\log n$

Then:

TEXT

$T(n)$
"H $\log 1 + \log 2 + \dots + \log n$

And:

TEXT

$\log(1 \times 2 \times \dots \times n)$
=
 $\log(n!)$

Using asymptotic reasoning:

TEXT

$\sim (n \log n)$

This is more advanced, but useful to recognize.

~~$O(n)$~~ NOW.

47. CASE 16 — RECURSIVE CALL INSIDE LOOP

Now a dangerous-looking pattern:

C

```
void f(int n)
{
    if (n == 0)
```

```

    return;

    for (int i = 0; i < 2; i++)
    {
        f(n - 1);
    }
}

```

How many recursive calls per invocation?

TEXT

2

So recurrence:

TEXT

$T(n) = 2T(n-1) + O(1)$

Therefore:

TEXT

$O(2^n)$

Even though the loop itself has only 2 iterations.

$O(2^n)$ is very important.

48. LOOP COUNT CAN BECOME BRANCHING FACTOR

If:

```

c
for (int i = 0; i < 3; i++)
{
    f(n - 1);
}

```

then:

TEXT

$T(n) = 3T(n-1) + O(1)$

So:

TEXT

$O(3^n)$

A constant-loop can still create exponential recursion if it repeatedly makes recursive calls.

49. VARIABLE NUMBER OF RECURSIVE CALLS

Consider conceptually:

```
c
for (int i = 0; i < n; i++)
{
    f(n - 1);
}
```

Now each invocation makes n recursive calls of size $n-1$.

This grows extremely rapidly.

Recurrence:

```
TEXT
T(n)=nT(n-1)+O(n)
```

which is factorial-like:

```
TEXT
O(n!)
```

roughly.

$\Theta = \text{Placement/backtracking connection.}$

50. PERMUTATION CONNECTION

Generating all permutations of n items produces about:

```
TEXT
n!
```

outputs.

Therefore no algorithm that explicitly generates every permutation can run in less than proportional-to-output work.

This is why permutation backtracking naturally has factorial-scale complexity.

51. RECURSION TREE — THREE QUESTIONS

Whenever you draw a recursion tree, ask:

1. How many children per node?

Branching factor.

2. How much does input shrink?

Controls height.

3. How much work is done at each node/level?

Controls total time.

$\Theta = \text{Yes}$ is the foundation of divide-and-conquer complexity.

52. EXAMPLE — $2T(n/2)+1$

Branching factor:

TEXT
2

Input shrinks:

TEXT
 $n \rightarrow n / 2$

Height:

TEXT
 $\log n$

Leaves:

TEXT
 $2^{(\log, n)}$
 $= n$

Total:

TEXT
 $O(n)$

53. EXAMPLE — $4T(n/2)+1$

Recurrence:

TEXT
 $T(n)=4T(n/2)+O(1)$

Height:

TEXT
 $\log, n$

Leaves:

TEXT
 $4^{(\log, n)}$

Since:

TEXT

$$4 = 2^2$$

TEXT

$$4^{\log_2 n} = n^2$$

So:

TEXT

$$T(n) = O(n^2)$$

✓ Nice GATE-style result.

54. GENERAL LEAF IDEA

For:

TEXT

$$T(n) = aT(n/b) + O(1)$$

height:

TEXT

$$\log_b n$$

number of leaves:

TEXT

$$a^{\log_b n}$$

which equals:

TEXT

$$n^{\log_b a}$$

This term becomes central in the:

Master Theorem

which we'll study deeply in the Algorithms subject.

For now, recognize the intuition.

55. GATE-STYLE EXAMPLE

Find complexity:


```

c
void f(int n)
{
    if (n <= 1)
        return;

    f(n / 2);
    f(n / 2);
    f(n / 2);
    f(n / 2);
}

```

Recurrence:

TEXT
 $T(n) = 4T(n/2) + O(1)$

Therefore:

TEXT
 Time = $O(n^2)$

Stack depth:

TEXT
 $O(\log n)$

So:

TEXT
 Space = $O(\log n)$

56. EXACT VS ASYMPTOTIC

Suppose exact calls:

TEXT
 $2^{(n+1)} - 1$

GATE may ask exact count.

But if asked complexity:

TEXT
 $O(2^n)$

Do not answer exact formula when only asymptotic complexity is requested.

And vice versa.

57. BIG-O VS THETA

If we know tight growth, prefer:

TEXT
 $\sim (n)$

rather than only:

TEXT
 $O(n)$

For example:

TEXT
 $T(n) = T(n-1) + 1$

is:

TEXT
 $\sim (n)$

GATE questions may distinguish tight bound from upper bound.

For our course:

- $O(\dots)$ = convenient complexity description
- $\sim (\dots)$ = tight asymptotic order

~~Θ~~ We'll formally study these in Complexity Module.

58. SOLVED EXAMPLE 1

```
c
void f(int n)
{
    if (n == 0)
        return;

    printf("");
    f(n - 1);
}
```

Recurrence:

TEXT
 $T(n) = T(n-1) + 1$

Time:

TEXT
 $\sim (n)$

Depth:

TEXT

$\sim (n)$

Space:

TEXT

$\sim (n)$

59. SOLVED EXAMPLE 2

```
c
void f(int n)
{
    if (n <= 1)
        return;

    f(n / 2);

    printf("");
}
```

Recurrence:

TEXT

$T(n) = T(n/2) + 1$

Time:

TEXT

$\sim (\log n)$

Space:

TEXT

$\sim (\log n)$

60. SOLVED EXAMPLE 3

```
c
void f(int n)
{
    if (n == 0)
        return;

    for (int i = 0; i < n; i++)
        work();

    f(n - 1);
}
```

Recurrence:

TEXT

$T(n) = T(n-1) + n$

Time:

TEXT

$\sim (n^2)$

Stack:

TEXT

$\sim (n)$

61. SOLVED EXAMPLE 4

```
c
void f(int n)
{
    if (n <= 1)
        return;

    for (int i = 0; i < n; i++)
        work();

    f(n / 2);
}
```

Recurrence:

TEXT

$T(n) = T(n/2) + n$

Time:

TEXT

$\sim (n)$

Stack:

TEXT

$\sim (\log n)$

62. SOLVED EXAMPLE 5

```
c
void f(int n)
{
    if (n <= 1)
        return;
```

```

    f(n / 2);
    f(n / 2);
}

```

Recurrence:

TEXT
 $T(n) = 2T(n/2) + 1$

Time:

TEXT
 $\sim (n)$

Stack:

TEXT
 $\sim (\log n)$

63. SOLVED EXAMPLE 6

```

C
void f(int n)
{
    if (n <= 1)
        return;

    for (int i = 0; i < n; i++)
        work();

    f(n / 2);
    f(n / 2);
}

```

Recurrence:

TEXT
 $T(n) = 2T(n/2) + n$

Time:

TEXT
 $\sim (n \log n)$

Stack:

TEXT
 $\sim (\log n)$

64. SOLVED EXAMPLE 7

```
c
void f(int n)
{
    if (n == 0)
        return;

    f(n - 1);
    f(n - 1);
}
```

Recurrence:

```
TEXT
T(n)=2T(n-1)+1
```

Time:

```
TEXT
~ ( 2 ^ n )
```

Stack:

```
TEXT
~ ( n )
```

65. GATE TRAP SHEET

Trap 1

Two recursive calls do **not** automatically mean $O(2^n)$.

```
TEXT
2 T ( n - 1 ) ! '   e x p o n e n t i a l
2 T ( n / 2 ) ! '   m a y   b e   l i n e a r
```

Trap 2

$n/2$ reduction generally creates logarithmic depth.

Trap 3

Time is not always equal to recursion depth.

Example:

TEXT

$2T(n-1)$

Time:

TEXT

$O(2^n)$

Depth:

TEXT

$O(n)$

Trap 4

A loop at every recursive level must be summed across levels.

Trap 5

$T(n/2) + n$ is $O(n)$, not $O(n \log n)$.

Trap 6

$2T(n/2) + n$ is $O(n \log n)$.

Trap 7

Constant number of recursive calls inside a loop can create exponential branching.

66. PLACEMENT CONNECTION

You will use these recurrence patterns in:

- binary search
- merge sort
- quicksort
- tree traversal
- DFS
- Fibonacci
- subset generation
- permutations
- backtracking
- divide-and-conquer

Examples:

TEXT

Binary Search
 $T(n) = T(n/2) + O(1)$
! ' $O(\log n)$

TEXT

Merge Sort
 $T(n) = 2T(n/2) + O(n)$
! ' $O(n \log n)$

TEXT

Naïve Fibonacci
 $T(n) = T(n-1) + T(n-2) + O(1)$
! ' exponential

67. MEMORY TRICKS

Time

Count the whole recursion tree.

Space

Count the deepest active path.

One smaller by 1

TEXT

n ! ' $n - 1$
CHAIN ! ' $O(n)$ depth

Divide by 2

TEXT

n ! ' $n / 2$
CHAIN ! ' $O(\log n)$ depth

Two $n-1$ calls

TEXT

BIG TREE ! ' exponential

Two $n/2$ calls

TEXT

balanced tree ! ' often polynomial

68. NOTEBOOK NOTES

Recurrence from code

C

$f(n - 1);$

TEXT

$T(n) = T(n-1) + \text{work}$

C

$f(n / 2);$

TEXT

$T(n) = T(n/2) + \text{work}$

C

$f(n / 2);$

$f(n / 2);$

TEXT

$T(n) = 2T(n/2) + \text{work}$

Key results

TEXT

$T(n) = T(n-1) + 1$
! ' ~ (n)

TEXT

$T(n) = T(n/2) + 1$
! ' ~ (\log n)

TEXT

$T(n) = T(n-1) + n$
! ' ~ (n^2)

TEXT

$T(n) = T(n/2) + n$
! ' ~ (n)

TEXT

$T(n) = 2T(n-1) + 1$
! ' ~ (2^n)

TEXT

$$T(n) = 2T(n/2) + 1$$

! ' ~ (n)

TEXT

$$T(n) = 2T(n/2) + n$$

! ' ~ (n \log n)

69. 30-SECOND REVISION

Remember these seven:

TEXT

- | | | |
|-----|------------------------|-------------------|
| 1 . | $T(n) = T(n - 1) + 1$ | ! ' $O(n)$ |
| 2 . | $T(n) = T(n / 2) + 1$ | ! ' $O(\log n)$ |
| 3 . | $T(n) = T(n - 1) + n$ | ! ' $O(n^2)$ |
| 4 . | $T(n) = T(n / 2) + n$ | ! ' $O(n)$ |
| 5 . | $T(n) = 2T(n - 1) + 1$ | ! ' $O(2^n)$ |
| 6 . | $T(n) = 2T(n / 2) + 1$ | ! ' $O(n)$ |
| 7 . | $T(n) = 2T(n / 2) + n$ | ! ' $O(n \log n)$ |

And:

TEXT

TIME = TOTAL TREE WORK
SPACE = MAX ACTIVE DEPTH

~~Q=Yes~~ These alone solve a large number of questions.

70. ACTIVE RECALL

Q1

Complexity?

TEXT

$$T(n) = T(n-1) + 1$$

Q2

Complexity?

TEXT

$$T(n) = T(n/2) + 1$$

Q3

Complexity?

TEXT

$$T(n) = T(n-1) + n$$

Q4

Complexity?

TEXT

$$T(n) = 2T(n/2) + n$$

Q5

If recursion has exponential number of calls but depth n , what is recursive stack space?

ANSWERS

A1

TEXT

$$\sim (n)$$

A2

TEXT

$$\sim (\log n)$$

A3

TEXT

$$\sim (n^2)$$

A4

TEXT

$$\sim (n \log n)$$

A5

TEXT

$$\sim (n)$$

71. PRACTICE

Q1

Find time and stack space:

```
c
void f(int n)
{
    if (n == 0)
        return;

    f(n - 1);
}
```

~~Q=Q2~~

Find time and stack space:

```
c
void f(int n)
{
    if (n <= 1)
        return;

    f(n / 2);
}
```

~~Q=Q3~~

Find time and stack space:

```
c
void f(int n)
{
    if (n == 0)
        return;

    for (int i = 0; i < n; i++)
        work();

    f(n - 1);
}
```

~~Q=Q4~~

Find time and stack space:

```
c
void f(int n)
{
    if (n <= 1)
        return;

    for (int i = 0; i < n; i++)
        work();

    f(n / 2);
}
```

```
}
```

Q5

Find time and stack space:

```
c
void f(int n)
{
    if (n == 0)
        return;

    f(n - 1);
    f(n - 1);
}
```

Q6

Find time and stack space:

```
c
void f(int n)
{
    if (n <= 1)
        return;

    f(n / 2);
    f(n / 2);
}
```

Q7 — GATE CLASSIC

Find time and recursive-stack complexity:

```
c
void f(int n)
{
    if (n <= 1)
        return;

    for (int i = 0; i < n; i++)
        work();

    f(n / 2);
    f(n / 2);
}
```

Also write the recurrence.

Q8 — FOUR BRANCHES

Find time and stack-space complexity:

```
c
void f(int n)
{
    if (n <= 1)
        return;

    f(n / 2);
    f(n / 2);
    f(n / 2);
    f(n / 2);
}
```

Hint:

TEXT
 $T(n)=4T(n/2)+O(1)$

0=09 — PLACEMENT THINKING

Why is:

TEXT
 $T(n)=2T(n-1)$

exponential, while:

TEXT
 $T(n)=2T(n/2)$

is only linear?

Explain using:

TEXT
tree height
+
number of leaves

PROGRESS

Recursion Module

' Lesson 17 — Fundamentals ' Lesson 18 — Tracing ' Lesson 19 — Types ' Lesson 20 — Recursion Complexity

NEXT — MODULE 1 / LESSON 21

Classical Recursion Problems

Next we complete the Recursion module with practical problems:

- factorial
- sum of first `n`
- sum of array
- reverse array/string recursively
- GCD using Euclid
- power
- optimized power / exponentiation by squaring
- binary search recursion
- Fibonacci
- Tower of Hanoi
- exact Tower of Hanoi recurrence:

TEXT

$T(n) = 2T(n-1) + 1$

and formula:

TEXT

Minimum moves = $2^n - 1$

We will connect every problem to:

TEXT

BASE CASE
RECURSIVE RELATION
TRACE
TIME
SPACE
GATE TRAP
PLACEMENT USE

Type **NEXT**.

USER

NEXT NEXT

ASSISTANT

You gave **two NEXTs**, so I'll continue through the **next two lessons** in sequence:

1. Lesson 21 — Classical Recursion Problems
2. Lesson 22 — Arrays Fundamentals & Memory

MODULE 1 — LESSON 21

Classical Recursion Problems

This lesson completes our main recursion foundation by applying everything we learned:

STACK SPACE

TEXT

$n! = n \times (n-1)!$

Base:

TEXT

$0! = 1$

3. FACTORIAL CODE

```
c
int fact(int n)
{
    if (n == 0)
        return 1;

    return n * fact(n - 1);
}
```

For:

TEXT

`fact(4)`

downward:

TEXT

```
fact(4)
= 4 × fact(3)

fact(3)
= 3 × fact(2)

fact(2)
= 2 × fact(1)

fact(1)
= 1 × fact(0)

fact(0)
= 1
```

upward:

TEXT

```
fact(1) = 1
fact(2) = 2
fact(3) = 6
fact(4) = 24
```

4. FACTORIAL COMPLEXITY

Recurrence:

TEXT

$$T(n) = T(n-1) + O(1)$$

Therefore:

TEXT

$$Time = \sim (n)$$

Recursion depth:

TEXT

$$\sim (n)$$

Auxiliary stack:

TEXT

$$\sim (n)$$

5. GATE TRAP — WRONG BASE VALUE

Wrong:

C

```
if (n == 0)
    return 0;
```

Then:

TEXT

```
1 × 0 = 0
2 × 0 = 0
...
```

Everything becomes zero.

Correct:

TEXT

$$0! = 1$$

6. SUM OF FIRST n NUMBERS

We want:

TEXT

$$1 + 2 + 3 + \dots + n$$

Recursive relation:

TEXT

```
sum(n) = n + sum(n-1)
```

Base:

TEXT

```
sum(0) = 0
```

Code:

C

```
int sum(int n)
{
    if (n == 0)
        return 0;

    return n + sum(n - 1);
}
```

7. TRACE `sum(4)`

TEXT

```
sum(4)
= 4 + sum(3)
= 4 + 3 + sum(2)
= 4 + 3 + 2 + sum(1)
= 4 + 3 + 2 + 1 + sum(0)
= 10
```

Complexity:

TEXT

```
Time   = ~ ( n )
Space  = ~ ( n )
```

8. SUM OF ARRAY RECURSIVELY

Suppose:

C

```
int a[] = {10, 20, 30, 40};
```

We can think:

TEXT

```
sum of first n elements
=
last element
+
sum of first n-1 elements
```

Code:

```
c
int arraySum(int a[], int n)
{
    if (n == 0)
        return 0;

    return a[n - 1] + arraySum(a, n - 1);
}
```

Call:

```
c
arraySum(a, 4);
```

9. TRACE

TEXT

```
arraySum(a, 4)
= a[3] + arraySum(a, 3)
= 40 + 30 + 20 + 10 + 0
= 100
```

Complexity:

TEXT

```
T i m e   =   ~ ( n )
S t a c k   =   ~ ( n )
```

10. POWER

We want:

TEXT

a^n

Basic relation:

TEXT

$a^n = a \times a^{(n-1)}$

Base:

TEXT

$a^0 = 1$

Code:

```
c
int power(int a, int n)
{
    if (n == 0)
        return 1;

    return a * power(a, n - 1);
}
```

11. TRACE `power(2,4)`

TEXT

```
2 × power(2,3)
2 × 2 × power(2,2)
2 × 2 × 2 × power(2,1)
2 × 2 × 2 × 2 × power(2,0)
```

Base:

TEXT

```
power(2,0)=1
```

Result:

TEXT

```
16
```

Complexity:

TEXT

```
T i m e   =   ~ ( n )
S p a c e   =   ~ ( n )
```

12. CAN WE DO POWER FASTER?

Yes.

Suppose:

TEXT

```
2^8
```

Instead of multiplying 2 eight times:

TEXT

```
2^8
=
(2^4)^2
```

and:

TEXT

```
2^4
=
(2^2)^2
```

We repeatedly halve the exponent.

This leads to:

Exponentiation by Squaring

13. FAST POWER

For even `n`:

TEXT

```
a^n = (a^(n/2))^2
```

For odd `n`:

TEXT

```
a^n = a * a^(n-1)
```

A clean implementation:

```
c
long long power(long long a, int n)
{
    if (n == 0)
        return 1;

    long long half = power(a, n / 2);

    if (n % 2 == 0)
        return half * half;

    return a * half * half;
}
```

14. WHY STORE `half`?

Bad version:

C

```
return power(a, n/2) * power(a, n/2);
```

This calculates the same subproblem twice.

Better:

```
c
long long half = power(a, n/2);
return half * half;
```

Only one recursive call.

$O(\log n)$ — Placement-important optimization.

15. FAST POWER COMPLEXITY

Recursive size:

```
TEXT
n !'  n / 2 !'  n / 4 !'  . . .
```

Therefore:

```
TEXT
Time = ~ ( log n )
```

Stack:

```
TEXT
~ ( log n )
```

Compare:

Method	Time	Stack
Basic recursive power	$\sim(n)$	$\sim(n)$
Fast power	$\sim(\log n)$	$\sim(\log n)$

16. GCD — EUCLID'S ALGORITHM

We want:

```
TEXT
gcd(a, b)
```

Key mathematical rule:

```
TEXT
gcd(a, b) = gcd(b, a mod b)
```

Base:

TEXT

$\text{gcd}(a, 0) = a$

Code:

```
c
int gcd(int a, int b)
{
    if (b == 0)
        return a;

    return gcd(b, a % b);
}
```

~~0-1%~~ Very important classical algorithm.

17. TRACE `gcd(48, 18)`

TEXT

`gcd(48, 18)`

```
!' gcd ( 18 , 48 % 18 )
!' gcd ( 18 , 12 )

!' gcd ( 12 , 6 )

!' gcd ( 6 , 0 )

!' 6
```

Answer:

TEXT

6

18. GCD COMPLEXITY

Euclid's algorithm is extremely efficient.

Time:

TEXT

$O(\log \min(a, b))$

approximately.

Recursive stack:

TEXT

$O(\log \min(a, b))$

~~0-1%~~ Useful placement fact.

19. REVERSE AN ARRAY RECURSIVELY

Suppose:

TEXT

[10, 20, 30, 40]

We can swap:

TEXT

f i r s t ! " l a s t

then recurse on inner portion:

TEXT

[20, 30]

Code:

C

```
void reverse(int a[], int left, int right)
{
    if (left >= right)
        return;

    int temp = a[left];
    a[left] = a[right];
    a[right] = temp;

    reverse(a, left + 1, right - 1);
}
```

20. TRACE

Initial:

TEXT

10 20 30 40
^ ^

Swap:

TEXT

40 20 30 10
 ^ ^

Swap:

TEXT

40 30 20 10

Stop when:

```
TEXT
left >= right
```

Complexity:

```
TEXT
Time = ~ ( n )
```

Number of recursive levels:

```
TEXT
"H  n / 2
```

SO:

```
TEXT
Stack = ~ ( n )
```

asymptotically.

21. REVERSE STRING

Because a C string is a character array, the same idea works.

For:

```
TEXT
GATE
```

swap:

```
TEXT
G  !"  E
A  !"  T
```

result:

```
TEXT
ETAG
```

The terminating:

```
C
'\0'
```

must remain at the end.

22. RECURSIVE BINARY SEARCH

Binary search works on a sorted array.

Suppose:

TEXT

[10, 20, 30, 40, 50, 60, 70]

Search 60.

Check middle:

TEXT

40

Since:

TEXT

60 > 40

discard left half.

Search:

TEXT

[50, 60, 70]

23. RECURSIVE BINARY SEARCH CODE

```
C
int binarySearch(int a[], int low, int high, int key)
{
    if (low > high)
        return -1;

    int mid = low + (high - low) / 2;

    if (a[mid] == key)
        return mid;

    if (key < a[mid])
        return binarySearch(a, low, mid - 1, key);

    return binarySearch(a, mid + 1, high, key);
}
```

24. BINARY SEARCH BASE CASE

```
C
if (low > high)
    return -1;
```

means:

Search range is empty.

$O(\log n)$ important.

25. BINARY SEARCH COMPLEXITY

Each call removes roughly half of remaining elements.

TEXT

n
 $n/2$
 $n/4$
 $\dots$
 1

Therefore:

TEXT

$\text{Time} = \sim (\log n)$

Recursive stack:

TEXT

$\sim (\log n)$

Iterative binary search can use:

TEXT

$O(1)$

auxiliary space.

26. FIBONACCI

Definition:

TEXT

$F(0)=0$
 $F(1)=1$
 $F(n)=F(n-1)+F(n-2)$

Sequence:

TEXT

0 1 1 2 3 5 8 13 ...

Naive recursive implementation:

```
c
int fib(int n)
{
    if (n <= 1)
        return n;

    return fib(n - 1) + fib(n - 2);
}
```

27. FIBONACCI RECURSION TREE

For:

```
TEXT
fib(5)
```

you repeatedly calculate:

```
TEXT
fib(3)
fib(2)
```

multiple times.

This repeated work causes inefficiency.

Recurrence:

```
TEXT
T(n)
=
T(n-1)
+
T(n-2)
+
O(1)
```

Time:

```
TEXT
exponential
```

A simple upper bound:

```
TEXT
O(2^n)
```

Stack:

```
TEXT
O(n)
```

28. PLACEMENT LESSON FROM FIBONACCI

Naive recursion teaches an important idea:

Recursion may solve the problem correctly but still repeat subproblems.

Solution later:

TEXT
MEMOIZATION
or
DYNAMIC PROGRAMMING

which can reduce Fibonacci to:

TEXT
 $O(n)$

time.

29. TOWER OF HANOI Ø=Ý%Ø=Ý%Ø=Ý%

This is the classic recursion problem.

We have:

TEXT
Source
Auxiliary
Destination

and n disks.

Rules:

1. Move one disk at a time.
2. Never place a larger disk on a smaller disk.
3. Goal: move all disks from source to destination.

30. STORY FOR 3 DISKS

To move 3 disks:

Step 1

Move top 2 disks:

TEXT
Source !' Auxiliary

Step 2

Move largest disk:

TEXT

S o u r c e ! ' D e s t i n a t i o n

Step 3

Move 2 disks:

TEXT

A u x i l i a r y ! ' D e s t i n a t i o n

Notice:

To solve Hanoi with n , solve Hanoi with $n-1$ twice.

That's recursion.

31. RECURSIVE RELATION

To move n disks:

TEXT

```
1 .   M o v e   n - 1   d i s k s   f r o m   A   ! '   B
2 .   M o v e   l a r g e s t   d i s k   A   ! '   C
3 .   M o v e   n - 1   d i s k s   B   ! '   C
```

So recurrence:

TEXT

```
T(n)
=
T(n-1)
+
1
+
T(n-1)
```

Therefore:

TEXT

$T(n) = 2T(n-1) + 1$

~~$O = M$~~ MUST KNOW.

32. HANOI CODE

C

```
void hanoi(int n, char source, char aux, char dest)
{
    if (n == 0)
        return;
```

```

    hanoi(n - 1, source, dest, aux);

    printf("Move %c to %c\n", source, dest);

    hanoi(n - 1, aux, source, dest);
}

```

33. HANOI FOR 2 DISKS

Moves:

TEXT

```

A  !'  B
A  !'  C
B  !'  C

```

Total:

TEXT

3

Formula:

TEXT

$2^2 - 1 = 3$

34. HANOI FOR 3 DISKS

Minimum moves:

TEXT

$2^3 - 1$
= 7

For 4:

TEXT

$2^4 - 1$
= 15

For n:

FORMULA BOX

TEXT

Minimum Hanoi moves = $2^n - 1$

Memorize.

35. SOLVING THE RECURRENCE

TEXT

$$T(n) = 2T(n-1) + 1$$

Base:

TEXT

$$T(0) = 0$$

Results:

TEXT

$$T(1) = 1$$

$$T(2) = 3$$

$$T(3) = 7$$

$$T(4) = 15$$

Pattern:

TEXT

$$T(n) = 2^n - 1$$

Thus:

TEXT

$$Time = \sim (2^n)$$

Stack depth:

TEXT

$$\sim (n)$$

36. GATE TRAP — TIME VS STACK

Hanoi makes:

TEXT

$$2^n - 1$$

moves.

So time is exponential.

But only one root-to-leaf path is active at once.

Therefore:

TEXT

$$Stack\ Space = \sim (n)$$

Not:

TEXT

$\sim (2^n)$

Ø=Imp important.

37. CLASSICAL RECURSION TABLE

Problem	Recurrence idea	Time	Stack
Factorial	$T(n-1)+1$	$\sim(n)$	$\sim(n)$
Sum 1...n	$T(n-1)+1$	$\sim(n)$	$\sim(n)$
Array sum	$T(n-1)+1$	$\sim(n)$	$\sim(n)$
Basic power	$T(n-1)+1$	$\sim(n)$	$\sim(n)$
Fast power	$T(n/2)+1$	$\sim(\log n)$	$\sim(\log n)$
GCD	shrinking modulo	$O(\log n)$	$O(\log n)$
Binary search	$T(n/2)+1$	$\sim(\log n)$	$\sim(\log n)$
Fibonacci naive	two unequal branches	exponential	$O(n)$
Hanoi	$2T(n-1)+1$	$\sim(2^n)$	$\sim(n)$

Ø=Copy this.

38. HOW TO CREATE A RECURSIVE SOLUTION

Use this template:

TEXT

1. DEFINE the problem for input n
2. FIND smallest/base input
3. SOLVE base directly
4. REDUCE current problem
5. CALL recursion on smaller problem
6. COMBINE smaller answer if necessary
7. VERIFY progress toward base
8. ANALYZE calls + depth

39. GATE TRAP SHEET

Trap 1

Factorial base:

TEXT

$0! = 1$

not zero.

Trap 2

Binary search requires sorted data.

Trap 3

Naive Fibonacci repeats subproblems.

Trap 4

Hanoi time:

TEXT

$\sim (2^n)$

but stack:

TEXT

$\sim (n)$

Trap 5

Fast power should avoid calculating:

c

power(a, n/2)

twice.

Store result.

40. 30-SECOND RECURSION MODULE REVISION

Recursion needs:

TEXT

BASE + PROGRESS

Trace:

TEXT

DOWN calls
UP returns

Complexity:

TEXT

TIME = total call-tree work
SPACE = deepest active path

Important recurrences:

TEXT

$T(n) = T(n - 1) + 1$
 $T(n) = T(n / 2) + 1$
 $T(n) = 2T(n - 1) + 1$
 $T(n) = 2T(n / 2) + n$

$! \sim (n)$
 $! \sim (\log n)$
 $! \sim (2^n)$
 $! \sim (n \log n)$

Hanoi:

TEXT

$2^n - 1$

41. ACTIVE RECALL

Q1

What is factorial base case?

Q2

What recurrence does recursive binary search follow?

Q3

Why is naive Fibonacci slow?

Q4

What is the minimum number of moves for Tower of Hanoi with n disks?

Q5

Time and stack complexity of Hanoi?

ANSWERS

A1

TEXT

$0! = 1$

A2

TEXT

$T(n) = T(n/2) + O(1)$

A3

Because the same Fibonacci subproblems are recursively recalculated many times.

A4

TEXT

$2^n - 1$

A5

TEXT

Time = $\sim (2^n)$
Stack = $\sim (n)$

42. PRACTICE

Q1

Evaluate:

TEXT

fact(6)

Q2

Evaluate:

TEXT

gcd(84, 30)

Q3

For:

TEXT

a = {5, 10, 15, 20}

find recursive array sum.

Q4

What is:

TEXT

power(3, 4)

?

Q5

What is minimum Hanoi moves for:

TEXT

n = 5

Q6

State time and space of recursive binary search.

Q7

Why is:

```
c
return power(a,n/2) * power(a,n/2);
```

worse than storing the recursive result once?

Q8 Ø=pe

How would memoization improve naive Fibonacci?

RECURSION MODULE COMPLETE '

Completed:

- Lesson 17 — Fundamentals
- Lesson 18 — Tracing
- Lesson 19 — Types
- Lesson 20 — Complexity
- Lesson 21 — Classical Problems

We now begin the next official syllabus topic:

ARRAYS

MODULE 2 — LESSON 22

Arrays Fundamentals — Memory, Indexing, Addresses & Complexity

Arrays are one of the most fundamental data structures.

Everything later connects to arrays:

- strings
- matrices
- stacks
- queues
- heaps
- graphs
- searching
- sorting
- dynamic programming

GATE importance: Ø=Ý%ØMÚ%ØKÍØW **Placement importance:** Ø=Ý%ØMÚ%ØKÍØW

1. STORY / INTUITION

Imagine 5 hostel rooms arranged side by side:

TEXT

Room 0
Room 1
Room 2
Room 3
Room 4

Every room stores one integer.

Instead of creating:

C

```
int a;  
int b;  
int c;  
int d;  
int e;
```

we create:

C

```
int arr[5];
```

Now we have one named collection:

TEXT

arr

with positions:

TEXT

0 1 2 3 4

This is an:

Array

2. WHAT IS AN ARRAY?

An array is:

A fixed-size sequence of elements of the same type stored contiguously in memory.

Three words matter:

TEXT

SAME TYPE
FIXED NUMBER OF ELEMENTS
CONTIGUOUS MEMORY

Ø=MUST KNOW.

3. DECLARATION

```
C
int a[5];
```

Meaning:

Create an array named `a` containing 5 integers.

Valid indices:

```
TEXT
0
1
2
3
4
```

Not:

```
TEXT
5
```

4. WHY INDEX STARTS AT ZERO?

Recall:

```
TEXT
a[i] = *(a+i)
```

For first element:

```
TEXT
a[0]
=
*(a+0)
```

No displacement from base address.

So index 0 naturally means:

```
TEXT
0 elements away from beginning
```

5. ARRAY INITIALIZATION


```
C
int a[5] = {10, 20, 30, 40, 50};
```

Memory conceptually:

```
TEXT
Index:
  0      1      2      3      4
+-----+ +-----+ +-----+ +-----+ +-----+
| 10 | | 20 | | 30 | | 40 | | 50 |
+-----+ +-----+ +-----+ +-----+ +-----+
```

6. ACCESS

```
C
a[0]
```

gives:

```
TEXT
10
```

```
C
a[3]
```

gives:

```
TEXT
40
```

7. MODIFY

```
C
a[2] = 100;
```

Array changes:

```
TEXT
10 20 100 40 50
```

Accessing any known index:

```
TEXT
Time = O(1)
```

~~O = One~~ One of array's greatest strengths.

8. WHY RANDOM ACCESS IS $O(1)$

Because array elements are contiguous.

Suppose:

TEXT

Base address = B
Element size = w

Address of:

TEXT

a[i]

can be calculated directly:

FORMULA BOX

TEXT

$LOC(a[i]) = B + i \times w$

No need to visit:

TEXT

a[0]
a[1]
...

first.

9. ADDRESS EXAMPLE

Suppose:

TEXT

Base address = 1000
sizeof(int) = 4

Find:

TEXT

&a[3]

Formula:

TEXT

$1000 + 3 \times 4$
 $= 1012$

Therefore:

```
TEXT
&a[3] = 1012
```

under those assumptions.

10. MEMORY TABLE

For:

```
C
int a[5];
```

Assume base 1000, int size 4.

Element	Address
a[0]	1000
a[1]	1004
a[2]	1008
a[3]	1012
a[4]	1016

This is contiguous storage.

11. ARRAY NAME

For:

```
C
int a[5];
```

in most expressions:

```
TEXT
a
```

decays to:

```
TEXT
&a[0]
```

So:

```
c
int *p = a;
```

means:

```
TEXT
p != a[0]
```

We already learned this during pointer lessons.

12. MASTER ARRAY RELATIONS

Memorize:

```
TEXT
a           != address of first element in most expressions
a + i       != address of a[i]
* ( a + i ) != value of a[i]
a [ i ]     != * ( a + i )
& a [ i ]   != a + i
```

13. ARRAY LENGTH

For a real local array:

```
c
int a[10];
```

you can calculate:

```
c
sizeof(a) / sizeof(a[0])
```

Assume:

```
TEXT
sizeof(int)=4
```

Then:

```
TEXT
sizeof(a) = 40
sizeof(a[0]) = 4

40/4 = 10
```

14. GATE TRAP — FUNCTION PARAMETER

Inside:

```
C
void f(int a[])
```

the parameter is adjusted to a pointer.

So inside `f`:

```
C
sizeof(a)
```

means pointer size, not size of caller's entire array.

~~0-MUST KNOW.~~

Therefore array size is commonly passed separately:

```
C
void f(int a[], int n)
```

15. PARTIAL INITIALIZATION

```
C
int a[5] = {10, 20};
```

What happens?

Remaining elements are zero-initialized:

```
TEXT
10 20 0 0 0
```

~~0-Must know~~ Important C fact.

16. ALL ZERO INITIALIZATION

Common shortcut:

```
C
int a[5] = {0};
```

Result:

```
TEXT
0 0 0 0 0
```

Useful practical pattern.

17. SIZE INFERENCE

```
C
int a[] = {10,20,30,40};
```

Compiler infers:

```
TEXT
size = 4
```

So equivalent conceptually to:

```
C
int a[4] = {10,20,30,40};
```

18. OUT-OF-BOUNDS ACCESS

For:

```
C
int a[5];
```

valid:

```
TEXT
a[0] ... a[4]
```

This is invalid:

```
C
a[5]
```

C generally does not automatically perform runtime bounds checking.

Accessing beyond array bounds gives:

```
TEXT
undefined behavior
```

Very important.

19. STORY

Array is a hotel with rooms:

TEXT

0 1 2 3 4

Trying:

TEXT

room 5

means leaving the hotel reservation and entering unrelated memory.

C won't necessarily stop you automatically.

That's why array bounds bugs are dangerous.

20. ARRAY OPERATIONS

Typical array operations:

- access
- update
- traversal
- search
- insertion
- deletion

Their complexities differ.

21. ACCESS

C

```
x = a[i];
```

Time:

TEXT

$O(1)$

because direct address formula exists.

22. UPDATE

C

```
a[i] = x;
```

Time:

TEXT

$O(1)$

23. TRAVERSAL

```
c
for (int i = 0; i < n; i++)
{
    printf("%d ", a[i]);
}
```

Visits n elements.

Time:

TEXT

$O(n)$

Auxiliary space:

TEXT

$O(1)$

24. LINEAR SEARCH

Search for key:

```
c
for (int i = 0; i < n; i++)
{
    if (a[i] == key)
        return i;
}
```

Worst case:

TEXT

key absent
or
key at last index

Need to inspect all n .

Worst-case time:

TEXT

$O(n)$

Best case:

TEXT

$O(1)$

25. INSERTION AT END

Suppose array has available capacity.

TEXT

10 20 30 _ _

Insert 40 at end:

TEXT

10 20 30 40 _

If we know current length:

TEXT

$O(1)$

assuming capacity exists.

26. INSERTION AT BEGINNING

Before:

TEXT

10 20 30 40

Insert 5 at beginning.

Need to shift:

TEXT

4	0	!
3	0	!
2	0	!
1	0	!

Result:

TEXT

5 10 20 30 40

Time:

TEXT

$O(n)$

27. INSERTION IN MIDDLE

Suppose:

```
TEXT
10 20 30 40
```

Insert 25 at index 2.

Shift:

```
TEXT
4 0 !' r i g h t
3 0 !' r i g h t
```

Then:

```
TEXT
10 20 25 30 40
```

Worst-case:

```
TEXT
O(n)
```

28. DELETION FROM BEGINNING

Before:

```
TEXT
10 20 30 40
```

Delete 10.

Need to shift:

```
TEXT
2 0 !' i n d e x 0
3 0 !' i n d e x 1
4 0 !' i n d e x 2
```

Time:

```
TEXT
O(n)
```

29. DELETION FROM END

If logical length is known:

TEXT

10 20 30 40

remove last logical element.

Usually:

TEXT

$O(1)$

because no shifting required.

30. MASTER ARRAY COMPLEXITY TABLE

Operation	Time
Access by index	$O(1)$
Update	$O(1)$
Traverse	$O(n)$
Linear search	$O(n)$ worst
Insert beginning	$O(n)$
Insert middle	$O(n)$
Insert end with space	$O(1)$
Delete beginning	$O(n)$
Delete middle	$O(n)$
Delete end	$O(1)$

Copy this.

31. WHY ARRAY INSERTION IS EXPENSIVE

Not because finding an index is difficult.

The expensive part is:

TEXT

SHIFTING ELEMENTS

This distinction becomes important when comparing:

TEXT

Array vs Linked List

later.

32. ARRAY ADVANTAGES

Arrays provide:

1. Fast random access

TEXT

$O(1)$

2. Good cache locality

Elements are contiguous.

3. Low per-element overhead

No extra pointer required between elements.

4. Simple representation

33. ARRAY DISADVANTAGES

- fixed size for ordinary static array
 - insertion/deletion can require shifting
 - needs contiguous memory
 - resizing ordinary arrays is not automatic
 - C has no built-in bounds checking
-

34. ARRAY VS LINKED LIST PREVIEW

Property	Array	Linked List
Storage	contiguous	nodes can be separate
Random access	$O(1)$	$O(n)$
Insert front	$O(n)$	potentially $O(1)$
Extra pointers	no	yes
Cache locality	strong	generally weaker

We'll study this deeply later.

35. FIXED SIZE VS DYNAMIC ARRAY

Ordinary:

```
C
int a[100];
```

has a fixed declared size.

Dynamic:

```
C
int *a = malloc(n * sizeof *a);
```

can allocate based on runtime `n`.

But dynamically allocated array is still:

```
TEXT
CONTIGUOUS MEMORY
```

for those `n` elements.

36. ARRAY MEMORY DOES NOT STORE ITS LENGTH AUTOMATICALLY FOR POINTER USE

If you have:

```
C
int *p;
```

C doesn't inherently know:

```
TEXT
how many integers are valid behind p
```

That information must be tracked separately.

Example:

```
C
void printArray(int a[], int n)
```

requires `n`.

0 = 16 important systems concept.

37. GATE ADDRESS FORMULA WITH LOWER BOUND

General theoretical array:

```
TEXT
A[L ... U]
```

If:

TEXT

Base = address of A[L]
w = element size

then:

FORMULA BOX

TEXT

$\text{LOC}(A[i])$
=
 $\text{Base} + (i-L) \times w$

For C:

TEXT

$L = 0$

So:

TEXT

$\text{LOC}(A[i])$
=
 $\text{Base} + i \times w$

38. GATE EXAMPLE

Suppose theoretical array:

TEXT

A[5...15]

Base address of A[5] :

TEXT

1000

Each element:

TEXT

4 bytes

Find address of:

TEXT

A[9]

Formula:

TEXT

$1000 + (9-5) \times 4$

TEXT

$= 1000 + 16$
 $= 1016$

Answer:

TEXT

1016

39. NUMBER OF ELEMENTS WITH GENERAL BOUNDS

For:

TEXT

$A[L \dots U]$

number of elements:

FORMULA BOX

TEXT

$U - L + 1$

Example:

TEXT

$A[5 \dots 15]$

contains:

TEXT

$15 - 5 + 1$
 $= 11$

40. ARRAYS & POINTERS — QUICK CONSOLIDATION

Suppose:

C

```
int a[] = {10, 20, 30, 40};  
int *p = a;
```

Then:

TEXT

```
*p      = 10
*(p+1)  = 20
p[2]    = 30
*(a+3)  = 40
```

And:

TEXT

p+2

points to:

TEXT

a[2]

41. a VS &a

For:

c

```
int a[5];
```

Remember:

TEXT

a+1

moves one integer.

But:

TEXT

&a+1

moves one entire array.

Assuming:

TEXT

```
sizeof(int)=4
```

one array contains:

TEXT

20 bytes

So from conceptual address 1000:

TEXT

```
a + 1      !'    1 0 0 4
& a + 1    !'    1 0 2 0
```

Ø=0% CTE classic.

42. ARRAY ASSIGNMENT

Suppose:

```
C
int a[3] = {1,2,3};
int b[3];
```

Can we write:

```
C
b = a;
```

No.

Ordinary arrays cannot be assigned directly like structures.

You must copy elements individually or use appropriate library operations when applicable.

43. ARRAY PASSED TO FUNCTION

```
C
void change(int a[])
{
    a[0] = 100;
}
```

Caller:

```
C
int x[] = {10,20,30};

change(x);
```

After call:

```
TEXT
x[0] = 100
```

Why?

Array argument decays to pointer to its first element.

C still passes:

TEXT

pointer value by value

not the whole array by reference as a special mechanism.

44. GATE TRAP — sizeof

C

```
int a[10];
```

Assume:

TEXT

```
sizeof(int)=4  
pointer size=8
```

In same scope:

TEXT

```
sizeof(a)=40
```

But:

C

```
void f(int a[])  
{  
    sizeof(a)  
}
```

inside f:

TEXT

8

under given pointer-size assumption.

0-8% extremely common.

45. GATE TRAP — PARTIAL INITIALIZATION

C

```
int a[5] = {1,2};
```

Values:

TEXT

```
1 2 0 0 0
```

Do not say remaining values are garbage.

Initialization rules zero-initialize the rest.

46. BUT UNINITIALIZED AUTOMATIC ARRAY

Different:

```
C
int a[5];
```

inside an ordinary function/block with automatic storage.

Elements have indeterminate values until assigned.

0=0 DON'T CONFUSE.

47. COMPARE

Partial initialization

```
C
int a[5] = {1,2};
```

Result:

```
TEXT
1 2 0 0 0
```

No initialization

```
C
int a[5];
```

Automatic local elements:

```
TEXT
indeterminate
```

48. PLACEMENT CONNECTION

Arrays are the foundation for high-frequency patterns:

- two pointers
- sliding window
- prefix sum
- frequency arrays
- binary search
- Kadane's algorithm
- sorting

- matrix traversal

We will learn these after core array memory and operations.

49. INTERVIEW QUESTIONS

Q1

Why is array random access $O(1)$?

Because the element address is computed directly using base address + index $\times$ element size.

Q2

Why is insertion at beginning $O(n)$?

Existing elements must be shifted.

Q3

Are C arrays contiguous?

Yes, array elements are contiguous.

Q4

Can ordinary arrays be assigned directly with `=` ?

No.

Q5

What happens when an array is passed to a function?

The array expression is converted to a pointer to its first element in that context.

Q6

Does an array automatically remember its length after decay to a pointer?

No.

50. MEMORY TRICKS

Array

Same type, side by side.

Random access

Address formula !' directly jump.

Insert/delete

Shifting is the enemy.

Pointer relation

```
TEXT
INDEX
=
MOVE + DEREFERENCE
```

```
TEXT
a[i] = *(a+i)
```

51. NOTEBOOK NOTES

Declaration

```
C
int a[5];
```

Indices:

```
TEXT
0 to 4
```

Address formula

```
TEXT
LOC(A[i])
=
Base + i×w
```

for zero-based arrays.

General:

```
TEXT
Base + (i-L)×w
```

Key properties

```
TEXT
same type
contiguous
fixed number of elements
O(1) random access
```

Complexity

TEXT

Access	$O(1)$
Update	$O(1)$
Traversal	$O(n)$
Search	$O(n)$
Insert front	$O(n)$
Delete front	$O(n)$

52. 30-SECOND REVISION

Remember:

1. Array elements have same type.
2. Elements are contiguous.
3. C indices begin at 0.
4. `a[i] = *(a+i)`.
5. Address calculation makes access $O(1)$.
6. Traversal is $O(n)$.
7. Insert/delete middle usually requires shifting.
8. Out-of-bounds access is undefined behavior.
9. Partial initialization zero-fills remaining elements.
10. Array parameter behaves as pointer parameter.

53. ACTIVE RECALL

Q1

For:

```
C
int a[5];
```

what are valid indices?

Q2

If base=1000 and int size=4, address of `a[4]` ?

Q3

Why is accessing `a[i]` $O(1)$?

Q4

Complexity of inserting at beginning?

Q5

What is:

TEXT

a[i]

equivalent to using pointer notation?

ANSWERS

A1

TEXT

0, 1, 2, 3, 4

A2

TEXT

1000 + 4×4
= 1016

A3

Because its address is calculated directly.

A4

TEXT

O(n)

A5

C

*(a+i)

54. PRACTICE

Q1

Given:

C

```
int a[] = {5, 10, 15, 20};
```

find:

TEXT

a[0]
a[3]
*(a+2)

Q2

Assume:

TEXT

```
&a[0] = 2000  
sizeof(int)=4
```

Find address of:

TEXT

```
a[6]
```

~~Q3~~ Q3

For:

C

```
int a[6] = {1,2};
```

write all six initial values.

~~Q4~~ Q4

Why is:

C

```
a[6]
```

invalid for:

C

```
int a[6];
```

?

~~Q5~~ Q5

Assume:

TEXT

```
sizeof(int)=4  
pointer size=8
```

For:

C

```
int a[10];  
int *p=a;
```

find:

TEXT

```
sizeof(a)  
sizeof(p)
```


Q=Q6

Why can:

```
c
void f(int a[])
{
    a[0] = 100;
}
```

modify the caller's array?

Q=Q7 — GATE ADDRESS

An array is declared conceptually as:

```
TEXT
A[10...30]
```

Base address of `A[10]` = 5000 .

Each element occupies 8 bytes.

Find address of:

```
TEXT
A[18]
```

Q=Q8 — PLACEMENT THINKING

Why are arrays usually better than linked lists for random indexing such as:

```
TEXT
give me element number 500
```

?

PROGRESS

MODULE 1 — RECURSION ' COMPLETE

Lessons 17–21 completed.

MODULE 2 — ARRAYS

' Lesson 22 — Fundamentals & Memory

NEXT — LESSON 23

1D Array Operations

Next we will implement and analyze:

- traversal

- insertion at beginning/middle/end
- deletion
- linear search
- maximum/minimum
- second largest
- reversing
- copying
- frequency counting basics
- duplicate handling intuition
- operation-by-operation complexity

Then:

Lesson 24 — 2D Arrays & Matrix Memory

including:

TEXT

```
a[i][j]
```

memory layout, rows, columns, matrix traversal, and preparation for the crucial **row-major/column-major GATE address formulas**.

USER

NEXT

ASSISTANT

MODULE 2 — LESSON 23

1D Array Operations — Traversal, Insertion, Deletion, Search, Reverse & Core Placement Patterns

We now continue the **Arrays** module.

[illegible]

Today we focus on actual operations performed on a one-dimensional array and their complexities.

1. WHERE WE ARE

Completed:

- C foundation
- pointers
- recursion
- array fundamentals and memory

Now:

1D Array Operations

We will cover:

- traversal
- reading/writing
- insertion
- deletion
- linear search
- maximum/minimum
- reversing
- copying
- second-largest
- frequency basics
- duplicates intuition
- complexity table
- placement

USER

NEXT

ASSISTANT

MODULE 2 — LESSON 23

1D Array Operations — Traversal, Insertion, Deletion, Search, Reverse & Core Placement Patterns

Continuing from exactly where Lesson 23 stopped.

GATE importance: Ø=Ý%ØMÚSÐKÍÓW **Placement importance:** Ø=Ý%ØMÚSÐKÍÓW

2. TRAVERSAL

Traversal means visiting every array element one by one.

Example:

```
C
int a[5] = {10, 20, 30, 40, 50};

for (int i = 0; i < 5; i++)
{
    printf("%d ", a[i]);
}
```

Output:

```
TEXT
10 20 30 40 50
```

Number of elements visited:

```
TEXT
n
```

Therefore:

```
TEXT
Time =  $O(n)$ 
Auxiliary Space =  $O(1)$ 
```

3. STORY / INTUITION

Imagine 5 rooms:

```
TEXT
[10] [20] [30] [40] [50]
```

Traversal means:

```
TEXT
visit room 0
visit room 1
visit room 2
visit room 3
visit room 4
```

You cannot inspect every element without visiting all `n` positions.

So:

```
TEXT
 $O(n)$ 
```

4. READING ARRAY INPUT

```
C
int a[5];

for (int i = 0; i < 5; i++)
{
    scanf("%d", &a[i]);
}
```

Remember:

TEXT

```
a [ i ]           !'   v a l u e
& a [ i ]         !'   a d d r e s s
```

scanf needs the address.

5. SUM OF ARRAY

C

```
int sum = 0;

for (int i = 0; i < n; i++)
{
    sum += a[i];
}
```

Example:

TEXT

10 20 30 40

Trace:

TEXT

sum = 0

+

USER

NEXT

ASSISTANT

MODULE 2 — LESSON 23

1D Array Operations — Traversal, Insertion, Deletion, Search, Reverse & Core Placement Patterns

Continuing exactly from the interrupted point.

5. SUM OF ARRAY

C

```
int sum = 0;
```

```
for (int i = 0; i < n; i++)
{
    sum += a[i];
}
```

Example:

TEXT
10 20 30 40

Trace:

TEXT
sum = 0
sum = 10
sum = 30
sum = 60
sum = 100

Therefore:

TEXT
Time = $O(n)$
Auxiliary Space = $O(1)$

6. FIND MAXIMUM ELEMENT

Suppose:

TEXT
8 3 15 6 10

We should not start with:

C
int max = 0;

because the array could contain only negative numbers.

Example:

TEXT
-10 -4 -20

If `max = 0`, answer would incorrectly remain 0.

~~0 = Better~~
Better:

C
int max = a[0];

for (int i = 1; i < n; i++)
{

```
    if (a[i] > max)
    {
        max = a[i];
    }
}
```

7. TRACE MAXIMUM

Array:

```
TEXT
8 3 15 6 10
```

Initially:

```
TEXT
max = 8
```

Compare:

```
TEXT
3 > 8?   No
1 5 > 8?   Y e s   ! '   m a x   =   1 5
6 > 15?  No
10 > 15? No
```

Answer:

```
TEXT
15
```

Complexity:

```
TEXT
Time = O(n)
Space = O(1)
```

8. FIND MINIMUM

Same idea:

```
C
int min = a[0];

for (int i = 1; i < n; i++)
{
    if (a[i] < min)
    {
        min = a[i];
    }
}
```

Complexity:

```
TEXT
0(n)
```

9. GATE / INTERVIEW TRAP — INITIAL VALUE

Bad:

```
C
int max = 0;
```

Good:

```
C
int max = a[0];
```

provided:

```
TEXT
n > 0
```

Similarly:

```
C
int min = a[0];
```

~~0 = 0~~ Always think about negative values.

10. LINEAR SEARCH

Suppose:

```
TEXT
a = [10, 40, 20, 70, 50]
key = 70
```

Start from first element and compare sequentially.

```
C
int linearSearch(int a[], int n, int key)
{
    for (int i = 0; i < n; i++)
    {
        if (a[i] == key)
            return i;
    }

    return -1;
}
```


Result:

TEXT
index = 3

11. LINEAR SEARCH COMPLEXITY

Best case

Key is first:

TEXT
 $O(1)$

Worst case

Key is last or absent:

TEXT
 $O(n)$

Average case

TEXT
 $O(n)$

Auxiliary space:

TEXT
 $O(1)$

~~$O(n)$~~ Linear search does **not require sorting**.

12. SEARCH STORY

Imagine checking student roll numbers from top to bottom.

If target is first:

TEXT
1 comparison

If target is last:

TEXT
n comparisons

Hence worst case:

TEXT

$O(n)$

13. INSERTION — CORE IDEA

Insertion in an array has two separate questions:

1. Is there enough capacity?
2. How many elements must move?

Suppose capacity is 10 but currently:

TEXT

$n = 5$

Array:

TEXT

10 20 30 40 50 _ _ _ _

We have free space.

14. INSERT AT END

Insert 60.

C

```
a[n] = 60;  
n++;
```

Result:

TEXT

10 20 30 40 50 60

No shifting.

Therefore:

TEXT

Time = $O(1)$

assuming available capacity and current length known.

15. INSERT AT BEGINNING

Before:

TEXT

10 20 30 40

Insert:

TEXT

5

We first shift everything one position right.

Important:

Shift from RIGHT TO LEFT.

Why?

Because otherwise we overwrite values.

16. CORRECT SHIFT

```
c
for (int i = n; i > 0; i--)
{
    a[i] = a[i - 1];
}

a[0] = value;
n++;
```

Example:

TEXT

Before:

10 20 30 40 _

Shift:

4 0 !'
3 0 !'
2 0 !'
1 0 !'

After:

5 10 20 30 40

Complexity:

TEXT

$O(n)$

17. WHY SHIFT RIGHT-TO-LEFT?

Suppose:

TEXT

10 20 30

If you incorrectly shift left-to-right:

C

```
a[1] = a[0];  
a[2] = a[1];
```

After first assignment:

TEXT

10 10 30

Then:

C

```
a[2] = a[1];
```

copies the already changed 10.

Result:

TEXT

10 10 10

Original information was destroyed.

0-100% right shifting:

Move from right to left.

18. INSERT AT POSITION

Suppose:

TEXT

10 20 30 40

Insert:

TEXT

25

at index:

TEXT

2

Desired:

TEXT

10 20 25 30 40

Code:

```
c
for (int i = n; i > pos; i--)
{
    a[i] = a[i - 1];
}

a[pos] = value;
n++;
```

19. TRACE INSERTION

Before:

TEXT

Index: 0 1 2 3
10 20 30 40

pos = 2.

Shift:

TEXT

a [4] = a [3] !' 4 0
a [3] = a [2] !' 3 0

Then:

TEXT

a[2] = 25

Result:

TEXT

10 20 25 30 40

20. INSERTION COMPLEXITY

If inserting at index pos, number of elements shifted:

TEXT

n - pos

Therefore:

Best case

Insert at end:

TEXT
 $O(1)$

Worst case

Insert at beginning:

TEXT
 $O(n)$

FORMULA BOX

For insertion at index i into an array with n existing elements:

TEXT
Elements shifted = $n - i$

assuming sufficient capacity.

21. DELETION — CORE IDEA

Suppose:

TEXT
10 20 30 40 50

Delete the element at index:

TEXT
2

which is:

TEXT
30

After removing it, we cannot leave a hole if we want the logical array contiguous:

TEXT
10 20 _ 40 50

So elements after it shift left.

22. DELETE AT POSITION

```

c
for (int i = pos; i < n - 1; i++)
{
    a[i] = a[i + 1];
}

n--;

```

Example:

TEXT

Before:

10 20 30 40 50

Delete index 2:

TEXT

a [2]	=	a [3]	!	4 0
a [3]	=	a [4]	!	5 0

Logical result:

TEXT

10 20 40 50

23. WHY SHIFT LEFT-TO-RIGHT?

For deletion, we copy later elements into earlier positions:

TEXT

a[i] = a[i+1]

So processing naturally goes:

TEXT

L E F T ! ' R I G H T

Memory trick:

TEXT

I N S E R T	!	s h i f t	R I G H T	!	t r a v e r s e	b a c k w a r d s
D E L E T E	!	s h i f t	L E F T	!	t r a v e r s e	f o r w a r d s

Very useful.

24. DELETION COMPLEXITY

Deleting at index `pos` shifts:

TEXT

$n - \text{pos} - 1$

elements.

So:

Delete beginning

TEXT

$O(n)$

Delete middle

TEXT

$O(n)$

Delete last element

TEXT

$O(1)$

FORMULA BOX

TEXT

Insertion shifts = $n - \text{pos}$

Deletion shifts = $n - \text{pos} - 1$

25. DELETE BY VALUE

Suppose:

TEXT

10 20 30 40

Delete value:

TEXT

30

First search for it.

Search:

TEXT

$O(n)$

Then shift:

TEXT

$O(n)$

Total:

TEXT

$O(n) + O(n)$
 $= O(n)$

~~$O = \times$~~ Sequential linear operations add, not multiply.

26. REVERSING AN ARRAY

Suppose:

TEXT

10 20 30 40 50

Desired:

TEXT

50 40 30 20 10

Naive idea:

Create another array.

But we can reverse **in-place** with two pointers/indices.

27. TWO-POINTER IDEA

Set:

TEXT

```
left = 0
right = n-1
```

Swap:

TEXT

```
a[ left ] ! " a[ right ]
```

Then:

TEXT

```
left++
right--
```

Continue while:

TEXT

left < right

28. C IMPLEMENTATION

```
c
void reverse(int a[], int n)
{
    int left = 0;
    int right = n - 1;

    while (left < right)
    {
        int temp = a[left];
        a[left] = a[right];
        a[right] = temp;

        left++;
        right--;
    }
}
```

29. TRACE REVERSE

Array:

TEXT

1 2 3 4 5

First:

TEXT

1 !" 5

becomes:

TEXT

5 2 3 4 1

Then:

TEXT

2 !" 4

becomes:

TEXT

5 4 3 2 1

Stop.

30. REVERSE COMPLEXITY

Number of swaps:

```
TEXT
floor(n/2)
```

Therefore:

```
TEXT
Time = O(n)
Auxiliary Space = O(1)
```

~~O=1/4~~ In-place reversal is a classic placement pattern.

31. COPY AN ARRAY

```
C
for (int i = 0; i < n; i++)
{
    b[i] = a[i];
}
```

Time:

```
TEXT
O(n)
```

Extra storage for destination:

```
TEXT
O(n)
```

Remember:

```
C
b = a;
```

is not valid ordinary array assignment in C.

32. FIND SECOND LARGEST — IMPORTANT PLACEMENT PROBLEM

Suppose:

```
TEXT
12 35 1 10 34 1
```

Largest:

TEXT

35

Second largest distinct:

TEXT

34

A simple efficient method uses two variables.

33. IDEA

Track:

TEXT

largest
second

Whenever a new largest appears:

TEXT

second = largest
largest = current

Otherwise, if current is between largest and second:

TEXT

second = current

34. SIMPLE IMPLEMENTATION

Assume at least two distinct values exist.

```
C
#include <limits.h>

int largest = INT_MIN;
int second = INT_MIN;

for (int i = 0; i < n; i++)
{
    if (a[i] > largest)
    {
        second = largest;
        largest = a[i];
    }
    else if (a[i] > second && a[i] < largest)
    {
        second = a[i];
    }
}
```

Time:

TEXT

$O(n)$

Space:

TEXT

$O(1)$

35. DON'T CONFUSE — SECOND LARGEST

For:

TEXT

5 5 4

If question asks:

second largest distinct element

answer:

TEXT

4

If duplicates are allowed as ranking positions, interpretation could differ.

0-1 Always read wording carefully.

36. FIND WHETHER ARRAY IS SORTED

For ascending order:

```
c
int sorted = 1;

for (int i = 1; i < n; i++)
{
    if (a[i] < a[i - 1])
    {
        sorted = 0;
        break;
    }
}
```

Why start from:

TEXT

$i = 1$

?

Because we compare:

TEXT
a[i] with a[i-1]

and index -1 is invalid.

37. COMPLEXITY

Best case:

First inversion found immediately:

TEXT
 $O(1)$

Worst case:

Array sorted:

TEXT
 $O(n)$

Auxiliary:

TEXT
 $O(1)$

38. COUNT OCCURRENCES

Suppose:

TEXT
2 3 2 5 2

Count 2.

```
C
int count = 0;

for (int i = 0; i < n; i++)
{
    if (a[i] == key)
        count++;
}
```

Result:

TEXT
3

Time:

TEXT
 $O(n)$

39. FREQUENCY ARRAY — PLACEMENT FOUNDATION

Suppose values are restricted to:

TEXT
0 to 9

Array:

TEXT
2 3 2 5 3 2

Create:

C
`int freq[10] = {0};`

Then:

C
`for (int i = 0; i < n; i++)
{
 freq[a[i]]++;
}`

After processing:

TEXT
`freq[2] = 3
freq[3] = 2
freq[5] = 1`

40. WHY THIS WORKS

We use the value itself as an index.

Example:

TEXT
`value = 3`

then:

```
c
freq[3]++;
```

This records how many 3 s occurred.

41. FREQUENCY COMPLEXITY

If value range is size k :

```
TEXT
Time =  $O(n)$ 
Extra Space =  $O(k)$ 
```

If k is fixed tiny constant like 10:

```
TEXT
 $O(1)$ 
```

auxiliary space asymptotically.

But if k grows with input:

```
TEXT
 $O(k)$ 
```

~~$O(k)$~~ Always define your variable range.

42. FREQUENCY ARRAY LIMITATION

Suppose values can be:

```
TEXT
-1000000000
+1000000000
```

You cannot realistically create a huge direct frequency array.

So frequency arrays work best when value range is:

```
TEXT
small and bounded
```

Later placement solutions may use maps/hashing for arbitrary values.

43. REMOVE DUPLICATES FROM SORTED ARRAY — PREVIEW

Suppose:

TEXT

1 1 2 2 2 3 4 4

Since it is sorted, duplicates are adjacent.

We can use two indices:

TEXT

slow
fast

Result logically becomes:

TEXT

1 2 3 4

This is a classic two-pointer problem.

We'll study it properly in Lesson 28.

44. ROTATION — PREVIEW

Array:

TEXT

1 2 3 4 5

Left rotate once:

TEXT

2 3 4 5 1

Right rotate once:

TEXT

5 1 2 3 4

Rotations appear often in placements.

For now just recognize the operation.

45. PREFIX SUM — PREVIEW

Suppose:

TEXT

a = 2 4 6 8

Prefix sums:

TEXT

2 6 12 20

Meaning:

TEXT

```
prefix[0] = 2
prefix[1] = 2+4 = 6
prefix[2] = 2+4+6 = 12
prefix[3] = 20
```

This lets many range-sum queries become very fast.

We'll study it later in placement-array patterns.

46. IMPORTANT ARRAY OPERATION COMPARISON

Array:

TEXT

[10][20][30][40][50]

Want element 4?

Direct address:

TEXT

0(1)

Want value 37?

Without special structure, search:

TEXT

0(n)

Want insert at beginning?

Shift:

TEXT

0(n)

Want append with capacity?

TEXT

0(1)

This is the core performance personality of arrays.

47. GATE TRAP — ACCESS VS SEARCH

Do not confuse:

TEXT

Access `a[500]`

with:

TEXT

Find where value 500 occurs

Access by index

TEXT

$O(1)$

Search by value

TEXT

$O(n)$

in an unsorted array.

Very common conceptual distinction.

48. GATE TRAP — SORTED ARRAY SEARCH

If array is sorted, we can use:

TEXT

Binary Search

giving:

TEXT

$O(\log n)$

instead of linear search.

But ordinary unsorted array:

TEXT

$O(n)$

We cover binary search fully in Lesson 27.

49. GATE TRAP — INSERTION INTO SORTED ARRAY

Suppose a sorted array has enough capacity.

Finding insertion position with binary search might take:

TEXT
 $O(\log n)$

But then shifting elements still costs:

TEXT
 $O(n)$

Therefore total:

TEXT
 $O(\log n) + O(n)$
 $= O(n)$

~~$O = \sqrt{O}$~~ Very important.

Faster search does not eliminate shifting cost.

50. GATE TRAP — DELETE AFTER SEARCH

Similarly, in sorted array:

TEXT
find target !' $O(\log n)$
shift elements !' $O(n)$

Total:

TEXT
 $O(n)$

51. ARRAY OPERATION MASTER TABLE

Operation	Best	Worst
Index access	$O(1)$	$O(1)$
Update index	$O(1)$	$O(1)$
Traverse	$O(n)$	$O(n)$
Linear search	$O(1)$	$O(n)$
Insert end with capacity	$O(1)$	$O(1)$
Insert beginning	$O(n)$	$O(n)$
Insert arbitrary position	$O(1)$ near end	$O(n)$
Delete end	$O(1)$	$O(1)$

Operation	Best	Worst
Delete beginning	$O(n)$	$O(n)$
Reverse	$O(n)$	$O(n)$
Find max/min	$O(n)$	$O(n)$

52. WHY MAXIMUM REQUIRES $O(n)$

For an unsorted array:

TEXT

5 2 100 4 7

Could you safely decide maximum without examining every element?

No.

An unseen element could always be larger.

Therefore comparison-based maximum finding requires:

TEXT

$n - 1$ comparisons

in the standard model.

~~$O = \Omega$~~ Exact fact:

FORMULA BOX

TEXT

Minimum comparisons to find maximum among n elements = $n - 1$

53. FIND MIN AND MAX SEPARATELY

Naive:

TEXT

Find max !' $n - 1$ comparisons
Find min !' $n - 1$ comparisons

Total:

TEXT

$2n - 2$ comparisons

There are optimized pairwise methods that reduce this to approximately:

TEXT

$3n/2$

comparisons.

Ø=Useful GATE concept; we can revisit in algorithms.

54. REVERSE — EXACT SWAPS

For n elements:

TEXT

number of swaps = floor($n/2$)

Examples:

TEXT

```
n = 5 !' 2 s w a p s
n = 6 !' 3 s w a p s
```

Still:

TEXT

$\sim (n)$

55. IN-PLACE ALGORITHM

An algorithm is often called **in-place** when it uses only a small constant amount of extra storage relative to input.

Our reverse:

C

```
int temp;
```

uses constant extra memory.

Therefore:

TEXT

Auxiliary Space = $O(1)$

and is considered in-place.

Ø=Placement terminology.

56. PLACEMENT PATTERN — TWO POINTERS

Our reversal algorithm already introduced:

TEXT

```
l e f t ! '   b e g i n n i n g
r i g h t ! '   e n d
```

Then:

TEXT

```
move both inward
```

This same pattern appears in:

- palindrome check
- sorted two-sum
- remove duplicates
- partitioning
- container problems
- merging-style scans

So this simple reverse is actually introducing a major interview technique.

57. SIMPLE PALINDROME ARRAY CHECK

Example:

TEXT

```
1 2 3 2 1
```

Check:

TEXT

```
first == last?
second == second-last?
```

Code:

```
c
int isPalindrome(int a[], int n)
{
    int left = 0;
    int right = n - 1;

    while (left < right)
    {
        if (a[left] != a[right])
            return 0;

        left++;
        right--;
    }

    return 1;
}
```

Time:

TEXT

$O(n)$

Space:

TEXT

$O(1)$

~~$O(1)$~~ Placement extension.

58. COMMON CODING BUG — WRONG LOOP LIMIT

Suppose:

C

```
for (int i = 0; i <= n; i++)
```

for array of `n` elements.

This accesses:

TEXT

`a[n]`

which is out of bounds.

Correct:

C

```
i < n
```

~~$O(1)$~~ Off-by-one errors are among the most common array bugs.

59. COMMON BUG — EMPTY ARRAY

Code:

C

```
int max = a[0];
```

assumes:

TEXT

`n > 0`

If:

TEXT

`n = 0`

then `a[0]` is invalid.

Real-world functions should establish a precondition or explicitly handle empty input.

60. COMMON BUG — INSERT WITHOUT CAPACITY

Suppose:

```
C
int a[5] = {1, 2, 3, 4, 5};
```

and current logical size is already:

```
TEXT
5
```

You cannot safely insert a sixth element into this fixed array.

Need:

- larger storage, or
- dynamic reallocation, or
- reject insertion.

Capacity and logical size are different concepts.

61. SIZE VS CAPACITY

Suppose:

```
C
int a[100];
```

but only 20 positions are currently used.

Then:

```
TEXT
Logical size = 20
Capacity = 100
```

Appending at:

```
TEXT
a[20]
```

is possible.

This distinction later becomes essential for dynamic arrays/vectors.

62. GATE FOCUS

Ø=MUST KNOW:

- array traversal
- random access $O(1)$
- linear search
- insertion shifting
- deletion shifting
- max/min
- reverse
- exact shift counts
- size vs capacity
- sorted-array insertion still $O(n)$
- $n-1$ comparisons for maximum

63. INTERVIEW QUESTIONS

Q1. Why is array insertion at beginning $O(n)$?

Because existing elements must shift right.

Q2. Why is deletion at end $O(1)$?

No elements after it need shifting.

Q3. How do you reverse an array in-place?

Swap first/last elements while moving two indices inward.

Q4. Complexity of finding maximum?

$O(n)$ time, $O(1)$ auxiliary space.

Q5. Difference between array size and capacity?

Size = number of logically stored elements; capacity = total available slots.

Q6. Why can linear search work on unsorted arrays?

It simply inspects elements one by one and requires no ordering assumption.

64. MEMORY TRICKS

Insert

Make room first.

Shift right.

Delete

Close the gap.

Shift left.

Reverse

Ends exchange and walk inward.

Search

Unsorted? Walk through.

Maximum

Champion fights every challenger.

Needs $n-1$ comparisons.

65. NOTEBOOK NOTES

Traversal

```
C
for (int i = 0; i < n; i++)
```

```
TEXT
O(n)
```

Linear Search

```
TEXT
Best  = O(1)
Worst = O(n)
```

Insert at position

```
C
for (int i = n; i > pos; i--)
    a[i] = a[i - 1];
```

```
TEXT
Worst O(n)
```

Delete position

C

```
for (int i = pos; i < n - 1; i++)  
    a[i] = a[i + 1];
```

TEXT

Worst $O(n)$

Reverse

Two pointers:

TEXT

```
left = 0  
right = n-1
```

TEXT

Time $O(n)$
Space $O(1)$

66. 30-SECOND REVISION

Remember:

1. Traversal = $O(n)$.
2. Access by index = $O(1)$.
3. Linear search worst = $O(n)$.
4. Insert at beginning requires right shifts.
5. Delete at beginning requires left shifts.
6. Append with capacity = $O(1)$.
7. Reverse with two pointers = $O(n)$, $O(1)$.
8. Max/min needs one scan.
9. Max requires $n-1$ comparisons.
10. Sorted insertion still worst-case $O(n)$ because shifting dominates.

67. ACTIVE RECALL

Q1

What direction should we shift while inserting?

Q2

What direction while deleting?

Q3

What is worst-case linear-search time?

Q4

How many comparisons are required to find maximum among n elements?

Q5

What is auxiliary space of in-place reversal?

ANSWERS

A1

Right-to-left while making space.

A2

Left-to-right while closing the gap.

A3

TEXT

$O(n)$

A4

TEXT

$n - 1$

A5

TEXT

$O(1)$

68. PRACTICE

Q1

Given:

TEXT

5 10 15 20

insert 12 at index 2.

Write final array.

Q2

Given:

TEXT

10 20 30 40 50

delete index 1.

Write final logical array.

~~Q3~~ Q3

How many elements must shift when inserting at index 3 into an array currently containing 10 elements?

~~Q4~~ Q4

How many elements shift when deleting index 3 from an array of 10 elements?

~~Q5~~ Q5

Find maximum and second-largest distinct values:

TEXT

8 10 3 10 7 9

~~Q6~~ Q6

Reverse in-place:

TEXT

1 2 3 4 5 6

How many swaps occur?

~~Q7~~ — GATE THINKING

A sorted array contains n elements and sufficient unused capacity.

You use binary search to find the correct position for a new element, then insert it while preserving sorted order.

What is the overall worst-case time?

Explain why it is not $O(\log n)$.

~~Q8~~ — PLACEMENT

Design an $O(n)$ time, $O(1)$ auxiliary-space method to determine whether:

TEXT

1 2 3 2 1

is a palindrome.

Which array pattern are you using?

PROGRESS

MODULE 2 — ARRAYS

' Lesson 22 — Array Fundamentals & Memory ' Lesson 23 — 1D Array Operations

NEXT — LESSON 24

2D Arrays — Matrices, Rows, Columns & Memory Layout

Next we move from:

```
C
int a[5];
```

to:

```
C
int a[3][4];
```

We will learn:

- rows and columns
- declaration
- initialization
- `a[i][j]`
- nested traversal
- matrix input/output
- memory representation
- how 2D arrays are actually stored linearly
- arrays of arrays
- passing 2D arrays to functions
- matrix operations
- transpose
- diagonal elements
- complexity
- preparation for the major GATE topic:

Row-major and Column-major Address Calculation

That will lead directly into Lesson 25's address formulas.

USER

NEXT

ASSISTANT

MODULE 2 — LESSON 24

2D Arrays in C — Matrices, Rows, Columns, Traversal & Memory Layout

We now move from a one-dimensional array:

C

```
int a[5];
```

to a two-dimensional array:

C

```
int a[3][4];
```

This is the foundation for:

- matrices
- grids
- image data
- dynamic programming tables
- graph matrices
- matrix-based GATE questions
- row-major and column-major address calculation

GATE importance: $\emptyset = \gamma\%$ **MUST KNOW** **Placement importance:** $\emptyset = \gamma\%$ **NOT IMPORTANT**

1. STORY / INTUITION

A 1D array looks like one row of rooms:

TEXT

[10] [20] [30] [40]

A 2D array looks like a table:

TEXT

```
Column
0    1    2
```

```
Row 0  [ ] [ ] [ ]
Row 1  [ ] [ ] [ ]
```



```
Row 2  [ ] [ ] [ ]
```

So instead of one index:

```
c
a[i]
```

we need two:

```
c
a[i][j]
```

where:

```
TEXT
i  !'  r o w
j  !'  c o l u m n
```

2. DECLARATION

Example:

```
c
int a[3][4];
```

means:

```
TEXT
3 rows
4 columns
```

Total elements:

```
TEXT
3 × 4 = 12
```

Memory trick:

First dimension = rows, second dimension = columns

for the usual matrix interpretation.

3. VALID INDICES

For:

```
C
int a[3][4];
```

row indices:

```
TEXT
0, 1, 2
```

column indices:

```
TEXT
0, 1, 2, 3
```

Therefore:

```
C
a[2][3]
```

is valid.

But:

```
C
a[3][0]
```

is out of bounds.

4. VISUALIZATION

```
C
int a[3][4];
```

Conceptually:

```
TEXT
      j
      0  1  2  3
i=0  [ ] [ ] [ ] [ ]
i=1  [ ] [ ] [ ] [ ]
i=2  [ ] [ ] [ ] [ ]
```

Access:

```
TEXT
a [ 0 ] [ 0 ] != first element
a [ 0 ] [ 3 ] != last element of first row
a [ 2 ] [ 0 ] != first element of last row
a [ 2 ] [ 3 ] != last element
```

5. INITIALIZATION

```
C
int a[2][3] =
{
    {10, 20, 30},
    {40, 50, 60}
};
```

Matrix:

```
TEXT
10  20  30
40  50  60
```

6. FLAT INITIALIZATION

You can also write:

```
C
int a[2][3] = {10, 20, 30, 40, 50, 60};
```

C fills elements in row-major order.

Result:

```
TEXT
10  20  30
40  50  60
```

0=0. This already hints at C's memory layout.

7. PARTIAL INITIALIZATION

```
C
int a[2][3] =
{
    {1, 2},
    {3}
};
```

Remaining elements are zero-initialized.

Result:

```
TEXT
1  2  0
3  0  0
```

8. SIMPLE PARTIAL FORM

```
C
int a[2][3] = {1, 2};
```

fills:

```
TEXT
1  2  0
0  0  0
```

because the initialization proceeds through the array elements and omitted elements become zero.

9. ACCESSING AN ELEMENT

Example:

```
C
int a[2][3] =
{
    {10, 20, 30},
    {40, 50, 60}
};
```

Then:

```
C
a[0][0]
```

is:

```
TEXT
10
```

```
C
a[1][2]
```

is:

```
TEXT
60
```

10. MODIFYING ELEMENTS

```
C
a[1][1] = 100;
```

Before:

TEXT

```
10 20 30
40 50 60
```

After:

TEXT

```
10 20 30
40 100 60
```

Access/update of a known element:

TEXT

```
0(1)
```

because its address can be calculated directly.

11. TRAVERSING A 2D ARRAY

Use nested loops.

```
c
for (int i = 0; i < rows; i++)
{
    for (int j = 0; j < cols; j++)
    {
        printf("%d ", a[i][j]);
    }

    printf("\n");
}
```

12. COMPLEXITY OF TRAVERSAL

For:

TEXT

```
R rows
C columns
```

total elements:

TEXT

```
R × C
```

So:

TEXT

```
Time = O(RC)
```

Auxiliary space:

TEXT

$O(1)$

if only loop variables are used.

13. IMPORTANT GATE POINT

If matrix is:

TEXT

$n \times n$

then:

TEXT

$R = n$

$C = n$

so traversal becomes:

TEXT

$O(n^2)$

~~Do not~~ Do not automatically say every 2D array operation is $O(n^2)$.

For an:

TEXT

$R \times C$

matrix, it is:

TEXT

$O(RC)$

14. INPUT OF MATRIX

```
c
for (int i = 0; i < rows; i++)
{
    for (int j = 0; j < cols; j++)
    {
        scanf("%d", &a[i][j]);
    }
}
```

Again:

TEXT

```
a [ i ] [ j ]    !'   v a l u e
& a [ i ] [ j ] !'   a d d r e s s
```

scanf requires the address.

15. SUM OF ALL ELEMENTS

C

```
int sum = 0;

for (int i = 0; i < rows; i++)
{
    for (int j = 0; j < cols; j++)
    {
        sum += a[i][j];
    }
}
```

Time:

TEXT

$O(RC)$

Space:

TEXT

$O(1)$

16. ROW SUM

Suppose:

TEXT

```
1 2 3
4 5 6
7 8 9
```

Row 1:

TEXT

4 5 6

sum:

TEXT

15

Code:

```

c
for (int i = 0; i < rows; i++)
{
    int sum = 0;

    for (int j = 0; j < cols; j++)
    {
        sum += a[i][j];
    }

    printf("%d\n", sum);
}

```

Total time:

TEXT
O(RC)

17. COLUMN SUM

```

c
for (int j = 0; j < cols; j++)
{
    int sum = 0;

    for (int i = 0; i < rows; i++)
    {
        sum += a[i][j];
    }

    printf("%d\n", sum);
}

```

Notice loop order is reversed.

Outer:

TEXT
columns

Inner:

TEXT
rows

Still:

TEXT
O(RC)

18. MAIN DIAGONAL

For a square matrix:

TEXT

```
1 2 3
4 5 6
7 8 9
```

main diagonal:

TEXT

```
1
5
9
```

Indices:

TEXT

```
a[0][0]
a[1][1]
a[2][2]
```

Condition:

FORMULA BOX

TEXT

```
i == j
```

19. MAIN DIAGONAL CODE

```
c
for (int i = 0; i < n; i++)
{
    printf("%d ", a[i][i]);
}
```

Time:

TEXT

```
O(n)
```

~~O=Not~~ Time: not $O(n^2)$ because we directly visit only n diagonal elements.

20. SECONDARY DIAGONAL

For:

TEXT

```
1 2 3
4 5 6
7 8 9
```

secondary diagonal:

TEXT

3
5
7

Indices:

TEXT

a[0][2]
a[1][1]
a[2][0]

For $n \times n$ matrix:

FORMULA BOX

TEXT

$i + j = n - 1$

or:

TEXT

$j = n - 1 - i$

21. CODE

C

```
for (int i = 0; i < n; i++)  
{  
    printf("%d ", a[i][n - 1 - i]);  
}
```

Time:

TEXT

$O(n)$

22. DIAGONAL GATE TRAP

For a 3×3 matrix:

center element:

TEXT

a[1][1]

belongs to both diagonals.

So if you're computing:

```
TEXT
sum of both diagonals
```

and simply add both full diagonals, center gets counted twice.

For odd `n`, subtract center once.

~~0 = 0%~~
Common matrix trap.

23. TRANSPOSE

Transpose converts rows into columns.

Original:

```
TEXT
1 2 3
4 5 6
```

Shape:

```
TEXT
2 × 3
```

Transpose:

```
TEXT
1 4
2 5
3 6
```

Shape:

```
TEXT
3 × 2
```

Definition:

FORMULA BOX

```
TEXT
B[j][i] = A[i][j]
```

24. TRANSPOSE USING SECOND ARRAY

```
C
for (int i = 0; i < rows; i++)
{
```

```

    for (int j = 0; j < cols; j++)
    {
        b[j][i] = a[i][j];
    }
}

```

Time:

TEXT
 $O(RC)$

Additional matrix space:

TEXT
 $O(RC)$

25. IN-PLACE TRANSPOSE OF SQUARE MATRIX

For:

TEXT
 $n \times n$

we can swap:

TEXT
 $a[i][j] \leftrightarrow a[j][i]$

Only process one side of diagonal.

```

c
for (int i = 0; i < n; i++)
{
    for (int j = i + 1; j < n; j++)
    {
        int temp = a[i][j];
        a[i][j] = a[j][i];
        a[j][i] = temp;
    }
}

```

26. WHY $j = i + 1$?

Because diagonal:

TEXT
 $a[i][i]$

doesn't need swapping.

And if we swap both:

TEXT

```
a[i][j]
```

and later:

TEXT

```
a[j][i]
```

we undo our work.

So process only one triangular half.

27. TRANSPOSE COMPLEXITY

Approximately:

TEXT

```
n(n-1)/2
```

swaps.

Therefore:

TEXT

```
Time =  $O(n^2)$ 
```

```
Space =  $O(1)$ 
```

for in-place square transpose.

28. MATRIX ADDITION

For matrices of same dimensions:

TEXT

```
c[i][j] = A[i][j] + B[i][j]
```

Code:

```
c
for (int i = 0; i < rows; i++)
{
    for (int j = 0; j < cols; j++)
    {
        c[i][j] = a[i][j] + b[i][j];
    }
}
```

Time:

TEXT

$O(RC)$

29. MATRIX MULTIPLICATION— INTUITION

Suppose:

TEXT

A is $R_1 \times C_1$

B is $R_2 \times C_2$

Multiplication is valid when:

FORMULA BOX

TEXT

$C_1 = R_2$

Result dimensions:

TEXT

$R_1 \times C_2$

~~$O = 100$~~ Important basic matrix rule.

30. MATRIX MULTIPLICATIONCODE SHAPE

For:

TEXT

$A[m][k]$

$B[k][n]$

result:

TEXT

$C[m][n]$

Core loops:

```
c
for (int i = 0; i < m; i++)
{
    for (int j = 0; j < n; j++)
    {
        c[i][j] = 0;

        for (int x = 0; x < k; x++)
        {
            c[i][j] += a[i][x] * b[x][j];
        }
    }
}
```

```
}  
  }  
}
```

Time:

TEXT
 $O(mkn)$

For two square $n \times n$ matrices using ordinary multiplication:

TEXT
 $O(n^3)$

31. GATE POINT

Do not see 3 nested loops and blindly say $O(n^3)$.

If loop ranges are:

TEXT
m
n
k

then:

TEXT
 $O(mnk)$

Only when all are n :

TEXT
 $O(n^3)$

32. HOW 2D ARRAYS ARE ACTUALLY STORED

This is crucial.

Although we visualize:

TEXT
10 20 30
40 50 60

physical memory is linear.

C stores this array's elements contiguously.

For:

```
c
int a[2][3] =
{
    {10, 20, 30},
    {40, 50, 60}
};
```

memory sequence is conceptually:

```
TEXT
10 20 30 40 50 60
```

This is:

Row-Major Order

0-0% uses row-major ordering for multidimensional arrays.

33. ROW-MAJOR STORY

Finish one entire row before moving to the next.

```
TEXT
Row 0:
10 20 30

then Row 1:
40 50 60
```

Linear memory:

```
TEXT
1 0 !' 2 0 !' 3 0 !' 4 0 !' 5 0 !' 6 0
```

Memory trick:

C completes a ROW before going BELOW.

34. ARRAY OF ARRAYS

This is the precise C view.

For:

```
c
int a[2][3];
```

a is:

an array of 2 elements

where each element is:

an array of 3 integers.

So:

TEXT
a

type:

TEXT
array of 2 arrays of 3 int

And:

TEXT
a[0]

is the first row:

TEXT
array of 3 int

35. IMPORTANT POINTER CONNECTION

For:

C
int a[2][3];

in most expressions:

TEXT
a

decays to a pointer to its first row.

Type conceptually:

C
int (*)[3]

Read:

pointer to array of 3 integers.

$\neq$ This is different from:

```
c
int *
```

36. VISUALIZATION

```
TEXT
a
|
v
+-----+
| a[0]   |
| int[3] |
+-----+
| a[1]   |
| int[3] |
+-----+
```

Then:

```
TEXT
a + 1
```

moves one entire row.

Not one integer.

37. ADDRESS MOVEMENT

Suppose:

```
c
int a[2][3];
```

Assume:

```
TEXT
sizeof(int)=4
```

One row contains:

```
TEXT
3 × 4 = 12 bytes
```

Therefore:

```
TEXT
a + 1
```

moves:

TEXT

12 bytes

conceptually.

But:

TEXT

$a[0] + 1$

moves one integer:

TEXT

4 bytes

Excellent pointer + 2D array trap.

38. RELATIONSHIP OF $a[i][j]$

Remember:

TEXT

$a[i][j]$

can be expanded.

First:

TEXT

$a[i]$
=
 $*(a+i)$

Then:

TEXT

$a[i][j]$
=
 $*(*(a+i)+j)$

FORMULA BOX

TEXT

$a[i][j] = *(*(a+i)+j)$

This is a very important pointer expression.

39. STEP-BY-STEP

Suppose:

TEXT

```
a[1][2]
```

Then:

TEXT

```
a + 1
```

points to row 1.

TEXT

```
*(a+1)
```

refers to row 1 and in the next expression context decays to pointer to first element of row 1.

Then:

TEXT

```
*(a+1) + 2
```

points to column 2.

Finally:

TEXT

```
*(*(a+1)+2)
```

gives the element value.

40. GATE POINTER IDENTITY

For:

C

```
int a[3][4];
```

TEXT

```
a[i][j]
```

equals:

TEXT

```
*(*(a+i)+j)
```

Remember.

41. `a`, `a[0]`, `&a[0][0]`

For:

```
C
int a[2][3];
```

these may have the same starting numerical address:

```
TEXT
a
a[0]
&a[0][0]
```

but their types differ.

Broadly:

```
TEXT
a           !'  p o i n t e r   t o   r o w
a [ 0 ]     !'  p o i n t e r   t o   f i r s t   i n t   o f   r o w   0   i n   m o s t   e x p r e s s
&a [ 0 ] [ 0 ] !'  p o i n t e r   t o   i n t
```

Types affect pointer arithmetic.

42. DON'T CONFUSE

```
TEXT
Same numerical starting address
" `
same pointer type
```

This was also true for:

```
TEXT
a
&a
```

in 1D arrays.

NOTE loves this idea.

43. PASSING 2D ARRAY TO FUNCTION

Suppose:

```
C
int a[3][4];
```

Function can be declared:

```
C
void print(int a[][4], int rows)
```

or equivalently:

```
C
void print(int (*a)[4], int rows)
```

Why must the number of columns usually be known?

Because compiler needs to know the size of one row to calculate:

```
TEXT
a[i][j]
```

44. WHY COLUMN SIZE MATTERS

To move from:

```
TEXT
row 0
```

to:

```
TEXT
row 1
```

compiler must know:

```
TEXT
how many elements are in one row
```

For:

```
C
int a[][4]
```

row size is:

```
TEXT
4 × sizeof(int)
```

So it can calculate row jumps.

~~Very~~ ^{Very} important GATE concept.

45. VALID FUNCTION DECLARATION

```
C
void display(int a[][4], int rows)
{
    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < 4; j++)
        {
```

```

        printf("%d ", a[i][j]);
    }
}

```

46. WHAT ABOUT FIRST DIMENSION?

For a function parameter like:

```

c
void f(int a[][4])

```

the first dimension does not need to be specified there because the parameter adjusts to a pointer to rows.

But row width `4` is needed for pointer arithmetic.

This is why:

```

c
int a[][4]

```

is meaningful as a function parameter.

47. MODERN VLA-STYLE FUNCTION PARAMETER

Where variable-length arrays are supported, you may see:

```

c
void f(int rows, int cols, int a[rows][cols])

```

This allows runtime dimensions.

Good to know, but fixed-column forms are more common in basic GATE-style questions.

48. SIZE OF 2D ARRAY

Suppose:

```

c
int a[3][4];

```

Assume:

```

TEXT
sizeof(int)=4

```

Total elements:

TEXT

$3 \times 4 = 12$

Therefore:

TEXT

```
sizeof(a)
=
12 × 4
=
48 bytes
```

49. SIZE OF ONE ROW

C

```
sizeof(a[0])
```

`a[0]` is an actual array of:

TEXT

4 ints

So:

TEXT

```
sizeof(a[0])
=
4 × sizeof(int)
=
16
```

under the assumption.

50. NUMBER OF ROWS

For actual array in same scope:

C

```
sizeof(a) / sizeof(a[0])
```

gives:

TEXT

```
48 / 16
= 3
```

ROWS.

51. NUMBER OF COLUMNS

```
C
sizeof(a[0]) / sizeof(a[0][0])
```

gives:

```
TEXT
16 / 4
= 4
```

columns.

Useful array-size formula.

FORMULA BOX

For actual 2D array:

```
TEXT
Rows =
sizeof(a) / sizeof(a[0])

Columns =
sizeof(a[0]) / sizeof(a[0][0])
```

inside the scope where `a` is truly an array.

52. FUNCTION PARAMETER TRAP AGAIN

If:

```
C
void f(int a[][4])
```

inside `f`, `a` is effectively a pointer parameter.

So:

```
C
sizeof(a)
```

returns pointer size, not full matrix size.

Same decay concept as 1D arrays.

53. ROW-MAJOR ADDRESS INTUITION

Suppose:

TEXT

A has 4 columns.

To reach:

TEXT

$A[i][j]$

in row-major order:

First skip:

TEXT

i full rows

Each row has:

TEXT

4 elements

So skipped elements:

TEXT

$i \times 4$

Then move:

TEXT

j more elements

Total element offset:

TEXT

$i \times 4 + j$

This leads directly to the row-major address formula.

54. GENERAL ROW-MAJOR OFFSET

For:

TEXT

$R \times C$

zero-based matrix:

TEXT

$A[i][j]$

element offset from $A[0][0]$ is:

FORMULA BOX

TEXT

$$i \times C + j$$

Byte offset:

TEXT

$$(i \times C + j) \times w$$

where:

TEXT

w = bytes per element

Address:

TEXT

$$\text{Base} + (iC + j)w$$

0-9% is the core of the next lesson.

55. EXAMPLE

Matrix:

TEXT

3 rows
4 columns

Want:

TEXT

A[2][1]

Element offset:

TEXT

$$2 \times 4 + 1 \\ = 9$$

So A[2][1] is the 10th element from the beginning because indexing starts at zero.

If:

TEXT

Base = 1000
w = 4

address:

TEXT

```
1000 + 9×4  
= 1036
```

56. VISUAL PROOF

For 3×4 :

TEXT

```
A[0][0] offset 0  
A[0][1] offset 1  
A[0][2] offset 2  
A[0][3] offset 3  
  
A[1][0] offset 4  
A[1][1] offset 5  
A[1][2] offset 6  
A[1][3] offset 7  
  
A[2][0] offset 8  
A[2][1] offset 9  
...
```

So:

TEXT

```
offset = row × columns + column
```

57. COLUMN-MAJOR — IMPORTANT GATE PREVIEW

C itself stores multidimensional arrays in:

TEXT

ROW-MAJOR

But GATE may give theoretical questions asking about:

TEXT

COLUMN-MAJOR

In column-major order, memory is:

TEXT

```
A[0][0]  
A[1][0]  
A[2][0]  
...  
then next column
```

So the address formula changes.

We'll derive both carefully in Lesson 25.

58. CACHE LOCALITY — PLACEMENT/SYSTEMS CONNECTION

Because C arrays are row-major, this traversal:

```
c
for (int i = 0; i < rows; i++)
{
    for (int j = 0; j < cols; j++)
    {
        use(a[i][j]);
    }
}
```

follows memory order.

That's typically cache-friendly.

59. COLUMN-FIRST TRAVERSAL

```
c
for (int j = 0; j < cols; j++)
{
    for (int i = 0; i < rows; i++)
    {
        use(a[i][j]);
    }
}
```

still visits every element:

```
TEXT
O(RC)
```

but for large row-major arrays it may have worse cache locality because accesses jump between rows.

~~O=Big~~ Systems/placement insight.

Same Big-O does not necessarily mean same real-world speed.

60. IMPORTANT DISTINCTION

Two algorithms can both be:

```
TEXT
O(n²)
```

yet one may run faster because of:

- cache behavior
- constants
- memory access pattern

Big-O describes growth, not every machine-level performance detail.

61. GATE TRAP — TOTAL ELEMENTS

For:

```
C
int a[5][10];
```

number of elements:

```
TEXT
5 × 10 = 50
```

not:

```
TEXT
15
```

0 = 5% simple but common.

62. GATE TRAP — DIMENSIONS

For:

```
C
int a[3][7];
```

usual matrix interpretation:

```
TEXT
3 rows
7 columns
```

not reverse.

63. GATE TRAP — NESTED LOOPS

```
C
for (i = 0; i < 3; i++)
    for (j = 0; j < n; j++)
```

Time:

```
TEXT
O(3n)
=
O(n)
```

not:

TEXT

$O(n^2)$

because outer loop is constant 3.

$O=O(n)$ complexity depends on actual bounds.

64. GATE TRAP — DIAGONAL COMPLEXITY

This:

C

```
for (int i = 0; i < n; i++)  
    printf("%d", a[i][i]);
```

is:

TEXT

$O(n)$

not $O(n^2)$.

Only n elements are visited.

65. GATE TRAP — MATRIX MULTIPLICATION DIMENSIONS

If:

TEXT

A = 2×3
B = 3×4

multiplication is valid.

Result:

TEXT

2×4

But:

TEXT

A = 2×3
B = 2×4

ordinary $A \times B$ is invalid because:

TEXT

3 " ` 2

66. GATE FOCUS

0-MUST KNOW:

- `a[i][j]`
- rows vs columns
- total elements
- nested traversal complexity
- row-major storage in C
- 2D array = array of arrays
- `a[i][j] = *(* (a+i)+j)`
- `a+1` moves one row
- passing a 2D array to functions
- column dimension requirement
- transpose
- main/secondary diagonals
- row-major offset intuition

67. INTERVIEW QUESTIONS

Q1. How is a 2D array stored in C?

In contiguous row-major order.

Q2. What does `int a[3][4]` mean?

An array of 3 elements, each of which is an array of 4 integers.

Q3. What is `a[i][j]` in pointer notation?

```
c
*(*(a + i) + j)
```

Q4. Why must column size usually be known when passing a 2D array?

The compiler needs row width to calculate address jumps between rows.

Q5. Complexity of traversing an $R \times C$ matrix?

```
TEXT
O(RC)
```

Q6. Complexity of printing only main diagonal of $n \times n$ matrix?

```
TEXT
O(n)
```

68. MEMORY TRICKS

2D Array

Rows of arrays.

Index

TEXT
`a[row][column]`

C memory

Finish ROW, then move DOWN.

Pointer

TEXT
`a + 1`
`! ' next whole row`

Matrix address

Skip complete rows, then move inside row.

69. NOTEBOOK NOTES

Declaration

C
`int a[R][C];`

Total elements

TEXT
 $R \times C$

Access

C
`a[i][j]`

Pointer form

```
C
*(*(a+i)+j)
```

Traversal

```
TEXT
O(RC)
```

Main diagonal

```
TEXT
i = j
```

Secondary diagonal

```
TEXT
i + j = n - 1
```

Transpose

```
TEXT
B[j][i] = A[i][j]
```

C storage

```
TEXT
Row-major
```

70. 30-SECOND REVISION

Remember:

1. `int a[3][4]` = 3 rows, 4 columns.
2. Total elements = 12.
3. `a[i][j]` accesses row `i`, column `j`.
4. 2D traversal = $O(RC)$.
5. C stores 2D arrays row-major.
6. A 2D array is an array of rows.
7. `a+1` moves one whole row.
8. `a[i][j] = (*(a+i)+j)`.
9. Main diagonal: `i==j`.
10. Secondary diagonal: `i+j=n-1`.

71. ACTIVE RECALL

Q1

For:

```
C
int a[4][6];
```

how many rows, columns and elements?

Q2

What is the complexity of scanning all elements?

Q3

What does:

```
C
a + 1
```

move over in a 2D array?

Q4

Write `a[i][j]` in pointer notation.

Q5

What storage order does C use?

ANSWERS

A1

```
TEXT
Rows = 4
Columns = 6
Elements = 24
```

A2

```
TEXT
O(24)
```

for that fixed example; generally:

```
TEXT
O(RC)
```

A3

One complete row.

A4

```
c
*(*(a+i)+j)
```

A5

Row-major.

72. PRACTICE

Q1

For:

```
c
int a[5][8];
```

find:

- rows
- columns
- total elements

Q2

For:

```
TEXT
1 2 3
4 5 6
7 8 9
```

write:

- main diagonal
- secondary diagonal

Q3

Transpose:

```
TEXT
1 2 3
4 5 6
```

What are the dimensions before and after?

Q4

For:

```
C
int a[3][4];
```

assume:

```
TEXT
sizeof(int)=4
```

find:

```
TEXT
sizeof(a)
sizeof(a[0])
```

~~Q5~~ — POINTER

For:

```
C
int a[2][3];
```

what is the difference between:

```
TEXT
a + 1
```

and:

```
TEXT
a[0] + 1
```

in terms of element movement?

~~Q6~~ — COMPLEXITY

Find complexity:

```
C
for (int i = 0; i < n; i++)
{
    printf("%d", a[i][i]);
}
```

~~Q7~~ — ROW-MAJOR OFFSET

For a zero-based matrix:

TEXT

A[5][8]

find the linear element offset of:

TEXT

A[3][6]

from A[0][0] .

Use:

TEXT

row × columns + column

Q8 — ADDRESS

Suppose:

TEXT

Base address of A[0][0] = 1000
A has 6 columns
element size = 4 bytes

Find address of:

TEXT

A[2][3]

in row-major order.

Q9 — PLACEMENT/SYSTEMS

Two programs traverse the same huge C matrix:

Program A

C

```
for (i = 0; i < rows; i++)  
    for (j = 0; j < cols; j++)  
        use(a[i][j]);
```

Program B

C

```
for (j = 0; j < cols; j++)  
    for (i = 0; i < rows; i++)  
        use(a[i][j]);
```

Both have $O(RC)$ time.

Why might Program A still be faster in practice?

PROGRESS

MODULE 2 — ARRAYS

' Lesson 22 — Fundamentals & Memory ' Lesson 23 — 1D Array Operations ' Lesson 24 — 2D Arrays & Matrix Memory

NEXT — LESSON 25

Row-Major & Column-Major Address Calculation — GATE Formula Masterclass

Next is a particularly high-yield GATE numerical topic.

We will derive and practice:

TEXT

1D:
 $LOC(A[i]) = \text{Base} + (i-L)w$

TEXT

2D Row Major:
 $LOC(A[i][j])$
 $= \text{Base} + [(i-LR) \times N + (j-LC)] \times w$

TEXT

2D Column Major:
 $LOC(A[i][j])$
 $= \text{Base} + [(j-LC) \times M + (i-LR)] \times w$

where:

TEXT

M = number of rows
N = number of columns

We will solve:

- zero-based arrays
- non-zero lower bounds
- negative lower bounds
- row-major questions
- column-major questions
- finding base address
- finding element indices from addresses
- 3D-array formula intuition
- common GATE traps

Type **NEXT**.

USER

next

ASSISTANT

MODULE 2 — LESSON 25

Row-Major & Column-Major Address Calculation — GATE Formula Masterclass

This is one of the most important **numerical topics** in Arrays for GATE.

You should be able to solve questions where they give:

- base address
- lower bounds
- upper bounds
- row/column indices
- element size
- row-major or column-major order

and ask for the address of an element.

GATE importance: 0=Ý% ÇAÝSDEKÝNÖW Placement importance: +P IMPORTANT

1. WHERE WE ARE

Completed:

- Array basics
- 1D operations
- 2D arrays
- matrix memory layout

Now we formally learn:

TEXT

- 1D address formula
- 2D row-major formula
- 2D column-major formula

The goal is not to blindly memorize formulas.

We will derive them so you can rebuild them in the exam if you forget.

2. FIRST IDEA — MEMORY IS LINEAR

Even if we write:

```
TEXT
A[i][j]
```

memory itself is one long sequence of bytes.

So the real question is:

How many elements must I skip before reaching $A[i][j]$?

Once we know the number of skipped elements:

```
TEXT
Address
=
Base Address
+
Number of skipped elements × element size
```

0-1% is the foundation of every formula.

3. UNIVERSAL ADDRESS IDEA

FORMULA BOX

```
TEXT
Address
=
Base
+
Offset in elements × Element Size
```

So almost every array-address problem has two steps:

1. Find **element offset**
2. Multiply by element size

4. 1D ARRAY — ZERO-BASED

Suppose:

```
TEXT
A[0 ... n-1]
```

Base address:

```
TEXT
B = address of A[0]
```

Element size:

TEXT

w bytes

Then:

TEXT

```
A [ 0 ] !'  s k i p  0  e l e m e n t s
A [ 1 ] !'  s k i p  1  e l e m e n t
A [ 2 ] !'  s k i p  2  e l e m e n t s
...
A [ i ] !'  s k i p  i  e l e m e n t s
```

Therefore:

FORMULA BOX

TEXT

$\text{LOC}(A[i]) = B + i \times w$

for zero-based arrays.

5. EXAMPLE

Given:

TEXT

Base = 1000
Element size = 4 bytes

Find:

TEXT

A[7]

Offset:

TEXT

7

Byte offset:

TEXT

$7 \times 4 = 28$

Address:

TEXT

$1000 + 28 = 1028$

Answer:

6. NON-ZERO LOWER BOUND

Now suppose:

TEXT

$A[5 \dots 20]$

and base means:

TEXT

$B = \text{address of } A[5]$

To reach:

TEXT

$A[8]$

you don't skip 8 elements.

You skip:

TEXT

$8 - 5 = 3$

elements.

Therefore:

FORMULA BOX

TEXT

$\text{LOC}(A[i])$

=

$B + (i - L) \times w$

where:

TEXT

$L = \text{lower bound}$

$\emptyset = \text{Must know.}$

7. EXAMPLE

Given:

TEXT

A[5 ... 20]
Base address of A[5] = 2000
Element size = 4 bytes

Find address of:

TEXT

A[12]

Offset:

TEXT

$12 - 5 = 7$ elements

Bytes:

TEXT

$7 \times 4 = 28$

Address:

TEXT

$2000 + 28 = 2028$

Answer:

2028

8. WHY LOWER BOUND IS SUBTRACTED

Think:

TEXT

A[5]

must have offset:

TEXT

0

Formula:

TEXT

$5 - 5 = 0$

Correct.

For:

TEXT

A[6]

offset:

TEXT

6 - 5 = 1

Correct.

So:

TEXT

index - lower bound

automatically converts arbitrary indexing into zero-based offset.

9. NUMBER OF ELEMENTS IN 1D ARRAY

For:

TEXT

A[L ... U]

number of elements:

FORMULA BOX

TEXT

$N = U - L + 1$

Example:

TEXT

A[5 ... 20]

contains:

TEXT

20 - 5 + 1
= 16 elements

$U - L + 1$ is often forgotten.

10. NEGATIVE LOWER BOUND

Suppose:

TEXT

A[-3 ... 5]

Find offset of:

TEXT

A[2]

Use:

TEXT

i - L

So:

TEXT

2 - (-3)
= 5

Therefore A[2] is 5 elements after A[-3].

0 = Negative lower bounds are not special; same formula works.

11. NOW 2D ARRAYS

Suppose:

TEXT

A[3][4]

Visual matrix:

TEXT

	Col			
	0	1	2	3
Row 0	a	b	c	d
Row 1	e	f	g	h
Row 2	i	j	k	l

There are:

TEXT

3 rows
4 columns

12. ROW-MAJOR ORDER

In row-major order, memory stores:

TEXT

a b c d e f g h i j k l

That means:

Complete the entire row before moving to the next row.

C uses row-major ordering for ordinary multidimensional arrays.

13. ROW-MAJOR STORY

To reach:

TEXT

A[i][j]

you first skip:

TEXT

i complete rows

If there are:

TEXT

C columns per row

each full row contains:

TEXT

C elements

So skipping i rows means:

TEXT

i × C

elements.

Then within the target row, skip:

TEXT

j

elements.

Therefore total element offset:

FORMULA BOX

$$\mathbf{i} \times \mathbf{C} + \mathbf{j}$$

14. ZERO-BASED ROW-MAJOR FORMULA

```
B = address of A[0][0]
C = number of columns
w = bytes per element
```

FORMULA BOX

$$\text{LOC}(A[i][j]) = B + (i \times C + j) \times w$$

15. EXAMPLE

A[3][4]

1000

4 bytes

A[2][1]

Page 948 of 2799
www.gpttopdf.in

TEXT

$C = 4$

Offset:

TEXT

$2 \times 4 + 1$
 $= 9$

Byte offset:

TEXT

$9 \times 4 = 36$

Address:

TEXT

$1000 + 36$
 $= 1036$

Answer:

1036

16. VISUAL CHECK

Memory order:

TEXT

```
0 f f s e t   0   !'   A [ 0 ] [ 0 ]
0 f f s e t   1   !'   A [ 0 ] [ 1 ]
0 f f s e t   2   !'   A [ 0 ] [ 2 ]
0 f f s e t   3   !'   A [ 0 ] [ 3 ]

0 f f s e t   4   !'   A [ 1 ] [ 0 ]
0 f f s e t   5   !'   A [ 1 ] [ 1 ]
0 f f s e t   6   !'   A [ 1 ] [ 2 ]
0 f f s e t   7   !'   A [ 1 ] [ 3 ]

0 f f s e t   8   !'   A [ 2 ] [ 0 ]
0 f f s e t   9   !'   A [ 2 ] [ 1 ]
```

Correct.

17. ROW-MAJOR WITH NON-ZERO LOWER BOUNDS

Now suppose:

TEXT

$A[LR \dots UR][LC \dots UC]$

where:

TEXT

LR = lower row bound
LC = lower column bound

Number of columns:

FORMULA BOX

TEXT

$N = UC - LC + 1$

Before row i , number of complete rows skipped:

TEXT

$i - LR$

Each row has:

TEXT

N

elements.

So row contribution:

TEXT

$(i - LR) \times N$

Within row, columns skipped:

TEXT

$j - LC$

Total offset:

TEXT

$(i - LR)N + (j - LC)$

18. GENERAL ROW-MAJOR FORMULA

FORMULA BOX

TEXT

$LOC(A[i][j])$
=
B
+
 $[(i - LR) \times N + (j - LC)] \times w$

where:

TEXT

N = number of columns
= UC - LC + 1

Row-major formula.

19. MEMORY TRICK FOR ROW MAJOR

ROW !' use number of COLUMNS

Because in row-major order, to skip one full row you must skip all columns.

So:

TEXT

Row Major
!' row displacement $\times$ number of columns

Mnemonic:

R uses C

Row-major formula uses number of **C**olumns.

20. SOLVED GATE EXAMPLE 1

Suppose:

TEXT

A[2 ... 5][3 ... 8]

stored in row-major order.

Given:

TEXT

Base address of A[2][3] = 1000
Element size = 4 bytes

Find address of:

TEXT

A[4][6]

21. STEP 1 — NUMBER OF COLUMNS

Column bounds:

TEXT

3 ... 8

Number of columns:

TEXT

$8 - 3 + 1$
 $= 6$

So:

TEXT

$N = 6$

22. STEP 2 — ELEMENT OFFSET

Formula:

TEXT

$(i-LR)N + (j-LC)$

Substitute:

TEXT

$(4-2) \times 6 + (6-3)$

TEXT

$= 2 \times 6 + 3$
 $= 15$

23. STEP 3 — BYTE OFFSET

TEXT

15×4
 $= 60$

24. STEP 4 — ADDRESS

TEXT

$1000 + 60$
 $= 1060$

Answer:

1060

25. FAST EXAM METHOD

Write:

TEXT

RM:

$B + [(\text{row jump} \times \text{columns}) + \text{column jump}] \times w$

For this problem:

TEXT

$\text{row jump} = 4 - 2 = 2$

$\text{columns} = 6$

$\text{column jump} = 6 - 3 = 3$

So:

TEXT

$1000 + [(2 \times 6) + 3] \times 4$

26. COLUMN-MAJOR ORDER

Now imagine storing the same matrix by finishing each column first.

Matrix:

TEXT

a b c

d e f

g h i

Column-major memory:

TEXT

a d g b e h c f i

So:

Finish one entire COLUMN before moving to the next column.

27. COLUMN-MAJOR STORY

To reach:

TEXT

$A[i][j]$

first skip:

TEXT

j complete columns

If there are:

TEXT

R rows

each complete column contains:

TEXT

R elements

So:

TEXT

$j \times R$

elements are skipped.

Then inside target column, move down:

TEXT

i

rows.

Total zero-based offset:

FORMULA BOX

TEXT

$j \times R + i$

28. ZERO-BASED COLUMN-MAJOR FORMULA

If:

TEXT

R = number of rows

then:

FORMULA BOX

TEXT

LOC(A[i][j])

=

$B + (j \times R + i) \times w$

Ø=1% Important theoretical GATE formula.

29. MEMORY TRICK FOR COLUMN MAJOR

COLUMN !' use number of ROWS

Because skipping one whole column skips every row.

Mnemonic:

C uses R

Column-major formula uses number of **Rows**.

30. ROW VS COLUMN MAJOR MEMORY TRICK

TEXT

ROW major:
 $(\text{row jump} \times \text{COLUMNS}) + \text{column jump}$

COLUMN major:
 $(\text{column jump} \times \text{ROWS}) + \text{row jump}$

Or simply:

R uses C

C uses R

Ø=1% This is one of the best shortcuts to memorize.

31. COLUMN-MAJOR WITH LOWER BOUNDS

Suppose:

TEXT

$A[\text{LR} \dots \text{UR}][\text{LC} \dots \text{UC}]$

Number of rows:

FORMULA BOX

TEXT

$M = \text{UR} - \text{LR} + 1$

Columns skipped:

TEXT

$j - LC$

Each column contains:

TEXT

M rows

So:

TEXT

$(j - LC) \times M$

Then row displacement:

TEXT

$i - LR$

Total:

TEXT

$(j - LC)M + (i - LR)$

32. GENERAL COLUMN-MAJOR FORMULA

FORMULA BOX

TEXT

```
LOC(A[i][j])
=
B
+
[(j - LC) × M + (i - LR)] × w
```

where:

TEXT

M = number of rows
 $= UR - LR + 1$

Ø=Ý%Ø=Ý%Ø=Ý%Ø=Ý%

33. SOLVED GATE EXAMPLE 2

Suppose:

TEXT

A[2 ... 5][3 ... 8]

stored column-major.

Given:

TEXT

Base address of A[2][3] = 1000
Element size = 4

Find:

TEXT

A[4][6]

34. NUMBER OF ROWS

Rows:

TEXT

2 ... 5

Count:

TEXT

$5 - 2 + 1$
 $= 4$

So:

TEXT

M = 4

35. OFFSET

Column-major formula:

TEXT

$(j - LC)M + (i - LR)$

Substitute:

TEXT

$(6 - 3) \times 4 + (4 - 2)$

TEXT

$$= 3 \times 4 + 2$$
$$= 14$$

36. ADDRESS

Byte offset:

TEXT

$$14 \times 4$$
$$= 56$$

Address:

TEXT

$$1000 + 56$$
$$= 1056$$

Answer:

1056

Notice same element had a different address under row-major because storage order changed.

37. MASTER COMPARISON

For:

TEXT

$A[LR \dots UR][LC \dots UC]$

Let:

TEXT

M = number of rows
N = number of columns

Row-major

TEXT

$$B + [(i-LR)N + (j-LC)]w$$

Column-major

TEXT

$$B + [(j-LC)M + (i-LR)]w$$

Copy this exact pair.

38. GATE TRAP — M AND N

Many textbooks use:

```
TEXT
M = rows
N = columns
```

But not every book uses the same letters.

So don't memorize letters blindly.

Memorize meaning:

```
TEXT
Row - major !' multiply row displacement by number of columns
Column - major !' multiply column displacement by number of rows
```

39. EXAMPLE WITH NEGATIVE LOWER BOUNDS

Suppose:

```
TEXT
A[-2 ... 2][-3 ... 3]
```

Row-major.

Base:

```
TEXT
500
```

Element size:

```
TEXT
2 bytes
```

Find address:

```
TEXT
A[1][2]
```

40. NUMBER OF COLUMNS

```
TEXT
-3 ... 3
```

Count:

TEXT

$$3 - (-3) + 1 \\ = 7$$

41. OFFSET

TEXT

$$(i-LR)N + (j-LC)$$

TEXT

$$(1 - (-2)) \times 7 + (2 - (-3))$$

TEXT

$$= 3 \times 7 + 5 \\ = 26$$

Byte offset:

TEXT

$$26 \times 2 = 52$$

Address:

TEXT

$$500 + 52 \\ = 552$$

Answer:

552

42. GATE TRAP — NEGATIVE BOUNDS

When lower bound is negative:

TEXT

$$i - (-3)$$

becomes:

TEXT

$$i + 3$$

Do not accidentally write:

TEXT

i - 3

0=Very common arithmetic error.

43. FIND BASE ADDRESS FROM ELEMENT ADDRESS

GATE may reverse the question.

Suppose row-major:

TEXT

$A[1 \dots 5][2 \dots 8]$

Element size:

TEXT

4 bytes

and:

TEXT

Address of $A[3][6] = 2088$

Find base address of:

TEXT

$A[1][2]$

44. NUMBER OF COLUMNS

TEXT

$2 \dots 8$

Count:

TEXT

$8 - 2 + 1$
 $= 7$

45. OFFSET OF $A[3][6]$

TEXT

$(3-1) \times 7 + (6-2)$

TEXT

$= 2 \times 7 + 4$
 $= 18 \text{ elements}$

Bytes:

TEXT

18×4
 $= 72$

We know:

TEXT

Element Address
 $=$
 $\text{Base} + 72$

So:

TEXT

2088
 $=$
 $\text{Base} + 72$

Therefore:

TEXT

$\text{Base} = 2016$

Answer:

2016

46. REVERSE FORMULA IDEA

If:

TEXT

$\text{Address} = \text{Base} + \text{offset}$

then:

TEXT

$\text{Base} = \text{Address} - \text{offset}$

Simple algebra.

Don't panic if GATE asks backward.

47. FIND INDEX FROM ADDRESS — 1D

Suppose:

TEXT
A[5...20]

Base:

TEXT
1000

Element size:

TEXT
4

Which index is stored at address:

TEXT
1040

?

Difference:

TEXT
 $1040 - 1000 = 40 \text{ bytes}$

Elements skipped:

TEXT
 $40 / 4 = 10$

Since lower bound is 5:

TEXT
 $i - 5 = 10$

so:

TEXT
 $i = 15$

Answer:

A[15]

48. FINDING 2D INDEX FROM LINEAR OFFSET — ROW MAJOR

For zero-based row-major with:

```
TEXT
C columns
```

linear offset:

```
TEXT
k
```

Then:

```
TEXT
row = k / C
column = k % C
```

using integer division.

Very useful.

49. EXAMPLE

Matrix has:

```
TEXT
4 columns
```

Linear offset:

```
TEXT
11
```

Then:

```
TEXT
row = 11 / 4 = 2
column = 11 % 4 = 3
```

So:

```
TEXT
offset 11 != A[2][3]
```

50. WHY?

Each complete row consumes:

```
TEXT
C
```

elements.

So quotient tells:

```
TEXT
number of complete rows
```

and remainder tells:

```
TEXT
position within current row
```

51. COLUMN-MAJOR REVERSE IDEA

For zero-based column-major with:

```
TEXT
R rows
```

and offset k :

```
TEXT
column = k / R
row    = k % R
```

Because each complete column contains R elements.

Again:

C uses R

52. ADDRESS DIFFERENCE BETWEEN TWO ELEMENTS

Suppose two elements belong to same 1D array:

```
TEXT
A[i]
A[j]
```

Difference:

```
TEXT
|i-j| × w
```

bytes.

For row-major 2D arrays, first convert both to linear offsets.

Example:

TEXT

$\text{offset}(i, j) = iC + j$

Then subtract offsets.

53. GATE EXAMPLE — ADDRESS DIFFERENCE

Zero-based:

TEXT

$A[5][6]$

row-major.

Element size:

TEXT

4 bytes

Find byte distance between:

TEXT

$A[1][2]$

and:

TEXT

$A[3][4]$

Offsets:

TEXT

$A[1][2]:$
 $1 \times 6 + 2 = 8$

$A[3][4]:$
 $3 \times 6 + 4 = 22$

Difference:

TEXT

$22 - 8 = 14$ elements

Byte distance:

TEXT

$14 \times 4 = 56$ bytes

54. 3D ARRAY — INTUITION

GATE can sometimes extend the same logic.

Suppose:

```
TEXT
A[X][Y][Z]
```

stored row-major.

Think of:

```
TEXT
X blocks
```

Each block contains:

```
TEXT
Y rows
```

Each row contains:

```
TEXT
Z elements
```

To reach:

```
TEXT
A[i][j][k]
```

skip:

```
TEXT
i complete blocks
+
j complete rows
+
k elements
```

55. ZERO-BASED 3D ROW-MAJOR OFFSET

One full block contains:

```
TEXT
Y × Z
```

elements.

Therefore:

FORMULA BOX

TEXT

Offset
=
 $i(YZ) + jZ + k$

Address:

TEXT

$B + [iYZ + jZ + k]w$

Ø=Übod to know.

56. EXAMPLE

TEXT

$A[3][4][5]$

Find offset of:

TEXT

$A[2][1][3]$

Formula:

TEXT

$2 \times (4 \times 5) + 1 \times 5 + 3$

TEXT

= $40 + 5 + 3$
= 48

So it is 48 elements after the base.

57. GENERAL LOWER-BOUND 3D IDEA

You don't need to memorize a giant formula.

Just follow:

TEXT

normalize every index
× sizes of all dimensions to its right

For row-major:

TEXT

$(i - L1) \times N2 \times N3$
+
 $(j - L2) \times N3$
+

(k-L3)

Then multiply by w .

$O=YN$ logic scales to any dimension.

58. MEMORY TRICK FOR MULTIDIMENSIONAL ROW MAJOR

For each index:

Multiply by the product of dimensions to its right.

Example:

TEXT

`A[i][j][k]`

TEXT

```
i !'  m u l t i p l y   b y   Y × Z
j !'  m u l t i p l y   b y   Z
k !'  m u l t i p l y   b y   1
```

Very powerful shortcut.

59. COLUMN-MAJOR MULTIDIMENSIONAL INTUITION

Column-major reverses which dimension varies fastest.

For 2D:

TEXT

`row index changes fastest`

For row-major:

TEXT

`column index changes fastest`

For GATE, 2D column-major is much more important than memorizing high-dimensional formulas.

60. WHAT C ACTUALLY USES

$O=YN$ important distinction:

In actual C:

C

```
int a[3][4];
```

is stored:

Row-major

But GATE may still ask theoretical column-major questions because data-structure address calculation is language-independent unless language/storage order is specified.

61. GATE TRAP — BASE ADDRESS MEANING

If the problem says:

TEXT

Base address of A = 1000

you must determine what A means from the declared lower bounds.

Usually base is the address of:

TEXT

A[LR][LC]

the first logical element.

Do not automatically treat it as:

TEXT

A[0][0]

if lower bounds are not zero.

$\emptyset = 0\%$ critical.

62. GATE TRAP — NUMBER OF COLUMNS

Suppose:

TEXT

A[2...7][4...9]

Number of columns:

TEXT

$9 - 4 + 1 = 6$

not:

TEXT

9

and not:

TEXT

$9 - 4 = 5$

Remember:

TEXT

Upper - Lower + 1

63. GATE TRAP — WRONG MULTIPLIER

For row-major:

Wrong:

TEXT

$(\text{row displacement} \times \text{rows})$

Correct:

TEXT

$(\text{row displacement} \times \text{columns})$

because every skipped row contains all columns.

64. GATE TRAP — COLUMN MAJOR

For column-major:

Wrong:

TEXT

$(\text{column displacement} \times \text{columns})$

Correct:

TEXT

$(\text{column displacement} \times \text{rows})$

because every skipped column contains all rows.

65. FAST VISUAL METHOD IF YOU FORGET FORMULA

Suppose row-major:

TEXT

$A[2 \dots 5][3 \dots 8]$

and want:

TEXT

$A[4][6]$

Ask:

How many complete rows before row 4?

TEXT

$4 - 2 = 2$

How many elements per row?

TEXT

6

So:

TEXT

$2 \times 6 = 12$

How far inside target row?

TEXT

$6 - 3 = 3$

Total:

TEXT

$12 + 3 = 15$

No formula memorization required.

$O = YN$ is the safest exam fallback.

66. COLUMN-MAJOR VISUAL METHOD

Ask:

How many complete columns before target column?

TEXT

$j - LC$

How many elements per column?

TEXT

number of rows

Multiply them.

Then move down:

TEXT

i-LR

Simple.

67. MASTER SOLVING PROCEDURE

For every GATE address question:

Step 1

Identify:

TEXT

Row-major or Column-major?

Step 2

Write lower bounds.

Step 3

Calculate:

TEXT

Number of rows
Number of columns

Step 4

Calculate element offset.

Step 5

Multiply by element size.

Step 6

Add to base address.

Never jump directly into arithmetic.

68. MASTER FORMULA SHEET

1D

TEXT

$LOC(A[i])$
 $=$
 $B + (i-L)w$

2D ROW MAJOR

TEXT

```
LOC(A[i][j])  
=  
B + [(i-LR)N + (j-LC)]w
```

where:

TEXT

N = number of columns

2D COLUMN MAJOR

TEXT

```
LOC(A[i][j])  
=  
B + [(j-LC)M + (i-LR)]w
```

where:

TEXT

M = number of rows

Number of elements

TEXT

U-L+1

Copy this entire box.

69. COMPLEXITY CONNECTION

Address calculation uses a fixed number of arithmetic operations.

Therefore accessing:

TEXT

A[i][j]

is:

TEXT

O(1)

even for multidimensional arrays.

This explains why multidimensional arrays still provide random access.

70. WHY LINKED LIST CANNOT DO THIS

Array:

TEXT

Base + formula

! ' i m m e d i a t e l y c a l c u l a t e l o c a t i o n

Linked list:

TEXT

h e a d ! ' n e x t ! ' n e x t ! ' n e x t . . .

To reach element `i`, traversal is required.

So:

TEXT

Array index access = $O(1)$

Linked-list positional access = $O(n)$

This is one of the central data-structure trade-offs.

71. SOLVED GATE-STYLE EXAMPLE 3

Array:

TEXT

`A[1...4][-2...3]`

Row-major.

Given:

TEXT

Base = 500

w = 2 bytes

Find address:

TEXT

`A[3][1]`

Number of columns:

TEXT

$3 - (-2) + 1$

= 6

Offset:

TEXT

$$(3-1) \times 6 + (1 - (-2))$$

TEXT

$$= 12 + 3$$
$$= 15$$

Bytes:

TEXT

$$15 \times 2$$
$$= 30$$

Address:

TEXT

$$500 + 30$$
$$= 530$$

Answer:

530

72. SOLVED COLUMN-MAJOR VERSION

Same:

TEXT

$$A[1 \dots 4][-2 \dots 3]$$

Column-major.

Rows:

TEXT

$$4 - 1 + 1$$
$$= 4$$

Offset:

TEXT

$$(1 - (-2)) \times 4 + (3 - 1)$$

TEXT

$$= 3 \times 4 + 2$$
$$= 14$$

Bytes:

TEXT

$14 \times 2 = 28$

Address:

TEXT

$500 + 28 = 528$

Answer:

528

73. GATE FOCUS

Ø= MUST KNOW:

- 1D formula
- lower-bound handling
- $U - L + 1$
- row-major formula
- column-major formula
- R uses C
- C uses R
- negative lower bounds
- reverse base-address questions
- linear offset interpretation
- address access = $O(1)$

+ IMPORTANT:

- finding index from offset
- address differences

Ø= KNOW:

- 3D formula intuition

74. INTERVIEW / VIVA QUESTIONS

Q1. Why does row-major formula multiply row displacement by number of columns?

Because skipping one row skips all elements in that row.

Q2. Why does column-major use number of rows?

Because skipping one column skips all rows in that column.

Q3. What is C's multidimensional array storage order?

Row-major.

Q4. What does $U - L + 1$ calculate?

Number of legal indices/elements between inclusive bounds.

Q5. Why is multidimensional array access $O(1)$?

The address can be directly computed from indices.

75. MEMORY TRICKS

Lower bound

Convert to zero first

TEXT

index - lower bound

Row major

R uses C

TEXT

Row displacement $\times$ Columns

Column major

C uses R

TEXT

Column displacement $\times$ Rows

Final address

Offset !' Bytes !' Base

TEXT

element offset
 $\times$ element size
 $+$ base

76. NOTEBOOK NOTES

1D

TEXT

$$B + (i-L)w$$

Row-major

TEXT

$$B + [(i-LR)N + (j-LC)]w$$

N = columns

Column-major

TEXT

$$B + [(j-LC)M + (i-LR)]w$$

M = rows

Element count

TEXT

$$U-L+1$$

Shortcut

TEXT

R	M	!	'	R	u	s	e	s	C
C	M	!	'	C	u	s	e	s	R

77. 30-SECOND REVISION

Remember:

1. Address = base + offset × element size.
2. Normalize index with `index - lower_bound`.
3. Count dimension with `upper - lower + 1`.
4. Row-major completes rows.
5. Column-major completes columns.
6. Row-major uses number of columns.
7. Column-major uses number of rows.
8. C uses row-major.
9. Negative lower bounds use the same formula.
10. Direct address calculation means `O(1)` access.

78. ACTIVE RECALL

Q1

What is the 1D address formula?

Q2

What is the zero-based row-major offset?

Q3

What does **R uses C** mean?

Q4

What is the general column-major formula?

Q5

How many elements are in:

TEXT

$A[-3 \dots 7]$

?

ANSWERS

A1

TEXT

$B + (i-L)w$

A2

TEXT

$iC + j$

A3

Row displacement is multiplied by the number of columns.

A4

TEXT

$B + [(j-LC)M + (i-LR)]w$

A5

TEXT

$7 - (-3) + 1$
 $= 11$

79. PRACTICE

~~Q1~~ — 1D

TEXT

A[4...20]
Base of A[4] = 100
w = 2 bytes

Find address of:

TEXT

A[14]

Q2 — ZERO-BASED ROW MAJOR

TEXT

A[5][8]
Base = 1000
w = 4

Find:

TEXT

A[3][6]

Q3 — NON-ZERO ROW MAJOR

TEXT

A[2...6][5...10]
Base = 2000
w = 4

Find:

TEXT

A[5][8]

Q4 — COLUMN MAJOR

Same array:

TEXT

A[2...6][5...10]
Base = 2000
w = 4

but stored column-major.

Find address of:

TEXT

A[5][8]

Q5 — NEGATIVE BOUNDS

TEXT

A[-3...4][-5...2]

row-major.

Base:

TEXT

5000

Element size:

TEXT

8

Find address of:

TEXT

A[1][-1]

Q6 — FIND BASE

Row-major:

TEXT

A[1...5][2...6]

Element size:

TEXT

4

Address of:

TEXT

A[4][5] = 1080

Find base address of:

TEXT

A[1][2]

Q7 — FIND INDEX

A zero-based row-major matrix has:

TEXT

7 columns

An element has linear offset:

TEXT

31

Find:

TEXT

row
column

Q8 — 3D

For zero-based row-major:

TEXT

A[4][5][6]

find the element offset of:

TEXT

A[2][3][4]

Use:

TEXT

i(YZ) + jZ + k

Q9 — CONCEPTUAL

Without memorizing the formula, explain how you would derive the row-major address of:

TEXT

A[i][j]

from:

1. number of full rows skipped,
2. number of elements per row,
3. movement inside target row.

PROGRESS

MODULE 2 — ARRAYS

' Lesson 22 — Fundamentals & Memory ' Lesson 23 — 1D Operations ' Lesson 24 — 2D Arrays ' **Lesson 25 — Row-Major & Column-Major Address Calculation**

NEXT — LESSON 26

Arrays + Pointers — Deep GATE Consolidation

Next we connect arrays and pointers at a deeper level:

```
c
a
&a
&a[0]
a + 1
&a + 1
*(a + i)
a[i]
```

and for 2D arrays:

```
c
a
a + 1
*a
*(a + 1)
a[i][j]
*(*(a+i)+j)
```

We will solve tricky GATE questions involving:

- array decay
- pointer arithmetic
- `sizeof`
- 1D vs 2D pointer types
- pointer-to-array syntax
- function parameters
- `int *p`
- `int (*p)[N]`
- why `int **` is **not** the same as `int a[R][C]`
- exact address movement
- common output questions

Then we move to **Searching** in Lesson 27.

USER

next

MODULE 2 — LESSON 26

Arrays + Pointers — Deep GATE Consolidation

This lesson connects everything we learned about:

- arrays
- addresses
- pointer arithmetic
- `sizeof`
- 2D arrays
- function parameters

This is a **high-value GATE trap area** because many expressions can have the **same numerical address but different types and pointer arithmetic behavior**.

GATE importance: MUST KNOW **Placement importance:** IMPORTANT

1. WHERE WE ARE

Completed:

- Array fundamentals
- 1D operations
- 2D arrays
- Row-major / column-major formulas

Now we deeply connect:

```
c
a
&a
&a[0]
a + 1
&a + 1
*(a + i)
a[i]
```

and 2D:

```
c
a
a + 1
*a
*(a + 1)
a[i][j]
*(*(a+i)+j)
```

2. START WITH A SIMPLE ARRAY

```
C
int a[5] = {10, 20, 30, 40, 50};
```

Conceptually assume:

```
TEXT
Base address = 1000
sizeof(int) = 4
```

Memory:

```
TEXT
Address      Element
1000         a[0] = 10
1004         a[1] = 20
1008         a[2] = 30
1012         a[3] = 40
1016         a[4] = 50
```

3. WHAT DOES `a` MEAN?

In most expressions:

```
C
a
```

is converted to a pointer to the first element:

```
C
&a[0]
```

So numerically:

```
TEXT
a  !'  1 0 0 0
```

and its resulting pointer type is:

```
C
int *
```

~~One~~ rule:

```
TEXT
Array name usually decays to pointer to first element.
```

4. `a` VS `&a[0]`

For:

```
c
int a[5];
```

in most ordinary expressions:

```
TEXT
a
```

and:

```
TEXT
&a[0]
```

point to the same first integer.

So both may numerically hold:

```
TEXT
1000
```

Their useful pointer type in that context:

```
c
int *
```

5. `a + 1`

Since:

```
TEXT
a !' i n t *
```

then:

```
c
a + 1
```

moves by one `int`.

Assume:

```
TEXT
sizeof(int)=4
```

Then:

```
TEXT
a      !'   1 0 0 0
a + 1 !'   1 0 0 4
```

```
a + 2 !' 1 0 0 8
```

6. DEREFERENCE

```
C
*(a + 2)
```

means:

```
TEXT
go to address 1008
read the int
```

Result:

```
TEXT
30
```

Thus:

FORMULA BOX

```
TEXT
a[i] = *(a + i)
```

Ø=NOT KNOWN.

7. NOW &a

This is where many students get confused.

For:

```
C
int a[5];
```

&a means:

address of the entire array object.

Its type is:

```
C
int (*)[5]
```

Read:

pointer to an array of 5 integers.

8. SAME ADDRESS, DIFFERENT TYPE

Assume array starts at:

```
TEXT
1000
```

Then numerically:

```
TEXT
a           !'  1 0 0 0
& a [ 0 ]  !'  1 0 0 0
& a        !'  1 0 0 0
```

But their types differ.

Broadly:

```
TEXT
a           !'  i n t  *
& a [ 0 ]  !'  i n t  *
& a        !'  i n t  ( * ) [ 5 ]
```

Very important:

Same starting address does not mean same pointer type.

9. WHY TYPE MATTERS

Pointer arithmetic depends on pointed-to type.

For:

```
C
int a[5];
```

a + 1

moves by:

```
TEXT
sizeof(int)
```

&a + 1

moves by:

```
sizeof(a)
```

10. NUMERICAL EXAMPLE

```
Base = 1000
sizeof(int)=4
```

```
sizeof(a)=5×4=20
```

$$a + 1 = 1004$$

```
&a + 1
= 1020
```

11. VISUALIZATION

```

a
|
v
+---+---+---+---+---+
| 10 | 20 | 30 | 40 | 50 |
+---+---+---+---+---+
1000                                1020

```

$$\begin{array}{c} \wedge \\ | \\ \text{second int} \end{array}$$
Page 990 of 2799
www.gpttopdf.in

TEXT

^
|
after whole array

12. `*(&a)`

What does:

```
C
*(&a)
```

mean?

`&a` points to whole array.

Dereferencing it gives:

TEXT
the array object `a`

In most further expressions, that array again decays to pointer to first element.

So expressions involving:

```
C
*(&a)
```

often behave similarly to `a` in the next expression context.

But conceptually:

TEXT
`&a` !' pointer to whole array
`* &a` !' whole array

13. `sizeof(a)` VS `sizeof(&a)`

Assume:

```
C
int a[5];
```

and:

TEXT
`sizeof(int)=4`
pointer size=8

Then:

TEXT

```
sizeof(a)=20
```

because `a` is an actual array when used with `sizeof`.

But:

TEXT

```
sizeof(&a)=8
```

because `&a` is a pointer.

`sizeof` is one of the contexts where the array does not decay.

14. ARRAY DECAY EXCEPTIONS

Array-to-pointer conversion does **not** happen in certain important contexts.

Most important for GATE:

TEXT

```
sizeof(array)
```

and:

TEXT

```
&array
```

So:

C

```
sizeof(a)
```

sees the entire array.

And:

C

```
&a
```

gives pointer to entire array.

15. `sizeof(a)/sizeof(a[0])`

For:

C

```
int a[10];
```

same scope:

```
C
sizeof(a) / sizeof(a[0])
```

gives:

```
TEXT
10
```

Example:

```
TEXT
sizeof(a)=40
sizeof(a[0])=4
40/4=10
```

16. FUNCTION PARAMETER TRAP

Suppose:

```
C
void f(int a[])
{
    printf("%zu", sizeof(a));
}
```

Inside the function parameter list:

```
C
int a[]
```

is adjusted to:

```
C
int *a
```

So `sizeof(a)` inside `f` gives:

```
TEXT
pointer size
```

not original array size.

Extremely common GATE trap.

17. THESE FUNCTION PARAMETERS ARE EQUIVALENT

For function parameters:

```
C
void f(int a[])
```

and:

```
C
void f(int a[100])
```

and:

```
C
void f(int *a)
```

all essentially declare the parameter as:

```
C
int *
```

The written `100` does not make the whole array get passed by value.

18. IMPORTANT

C does **not** pass an entire ordinary array by value in a normal parameter like:

```
C
void f(int a[])
```

Instead the function receives a copy of a pointer to the first element.

Therefore modifying:

```
C
a[0]
```

modifies the caller's underlying array element.

19. EXAMPLE

```
C
void change(int a[])
{
    a[0] = 100;
}

int main(void)
{
    int x[] = {10, 20, 30};

    change(x);

    printf("%d", x[0]);
}
```

```
}
```

Output:

```
TEXT
100
```

But C is still:

```
TEXT
call by value
```

because the **pointer value** was copied.

20. CHANGING THE PARAMETER POINTER

Consider:

```
C
void f(int *p)
{
    p = NULL;
}
```

Caller:

```
C
int a[3];
int *p = a;

f(p);
```

Does caller's `p` become `NULL`?

No.

Because function received a copy of the pointer.

Same principle from our earlier pointer lessons.

21. POINTER SUBTRACTION

If two pointers point into the same array:

```
C
int *p = &a[4];
int *q = &a[1];
```

then:

```
C
p - q
```

gives element distance:

```
TEXT
3
```

not byte distance.

Pointer subtraction is measured in elements.

22. EXAMPLE

Addresses:

```
TEXT
p = 1016
q = 1004
```

Byte difference:

```
TEXT
12 bytes
```

Since each int is 4:

```
TEXT
12/4 = 3 elements
```

C pointer subtraction gives:

```
TEXT
3
```

23. ONE-PAST-ENDPOINTER

For:

```
C
int a[5];
```

this pointer may be formed:

```
C
a + 5
```

It points one past the last element.

Valid to form/use for comparison and loop termination.

But:

```
c
*(a + 5)
```

is invalid.

Don't dereference one-past-end.

24. COMMON POINTER LOOP

```
c
for (int *p = a; p < a + n; p++)
{
    printf("%d ", *p);
}
```

Here:

```
TEXT
p starts at a[0]
```

and stops before:

```
TEXT
a+n
```

This is pointer-style traversal.

Time:

```
TEXT
O(n)
```

25. ARRAY INDEX SYMMETRY TRAP

Remember:

```
c
a[i]
```

is defined using:

```
c
*(a+i)
```

Addition is commutative:

TEXT

```
a+i = i+a
```

Therefore bizarre but valid:

C

```
i[a]
```

means:

C

```
*(i+a)
```

which equals:

C

```
a[i]
```

0-6% TE curiosity.

26. NOW MOVE TO 2D ARRAYS

Consider:

C

```
int a[3][4];
```

This means:

array of 3 elements

where each element is:

TEXT

```
array of 4 int
```

So one row has type:

C

```
int [4]
```

27. WHAT DOES `a` DECAY TO?

`a` is an array of 3 rows.

Therefore in most expressions:

```
c
a
```

decays to pointer to its first row.

Type:

```
c
int (*)[4]
```

Read:

pointer to array of 4 integers.

~~0=1%~~ This is not `int *`.

28. WHY `a + 1` MOVES A WHOLE ROW

Since `a` points to:

```
TEXT
one row = array of 4 ints
```

then:

```
c
a + 1
```

moves by:

```
TEXT
sizeof(int[4])
```

Assume int size 4:

```
TEXT
4 × 4 = 16 bytes
```

So:

```
TEXT
a + 1  !'  n e x t   r o w
```

29. MEMORY EXAMPLE

```
c
int a[3][4];
```

Assume:

TEXT

```
Base = 1000
sizeof(int)=4
```

Each row:

TEXT

```
4 ints = 16 bytes
```

Therefore:

TEXT

```
a      !'    1 0 0 0
a  +  1  !'    1 0 1 6
a  +  2  !'    1 0 3 2
```

30. WHAT IS `*a` ?

Since:

TEXT

```
a !'  p o i n t e r   t o   f i r s t   r o w
```

then:

C

```
*a
```

is:

TEXT

```
first row
```

That row is an array of 4 ints.

In most further expressions, it decays to:

C

```
int *
```

pointing to:

TEXT

```
a[0][0]
```

31. NUMERICAL ADDRESSES

For:

```
C
int a[3][4];
```

these may all begin numerically at 1000:

```
TEXT
a
*a
&a[0]
&a[0][0]
```

But types differ.

Conceptually:

```
TEXT
a           !'  p o i n t e r   t o   r o w
&a [ 0 ]    !'  p o i n t e r   t o   r o w
*a          !'  f i r s t   r o w ,   t h e n   o f t e n   d e c a y s   t o   i n t *
&a [ 0 ] [ 0 ] !'  i n t *
```

0=1000: same address, different type.

32. `a + 1` VS `*a + 1`

Assume:

```
TEXT
Base = 1000
row size = 16
int size = 4
```

Then:

```
TEXT
a + 1
```

moves one row:

```
TEXT
1016
```

But:

```
C
*a + 1
```

moves one integer within first row:

```
TEXT
1004
```

33. EXPANDING `a[i][j]`

We know:

```
c
a[i]
```

means:

```
c
*(a+i)
```

Then:

```
c
a[i][j]
```

means:

```
c
*(*(a+i)+j)
```

FORMULA BOX

```
TEXT
a[i][j]
=
*(*(a+i)+j)
```

Ø=M MUST KNOW.

34. TRACE `a[2][3]`

Suppose:

```
c
int a[3][4];
```

Toget:

```
c
a[2][3]
```

Step 1:

TEXT

$a + 2$

moves to row 2.

Step 2:

TEXT

$*(a+2)$

gets row 2.

Step 3:

TEXT

$*(a+2) + 3$

moves to column 3.

Step 4:

TEXT

$*(*(a+2)+3)$

gets the integer.

35. ADDRESS FORM

Similarly:

C

$\&a[i][j]$

is equivalent to:

C

$*(a+i) + j$

because:

TEXT

$*(a+i)$

decays to pointer to first element of row `i`.

36. POINTER-TO-ARRAY DECLARATION

Suppose:

```
c
int (*p)[4];
```

How do we read it?

Start from `p`:

```
TEXT
p is a pointer
```

because:

```
TEXT
(*p)
```

then:

```
TEXT
[4]
```

means:

pointer to array of 4 ints.

So:

```
c
int (*p)[4];
```

is the correct pointer type for rows of:

```
c
int a[][4];
```

37. ASSIGNING 2D ARRAY TO ROW POINTER

```
c
int a[3][4];

int (*p)[4] = a;
```

Valid.

Because `a` decays to:

```
c
int (*)[4]
```

Then:

TEXT

```
p      !'   r o w   0
p  +   1  !'   r o w   1
p  +   2  !'   r o w   2
```

38. ACCESS THROUGH `p`

C

```
p[1][2]
```

works.

Why?

TEXT

```
p[1]
=
*(p+1)
```

which is row 1.

Then:

TEXT

```
p[1][2]
```

selects column 2.

Equivalent:

C

```
*( *(p+1)+2)
```

39. HUGE GATE TRAP — `int **` IS NOT A 2D ARRAY POINTER

Suppose:

C

```
int a[3][4];
```

Can we correctly treat `a` as:

C

```
int **
```

?

No.

~~0=Ý%0450KN0.~~
~~0=Ý%0450KN0.~~

a decays to:

```
c
int (*)[4]
```

not:

```
c
int **
```

These represent fundamentally different layouts/types.

40. WHY `int **` IS DIFFERENT

A true:

```
c
int **p;
```

means:

pointer to pointer to int.

Common conceptual structure:

```
TEXT
p
|
v
+-----+
| pointer | ---> row data
+-----+
| pointer | ---> another row
+-----+
```

There may be separate row pointers and separate allocations.

But:

```
c
int a[3][4];
```

is one contiguous block:

```
TEXT
row0 row1 row2
```

with no row-pointer objects stored between them.

41. VISUALIZATION — REAL 2D ARRAY

```
c
int a[3][4];
```

Memory:

```
TEXT
+-----+
| 4 ints row 0 |
+-----+
| 4 ints row 1 |
+-----+
| 4 ints row 2 |
+-----+
```

`a` points directly to first row.

No extra array of pointers exists.

42. VISUALIZATION — POINTER-TO-POINTER STYLE

Conceptually:

```
c
int **p;
```

could represent:

```
TEXT
p
|
v
+-----+ +-----+
| ptr | --> | row 0 ints |
+-----+ +-----+
| ptr | --> | row 1 ints |
+-----+ +-----+
| ptr | --> | row 2 ints |
+-----+ +-----+
```

Completely different.

43. FUNCTION PARAMETER FOR 2D ARRAY

Given:

```
c
int a[3][4];
```

valid function form:

```
c
void f(int a[][4])
```

Equivalent parameter type:

```
c
void f(int (*a)[4])
```

Not:

```
c
void f(int **a)
```

for the same actual 2D array representation.

Ø=important.

44. WHY NUMBER OF COLUMNS IS NEEDED

Compiler translates:

```
c
a[i][j]
```

roughly into:

```
c
*(*(a+i)+j)
```

To calculate:

```
c
a+i
```

it must know how large one row is.

For:

```
c
int a[][4]
```

one row size:

```
TEXT
4 × sizeof(int)
```

Thus column count is essential.

45. POINTER DECLARATION PRECEDENCE TRAP

Compare:

```
C
int *p[4];
```

and:

```
C
int (*p)[4];
```

They are completely different.

0-6% favorite.

46. `int *p[4]`

Because `[]` has higher precedence:

```
C
int *p[4];
```

means:

p is an array of 4 pointers to int.

Visualization:

```
TEXT
p [ 0 ] !' i n t *
p [ 1 ] !' i n t *
p [ 2 ] !' i n t *
p [ 3 ] !' i n t *
```

47. `int (*p)[4]`

Parentheses force:

```
TEXT
*p
```

first.

So:

```
C
int (*p)[4];
```

means:

p is a pointer to an array of 4 ints.

Visualization:

TEXT

```
p  
|  
v  
[int][int][int][int]
```

0-100% parentheses completely change meaning.

48. MEMORY TRICK

TEXT

```
int *p[4]
```

ARRAY first !' array of pointers.

TEXT

```
int (*p)[4]
```

POINTER first !' pointer to array.

49. sizeof WITH 2D ARRAY

Suppose:

C

```
int a[3][4];
```

Assume:

TEXT

```
sizeof(int)=4
```

Then:

TEXT

```
sizeof(a)=3×4×4=48
```

One row:

TEXT

```
sizeof(a[0])=4×4=16
```

One integer:

TEXT

```
sizeof(a[0][0])=4
```

50. DERIVING DIMENSIONS

Same scope:

C

```
sizeof(a) / sizeof(a[0])
```

gives rows:

TEXT

```
48 / 16 = 3
```

And:

C

```
sizeof(a[0]) / sizeof(a[0][0])
```

gives columns:

TEXT

```
16 / 4 = 4
```

51. sizeof(*a)

For:

C

```
int a[3][4];
```

*a is first row.

Under `sizeof`, the row does not decay.

Therefore:

C

```
sizeof(*a)
```

equals:

TEXT

```
sizeof(int[4])
```

So under 4-byte ints:

TEXT

16

Ø=Not a GATE expression.

52. sizeof(**a)

**a is:

TEXT

a[0][0]

an int.

So:

C

sizeof(**a)

is:

TEXT

sizeof(int)

Under assumption:

TEXT

4

53. GATE TABLE

For:

C

int a[3][4];

assuming sizeof(int)=4 and pointer size 8:

Expression	Meaning	Size
sizeof(a)	whole matrix	48
sizeof(a[0])	one row	16
sizeof(a[0][0])	one int	4
sizeof(*a)	one row	16
sizeof(**a)	one int	4
sizeof(&a)	pointer	8

Copy this.

54. &a FOR 2D ARRAY

For:

```
c
int a[3][4];
```

what is type of:

```
c
&a
```

?

It is pointer to the whole 3×4 array:

```
c
int (*)(3)[4]
```

So:

```
c
&a + 1
```

moves past the entire matrix.

55. COMPARE MOVEMENT

Assume:

```
TEXT
int size = 4
matrix = 3×4
```

Then:

```
TEXT
a + 1
```

moves one row:

```
TEXT
4 ints = 16 bytes
```

But:

TEXT

&a + 1

moves whole matrix:

TEXT

3×4×4 = 48 bytes

0 = No other GATE trap.

56. NUMERICAL EXAMPLE

Base:

TEXT

1000

Then:

TEXT

a + 1
! ' 1 0 1 6

while:

TEXT

&a + 1
! ' 1 0 4 8

under assumptions.

57. POINTER ARITHMETIC AND ROW-MAJOR STORAGE

For:

C

```
int a[3][4];
```

row-major storage naturally matches:

C

a + i

because each increment skips one complete row.

Then:

C

*(a+i) + j

moves within that row.

This pointer logic is exactly why row-major formula is:

TEXT

$i \times \text{columns} + j$

58. OUTPUT TRACE EXAMPLE

```
C
#include <stdio.h>

int main(void)
{
    int a[] = {10, 20, 30, 40};
    int *p = a;

    printf("%d %d %d",
           *p,
           *(p + 2),
           p[3]);

    return 0;
}
```

Output:

TEXT

10 30 40

59. OUTPUT TRACE — POINTER INCREMENT

```
C
int a[] = {10, 20, 30};
int *p = a;

printf("%d ", *p);
p++;
printf("%d ", *p);
```

Output:

TEXT

10 20

because `p++` moves one integer.

60. PRECEDENCE TRAP

```
C
*p++
```

means:

```
C
*(p++)
```

because postfix `++` binds tighter than unary `*`.

So it:

1. dereferences current pointer,
2. then increments pointer.

61. COMPARE FOUR EXPRESSIONS

Suppose:

```
C
int a[] = {10, 20, 30};
int *p = a;
```

***p++**

Equivalent:

```
C
*(p++)
```

use current value, then advance pointer.

(*p)++

Increment pointed value.

Array becomes:

```
TEXT
11 20 30
```

Pointer unchanged.

***++p**

Increment pointer first.

Then dereference.

Result:

TEXT

20

++*p

Equivalent:

```
C
++(*p)
```

increment pointed value first.

Ø=No visit from pointer module.

62. ARRAY NAME CANNOT BE INCREMENTED

This is invalid:

```
C
a++;
```

because array name is not a modifiable pointer variable.

But:

```
C
int *p = a;
p++;
```

is valid.

Ø=No array name may **behave like an address** in expressions, but it is not a pointer variable.

63. ARRAY NAME VS POINTER VARIABLE

```
C
int a[5];
int *p = a;
```

Both may initially represent address of first element.

But:

```
C
p = p + 1;
```

valid.

```
c
a = a + 1;
```

invalid.

Why?

`p` stores an address as a modifiable pointer object.

`a` is the array object name.

64. GATE TRAP — `sizeof`

For:

```
c
int a[5];
int *p = a;
```

assume `int=4`, `pointer=8`:

```
TEXT
sizeof(a) = 20
sizeof(p) = 8
```

Even though both may indicate the same starting address.

`0x5000` same address "" same object/type.

65. ARRAY POINTER TO SPECIFIC ROW SIZE

Suppose:

```
c
int (*p)[5];
```

Then:

```
c
p + 1
```

always moves:

```
TEXT
5 integers
```

because pointed-to type is:

```
c
int[5]
```

This is why pointer type carries important structural information.

66. PLACEMENT CONNECTION

Understanding arrays/pointers deeply helps with:

- dynamic arrays
- strings
- matrix manipulation
- binary search
- C interviews
- memory bugs
- buffer boundaries
- linked structures
- pointer-based APIs

For pure coding interviews, you may rarely manually use pointer-to-array syntax, but it is highly useful for **GATE C questions**.

67. GATE TRAP SHEET

Memorize these traps:

Trap 1

TEXT
a

usually decays to pointer to first element.

Trap 2

TEXT
&a

points to whole array.

Trap 3

`a+1` and `&a+1` move different distances.

Trap 4

`sizeof(a)` gives whole array size in same scope.

Trap 5

Array parameter becomes pointer parameter.

Trap 6

For 2D array:

TEXT

a !' p o i n t e r t o r o w

not pointer to first int type-wise.

Trap 7

C

```
int **p
```

is not equivalent to:

C

```
int a[R][C]
```

Trap 8

C

```
int *p[N]
```

”

C

```
int (*p)[N]
```

Trap 9

TEXT

```
a[i][j] = *(*a+i)+j)
```

Trap 10

One-past-end pointer can be formed but not dereferenced.

68. MASTER TYPE TABLE — 1D

For:

C

```
int a[5];
```

Expression	Conceptual type/value
a	usually int * to first element
&a[0]	int *
a+1	pointer to next int
&a	int (*)[5]

Expression	Conceptual type/value
&a+1	pointer after whole array
*a	first int
a[i]	int element

69. MASTER TYPE TABLE — 2D

For:

```
c
int a[3][4];
```

Expression	Meaning
a	pointer to first row, int (*)[4]
a+1	pointer to second row
*a	first row
*a + 1	pointer to a[0][1]
*(a+1)	second row
**a	a[0][0]
a[i][j]	element
&a	pointer to whole 3×4 array

Ø=Very high-value table.

70. MEMORY TRICKS

1D

TEXT

```
a+1
!' o n e   e l e m e n t
```

TEXT

```
&a+1
!' o n e   w h o l e   a r r a y
```

2D

TEXT

```
a+1
!' one row
```

TEXT

```
*a+1
!' one element inside row
```

Pointer declaration

TEXT

```
*p[N]
!' array of pointers

(*p)[N]
!' pointer to array
```

71. NOTEBOOK NOTES

1D

TEXT

```
a[i] = *(a+i)
```

TEXT

```
a           !' first element address in most expressions
&a          !' whole array address
```

If `int a[N]`:

TEXT

```
a + 1      !' sizeof (int)
&a + 1     !' sizeof (a)
```

2D

For:

C

```
int a[R][C];
```

TEXT

```
a !' int ( * ) [ C ]
```

TEXT

```
a[i][j] = (*(a+i)+j)
```

TEXT

a + 1 ! ' n e x t r o w

Function

```
C
void f(int a[])
```

means pointer parameter.

For 2D:

```
C
void f(int a[][C])
```

equivalent to:

```
C
void f(int (*a)[C])
```

72. 30-SECOND REVISION

Remember:

1. `a[i] = *(a+i)`.
2. `a` usually becomes pointer to first element.
3. `&a` points to whole array.
4. `a+1` moves one element in 1D.
5. `&a+1` moves one entire array.
6. In 2D, `a+1` moves one row.
7. `a[i][j] = (*(a+i)+j)`.
8. 2D array decays to pointer to row.
9. `int **` is not same as actual 2D array.
10. `int *p[N]` and `int (*p)[N]` are different.

73. ACTIVE RECALL

Q1

For:

```
C
int a[5];
```

what is the type of:

```
c
&a
```

?

Q2

If int size is 4 and base is 1000, what numerical address does:

```
c
a + 2
```

represent?

Q3

If:

```
c
int a[3][4];
```

what does:

```
c
a + 1
```

move over?

Q4

Write:

```
c
a[i][j]
```

in pointer notation.

Q5

Are:

```
c
int **p
```

and:

```
c
int a[3][4]
```

equivalent representations?

ANSWERS

A1

```
c
int (*)[5]
```

pointer to array of 5 ints.

A2

```
TEXT
1000 + 2×4
= 1008
```

A3

One complete row of 4 integers.

A4

```
c
*(*(a+i)+j)
```

A5

No.

74. PRACTICE

Q1

Given:

```
c
int a[6];
```

assume:

```
TEXT
base = 2000
sizeof(int)=4
```

Find numerical values of:

```
TEXT
a
a+1
a+5
&a+1
```

Q2

Given:

```
C
int a[] = {5,10,15,20};
```

find:

```
TEXT
*(a+3)
2[a]
```

Q3

Assume:

```
TEXT
sizeof(int)=4
pointer size=8
```

For:

```
C
int a[10];
int *p = a;
```

find:

```
TEXT
sizeof(a)
sizeof(p)
sizeof(&a)
```

Q4 — 2D MOVEMENT

Given:

```
C
int a[3][5];
```

with int size 4 and base 1000, find numerical addresses of:

```
TEXT
a
a+1
*a
*a+1
```

Q5 — POINTER NOTATION

Convert:

```
C
a[2][3]
```

fully into pointer notation.

Q6 — DECLARATION

Explain the difference:

```
C
int *p[10];
```

versus:

```
C
int (*p)[10];
```

Q7 — GATE sizeof

Assume:

```
TEXT
sizeof(int)=4
pointer size=8
```

Given:

```
C
int a[3][4];
```

find:

```
TEXT
sizeof(a)
sizeof(a[0])
sizeof(*a)
sizeof(**a)
sizeof(&a)
```

Q8 — FUNCTION PARAMETER

Why does:

```
C
void f(int a[100])
{
    printf("%zu", sizeof(a));
}
```

```
}
```

not print the size of 100 integers?

What object does parameter `a` actually represent inside the function?

0=09 — CONCEPTUAL

Explain why:

```
c
int **p;
```

cannot automatically be used as the same representation as:

```
c
int a[3][4];
```

Draw both conceptual memory layouts.

PROGRESS

MODULE 2 — ARRAYS

' Lesson 22 — Fundamentals & Memory ' Lesson 23 — 1D Operations ' Lesson 24 — 2D Arrays ' Lesson 25 — Row/Column-Major Address Formulas ' **Lesson 26 — Arrays + Pointers Deep Consolidation**

NEXT — LESSON 27

Searching in Arrays — Linear Search & Binary Search

Next we cover:

- linear search
- binary search
- iterative binary search
- recursive binary search
- precondition: sorted array
- `low`, `high`, `mid`
- why `mid = low + (high-low)/2`
- best/worst-case comparisons
- `O(n)` vs `O(log n)`
- exact iteration intuition
- unsuccessful search
- duplicate-element issues
- first/last occurrence extension

- common GATE traps
- placement questions

Then **Lesson 28** will cover the high-frequency placement array patterns: **Two Pointers, Sliding Window, Prefix Sum, Frequency Arrays and Kadane's Algorithm.**

USER

next

ASSISTANT

MODULE 2 — LESSON 27

Searching in Arrays — Linear Search & Binary Search

This is one of the most important array topics for both **GATE and placements**.

You must understand not only the code, but also:

- when each search can be used
- why binary search is faster
- why binary search requires sorted data
- how `low`, `high`, and `mid` move
- successful vs unsuccessful search
- best/worst-case complexity
- duplicate-element behavior
- common GATE traps
- placement extensions like first/last occurrence

GATE importance: Ø=Ý%ØMÚ%ØKÍØW **Placement importance:** Ø=Ý%ØMÚ%ØKÍØW

1. WHERE WE ARE

Completed:

- Array fundamentals
- 1D operations
- 2D arrays
- Row-major/column-major addresses
- Arrays + pointers

Now:

Searching

We study:

1. Linear Search
2. Binary Search

2. STORY / INTUITION

Suppose you have these roll numbers:

TEXT

52 17 88 31 64

You want to find:

TEXT

31

There is no useful order.

So you may have to check:

TEXT

52? No
17? No
88? No
31? Yes

This is:

Linear Search

Now suppose the numbers are sorted:

TEXT

10 20 30 40 50 60 70

Want:

TEXT

60

Instead of starting from 10, check middle:

TEXT

40

Since:

TEXT

60 > 40

everything left of 40 can be discarded.

Then search only:

TEXT

50 60 70

This is:

Binary Search

3. LINEAR SEARCH

Linear search examines array elements sequentially until:

- key is found, or
- array ends

Example:

```
C
int linearSearch(int a[], int n, int key)
{
    for (int i = 0; i < n; i++)
    {
        if (a[i] == key)
            return i;
    }

    return -1;
}
```

4. TRACE LINEAR SEARCH

Array:

TEXT

10 40 20 70 50

Search:

TEXT

70

Trace:

TEXT

i = 0	!	10	==	70?	No
i = 1	!	40	==	70?	No
i = 2	!	20	==	70?	No
i = 3	!	70	==	70?	Yes

Return:

TEXT

3

5. LINEAR SEARCH COMPLEXITY

Best case

Key is first:

TEXT

1 comparison

So:

TEXT

$\sim (1)$

Worst case

Key is:

- last element, or
- absent

Need:

TEXT

n comparisons

So:

TEXT

$\sim (n)$

Average case

Still linear:

TEXT

$\sim (n)$

Auxiliary space:

TEXT

$\sim (1)$

6. LINEAR SEARCH DOES NOT REQUIRE SORTING

Example:

TEXT

50 3 89 1 72

Linear search works perfectly.

It only asks:

TEXT

Is this the key?

So:

Linear search works on sorted and unsorted arrays.

Ø=Ý%Ø=Ý%Ø=Ý%
Important.

7. WHEN LINEAR SEARCH IS REASONABLE

Linear search is useful when:

- array is small
- array is unsorted
- search happens only once/few times
- sorting would cost more than the search benefit

Do not automatically binary-search every array.

8. NOW BINARY SEARCH Ø=Ý%Ø=Ý%Ø=Ý%

Binary search repeatedly divides the search interval into two halves.

Main requirement:

The array must be sorted according to the same ordering used by the search.

Example:

TEXT

10 20 30 40 50 60 70

Search:

TEXT

60

9. INITIAL VARIABLES

For array of `n` elements:

```
C
int low = 0;
int high = n - 1;
```

Then compute:

```
C
int mid = low + (high - low) / 2;
```

We compare:

```
TEXT
a[mid]
```

with:

```
TEXT
key
```

10. THREE CASES

Case 1

```
C
a[mid] == key
```

Found.

Case 2

```
C
key < a[mid]
```

Search left half:

```
C
high = mid - 1;
```

Case 3

```
C
key > a[mid]
```

Search right half:

```
C
low = mid + 1;
```

0-100% This is the entire binary-search logic.

11. ITERATIVE BINARY SEARCH

```
c
int binarySearch(int a[], int n, int key)
{
    int low = 0;
    int high = n - 1;

    while (low <= high)
    {
        int mid = low + (high - low) / 2;

        if (a[mid] == key)
            return mid;

        if (key < a[mid])
            high = mid - 1;
        else
            low = mid + 1;
    }

    return -1;
}
```

12. TRACE BINARY SEARCH

Array:

TEXT

Index:

0 1 2 3 4 5 6

10 20 30 40 50 60 70

Search:

TEXT

60

Initially:

TEXT

low = 0

high = 6

Mid:

TEXT

mid = 0 + (6-0)/2
= 3

So:

TEXT

```
a[3] = 40
```

Since:

TEXT

```
60 > 40
```

discard:

TEXT

```
10 20 30 40
```

Now:

TEXT

```
low = 4  
high = 6
```

13. SECOND ITERATION

TEXT

```
mid = 4 + (6-4)/2  
    = 5
```

TEXT

```
a[5] = 60
```

Found.

Return:

TEXT

```
5
```

Only:

TEXT

```
2 comparisons with middle elements
```

instead of potentially scanning 6 elements.

14. WHY BINARY SEARCH IS FAST

Every unsuccessful comparison cuts the remaining search region roughly in half.

TEXT

```
n
!"
n/2
!"
n/4
!"
n/8
!"
...
1
```

Number of halvings:

TEXT

```
l o g ,   n
```

Therefore:

FORMULA BOX

TEXT

```
B i n a r y   S e a r c h   T i m e   =   ~ ( l o g   n )
```

in the worst case.

15. EXAMPLE WITH 1,024 ELEMENTS

Linear search might inspect:

TEXT

```
1024
```

elements.

Binary search needs roughly:

TEXT

```
l o g , 1 0 2 4   =   1 0
```

halving steps.

That's the power of logarithmic search.

16. BINARY SEARCH COMPLEXITY

Best case

Key is exactly at first middle checked.

TEXT

$\sim (1)$

Worst case

Repeatedly halve until one/no element remains.

TEXT

$\sim (\log n)$

Auxiliary space — iterative version

Only uses:

TEXT

low, high, mid

So:

TEXT

$\sim (1)$

17. RECURSIVE BINARY SEARCH

```
C
int binarySearch(int a[], int low, int high, int key)
{
    if (low > high)
        return -1;

    int mid = low + (high - low) / 2;

    if (a[mid] == key)
        return mid;

    if (key < a[mid])
        return binarySearch(a, low, mid - 1, key);

    return binarySearch(a, mid + 1, high, key);
}
```

18. RECURSIVE COMPLEXITY

Each call handles approximately half:

TEXT

$T(n) = T(n/2) + O(1)$

Therefore:

TEXT

`Time = ~ ( log n )`

Recursion depth:

TEXT

`~ ( log n )`

So recursive auxiliary stack:

TEXT

`~ ( log n )`

19. ITERATIVE VS RECURSIVE

Version	Time	Auxiliary space
Iterative binary search	$\sim(\log n)$	$\sim(1)$
Recursive binary search	$\sim(\log n)$	$\sim(\log n)$

0-Placement/interview favorite.

If both are equally clear, iterative binary search is usually more space-efficient.

20. WHY `low <= high`?

Suppose only one element remains:

TEXT

`low == high`

That element still needs to be checked.

So condition must allow:

TEXT

`low == high`

Hence:

C

`while (low <= high)`

not blindly:

C

`while (low < high)`

for this standard implementation.

21. UNSUCCESSFUL SEARCH

Array:

```
TEXT
10 20 30 40 50
```

Search:

```
TEXT
35
```

Eventually you may reach:

```
TEXT
low > high
```

Example:

```
TEXT
low = 3
high = 2
```

This means:

Search interval is empty.

Return:

```
TEXT
-1
```

22. BASE CASE OF RECURSIVE BINARY SEARCH

```
C
if (low > high)
    return -1;
```

means:

There are no valid elements left to search.

$\emptyset = \text{Must understand, not just memorize.}$

23. WHY SORTING IS REQUIRED

Suppose:

TEXT

10 80 20 70 30

Search:

TEXT

20

Middle:

TEXT

20

might luckily work.

But search:

TEXT

80

could fail if we use ordering logic incorrectly.

Binary search assumes:

TEXT

```
if key < middle
!' key cannot be on right

if key > middle
!' key cannot be on left
```

That conclusion is only valid because the array is sorted.

24. GATE TRAP

Binary search on arbitrary unsorted data is:

TEXT

not valid

Even if one specific input accidentally produces the right answer.

The algorithm's correctness depends on sorted ordering.

25. ASCENDING VS DESCENDING

Binary search can also work on a descending array.

Example:

TEXT

90 80 70 60 50 40

But comparison directions must be adjusted.

So the deeper requirement is:

Data must be ordered, and your branching logic must match that order.

26. MIDPOINT FORMULA

You often see:

C

```
mid = (low + high) / 2;
```

Mathematically fine.

But in fixed-width integer arithmetic:

TEXT

low + high

can overflow for very large indices.

Safer:

C

```
mid = low + (high - low) / 2;
```

✓
Very common interview expectation.

27. WHY SAFER FORM WORKS

Since:

TEXT

```
high >= low
```

within active search,

TEXT

```
high - low
```

is the width of current interval.

Half of it:

TEXT

```
(high-low)/2
```

moves from `low` toward the middle.

So:

TEXT

```
low + half
```

produces middle without adding two potentially very large indices first.

28. FLOOR BEHAVIOR

With integers:

TEXT

```
low = 0  
high = 5
```

Then:

TEXT

```
mid = (0+5)/2  
= 2
```

because integer division truncates.

So lower middle is chosen.

For 6 elements:

TEXT

```
indices 0...5
```

possible middle pair:

TEXT

```
2 and 3
```

this formula selects:

TEXT

```
2
```

That's perfectly valid.

29. UPPER MID

Sometimes algorithms intentionally use:

```
c
```

```
mid = low + (high - low + 1) / 2;
```

to choose the upper middle.

This is useful in some boundary-search variants.

0-1 know that different midpoint choices can be correct depending on update rules.

For standard search, use the familiar lower-mid version.

30. BINARY SEARCH INVARIANT

A powerful way to understand binary search:

If the key exists, it must remain inside the current search interval `[low, high]`.

Every update preserves this idea.

If:

```
TEXT
```

```
key < a[mid]
```

then `mid` and everything right can be removed.

So:

```
c
```

```
high = mid - 1;
```

31. WHY `mid - 1` AND `mid + 1`?

Because we already checked:

```
TEXT
```

```
a[mid]
```

and it wasn't equal.

So there is no reason to include `mid` again.

Use:

```
c
```

```
high = mid - 1;
```

or:

```
c
```

```
low = mid + 1;
```


~~O(1)~~ avoids repeated middle and possible infinite loops.

32. COMMON WRONG VERSION

```
C
if (key < a[mid])
    high = mid;
else
    low = mid;
```

With certain interval conventions this can fail to make progress.

Example:

```
TEXT
low = 4
high = 5
mid = 4
```

Then:

```
TEXT
low = mid = 4
```

No change.

Loop may repeat forever.

~~O(1)~~ Binary search correctness depends on matching:

- interval definition
- midpoint choice
- update rules

33. STANDARD SAFE TEMPLATE

For inclusive interval:

```
TEXT
[low, high]
```

use:

```
C
while (low <= high)
{
    mid = low + (high-low)/2;

    if (a[mid] == key)
        return mid;
    else if (key < a[mid])
        high = mid - 1;
    else
        low = mid + 1;
```

```
}
```

Memorize this template.

34. LINEAR VS BINARY SEARCH

Property	Linear	Binary
Requires sorted data	No	Yes
Best	$O(1)$	$O(1)$
Worst	$O(n)$	$O(\log n)$
Easy on linked list	Yes logically	Not efficient for ordinary linked list
Uses random access well	Not necessary	Yes
Suitable for unsorted one-time search	Yes	No

35. WHY BINARY SEARCH FITS ARRAYS

Binary search repeatedly needs:

TEXT
middle element

Arrays support:

TEXT
`a[mid]`

in:

TEXT
 $O(1)$

because of random access.

That makes binary search efficient.

36. BINARY SEARCH ON LINKED LIST — GATE CONCEPT

Suppose you have a sorted singly linked list.

Can you conceptually find a middle?

Yes.

But getting to the middle itself requires traversal:

TEXT

$O(n)$

So standard binary search loses much of its advantage.

Binary search is especially suited to random-access structures like arrays.

37. EXACT COMPARISON INTUITION

Suppose:

TEXT

$n = 8$

Binary search reduces approximately:

TEXT

8
4
2
1

So around:

TEXT

$\log_2 8 + 1$

middle checks in the worst-case depending on exact success/failure and convention.

For asymptotic questions:

TEXT

$\sim (\log_2 n)$

is what matters.

38. UNSUCCESSFUL SEARCH COMPARISONS

For unsuccessful binary search, the search continues until:

TEXT

interval becomes empty

So worst-case comparisons are on the order of:

TEXT

$\text{floor}(\log_2 n) + 1$

for common formulations, with exact counts depending on n and implementation.

Do not overgeneralize an exact formula without checking convention.

39. GATE TREE INTERPRETATION

Binary search decisions can be visualized as a decision tree.

Each comparison chooses:

```
TEXT
left half
or
right half
```

A balanced decision tree with n candidate positions has height:

```
TEXT
~ ( log n )
```

This gives another explanation for logarithmic complexity.

40. DUPLICATE VALUES

Suppose:

```
TEXT
1 2 2 2 2 3 4
```

Search for:

```
TEXT
2
```

Standard binary search may return **any occurrence** it encounters first.

Possible index:

```
TEXT
3
```

It does not automatically guarantee:

- first occurrence
- last occurrence

Ø = Important placement trap.

41. FIRST OCCURRENCE

Suppose we want the leftmost 2 .

Array:

TEXT

1 2 2 2 2 3

When we find 2, do not immediately stop.

Instead:

TEXT

record index
continue searching LEFT

42. FIRST OCCURRENCE CODE

```
C
int firstOccurrence(int a[], int n, int key)
{
    int low = 0;
    int high = n - 1;
    int ans = -1;

    while (low <= high)
    {
        int mid = low + (high - low) / 2;

        if (a[mid] == key)
        {
            ans = mid;
            high = mid - 1;
        }
        else if (a[mid] < key)
        {
            low = mid + 1;
        }
        else
        {
            high = mid - 1;
        }
    }

    return ans;
}
```

Complexity:

TEXT

$O(\log n)$

43. MEMORY TRICK

Found target but want:

First occurrence

Found? Go LEFT.

```
C
high = mid - 1;
```

Last occurrence

Found? Go RIGHT.

```
C
low = mid + 1;
```

Great placement shortcut.

44. LAST OCCURRENCE CODE

```
C
int lastOccurrence(int a[], int n, int key)
{
    int low = 0;
    int high = n - 1;
    int ans = -1;

    while (low <= high)
    {
        int mid = low + (high - low) / 2;

        if (a[mid] == key)
        {
            ans = mid;
            low = mid + 1;
        }
        else if (a[mid] < key)
        {
            low = mid + 1;
        }
        else
        {
            high = mid - 1;
        }
    }

    return ans;
}
```

45. COUNT OCCURRENCES IN SORTED ARRAY

If key exists from:

```
TEXT
first
```

to:

TEXT

last

inclusive, count:

FORMULA BOX

TEXT

count = last - first + 1

Both first and last can be found in:

TEXT

$O(\log n)$

So total remains:

TEXT

$O(\log n)$

46. EXAMPLE

Array:

TEXT

1 2 2 2 2 3 4

For key 2:

TEXT

first = 1

last = 4

Count:

TEXT

$4 - 1 + 1$
 $= 4$

47. LOWER BOUND INTUITION — PLACEMENT EXTENSION

Another useful binary-search problem:

Find first position where value is not less than key.

This is often called:

TEXT

lower bound

Example:

TEXT

1 3 5 7

Key:

TEXT

4

First value ≥ 4 is:

TEXT

5

index:

TEXT

2

This concept is widely used in interview problems.

Ø=We'll revisit during placement pattern practice.

48. UPPER BOUND INTUITION

Upper bound commonly means:

First position whose value is strictly greater than key.

Example:

TEXT

1 2 2 2 5

For key 2:

first value greater than 2:

TEXT

5

So upper-bound position points after all 2 s.

Ø=Placement extension.

49. BINARY SEARCH IS MORE GENERAL THAN "FIND KEY"

This is crucial for placements.

Binary search can search for:

- exact value
- first occurrence
- last occurrence
- insertion position
- first value satisfying condition
- minimum feasible answer
- maximum feasible answer

This broader pattern is sometimes called:

Binary Search on Answer / Monotonic Condition

We'll revisit it later.

50. GATE TRAP — SORTING COST

Suppose you have an unsorted array and want to search **once**.

Could you sort it first and binary search?

Sorting typically costs:

TEXT

$O(n \log n)$

then binary search:

TEXT

$O(\log n)$

Total:

TEXT

$O(n \log n)$

Linear search alone:

TEXT

$O(n)$

So sorting first for only one search can be worse.

0-1% important algorithmic reasoning.

51. MANY SEARCH QUERIES

Suppose you will search the same data many times.

You might sort once:

```
TEXT
O(n log n)
```

then perform each search:

```
TEXT
O(log n)
```

If there are many queries, the preprocessing cost can become worthwhile.

This is a common design trade-off.

52. EXAMPLE — 1,000 QUERIES

Unsorted array size `n`.

Linear searching every query:

```
TEXT
O(1000n)
```

Sort once + binary search:

```
TEXT
O(n log n + 1000 log n)
```

Depending on `n` and number of queries, the second can be much better.

53. GATE TRAP — BINARY SEARCH WORST CASE

Some students say:

```
TEXT
O(n/2)
```

because half is removed.

Wrong.

The process repeatedly halves:

```
TEXT
n
n/2
n/4
n/8
...
```

Number of steps is:

TEXT

$\log n$

not $n/2$.

54. GATE TRAP — BINARY SEARCH SPACE

Iterative:

TEXT

$O(1)$

Recursive:

TEXT

$O(\log n)$

Do not blindly say binary search always uses $O(\log n)$ space.

Depends on implementation.

55. GATE TRAP — KEY FOUND AT MIDDLE

If first comparison finds key:

TEXT

Best time = $O(1)$

Do not answer $O(\log n)$ for best-case time.

Worst:

TEXT

$O(\log n)$

56. GATE TRAP — EMPTY ARRAY

For:

TEXT

$n = 0$

standard initialization:

C

```
low = 0;
high = n - 1;
```

gives conceptually:

TEXT

```
high = -1
```

Then:

TEXT

```
low <= high
```

is false.

Search correctly returns:

TEXT

```
-1
```

assuming signed integer indices as written.

57. GATE TRAP — DUPLICATES

For:

TEXT

```
1 2 2 2 3
```

standard binary search returning index of 2 does **not** guarantee first index.

If question explicitly asks:

TEXT

```
first occurrence
```

you must modify binary search.

58. GATE TRAP — DESCENDING ORDER

If array is descending:

TEXT

```
90 80 70 60
```

this standard ascending comparison:

C

```
if (key < a[mid])  
    high = mid - 1;
```

is no longer correct.

Directions must reverse appropriately.

59. GATE TRAP — OVERFLOW-SAFE MID

Preferred:

```
C
mid = low + (high - low) / 2;
```

instead of:

```
C
mid = (low + high) / 2;
```

for robustness.

60. COMPARISON — SEARCHING COMPLEXITIES

Algorithm	Requirement	Best	Worst	Space
Linear Search	none	$O(1)$	$O(n)$	$O(1)$
Binary Search iterative	sorted	$O(1)$	$O(\log n)$	$O(1)$
Binary Search recursive	sorted	$O(1)$	$O(\log n)$	$O(\log n)$

Copy this.

61. SEARCHING AND ARRAYS VS LINKED LISTS

Array

```
TEXT
middle access =  $O(1)$ 
```

So binary search works efficiently.

Linked list

To reach middle position requires traversal.

So random middle access is not $O(1)$.

This is a major reason arrays support binary search naturally.

62. PLACEMENT PROBLEM — SEARCH INSERT POSITION

Array:

```
TEXT
1 3 5 6
```

Key:

TEXT

5

answer:

TEXT

2

Key:

TEXT

2

where should it be inserted?

TEXT

1 2 3 5 6

index:

TEXT

1

Binary search can find this position in:

TEXT

$O(\log n)$

63. PLACEMENT PROBLEM — FIRST TRUE

Suppose you have a monotonic condition:

TEXT

False False False True True True

You want:

TEXT

first True

Binary search can find the boundary.

This pattern appears in problems like:

- minimum feasible capacity
- earliest day
- minimum speed
- smallest valid answer

~~0-50~~ Extremely important later.

64. HOW TO RECOGNIZE BINARY SEARCH IN INTERVIEWS

Ask:

1. Is the data sorted?
2. Or is there a monotonic true/false condition?
3. Can I eliminate half the possibilities after one check?

If yes, binary search may apply.

65. INTERVIEW QUESTIONS

Q1. Why does binary search require sorted data?

Because comparison with the middle must tell which half can be safely discarded.

Q2. Worst-case binary search time?

TEXT

$O(\log n)$

Q3. Iterative auxiliary space?

TEXT

$O(1)$

Q4. Recursive auxiliary space?

TEXT

$O(\log n)$

Q5. Why use `low + (high - low) / 2`?

To avoid possible overflow from `low + high`.

Q6. Can binary search return any duplicate occurrence?

Standard implementation may return whichever matching middle it encounters.

66. MEMORY TRICKS

Linear Search

LOOK ONE BY ONE

TEXT

$O(n)$

Binary Search

MIDDLE !' THROW HALF AWAY

TEXT

$O(\log n)$

First occurrence

Found? Go LEFT

Last occurrence

Found? Go RIGHT

Empty interval

TEXT

low > high

67. NOTEBOOK NOTES

Linear Search

```
C
for (i = 0; i < n; i++)
    if (a[i] == key)
        return i;
```

Worst:

TEXT

$O(n)$

Binary Search

TEXT

Requirement:
SORTED ARRAY


```

c
low = 0;
high = n - 1;

while (low <= high)
{
    mid = low + (high-low)/2;

    if (a[mid] == key)
        found;
    else if (key < a[mid])
        high = mid - 1;
    else
        low = mid + 1;
}

```

Worst:

TEXT
 $O(\log n)$

68. BINARY SEARCH RECURRENCE

Recursive:

TEXT
 $T(n)$
 $=$
 $T(n/2) + O(1)$

Therefore:

TEXT
 $\sim (\log n)$

69. 30-SECOND REVISION

Remember:

1. Linear search works on unsorted data.
2. Linear worst case = $O(n)$.
3. Binary search requires ordered data.
4. Binary search checks middle.
5. Every failure removes roughly half.
6. Binary worst case = $O(\log n)$.
7. Iterative space = $O(1)$.
8. Recursive space = $O(\log n)$.
9. Standard duplicate search may return any match.
10. First match !' keep going left; last match !' keep going right.

70. ACTIVE RECALL

Q1

Can binary search be correctly used on arbitrary unsorted arrays?

Q2

What is worst-case linear-search time?

Q3

What is worst-case binary-search time?

Q4

Why do we use:

```
C
high = mid - 1;
```

rather than `high = mid` in the standard inclusive version?

Q5

If target is found but we want its first occurrence, which direction do we continue?

ANSWERS

A1

No.

A2

```
TEXT
O(n)
```

A3

```
TEXT
O(log n)
```

A4

`mid` has already been checked and is not the target, so excluding it guarantees progress.

A5

Left:

```
C
high = mid - 1;
```

71. SOLVED GATE-STYLE EXAMPLE 1

Sorted array:

TEXT

2 5 8 12 16 23 38 56 72 91

Search:

TEXT

23

Indices:

TEXT

0 1 2 3 4 5 6 7 8 9

Initially:

TEXT

low=0
high=9
mid=4

TEXT

a[4]=16

23 > 16:

TEXT

low=5

Now:

TEXT

low=5
high=9
mid=7

TEXT

a[7]=56

23 < 56:

TEXT

high=6

Now:

TEXT

low=5
high=6
mid=5

TEXT

a[5]=23

Found.

Index:

5

72. SOLVED EXAMPLE 2 — ABSENT KEY

Same array.

Search:

TEXT

20

Sequence of checked middle values:

TEXT

16
56
23

Eventually interval becomes empty.

Return:

TEXT

-1

73. PRACTICE

Q1 — LINEAR SEARCH

Array:

TEXT

12 5 7 20 9

Search:

TEXT

20

How many comparisons are required?

What index is returned?

Q2 — BINARY SEARCH

Sorted array:

TEXT

10 20 30 40 50 60 70

Search:

TEXT

50

Trace:

TEXT

low
high
mid
a[mid]

for every iteration.

Q3 — UNSUCCESSFUL SEARCH

Sorted:

TEXT

5 10 15 20 25 30

Search:

TEXT

18

Trace until:

TEXT

low > high

Q4 — COMPLEXITY

For an array of:

TEXT

1,048,576 = 2^{20}

elements, approximately how many halvings does binary search require?

Compare that with worst-case linear search.

Q5 — RECURSIVE VS ITERATIVE

State:

- time complexity
- auxiliary space

for:

1. iterative binary search
2. recursive binary search

Q6 — DUPLICATES

Array:

TEXT

1 2 2 2 2 3 4

What modification is required if the question asks for the **first** occurrence of 2 rather than any occurrence?

Q7 — GATE TRAP

Why is this potentially unsafe?

C

```
int mid = (low + high) / 2;
```

What is the preferred alternative?

Q8 — SORTING TRADE-OFF

You have an unsorted array and need to search for only one key.

Compare:

Method A

Linear search directly.

Method B

Sort first, then binary search.

What are their broad time complexities, and which is asymptotically better for only one search?

0=29 — PLACEMENT

Given:

TEXT

1 3 5 6

find the insertion position for:

TEXT

2

using binary-search thinking.

PROGRESS

MODULE 2 — ARRAYS

' Lesson 22 — Fundamentals & Memory ' Lesson 23 — 1D Operations ' Lesson 24 — 2D Arrays ' Lesson 25 — Row/Column-Major Address Formulas ' Lesson 26 — Arrays + Pointers ' **Lesson 27 — Searching**

NEXT — LESSON 28

Placement Array Patterns — Two Pointers, Sliding Window, Prefix Sum, Frequency Arrays, Kadane & Binary-Search Extensions

This is the **placement-heavy final lesson of the Arrays module**.

We will cover:

- two-pointer technique
- reverse/palindrome
- two-sum in sorted array
- remove duplicates
- move zeros
- sliding window
- fixed-size window maximum sum
- variable-window intuition
- prefix sums
- `rangeSum(L, R)`
- frequency arrays
- counting occurrences
- Kadane's algorithm
- maximum subarray sum
- first/last occurrence via binary search
- common interview pattern-recognition shortcuts

After Lesson 28, we begin:

MODULE 3 — LINKED LISTS

with **Lesson 29 — Linked List Intuition, Nodes, Memory & Why Arrays Are Not Enough.**

USER

next

USER

next

ASSISTANT

MODULE 2 — LESSON 28

Placement Array Patterns — Two Pointers, Sliding Window, Prefix Sum, Frequency Array & Kadane’s Algorithm

This is the **placement-heavy final lesson of the Arrays module.**

Until now, we learned what an array is and how basic operations work.

Now the important placement question becomes:

When I see an array coding problem, how do I recognize the right pattern instead of trying random loops?

Today we build that pattern recognition.

GATE importance: +IMPORTANT Placement importance: 0=Y%0450KNOW

1. WHERE WE ARE

Completed:

- Array fundamentals
- 1D operations
- 2D arrays
- Address calculations
- Arrays + pointers
- Linear and binary search

Now:

High-frequency array problem-solving patterns

We will study:

1. Two Pointers
2. Remove Duplicates
3. Move Zeros
4. Sliding Window
5. Prefix Sum
6. Frequency Array
7. Kadane's Algorithm
8. Binary-search extensions
9. Pattern-recognition shortcuts

2. WHY PATTERNS MATTER

Suppose interviewer asks:

Find two numbers whose sum is 10.

A beginner may immediately try:

```
c
for (int i = 0; i < n; i++)
{
    for (int j = i + 1; j < n; j++)
    {
        if (a[i] + a[j] == 10)
            ...
    }
}
```

Time:

TEXT
 $O(n^2)$

But if the array is sorted, a **two-pointer** solution can do:

TEXT
 $O(n)$

The difference comes from recognizing structure.

3. PATTERN1 — TWO POINTERS

Two pointers means:

Maintain two positions in the array and move them intelligently instead of repeatedly rescanning elements.

Typical forms:

5. PALINDROME USING TWO POINTERS

Array:

TEXT

1 2 3 2 1

Compare:

TEXT

a[left] with a[right]

Then move inward.

C

```
int isPalindrome(int a[], int n)
{
    int left = 0;
    int right = n - 1;

    while (left < right)
    {
        if (a[left] != a[right])
            return 0;

        left++;
        right--;
    }

    return 1;
}
```

Complexity:

TEXT

Time = O(n)
Space = O(1)

6. SORTED TWO-SUM Ø=Ý%

Suppose sorted array:

TEXT

1 2 4 6 8 10

Target:

TEXT

12

Set:

TEXT

```
left  = 0  
right = n-1
```

Current:

TEXT

```
1 + 10 = 11
```

Too small.

To increase the sum:

TEXT

```
left++
```

Now:

TEXT

```
2 + 10 = 12
```

Found.

7. WHY MOVEMENTS WORK

For sorted ascending array:

If:

TEXT

```
a[left] + a[right] < target
```

we need a larger sum.

So move:

TEXT

```
left++
```

If:

TEXT

```
a[left] + a[right] > target
```

we need a smaller sum.

So:

TEXT

```
right--
```

This reasoning depends on sorting.

8. C IMPLEMENTATION

```
c
int twoSumSorted(int a[], int n, int target)
{
    int left = 0;
    int right = n - 1;

    while (left < right)
    {
        int sum = a[left] + a[right];

        if (sum == target)
            return 1;

        if (sum < target)
            left++;
        else
            right--;
    }

    return 0;
}
```

Time:

```
TEXT
O(n)
```

Space:

```
TEXT
O(1)
```

9. WHY NOT $O(n^2)$?

Although two indices are used, they do not form nested independent loops.

Across the entire algorithm:

```
TEXT
left moves at most n times
right moves at most n times
```

They only move inward.

Therefore total work:

```
TEXT
O(n)
```

~~$O(n^2)$~~ Important complexity trap.

10. TWO-SUM COMPARISON

Brute force

TEXT

Check every pair
!' $O(n^2)$

Sorted + two pointers

TEXT

$O(n)$

But if sorting is required first:

TEXT

Sorting $O(n \log n)$
+
Two pointers $O(n)$

Overall:

TEXT

$O(n \log n)$

11. PATTERN2 — SLOW & FAST POINTERS

Suppose sorted array:

TEXT

1 1 2 2 2 3 4 4

Want unique elements:

TEXT

1 2 3 4

Because duplicates are adjacent, we can use:

TEXT

slow !' last unique position
fast !' scans new elements

12. REMOVE DUPLICATES FROM SORTED ARRAY

Start:

TEXT

```
1 1 2 2 2 3 4 4
^ ^
S F
```

slow marks last unique element.

When:

TEXT

```
a[fast] != a[slow]
```

we discovered a new value.

Move slow:

TEXT

```
slow++
```

then copy:

TEXT

```
a[slow] = a[fast]
```

13. CODE

```
C
int removeDuplicates(int a[], int n)
{
    if (n == 0)
        return 0;

    int slow = 0;

    for (int fast = 1; fast < n; fast++)
    {
        if (a[fast] != a[slow])
        {
            slow++;
            a[slow] = a[fast];
        }
    }

    return slow + 1;
}
```

14. TRACE

Input:

TEXT

```
1 1 2 2 3
```

Initially:

TEXT

```
s l o w   =   0   ! '   v a l u e   1
```

fast=1:

TEXT

```
1 == 1  
skip
```

fast=2:

TEXT

```
2 != 1  
slow=1  
a[1]=2
```

Logical array now:

TEXT

```
1 2 ...
```

Later 3:

TEXT

```
slow=2  
a[2]=3
```

New logical length:

TEXT

```
3
```

Result:

TEXT

```
1 2 3
```

15. COMPLEXITY

fast scans once.

TEXT

```
Time = O(n)
```

Only variables:

TEXT

```
slow, fast
```


So:

```
TEXT
Space = O(1)
```

Very common placement problem.

16. WHY SORTED ARRAY MATTERS

For:

```
TEXT
1 1 2 2 3 3
```

duplicates are adjacent.

But unsorted:

```
TEXT
1 2 1 3 2
```

the same simple adjacent-comparison method does not remove all duplicates.

So always inspect input conditions.

17. MOVE ZEROS $O(N)$

Input:

```
TEXT
0 1 0 3 12
```

Desired:

```
TEXT
1 3 12 0 0
```

while keeping nonzero values in relative order.

Pattern:

```
TEXT
slow = position where next non-zero should go
fast = scans array
```

18. SIMPLE IMPLEMENTATION

```
c
void moveZeros(int a[], int n)
{
    int pos = 0;

    for (int i = 0; i < n; i++)
    {
        if (a[i] != 0)
        {
            a[pos] = a[i];
            pos++;
        }
    }

    while (pos < n)
    {
        a[pos] = 0;
        pos++;
    }
}
```

19. TRACE

Input:

TEXT
0 1 0 3 12

Collect nonzeros:

TEXT
1 3 12

Then fill remaining:

TEXT
0 0

Result:

TEXT
1 3 12 0 0

Time:

TEXT
 $O(n)$

Auxiliary space:

TEXT
 $O(1)$

20. MEMORY TRICK — SLOW/FAST

Think:

FAST discovers. SLOW stores.

Useful for:

- remove duplicates
 - move zeros
 - filtering
 - partitioning
-

21. PATTERN3 — SLIDING WINDOW Ø=Ý%Ø=Ý%Ø=Ý%

Suppose array:

TEXT

2 1 5 1 3 2

Question:

Find maximum sum of any 3 consecutive elements.

Brute force:

TEXT

2+1+5 = 8
1+5+1 = 7
5+1+3 = 9
1+3+2 = 6

Answer:

TEXT

9

But notice neighboring windows overlap heavily.

22. WINDOW IDEA

First window:

TEXT

[2 1 5] 1 3 2

Sum:

TEXT

8

Next:

TEXT

2 [1 5 1] 3 2

Instead of recalculating:

TEXT

1+5+1

from scratch:

TEXT

```
new sum
=
old sum
- element leaving
+ element entering
```

So:

TEXT

8 - 2 + 1 = 7

$O(1)$ is sliding window.

23. FIXED-SIZE SLIDING WINDOW CODE

```
C
int maxWindowSum(int a[], int n, int k)
{
    if (k > n || k <= 0)
        return 0;

    int sum = 0;

    for (int i = 0; i < k; i++)
        sum += a[i];

    int maxSum = sum;

    for (int i = k; i < n; i++)
    {
        sum += a[i];
        sum -= a[i - k];

        if (sum > maxSum)
            maxSum = sum;
    }

    return maxSum;
}
```

24. COMPLEXITY

First window:

TEXT

$O(k)$

Sliding through remainder:

TEXT

$O(n-k)$

Total:

TEXT

$O(k + n-k)$
 $= O(n)$

Space:

TEXT

$O(1)$

25. BRUTE FORCE VS SLIDING WINDOW

If every window has k elements:

Brute force:

TEXT

$O(nk)$

Sliding window:

TEXT

$O(n)$

$O=10\%$ improvement.

26. WHEN TO THINK SLIDING WINDOW

Look for words like:

- contiguous subarray
- consecutive elements
- window of size k
- longest substring/subarray satisfying...
- shortest contiguous segment satisfying...

Memory trick:

CONTIGUOUS + repeated window work != SLIDING WINDOW

27. FIXED VS VARIABLE WINDOW

Fixed window

Size always:

```
TEXT
k
```

Example:

Maximum sum of 5 consecutive elements.

Variable window

Window grows/shrinks based on condition.

Example:

Longest contiguous subarray with sum at most K .

Placement extension.

28. VARIABLE WINDOW INTUITION

Imagine:

```
TEXT
left
!"
[ ... current window ... ]
! '
right
```

Usually:

1. expand `right`
2. update window information
3. while condition becomes invalid, move `left`
4. record best answer

This works particularly well when the relevant property behaves monotonically, such as sums with non-negative values.

29. DON'T BLINDLY USE SLIDING WINDOW FOR NEGATIVE VALUES

For some sum-based variable-window problems, negative numbers break simple grow/shrink reasoning.

Why?

Because removing an element does not necessarily reduce the sum when negatives exist.

Ø=Ý%Ø=Ý%Ø=Ý%
Interview trap:

Sliding window depends on problem properties, not merely on the word “subarray.”

30. PATTERN4 — PREFIX SUM Ø=Ý%Ø=Ý%Ø=Ý%

Suppose:

TEXT

a = 2 4 6 8 10

You repeatedly receive questions like:

What is sum from index 1 to index 3?

Without preprocessing:

TEXT

4 + 6 + 8
= 18

For many queries, repeatedly summing can be expensive.

31. BUILD PREFIX SUM

Define:

TEXT

prefix[i]
=
sum of a[0] through a[i]

For:

TEXT

2 4 6 8 10

prefix:

TEXT

2 6 12 20 30

because:

TEXT

```
prefix[0] = 2
prefix[1] = 2+4 = 6
prefix[2] = 12
prefix[3] = 20
prefix[4] = 30
```

32. PREFIX BUILD CODE

C

```
void buildPrefix(int a[], int prefix[], int n)
{
    if (n == 0)
        return;

    prefix[0] = a[0];

    for (int i = 1; i < n; i++)
    {
        prefix[i] = prefix[i - 1] + a[i];
    }
}
```

Time:

TEXT

$O(n)$

Additional space:

TEXT

$O(n)$

33. RANGE SUM FORMULA

Want:

TEXT

$\text{sum}(L \dots R)$

We know:

TEXT

```
prefix[R]
=
sum(0 ... R)
```

But it includes elements before `L`.

Subtract:

TEXT

prefix[L-1]

Therefore:

FORMULA BOX

For $L > 0$:

TEXT

RangeSum(L, R)
=
prefix[R] - prefix[L-1]

If:

TEXT

$L = 0$

then:

TEXT

RangeSum(0, R) = prefix[R]

$0 \leq R \leq N-1$ must know.

34. EXAMPLE

Array:

TEXT

2 4 6 8 10

Prefix:

TEXT

2 6 12 20 30

Find sum:

TEXT

L=1
R=3

Formula:

TEXT

prefix[3] - prefix[0]

TEXT

20 - 2
= 18

Correct:

TEXT

4+6+8=18

35. WHY PREFIX SUM IS POWERFUL

Build once:

TEXT

$O(n)$

Then each range query:

TEXT

$O(1)$

Without prefix:

Each range might cost:

TEXT

$O(n)$

If there are q queries:

Naive

TEXT

$O(nq)$

worst case.

Prefix

TEXT

$O(n + q)$

$O(1)$ for placement technique.

36. ALTERNATIVE PREFIX STYLE

Sometimes define:

TEXT

```
prefix[0] = 0
prefix[i+1] = prefix[i] + a[i]
```

Example:

TEXT

```
a =      2  4  6  8
prefix = 0  2  6 12 20
```

Then range `[L, R]` becomes beautifully:

TEXT

```
prefix[R+1] - prefix[L]
```

This convention avoids a special `L==0` case.

Very useful in coding interviews.

37. PREFIX SUM IS NOT ONLY FOR SUM

The broader idea is:

Precompute cumulative information so repeated queries become cheap.

Variants include:

- prefix counts
- prefix XOR
- cumulative frequency
- 2D prefix sums

Placement extension.

38. 2D PREFIX SUM — INTUITION

For a matrix, we can preprocess rectangle sums.

Then queries like:

Sum of all elements inside rectangle `(r1, c1)` to `(r2, c2)`

can be answered much faster.

Formal 2D prefix formulas are useful but not essential for core GATE DS syllabus, so keep this as a placement extension for now.

39. PATTERN5 — FREQUENCY ARRAY

Suppose:

TEXT

2 4 2 3 4 2

Values are guaranteed between:

TEXT

0 and 5

Create:

C

```
int freq[6] = {0};
```

Process:

C

```
for (int i = 0; i < n; i++)  
{  
    freq[a[i]]++;  
}
```

Result:

TEXT

```
freq[2] = 3  
freq[3] = 1  
freq[4] = 2
```

40. COMPLEXITY

Building:

TEXT

$O(n)$

If frequency range has size K :

TEXT

Space = $O(K)$

Then asking:

How often does value 4 occur?

becomes:

TEXT

$O(1)$

by reading:

```
c
freq[4]
```

41. WHEN TO USE FREQUENCY ARRAY

Best when:

- values are integers
- range is small
- bounds are known

Examples:

```
TEXT
digits 0-9
lowercase letters 0-25
marks 0-100
small IDs
```

42. CHARACTER FREQUENCY

For lowercase English letters:

```
c
int freq[26] = {0};
```

For character:

```
c
char ch = 'c';
```

index:

```
TEXT
ch - 'a'
```

So:

```
c
freq[ch - 'a']++;
```

This pattern appears everywhere in string placement questions.

43. WHY `ch - 'a'`?

Character codes are sequential.

So conceptually:

TEXT

```
'a' - 'a' = 0
'b' - 'a' = 1
'c' - 'a' = 2
...
'z' - 'a' = 25
```

Thus letters map naturally into a 26-element array.

44. FREQUENCY ARRAY VS HASHING

Frequency array:

TEXT

Fast and simple
but requires manageable value range.

Hashing:

TEXT

Handles arbitrary keys more flexibly.

Hashing is separately part of your Algorithms syllabus, so we'll study it in depth there.

45. PATTERN6 — KADANE'S ALGORITHM Ø=Ý%Ø=Ý%Ø=Ý%

Problem:

Find the maximum sum of a contiguous subarray.

Example:

TEXT

-2 1 -3 4 -1 2 1 -5 4

Best contiguous part:

TEXT

4 -1 2 1

Sum:

TEXT

6

This is one of the most famous array interview problems.

46. BRUTE FORCE IDEA

Try every possible subarray.

There are roughly:

TEXT
 $O(n^2)$

different start/end pairs.

If summing each separately, naive can even become:

TEXT
 $O(n^3)$

With improvements:

TEXT
 $O(n^2)$

But Kadane solves it in:

TEXT
 $O(n)$

47. KADANE INTUITION

At every position, ask:

Is it better to continue the previous subarray, or start fresh from the current element?

Suppose:

TEXT
currentSum

is negative.

If you carry a negative sum forward, it only hurts future sums.

So we can abandon it and restart.

Memory trick:

Negative baggage? Drop it.

48. SAFE KADANE VERSION

A version that correctly handles arrays containing all negative values:

```

c
int maxSubarraySum(int a[], int n)
{
    int current = a[0];
    int best = a[0];

    for (int i = 1; i < n; i++)
    {
        if (current + a[i] > a[i])
            current = current + a[i];
        else
            current = a[i];

        if (current > best)
            best = current;
    }

    return best;
}

```

Assume:

```

TEXT
n > 0

```

49. CORE KADANE RECURRENCE

At index **i**:

```

TEXT
current =
max(
    a[i],
    current + a[i]
)

```

Then:

```

TEXT
best =
max(best, current)

```

This is the entire algorithm.

50. TRACE KADANE

Array:

```

TEXT
-2 1 -3 4 -1 2 1 -5 4

```

Start:

TEXT

```
current = -2  
best = -2
```

At 1:

TEXT

```
max(1, -2+1)  
= max(1, -1)  
=1
```

So:

TEXT

```
current=1  
best=1
```

At -3:

TEXT

```
max(-3, 1-3)  
=max(-3, -2)  
=-2
```

TEXT

```
current=-2  
best=1
```

At 4:

TEXT

```
max(4, -2+4)  
=max(4, 2)  
=4
```

TEXT

```
best=4
```

At -1:

TEXT

```
current=3  
best=4
```

At 2:

TEXT

```
current=5  
best=5
```

At 1:

TEXT

```
current=6  
best=6
```

Answer:

6

51. KADANE COMPLEXITY

One scan:

TEXT

```
Time = O(n)
```

Only constant variables:

TEXT

```
Space = O(1)
```

0-1 placement essential.

52. HUGE KADANE TRAP — ALL NEGATIVE ARRAY

Suppose:

TEXT

```
-8 -3 -5 -2
```

Correct maximum subarray sum:

TEXT

```
-2
```

because a subarray must contain at least one element.

If you initialize:

C

```
int current = 0;  
int best = 0;
```

you may incorrectly answer:

TEXT

```
0
```

even though no zero exists.

Enter initialize:

```
C
current = a[0];
best = a[0];
```

53. KADANE VS SLIDING WINDOW

Both deal with contiguous subarrays, but they solve different structures.

Sliding Window

Useful when window movement has a predictable condition, especially fixed size.

Kadane

Specifically optimized for:

```
TEXT
maximum contiguous subarray sum
```

with arbitrary positive/negative values.

Don't treat them as interchangeable.

54. PATTERN7 — BINARY SEARCH EXTENSIONS

We learned exact-key search.

Now recognize three important extensions:

1. First occurrence
2. Last occurrence
3. Search insertion position

55. FIRST OCCURRENCE

Array:

```
TEXT
1 2 2 2 2 4
```

When:

```
TEXT
a[mid] == key
```

record:

```
TEXT
ans = mid
```

but search:

```
TEXT
LEFT
```

because an earlier occurrence may exist.

```
C
high = mid - 1;
```

56. LAST OCCURRENCE

Same idea, but after finding:

```
TEXT
ans = mid
```

continue:

```
TEXT
RIGHT
```

```
C
low = mid + 1;
```

Memory trick:

```
TEXT
F I R S T  ! '   L E F T
L A S T    ! '   R I G H T
```

57. SEARCH INSERT POSITION

Sorted:

```
TEXT
1 3 5 6
```

Target:

```
TEXT
2
```

Desired insertion:

TEXT

1 2 3 5 6

position:

TEXT

1

Binary search can determine where target should appear in:

TEXT

$O(\log n)$

even when target is absent.

58. PATTERN RECOGNITION CHEAT SHEET Ø=Ý%Ø=Ý%Ø=Ý%

When you see...

“Sorted array + pair sum”

Think:

TEXT

Two Pointers

“Palindrome / reverse”

Think:

TEXT

Two Pointers from ends

“Remove duplicates / move zeros”

Think:

TEXT

Slow + Fast pointers

“Consecutive / contiguous elements of fixed length k”

Think:

TEXT

Sliding Window

“Many range-sum queries”

Think:

TEXT

Prefix Sum

“Count occurrences, small bounded values”

Think:

TEXT

Frequency Array

“Maximum sum contiguous subarray”

Think:

TEXT

Kadane

“Sorted array + find value/boundary”

Think:

TEXT

Binary Search

0-100% Copy this cheat sheet.

59. PLACEMENT EXAMPLE 1 — TWO POINTERS

Problem:

TEXT

Sorted array:
1 2 3 4 6

Target = 6

Possible pair:

TEXT

2 + 4

Trace:

TEXT

left=1
right=6

```
s u m = 7 !'   t o o   b i g
```

Move right.

TEXT

```
1 + 4 = 5 !'   t o o   s m a l l
```

Move left.

TEXT

```
2 + 4 = 6 !'   f o u n d
```

Time:

TEXT

$O(n)$

60. PLACEMENT EXAMPLE 2 — WINDOW

Array:

TEXT

```
1 4 2 10 2 3 1 0 20
```

Find maximum sum of 4 consecutive elements.

First window:

TEXT

```
1+4+2+10 = 17
```

Next:

TEXT

```
17 - 1 + 2 = 18
```

Next:

TEXT

```
18 - 4 + 3 = 17
```

Continue similarly.

No need to recalculate all four values every time.

61. PLACEMENT EXAMPLE 3 — PREFIX QUERY

Array:

TEXT

3 1 4 1 5

Prefix:

TEXT

3 4 8 9 14

Query:

TEXT

sum index 2...4

TEXT

prefix[4]-prefix[1]

=14-4

=10

Check:

TEXT

4+1+5=10

62. PLACEMENT EXAMPLE 4 — FREQUENCY

Array:

TEXT

3 1 3 2 1 3

Frequency:

TEXT

1 !' 2

2 !' 1

3 !' 3

Most frequent:

TEXT

3

63. PLACEMENT EXAMPLE 5 — KADANE

Array:

TEXT

-4 2 3 -2 5 -10

Best:

TEXT

2 + 3 - 2 + 5
= 8

Kadane finds this in one scan.

64. GATE CONNECTION — PREFIX SUM COMPLEXITY

Suppose q range-sum queries are asked on array of n .

Without preprocessing:

TEXT

$O(n)$ per query

Total:

TEXT

$O(nq)$

With prefix:

TEXT

Build = $O(n)$
Queries = $O(q)$

Total:

TEXT

$O(n+q)$

~~$O(n+q)$~~ Common algorithmic comparison.

65. GATE CONNECTION — TWO POINTER COMPLEXITY

Code may look like:

```
c
while (left < right)
{
    if (...)
        left++;
    else
        right--;
}
```

Worst case:

```
TEXT
0(n)
```

because distance:

```
TEXT
right-left
```

decreases each iteration.

Do not count two indices as $O(n^2)$.

66. GATE CONNECTION — NESTED-LOOKING WINDOW

Sometimes code has:

```
C
for (...)
{
    while (...)
    {
        left++;
    }
}
```

You may be tempted to say:

```
TEXT
 $O(n^2)$ 
```

But if `left` only moves forward globally and never resets, the inner `while` may execute only $O(n)$ total times.

Thus overall:

```
TEXT
 $O(n)$ 
```

$O=Vn$ is an important amortized-style intuition.

67. EXAMPLE

```
C
int left = 0;

for (int right = 0; right < n; right++)
{
    while (left <= right && condition)
    {
        left++;
    }
}
```

right moves:

TEXT
n times

left moves:

TEXT
at most n times total

So:

TEXT
 $O(n+n)$
=
 $O(n)$

not automatically $O(n^2)$.

68. DON'T CONFUSE SUBARRAY, SUBSEQUENCE & SUBSET

Very important placement terminology.

Subarray

Must be contiguous.

From:

TEXT
1 2 3 4

valid:

TEXT
2 3

Not:

TEXT
1 3

Subsequence

Maintains relative order but need not be contiguous.

Valid:

TEXT
1 3 4

Subset

Ordering/contiguity usually not relevant.

Example:

```
TEXT
{1, 4}
```

Memory trick:

```
TEXT
SUBARRAY = STAYS TOGETHER
SUBSEQUENCE = ORDER STAYS
SUBSET = CHOOSE ITEMS
```

Ø = Important.

69. KADANE FINDS SUBARRAY

Kadane solves:

```
TEXT
maximum contiguous subarray sum
```

not arbitrary subsequence sum.

For:

```
TEXT
-5 10 -100 20
```

maximum positive subsequence could choose:

```
TEXT
10 + 20 = 30
```

But they are not contiguous.

Maximum subarray is:

```
TEXT
20
```

or 10, depending contiguous ranges.

So read terminology carefully.

70. IN-PLACE VS EXTRA ARRAY

Many placement questions specifically ask:

Solve in-place.

That usually means avoid an extra array proportional to n .

Examples:

Reverse

TEXT

$O(1)$ extra space

Remove duplicates sorted

TEXT

$O(1)$ extra space

Move zeros

TEXT

$O(1)$ extra space

Two-pointer techniques are excellent for in-place solutions.

71. COMMON MISTAKE — MODIFYING WHILE SCANNING

When shifting/removing elements, be careful not to overwrite data still needed.

This is why slow/fast patterns are valuable:

TEXT

fast reads future elements
slow writes finalized positions

This separation makes in-place modification safer.

72. GATE / INTERVIEW TRAP SHEET

Trap 1

Two pointers "" automatically $O(n^2)$.

Trap 2

Sliding window is mainly for contiguous regions.

Trap 3

Prefix preprocessing costs $O(n)$ but each range query becomes $O(1)$.

Trap 4

Frequency arrays need a manageable key range.

Trap 5

Kadane must handle all-negative arrays correctly.

Trap 6

Standard binary search doesn't guarantee first/last duplicate.

Trap 7

A nested-looking variable window can still be $O(n)$ if both pointers only move forward.

Trap 8

Subarray "subsequence."

73. MASTER COMPLEXITY TABLE

Pattern	Typical Time	Extra Space
Reverse with two pointers	$O(n)$	$O(1)$
Sorted two-sum	$O(n)$	$O(1)$
Remove duplicates sorted	$O(n)$	$O(1)$
Move zeros	$O(n)$	$O(1)$
Fixed sliding window	$O(n)$	$O(1)$
Build prefix sum	$O(n)$	$O(n)$
Prefix range query	$O(1)$	—
Build frequency array	$O(n)$	$O(K)$
Kadane	$O(n)$	$O(1)$
Binary search boundary	$O(\log n)$	$O(1)$ iterative

Copy this table.

74. INTERVIEW QUESTIONS

Q1. When should you think of two pointers?

When two positions can move systematically through an array without reconsidering discarded elements.

Q2. Sliding window advantage?

Reuse information from overlapping contiguous windows instead of recalculating from scratch.

Q3. Why prefix sum?

Pay $O(n)$ once so repeated range-sum queries become $O(1)$.

Q4. Kadane complexity?

TEXT

$O(n)$ time
 $O(1)$ auxiliary space

Q5. Why initialize Kadane from `a[0]` ?

To correctly handle arrays containing only negative values.

Q6. Difference between subarray and subsequence?

Subarray must be contiguous; subsequence need only preserve order.

75. MEMORY TRICKS

Two pointers

Two eyes, one scan.

Slow/Fast

FAST finds, SLOW fixes.

Sliding Window

Remove left, add right.

Prefix

Pay once, answer fast.

Frequency

Value becomes index.

Kadane

Carry profit, drop harmful baggage.

Binary boundary

Found doesn't always mean stop.

76. NOTEBOOK NOTES

Two Sum — sorted

TEXT

```
s u m < t a r g e t ! ' l e f t + +
s u m > t a r g e t ! ' r i g h t - -
s u m = t a r g e t ! ' f o u n d
```

TEXT

$O(n)$

Sliding window

TEXT

```
newWindow
=
oldWindow
- outgoing
+ incoming
```

Prefix

TEXT

```
prefix[i]
=
prefix[i-1] + a[i]
```

Range:

TEXT

```
prefix[R] - prefix[L-1]
```

Frequency

C

```
freq[a[i]]++;
```

Kadane

TEXT

```
current = max(a[i], current+a[i])
best    = max(best,current)
```


TEXT

Time $O(n)$
Space $O(1)$

77. 30-SECOND ARRAY PATTERN REVISION

When you see:

TEXT

sorted pair problem
!' TWO POINTERS

TEXT

remove/move/filter in-place
!' SLOW + FAST

TEXT

consecutive fixed-size region
!' SLIDING WINDOW

TEXT

many range sums
!' PREFIX SUM

TEXT

small bounded frequency
!' FREQUENCY ARRAY

TEXT

maximum contiguous sum
!' KADANE

TEXT

sorted search/boundary
!' BINARY SEARCH

~~O=You~~ is your placement array pattern map.

78. ACTIVE RECALL

Q1

What pattern is best for pair sum in a sorted array?

Q2

What is fixed sliding-window complexity?

Q3

After prefix preprocessing, complexity of one range-sum query?

Q4

What does Kadane solve?

Q5

What is the difference between subarray and subsequence?

ANSWERS

A1

TEXT

Two pointers

A2

TEXT

$O(n)$

A3

TEXT

$O(1)$

A4

Maximum sum of a contiguous subarray.

A5

Subarray must be contiguous; subsequence need only preserve order.

79. PRACTICE — EASY

Q1

Reverse using two-pointer thinking:

TEXT

10 20 30 40 50

How many swaps?

Q2

Sorted:

TEXT

1 2 4 7 11

Target:

TEXT

9

Which pair is found by two pointers?

Q3

Array:

TEXT

1 1 2 2 3 3 3 4

What is the logical array after removing sorted duplicates?

80. PRACTICE — MODERATE

Q4 — SLIDING WINDOW

Array:

TEXT

2 1 5 1 3 2

Find maximum sum of any $k=3$ consecutive elements.

Q5 — PREFIX

Array:

TEXT

5 2 7 3 6

Construct prefix sums.

Then find sum from index:

TEXT

1 to 3

Q6 — FREQUENCY

Array:

TEXT

2 3 2 5 3 2 5 5

Find frequencies of:

TEXT

2, 3, 5

81. PRACTICE — TRICKY

Q7 — KADANE

Find maximum contiguous subarray sum:

TEXT

-2 -3 4 -1 -2 1 5 -3

Also identify the best subarray.

Q8 — ALL NEGATIVE

Find maximum subarray sum:

TEXT

-8 -2 -6 -5

Why would initializing `best=0` be incorrect?

Q9 — COMPLEXITY TRAP

Consider:

```
c
int left = 0;

for (int right = 0; right < n; right++)
{
    while (left < right && condition)
    {
        left++;
    }
}
```

Assuming `left` never moves backward, is this automatically $O(n^2)$?

Explain.

82. PLACEMENT MINI-CHALLENGE

For each problem, identify the pattern only.

A

“Find if a sorted array contains two values summing to X.”

B

“Find maximum sum of exactly K consecutive elements.”

C

“Answer 10 range-sum queries.”

D

“Find maximum contiguous subarray sum.”

E

“Remove duplicates from a sorted array in-place.”

F

“Find first occurrence of X in sorted array.”

ANSWERS

TEXT

```
A !'  T w o   P o i n t e r s
B !'  S l i d i n g   W i n d o w
C !'  P r e f i x   S u m
D !'  K a d a n e
E !'  S l o w / F a s t   T w o   P o i n t e r s
F !'  M o d i f i e d   B i n a r y   S e a r c h
```

MODULE 2 — ARRAYS COMPLETE '

You have now covered the full GATE array foundation plus the important placement extensions:

' Array memory ' 1D operations ' 2D arrays ' Row-major / column-major formulas ' Arrays + pointers ' Linear search
' Binary search ' Two pointers ' Sliding window ' Prefix sum ' Frequency arrays ' Kadane

NEXT — MODULE 3 / LESSON 29

Linked Lists — Intuition, Nodes, Dynamic Memory & Why Arrays Are Not Enough

Next we begin an entirely new official GATE topic:

TEXT

LINKED LISTS

We will start from absolute basics:

```

c
struct Node
{
    int data;
    struct Node *next;
};

```

and understand visually:



The key idea will be:

Array connects elements through physical adjacency. Linked list connects nodes through stored addresses.

Then we will build every pointer update using:

BEFORE !' POINTER CHANGES !' AFTER

which will make insertion, deletion, reversal and Floyd cycle problems much easier.

USER

next

ASSISTANT

MODULE 3 — LESSON 29

Linked Lists — Intuition, Nodes, Dynamic Memory & Why Arrays Are Not Enough

We now begin a completely new official GATE topic:

LINKED LISTS

This is one of the most important places where your previous topics combine:

- structures
- pointers
- dynamic memory

- `malloc`
- `NULL`
- addresses
- time complexity

GATE importance: Ø=Ý%ØMÚ%ØKÍØW **Placement importance:** Ø=Ý%ØMÚ%ØKÍØW

1. WHERE WE ARE

Completed:

' C Foundation ' Pointers ' Dynamic Memory ' Recursion ' Arrays

Now:

MODULE 3 — LINKED LISTS

Roadmap:

- Lesson 29 — Linked-list intuition, node memory
- Lesson 30 — Singly linked list
- Lesson 31 — Insertion
- Lesson 32 — Deletion
- Lesson 33 — Reversal
- Lesson 34 — Doubly linked list
- Lesson 35 — Circular linked list
- Lesson 36 — Placement problems

2. STORY — WHY DO WE NEED LINKED LISTS?

Suppose an array contains:

TEXT

10 20 30 40

Memory might look conceptually like:

TEXT

1000 1004 1008 1012

[10] [20] [30] [40]

The elements are physically:

CONTIGUOUS

This gives fast access.

For example:

TEXT

a[3]

can be located immediately using an address formula.

So arrays are excellent for:

TEXT

Random access

But arrays have another problem.

3. ARRAY INSERTION PROBLEM

Suppose:

TEXT

10 20 30 40

and you want to insert:

TEXT

15

after 10 .

Desired:

TEXT

10 15 20 30 40

Existing elements may need shifting:

TEXT

4 0 !'
3 0 !'
2 0 !'

That costs:

TEXT

$O(n)$

Similarly, deletion may require shifting.

What if instead each element could live anywhere in memory and simply store:

Where is the next element?

That gives us:

Linked List

4. CORE DIFFERENCE

Array

Connection comes from:

TEXT
PHYSICAL ADJACENCY

Example:

TEXT
[10][20][30][40]

The next element is physically next.

Linked List

Connection comes from:

TEXT
STORED ADDRESSES

Example:

TEXT
[10 | 5000] [20 | 8000] [30 | NULL]
 | |
 v v
address of address of
next node next node

Nodes do not need to be adjacent.

Central idea:

Array = position creates connection. Linked list = pointer creates connection.

5. LINKED-LIST STORY

Imagine students standing anywhere on a campus.

They are not standing in one straight physical line.

But every student carries a note:

"The next student is standing at Room 205."

That student carries another note:

“Next student is at Library.”

The physical locations can be completely different.

The chain is created by:

```
TEXT
next-address information
```

This is exactly a linked list.

6. WHAT IS A NODE?

Each linked-list element is called a:

NODE

A basic singly linked-list node has two parts:

```
TEXT
DATA
+
ADDRESS OF NEXT NODE
```

Visualization:

```
TEXT
+-----+-----+
| data   | next   |
+-----+-----+
```

Example:

```
TEXT
+-----+-----+
|  10   | 5000   |
+-----+-----+
```

means:

```
TEXT
data = 10
next node is at address 5000
```

7. NODE IN C

We use a structure:

```
c
struct Node
{
    int data;
    struct Node *next;
};
```

0-7% This definition is fundamental.

8. UNDERSTAND EVERY PART

```
c
struct Node
```

defines a structure named `Node`.

Inside:

```
c
int data;
```

stores the actual value.

Then:

```
c
struct Node *next;
```

stores an address of another `struct Node`.

So `next` is:

```
TEXT
pointer to another node
```

9. WHY `next` MUST BE A POINTER

Could we write:

```
c
struct Node
{
    int data;
    struct Node next;
};
```

No.

Why?

Then one `Node` contains another complete `Node`.

That inner node contains another complete node.

Which contains another.

It becomes infinitely recursive in size.

Compiler cannot determine finite structure size.

10. POINTER SOLVES THE PROBLEM

Instead:

```
C
struct Node *next;
```

has a fixed pointer size.

So a node contains:

```
TEXT
one integer
+
one pointer
```

regardless of how many nodes exist later.

~~0=100~~ This is why self-referential structures use pointers.

11. MEMORY VISUALIZATION

Suppose three nodes happen to exist at:

```
TEXT
Node 1 !' address 1 0 0 0
Node 2 !' address 7 0 0 0
Node 3 !' address 4 2 0 0
```

Notice:

```
TEXT
not contiguous
```

Memory:

```
TEXT
Address 1000
+-----+-----+
| 10   | 7000 |
+-----+-----+

Address 7000
+-----+-----+
| 20   | 4200 |
+-----+-----+

Address 4200
+-----+-----+
```

```
| 30 | NULL |  
+-----+-----+
```

Logical list:

```
TEXT  
1 0 !' 2 0 !' 3 0 !' N U L L
```

Physical memory order doesn't matter.

12. WHAT DOES NULL MEAN HERE?

For the final node:

```
TEXT  
next = NULL
```

means:

There is no next node.

So:

```
TEXT  
NULL
```

marks the end of a normal singly linked list.

Visualization:

```
TEXT  
HEAD  
!"  
+-----+ +-----+ +-----+  
| 10 | •----->| 20 | •----->| 30 | NULL |  
+-----+ +-----+ +-----+
```

13. WHAT IS HEAD ?

How do we find the first node?

We store its address in a pointer called:

```
C  
head
```

Example:

```
C
struct Node *head;
```

If first node is at address 1000 :

```
TEXT
head = 1000
```

Then:

```
TEXT
head
! "
[10 | next]
```

head is not the first node itself.

It is:

a pointer containing the address of the first node.

Ø=NOT KNOWN.

14. EMPTY LINKED LIST

If there are no nodes:

```
C
head = NULL;
```

So:

```
TEXT
head == NULL
```

means:

Empty List

15. ONE-NODE LIST

Suppose one node:

```
TEXT
head
! "
+-----+-----+
| 10    | NULL  |
+-----+-----+
```

Then:

```
TEXT
head != NULL
```

but:

```
TEXT
head->next == NULL
```

This tells us:

```
TEXT
exactly one node
```

16. TWO-NODE LIST

```
TEXT
head
!"
+-----+-----+-----+
| 10 | •-----> | 20 | NULL |
+-----+-----+-----+
```

So:

```
TEXT
head
```

points to first node.

```
TEXT
head->next
```

points to second node.

```
TEXT
head->next->next
```

is:

```
TEXT
NULL
```

17. . VS ->

Remember structures.

Suppose:

```
c
struct Node n;
```

`n` is actual structure object.

Use:

```
c
n.data
```

But if:

```
c
struct Node *p = &n;
```

`p` is a pointer.

Use:

```
c
p->data
```

Equivalent:

```
c
(*p).data
```

So:

FORMULA BOX

```
TEXT
p->data == (*p).data
```

Essential for linked lists.

18. LINKED-LIST ACCESS EXAMPLES

Suppose:

```
TEXT
head !' node containing 10
```

Then:

```
c
head->data
```

gives:

TEXT

10

And:

C

head->next

gives address of second node.

Then:

C

head->next->data

gives data of second node.

19. POINTER CHAIN

For:

TEXT

1 0 !' 2 0 !' 3 0 !' N U L L

we can conceptually write:

TEXT

head->data
= 10

TEXT

head->next->data
= 20

TEXT

head->next->next->data
= 30

TEXT

head->next->next->next
= NULL

20. WHY LINKED LISTS COMMONLY USE HEAP MEMORY

Nodes often need to:

- be created at runtime
- survive after helper functions return

- exist until explicitly deleted

Therefore we usually create nodes dynamically:

```
C
malloc(...)
```

This means node objects typically live in dynamically allocated memory.

21. CREATE ONE NODE

```
C
struct Node *p;

p = malloc(sizeof(struct Node));
```

Better C style:

```
C
struct Node *p = malloc(sizeof *p);
```

Then check:

```
C
if (p == NULL)
{
    /* allocation failed */
}
```

22. INITIALIZE NODE

```
C
p->data = 10;
p->next = NULL;
```

Now conceptually:

```
TEXT
p
!"
+-----+-----+
| 10    | NULL |
+-----+-----+
```

23. COMPLETE BASIC EXAMPLE

```
C
#include <stdio.h>
#include <stdlib.h>
```

```

struct Node
{
    int data;
    struct Node *next;
};

int main(void)
{
    struct Node *head = malloc(sizeof *head);

    if (head == NULL)
        return 1;

    head->data = 10;
    head->next = NULL;

    printf("%d", head->data);

    free(head);

    return 0;
}

```

Output:

```

TEXT
10

```

24. LINE-BY-LINE

```

C
struct Node *head

```

`head` is a pointer to a node.

```

C
malloc(sizeof *head)

```

allocates enough dynamic memory for one node.

```

C
head->data = 10;

```

stores 10.

```

C
head->next = NULL;

```

marks this as final node.

```
C
free(head);
```

releases the dynamically allocated node.

25. MEMORY LAYOUT — POINTER VS NODE

Important:

```
C
struct Node *head = malloc(...);
```

There are two different things.

head

A pointer variable.

Conceptually may be local/stack-associated.

allocated node

Dynamic object in heap-associated storage.

Visualization:

```
TEXT
Stack-like frame                                Heap-like dynamic memory

head
+-----+
| 5000 | -----> +-----+-----+
+-----+         | 10  | NULL |
+-----+         +-----+-----+
                        addr 5000
```

Do not say:

“head is the node.”

head stores the node's address.

26. CREATE SECOND NODE

```
C
struct Node *second = malloc(sizeof *second);

second->data = 20;
second->next = NULL;
```

Initially:

TEXT

```
h e a d !' [ 1 0 | N U L L ]
s e c o n d !' [ 2 0 | N U L L ]
```

They are two separate nodes.

Not connected yet.

27. CONNECT THE NODES

Do:

```
C
head->next = second;
```

Now:

TEXT

```
head
!'"
[ 1 0 | • ] !' [ 2 0 | N U L L ]
                !'
                second
```

This single pointer assignment creates the list connection.

Ø=Very important intuition.

28. BEFORE !' POINTER CHANGE !' AFTER

We will use this method for every linked-list operation.

BEFORE

TEXT

```
h e a d !' [ 1 0 | N U L L ]
s e c o n d !' [ 2 0 | N U L L ]
```

POINTER CHANGE

```
C
head->next = second;
```

AFTER

TEXT

```
h e a d !' [ 1 0 | • ] !' [ 2 0 | N U L L ]
```

Ø=For insertion/deletion, always think this way.

29. BUILD THREE NODES MANUALLY

```
C
struct Node *first = malloc(sizeof *first);
struct Node *second = malloc(sizeof *second);
struct Node *third = malloc(sizeof *third);
```

Values:

```
C
first->data = 10;
second->data = 20;
third->data = 30;
```

Connect:

```
C
first->next = second;
second->next = third;
third->next = NULL;
```

Then:

```
C
struct Node *head = first;
```

Logical list:

```
TEXT
HEAD
!"
1 0  !'  2 0  !'  3 0  !'  N U L L
```

30. FULL CODE

```
C
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node *next;
};

int main(void)
{
    struct Node *first = malloc(sizeof *first);
    struct Node *second = malloc(sizeof *second);
    struct Node *third = malloc(sizeof *third);

    if (first == NULL || second == NULL || third == NULL)
        return 1;
```

```

first->data = 10;
second->data = 20;
third->data = 30;

first->next = second;
second->next = third;
third->next = NULL;

struct Node *head = first;

printf("%d %d %d",
        head->data,
        head->next->data,
        head->next->next->data);

free(third);
free(second);
free(first);

return 0;
}

```

Output:

```

TEXT
10 20 30

```

31. WHY DON'T NODES NEED CONTIGUOUS MEMORY?

Because each node stores:

```

TEXT
address of next node

```

Suppose:

```

TEXT
first  = address 9000
second = address 2000
third  = address 7300

```

Still:

```

TEXT
9 0 0 0  !'  2 0 0 0  !'  7 3 0 0  !'  N U L L

```

works perfectly.

Logical order is determined by pointers.

Not physical numerical address order.

32. LINKED LIST DOES NOT HAVE INDEX-BASED RANDOM ACCESS

Suppose:

TEXT

1 0 !' 2 0 !' 3 0 !' 4 0 !' 5 0

Want the 5th element.

Array:

c

a[4]

directly:

TEXT

0(1)

Linked list:

TEXT

head

!' s e c o n d

!' t h i r d

!' f o u r t h

!' f i f t h

Need traversal.

So:

TEXT

0(n)

worst case.

~~O(n)~~ Major disadvantage.

33. WHY CAN'T WE CALCULATE THE ADDRESS DIRECTLY?

Array:

TEXT

Base + i × element_size

works because elements are contiguous.

Linked-list nodes might be anywhere:

TEXT

1000

7800

2500

9200

There is no formula such as:

```
TEXT
base + i×node_size
```

for logical linked-list position.

You must follow links.

34. ARRAY VS LINKED LIST

Property	Array	Linked List
Physical storage	Contiguous	Nodes need not be contiguous
Connection	Position	Pointers
Index access	$O(1)$	$O(n)$
Insert beginning	$O(n)$	potentially $O(1)$
Delete beginning	$O(n)$	potentially $O(1)$
Extra pointer memory	No per element	Yes
Cache locality	Usually good	Usually weaker
Fixed-size ordinary C array	Yes	Nodes can grow dynamically

Copy this table.

35. IMPORTANT QUALIFICATION — INSERTION COMPLEXITY

You may hear:

Linked-list insertion is $O(1)$.

That statement needs context.

If you already have the pointer to the correct insertion position, changing links can be:

```
TEXT
 $O(1)$ 
```

But if you must first search for that position:

```
TEXT
 $O(n)$ 
```

So overall operation may become:

TEXT

$O(n)$

~~$O(n)$~~ Important GATE/interview precision.

36. EXAMPLE

Want to insert after node containing:

TEXT

999

You may first need to search:

TEXT

h e a d ! ' . . . ! ' n o d e 9 9 9

Search:

TEXT

$O(n)$

Then link update:

TEXT

$O(1)$

Total:

TEXT

$O(n)$

37. INSERTION AT HEAD — WHY LINKED LIST IS POWERFUL

Suppose:

TEXT

h e a d ! ' 1 0 ! ' 2 0 ! ' 3 0

Want:

TEXT

5 ! ' 1 0 ! ' 2 0 ! ' 3 0

No shifting.

Only pointer changes are needed.

Conceptually:

TEXT

```
newNode->next = head  
head = newNode
```

Number of pointer operations is constant.

So:

TEXT

$O(1)$

We will implement this properly in Lesson 31.

38. DELETION AT HEAD

Before:

TEXT

h e a d ! ' 1 0 ! ' 2 0 ! ' 3 0

Want remove 10 .

Conceptually:

TEXT

```
temp = head  
head = head->next  
free(temp)
```

After:

TEXT

h e a d ! ' 2 0 ! ' 3 0

Again:

TEXT

$O(1)$

No shifting.

39. WHY DYNAMIC MEMORY MAKES LISTS FLEXIBLE

Array:

C

```
int a[100];
```

capacity chosen ahead of time.

Linked list:

Create nodes as needed:

```
c
malloc(...)
```

Again:

```
c
malloc(...)
```

Again:

```
c
malloc(...)
```

The structure grows node by node.

This is useful when size is unknown beforehand.

40. BUT DYNAMIC MEMORY HAS COSTS

Linked lists are not automatically better.

Each node requires extra pointer storage.

For:

```
c
struct Node
{
    int data;
    struct Node *next;
};
```

we store:

```
TEXT
data
+
pointer
```

There may also be structure padding/alignment.

So linked lists have more memory overhead per stored value.

41. CACHE LOCALITY

Array:

```
TEXT
[10][20][30][40]
```

elements are adjacent.

Modern hardware can load nearby data efficiently.

Linked list:

TEXT

```
10 at 1000
20 at 8000
30 at 3200
```

following links may jump around memory.

So even if two algorithms are both:

TEXT

$O(n)$

array traversal can often be faster in practice because of cache locality.

$O(1)$ placement/systems insight.

42. TRAVERSAL IDEA

To visit a linked list:

Start:

```
C
struct Node *temp = head;
```

Then repeatedly:

```
C
temp = temp->next;
```

until:

```
TEXT
temp == NULL
```

43. BASIC TRAVERSAL CODE

```
C
void printList(struct Node *head)
{
    struct Node *temp = head;

    while (temp != NULL)
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }
}
```

Ø=7% This loop pattern will appear constantly.

44. TRACE TRAVERSAL

List:

```
TEXT
1 0 !' 2 0 !' 3 0 !' N U L L
```

Initially:

```
TEXT
t e m p !' 1 0
```

Print:

```
TEXT
10
```

Then:

```
C
temp = temp->next;
```

Now:

```
TEXT
t e m p !' 2 0
```

Print:

```
TEXT
20
```

Then:

```
TEXT
t e m p !' 3 0
```

Print:

```
TEXT
30
```

Then:

```
TEXT
temp = NULL
```

Loop stops.

45. TRAVERSAL COMPLEXITY

For `n` nodes:

```
TEXT
Time = O(n)
```

Extra variables:

```
TEXT
temp
```

So:

```
TEXT
Auxiliary Space = O(1)
```

for iterative traversal.

46. WHY USE `temp` INSTEAD OF MOVING `head` ?

You could write:

```
C
while (head != NULL)
{
    printf("%d", head->data);
    head = head->next;
}
```

Inside the function:

```
C
head
```

is only a copied pointer parameter, so caller's head would not change.

But using `temp` is clearer and safer conceptually:

```
TEXT
head = permanent entry
temp = traveller
```

Memory trick:

HEAD knows the home. TEMP goes on the journey.

47. COUNT NODES

```

c
int countNodes(struct Node *head)
{
    int count = 0;
    struct Node *temp = head;

    while (temp != NULL)
    {
        count++;
        temp = temp->next;
    }

    return count;
}

```

Time:

TEXT
 $O(n)$

Space:

TEXT
 $O(1)$

48. SEARCH IN LINKED LIST

```

c
int search(struct Node *head, int key)
{
    struct Node *temp = head;

    while (temp != NULL)
    {
        if (temp->data == key)
            return 1;

        temp = temp->next;
    }

    return 0;
}

```

Worst case:

TEXT
 $O(n)$

49. BEST/WORST SEARCH

Best

Key at first node:

TEXT

$O(1)$

Worst

Key:

- at last node, or
- absent

TEXT

$O(n)$

Same asymptotic search behavior as unsorted array.

50. FIND LAST NODE

How do we identify last node?

Last node satisfies:

```
C
temp->next == NULL
```

So:

```
C
while (temp->next != NULL)
{
    temp = temp->next;
}
```

After loop:

```
TEXT
temp != last node
```

provided list is nonempty.

51. GATE TRAP — TWO DIFFERENT LOOP CONDITIONS

Traverse all nodes

```
C
while (temp != NULL)
```

Stop at final node

```
C
while (temp->next != NULL)
```

They are not the same.

0-100% common bug.

52. EMPTY LIST BUG

Suppose:

```
TEXT
head = NULL
```

and you write:

```
C
while (head->next != NULL)
```

Invalid.

Why?

You dereference:

```
TEXT
NULL
```

before checking.

Correct logic must first establish:

```
C
head != NULL
```

before using:

```
C
head->next
```

0-100% inter-safety trap.

53. SAFE LAST-NODE EXAMPLE

```
C
struct Node *lastNode(struct Node *head)
{
    if (head == NULL)
        return NULL;

    struct Node *temp = head;

    while (temp->next != NULL)
    {
        temp = temp->next;
    }
}
```

```
    return temp;
}
```

54. WHAT HAPPENS IF WE LOSE head?

Suppose:

TEXT

```
h e a d !' 1 0 !' 2 0 !' 3 0
```

Then accidentally:

C

```
head = head->next;
```

Now:

TEXT

```
h e a d !' 2 0 !' 3 0
```

If no other pointer remembers node 10, you have lost access to it.

If it was dynamically allocated and not freed:

TEXT

```
memory leak
```

0=important.

55. MEMORY-LEAK EXAMPLE

Before:

TEXT

```
head
!"
```

```
[ 1 0 ] !' [ 2 0 ] !' N U L L
```

Execute:

C

```
head = head->next;
```

After:

TEXT

```
[ 1 0 ] h e a d !' [ 2 0 ] !' N U L L
```

Node 10 may still exist in heap memory but no pointer reaches it.

Therefore it cannot be freed normally anymore.

This is a:

Memory Leak

56. SAFE REMOVAL IDEA

Before changing `head`, preserve the old address:

```
C
struct Node *temp = head;
head = head->next;
free(temp);
```

This is why linked-list pointer-update order matters enormously.

57. POINTER UPDATE ORDER

In arrays, many operations involve shifting values.

In linked lists, operations involve changing links.

If you change a pointer too early, you may lose the rest of the list.

Therefore our rule for every operation will be:

BEFORE !' SAVE NEEDED ADDRESS !' CHANGE LINKS !' FREE IF REQUIRED !' AFTER

~~Don't~~ Make this a habit now.

58. `typedef` SHORTCUT

Instead of repeatedly writing:

```
C
struct Node
```

we can define:

```
C
typedef struct Node
{
    int data;
    struct Node *next;
} Node;
```

Then:

```
C
Node *head;
Node *temp;
```

instead of:

```
C
struct Node *head;
```

Both are fine.

For GATE questions, be comfortable with both styles.

59. ANOTHER COMMON STYLE

```
C
typedef struct Node
{
    int data;
    struct Node *next;
} Node;
```

Then create:

```
C
Node *newNode = malloc(sizeof *newNode);
```

This is concise and readable.

60. CREATE-NODE HELPER

A reusable function:

```
C
Node *createNode(int value)
{
    Node *newNode = malloc(sizeof *newNode);

    if (newNode == NULL)
        return NULL;

    newNode->data = value;
    newNode->next = NULL;

    return newNode;
}
```

61. WHY RETURNING THIS POINTER IS SAFE

We return:

```
c
newNode
```

which points to dynamically allocated memory.

The local pointer variable `newNode` disappears when the function returns.

But the allocated node survives until:

```
c
free(...)
```

So returning the heap allocation address is valid.

Recall Lesson 16.

62. DON'T DO THIS

```
c
Node *createNode(int value)
{
    Node n;

    n.data = value;
    n.next = NULL;

    return &n;
}
```

Wrong.

`n` has automatic lifetime.

After function returns:

```
TEXT
n no longer exists
```

Returned pointer becomes dangling.

0-Minor C + linked-list trap.

63. NODE SIZE

Suppose:

```
c
struct Node
{
    int data;
    struct Node *next;
};
```

Do not blindly calculate:

TEXT

```
sizeof(int) + sizeof(pointer)
```

as exact structure size in every implementation.

Padding/alignment may exist.

Therefore use:

C

```
sizeof(struct Node)
```

or better:

C

```
sizeof *p
```

with `malloc`.

64. BASIC LINKED-LIST TERMINOLOGY

~~Ø=K%~~ Know these terms:

Node

Individual list element.

Data / info field

Stores value.

Link / next field

Stores address of next node.

Head

Pointer to first node.

Tail

Usually the last node.

NULL link

Marks end in a standard singly linked list.

Empty list

```
head == NULL .
```

65. WHAT IS A SINGLY LINKED LIST?

A linked list where each node stores:

TEXT

one next pointer

and traversal is naturally forward:

TEXT

h e a d ! ' n e x t ! ' n e x t ! ' . . .

This is:

Singly Linked List

Our current structure:

C

```
struct Node
{
    int data;
    struct Node *next;
};
```

is a singly linked-list node.

66. OTHER LINKED-LIST TYPES — PREVIEW

Later we study:

Doubly Linked List

TEXT

N U L L ! • 1 0 ! Ä 2 0 ! Ä 3 0 ! ' N U L L

Each node has:

TEXT

prev
next

Circular Linked List

Last node points back:

TEXT

1 0 ! ' 2 0 ! ' 3 0
! ' ! "
% % % % % % % % % %

No **NULL** at the logical cycle end.

67. LINKED LIST VS STACK/QUEUE

Linked list is a data representation.

We can use linked nodes to implement:

- stack
- queue
- adjacency lists
- hash chains
- graph structures

So understanding linked lists helps later modules.

68. GATE FOCUS

Ø=MUST KNOW:

- self-referential node structure
- head
- next
- NULL
- ->
- dynamic allocation
- non-contiguous nodes
- traversal
- array vs linked-list access
- pointer safety
- search $O(n)$
- insert-at-head concept $O(1)$
- loss of head / memory leak

69. GATE TRAP — head TYPE

For:

```
C
struct Node
{
    int data;
    struct Node *next;
};
```

Typical:

```
C
struct Node *head;
```

head is:

TEXT

pointer to node

not:

TEXT

node object

70. GATE TRAP — next TYPE

C

head->next

has type:

C

struct Node *

But:

C

head->next->data

has type:

C

int

assuming data is int.

71. GATE TRAP — NULL

If:

C

head == NULL

then:

C

head->data

is invalid.

Never dereference a NULL pointer.

72. GATE TRAP — LAST NODE

Last node:

```
TEXT
node->next == NULL
```

not:

```
TEXT
node == NULL
```

`node == NULL` means there is no node at that pointer.

73. GATE TRAP — LINKED LIST ACCESS

Even if list has `n` nodes, accessing logical element `i` generally requires:

```
TEXT
O(i)
```

traversal from head.

Worst:

```
TEXT
O(n)
```

No constant-time random indexing.

74. GATE TRAP — SEARCH + INSERT

If location already known:

```
TEXT
link changes = O(1)
```

If location must be searched:

```
TEXT
search O(n) + link O(1)
= O(n)
```

Do not blindly say every linked-list insertion is `O(1)`.

75. GATE TRAP — `malloc`

Always remember:

C

```
Node *p = malloc(sizeof *p);
```

can return:

TEXT

NULL

on failure.

For conceptual exam code, failure checks may be omitted for brevity, but robust C should check.

76. ARRAY VS LINKED LIST — MEMORY DIAGRAM

Array

TEXT

base

!"

```
+---+---+---+---+
| 10 | 20 | 30 | 40 |
+---+---+---+---+
```

Connection:

TEXT

physical positions

Linked List

TEXT

head

!"

```
+---+---+      +---+---+      +---+---+
| 10 | •----->| 20 | •----->| 30 | NULL |
+---+---+      +---+---+      +---+---+
```

Connection:

TEXT

stored addresses

77. COMPLEXITY TABLE — BASIC SINGLY LIST

Assuming only `head` pointer and no extra tail pointer:

Operation	Complexity
Access first node	$O(1)$
Access kth node	$O(k)$
Search	$O(n)$
Traverse	$O(n)$
Insert at head	$O(1)$
Delete head	$O(1)$
Find last node	$O(n)$
Insert at end	$O(n)$
Delete last	$O(n)$

We'll refine these in subsequent lessons.

78. WHAT IF WE STORE A TAIL POINTER?

If we maintain:

```
C
Node *tail;
```

pointing to last node, insertion at end can often become:

```
TEXT
O(1)
```

because we don't need to traverse to find the last node.

Linked-list performance depends partly on what metadata/pointers we maintain.

79. BUT DELETE LAST IN SINGLY LIST?

Even with a tail pointer, to delete the last node in a singly linked list you generally need the node **before** tail.

Without a `prev` link, finding that predecessor usually takes:

```
TEXT
O(n)
```

This motivates doubly linked lists.

80. PLACEMENT CONNECTION

Linked-list interview questions heavily test pointer thinking:

- reverse list
- find middle
- detect cycle
- merge sorted lists
- nth node from end
- intersection
- palindrome
- remove duplicates
- reverse in groups

If your node/pointer fundamentals are strong, these become much easier.

81. MEMORY TRICKS

Linked list

VALUE + WHERE NEXT?

Head

HEAD knows the first address.

Last node

NEXT knows nobody !' NULL.

Traversal

FOLLOW THE ARROWS.

Array vs list

Array: neighbors by location. List: neighbors by address.

Safety

Never cut an arrow before saving where it leads.

Ø=This last mnemonic will be crucial for deletion/reversal.

82. NOTEBOOK NOTES

Node

```
C
struct Node
{
    int data;
    struct Node *next;
};
```

Head

```
C
struct Node *head = NULL;
```

Empty list:

```
TEXT
head == NULL
```

Last node

```
TEXT
node->next == NULL
```

Traversal

```
C
Node *temp = head;

while (temp != NULL)
{
    printf("%d ", temp->data);
    temp = temp->next;
}
```

Complexity

```
TEXT
Traversal = O(n)
Search    = O(n)
Access kth= O(n) worst
Insert head = O(1)
Delete head = O(1)
```

83. 30-SECOND REVISION

Remember:

1. Linked-list nodes need not be contiguous.
2. Every singly node stores data + next pointer.
3. `head` stores first node address.

4. `head==NULL` means empty list.
 5. Last node has `next==NULL`.
 6. `p->data == (*p).data`.
 7. Nodes are commonly created using `malloc`.
 8. Traversal follows `next`.
 9. Random access is $O(n)$, not $O(1)$.
 10. Pointer-update order matters because losing an address can lose the rest of the list.
-

84. ACTIVE RECALL

Q1

Why can't this be used?

```
C
struct Node
{
    int data;
    struct Node next;
};
```

Q2

What does:

```
C
head == NULL
```

mean?

Q3

How do you identify the last node?

Q4

What is the worst-case time to access the kth node?

Q5

Why can inserting at head be $O(1)$?

ANSWERS

A1

A node would contain another complete node recursively forever, so its size could not be finite. A pointer to the next node is required.

A2

The linked list is empty.

A3

Its:

```
c
next
```

pointer is `NULL`.

A4

```
TEXT
O(n)
```

in the worst case.

A5

No shifting/traversal is required if the new node is already created; only a constant number of pointer changes are needed.

85. PRACTICE — EASY

Q1

For:

```
TEXT
h e a d ! ' 1 0 ! ' 2 0 ! ' 3 0 ! ' N U L L
```

what are:

```
TEXT
head->data
head->next->data
head->next->next->data
```

?

Q2

What does:

```
TEXT
head->next->next->next
```

contain for that list?

Q3

If:

TEXT

```
head = NULL
```

how many nodes are in the list?

86. PRACTICE — MODERATE

Q4

Suppose:

TEXT

```
head ! ' A ! ' B ! ' C ! ' NULL
```

What happens conceptually if you execute:

C

```
head = head->next;
```

without saving or freeing node A?

Q5

Why is accessing the 500th node of a linked list not $O(1)$ like `a[499]` in an array?

Q6

What is the difference between:

C

```
Node n;
```

and:

C

```
Node *p;
```

?

87. PRACTICE — GATE TRAPS

Q7

Suppose `temp` points to the final node.

Which is true?

A.

TEXT

```
temp == NULL
```

B.

TEXT

```
temp->next == NULL
```

Q8

For:

C

```
Node *p = malloc(sizeof *p);
```

where are these two logically different objects?

1. pointer variable `p`
2. dynamically allocated node

Q9

Why is:

C

```
return &localNode;
```

unsafe if `localNode` is an automatic local structure?

88. PLACEMENT THINKING

Consider:

TEXT

```
1  !'  2  !'  3  !'  4  !'  5  !'  N U L L
```

Without writing code yet, think:

How could we find the middle node without first counting all nodes?

Hint:

TEXT

```
one pointer moves 1 step  
another pointer moves 2 steps
```

When the faster one reaches the end:

TEXT

slow is near the middle

This is the famous:

Slow & Fast Pointer Technique

We'll formally use it later.

PROGRESS

MODULE 2 — ARRAYS' COMPLETE

MODULE 3 — LINKED LISTS

' Lesson 29 — Intuition, Nodes & Memory

NEXT — LESSON 30

Singly Linked List — Creation, Traversal, Length, Search & Core Pointer Operations

Next we turn the concept into a proper working singly linked list.

We will build:

TEXT

head

!"

1 0 !' 2 0 !' 3 0 !' 4 0 !' N U L L

and learn:

- create nodes cleanly
- connect nodes
- build a list
- iterative traversal
- recursive traversal
- count nodes
- search
- find last node
- access kth node
- head vs temp
- tail pointer
- exact complexities
- NULL/empty/single-node edge cases

Then Lesson 31 begins the most important part:

2. BASIC NODE STRUCTURE

We use:

```
c
typedef struct Node
{
    int data;
    struct Node *next;
} Node;
```

Each node contains:

```
TEXT
+-----+-----+
|  data  |  next  |
+-----+-----+
```

Example:

```
TEXT
+-----+-----+
|    10    | • - - - - - !'  n e x t   n o d e
+-----+-----+
```

3. WHAT MAKES IT "SINGLY"?

A singly linked-list node contains only one link:

```
c
Node *next;
```

So movement is naturally:

```
TEXT
FORWARD ONLY
```

Example:

```
TEXT
head
!"
10 !' 20 !' 30 !' 40 !' N U L L
```

From 20, we know where 30 is.

But 20 does not automatically store the address of 10.

That is why it is called:

Singly Linked List

4. CREATE NODE HELPER

Instead of repeatedly writing `malloc`, create a function:

```
C
Node *createNode(int value)
{
    Node *newNode = malloc(sizeof *newNode);

    if (newNode == NULL)
        return NULL;

    newNode->data = value;
    newNode->next = NULL;

    return newNode;
}
```

5. STORY

Calling:

```
C
createNode(10);
```

conceptually creates:

```
TEXT
+-----+-----+
|  10  | NULL |
+-----+-----+
```

and returns its address.

So:

```
C
Node *p = createNode(10);
```

means:

```
TEXT
p
!"
+-----+-----+
|  10  | NULL |
+-----+-----+
```

6. WHY `newNode->next = NULL`?

When a node is created, we don't yet know its next node.

So initialize:

```
C
newNode->next = NULL;
```

This avoids leaving `next` as an indeterminate pointer.

Very important pointer safety habit.

7. BUILD A LIST MANUALLY

Let's create:

```
TEXT
1 0  !'  2 0  !'  3 0  !'  N U L L
```

Code:

```
C
Node *first = createNode(10);
Node *second = createNode(20);
Node *third = createNode(30);
```

Initially:

```
TEXT
f i r s t    !'  [ 1 0  |  N U L L ]
s e c o n d !'  [ 2 0  |  N U L L ]
t h i r d    !'  [ 3 0  |  N U L L ]
```

They are not connected.

8. CONNECT THEM

```
C
first->next = second;
second->next = third;
```

`third->next` is already `NULL`.

Now:

```
TEXT
first
!"
1 0  !'  2 0  !'  3 0  !'  N U L L
```

Set:


```
C
Node *head = first;
```

Then:

```
TEXT
head !' 10 !' 20 !' 30 !' NULL
```

9. FULL BASIC PROGRAM

```
C
#include <stdio.h>
#include <stdlib.h>

typedef struct Node
{
    int data;
    struct Node *next;
} Node;

Node *createNode(int value)
{
    Node *newNode = malloc(sizeof *newNode);

    if (newNode == NULL)
        return NULL;

    newNode->data = value;
    newNode->next = NULL;

    return newNode;
}

int main(void)
{
    Node *first = createNode(10);
    Node *second = createNode(20);
    Node *third = createNode(30);

    if (first == NULL || second == NULL || third == NULL)
        return 1;

    first->next = second;
    second->next = third;

    Node *head = first;

    return 0;
}
```

This creates the list.

10. TRAVERSAL

Traversal means:

Visit every node from head until NULL .

Pattern:

```
C
Node *temp = head;

while (temp != NULL)
{
    // process node

    temp = temp->next;
}
```

Memorize this template.

11. PRINT LIST

```
C
void printList(Node *head)
{
    Node *temp = head;

    while (temp != NULL)
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }
}
```

For:

```
TEXT
1 0 !' 2 0 !' 3 0 !' N U L L
```

output:

```
TEXT
10 20 30
```

12. TRACE TRAVERSAL

Initially:

```
TEXT
temp = head
```

So:

TEXT

```
t e m p ! ' 1 0
```

Loop 1:

TEXT

```
print 10  
temp = temp->next
```

Now:

TEXT

```
t e m p ! ' 2 0
```

Loop 2:

TEXT

```
print 20  
t e m p ! ' 3 0
```

Loop 3:

TEXT

```
print 30  
t e m p ! ' N U L L
```

Loop stops.

13. WHY `while (temp != NULL)`?

Because every valid node address is non-NULL.

Once:

TEXT

```
temp == NULL
```

there is no node left.

So:

TEXT

```
temp != NULL
```

means:

A current node exists. Process it.

14. TRAVERSAL COMPLEXITY

For `n` nodes:

TEXT

```
T i m e   =   ~ ( n )
```

We use only:

TEXT

```
temp
```

So:

TEXT

```
A u x i l i a r y   S p a c e   =   ~ ( 1 )
```

for iterative traversal.

15. HEAD VS TEMP

Suppose:

TEXT

```
h e a d ! '   1 0 ! '   2 0 ! '   3 0
```

We usually do:

C

```
Node *temp = head;
```

Why?

Because:

TEXT

```
head = entry point  
temp = traveller
```

If we preserve `head`, we can still reach the entire list afterward.

Memory trick:

HEAD stays home; TEMP travels.

16. PASSING `head` TO A FUNCTION

Consider:

C

```
void printList(Node *head)  
{
```

```

while (head != NULL)
{
    printf("%d ", head->data);
    head = head->next;
}

```

Does this destroy the caller's head?

No.

Because C passes the pointer value by value.

The function's `head` is a copy.

Changing it does not change caller's pointer variable.

Still, using `temp` often makes intent clearer.

17. RECURSIVE TRAVERSAL

We can also traverse recursively:

```

c
void printRecursive(Node *head)
{
    if (head == NULL)
        return;

    printf("%d ", head->data);
    printRecursive(head->next);
}

```

For:

```

TEXT
1 0  !'  2 0  !'  3 0

```

calls:

```

TEXT
print(10)
!'  p r i n t ( 2 0 )
!'  p r i n t ( 3 0 )
!'  p r i n t ( N U L L )

```

Output:

```

TEXT
10 20 30

```

18. RECURSIVE TRAVERSAL COMPLEXITY

Visits each node once:

TEXT

`T i m e = ~ ( n )`

But recursion depth:

TEXT

`~ ( n )`

Therefore:

TEXT

`A u x i l i a r y   S t a c k = ~ ( n )`

Compare:

Traversal	Time	Extra Space
Iterative	$\sim(n)$	$\sim(1)$
Recursive	$\sim(n)$	$\sim(n)$

0-1 placement/GATE distinction.

19. PRINT LIST IN REVERSE — RECURSION PREVIEW

Interesting:

C

```
void printReverse(Node *head)
{
    if (head == NULL)
        return;

    printReverse(head->next);

    printf("%d ", head->data);
}
```

For:

TEXT

`1 0 !' 2 0 !' 3 0`

output:

TEXT

`30 20 10`

Why?

Printing happens while recursion unwinds.

This does **not reverse the links**.

It only prints values in reverse order.

Don't confuse reverse printing with reversing the linked list.

20. COUNT NODES

We want length:

TEXT

1 0 !' 2 0 !' 3 0 !' 4 0 !' N U L L

answer:

TEXT

4

Code:

```
C
int length(Node *head)
{
    int count = 0;
    Node *temp = head;

    while (temp != NULL)
    {
        count++;
        temp = temp->next;
    }

    return count;
}
```

21. TRACE LENGTH

TEXT

count = 0

v i s i t	1 0 !'	c o u n t	=	1
v i s i t	2 0 !'	c o u n t	=	2
v i s i t	3 0 !'	c o u n t	=	3
v i s i t	4 0 !'	c o u n t	=	4

Then:

TEXT

temp = NULL

return:

TEXT

4

22. LENGTH COMPLEXITY

TEXT

```
Time =  $O(n)$   
Space =  $O(1)$ 
```

Why not $O(1)$?

Because a basic singly linked list does not automatically store its length.

We must count nodes.

23. CAN LENGTH BE $O(1)$?

Yes, if the list data structure maintains a separate variable:

C

```
int size;
```

and updates it during insertion/deletion.

Then querying size can become:

TEXT

```
 $O(1)$ 
```

~~$O(1)$~~ Data-structure complexity depends on stored metadata.

24. RECURSIVE LENGTH

C

```
int lengthRecursive(Node *head)  
{  
    if (head == NULL)  
        return 0;  
  
    return 1 + lengthRecursive(head->next);  
}
```

For:

TEXT

```
1 0 !' 2 0 !' 3 0
```

TEXT

```
length(10)  
= 1 + length(20)  
  
= 1 + 1 + length(30)
```



```
= 1 + 1 + 1 + length(NULL)

= 3
```

Time:

```
TEXT
O(n)
```

Stack:

```
TEXT
O(n)
```

25. SEARCH IN SINGLY LINKED LIST

Want to find:

```
TEXT
key = 30
```

in:

```
TEXT
1 0 !'  2 0 !'  3 0 !'  4 0 !'  N U L L
```

We must follow links.

```
C
int search(Node *head, int key)
{
    Node *temp = head;

    while (temp != NULL)
    {
        if (temp->data == key)
            return 1;

        temp = temp->next;
    }

    return 0;
}
```

26. SEARCH TRACE

Search 30.

```
TEXT
10 == 30? No
```

20 == 30? No

30 == 30? Yes

Return:

TEXT

1

27. SEARCH COMPLEXITY

Best case

First node:

TEXT

$O(1)$

Worst case

Last node or absent:

TEXT

$O(n)$

Average:

TEXT

$O(n)$

Auxiliary:

TEXT

$O(1)$

28. WHY SORTED LINKED LIST STILL DOESN'T GIVE ORDINARY $O(\log n)$ SEARCH

Suppose sorted:

TEXT

1 0 !' 2 0 !' 3 0 !' 4 0 !' 5 0

You may think:

Use binary search.

Problem:

To inspect middle node, you cannot do:

```
TEXT
list[2]
```

in $O(1)$.

You must traverse.

So the random-access advantage required by standard array binary search is absent.

~~$O(1)$~~ Important GATE concept.

29. ACCESS kth NODE

Suppose:

```
TEXT
1 0 !' 2 0 !' 3 0 !' 4 0 !' 5 0
```

Find node at zero-based index:

```
TEXT
3
```

Need:

```
TEXT
40
```

Code:

```
C
Node *getNode(Node *head, int index)
{
    int i = 0;
    Node *temp = head;

    while (temp != NULL)
    {
        if (i == index)
            return temp;

        i++;
        temp = temp->next;
    }

    return NULL;
}
```

30. COMPLEXITY OF kth ACCESS

If index is k :

TEXT

$O(k)$

Worst case:

TEXT

$O(n)$

Compare array:

TEXT

`a [ k ] != 0 ( 1 )`

$O(1)$ is the fundamental array vs linked-list trade-off.

31. FIND LAST NODE

List:

TEXT

`1 0 != 2 0 != 3 0 != NULL`

Last node is `30`.

Condition:

TEXT

`node->next == NULL`

Code:

C

```
Node *lastNode(Node *head)
{
    if (head == NULL)
        return NULL;

    Node *temp = head;

    while (temp->next != NULL)
    {
        temp = temp->next;
    }

    return temp;
}
```

32. TRACE FIND LAST

Initially:

TEXT

```
t e m p != 10
```

Check:

TEXT

```
10.next != NULL
```

move to 20.

TEXT

```
20.next != NULL
```

move to 30.

Now:

TEXT

```
30.next == NULL
```

stop.

Return node 30.

33. WHY NOT `while(temp != NULL)` HERE?

If we use:

C

```
while (temp != NULL)
    temp = temp->next;
```

after loop:

TEXT

```
temp == NULL
```

We have gone **past** the final node.

If we want the actual last node, stop when:

TEXT

```
temp->next == NULL
```

Common GATE/coding distinction.

34. EMPTY LIST EDGE CASE

This code is dangerous:

```
C
Node *temp = head;

while (temp->next != NULL)
```

if:

```
TEXT
head == NULL
```

because `temp` is NULL.

Therefore:

```
C
if (head == NULL)
    return NULL;
```

must happen first.

35. EMPTY LIST

Representation:

```
TEXT
head != NULL
```

Operations:

Traverse

Does nothing.

Length

```
TEXT
0
```

Search

Not found.

Last node

```
TEXT
NULL
```

~~0 = 1~~ Always consider empty list as an edge case.

36. SINGLE-NODE LIST

TEXT

```
head
! "
1 0 ! ' N U L L
```

Properties:

TEXT

```
head != NULL
head->next == NULL
```

Therefore same node is:

- first node
- last node
- head
- tail if tail maintained

37. TWO-NODE LIST

TEXT

```
head
! "
1 0 ! ' 2 0 ! ' N U L L
```

Then:

TEXT

```
head->next
```

points to 20.

And:

TEXT

```
head->next->next
```

is NULL.

38. TAIL POINTER

Instead of only storing:

C

```
Node *head;
```

we may also maintain:

```
C
Node *tail;
```

where:

```
TEXT
t a i l ! ' l a s t n o d e
```

Example:

```
TEXT
head          tail
! "
1 0 ! ' 2 0 ! ' 3 0 ! ' 4 0 ! ' N U L L ! "
```

39. WHY TAIL IS USEFUL

Without tail, to find last node:

```
TEXT
O(n)
```

With tail:

```
TEXT
O(1)
```

because:

```
TEXT
tail
```

already stores the last-node address.

This is especially useful for appending.

40. APPEND WITH ONLY HEAD

To add node at end:

```
TEXT
1 0 ! ' 2 0 ! ' 3 0
```

want:

```
TEXT
1 0 ! ' 2 0 ! ' 3 0 ! ' 4 0
```

Need to first reach node 30:

TEXT

$O(n)$

Then change its `next` :

TEXT

$O(1)$

Total:

TEXT

$O(n)$

41. APPEND WITH TAIL

If:

TEXT

`tail -> next = 30`

then:

TEXT

`newNode -> next = 40`

we can do:

C

```
tail->next = newNode;
tail = newNode;
```

Both constant operations.

Therefore:

TEXT

$O(1)$

$O(1)$ - Important performance improvement.

42. BEFORE !' POINTER CHANGES !' AFTER

Suppose:

TEXT

BEFORE

```
head      tail
! "      ! "
10 !'  20 !'  30 !'  NULL
```

```
newNode
!"
4 0 !" NULL
```

Pointer changes:

```
C
tail->next = newNode;
tail = newNode;
```

After:

```
TEXT
head                                tail                                !"
!"
1 0 !" 2 0 !" 3 0 !" 4 0 !" NULL
```

43. EMPTY LIST WITH HEAD + TAIL

If empty:

```
TEXT
head = NULL
tail = NULL
```

When inserting first node:

```
TEXT
head = newNode
tail = newNode
```

Both point to same node.

Example:

```
TEXT
head
!"
1 0 !" NULL
!"
tail
```

$\emptyset = \text{Edge case}$.

44. SEARCH RETURNING NODE POINTER

Instead of merely returning `1/0`, we can return the matching node:

```

c
Node *findNode(Node *head, int key)
{
    Node *temp = head;

    while (temp != NULL)
    {
        if (temp->data == key)
            return temp;

        temp = temp->next;
    }

    return NULL;
}

```

This is often more useful because caller can modify or insert after the found node.

45. EXAMPLE

Suppose:

```

TEXT
1 0  !'   2 0  !'   3 0

```

Call:

```

c
Node *p = findNode(head, 20);

```

Then:

```

TEXT
p  !'   n o d e   2 0

```

So:

```

c
p->data

```

is:

```

TEXT
20

```

and:

```

c
p->next

```

points to node 30.

46. WHY RETURN POINTER?

If we only return index:

TEXT

1

we would still need traversal to reach the node again.

Returning the node's pointer gives direct access once it is found.

0=Placement insight.

47. RECURSIVE SEARCH

```
C
Node *searchRecursive(Node *head, int key)
{
    if (head == NULL)
        return NULL;

    if (head->data == key)
        return head;

    return searchRecursive(head->next, key);
}
```

Time:

TEXT

$O(n)$

Stack:

TEXT

$O(n)$

48. FIND MINIMUM / MAXIMUM

A linked list can be scanned like an array.

Example:

```
C
int maximum(Node *head)
{
    int max = head->data;

    Node *temp = head->next;

    while (temp != NULL)
    {
        if (temp->data > max)
            max = temp->data;
    }
}
```

```

    temp = temp->next;
}

return max;
}

```

Assumes:

```

TEXT
head != NULL

```

Time:

```

TEXT
O(n)

```

49. GATE TRAP — ACCESS FIRST VS LAST

With only `head` :

First node

```

TEXT
head

```

Immediate:

```

TEXT
O(1)

```

Last node

must traverse:

```

TEXT
O(n)

```

Unless a `tail` pointer is maintained.

50. GATE TRAP — SEARCH

Even if key values happen to be sorted:

```

TEXT
O(n)

```

for ordinary linked-list sequential search.

Do not write `O(log n)` just because sorted.

51. GATE TRAP — LENGTH

Without stored size:

```
TEXT
0(n)
```

With maintained size field:

```
TEXT
0(1)
```

Always inspect implementation assumptions.

52. GATE TRAP — `temp->next` AFTER TEMP BECOMES NULL

Bad:

```
C
while (temp != NULL)
{
    temp = temp->next;
}

printf("%d", temp->data);
```

After loop:

```
TEXT
temp == NULL
```

So:

```
C
temp->data
```

is invalid.

Very common pointer bug.

53. GATE TRAP — WRONG ORDER

Suppose:

```
TEXT
temp != current node
```

You execute:

```
c
temp = temp->next;
printf("%d", temp->data);
```

Now you're printing the **next node**, not the original current node.

Order matters:

```
c
printf("%d", temp->data);
temp = temp->next;
```

for normal traversal.

54. GATE TRAP — NODE POINTER VS DATA

```
c
temp
```

is an address.

```
c
temp->data
```

is the stored value.

```
c
temp->next
```

is another node address.

Do not mix these.

55. TRACE TABLE METHOD

For linked-list questions, use a pointer table.

Example:

```
TEXT
head !' 1 0 !' 2 0 !' 3 0 !' N U L L
temp = head
```

Step	temp->data	temp->next points to
1	10	node 20
2	20	node 30
3	30	NULL

This method will become extremely useful in insertion/deletion/reversal.

56. CORE COMPLEXITY TABLE

Assuming only `head` and no size/tail metadata:

Operation	Time
Check empty	$O(1)$
First node access	$O(1)$
kth node access	$O(n)$ worst
Traverse	$O(n)$
Length	$O(n)$
Search	$O(n)$
Find last	$O(n)$
Insert at head	$O(1)$
Delete head	$O(1)$
Append	$O(n)$

57. WITH TAIL POINTER

Operation	Time
Access first	$O(1)$
Access last	$O(1)$
Append	$O(1)$
Search	$O(n)$
kth access	$O(n)$
Delete last	usually $O(n)$

Why delete last still $O(n)$?

Because in a singly linked list you need the **previous node** to update its `next = NULL`.

58. WHY TAIL DOESN'T SOLVE DELETE-LAST

Suppose:

TEXT

```
1 0 !' 2 0 !' 3 0 !' 4 0 !' N U L L
      !'
      tail
```

To delete 40:

We need node 30 so we can do:

C

```
30->next = NULL;
```

But tail only gives 40.

There is no backward link.

So we must find predecessor:

TEXT

```
0(n)
```

This motivates:

Doubly Linked List

later.

59. ARRAYS VS LIST — SEARCH & ACCESS

Array:

TEXT

```
Index access = 0(1)
Search unsorted = 0(n)
```

Linked list:

TEXT

```
Position access = 0(n)
Search = 0(n)
```

The linked list's advantage is not fast search.

Its strengths are flexible link updates and dynamic growth.

60. MEMORY MANAGEMENT — FREEING AN ENTIRE LIST

Every `malloc`ed node should eventually be freed.

Suppose:

TEXT

```
1 0 !' 2 0 !' 3 0 !' N U L L
```

We must not simply:

```
C
free(head);
```

because that frees only first node.

The remaining nodes would leak if we lose their addresses.

61. CORRECT FREE-LIST IDEA

Before freeing current node, save next node.

```
C
void freeList(Node *head)
{
    Node *temp;

    while (head != NULL)
    {
        temp = head;
        head = head->next;
        free(temp);
    }
}
```

~~0=1%~~ This already demonstrates our pointer-order rule.

62. BEFORE !' SAVE !' MOVE !' FREE

For:

TEXT

```
h e a d !' 1 0 !' 2 0 !' 3 0
```

Step 1:

```
C
temp = head;
```

Now both point to 10.

Step 2:

```
C
head = head->next;
```

Now:

TEXT

```
h e a d ! ' 2 0
t e m p ! ' 1 0
```

Step 3:

```
c
free(temp);
```

Node 10 released safely.

Continue.

63. WHY NOT FREE FIRST?

Wrong:

```
c
free(head);
head = head->next;
```

After:

```
c
free(head);
```

`head` is dangling.

Using:

```
c
head->next
```

after free is use-after-free.

Very important C linked-list bug.

64. SAFE RULE

Before freeing a node:

Save anything you will need from it first.

Memory trick:

SAVE !' MOVE !' FREE

This will reappear during deletion.

65. RECURSIVE FREE — OPTIONAL

You could conceptually write:

```
C
void freeList(Node *head)
{
    if (head == NULL)
        return;

    freeList(head->next);
    free(head);
}
```

This works for an acyclic list.

But:

```
TEXT
Stack space = O(n)
```

Iterative freeing:

```
TEXT
O(1)
```

extra space.

For long lists, iterative is generally safer.

66. WHAT IF LIST HAS A CYCLE?

Suppose:

```
TEXT
1 0  !'   2 0  !'   3 0
      !'       !"
      % % % % % %
```

Then traversal:

```
C
while (temp != NULL)
```

never reaches NULL.

It can run forever.

This is why cycle detection is an important linked-list interview topic.

We'll study Floyd's algorithm in Lesson 36.

67. LINKED LIST INVARIANT

For a normal acyclic singly linked list:

TEXT

starting from head
and repeatedly following next
must eventually reach NULL

That is the structural assumption behind normal traversal.

68. INTERVIEW QUESTIONS

Q1. Complexity of linked-list traversal?

TEXT

$O(n)$

Q2. Complexity of accessing kth node?

TEXT

$O(n)$

worst case.

Q3. Why can append be $O(1)$ with a tail pointer?

Because last node is already directly available.

Q4. Why is delete-last still $O(n)$ with only singly links?

Need the predecessor of tail.

Q5. Why must you save `next` before freeing a node?

Because after `free`, accessing the node's fields is invalid.

69. MEMORY TRICKS

Traversal

PROCESS !' FOLLOW NEXT

Length

ONE NODE = ONE COUNT

Search

FOLLOW until FOUND or NULL

Head

First door

Tail

Last door

Freeing

SAVE !' MOVE !' FREE

kth access

No jumping; follow arrows.

70. NOTEBOOK NOTES

Traversal

```
C
Node *temp = head;

while (temp != NULL)
{
    printf("%d ", temp->data);
    temp = temp->next;
}
```

TEXT

Time $O(n)$
Space $O(1)$

Length

```
C
while (temp != NULL)
{
    count++;
    temp = temp->next;
}
```

TEXT

$O(n)$

Search

TEXT

Best $O(1)$
Worst $O(n)$

Last node

```
C
while (temp->next != NULL)
{
    temp = temp->next;
}
```

Need:

TEXT

head != NULL

Free list

```
C
while (head != NULL)
{
    temp = head;
    head = head->next;
    free(temp);
}
```

71. 30-SECOND REVISION

Remember:

1. Singly list has one `next` pointer per node.
2. Traversal starts from `head`.
3. Stop at `NULL`.
4. Traversal = $O(n)$.
5. kth node = $O(n)$ worst case.
6. Search = $O(n)$ worst case.
7. Last node has `next == NULL`.
8. Tail can make append $O(1)$.
9. Tail does not make delete-last $O(1)$ in singly list.
10. Before freeing: **save what you need first**.

72. ACTIVE RECALL

Q1

What loop condition visits every node?

Q2

What loop condition stops on the last node?

Q3

What is recursive traversal's auxiliary space?

Q4

Why is kth-node access not $O(1)$?

Q5

What is the safe order when freeing the first node?

ANSWERS

A1

```
C
while (temp != NULL)
```

A2

```
C
while (temp->next != NULL)
```

after ensuring `temp` is not NULL.

A3

```
TEXT
O(n)
```

A4

Nodes are not contiguous/random-accessible; links must be followed sequentially.

A5

```
TEXT
SAVE !'  MOVE  HEAD !'  FREE  OLD  NODE
```

73. PRACTICE

Q1

Given:

TEXT

h e a d ! ' 5 ! ' 1 0 ! ' 1 5 ! ' N U L L

what values are printed by normal traversal?

Q=Q2

How many nodes?

TEXT

1 0 ! ' 2 0 ! ' 3 0 ! ' 4 0 ! ' 5 0 ! ' N U L L

What is the time complexity of computing this length if size is not stored?

Q=Q3

For:

TEXT

h e a d ! ' 1 0 ! ' 2 0 ! ' 3 0 ! ' N U L L

what does this point to?

C

head->next->next

What does this contain?

C

head->next->next->data

Q=Q4

Why is:

C

while (temp->next != NULL)

unsafe when the list may be empty?

Q=Q5

Suppose only `head` is maintained.

State complexity of:

- access first
- access last

- append at end
- search

Q6

Now suppose both `head` and `tail` are maintained.

Which of those operations improve to $O(1)$?

Q7 — MEMORY SAFETY

Explain why this is wrong:

```
C
free(head);
head = head->next;
```

Write the safe conceptual order.

Q8 — RECURSION

What does this print?

```
C
void f(Node *head)
{
    if (head == NULL)
        return;

    f(head->next);
    printf("%d ", head->data);
}
```

for:

```
TEXT
1 0 !'  2 0 !'  3 0 !'  N U L L
```

Also state time and stack complexity.

Q9 — PLACEMENT

You need to find a value, and after finding it you immediately want to insert a node after it.

Why might returning a `Node *` from `search` be more useful than returning only `1/0`?

PROGRESS

MODULE 3 — LINKED LISTS

NEXT — LESSON 31

Insertion in Singly Linked List — Head, End, Position & After a Node

Next is one of the most important pointer-update lessons.

We will use the permanent format:

BEFORE !' POINTER CHANGES !' AFTER

for:

- insert into empty list
- insert at head
- insert at end
- insert at end with tail
- insert after a known node
- insert before a position
- insertion by index
- invalid positions
- maintaining head/tail
- exact complexity
- critical pointer-order traps

The first fundamental insertion will be:

TEXT

BEFORE

```
h e a d !' 1 0 !' 2 0 !' N U L L
n e w !' 5 !' N U L L
```

Pointer changes:

C

```
new->next = head;
head = new;
```

After:

TEXT

```
h e a d !' 5 !' 1 0 !' 2 0 !' N U L L
```

That lesson will build the pointer discipline needed for every later linked-list problem.

USER

next

MODULE 3 — LESSON 31

Insertion in Singly Linked List — Head, End, Position & After a Node

This is one of the most important linked-list lessons because **insertion is mainly about pointer order**.

Our permanent method:

BEFORE !' POINTER CHANGES !' AFTER

GATE importance: Ø=Ý%ØMÚ%ØKØW **Placement importance:** Ø=Ý%ØMÚ%ØKØW

1. CORE IDEA OF INSERTION

Suppose:

TEXT

h e a d !' 1 0 !' 2 0 !' 3 0 !' N U L L

We want to insert:

TEXT

15

between 10 and 20.

Unlike arrays, we do **not shift** 20 and 30.

We only change links.

That is the central linked-list advantage.

2. CREATE A NEW NODE

We assume:

```
c
typedef struct Node
{
    int data;
    struct Node *next;
} Node;
```

Helper:

```
c
Node *createNode(int value)
{
    Node *newNode = malloc(sizeof *newNode);

    if (newNode == NULL)
```

```

        return NULL;

    newNode->data = value;
    newNode->next = NULL;

    return newNode;
}

```

3. INSERTION AT HEAD Ø=Ý%

Before:

```

TEXT
head
! "
1 0 !' 2 0 !' 3 0 !' N U L L

newNode
! "
5 !' N U L L

```

Desired:

```

TEXT
head
! "
5 !' 1 0 !' 2 0 !' 3 0 !' N U L L

```

4. POINTER CHANGES

First:

```

C
newNode->next = head;

```

Now:

```

TEXT
n e w N o d e !' 5 !' 1 0 !' 2 0 !' 3 0 !' N U L L

head -----^

```

Then:

```

C
head = newNode;

```

Final:

TEXT

```
head
! "
5 !' 1 0 !' 2 0 !' 3 0 !' N U L L
```

Order matters.

5. INSERT-AT-HEADFUNCTION

```
C
Node *insertHead(Node *head, int value)
{
    Node *newNode = createNode(value);

    if (newNode == NULL)
        return head;

    newNode->next = head;
    head = newNode;

    return head;
}
```

Call:

```
C
head = insertHead(head, 5);
```

6. WHY RETURN head ?

Because insertion at beginning changes the first-node address.

Before:

```
TEXT
h e a d !' 1 0
```

After:

```
TEXT
h e a d !' 5
```

A function receives a copy of the pointer, so if it changes its local `head`, the caller must receive the new value.

Hence:

```
C
return head;
```

and caller:

```
C
head = insertHead(head, 5);
```

7. DOUBLE-POINTER ALTERNATIVE

We could instead write:

```
C
void insertHead(Node **head, int value)
{
    Node *newNode = createNode(value);

    if (newNode == NULL)
        return;

    newNode->next = *head;
    *head = newNode;
}
```

Call:

```
C
insertHead(&head, 5);
```

Why `Node **`?

Because we want to modify the caller's `Node *head`.

Recall:

```
TEXT
h e a d      ! '   N o d e   *
& h e a d    ! '   N o d e   * *
```

Ø=Important pointer-to-pointer connection.

8. GATE TAKEAWAY

Two valid design styles:

Return new head

```
C
head = insertHead(head, value);
```

Modify caller's head through `Node **`

```
C
insertHead(&head, value);
```

Both are correct.

9. INSERT INTO EMPTY LIST

Suppose:

```
TEXT
head = NULL
```

Insert 10.

New node:

```
TEXT
10 !' NULL
```

For head insertion:

```
C
newNode->next = head;
```

Since:

```
TEXT
head = NULL
```

new node remains:

```
TEXT
10 !' NULL
```

Then:

```
C
head = newNode;
```

Result:

```
TEXT
head !' 10 !' NULL
```

So the same head-insertion logic naturally handles empty lists.

~~Ø=Nil~~ property.

10. COMPLEXITY — INSERT AT HEAD

Operations:

```
TEXT
newNode->next = head
head = newNode
```


constant number of pointer assignments.

Therefore:

```
TEXT
Time = O(1)
```

Ignoring allocator internals, linked-list algorithmic insertion work is constant.

11. INSERT AT END — ONLY HEAD AVAILABLE

Before:

```
TEXT
head !' 1 0 !' 2 0 !' 3 0 !' NULL
```

Insert:

```
TEXT
40
```

Desired:

```
TEXT
head !' 1 0 !' 2 0 !' 3 0 !' 4 0 !' NULL
```

We need to find the current last node.

12. BEFORE !' TRAVERSE !' CHANGE !' AFTER

BEFORE

```
TEXT
head !' 1 0 !' 2 0 !' 3 0 !' NULL
newNode !' 4 0 !' NULL
```

Find last

```
TEXT
temp !' 1 0
      !"
      20
      !"
      30
```

At 30:

```
TEXT
temp->next == NULL
```

Pointer change

```
C
temp->next = newNode;
```

AFTER

```
TEXT
h e a d ! ' 1 0 ! ' 2 0 ! ' 3 0 ! ' 4 0 ! ' N U L L
```

13. INSERT-END FUNCTION

```
C
Node *insertEnd(Node *head, int value)
{
    Node *newNode = createNode(value);

    if (newNode == NULL)
        return head;

    if (head == NULL)
        return newNode;

    Node *temp = head;

    while (temp->next != NULL)
    {
        temp = temp->next;
    }

    temp->next = newNode;

    return head;
}
```

14. WHY HANDLE EMPTY LIST?

If:

```
TEXT
head == NULL
```

this is invalid:

```
C
while (temp->next != NULL)
```

because:

```
TEXT
temp == NULL
```

So empty list needs special handling.

For empty list:

```
C
return newNode;
```

because the new node becomes the first node.

15. COMPLEXITY — INSERT AT END

With only `head` :

Need to traverse:

```
TEXT
n nodes
```

Worst case:

```
TEXT
O(n)
```

Then pointer assignment:

```
TEXT
O(1)
```

Total:

```
TEXT
O(n)
```

16. INSERT AT END WITH TAIL POINTER Ø=Ý%

Suppose we maintain:

```
TEXT
head !' 1 0 !' 2 0 !' 3 0 !' N U L L
      !'
      tail
```

New node:

```
TEXT
4 0 !' N U L L
```

Pointer changes:

```
C
tail->next = newNode;
tail = newNode;
```

Result:

```
TEXT
head !' 1 0 !' 2 0 !' 3 0 !' 4 0 !' N U L L
      !'
      tail
```

No traversal.

Therefore:

```
TEXT
0(1)
```

17. EMPTY LIST WITH HEAD + TAIL

Before:

```
TEXT
head = NULL
tail = NULL
```

Insert first node:

```
TEXT
newNode !' 1 0 !' N U L L
```

Set:

```
C
head = newNode;
tail = newNode;
```

Now:

```
TEXT
head
!'"
1 0 !' N U L L
!'"
tail
```

First-node insertion is a special case when maintaining both pointers.

18. INSERT AFTER A KNOWN NODE

Suppose:

```
TEXT
1 0 !' 2 0 !' 3 0 !' N U L L
      !'
```

p

We already have pointer:

TEXT

p !' n o d e 2 0

Want insert:

TEXT

25

Desired:

TEXT

1 0 !' 2 0 !' 2 5 !' 3 0 !' N U L L

19. CRITICAL POINTER ORDER

BEFORE

TEXT

p
!"
2 0 % % % % !' 3 0

newNode
!"
2 5 !' N U L L

First:

C

```
newNode->next = p->next;
```

Now:

TEXT

2 0 % % % % !' 3 0
 !"
2 5 % % % % % % %

Then:

C

```
p->next = newNode;
```

Final:

TEXT

2 0 !' 2 5 !' 3 0

Correct order.

20. WHY THIS ORDER?

We must preserve the address of node 30.

Originally:

TEXT

p->next

contains node 30's address.

So first save that address into:

c

newNode->next

Then safely overwrite:

c

p->next

21. WRONG ORDER

Suppose you do:

c

```
p->next = newNode;
newNode->next = p->next;
```

After first line:

TEXT

p->next == newNode

So second line becomes:

TEXT

newNode->next = newNode

You create a self-loop:

TEXT

2 0 !' 2 5
! ' !"
% % %

And node 30 is lost from that chain.

0=Major GATE/pointer trap.

22. MEMORY TRICK

For insertion after a node:

NEW connects FORWARD first. OLD connects NEW second.

```
C
newNode->next = p->next;
p->next = newNode;
```

Memorize this pair.

23. FUNCTION — INSERT AFTER KNOWN NODE

```
C
int insertAfter(Node *p, int value)
{
    if (p == NULL)
        return 0;

    Node *newNode = createNode(value);

    if (newNode == NULL)
        return 0;

    newNode->next = p->next;
    p->next = newNode;

    return 1;
}
```

If p is already given:

```
TEXT
Time = O(1)
```

24. BUT IF WE MUST SEARCH FOR p?

Suppose user says:

Insert 25 after value 20 .

We must first find node containing 20.

Search:

TEXT

$O(n)$

Insertion after found node:

TEXT

$O(1)$

Total:

TEXT

$O(n)$

Don't blindly say insertion is $O(1)$.

25. INSERT AT A GIVEN INDEX

Suppose zero-based indices.

Current list:

TEXT

Index: 0 1 2
 1 0 !' 2 0 !' 3 0

Insert 15 at index:

TEXT

1

Desired:

TEXT

Index: 0 1 2 3
 1 0 !' 1 5 !' 2 0 !' 3 0

26. KEY IDEA

To insert at index $pos > 0$, reach:

TEXT

node at index $pos-1$

because we need to modify its `next`.

For position 1:

TEXT

$pos-1 = 0$

so we need node 10.

27. POINTER CHANGES

Before:

```
TEXT
prev
!"
1 0  !'   2 0  !'   3 0

newNode
!"
1 5  !'   N U L L
```

Then:

```
C
newNode->next = prev->next;
```

and:

```
C
prev->next = newNode;
```

Result:

```
TEXT
1 0  !'   1 5  !'   2 0  !'   3 0
```

Same core insertion pattern.

28. FUNCTION — INSERT AT INDEX

```
C
Node *insertAt(Node *head, int pos, int value)
{
    if (pos < 0)
        return head;

    Node *newNode = createNode(value);

    if (newNode == NULL)
        return head;

    if (pos == 0)
    {
        newNode->next = head;
        return newNode;
    }

    Node *temp = head;

    for (int i = 0; i < pos - 1 && temp != NULL; i++)
```

```

{
    temp = temp->next;
}

if (temp == NULL)
{
    free(newNode);
    return head;
}

newNode->next = temp->next;
temp->next = newNode;

return head;
}

```

29. WHY pos-1?

For insertion at index 3:

```

TEXT
0 !' 1 !' 2 !' [ N E W ] !' o l d 3

```

Need node at index:

```

TEXT
2

```

because:

```

TEXT
node2->next

```

currently points to old node3.

We splice new node into that link.

30. INSERTING AT THE END USING POSITION

Suppose length = 3:

```

TEXT
1 0 !' 2 0 !' 3 0

```

Valid insertion positions are:

```

TEXT
0,1,2,3

```

Position 3 means append:

TEXT

1 0 !' 2 0 !' 3 0 !' N E W

We need predecessor at:

TEXT

2

The same general algorithm works.

31. INVALID POSITION

For list length 3:

TEXT

valid insertion positions:
0,1,2,3

Trying position:

TEXT

5

is invalid under a strict positional API.

The traversal won't find a predecessor node.

So:

C

temp == NULL

and insertion is rejected.

32. INSERT BEFORE A KNOWN NODE — SINGLY LIST ISSUE

Suppose:

TEXT

1 0 !' 2 0 !' 3 0
!'
target

Want insert:

TEXT

15

before target.

Desired:

TEXT

1 0 !' 1 5 !' 2 0 !' 3 0

If you only know:

TEXT

t a r g e t !' n o d e 2 0

can you directly find node 10?

No.

Singly linked list has no `prev` pointer.

So usually you must traverse from head to find predecessor.

Worst case:

TEXT

$O(n)$

Important motivation for doubly linked lists.

33. INSERT BEFORE HEAD

Special case:

TEXT

target == head

Then inserting before it is simply:

TEXT

insert at head

which is:

TEXT

$O(1)$

34. INSERT AFTER LAST NODE

Suppose:

TEXT

p !' l a s t n o d e

and:

TEXT

```
p->next == NULL
```

Use normal after-node insertion:

C

```
newNode->next = p->next;
```

becomes:

TEXT

```
newNode->next = NULL
```

Then:

C

```
p->next = newNode;
```

Works perfectly.

So insertion-after logic naturally handles inserting after the tail.

35. MAINTAINING TAIL DURING INSERT AFTER

If:

TEXT

```
p == tail
```

then after:

C

```
p->next = newNode;
```

we must also update:

C

```
tail = newNode;
```

Otherwise `tail` would still point to old last node.

Metadata must remain consistent.

36. HEAD AND TAIL INVARIANTS

For empty list:

TEXT

```
head == NULL
```

```
tail == NULL
```

For non-empty list:

TEXT

```
head != NULL
tail != NULL
tail->next == NULL
```

For one-node list:

TEXT

```
head == tail
```

Useful invariant questions.

37. INSERT-HEAD WITH TAIL MAINTENANCE

Suppose using both head and tail.

If empty:

TEXT

```
head = tail = newNode;
```

Otherwise:

C

```
newNode->next = head;
head = newNode;
```

Tail does not change for ordinary head insertion into nonempty list.

38. INSERT-END WITH TAIL MAINTENANCE

If empty:

C

```
head = newNode;
tail = newNode;
```

Else:

C

```
tail->next = newNode;
tail = newNode;
```

Time:

TEXT

```
O(1)
```

39. COMPLETE LIST WRAPPER — PLACEMENT EXTENSION

A clean design might be:

```
C
typedef struct
{
    Node *head;
    Node *tail;
    int size;
} List;
```

Then we maintain:

```
TEXT
head
tail
size
```

This can give:

```
TEXT
length query = O(1)
append = O(1)
insert head = O(1)
```

Useful software-design insight.

40. COST OF MAINTAINING SIZE

Each successful insertion:

```
C
list->size++;
```

Each deletion:

```
C
list->size--;
```

Then:

```
TEXT
get size
```

is immediate:

```
TEXT
O(1)
```

This is a classic time-vs-metadata design trade-off.

41. GATE QUESTION — WHICH INSERTIONS ARE $O(1)$?

For singly linked list, assuming relevant pointers are already available:

Insert at head

TEXT

$O(1)$

Insert after known node

TEXT

$O(1)$

Insert at end with tail pointer

TEXT

$O(1)$

Insert at kth position with only head

Usually:

TEXT

$O(n)$

due to traversal.

42. GATE TRAP — END INSERTION

Statement:

Insertion at end of linked list is always $O(1)$.

False.

With only head:

TEXT

$O(n)$

With maintained tail:

TEXT

$O(1)$

~~Always~~ Always inspect assumptions.

43. GATE TRAP — POSITION SEARCH

Suppose:

TEXT

Insert after 100th node

Even though the pointer rewiring is constant:

TEXT

$O(1)$

finding 100th node from head requires traversal.

For variable n position worst-case:

TEXT

$O(n)$

44. GATE TRAP — MALLOC "LINK

Creating:

C

```
Node *newNode = malloc(...);
```

does **not** automatically insert the node into the list.

You must explicitly change pointer fields.

Before link:

TEXT

```
l i s t :   1 0   !'   2 0   !'   N U L L
```

```
n e w N o d e :   1 5   !'   N U L L
```

Still separate.

Only assignments like:

C

```
newNode->next = ...
```

```
...
```

connect it.

45. GATE TRAP — LOSING THE REST OF LIST

Suppose:

TEXT

```
1 0   !'   2 0   !'   3 0   !'   N U L L
           !'
```

```
p
```

If you overwrite:

```
c
p->next = newNode;
```

before preserving old `p->next`, node 30 may become unreachable.

Hence:

SAVE/CONNECT OLD NEXT BEFORE OVERWRITING IT

46. ANOTHER SAFE PATTERN WITH TEMP

You can explicitly save:

```
c
Node *nextNode = p->next;

p->next = newNode;
newNode->next = nextNode;
```

This also works.

However, the compact standard pattern is:

```
c
newNode->next = p->next;
p->next = newNode;
```

Both express the same logic.

47. POINTER-CHANGE COUNT

Insertion after known node requires two main link writes:

```
TEXT
1. newNode->next = old successor
2. predecessor->next = newNode
```

So complexity is independent of list length once predecessor is known.

That's why:

```
TEXT
O(1)
```

48. COMPARE WITH ARRAY INSERTION

Suppose correct insertion location already known.

Array

May shift:

TEXT
 $O(n)$

Linked list

If predecessor pointer already available:

TEXT
 $O(1)$

This is a fundamental difference.

49. BUT ARRAY SEARCH BY INDEX

Array can directly get insertion index location:

TEXT
 $O(1)$

while linked list may need:

TEXT
 $O(n)$

to reach the predecessor.

So neither structure is universally better.

50. LINKED LIST INSERTION COMPLEXITY TABLE

Assume only `head` unless specified.

Operation	Time
Insert into empty list	$O(1)$
Insert at head	$O(1)$
Insert after known node	$O(1)$
Insert at kth position	$O(n)$ worst
Insert after searched value	$O(n)$
Insert at end	$O(n)$
Insert at end with tail	$O(1)$

Copy this.

51. MEMORY COST

Each inserted node requires dynamic storage.

For `n` nodes:

TEXT

`O(n)`

total list storage.

But each node also stores pointer overhead.

For singly list:

TEXT

`data + next pointer`

52. EDGE CASE CHECKLIST

Before writing any insertion, ask:

1. Is list empty?
2. Is insertion at head?
3. Is insertion at tail?
4. Is position valid?
5. Does `head` change?
6. Does `tail` change?
7. Have I preserved old `next`?
8. Did allocation succeed?

Excellent coding checklist.

53. EXAMPLE — INSERT HEAD TRACE

Before:

TEXT

```
head !' 1 0 !' 2 0 !' NULL
new !' 5 !' NULL
```

Step 1:

C

```
new->next = head;
```

TEXT

```
new !' 5 !' 10 !' 20 !' NULL
      !'
      head
```

Step 2:

C

```
head = new;
```

After:

TEXT

```
head !' 5 !' 10 !' 20 !' NULL
```

54. EXAMPLE — INSERT AFTER TRACE

Before:

TEXT

```
10 !' 20 !' 30 !' NULL
      !'
      p
new !' 25 !' NULL
```

Step 1:

C

```
new->next = p->next;
```

TEXT

```
new !' 25 !' 30
```

Step 2:

C

```
p->next = new;
```

After:

TEXT

```
10 !' 20 !' 25 !' 30 !' NULL
```

55. EXAMPLE — INSERT AT POSITION 2

Before:

TEXT

Index

0	1	2
1 0	!'	2 0 !' 3 0

Insert:

TEXT

15 at index 2

Need predecessor:

TEXT

i n d e x 1 !' n o d e 2 0

Set:

C

```
new->next = temp->next;
temp->next = new;
```

After:

TEXT

0	1	2	3
1 0	!'	2 0 !' 1 5 !' 3 0	

56. GATE OUTPUT / POINTER TRACE

Suppose:

TEXT

h e a d !' 1 0 !' 2 0 !' 3 0 !' N U L L

and:

C

```
Node *p = head->next;
Node *q = createNode(25);

q->next = p->next;
p->next = q;
```

What list?

p points to:

TEXT

20

q->next becomes:

TEXT

30

then:

TEXT

20->next = 25

Final:

TEXT

1 0 !' 2 0 !' 2 5 !' 3 0 !' N U L L

57. GATE POINTER TRACE — WRONG ORDER

Same list:

C

```
q->next = p;  
p = q;
```

Does this insert **q** into the actual list before node **p**?

Not necessarily.

Why?

Only local pointer **p** changes.

The predecessor's **next** still points to old node.

Example:

TEXT

1 0 !' 2 0 !' 3 0

if **p** originally 20:

TEXT

p !' 2 0

after:

TEXT

q !' 2 5 !' 2 0
p !' 2 5

but list still remains:

TEXT

h e a d !' 1 0 !' 2 0 !' 3 0

because node10's **next** wasn't changed.

Ø=Extremely important.

58. POINTER VARIABLE VS LIST LINK

Changing:

```
c
p = q;
```

changes the pointer variable `p`.

Changing:

```
c
someNode->next = q;
```

changes the actual list structure.

Ø=Keep these separate.

59. INSERT BEFORE KNOWN NODE — WHY POINTER-TO-PREV MATTERS

Suppose:

```
TEXT
prev !' 10 !' target ( 20 )
```

If both `prev` and `target` are known:

```
c
newNode->next = target;
prev->next = newNode;
```

Insertion:

```
TEXT
O(1)
```

But if only `target` is known in a singly linked list:

```
TEXT
finding prev !' O(n)
```

unless special tricks/data constraints apply.

60. INTERVIEW QUESTION — CAN WE INSERT BEFORE A NODE IN O(1)?

Interesting answer:

In a singly linked list, if only a pointer to the target node is given, normally predecessor isn't available.

For generic insertion **before** target:

```
TEXT
not straightforward O(1)
```

without additional information.

There are value-copy tricks in some special interview questions, but they have restrictions and do not represent general link insertion before the exact node.

0=00 Good interview nuance.

61. EMPTY LIST + INVALID POSITION

Suppose:

```
TEXT
head = NULL
```

Valid insertion position:

```
TEXT
0
```

If API uses zero-based strict positions.

Position 1 is invalid because there is no predecessor.

Always define position convention clearly.

62. ONE-BASED VS ZERO-BASED POSITIONS

Some GATE/books say:

```
TEXT
insert at position 1
```

meaning first node.

Coding interviews often use:

```
TEXT
index 0
```

meaning first node.

0=1000 Never assume.

Read whether position/index is zero-based or one-based.

63. ONE-BASED INSERTION FORMULA

If position is one-based:

TEXT

```
p o s i t i o n   1   ! '   h e a d
p o s i t i o n   2   ! '   a f t e r   f i r s t   n o d e
p o s i t i o n   k   ! '   a f t e r   n o d e   k - 1
```

If zero-based:

TEXT

```
i n d e x   0   ! '   h e a d
i n d e x   k   ! '   a f t e r   i n d e x   k - 1
```

Same idea, different numbering.

64. PLACEMENT CONNECTION

Insertion concepts directly support:

- sorted linked-list insertion
- merge two sorted lists
- adjacency-list graph construction
- implementing stack using linked list
- implementing queue
- hash-table chaining
- LRU-style structures with doubly lists later

65. SORTED LINKED-LIST INSERTION — PREVIEW

Suppose sorted:

TEXT

```
1 0   ! '   2 0   ! '   4 0   ! '   N U L L
```

Insert:

TEXT

30

Need to find:

TEXT

```
prev = 20
current = 40
```

Then:

C

```
newNode->next = current;
prev->next = newNode;
```

Result:

TEXT

1 0 !' 2 0 !' 3 0 !' 4 0

Search position:

TEXT

0(n)

Link update:

TEXT

0(1)

Total:

TEXT

0(n)

66. HEAD INSERTION AND CALL-BY-VALUE

Suppose:

```
c
void badInsert(Node *head)
{
    Node *newNode = createNode(5);
    newNode->next = head;
    head = newNode;
}
```

Caller:

```
c
badInsert(head);
```

Does caller's head change?

No.

Because local `head` is only a copy.

~~0-Must~~ Must return new head or use `Node **`.

67. CORRECT RETURN STYLE

```
c
Node *insertHead(Node *head, int value)
{
    Node *newNode = createNode(value);
```

```

newNode->next = head;

return newNode;
}

```

Caller:

```

c
head = insertHead(head, 5);

```

68. CORRECT DOUBLE-POINTER STYLE

```

c
void insertHead(Node **head, int value)
{
    Node *newNode = createNode(value);

    newNode->next = *head;
    *head = newNode;
}

```

Caller:

```

c
insertHead(&head, 5);

```

Again:

```

TEXT
Node *      !'  node address
Node **    !'  address of head pointer

```

69. GATE TRAP SHEET

Trap 1

Insert after known node:

```

c
new->next = p->next;
p->next = new;

```

Order matters.

Trap 2

Changing local `p` does not change predecessor's link.

Trap 3

Changing local function `head` does not change caller head.

Trap 4

End insertion isn't always $O(1)$.

Trap 5

Position insertion includes traversal cost.

Trap 6

If tail maintained and you append, update tail.

Trap 7

Allocation alone doesn't insert a node.

Trap 8

Don't lose old successor before connecting new node to it.

70. MASTER INSERTION RULES

Insert at head

```
C
new->next = head;
head = new;
```

Insert after p

```
C
new->next = p->next;
p->next = new;
```

Append with tail

```
C
tail->next = new;
tail = new;
```

✓ These three patterns solve most singly-list insertion questions.

71. MEMORY TRICKS

Head

NEW points OLD HEAD, HEAD points NEW

Middle

NEW catches NEXT before PREV lets go

Tail

OLD TAIL points NEW, NEW becomes TAIL

Function head modification

Change caller's pointer? Return it or use ``.**

72. NOTEBOOK NOTES

Insert Head

```
C
new->next = head;
head = new;
```

TEXT
 $O(1)$

Insert After Known Node

```
C
new->next = p->next;
p->next = new;
```

TEXT
 $O(1)$

Insert End — only head

TEXT
Traverse + link
= $O(n)$

Insert End — tail

TEXT
 $O(1)$

Insert At Position

TEXT
Find predecessor $O(n)$
Link $O(1)$
Total $O(n)$

73. 30-SECOND REVISION

Remember:

1. Linked-list insertion changes links, not shifting values.
 2. Insert head: `new->next=head; head=new`.
 3. Insert after: `new->next=p->next; p->next=new`.
 4. Preserve old successor before overwriting.
 5. Head insertion is $O(1)$.
 6. After-known-node insertion is $O(1)$.
 7. End insertion with only head is $O(n)$.
 8. Tail can reduce append to $O(1)$.
 9. Positional insertion usually includes traversal.
 10. To modify caller's head: return new head or use `Node **`.
-

74. ACTIVE RECALL

Q1

What are the two pointer changes for insertion at head?

Q2

What is the correct order for insertion after known node `p`?

Q3

Why can inserting at position `k` be $O(n)$?

Q4

How does tail improve append complexity?

Q5

Why doesn't changing a local `Node *head` automatically change caller's head?

ANSWERS

A1

```
C
newNode->next = head;
head = newNode;
```

A2

```
C
newNode->next = p->next;
p->next = newNode;
```

A3

Because we may need to traverse from head to find the predecessor.

A4

It removes the need to traverse to the last node.

A5

C passes the pointer value by value.

75. PRACTICE — EASY

Q1

Before:

TEXT

h e a d ! ' 2 0 ! ' 3 0 ! ' N U L L

Insert 10 at head.

Write:

- pointer changes
- final list

Q2

Before:

TEXT

1 0 ! ' 2 0 ! ' 4 0 ! ' N U L L
! '
p

Insert 30 after p.

Write the exact two assignments.

76. PRACTICE — MODERATE

Q3

List:

TEXT

1 0 ! ' 2 0 ! ' 3 0 ! ' N U L L

Insert 15 at zero-based index 1.

Which node must temp point to before link updates?

Q4

List has `n` nodes and only `head`.

What is worst-case time to insert at end?

Why?

Q5

Same list maintains `tail`.

What does append become?

Which two pointer updates are required?

77. PRACTICE — TRICKY

Q6

Before:

```
TEXT
1 0 !'  2 0 !'  3 0 !'  N U L L
      !'
      p
```

You create `q`.

Then:

```
C
p->next = q;
q->next = p->next;
```

What structural error occurs?

Q7

Why does:

```
C
p = newNode;
```

not automatically insert `newNode` before node `p` in the actual list?

Q8

A list maintains `head` and `tail`.

It currently contains one node.

What relationship is true between `head` and `tail`?

78. GATE-STYLE POINTER QUESTION

Initially:

TEXT

h e a d ! ' 1 ! ' 2 ! ' 3 ! ' N U L L

p points to node 2.

A new node q stores 5.

Execute:

C

```
q->next = p->next;  
p->next = q;
```

Final list?

Answer:

TEXT

1 ! ' 2 ! ' 5 ! ' 3 ! ' N U L L

79. PLACEMENT MINI-CHALLENGE

You are given:

TEXT

1 0 ! ' 2 0 ! ' 3 0 ! ' 4 0 ! ' N U L L

and a direct pointer to node 20.

For each, state complexity:

1. Insert after node 20.
2. Insert at head.
3. Insert before node 20 if predecessor is not given.
4. Append if tail is not maintained.
5. Append if tail is maintained.

Answers

TEXT

1. $O(1)$
2. $O(1)$
3. $O(n)$ generally
4. $O(n)$
5. $O(1)$

PROGRESS

MODULE 3 — LINKED LISTS

' Lesson 29 — Intuition, Nodes & Memory ' Lesson 30 — Singly Linked List Fundamentals ' **Lesson 31 — Insertion**

NEXT — LESSON 32

Deletion in Singly Linked List — Head, End, Position & By Value

Next is another major pointer-safety lesson.

We will cover:

- delete first node
- delete last node
- delete at position
- delete by value
- delete after known predecessor
- empty-list case
- single-node case
- head/tail maintenance
- memory leaks
- dangling pointers
- use-after-free
- why deletion needs **SAVE !' RELINK !' FREE**

Example:

```
TEXT
BEFORE
head !' 1 0 !' 2 0 !' 3 0 !' N U L L
      !'
      delete
```

Safe logic:

```
C
temp = head;
head = head->next;
free(temp);
```

After:

```
TEXT
head !' 2 0 !' 3 0 !' N U L L
```

We will use:

BEFORE !' SAVE !' RELINK !' FREE !' AFTER

USER

next

ASSISTANT

MODULE 3 — LESSON 32

Deletion in Singly Linked List — Head, End, Position & By Value

Deletion is more dangerous than insertion because after removing a node we also have to manage its memory correctly.

Our permanent rule:

BEFORE !' SAVE !' RELINK !' FREE !' AFTER

GATE importance: Ø=Ý%ØMÚ%ØKØW **Placement importance:** Ø=Ý%ØMÚ%ØKØW

1. CORE IDEA

Suppose:

TEXT

h e a d !' 1 0 !' 2 0 !' 3 0 !' N U L L

We want to delete 20 .

We cannot simply erase the number 20 .

We must make node 10 bypass node 20 :

TEXT

1 0 % % % % % % % !' 3 0

and then release node 20 .

Final:

TEXT

h e a d !' 1 0 !' 3 0 !' N U L L

2. WHY DELETION IS TRICKY

Every node contains the address of the next node.

If you free or overwrite a pointer too early, you can:

- lose part of the list
- create dangling pointers

- cause memory leaks
- perform use-after-free
- crash the program

So pointer order is critical.

3. DELETE FIRST NODE Ø=Ý%

Before:

```
TEXT
head
! "
1 0 !' 2 0 !' 3 0 !' N U L L
```

We want to delete node 10.

4. SAFE STEPS

BEFORE

```
TEXT
h e a d !' 1 0 !' 2 0 !' 3 0 !' N U L L
```

SAVE

Save old head:

```
C
Node *temp = head;
```

Now:

```
TEXT
temp
! "
1 0 !' 2 0 !' 3 0
!'
head
```

RELINK

Move head:

```
C
head = head->next;
```

Now:

TEXT

```
t e m p ! ' 1 0 ! ' 2 0 ! ' 3 0
                ! '
            head
```

FREE

```
C
free(temp);
```

AFTER

TEXT

```
h e a d ! ' 2 0 ! ' 3 0 ! ' N U L L
```

Correct pattern.

5. DELETE-HEAD FUNCTION

```
C
Node *deleteHead(Node *head)
{
    if (head == NULL)
        return NULL;

    Node *temp = head;
    head = head->next;

    free(temp);

    return head;
}
```

Caller:

```
C
head = deleteHead(head);
```

6. WHY EMPTY-LIST CHECK?

If:

```
TEXT
head == NULL
```

then:

```
C
head->next
```

would dereference NULL.

So always handle:

```
C
if (head == NULL)
```

first.

7. COMPLEXITY

Delete head requires:

```
TEXT
constant pointer changes
```

Therefore:

```
TEXT
Time = O(1)
Auxiliary Space = O(1)
```

8. SINGLE-NODE LIST

Suppose:

```
TEXT
head !' 1 0 !' N U L L
```

Delete head.

```
C
temp = head;
head = head->next;
```

Since:

```
TEXT
head->next == NULL
```

new head becomes:

```
TEXT
NULL
```

Then:

```
c
free(temp);
```

Final:

```
TEXT
h e a d ! ' N U L L
```

So the same delete-head logic handles a one-node list naturally.

9. DELETE LAST NODE

Before:

```
TEXT
h e a d ! ' 1 0 ! ' 2 0 ! ' 3 0 ! ' N U L L
                delete
```

Problem:

To delete node `30`, we need node `20`.

Why?

Because after deletion:

```
TEXT
20->next
```

must become:

```
TEXT
NULL
```

10. WHY TAIL ALONE IS NOT ENOUGH

Even if:

```
TEXT
t a i l ! ' 3 0
```

we still need:

```
TEXT
node 20
```

to do:


```
C
20->next = NULL;
```

A singly linked list does not provide a backward pointer.

Therefore predecessor must usually be found by traversal.

Worst case:

```
TEXT
O(n)
```

11. DELETE LAST — TWO POINTER APPROACH

Use:

```
TEXT
prev
curr
```

Initially:

```
TEXT
prev = NULL
curr = head
```

Traverse until:

```
TEXT
curr->next == NULL
```

Then:

```
TEXT
curr = last node
prev = second-last node
```

12. VISUALIZATION

List:

```
TEXT
1 0 !'  2 0 !'  3 0 !'  N U L L
```

Eventually:

```
TEXT
prev
!"
2 0 !'  3 0 !'  N U L L
```

```
curr->next = !'
```

Then:

```
c
prev->next = NULL;
free(curr);
```

Final:

```
TEXT
1 0  !'  2 0  !'  N U L L
```

13. DELETE-LAST FUNCTION

```
c
Node *deleteLast(Node *head)
{
    if (head == NULL)
        return NULL;

    if (head->next == NULL)
    {
        free(head);
        return NULL;
    }

    Node *prev = NULL;
    Node *curr = head;

    while (curr->next != NULL)
    {
        prev = curr;
        curr = curr->next;
    }

    prev->next = NULL;
    free(curr);

    return head;
}
```

14. WHY SINGLE NODE NEEDS SPECIAL CASE?

For:

```
TEXT
h e a d  !'  1 0  !'  N U L L
```

there is no predecessor.

If we tried using:

```
C
prev->next = NULL;
```

then `prev` would still be `NULL`.

Invalid.

So:

```
C
if (head->next == NULL)
```

must be handled separately.

15. DELETE LAST COMPLEXITY

Need traversal to the end:

```
TEXT
O(n)
```

Freeing and link change:

```
TEXT
O(1)
```

Total:

```
TEXT
O(n)
```

16. DELETE AFTER A KNOWN PREDECESSOR Ø=Ý%

Suppose:

```
TEXT
1 0 !' 2 0 !' 3 0 !' 4 0 !' N U L L
      !'
      p
```

We want to delete the node after `p`.

That means delete:

```
TEXT
30
```

If `p` is already known, this can be done in:

TEXT

0(1)

17. BEFORE !' SAVE !' RELINK !' FREE

BEFORE

TEXT

```
p
!"
2 0  !'  3 0  !'  4 0
```

SAVE NODE TO DELETE

C

```
Node *temp = p->next;
```

So:

TEXT

```
t e m p  !'  3 0
```

RELINK

C

```
p->next = temp->next;
```

Now:

TEXT

```
2 0  % % % % !'  4 0
```

FREE

C

```
free(temp);
```

AFTER

TEXT

```
1 0  !'  2 0  !'  4 0  !'  N U L L
```

Essential deletion pattern.

18. FUNCTION

```
c
int deleteAfter(Node *p)
{
    if (p == NULL || p->next == NULL)
        return 0;

    Node *temp = p->next;

    p->next = temp->next;

    free(temp);

    return 1;
}
```

19. WHY CHECK `p->next == NULL` ?

If:

```
TEXT
p != last node
```

then:

```
TEXT
p->next == NULL
```

There is no node after it.

So nothing can be deleted.

20. MEMORY TRICK

For deleting after predecessor:

SAVE VICTIM !' SKIP VICTIM !' FREE VICTIM

```
c
temp = p->next;
p->next = temp->next;
free(temp);
```

Ø-Memorize.

21. DELETE AT A GIVEN INDEX

Suppose zero-based indices:

TEXT

Index:

0	1	2	3
1 0	!'	2 0	!'
3 0	!'	4 0	

Delete index:

TEXT

2

Need to delete:

TEXT

30

We need its predecessor:

TEXT

i n d e x 1 !' n o d e 2 0

22. LOGIC

If:

TEXT

pos == 0

delete head.

Otherwise reach:

TEXT

pos - 1

Then use delete-after logic.

23. FUNCTION — DELETE AT INDEX

C

```
Node *deleteAt(Node *head, int pos)
{
    if (head == NULL || pos < 0)
        return head;

    if (pos == 0)
    {
        Node *temp = head;
        head = head->next;
        free(temp);
        return head;
    }
}
```

```

Node *prev = head;

for (int i = 0; i < pos - 1 && prev != NULL; i++)
{
    prev = prev->next;
}

if (prev == NULL || prev->next == NULL)
    return head;

Node *temp = prev->next;

prev->next = temp->next;

free(temp);

return head;
}

```

24. TRACE DELETE INDEX 2

Before:

```

TEXT
1 0  !'  2 0  !'  3 0  !'  4 0  !'  N U L L

```

pos = 2.

Find:

```

TEXT
p r e v  !'  2 0

```

Then:

```

c
temp = prev->next;

```

So:

```

TEXT
t e m p  !'  3 0

```

Then:

```

c
prev->next = temp->next;

```

becomes:

```

TEXT
2 0  !'  4 0

```

Then:

```
c
free(temp);
```

Final:

```
TEXT
1 0 !' 2 0 !' 4 0 !' N U L L
```

25. COMPLEXITY — POSITION DELETION

Finding predecessor:

```
TEXT
O(n)
```

Link change + free:

```
TEXT
O(1)
```

Total worst case:

```
TEXT
O(n)
```

26. DELETE BY VALUE

Suppose:

```
TEXT
1 0 !' 2 0 !' 3 0 !' 4 0 !' N U L L
```

Delete:

```
TEXT
key = 30
```

We don't know where it is.

Need to search.

While searching, keep:

```
TEXT
prev
curr
```


27. TWO-POINTER SEARCH

Initially:

```
TEXT
prev = NULL
curr = head
```

Move until:

```
TEXT
curr->data == key
```

or:

```
TEXT
curr == NULL
```

28. TRACE

List:

```
TEXT
1 0 !' 2 0 !' 3 0 !' 4 0
```

Search 30.

First:

```
TEXT
prev = NULL
curr = 10
```

Move:

```
TEXT
prev = 10
curr = 20
```

Move:

```
TEXT
prev = 20
curr = 30
```

Found.

Now:

```
TEXT
p r e v !' 2 0
c u r r !' 3 0
```

Relink:

```
c
prev->next = curr->next;
```

Then:

```
c
free(curr);
```

29. DELETE-BY-VALUE FUNCTION

```
c
Node *deleteValue(Node *head, int key)
{
    if (head == NULL)
        return NULL;

    if (head->data == key)
    {
        Node *temp = head;
        head = head->next;
        free(temp);
        return head;
    }

    Node *prev = head;
    Node *curr = head->next;

    while (curr != NULL && curr->data != key)
    {
        prev = curr;
        curr = curr->next;
    }

    if (curr == NULL)
        return head;

    prev->next = curr->next;
    free(curr);

    return head;
}
```

30. WHY HEAD IS A SPECIAL CASE?

If the value to delete is in first node:

```
TEXT
head !' 1 0
```

there is no predecessor.

So ordinary:

```
c
prev->next = ...
```

cannot be used.

We must update:

```
TEXT
head
```

directly.

~~0=first~~ node frequently needs special handling in singly linked lists.

31. DELETE FIRST OCCURRENCE VS ALL OCCURRENCES

Suppose:

```
TEXT
1 0 !' 2 0 !' 2 0 !' 3 0
```

If requirement says:

Delete value 20.

Usually ambiguous.

Could mean:

First occurrence

```
TEXT
1 0 !' 2 0 !' 3 0
```

or:

All occurrences

```
TEXT
1 0 !' 3 0
```

~~0=first~~ Always read wording carefully.

32. DELETE ALL OCCURRENCES — INTUITION

For:

```
TEXT
2 0 !' 2 0 !' 1 0 !' 2 0 !' 3 0
```

the head itself may need to change repeatedly.

A common strategy:

1. remove matching nodes from beginning
2. then traverse remaining list
3. whenever next node matches, unlink and free it

This is a placement extension.

33. WHY FREE COMES AFTER RELINK

Suppose:

```
TEXT
2 0 !' 3 0 !' 4 0
      !'
temp
```

Wrong:

```
C
free(temp);
prev->next = temp->next;
```

After `free(temp)`:

```
TEXT
temp
```

is dangling.

Accessing:

```
C
temp->next
```

is use-after-free.

Undefined behavior.

Ø=Critical.

34. CORRECT ORDER

```
C
prev->next = temp->next;
free(temp);
```

or save successor first:

```
C
Node *next = temp->next;
free(temp);
```

```
prev->next = next;
```

But generally relinking before free is clearer.

35. MEMORY TRICK

Don't ask a dead node where it points.

Once:

```
c
free(node);
```

you cannot safely read:

```
c
node->next
```

or:

```
c
node->data
```

Excellent linked-list safety rule.

36. DANGLING POINTER AFTER DELETION

Suppose:

```
c
Node *p = curr;
```

Then:

```
c
free(curr);
```

Now both:

```
TEXT
curr
p
```

may still contain the old numerical address.

But the node no longer exists as a live allocated object.

So both are dangling.

Setting:

```
c
curr = NULL;
```

does not automatically fix:

```
c
p
```

Recall aliasing from pointer lessons.

37. DOUBLE FREE

Wrong:

```
c
free(temp);
free(temp);
```

Second free is invalid.

Likewise, if two aliases point to same node:

```
c
Node *p = temp;
```

then:

```
c
free(temp);
free(p);
```

is also a double free.

Memory safety.

38. MEMORY LEAK

Suppose:

```
TEXT
1 0 !' 2 0 !' 3 0
```

and you do:

```
c
head->next = head->next->next;
```

This makes:

TEXT

1 0 !' 3 0

But what happened to node 20?

If you did not save its address and `free` it:

TEXT

20

becomes unreachable.

That's a:

Memory Leak

39. SAFE VERSION

C

```
Node *temp = head->next;
```

```
head->next = temp->next;
```

```
free(temp);
```

Now node 20 is correctly removed and released.

40. DELETE HEAD WITH TAIL POINTER

Suppose we maintain:

TEXT

head

tail

If list has more than one node:

TEXT

```
h e a d !' 1 0 !' 2 0 !' 3 0  
tail
```

Deleting head changes only:

TEXT

head

Tail stays.

41. SINGLE-NODE HEAD+TAIL CASE

Before:

```
TEXT
head
!''
1 0 !' NULL
!''
tail
```

Delete node.

After:

```
TEXT
head = NULL
tail = NULL
```

Ø=If list becomes empty, both pointers must be updated.

42. DELETE LAST WITH HEAD + TAIL

Suppose:

```
TEXT
head !' 1 0 !' 2 0 !' 3 0 !' NULL
!''
tail
```

Need predecessor:

```
TEXT
20
```

After traversal:

```
C
free(tail);
tail = prev;
tail->next = NULL;
```

But be careful about using freed pointer.

Better logical order:

```
C
Node *oldTail = tail;
tail = prev;
tail->next = NULL;
free(oldTail);
```

For one-node list:

TEXT

```
head = tail = NULL
```

43. DELETE LAST STILL $O(n)$

Even with tail:

TEXT

$O(n)$

because singly linked list cannot directly move backward to predecessor.

This is one of the strongest motivations for:

Doubly Linked List

44. DELETION AFTER KNOWN PREDECESSOR

If:

TEXT

```
p != predecessor
```

is given:

C

```
temp = p->next;  
p->next = temp->next;  
free(temp);
```

Time:

TEXT

$O(1)$

Important complexity distinction.

45. DELETE KNOWN NODE WITH PREDECESSOR

If both:

TEXT

```
prev  
curr
```

are known:

```
c
prev->next = curr->next;
free(curr);
```

Again:

```
TEXT
0(1)
```

The expensive part is often finding the node/predecessor.

46. CAN WE DELETE A GIVEN NODE IN $O(1)$ WITHOUT PREDECESSOR?

Interesting placement question.

Suppose singly linked list:

```
TEXT
1 0 !'  2 0 !'  3 0 !'  4 0
      !'
      p
```

You are given only pointer to node 20.

One trick for a **non-tail node**:

1. copy next node's data into current node
2. delete next node

Conceptually:

```
TEXT
1 0 !'  2 0 !'  3 0 !'  4 0
```

Copy 30 into node currently storing 20:

```
TEXT
1 0 !'  3 0 !'  3 0 !'  4 0
```

Then delete the second 30.

Final logical values:

```
TEXT
1 0 !'  3 0 !'  4 0
```

47. $O(1)$ DELETE-GIVEN-NODE TRICK

```
c
void deleteGivenNode(Node *p)
{
```

```

    if (p == NULL || p->next == NULL)
        return;

    Node *temp = p->next;

    p->data = temp->data;
    p->next = temp->next;

    free(temp);
}

```

Time:

TEXT
O(1)

Ø=Placement extension.

48. IMPORTANT LIMITATION

This trick does **not** work for the tail node.

Why?

Tail has:

TEXT
p->next == NULL

There is no next node whose data/link can be copied.

Ø=Interview favorite.

49. DEEPER LIMITATION

This trick technically does not delete the exact node object `p`.

It keeps that node and changes its data, while deleting its successor.

So it may be inappropriate when:

- node identity matters
- external pointers reference specific nodes
- data cannot simply be copied
- object semantics are more complex

For textbook coding interviews, it is a common trick, but understand what really happens.

50. DELETE COMPLEXITY TABLE

Assuming only `head` unless specified:

Operation	Time
Delete head	$O(1)$
Delete after known predecessor	$O(1)$
Delete known non-tail node using copy trick	$O(1)$ special case
Delete by index	$O(n)$ worst
Delete by value	$O(n)$
Delete last	$O(n)$
Delete last with tail in singly list	still $O(n)$
Search + delete	$O(n)$

0-0% Copy this table.

51. ARRAY VS LINKED-LIST DELETION

Array:

TEXT
10 20 30 40

Delete 20 :

Need shifting:

TEXT
3 0 !•
4 0 !•

Worst:

TEXT
 $O(n)$

Linked list:

If predecessor already known:

TEXT
1 0 !' 2 0 !' 3 0

we can bypass node with:

TEXT
 $O(1)$

link changes.

But finding predecessor may cost:

TEXT

$O(n)$

52. DELETE AT BEGINNING — ARRAY VS LIST

Array

Shift remaining $n-1$ elements:

TEXT

$O(n)$

Singly linked list

```
c
temp = head;
head = head->next;
free(temp);
```

TEXT

$O(1)$

$O(1)$ for linked-list strength.

53. DELETE AT END — ARRAY VS SINGLY LIST

Interesting trade-off:

Array with logical size known

Remove last:

TEXT

$O(1)$

Singly linked list with only head

Need predecessor:

TEXT

$O(n)$

So array is actually better here.

$O(1)$ linked lists are not universally faster at deletion.

54. POINTER TRACE EXAMPLE

Initial:

TEXT

h e a d ! ' 1 ! ' 2 ! ' 3 ! ' N U L L

Let:

C

```
Node *p = head;
Node *q = p->next;
```

So:

TEXT

p ! ' 1
q ! ' 2

Execute:

C

```
p->next = q->next;
free(q);
```

Final list:

TEXT

1 ! ' 3 ! ' N U L L

Node 2 deleted.

55. POINTER TRACE WITH ALIAS

Suppose:

C

```
Node *q = head->next;
Node *r = q;
```

Then:

C

```
free(q);
q = NULL;
```

What about **r**?

r is:

TEXT

dangling

because it still contains the freed node's old address.

~~0=Set~~ Setting one alias to NULL does not update others.

56. WRONG CODE — MEMORY LEAK

```
C
head = head->next;
```

when removing first node, without freeing old head.

Logical list changes correctly, but the removed dynamic node leaks.

Need:

```
C
Node *temp = head;
head = head->next;
free(temp);
```

57. WRONG CODE — USE AFTER FREE

```
C
free(head);
head = head->next;
```

Wrong.

Need `head->next` before freeing.

Correct:

```
C
Node *temp = head;
head = head->next;
free(temp);
```

58. WRONG CODE — LOST REST OF LIST

Suppose:

```
TEXT
p r e v ! '   2 0 ! '   3 0 ! '   4 0
```

You want delete 30.

If:

```
C
prev->next = NULL;
```

then you disconnect:

TEXT

3 0 ! ' 4 0

from the list.

Unless saved elsewhere, nodes 30 and 40 become unreachable.

Correct:

C

```
prev->next = temp->next;
```

which preserves node 40.

59. EDGE CASE CHECKLIST

Before deletion, ask:

1. Is list empty?
2. Is target head?
3. Is target tail?
4. Is there only one node?
5. Do I have predecessor?
6. Do I need to update head?
7. Do I need to update tail?
8. Have I saved all necessary addresses?
9. Did I free exactly once?

Ø=Excellent placement checklist.

60. GATE TRAP — LAST NODE VS NULL

Remember:

Last node:

TEXT

```
curr != NULL  
curr->next == NULL
```

After moving beyond last:

TEXT

```
curr == NULL
```

Do not confuse them.

61. GATE TRAP — DELETE AFTER p

If:

```
TEXT
p->next == NULL
```

there is nothing after `p`.

So:

```
C
Node *temp = p->next;
```

would give:

```
TEXT
NULL
```

and dereferencing:

```
C
temp->next
```

would be invalid.

62. GATE TRAP — `free()` DOES NOT SET POINTER TO NULL

After:

```
C
free(temp);
```

`temp` is not automatically:

```
TEXT
NULL
```

It becomes a dangling pointer.

You may explicitly do:

```
C
temp = NULL;
```

if useful.

63. GATE TRAP — FREEING NON-HEAP NODE

Suppose:

```
C
Node x;
```

automatic local object.

You must not:

```
C
free(&x);
```

`free` is only for suitable dynamically allocated memory returned by allocation functions.

64. GATE TRAP — FREEING INTERIOR POINTER

If:

```
C
Node *p = malloc(...);
```

only the allocation pointer value corresponding to the allocated block may be appropriately freed.

Do not perform arbitrary:

```
C
free(p + 1);
```

Same dynamic-allocation rule from Lesson 16.

65. GATE TRAP — HEAD MODIFICATION IN FUNCTION

Deleting head changes caller's head.

So function must either:

Return new head

```
C
head = deleteHead(head);
```

or:

Take Node **

```
C
deleteHead(&head);
```

Same principle as insertion.

66. DOUBLE-POINTER DELETE HEAD

```

C
void deleteHead(Node **head)
{
    if (head == NULL || *head == NULL)
        return;

    Node *temp = *head;

    *head = (*head)->next;

    free(temp);
}

```

Call:

```

C
deleteHead(&head);

```

67. WHY CHECK BOTH?

```

C
head == NULL

```

means function itself did not receive a valid pointer-to-head variable.

```

C
*head == NULL

```

means actual list is empty.

Ø=Øbod C robustness detail.

68. INTERVIEW QUESTIONS

Q1. Delete-head complexity?

```

TEXT
O(1)

```

Q2. Delete-last complexity in singly linked list?

```

TEXT
O(n)

```

Q3. Why must predecessor be known?

To reconnect predecessor directly to successor.

Q4. Why free after relinking?

Because reading a freed node's fields is invalid.

Q5. Can a given non-tail node be deleted in $O(1)$ without predecessor?

Using the data-copy successor trick, yes in the standard interview formulation.

Q6. Why not tail?

There is no successor to copy.

69. MEMORY TRICKS

Delete head

SAVE HEAD !' MOVE HEAD !' FREE OLD

Delete middle

SAVE VICTIM !' SKIP VICTIM !' FREE

General safety

Never ask a dead node for NEXT.

Singly list tail deletion

To remove LAST, find SECOND-LAST.

70. NOTEBOOK NOTES

Delete Head

```
C
temp = head;
head = head->next;
free(temp);
```

TEXT
 $O(1)$

Delete After Known Node

```
C
temp = p->next;
p->next = temp->next;
free(temp);
```

TEXT

$O(1)$

Delete at Position

TEXT

Find predecessor $O(n)$

Relink/free $O(1)$

Total $O(n)$

Delete Last

TEXT

Find predecessor $O(n)$

Total $O(n)$

71. 30-SECOND REVISION

Remember:

1. Deletion means bypassing a node and releasing memory.
2. Delete head = $O(1)$.
3. Delete last in singly list = $O(n)$.
4. Delete after known predecessor = $O(1)$.
5. Save victim pointer before changing links.
6. Relink before freeing.
7. Never access fields after `free`.
8. `free` does not set pointer to NULL.
9. Head/tail metadata must remain consistent.
10. Single-node and empty lists are critical edge cases.

72. ACTIVE RECALL

Q1

What is the safe three-step pattern to delete head?

Q2

To delete node after `p`, what assignments are required?

Q3

Why is delete-last $O(n)$ in singly linked list?

Q4

What happens if you free a node and then read `node->next` ?

Q5

What happens to aliases after one pointer to a node is freed?

ANSWERS

A1

TEXT

```
SAVE old head
MOVE head
FREE old head
```

A2

C

```
temp = p->next;
p->next = temp->next;
free(temp);
```

A3

We must find its predecessor by forward traversal.

A4

Use-after-free / undefined behavior.

A5

They become dangling unless separately updated.

73. PRACTICE — EASY

Q1

Before:

TEXT

```
h e a d ! ' 5 ! ' 1 0 ! ' 1 5 ! ' N U L L
```

Delete head.

Write:

- `temp`
- new `head`
- final list

Q2

Before:

TEXT

```
1 0 !' 2 0 !' 3 0 !' N U L L
      !'
      p
```

Delete the node after `p`.

What node gets deleted?

Write the exact three operations.

74. PRACTICE — MODERATE

Q3

Delete zero-based index 2 from:

TEXT

```
1 !' 2 !' 3 !' 4 !' N U L L
```

What node must `prev` point to?

What does final list become?

Q4

Why is deletion of the first node $O(1)$ but deletion of the last node $O(n)$ in a singly linked list?

Q5

A list has one node and maintains both `head` and `tail`.

What must both pointers become after deletion?

75. PRACTICE — GATE TRAPS

Q6

Why is this wrong?

C

```
free(temp);
prev->next = temp->next;
```

Q7

Initially:

TEXT

```
1 !' 2 !' 3 !' N U L L
```

Execute:

C

```
Node *p = head;
Node *q = p->next;

p->next = q->next;
free(q);
```

What is the final list?

Q8

Suppose:

C

```
Node *q = head->next;
Node *r = q;

free(q);
q = NULL;
```

What is the state of `r`?

76. PLACEMENT CHALLENGE

You are given only a pointer `p` to this node:

TEXT

```
1 0  !'  2 0  !'  3 0  !'  4 0
      !'
      p
```

No head and no predecessor.

Can you logically remove value `20` in `0(1)`?

Explain the copy-successor trick and why it fails if `p` points to `40`.

PROGRESS

MODULE 3 — LINKED LISTS

' Lesson 29 — Intuition, Nodes & Memory ' Lesson 30 — Singly Linked List Fundamentals ' Lesson 31 — Insertion ' Lesson 32 — Deletion

NEXT — LESSON 33

Reversing a Singly Linked List — `prev`, `curr`, `next`

Next we learn the most famous linked-list pointer problem:

TEXT

BEFORE

```
head
!"
1 0 !' 2 0 !' 3 0 !' N U L L
```

Desired:

TEXT

```
head
!"
3 0 !' 2 0 !' 1 0 !' N U L L
```

The entire algorithm is built around three pointers:

```
c
prev
curr
next
```

and one critical safety rule:

SAVE NEXT BEFORE REVERSING THE ARROW

Core loop:

```
c
next = curr->next;
curr->next = prev;
prev = curr;
curr = next;
```

We will trace every pointer after every line, understand why wrong ordering loses the list, analyze $O(n)$ time / $O(1)$ space, and then compare iterative and recursive reversal.

USER

next

ASSISTANT

MODULE 3 — LESSON 33

Reversing a Singly Linked List — prev, curr, next

This is one of the **most important linked-list problems** for GATE, placements, and interviews.

If you understand this properly, many later problems become easier:

- reverse list
- reverse sublist

- palindrome linked list
- reverse in groups
- reorder list
- recursive reversal

GATE importance: 0=Ý%0MÚ%0KÍ0W **Placement importance:** 0=Ý%0MÚ%0KÍ0W

1. WHERE WE ARE

Completed:

' Linked-list basics ' Traversal ' Insertion ' Deletion

Now:

REVERSAL

Before:

```
TEXT
head
! "
1 0 !' 2 0 !' 3 0 !' N U L L
```

After:

```
TEXT
head
! "
3 0 !' 2 0 !' 1 0 !' N U L L
```

Notice:

We do not move node data.

We reverse the **links**.

2. STORY / INTUITION

Imagine people holding arrows pointing to the next person:

```
TEXT
1 0 !' 2 0 !' 3 0
```

To reverse the list, every arrow must point backward:

```
TEXT
1 0 !• 2 0 !• 3 0
```

Finally:

TEXT

3 0 !' 2 0 !' 1 0 !' N U L L

The challenge:

The moment you reverse one arrow, you might lose the address of the remaining list.

Therefore we need three pointers.

3. THE THREE POINTERS Ø=Ý%

Use:

C

```
Node *prev;  
Node *curr;  
Node *next;
```

Meaning:

TEXT

prev !' already reversed portion
curr !' node currently being processed
next !' saves remaining unreversed portion

Memory trick:

****PREV = past**

CURR = present NEXT = future**

4. INITIAL STATE

For:

TEXT

1 0 !' 2 0 !' 3 0 !' N U L L

initialize:

C

```
Node *prev = NULL;  
Node *curr = head;  
Node *next = NULL;
```

Visualization:

TEXT

```
prev
!"
NULL
```

```
curr
!"
1 0 !' 2 0 !' 3 0 !' N U L L
```

5. WHY `prev = NULL`?

Original first node:

TEXT

10

will become the **last node** after reversal.

Last node must point to:

TEXT

NULL

Therefore eventually:

C

```
10->next = NULL;
```

By initializing:

C

```
prev = NULL;
```

the first reversal naturally creates this.

Ø = ~~Empty~~ elegant design.

6. THE CORE FOUR LINES

Memorize the logic:

C

```
next = curr->next;
curr->next = prev;
prev = curr;
curr = next;
```

Repeat until:

```
c
curr == NULL
```

Then:

```
c
head = prev;
```

This is the complete iterative reversal algorithm.

7. WHY EXACTLY THIS ORDER?

Let's understand every line.

Line 1

```
c
next = curr->next;
```

Save the rest of the list.

Line 2

```
c
curr->next = prev;
```

Reverse current arrow.

Line 3

```
c
prev = curr;
```

Expand reversed portion.

Line 4

```
c
curr = next;
```

Move to remaining list.

Memory trick:

SAVE !' REVERSE !' ADVANCE PREV !' ADVANCE CURR

8. BEFORE FIRST ITERATION

```
TEXT
prev
!"
NULL

curr
!"
1 0 !' 2 0 !' 3 0 !' N U L L
```

9. STEP 1 — SAVE NEXT

```
c
next = curr->next;
```

Now:

```
TEXT
p r e v !' N U L L

c u r r !' 1 0 !' 2 0 !' 3 0 !' N U L L
              !'
              next
```

More precisely:

```
TEXT
c u r r !' 1 0
n e x t !' 2 0
```

We have safely remembered node 20.

10. STEP 2 — REVERSE CURRENT LINK

```
c
curr->next = prev;
```

Originally:

```
TEXT
1 0 !' 2 0
```

Now:

```
TEXT
1 0 !' N U L L
```

Visualization:

TEXT

```
p r e v ! '   N U L L

c u r r ! '   1 0 ! '   N U L L

n e x t ! '   2 0 ! '   3 0 ! '   N U L L
```

0=We did not lose node 20 because `next` saved it.

11. STEP 3 — MOVE PREV

```
c
prev = curr;
```

Now:

TEXT

```
prev
!"
1 0 ! '   N U L L

curr
!"
1 0 ! '   N U L L

next
!"
2 0 ! '   3 0 ! '   N U L L
```

`prev` now represents the reversed part.

12. STEP 4 — MOVE CURR

```
c
curr = next;
```

Now:

TEXT

```
prev
!"
1 0 ! '   N U L L

curr
!"
2 0 ! '   3 0 ! '   N U L L
```

Ready for next iteration.

13. SECOND ITERATION

Current state:

TEXT

```
p r e v ! '   1 0 ! '   N U L L
c u r r ! '   2 0 ! '   3 0 ! '   N U L L
```

First:

```
c
next = curr->next;
```

So:

TEXT

```
n e x t ! '   3 0
```

Then:

```
c
curr->next = prev;
```

Now:

TEXT

```
2 0 ! '   1 0 ! '   N U L L
```

Then:

```
c
prev = curr;
```

Now:

TEXT

```
p r e v ! '   2 0 ! '   1 0 ! '   N U L L
```

Then:

```
c
curr = next;
```

Now:

TEXT

```
c u r r ! '   3 0 ! '   N U L L
```

14. THIRD ITERATION

Before:

TEXT

```
p r e v ! '   2 0 ! '   1 0 ! '   N U L L
c u r r ! '   3 0 ! '   N U L L
```

Save:

```
c
next = curr->next;
```

So:

```
TEXT
next = NULL
```

Reverse:

```
c
curr->next = prev;
```

Now:

```
TEXT
3 0 ! '   2 0 ! '   1 0 ! '   N U L L
```

Move:

```
c
prev = curr;
```

So:

```
TEXT
p r e v ! '   3 0 ! '   2 0 ! '   1 0 ! '   N U L L
```

Move:

```
c
curr = next;
```

So:

```
TEXT
curr = NULL
```

Loop ends.

15. FINAL STEP

At this point:

TEXT

```
prev !' 3 0 !' 2 0 !' 1 0 !' NULL
curr = NULL
```

Therefore:

```
C
head = prev;
```

Final:

TEXT

```
head
!"
3 0 !' 2 0 !' 1 0 !' NULL
```

0% complete.

16. ITERATIVE C IMPLEMENTATION

```
C
Node *reverseList(Node *head)
{
    Node *prev = NULL;
    Node *curr = head;

    while (curr != NULL)
    {
        Node *next = curr->next;

        curr->next = prev;

        prev = curr;
        curr = next;
    }

    return prev;
}
```

Caller:

```
C
head = reverseList(head);
```

17. WHY RETURN prev?

When loop finishes:

TEXT

```
curr == NULL
```

but:

TEXT

prev

points to the old last node.

That node is now the new first node.

Therefore:

c

return prev;

18. TRACE TABLE Ø=Ý%

Original:

TEXT

1 0 !' 2 0 !' 3 0 !' N U L L

Iteration	prev	curr	next after save
Start	NULL	10	—
1	10	20	20
2	20	30	30
3	30	NULL	NULL

The reversed chains represented by prev become:

TEXT

NULL

1 0 !' N U L L

2 0 !' 1 0 !' N U L L

3 0 !' 2 0 !' 1 0 !' N U L L

19. MOST IMPORTANT SAFETY RULE

SAVE NEXT BEFORE REVERSING THE ARROW

Why?

Suppose:

TEXT

1 0 !' 2 0 !' 3 0

and `curr` points to 10.

If you immediately do:

```
c
curr->next = prev;
```

then:

```
TEXT
1 0 !' N U L L
```

Where is node 20 now?

If no other pointer saved it:

```
TEXT
2 0 !' 3 0
```

has been lost.

~~0=10~~ destroys access to the remaining list.

20. WRONG ORDER

Bad:

```
c
curr->next = prev;
curr = curr->next;
```

After first line:

```
TEXT
curr->next
```

now points backward.

So:

```
c
curr = curr->next;
```

moves backward instead of into the remaining list.

The unreversed nodes are lost.

21. MEMORY TRICK

Don't turn around until you remember where you were going.

Translation:

```
c
next = curr->next;
```

must happen before:

```
c
curr->next = prev;
```

22. COMPLEXITY

Each node is processed exactly once.

For n nodes:

```
TEXT
Time =  $\sim (n)$ 
```

Pointers used:

```
TEXT
prev
curr
next
```

constant number.

So:

```
TEXT
Auxiliary Space =  $\sim (1)$ 
```

~~$O(n)$~~ Iterative reversal is **in-place**.

23. WHY NOT $O(n^2)$?

Although multiple pointer operations happen for every node, each iteration does constant work.

So:

```
TEXT
n nodes  $\times O(1)$ 
=
 $O(n)$ 
```

24. EMPTY LIST

Suppose:

TEXT

```
head = NULL
```

Initialize:

TEXT

```
prev = NULL  
curr = NULL
```

Condition:

C

```
while (curr != NULL)
```

is immediately false.

Return:

TEXT

```
prev = NULL
```

Correct.

So the same function handles an empty list naturally.

25. SINGLE-NODE LIST

Before:

TEXT

```
head !' 10 !' NULL
```

Initialize:

TEXT

```
prev = NULL  
curr = 10
```

Save:

TEXT

```
next = NULL
```

Reverse:

TEXT

```
10->next = NULL
```

which is unchanged.

Then:

TEXT

```
prev = 10  
curr = NULL
```

Return 10.

Result:

TEXT

```
head !' 10 !' NULL
```

Correct.

26. TWO-NODE LIST

Before:

TEXT

```
10 !' 20 !' NULL
```

Iteration 1:

TEXT

```
10 !' NULL
```

```
curr !' 20  
prev !' 10
```

Iteration 2:

TEXT

```
20 !' 10 !' NULL
```

Return:

TEXT

```
20
```

Final:

TEXT

```
20 !' 10 !' NULL
```

27. DOES REVERSAL CHANGE NODE ADDRESSES?

No.

Nodes themselves stay wherever they are in memory.

Only their:

TEXT

next

fields change.

Before:

TEXT

Node A address 1000

Node B address 8000

Node C address 3000

Pointers:

TEXT

A	!	B
B	!	C
C	!	N U L L

After reversal:

TEXT

C	!	B
B	!	A
A	!	N U L L

Node addresses remain:

TEXT

1000, 8000, 3000

~~0-link~~ Linked-list reversal changes **links**, not physical storage.

28. DOES REVERSAL COPY DATA?

No.

We do not perform:

TEXT

swap data values

We alter:

TEXT

next pointers

This distinction can matter if nodes contain large objects or node identity matters.

29. REVERSING VALUES VS REVERSING LINKS

Suppose:

TEXT

1 0 !' 2 0 !' 3 0

One alternative could swap data:

TEXT

3 0 !' 2 0 !' 1 0

while leaving node links unchanged.

Values appear reversed, but node order hasn't actually changed.

In genuine linked-list reversal:

TEXT

links themselves reverse.

0=1%
0=1% interviewers usually expect link reversal.

30. RECURSIVE REVERSAL 0=1%

Now let's understand the famous recursive version.

For:

TEXT

1 !' 2 !' 3 !' N U L L

Idea:

Ask recursion to reverse everything after the first node.

So reverse:

TEXT

2 !' 3

into:

TEXT

3 !' 2

Then attach node 1 at the end:

TEXT

3 !' 2 !' 1

31. RECURSIVE CODE

```
C
Node *reverseRecursive(Node *head)
{
    if (head == NULL || head->next == NULL)
        return head;

    Node *newHead = reverseRecursive(head->next);

    head->next->next = head;
    head->next = NULL;

    return newHead;
}
```

0-7% This code is small but conceptually tricky.

32. BASE CASE

```
C
if (head == NULL || head->next == NULL)
    return head;
```

Two cases:

Empty list

```
TEXT
NULL
```

already reversed.

One-node list

```
TEXT
1 0 !' N U L L
```

already reversed.

33. RECURSIVE TRACE — 1 !' 2 !' 3

Call:

```
TEXT
reverse(1)
```

which calls:

```
TEXT
reverse(2)
```

which calls:

```
TEXT
reverse(3)
```

For node 3:

```
TEXT
3->next == NULL
```

so return:

```
TEXT
newHead = 3
```

34. UNWIND AT NODE 2

At this point conceptually:

```
TEXT
2  !'  3  !'  N U L L
```

We want:

```
TEXT
3  !'  2  !'  N U L L
```

Current:

```
TEXT
head = 2
head->next = 3
```

Execute:

```
C
head->next->next = head;
```

This means:

```
TEXT
3->next = 2
```

Now temporarily:

```
TEXT
2  !'  3
!'  !"
% % % %
```

a cycle exists.

Therefore immediately:

```
C
head->next = NULL;
```

which means:

```
TEXT
2->next = NULL
```

Result:

```
TEXT
3  !'  2  !'  N U L L
```

35. UNWIND AT NODE 1

Now recursion has produced:

```
TEXT
3  !'  2  !'  N U L L
```

but original node 1 still points to 2.

Execute:

```
C
head->next->next = head;
```

Here:

```
TEXT
head = 1
head->next = 2
```

So:

```
TEXT
2->next = 1
```

Then:

```
C
head->next = NULL;
```

gives:

```
TEXT
1->next = NULL
```

Final:

TEXT

3 !' 2 !' 1 !' N U L L

36. WHY `head->next->next = head` ?

Suppose original local pair:

TEXT

h e a d !' n e x t
1 !' 2

To reverse their relationship:

TEXT

2 !' 1

`head->next` is node 2.

So:

C

`head->next->next = head;`

means:

TEXT

`2->next = 1`

Read it step by step.

37. WHY `head->next = NULL` ?

If we don't clear the original link:

TEXT

1 !' 2

while also creating:

TEXT

2 !' 1

we create a cycle:

TEXT

1 !Ä 2

Therefore:

```
c
head->next = NULL;
```

breaks the old forward link.

38. RECURSIVE COMPLEXITY

Each node processed once:

```
TEXT
Time = ~ ( n )
```

But recursion depth:

```
TEXT
~ ( n )
```

So:

```
TEXT
Auxiliary Space = ~ ( n )
```

due to call stack.

39. ITERATIVE VS RECURSIVE REVERSAL

Version	Time	Auxiliary Space
Iterative	$O(n)$	$O(1)$
Recursive	$O(n)$	$O(n)$

For $n > 1000$ placements, iterative is often preferred because it uses constant auxiliary space.

Recursive is important for understanding pointer recursion.

40. REVERSE PRINTING VS ACTUAL REVERSAL

Recall:

```
c
void printReverse(Node *head)
{
    if (head == NULL)
        return;

    printReverse(head->next);
    printf("%d ", head->data);
}
```

This prints:

```
TEXT
30 20 10
```

but list remains:

```
TEXT
1 0 !' 2 0 !' 3 0
```

Actual reversal changes links to:

```
TEXT
3 0 !' 2 0 !' 1 0
```

Don't confuse.

41. TAIL AFTER REVERSAL

Before:

```
TEXT
h e a d !' 1 0 !' 2 0 !' 3 0 !' N U L L
           !'
           t a i l
```

After reversal:

```
TEXT
h e a d !' 3 0 !' 2 0 !' 1 0 !' N U L L
           !'
           t a i l
```

So:

```
TEXT
o l d   t a i l !'   n e w   h e a d
o l d   h e a d !'   n e w   t a i l
```

Important if maintaining tail pointer.

42. HEAD/TAIL UPDATE TRICK

If maintaining both:

```
TEXT
oldHead = head
```

Reverse list.

Then:

TEXT

```
tail = oldHead  
head = newHead
```

because old first node becomes new last node.

43. REVERSING TWICE

If:

TEXT

```
1 0 !' 2 0 !' 3 0
```

reverse once:

TEXT

```
3 0 !' 2 0 !' 1 0
```

reverse again:

TEXT

```
1 0 !' 2 0 !' 3 0
```

Therefore reversal is its own inverse.

Ø=Nice conceptual property.

44. GATE POINTER TRACE

Initial:

TEXT

```
1 !' 2 !' 3 !' N U L L
```

Suppose:

C

```
Node *p = NULL;  
Node *q = head;  
Node *r;
```

Loop executes:

C

```
r = q->next;  
q->next = p;  
p = q;  
q = r;
```

This is exactly reversal with:

TEXT

```
p = prev  
q = curr  
r = next
```

After completion:

TEXT

```
p != reversed list  
q = NULL
```

Therefore new head:

TEXT

```
p
```

Variable names may change; algorithm stays same.

45. GATE TRAP — WHICH POINTER IS NEW HEAD?

After loop:

TEXT

```
prev != old last node  
curr = NULL
```

Therefore new head is:

TEXT

```
prev
```

not:

TEXT

```
curr
```

because `curr` has moved beyond the list.

46. GATE TRAP — OLD HEAD

After reversal, original `head` pointer may still point to old first node unless you assign the return value.

Example:

C

```
reverseList(head);
```

without:

```
C
head = reverseList(head);
```

The list's links may be reversed, but caller's `head` still refers to old first node, which is now the tail.
You must capture new head.

47. EXAMPLE

Original:

```
TEXT
h e a d ! ' 1 ! ' 2 ! ' 3
```

After internal reversal:

```
TEXT
3 ! ' 2 ! ' 1 ! ' N U L L
```

But if caller's `head` still points to `1`:

```
TEXT
h e a d ! ' 1 ! ' N U L L
```

From that pointer, the rest of reversed list isn't reachable forward.

Hence:

```
C
head = reverseList(head);
```

48. GATE TRAP — WRONG INITIAL PREV

If:

```
C
Node *prev = head;
Node *curr = head;
```

instead of:

```
C
prev = NULL;
curr = head;
```

the logic is wrong and can create cycles/self-links.

Correct initialization:

TEXT

```
prev = NULL  
curr = head
```

~~O(1)~~ Memorize.

49. GATE TRAP — FAILURE TO CLEAR OLD HEAD LINK

Iterative algorithm naturally handles this because first iteration does:

```
c  
curr->next = prev;
```

where:

```
TEXT  
prev = NULL
```

Thus old head becomes:

```
TEXT  
oldHead->next = NULL
```

correct new tail.

50. GATE TRAP — RECURSIVE SPACE

Some may say:

```
TEXT  
Recursive reversal = O(1) extra space
```

because no explicit array is used.

Wrong.

Call stack consumes:

```
TEXT  
O(n)
```

auxiliary space.

~~O(1)~~ Recursion stack always matters in complexity unless optimization explicitly applies, and this recursive form has pending work anyway.

51. LOOP INVARIANT — DEEP UNDERSTANDING

During iterative reversal:

TEXT

prev

always points to:

correctly reversed prefix.

And:

TEXT

curr

always points to:

first node of the still-unreversed suffix.

Example midway:

TEXT

```
3 0 !• 2 0 !• 1 0          4 0 !' 5 0 !' N U L L
           !'              !'
prev?
```

Better directional representation:

TEXT

prev

!"

2 0 !' 1 0 !' N U L L

curr

!"

3 0 !' 4 0 !' 5 0 !' N U L L

At every iteration, one node moves from the **curr** side to the **prev** side.

$\emptyset = \text{True}$ invariant is useful for proving correctness.

52. CORRECTNESS INTUITION

Initially:

TEXT

reversed prefix = empty
unreversed suffix = entire list

Every iteration:

TEXT

moves current node from suffix to front of reversed prefix

At termination:

TEXT

unreversed suffix = empty
reversed prefix = entire list

Therefore list is reversed.

53. PARTIAL REVERSAL PREVIEW

Suppose:

TEXT

1 !' 2 !' 3 !' 4 !' 5

Reverse only:

TEXT

2 !' 3 !' 4

Desired:

TEXT

1 !' 4 !' 3 !' 2 !' 5

This uses the same:

TEXT

prev/curr/next

idea, plus reconnecting boundaries.

0=0 common placement problem later.

54. REVERSE FIRST K NODES — PREVIEW

Example:

TEXT

1 !' 2 !' 3 !' 4 !' 5

k=3

Result:

TEXT

3 !' 2 !' 1 !' 4 !' 5

Again, same reversal loop, but run exactly **k** iterations and reconnect.

55. REVERSE IN GROUPS — PREVIEW

Example:

TEXT

1 !' 2 !' 3 !' 4 !' 5 !' 6

k=2

Result:

TEXT

2 !' 1 !' 4 !' 3 !' 6 !' 5

This is a classic interview extension.

We'll revisit advanced linked-list patterns in Lesson 36.

56. REVERSAL AND PALINDROME CHECK

For:

TEXT

1 !' 2 !' 3 !' 2 !' 1

a common $O(1)$ -extra-space approach:

1. find middle
2. reverse second half
3. compare halves
4. optionally restore list

So today's reversal algorithm becomes a building block for palindrome checking.

57. REVERSAL AND REORDER LIST

Problem like:

TEXT

1 !' 2 !' 3 !' 4 !' 5

reorder into:

TEXT

1 !' 5 !' 2 !' 4 !' 3

often uses:

1. find middle
2. reverse second half
3. merge halves

Again reversal is a core building block.

58. GATE FOCUS

Ø=141 MUST KNOW:

- three-pointer reversal
 - initialization
 - exact four-line loop
 - why next must be saved first
 - new head is `prev`
 - iterative complexity
 - recursive reversal
 - recursive space
 - old head becomes new tail
 - pointer-order traps
-

59. MASTER ITERATIVE TEMPLATE

```
c
Node *reverseList(Node *head)
{
    Node *prev = NULL;
    Node *curr = head;

    while (curr != NULL)
    {
        Node *next = curr->next;

        curr->next = prev;

        prev = curr;
        curr = next;
    }

    return prev;
}
```

Ø=141 Learn this well enough to write without looking.

60. MASTER RECURSIVE TEMPLATE

```
c
Node *reverseRecursive(Node *head)
{
    if (head == NULL || head->next == NULL)
        return head;

    Node *newHead = reverseRecursive(head->next);

    head->next->next = head;
    head->next = NULL;
}
```

```
    return newHead;
}
```

Understand it rather than blindly memorizing.

61. COMPARISON TABLE

Property	Iterative	Recursive
Time	$O(n)$	$O(n)$
Extra space	$O(1)$	$O(n)$
Main idea	3 pointers	reverse rest, attach current
Stack overflow risk	No recursion	Possible for very long list
Placement preference	Usually stronger default	Common conceptual question

62. MEMORY TRICKS

Three pointers

PAST — PRESENT — FUTURE

TEXT

prev curr next

Four operations

SAVE !' TURN !' STEP PAST !' STEP PRESENT

More directly:

TEXT

SAVE next
REVERSE current link
MOVE prev
MOVE curr

Critical safety

Remember the road before turning the sign.

63. NOTEBOOK NOTES

Iterative reverse

```
C
prev = NULL;
curr = head;

while (curr != NULL)
{
    next = curr->next;
    curr->next = prev;
    prev = curr;
    curr = next;
}

head = prev;
```

Complexity

TEXT

Time = $O(n)$
Space = $O(1)$

Recursive

TEXT

Base:
NULL or one node

```
C
newHead = reverse(head->next);

head->next->next = head;
head->next = NULL;
```

TEXT

Time = $O(n)$
Space = $O(n)$

64. 30-SECOND REVISION

Remember:

1. Reversal changes links, not node addresses.
2. Use `prev`, `curr`, `next`.
3. Start `prev=NULL`, `curr=head`.
4. Save `next` first.
5. Reverse `curr->next`.
6. Move `prev`.
7. Move `curr`.

8. Stop when `curr==NULL` .
 9. New head is `prev` .
 10. Iterative reversal is $O(n)$ time, $O(1)$ extra space.
-

65. ACTIVE RECALL

Q1

Why must `next = curr->next` happen before reversing `curr->next` ?

Q2

What should `prev` initially contain?

Q3

At loop termination, which pointer is the new head?

Q4

What are iterative reversal's time and auxiliary-space complexities?

Q5

What does old head become after reversal?

ANSWERS

A1

Because otherwise we can lose the address of the remaining unreversed list.

A2

TEXT
NULL

A3

TEXT
prev

A4

TEXT
Time = $O(n)$
Space = $O(1)$

A5

The new tail.

66. PRACTICE — EASY

Q1

Reverse:

TEXT

1 !' 2 !' 3 !' N U L L

Write the final list.

Q2

At the beginning of iterative reversal:

TEXT

h e a d !' 1 0 !' 2 0 !' N U L L

What should these be?

TEXT

prev = ?

curr = ?

Q3

After first iteration on:

TEXT

1 0 !' 2 0 !' 3 0 !' N U L L

draw:

- prev
- curr
- links

67. PRACTICE — MODERATE

Q4

For:

TEXT

1 0 !' 2 0 !' 3 0 !' 4 0 !' N U L L

after exactly two iterations of reversal, what does:

TEXT

prev

point to?

What does:

TEXT

curr

point to?

Q5

Why is this wrong?

C

```
curr->next = prev;  
curr = curr->next;
```

Q6

Why must caller write:

C

```
head = reverseList(head);
```

instead of merely:

C

```
reverseList(head);
```

if function returns the new head?

68. PRACTICE — GATE POINTER TRACE

Initially:

TEXT

h e a d ! ' A ! ' B ! ' C ! ' N U L L

and:

C

```
Node *p = NULL;  
Node *q = head;  
Node *r;
```

One iteration executes:

```
c
r = q->next;
q->next = p;
p = q;
q = r;
```

After this iteration:

1. What does `p` point to?
2. What does `q` point to?
3. What does `A->next` contain?

ANSWERS

TEXT

```
p !' A
q !' B
A->next = NULL
```

69. PRACTICE — RECURSION

For:

TEXT

```
1 !' 2 !' 3 !' N U L L
```

during recursive unwind at node `2`, what does:

C

```
head->next->next = head;
```

do?

Answer:

TEXT

```
3->next = 2
```

Then:

C

```
head->next = NULL;
```

sets:

TEXT

```
2->next = NULL
```

producing:

TEXT

3 !' 2 !' N U L L

for that stage.

70. PLACEMENT CHALLENGE

Suppose:

TEXT

1 !' 2 !' 3 !' 4 !' 5 !' N U L L

After three iterations of the standard iterative reversal loop, draw both parts.

Answer:

TEXT

prev

!"

3 !' 2 !' 1 !' N U L L

curr

!"

4 !' 5 !' N U L L

0-1% If you can visualize this immediately, your reversal fundamentals are strong.

PROGRESS

MODULE 3 — LINKED LISTS

' Lesson 29 — Intuition, Nodes & Memory ' Lesson 30 — Singly Linked List Fundamentals ' Lesson 31 — Insertion ' Lesson 32 — Deletion ' **Lesson 33 — Reversal**

NEXT — LESSON 34

Doubly Linked Lists — prev, next, Bidirectional Traversal & O(1) Deletion with Known Node

Next we upgrade:

TEXT

1 0 !' 2 0 !' 3 0

into:

TEXT

N U L L !• 1 0 !Ä 2 0 !Ä 3 0 !' N U L L

Each node will contain:

```

c
struct DNode
{
    int data;
    struct DNode *prev;
    struct DNode *next;
};

```

We will study:

- why `prev` is useful
- forward and backward traversal
- insert at head/end
- insert before/after known node
- delete known node
- head/tail cases
- four-pointer rewiring
- singly vs doubly linked lists
- memory overhead
- GATE traps
- placement applications such as browser history and LRU-cache structure intuition.

USER

next

ASSISTANT

MODULE 3 — LESSON 34

Doubly Linked Lists — `prev`, `next`, Bidirectional Traversal & $O(1)$ Deletion with Known Node

We now upgrade from a singly linked list:

TEXT

1 0 !' 2 0 !' 3 0 !' N U L L

to a doubly linked list:

TEXT

N U L L !• 1 0 !Ä 2 0 !Ä 3 0 !' N U L L

Each node can move in **both directions**.

GATE importance: $\emptyset = \text{YES}$ **MUST KNOW** **Placement importance:** $\emptyset = \text{YES}$ **IMPORTANT**

1. WHY DOUBLY LINKED LIST?

In a singly linked list, each node knows only:

TEXT

next

So from node 30, we cannot directly move back to 20.

This causes problems like:

- deleting the last node
- inserting before a known node
- backward traversal

A doubly linked list solves this by storing:

TEXT

prev

next

2. NODE STRUCTURE

C

```
typedef struct DNode
{
    int data;
    struct DNode *prev;
    struct DNode *next;
} DNode;
```

Each node:

TEXT

```
+-----+-----+-----+
| prev | data | next |
+-----+-----+-----+
```

3. VISUALIZATION

TEXT

N U L L ! • [1 0] ! Ä [2 0] ! Ä [3 0] ! ' N U L L

For node 20:

TEXT

```
p r e v ! ' 1 0
data = 20
n e x t ! ' 3 0
```

So:


```
C
node20->prev
```

points to 10.

```
C
node20->next
```

points to 30.

4. HEAD AND TAIL

Usually maintain:

```
C
DNode *head;
DNode *tail;
```

Example:

```
TEXT
head          tail
!"
N U L L !• 1 0 !Ä 2 0 !Ä 3 0 !' N U L L !"
!"
```

Important invariants:

```
TEXT
head->prev == NULL
tail->next == NULL
```

For one node:

```
TEXT
head == tail
```

For empty list:

```
TEXT
head == NULL
tail == NULL
```

5. CREATE NODE

```
C
DNode *createDNode(int value)
{
    DNode *p = malloc(sizeof *p);

    if (p == NULL)
```

```

        return NULL;

    p->data = value;
    p->prev = NULL;
    p->next = NULL;

    return p;
}

```

6. FORWARD TRAVERSAL

Same basic idea as singly list:

```

c
void printForward(DNode *head)
{
    DNode *temp = head;

    while (temp != NULL)
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }
}

```

For:

```

TEXT
1 0 !Ä 2 0 !Ä 3 0

```

output:

```

TEXT
10 20 30

```

Time:

```

TEXT
O(n)

```

7. BACKWARD TRAVERSAL

This is the new advantage.

Start from:

```

TEXT
tail

```

and follow:

TEXT

prev

C

```
void printBackward(DNode *tail)
{
    DNode *temp = tail;

    while (temp != NULL)
    {
        printf("%d ", temp->data);
        temp = temp->prev;
    }
}
```

Output:

TEXT

30 20 10

Time:

TEXT

$O(n)$

8. KEY DIFFERENCE FROM SINGLY LIST

Singly:

TEXT

1 0 !' 2 0 !' 3 0

Can naturally move:

TEXT

forward only

Doubly:

TEXT

1 0 !Ä 2 0 !Ä 3 0

Can move:

TEXT

forward and backward

Ø=Min advantage.

9. INSERT AT HEAD

Before:

```
TEXT
head
!"
NULL !• 1 0 !Ä 2 0 !' NULL
```

New:

```
TEXT
5
```

Desired:

```
TEXT
NULL !• 5 !Ä 1 0 !Ä 2 0 !' NULL
head
```

10. POINTER CHANGES

Our rule:

BEFORE !' POINTER CHANGES !' AFTER

For non-empty list:

```
C
newNode->next = head;
head->prev = newNode;
head = newNode;
```

`newNode->prev` is already NULL.

11. VISUAL TRACE

Before:

```
TEXT
new !' [ 5 ]
head !' [ 1 0 ] !Ä [ 2 0 ]
```

First:

```
C
new->next = head;
```

Then:

TEXT

5 !' 1 0

Next:

C

```
head->prev = new;
```

Now:

TEXT

5 !Ä 1 0

Finally:

C

```
head = new;
```

Result:

TEXT

head

!"

N U L L !• 5 !Ä 1 0 !Ä 2 0 !' N U L L

12. EMPTY-LIST CASE

If:

TEXT

```
head == NULL
```

then first node must become both:

TEXT

head

tail

So:

C

```
head = newNode;
```

```
tail = newNode;
```

Result:

TEXT

head

!"

10

!'

```
tail
```

with:

```
TEXT
prev = NULL
next = NULL
```

13. INSERT AT END

With tail pointer, append is very easy.

Before:

```
TEXT
head      tail
! "
1 0  !Ä  2 0  !Ä  3 0  !'  N U L L      ! "
```

New:

```
TEXT
40
```

Pointer changes:

```
C
newNode->prev = tail;
tail->next = newNode;
tail = newNode;
```

Result:

```
TEXT
1 0  !Ä  2 0  !Ä  3 0  !Ä  4 0
                                ! '
                        tail
```

Time:

```
TEXT
O(1)
```

14. INSERT AFTER KNOWN NODE

Suppose:

```
TEXT
1 0  !Ä  2 0  !Ä  3 0
          ! '
```

p

Insert:

TEXT

25

Desired:

TEXT

1 0 !Ä 2 0 !Ä 2 5 !Ä 3 0

This requires maintaining both forward and backward links.

15. POINTER CHANGES — INSERT AFTER

Assume p is not tail.

C

```
newNode->next = p->next;  
newNode->prev = p;
```

```
p->next->prev = newNode;  
p->next = newNode;
```

Our relationships must become correct.

16. WHY THIS ORDER?

Original:

TEXT

2 0 !Ä 3 0

We need:

TEXT

2 0 !Ä 2 5 !Ä 3 0

New node must know:

TEXT

```
previous = 20  
next = 30
```

And neighbors must know:

TEXT

```
20.next = 25  
30.prev = 25
```

So all four directional relationships are updated.

17. AFTER INSERT TRACE

Before:

```
TEXT
2 0 !Ä 3 0
```

After:

```
TEXT
2 0 !Ä 2 5 !Ä 3 0
```

Check consistency:

```
TEXT
20.next = 25
25.prev = 20

25.next = 30
30.prev = 25
```

Ø=Å% doubly linked list must remain correct in both directions.

18. INSERT AFTER TAIL

If:

```
TEXT
p == tail
```

there is no `p->next` node.

So use append logic:

```
C
newNode->prev = tail;
tail->next = newNode;
tail = newNode;
```

Do not execute:

```
C
p->next->prev
```

when `p->next == NULL`.

19. INSERT BEFORE KNOWN NODE Ø=Ý%

This is where doubly linked lists are much better than singly lists.

Suppose:

TEXT

```
1 0 !Ä 2 0 !Ä 3 0
      !'
      p
```

Want insert 15 before 20.

Because:

```
c
p->prev
```

directly gives node 10.

No traversal needed.

If p is known:

TEXT

```
0(1)
```

20. POINTER CHANGES — INSERT BEFORE

Assuming p is not head:

```
c
newNode->prev = p->prev;
newNode->next = p;

p->prev->next = newNode;
p->prev = newNode;
```

Final:

TEXT

```
1 0 !Ä 1 5 !Ä 2 0 !Ä 3 0
```

21. COMPARE WITH SINGLY LIST

Given only pointer to node 20:

Singly

TEXT

```
1 0 !' 2 0 !' 3 0
```

Cannot directly find node 10.

May need:

TEXT

$O(n)$

traversal from head.

Doubly

TEXT

1 0 !Ä 2 0 !Ä 3 0

Use:

C

p->prev

immediately.

So insert-before-known-node:

TEXT

$O(1)$

Ø=Major advantage.

22. DELETE HEAD

Before:

TEXT

head

!“

N U L L !• 1 0 !Ä 2 0 !Ä 3 0 !' N U L L

Delete 10.

Need:

TEXT

head = 20

20.prev = NULL

23. SAFE DELETE HEAD

C

```
DNode *temp = head;
```

```
head = head->next;
```

```
if (head != NULL)
```

```
    head->prev = NULL;
```

```
free(temp);
```

If list becomes empty:

```
TEXT
tail
```

must also become:

```
TEXT
NULL
```

if tail is maintained.

24. ONE-NODE CASE

Before:

```
TEXT
head
! "
10
! '
tail
```

Delete it.

After:

```
TEXT
head = NULL
tail = NULL
```

Both pointers must change.

25. DELETE TAIL

This is much easier than singly linked list.

Before:

```
TEXT
1 0 !Ä 2 0 !Ä 3 0
! '
tail
```

Because:

```
c
tail->prev
```

directly gives node 20.

So:

```

c
DNode *temp = tail;

tail = tail->prev;

if (tail != NULL)
    tail->next = NULL;

free(temp);

```

For non-empty result.

26. COMPLEXITY — DELETE TAIL

With maintained tail:

```

TEXT
O(1)

```

Compare with singly linked list:

```

TEXT
O(n)

```

because singly list has no backward pointer.

This is one of the strongest advantages of DLL.

27. DELETE A KNOWN MIDDLE NODE

Suppose:

```

TEXT
1 0 !Ä 2 0 !Ä 3 0 !Ä 4 0
      !'
      p

```

Want delete node 20.

Because node itself gives:

```

TEXT
p->prev
p->next

```

we can reconnect neighbors directly.

28. BEFORE !' RELINK BOTH SIDES !' FREE

Before:

TEXT

```
1 0 !Ä 2 0 !Ä 3 0
      !'
      p
```

Need:

TEXT

```
1 0 !Ä 3 0
```

Forward reconnection:

C

```
p->prev->next = p->next;
```

Backward reconnection:

C

```
p->next->prev = p->prev;
```

Then:

C

```
free(p);
```

Time:

TEXT

```
0(1)
```

if `p` is already known and is an interior node.

29. VISUAL PROOF

Originally:

TEXT

```
10.next = 20
20.prev = 10

20.next = 30
30.prev = 20
```

After:

TEXT

```
10.next = 30
30.prev = 10
```

Node 20 becomes disconnected.

Then free it.

30. GENERAL DELETE KNOWN NODE

A robust deletion must consider:

- node is head
- node is tail
- node is middle
- one-node list

Conceptual general rules:

TEXT

```
If previous exists:
    previous.next = current.next
Else:
    current is head

If next exists:
    next.prev = current.prev
Else:
    current is tail
```

Then free current.

31. GENERAL FUNCTION IDEA

```
C
void deleteNode(DNode **head, DNode **tail, DNode *p)
{
    if (p == NULL)
        return;

    if (p->prev != NULL)
        p->prev->next = p->next;
    else
        *head = p->next;

    if (p->next != NULL)
        p->next->prev = p->prev;
    else
        *tail = p->prev;

    free(p);
}
```

This handles head, middle, tail, and one-node cases.

32. WHY THIS FUNCTION IS ELEGANT

If `p` has no predecessor:

TEXT

`p` is head

So update:

```
TEXT
head
```

If `p` has no successor:

```
TEXT
p is tail
```

So update:

```
TEXT
tail
```

Otherwise update neighboring links.

This is a clean general deletion pattern.

33. DELETE KNOWN NODE COMPLEXITY

If direct pointer `p` is already given:

```
TEXT
O(1)
```

No traversal needed.

Contrast with singly linked list where deleting a known tail or inserting before a node is problematic.

34. SEARCH STILL $O(n)$

Doubly linked lists improve pointer manipulation.

They do **not** make value search logarithmic.

To find key:

```
TEXT
1 0 !Ä 2 0 !Ä 3 0 !Ä 4 0
```

still may require:

```
TEXT
O(n)
```

traversal.

35. kth ACCESS STILL $O(n)$

Even though we can move backward:

TEXT

prev

there is still no direct array-like address formula.

So random positional access remains:

TEXT

$O(n)$

worst case.

36. CAN TAIL HELP kth ACCESS?

Yes slightly in practice.

If list has known length n and you want index k :

- if k near start, traverse from head
- if k near end, traverse backward from tail

Cost:

TEXT

$O(\min(k, n-1-k))$

Worst case still:

TEXT

$O(n)$

Useful implementation insight.

37. MEMORY OVERHEAD

Singly node:

```
C
struct Node
{
    int data;
    struct Node *next;
};
```

stores one link.

Doubly node:

```
C
struct DNode
{
    int data;
    struct DNode *prev;
```



```

    struct DNode *next;
};

```

stores two links.

Therefore DLL requires more memory per node.

Trade-off:

Faster/easier backward operations in exchange for extra pointer memory and more link updates.

38. INSERTION ERROR RISK

In singly list, insertion may require 2 core link writes.

In doubly list, middle insertion may require 4 relationships.

That gives more flexibility but also more chance of bugs.

You must maintain:

```

TEXT
forward consistency
+
backward consistency

```

39. DLL CONSISTENCY PROPERTY

For neighboring nodes A and B :

If:

```

TEXT
A.next == B

```

then normally:

```

TEXT
B.prev == A

```

Useful invariant.

If one direction says something different, structure is corrupted.

40. EXAMPLE CORRUPTION

Suppose you insert 25:

```

TEXT
2 0 !Ä 2 5 !' 3 0

```

but forget:

```
C
30->prev = newNode;
```

Forward traversal may appear correct:

```
TEXT
2 0 !' 2 5 !' 3 0
```

But backward traversal:

```
TEXT
3 0 !' 2 0
```

skips 25.

The DLL is inconsistent.

0-6% TE/output questions can exploit this.

41. POINTER ORDER FOR INSERT AFTER

A safe compact order:

```
C
newNode->prev = p;
newNode->next = p->next;

if (p->next != NULL)
    p->next->prev = newNode;

p->next = newNode;
```

If using tail:

```
C
if (newNode->next == NULL)
    tail = newNode;
```

42. POINTER ORDER FOR INSERT BEFORE

```
C
newNode->next = p;
newNode->prev = p->prev;

if (p->prev != NULL)
    p->prev->next = newNode;
else
    head = newNode;

p->prev = newNode;
```

This naturally handles inserting before head.

43. DELETE GENERAL PATTERN

For node `p`:

```
c
if (p->prev != NULL)
    p->prev->next = p->next;
else
    head = p->next;

if (p->next != NULL)
    p->next->prev = p->prev;
else
    tail = p->prev;

free(p);
```

Excellent pattern to understand.

44. SINGLY VS DOUBLY — OPERATION TABLE

Assume direct relevant pointers are available where stated.

Operation	Singly	Doubly
Forward traversal	$O(n)$	$O(n)$
Backward traversal	Not directly	$O(n)$
Search	$O(n)$	$O(n)$
Insert head	$O(1)$	$O(1)$
Append with tail	$O(1)$	$O(1)$
Delete head	$O(1)$	$O(1)$
Delete tail with tail ptr	$O(n)$	$O(1)$
Insert after known node	$O(1)$	$O(1)$
Insert before known node	usually need predecessor/search	$O(1)$
Delete known interior node	need predecessor unless special trick	$O(1)$

High-value comparison.

45. WHY DELETE TAIL IMPROVES

Singly:

TEXT

1 0 !' 2 0 !' 3 0

tail points to 30.

Need predecessor 20.

Cannot access it directly.

Doubly:

TEXT

1 0 !Ä 2 0 !Ä 3 0
!'
tail

Use:

c

tail->prev

directly.

Hence:

TEXT

0(1)

46. REVERSING A DOUBLY LINKED LIST — INTUITION

Original:

TEXT

N U L L !• 1 0 !Ä 2 0 !Ä 3 0 !' N U L L

To reverse, every node's:

TEXT

next
prev

can be swapped.

Then head/tail exchange roles.

Result:

TEXT

N U L L !• 3 0 !Ä 2 0 !Ä 1 0 !' N U L L

Ø=Placement extension.

47. REVERSAL BASIC IDEA

For every node:

TEXT

```
swap(node->prev, node->next)
```

Then move through the list carefully.

At end:

TEXT

```
old tail = new head  
old head = new tail
```

Time:

TEXT

$O(n)$

Space:

TEXT

$O(1)$

Detailed implementation is less important than singly reversal for core GATE, but know the concept.

48. APPLICATION — BROWSER HISTORY

Imagine:

TEXT

P a g e A !Ä P a g e B !Ä P a g e C

From Page B:

TEXT

B a c k !' P a g e A
F o r w a r d !' P a g e C

This naturally resembles a doubly linked list.

Common intuition example.

49. APPLICATION — UNDO/REDO

State history:

TEXT

S t a t e 1 !Ä S t a t e 2 !Ä S t a t e 3

You can move:

```
TEXT
u n d o !•
r e d o !'
```

Again DLL is a natural conceptual fit.

50. APPLICATION — LRU CACHE INTUITION Ø=pe

A common LRU cache design combines:

```
TEXT
hash table
+
doubly linked list
```

Why doubly list?

Because once a cache entry/node is found, we often want to:

```
TEXT
remove it
move it to front
```

in:

```
TEXT
O(1)
```

A DLL allows known-node deletion without searching for predecessor.

We will study hashing separately later, but remember this connection.

51. APPLICATION — MUSIC PLAYLIST

Conceptually:

```
TEXT
P r e v i o u s !• C u r r e n t !' N e x t
```

is naturally modeled with bidirectional links.

52. GATE TRAP — prev OF HEAD

For normal DLL:

```
TEXT
head->prev == NULL
```

Ø=Must know.

53. GATE TRAP — next OF TAIL

TEXT

```
tail->next == NULL
```

54. GATE TRAP — ONE NODE

If one node:

TEXT

```
head == tail
```

and both:

TEXT

```
head->prev == NULL  
head->next == NULL
```

55. GATE TRAP — DELETE KNOWN NODE

Statement:

Deletion in a doubly linked list is always $O(1)$.

Not fully correct.

If node pointer is already given:

TEXT

```
 $O(1)$ 
```

But if you first need to find the value:

TEXT

```
search  $O(n)$   
+  
delete  $O(1)$   
=  
 $O(n)$ 
```

~~$O(1)$~~ Some search-vs-rewiring distinction.

56. GATE TRAP — EXTRA MEMORY

DLL uses more memory than singly list because each node needs:

TEXT

```
prev + next
```

rather than only:

TEXT

```
next
```

Plus possible padding/alignment.

57. GATE TRAP — MODIFY BOTH SIDES

If inserting/deleting between nodes, you usually must update both:

TEXT

```
next relationship
prev relationship
```

Updating only one direction corrupts structure.

58. GATE POINTER TRACE

Initial:

TEXT

```
N U L L !•  A !Ä  B !Ä  C !'  N U L L
```

Let:

TEXT

```
p !'  B
```

Execute:

C

```
p->prev->next = p->next;
p->next->prev = p->prev;
```

What happens?

First:

TEXT

```
A.next = C
```

Second:

TEXT

```
C.prev = A
```


So B is bypassed:

TEXT

A !Ä C

But B is not yet freed.

0-1% linking and freeing are separate operations.

59. MEMORY LEAK TRAP

If after bypassing B you lose the only pointer to B without:

C

```
free(B);
```

B becomes an unreachable dynamically allocated node.

Memory leak.

So:

TEXT

R E L I N K ! ' F R E E

still matters.

60. DANGLING POINTER TRAP

If another pointer:

C

```
q = p;
```

and then:

C

```
free(p);
```

q becomes dangling.

Same pointer alias rule as before.

61. INTERVIEW QUESTION — WHY NOT ALWAYS USE DLL?

Because DLL has trade-offs:

- extra memory
- more pointer assignments
- more implementation complexity
- potentially more chances for corruption

If backward traversal/deletion-before are not needed, singly list may be simpler.

62. INTERVIEW QUESTION — DELETE NODE COMPLEXITY

Given direct pointer `p` to an interior DLL node:

TEXT

$O(1)$

because:

TEXT

`p->prev`

`p->next`

give both neighbors directly.

This is a classic answer.

63. INTERVIEW QUESTION — SEARCH COMPLEXITY

Still:

TEXT

$O(n)$

unless a separate indexing/search structure is used.

DLL does not magically provide random access.

64. MEMORY TRICKS

Doubly list

Every node knows BEFORE and AFTER.

Head

Nobody before HEAD.

TEXT

`head->prev = NULL`

Tail

Nobody after TAIL.

TEXT

```
tail->next = NULL
```

Delete middle

LEFT skips me; RIGHT skips me backward.

Insert middle

New node shakes hands with BOTH neighbors.

65. NOTEBOOK NOTES

Structure

```
C
struct DNode
{
    int data;
    struct DNode *prev;
    struct DNode *next;
};
```

Head/Tail

TEXT

```
head->prev = NULL
tail->next = NULL
```

Insert after p

```
C
new->prev = p;
new->next = p->next;

p->next->prev = new;
p->next = new;
```

for interior case.

Delete known interior p

```
C
p->prev->next = p->next;
p->next->prev = p->prev;
free(p);
```

66. COMPLEXITIES

TEXT

```
Forward traversal  O(n)
Backward traversal O(n)
Search            O(n)
Insert head       O(1)
Delete head       O(1)
Append with tail  O(1)
Delete tail       O(1)
Insert before known node O(1)
Delete known node      O(1)
```

assuming required node/tail pointers are already available.

67. 30-SECOND REVISION

Remember:

1. DLL node has `prev` and `next`.
2. It supports forward and backward traversal.
3. Head has `prev=NULL`.
4. Tail has `next=NULL`.
5. Known-node delete can be $O(1)$.
6. Delete tail with tail pointer is $O(1)$.
7. Insert before known node is $O(1)$.
8. Search remains $O(n)$.
9. DLL requires extra memory.
10. Both forward and backward links must remain consistent.

68. ACTIVE RECALL

Q1

What three fields does a basic DLL integer node contain?

Q2

What should `head->prev` contain?

Q3

Why can delete-tail be $O(1)$ in DLL?

Q4

If a known interior node `p` is deleted, which two neighbor links must be changed?

Q5

Does DLL provide $O(1)$ random positional access?

ANSWERS

A1

TEXT

```
prev
data
next
```

A2

TEXT

NULL

A3

Because:

C

```
tail->prev
```

directly gives its predecessor.

A4

C

```
p->prev->next = p->next;
p->next->prev = p->prev;
```

A5

No. Worst-case positional access remains $O(n)$.

69. PRACTICE — EASY

Q1

For:

TEXT

```
N U L L ! • 1 0 ! Ä 2 0 ! Ä 3 0 ! ' N U L L
```

if `p` points to 20, what values do these nodes contain?

TEXT

```
p->data
p->prev->data
p->next->data
```

Q2

If only one node exists, what relationship exists between:

TEXT

head
tail

?

70. PRACTICE — MODERATE

Q3

Before:

TEXT

```
1 0 !Ä 2 0 !Ä 4 0
      !'
      p
```

Insert 30 after p.

Write the four conceptual link relationships that must hold afterward.

Q4

Before:

TEXT

```
1 0 !Ä 2 0 !Ä 3 0
      !'
      p
```

Delete node 20.

What should:

TEXT

10.next
30.prev

become?

Q5

Why is deleting tail $O(n)$ in a singly linked list but $O(1)$ in a DLL with tail pointer?

71. PRACTICE — GATE TRAP

Initial:

TEXT

A !Ä B !Ä C

Execute only:

```
C
A->next = C;
```

What is wrong?

Forward traversal may become:

```
TEXT
A !' C
```

but:

```
TEXT
C->prev
```

still points to B.

The list is inconsistent.

The missing assignment is:

```
C
C->prev = A;
```

Both directions matter.

72. PLACEMENT CHALLENGE

You are given only a pointer to node `X` in a non-empty doubly linked list and also have access to `head` and `tail`.

Can you delete `X` in $O(1)$?

Answer

Yes.

- If `X` has a predecessor, connect predecessor to successor.
- Otherwise update `head`.
- If `X` has a successor, connect successor back to predecessor.
- Otherwise update `tail`.
- Free `X`.

No search is required because `X` itself stores both neighboring addresses.

PROGRESS

MODULE 3 — LINKED LISTS

' Lesson 29 — Intuition, Nodes & Memory ' Lesson 30 — Singly Linked List Fundamentals ' Lesson 31 — Insertion ' Lesson 32 — Deletion ' Lesson 33 — Reversal ' **Lesson 34 — Doubly Linked Lists**

NEXT — LESSON 35

Circular Linked Lists — No NULL End, Circular Traversal & Pointer Traps

Next we study:

```
TEXT
head
! "
1 0 ! ' 2 0 ! ' 3 0
! ' ! "
% % % % % % % % % %
```

The key difference:

```
TEXT
last->next = head
```

instead of:

```
TEXT
last->next = NULL
```

We will cover:

- circular singly linked list
- empty and one-node cases
- traversal without infinite loops
- `do...while`
- insert at head/end
- tail-based representation
- delete head/end
- circular doubly linked list preview
- applications such as round-robin scheduling
- GATE traps involving termination conditions and cycle structure.

```
USER
next
```

ASSISTANT

MODULE 3 — LESSON 35

Circular Linked Lists — No NULL End, Circular Traversal & Pointer Traps

We now study the final basic linked-list type:

Circular Linked List

Unlike a normal singly linked list, the last node does **not** point to `NULL`.

Instead:

TEXT

```
last->next = head
```

So the list forms a circle.

GATE importance: $\emptyset = \text{NOT IMPORTANT}$ **Placement importance:** $++ = \text{IMPORTANT}$

1. NORMAL SINGLY VS CIRCULAR

Normal singly linked list:

TEXT

```
head
! "
1 0 !' 2 0 !' 3 0 !' N U L L
```

Circular singly linked list:

TEXT

```
head
! "
1 0 !' 2 0 !' 3 0
!' ! "
% % % % % % % % %
```

So:

TEXT

```
30->next = head
```

instead of:

TEXT

```
30->next = NULL
```

2. CORE DIFFERENCE

For a normal singly linked list:

Repeatedly following next eventually reaches NULL.

For a circular linked list:

Repeatedly following next eventually returns to the starting node.

0=7% This changes traversal logic completely.

3. NODE STRUCTURE

The node itself is the same as a singly linked-list node:

```
c
typedef struct Node
{
    int data;
    struct Node *next;
} Node;
```

What makes the list circular is not a new field.

It is the way the links are connected.

4. THREE-NODE CIRCULAR LIST

Suppose:

```
TEXT
1 0 !' 2 0 !' 3 0
!'          !"
% % % % % % % % %
```

Then:

```
TEXT
head->data = 10
head->next->data = 20
head->next->next->data = 30
```

and crucially:

```
TEXT
head->next->next->next == head
```

0=7% common GATE-style pointer relation.

5. THERE IS NO NULL AT THE END

In a standard non-empty circular singly linked list:

```
TEXT
last->next == head
```

not:

TEXT

```
last->next == NULL
```

Therefore this normal traversal is wrong:

```
C
while (temp != NULL)
{
    temp = temp->next;
}
```

Why?

Because `temp` never becomes `NULL`.

It keeps cycling.

6. INFINITE LOOP TRAP Ø=Ý%

Suppose:

TEXT

```
1 0 !' 2 0 !' 3 0
!'          !"
% % % % % % % % %
```

Start at 10:

TEXT

```
10
20
30
10
20
30
10
...
```

So:

C

```
while (temp != NULL)
```

never stops.

Ø=Ý% is the single biggest circular-list trap.

7. HOW DO WE KNOW WE COMPLETED ONE CYCLE?

If traversal starts at:

TEXT

head

we stop when we return to:

TEXT

head

So the condition becomes conceptually:

TEXT

process until temp becomes head again

8. WHY `do...while` FITS PERFECTLY

Suppose list is non-empty.

We need to process the first node **before** checking whether we have returned to it.

So:

```
C
Node *temp = head;

do
{
    printf("%d ", temp->data);
    temp = temp->next;
} while (temp != head);
```

~~0-1%~~ This is the classic circular-list traversal.

9. FULL SAFE TRAVERSAL

```
C
void printCircular(Node *head)
{
    if (head == NULL)
        return;

    Node *temp = head;

    do
    {
        printf("%d ", temp->data);
        temp = temp->next;
    } while (temp != head);
}
```

For:

TEXT

```
1 0 !' 2 0 !' 3 0
!'      !"
% % % % % % % % %
```

output:

TEXT

```
10 20 30
```

exactly once each.

10. WHY NOT SIMPLE `while(temp != head)` ?

If we do:

```
C
Node *temp = head;

while (temp != head)
{
    ...
}
```

the condition is false immediately.

Because initially:

TEXT

```
temp == head
```

So loop executes zero times.

That's why:

DO-WHILE IS NATURAL FOR CIRCULAR TRAVERSAL

11. ALTERNATIVE WITHOUT DO-WHILE

You could write:

```
C
Node *temp = head;

if (temp != NULL)
{
    printf("%d ", temp->data);
    temp = temp->next;

    while (temp != head)
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }
}
```

```
}  
}
```

But `do...while` is cleaner.

12. EMPTY CIRCULAR LIST

Empty list:

```
TEXT  
head = NULL
```

There is no circle.

So always check:

```
C  
if (head == NULL)
```

before circular traversal.

13. ONE-NODE CIRCULAR LIST $\emptyset \rightarrow Y\%$

This is very important.

One node:

```
TEXT  
head  
! "  
+----+  
| 10 |  
+----+  
! ' |  
% % %
```

Its `next` points to itself:

```
C  
head->next = head;
```

So:

```
TEXT  
head->next == head
```

$\emptyset \rightarrow Y\%$ is how a one-node circular list differs from a one-node normal singly list.

Normal singly:

TEXT

```
head->next == NULL
```

Circular:

TEXT

```
head->next == head
```

14. CREATE FIRST NODE

Suppose:

C

```
Node *newNode = createNode(10);
```

For the first circular node:

C

```
newNode->next = newNode;  
head = newNode;
```

Now:

TEXT

```
10 points to itself
```

15. FIND LAST NODE

In circular list, last node satisfies:

C

```
temp->next == head
```

not:

C

```
temp->next == NULL
```

So:

C

```
Node *temp = head;  
  
while (temp->next != head)  
{  
    temp = temp->next;  
}
```

After loop:

TEXT

```
temp != last node
```

assuming non-empty list.

16. COMPLEXITY FIND LAST

Without tail pointer:

TEXT

```
O(n)
```

because we may traverse the whole cycle.

With a maintained `tail` pointer:

TEXT

```
O(1)
```

to access last node.

17. TAIL-BASED REPRESENTATION

Circular singly linked lists are often represented using:

C

```
Node *tail;
```

Instead of head.

Why?

Because:

TEXT

```
head = tail->next
```

So with one pointer `tail`, we can get:

TEXT

```
last node directly  
and  
first node directly
```

This is very useful.

18. VISUALIZATION WITH TAIL

TEXT

```
tail
1 0 !' 2 0 !' 3 0 !"
!'      !"
% % % % % % % % %
```

Then:

C

```
tail->data
```

gives last node.

And:

C

```
tail->next
```

gives head.

So:

FORMULA BOX

TEXT

```
head = tail->next
```

for a non-empty circular singly linked list represented by tail.

$\emptyset$ = Null value concept.

19. EMPTY LIST WITH TAIL REPRESENTATION

Empty:

TEXT

```
tail = NULL
```

Then there is no head either.

For non-empty:

TEXT

```
head = tail->next
```

20. ONE NODE WITH TAIL

One node:

TEXT

```
tail
  !"
10
! ' !"
% % %
```

So:

C

```
tail->next = tail;
```

And since:

TEXT

```
head = tail->next
```

we get:

TEXT

```
head == tail
```

for one-node list.

21. INSERT AT HEAD — USING ONLY HEAD

Suppose:

TEXT

```
head
  !"
 1 0 !'  2 0 !'  3 0
! '           !"
% % % % % % % % %
```

Insert:

TEXT

```
5
```

Desired:

TEXT

```
head
  !"
 5 !'  1 0 !'  2 0 !'  3 0
! '           !"
% % % % % % % % % % % %
```

Problem:

Last node currently points to old head.

After changing head, last must point to new head.

22. INSERT HEAD — STEPS

If only `head` is maintained:

1. find last node
2. connect new node to old head
3. connect last node to new node
4. update head

Conceptually:

```
c
newNode->next = head;
last->next = newNode;
head = newNode;
```

But finding `last` costs:

```
TEXT
O(n)
```

unless tail is maintained.

23. INSERT HEAD WITH TAIL POINTER Ø=Ý%

This becomes much easier.

Suppose:

```
TEXT
t a i l ! ' 3 0
h e a d = t a i l - > n e x t ! ' 1 0
```

Insert 5.

Before:

```
TEXT
tail
! "
1 0 ! ' 2 0 ! ' 3 0
! ' ! "
% % % % % % % % % %
```

New:

```
TEXT
5
```

Pointer changes:

```
C
newNode->next = tail->next;
tail->next = newNode;
```

That's it.

Tail stays 30.

New head becomes:

```
TEXT
tail->next
```

which is 5.

$O(1)$ time:

```
TEXT
O(1)
```

24. INSERT HEAD — EMPTY TAIL CASE

If:

```
TEXT
tail == NULL
```

new node is first node.

Set:

```
C
tail = newNode;
newNode->next = newNode;
```

Now:

```
TEXT
head = tail->next = newNode
```

25. INSERT AT END USING TAIL $O(1)$

Before:

```
TEXT
tail
!
1 0 !' 2 0 !' 3 0
!' !
% % % % % % % % %
```

Insert:

```
TEXT
40
```

Desired:

```
TEXT
tail
1 0 !' 2 0 !' 3 0 !' 4 0
!' !"
% % % % % % % % % % % % %
```

26. POINTER CHANGES — INSERT END

New node should point to head:

```
C
newNode->next = tail->next;
```

Old tail should point to new:

```
C
tail->next = newNode;
```

Then:

```
C
tail = newNode;
```

✓
O(1) free constant-time updates.

27. BEFORE !' POINTER CHANGES !' AFTER

Before:

```
TEXT
tail = 30
head = 10

1 0 !' 2 0 !' 3 0
!' !"
% % % % % % % % % %

new = 40
```

Step 1:

```
C
new->next = tail->next;
```

SO:

```
TEXT
4 0 !' 1 0
```

Step 2:

```
C
tail->next = new;
```

SO:

```
TEXT
3 0 !' 4 0
```

Step 3:

```
C
tail = new;
```

Now tail is 40.

Final cycle:

```
TEXT
1 0 !' 2 0 !' 3 0 !' 4 0
!'                                     !"
% % % % % % % % % % % % % % % %
```

28. COMPLEXITY — INSERT END

With tail:

```
TEXT
O(1)
```

Without tail:

need find last:

```
TEXT
O(n)
```

$O=V$ representation is especially useful in circular lists.

29. INSERT AFTER KNOWN NODE

Suppose:

TEXT

```
1 0 !' 2 0 !' 3 0
!'          !"
% % % % % % % % %
      !'
    p = 20
```

Insert 25 after 20.

Same singly-list pointer logic:

C

```
newNode->next = p->next;
p->next = newNode;
```

Result:

TEXT

```
1 0 !' 2 0 !' 2 5 !' 3 0
!'          !"
% % % % % % % % % % % %
```

If `p == tail`, then update:

C

```
tail = newNode;
```

30. DELETE HEAD USING HEAD ONLY

Suppose:

TEXT

```
head
!"
1 0 !' 2 0 !' 3 0
!'          !"
% % % % % % % % %
```

Delete 10.

Need:

TEXT

```
new head = 20
```

but last node must be changed from:

TEXT

```
3 0 !' 1 0
```

to:

TEXT

3 0 !' 2 0

So if only head is maintained:

1. find last
2. save old head
3. move head
4. last->next = new head
5. free old head

Finding last:

TEXT

$O(n)$

31. DELETE HEAD WITH TAIL $\emptyset = \text{!}'\%$

Since:

TEXT

head = tail->next

we already know both last and first.

Before:

TEXT

t a i l !' 3 0
h e a d !' 1 0

Save victim:

C

```
Node *temp = tail->next;
```

Move head by changing tail's link:

C

```
tail->next = temp->next;
```

Then:

C

```
free(temp);
```

New head automatically is:

TEXT

```
tail->next
```

Time:

TEXT

```
O(1)
```

32. SINGLE-NODE DELETE HEAD

This is special.

Suppose:

TEXT

```
tail
! "
10
! ' ! "
% % %
```

Then:

TEXT

```
tail->next == tail
```

If deleting the only node:

C

```
free(tail);
tail = NULL;
```

Now list empty.

~~O(1)~~ Can not use normal multi-node head-deletion logic blindly.

33. DELETE END USING TAIL

Interesting.

We have direct pointer to tail, but to remove it we need the node before tail.

In circular singly list:

TEXT

```
1 0 ! ' 2 0 ! ' 3 0
! ' ! "
% % % % % % % % % %
! '
tail
```

Need node:

TEXT

20

There is no `prev`.

Therefore must traverse to find:

TEXT

node whose `next == tail`

So delete-tail remains:

TEXT

$O(n)$

in circular singly list.

~~$O(1)$~~ Some fundamental singly-list limitation.

34. DELETE LAST — TRACE

Before:

TEXT

```
1 0 !' 2 0 !' 3 0
!'                               !"
% % % % % % % % % %
                               !'
tail
```

Find:

TEXT

p r e v !' 2 0

Then:

C

```
prev->next = tail->next;
```

Remember:

TEXT

```
tail->next = head
```

So this makes:

TEXT

2 0 !' h e a d

Then:

```
c
free(tail);
tail = prev;
```

Final:

```
TEXT
1 0 !' 2 0
!' !"
% % % % %
```

35. DELETE AFTER KNOWN NODE

Suppose:

```
TEXT
1 0 !' 2 0 !' 3 0 !' 4 0
!' !"
% % % % % % % % % % % % %
!'
p
```

Delete node after 20, i.e. 30.

Standard pattern:

```
c
Node *temp = p->next;
p->next = temp->next;
free(temp);
```

Works.

But if deleted node is `tail`, update:

```
c
tail = p;
```

Ø=Val metadata must remain correct.

36. CIRCULAR LIST TRAVERSAL FROM ANY NODE

Interesting property:

If the list is circular and connected, you don't necessarily need to start at the designated head.

Suppose:

```
TEXT
1 0 !' 2 0 !' 3 0
!' !"
% % % % % % % % %
```

Start at node 20:

```
TEXT
2 0 !' 3 0 !' 1 0 !' 2 0
```

You can still visit every node exactly once by stopping when you return to 20.

Ø=10% is useful in round-robin applications.

37. ROUND-ROBIN SCHEDULING INTUITION

Suppose processes:

```
TEXT
P 1 !' P 2 !' P 3 !' P 1 !' . . .
```

Each gets a turn, then scheduler moves to the next.

After P3:

```
TEXT
next = P1
```

This naturally resembles:

Circular Linked List

Ø=10% common application example.

38. MUSIC / GAME TURNS

Players:

```
TEXT
A !' B !' C !' D
!' !"
% % % % % % % % % % % % % %
```

After D's turn:

```
TEXT
back to A
```

Again circular structure fits naturally.

39. JOSEPHUS PROBLEM CONNECTION Ø=10%

Classic problem:

People stand in a circle and every kth person is eliminated until one remains.

Circular linked lists give a natural representation.

The Josephus problem is also solvable mathematically/recursively, but circular-list simulation is intuitive.

Ø=Übod placement/DS connection.

40. COUNT NODES IN CIRCULAR LIST

```
c
int countCircular(Node *head)
{
    if (head == NULL)
        return 0;

    int count = 0;
    Node *temp = head;

    do
    {
        count++;
        temp = temp->next;
    } while (temp != head);

    return count;
}
```

Time:

```
TEXT
O(n)
```

Space:

```
TEXT
O(1)
```

41. SEARCH IN CIRCULAR LIST

```
c
int searchCircular(Node *head, int key)
{
    if (head == NULL)
        return 0;

    Node *temp = head;

    do
    {
        if (temp->data == key)
            return 1;
    }
}
```

```

    temp = temp->next;

} while (temp != head);

return 0;
}

```

Worst:

TEXT
 $O(n)$

42. WHY SEARCH MUST STOP AT START

If key is absent and you search with:

C
while (temp != NULL)

you never stop.

Correct failure condition is:

I have returned to my starting node without finding the key.

43. CIRCULAR LIST INVARIANT

For a valid non-empty circular singly linked list:

TEXT
following next repeatedly from any node
eventually returns to that node

In a simple n -node cycle, after exactly n next-steps from a node:

TEXT
you return to that same node

Useful GATE relation.

44. GATE EXAMPLE

Suppose circular list has 5 nodes and:

TEXT
p

points to any node.

What is:

TEXT

p advanced through next exactly 5 times

?

Answer:

TEXT

p returns to the same node

For 10 steps:

TEXT

same node again

because:

TEXT

$10 \bmod 5 = 0$

45. MODULO INTUITION

If circular list has n nodes, moving k positions around the cycle is equivalent to:

TEXT

$k \bmod n$

effective forward steps from the starting position.

Example:

TEXT

$n = 4$
 $k = 10$

TEXT

$10 \bmod 4 = 2$

So 10 forward moves land where 2 forward moves would.

Nice conceptual connection.

46. CIRCULAR DOUBLY LINKED LIST — PREVIEW

We can also make a doubly linked list circular.

Instead of:

TEXT

```
head->prev = NULL
tail->next = NULL
```

we set:

TEXT

```
head->prev = tail
tail->next = head
```

Visualization:

TEXT

```

          % % % % % % % % % % % % % % % %
          ! "
1 0 !Ä 2 0 !Ä 3 0
! '
% % % % % % % % % % % % % % % %
```

More simply:

TEXT

```
head.prev = tail
tail.next = head
```

47. CIRCULAR DOUBLY INVARIANTS

For non-empty circular DLL:

C

```
head->prev == tail
tail->next == head
```

And for neighboring nodes:

TEXT

```
A.next = B
B.prev = A
```

There are no NULL end links.

48. ONE-NODE CIRCULAR DOUBLY LIST

Single node points to itself in both directions:

TEXT

```
node->next = node
node->prev = node
```

$\emptyset \neq \text{Pot}$ Potential GATE edge case.

49. NORMAL VS CIRCULAR COMPARISON

Property	Normal Singly	Circular Singly
Last next	NULL	head
Natural stopping condition	NULL	return to start
One-node next	NULL	itself
Round-robin traversal	less natural	natural
while(temp != NULL)	correct	can loop forever

50. TAIL REPRESENTATION COMPLEXITIES

For circular singly list with `tail`:

Operation	Time
Access tail	O(1)
Access head (tail->next)	O(1)
Insert at head	O(1)
Insert at tail	O(1)
Delete head	O(1)
Delete tail	O(n)
Search	O(n)
Traverse	O(n)

$O = \infty$ - value table.

51. WHY TAIL REPRESENTATION IS SO GOOD

Normal singly list often chooses:

TEXT
head

as the essential pointer.

Circular singly list frequently benefits from:

TEXT

tail

because tail immediately gives both:

TEXT

tail

and:

TEXT

tail->next = head

So with one pointer we know both ends.

Very useful insight.

52. INSERT HEAD WITH ONLY TAIL — MASTER FORM

```
C
if (tail == NULL)
{
    tail = newNode;
    newNode->next = newNode;
}
else
{
    newNode->next = tail->next;
    tail->next = newNode;
}
```

Notice:

TEXT

tail does not change

for nonempty head insertion.

53. INSERT END WITH ONLY TAIL — MASTER FORM

```
C
if (tail == NULL)
{
    tail = newNode;
    newNode->next = newNode;
}
else
{
    newNode->next = tail->next;
    tail->next = newNode;
    tail = newNode;
}
```

Compare with head insertion.

Only extra final step:

```
c
tail = newNode;
```

~~Ø=Nil~~ symmetry.

54. GATE TRAP — LAST NODE CONDITION

Normal singly:

```
c
last->next == NULL
```

Circular singly:

```
c
last->next == head
```

If tail representation:

```
c
tail->next == head
```

always for nonempty list.

55. GATE TRAP — SINGLE NODE

Normal singly one-node:

```
TEXT
head->next = NULL
```

Circular singly one-node:

```
TEXT
head->next = head
```

~~Ø=Nil~~ must distinguish.

56. GATE TRAP — EMPTY LIST

Do not try:

```
c
tail->next
```

if:

```
TEXT
tail == NULL
```

because there is no head.

Empty cases still need explicit checking.

57. GATE TRAP — CYCLE VS CIRCULAR LIST

A circular linked list intentionally contains a cycle.

But later, when we study cycle detection, a normal singly linked list may accidentally contain a cycle due to a bug.

So:

```
TEXT
cycle
```

does not automatically mean:

```
TEXT
proper circular linked list
```

A circular list has intentionally defined structural semantics.

58. GATE TRAP — `do...while`

Do-while is ideal because:

```
TEXT
first node must be processed once
```

before checking:

```
TEXT
have we returned to head?
```

A plain `while(temp != head)` starting at head executes zero times.

59. GATE POINTER QUESTION

Circular list:

```
TEXT
A  !'  B  !'  C  !'  A
```

Suppose:

TEXT

```
p = A
```

Then:

TEXT

```
p->next = B
p->next->next = C
p->next->next->next = A
```

Therefore:

C

```
p->next->next->next == p
```

is true.

0-0% Common type of reasoning.

60. GATE POINTER QUESTION — 4 NODES

Suppose:

TEXT

```
A !' B !' C !' D !' A
```

What is:

TEXT

```
A->next->next->next->next
```

?

Answer:

TEXT

```
A
```

because four next steps complete the circle.

61. GATE TRAP — TRAVERSAL COMPLEXITY

Circular traversal is still:

TEXT

```
O(n)
```

for one complete cycle.

The fact that traversal could continue forever does not change intended one-cycle complexity.

The algorithm deliberately stops after one full cycle.

62. MEMORY MANAGEMENT

If deleting all nodes in a circular list, normal:

```
C
while (head != NULL)
```

free logic cannot be copied blindly because the list does not naturally reach NULL.

You must carefully break or iterate through exactly one cycle while preserving addresses.

Ø=unimportant implementation caution.

63. SIMPLE FREE IDEA WITH TAIL

One approach:

1. identify head
2. break the circle
3. free like ordinary singly list

If:

```
C
Node *head = tail->next;
tail->next = NULL;
```

the circular list becomes:

```
TEXT
h e a d ! ' . . . ! ' t a i l ! ' N U L L
```

Then normal free-list code can be used.

Afterward:

```
TEXT
tail = NULL
```

This is conceptually clean.

64. APPLICATIONS

Circular linked lists are useful conceptually for:

- round-robin CPU scheduling
 - multiplayer turns
 - cyclic playlists
 - repeated resource allocation
 - token passing
-

- Josephus simulation
- circular buffers at a conceptual level

Note: circular arrays are often preferable for queues/buffers due to cache locality, but the cyclic idea is similar.

65. INTERVIEW QUESTIONS

Q1

How do you identify last node in circular singly list?

TEXT

```
node->next == head
```

Q2

How do you traverse exactly once?

Use `do...while` and stop when pointer returns to start.

Q3

Why can `while(temp != NULL)` fail?

Because no nonempty node points to NULL.

Q4

If tail is maintained, how do you obtain head?

C

```
tail->next
```

Q5

Insert-at-end complexity with tail?

TEXT

```
O(1)
```

Q6

Delete-tail complexity?

TEXT

```
O(n)
```

for singly circular list, because predecessor must be found.

66. MEMORY TRICKS

Circular list

LAST points FIRST.

Traversal

Stop when you come HOME.

One node

Points to ITSELF.

Tail representation

TAIL knows LAST; TAIL->NEXT knows FIRST.

Do-while

Visit first, then ask: "Am I back?"

67. NOTEBOOK NOTES

Circular Singly

TEXT

```
last->next = head
```

One node

TEXT

```
head->next = head
```

Traversal

```
c
if (head != NULL)
{
    Node *temp = head;

    do
    {
        process(temp);
        temp = temp->next;
    } while (temp != head);
}
```


Tail representation

TEXT

```
head = tail->next
```

68. INSERT HEAD — TAIL REPRESENTATION

C

```
newNode->next = tail->next;
tail->next = newNode;
```

Non-empty case.

Complexity:

TEXT

$O(1)$

69. INSERT TAIL

C

```
newNode->next = tail->next;
tail->next = newNode;
tail = newNode;
```

Complexity:

TEXT

$O(1)$

70. COMPLEXITY MASTER TABLE

Operation	Circular Singly with Tail
Head access	$O(1)$
Tail access	$O(1)$
Traverse once	$O(n)$
Search	$O(n)$
Insert head	$O(1)$
Insert tail	$O(1)$
Delete head	$O(1)$
Delete tail	$O(n)$

71. 30-SECOND REVISION

Remember:

1. Circular list has no normal NULL end.
 2. Last node points to head.
 3. One-node list points to itself.
 4. `while(temp != NULL)` can loop forever.
 5. `do...while` is natural for traversal.
 6. Stop after returning to the start node.
 7. With tail, `head = tail->next`.
 8. Insert head with `tail = 0(1)`.
 9. Insert tail with `tail = 0(1)`.
 10. Delete tail remains `0(n)` in singly circular list.
-

72. ACTIVE RECALL

Q1

What does the last node point to?

Q2

What does a single circular node's `next` point to?

Q3

Why is `do...while` useful?

Q4

If `tail` is maintained, how do you access head?

Q5

What is delete-tail complexity in circular singly list?

ANSWERS

A1

TEXT
head

A2

Itself.

C
`head->next = head;`

A3

Because the starting node must be processed before checking whether traversal has returned to it.

A4

```
C
tail->next
```

A5

```
TEXT
O(n)
```

because predecessor must be located.

73. PRACTICE — EASY

Q1

For:

```
TEXT
A !' B !' C !' A
```

what does:

```
TEXT
A->next->next->next
```

point to?

Q2

If a circular singly linked list has one node `X`, what is:

```
TEXT
X->next
```

?

Q3

Why doesn't:

```
C
while (temp != NULL)
```

properly terminate normal circular traversal?

74. PRACTICE — MODERATE

Q4

Suppose:

TEXT

```
t a i l ! ' 3 0
```

and:

TEXT

```
t a i l - > n e x t ! ' 1 0
```

for:

TEXT

```
1 0 ! ' 2 0 ! ' 3 0 ! ' 1 0
```

Insert 5 at head.

Write the two nonempty-case pointer assignments.

Q5

Insert 40 at end using tail.

Write all three pointer assignments.

Q6

Why can head deletion be $O(1)$ with tail, but tail deletion remains $O(n)$?

75. PRACTICE — GATE TRAPS

Q7

A circular list has 6 nodes.

Starting from any node `p`, after 14 `next` movements, how many effective positions forward from `p` are you?

Use:

TEXT

```
14 mod 6
```

Q8

Why does this print nothing?

```
C
Node *temp = head;

while (temp != head)
{
    printf("%d", temp->data);
    temp = temp->next;
}
```

Q9

For a non-empty tail-based circular singly list, which must be true?

A.

```
TEXT
tail->next == NULL
```

B.

```
TEXT
tail->next == head
```

Correct:

B

PROGRESS

MODULE 3 — LINKED LISTS

' Lesson 29 — Intuition, Nodes & Memory ' Lesson 30 — Singly Linked List Fundamentals ' Lesson 31 — Insertion ' Lesson 32 — Deletion ' Lesson 33 — Reversal ' Lesson 34 — Doubly Linked Lists ' **Lesson 35 — Circular Linked Lists**

NEXT — LESSON 36

Linked-List Placement Problems — Middle, Floyd Cycle Detection, nth From End, Merge, Intersection & Palindrome

Next we complete the Linked Lists module with the highest-value interview patterns:

- slow and fast pointers
- find middle in one pass
- detect cycle using Floyd
- why fast catches slow
- find cycle starting node
- nth node from end
- two-pointer gap method

- merge two sorted linked lists
- intersection of two linked lists
- palindrome checking
- reversal as a building block
- exact time/space analysis
- GATE traps involving pointer meeting and list lengths

Then we begin:

MODULE 4 — STACK

with **Lesson 37 — Stack Fundamentals, LIFO, Push/Pop/Peek, Overflow & Underflow.**

USER

next

ASSISTANT

MODULE 3 — LESSON 36

Linked-List Placement Problems — Middle, Floyd Cycle Detection, nth From End, Merge, Intersection & Palindrome

This is the **placement-heavy final lesson of Linked Lists.**

You already know:

- nodes
- traversal
- insertion
- deletion
- reversal
- doubly linked lists
- circular linked lists

Now we use those basics to solve the most common linked-list interview patterns.

GATE importance: +IMPORTANT Placement importance: 0=Y%04USDKW

1. TODAY'S HIGH-VALUE PATTERNS

We will cover:

1. Find middle node
2. Slow/Fast pointer pattern
3. Detect cycle — Floyd's algorithm
4. Find cycle starting node

5. nth node from end
6. Merge two sorted linked lists
7. Intersection of two linked lists
8. Palindrome linked list
9. Pattern-recognition shortcuts

2. PATTERN1 — FIND MIDDLE NODE

Suppose:

TEXT

1 0 !' 2 0 !' 3 0 !' 4 0 !' 5 0 !' N U L L

Middle:

TEXT

3 0

Simple method:

1. count nodes
2. traverse again to middle

Time:

TEXT

$O(n) + O(n) = O(n)$

But it needs two passes.

Can we find middle in **one traversal**?

Yes.

Use:

Slow and Fast Pointers

3. SLOW / FAST IDEA

Create:

C

```
Node *slow = head;
Node *fast = head;
```

Move:

TEXT

s l o w !' 1 n o d e a t a t i m e
f a s t !' 2 n o d e s a t a t i m e

When fast reaches the end:

```
TEXT
slow will be around the middle
```

Memory trick:

FAST runs twice; SLOW reaches halfway.

4. TRACE — ODD NUMBER OF NODES

List:

```
TEXT
1  !'  2  !'  3  !'  4  !'  5  !'  N U L L
```

Initially:

```
TEXT
slow = 1
fast = 1
```

After one iteration:

```
TEXT
slow = 2
fast = 3
```

After second:

```
TEXT
slow = 3
fast = 5
```

Next fast cannot move two steps.

So middle:

3

5. CODE

```
C
Node *middleNode(Node *head)
{
    Node *slow = head;
    Node *fast = head;

    while (fast != NULL && fast->next != NULL)
    {
```



```

        slow = slow->next;
        fast = fast->next->next;
    }

    return slow;
}

```

Time:

TEXT
 $O(n)$

Auxiliary space:

TEXT
 $O(1)$

6. WHY THE LOOP CONDITION?

We write:

```

C
while (fast != NULL && fast->next != NULL)

```

because inside:

```

C
fast = fast->next->next;

```

we need:

TEXT
fast
and
fast->next

to be valid.

Short-circuit evaluation also matters.

If `fast == NULL`, C will not evaluate:

```

C
fast->next

```

because of `&&`.

7. EVEN NUMBER OF NODES

Suppose:

TEXT

1 !' 2 !' 3 !' 4 !' N U L L

Trace:

TEXT

start:
slow=1
fast=1

iteration 1:
slow=2
fast=3

iteration 2:
slow=3
fast=NULL

Returned:

TEXT

3

There are two middle candidates:

TEXT

2 and 3

This implementation returns the:

second middle

0=1/4
Interview trap.

8. FIRST MIDDLE VS SECOND MIDDLE

For:

TEXT

1 !' 2 !' 3 !' 4

Standard:

C

slow = head;
fast = head;

often returns:

TEXT

3

second middle.

If a problem explicitly asks for first middle, the initialization/condition can be adjusted.

Do not assume wording.

9. PATTERN RECOGNITION

Whenever you see:

- middle of linked list
- split linked list
- palindrome list
- cycle detection

think:

SLOW + FAST

10. PATTERN 2 — CYCLE DETECTION $\emptyset = Y\% \emptyset = Y\% \emptyset = Y\%$

Normal list:

```
TEXT
1  !'  2  !'  3  !'  4  !'  N U L L
```

No cycle.

But suppose:

```
TEXT
1  !'  2  !'  3  !'  4
           !'      !"
           % % % % % %
```

Node 4 points back to node 3.

Now traversal:

```
C
while (temp != NULL)
```

will never terminate.

We need to detect whether a cycle exists.

11. BRUTE-FORCE IDEA

One option:

Remember every node address already visited.

If an address repeats:

```
TEXT
cycle exists
```

This can use a hash set:

```
TEXT
Time  = O(n)
Space = O(n)
```

But we can do it with:

```
TEXT
O(1) extra space
```

using Floyd's algorithm.

12. FLOYD'S CYCLE DETECTION

Also called:

Tortoise and Hare Algorithm

Use:

```
TEXT
slow !' 1 step
fast !' 2 steps
```

If no cycle:

```
TEXT
fast eventually reaches NULL
```

If cycle exists:

```
TEXT
fast eventually meets slow
```

$\emptyset = \text{NULL}$ must know.

13. CODE

```
C
int hasCycle(Node *head)
{
    Node *slow = head;
    Node *fast = head;
```

```

while (fast != NULL && fast->next != NULL)
{
    slow = slow->next;
    fast = fast->next->next;

    if (slow == fast)
        return 1;
}

return 0;
}

```

Time:

TEXT
 $O(n)$

Space:

TEXT
 $O(1)$

14. WHY DOES FAST CATCH SLOW?

Imagine both enter a circular track.

Every iteration:

TEXT
slow advances 1
fast advances 2

So relative to slow, fast gains:

TEXT
1 node per iteration

Inside a finite cycle, the relative position cannot avoid repeating forever.

Eventually:

TEXT
fast == slow

Think of runners on a circular track.

0-1% The key is **relative speed**, not absolute speed.

15. SIMPLE CYCLE TRACE

Cycle:

TEXT

```
1 !' 2 !' 3 !' 4 !' 5
      !'
      % % % % % % % % %
```

Suppose cycle is:

TEXT

```
3 !' 4 !' 5 !' 3
```

Eventually slow and fast both enter this loop.

Fast keeps gaining one position per iteration.

Therefore they meet.

16. NO-CYCLE CASE

List:

TEXT

```
1 !' 2 !' 3 !' 4 !' N U L L
```

Fast moves:

TEXT

```
1 !' 3 !' N U L L
```

or reaches a node whose `next` is NULL.

So loop ends.

Return:

TEXT

```
0
```

17. IMPORTANT DISTINCTION

A circular linked list intentionally has a cycle.

A normal linked list may accidentally have a cycle.

Floyd detects:

TEXT

```
cycle existence
```

It does not care whether the cycle was intentional.

18. FLOYD COMPLEXITY

At most linear number of pointer moves relative to number of reachable nodes.

So:

TEXT

Time = $O(n)$

Only two main pointers:

TEXT

slow
fast

Thus:

TEXT

Space = $O(1)$

Classic interview answer.

19. PATTERN3 — FIND START OF CYCLE

Detecting a cycle is one problem.

Another:

At which node does the cycle begin?

Example:

TEXT

```
1  !'  2  !'  3  !'  4  !'  5
           !'               !"
          % % % % % % % % % %
```

Cycle starts at:

TEXT

3

Floyd can find this too.

20. FLOYD PHASE 1

Run slow and fast until they meet inside cycle.

Suppose meeting occurs at some node.

Then:

Phase 2

Set one pointer back to:

TEXT

head

Keep the other at meeting point.

Now move both:

TEXT

one step at a time

Their next meeting point is:

cycle entry

Very famous result.

21. CODE — CYCLE START

```
c
Node *cycleStart(Node *head)
{
    Node *slow = head;
    Node *fast = head;

    while (fast != NULL && fast->next != NULL)
    {
        slow = slow->next;
        fast = fast->next->next;

        if (slow == fast)
        {
            slow = head;

            while (slow != fast)
            {
                slow = slow->next;
                fast = fast->next;
            }

            return slow;
        }
    }

    return NULL;
}
```

22. WHY DOES PHASE 2 WORK?

Let:

TEXT

```
¼ = distance from head to cycle start
» = cycle length
```

When slow and fast meet, their traveled distances satisfy a relationship involving complete cycle laps.

The mathematical consequence is:

Distance from head to cycle entry equals an appropriate forward distance from meeting point to cycle entry modulo the cycle length.

Therefore:

TEXT

```
pointer from head
+
pointer from meeting point
```

moving one step together meet at entry.

For GATE, understand the result more than memorizing the full proof unless a question asks for it.

23. PATTERN4 — nth NODE FROM END

Suppose:

TEXT

```
1 0 !' 2 0 !' 3 0 !' 4 0 !' 5 0 !' N U L L
```

Find:

TEXT

```
2nd node from end
```

Answer:

TEXT

```
40
```

Naive:

1. count length
2. convert to forward position
3. traverse again

Two passes.

Can we do one pass?

Yes.

24. TWO-POINTER GAP METHOD

Use two pointers:

```
TEXT
first
second
```

Create a gap of `n` nodes.

Then move both together.

When `first` reaches NULL:

```
TEXT
second points to nth node from end
```

Memory trick:

Make a gap, then march together.

25. EXAMPLE — 2nd FROM END

List:

```
TEXT
1 0 !'  2 0 !'  3 0 !'  4 0 !'  5 0
```

Advance `first` by 2 steps:

```
TEXT
first !'  3 0
second !'  1 0
```

Now move both:

```
TEXT
first !'  4 0
second !'  2 0
```

Then:

```
TEXT
first !'  5 0
second !'  3 0
```

Then:

```
TEXT
first !'  NULL
second !'  4 0
```

Answer:

40

26. CODE

Using `n=1` for last node:

```
C
Node *nthFromEnd(Node *head, int n)
{
    if (n <= 0)
        return NULL;

    Node *first = head;
    Node *second = head;

    for (int i = 0; i < n; i++)
    {
        if (first == NULL)
            return NULL;

        first = first->next;
    }

    while (first != NULL)
    {
        first = first->next;
        second = second->next;
    }

    return second;
}
```

27. EDGE CASES

If:

```
TEXT
n = 1
```

returns last node.

If:

```
TEXT
n = length
```

returns head.

If:

```
TEXT
n > length
```

invalid:

```
TEXT
NULL
```

If:

```
TEXT
n <= 0
```

also reject under this convention.

28. COMPLEXITY

`first` traverses at most `n` nodes.

Then both move through remainder.

Overall:

```
TEXT
O(length)
```

So:

```
TEXT
Time = O(n_list)
Space = O(1)
```

One pass after establishing gap.

29. REMOVE nth NODE FROM END

Very common placement problem.

Example:

```
TEXT
1 !' 2 !' 3 !' 4 !' 5
```

Remove 2nd from end:

```
TEXT
4
```

Result:

```
TEXT
1 !' 2 !' 3 !' 5
```

The same gap technique can place one pointer at predecessor.

A dummy node is often useful for handling deletion of head.

Very important interview extension.

30. DUMMY / SENTINEL NODE IDEA

Suppose deletion might remove head.

Instead of special-casing head repeatedly, create:

TEXT

d u m m y ! ' h e a d

Example:

TEXT

d u m m y ! ' 1 ! ' 2 ! ' 3

Then even the real head has a predecessor:

TEXT

dummy

This simplifies many linked-list algorithms.

Very useful placement technique.

31. PATTERN5 — MERGE TWO SORTED LINKED LISTS

Given:

TEXT

List A:

1 ! ' 3 ! ' 5 ! ' N U L L

List B:

2 ! ' 4 ! ' 6 ! ' N U L L

Merge into:

TEXT

1 ! ' 2 ! ' 3 ! ' 4 ! ' 5 ! ' 6 ! ' N U L L

This is similar to the merge step of merge sort.

32. CORE IDEA

Compare current nodes:

TEXT

A current
B current

Choose smaller node.

Append it to result.

Advance only the pointer from the chosen list.

Repeat.

33. DUMMY NODE MAKES THIS EASY

Create:

```
C
Node dummy;
Node *tail = &dummy;

dummy.next = NULL;
```

Then `tail` always points to last node of merged list.

At end:

```
TEXT
merged head = dummy.next
```

The dummy is not part of final logical data.

34. MERGE CODE

```
C
Node *mergeSorted(Node *a, Node *b)
{
    Node dummy;
    dummy.next = NULL;

    Node *tail = &dummy;

    while (a != NULL && b != NULL)
    {
        if (a->data <= b->data)
        {
            tail->next = a;
            a = a->next;
        }
        else
        {
            tail->next = b;
            b = b->next;
        }

        tail = tail->next;
    }
}
```

```

if (a != NULL)
    tail->next = a;
else
    tail->next = b;

return dummy.next;
}

```

35. WHY NO NEW NODES ARE REQUIRED?

We can reuse existing nodes by changing their:

TEXT
next

links.

So aside from a few pointer variables:

TEXT
Auxiliary space = $O(1)$

for this iterative merge.

No need to copy all data.

36. COMPLEXITY OF MERGE

If lengths are:

TEXT
m
n

Each node is processed at most once.

Therefore:

TEXT
Time = $O(m+n)$

Auxiliary:

TEXT
 $O(1)$

excluding the existing nodes.

~~$O(M+N)$~~ Must know.

37. WHY APPEND REMAINDER DIRECTLY?

Suppose A becomes NULL first.

Remaining B is already sorted:

TEXT

7 !' 8 !' 9

There is no need to compare those nodes further.

Simply:

C

```
tail->next = b;
```

Done.

38. RECURSIVE MERGE — PREVIEW

Recursive version:

C

```
Node *merge(Node *a, Node *b)
{
    if (a == NULL)
        return b;

    if (b == NULL)
        return a;

    if (a->data <= b->data)
    {
        a->next = merge(a->next, b);
        return a;
    }
    else
    {
        b->next = merge(a, b->next);
        return b;
    }
}
```

Time:

TEXT

$O(m+n)$

Stack:

TEXT

$O(m+n)$

worst case.

Iterative version uses:

TEXT

0(1)

auxiliary space.

39. PATTERN6 — INTERSECTION OF TWO LINKED LISTS Ø=Ý%

Suppose two lists eventually share the same nodes:

TEXT

```
A:
 1 !'  2 !'  8 !'  9 !'  N U L L
      |
B:
 5 !'  6 % % % %
```

The intersection begins at node:

TEXT

8

Important:

Intersection means same node object/address, not merely same data value.

Ø=Ý%y important.

40. SAME VALUE "INTERSECTION

Suppose:

TEXT

```
A :  1 !'  [ 5 ] !'  8
B :  2 !'  [ 5 ] !'  9
```

If those two nodes storing 5 have different addresses:

TEXT

not an intersection

Even though values are equal.

Intersection asks:

TEXT

pointer equality

not data equality.

41. LENGTH-DIFFERENCE METHOD

Suppose:

TEXT

```
length A = m  
length B = n
```

If one list is longer, advance its pointer by:

TEXT

```
|m-n|
```

nodes.

Now both pointers have equal distance to the end.

Move both one step together.

First point where:

c

```
p == q
```

is intersection.

42. EXAMPLE

A:

TEXT

```
1  !'  2  !'  3  !'  8  !'  9
```

B:

TEXT

```
5  !'  8  !'  9
```

A is longer by:

TEXT

```
2
```

Advance A pointer:

TEXT

```
A  p o i n t e r !'  3  
B  p o i n t e r !'  5
```

Now move together:

TEXT

3 vs 5
8 vs 8

Intersection:

TEXT

8

43. COMPLEXITY

Need lengths:

TEXT

$O(m+n)$

Then aligned traversal:

TEXT

$O(m+n)$

Overall:

TEXT

$O(m+n)$

Space:

TEXT

$O(1)$

44. POINTER-SWITCHING TRICK Ø=Þ€

A beautiful method avoids explicit length calculation.

Set:

C

```
p = headA;  
q = headB;
```

Then repeatedly:

C

```
p = (p == NULL) ? headB : p->next;  
q = (q == NULL) ? headA : q->next;
```

Eventually:

TEXT

$p == q$

at intersection or both NULL.

Why?

Each pointer walks:

TEXT

$A + B$

total distance.

So length difference cancels out.

Very common interview trick.

45. PATTERN7 — PALINDROME LINKED LIST

Suppose:

TEXT

1 !' 2 !' 3 !' 2 !' 1

Palindrome?

Yes.

Simplest method:

- copy values into array
- compare from ends

Time:

TEXT

$O(n)$

Extra space:

TEXT

$O(n)$

Can we do:

TEXT

$O(1)$

extra space?

Yes.

Use multiple linked-list patterns together.

46. PALINDROME — O(1) EXTRA-SPACE STRATEGY

Steps:

1. Find middle using slow/fast.
2. Reverse second half.
3. Compare first half with reversed second half.
4. Optionally restore second half.

~~O(1)~~ This shows why middle + reversal are essential building blocks.

47. EXAMPLE

Original:

TEXT

1 !' 2 !' 3 !' 2 !' 1

Middle:

TEXT

3

Second half:

TEXT

2 !' 1

Reverse second half:

TEXT

1 !' 2

Compare:

TEXT

f	i	r	s	t		h	a	l	f	:			1	!	'		2	
r	e	v	e	r	s	e	d		h	a	l	f	:		1	!	'	2

Equal.

Palindrome.

48. EVEN-LENGTH EXAMPLE

TEXT

1 !' 2 !' 2 !' 1

Standard slow/fast method may place slow at second half:

TEXT

s l o w ! ' t h i r d n o d e (2)

Reverse:

TEXT

2 ! ' 1

Compare with first:

TEXT

1 ! ' 2

after correct boundary handling.

The exact implementation must define where second half begins carefully.

49. PALINDROME IMPLEMENTATIONIDEA

```
C
int isPalindrome(Node *head)
{
    if (head == NULL || head->next == NULL)
        return 1;

    Node *slow = head;
    Node *fast = head;

    while (fast != NULL && fast->next != NULL)
    {
        slow = slow->next;
        fast = fast->next->next;
    }

    Node *second = reverseList(slow);
    Node *first = head;
    Node *check = second;

    while (check != NULL)
    {
        if (first->data != check->data)
        {
            reverseList(second);
            return 0;
        }

        first = first->next;
        check = check->next;
    }

    reverseList(second);

    return 1;
}
```

This illustrates the pattern; exact split/restoration style can vary.

50. PALINDROME COMPLEXITY

Find middle:

TEXT

$O(n)$

Reverse half:

TEXT

$O(n)$

Compare:

TEXT

$O(n)$

Restore:

TEXT

$O(n)$

Overall:

TEXT

$O(n)$

because constants are ignored.

Auxiliary:

TEXT

$O(1)$

for iterative reversal.

$O(1)$ important.

51. PATTERN8 — FIND Kth NODE FROM START

This one is simple:

```
C
Node *kth(Node *head, int k)
{
    Node *temp = head;

    for (int i = 0; i < k && temp != NULL; i++)
        temp = temp->next;

    return temp;
}
```

For zero-based k .

Complexity:

TEXT
 $O(k)$

Worst:

TEXT
 $O(n)$

Compared with nth-from-end:

TEXT
two-pointer gap

is more interesting because list length may be unknown.

52. PATTERN9 — REMOVE DUPLICATES FROM SORTED LIST

Suppose:

TEXT
1 !' 1 !' 2 !' 2 !' 3 !' N U L L

Because duplicates are adjacent:

```
C
Node *curr = head;

while (curr != NULL && curr->next != NULL)
{
    if (curr->data == curr->next->data)
    {
        Node *temp = curr->next;
        curr->next = temp->next;
        free(temp);
    }
    else
    {
        curr = curr->next;
    }
}
```

Result:

TEXT
1 !' 2 !' 3

Time:

TEXT

$O(n)$

Space:

TEXT

$O(1)$

53. WHY DON'T WE ALWAYS MOVE CURR?

If duplicate is deleted:

TEXT

1 !' 1 !' 1 !' 2

after deleting one duplicate, the current node may still have another duplicate next to it.

So keep `curr` at same node and check again.

Move only when values differ.

Common pointer-loop logic.

54. PATTERN10 — REORDER LIST PREVIEW

Given:

TEXT

1 !' 2 !' 3 !' 4 !' 5

Want:

TEXT

1 !' 5 !' 2 !' 4 !' 3

Typical strategy:

1. find middle
2. reverse second half
3. merge alternating nodes

This is a very good example of combining known patterns.

55. PATTERN11 — MERGE SORT ON LINKED LIST

Linked lists are well-suited to merge sort.

Why?

1. Find middle using slow/fast.
2. Split list.

3. Recursively sort halves.
4. Merge sorted lists efficiently.

Typical:

TEXT

Time = $O(n \log n)$

Why merge sort rather than array-style random-access algorithms?

Linked lists do not provide cheap random indexing, but splitting/merging through links is natural.

Ø=Good placement and algorithm connection.

56. LINKED LIST VS ARRAY FOR BINARY SEARCH

Even if sorted, ordinary linked-list binary search isn't efficient because:

TEXT

middle access = $O(n)$

But merge sort works well because it mainly needs:

TEXT

sequential traversal
+
link merging

This highlights an important rule:

Algorithm choice depends on data-structure capabilities.

57. COMMON POINTER TECHNIQUES SUMMARY

One pointer

TEXT

Traversal/search

prev + curr

TEXT

Deletion

prev + curr + next

TEXT

Reversal

slow + fast

TEXT

Middle
Cycle detection
Palindrome

first + second with gap

TEXT

nth from end

Two current pointers + tail

TEXT

Merge two sorted lists

0-100% Copy this.

58. GATE TRAP — FAST CONDITION

Correct:

C

```
while (fast != NULL && fast->next != NULL)
```

Why order matters:

C evaluates left to right with short circuit.

Bad if you write/access:

C

```
fast->next
```

when `fast` may already be NULL.

59. GATE TRAP — MIDDLE OF EVEN LIST

Different implementations can return:

TEXT

first middle
or
second middle

For:

TEXT

```
1  !'  2  !'  3  !'  4
```

our standard initialization:

```
c
slow=head;
fast=head;
```

with the usual loop returns:

```
TEXT
3
```

second middle.

Read exact question.

60. GATE TRAP — CYCLE MEETING POINT

The node where slow and fast first meet is **not necessarily** the cycle start.

Ø=Very important.

Phase 1:

```
TEXT
detect meeting
```

Phase 2:

```
TEXT
find cycle entry
```

Do not confuse them.

61. GATE TRAP — INTERSECTION

Compare:

```
c
p == q
```

not:

```
c
p->data == q->data
```

Intersection is based on same node address.

Two nodes can contain identical data without intersecting.

62. GATE TRAP — nth FROM END OFF-BY-ONE

If `n=1` means last node:

Create gap of exactly:

TEXT
`n nodes`

between `first` and `second`.

Test manually on a small list to avoid off-by-one errors.

63. GATE TRAP — MERGE COMPLEXITY

For lengths:

TEXT
`m and n`

merging sorted lists is:

TEXT
`O(m+n)`

not:

TEXT
`O(mn)`

because each pointer only moves forward.

64. GATE TRAP — PALINDROME

Reversing only the second half gives:

TEXT
`O(1)`

auxiliary pointer space.

But if recursive reversal is used:

TEXT
`O(n)`

stack space may appear.

Implementation choice matters.

65. PLACEMENT PATTERNMAP Ø=Ý%Ø=Ý%Ø=Ý%

When you see:

“Middle of linked list”

Think:

TEXT

Slow + Fast

“Cycle?”

Think:

TEXT

Floyd Slow + Fast

“Cycle starting point?”

Think:

TEXT

D e t e c t m e e t i n g ! ' r e s e t o n e t o h e a d ! ' b o t h m o v e 1

“nth from end”

Think:

TEXT

Two pointers with n-gap

“Merge sorted lists”

Think:

TEXT

Two current pointers + dummy/tail

“Intersection”

Think:

TEXT

Align lengths
or pointer switching

“Palindrome”

Think:

TEXT

Middle + Reverse second half + Compare

0-70% is your linked-list interview map.

66. MASTER COMPLEXITY TABLE

Problem	Time	Extra Space
Find middle	$O(n)$	$O(1)$
Detect cycle — Floyd	$O(n)$	$O(1)$
Find cycle start	$O(n)$	$O(1)$
nth from end	$O(n)$	$O(1)$
Merge sorted lists	$O(m+n)$	$O(1)$ iterative
Intersection	$O(m+n)$	$O(1)$
Palindrome optimized	$O(n)$	$O(1)$
Remove sorted duplicates	$O(n)$	$O(1)$

$O(1)$ — high-value revision table.

67. INTERVIEW QUESTIONS

Q1. How to find middle in one pass?

Slow moves one, fast moves two.

Q2. Floyd cycle detection complexity?

TEXT

$O(n)$ time
 $O(1)$ space

Q3. Is first slow-fast meeting necessarily cycle start?

No.

Q4. How to find nth node from end?

Advance one pointer n steps, then move both until first reaches NULL.

Q5. What does linked-list intersection compare?

Node addresses/pointers, not values.

Q6. How can palindrome be checked with $O(1)$ extra space?

Find middle, reverse second half, compare halves.

68. MEMORY TRICKS

Middle

FAST finishes; **SLOW** is halfway.

Cycle

On a circle, the faster runner catches the slower runner.

Cycle entry

Meet !' reset !' walk together.

nth from end

MAKE GAP !' MOVE PAIR.

Merge

Pick smaller, move that list.

Intersection

Same NODE, not same NUMBER.

Palindrome

HALF !' FLIP !' COMPARE.

69. NOTEBOOK NOTES

Middle

```
C
slow = head;
fast = head;

while (fast != NULL && fast->next != NULL)
{
    slow = slow->next;
    fast = fast->next->next;
}
```

Cycle

```
C
if (slow == fast)
    cycle exists;
```

after movements.

nth From End

TEXT

```
first ahead by n
move first + second together
when first == NULL
second = answer
```

Merge

TEXT

```
compare a->data and b->data
attach smaller
advance selected list
```

TEXT

$O(m+n)$

70. 30-SECOND REVISION

Remember:

1. Slow/fast finds middle.
2. Standard even-length version often returns second middle.
3. Floyd detects cycles in $O(1)$ extra space.
4. Meeting point isn't necessarily cycle start.
5. To find entry: reset one pointer to head, move both one step.
6. nth from end uses a fixed pointer gap.
7. Sorted-list merge is $O(m+n)$.
8. Intersection means pointer equality.
9. Palindrome = middle + reverse + compare.
10. Most powerful linked-list tricks are just smart pointer movement.

71. ACTIVE RECALL

Q1

In Floyd's algorithm, how fast do slow and fast move?

Q2

What is the loop condition commonly used for middle/cycle algorithms?

Q3

If the first slow-fast meeting happens inside a cycle, is it guaranteed to be cycle entry?

Q4

What gap is created to find nth node from end?

Q5

What three patterns combine to solve palindrome efficiently?

ANSWERS

A1

TEXT

```
slow = 1 step  
fast = 2 steps
```

A2

C

```
fast != NULL && fast->next != NULL
```

A3

No.

A4

n nodes, if n=1 denotes the last node.

A5

TEXT

```
Find middle  
Reverse second half  
Compare halves
```

72. PRACTICE — MIDDLE

For:

TEXT

```
1 0 !' 2 0 !' 3 0 !' 4 0 !' 5 0 !' N U L L
```

Trace slow and fast and identify middle.

Then repeat for:

TEXT

```
1 0 !' 2 0 !' 3 0 !' 4 0 !' N U L L
```

Which middle does our standard code return?

73. PRACTICE — CYCLE

Suppose:

TEXT

```
1 !' 2 !' 3 !' 4 !' 5
      !'
      % % % % % % % % %
```

Answer:

1. Does a normal `while(temp != NULL)` terminate?
2. Will Floyd detect the cycle?
3. Is the first slow-fast meeting guaranteed to be node 3?

74. PRACTICE — nth FROM END

List:

TEXT

```
1 !' 2 !' 3 !' 4 !' 5 !' 6 !' N U L L
```

Find:

TEXT

3rd node from end

using the two-pointer gap method.

75. PRACTICE — MERGE

Merge:

TEXT

```
A : 1 !' 4 !' 7
B : 2 !' 3 !' 8
```

Write the final merged sequence.

Also state complexity.

76. PRACTICE — INTERSECTION

Suppose:

TEXT

```
A : 1 0 !' 2 0 !' X !' Y !' N U L L
B : 5 !' X !' Y !' N U L L
```

where `X` is literally the same node object in both lists.

At what node do they intersect?

What if B contains another separate node whose data equals `X->data` but address differs?

77. PRACTICE — PALINDROME

Determine whether:

TEXT

1 !' 2 !' 3 !' 2 !' 1

is palindrome using:

TEXT

middle
reverse second half
compare

Then try:

TEXT

1 !' 2 !' 3 !' 4 !' N U L L

78. GATE-STYLE TRAP

For:

TEXT

1 !' 2 !' 3 !' 4

using:

C

```
slow = head;
fast = head;

while (fast != NULL && fast->next != NULL)
{
    slow = slow->next;
    fast = fast->next->next;
}
```

What does slow point to?

Answer:

3

The second middle.

79. PLACEMENT MINI-CHALLENGE

Identify only the technique.

A

"Find middle without calculating length."

B

"Check whether a linked list loops forever."

C

"Find 4th node from the end in one pass."

D

"Combine two sorted linked lists."

E

"Check whether two lists eventually share nodes."

F

"Check if list values read the same forward/backward with O(1) extra space."

Answers

TEXT

```
A !' S l o w / F a s t
B !' F l o y d
C !' T w o - p o i n t e r   g a p
D !' M e r g e   w i t h   t w o   l i s t   p o i n t e r s
E !' L e n g t h   a l i g n m e n t   /   p o i n t e r   s w i t c h i n g
F !' M i d d l e   +   r e v e r s e   +   c o m p a r e
```

MODULE 3 — LINKED LISTS COMPLETE '

You have now covered:

' Node structure ' head , next , NULL ' Traversal ' Search ' Length ' Singly linked list ' Insertion ' Deletion ' Reversal ' Doubly linked list ' Circular linked list ' Slow/Fast pointers ' Middle ' Floyd cycle detection ' nth from end ' Sorted merge ' Intersection ' Palindrome

NEXT — MODULE 4 / LESSON 37

Stack Fundamentals — LIFO, Push, Pop, Peek, Overflow & Underflow

We now start another official GATE topic:

STACK

Core idea:

TEXT

TOP

!"

+-----+

| 4 0 | !• l a t e s t i n s e r t e d

+-----+

| 30 |

+-----+

| 20 |

+-----+

| 10 |

+-----+

Rule:

LIFO — Last In, First Out

We will learn:

- stack ADT
- push
- pop
- peek/top
- overflow
- underflow
- array representation
- top
- exact operation complexities
- stack vs queue
- real-life intuition
- recursion/call-stack connection
- GATE sequences and valid/invalid stack operations

Then Lesson 38 will implement a **Stack Using Array** in C.

USER

next

ASSISTANT

MODULE 4 — LESSON 37

Stack Fundamentals — LIFO, Push, Pop, Peek, Overflow & Underflow

We now begin another official GATE topic:

STACK

It appears in:

- GATE importance:** Ø=Ý%ØÙ\$ÚÓÏØW **Placement importance:** Ø=Ý%ØÙ\$ÚÓÏØW

A stack follows:

Think of a pile of plates.

You place plates on top:

Plate 1
Plate 2
Plate 3

The last plate placed:

Plate 3

is the first one removed.

That is LIFO.

Suppose we insert:

10
20
30
40

Stack becomes:

TEXT

```
      TOP
      ! "
+-----+
|  40  |
+-----+
|  30  |
+-----+
|  20  |
+-----+
|  10  |
+-----+
```

The only end where normal insertion/deletion happens is:

TOP

3. STACK ADT

A stack is an:

Abstract Data Type

The essential operations are:

TEXT

```
push
pop
peek / top
isEmpty
```

Sometimes also:

TEXT

```
isFull
size
```

4. PUSH

`push(x)` means:

Insert `x` on top of the stack.

Before:

TEXT

```
TOP
! "
30
20
10
```


Push:

```
TEXT
40
```

After:

```
TEXT
TOP
! "
40
30
20
10
```

5. POP

`pop()` means:

Remove the top element.

Before:

```
TEXT
TOP
! "
40
30
20
10
```

Pop removes:

```
TEXT
40
```

After:

```
TEXT
TOP
! "
30
20
10
```

~~Only~~ Only the top element is directly removed.

6. PEEK / TOP

`peek()` means:

Read the current top value without removing it.

Stack:

```
TEXT
TOP
! "
30
20
10
```

Then:

```
TEXT
peek() = 30
```

Stack remains unchanged.

7. PUSH VS POP VS PEEK

Operation	Meaning
push(x)	Add x
pop()	Remove top
peek()	Read top
isEmpty()	Check whether stack has no elements

Ø = 0 is fundamental.

8. SIMPLE OPERATION TRACE

Initially:

```
TEXT
empty
```

Operations:

```
TEXT
push(10)
push(20)
push(30)
pop()
push(40)
```

Trace:

TEXT

```
push(10)
10
```

TEXT

```
push(20)
20
10
```

TEXT

```
push(30)
30
20
10
```

TEXT

```
pop()
removed 30

20
10
```

TEXT

```
push(40)

40
20
10
```

Final top:

TEXT

```
40
```

9. STACK ORDER TRICK

If values are pushed:

TEXT

```
A
B
C
D
```

in that order,

then normal popping gives:

TEXT

```
D
C
B
```

A

So a stack naturally reverses order.

0-16 This is why stacks are useful for reverse-like operations.

10. REAL-LIFE INTUITION

Imagine books stacked:

TEXT

Book D
Book C
Book B
Book A

To reach Book A at the bottom, you must first remove:

TEXT

D
C
B

This means a stack does **not** provide arbitrary removal efficiently.

Only top is directly accessible.

11. STACK VS ARRAY

An array is a storage structure.

A stack is an access discipline.

You can implement a stack using:

- array
- linked list

So:

TEXT

S T A C K " ` A R R A Y

Rather:

TEXT

Array can IMPLEMENT Stack

0-16 Important conceptual distinction.

12. STACK USING ARRAY — PREVIEW

Suppose:

```
c
int stack[5];
```

We maintain:

```
c
int top;
```

Typical initialization:

```
c
top = -1;
```

Why `-1`?

Because no valid element exists yet.

13. TOP VARIABLE

For stack:

```
TEXT
10 20 30
```

stored:

```
TEXT
i n d e x   0   !'   1 0
i n d e x   1   !'   2 0
i n d e x   2   !'   3 0
```

Then:

```
TEXT
top = 2
```

So:

```
TEXT
stack[top] = 30
```

14. EMPTY STACK CONDITION

If:

```
TEXT
top = -1
```

stack is empty.

FORMULA BOX

TEXT

Empty Stack:
 $\text{top} == -1$

Remember for array-based stack.

15. FULL STACK CONDITION

Suppose capacity:

TEXT

N

Valid indices:

TEXT

$0 \dots N-1$

If:

TEXT

$\text{top} == N-1$

stack is full.

FORMULA BOX

TEXT

Full Stack:
 $\text{top} == \text{capacity} - 1$

16. PUSH IN ARRAY STACK

Conceptually:

C

```
top++;  
stack[top] = value;
```

Or compactly:

C

```
stack[++top] = value;
```

But for learning, prefer explicit steps.

17. PUSH TRACE

Capacity:

```
TEXT
5
```

Initially:

```
TEXT
top = -1
```

Push 10:

```
TEXT
top = 0
stack[0] = 10
```

Push 20:

```
TEXT
top = 1
stack[1] = 20
```

Push 30:

```
TEXT
top = 2
stack[2] = 30
```

18. POP IN ARRAY STACK

To remove top:

```
C
value = stack[top];
top--;
```

Example:

```
TEXT
top = 2
stack[2] = 30
```

Pop:

```
TEXT
value = 30
top = 1
```

Logical stack now contains:

```
TEXT
10 20
```

19. PHYSICAL VALUE MAY STILL REMAIN

After:

```
TEXT
top--
```

the old memory slot may physically still contain:

```
TEXT
30
```

Example:

```
TEXT
stack[2] = 30
```

But logical stack has:

```
TEXT
top = 1
```

So index 2 is no longer considered part of the stack.

☞ Some physical-vs-logical-size idea from arrays.

20. PEEK IN ARRAY STACK

If stack is not empty:

```
c
stack[top]
```

returns top value.

No change to `top`.

Example:

```
TEXT
top = 2
```

then:

TEXT

```
peek = stack[2]
```

21. STACK UNDERFLOW Ø=Ý%

What happens if we try:

TEXT

```
pop()
```

from an empty stack?

Example:

TEXT

```
top = -1
```

There is no valid top element.

This condition is called:

STACK UNDERFLOW

22. UNDERFLOW CONDITION

For array stack:

TEXT

```
top == -1
```

and operation attempts:

TEXT

```
pop  
or  
peek
```

Then:

TEXT

UNDERFLOW

Ø=Ý% terminology.

23. STACK OVERFLOW Ø=Ý%

Suppose capacity is:

TEXT

5

and:

TEXT

top = 4

All positions filled.

Trying another push causes:

STACK OVERFLOW

24. OVERFLOW CONDITION

For fixed-capacity array stack:

TEXT

top == capacity - 1

and:

TEXT

push attempted

Then:

TEXT

OVERFLOW

25. DON'T CONFUSE OVERFLOW TYPES

"Stack overflow" can mean two related but different contexts.

Data-structure stack overflow

Trying to push into a full fixed-size stack.

Call-stack overflow

Too many nested function/recursive calls exhaust runtime call-stack memory.

Same broad idea—stack capacity exhausted—but context differs.

26. BASIC PUSH ALGORITHM

TEXT

PUSH(x)

```
if top == capacity-1
    Overflow
else
    top = top + 1
    stack[top] = x
```

Time:

TEXT

$O(1)$

27. BASIC POP ALGORITHM

TEXT

POP()

```
if top == -1
    Underflow
else
    x = stack[top]
    top = top - 1
    return x
```

Time:

TEXT

$O(1)$

28. BASIC PEEK ALGORITHM

TEXT

PEEK()

```
if top == -1
    Underflow
else
    return stack[top]
```

Time:

TEXT

$O(1)$

29. STACK OPERATION COMPLEXITIES

For standard stack:

Operation	Time
Push	$O(1)$
Pop	$O(1)$
Peek	$O(1)$
isEmpty	$O(1)$
isFull	$O(1)$
Search arbitrary value	$O(n)$

$O=1$ Must know.

30. WHY SEARCH IS $O(n)$

A stack ADT only guarantees direct access to:

TEXT
top

If you want some arbitrary value lower down, you may need to inspect multiple elements.

So:

TEXT
search = $O(n)$

worst case.

31. LIFO EXAMPLE

Push:

TEXT
1 2 3 4 5

Pop sequence:

TEXT
5 4 3 2 1

A stack reverses the insertion order.

This leads to many GATE questions involving possible output sequences.

32. VALID STACK POP SEQUENCE — INTUITION

Suppose input values are pushed in order:

TEXT

1 2 3

Could output be:

TEXT

3 2 1

Yes.

Push all:

TEXT

1
2
3

then pop all.

33. COULD OUTPUT BE 2 1 3?

Push:

TEXT

1
2

Pop:

TEXT

2
1

Then push 3 and pop:

TEXT

3

So output:

TEXT

2 1 3

is valid.

Stack output does not have to be fully reversed.

Pushes and pops can interleave.

34. INVALID SEQUENCE EXAMPLE

Input push order:

TEXT

1 2 3

Could pop order be:

TEXT

3 1 2

Let's test.

Push all:

TEXT

1 2 3

Pop 3.

Now stack:

TEXT

2
1

Top is 2.

But desired next output is 1.

Cannot remove 1 without first removing 2.

Therefore:

TEXT

3 1 2

is impossible.

0 = 0% Classic stack-permutation reasoning.

35. STACK PERMUTATION RULE

At any moment:

Only the top element can be popped.

So if an element blocks another element beneath it, the lower element cannot come out first.

This is the key logic for GATE stack-sequence questions.

36. GATE-STYLE TRACE

Input arrives:

TEXT

1 2 3 4

Desired output:

TEXT

2 1 4 3

Possible?

Push 1:

TEXT

1

Push 2:

TEXT

2

1

Pop:

TEXT

2

Pop:

TEXT

1

Push 3:

TEXT

3

Push 4:

TEXT

4

3

Pop:

TEXT

4

Pop:

TEXT

3

Yes.

37. STACK AND RECURSION Ø=Ý%

When functions call each other, runtime uses a:

CALL STACK

Example:

```
TEXT
main
calls A
A calls B
B calls C
```

Active calls:

```
TEXT
TOP
! "
C
B
A
main
```

When C finishes:

```
TEXT
C pops
```

Then B resumes.

This is exactly LIFO.

38. WHY FUNCTION RETURNS ARE LIFO

If:

```
TEXT
A calls B
B calls C
```

C must finish before B can continue.

B must finish before A can continue.

So return order:

```
TEXT
C
B
A
```


which is reverse of call order:

TEXT

A
B
C

That is stack behavior.

39. ACTIVATION RECORDS

Each function call creates an activation record / stack frame containing things such as:

- parameters
- local variables
- return address
- saved execution state

Those frames are pushed onto the runtime call stack.

When function returns:

TEXT

frame is popped

~~Ø=Link~~ Links C/recursion lessons to stack data structure.

40. RECURSION EXAMPLE

For:

C

f(3)

calling:

TEXT

f(3)
! ' f (2)
! ' f (1)
! ' f (0)

call stack grows:

TEXT

f(3)
f(2)
f(1)
f(0)

Then unwinds:

TEXT

```
f(0)
f(1)
f(2)
f(3)
```

LIFO.

41. STACK OVERFLOW IN RECURSION

If recursion has no proper base case:

```
c
void f(void)
{
    f();
}
```

calls continue:

TEXT

```
f
f
f
f
...
```

Frames keep accumulating.

Eventually runtime stack memory may be exhausted.

This is:

Call Stack Overflow

42. STACK AND EXPRESSION EVALUATION

Stacks are heavily used for:

- parentheses matching
- infix to postfix conversion
- postfix evaluation
- prefix evaluation
- syntax parsing

Example:

TEXT

```
(a + b) * c
```

Operators may need temporary storage according to precedence.

Stacks provide the required LIFO behavior.

We will cover this deeply in Lessons 40–42.

43. STACK AND PARENTHESES

Expression:

```
TEXT
({[]})
```

When opening symbol appears:

```
TEXT
push it
```

When closing symbol appears:

```
TEXT
check/pop matching opening symbol
```

Why stack?

The most recently opened bracket must be closed first.

Example:

```
TEXT
(
{
[
```

must close:

```
TEXT
]
}
)
```

LIFO.

✓
Very natural stack application.

44. STACK AND UNDO

Suppose actions:

```
TEXT
A
B
C
```

Most recent action:

TEXT

C

should be undone first.

So:

TEXT

Undo stack

T O P ! ' C

B

A

Again LIFO.

45. BROWSER HISTORY — STACK INTUITION

If you visit:

TEXT

Page A

Page B

Page C

Back action conceptually accesses most recent page transition first.

Real browsers use richer structures, but stack behavior is a useful model.

46. DFS CONNECTION

Depth First Search follows:

Go deep, then backtrack.

That naturally uses:

- explicit stack

or

- recursion, whose call stack acts as a stack

We'll revisit during Graphs.

47. STACK VS QUEUE

Stack:

TEXT

LIFO

Queue:

TEXT
FIFO

Suppose insert:

TEXT
10
20
30

Stack removal

TEXT
30
20
10

Queue removal

TEXT
10
20
30

Ø = Fundamental distinction.

48. STACK TERMINOLOGY

Top

Current accessible end.

Push

Insert at top.

Pop

Remove top.

Peek

Read top without removal.

Overflow

Push into full fixed-capacity stack.

Underflow

Pop/read from empty stack.

49. STACK REPRESENTATION— ARRAY

Conceptually:

TEXT

```
index
0    [10]
1    [20]
2    [ 3 0 ] !• t o p
3    [  ]
4    [  ]
```

Then:

TEXT

```
top = 2
capacity = 5
```

50. STACK REPRESENTATION— LINKED LIST

We can also implement:

TEXT

```
top
!"
3 0 !' 2 0 !' 1 0 !' N U L L
```

Push at head:

TEXT

```
0(1)
```

Pop head:

TEXT

```
0(1)
```

This will be Lesson 39.

51. ARRAY STACK VS LINKED STACK — PREVIEW

Array stack

Advantages:

- simple
- cache-friendly
- no pointer overhead

Disadvantage:

- fixed capacity if static array

Linked stack

Advantages:

- dynamic growth

Disadvantages:

- pointer overhead
- dynamic allocation cost
- weaker cache locality

52. DOES LINKED STACK HAVE OVERFLOW?

Conceptually, a linked stack does not have a fixed N -element capacity like an array stack.

But allocation can fail because memory is finite.

So practical overflow may occur due to inability to allocate a new node.

Don't say "linked stack can never overflow" absolutely.

53. GATE TRAP — TOP INITIAL VALUE

For zero-based array stack:

```
TEXT
top = -1
```

means empty.

After first push:

```
TEXT
top = 0
```

After n pushes:

```
TEXT
top = n - 1
```

provided no pops.

54. STACK SIZE FROM TOP

For array stack with:

```
TEXT
top = -1
```

when empty,

number of elements is:

FORMULA BOX

TEXT

size = top + 1

Example:

TEXT

top = 3

then:

TEXT

size = 4

55. EMPTY/ONE/FULL STATES

Capacity 5.

Empty

TEXT

top = -1

size = 0

One element

TEXT

top = 0

size = 1

Full

TEXT

top = 4

size = 5

56. PUSH EFFECT ON TOP

Every successful push:

TEXT

top increases by 1

Every successful pop:

TEXT

top decreases by 1

So after arbitrary valid operations:

FORMULA BOX

TEXT

```
final size
=
initial size
+ successful pushes
- successful pops
```

Useful numerical shortcut.

57. GATE NUMERICAL EXAMPLE

Initially empty.

Operations:

TEXT

```
push
push
push
pop
push
push
pop
```

Successful pushes:

TEXT

5

Successful pops:

TEXT

2

Final size:

TEXT

5 - 2 = 3

So:

TEXT

top = 2

for zero-based array stack.

58. BUT WATCH UNDERFLOW

Suppose:

TEXT

pop
push

starting from empty.

First pop fails underflow.

If unsuccessful operations do not modify stack, it should not count as a successful pop.

So final size:

TEXT

1

not 0.

~~0 = GATE trap.~~

59. OVERFLOW ALSO DOES NOT CHANGE STACK

Capacity 3.

Already full:

TEXT

10
20
30

Attempt:

TEXT

push(40)

If overflow is detected correctly:

TEXT

stack remains unchanged

So:

TEXT

size = 3

60. GATE TRAP — POP RETURNS WHAT?

Suppose:

TEXT

10
20

```
3 0 !• t o p
```

Pop returns/removes:

```
TEXT
30
```

not 10.

LIFO.

Seems basic, but stack-operation traces often exploit this.

61. GATE TRAP — PEEK DOES NOT POP

Stack:

```
TEXT
30
20
10
```

Operations:

```
TEXT
peek
peek
```

Both return:

```
TEXT
30
```

Stack remains unchanged.

62. GATE TRAP — `top++` VS `++top`

Suppose:

```
TEXT
top = -1
```

For push, a common correct compact expression:

```
c
stack[++top] = value;
```

Why?

`++top` changes:

TEXT

```
- 1 !' 0
```

before indexing.

But:

C

```
stack[top++] = value;
```

would initially attempt:

TEXT

```
stack[-1]
```

before incrementing.

Wrong for this representation.

Ø=0% stack output trap.

63. POP COMPACT EXPRESSION

For valid nonempty stack:

C

```
return stack[top--];
```

This:

1. uses current `top`
2. then decrements it

Example:

TEXT

```
top = 2
```

returns:

TEXT

```
stack[2]
```

then:

TEXT

```
top = 1
```

This compact form matches pop semantics.

64. PRECEDENCE CONNECTION

Recall postfix:

```
c
top--
```

returns old value for the expression, then decrements.

So:

```
c
stack[top--]
```

indexes current top first.

Good for pop.

Meanwhile push commonly uses prefix:

```
c
stack[++top]
```

increment then use.

Memory trick:

PUSH: move top first. POP: use top first.

Very useful.

65. BASIC C STRUCTURE — PREVIEW

```
c
#define MAX 100

int stack[MAX];
int top = -1;
```

Push:

```
c
if (top == MAX - 1)
{
    /* overflow */
}
else
{
    top++;
    stack[top] = value;
}
```

Pop:

```

c
if (top == -1)
{
    /* underflow */
}
else
{
    value = stack[top];
    top--;
}

```

We'll implement this fully in Lesson 38.

66. STACK ADT VS IMPLEMENTATION

Abstractly, stack promises:

TEXT
push/pop/peek

It does not require:

TEXT
array

or:

TEXT
linked list

Those are implementation choices.

NOTE theory point:

ADT describes behavior/operations; implementation describes storage details.

67. STACK APPLICATION MAP

Problem	Why Stack?
Function calls	Last call returns first
Recursion	Activation records
Parentheses	Latest opener closes first
Undo	Latest action undone first
DFS	Explore/defer paths
Postfix evaluation	Latest operands/operators

Problem	Why Stack?
Infix conversion	Operator precedence handling
Backtracking	Undo latest decision

68. GATE-STYLE OPERATION TRACE

Stack initially empty.

Execute:

```
TEXT
push(5)
push(8)
pop()
push(3)
push(9)
pop()
peek()
```

Trace:

After `push(5)` :

```
TEXT
5
```

After `push(8)` :

```
TEXT
8
5
```

`pop()` returns:

```
TEXT
8
```

Stack:

```
TEXT
5
```

Push 3:

```
TEXT
3
5
```

Push 9:

TEXT

9
3
5

Pop returns:

TEXT

9

Stack:

TEXT

3
5

Peek:

TEXT

3

Final top:

3

69. GATE-STYLE PERMUTATION QUESTION

Input:

TEXT

1 2 3 4

Can output:

TEXT

2 1 4 3

be generated using one stack?

Yes.

Can:

TEXT

3 1 4 2

be generated?

Let's check.

To output 3:

TEXT

push 1
push 2
push 3
pop 3

Now top is:

TEXT

2

But desired next:

TEXT

1

Cannot pop 1 before 2.

Therefore impossible.

Exact reasoning style for GATE.

70. FORMULA BOX — ARRAY STACK

For capacity N :

TEXT

Empty:
 $\text{top} = -1$

TEXT

Full:
 $\text{top} = N-1$

TEXT

Size:
 $\text{top} + 1$

TEXT

Push:
 $\text{top}++$
insert

TEXT

Pop:
remove
 $\text{top}--$

71. MASTER COMPLEXITY TABLE

Stack operation	Complexity
Push	O(1)
Pop	O(1)
Peek	O(1)
isEmpty	O(1)
isFull (array)	O(1)
Size with top	O(1)
Search arbitrary element	O(n)

0-00 Copy this.

72. DON'T CONFUSE

Stack vs Queue

TEXT

Stack !' L I F O
Queue !' F I F O

Pop vs Peek

TEXT

Pop !' removes
Peek !' only reads

Overflow vs Underflow

TEXT

Overflow !' push when full
Underflow !' pop / peek when empty

Stack ADT vs Call Stack

One is the general abstract data structure; runtime call stack is a major application of stack behavior.

73. MEMORY TRICKS

Stack

LAST IN !' FIRST OUT

Push

Put on TOP

Pop

Pull from TOP

Peek

See TOP, don't touch

Underflow

Nothing to remove

Overflow

No room to add

Array top

Empty starts below zero: `top = -1`

74. NOTEBOOK NOTES

Stack

TEXT

LIFO

Operations

TEXT

```
Push    !'   Insert at top
Pop     !'   Delete top
Peek    !'   Read top
```

Array representation

TEXT

```
top = -1 !' empty
top = N - 1 !' full
```

Complexity

TEXT

Push = $O(1)$

Pop = $O(1)$

Peek = $O(1)$

75. 30-SECOND REVISION

Remember:

1. Stack follows LIFO.
2. Push inserts at top.
3. Pop removes top.
4. Peek reads top without removal.
5. Array stack often starts with `top=-1`.
6. Full when `top=capacity-1`.
7. Underflow means removing from empty stack.
8. Overflow means pushing into full fixed stack.
9. Push/pop/peek are $O(1)$.
10. Function calls and recursion use stack behavior.

76. ACTIVE RECALL

Q1

What does LIFO mean?

Q2

What is array-stack empty condition?

Q3

What is full condition for capacity `N`?

Q4

What is the difference between pop and peek?

Q5

Why does recursion naturally use a stack?

ANSWERS

A1

Last In, First Out.

A2

TEXT

`top == -1`

A3

TEXT

```
top == N-1
```

A4

Pop removes top; peek only reads it.

A5

The most recently called active function must return before its caller resumes.

77. PRACTICE — EASY

Q1

Starting empty:

TEXT

```
push(10)
push(20)
push(30)
pop()
```

What value is popped?

What is final top value?

Q2

Capacity = 5.

If:

TEXT

```
top = 4
```

what happens on push?

Q3

If:

TEXT

```
top = -1
```

what happens on pop?

78. PRACTICE — MODERATE

Q4

Starting empty:

TEXT

```
push(1)
push(2)
pop()
push(3)
push(4)
pop()
peek()
```

Find:

- first popped value
- second popped value
- peek value
- final stack

Q5

For an array stack with:

TEXT

```
top = 6
```

how many elements are logically present?

Q6

Why is:

C

```
stack[++top] = x;
```

appropriate for push when empty is represented by `top=-1`, whereas:

C

```
stack[top++] = x;
```

is not?

79. PRACTICE — GATE STACK PERMUTATION

Input order:

TEXT

```
1 2 3 4
```

Which of these can be stack pop sequences?

A.

TEXT

4 3 2 1

B.

TEXT

2 1 4 3

C.

TEXT

3 1 2 4

Think using push/pop simulation.

80. PLACEMENT THINKING

Identify whether stack is a natural choice:

A

Check balanced brackets.

B

Process customers in arrival order.

C

Undo latest text-editor action.

D

Depth-first traversal.

E

Serve printer jobs strictly first-come-first-served.

Answers:

TEXT

```
A !' S t a c k
B !' Q u e u e
C !' S t a c k
D !' S t a c k / r e c u r s i o n
E !' Q u e u e
```

PROGRESS

MODULE 4 — STACK

' Lesson 37 — Stack Fundamentals

NEXT — LESSON 38

Stack Using Array in C — Complete Implementation & GATE Output Tracing

Next we implement a proper array-based stack and master:

```
C
push()
pop()
peek()
isEmpty()
isFull()
display()
```

We will cover:

- `top = -1`
- exact C implementation
- static vs dynamic capacity
- push/pop return values
- overflow/underflow
- `++top` vs `top++`
- `top--`
- logical vs physical contents
- output-tracing questions
- GATE operation sequences
- array-stack complexity and limitations

Then Lesson 39 will implement **Stack Using Linked List**.

USER

next

ASSISTANT

MODULE 4 — LESSON 38

Stack Using Array in C — Complete Implementation & GATE Output Tracing

Now we implement the stack properly using an array.

This lesson is important because GATE often mixes:

- stack concepts
- C code
- `top`
- `++top`
- `top--`

- GATE importance: Ø=Ý% MUST KNOW Placement importance: Ø=Ý% IMPORTANT**

Suppose maximum stack capacity is:

We create:

Why:

?

Because initially:

2. VISUALIZATION

Initially:

After:

we get:

TEXT

Index: 0 1 2 3 4

[10] [20] [30] [] []

!'

top

top = 2

3. STACK SIZE

Since indices start at 0:

FORMULA BOX

TEXT

size = top + 1

Examples:

TEXT

top = -1 !' size = 0

top = 0 !' size = 1

top = 3 !' size = 4

4. EMPTY CONDITION

C

top == -1

means:

STACK EMPTY

5. FULL CONDITION

For capacity MAX :

C

top == MAX - 1

means:

STACK FULL

Example:

TEXT

```
MAX = 5
top = 4
```

All 5 slots are logically occupied.

6. isEmpty()

```
C
int isEmpty(void)
{
    return top == -1;
}
```

Returns:

TEXT

```
1 !'   e m p t y
0 !'   n o t   e m p t y
```

Time:

TEXT

O(1)

7. isFull()

```
C
int isFull(void)
{
    return top == MAX - 1;
}
```

Time:

TEXT

O(1)

8. PUSH OPERATION Ø=Ý%

Algorithm:

TEXT

```
if full
    overflow
else
    top = top + 1
    stack[top] = value
```

C:

```
c
void push(int value)
{
    if (top == MAX - 1)
    {
        printf("Stack Overflow\n");
        return;
    }

    top++;
    stack[top] = value;
}
```

9. PUSH TRACE

Initially:

```
TEXT
top = -1
```

Execute:

```
c
push(10);
```

Step 1:

```
TEXT
top++
```

SO:

```
TEXT
top = 0
```

Step 2:

```
TEXT
stack[0] = 10
```

Result:

```
TEXT
[10] [ ] [ ] [ ] [ ] [ ]
!
top
```

10. SECOND PUSH

```
C
push(20);
```

Then:

```
TEXT
top = 1
stack[1] = 20
```

Result:

```
TEXT
[10] [20] [ ] [ ] [ ]
      !'
    top
```

11. COMPACT PUSH FORM

Instead of:

```
C
top++;
stack[top] = value;
```

we can write:

```
C
stack[++top] = value;
```

Why?

Because:

```
TEXT
++top
```

increments `top` **before** using it as index.

If initially:

```
TEXT
top = -1
```

then:

```
TEXT
++top !' 0
```

so:

TEXT

```
stack[0]
```

is used.

12. WRONG PUSH FORM

Suppose:

C

```
stack[top++] = value;
```

Initially:

TEXT

```
top = -1
```

Post-increment uses the old value first.

So it tries:

TEXT

```
stack[-1]
```

before changing top to 0.

Invalid.

Ø=Ý% Important C + stack trap.

13. MEMORY TRICK

For this representation:

PUSH !' move TOP first

C

```
stack[++top] = value;
```

14. POP OPERATION Ø=Ý%

Pop means:

1. read current top value
2. decrease top

Algorithm:

TEXT

```
if empty
    underflow
else
    value = stack[top]
    top = top - 1
    return value
```

15. POP FUNCTION

C

```
int pop(void)
{
    if (top == -1)
    {
        printf("Stack Underflow\n");
        return -1;
    }

    int value = stack[top];
    top--;

    return value;
}
```

For learning purposes, we use `-1` as an error value, though in real software this can be ambiguous if `-1` is a valid stack value.

16. POP TRACE

Before:

TEXT

```
Index:
0      1      2
10     20     30
           !
           top

top = 2
```

Execute:

C

```
x = pop();
```

Then:

TEXT

```
x = stack[2] = 30
top = 1
```

Logical stack:

TEXT

10 20

Returned:

TEXT

30

17. IMPORTANT — SLOT 2 MAY STILL CONTAIN 30

After pop, memory may physically look like:

TEXT

```
[10] [20] [30] [ ] [ ]  
          !'  
        top
```

The value 30 may still remain in `stack[2]`.

But it is no longer logically part of the stack because:

TEXT

`top = 1`

Stack membership is determined by `top`.

18. COMPACT POP

This:

C

```
return stack[top--];
```

works for a valid nonempty stack.

Why?

Post-decrement:

1. uses current `top`
2. then decreases it

Example:

TEXT

`top = 2`

then:


```
TEXT
stack[top--]
```

means:

```
TEXT
stack[2]
```

then:

```
TEXT
top = 1
```

Exactly what pop needs.

19. MEMORY TRICK

POP !' use TOP first, then move it down

```
C
stack[top--]
```

Compare:

```
TEXT
P U S H   !'   + + t o p
P O P     !'   t o p - -
```

0=Min high-value shortcut.

20. PEEK OPERATION

Peek reads top without removing it.

```
C
int peek(void)
{
    if (top == -1)
    {
        printf("Stack Underflow\n");
        return -1;
    }

    return stack[top];
}
```

Time:

TEXT

0(1)

21. POP VS PEEK TRACE

Stack:

TEXT

3 0 !• t o p
20
10

First:

c

peek();

returns:

TEXT

30

Stack remains:

TEXT

30
20
10

Then:

c

pop();

returns:

TEXT

30

Stack becomes:

TEXT

2 0 !• t o p
10

0-peek does not modify top.

22. DISPLAY STACK

A display function from top to bottom:

```

c
void display(void)
{
    if (top == -1)
    {
        printf("Stack is empty\n");
        return;
    }

    for (int i = top; i >= 0; i--)
    {
        printf("%d ", stack[i]);
    }
}

```

If stack contains:

```

TEXT
10 20 30

```

with 30 on top, output:

```

TEXT
30 20 10

```

23. DISPLAY FROM BOTTOM TO TOP

If instead:

```

c
for (int i = 0; i <= top; i++)

```

output becomes:

```

TEXT
10 20 30

```

Both are possible display conventions.

But top-to-bottom better reflects pop order.

24. COMPLETE ARRAY STACK PROGRAM

```

c
#include <stdio.h>

#define MAX 5

int stack[MAX];
int top = -1;

int isEmpty(void)
{

```

```

        return top == -1;
    }

    int isFull(void)
    {
        return top == MAX - 1;
    }

    void push(int value)
    {
        if (isFull())
        {
            printf("Stack Overflow\n");
            return;
        }

        stack[++top] = value;
    }

    int pop(void)
    {
        if (isEmpty())
        {
            printf("Stack Underflow\n");
            return -1;
        }

        return stack[top--];
    }

    int peek(void)
    {
        if (isEmpty())
        {
            printf("Stack Underflow\n");
            return -1;
        }

        return stack[top];
    }

    void display(void)
    {
        for (int i = top; i >= 0; i--)
            printf("%d ", stack[i]);
    }
}

```

0-7% is the core array-stack implementation.

25. OPERATION TRACE 1

Initially:

TEXT

top = -1

Execute:

TEXT

```
push(10)
push(20)
push(30)
```

Trace:

TEXT

```
After push(10):
top = 0
```

```
After push(20):
top = 1
```

```
After push(30):
top = 2
```

Stack:

TEXT

```
TOP
! "
30
20
10
```

26. THEN POP

C

```
printf("%d", pop());
```

Output:

TEXT

```
30
```

Now:

TEXT

```
top = 1
```

Stack:

TEXT

```
20
10
```

27. THEN PUSH 40

```
C
push(40);
```

Since:

```
TEXT
top = 1
```

`++top` gives:

```
TEXT
2
```

Then:

```
TEXT
stack[2] = 40
```

Logical stack:

```
TEXT
40
20
10
```

Even though slot 2 previously contained 30, it is overwritten by 40.

28. GATE OUTPUT QUESTION 1

Given:

```
C
int s[5];
int top = -1;

s[++top] = 10;
s[++top] = 20;
s[++top] = 30;

printf("%d ", s[top--]);
printf("%d ", s[top]);
```

Trace.

After pushes:

```
TEXT
top = 2

s[0]=10
s[1]=20
s[2]=30
```

First:

```
C
s[top--]
```

uses:

```
TEXT
s[2] = 30
```

then:

```
TEXT
top = 1
```

prints:

```
TEXT
30
```

Second:

```
C
s[top]
```

is:

```
TEXT
s[1] = 20
```

Output:

```
TEXT
30 20
```

Ø=Typical GATE C-stack trace.

29. GATE OUTPUT QUESTION 2

```
C
int s[5];
int top = -1;

s[++top] = 5;
s[++top] = 8;

printf("%d ", s[top]);
printf("%d ", s[top--]);
printf("%d ", top);
```

After pushes:

TEXT

```
top = 1
```

First:

TEXT

```
s[top] = s[1] = 8
```

prints 8.

Second:

TEXT

```
s[top--]
```

prints:

TEXT

```
8
```

then:

TEXT

```
top = 0
```

Third prints:

TEXT

```
0
```

Final output:

TEXT

```
8 8 0
```

30. GATE OUTPUT QUESTION 3

C

```
int top = -1;
int s[5];

s[++top] = 10;
s[++top] = 20;

int x = s[top--];
int y = s[++top];

printf("%d %d", x, y);
```

Trace carefully.

After pushes:

TEXT

```
top = 1
```

Then:

C

```
x = s[top--];
```

So:

TEXT

```
x = s[1] = 20  
top = 0
```

Then:

C

```
y = s[++top];
```

top becomes:

TEXT

```
1
```

So:

TEXT

```
y = s[1] = 20
```

Output:

TEXT

```
20 20
```

Physical array values matter even when logical stack size changes.

31. IMPORTANT GATE LESSON

After pop:

TEXT

```
top decreases
```

but C does not automatically erase:

TEXT

```
stack[oldTop]
```

So later changing `top` may expose that old value again unless overwritten.

Data-structure logic assumes you manipulate `top` correctly.

32. STACK OVERFLOW TRACE

Capacity:

```
TEXT
MAX = 3
```

Start:

```
TEXT
top = -1
```

Push:

```
TEXT
10
20
30
```

Now:

```
TEXT
top = 2 = MAX-1
```

Stack full.

Attempt:

```
TEXT
push(40)
```

Correct implementation detects:

```
TEXT
top == MAX-1
```

and does not change the stack.

Final:

```
TEXT
30
20
10
```

33. STACK UNDERFLOW TRACE

Start:

TEXT

```
top = -1
```

Attempt:

TEXT

```
pop()
```

Correct code detects empty.

`top` should remain:

TEXT

```
-1
```

No valid value is removed.

34. IMPORTANT ERROR-DESIGN NOTE

Our simple function uses:

C

```
return -1;
```

for underflow.

But what if valid stack data can contain:

TEXT

```
-1
```

?

Then caller cannot distinguish:

TEXT

```
valid -1
```

from:

TEXT

```
error
```

In robust software, better approaches include:

- return success/failure separately
- use output parameter
- store status
- use a wider API design

0-Placement/software-design insight.

35. BETTER POP API

Example:

```
c
int pop(int *value)
{
    if (top == -1)
        return 0;

    *value = stack[top--];
    return 1;
}
```

Caller:

```
c
int x;

if (pop(&x))
    printf("%d", x);
else
    printf("Underflow");
```

Now any integer, including `-1`, can be a valid stack value.

36. STACK AS STRUCTURE

Instead of global variables, a cleaner implementation:

```
c
typedef struct
{
    int data[MAX];
    int top;
} Stack;
```

Initialize:

```
c
Stack s;
s.top = -1;
```

This lets us create multiple independent stacks.

37. WHY GLOBAL VERSION IS LIMITED

With:

```
c
int stack[MAX];
int top = -1;
```

we effectively have one global stack.

Using:

```
c
typedef struct
{
    int data[MAX];
    int top;
} Stack;
```

we can create:

```
c
Stack s1;
Stack s2;
Stack s3;
```

Each has its own `top`.

Ø= Better software design.

38. STRUCT-BASED PUSH

```
c
int push(Stack *s, int value)
{
    if (s->top == MAX - 1)
        return 0;

    s->data[++s->top] = value;

    return 1;
}
```

Notice:

```
TEXT
Stack *
```

because the function modifies the actual stack object.

39. STRUCT-BASED POP

```
c
int pop(Stack *s, int *value)
{
    if (s->top == -1)
        return 0;

    *value = s->data[s->top--];

    return 1;
}
```

This is a practical C implementation.

40. STATIC ARRAY LIMITATION

Suppose:

```
C
#define MAX 100
```

The stack can hold at most:

```
TEXT
100 elements
```

even if the system has much more memory.

This is a fixed-capacity stack.

41. DYNAMIC ARRAY STACK — PLACEMENT EXTENSION

A stack can also use dynamically allocated arrays.

Conceptually:

```
TEXT
capacity = 4
```

If full:

```
TEXT
allocate/reallocate larger storage
```

For example:

```
TEXT
4  !'  8  !'  1 6  !'  3 2
```

Then individual push is usually:

```
TEXT
O(1) amortized
```

although a resizing push itself may cost:

```
TEXT
O(n)
```

↳ Important dynamic-array insight.

42. AMORTIZED PUSH INTUITION

Most pushes:

```
TEXT
0(1)
```

Occasionally capacity fills and all elements are copied:

```
TEXT
0(n)
```

But if capacity doubles geometrically, expensive resizing doesn't happen every push.

Across many pushes, average cost per push remains:

```
TEXT
0(1) amortized
```

We'll cover amortized complexity more deeply in Algorithms if needed.

43. FIXED ARRAY STACK SPACE

Capacity:

```
TEXT
MAX
```

Array storage:

```
TEXT
0(MAX)
```

Current logical stack may contain only:

```
TEXT
n <= MAX
```

elements.

So allocated capacity and logical size are different.

Recall arrays:

```
TEXT
capacity "` logical size
```

44. STACK USING ARRAY MEMORY

Array elements are contiguous:

TEXT

```
[10][20][30][40]
```

This provides good cache locality.

No pointer per element is required.

This is one practical advantage over a linked stack.

45. GATE TRAP — ISFULL CONDITION

For `MAX = 10`:

Valid indices:

TEXT

```
0 ... 9
```

So full when:

TEXT

```
top == 9
```

not:

TEXT

```
top == 10
```

~~0=0%~~ by-one trap.

46. GATE TRAP — EMPTY CONDITION

With this representation:

TEXT

```
top == -1
```

not:

TEXT

```
top == 0
```

Why?

`top=0` means one element exists at index 0.

47. GATE TRAP — NUMBER OF ELEMENTS

If:

TEXT

```
top = 7
```

number of elements:

TEXT

```
8
```

because:

TEXT

```
0 ... 7
```

contains 8 positions.

Formula:

TEXT

```
top + 1
```

48. GATE TRAP — POP DOES NOT NECESSARILY ZERO SLOT

This is sufficient:

C

```
top--;
```

You do not need:

C

```
stack[top] = 0;
```

for logical deletion.

And setting it to zero may even be incorrect conceptually if zero is valid data.

49. GATE TRAP — PUSH ORDER

Correct:

C

```
top++;  
stack[top] = x;
```

Wrong for `top=-1` representation:

C

```
stack[top] = x;  
top++;
```

because first push accesses:

```
TEXT
stack[-1]
```

50. GATE TRAP — POP ORDER

Correct:

```
C
x = stack[top];
top--;
```

Wrong:

```
C
top--;
x = stack[top];
```

because it returns the element below the old top.

Example:

```
TEXT
10 20 30
      ! '
      top
```

If top decremented first:

```
TEXT
top points to 20
```

and you'd incorrectly return 20 instead of 30.

Very important.

51. PUSH AND POP SHORTCUT

Memorize:

```
TEXT
PUSH:
stack[++top] = x
```

```
TEXT
POP:
x = stack[top--]
```

For this standard representation.

52. STACK PERMUTATION CHECKING

Suppose input:

```
TEXT
1 2 3 4
```

Desired output:

```
TEXT
2 4 3 1
```

Let's simulate.

Push 1:

```
TEXT
1
```

Push 2:

```
TEXT
2
1
```

Pop 2.

Stack:

```
TEXT
1
```

Desired next 4.

Push 3:

```
TEXT
3
1
```

Push 4:

```
TEXT
4
3
1
```

Pop 4.

Desired next 3.

Pop 3.

Desired next 1.

Pop 1.

So valid.

53. STACK PERMUTATION— IMPOSSIBLE EXAMPLE

Input:

```
TEXT
1 2 3 4
```

Output:

```
TEXT
3 1 4 2
```

To output 3:

```
TEXT
push 1
push 2
push 3
pop 3
```

Stack now:

```
TEXT
2
1
```

Desired next:

```
TEXT
1
```

But top is:

```
TEXT
2
```

Cannot bypass it.

So impossible.

54. GATE COUNTING CONNECTION

For distinct input elements pushed in fixed order, not every permutation is a valid stack permutation.

For small n , stack-permutation counts relate to:

Catalan numbers

Ø=0 This is more Discrete Mathematics / combinatorics flavored, but GATE can connect data structures with counting.
For now, just remember the conceptual link.

55. TWO STACKS IN ONE ARRAY — PLACEMENT/GATE EXTENSION

Suppose one array:

```
TEXT
0 ..... N-1
```

We can implement two stacks:

```
TEXT
Stack 1 grows !'
Stack 2 grows !•
```

Initialize:

```
TEXT
top1 = -1
top2 = N
```

Stack 1 push:

```
TEXT
top1++
```

Stack 2 push:

```
TEXT
top2--
```

56. FULL CONDITION FOR TWO STACKS

They are full when:

```
TEXT
top1 + 1 == top2
```

because there is no free position between them.

Ø=0 Classic interview/GATE extension.

57. VISUALIZATION

Array capacity 8:

TEXT

Index:

0 1 2 3 4 5 6 7

S 1 ! ' ! • S 2

Example:

TEXT

[1][2][3][][][8][7][6]

! ' ! '
top1 top2

Unused middle space is shared dynamically between both stacks.

58. WHY TWO STACKS IN ONE ARRAY?

If we gave each stack half the array, one could overflow while the other half remained mostly empty.

Growing from opposite ends lets both share unused capacity.

Good design insight.

59. STACK IMPLEMENTATION COMPLEXITY

Fixed array:

Operation	Time
Push	$O(1)$
Pop	$O(1)$
Peek	$O(1)$
isEmpty	$O(1)$
isFull	$O(1)$
Display	$O(n)$

Space capacity:

TEXT

$O(\text{MAX})$

60. WHY DISPLAY IS $O(n)$

To print all stack elements:

TEXT

```
top
top-1
...
0
```

we visit every element.

Therefore:

TEXT

$O(n)$

This does not change the fact that fundamental stack ADT operations remain $O(1)$.

61. INTERVIEW QUESTIONS

Q1

Why is `top=-1` used?

It represents no valid top index.

Q2

Why `stack[++top]` for push?

Top must move to the new free logical position before storing.

Q3

Why `stack[top--]` for pop?

Current top must be read before moving top downward.

Q4

Is the old popped value physically erased?

Not necessarily.

Q5

What is fixed-array stack overflow?

Attempting to push when:

TEXT

```
top == MAX-1
```

62. MEMORY TRICKS

Empty

Top is below the array: `-1`.

Full

Top reached last valid index.

Push

MOVE then STORE.

Pop

TAKE then MOVE.

Peek

LOOK, don't move.

Logical deletion

TOP decides what exists.

63. NOTEBOOK NOTES

Representation

```
C
int stack[MAX];
int top = -1;
```

Push

```
C
if (top == MAX - 1)
    overflow;
else
    stack[++top] = x;
```

Pop

```
C
if (top == -1)
    underflow;
else
    x = stack[top--];
```

Peek

C
stack[top]

64. FORMULA BOX

TEXT
Empty:
top = -1

TEXT
Full:
top = MAX - 1

TEXT
Size:
top + 1

TEXT
Push:
++top

TEXT
Pop:
top--

65. 30-SECOND REVISION

Remember:

1. Array stack uses `top`.
2. Initial `top=-1`.
3. First push makes `top=0`.
4. Full when `top=MAX-1`.
5. Push: increment then store.
6. Pop: read then decrement.
7. Peek does not modify `top`.
8. Popped slot need not be physically erased.
9. Push/pop/peek = $O(1)$.
10. Display/search = $O(n)$.

66. ACTIVE RECALL

Q1

For capacity 20, what value of `top` means full?

Q2

If `top=5` , how many elements exist?

Q3

Why is this wrong for first push?

```
C
stack[top++] = x;
```

when `top=-1`.

Q4

After:

```
C
x = stack[top--];
```

what happens first: access or decrement?

Q5

Does `top--` erase the old array value?

ANSWERS

A1

```
TEXT
19
```

A2

```
TEXT
6
```

A3

It attempts to access `stack[-1]` .

A4

Access current `stack[top]` first, then decrement.

A5

No. It only changes the logical stack boundary.

67. PRACTICE — EASY

Initially:

TEXT

```
top = -1
```

Execute:

TEXT

```
push(4)
push(7)
push(9)
pop()
```

Find:

- popped value
- final `top`
- final logical stack

68. PRACTICE — C TRACE

C

```
int s[10];
int top = -1;

s[++top] = 2;
s[++top] = 4;
s[++top] = 6;

printf("%d ", s[top--]);
printf("%d ", s[++top]);
printf("%d", top);
```

Find the exact output.

69. PRACTICE — OVERFLOW

Capacity:

TEXT

4

Operations:

TEXT

```
push(1)
push(2)
push(3)
push(4)
push(5)
```

Assuming proper overflow handling:

- what is final stack?
- what is final `top`?

- which operation causes overflow?

70. PRACTICE — UNDERFLOW

Start empty:

TEXT

```
pop()  
push(10)  
pop()  
pop()
```

Which operations cause underflow?

What are the successful popped values?

71. PRACTICE — STACK PERMUTATION

Input order:

TEXT

1 2 3 4 5

Determine whether these can be generated using one stack:

A

TEXT

5 4 3 2 1

B

TEXT

2 1 4 3 5

C

TEXT

3 1 2 5 4

Use actual push/pop reasoning, not guessing.

72. GATE-STYLE QUESTION

An array stack of capacity 10 starts empty.

Operations:

TEXT

8 pushes
3 pops
2 pushes
4 pops

Assume no overflow/underflow.

Final number of elements:

TEXT

$0 + 8 - 3 + 2 - 4$
 $= 3$

Thus:

TEXT

$\text{top} = 2$

~~0 = 3~~ Very quick numerical shortcut.

PROGRESS

MODULE 4 — STACK

' Lesson 37 — Stack Fundamentals ' Lesson 38 — Stack Using Array

NEXT — LESSON 39

Stack Using Linked List — Dynamic Stack, `top` Pointer, Push/Pop & Memory Management

Next we implement:

TEXT

`top`
`!"`
`3 0 !' 2 0 !' 1 0 !' N U L L`

using nodes.

We will learn:

- why linked-list head naturally acts as stack top
- `push` = insert at head
- `pop` = delete head
- why both are $O(1)$
- dynamic memory
- malloc failure
- underflow

- linked stack vs array stack
- fixed capacity vs dynamic growth
- memory overhead
- GATE pointer/output questions

Then we move to the major Stack applications:

Lessons 40–42 — Infix, Prefix, Postfix Conversion & Evaluation

USER

next

ASSISTANT

MODULE 4 — LESSON 39

Stack Using Linked List — Dynamic Stack, top Pointer, Push/Pop & Memory Management

Now we implement a stack using a linked list.

This is the perfect connection between:

- linked lists
- dynamic memory
- stack ADT
- pointer manipulation

GATE importance: $\emptyset = \text{Yes} \%$ **MUST KNOW** **Placement importance:** $\emptyset = \text{Yes} \%$ **IMPORTANT**

1. CORE IDEA

For a linked-list stack, the best place to perform stack operations is the:

HEAD

Why?

Because:

TEXT

```
insert at head = O(1)
delete head    = O(1)
```

So we use the linked-list head as:

STACK TOP

2. VISUALIZATION

Suppose we push:

```
TEXT
10
20
30
```

Stack:

```
TEXT
top
!"
3 0 !' 2 0 !' 1 0 !' N U L L
```

The most recently inserted element:

```
TEXT
30
```

is at the front.

Therefore:

```
TEXT
LIFO
```

works naturally.

3. NODE STRUCTURE

```
C
typedef struct Node
{
    int data;
    struct Node *next;
} Node;
```

Stack top:

```
C
Node *top = NULL;
```

Initially:

```
TEXT
top = NULL
```

means:

EMPTY STACK

4. WHY `top == NULL`?

Unlike array stack:

```
TEXT
top = -1
```

linked stack uses a pointer.

So empty condition:

```
C
top == NULL
```

because there is no first node.

Don't confuse array `top` index with linked-list `top` pointer.

5. PUSH = INSERT AT HEAD

Suppose:

```
TEXT
top
!"
2 0 !' 1 0 !' N U L L
```

Push:

```
TEXT
3 0
```

Create node:

```
TEXT
newNode !' 3 0 !' N U L L
```

Then:

```
C
newNode->next = top;
top = newNode;
```

After:

```
TEXT
top
!"
3 0 !' 2 0 !' 1 0 !' N U L L
```


Exactly like linked-list head insertion.

6. PUSH FUNCTION

```
c
int push(Node **top, int value)
{
    Node *newNode = malloc(sizeof *newNode);

    if (newNode == NULL)
        return 0;

    newNode->data = value;
    newNode->next = *top;

    *top = newNode;

    return 1;
}
```

Call:

```
c
push(&top, 30);
```

7. WHY Node **top?

Caller has:

```
c
Node *top;
```

Push changes that pointer itself.

So function needs its address:

```
TEXT
&top
```

which has type:

```
TEXT
Node **
```

Same pointer-to-pointer idea from linked-list insertion.

8. RETURN-STYLE ALTERNATIVE

Could also write:

```

c
Node *push(Node *top, int value)
{
    Node *newNode = malloc(sizeof *newNode);

    if (newNode == NULL)
        return top;

    newNode->data = value;
    newNode->next = top;

    return newNode;
}

```

Then:

```

c
top = push(top, 30);

```

Both designs are valid.

9. PUSH COMPLEXITY

No traversal.

Only:

- allocate node
- set data
- change two pointers

Algorithmically:

```

TEXT
O(1)

```

Auxiliary pointer space:

```

TEXT
O(1)

```

The new node itself contributes to stack storage.

10. POP = DELETE HEAD

Suppose:

```

TEXT
top
!"
3 0 !' 2 0 !' 1 0 !' N U L L

```

Pop should remove:

TEXT

30

Safe pattern:

SAVE !' MOVE TOP !' FREE

11. POP POINTER STEPS

Before:

TEXT

top

!"

3 0 !' 2 0 !' 1 0

Save:

C

```
Node *temp = top;
```

Then:

C

```
top = top->next;
```

Now:

TEXT

t e m p !' 3 0

t o p !' 2 0 !' 1 0

Save value:

C

```
int value = temp->data;
```

Then:

C

```
free(temp);
```

Return:

TEXT

30

12. POP FUNCTION

```

c
int pop(Node **top, int *value)
{
    if (*top == NULL)
        return 0;

    Node *temp = *top;

    *value = temp->data;
    *top = temp->next;

    free(temp);

    return 1;
}

```

Call:

```

c
int x;

if (pop(&top, &x))
    printf("%d", x);

```

13. WHY OUTPUT PARAMETER?

We avoid using:

```

c
return -1;

```

as an error code because `-1` may be valid data.

So:

```

TEXT
function return value
!' s u c c e s s / f a i l u r e

```

and:

```

TEXT
*value
!' p o p p e d   d a t a

```

This is a cleaner API.

14. UNDERFLOW

If:

TEXT

top == NULL

and we attempt:

TEXT

pop

there is no node.

This is:

STACK UNDERFLOW

Linked stack still has underflow.

15. PEEK

Peek reads top without removing it.

```
c
int peek(Node *top, int *value)
{
    if (top == NULL)
        return 0;

    *value = top->data;

    return 1;
}
```

Time:

TEXT

O(1)

16. isEmpty

```
c
int isEmpty(Node *top)
{
    return top == NULL;
}
```

Time:

TEXT

O(1)

17. COMPLETE LINKED STACK

```
C
#include <stdio.h>
#include <stdlib.h>

typedef struct Node
{
    int data;
    struct Node *next;
} Node;

int push(Node **top, int value)
{
    Node *newNode = malloc(sizeof *newNode);

    if (newNode == NULL)
        return 0;

    newNode->data = value;
    newNode->next = *top;

    *top = newNode;

    return 1;
}

int pop(Node **top, int *value)
{
    if (*top == NULL)
        return 0;

    Node *temp = *top;

    *value = temp->data;
    *top = temp->next;

    free(temp);

    return 1;
}

int peek(Node *top, int *value)
{
    if (top == NULL)
        return 0;

    *value = top->data;

    return 1;
}
```

18. OPERATION TRACE

Initially:

```
TEXT
top = NULL
```

Push 10:

TEXT

```
top
!"
1 0 !' N U L L
```

Push 20:

TEXT

```
top
!"
2 0 !' 1 0 !' N U L L
```

Push 30:

TEXT

```
top
!"
3 0 !' 2 0 !' 1 0 !' N U L L
```

Pop:

TEXT

removes 30

Final:

TEXT

```
top
!"
2 0 !' 1 0 !' N U L L
```

19. PUSH ORDER VS POP ORDER

Push:

TEXT

```
10
20
30
40
```

Linked structure:

TEXT

```
top
!"
4 0 !' 3 0 !' 2 0 !' 1 0
```

Repeated pops:

TEXT

40
30
20
10

LIFO preserved.

20. LINKED STACK DOES NOT NEED CONTIGUOUS MEMORY

Nodes may exist at addresses:

TEXT

1000
8000
3000
6000

but links define stack order.

Example:

TEXT

top=6000

!"

4 0 !' n o d e a t 3 0 0 0

!"

3 0 !' n o d e a t 8 0 0 0

!"

2 0 !' . . .

Physical location doesn't matter.

21. ARRAY STACK VS LINKED STACK

Array:

TEXT

[10][20][30][]

Linked:

TEXT

3 0 !' 2 0 !' 1 0 !' N U L L

Both implement the same:

STACK ADT

Different storage strategy.

22. MAIN ARRAY STACK LIMITATION

Fixed array:

```
C
int stack[100];
```

can hold at most:

```
TEXT
100
```

elements.

If full:

```
TEXT
overflow
```

even if other system memory exists.

23. LINKED STACK GROWTH

Linked stack creates one node per push:

```
C
malloc(...)
```

So it can continue growing while memory allocation succeeds.

No fixed predefined element capacity is required.

24. DOES LINKED STACK NEVER OVERFLOW?

Do **not** say this absolutely.

Memory is finite.

If:

```
C
malloc(...)
```

returns:

```
TEXT
NULL
```

push fails.

That is the practical linked-stack equivalent of inability to grow further.

Ø=important nuance.

25. LINKED STACK MEMORY OVERHEAD

Each node stores:

```
TEXT
data
+
next pointer
```

So compared with an array of only data:

```
TEXT
linked stack uses extra pointer memory per element
```

Possible structure padding also exists.

26. CACHE LOCALITY

Array stack:

```
TEXT
contiguous memory
```

usually has good cache locality.

Linked stack:

```
TEXT
nodes may be scattered
```

so traversal/access patterns may have poorer locality.

For push/pop complexity both are $O(1)$, but real performance may differ.

$\emptyset$ -Systems insight.

27. POP MEMORY SAFETY $\emptyset=\checkmark\%$

Wrong:

```
C
free(*top);
*top = (*top)->next;
```

Why?

After:

```
C
free(*top);
```

*top is dangling.

Then reading:

```
c
(*top)->next
```

is use-after-free.

28. CORRECT POP ORDER

```
c
Node *temp = *top;
*top = temp->next;
free(temp);
```

Memory trick:

SAVE !' MOVE !' FREE

Same as linked-list deletion.

29. MEMORY LEAK TRAP

Wrong:

```
c
*top = (*top)->next;
```

without freeing old top.

The stack logically pops the node, but old node becomes unreachable.

That is a:

Memory Leak

Correct:

```
c
Node *temp = *top;
*top = temp->next;
free(temp);
```

30. PEEK MUST NOT CHANGE TOP

Wrong peek:

```

c
int peek(Node **top)
{
    int x = (*top)->data;
    *top = (*top)->next;
    return x;
}

```

This is actually behaving like pop without free.

A proper peek only reads:

```

c
top->data

```

and leaves all links unchanged.

31. DISPLAY LINKED STACK

```

c
void display(Node *top)
{
    Node *temp = top;

    while (temp != NULL)
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }
}

```

Since traversal starts at top, it prints in pop order.

For:

```

TEXT
t o p ! ' 3 0 ! ' 2 0 ! ' 1 0

```

output:

```

TEXT
30 20 10

```

Time:

```

TEXT
O(n)

```

32. SIZE

If size is not maintained separately, count nodes:

```

c
int size(Node *top)
{
    int count = 0;

    while (top != NULL)
    {
        count++;
        top = top->next;
    }

    return count;
}

```

Time:

```

TEXT
O(n)

```

33. MAINTAIN SIZE VARIABLE

If stack structure stores:

```

c
int size;

```

and updates it on push/pop:

```

TEXT
p u s h   !'   s i z e + +
p o p     !'   s i z e - -

```

then size query becomes:

```

TEXT
O(1)

```

Same metadata trade-off as linked lists.

34. CLEAN STACK STRUCTURE

A more complete design:

```

c
typedef struct
{
    Node *top;
    int size;
} Stack;

```

Initialize:

```

c
Stack s;

s.top = NULL;
s.size = 0;

```

35. PUSH WITH STACK WRAPPER

```

c
int push(Stack *s, int value)
{
    Node *newNode = malloc(sizeof *newNode);

    if (newNode == NULL)
        return 0;

    newNode->data = value;
    newNode->next = s->top;

    s->top = newNode;
    s->size++;

    return 1;
}

```

36. POP WITH WRAPPER

```

c
int pop(Stack *s, int *value)
{
    if (s->top == NULL)
        return 0;

    Node *temp = s->top;

    *value = temp->data;
    s->top = temp->next;

    free(temp);
    s->size--;

    return 1;
}

```

This is a clean practical implementation.

37. LINKED STACK COMPLEXITY TABLE

Operation	Time
Push	O(1)
Pop	O(1)

Operation	Time
Peek	$O(1)$
isEmpty	$O(1)$
Size if stored	$O(1)$
Size if not stored	$O(n)$
Display	$O(n)$
Search	$O(n)$

0=100% Must know.

38. ARRAY VS LINKED STACK — MASTER TABLE

Property	Array Stack	Linked Stack
Storage	Contiguous	Non-contiguous
Push	$O(1)$	$O(1)$
Pop	$O(1)$	$O(1)$
Peek	$O(1)$	$O(1)$
Fixed capacity	Usually yes for static array	No fixed element limit
Overflow	When array full	Allocation failure
Pointer overhead	No per element	Yes
Cache locality	Better	Usually weaker
Dynamic memory	Not required	Required normally

0=100% Excellent revision table.

39. WHY TOP SHOULD BE HEAD, NOT TAIL

Suppose singly linked list:

TEXT

1 0 !' 2 0 !' 3 0

If stack top were tail:

TEXT

3 0

Push at tail with only head:

TEXT

$O(n)$

Pop tail:

TEXT

$O(n)$

bad.

If top is head:

TEXT

push head = $O(1)$

pop head = $O(1)$

Therefore head is the natural stack top.

~~$O(n)$~~ GATE reasoning question.

40. WHAT IF WE MAINTAIN TAIL TOO?

Even with tail, popping tail in singly linked list requires predecessor:

TEXT

$O(n)$

So tail is still a bad stack-top choice for singly linked implementation.

Use head.

41. COULD DOUBLY LINKED LIST USE TAIL AS TOP?

Yes.

With a DLL and tail:

TEXT

push tail = $O(1)$

pop tail = $O(1)$

But that's unnecessary overhead if a singly list with head already gives $O(1)$ operations.

So simplest linked stack:

TEXT

singly linked list + head as top

42. GATE POINTER TRACE

Initially:

TEXT

t o p ! ' 2 0 ! ' 1 0 ! ' N U L L

Create:

TEXT

p ! ' 3 0 ! ' N U L L

Execute:

C

```
p->next = top;  
top = p;
```

Final:

TEXT

t o p ! ' 3 0 ! ' 2 0 ! ' 1 0 ! ' N U L L

This is:

TEXT

push(30)

43. GATE POINTER TRACE — POP

Initially:

TEXT

t o p ! ' A ! ' B ! ' C ! ' N U L L

Execute:

C

```
Node *p = top;  
top = top->next;  
free(p);
```

Final:

TEXT

t o p ! ' B ! ' C ! ' N U L L

Node A removed.

This is exactly:

POP

44. GATE TRAP — `top = top->next` ONLY

If you do:

```
C
top = top->next;
```

stack logically removes first node but doesn't free it.

If dynamically allocated and no alias remains:

```
TEXT
memory leak
```

45. GATE TRAP — LINKED STACK EMPTY

Array:

```
TEXT
top == -1
```

Linked:

```
TEXT
top == NULL
```

Do not mix representations.

46. GATE TRAP — FULL CONDITION

There is no simple condition like:

```
TEXT
top == MAX-1
```

unless you artificially impose a maximum size.

Push fails when node allocation cannot be made or when an imposed capacity is reached.

47. GATE TRAP — POP AFTER FREE

Wrong:

```
C
free(top);
top = top->next;
```

Undefined behavior.

Correct:

```
C
temp = top;
top = top->next;
free(temp);
```

48. GATE TRAP — PUSH ALLOCATION FAILURE

Suppose:

```
C
newNode = malloc(...);
```

returns:

```
TEXT
NULL
```

You must not:

```
C
newNode->data = value;
```

because that dereferences NULL.

Check allocation first.

49. GATE TRAP — STACK ADT COMPLEXITY

The implementation may differ, but a properly designed linked stack still supports:

```
TEXT
push = O(1)
pop  = O(1)
peek = O(1)
```

Do not say linked list traversal is $O(n)$, therefore stack push/pop are $O(n)$.

They occur at head.

50. FREE ENTIRE STACK

At program end:

```
C
void freeStack(Node **top)
{
    while (*top != NULL)
    {
        Node *temp = *top;
        *top = temp->next;
    }
}
```

```
    free(temp);  
  }  
}
```

After:

```
TEXT  
*top = NULL
```

Stack empty.

Time:

```
TEXT  
O(n)
```

51. USING POP TO FREE STACK

Could repeatedly call:

```
TEXT  
pop
```

until empty.

If each pop:

```
TEXT  
O(1)
```

and there are n nodes:

```
TEXT  
O(n)
```

total.

52. APPLICATION LINK

Why does linked-stack implementation matter?

Because later for:

- expression conversion
- evaluation
- parentheses
- DFS

we may need a stack whose capacity isn't conveniently known ahead of time.

Array stack is usually simpler, but linked stack is an important implementation option.

53. GATE-STYLE QUESTION

A stack is implemented with a singly linked list.

Which end should represent the top to guarantee $O(1)$ push and pop?

Answer:

HEAD / BEGINNING

Because insertion/deletion at beginning are both constant time.

54. GATE-STYLE QUESTION

If stack has n elements and is represented as:

TEXT

t o p ! ' n o d e 1 ! ' n o d e 2 ! ' . . . ! ' n o d e N

what is complexity of retrieving bottom element without extra metadata?

Need traverse:

TEXT

n-1 links

So:

TEXT

$O(n)$

Only top is directly accessible.

55. LIFO TRACE

Start empty.

Operations:

TEXT

push(5)
push(8)
push(2)
pop()
push(7)
pop()

After pushes:

TEXT

2 !' 8 !' 5

First pop:

TEXT

2

Remaining:

TEXT

8 !' 5

Push 7:

TEXT

7 !' 8 !' 5

Second pop:

TEXT

7

Remaining:

TEXT

8 !' 5

Final top:

TEXT

8

56. ARRAY VS LINKED TRACE

Push:

TEXT

10 20 30

Array stack:

TEXT

[10][20][30]

top

!'

Linked stack:

TEXT

```
top
!"
3 0 !' 2 0 !' 1 0
```

The physical representation looks reversed because linked push happens at head.

But logical top in both cases is:

TEXT

30

Ø=Nil conceptual point.

57. MEMORY TRICKS

Linked stack

TOP = HEAD

Push

New node becomes HEAD

TEXT

```
new->next = top
top = new
```

Pop

SAVE TOP !' MOVE TOP !' FREE OLD TOP

Empty

No node at TOP !' `NULL`

Why head?

Head gives O(1) in and O(1) out.

58. NOTEBOOK NOTES

Representation

```
C
Node *top = NULL;
```

Push

```
C
new->next = top;
top = new;
```

```
TEXT
O(1)
```

Pop

```
C
temp = top;
top = top->next;
free(temp);
```

```
TEXT
O(1)
```

Peek

```
C
top->data
```

```
TEXT
O(1)
```

59. DON'T CONFUSE

Array empty:

```
TEXT
top == -1
```

Linked empty:

```
TEXT
top == NULL
```

Array overflow:

TEXT

```
top == MAX-1
```

Linked push failure:

TEXT

```
malloc fails
```

Linked top:

TEXT

```
pointer to first node
```

not an integer index.

60. 30-SECOND REVISION

Remember:

1. Linked stack uses a singly linked list.
2. Head is used as top.
3. Empty when `top == NULL`.
4. Push = insert at head.
5. Pop = delete head.
6. Push = $O(1)$.
7. Pop = $O(1)$.
8. Peek = $O(1)$.
9. Linked stack has pointer overhead.
10. It grows dynamically while memory allocation succeeds.

61. ACTIVE RECALL

Q1

Why is head chosen as stack top?

Q2

What is linked-stack empty condition?

Q3

What are the two pointer changes for push?

Q4

What is the safe order for pop?

Q5

Does linked stack have a fixed-size overflow condition like array stack?

ANSWERS

A1

Because both insertion and deletion at head are $O(1)$.

A2

```
C
top == NULL
```

A3

```
C
newNode->next = top;
top = newNode;
```

A4

```
TEXT
SAVE top
MOVE top
FREE old top
```

A5

Not normally. Growth fails when allocation fails or an explicit capacity limit is imposed.

62. PRACTICE — EASY

Start:

```
TEXT
t o p ! ' 2 0 ! ' 1 0 ! ' N U L L
```

Push 30.

Draw final stack.

63. PRACTICE — POP

Start:

```
TEXT
t o p ! ' 4 0 ! ' 3 0 ! ' 2 0 ! ' N U L L
```

Perform one pop.

State:

- popped value
-

- new top
- final list

64. PRACTICE — POINTER SAFETY

Why is this wrong?

```
C
free(top);
top = top->next;
```

Write the correct order.

65. PRACTICE — COMPLEXITY

For linked stack with n nodes, state complexity of:

- push
- pop
- peek
- search arbitrary value
- display all elements
- bottom element access

66. GATE-STYLE QUESTION

A stack is implemented using singly linked list. A pointer to only the first node is maintained.

Which operations can be done in $O(1)$?

A. Push B. Pop C. Peek D. Search arbitrary value

Answer:

```
TEXT
A, B, C
```

Search remains:

```
TEXT
O(n)
```

PROGRESS

MODULE 4 — STACK

' Lesson 37 — Fundamentals ' Lesson 38 — Array Implementation ' **Lesson 39 — Linked-List Implementation**

NEXT — LESSON 40

Infix, Prefix & Postfix Expressions — Precedence, Associativity & Stack Foundations

Next we enter one of the most important GATE stack areas.

We will learn from zero:

TEXT

Infix:

$A + B$

Prefix:

$+ A B$

Postfix:

$A B +$

Then:

- operator precedence
- associativity
- parentheses
- why postfix/prefix do not need parentheses
- expression trees intuition
- operand/operator ordering
- infix vs prefix vs postfix
- common GATE traps
- manual conversions

After that:

- Lesson 41 — **Infix !' Postfix/Prefix Conversion using Stack**
- Lesson 42 — **Postfix & Prefix Evaluation**

USER

next

ASSISTANT

MODULE 4 — LESSON 40

Infix, Prefix & Postfix Expressions — Precedence, Associativity & Stack Foundations

This is one of the most important stack topics for GATE.

Before we learn conversion algorithms, you must clearly understand:

- what infix means
- what prefix means
- what postfix means
- precedence
- associativity
- operand order
- why prefix/postfix are unambiguous
- how expressions are evaluated conceptually

GATE importance: 0=1% NOT IMPORTANT

1. WHERE WE ARE

Completed:

' Stack fundamentals ' Stack using array ' Stack using linked list

Now:

Expression Representation

The same arithmetic expression can be written in three common forms:

TEXT

- Infix
- Prefix
- Postfix

2. START WITH A SIMPLE EXPRESSION

Suppose:

TEXT

$$A + B$$

This is the normal mathematical notation we use.

Operator:

TEXT

+

appears **between** operands.

Therefore:

INFIX

TEXT

A + B

3. PREFIX

Move operator before its operands:

TEXT

+ A B

This is called:

PREFIX

because operator comes **before** operands.

4. POSTFIX

Move operator after operands:

TEXT

A B +

This is called:

POSTFIX

because operator comes **after** operands.

5. MASTER MEMORY TRICK

For:

TEXT

A + B

Infix

TEXT

A + B

Operator **IN** the middle.

Prefix

TEXT

+ A B

Operator comes **PRE** = before.

Postfix

TEXT

A B +

Operator comes **POST** = after.

Ø=Memorize:

****PREFIX !' operator first**

INFIX !' operator middle POSTFIX !' operator last**

6. BASIC TABLE

Form	Expression
Infix	A + B
Prefix	+ A B
Postfix	A B +

7. ANOTHER EXAMPLE

Infix:

TEXT

A * B

Prefix:

TEXT

* A B

Postfix:

TEXT

A B *

8. WHY DO WE NEED DIFFERENT FORMS?

Consider:

TEXT

A + B * C

What happens first?

Multiplication:

TEXT

B * C

Then:

TEXT

A + result

To interpret infix correctly, we need rules like:

- precedence
- associativity
- parentheses

Prefix and postfix encode the order more explicitly.

9. OPERATOR PRECEDENCE Ø=Ý%

Typical arithmetic precedence:

From highest to lowest:

TEXT

()
^
* / %
+ -

A useful memory:

BODMAS / PEMDAS intuition

But for GATE/C-expression conversion, think directly in operator levels.

10. COMMON PRECEDENCE TABLE

Operator	Meaning	Precedence
()	Parentheses	Highest grouping
^	Exponent	High
* / %	Multiply/Divide/Modulo	Medium
+ -	Add/Subtract	Low

⚠️ In actual C, `^` is bitwise XOR, not exponentiation. In data-structure expression-conversion questions, `^` is often used **mathematically as exponent** by convention.

⚠️ Do not confuse C operator meaning with expression-conversion notation.

11. PRECEDENCE EXAMPLE

Expression:

```
TEXT
A + B * C
```

Because:

```
TEXT
* > +
```

evaluate:

```
TEXT
B * C
```

first.

Equivalent grouping:

```
TEXT
A + (B * C)
```

12. POSTFIX OF `A + B * C`

First:

```
TEXT
B * C
```

becomes:

```
TEXT
B C *
```

Then add A:

```
TEXT
A B C * +
```

Therefore:

POSTFIX

TEXT
ABC*+

or spaced:

TEXT
A B C * +

13. PREFIX OF A + B * C

Grouping:

TEXT
A + (B * C)

Inner:

TEXT
B * C

prefix:

TEXT
* B C

Whole:

TEXT
+ A * B C

So:

PREFIX

TEXT
+A*BC

14. PARENTHESES CAN CHANGE EVERYTHING

Compare:

TEXT
A + B * C

with:

TEXT

$(A + B) * C$

First expression groups:

TEXT

$A + (B * C)$

Second groups:

TEXT

$(A+B) * C$

Different answers.

15. POSTFIX OF $(A+B) * C$

First:

TEXT

$A+B$

!'

TEXT

$AB+$

Then multiply by C:

TEXT

$AB+C*$

So:

TEXT

$A B + C *$

16. PREFIX OF $(A+B) * C$

First:

TEXT

$A+B$

!'

TEXT

+AB

Then multiply:

TEXT

*+ABC

So:

TEXT

* + A B C

17. COMPARE THE TWO EXPRESSIONS

A + B * C

Postfix:

TEXT

ABC*+

Prefix:

TEXT

+A*BC

(A+B) * C

Postfix:

TEXT

AB+C*

Prefix:

TEXT

*+ABC

Paraphrasing: Parentheses change the expression structure.

18. ASSOCIATIVITY

What if operators have the same precedence?

Example:

TEXT

$A - B - C$

Both `-` operators have same precedence.

So associativity decides grouping.

For subtraction:

TEXT

left associative

Therefore:

TEXT

$(A - B) - C$

not:

TEXT

$A - (B - C)$

19. LEFT ASSOCIATIVITY

Operators commonly left-associative:

TEXT

$+ \\ - \\ * \\ / \\ \%$

Therefore:

TEXT

$A / B * C$

groups as:

TEXT

$(A / B) * C$

not:

TEXT

$A / (B * C)$

because `/` and `*` have same precedence and are processed left to right.

20. EXPONENT ASSOCIATIVITY

Exponentiation is commonly treated as:

RIGHT ASSOCIATIVE

So:

TEXT

$A \wedge B \wedge C$

means:

TEXT

$A \wedge (B \wedge C)$

not:

TEXT

$(A \wedge B) \wedge C$

Ø=No infix-conversion trap.

21. LEFT VS RIGHT ASSOCIATIVE

Left associative

TEXT

$A - B - C$

means:

TEXT

$(A - B) - C$

Right associative

TEXT

$A \wedge B \wedge C$

means:

TEXT

$A \wedge (B \wedge C)$

22. DON'T CONFUSE PRECEDENCE AND ASSOCIATIVITY

Precedence answers:

Which different operator type wins?

Example:

TEXT

A + B * C

* wins over +.

Associativity answers:

When operators have equal precedence, which direction groups?

Example:

TEXT

A - B - C

left to right.

Very important distinction.

23. PREFIX/POSTFIX DO NOT NEED NORMAL PARENTHESES

Consider postfix:

TEXT

A B C * +

We know:

1. B C *
2. then A + result

No parentheses needed.

Similarly prefix:

TEXT

+ A * B C

The structure is already encoded by operator positions and arity.

One major advantage of prefix/postfix.

24. WHY POSTFIX IS UNAMBIGUOUS

Suppose:

TEXT

A B + C *

Scan conceptually:

TEXT

A B +

produces:

TEXT

(A+B)

then:

TEXT

C *

produces:

TEXT

(A+B)*C

No precedence table is needed during postfix evaluation.

25. WHY PREFIX IS UNAMBIGUOUS

Prefix:

TEXT

* + A B C

First operator:

TEXT

*

needs two operands.

First operand is expression:

TEXT

+ A B

Second is:

TEXT

C

Thus:

TEXT

$(A+B)*C$

Again structure is explicit.

26. OPERAND ORDER MATTERS Ø=Ý%

For commutative operations like:

TEXT

$A+B$

order may not visibly matter.

But for:

TEXT

$A-B$

A/B

it matters greatly.

Postfix:

TEXT

$A\ B\ -$

means:

TEXT

$A - B$

not:

TEXT

$B - A$

Ø=Ý% becomes critical in evaluation.

27. POSTFIX EVALUATION OPERAND ORDER PREVIEW

Suppose stack pops:

TEXT

`operand2 = pop()`

`operand1 = pop()`

For operator `-`:

TEXT

operand1 - operand2

not:

TEXT

operand2 - operand1

Example:

TEXT

8 3 -

Correct:

TEXT

8 - 3 = 5

not 3-8.

We'll master this in Lesson 42.

28. PREFIX OPERAND ORDER PREVIEW

For prefix evaluation, scanning direction differs.

But the same mathematical structure must preserve:

TEXT

left operand
operator
right operand

Be especially careful for:

TEXT

-
/
^

29. MANUAL CONVERSION METHOD — THINK IN SUBEXPRESSIONS

Before learning stack algorithms, use this reliable conceptual method:

1. Identify smallest subexpression.
2. Convert it.
3. Treat result as one operand.
4. Continue outward.

Example:

TEXT

$A + B * C$

Smallest/highest-priority:

TEXT

$B * C$

Postfix:

TEXT

$BC *$

Then:

TEXT

$A + [BC *]$

becomes:

TEXT

$ABC * +$

30. EXAMPLE — $A * B + C$

Precedence:

TEXT

$*$

first.

Grouping:

TEXT

$(A * B) + C$

Postfix:

TEXT

$AB * C +$

Prefix:

TEXT

$+ * ABC$

31. EXAMPLE — $A * (B + C)$

Parentheses first.

$B + C$:

Postfix:

TEXT

$BC +$

Then multiply A:

TEXT

$ABC + *$

Prefix:

TEXT

$*A + BC$

32. EXAMPLE — $(A + B) * (C - D)$

Left subexpression:

TEXT

$A + B$

Postfix:

TEXT

$AB +$

Right:

TEXT

$C - D$

Postfix:

TEXT

$CD -$

Multiply:

TEXT

$AB + CD - *$

33. PREFIX OF SAME EXPRESSION

TEXT

$(A+B) * (C-D)$

Left:

TEXT

$+AB$

Right:

TEXT

$-CD$

Whole:

TEXT

$*+AB-CD$

Classic conversion example.

34. EXPRESSION TREE INTUITION

Expression:

TEXT

$(A+B) * C$

can be represented:

TEXT



Now observe:

Inorder traversal

TEXT

$A + B * C$

with parentheses needed to preserve exact grouping.

Preorder

TEXT

$* + A B C$

= prefix.

Postorder

TEXT

A B + C *

= postfix.

Extremely powerful relationship.

35. FORMULA BOX — TREE CONNECTION

For an expression tree:

TEXT

I n o r d e r	!	I n f i x
P r e o r d e r	!	P r e f i x
P o s t o r d e r	!	P o s t f i x

This will become even more useful during Trees.

36. EXAMPLE TREE

Expression:

TEXT

A + B * C

Tree:

TEXT



Preorder:

TEXT

+ A * B C

Prefix.

Postorder:

TEXT

A B C * +

Postfix.

37. OPERANDS AND OPERATORS

Operands can be:

TEXT

A
B
x
y
5
23
variables
constants

Operators:

TEXT

+
-
*
/
%
^

In simple GATE problems, operands are often single characters to make conversion easy.

38. MULTI-DIGIT OPERANDS NEED SEPARATION

Postfix:

TEXT

12 34 +

is clear.

Without spaces:

TEXT

1234+

we don't know whether operands are:

TEXT

1, 2, 3, 4

or:

TEXT

12, 34

So practical tokenization matters.

Ø=GATE often uses single-letter operands to avoid this issue.

39. UNARY VS BINARY OPERATORS

Most beginner conversion problems assume binary operators:

TEXT

A+B
A-B
A*B

Each operator has exactly two operands.

Unary operators complicate parsing.

Example:

TEXT

-A

has one operand.

Unless question explicitly includes unary operators, basic stack-conversion algorithms generally assume standard binary operators.

40. GATE TRAP — $A+B*C$

Do not convert left-to-right blindly.

Wrong idea:

TEXT

$AB+C*$

That represents:

TEXT

$(A+B)*C$

But original is:

TEXT

$A+(B*C)$

Correct postfix:

TEXT

$ABC*+$

Precedence matters before conversion.

41. GATE TRAP — $(A+B)*C$

Correct postfix:

TEXT
AB+C*

not:

TEXT
ABC*+

Parentheses override precedence.

42. GATE TRAP — A-B-C

Because subtraction is left-associative:

TEXT
(A-B)-C

Postfix:

TEXT
AB-C-

Prefix:

TEXT
--ABC

Let's verify prefix:

TEXT
(A-B)-C

Left:

TEXT
-AB

whole:

TEXT
- - A B C

!'

TEXT
--ABC

43. COMPARE A - (B - C)

This is different.

Postfix:

TEXT
ABC - -

Let's derive:

B - C :

TEXT
BC -

Then:

TEXT
A [BC -] -

!'

TEXT
ABC - -

Prefix:

TEXT
-A - BC

Some operators, different grouping.

44. GATE TRAP — EXPONENT

TEXT
 $A^B{}^C$

Assuming $\wedge$ exponent and right associativity:

TEXT
 $A^{(B^C)}$

Postfix:

TEXT
 $ABC^{^^}$

Prefix:

TEXT
 $^A^B C$

45. WHAT IF $(A^B)^C$?

Explicit parentheses force:

TEXT
 $(A^B)^C$

Postfix:

TEXT
 $A B^C^A$

Prefix:

TEXT
 $^A^A B C$

Compare:

TEXT
 $A^B (A^C)$
Postfix = $A B C^A$

versus:

TEXT
 $(A^B)^C$
Postfix = $A B^A C^A$

~~Very~~ high-value associativity trap.

46. EXAMPLE — $A+B^*C-D$

Precedence:

TEXT
 B^*C

First.

Expression becomes:

TEXT
 $A + (B^*C) - D$

Since $+$ and $-$ same precedence and left associative:

TEXT

$(A + (B * C)) - D$

Postfix:

TEXT

$ABC * + D -$

Prefix:

TEXT

$- + A * B C D$

47. EXAMPLE — $A * (B + C) / D$

Parentheses:

TEXT

$B + C$

Then multiplication/division are same precedence, left associative:

TEXT

$(A * (B + C)) / D$

Postfix:

TEXT

$ABC + * D /$

Prefix:

TEXT

$/ * A + B C D$

Let's read prefix spaced:

TEXT

$/ * A + B C D$

48. EXAMPLE — $A + B * (C - D)$

Inner:

TEXT

$C - D$

Postfix:

TEXT

CD-

Then:

TEXT

B * (C-D)

!'

TEXT

BCD- *

Then add A:

TEXT

ABCD- *+

Prefix:

TEXT

+A*B-CD

49. EXAMPLE — $(A+B)/(C-D)$

Postfix:

TEXT

AB+CD- /

Prefix:

TEXT

/+AB-CD

50. HOW STACK WILL HELP

When converting infix to postfix:

- operands go directly to output
- operators wait on stack
- parentheses control when operators leave stack
- precedence/associativity decide whether to pop existing operators

Example conceptually:

TEXT

A + B * C

When + arrives:

TEXT

stack: +

When * arrives:

Since * has higher precedence:

TEXT

keep + underneath
push *

At end:

TEXT

pop *
then +

producing:

TEXT

ABC*+

Lesson 41 will do this systematically.

51. WHY OPERATORS NEED A STACK

Consider:

TEXT

A+B*C

When we see +, we cannot output it immediately because we don't yet know whether a higher-precedence operation will appear.

So + waits.

Later * arrives and must occur first.

Stack helps temporarily hold pending operators.

One intuition for infix-to-postfix conversion.

52. PARENTHESES IN OPERATOR STACK

For:

TEXT

$A * (B + C)$

when (appears:

TEXT

push (

Operators inside parentheses remain isolated until:

TEXT

)

appears.

Then pop until matching (.

The parentheses themselves are not included in postfix output.

Ø=important preview.

53. POSTFIX DOES NOT CONTAIN PARENTHESES

For:

TEXT

$(A + B) * C$

postfix:

TEXT

$AB + C *$

not:

TEXT

$(AB +) C *$

Parentheses are unnecessary because order is encoded.

Similarly prefix:

TEXT

$* + ABC$

54. PREFIX CONVERSION STRATEGY PREVIEW

One common algorithmic method:

1. reverse infix
2. swap (with)

3. convert modified expression to postfix using appropriately handled associativity

4. reverse result

But there are subtleties around equal precedence and right-associative operators.

We will cover this carefully in Lesson 41 rather than blindly using a shortcut.

55. GATE QUESTION — IDENTIFY FORM

Expression:

TEXT

$AB+C^*$

Operators come after operands/subexpressions.

So:

TEXT

Postfix

It represents:

TEXT

$(A+B)*C$

56. IDENTIFY PREFIX

Expression:

TEXT

$*+ABC$

First operator is $*$.

Its first operand expression:

TEXT

$+AB$

Second operand:

TEXT

C

Therefore:

TEXT

$(A+B)*C$

57. IDENTIFY POSTFIX STRUCTURE

Expression:

TEXT
ABC*+

Read:

TEXT
B C *

!' B*C

Then:

TEXT
A [B*C] +

!'

TEXT
A+B*C

58. VALIDITY INTUITION — POSTFIX

For binary operators, scanning postfix from left to right:

- operand increases available operand count by 1
- operator needs 2 available operands and reduces count by 1

At the end, a valid single expression should leave exactly:

TEXT
1 result

This becomes a useful validation/evaluation property.

59. EXAMPLE VALID POSTFIX

TEXT
A B + C *

Stack-count reasoning:

TEXT
A !' 1
B !' 2
+ !' c o n s u m e s 2 , p r o d u c e s 1 !' 1
C !' 2
* !' 1

Valid.

60. INVALID POSTFIX EXAMPLE

TEXT

A + B

Scan:

TEXT

A ! ' o n e o p e r a n d
+ ! ' n e e d s t w o

Invalid postfix expression.

0-6% Good conceptual check.

61. PREFIX VALIDITY INTUITION

For prefix, scanning naturally from right to left gives a similar property.

Operand:

TEXT

push

Operator:

TEXT

needs two operands/results

Exactly why stacks work so naturally for evaluation.

62. GATE TRAP — INFIX IS NOT ALWAYS FULLY PARENTHESESIZED

Expression:

TEXT

A+B*C

is infix even without parentheses.

Precedence rules determine structure.

A fully parenthesized form would be:

TEXT

A+(B*C)

63. FULLY PARENTHESESIZED INFIX

An expression like:

TEXT

$((A+B)*(C-D))$

makes every operation's grouping explicit.

Then conversion becomes much easier because precedence ambiguity disappears.

64. QUICK CONVERSION TABLE

Infix	Prefix	Postfix
A+B	+AB	AB+
A-B	-AB	AB-
A*B	*AB	AB*
A+B*C	+A*BC	ABC*+
(A+B)*C	*+ABC	AB+C*
(A+B)*(C-D)	*+AB-CD	AB+CD-*

Copy this table.

65. PRECEDENCE + ASSOCIATIVITY TABLE

For our expression-conversion convention:

Operator	Precedence	Associativity
$\wedge$	Highest operator	Right
$*$ / $\%$	Next	Left
$+$ -	Lowest	Left

Parentheses override these rules.

66. DON'T CONFUSE

Precedence

TEXT

* before +

Associativity

TEXT

- groups left
^ groups right

Prefix

TEXT

operator before operands

Postfix

TEXT

operator after operands

Parentheses

Needed in infix for explicit grouping, normally unnecessary in valid prefix/postfix.

67. GATE TRAP — C'S $\wedge$

Again:

In C:

c
a $\wedge$ b

means:

TEXT

bitwise XOR

Not exponentiation.

In many DS textbook/GATE-style expression examples, $\wedge$ is treated symbolically as exponent operator.

Do not interpret according to the problem's notation.

68. EXPRESSION TREE CONNECTION — MUST REMEMBER

TEXT

OP
/ \
left right

Then:

Prefix

TEXT

OP left right

Infix

TEXT

left OP right

Postfix

TEXT

left right OP

This is the deepest intuition behind all three forms.

69. MEMORY TRICK

Imagine operator is the parent:

TEXT

```
  +
 / \
A   B
```

Parent first

TEXT

+AB

= PREFIX / preorder.

Parent middle

TEXT

A+B

= INFIX / inorder.

Parent last

TEXT

AB+

= POSTFIX / postorder.

Excellent memory connection.

70. NOTEBOOK NOTES

Forms

TEXT

Infix: $A+B$
Prefix: $+AB$
Postfix: $AB+$

Precedence

TEXT

$\wedge$
 $* / \%$
 $+ -$

Associativity

TEXT

$\wedge$	!	r	i	g	h	t
$*$	!	l	e	f	t	
$+$	!	l	e	f	t	

Tree relation

TEXT

Inorder = infix
Preorder = prefix
Postorder = postfix

71. 30-SECOND REVISION

Remember:

1. Infix operator is between operands.
2. Prefix operator comes before operands.
3. Postfix operator comes after operands.
4. $* / \%$ have higher precedence than $+ -$.
5. $\wedge$ is commonly highest and right-associative in DS notation.
6. $+ - * / \%$ are generally left-associative.
7. Parentheses override precedence.
8. Prefix/postfix normally don't need parentheses.
9. Expression tree preorder gives prefix.
10. Expression tree postorder gives postfix.

72. ACTIVE RECALL

Q1

Convert:

TEXT

A+B

to prefix and postfix.

Q2

What is postfix of:

TEXT

A+B*C

?

Q3

What is postfix of:

TEXT

(A+B)*C

?

Q4

How does:

TEXT

A-B-C

associate?

Q5

How does:

TEXT

A^B^C

associate under standard exponent notation?

ANSWERS

A1

TEXT

Prefix = +AB

Postfix = AB+

A2

TEXT
ABC*+

A3

TEXT
AB+C*

A4

TEXT
(A-B)-C

A5

TEXT
 $A^{\wedge}(B^{\wedge}C)$

73. PRACTICE — EASY

Convert to postfix:

A

TEXT
A-B

B

TEXT
 $A*B+C$

C

TEXT
 $A*(B+C)$

D

TEXT
 $(A+B)*(C-D)$

74. PRACTICE — PREFIX

Convert:

A

TEXT

A/B

B

TEXT

$A+B*C$

C

TEXT

$(A-B)/C$

D

TEXT

$A*(B+C-D)$

Be careful with left associativity inside:

TEXT

$B+C-D$

75. PRACTICE — ASSOCIATIVITY

Write the fully parenthesized form.

A

TEXT

$A-B+C$

Answer should follow equal-precedence left associativity.

B

TEXT

$A/B*C$

C

TEXT

A^B^C

D

TEXT

A^B*C+D

76. GATE-STYLE QUESTION

Which postfix expression represents:

TEXT

$A * (B + C)$

A.

TEXT

$AB * C +$

B.

TEXT

$ABC + *$

C.

TEXT

$AB + C *$

D.

TEXT

$*ABC +$

Correct:

$*B \text{ — } `ABC + `$**

Because:

TEXT

$B + C$

must occur before multiplication by A.

77. GATE-STYLE TRAP

Which infix expression corresponds to:

TEXT

$AB - C -$

Start:

TEXT

AB-

means:

TEXT

A-B

Then:

TEXT

[A-B] C -

means:

TEXT

(A-B) - C

Not:

TEXT

A - (B - C)

Prefix encodes associativity directly.

78. PLACEMENT MINI-CHALLENGE

Identify the form and reconstruct infix.

A

TEXT

AB+CD - *

B

TEXT

*+AB - CD

Both represent:

TEXT

(A+B) * (C-D)

A is postfix.

B is prefix.

PROGRESS

MODULE 4 — STACK

' Lesson 37 — Stack Fundamentals ' Lesson 38 — Stack Using Array ' Lesson 39 — Stack Using Linked List '
Lesson 40 — Infix, Prefix & Postfix Fundamentals

NEXT — LESSON 41

Infix !' Postfix and Prefix Conversion Using Stack

Next is the main GATE conversion lesson.

We will master:

- operand !' output
- operator !' stack
- (push
-) pop until (
- precedence comparison
- equal-precedence handling
- left vs right associativity
- exact conversion tables
- infix !' postfix algorithm
- infix !' prefix algorithm
- examples from easy to tricky
- $A+B*C$
- $(A+B)*(C-D)$
- A^B^C
- nested parentheses
- GATE traps

The key rule we will formalize is:

Before pushing an operator, decide which operators already on the stack must come out first.

USER

next

ASSISTANT

MODULE 4 — LESSON 41

Infix !' Postfix & Prefix Conversion Using Stack

This is one of the **highest-value GATE stack lessons**.

Today we convert infix expressions like:

TEXT

$$A + B \rightarrow C$$

into:

TEXT

Postfix: A B C * +

Prefix: + A * B C

We will do it using a **stack**, not by guessing.

GATE importance: $\emptyset = \gamma\%$ MUST KNOW Placement importance: $\emptyset = \gamma\%$ IMPORTANT

1. WHERE WE ARE

You already know:

TEXT

Infix: $A + B$

Prefix: + A B

Postfix: A B +

and:

- precedence
- associativity
- parentheses
- expression-tree intuition

Now we learn the actual conversion algorithm.

2. WHY DO WE NEED A STACK?

Consider:

TEXT

$$A + B \rightarrow C$$

When we encounter:

TEXT

+

we cannot immediately place it in postfix output.

Why?

Because later we discover:

TEXT

*

which has higher precedence.

So some operators must temporarily wait.

Where?

STACK

Memory trick:

Operands go OUT. Operators WAIT.

3. INFIX !' POSTFIX MASTER RULES Ø=Ý%

While scanning infix from **left to right**:

Rule 1 — Operand

Send directly to output.

TEXT

A !' o u t p u t

Rule 2 — (

Push onto stack.

Rule 3 —)

Pop operators to output until (is found.

Discard the matching parentheses.

Rule 4 — Operator

Before pushing it, pop operators that must be performed before it according to:

- precedence
- associativity

Rule 5 — End

Pop all remaining operators to output.

Ø=Ý% These five rules solve infix-to-postfix.

4. PRECEDENCE TABLE

For our standard DS notation:

Operator	Precedence	Associativity
$\wedge$	Highest	Right
$* / \%$	Next	Left
$+ -$	Lowest	Left

Parentheses control grouping.

5. THE MOST IMPORTANT OPERATOR RULE

Suppose current operator is:

TEXT

op

and stack top contains:

TEXT

topOp

For infix !' postfix, pop `topOp` before pushing `op` if:

TEXT

`precedence(topOp) > precedence(op)`

OR if:

TEXT

`precedence(topOp) == precedence(op)`

and the **current operator is left-associative**.

~~0-1%~~ This equal-precedence rule is where many mistakes happen.

6. EASY EXAMPLE — `A+B`

Scan left to right.

Initial:

TEXT

Output:

Stack:

Read `A`:

TEXT

Output: `A`

Stack:

Read +:

TEXT

Output: A
Stack: +

Read B:

TEXT

Output: AB
Stack: +

End:

Pop +.

TEXT

Output: AB+

Answer

TEXT

Postfix = AB+

7. EXAMPLE — A+B*C

Expression:

TEXT

A + B * C

We know answer should respect multiplication first.
Let's use the stack.

8. STEP TABLE

Symbol	Action	Stack	Output
A	operand ' output	—	A
+	push	+	A
B	output	+	AB
*	higher than +, push	+ *	AB
C	output	+ *	ABC
End	pop all	—	ABC*+

So:

POSTFIX

TEXT

ABC*+

9. WHY DID + STAY?

When current operator was:

TEXT

*

stack top was:

TEXT

+

Since:

TEXT

precedence(*) > precedence(+)

* must happen first.

So we simply push * above +.

Stack:

TEXT

TOP

!“

*

+

At the end:

TEXT

* pops before +

giving correct order.

10. EXAMPLE — A*B+C

Scan:

TEXT

A * B + C

After reading $A*B$:

TEXT

Output: AB

Stack: *

Now current operator is:

TEXT

+

Stack top:

TEXT

*

Since:

TEXT

* > +

pop * first.

Output becomes:

TEXT

AB*

Then push +.

Continue with C.

Final:

TEXT

AB*C+

High-precedence operators leave the stack before lower-precedence operators are pushed.

11. PARENTHESES — $A*(B+C)$

Expression:

TEXT

A * (B + C)

12. CONVERSION TABLE

Symbol	Action	Stack	Output
A	output	—	A
*	push	*	A
(	push	* (	A
B	output	* (	AB
+	push	* (+	AB
C	output	* (+	ABC
)	pop until (	*	ABC+
End	pop *	—	ABC+*

Answer

TEXT

Postfix = ABC+*

13. WHY PARENTHESES DO NOT APPEAR IN POSTFIX

They only guide operator ordering.

When:

TEXT

)

is found:

1. pop operators until (
2. discard (
3. do not output either parenthesis

So:

TEXT

(A+B)

becomes simply:

TEXT

AB+

14. EXAMPLE — (A+B) * (C-D)

Expression:

TEXT

$(A+B) * (C-D)$

Let's convert.

15. STEP TABLE

Symbol	Stack	Output
(	(	
A	(	A
+	(+	A
B	(+	AB
)	—	AB+
*	*	AB+
(	* (	AB+
C	* (	AB+C
-	* (-	AB+C
D	* (-	AB+CD
)	*	AB+CD-
End	—	AB+CD-*

Answer

TEXT

Postfix = $AB+CD-*$

0-0% Classic GATE expression.

16. EQUAL PRECEDENCE — LEFT ASSOCIATIVITY

Consider:

TEXT

$A-B-C$

Subtraction is left-associative:

TEXT
(A-B)-C

Expected postfix:

TEXT
AB-C-

Let's see why.

17. TRACE

Read first - :

TEXT
Stack: -
Output: A

Read B:

TEXT
Output: AB

Now second - arrives.

Stack top is also:

TEXT
-

Same precedence.

Because subtraction is **left associative**, pop old - first.

TEXT
Output: AB-

Then push new - .

Finally C and pop.

TEXT
AB-C-

Equal precedence + left associativity means:

POP FIRST

18. EXAMPLE — A/B*C

`/` and `*` have equal precedence and are left-associative.

Expression groups:

TEXT
 $(A/B)*C$

During second operator `*`, stack has `/`.

Same precedence + left associative:

TEXT
pop `/`

So postfix:

TEXT
 $AB/C*$

not:

TEXT
 $ABC*/$

19. RIGHT ASSOCIATIVITY — BIG TRAP $\emptyset = \dot{Y}\% \emptyset = \dot{Y}\% \emptyset = \dot{Y}\%$

Consider:

TEXT
 $A^B{}^C$

Exponentiation is right-associative:

TEXT
 $A^{\left(B^C\right)}$

Expected postfix:

TEXT
 $ABC^{^^}$

20. WHY WE DO NOT POP EQUAL `^`

After reading:

TEXT
 $A \wedge B$

stack contains:

TEXT

^

Then another ^ arrives.

Equal precedence.

But current operator ^ is **right associative**.

So we do **not** pop the previous ^.

Instead push another:

TEXT

TOP

!"

^

^

Then C.

At end both operators pop:

TEXT

ABC^^

☐=☐ postfix conversion:

Equal-precedence right-associative operator stays stacked.

21. LEFT VS RIGHT RULE

Current operator left-associative

On equal precedence:

TEXT

POP

Current operator right-associative

On equal precedence:

TEXT

DO NOT POP

Memory trick:

LEFT resolves now. RIGHT waits.

22. MASTER INFIX !' POSTFIX ALGORITHM

For each token `x`:

TEXT

```
If x is operand:
    output x

Else if x == '(':
    push x

Else if x == ')':
    while stack top != '(':
        output pop()
    remove '('

Else if x is operator:
    while stack not empty
        and top != '('
        and (
            precedence(top) > precedence(x)
            OR
            precedence(top) == precedence(x)
            AND x is left-associative
        ):
        output pop()

    push x

At end:
    pop all operators to output
```

Learn the logic, not just code.

23. C-LIKE HELPER — PRECEDENCE

```
C
int precedence(char op)
{
    if (op == '^')
        return 3;

    if (op == '*' || op == '/' || op == '%')
        return 2;

    if (op == '+' || op == '-')
        return 1;

    return 0;
}
```

24. ASSOCIATIVITY HELPER

```
C
int isRightAssociative(char op)
{
    return op == '^';
}
```



```
}
```

For our standard notation.

25. CORE CONDITION IN CODE

Conceptually:

```
C
while (top != -1 &&
      stack[top] != '(' &&
      (precedence(stack[top]) > precedence(current) ||
       (precedence(stack[top]) == precedence(current) &&
        !isRightAssociative(current))))
{
    output[k++] = stack[top--];
}
```

Then:

```
C
stack[++top] = current;
```

0-1% This is the critical part.

26. GATE TRAP — USING `>=` BLINDLY

Many students memorize:

```
TEXT
while precedence(top) >= precedence(current):
    pop
```

This fails for:

```
TEXT
A^B^C
```

because equal `^` operators should not pop under right associativity.

So:

```
TEXT
>=
```

cannot be used blindly unless associativity is handled separately.

0-1% Very important.

27. TRICKY EXAMPLE — `A+B*C-D/E`

Precedence groups:

TEXT
(A + B*C) - (D/E)

Let's convert.

B*C :

TEXT
BC*

Then A +:

TEXT
ABC*+

D/E :

TEXT
DE/

Finally subtract:

TEXT
ABC*+DE/-

Let's confirm stack-wise.

28. RESULT

TEXT
Postfix = ABC*+DE/-

Prefix later:

TEXT
-+A*BC/DE

29. TRICKY EXAMPLE — A*(B+C)/D

Expression:

TEXT
(A*(B+C))/D

Postfix:

Inside:

TEXT

B + C ! ' B C +

Multiply A:

TEXT

ABC+*

Then divide D:

TEXT

ABC+*D/

Answer

TEXT

ABC+*D/

30. NESTED PARENTHESES

Expression:

TEXT

A*(B+(C-D))

Innermost:

TEXT

C - D ! ' C D -

Then:

TEXT

B+(C-D)

!'

TEXT

BCD-+

Then multiply A:

TEXT

ABCD-+*

Postfix

TEXT

ABCD - + *

31. GATE TABLE METHOD

During examination, use four columns:

Token	Stack	Output	Reason
-------	-------	--------	--------

Example:

TEXT

A+B*C

This prevents mistakes.

Especially use it for:

- nested parentheses
- repeated operators
- $\wedge$
- mixed equal precedence operators

32. NOW INFIX !' PREFIX $\emptyset = \dot{Y}\% \emptyset = \dot{Y}\% \emptyset = \dot{Y}\%$

Example:

TEXT

A+B*C

Prefix:

TEXT

+A*BC

We can derive it manually from expression structure.

But GATE may ask systematic conversion.

Two main approaches:

Method 1

Use expression tree / recursive grouping.

Method 2

Use a stack-based reverse-and-convert technique.

For exams, both are useful.

33. MANUAL PREFIX METHOD

Take:

TEXT
A+B*C

Highest precedence:

TEXT
B*C

Prefix:

TEXT
*BC

Now:

TEXT
A + [*BC]

Prefix whole expression:

TEXT
+A*BC

For moderate expressions, this is often faster than blindly applying a trick.

34. PREFIX USING EXPRESSION TREE

For:

TEXT
A+B*C

tree:



Preorder:

TEXT
+ A * B C

Therefore prefix:

TEXT

+A*BC

0-50% best conceptual method.

35. COMMON STACK SHORTCUT FOR PREFIX

A widely taught strategy is:

1. Reverse infix token order.
2. Swap (and) .
3. Convert the transformed expression using a modified postfix conversion.
4. Reverse the resulting postfix.

But:

Equal-precedence associativity must be handled correctly.

This is the part people often memorize incorrectly.

36. WHY ASSOCIATIVITY CHANGES DURING THE REVERSE METHOD

Original infix is processed:

TEXT

l e f t ! ' r i g h t

But after reversing it, you're effectively processing the original expression from:

TEXT

r i g h t ! ' l e f t

So the equal-precedence handling must be mirrored.

For prefix conversion through this reversed scan:

Original left-associative operators

On equal precedence in the reversed conversion:

TEXT

DO NOT POP

Original right-associative operator ^

On equal precedence in the reversed conversion:

TEXT

POP

0-70% is opposite to ordinary i n f i x ! ' p o s t f i x equal-precedence handling.

37. PREFIX SHORTCUT RULE

When using:

TEXT

r e v e r s e ! ' s w a p b r a c k e t s ! ' p o s t f i x - l i k e c o n v e r s i o n ! ' r e v

the operator-pop condition becomes:

Pop when:

TEXT

precedence(top) > precedence(current)

or when equal precedence and the **original current operator is right-associative**.

Thus:

- equal +, -, *, /, % !' do not pop
- equal ^ !' pop

This preserves the original expression's associativity.

38. PREFIX EXAMPLE — A+B*C

Original:

TEXT

A+B*C

Reverse:

TEXT

C*B+A

No parentheses to swap.

Convert this using prefix-shortcut equal-precedence rules.

Postfix of transformed expression:

TEXT

CB*A+

Reverse:

TEXT

+A*BC

Correct prefix.

39. PREFIX EXAMPLE — $(A+B)*C$

Original:

TEXT
 $(A+B)*C$

Reverse:

TEXT
 $C*)B+A($

Swap parentheses:

TEXT
 $C*(B+A)$

Convert to postfix:

TEXT
 $CBA+*$

Reverse:

TEXT
 $*+ABC$

Prefix

TEXT
 $*+ABC$

40. PREFIX EXAMPLE — $(A+B)*(C-D)$

Original:

TEXT
 $(A+B)*(C-D)$

Reverse token order:

TEXT
 $)D-C(*)B+A($

Swap parentheses:

TEXT
 $(D-C)*(B+A)$

Postfix of transformed:

TEXT
DC-BA+*

Reverse:

TEXT
*+AB-CD

Prefix

TEXT
*+AB-CD

Doesn't match expression-tree method.

41. CRITICAL PREFIX ASSOCIATIVITY EXAMPLE — A-B-C

Original is left associative:

TEXT
(A-B)-C

Correct prefix:

TEXT
--ABC

Let's verify the reverse technique.

Original:

TEXT
A-B-C

Reverse:

TEXT
C-B-A

During transformed postfix conversion, equal - must **not** pop under the prefix shortcut.

This makes the transformed grouping effectively:

TEXT
C-(B-A)

Postfix:

TEXT

CBA - -

Reverse:

TEXT

- -ABC

Correct.

⚠️ If you blindly use ordinary left-associative popping here, you can get the wrong prefix.

42. CRITICAL PREFIX EXPONENT EXAMPLE — $A^B{}^C$

Original right-associative:

TEXT

$A^B{}^C$

Correct prefix:

TEXT

${}^A{}^B C$

Reverse:

TEXT

$C{}^B{}^A$

For the prefix shortcut, equal A **must pop**.

Transformed postfix becomes:

TEXT

$C B {}^A {}^A$

Reverse:

TEXT

${}^A {}^A B C$

Correct.

⚠️ This is why prefix conversion requires careful associativity handling.

43. SAFEST GATE STRATEGY FOR PREFIX

For small-to-medium expressions:

Understand the grouping first, then build prefix from the expression tree/subexpressions.

For long expressions:

Use the reverse + swap + modified-postfix algorithm carefully.

Do not blindly memorize:

```
TEXT
reverse !' postfix !' reverse
```

without associativity rules.

44. PREFIX — DIRECT STRUCTURAL RULE

For any binary expression:

```
TEXT
LEFT op RIGHT
```

prefix is:

```
TEXT
op PREFIX(LEFT) PREFIX(RIGHT)
```

Postfix is:

```
TEXT
POSTFIX(LEFT) POSTFIX(RIGHT) op
```

Ø=1% This recursive rule is mathematically clean.

45. EXAMPLE — $A * (B + C - D)$

Inside parentheses:

```
TEXT
B+C-D
```

Since $+$ and $-$ are left-associative:

```
TEXT
(B+C)-D
```

Prefix of inner:

```
TEXT
-+BCD
```

Whole expression:

TEXT

$A * [(B+C) - D]$

Prefix:

TEXT

$*A - +BCD$

Postfix:

Inner:

TEXT

$BC+D -$

Whole:

TEXT

$ABC+D - *$

46. EXAMPLE — $A/B * C + D$

Same-precedence $/$ and $*$ group left:

TEXT

$((A/B) * C) + D$

Postfix:

TEXT

$AB/C * D +$

Prefix:

TEXT

$+ * / ABCD$

Spaced:

TEXT

$+ * / A B C D$

47. EXAMPLE — $A + B * (C^D - E)$

Let's solve carefully.

Inside:

TEXT
 $C^A D - E$

^ first:

TEXT
 $(C^A D) - E$

Then:

TEXT
 $B * ((C^A D) - E)$

Then add A.

48. POSTFIX

$C^A D$:

TEXT
 CD^A

Subtract E:

TEXT
 $CD^A E -$

Multiply B:

TEXT
 $BCD^A E - *$

Add A:

TEXT
 $ABCD^A E - * +$

Postfix

TEXT
 $ABCD^A E - * +$

49. PREFIX

$C^A D$:

TEXT

$\wedge CD$

Subtract E:

TEXT

$- \wedge CDE$

Multiply B:

TEXT

$* B - \wedge CDE$

Add A:

TEXT

$+ A * B - \wedge CDE$

Prefix

TEXT

$+ A * B - \wedge CDE$

50. GATE TRAP — PARENTHESES ON STACK

When current token is:

TEXT

)

you pop until:

TEXT

(

But do **not** send (to output.

It is simply discarded.

Wrong:

TEXT

$AB+ (. . .$

Postfix normally contains no parentheses.

51. GATE TRAP — STACK TOP (

Suppose an operator arrives and stack top is:

TEXT

(

Do not compare normal arithmetic precedence through the parenthesis.

(acts as a boundary.

Push the operator above it.

0-10 Parentheses isolate operators inside the group.

52. GATE TRAP — UNMATCHED PARENTHESES

Expression:

TEXT

A+(B*C

contains unmatched (.

A robust conversion algorithm should detect invalid input rather than simply outputting (as an operator.

Similarly:

TEXT

A+B)

has an unmatched closing parenthesis.

0-10 Validity checking may be part of implementation questions.

53. GATE TRAP — ^ IN C

Again:

For expression-conversion questions:

TEXT

^

often means exponentiation and is right-associative.

In actual C:

TEXT

^

means bitwise XOR and has different precedence rules.

Always follow the notation defined by the problem.

54. GATE TRAP — SAME PRECEDENCE DOES NOT MEAN SAME RESULT

Compare:

TEXT
 $A - B + C$

Since $-$ and $+$ have same precedence and are left-associative:

TEXT
 $(A - B) + C$

Postfix:

TEXT
 $AB - C +$

Prefix:

TEXT
 $+ - ABC$

55. ANOTHER SAME-PRECEDENCE EXAMPLE

TEXT
 $A / B * C$

groups:

TEXT
 $(A / B) * C$

Postfix:

TEXT
 $AB / C *$

Prefix:

TEXT
 $*/ABC$

Do not group:

TEXT
 $A / (B * C)$

56. OPERATOR STACK DOES NOT STORE OPERANDS

In standard infix to postfix conversion:

Operands

TEXT

d i r e c t l y ! ' o u t p u t

Operators / parentheses

TEXT

! ' s t a c k

0-6% Common GATE conceptual question.

57. SPACE COMPLEXITY

For an expression containing n tokens, operator stack can hold up to:

TEXT

$O(n)$

tokens in worst case.

Output also contains:

TEXT

$O(n)$

tokens.

So conversion:

TEXT

Time = $O(n)$
Auxiliary stack space = $O(n)$

assuming each token is processed/pushed/popped a constant number of times.

58. WHY TIME IS $O(n)$, NOT $O(n^2)$

The algorithm has a `while` loop inside the scan, but each operator:

TEXT

is pushed at most once
and
popped at most once

Across the entire expression.

So total stack activity is linear.

This is the same aggregate reasoning you saw with some two-pointer loops.

Important complexity insight.

59. INFIX! POSTMASTER TABLE

Token type	Action
Operand	Send to output
(	Push
)	Pop until (
Higher-precedence operator	Usually push
Lower-precedence operator	Pop stronger stack operators first
Equal precedence + left associative	Pop
Equal precedence + right associative	Don't pop
End	Pop remaining operators

60. INFIX! PRESHORTCUT TABLE

Using reverse + swap + postfix-like conversion:

Equal precedence	Action in transformed scan
Original left-associative operator	Don't pop
Original right-associative operator	Pop

Then reverse generated postfix.

Keep this separate from ordinary postfix conversion.

61. MANUAL CONVERSION SHORTCUT

For exams, you can often solve faster by fully parenthesizing.

Example:

TEXT
A+B*C-D

Fully group:

TEXT
(A+(B*C))-D

Then postfix:

TEXT

A (BC*) + D -

!'

TEXT

ABC*+D-

Prefix:

TEXT

- (+ A (* B C)) D

!'

TEXT

--A*BCD

This is very reliable.

62. EXPRESSION TREE CHECK

If uncertain, mentally build tree.

For:

TEXT

A+B*C-D

tree:

TEXT



Preorder:

TEXT

- + A * B C D

Prefix:

TEXT

--A*BCD

Postorder:

TEXT

A B C * + D -

Postfix:

TEXT

ABC*+D-

Excellent verification technique.

63. GATE-STYLE QUESTION 1

Convert:

TEXT

A+B*(C-D)

Postfix:

TEXT

ABCD- *+

Prefix:

TEXT

+A*B-CD

64. GATE-STYLE QUESTION 2

Convert:

TEXT

(A+B)/(C-D)

Postfix:

TEXT

AB+CD- /

Prefix:

TEXT

/+AB-CD

65. GATE-STYLE QUESTION 3

Convert:

TEXT
 $A^B \wedge C$

assuming exponentiation is right associative.

Postfix:

TEXT
 $ABC \wedge \wedge$

Prefix:

TEXT
 $\wedge \wedge ABC$

~~Must~~ Must remember.

66. GATE-STYLE QUESTION 4

Convert:

TEXT
 $(A \wedge B) \wedge C$

Explicit grouping.

Postfix:

TEXT
 $AB \wedge C \wedge$

Prefix:

TEXT
 $\wedge \wedge ABC$

Compare carefully with previous example.

67. GATE-STYLE QUESTION 5

Convert:

TEXT
 $A - B - C$

Postfix:

TEXT

AB - C -

Prefix:

TEXT

- - ABC

Because:

TEXT

(A - B) - C

68. GATE-STYLE QUESTION 6

Convert:

TEXT

A - (B - C)

Postfix:

TEXT

ABC - -

Prefix:

TEXT

- A - BC

Ø = Same symbols, different structure.

69. COMMON WRONG ANSWERS

For:

TEXT

A + B * C

Wrong postfix:

TEXT

AB + C *

because that means:

TEXT
(A+B)*C

Correct:

TEXT
ABC*+

70. COMMON WRONG PREFIX

For:

TEXT
A+B*C

Wrong:

TEXT
*+ABC

That represents:

TEXT
(A+B)*C

Correct:

TEXT
+A*BC

71. MEMORY TRICKS

Postfix conversion

Operand OUT, operator WAIT.

Parentheses

Open enters, close empties until open.

Precedence

Stronger stack operator comes out first.

Equal precedence

LEFT resolves now; RIGHT waits.

for ordinary infix! postfix.

Prefix

Operator becomes parent first.

72. NOTEBOOK NOTES

Infix !' Postfix

TEXT

```
O p e r a n d ! ' o u t p u t
(           ! ' p u s h
)           ! ' p o p u n t i l (
Operator:
  pop stronger operators
  pop equal if current is left-associative
  push current
End:
  pop everything
```

73. FORMULA BOX — POSTFIX POP CONDITION

Pop top operator when:

TEXT

```
prec(top) > prec(current)
```

or:

TEXT

```
prec(top) == prec(current)
AND current is left-associative
```

Do not pop equal precedence when current is right-associative.

74. COMPLEXITY BOX

For n expression tokens:

TEXT

```
Time =  $O(n)$ 
Stack space =  $O(n)$ 
```

because each operator is pushed/popped at most once.

75. 30-SECOND REVISION

Remember:

1. Scan infix left to right for postfix.
 2. Operand goes directly to output.
 3. (goes to stack.
 4.) pops until (.
 5. Higher stack precedence gets popped.
 6. Equal precedence left-associative gets popped.
 7. Equal ^ does not pop in ordinary postfix conversion.
 8. At end, pop remaining operators.
 9. Prefix can be derived by tree/grouping or careful reverse technique.
 10. Conversion is $O(n)$ time and $O(n)$ stack space.
-

76. ACTIVE RECALL

Q1

When an operand is encountered during 'infix' pop ~~where~~ does it go?

Q2

When) is encountered, what happens?

Q3

For equal-precedence - , do we pop the existing - ?

Q4

For equal-precedence ^ , do we pop existing ^ during ordinary postfix conversion?

Q5

Why is the overall algorithm $O(n)$ despite inner popping loops?

ANSWERS

A1

Directly to output.

A2

Pop operators until matching (and discard parentheses.

A3

Yes, because - is left-associative.

A4

No, because exponentiation is right-associative in this notation.

A5

Each operator is pushed and popped at most once overall.

77. PRACTICE — EASY

Convert to postfix:

A

TEXT

$A+B$

B

TEXT

$A+B*C$

C

TEXT

$A*B+C$

D

TEXT

$A*(B+C)$

78. PRACTICE — MODERATE

Convert to postfix and prefix:

A

TEXT

$(A+B)*(C-D)$

B

TEXT

$A+B*C-D$

C

TEXT

$A*(B-C+D)$

D

TEXT

$(A+B)/(C-D)$

79. PRACTICE — TRICKY

Convert carefully:

A

TEXT

$A-B-C-D$

B

TEXT

$A/B*C/D$

C

TEXT

$A^B \wedge C \wedge D$

D

TEXT

$(A^B) \wedge (C^D)$

E

TEXT

$A^B \wedge (C^D) - D * E + F$

80. GATE STACK-TRACE PRACTICE

For:

TEXT

$A+B*C-D$

make a table with:

TEXT

Token | Stack | Output

Do not skip steps.

Your final postfix should be:

TEXT

ABC*+D-

81. PLACEMENT CONNECTION

Compilers and parsers often convert or internally represent expressions in forms that make evaluation easier.

Why postfix is useful:

TEXT

A B C * +

can be evaluated with a stack without repeatedly checking:

- precedence
- associativity
- parentheses

That leads directly to the next lesson.

PROGRESS

MODULE 4 — STACK

' Lesson 37 — Stack Fundamentals ' Lesson 38 — Stack Using Array ' Lesson 39 — Stack Using Linked List '
Lesson 40 — Infix, Prefix & Postfix Fundamentals ' **Lesson 41 — Infix !' Postfix/Prefix Conversion**

NEXT — LESSON 42

Postfix & Prefix Evaluation Using Stack

Next we master numerical evaluation.

Example:

TEXT

Postfix:
2 3 4 * +

Evaluation:

TEXT

$3 \times 4 = 12$
 $2 + 12 = 14$

But the real GATE trap is operand order:

TEXT

8 2 /

must be:

TEXT

8 / 2

not:

TEXT

2 / 8

We will cover:

- postfix 'left to right' scanning
- operand push
- operator pops
- operand1 op operand2
- prefix 'right to left' scanning
- subtraction/division traps
- multi-digit operands
- exact stack traces
- invalid expressions
- time/space complexity
- combined conversion + evaluation problems.

USER

did it complete more how much syllabus there

ASSISTANT

Yes— we have completed a **large portion**, but the full subject is not finished yet.

Your official syllabus is:

Programming in C, Recursion, Arrays, Stacks, Queues, Linked Lists, Trees, Binary Search Trees, Binary Heaps, Graphs

Current progress

Official GATE Topic	Status
Programming in C	' Complete
Recursion	' Complete
Arrays	' Complete
Linked Lists	' Complete
Stacks	Ø=ßá Almost complete

Official GATE Topic	Status
Queues	#6 Remaining
Trees	#6 Remaining
Binary Search Trees	#6 Remaining
Binary Heaps	#6 Remaining
Graphs	#6 Remaining

For **Stacks**, we have completed Lessons 37–41. Only these major parts remain:

- **Lesson 42:** Postfix & Prefix Evaluation
- **Lesson 43:** Placement stack patterns — monotonic stack, Next Greater Element, Stock Span, Largest Rectangle

Then we move to **Queues**.

In terms of our detailed course

We have completed:

Lesson 1 !' Lesson 41 '

The core syllabus roadmap goes approximately up to:

Lesson 78 — Graphs

So:

41 / 78 lessons completed "53% of the detailed core course.

About **37 core lessons remain**.

The remaining order is:

Stack finish !' Queue !' Trees !' BST !' Binary Heaps !' Graphs

After those, the **official topic coverage is essentially complete**. Then I had planned extra high-value sections for you:

Lessons 79–84: Complexity revision **Lessons 85–92:** GATE mixed consolidation/tests **Lessons 93–98:** Placement/interview preparation **Final:** formula sheets + revision sheets

So don't worry—we are **more than halfway through our detailed core teaching roadmap**, and everything you've completed so far is foundational. Continue with **NEXT**, and we'll do **Lesson 42 — Postfix & Prefix Evaluation**.

USER

Okay give next

ASSISTANT

MODULE 4 — LESSON 42

Postfix & Prefix Evaluation Using Stack

This is a very common GATE area because it mixes:

- GATE importance: $\phi = 7\%$ ~~DO NOT KNOW~~ Placement importance: $\phi = 7\%$ ~~DO NOT KNOW~~**

Completed:

' Infix / Prefix / Postfix basics ' Infix !' Postfix conversion ' Infix !' Prefix conversion

Now:

Example:

```
```text id="ie94cx" Postfix: 2 3 4 * +
```

means :

```
```text id="ur8sl4"
2 + (3*4)
```

Answer:

```
``text id="1ziq1s" 14
```

— — —

2. WHY STACK IS PERFECT HERE

Consider postfix:

```
```text id="avqun7"
2 3 4 * +
```

When we encounter numbers:

```
```text id="vhabn9" 2 3 4
```

we don't immediately know when each will be used.

So we temporarily store them.

Where?

```
# **STACK**
```

When an operator appears, the most recently available operands are required first.

That naturally fits LIFO.

```
# 3 . P O S T F I X E V A L U A T I O N M A S T E R R U L E Ø=Ý%
```

Scan:

```
# * * L E F T ! ' R I G H T * *
```

For every token:

If token is an operand

```
```text id="rmk5kg"
PUSH it
```

### If token is an operator

```
```text id="d7vj2c" operand2 = POP() operand1 = POP()
```

result = operand1 operator operand2

PUSH result

At the end:

```
```text id="4r6nzj"
only one value should remain
```

That value is the answer.

Ø=Ý% Memorize:

## POSTFIX = L E F T ! ' R I G H T

## 4. WHY POP ORDER MATTERS Ø=Ý%Ø=Ý%Ø=Ý%

Suppose postfix:

```
```text id="vbiqh8" 8 2 -
```

Stack becomes:

```
```text id="3ii5j3"
TOP
!"
2
8
```

When - appears:

First pop:



```
```text id="jrsr57" operand2 = 2
```

Second pop:

```
```text id="rth2ue"
operand1 = 8
```

Then:

```
```text id="65aocz" operand1 - operand2 = 8 - 2 = 6
```

Correct.

5. BIGGEST POSTFIX TRAP

Do NOT calculate:

```
```text id="dvbrn"
operand2 - operand1
```

That would give:

```
```text id="m2152k" 2 - 8 = -6
```

Wrong.

Ø=Ÿ% T h e s t a n d a r d r u l e i s :

```
```text id="pipp18"
```

```
b = pop()
```

```
a = pop()
```

```
answer = a op b
```

Memory trick:

**\*\*First pop goes RIGHT.**

Second pop goes LEFT.\*\*

## 6. SIMPLE EXAMPLE — 2 3 +

Postfix:

```
```text id="awl0c" 2 3 +
```

Scan 2:

```
```text id="p30lrv"
```

Stack:

2

Scan 3:

```
```text id="0gczgs" Stack: 3 2
```

Scan `+`:

```
```text id="pcr1w7"
b = 3
a = 2
```

Calculate:

```
```text id="6qnas0" 2 + 3 = 5
```

Push 5:

```
```text id="cqa5g9"
Stack:
5
```

Final:

**5**

## 7. FULL TRACE — 2 3 4 \* +

Expression:

```
```text id="r1yq2c" 2 3 4 * +
```

Read 2

```
```text id="gjm4j1"
Stack:
2
```

**Read 3**

```
```text id="xjo7ur" 3 2
```

Read 4

```
```text id="519waf"
4
3
2
```

**Read \***

Pop:

```
```text id="zmkm8x" b = 4 a = 3
```

Calculate:

```
```text id="p04gje"
3*4 = 12
```

Push 12:

```
```text id="shxefk" 12 2
```

```
### Read `+`

```text id="y51a3f"
b = 12
a = 2
```

```
```text id="cthkk5" 2+12 = 14
```

Push:

```
```text id="f7ta2e"
14
```

Answer:

**14**

## 8. STEP TABLE METHOD

For GATE, use:

Token	Action	Stack
2	Push	2
3	Push	2,3
4	Push	2,3,4
*	$3*4=12$	2,12
+	$2+12=14$	14

This dramatically reduces mistakes.

## 9. POSTFIX EXAMPLE — SUBTRACTION

Expression:

```
```text id="ev4ht6" 10 4 2 + -
```

Scan operands:

```
```text id="qi3awk"
10
4
2
```

At +:

```
```text id="pu1drj" 4 + 2 = 6
```

Stack:

```
```text id="q6hnq9"
10
6
```

At -:

```
```text id="n50t2c" 10 - 6 = 4
```

Answer:

4

This represents:

```
```text id="bcji4x"
10 - (4+2)
```

---

## 10. POSTFIX EXAMPLE — DIVISION

---

```
```text id="1zdtfa" 20 5 /
```

First pop:

```
```text id="ac0jom"
5
```

Second:

```
```text id="42fofk" 20
```

Then:

```
```text id="r98u18"
20/5 = 4
```

Not:

```
```text id="gleuyx" 5/20
```

Ø=Ý% D i v i s i o n e x p o s e s o p e r a n d - o r d e r m i s t a k e s i m m e d i a t e l y .

11. POSTFIX EXAMPLE — MIXED

Evaluate:

```
```text id="v8yqtq"
5 6 2 + * 12 4 / -
```

Let's trace.

## 12. TRACE

Push:

```
```text id="tdlrj4" 5 6 2
```

At `+`:

```
```text id="6jt57x"
6+2 = 8
```

Stack:

```
```text id="v8v6iw" 5 8
```

At `\*`:

```
```text id="ji47ko"
5*8 = 40
```

Stack:

```
```text id="zsqqpt" 40
```

Push:

```
```text id="ag7kfq"
12
4
```

At `/`:

```
```text id="oqizd7" 12/4 = 3
```

Stack:

```
```text id="5bos40"
40
3
```

At - :

```
```text id="f5w2pi" 40-3 = 37
```

```
# Answer
```

```
```text id="3mj1kl"
37
```

## 13. RECONSTRUCT INFIX TO CHECK

Postfix:

```
```text id="r08oiq" 5 6 2 + * 12 4 / -
```

means:

```
```text id="k8g665"
5*(6+2) - (12/4)
```

```
```text id="1tqrl8" = 5*8 - 3 = 40 - 3 = 37
```

Useful verification.

14. POSTFIX IMPLEMENTATION IDEA

Pseudo-code:

```
```text id="8wgr1l"
for each token:
 if operand:
 push(token)

 else if operator:
 b = pop()
 a = pop()

 push(a op b)

answer = pop()
```

## 15. C-LIKE CODE FOR SINGLE-DIGIT OPERANDS

```
```c id="6j7dse" int evaluatePostfix(char exp[]) { int stack[100]; int top = -1;

for (int i = 0; exp[i] != '\0'; i++) { char ch = exp[i];

if (ch >= '0' && ch <= '9') { stack[++top] = ch - '0'; } else { int b = stack[top--]; int a = stack[top--];

if (ch == '+') stack[++top] = a + b; else if (ch == '-') stack[++top] = a - b; else if (ch == '*') stack[++top] = a * b; else if (ch == '/') stack[++top] = a / b; } }

return stack[top]; }
```

```

---
# 16. WHY `ch - '0'?
Character:
```text id="719pim"
'7'

```

is not integer value 7 directly.

So:

```
```c id="z8qifo" ch - '0'
```

converts digit character to its numeric digit value.

Example:

```
```text id="rqnm7p"
'7' - '0' = 7

```

Ø=C% connection.

## 17. LIMITATION OF SIMPLE CHARACTER CODE

Expression:

```
```text id="ewdcjg" 23 10 +
```

contains multi-digit numbers.

A character-by-character implementation would see:

```
```text id="b8k26w"
'2'
'3'
'1'
'0'

```

as four separate digits unless tokenization is handled.

So practical evaluators often use:

- spaces
- token parsing
- `strtol`
- scanner/tokenizer logic

Ø=Ö For GATE hand calculations, operands are often single digits or clearly spaced.

## 18. VALID POSTFIX CONDITION

For a valid binary-operator postfix expression:

Every operator needs:

```
```text id="j3en47" 2 operands/results
```

At end:

```
```text id="7na4st"
exactly 1 result
```

should remain.

---

## 19. INVALID POSTFIX — TOO FEW OPERANDS

---

```
```text id="lfbr6u" 5 +
```

Scan 5:

```
```text id="8d6ke1"
stack has one operand
```

Then `+` requires two operands.

Invalid.

This causes:

### Stack underflow during evaluation

---

---

## 20. INVALID POSTFIX — EXTRA OPERANDS

---

```
```text id="yxtt2y" 5 6 7 +
```

At ``+``:

```
```text id="5ynryx"
6? Actually top two are 6? Let's trace:
stack = 5,6,7
7+6 = 13
```

Remaining:

```
```text id="blbp46" 5 13
```

Two final values remain.

So this is not a single complete binary expression.

$\emptyset = \forall\%$ At completion, stack size must normally be 1.

21. OPERAND / OPERATOR COUNT PROPERTY

For a valid expression using only **binary operators**:

FORMULA BOX

```
```text id="e8igj8"
number of operands
=
number of operators + 1
```

Example:

```
```text id="wozkuv" A B C * +
```

Operands:

```
```text id="tyvl3l"
3
```

Operators:

```
```text id="e798r6" 2
```

Correct:

```
```text id="a5eaif"
3 = 2 + 1
```

Useful GATE property.

## 22. POSTFIX VALIDITY PREFIX CONDITION

While scanning postfix `l e f t ! ' r i g h t :`

At every point:

```
```text id="j0rxv5" available operands/results must be enough for any operator
```

For binary operators, before applying an operator:

```
```text id="vekase"
stack size >= 2
```

At end:

```
```text id="7o5aol" stack size == 1
```

```
---
# 2 3 . P R E F I X E V A L U A T I O N 0=Ý%0=Ý%0=Ý%
```

Now prefix.

Example:

```
```text id="3onxyx"
+ 2 * 3 4
```

means:

```
```text id="dz1g0w" 2 + (3*4)
```

```
= 14.
```

For prefix evaluation, scan:

```
#   * * R I G H T ! '   L E F T * *

Ø=Ý%   O p p o s i t e   o f   p o s t f i x .

---

# 24. PREFIX MASTER RULE

Scan from right to left.

### If operand

```text id="0wi4rc"
PUSH
```

### If operator

```
```text id="ep8i8b" operand1 = POP() operand2 = POP()
```

result = operand1 op operand2

PUSH result

At end, one answer remains.

Notice operand-order handling differs in how we name pops because scanning direction is reversed.

```
---

# 25. MEMORY TRICK

#   * * P O S T F I X :   s c a n ! '
P R E F I X :   s c a n ! • * *
```

That alone prevents many mistakes.

```
---

# 26. PREFIX EXAMPLE – `+ 2 3`
```

Scan right to left.

Read 3:

```
```text id="02ezzn"
Stack:
3
```

Read 2:

```text id="2e3p7l" 2 3

Read `+`:

First pop:

```text id="8z33so"  
a = 2

Second:

```text id="n5e8wd" b = 3

Calculate:

```text id="328bkz"  
2+3 = 5

Answer:

5

## 27. PREFIX SUBTRACTION TRAP

Prefix:

```text id="a37gvt"

- 8 3

S c a n r i g h t ! ' l e f t :

Push 3.

Push 8.

Stack top:

```text id="i9c28i"  
8  
3

Operator -.

First pop:

```text id="hqc7h7" operand1 = 8

Second:

```text id="tsqsni"  
operand2 = 3

Then:

```
```text id="4maiyx" 8 - 3 = 5
```

Correct.

Ø=Ý% I n p r e f i x r i g h t - t o - l e f t e v a l u a t i o n :

First pop is the LEFT operand.

28. COMPARE POSTFIX AND PREFIX POP ORDER

Postfix

S c a n l e f t ! ' r i g h t :

```
```text id="08s4j0"
```

8 3 -

Stack top before operator:

```
```text id="l3nv42" 3 8
```

So:

```
```text id="xq8ka6"
```

right = first pop

left = second pop

## Prefix

Scan right! left:

```
```text id="1x16sy"
```

- 8 3

Stack top before operator:

```
```text id="0cpxh0"
```

8

3

So:

```
```text id="cocf2e" left = first pop right = second pop
```

Ø=Ý% V e r y i m p o r t a n t c o m p a r i s o n .

29. FORMULA BOX — OPERAND ORDER

Postfix

```
```text id="2cq6kt"
```

b = pop()

```
a = pop()
```

```
a op b
```

## Prefix

```
```text id="w3p4i9" a = pop() b = pop()
```

```
a op b
```

```
a s s u m i n g s t a n d a r d l e f t / r i g h t n a m i n g a n d r i g h t ! ' l e f t s c a n
```

```
---
```

```
# 30. PREFIX EXAMPLE - `+ 2 * 3 4`
```

Scan from right.

```
### 4
```

```
```text id="jq3wm8"
```

```
4
```

## 3

```
```text id="ywlcc5" 3 4
```

```
### `*`
```

```
```text id="hjjocx"
```

```
3 * 4 = 12
```

Stack:

```
```text id="5t8imx" 12
```

```
### 2
```

```
```text id="ey37ig"
```

```
2
```

```
12
```

+

```
```text id="bs3ddf" 2 + 12 = 14
```

Answer:

```
# **14**
```

```
---
```

```
# 31. PREFIX COMPLEX EXAMPLE
```

Evaluate:

```
```text id="t1xk18"
- * 5 + 6 2 / 12 4
```

Expression structure:

```
```text id="776z4f" [5*(6+2)] - [12/4]
```

Expected:

```
```text id="2y244y"
37
```

Let's evaluate from right to left.

## 32. PREFIX TRACE

Tokens from right:

```
```text id="wwfc9m" 4 12 / 2 6 + 5 * -
```

Push 4.

Push 12.

At `/`:

```
```text id="uu9b73"
12 / 4 = 3
```

Stack:

```
```text id="zf9xmv" 3
```

Push 2.

Push 6.

At `+`:

```
```text id="u84flt"
6 + 2 = 8
```

Stack has:

```
```text id="xeosnk" 8 3
```

Push 5.

At `\*`:

```
```text id="mb00f4"
5*8 = 40
```

Stack:

```text id="r1j0qf" 40 3

At ``-``:

```text id="fg6l0s"  
40-3 = 37

## Answer

```text id="pft89y" 37

33. PREFIX IMPLEMENTATION IDEA

Pseudo-code:

```text id="y8h57w"  
scan tokens from right to left

if operand:  
    push

if operator:  
    a = pop()  
    b = pop()

    push(a op b)

## 34. POSTFIX VS PREFIX MASTER TABLE

| Feature        | Postfix       | Prefix        |
|----------------|---------------|---------------|
| Scan direction | Left !' Right | Right !' Left |
| Operand        | Push          | Push          |
| Operator       | Pop 2         | Pop 2         |
| First pop      | Right operand | Left operand  |
| Second pop     | Left operand  | Right operand |
| Final stack    | One result    | One result    |

0-00 Copy this.

## 35. EVALUATING ^

Suppose postfix:

```text id="aoj1z5" 2 3 ^

assuming `^` means exponent:

```
```text id="kx8rze"
2^3 = 8
```

For actual C implementation, remember:

```
```c id="g4e4ag" 2 ^ 3
```

would perform XOR, not exponentiation.

To compute mathematical power in C, use suitable logic/library functions.

Ø=Ý% Same notation warning .

36. INTEGER DIVISION TRAP

Suppose postfix:

```
```text id="g6x1z3"
5 2 /
```

If evaluation uses C integer arithmetic:

```
```text id="n3pbzr" 5 / 2 = 2
```

not:

```
```text id="71l7ax"
2.5
```

If floating-point operands are intended:

```
```text id="5dy47z" 2.5
```

may be correct.

Ø=Ý% Always check operand types / question context .

37. NEGATIVE VALUES

Example postfix:

```
```text id="3kkbcb"
3 8 -
```

Calculation:

```
```text id="xao9jj" 3 - 8 = -5
```

Negative results are perfectly valid.

Do not confuse negative result with an error code in a naive implementation.

38. DIVISION BY ZERO

Example:

```
```text id="crseda"
10 0 /
```

Arithmetic operation is invalid for integer division.

A robust evaluator must detect division by zero before performing it.

Ø=Ø Programming safety point.

---

## 39. COMBINED INFIX !' POSTFIX !' EVALUATE

---

Suppose infix:

```
```text id="a12i16" 2 + 3 * 4
```

Step 1 – Convert:

```
```text id="xezb8j"
2 3 4 * +
```

Step 2 — Evaluate:

```
```text id="4x3qo5" 3*4 = 12 2+12 = 14
```

Answer:

14

40. COMBINED EXAMPLE — `(2+3)\*4`

Infix:

```
```text id="hjcv8j"
(2+3)*4
```

Postfix:

```
```text id="0sm25m" 2 3 + 4 *
```

Evaluation:

```
```text id="s17vhm"
2+3 = 5
5*4 = 20
```

Answer:

## 20

Parentheses changed result from 14 to 20.

## 41. COMBINED PREFIX

Same infix:

```
```text id="43u5zd" (2+3)*4
```

Prefix:

```
```text id="4hz3g3"  
* + 2 3 4
```

Evaluate right to left:

```
```text id="3s1y5k" 2+3 = 5 5*4 = 20
```

Same answer.

42. ALL THREE FORMS REPRESENT SAME EXPRESSION

For:

```
```text id="0r0l6h"  
(2+3)*4
```

### Infix

```
```text id="prqdfd" (2+3)*4
```

Prefix

```
```text id="c1myob"  
* + 2 3 4
```

### Postfix

```
```text id="2k75qh" 2 3 + 4 *
```

All evaluate to:

```
```text id="ryxgpj"  
20
```

Only notation differs.

## 43. EXPRESSION TREE CONNECTION

```
```text id="az7jca" */ \
```

– 4

/ \ 2 3

Preorder:

```
```text id="wd35yf"
* + 2 3 4
```

Prefix.

Postorder:

```
```text id="wv8ayr" 2 3 + 4 *
```

Postfix.

Evaluation is essentially reducing this tree from its leaves upward.

Ø=Ý% N i c e d e e p c o n n e c t i o n .

44. STACK SIZE DURING POSTFIX EVALUATION

For binary operators:

- operand: stack size +1
- operator: pops 2 and pushes 1

Net effect of operator:

```
```text id="bkvrgd"
-1
```

If there are:

```
```text id="wfg1xy" m operands
```

and:

```
```text id="b80b2v"
k binary operators
```

final stack size:

```
```text id="byw0ck" m - k
```

For a complete valid expression:

```
```text id="e99prm"
m-k = 1
```

Therefore:

```
```text id="b6as7q" m = k+1
```

This explains the earlier formula.

45. MAXIMUM STACK SIZE

Example postfix:

```
```text id="9fdv5o"
A B C D + * +
```

Before operators start, multiple operands can accumulate.

Worst-case auxiliary stack can contain:

```
```text id="h71125" O(n)
```

```
items.
```

Therefore evaluation:

```
```text id="id0ko4"
Space = O(n)
```

worst case.

---

## 46. TIME COMPLEXITY

Each token is examined once.

Every operand may be pushed once.

Every intermediate result may be popped/pushed a constant number of times.

So:

---

## FORMULA BOX

```
```text id="dudz4" Time = O(n) Auxiliary Stack Space = O(n)
```

```
for `n` tokens.
```

```
Ø=Ý% M u s t k n o w .
```

47. WHY NOT $O(n^2)$?

Stack push/pop each take:

```
```text id="h8pfwm"
O(1)
```

and we process:

```
```text id="46jgar" n tokens
```

So total:

```
```text id="p2m4u2"
0(n)
```

---

## 48. GATE TRAP — POSTFIX SCAN DIRECTION

---

Postfix:

```
```text id="u1jz0z" A B +
```

Scan:

```
```text id="ymlqmt"
L E F T ! ' R I G H T
```

Not right! left.

---

## 49. GATE TRAP — PREFIX SCAN DIRECTION

---

Prefix:

```
```text id="nhdaep"
```

- A B

Standard stack evaluation scans:

```
```text id="1cipl8"
R I G H T ! ' L E F T
```

Not left! right.

---

## 50. GATE TRAP — POSTFIX POP ORDER

---

For:

```
```text id="0vl38m" 8 4 /
```

First pop:

```
```text id="yf4mv0"
4
```

Second:

```
```text id="wbyxmd" 8
```

Calculate:

```
```text id="dovknu"
8/4
```

not  $4/8$ .

---

## 51. GATE TRAP — PREFIX POP ORDER

---

For:

```
```text id="7qtnh3" / 8 4
```

```
scan from right.
```

```
Stack before `/`:
```

```
```text id="xzjmat"
8 !• t o p
4
```

First pop:

```
```text id="0dhku7" 8
```

```
second:
```

```
```text id="4v8ekn"
4
```

calculate:

```
```text id="byksjf" 8/4
```

```
---
```

```
# 52. GATE TRAP — COMMUTATIVE OPERATORS HIDE ERRORS
```

```
For:
```

```
```text id="wtcmdq"
+
*
```

wrong operand order still gives same result:

```
```text id="kwbl85" a+b = b+a
```

```
and:
```

```
```text id="staz4u"
a*b = b*a
```

So you may think your algorithm is correct.

But it fails for:

```
```text id="nj7wc4" - / ^
```

```
0=Ÿ%   T e s t   y o u r   u n d e r s t a n d i n g   u s i n g   s u b t r a c t i o n / d i v i s i o n .
```

```
---
```

```
# 53. POSTFIX TRACE QUESTION
```

Evaluate:

```
```text id="51mfhu"
8 2 3 * + 4 -
```

Start:

```
```text id="oq6pp4" 8 2 3
```

```
At `*`:
```

```
```text id="n2kt7a"
2*3 = 6
```

Stack:

```
```text id="wr9c0r" 8 6
```

```
At `+`:
```

```
```text id="8l6y3u"
8+6 = 14
```

Push 4.

```
At `-:
```

```
```text id="djzbik" 14-4 = 10
```

```
# Answer
```

```
```text id="u3xg39"
10
```

---

## 54. PREFIX TRACE QUESTION

Evaluate:

```
```text id="nyx8v6"
```

- + 8 * 2 3 4

Structure:

```
```text id="76ixnu"
(8 + (2*3)) - 4
```

Calculate:

```
```text id="f66oym" 2*3 = 6 8+6 = 14 14-4 = 10
```

Answer :

```
# **10**
```

```
# 55. GATE-STYLE EXAMPLE – POSTFIX
```

Evaluate:

```
```text id="oc5boq"
6 2 / 3 - 4 2 * +
```

First:

```
```text id="fb2kh7" 6/2 = 3
```

Then:

```
```text id="v7eeq9"
3-3 = 0
```

Then:

```
```text id="btf3js" 4*2 = 8
```

Then:

```
```text id="5z69xm"
0+8 = 8
```

Answer:

**8**

## 56. CONVERT BACK TO INFIX METHOD

Sometimes values/variables make an expression hard to understand.

You can use a stack of subexpressions conceptually.

Postfix:

```
```text id="lr4l4z" A B C * +
```


Push:

```
```text id="983a1l"
A
B
C
```

At \*:

```
```text id="79zj0t" (B*C)
```

At `+`:

```
```text id="lnzmvh"
(A+(B*C))
```

This helps verify the intended structure.

---

## 57. POSTFIX !' INFIX WITH STACK

---

Algorithm:

```
```text id="xv4r0e" Scan l e f t ! ' r i g h t
```

operand: push operand/string

operator: right = pop() left = pop()

push("(" + left + op + right + ")")

Final string is infix.

Ø=Ý% S a m e o p e r a n d - o r d e r p r i n c i p l e .

5 8 . P R E F I X ! ' I N F I X

S c a n r i g h t ! ' l e f t .

For operator:

```
```text id="gnwp1i"
```

```
left = pop()
```

```
right = pop()
```

```
push("(" + left + op + right + ")")
```

Again, structure follows evaluation rules.

Ø=Übod placement extension.

---

## 59. POSTFIX !' PREFIX

---

Another stack transformation:

Scan postfix l e f t! ' r i g h t .

Operand:

```
```text id="ylwcj5" push
```

Operator:

```
```text id="oygibf"
right = pop()
left = pop()

push(op + left + right)
```

Example:

```
```text id="t1ymh4" AB+C*
```

Eventually:

```
```text id="yq35j2"
*+ABC
```

Ø=Useful conversion connection.

---

## 60. PREFIX !' POSTFIX

Scan prefix r i g h t! ' l e f t .

Operand:

```
```text id="fz89ru" push
```

Operator:

```
```text id="rmpkjl"
left = pop()
right = pop()

push(left + right + op)
```

Again stack handles subexpressions.

---

## 61. WHY ALL THESE ALGORITHMS USE STACK

Because an operator combines the **most recently completed subexpressions**.

Those subexpressions are naturally managed in LIFO order.

That's the deeper common principle.

---

## 62. GATE QUESTION — VALIDITY

Which of these is valid postfix for binary operators?

**A**

```text id="sfcr5w" A B +

Valid.

B

```text id="4lt3pv"  
A + B

Not valid postfix.

**C**

```text id="jdppvq" A B C + \*

Valid.

D

```text id="i6h2ib"  
A B + \*

Invalid because \* lacks two operands/results.

## 63. GATE QUESTION — NUMBER COUNTS

A valid postfix expression contains:

```text id="tqnvcy" 15 binary operators

How many operands must it contain?

Using:

```text id="j58616"  
operands = operators + 1

Answer:

```text id="ifhf8w" 16

Ø=Ý% V e r y q u i c k n u m e r i c a l q u e s t i o n .

64. ANOTHER COUNT QUESTION

Expression has:

```text id="pmsux8"  
20 operands

and only binary operators.

For one complete expression:

```
```text id="f59ghy" operators = 20-1 = 19
```

```
---
```

```
# 65. WHAT IF UNARY OPERATORS EXIST?
```

Then:

```
```text id="5vsn1n"
operands = binary operators + 1
```

alone is no longer sufficient to count total operators because unary operators consume only one operand.

$O = \frac{N}{2} + 1$  formula assumes all operators are binary.

Read question carefully.

---

## 66. PLACEMENT APPLICATIONS

Expression evaluation concepts appear in:

- calculators
- interpreters
- compilers
- parsers
- syntax trees
- virtual machines
- expression engines

Even if actual implementations are more sophisticated, stack-based evaluation is foundational.

---

## 67. MASTER POSTFIX TEMPLATE

```
```text id="oey1pp" SCAN LEFT !' RIGHT
```

Operand: PUSH

Operator: b = POP a = POP PUSH(a op b)

Final: one value remains

```
Ø=Ý% M e m o r i z e .
```

```
---
```

```
# 68. MASTER PREFIX TEMPLATE
```

```
```text id="53qh5q"
S C A N R I G H T !' L E F T
```

Operand:

PUSH

Operator:

a = POP

b = POP

PUSH(a op b)

Final:

one value remains

0-Memorize.

## 69. MEMORY TRICKS

### Postfix

**POST goes forward.**

```text id="upbvzw" Left !' Right

Prefix

> **PRE goes backward.**

```text id="5m8yd1"

R i g h t !' L e f t

### Postfix operator

**First pop is RIGHT.**

### Prefix operator

**First pop is LEFT.**

### Valid expression

**Everything collapses to ONE.**

## 70. NOTEBOOK NOTES

### Postfix Evaluation

```text id="m3ibak" Scan L!' R

Operand !' push

Operator: b = pop a = pop push(a op b)

```

---

## Prefix Evaluation

```text id="ezvtg7"
S c a n R ! ' L

O p e r a n d ! ' p u s h

Operator:
 a = pop
 b = pop
 push(a op b)

```

## 71. FORMULA BOX

For valid expressions containing only binary operators:

```
```text id="oqazpi" Operands = Operators + 1
```

```

Complexity:

```text id="mq0k4j"
Time = O(n)
Space = O(n)

```

## 72. DON'T CONFUSE

Topic	Postfix	Prefix
Scan	L! ' R	R! ' L
Operator position	after operands	before operands
First pop	right operand	left operand
Second pop	left operand	right operand

0-00 Copy this table.

## 73. 30-SECOND REVISION

Remember:

1. Postfix evaluation scans l e f t ! ' r i g h t .
2. Prefix evaluation scans r i g h t ! ' l e f t .
3. Operands are pushed.
4. Operator consumes two stack values.
5. Postfix: first pop is right operand.
6. Prefix: first pop is left operand.

7. Push result back.
  8. Final stack should have exactly one result.
  9. Binary expression operands = operators + 1.
  10. Evaluation is  $O(n)$  time and  $O(n)$  stack space.
- 

## 74. ACTIVE RECALL

---

### Q1

Which direction do we scan postfix?

### Q2

Which direction do we scan prefix?

### Q3

In postfix:

```text id="sxcj3e" 8 2 -

what is the answer?

Q4

In prefix:

```text id="z1m559"  
- 8 2

what is the answer?

### Q5

A valid binary postfix expression has 9 operators. How many operands?

---

## ANSWERS

---

### A1

```text id="pfjjpq" Left !' Right

A2

```text id="nmuhd6"  
R i g h t ! ' L e f t

### A3

```text id="mvqm0f" 8-2 = 6

A4

```
```text id="h7on7z"
8-2 = 6
```

## A5

```
```text id="hvnk2z" 10 operands
```

```
---

# 75. PRACTICE – EASY

Evaluate postfix:

### A

```text id="13iekh"
2 3 +
```

## B

```
```text id="mfs674" 8 2 -
```

```
### C

```text id="wlc bw1"
6 3 /
```

## D

```
```text id="jr5vtn" 2 3 4 * +
```

```
---

# 76. PRACTICE – MODERATE POSTFIX

Evaluate:

### A

```text id="1w86k5"
5 2 + 3 *
```

## B

```
```text id="0mrkk4" 10 2 3 * -
```

```
### C

```text id="91g9us"
8 4 / 2 + 3 *
```

## D

```
```text id="jrz111" 5 6 2 + * 12 4 / -
```

77. PRACTICE — PREFIX

Evaluate:

A

```
```text id="m4030i"
+ 2 3
```

## B

```
```text id="sgh9nw"
```

- 10 4

C

```
```text id="xyv4qp"
+ 2 * 3 4
```

## D

```
```text id="ss211u"
```

- * 5 + 6 2 / 12 4

78. GATE TRAP PRACTICE

Evaluate:

```
```text id="za2457"
20 5 2 - /
```

First:

```
```text id="8bpcqf" 5-2 = 3
```

Then:

```
```text id="klg9z4"
20/3
```

If integer arithmetic:

```
```text id="q8q9b9" 6
```

If real arithmetic:

```
```text id="2x37mf"
6.666...
```

Ø = Type/context matters.

## 79. GATE-STYLE COMBINED QUESTION

Given infix:

```
```text id="14jxo7" (8+2)*(6-3)
```

```
### Step 1 – Postfix
```

```
```text id="4emq2j"
8 2 + 6 3 - *
```

### Step 2 — Evaluate

```
```text id="w76udc" 8+2 = 10 6-3 = 3 10*3 = 30
```

Answer :

```
# **30**
```

```
# 80. PLACEMENT MINI-CHALLENGE
```

Which algorithm should you think of immediately?

```
### A
```

Evaluate postfix expression.

```
```text id="nngh03"
Stack, scan left! right
```

### B

Evaluate prefix expression.

```
```text id="temu2v" Stack, scan right! left
```

```
### C
```

Convert infix to postfix.

```
```text id="hgbhxo"
Operator stack + precedence
```

### D

Check parentheses.

```
```text id="7x5ics" Stack of opening brackets
```

That last application leads naturally into the next stack-pattern lesson.

```
# PROGRESS
```

```
## MODULE 4 – STACK
```

```
' Lesson 37 – Stack Fundamentals
' Lesson 38 – Stack Using Array
' Lesson 39 – Stack Using Linked List
' Lesson 40 – Infix / Prefix / Postfix
' Lesson 41 – Infix !' Postfix / Prefix Conversion
' ** Lesson 42 – Postfix & Prefix Evaluation **
```

```
# NEXT – LESSON 43
```

```
# Stack Applications & Placement Patterns – Parentheses, Next Greater Element, Stock Span & Monotonic Stack
```

Next we complete the Stack module with:

- balanced parentheses
- multiple bracket types
- why matching brackets require LIFO
- Next Greater Element
- brute force $O(n^2)$ vs stack $O(n)$
- monotonic stack
- stock span
- nearest smaller/greater element
- largest rectangle in histogram intuition
- stack pattern recognition
- GATE traps
- placement problem roadmap

After Lesson 43:

```
# MODULE 5 – QUEUES
```

starts with:

```
**Lesson 44 – Queue Fundamentals, FIFO, Front/Rear, Enqueue/Dequeue.**
```

USER

next

ASSISTANT

MODULE 4 — LESSON 43

Stack Applications & Placement Patterns — Parentheses, Monotonic Stack, Next Greater Element, Stock Span

This is the **final Stack lesson**.

Today we move from basic stack operations to the most important real applications and interview patterns.

GATE importance: $O=Y\%$ **Placement importance:** $O=Y\%$ **MUST KNOW**

1. WHERE WE ARE

Completed:

' Stack fundamentals ' Array implementation ' Linked-list implementation ' Infix / Prefix / Postfix ' Infix !' Postfix / Prefix ' Prefix / Postfix evaluation

Now we complete Stack with:

1. Balanced parentheses
2. Next Greater Element
3. Monotonic Stack
4. Stock Span
5. Nearest Smaller / Greater patterns
6. Largest Rectangle in Histogram intuition
7. How to recognize stack problems

2. APPLICATION 1 — BALANCED PARENTHESES

Consider:

```
TEXT
({[]})
```

This is balanced.

Why?

Each closing bracket matches the **most recently opened unmatched bracket**.

Example opening order:

```
TEXT
(
{
[
```

Closing order must be:

```
TEXT
]
}
)
```

That is:

LIFO

Therefore stack is the natural data structure.

3. UNBALANCED EXAMPLES

```
TEXT
([
```

Wrong type.

TEXT

((()))

Some openings never close.

TEXT

()))(

A closing bracket appears when no matching opening exists.

4. CORE ALGORITHM

Scan expression from left to right.

Opening bracket

TEXT

(
{
[

Push it.

Closing bracket

TEXT

)
}
]

Check top.

It must contain the corresponding opening bracket.

Then pop it.

At end:

TEXT

stack must be empty

Two failure conditions:

1. closing bracket has no matching opening
2. openings remain after scan

5. EXAMPLE — ({[]})

Trace:

Symbol	Action	Stack
(	push	(

Symbol	Action	Stack
{	push	{{
[	push	{{[
]	pop [	{{
}	pop {	{
)	pop (	empty

Final stack empty.

Balanced '

6. EXAMPLE — ([)]

Trace:

TEXT

```
( !' p u s h
[ !' p u s h
```

Now:

TEXT

)

arrives.

Stack top is:

TEXT

[

But) requires:

TEXT

(

Mismatch.

So:

Not Balanced 'L

Ø=Yes proves that merely counting opening and closing brackets is not enough.

7. C IMPLEMENTATION

```

c
int isMatching(char open, char close)
{
    return (open == '(' && close == ')') ||
           (open == '[' && close == ']') ||
           (open == '{' && close == '}');
}

```

Core:

```

c
int isBalanced(char exp[])
{
    char stack[100];
    int top = -1;

    for (int i = 0; exp[i] != '\0'; i++)
    {
        char ch = exp[i];

        if (ch == '(' || ch == '[' || ch == '{')
        {
            stack[++top] = ch;
        }
        else if (ch == ')' || ch == ']' || ch == '}')
        {
            if (top == -1)
                return 0;

            if (!isMatching(stack[top], ch))
                return 0;

            top--;
        }
    }

    return top == -1;
}

```

8. COMPLEXITY

Each symbol processed once.

```

TEXT
Time = O(n)

```

Worst case all symbols are opening brackets:

```

TEXT
(((((((

```

So stack:

```

TEXT
O(n)

```

Therefore:

FORMULA BOX

TEXT

Balanced Parentheses:

Time = $O(n)$

Space = $O(n)$

9. GATE TRAP — EQUAL COUNTS ARE NOT ENOUGH

Expression:

TEXT

([])

Counts:

TEXT

(= 1

) = 1

[= 1

] = 1

Counts match.

Still invalid because nesting order is wrong.

Stack checks **structure**, not just counts.

10. GATE TRAP — EARLY CLOSING

Expression:

TEXT

) (

Number of opening and closing parentheses is equal.

Still invalid.

At first symbol:

TEXT

)

stack is empty.

Immediate failure.

11. APPLICATION 2 — NEXT GREATER ELEMENT

Given:

TEXT

4 5 2 10 8

For each element, find the first greater element to its right.

Results:

TEXT

4	!	'	5
5	!	'	10
2	!	'	10
10	!	'	-1
8	!	'	-1

This is:

Next Greater Element — NGE

12. BRUTE FORCE

For each element, scan right until you find a greater element.

Example:

TEXT

4	!	'	check	5
5	!	'	check	2, 10
2	!	'	check	10
...				

Worst case:

TEXT

$O(n^2)$

Can stack reduce this?

Yes.

13. MONOTONIC STACK IDEA Ø=Ý%Ø=Ý%Ø=Ý%

A monotonic stack maintains elements in a consistent increasing or decreasing order.

Two common forms:

Monotonic increasing

TEXT

small !' large

Monotonic decreasing

TEXT

l a r g e ! ' s m a l l

The exact orientation depends on whether you're reading stack b o t t o m ! ' d r o p p l ! ' b o t t o m focus on the invariant rather than memorizing wording blindly.

14. NGE — RIGHT TO LEFT METHOD

For each element `a[i]`, scan from right to left.

Maintain a stack containing useful candidates.

For current value:

1. Pop all stack elements `<= a[i]`
2. If stack is empty !' no greater element
3. Else top is NGE
4. Push current element

~~0-100%~~ Why pop smaller values?

Because they can never be useful as "greater" for the current or certain earlier larger elements.

15. EXAMPLE

Array:

TEXT

4 5 2 10 8

Scan r i g h t ! ' l e f t .

Current = 8

Stack empty.

TEXT

NGE(8) = -1

Push 8.

Stack:

TEXT

8

16. CURRENT = 10

Top:

TEXT

8

Since:

TEXT

$8 \leq 10$

pop 8.

Stack empty.

TEXT

$\text{NGE}(10) = -1$

Push 10.

17. CURRENT = 2

Stack top:

TEXT

10

$10 > 2$.

So:

TEXT

$\text{NGE}(2) = 10$

Push 2.

Stack conceptually:

TEXT

10

2 !• t o p

18. CURRENT = 5

Top:

TEXT

2

Since:

TEXT

2 <= 5

pop 2.

Now top:

TEXT

10

10 > 5.

Therefore:

TEXT

NGE(5) = 10

Push 5.

19. CURRENT = 4

Top:

TEXT

5

5 > 4.

Therefore:

TEXT

NGE(4) = 5

Final answers:

TEXT

5 10 10 -1 -1

20. CODE

```
C
void nextGreater(int a[], int n)
{
    int stack[100];
    int top = -1;
    int ans[100];

    for (int i = n - 1; i >= 0; i--)
    {
        while (top != -1 && stack[top] <= a[i])
            top--;
    }
}
```

```

        if (top == -1)
            ans[i] = -1;
        else
            ans[i] = stack[top];

        stack[++top] = a[i];
    }

    for (int i = 0; i < n; i++)
        printf("%d ", ans[i]);
}

```

21. WHY NGE IS $O(n)$, NOT $O(n^2)$

There is a `while` loop inside a `for` loop.

Looks like $O(n^2)$.

But each element:

TEXT

pushed at most once
popped at most once

So total stack operations:

TEXT

$O(n)$

Therefore:

FORMULA BOX

TEXT

NGE:
Time = $O(n)$
Space = $O(n)$

~~$O = \sum$~~ Extremely important amortized/aggregate reasoning.

22. MEMORY TRICK — NGE

Remove everyone too small to help.

For current `x`:

TEXT

`pop <= x`

Then surviving top is the nearest greater candidate.

23. VALUES VS INDICES

In many interview problems, storing values is enough.

But often we need:

- distance
- position
- span
- answer array index

Then store:

```
TEXT
indices
```

instead of values.

Example:

```
C
stack[++top] = i;
```

Then compare:

```
C
a[stack[top]]
```

~~O = index~~ Index stacks are extremely common.

24. NEXT GREATER INDEX

Suppose:

```
TEXT
2 1 5
```

For value 2:

greater value:

```
TEXT
5
```

greater index:

```
TEXT
2
```

distance:

TEXT

2 - 0 = 2

If stack stores indices, all three can be obtained.

25. MONOTONIC STACK PATTERNMAP

When question asks:

TEXT

nearest greater
nearest smaller
next greater
previous greater
next smaller
previous smaller

immediately think:

MONOTONIC STACK

Ø=No placement pattern.

26. FOUR CLASSIC VARIANTS

Next Greater to Right

TEXT

NGR

Next Greater to Left

TEXT

NGL

Next Smaller to Right

TEXT

NSR

Next Smaller to Left

TEXT

NSL

Direction determines scan direction.

Greater/smaller determines what you pop.

27. GENERAL MEMORY RULE

If looking for:

GREATER

remove candidates that are:

TEXT
<= current

If looking for:

SMALLER

remove candidates that are:

TEXT
>= current

subject to whether strict or non-strict comparison is required.

~~Always~~ Always inspect duplicate semantics.

28. DUPLICATE TRAP

Question may ask:

next greater

Meaning strictly greater:

TEXT
>

Then equal elements do not qualify.

So often pop:

TEXT
<= current

But if question asks:

next greater or equal

then equal is acceptable.

You would pop only:

TEXT

< current

Comparison sign matters.

29. APPLICATION 3 — STOCK SPAN

Stock prices:

TEXT

100 80 60 70 60 75 85

For each day, stock span is:

Number of consecutive days ending today for which price was less than or equal to today's price.

Expected spans:

TEXT

1 1 1 2 1 4 6

30. UNDERSTAND SPAN

For price:

TEXT

75

look backward:

TEXT

75 >= 60

75 >= 70

75 >= 60

but:

TEXT

75 < 80

So span:

TEXT

4

including current day.

31. BRUTE FORCE

For each day, move backward until a higher price appears.

Worst:

```
TEXT
O(n2)
```

Again monotonic stack gives:

```
TEXT
O(n)
```

32. STOCK SPAN KEY IDEA

For each day `i`, find the:

Previous Greater Element

specifically the nearest previous day having price:

```
TEXT
> price[i]
```

Then:

```
TEXT
span = i - previousGreaterIndex
```

If none:

```
TEXT
span = i + 1
```

Stock span is essentially a previous-greater problem.

33. EXAMPLE

Prices:

```
TEXT
100 80 60 70
```

For 70 at index 3:

Nearest previous greater is:

TEXT

80 at index 1

Therefore:

TEXT

span = 3 - 1 = 2

Correct.

34. STOCK SPAN STACK

Store indices.

For each day `i`:

TEXT

```
while stack not empty
and price[stack.top] <= price[i]:
    pop
```

Then:

TEXT

```
if empty:
    span[i] = i + 1
else:
    span[i] = i - stack.top
```

Then push `i`.

35. CODE

C

```
void stockSpan(int price[], int n)
{
    int stack[100];
    int top = -1;
    int span[100];

    for (int i = 0; i < n; i++)
    {
        while (top != -1 &&
            price[stack[top]] <= price[i])
        {
            top--;
        }

        if (top == -1)
            span[i] = i + 1;
        else
            span[i] = i - stack[top];

        stack[++top] = i;
    }
}
```

```

    }

    for (int i = 0; i < n; i++)
        printf("%d ", span[i]);
}

```

Time:

TEXT
O(n)

Space:

TEXT
O(n)

36. STOCK SPAN TRACE

Prices:

TEXT
100 80 60 70 60 75 85

Results:

TEXT

1	0	0	!	'	1
8	0		!	'	1
6	0		!	'	1
7	0		!	'	2
6	0		!	'	1
7	5		!	'	4
8	5		!	'	6

Final:

TEXT
1 1 1 2 1 4 6

37. WHY STORE INDICES?

Need:

TEXT
distance

For stock span:

TEXT

```
i - previousGreaterIndex
```

Values alone cannot directly give this distance.

Thus:

Distance problem !' strongly consider storing indices.

38. APPLICATION 4 — PREVIOUS SMALLER ELEMENT

Array:

TEXT

```
4 5 2 10 8
```

Previous smaller for each:

TEXT

```
4    !'   - 1
5    !'   4
2    !'   - 1
1 0  !'   2
8    !'   2
```

This uses same monotonic-stack idea.

39. APPLICATION 5 — LARGEST RECTANGLE IN HISTOGRAM $\emptyset = \text{PE} \emptyset = \text{Y} \%$

Histogram heights:

TEXT

```
2 1 5 6 2 3
```

Each bar has width 1.

Goal:

Find largest rectangle area formed using consecutive bars.

Answer here:

TEXT

```
10
```

from heights:

TEXT

```
5 and 6
```

using minimum height 5 and width 2.

40. BRUTE FORCE INTUITION

For every bar, determine how far it can extend:

```
TEXT
left
right
```

until a smaller bar blocks it.

For bar height $h[i]$, if we know:

```
TEXT
previous smaller index = L
next smaller index = R
```

then width is:

FORMULA BOX

```
TEXT
width = R - L - 1
```

Area:

```
TEXT
area = h[i] * width
```

∅=Y% This reduces histogram to:

```
TEXT
Previous Smaller + Next Smaller
```

which is monotonic stack territory.

41. WHY $R-L-1$?

Suppose smaller boundary indices:

```
TEXT
L      i      R
```

Bar can occupy positions:

```
TEXT
L+1 ... R-1
```

Number of positions:

TEXT

$$(R-1) - (L+1) + 1$$

Simplify:

TEXT

$$R - L - 1$$

42. HISTOGRAM COMPLEXITY

Using monotonic stack:

TEXT

Time = $O(n)$
Space = $O(n)$

instead of common brute-force:

TEXT

$$O(n^2)$$

Placement classic.

43. WHY MONOTONIC STACK WORKS

The stack retains only candidates that have not become useless.

When a new value arrives:

- some older values can never be the nearest required answer anymore
- those values are popped permanently

Thus each element participates in only a small number of stack operations.

This gives linear total complexity.

44. APPLICATION 6 — DAILY TEMPERATURES

Classic placement problem:

Temperatures:

TEXT

73 74 75 71 69 72 76 73

For each day:

How many days until a warmer temperature?

Answer:

TEXT

1 1 4 2 1 1 0 0

This is basically:

Next Greater Element by index distance

Use monotonic stack storing indices.

0-50% same pattern, different story.

45. DAILY TEMPERATURES INSIGHT

When current temperature:

TEXT

76

arrives, it resolves many earlier colder days.

So while:

TEXT

```
current > temperature[stack.top]
```

pop previous index and calculate:

TEXT

```
distance = currentIndex - previousIndex
```

This is a 'Left to Right' version of NGE.

46. TWO VALID NGE STYLES

You can solve NGE by:

Method A

Scan right and directly find current element's answer.

Method B

Scan Left to Right and let current element resolve earlier waiting elements.

Both can be $O(n)$.

0-50% Do not assume only one orientation is correct.

47. LEFT TO RIGHT OR NOT IDEA

Array:

TEXT

4 5 2 10

Maintain stack of unresolved indices.

Read 4:

TEXT

stack: index 0

Read 5:

5 > 4, so:

TEXT

NGE(4)=5

pop 4's index.

Push 5.

Read 2:

push.

Read 10:

10 resolves 2 and 5.

This is often intuitive for "current resolves previous".

48. MONOTONIC STACK VS NORMAL STACK

A normal stack only requires:

TEXT

LIFO

A monotonic stack additionally maintains ordering such as:

TEXT

increasing

or:

TEXT

decreasing

by popping elements that violate the invariant.

◊=Monotonic stack is not a new data structure; it is a usage pattern.

49. GATE TRAP — NESTED LOOPS

Code:

```
c
for (...)
{
    while (...)
        pop();
}
```

does not automatically mean:

```
TEXT
O(n2)
```

For a monotonic stack, each element is pushed once and popped at most once.

Thus:

```
TEXT
O(n)
```

Very important complexity trap.

50. GATE TRAP — STACK STORES "USEFUL" ELEMENTS

Stack does not necessarily contain every previous element.

Elements that can no longer answer future nearest-greater/smaller queries are removed.

That's the central optimization.

51. GATE TRAP — GREATER VS NEXT GREATER

"Greater element" could be any larger value.

"Next greater element" means:

first greater element encountered in that direction

Example:

```
TEXT
2 5 7
```

For 2, both 5 and 7 are greater.

Next greater is:

```
TEXT
5
```

not maximum greater value 7.

Nearest in direction.

52. GATE TRAP — STOCK SPAN IS CONSECUTIVE

Stock span is not:

count of all previous prices "current.

It is:

count of consecutive previous days ending today satisfying the condition.

Example:

TEXT

100 20 30

For 30:

Previous values "d 30 include:

TEXT

20

plus current day.

Span:

TEXT

2

100 blocks further extension.

53. APPLICATION 7 — DFS CONNECTION

Depth First Search can use:

TEXT

explicit stack

or:

TEXT

recursion

because recursion itself uses the call stack.

We'll cover DFS fully in Graphs.

54. APPLICATION 8 — BACKTRACKING

Problems such as:

- maze search
- Sudoku
- permutations
- DFS-style exploration

often need:

remember the most recent decision so it can be undone first.

That's LIFO.

Hence stack/call-stack behavior.

55. APPLICATION 9 — UNDO / REDO

Undo naturally uses a stack:

TEXT

latest action

must be undone first.

Often systems conceptually use two stacks:

TEXT

Undo stack

Redo stack

Undo moves an action from undo to redo.

Redo moves it back.

Common interview explanation.

56. APPLICATION 10 — MIN STACK

Interview question:

Design a stack supporting `getMin()` in $O(1)$.

Naive:

TEXT

scan whole stack

would be:

TEXT

$O(n)$

One solution maintains an auxiliary stack of minimum values.

Example values:

```
TEXT
push 5
push 3
push 7
push 2
```

Min stack tracks:

```
TEXT
5
3
3
2
```

or stores only new minima with suitable duplicate handling.

Thus:

```
TEXT
push 0(1)
pop 0(1)
top 0(1)
getMin 0(1)
```

Ø=Placement extension.

57. MIN STACK DUPLICATE TRAP

Suppose:

```
TEXT
5 3 3
```

If auxiliary stack stores minima improperly and one 3 is popped, you must still know another 3 remains.

So duplicate minima require careful tracking.

Possible approaches:

- push every current minimum
- push duplicates when `<=`
- store `{value, currentMin}` pairs

58. WHEN SHOULD YOU THINK "STACK"?

Strong signals:

- nested structures
- matching brackets
- undo latest

- reverse order
- recursion simulation
- DFS
- nearest greater/smaller
- unresolved elements waiting for future answer
- expression parsing/evaluation

Pattern recognition matters more than memorizing isolated code.

59. STACK PATTERNMAP

Problem	Stack Idea
Balanced parentheses	Match latest opening
Infix !' postfix	Pending operators
Postfix evaluation	Pending operands/results
DFS	Pending vertices
Next Greater	Monotonic stack
Stock Span	Previous greater index
Daily Temperatures	Next greater + distance
Histogram	Previous/next smaller
Undo	Latest action
Backtracking	Latest decision

60. BALANCED BRACKET MEMORY TRICK

****Open = PUSH**

Close = MATCH + POP**

Final stack:

TEXT
EMPTY

means all openings were matched.

61. MONOTONIC STACK MEMORY TRICK

If the top cannot help anymore, remove it forever.

That is the central idea.

62. STOCK SPAN MEMORY TRICK

Find the nearest previous taller wall.

Then:

```
TEXT
span = current index - wall index
```

If no wall:

```
TEXT
span = i+1
```

63. HISTOGRAM MEMORY TRICK

For each bar:

How far can I expand before a smaller bar stops me?

Need:

```
TEXT
previous smaller
next smaller
```

Then:

```
TEXT
width = right - left - 1
```

64. COMPLEXITY MASTER TABLE

Problem	Time	Space
Balanced Parentheses	$O(n)$	$O(n)$
NGE brute force	$O(n^2)$	$O(1)$
NGE stack	$O(n)$	$O(n)$
Stock Span stack	$O(n)$	$O(n)$
Daily Temperatures	$O(n)$	$O(n)$
Histogram optimal	$O(n)$	$O(n)$

Problem	Time	Space
Min Stack operations	O(1) each	O(n)

0=Min-value placement table.

65. GATE-STYLE QUESTION

For:

TEXT

3 1 4 2

find Next Greater to Right.

3

next greater:

TEXT

4

1

next greater:

TEXT

4

4

none:

TEXT

-1

2

none:

TEXT

-1

Answer:

TEXT

4 4 -1 -1

66. GATE-STYLE QUESTION — STOCK SPAN

Prices:

TEXT

10 4 5 90 120 80

Spans:

10

TEXT

1

4

TEXT

1

5

5 covers:

TEXT

4, 5

SO:

TEXT

2

90

covers:

TEXT

10, 4, 5, 90

SO:

TEXT

4

120

covers all five days:

TEXT

5

80

previous day is 120 > 80.

So:

TEXT

1

Answer:

TEXT

1 1 2 4 5 1

67. GATE-STYLE COMPLEXITY QUESTION

Suppose monotonic-stack algorithm:

C

```
for (i = 0; i < n; i++)
{
    while (!empty() && condition)
        pop();

    push(a[i]);
}
```

If each element is pushed once and popped at most once:

Total operations:

TEXT

"d 2 n

Therefore:

TEXT

$\sim (n)$

not $\sim (n^2)$.

$\mathcal{O} = \sqrt{\sim}$ likely reasoning style.

68. INTERVIEW QUESTION

Why is stack ideal for matching parentheses?

Because the latest unmatched opening bracket must be matched before earlier openings.

That is exactly:

TEXT

LIFO

69. INTERVIEW QUESTION

What is a monotonic stack?

A normal stack whose elements/indices are maintained in monotonically increasing or decreasing order by popping elements that violate the required invariant.

70. INTERVIEW QUESTION

Why are indices often stored instead of values?

Because many problems need:

- position
- distance
- span
- width

Example:

TEXT

```
stock span
daily temperatures
histogram
```

71. NOTEBOOK NOTES

Balanced Parentheses

TEXT

```
O p e n i n g ! '   p u s h
```

Closing:

```
    i f   s t a c k   e m p t y ! '   i n v a l i d
    i f   m i s m a t c h ! '   i n v a l i d
    e l s e p o p
```

End:

```
    s t a c k   e m p t y ! '   v a l i d
```

72. NEXT GREATER

R i g h t ! ' v e r s i o n :

TEXT

```
while top <= current:
    pop
```

```
answer = stack top or -1
```

```
push current
```

TEXT

Time = $O(n)$

73. STOCK SPAN

TEXT

```
while price[topIndex] <= currentPrice:
    pop

if empty:
    span = i+1
else:
    span = i-topIndex

push i
```

74. MONOTONIC STACK PROPERTY

TEXT

Each element:
push once
pop at most once

Therefore:

TEXT

$O(n)$

75. 30-SECOND REVISION

Remember:

1. Balanced brackets use LIFO matching.
2. Opening brackets push.
3. Closing brackets match/pop.
4. Final stack must be empty.
5. Monotonic stack solves nearest greater/smaller patterns.
6. NGE can be $O(n)$, not $O(n^2)$.
7. Each element is pushed once and popped at most once.
8. Stock span = previous greater index problem.
9. Store indices when distance/span is required.
10. Histogram uses previous/next smaller elements.

76. ACTIVE RECALL

Q1

Why is `([)])` invalid?

Q2

What is the NGE of 5 in:

TEXT

5 2 8

?

Q3

What does a monotonic stack maintain?

Q4

Why is the NGE stack algorithm $O(n)$ despite a nested while loop?

Q5

Stock span is closely related to which nearest-element problem?

ANSWERS

A1

Because `)` arrives while `[` is the most recent unmatched opening bracket.

A2

TEXT

8

A3

Elements/indices in a monotonic increasing/decreasing invariant.

A4

Each element is pushed once and popped at most once.

A5

Previous Greater Element.

77. PRACTICE — BALANCED BRACKETS

Determine whether balanced:

A

TEXT

`{[()]}`

B

TEXT
{[(())]}

C

TEXT
((()))

D

TEXT
((()))

78. PRACTICE — NGE

Find next greater element:

TEXT
2 7 3 5 4 6 8

Try both:

1. brute force
2. monotonic-stack reasoning

Compare complexity.

79. PRACTICE — PREVIOUS SMALLER

For:

TEXT
4 10 5 8 20 15 3

find the nearest smaller element to the left of each value.

80. PRACTICE — STOCK SPAN

Find spans:

TEXT
31 27 14 21 30 22 25

Use previous-greater logic.

REAR

Rememberize:

Enter from REAR, leave from FRONT.

3. QUEUE ADT

A queue is an:

Abstract Data Type

Common operations:

TEXT

```
enqueue(x)
dequeue()
front() / peek()
isEmpty()
```

Depending on implementation:

TEXT

```
isFull()
size()
```

may also be supported.

4. ENQUEUE

`enqueue(x)` means:

Add `x` at the rear of the queue.

Before:

TEXT

FRONT		REAR	
10	! "		! "
20			
30			

Enqueue:

TEXT

40

After:

TEXT

FRONT

REAR

! "
10 20 30 40

! "

5. DEQUEUE

`dequeue()` means:

Remove the element at the front.

Before:

TEXT

FRONT

REAR

! "
10 20 30 40

! "

Dequeue removes:

TEXT

10

After:

TEXT

FRONT

REAR

! "
20 30 40

! "

~~0 = 0~~ First element leaves first.

6. FRONT / PEEK

`front()` reads the element at front without removing it.

Queue:

TEXT

FRONT

! "
20 30 40

Then:

TEXT

`front() = 20`

Queue remains unchanged.

7. QUEUE OPERATIONS TABLE

Operation	Meaning
enqueue(x)	Insert at rear
dequeue()	Remove front
front()	Read front
isEmpty()	Check whether queue has no elements

8. SIMPLE TRACE

Initially:

```
TEXT
empty
```

Operations:

```
TEXT
enqueue(10)
enqueue(20)
enqueue(30)
dequeue()
enqueue(40)
```

Trace:

After enqueue(10) :

```
TEXT
10
```

After enqueue(20) :

```
TEXT
10 20
```

After enqueue(30) :

```
TEXT
10 20 30
```

Dequeue removes:

```
TEXT
10
```

Remaining:

TEXT

20 30

Enqueue 40:

TEXT

20 30 40

Final front:

TEXT

20

Final rear:

TEXT

40

9. FIFO ORDER

If values enter:

TEXT

A
B
C
D

in this order,

they normally leave:

TEXT

A
B
C
D

Same order.

Compare with stack:

TEXT

D
C
B
A

Queue preserves arrival order.

10. STACK VS QUEUE

Property	Stack	Queue
Rule	LIFO	FIFO
Insert	Top	Rear
Remove	Top	Front
Example	Undo	Waiting line

Memory trick:

TEXT

```

S T A C K ! '   L a s t   c o m e s   o u t   f i r s t
Q U E U E ! '   F i r s t   c o m e s   o u t   f i r s t

```

11. TWO IMPORTANT POINTERS / INDICES

Queue usually maintains:

TEXT

```

front
rear

```

front

Identifies the oldest element.

rear

Identifies the newest element.

Example:

TEXT

```

front
! "
10 20 30 40
           ! '
           rear

```

12. ARRAY QUEUE PREVIEW

Suppose:

C

```

int queue[5];

```

We can maintain:

```
c
int front;
int rear;
```

A common simple representation starts with:

```
TEXT
front = -1
rear  = -1
```

for an empty queue.

But there are multiple valid conventions.

Important GATE rule:

Never memorize only one front/rear formula without checking the representation used.

13. ONE COMMON LINEAR QUEUE REPRESENTATION

Initially:

```
TEXT
front = -1
rear  = -1
```

First enqueue:

```
TEXT
front = 0
rear  = 0
```

Example after enqueueing 10,20,30:

```
TEXT
Index:  0    1    2    3    4
        [10] [20] [30] [ ] [ ]
              !'
        front      rear
```

So:

```
TEXT
front = 0
rear  = 2
```

14. ENQUEUE IN LINEAR ARRAY QUEUE

For a non-full queue:

TEXT

```
rear = rear + 1
queue[rear] = value
```

On first insertion, if queue was empty, initialize:

TEXT

```
front = 0
```

15. DEQUEUE IN LINEAR ARRAY QUEUE

Read:

TEXT

```
queue[front]
```

Then move:

TEXT

```
front = front + 1
```

This is conceptually simple.

But it creates an important problem.

16. THE WASTED-SPACE PROBLEM

Suppose queue capacity:

TEXT

```
5
```

Initially after 5 enqueues:

TEXT

```
Index: 0    1    2    3    4
       [10] [20] [30] [40] [50]
           ! '
       front                rear
```

So:

TEXT

```
front = 0
rear = 4
```

Now dequeue three elements:

TEXT

10
20
30

Queue logically becomes:

TEXT

```
Index: 0    1    2    3    4
       [10] [20] [30] [40] [50]
               ! '      ! '
           front  rear
```

Logical queue:

TEXT

40 50

And:

TEXT

front = 3
rear = 4

17. CAN WE ENQUEUE 60?

There are physically unused locations:

TEXT

index 0
index 1
index 2

But in a simple linear queue, rear is already:

TEXT

4 = capacity - 1

So normal enqueue cannot move further right.

It may report:

QUEUE OVERFLOW

even though earlier positions are unused.

0 = 4 - 1. This is called **wasted space** or false overflow in a naive linear array queue.

18. WHY NOT SHIFT ELEMENTS LEFT?

After every dequeue, we could shift remaining elements left:

TEXT

40 50

from indices:

TEXT

3, 4

to:

TEXT

0, 1

But shifting costs:

TEXT

$O(n)$

per dequeue.

That destroys the desired:

TEXT

$O(1)$

queue operation.

19. SOLUTION — CIRCULAR QUEUE

Instead of treating the array as:

TEXT

0 1 2 3 4

with a hard end,

we conceptually connect:

TEXT

i n d e x 4 ! ' i n d e x 0

forming a circle.

Then rear can wrap around and reuse freed positions.

~~$O(n)$~~ is why circular queue exists.

We study it fully in Lesson 46.

20. QUEUE UNDERFLOW

If queue is empty and we attempt:

TEXT

```
dequeue( )
```

there is nothing to remove.

This is:

QUEUE UNDERFLOW

Same term as stack underflow.

21. QUEUE OVERFLOW

For a fixed-capacity queue:

Attempting to enqueue when no free queue position exists.

This is:

QUEUE OVERFLOW

For a naive linear queue, overflow may occur when:

TEXT

```
rear == capacity - 1
```

even if freed slots exist before `front`.

That's the inefficiency circular queue fixes.

22. EMPTY QUEUE

Depending on implementation, empty may be represented as:

TEXT

```
front = -1  
rear = -1
```

or sometimes:

TEXT

```
front > rear
```

after dequeues.

The exact condition depends on design.

~~NOTE~~ Some questions may define a specific implementation—follow it.

23. QUEUE USING LINKED LIST PREVIEW

A linked queue typically maintains:

```
TEXT
front
rear
```

Example:

```
TEXT
front      rear
!"         !"
1 0 !'    2 0 !'    3 0 !'    4 0 !'    N U L L
```

Enqueue:

```
TEXT
insert at rear
```

Dequeue:

```
TEXT
delete at front
```

With both pointers:

```
TEXT
enqueue = 0(1)
dequeue = 0(1)
```

We'll study this in Lesson 47.

24. WHY DO WE NEED BOTH FRONT AND REAR?

If only `front` is maintained in a singly linked queue:

```
TEXT
f r o n t !'    1 0 !'    2 0 !'    3 0
```

to enqueue at end we need to find last node:

```
TEXT
O(n)
```

If `rear` is also maintained:

```
TEXT
r e a r !'    3 0
```

append becomes:

TEXT

$O(1)$

Important design reasoning.

25. WHY NOT INSERT AT FRONT?

Suppose we inserted new items at front and also removed front.

Then newest item would leave first.

That becomes:

TEXT

LIFO

which behaves like a stack, not a queue.

For FIFO:

TEXT

insert rear
remove front

26. QUEUE COMPLEXITY GOAL

A well-designed queue should support:

TEXT

enqueue $O(1)$
dequeue $O(1)$
front $O(1)$

For:

- circular array queue
- linked queue with front and rear

Must know.

27. LINEAR ARRAY QUEUE COMPLEXITY

If implemented with moving indices and no shifting:

Operation	Time
Enqueue	$O(1)$
Dequeue	$O(1)$
Front	$O(1)$

But:

TEXT

space reuse is poor

That's why circular queue is preferred.

28. BAD ARRAY IMPLEMENTATION

Suppose dequeue physically shifts all elements:

TEXT

[20][30][40]

left after removing 10.

Then dequeue becomes:

TEXT

$O(n)$

This is unnecessary.

~~$O(n)$~~ Good implementation uses indices rather than shifting.

29. APPLICATION — CPU SCHEDULING

Processes waiting:

TEXT

P1 P2 P3 P4

If served in arrival order:

TEXT

P1
then P2
then P3
then P4

queue is natural.

Round-robin scheduling uses a circular queue-like idea because processes may repeatedly return to the rear.

30. APPLICATION — PRINTER QUEUE

Print jobs arrive:

TEXT

Job A
Job B

Job C

Usually oldest waiting job is handled first.

So:

TEXT

FIFO

31. APPLICATION — NETWORK PACKETS

Packets may arrive and wait in a buffer:

TEXT

Packet1

Packet2

Packet3

Processing in arrival order naturally uses a queue abstraction.

Real networking systems can use more sophisticated scheduling, but queue is foundational.

32. APPLICATION — PRODUCER / CONSUMER

Producer generates items:

TEXT

enqueue

Consumer takes items:

TEXT

dequeue

Conceptual buffer:

TEXT

P r o d u c e r ! ' Q U E U E ! ' C o n s u m e r

Very important operating-systems connection.

33. APPLICATION — BFS

Breadth First Search explores nodes level by level.

Suppose graph/tree:

TEXT



BFS:

TEXT

A B C D E

Why queue?

After visiting A:

TEXT

enqueue B
enqueue C

B entered before C, so B is explored first.

Then B's children are added behind existing work.

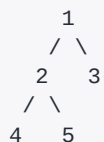
FIFO gives level-order exploration.

Queue ! BFS is a must-know connection.

34. TREE LEVEL-ORDER CONNECTION

For a binary tree:

TEXT



Level order:

TEXT

1 2 3 4 5

uses queue.

We'll revisit during Trees.

35. STACK VS QUEUE IN GRAPH TRAVERSAL

DFS

TEXT
Stack

or recursion.

BFS

TEXT
Queue

Memory trick:

****DFS dives !' Stack**

BFS spreads !' Queue**

Ø=Excellent shortcut.

36. QUEUE OPERATION TRACE

Initially empty.

Operations:

TEXT
enqueue(5)
enqueue(8)
enqueue(2)
dequeue()
enqueue(7)
dequeue()

Start:

TEXT
[]

Enqueue 5:

TEXT
[5]

Enqueue 8:

TEXT
[5, 8]

Enqueue 2:

TEXT
[5, 8, 2]

Dequeue returns:

TEXT
5

Remaining:

TEXT
[8, 2]

Enqueue 7:

TEXT
[8, 2, 7]

Dequeue returns:

TEXT
8

Final:

TEXT
[2, 7]

Front:

TEXT
2

Rear:

TEXT
7

37. GATE TRAP — DEQUEUE VALUE

Suppose queue:

TEXT
10 20 30

Front = 10.

Dequeue returns:

TEXT
10

not:

TEXT

30

That's FIFO.

38. GATE TRAP — FRONT VS REAR

After:

TEXT

enqueue 10

enqueue 20

enqueue 30

front:

TEXT

10

rear:

TEXT

30

Oldest vs newest.

39. GATE TRAP — QUEUE DOES NOT REVERSE

Stack naturally reverses insertion order.

Queue preserves order.

Input:

TEXT

1 2 3 4

Output via queue:

TEXT

1 2 3 4

Output via stack:

TEXT

4 3 2 1

40. GATE TRAP — REMOVE FROM REAR?

Standard queue `dequeue` removes:

TEXT
front

not rear.

A structure allowing insertion/deletion at both ends is:

Deque

We'll study it later.

41. GATE TRAP — PRIORITY QUEUE

A priority queue is not ordinary FIFO.

Removal is based primarily on:

TEXT
priority

rather than only arrival order.

We'll introduce it in Lesson 48 and study binary heap later.

42. QUEUE SIZE

If a simple non-circular linear queue currently uses contiguous logical positions from:

TEXT
front

through:

TEXT
rear

then:

FORMULA BOX

TEXT
$$\text{size} = \text{rear} - \text{front} + 1$$

provided queue is non-empty under that representation.

Example:

TEXT

```
front = 2
rear = 5
```

then:

TEXT

```
size = 5 - 2 + 1 = 4
```

∅ = 0. This formula does **not** directly work for wrapped circular queues.

43. LINEAR QUEUE ADDRESS EXAMPLE

Capacity:

TEXT

```
5
```

State:

TEXT

```
front = 1
rear = 3
```

Logical elements occupy:

TEXT

```
queue[1]
queue[2]
queue[3]
```

So size:

TEXT

```
3
```

Even if:

TEXT

```
queue[0]
```

still physically contains an old dequeued value, it is not logically part of queue.

Same logical-vs-physical idea from stacks.

44. PHYSICAL VALUE AFTER DEQUEUE

Queue:

TEXT

```
Index: 0      1      2
       [10] [20] [30]
           ! '      ! '
       front      rear
```

Dequeue 10.

We may simply set:

TEXT

```
front = 1
```

Memory may still contain:

TEXT

```
queue[0] = 10
```

But logical queue:

TEXT

```
20 30
```

~~0=10%~~ No need to erase old slot for logical deletion.

45. QUEUE WITH ONE ELEMENT

Suppose:

TEXT

```
front = rear = 2
```

Then one element exists at:

TEXT

```
queue[2]
```

After dequeue, queue becomes empty.

Implementation must reset or otherwise mark empty correctly.

Often:

TEXT

```
front = rear = -1
```

again.

~~0=0%~~ One-element case is important in implementation.

46. ONE-ELEMENT LINKED QUEUE

TEXT

```
front
!"
1 0 !' NULL
!'
rear
```

So:

TEXT

```
front == rear
```

After dequeue:

TEXT

```
front = NULL
rear = NULL
```

Both must change.

Same kind of edge case as linked-list head/tail.

47. QUEUE USING TWO STACKS — PLACEMENT PREVIEW Ø=pe

Interesting interview problem:

Implement queue using stacks.

One common design:

- one stack for incoming elements
- another for outgoing elements

It can support FIFO using two LIFO structures.

We'll mention this again in queue applications.

48. STACK USING QUEUES — ANOTHER PREVIEW

Conversely:

Implement stack using queues.

This is also a classic interview exercise.

These problems test whether you understand the behavior of ADTs, not just implementations.

49. QUEUE VARIANTS

We will cover:

Linear Queue

TEXT

simple front/rear movement

Circular Queue

TEXT

indices wrap around

Deque

TEXT

insert/delete both ends

Priority Queue

TEXT

remove according to priority

Linked Queue

TEXT

dynamic nodes

50. LINEAR VS CIRCULAR QUEUE

Linear:

TEXT

0 !' 1 !' 2 !' 3 !' 4 !' E N D

Circular:

TEXT

0 !' 1 !' 2 !' 3 !' 4
!' !"
% % % % % % % % % % % % % %

Circular queue reuses earlier freed slots.

Ø=No GATE topic.

51. WHY MODULO APPEARS IN CIRCULAR QUEUE

If capacity:

TEXT

N

then moving one position circularly:

TEXT

$(\text{index} + 1) \% N$

Example:

TEXT

N = 5

index = 4

Then:

TEXT

$(4+1)\%5 = 0$

wraps back to beginning.

We'll derive all formulas next lessons.

52. QUEUE MEMORY TRICKS

Queue rule

FIRST COME, FIRST SERVED.

Insertion

Join at REAR.

Removal

Leave from FRONT.

BFS

First discovered, first expanded.

Circular queue

End wraps to beginning.

53. QUEUE VS STACK MEMORY TRICK

Think of:

Stack

TEXT

Pile of plates

one end only.

Queue

TEXT

Ticket line

enter from back, leave from front.

54. MASTER COMPLEXITY TABLE

For well-designed standard queue:

Operation	Time
Enqueue	$O(1)$
Dequeue	$O(1)$
Front/Peek	$O(1)$
isEmpty	$O(1)$
Search arbitrary value	$O(n)$

$O=1$ Must know.

55. WHY SEARCH IS $O(n)$

Queue directly exposes:

TEXT

front

and rear metadata.

It does not provide a search index structure.

To find arbitrary value, may need scan:

TEXT

$O(n)$

56. GATE-STYLE TRACE

Initially empty.

Operations:

TEXT

```
enqueue(1)
enqueue(2)
dequeue()
enqueue(3)
enqueue(4)
dequeue()
```

Trace:

TEXT

```
e n q u e u e   1   !'   [ 1 ]
e n q u e u e   2   !'   [ 1 , 2 ]
d e q u e u e       !'   r e t u r n s   1   !'   [ 2 ]
e n q u e u e   3   !'   [ 2 , 3 ]
e n q u e u e   4   !'   [ 2 , 3 , 4 ]
d e q u e u e       !'   r e t u r n s   2   !'   [ 3 , 4 ]
```

Final:

TEXT

```
front = 3
rear = 4
```

as logical values.

57. GATE-STYLE QUESTION — OPERATIONS COUNT

Queue begins empty.

Operations:

TEXT

```
7 successful enqueues
3 successful dequeues
```

Final size:

TEXT

```
7 - 3 = 4
```

General:

FORMULA BOX

TEXT

```
final size
=
```

```
initial size
+ successful enqueues
- successful dequeues
```

Same size-count principle as stack.

58. WATCH UNDERFLOW

If queue is empty and dequeue is attempted:

```
TEXT
unsuccessful dequeue
```

should not reduce logical size below zero.

So count only:

```
TEXT
successful operations
```

in numerical questions.

59. GATE-STYLE CONCEPT QUESTION

Which data structure is naturally used for BFS?

A. Stack B. Queue C. Heap D. BST

Answer: B — Queue

Because nodes are processed in order of discovery, level by level.

60. GATE-STYLE CONCEPT QUESTION

Which combination gives standard queue behavior?

A. Insert front, delete front B. Insert rear, delete front C. Insert rear, delete rear D. Insert front, delete rear only under all implementations

Correct:

B

Standard queue:

```
TEXT
enqueue at rear
dequeue at front
```

61. INTERVIEW QUESTION

Why does a naive array queue waste space?

Because `front` moves forward after dequeues but `rear` cannot wrap around to reuse freed slots at earlier array indices.

62. INTERVIEW QUESTION

How do we solve wasted space without shifting elements?

Use:

Circular Queue

with modular index movement.

63. INTERVIEW QUESTION

Why maintain both front and rear in linked queue?

To make:

TEXT

```
dequeue front = 0(1)
enqueue rear  = 0(1)
```

Without rear, appending could cost $O(n)$.

64. DON'T CONFUSE

Queue

TEXT

FIFO

Stack

TEXT

LIFO

Enqueue

TEXT

insert rear

Dequeue

TEXT

remove front

Underflow

TEXT

dequeue empty queue

Overflow

TEXT

enqueue full fixed queue

65. NOTEBOOK NOTES

Queue

TEXT

FIFO

Operations

TEXT

```
Enqueue !' insert at rear
Dequeue !' remove from front
Front   !' read oldest element
```

Goal

TEXT

enqueue = $O(1)$
dequeue = $O(1)$
front = $O(1)$

66. LINEAR ARRAY ISSUE

TEXT

rear reaches last array index

but:

TEXT

free slots may exist before front

Result:

TEXT

wasted space

Solution:

TEXT

Circular Queue

67. 30-SECOND REVISION

Remember:

1. Queue follows FIFO.
2. Enqueue inserts at rear.
3. Dequeue removes from front.
4. Front is oldest element.
5. Rear is newest element.
6. Underflow = dequeue from empty queue.
7. Overflow = enqueue to full fixed queue.
8. Naive linear array queue wastes freed front space.
9. Circular queue fixes reuse using wrap-around.
10. BFS uses a queue.

68. ACTIVE RECALL

Q1

What does FIFO mean?

Q2

Where does enqueue happen?

Q3

Where does dequeue happen?

Q4

Why does naive linear array queue waste space?

Q5

Which graph traversal naturally uses a queue?

ANSWERS

A1

First In, First Out.

A2

At rear.

A3

At front.

A4

Because front advances, but rear cannot wrap around to reuse earlier freed array positions.

A5

BFS.

69. PRACTICE — EASY

Start empty:

TEXT

```
enqueue(10)
enqueue(20)
enqueue(30)
dequeue()
enqueue(40)
```

Find:

- dequeued value
- final queue
- front
- rear

70. PRACTICE — STACK VS QUEUE

Insert values:

TEXT

```
1 2 3 4
```

What removal order results from:

A. Stack

B. Queue

71. PRACTICE — LINEAR ARRAY

Capacity:

TEXT

```
5
```


After:

TEXT

```
enqueue 10
enqueue 20
enqueue 30
enqueue 40
enqueue 50
```

then three dequeues,

state:

TEXT

```
front = 3
rear = 4
```

Why can naive linear implementation reject another enqueue even though indices 0, 1, 2 are logically free?

72. PRACTICE — COMPLEXITY

State complexity of:

- enqueue
- dequeue
- front
- search arbitrary element

for a properly designed queue.

PROGRESS

MODULE 5 — QUEUE

' Lesson 44 — Queue Fundamentals

NEXT — LESSON 45

Linear Queue Using Array in C — front , rear , Enqueue/Dequeue & Wasted Space

Next we implement a full array queue and master:

- exact front / rear initialization
- first enqueue
- normal enqueue
- dequeue
- one-element queue
- resetting empty queue
- overflow / underflow

- physical vs logical contents
- complete C implementation
- output tracing
- why `rear == MAX-1` creates false overflow
- why shifting is inefficient
- GATE traps

Then:

Lesson 46 — Circular Queue

which is one of the most important queue topics for GATE.

USER

next

ASSISTANT

MODULE 5 — LESSON 45

Linear Queue Using Array in C — `front` , `rear` , Enqueue, Dequeue & Wasted Space

Now we implement a **linear queue using an array**.

This lesson is important because many GATE questions ask:

- values of `front` and `rear`
- overflow / underflow
- output after enqueue/dequeue
- one-element queue handling
- why linear queue wastes space
- why circular queue is better

GATE importance: 0-1% (MUST KNOW) **Placement importance:** +1% (IMPORTANT)

1. BASIC REPRESENTATION

Suppose:

```
C
#define MAX 5

int queue[MAX];
int front = -1;
int rear = -1;
```

We use:

TEXT

```
front !' first logical element
rear  !' last logical element
```

Initially:

TEXT

```
front = -1
rear  = -1
```

means:

EMPTY QUEUE

2. VISUALIZATION

Initially:

TEXT

```
Index:   0    1    2    3    4
         [ ] [ ] [ ] [ ] [ ]

front = -1
rear  = -1
```

After:

TEXT

```
enqueue(10)
enqueue(20)
enqueue(30)
```

we get:

TEXT

```
Index:   0    1    2    3    4
         [10] [20] [30] [ ] [ ]

         front      rear      !'
```

So:

TEXT

```
front = 0
rear  = 2
```

3. FIRST ENQUEUE IS SPECIAL

Suppose queue is empty:

TEXT

```
front = -1  
rear = -1
```

Enqueue 10.

We need:

TEXT

```
front = 0  
rear = 0
```

and:

TEXT

```
queue[0] = 10
```

Why set `front` ?

Because before first enqueue there was no first element.

4. ENQUEUE ALGORITHM

For this representation:

TEXT

```
If rear == MAX-1:  
    Overflow  
  
Else:  
    if queue empty:  
        front = 0  
  
    rear = rear + 1  
    queue[rear] = value
```

5. ENQUEUE FUNCTION

C

```
void enqueue(int value)  
{  
    if (rear == MAX - 1)  
    {  
        printf("Queue Overflow\n");  
        return;  
    }  
  
    if (front == -1)  
        front = 0;  
  
    rear++;  
    queue[rear] = value;  
}
```

Time:

```
TEXT
0(1)
```

6. ENQUEUE TRACE

Initially:

```
TEXT
front = -1
rear = -1
```

enqueue(10)

Since empty:

```
TEXT
front = 0
```

Then:

```
TEXT
rear = 0
queue[0] = 10
```

State:

```
TEXT
[10][ ][ ][ ][ ]
!
front/rear
```

7. SECOND ENQUEUE

```
TEXT
enqueue(20)
```

Queue is not empty.

So:

```
TEXT
rear = 1
queue[1] = 20
```

State:

TEXT

```
[10][20][ ][ ][ ]
! ' ! '
F R
```

8. THIRD ENQUEUE

TEXT

```
enqueue(30)
```

Result:

TEXT

```
[10][20][30][ ][ ]
! ' ! '
F R
```

TEXT

```
front = 0
rear = 2
```

9. DEQUEUE

Queue:

TEXT

```
front
! "
10 20 30
! '
rear
```

Deque removes:

TEXT

```
10
```

Then `front` moves forward.

Conceptually:

TEXT

```
value = queue[front]
front++
```

10. DEQUEUE FUNCTION

```

c
int dequeue(void)
{
    if (front == -1)
    {
        printf("Queue Underflow\n");
        return -1;
    }

    int value = queue[front];

    if (front == rear)
    {
        front = -1;
        rear = -1;
    }
    else
    {
        front++;
    }

    return value;
}

```

Notice the special:

TEXT

```
front == rear
```

case.

11. WHY `front == rear` IS SPECIAL

Suppose queue has one element:

TEXT

```

Index: 2
      [50]
      !
front=rear

```

After removing 50, queue becomes empty.

We should not leave:

TEXT

```
front = 3
rear = 2
```

under this chosen representation.

Instead reset:

TEXT

```
front = -1
rear = -1
```

This makes future empty checks simple.

12. ONE-ELEMENT TRACE

Before:

```
TEXT
front = 2
rear = 2
queue[2] = 50
```

Execute:

```
TEXT
dequeue()
```

Returns:

```
TEXT
50
```

Then:

```
TEXT
front = -1
rear = -1
```

Queue empty.

0-100 Important GATE edge case.

13. NORMAL DEQUEUE TRACE

Before:

```
TEXT
Index: 0    1    2
       [10] [20] [30]
           ! '   ! '
         F     R
```

Dequeue:

```
TEXT
value = 10
```

Since:

```
TEXT
front != rear
```


we do:

```
TEXT
front++
```

So:

```
TEXT
front = 1
rear = 2
```

Logical queue:

```
TEXT
20 30
```

14. PHYSICAL MEMORY AFTER DEQUEUE

The array may still physically contain:

```
TEXT
[10][20][30]
```

But logically:

```
TEXT
front = 1
rear = 2
```

So active queue is only:

```
TEXT
20 30
```

Same idea as stack:

Metadata decides what is logically valid.

15. FRONT / PEEK

```
C
int peek(void)
{
    if (front == -1)
    {
        printf("Queue is empty\n");
        return -1;
    }
}
```

```
    return queue[front];  
}
```

Time:

```
TEXT  
O(1)
```

Peek does not modify `front`.

16. DISPLAY

```
C  
void display(void)  
{  
    if (front == -1)  
    {  
        printf("Queue is empty\n");  
        return;  
    }  
  
    for (int i = front; i <= rear; i++)  
        printf("%d ", queue[i]);  
}
```

Time:

```
TEXT  
O(n)
```

17. COMPLETE PROGRAM

```
C  
#include <stdio.h>  
  
#define MAX 5  
  
int queue[MAX];  
int front = -1;  
int rear = -1;  
  
void enqueue(int value)  
{  
    if (rear == MAX - 1)  
    {  
        printf("Queue Overflow\n");  
        return;  
    }  
  
    if (front == -1)  
        front = 0;  
  
    queue[++rear] = value;  
}
```

```

int dequeue(void)
{
    if (front == -1)
    {
        printf("Queue Underflow\n");
        return -1;
    }

    int value = queue[front];

    if (front == rear)
    {
        front = -1;
        rear = -1;
    }
    else
    {
        front++;
    }

    return value;
}

int peek(void)
{
    if (front == -1)
        return -1;

    return queue[front];
}

void display(void)
{
    if (front == -1)
        return;

    for (int i = front; i <= rear; i++)
        printf("%d ", queue[i]);
}

```

18. BASIC OPERATION TRACE

Initially:

TEXT

F = -1
R = -1

Operations:

TEXT

enqueue(10)
enqueue(20)
enqueue(30)
dequeue()
enqueue(40)

After first three:

TEXT

```
[10][20][30][ ][ ]  
F      R
```

After dequeue:

TEXT

```
[10][20][30][ ][ ]  
      F  R
```

Logical:

TEXT

20 30

Enqueue 40:

TEXT

```
[10][20][30][40][ ]  
      F      R
```

Logical queue:

TEXT

20 30 40

19. QUEUE SIZE

For a non-empty linear queue under this representation:

FORMULA BOX

TEXT

size = rear - front + 1

Example:

TEXT

```
front = 1  
rear  = 3
```

Then:

TEXT

```
size = 3 - 1 + 1  
      = 3
```

20. SIZE WHEN EMPTY

If:

```
TEXT
front = rear = -1
```

do not apply:

```
TEXT
rear - front + 1
```

blindly.

It gives:

```
TEXT
-1 - (-1) + 1 = 1
```

which is wrong.

So:

```
TEXT
if empty !' size = 0
else !' rear - front + 1
```

Formula precondition matters.

21. QUEUE FULL CONDITION

For this **naïve linear queue**:

```
TEXT
rear == MAX - 1
```

is treated as full.

Example:

```
TEXT
MAX = 5
rear = 4
```

No further rightward array slot exists.

But here's the problem.

22. WASTED SPACE / FALSE OVERFLOW

Capacity 5.

First enqueue five values:

TEXT

```
[10][20][30][40][50]
F              R
```

Now dequeue three:

TEXT

```
10
20
30
```

State:

TEXT

```
[10][20][30][40][50]
              F    R
```

Logical queue:

TEXT

```
40 50
```

Unused logical positions:

TEXT

```
0
1
2
```

But:

TEXT

```
rear = 4
```

So this implementation says:

TEXT

```
rear == MAX-1
```

and rejects another enqueue.

23. WHY IS THIS BAD?

Array capacity:

TEXT

```
5
```

Current logical elements:

TEXT

2

So theoretically:

TEXT

3 free slots

exist.

But linear rear cannot move backward or wrap around.

Hence:

FALSE OVERFLOW / WASTED SPACE

0-1% is the main limitation of linear array queues.

24. COULD WE RESET REAR?

No, not while elements remain at indices 3 and 4.

If we simply set:

TEXT

rear = 1

new enqueues might overwrite logical data or destroy order.

So we need a systematic reuse mechanism.

25. OPTION 1 — SHIFT ELEMENTS

After dequeuing:

TEXT

40 50

we could shift them:

TEXT

Before:

[][][][40][50]

After:

[40][50][][][]

Then:

TEXT

front = 0

rear = 1

Now free space is reusable.

But shifting costs:

TEXT

$O(n)$

Ø=Not desirable.

26. OPTION 2 — CIRCULAR QUEUE

Better solution:

Treat array as circular:

TEXT

```
0  !'  1  !'  2  !'  3  !'  4
! '                                ! "
% % % % % % % % % % % % % % % %
```

Then after rear reaches 4:

TEXT

```
next rear = 0
```

if that slot is logically free.

This preserves:

TEXT

```
enqueue  $O(1)$ 
dequeue  $O(1)$ 
```

without shifting.

Ø=Yes is why Lesson 46 is extremely important.

27. GATE TRAP — `rear == MAX-1`

Statement:

If `rear == MAX-1`, the queue is always genuinely full.

Wrong in general.

It is true only for this naive **linear queue implementation's enqueue rule**.

In a circular queue, free slots may still exist.

Ø=No ways identify representation.

28. GATE TRAP — EMPTY AFTER DEQUEUE

Suppose:

```
TEXT
front = rear = 3
```

One element exists.

After dequeue, implementation resets:

```
TEXT
front = rear = -1
```

If you forget this, later operations can become inconsistent.

29. GATE TRAP — `front == rear`

Does:

```
TEXT
front == rear
```

mean empty?

Not in this representation.

It means:

Exactly one element exists

provided:

```
TEXT
front != -1
```

Empty is:

```
TEXT
front == -1
```

$\emptyset$ = Major trap.

30. STATES

Empty

```
TEXT
front = -1
rear = -1
```

One element

TEXT

```
front == rear >= 0
```

Multiple elements

TEXT

```
front < rear
```

for this non-wrapped linear representation.

31. GATE OUTPUT TRACE

Suppose:

C

```
enqueue(10);
enqueue(20);
enqueue(30);

printf("%d ", dequeue());
printf("%d ", peek());
printf("%d %d", front, rear);
```

After enqueue:

TEXT

```
front = 0
rear = 2
```

`dequeue()` returns:

TEXT

```
10
```

Then:

TEXT

```
front = 1
rear = 2
```

`peek()` returns:

TEXT

```
20
```

Output:

TEXT

```
10 20 1 2
```

32. TRACE WITH EMPTY RESET

Start empty.

```
TEXT
enqueue(5)
```

Now:

```
TEXT
front = 0
rear = 0
```

Then:

```
TEXT
dequeue( )
```

returns:

```
TEXT
5
```

and resets:

```
TEXT
front = -1
rear = -1
```

Then enqueue 8:

First enqueue rules apply again:

```
TEXT
front = 0
rear = 0
queue[0] = 8
```

Resetting lets the queue reuse the array after becoming completely empty.

33. IMPORTANT NUANCE

Even the naive linear queue can reuse positions after the queue becomes completely empty, because we reset:

```
TEXT
front = rear = -1
```

The wasted-space problem occurs when:

```
TEXT
rear reaches end
```

while some elements still remain in the queue.

34. EXAMPLE

Capacity:

TEXT

5

Operations:

TEXT

enqueue 1
enqueue 2
enqueue 3
enqueue 4
enqueue 5

dequeue
dequeue
dequeue
dequeue
dequeue

Queue becomes fully empty.

Reset:

TEXT

front = rear = -1

Now array can be reused from index 0.

No problem.

35. PROBLEM CASE

But:

TEXT

enqueue 1 2 3 4 5
dequeue
dequeue
dequeue

still leaves:

TEXT

4 5

Queue not empty.

Rear remains at last index.

So enqueue 6 is rejected in naive linear queue.

$O(1)$ That's the actual wasted-space case.

36. ENQUEUE COMPLEXITY

Without shifting:

```
TEXT
0(1)
```

because:

```
TEXT
rear++
queue[rear] = x
```

constant work.

37. DEQUEUE COMPLEXITY

Using index movement:

```
TEXT
0(1)
```

because:

```
TEXT
front++
```

No shifting.

38. DISPLAY COMPLEXITY

If there are n active elements:

```
TEXT
0(n)
```

because every active slot is visited.

39. SEARCH COMPLEXITY

Search key:

```
TEXT
front ... rear
```

Worst case:

TEXT

$O(n)$

40. COMPLEXITY TABLE

Operation	Linear Array Queue
Enqueue	$O(1)$
Dequeue	$O(1)$
Peek	$O(1)$
isEmpty	$O(1)$
Display	$O(n)$
Search	$O(n)$

Space:

TEXT

$O(\text{MAX})$

41. LOGICAL VS PHYSICAL CONTENTS

Suppose:

TEXT

```
queue = [10, 20, 30, 40, 50]
front = 3
rear = 4
```

Physically array contains 5 numbers.

Logically queue contains only:

TEXT

40 50

~~0-100~~ We never determine queue contents by looking only at raw array values.

Use:

TEXT

front
rear

42. ARRAY QUEUE VS ARRAY STACK

Array stack uses:

TEXT
top

Queue uses:

TEXT
front
rear

Why two indices?

Because:

TEXT
Stack:
insert and delete at same end

Queue:

TEXT
insert and delete at opposite ends

Ø = Excellent conceptual difference.

43. STACK ARRAY STATE

TEXT
[10][20][30]
! '
top

Push and pop use:

TEXT
top

44. QUEUE ARRAY STATE

TEXT
[10][20][30]
! '
F R

Enqueue uses:

TEXT
rear

Deque uses:

```
TEXT
front
```

45. GATE TRAP — DEQUEUE WITH SHIFTING

If implementation says:

On every dequeue, shift all elements one position left.

Then:

```
TEXT
dequeue = O(n)
```

Do not automatically answer $O(1)$.

~~$O(1)$~~ Complexity depends on actual implementation.

Standard efficient linear-index queue:

```
TEXT
O(1)
```

46. GATE TRAP — DIFFERENT FRONT/REAR CONVENTIONS

Some textbooks use:

```
TEXT
front = 0
rear = -1
```

and interpret `rear` as last occupied slot.

Others define `rear` as next free slot.

Circular queues also have multiple representations.

Therefore in GATE:

Read initialization and update rules before applying formulas.

~~$O(1)$~~ Concept > rote formula.

47. BETTER ERROR HANDLING

Like stack, returning:

TEXT

-1

for underflow is ambiguous if `-1` is valid queue data.

A cleaner API:

```
c
int dequeue(int *value)
{
    if (front == -1)
        return 0;

    *value = queue[front];

    if (front == rear)
        front = rear = -1;
    else
        front++;

    return 1;
}
```

Now:

TEXT

```
return 1 !' success
return 0 !' failure
```

and actual data goes through `*value`.

48. STRUCT-BASED IMPLEMENTATION

Better design:

```
c
typedef struct
{
    int data[MAX];
    int front;
    int rear;
} Queue;
```

Initialize:

```
c
Queue q;

q.front = -1;
q.rear = -1;
```

This supports multiple queues.

49. WHY GLOBAL VERSION IS LIMITED

Globals:

```
C
int queue[MAX];
int front;
int rear;
```

represent one queue.

Struct:

```
C
Queue q1;
Queue q2;
```

allows independent queues.

Ø=Software-design improvement.

50. GATE-STYLE NUMERICAL

Capacity:

```
TEXT
MAX = 6
```

After operations:

```
TEXT
enqueue A
enqueue B
enqueue C
enqueue D
dequeue
dequeue
```

State:

```
TEXT
front = 2
rear = 3
```

Number of elements:

```
TEXT
rear-front+1
= 3-2+1
= 2
```

Values:

```
TEXT
C D
```

51. ANOTHER NUMERICAL

Suppose:

```
TEXT
front = 4
rear = 7
```

in a non-empty linear queue.

Size:

```
TEXT
7 - 4 + 1
= 4
```

52. DON'T APPLY THAT TO CIRCULAR QUEUE

Suppose later:

```
TEXT
front = 4
rear = 1
```

in a wrapped circular queue.

Then:

```
TEXT
rear - front + 1
```

becomes negative.

But queue can be perfectly valid.

~~O = O(n)~~ Circular queue needs modular reasoning.

53. INTERVIEW QUESTION

Why is circular queue superior to simple linear array queue?

Because it:

- reuses freed array slots
- avoids $O(n)$ shifting
- preserves $O(1)$ enqueue/dequeue

54. INTERVIEW QUESTION

Why isn't shifting after every dequeue desirable?

Because removing one element can require moving $O(n)$ remaining elements.

55. INTERVIEW QUESTION

What happens when the only queue element is dequeued?

Both queue indices should represent empty again under this implementation:

TEXT

```
front = rear = -1
```

56. MEMORY TRICKS

Empty

Both are outside: `-1, -1`.

First enqueue

Bring FRONT in, then move REAR.

Enqueue

REAR moves right.

Dequeue

FRONT moves right.

Last dequeue

Both go back to `-1`.

Linear queue problem

FRONT leaves holes; REAR can't return.

Great memory line.

57. NOTEBOOK NOTES

Initialization

```
C
front = -1;
rear = -1;
```

Enqueue

```
C
if (rear == MAX-1)
    overflow;

if (front == -1)
    front = 0;

queue[++rear] = x;
```

58. DEQUEUE

```
C
if (front == -1)
    underflow;

x = queue[front];

if (front == rear)
    front = rear = -1;
else
    front++;
```

59. FORMULA BOX

For non-empty simple linear queue:

```
TEXT
size = rear - front + 1
```

Full according to naive implementation:

```
TEXT
rear == MAX-1
```

Empty:

```
TEXT
front == -1
```

60. DON'T CONFUSE

front == -1

TEXT

Empty

front == rear != -1

TEXT

One element

rear == MAX-1

TEXT

No more right-side array positions

Not necessarily true physical capacity exhaustion if earlier slots were freed.

61. 30-SECOND REVISION

Remember:

1. Linear array queue uses **front** and **rear**.
2. Empty often starts at **-1, -1**.
3. First enqueue sets front to 0.
4. Enqueue increments rear.
5. Dequeue reads front then advances it.
6. If last element removed, reset both to -1.
7. Enqueue/dequeue are $O(1)$ without shifting.
8. Naive linear queue wastes earlier freed positions.
9. Shifting fixes reuse but makes dequeue $O(n)$.
10. Circular queue solves this efficiently.

62. ACTIVE RECALL

Q1

What do **front=-1** and **rear=-1** mean?

Q2

What happens to **front** during first enqueue?

Q3

Why is **front==rear** not an empty condition here?

Q4

Why can **rear==MAX-1** cause false overflow?

Q5

What data structure solves this wasted-space problem?

ANSWERS

A1

Queue is empty.

A2

It becomes 0.

A3

It represents one remaining element when `front != -1`.

A4

Because slots before `front` may have become free but cannot be reused by a non-wrapping rear.

A5

Circular Queue.

63. PRACTICE — TRACE

Capacity 5.

Start:

TEXT

F=-1, R=-1

Operations:

TEXT

```
enqueue(10)
enqueue(20)
enqueue(30)
dequeue()
dequeue()
enqueue(40)
```

Find:

- values dequeued
- final front
- final rear
- logical queue

64. PRACTICE — LAST ELEMENT

Start with:

TEXT

```
front = 3
rear = 3
queue[3] = 90
```

Perform dequeue.

What are final:

TEXT

```
front
rear
```

?

65. PRACTICE — WASTED SPACE

Capacity:

TEXT

5

State:

TEXT

```
front = 3
rear = 4
```

How many logical elements exist?

How many array slots are physically before `front` ?

Why can't the naive linear queue immediately reuse them?

66. GATE-STYLE QUESTION

For a simple linear array queue of capacity 10:

TEXT

```
front = 6
rear = 9
```

Number of logical queue elements:

TEXT

$9 - 6 + 1 = 4$

How many indices `0..5` are outside the current logical queue?

TEXT

6

Yet another enqueue is rejected under:

```
TEXT
rear == MAX-1
```

This illustrates:

wasted space

PROGRESS

MODULE 5 — QUEUE

' Lesson 44 — Queue Fundamentals ' Lesson 45 — Linear Queue Using Array

NEXT — LESSON 46

Circular Queue — Wrap-Around, Modulo, Full/Empty Conditions & GATE Numericals

This is one of the most important Queue lessons.

We will master:

```
TEXT
nextIndex = (index + 1) % N
```

and understand exactly why:

```
TEXT
rear = N-1
```

can move to:

```
TEXT
0
```

We will cover:

- circular array intuition
- wrap-around
- enqueue/dequeue
- empty and full cases
- one-slot-empty representation
- full condition

```
((rear + 1) % N == front)
```

- modulo arithmetic
- exact front/rear traces

- queue size
- GATE numerical questions
- common ambiguity: capacity `N` but only `N-1` usable slots in one representation
- alternative count-based representation

Then Lesson 47 — **Queue Using Linked List**.

USER

next

ASSISTANT

MODULE 5 — LESSON 46

Circular Queue — Wrap-Around, Modulo, Full/Empty Conditions & GATE Numericals

This is one of the **most important Queue topics for GATE**.

A circular queue solves the major problem of a linear array queue:

Wasted space after dequeues

GATE importance: $\emptyset = \text{Yes\%} \rightarrow \text{Must Know}$ **Placement importance:** $\emptyset = \text{Yes\%} \rightarrow \text{Important}$

1. WHY CIRCULAR QUEUE?

Recall linear queue:

TEXT

```
Index: 0   1   2   3   4
       [ ] [ ] [ ] [40] [50]
               !'      !'
               F       R
```

Suppose:

TEXT

```
front = 3
rear  = 4
```

Indices:

TEXT

```
0, 1, 2
```

are free.

But `rear` already reached:

TEXT

N-1

A naive linear queue cannot use them.

Circular queue solves this by making:

TEXT

4 !' 0

conceptually.

2. CIRCULAR ARRAY INTUITION

For capacity 5:

TEXT

0
!- !~
4 1
!' !"
3 !•% % % 2

Or:

TEXT

0 !' 1 !' 2 !' 3 !' 4
!' !"
% % % % % % % % % % % % % % %

After index 4:

TEXT

next = 0

~~0-4~~ May itself is still physically linear.

Only the **index logic** is **circular**.

3. MOST IMPORTANT FORMULA

To move one position forward:

FORMULA BOX

TEXT

next = (index + 1) % N

where:

TEXT

$N = \text{queue array size}$

Example:

TEXT

$N = 5$

Then:

TEXT

$(0+1)\%5 = 1$
 $(1+1)\%5 = 2$
 $(2+1)\%5 = 3$
 $(3+1)\%5 = 4$
 $(4+1)\%5 = 0$

$0 \equiv 5$ Modulo causes wrap-around.

4. WHY % N?

Because:

TEXT

$5 \% 5 = 0$

So from last index:

TEXT

4

we calculate:

TEXT

$(4+1)\%5$
 $= 5\%5$
 $= 0$

That's the entire circular-queue idea.

5. TWO COMMON REPRESENTATIONS

Circular queues can be implemented in different ways.

This matters a lot in GATE.

We will study two common designs:

Representation A

One slot intentionally left empty

Very common textbook approach.

Representation B

Maintain a separate:

```
TEXT
count
```

Then all `N` slots can be used.

~~0-100%~~ Always identify which representation the question uses.

6. REPRESENTATION A — ONE SLOT LEFT EMPTY

Suppose array size:

```
TEXT
N
```

We use `front` and `rear`.

One common convention:

```
TEXT
front = index of first element
rear  = index of last element
```

Empty:

```
TEXT
front = -1
rear  = -1
```

Full condition:

FORMULA BOX

```
TEXT
(rear + 1) % N == front
```

~~0-100%~~ Must know.

7. WHY IS THIS FULL?

Imagine:

```
TEXT
N = 5
```

State:

TEXT

```
front = 1  
rear = 0
```

Then:

TEXT

```
(rear+1)%5  
= 1
```

which equals `front`.

That means if rear moves one more step:

TEXT

rear would collide with front

So we declare queue full.

8. IMPORTANT CONSEQUENCE

If array has:

TEXT

N slots

and we use this representation, maximum stored elements are:

N - 1

because one slot is sacrificed to distinguish:

TEXT

full

from:

TEXT

empty

0 = 6%N favorite.

9. EXAMPLE

Array size:

TEXT

N = 5

Maximum usable queue elements:

TEXT

4

not 5.

Because one slot remains unused as the distinguishing gap.

10. INITIAL EMPTY STATE

Let's use:

C

```
int front = -1;
int rear = -1;
```

Empty when:

TEXT

```
front == -1
```

and usually:

TEXT

```
rear == -1
```

too.

11. FIRST ENQUEUE

Suppose empty.

Enqueue 10.

Set:

TEXT

```
front = 0
rear = 0
```

Then:

TEXT

```
queue[0] = 10
```

State:

TEXT

```
Index: 0   1   2   3   4
       [10][ ][ ][ ][ ]
           !
       F, R
```

12. NORMAL ENQUEUE

If queue is not full:

```
c
rear = (rear + 1) % N;
queue[rear] = value;
```

~~O(N)~~ Difference from linear queue:

```
TEXT
rear++
```

becomes:

```
TEXT
rear = (rear+1)%N
```

13. ENQUEUE TRACE

Capacity 5.

Start:

```
TEXT
F=-1 R=-1
```

Enqueue 10:

```
TEXT
F=0 R=0
```

Enqueue 20:

```
TEXT
F=0 R=1
```

Enqueue 30:

```
TEXT
F=0 R=2
```

Enqueue 40:

```
TEXT
F=0 R=3
```

Can we enqueue 50?

Check:

TEXT

```
(R+1)%5
= (3+1)%5
= 4
```

Is 4 equal to front 0?

No.

So enqueue 50 at index 4?

Careful: under this `front=first`, `rear=last` plus `empty=-1` formulation, doing so yields 5 stored slots and then `(4+1)%5 = 0 = front`, meaning it has become full after the insertion. This representation can indeed use all N slots if empty is separately encoded with `-1`; the one-slot-empty rule belongs to a different convention where empty is `front == rear` or `rear` denotes the next insertion position.

~~0=0~~ **DON'T CONFUSE:** Circular-queue formulas depend on the exact representation.

Let's therefore separate the two clean conventions properly.

14. CLEAN CONVENTION 1 — `rear` IS LAST OCCUPIED, `EMPTY = -1`

This is a common practical implementation.

Definitions:

TEXT

```
front = first occupied index
rear  = last occupied index
```

Empty:

TEXT

```
front = rear = -1
```

Full:

TEXT

```
(rear + 1) % N == front
```

With the explicit `-1` empty state, this convention can use all N slots.

Example $N=5$:

TEXT

```
front=0 rear=4
```

Check:

TEXT

```
(4+1)%5 = 0 = front
```

So queue is full.

Number of elements = 5.

Ø=100% all N slots are usable.

15. CLEAN CONVENTION 2 — front AND rear AS BOUNDARIES / ONE SLOT EMPTY

Another common textbook design uses:

```
TEXT
front == rear
```

for empty.

Then full is often:

```
TEXT
(rear + 1) % N == front
```

But now empty and full would otherwise conflict, so one slot is deliberately unused.

Thus usable capacity:

```
TEXT
N-1
```

Ø=99% one-looking full formula, different empty convention.

This is why blindly memorizing:

```
TEXT
N-1 usable
```

is dangerous.

16. GATE RULE

Before solving a circular queue problem, write:

1. What does front point to?
2. What does rear point to?
3. What is the empty condition?
4. What is the full condition?

Only then calculate.

Ø=100% Very important exam strategy.

17. WE WILL USE THIS MAIN IMPLEMENTATION

For clarity, we'll use:

TEXT

```
front = first element
rear = last element
empty = front == -1
full = (rear+1)%N == front
```

This uses all **N** slots.

18. ENQUEUE ALGORITHM

TEXT

```
ENQUEUE(x)

if (rear+1)%N == front:
    Overflow

else if queue empty:
    front = rear = 0
    queue[rear] = x

else:
    rear = (rear+1)%N
    queue[rear] = x
```

19. C IMPLEMENTATION

```
c
void enqueue(int value)
{
    if ((rear + 1) % MAX == front)
    {
        printf("Queue Overflow\n");
        return;
    }

    if (front == -1)
    {
        front = rear = 0;
    }
    else
    {
        rear = (rear + 1) % MAX;
    }

    queue[rear] = value;
}
```

20. DEQUEUE

Dequeue removes:

TEXT

```
queue[front]
```

If only one element:

TEXT

```
front == rear
```

after removal:

TEXT

```
front = rear = -1
```

Otherwise:

TEXT

```
front = (front + 1) % N
```

$\emptyset = \text{front}$ in modulo enables wrap-around.

21. DEQUEUE IMPLEMENTATION

C

```
int dequeue(void)
{
    if (front == -1)
    {
        printf("Queue Underflow\n");
        return -1;
    }

    int value = queue[front];

    if (front == rear)
    {
        front = rear = -1;
    }
    else
    {
        front = (front + 1) % MAX;
    }

    return value;
}
```

22. WRAP-AROUND EXAMPLE $\emptyset = \text{front}$

Capacity:

TEXT

```
5
```

Suppose:

TEXT

```
front = 2
rear = 4
```

Queue occupies:

TEXT

2, 3, 4

Now enqueue one value.

Calculate:

TEXT

```
rear = (4+1)%5
      = 0
```

New rear:

TEXT

0

So logical queue occupies:

TEXT

2 !' 3 !' 4 !' 0

This is perfectly valid.

23. VISUALIZATION

Physical array:

TEXT

```
Index: 0   1   2   3   4
       [50] [ ] [20] [30] [40]
           !'           !'
         R           F
```

Logical order is:

TEXT

2 0 !' 3 0 !' 4 0 !' 5 0

not:

TEXT

5 0 !' 2 0 !' 3 0 !' 4 0

Physical index order "logical queue order after wrap-around."

24. NEXT ENQUEUE

Current:

TEXT

F=2
R=0

Check full:

TEXT

$(R+1)\%5$
= 1

Not equal to F=2.

So enqueue at index 1.

Now:

TEXT

F=2
R=1

Physical:

TEXT

Index:	0	1	2	3	4
	[50]	[60]	[20]	[30]	[40]
				!	!
		R	F		

Logical order:

TEXT

20 30 40 50 60

Queue full.

Check:

TEXT

$(R+1)\%5$
= $(1+1)\%5$
= 2
= F

~~Queue~~ full.

25. FULL DOES NOT REQUIRE `rear > front`

Here:

TEXT

```
front = 2  
rear = 1
```

Queue is full.

So in circular queue:

TEXT

```
rear < front
```

can be completely valid.

Ø=No GATE trap.

26. DEQUEUE AFTER WRAP

From full queue:

TEXT

```
logical:  
20 30 40 50 60
```

F=2.

Dequeue removes:

TEXT

```
20
```

Then:

TEXT

```
front = (2+1)%5 = 3
```

Now:

TEXT

```
F=3  
R=1
```

Logical queue:

TEXT

```
30 40 50 60
```

27. CONTINUE DEQUEUE

Remove 30:

TEXT

```
front = 4
```

Logical:

TEXT

```
40 50 60
```

Remove 40:

TEXT

```
front = (4+1)%5  
      = 0
```

Logical:

TEXT

```
50 60
```

front wraps too.

28. BOTH FRONT AND REAR ARE CIRCULAR

Rear movement:

TEXT

```
rear = (rear+1)%N
```

Front movement:

TEXT

```
front = (front+1)%N
```

Both can wrap.

29. FULL CONDITION DERIVATION

If moving rear one step would land on front:

TEXT

```
nextRear == front
```

then there is no free slot remaining.

Since:

TEXT

```
nextRear = (rear+1)%N
```

full condition:

FORMULA BOX

TEXT

```
(rear+1)%N == front
```

for this last-occupied/ -1 empty representation.

30. EMPTY CONDITION

Under our main representation:

TEXT

```
front == -1
```

and correspondingly:

TEXT

```
rear == -1
```

Queue empty.

31. ONE ELEMENT CONDITION

If:

TEXT

```
front == rear
```

and not -1:

TEXT

```
one element
```

Example:

TEXT

```
front=3  
rear=3
```

one element exists at index 3.

One as linear queue in this representation.

32. SIZE OF CIRCULAR QUEUE

This is a major GATE formula.

If empty:

TEXT

```
size = 0
```

If:

TEXT

```
rear >= front
```

then:

TEXT

```
size = rear - front + 1
```

If wrapped:

TEXT

```
rear < front
```

then:

TEXT

```
size = N - front + rear + 1
```

33. UNIFIED SIZE FORMULA

For non-empty queue:

FORMULA BOX

TEXT

```
size = ((rear - front + N) % N) + 1
```

Example:

TEXT

```
N=5  
front=3  
rear=1
```

Then:

TEXT

```
((1-3+5)%5)+1  
= (3%5)+1  
=4
```

Logical elements = 4.

0-4%5
Excellent GATE formula.

34. CHECK NON-WRAPPED SIZE

TEXT

```
N=10
front=2
rear=5
```

Formula:

TEXT

```
((5-2+10)%10)+1
=13%10+1
=3+1
=4
```

Correct.

35. CHECK FULL SIZE

TEXT

```
N=5
front=2
rear=1
```

Formula:

TEXT

```
((1-2+5)%5)+1
=4%5+1
=5
```

Full capacity.

Correct for our chosen representation.

36. ALTERNATIVE COUNT-BASED REPRESENTATION $\emptyset = Y\%$

Another very clean design maintains:

C

```
int count = 0;
```

Then:

TEXT

```
empty !' count == 0
full !' count == N
```

All N slots can be used.

No ambiguity.

Enqueue:

```
TEXT
rear = (rear+1)%N
count++
```

Dequeue:

```
TEXT
front = (front+1)%N
count--
```

37. WHY COUNT IS USEFUL

Without count, front/rear relationships must encode both:

```
TEXT
position
+
state
```

With count:

```
TEXT
front / rear !' position
count !' number of elements
```

This simplifies full/empty distinction.

Trade-off:

```
TEXT
extra integer metadata
```

38. ONE-SLOT-EMPTY VERSION

Some textbooks instead initialize:

```
TEXT
front = rear = 0
```

and define:

```
TEXT
empty:
front == rear
```

Enqueue places at rear and advances it.

Full:

TEXT

```
(rear+1)%N == front
```

Now one slot must remain unused.

Thus:

TEXT

```
usable capacity = N-1
```

Memorize this **with its convention**, not alone.

39. GATE EXAM TRAP

Question:

Circular queue array size = 10. Full condition is $(rear+1)\%10 == front$. What is maximum number of elements?

You **cannot answer confidently from the full condition alone** without knowing what rear/front mean and the empty convention.

If the question also states:

TEXT

```
empty when front == rear
```

then:

TEXT

```
maximum = 9
```

If empty is separately encoded as `-1` and rear denotes last occupied index, maximum may be 10.

Representation details matter.

40. COMPLETE C IMPLEMENTATION

Using our main representation:

```
C
#include <stdio.h>

#define MAX 5

int queue[MAX];
int front = -1;
int rear = -1;

int isEmpty(void)
{
    return front == -1;
}
```

```

int isFull(void)
{
    return !isEmpty() &&
           (rear + 1) % MAX == front;
}

void enqueue(int value)
{
    if (isFull())
    {
        printf("Queue Overflow\n");
        return;
    }

    if (isEmpty())
    {
        front = rear = 0;
    }
    else
    {
        rear = (rear + 1) % MAX;
    }

    queue[rear] = value;
}

int dequeue(void)
{
    if (isEmpty())
    {
        printf("Queue Underflow\n");
        return -1;
    }

    int value = queue[front];

    if (front == rear)
    {
        front = rear = -1;
    }
    else
    {
        front = (front + 1) % MAX;
    }

    return value;
}

```

41. DISPLAY CIRCULAR QUEUE

A normal loop:

```

c
for(i=front; i<=rear; i++)

```

fails when:

TEXT

```
rear < front
```

after wrap-around.

Instead:

```
C
void display(void)
{
    if (front == -1)
        return;

    int i = front;

    while (1)
    {
        printf("%d ", queue[i]);

        if (i == rear)
            break;

        i = (i + 1) % MAX;
    }
}
```

0-100% follows logical queue order.

42. GATE TRACE 1

Capacity:

TEXT

```
N = 5
```

Start:

TEXT

```
F=-1 R=-1
```

Operations:

TEXT

```
enqueue 10
enqueue 20
enqueue 30
dequeue
dequeue
enqueue 40
enqueue 50
enqueue 60
enqueue 70
```

Let's trace.

43. FIRST THREE

After:

TEXT

10 20 30

state:

TEXT

F=0

R=2

Physical:

TEXT

[10][20][30][][]

44. TWO DEQUEUES

Deque 10:

TEXT

F=1

Deque 20:

TEXT

F=2

Logical:

TEXT

30

State:

TEXT

F=2

R=2

45. ENQUEUE 40

TEXT

$R = (2 + 1) \% 5 = 3$

Logical:

TEXT

30 40

46. ENQUEUE 50

TEXT

R=4

Logical:

TEXT

30 40 50

47. ENQUEUE 60

TEXT

$R = (4+1) \% 5 = 0$

Logical:

TEXT

30 40 50 60

Physical:

TEXT

[60][20][30][40][50]
! ' ! '
R F

48. ENQUEUE 70

Check:

TEXT

$\text{nextRear} = (0+1) \% 5 = 1$

front =2.

Not full yet.

So:

TEXT

R=1
queue[1]=70

Logical:

```
TEXT
30 40 50 60 70
```

Now:

```
TEXT
F=2
R=1
```

Queue full.

49. NEXT ENQUEUE?

Check:

```
TEXT
(R+1)%5
=2
=F
```

So:

```
TEXT
Overflow
```

Do not do exact wrap-around trace.

50. GATE TRACE 2

Suppose:

```
TEXT
N=8
front=6
rear=2
```

Queue is non-empty.

How many elements?

Use:

```
TEXT
size = ((rear-front+N)%N)+1
```

```
TEXT
=((2-6+8)%8)+1
=(4%8)+1
=5
```

Logical occupied indices:

TEXT

6, 7, 0, 1, 2

~~0 = 10%~~ Answer = 5.

51. GATE TRACE 3

Capacity:

TEXT

N=6

State:

TEXT

front=4
rear=1

Logical indices:

TEXT

4, 5, 0, 1

Size:

TEXT

4

Next enqueue index:

TEXT

$(\text{rear}+1)\%6$
=2

If 2 != front 4:

TEXT

enqueue allowed

52. GATE TRAP — rear < front

This does **not** mean:

TEXT

invalid queue

It usually indicates:

wrapped state

Example:

```
TEXT
front=4
rear=1
```

perfectly valid.

53. GATE TRAP — `front > rear`

Same.

In linear queue:

```
TEXT
front > rear
```

may indicate empty depending on convention.

In circular queue:

```
TEXT
front > rear
```

can mean active elements cross the array boundary.

~~0=Never~~ Never use linear reasoning for circular queues.

54. GATE TRAP — MODULO PRECEDENCE

Correct:

```
C
rear = (rear + 1) % MAX;
```

Not conceptually:

```
TEXT
rear + (1 % MAX)
```

Parentheses make intent obvious.

55. GATE TRAP — NEGATIVE MODULO

Unified size formula uses:

TEXT

```
rear - front + N
```

before $\% N$.

Why add N?

Because:

TEXT

```
rear-front
```

may be negative.

Adding N normalizes it into nonnegative circular distance.

~~0-500~~ Some normalization idea as lower-bound array addressing.

56. CIRCULAR DISTANCE INTUITION

If:

TEXT

```
front=4  
rear=1  
N=6
```

Forward path:

TEXT

```
4 !' 5 !' 0 !' 1
```

Number of edges:

TEXT

```
3
```

Number of elements:

TEXT

```
edges + 1 = 4
```

Formula:

TEXT

```
((1-4+6)%6)+1  
=3+1  
=4
```

57. CIRCULAR QUEUE VS CIRCULAR LINKED LIST

Both use circularity, but differently.

Circular queue:

TEXT

array indices wrap with modulo

Circular linked list:

TEXT

last node's next points to first node

Same conceptual cycle, different implementation.

58. WHY CIRCULAR QUEUE IS GOOD FOR BUFFERS

Suppose fixed buffer size:

TEXT

N

Producer writes new data.

Consumer removes old data.

As old slots become free, producer can reuse them by wrapping.

Thus circular queue is ideal for:

- streaming buffers
- device buffers
- producer-consumer systems
- scheduling

59. ROUND ROBIN CONNECTION

Processes:

TEXT

P1 P2 P3

After P1 gets CPU time, if unfinished:

TEXT

enqueue P1 at rear

Queue cycles:

TEXT

P2 P3 P1 ...

Circular queue behavior fits this naturally.

60. COMPLEXITY

Circular queue:

Operation	Time
Enqueue	$O(1)$
Dequeue	$O(1)$
Front	$O(1)$
Rear	$O(1)$
isEmpty	$O(1)$
isFull	$O(1)$
Display	$O(n)$

Space:

TEXT
 $O(N)$

61. WHY CIRCULAR QUEUE IS BETTER THAN SHIFTING

Linear queue with shifting:

TEXT
dequeue = $O(n)$

Circular queue:

TEXT
dequeue = $O(1)$

because only:

TEXT
front

changes.

No element movement required.

62. MEMORY TRICKS

Circular queue

When REAR reaches the wall, wrap to zero.

Index movement

MOVE + MOD

TEXT
 $(i+1)\%N$

Full

If REAR's next step hits FRONT, stop.

Empty

For our implementation:

TEXT
`front = rear = -1`

One element

TEXT
`front == rear != -1`

63. NOTEBOOK NOTES

Move front/rear

TEXT
`next = (index+1)%N`

Full

For our chosen last-occupied/ `-1` representation:

TEXT
`(rear+1)%N == front`

Empty

TEXT
`front == -1`

One element

TEXT

```
front == rear != -1
```

64. SIZE FORMULA

Non-empty:

TEXT

```
size = ((rear-front+N)%N)+1
```

Examples:

TEXT

```
F=2, R=4, N=5  
size=3
```

TEXT

```
F=4, R=1, N=5  
size=3
```

(indices 4,0,1).

65. REPRESENTATION WARNING Ø=Ý%

Do not write in your notes:

Circular queue of size N always stores N-1 elements.

Instead write:

Some implementations intentionally leave one slot empty, giving usable capacity N-1. Other implementations use a count or explicit empty state and can use all N slots.

Ø=Ý% is the correct GATE-safe statement.

66. DON'T CONFUSE

Linear queue

TEXT

```
r e a r   r e a c h e s   N - 1   ! '   c a n n o t   w r a p
```

Circular queue

TEXT

```
r e a r   r e a c h e s   N - 1 ! '   n e x t   m a y   b e   0
```

Full

Not simply:

TEXT

```
rear == N-1
```

Wrapped

TEXT

```
rear < front
```

can be valid.

67. 30-SECOND REVISION

Remember:

1. Circular queue reuses freed array slots.
2. Wrap with $(\text{index}+1)\%N$.
3. Both front and rear can wrap.
4. `rear < front` can be valid.
5. Full condition depends on representation.
6. A common full condition is `(rear+1)%N == front`.
7. Empty can be encoded as `front=rear=-1`.
8. One-slot-empty designs store only `N-1`.
9. Count-based designs can store all `N`.
10. Enqueue/dequeue remain $O(1)$.

68. ACTIVE RECALL

Q1

For `N=5`, what comes after index 4?

Q2

What does:

TEXT

```
(index+1)%N
```

provide?

Q3

Can `rear < front` be valid?

Q4

Why might a circular queue of array size N store only N-1 elements?

Q5

What is size for:

```
TEXT
N=7
front=5
rear=1
```

using our main representation?

ANSWERS

A1

```
TEXT
0
```

A2

Next circular index.

A3

Yes. It indicates wrap-around.

A4

If the chosen representation deliberately leaves one slot empty to distinguish full from empty.

A5

```
TEXT
((1-5+7)%7)+1
=3+1
=4
```

Indices:

```
TEXT
5, 6, 0, 1
```

69. PRACTICE — MODULO

For:

```
TEXT
N=8
```

find next index after:

A

3

B

7

C

0

Use:

TEXT

$(i+1)\%8$

70. PRACTICE — SIZE

Find circular queue size:

A

TEXT

N=10

F=2

R=6

B

TEXT

N=10

F=8

R=1

C

TEXT

N=6

F=4

R=3

under our main full-capacity representation.

71. PRACTICE — TRACE

Capacity:

TEXT

N=5

Start empty.

Perform:

TEXT

```
enqueue 1
enqueue 2
enqueue 3
dequeue
dequeue
enqueue 4
enqueue 5
enqueue 6
enqueue 7
```

Find:

- final **front**
- final **rear**
- logical order
- whether another enqueue succeeds

72. GATE-STYLE QUESTION

Circular queue:

TEXT

```
N=8
front=6
rear=3
```

How many active elements?

Formula:

TEXT

```
((3-6+8)%8)+1
=5+1
=6
```

Logical indices:

TEXT

```
6, 7, 0, 1, 2, 3
```

Answer:

6

73. GATE-STYLE TRAP

A student says:

front > rear , therefore queue is empty.

Wrong for circular queue.

Example:

```
TEXT
front=6
rear=2
```

could represent:

```
TEXT
6, 7, 0, 1, 2
```

five valid elements.

~~0 = 10~~ ways think circularly.

PROGRESS

MODULE 5 — QUEUE

' Lesson 44 — Queue Fundamentals ' Lesson 45 — Linear Queue Using Array ' **Lesson 46 — Circular Queue**

NEXT — LESSON 47

Queue Using Linked List — front, rear, O(1) Enqueue/Dequeue & Pointer Updates

Next we implement:

```
TEXT
front      rear
! "
1 0  !'   2 0  !'   3 0  !'   4 0  !'   N U L L   ! "
```

and master:

- front + rear
- enqueue at tail
- dequeue at head
- first-node case
- last-node deletion
- why both pointers become NULL
- malloc failure
- memory leaks/dangling pointers
- exact O(1) complexity
- array queue vs linked queue
- GATE pointer traces

Then:

TEXT
front

and insert at:

TEXT
rear

So:

TEXT
E N Q U E U E ! ' r e a r
D E Q U E U E ! ' f r o n t

2. WHY MAINTAIN TWO POINTERS?

Suppose we maintain only:

TEXT
front

Example:

TEXT
front
!"
1 0 ! ' 2 0 ! ' 3 0 ! ' N U L L

To enqueue 40 at the end, we would have to traverse:

TEXT
1 0 ! ' 2 0 ! ' 3 0

to find the last node.

That costs:

TEXT
 $O(n)$

But if we also maintain:

TEXT
r e a r ! ' 3 0

then appending 40 is immediate.

So:

****front gives $O(1)$ dequeue**

rear gives $O(1)$ enqueue**

$O(1)$ is the key design idea.

3. NODE STRUCTURE

```
c
typedef struct Node
{
    int data;
    struct Node *next;
} Node;
```

Queue pointers:

```
c
Node *front = NULL;
Node *rear = NULL;
```

4. EMPTY QUEUE

Empty queue:

```
TEXT
front = NULL
rear  = NULL
```

There are no nodes.

$O(1)$ important invariant:

```
TEXT
empty queue:
front == NULL
rear  == NULL
```

5. ONE-ELEMENT QUEUE

Suppose we enqueue 10 into an empty queue.

Then:

```
TEXT
front
! "
1 0 !'  N U L L
! '
rear
```

Therefore:

TEXT

```
front == rear
```

when exactly one node exists.

This is similar to maintaining `head` and `tail` in linked lists.

6. ENQUEUE — BASIC IDEA

Suppose queue:

TEXT

```
front      rear
! "
1 0 !' 2 0 !' 3 0 !' N U L L ! "
```

Enqueue:

TEXT

```
4 0
```

Create:

TEXT

```
newNode
! "
4 0 !' N U L L
```

Then:

C

```
rear->next = newNode;
rear = newNode;
```

Final:

TEXT

```
front      rear
! "
1 0 !' 2 0 !' 3 0 !' 4 0 !' N U L L ! "
```

~~O(1)~~ No traversal required.

7. ENQUEUE — BEFORE !' POINTER CHANGES !' AFTER

Before:

TEXT

```
front      rear
! "
1 0 !' 2 0 !' 3 0 !' N U L L
```

```
n e w ! ' 4 0 ! ' N U L L
```

First:

```
c
rear->next = newNode;
```

Now:

```
TEXT
1 0 ! ' 2 0 ! ' 3 0 ! ' 4 0 ! ' N U L L
                        ! '
                    old rear
```

Then:

```
c
rear = newNode;
```

Result:

```
TEXT
front                rear
! "                  ! "
1 0 ! ' 2 0 ! ' 3 0 ! ' 4 0 ! ' N U L L
```

8. ENQUEUE INTO EMPTY QUEUE Ø=Ý%

This is the special case.

Initially:

```
TEXT
front = NULL
rear = NULL
```

Create:

```
TEXT
n e w ! ' 1 0 ! ' N U L L
```

Since it is the only node:

```
c
front = newNode;
rear = newNode;
```

Result:

TEXT

```
front
!"
1 0 !" NULL
!'
rear
```

Both pointers must point to the first node.

9. FULL ENQUEUE FUNCTION

C

```
int enqueue(Node **front, Node **rear, int value)
{
    Node *newNode = malloc(sizeof *newNode);

    if (newNode == NULL)
        return 0;

    newNode->data = value;
    newNode->next = NULL;

    if (*rear == NULL)
    {
        *front = newNode;
        *rear = newNode;
    }
    else
    {
        (*rear)->next = newNode;
        *rear = newNode;
    }

    return 1;
}
```

10. WHY CHECK `rear == NULL`?

For a valid queue invariant:

TEXT

```
rear == NULL
```

means queue is empty.

Because if there is at least one node:

TEXT

```
rear
```

must point to the last node.

So first enqueue needs special initialization.

11. ALTERNATIVE CHECK

Could check:

```
C
if (*front == NULL)
```

instead.

If queue invariants are correct:

```
TEXT
front == NULL !& rear == NULL
```

for empty queue.

Either works.

12. ENQUEUE COMPLEXITY

Operations:

- allocate one node
- set fields
- change constant number of pointers

No traversal.

Therefore:

FORMULA BOX

```
TEXT
Enqueue:
Time =  $O(1)$ 
```

Auxiliary pointer variables:

```
TEXT
 $O(1)$ 
```

The newly allocated node is part of the queue itself.

13. DEQUEUE — BASIC IDEA

Suppose:

```
TEXT
front      rear
! "
1 0 !' 2 0 !' 3 0 !' 4 0 !' N U L L ! "
```

FIFO requires removing:

```
TEXT
10
```

We already know the first node from:

```
TEXT
front
```

So no traversal.

14. SAFE DEQUEUE RULE

Recall linked-list deletion:

SAVE !' MOVE !' FREE

Use exactly the same rule.

Before:

```
TEXT
front
!"
1 0  !'  2 0  !'  3 0
```

Save:

```
C
Node *temp = front;
```

Move:

```
C
front = front->next;
```

Then:

```
C
free(temp);
```

15. VISUAL TRACE

Before:

```
TEXT
front                rear
!"
1 0  !'  2 0  !'  3 0  !'  N U L L      !"

```

Save:

TEXT

```
temp !' 10
front !' 10
```

Move:

TEXT

```
temp !' 10
front !' 20 !' 30
```

Save value:

TEXT

```
value = 10
```

Free:

TEXT

```
free(temp)
```

Final:

TEXT

```
front      rear
!"
20 !' 30 !' NULL      !"
```

16. DEQUEUE FUNCTION

C

```
int dequeue(Node **front, Node **rear, int *value)
{
    if (*front == NULL)
        return 0;

    Node *temp = *front;

    *value = temp->data;
    *front = temp->next;

    if (*front == NULL)
        *rear = NULL;

    free(temp);

    return 1;
}
```

Ø=The line:

```
C
if (*front == NULL)
    *rear = NULL;
```

is extremely important.

17. WHY MUST REAR BECOME NULL?

Suppose queue has one node:

```
TEXT
front
! "
1 0 ! ' N U L L
! '
rear
```

After:

```
C
front = front->next;
```

we get:

```
TEXT
front = NULL
```

But `rear` still points to old node 10.

Then we free node 10.

If we don't change rear:

```
TEXT
rear
```

becomes a:

dangling pointer

Ø= serious bug.

Correct final state:

```
TEXT
front = NULL
rear = NULL
```

18. ONE-ELEMENT DEQUEUE TRACE

Before:

TEXT

```
front = rear = address of node 10
```

Step 1:

C

```
temp = front;
```

Step 2:

C

```
front = front->next;
```

Since:

TEXT

```
10->next = NULL
```

now:

TEXT

```
front = NULL
```

Therefore:

C

```
rear = NULL;
```

Then:

C

```
free(temp);
```

Final:

TEXT

```
front = NULL  
rear  = NULL
```

Correct.

19. DEQUEUE COMPLEXITY

No search or traversal.

Therefore:

TEXT

```
Dequeue =  $O(1)$ 
```

~~$O(1)$~~ This is why linked queue maintains front directly.

20. PEEK / FRONT VALUE

To read the next element without removing it:

```
C
int peek(Node *front, int *value)
{
    if (front == NULL)
        return 0;

    *value = front->data;
    return 1;
}
```

Time:

```
TEXT
O(1)
```

21. REAR VALUE

If rear pointer is maintained:

```
C
rear->data
```

gives newest queue element in:

```
TEXT
O(1)
```

assuming queue non-empty.

22. DISPLAY

```
C
void display(Node *front)
{
    Node *temp = front;

    while (temp != NULL)
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }
}
```

For:

TEXT

f r o n t ! ' 1 0 ! ' 2 0 ! ' 3 0 ! ' N U L L

output:

TEXT

10 20 30

This is queue order.

Time:

TEXT

O(n)

23. COMPLETE LINKED QUEUE PROGRAM

```
C
#include <stdio.h>
#include <stdlib.h>

typedef struct Node
{
    int data;
    struct Node *next;
} Node;

int enqueue(Node **front, Node **rear, int value)
{
    Node *newNode = malloc(sizeof *newNode);

    if (newNode == NULL)
        return 0;

    newNode->data = value;
    newNode->next = NULL;

    if (*rear == NULL)
    {
        *front = newNode;
        *rear = newNode;
    }
    else
    {
        (*rear)->next = newNode;
        *rear = newNode;
    }

    return 1;
}

int dequeue(Node **front, Node **rear, int *value)
{
    if (*front == NULL)
        return 0;

    Node *temp = *front;
```

```

    *value = temp->data;
    *front = temp->next;

    if (*front == NULL)
        *rear = NULL;

    free(temp);

    return 1;
}

int peek(Node *front, int *value)
{
    if (front == NULL)
        return 0;

    *value = front->data;
    return 1;
}

```

24. OPERATION TRACE

Start:

```

TEXT
front = NULL
rear = NULL

```

Enqueue 10:

```

TEXT
front
!"
1 0  !"  N U L L
!'
rear

```

Enqueue 20:

```

TEXT
front      rear
!"
1 0  !"  2 0  !"  N U L L

```

Enqueue 30:

```

TEXT
front      rear
!"
1 0  !"  2 0  !"  3 0  !"  N U L L

```

25. DEQUEUE

Dequeue returns:

TEXT

10

Remaining:

TEXT

```
front      rear
! "        ! "
2 0 !'    3 0 !'    N U L L
```

Next dequeue:

TEXT

20

Remaining:

TEXT

```
front
! "
3 0 !'    N U L L
! '
rear
```

Next dequeue:

TEXT

30

Final:

TEXT

```
front = NULL
rear = NULL
```

Ø=71% clearly visible.

26. ARRAY QUEUE VS LINKED QUEUE

Property	Array Queue	Linked Queue
Storage	contiguous	non-contiguous
Enqueue	O(1)	O(1)
Dequeue	O(1)	O(1)
Fixed capacity	often yes	no fixed count
Circular logic needed	yes for efficient reuse	no
Pointer overhead	no per element	one next per node

Property	Array Queue	Linked Queue
Allocation	not per operation	malloc per enqueue
Cache locality	usually better	usually weaker

27. DOES LINKED QUEUE HAVE OVERFLOW?

Do not say:

Linked queue never overflows.

Memory is finite.

If:

```
C
malloc(...)
```

returns:

```
TEXT
NULL
```

enqueue fails.

So there is no fixed-array full condition, but allocation failure can prevent growth.

Ø=Some nuance as linked stack.

28. UNDERFLOW

If:

```
TEXT
front == NULL
```

and we attempt dequeue:

```
TEXT
QUEUE UNDERFLOW
```

No node exists to remove.

29. MEMORY LEAK TRAP Ø=Ý%

Wrong dequeue:

```
c
front = front->next;
```

without saving/freeing old front.

The old node becomes unreachable.

Result:

Memory leak

Correct:

```
c
temp = front;
front = front->next;
free(temp);
```

30. USE-AFTER-FREE TRAP

Wrong:

```
c
free(front);
front = front->next;
```

After:

```
c
free(front);
```

you must not dereference:

```
TEXT
front->next
```

because front points to freed memory.

Correct:

```
c
temp = front;
front = temp->next;
free(temp);
```

~~0=SAVE~~ !' MOVE !' FREE

31. DANGLING REAR TRAP

Single node:

TEXT

```
front = rear = X
```

If dequeue does:

C

```
front = NULL;  
free(X);
```

but leaves:

TEXT

```
rear = X
```

then rear is dangling.

Correct:

TEXT

```
front = NULL  
rear = NULL
```

32. GATE POINTER TRACE

Initial:

TEXT

```
front !' A !' B !' C !' N U L L  
      !'  
      rear
```

Execute:

C

```
Node *p = front;  
front = front->next;  
free(p);
```

Assuming more than one node, final:

TEXT

```
front !' B !' C !' N U L L  
      !'  
      rear
```

This is:

DEQUEUE

33. GATE POINTER TRACE — ENQUEUE

Initial:

TEXT

```
f r o n t ! '   A ! '   B ! '   N U L L
                ! '
                r e a r
```

New node:

TEXT

```
p ! '   C ! '   N U L L
```

Execute:

C

```
rear->next = p;
rear = p;
```

Final:

TEXT

```
f r o n t ! '   A ! '   B ! '   C ! '   N U L L
                ! '
                r e a r
```

This is:

ENQUEUE

34. GATE TRAP — WHAT IF ONLY FRONT IS MAINTAINED?

Queue:

TEXT

```
f r o n t ! '   A ! '   B ! '   C ! '   N U L L
```

Dequeue:

TEXT

0(1)

Enqueue at end:

TEXT

0(n)

because last node must be found.

So a proper linked queue normally maintains:

TEXT

front + rear

35. WHAT IF ONLY REAR IS MAINTAINED?

In a normal singly linked list:

TEXT

rear

alone doesn't directly give front.

So ordinary dequeue becomes difficult.

However a **circular singly linked list** can maintain only tail/rear because:

TEXT

front = rear->next

Remember Circular Linked List?

Ø=No connection.

36. QUEUE USING CIRCULAR LINKED LIST Ø=No

Represent:

TEXT

rear

!"

1 0 !' 2 0 !' 3 0

!' "

!"

% % % % % % % % %

Then:

TEXT

front = rear->next

So with only rear :

- front access O(1)
- rear access O(1)
- enqueue O(1)
- dequeue O(1)

This is a beautiful implementation.

Ø=No placement/GATE extension.

37. ENQUEUE IN CIRCULAR LINKED QUEUE

For non-empty:

```
C
newNode->next = rear->next;
rear->next = newNode;
rear = newNode;
```

Remember:

```
TEXT
rear->next = front
```

38. DEQUEUE IN CIRCULAR LINKED QUEUE

Front:

```
TEXT
rear->next
```

Save:

```
C
temp = rear->next;
```

If one node:

```
TEXT
rear->next == rear
```

then delete and:

```
TEXT
rear = NULL
```

Otherwise:

```
C
rear->next = temp->next;
free(temp);
```

All $O(1)$.

Excellent connection to Lesson 35.

39. SIZE OF LINKED QUEUE

If no size metadata exists:

TEXT

count nodes from front

cost:

TEXT

$O(n)$

If queue stores:

C

```
int size;
```

and updates on enqueue/dequeue:

TEXT

```
enqueue !' size++  
dequeue !' size--
```

then:

TEXT

size query = $O(1)$

40. QUEUE WRAPPER STRUCTURE

A cleaner design:

C

```
typedef struct  
{  
    Node *front;  
    Node *rear;  
    int size;  
} Queue;
```

Initialize:

C

```
Queue q;  
  
q.front = NULL;  
q.rear = NULL;  
q.size = 0;
```

41. ENQUEUE WITH WRAPPER

```

c
int enqueue(Queue *q, int value)
{
    Node *newNode = malloc(sizeof *newNode);

    if (newNode == NULL)
        return 0;

    newNode->data = value;
    newNode->next = NULL;

    if (q->rear == NULL)
    {
        q->front = newNode;
        q->rear = newNode;
    }
    else
    {
        q->rear->next = newNode;
        q->rear = newNode;
    }

    q->size++;
    return 1;
}

```

42. DEQUEUE WITH WRAPPER

```

c
int dequeue(Queue *q, int *value)
{
    if (q->front == NULL)
        return 0;

    Node *temp = q->front;

    *value = temp->data;
    q->front = temp->next;

    if (q->front == NULL)
        q->rear = NULL;

    free(temp);
    q->size--;

    return 1;
}

```

Clean and reusable.

43. QUEUE INVARIANTS Ø=Ý%

For a valid linked queue:

Empty

TEXT

```
front == NULL
rear == NULL
```

Non-empty

TEXT

```
front != NULL
rear != NULL
```

Rear node

TEXT

```
rear->next == NULL
```

One node

TEXT

```
front == rear
```

These properties help solve GATE pointer questions.

44. GATE TRAP — rear->next

For a normal linked queue:

TEXT

```
rear->next == NULL
```

But for a circular linked queue:

TEXT

```
rear->next == front
```

Main representation matters.

45. OPERATION COMPLEXITY TABLE

For linked queue maintaining both front and rear:

Operation	Time
Enqueue	O(1)
Dequeue	O(1)
Peek front	O(1)
Peek rear	O(1)

Operation	Time
isEmpty	$O(1)$
Size if stored	$O(1)$
Search	$O(n)$
Display	$O(n)$

46. WHY SEARCH REMAINS $O(n)$

Even though we have front and rear, they only tell us the ends.

To find arbitrary:

```
TEXT
value = 75
```

we may need traverse:

```
TEXT
front !' . . . !' rear
```

So:

```
TEXT
 $O(n)$ 
```

47. ARRAY VS LINKED — WHICH IS BETTER?

Neither is always better.

Array queue

Good when:

- maximum capacity known
- cache locality important
- allocation overhead should be avoided

Linked queue

Good when:

- dynamic growth useful
- maximum size unknown
- pointer overhead acceptable

~~$O(1)$~~ ~~$O(n)$~~ No structure choice depends on constraints.

48. QUEUE AND BFS

In BFS, queue may contain vertices waiting to be processed.

Conceptually:

TEXT

```
front !' next vertex to explore  
rear  !' newest discovered vertex
```

When new neighbor discovered:

TEXT

```
enqueue rear
```

When processing:

TEXT

```
dequeue front
```

This is why queue implementation matters for graph traversal.

49. QUEUE AND LEVEL ORDER

For tree:

TEXT

```
  A  
 / \  
B   C
```

Start queue:

TEXT

```
[A]
```

Dequeue A.

Enqueue B,C:

TEXT

```
[B, C]
```

Dequeue B, then C.

FIFO preserves tree levels.

50. MEMORY TRICKS

Linked queue

FRONT serves, REAR receives.

Enqueue

Old rear connects new; new becomes rear.

TEXT

```
rear->next = new  
rear = new
```

Dequeue

SAVE FRONT !' MOVE FRONT !' FREE

Last dequeue

If FRONT disappears, REAR must disappear too.

Ø = 0% at memory rule.

51. NOTEBOOK NOTES

Structure

```
c  
struct Node  
{  
    int data;  
    struct Node *next;  
};
```

Queue:

TEXT

```
f r o n t   !'   f i r s t   n o d e  
r e a r   !'   l a s t   n o d e
```

52. ENQUEUE

Non-empty:

```
c  
rear->next = newNode;  
rear = newNode;
```

Empty:

```
c
front = rear = newNode;
```

Time:

```
TEXT
O(1)
```

53. DEQUEUE

```
c
temp = front;
front = front->next;

if (front == NULL)
    rear = NULL;

free(temp);
```

Time:

```
TEXT
O(1)
```

54. DON'T CONFUSE

Linked stack

```
TEXT
top = head
```

Push/pop at same end.

Linked queue

```
TEXT
front = head
rear = tail
```

Insert rear, delete front.

55. ARRAY STACK VS LINKED STACK VS LINKED QUEUE

```
TEXT
STACK:
top !' newest
```

TEXT

QUEUE:

```
f r o n t  ! '   o l d e s t
r e a r    ! '   n e w e s t
```

0-1% relationship is worth memorizing.

56. 30-SECOND REVISION

Remember:

1. Linked queue uses a singly linked list.
2. Maintain both front and rear.
3. Empty: both NULL.
4. One node: front == rear.
5. Enqueue occurs at rear.
6. Dequeue occurs at front.
7. Enqueue = $O(1)$.
8. Dequeue = $O(1)$.
9. Last dequeue must set rear=NULL.
10. rear->next=NULL in normal linked queue.

57. ACTIVE RECALL

Q1

Why maintain rear?

Q2

What does front point to?

Q3

What happens on first enqueue?

Q4

What happens to rear when the final node is dequeued?

Q5

What are enqueue/dequeue complexities?

ANSWERS

A1

To append in $O(1)$ instead of traversing.

A2

The oldest/next-to-be-dequeued node.

A3

Both front and rear point to the new node.

A4

Rear becomes NULL.

A5

TEXT

0(1), 0(1)

58. PRACTICE — EASY

Start:

TEXT

front = rear = NULL

Perform:

TEXT

enqueue 10
enqueue 20
enqueue 30

Draw final pointers.

59. PRACTICE — DEQUEUE

Starting:

TEXT

front !' 5 !' 8 !' 1 2 !' NULL
 !'
 rear

Perform two dequeues.

Find:

- values removed
- final front
- final rear

60. PRACTICE — ONE NODE

Starting:

TEXT

```
front = rear != 50 ? NULL
```

Perform dequeue.

What should final:

TEXT

```
front  
rear
```

be?

61. GATE POINTER TRAP

Suppose queue contains one dynamically allocated node and dequeue does:

C

```
Node *temp = front;  
front = front->next;  
free(temp);
```

but does not update rear.

What is wrong?

Answer

After dequeue:

TEXT

```
front = NULL
```

but:

TEXT

```
rear
```

still holds the address of freed memory.

So rear becomes dangling.

62. GATE COMPLEXITY QUESTION

Queue implemented with singly linked list and only front pointer.

What is enqueue-at-rear complexity?

TEXT

```
O(n)
```

If both front and rear pointers maintained:

TEXT

$O(1)$

Very common reasoning question.

PROGRESS

MODULE 5 — QUEUE

' Lesson 44 — Queue Fundamentals ' Lesson 45 — Linear Queue Using Array ' Lesson 46 — Circular Queue '
Lesson 47 — Queue Using Linked List

NEXT — LESSON 48

Deque & Priority Queue — Double-Ended Operations, Variants & GATE Differences

Next we learn two important queue variants:

Deque

TEXT

`! • insert / delete [ QUEUE ] insert / delete !`

Operations at both ends.

We will cover:

- input-restricted deque
- output-restricted deque
- circular-array deque
- front/rear operations
- $O(1)$ complexities

Then:

Priority Queue

where removal depends on:

TEXT

priority

instead of pure FIFO.

We will connect priority queues to:

Binary Heaps

which we study deeply later.

USER

next

ASSISTANT

MODULE 5 — LESSON 48

Deque & Priority Queue — Double-Ended Queue, Restricted Deques & Priority-Based Removal

We now study two important queue variants:

1. Deque

2. Priority Queue

These are different from a normal FIFO queue.

GATE importance: $\emptyset = \text{NOT IMPORTANT}$ **Placement importance:** $\emptyset = \text{NOT IMPORTANT}$

1. NORMAL QUEUE RECAP

Normal queue:

```
TEXT
FRONT                REAR
! "
1 0 !' 2 0 !' 3 0 !' 4 0
```

Operations:

```
TEXT
e n q u e u e !'   r e a r
d e q u e u e !'   f r o n t
```

Rule:

FIFO

2. WHAT IS A DEQUE?

Deque means:

Double Ended Queue

Pronounced roughly:

TEXT
"deck"

A deque allows operations at:

TEXT
BOTH ENDS

So:

TEXT
! • F R O N T [Q U E U E] R E A R !'

We may:

- insert at front
- insert at rear
- delete from front
- delete from rear

Ø = Yes is the main difference.

3. VISUALIZATION

Suppose deque contains:

TEXT
FRONT REAR
! "
10 20 30 40 ! "

Possible operations:

TEXT
insertFront(5)

Result:

TEXT
5 10 20 30 40

Possible:

TEXT
insertRear(50)

Result:

TEXT
5 10 20 30 40 50

Possible:

```
TEXT
deleteFront()
```

removes 5.

Possible:

```
TEXT
deleteRear()
```

removes 50.

4. DEQUE OPERATIONS

Typical operations:

```
TEXT
insertFront(x)
insertRear(x)

deleteFront()
deleteRear()

getFront()
getRear()

isEmpty()
isFull()
```

A properly designed deque aims for:

```
TEXT
O(1)
```

at both ends.

5. DEQUE CAN BEHAVE LIKE STACK

A stack needs insertion and deletion at the same end.

Using deque rear:

```
TEXT
push !' insertRear
pop !' deleteRear
```

Therefore deque can implement:

STACK

6. DEQUE CAN ALSO BEHAVE LIKE QUEUE

Normal queue:

TEXT

```
enqueue !' insertRear
dequeue !' deleteFront
```

So deque can implement:

QUEUE

Therefore a deque is more flexible than a standard queue.

7. IMPORTANT RELATION

TEXT

Deque

```
% % %   c a n   b e h a v e   l i k e   S t a c k
% % %   c a n   b e h a v e   l i k e   Q u e u e
```

But that does not mean stack and queue are identical.

Deque simply supports a richer set of operations.

8. TYPES OF DEQUE

Two classic restricted variants:

Input-Restricted Deque

Output-Restricted Deque

These names are often asked directly.

9. INPUT-RESTRICTED DEQUE $\emptyset = \text{Y}\%$

"Input restricted" means:

Insertion is restricted to only one end.

But deletion can happen at both ends.

For example:

TEXT

```
Insertion:
rear only
```

Deletion:
front or rear

So:

TEXT

```
i n s e r t R e a r '
i n s e r t F r o n t ' L

d e l e t e F r o n t '
d e l e t e R e a r 'L
```

10. MEMORY TRICK

Input-restricted !' restricted insertion

"Input" means entering/inserting.

Therefore:

TEXT

```
one insertion end
two deletion ends
```

11. OUTPUT-RESTRICTED DEQUE Ø=Ý%

"Output restricted" means:

Deletion is restricted to only one end.

Insertion can happen at both ends.

Example:

TEXT

```
Insertion:
front or rear
```

```
Deletion:
front only
```

Thus:

TEXT

```
i n s e r t F r o n t '
i n s e r t R e a r '

d e l e t e F r o n t '
d e l e t e R e a r 'L
```

12. MEMORY TRICK

Output-restricted !' restricted deletion

"Output" means removing/leaving.

Thus:

TEXT

two insertion ends
one deletion end

13. MASTER TABLE

Type	Insert Front	Insert Rear	Delete Front	Delete Rear
Normal Deque	'	'	'	'
Input-Restricted	'L *	' *	'	'
Output-Restricted	'	'	' *	'L *

* The exact chosen permitted end may be reversed by definition/implementation, but the key property is:

- input-restricted !' insertion only one end
- output-restricted !' deletion only one end

Ø=6%
NOTE safest way to remember.

14. HOW TO IMPLEMENT DEQUE?

Common implementations:

Circular Array

Very common.

Doubly Linked List

Natural because both ends support $O(1)$ updates.

A singly linked list is less convenient for deleting the rear because predecessor access costs $O(n)$.

15. WHY DOUBLY LINKED LIST FITS DEQUE?

DLL:

TEXT

N U L L ! • 1 0 !Ä 2 0 !Ä 3 0 !' N U L L
!'
front rear

Operations:

TEXT

```
insert front = 0(1)
insert rear  = 0(1)
delete front = 0(1)
delete rear  = 0(1)
```

because each node has:

TEXT

```
prev
next
```

0-50% Strong connection to Lesson 34.

16. SINGLY LINKED LIST PROBLEM

Suppose:

TEXT

```
front !' 10 !' 20 !' 30 !' NULL
      !'
      rear
```

Deleting rear requires finding:

TEXT

20

the predecessor of 30.

In singly linked list:

TEXT

$O(n)$

So a singly linked list is not ideal for a full deque unless a more specialized structure is used.

17. CIRCULAR ARRAY DEQUE

Suppose:

TEXT

$N = 5$

Indices:

TEXT

0 1 2 3 4

Both:

TEXT

front
rear

can move circularly.

Rear forward:

TEXT

$(\text{rear} + 1) \% N$

Front forward:

TEXT

$(\text{front} + 1) \% N$

For moving backward we need another formula.

18. PREVIOUS CIRCULAR INDEX $\emptyset = Y\%$

To move backward:

FORMULA BOX

TEXT

$\text{previous} = (\text{index} - 1 + N) \% N$

Why $+N$?

To avoid a negative intermediate index.

Example:

TEXT

$N=5$
 $\text{index}=0$

Then:

TEXT

$(0 - 1 + 5) \% 5$
 $= 4 \% 5$
 $= 4$

So before 0 comes:

TEXT

4

$\emptyset = 4$ Must know for deque.

19. TWO CIRCULAR MOVEMENT FORMULAS

FORMULA BOX

Move forward:

```
TEXT
(i + 1) % N
```

Move backward:

```
TEXT
(i - 1 + N) % N
```

Memorize:

****Forward +1**

Backward -1 + N**

20. INSERT AT REAR

Same circular-queue idea:

```
TEXT
rear = (rear + 1) % N
```

then:

```
TEXT
a[rear] = x
```

provided deque is not full.

21. INSERT AT FRONT

To create space before current front:

```
TEXT
front = (front - 1 + N) % N
```

then:

```
TEXT
a[front] = x
```

Example:

TEXT

```
N=5  
front=0
```

Insert at front:

TEXT

```
front=(0-1+5)%5  
=4
```

New value goes to index 4.

$0 = \text{front}$ wraps backward.

22. DELETE FRONT

Read:

TEXT

```
a[front]
```

then move forward:

TEXT

```
front=(front+1)%N
```

unless removing the last element.

23. DELETE REAR

Read:

TEXT

```
a[rear]
```

then move backward:

TEXT

```
rear=(rear-1+N)%N
```

unless removing the last element.

24. ONE-ELEMENT CASE

If:

TEXT

```
front == rear
```


and deque is non-empty,
there is exactly one element.

After deleting it:

```
TEXT
front = rear = -1
```

under our familiar representation.

25. DEQUE FULL CONDITION

If using the same full-capacity representation from Circular Queue Lesson 46:

```
TEXT
front = first occupied
rear = last occupied
empty = -1
```

then full when:

```
TEXT
(rear + 1) % N == front
```

Equivalent geometric idea:

No free position remains between rear's next position and front.

Again, representation matters.

26. BASIC DEQUE TRACE

Capacity 5.

Start empty.

```
TEXT
insertRear(10)
```

State:

```
TEXT
[10]
```

Then:

```
TEXT
insertRear(20)
```

TEXT
10 20

Then:

TEXT
insertFront(5)

Logical deque:

TEXT
5 10 20

Then:

TEXT
deleteRear()

returns:

TEXT
20

Remaining:

TEXT
5 10

Then:

TEXT
deleteFront()

returns:

TEXT
5

Remaining:

TEXT
10

27. GATE TRAP — DEQUE IS NOT ALWAYS FIFO

Because you may remove from rear.

Example:

TEXT

```
insertRear(10)
insertRear(20)
deleteRear()
```

returns:

TEXT

20

That looks like stack behavior.

So a general deque does not enforce strict FIFO.

28. APPLICATION — SLIDING WINDOW MAXIMUM Ø=Ý%Ø=Ý%Ø=Ý%

One of the most important deque placement problems:

Given:

TEXT

1 3 -1 -3 5 3 6 7

window size:

TEXT

k=3

Find maximum of every consecutive window.

Windows:

TEXT

[1 , 3 , - 1]	!' 3
[3 , - 1 , - 3]	!' 3
[- 1 , - 3 , 5]	!' 5
[- 3 , 5 , 3]	!' 5
[5 , 3 , 6]	!' 6
[3 , 6 , 7]	!' 7

Answer:

TEXT

3 3 5 5 6 7

Optimal solution uses:

Deque of indices

29. WHY DEQUE FOR SLIDING WINDOW?

We need to remove:

1. old indices from the **front**
2. useless smaller values from the **rear**

So both ends are required.

That's exactly what deque provides.

Ø=No pattern.

30. SLIDING WINDOW MAX IDEA

Maintain indices such that corresponding values are decreasing:

TEXT

l a r g e s t ! ' . . . ! ' s m a l l e s t

Then:

TEXT

front of deque

always gives current window maximum.

When a new element arrives:

- remove indices outside window from front
- remove smaller/equal elements from rear
- add current index at rear

31. WHY REMOVE SMALLER FROM REAR?

Suppose current value is 8 and deque's rear corresponds to 5.

Since 8 is:

- newer
- larger

5 can never become maximum before 8 leaves the window.

So 5 is useless.

Pop it from rear.

This is related to monotonic-stack logic, but here we need removals at both ends.

32. COMPLEXITY OF SLIDING WINDOW MAX

Each index:

TEXT

inserted once
removed at most once from each relevant direction

Total:

TEXT

$O(n)$

instead of brute-force:

TEXT

$O(nk)$

or $O(nk)$ -style scanning.

$O=1$ placement favorite.

33. NOW PRIORITY QUEUE

A priority queue is another queue-like ADT, but unlike normal queue:

Removal is based on priority

not simply arrival order.

34. SIMPLE EXAMPLE

Suppose jobs arrive:

TEXT

Job A priority 2
Job B priority 5
Job C priority 1

If larger number means higher priority:

TEXT

Job B

is served first even though it arrived second.

So this is not pure FIFO.

35. PRIORITY QUEUE DEFINITION

Each element has:

TEXT

value
priority

When removing:

TEXT

highest-priority

or:

TEXT

lowest-priority

element is selected depending on the type.

36. TWO MAIN TYPES

Max-Priority Queue

Highest key/priority is removed first.

Example:

TEXT

10, 40, 20

remove:

TEXT

40

first.

Min-Priority Queue

Lowest key/priority is removed first.

Example:

TEXT

10, 40, 20

remove:

TEXT

10

first.

37. CONNECTION TO HEAPS

The most common efficient implementation is:

TEXT

Binary Heap

Max-priority queue

Use:

TEXT

Max Heap

Min-priority queue

Use:

TEXT

Min Heap

We will study binary heaps deeply in:

MODULE 8

So for now, understand the ADT.

38. PRIORITY QUEUE OPERATIONS

Typical operations:

TEXT

```
insert(x)
peekMin / peekMax
extractMin / extractMax
isEmpty
```

Depending on implementation.

39. HEAP-BASED COMPLEXITY PREVIEW

For binary heap:

Operation	Time
Peek highest/lowest priority	$O(1)$
Insert	$O(\log n)$
Extract min/max	$O(\log n)$

~~$O(n)$~~ Must know preview.

We'll derive why later.

40. PRIORITY QUEUE VS NORMAL QUEUE

Property	Normal Queue	Priority Queue
Removal basis	Arrival order	Priority
Rule	FIFO	Highest/lowest priority
Common implementation	Array/list/circular queue	Heap
Typical dequeue	Oldest	Best-priority element

41. WHAT IF TWO ELEMENTS HAVE SAME PRIORITY?

This depends on implementation/specification.

Often a **stable priority queue** preserves FIFO among equal-priority elements.

Example:

TEXT

A priority 5 arrives first
B priority 5 arrives later

A stable priority queue returns:

TEXT

A before B

But stability is not automatically guaranteed by every priority-queue implementation.

Read question carefully.

42. GATE TRAP — PRIORITY "ARRIVAL TIME

Suppose:

TEXT

A priority 1
B priority 9

A arrives first.

If higher number = higher priority:

TEXT

B

may leave first.

So:

TEXT

FIFO

does not control ordinary priority queue removal.

43. GATE TRAP — HIGHER NUMBER MAY NOT MEAN HIGHER PRIORITY

Some systems define:

TEXT
priority 1

as highest.

Others define:

TEXT
larger number

as highest.

Therefore:

Never infer priority direction unless the question defines it.

Ø=Very important.

44. IMPLEMENTING PRIORITY QUEUE WITH UNSORTED ARRAY

Suppose:

TEXT
[4, 10, 2, 8]

If we simply append:

TEXT
insert = $O(1)$

But to remove maximum:

TEXT
search whole array

cost:

TEXT
 $O(n)$

45. SORTED ARRAY IMPLEMENTATION

If array kept sorted:

TEXT

2 4 8 10

maximum/minimum can be found at an end:

TEXT

$O(1)$

But insertion may require shifting:

TEXT

$O(n)$

So there's a trade-off.

46. UNSORTED LINKED LIST

Insertion at head:

TEXT

$O(1)$

But finding best priority:

TEXT

$O(n)$

47. SORTED LINKED LIST

Keep list ordered by priority.

Then:

TEXT

extract highest priority = $O(1)$

if highest is at head.

But inserting in correct position:

TEXT

$O(n)$

48. WHY HEAP IS ATTRACTIVE

Heap balances operations:

TEXT

```
insert  O(log n)
extract O(log n)
peek    O(1)
```

Much better than forcing either insertion or extraction to $O(n)$ in many simple representations.

$O(\log n)$ is the motivation for binary heaps.

49. PRIORITY QUEUE APPLICATION — CPU SCHEDULING

Processes may have priorities.

Example:

TEXT

```
P1 priority 2
P2 priority 5
P3 priority 3
```

Scheduler may select:

TEXT

P2

first.

Real operating systems involve more sophisticated policies, but priority queues are conceptually important.

50. APPLICATION — DIJKSTRA

Shortest-path algorithms often need to repeatedly obtain the vertex with:

TEXT

smallest tentative distance

A min-priority queue is useful.

We'll revisit during Graphs.

51. APPLICATION — PRIM'S MST

Prim's algorithm repeatedly selects a minimum-weight candidate edge/vertex key.

A min-priority queue can make this efficient.

We'll revisit in Graph/Algorithms material.

52. APPLICATION — HUFFMAN CODING

Huffman coding repeatedly selects:

TEXT

two minimum-frequency nodes

So a:

TEXT

min-priority queue

is natural.

Ø=Useful cross-subject connection.

53. APPLICATION — EVENT SIMULATION

Events scheduled at different times:

TEXT

```
time = 2
time = 10
time = 5
```

Next event should be smallest timestamp.

A min-priority queue handles this naturally.

54. DEQUE VS PRIORITY QUEUE

These are not the same.

Deque

Choice is based on:

TEXT

front or rear

Priority Queue

Choice is based on:

TEXT

priority/key

Ø=Common conceptual trap.

55. DEQUE VS CIRCULAR QUEUE

Circular queue:

TEXT

```
insert rear  
delete front
```

but array indices wrap.

Deque:

TEXT

```
insert/delete both ends
```

It can also use a circular array internally.

~~0-1%~~ Circular is implementation behavior; deque is an ADT capability.

56. GATE-STYLE QUESTION

Which structure allows insertion and deletion at both ends?

A. Stack B. Queue C. Deque D. Heap

Answer

C — Deque

57. GATE-STYLE QUESTION

In an input-restricted deque:

A. Insertion at both ends, deletion at one end B. Insertion at one end, deletion at both ends C. Both restricted to one end D. Priority determines deletion

Answer

B

58. GATE-STYLE QUESTION

In output-restricted deque:

A. Insertion at both ends, deletion at one end B. Insertion at one end, deletion at both ends C. Only rear insertion and rear deletion D. FIFO only

Answer

A

59. GATE-STYLE QUESTION

Which is the most common efficient general implementation for priority queue?

A. Stack B. Binary Heap C. Circular Linked List only D. Array with shifting always

Answer

B — Binary Heap

60. GATE-STYLE COMPLEXITY

A priority queue implemented using an unsorted array of n elements.

Insert new element at next free position:

TEXT

$O(1)$

Extract maximum requires scanning:

TEXT

$O(n)$

If sorted array:

TEXT

insert = $O(n)$
extract-max = $O(1)$

~~$O(n)$~~ Classic trade-off question.

61. DEQUE COMPLEXITY

With circular array or doubly linked list:

Operation	Time
Insert front	$O(1)$
Insert rear	$O(1)$
Delete front	$O(1)$
Delete rear	$O(1)$
Get front	$O(1)$
Get rear	$O(1)$

Search arbitrary element:

TEXT

$O(n)$

62. CIRCULAR DEQUE INDEX FORMULAS

FORMULA BOX

Forward:

TEXT
 $(i + 1) \% N$

Backward:

TEXT
 $(i - 1 + N) \% N$

These are among the most important formulas in this lesson.

63. DEQUE MEMORY TRICK

DEQUE = DO BOTH ENDS

Not literally spelling, but easy to remember:

TEXT
front operations '
rear operations '

64. RESTRICTED DEQUE MEMORY TRICK

Input restricted

Input has one door.

Deletion can use both doors.

Output restricted

Output has one door.

Insertion can use both doors.

$\emptyset = \text{Easy}$ exam mnemonic.

65. PRIORITY QUEUE MEMORY TRICK

Normal queue:

Who came first?

Priority queue:

Who matters first?

Algorithmically:

TEXT

FIFO vs priority

66. NOTEBOOK NOTES

Deque

TEXT

Double Ended Queue

supports:

TEXT

insert front
insert rear
delete front
delete rear

All $O(1)$ in good implementations.

67. RESTRICTED DEQUES

TEXT

Input-restricted:
insert one end
delete both ends

TEXT

Output-restricted:
insert both ends
delete one end

68. PRIORITY QUEUE

Removal based on:

TEXT

priority

Types:

TEXT

Min Priority Queue

Max Priority Queue

Common implementation:

TEXT

Binary Heap

69. PRIORITY QUEUE COMPLEXITY PREVIEW

Heap implementation:

TEXT

peek $O(1)$

insert $O(\log n)$

extract $O(\log n)$

We'll prove these later.

70. DON'T CONFUSE

Queue

TEXT

FIFO

Deque

TEXT

both ends

Priority Queue

TEXT

priority based

Circular Queue

TEXT

queue whose indices wrap

These are different ideas.

71. 30-SECOND REVISION

Remember:

1. Deque = double-ended queue.
 2. Insert/delete can happen at both ends.
 3. Input-restricted deque restricts insertion to one end.
 4. Output-restricted deque restricts deletion to one end.
 5. Circular array and DLL are good deque implementations.
 6. Circular backward index = $(i - 1 + N) \% N$.
 7. Priority queue removes by priority, not pure FIFO.
 8. Min-PQ removes smallest priority key.
 9. Max-PQ removes largest.
 10. Binary heap is the standard efficient PQ implementation.
-

72. ACTIVE RECALL

Q1

What does deque stand for?

Q2

What is restricted in an input-restricted deque?

Q3

What is restricted in an output-restricted deque?

Q4

What is the backward circular-index formula?

Q5

Which data structure commonly implements a priority queue efficiently?

ANSWERS

A1

Double Ended Queue.

A2

Insertion/input is allowed at only one end.

A3

Deletion/output is allowed at only one end.

A4

TEXT

$(i - 1 + N) \% N$

A5

Binary Heap.

73. PRACTICE — DEQUE TRACE

Start empty:

TEXT

```
insertRear(10)
insertRear(20)
insertFront(5)
deleteRear()
insertFront(2)
deleteFront()
```

Find:

- values deleted
- final deque
- front value
- rear value

74. PRACTICE — IDENTIFY TYPE

A structure allows:

TEXT

```
insert only at rear
delete at front or rear
```

Which deque variant?

Answer:

TEXT

Input-restricted deque

75. PRACTICE — PRIORITY QUEUE

Elements:

TEXT

```
A priority 3
B priority 8
C priority 5
```

If larger number means higher priority, find removal order.

Answer:

TEXT

B
C
A

assuming no other changes.

76. PRACTICE — IMPLEMENTATION TRADE-OFF

For a max-priority queue:

Unsorted array

What are:

TEXT

insert?
extractMax?

Sorted array

What are:

TEXT

insert?
extractMax?

Answers:

TEXT

Unsorted:
insert $O(1)$
extractMax $O(n)$

Sorted:
insert $O(n)$
extractMax $O(1)$

PROGRESS

MODULE 5 — QUEUE

' Lesson 44 — Queue Fundamentals ' Lesson 45 — Linear Queue Using Array ' Lesson 46 — Circular Queue '
Lesson 47 — Queue Using Linked List ' **Lesson 48 — Deque & Priority Queue**

NEXT — LESSON 49

Queue Applications & Placement Patterns — BFS, Level Order, Producer–Consumer, Queue Using Stacks

This will complete the **Queue module**.

We will cover:

- BFS intuition
- tree level-order traversal
- shortest path in unweighted graphs intuition
- producer-consumer buffering
- round-robin scheduling
- queue using two stacks
- stack using queues
- sliding-window deque recap
- GATE application traps
- final queue complexity comparison

After Lesson 49:

MODULE 6 — TREES

starts with:

Lesson 50 — Tree Terminology: Root, Parent, Child, Leaf, Degree, Level, Height, Depth, Ancestor, Descendant & Subtree.

USER

next

ASSISTANT

MODULE 5 — LESSON 49

Queue Applications & Placement Patterns — BFS, Level Order, Producer–Consumer, Queue Using Stacks

This is the **final Queue lesson**.

Today we connect queue concepts to actual GATE and placement problems.

GATE importance: MUST KNOW **Placement importance:** MUST KNOW

1. WHERE WE ARE

Completed:

' Queue fundamentals ' Linear queue using array ' Circular queue ' Linked queue ' Deque ' Priority queue

Now we complete queues with applications:

1. BFS
2. Tree level-order traversal
3. Shortest path in unweighted graphs
4. Producer–consumer buffering
5. Round-robin scheduling
6. Queue using two stacks
7. Stack using queues
8. Sliding-window deque recap
9. Final queue complexity comparison

2. APPLICATION 1 — BFS

BFS stands for:

Breadth First Search

It explores a graph:

TEXT

level by level

or:

TEXT

nearest vertices first

The natural data structure is:

QUEUE

3. WHY QUEUE FOR BFS?

Suppose:

TEXT



Start at:

TEXT

A

After visiting A, we discover:

TEXT

B
C

Both wait for processing.

Since B was discovered before C:

TEXT

B must be processed first

That is:

TEXT

FIFO

Therefore queue.

4. BFS STEP-BY-STEP

Graph/tree:

TEXT



Start:

TEXT

Queue: [A]

Remove A:

TEXT

Visit A

Add B,C:

TEXT

Queue: [B,C]

Remove B:

TEXT

Visit B

Add D,E:

TEXT

Queue: [C,D,E]

Remove C:

TEXT

Visit C

Add F:

TEXT

Queue: [D,E,F]

Then:

TEXT

D,E,F

Final BFS order:

A B C D E F

5. BFS MASTER ALGORITHM

Conceptually:

TEXT

```
mark start visited
enqueue(start)

while queue not empty:
    u = dequeue()

    process u

    for each neighbor v of u:
        if v not visited:
            mark v visited
            enqueue(v)
```

Important:

Mark a vertex visited when you enqueue it, not repeatedly later.

Otherwise the same vertex may be enqueued multiple times in a graph.

6. WHY VISITED ARRAY?

Consider:

TEXT

```
A -- B
|    |
C -- D
```

Without visited tracking, BFS could repeatedly revisit:

TEXT

```
A !'   B !'   D !'   C !'   A . . .
```

So graph BFS normally uses:

TEXT

```
visited[]
```

to avoid cycles/repeated work.

We'll study this deeply in Graphs.

7. BFS COMPLEXITY PREVIEW

Using adjacency list:

TEXT

```
Time =  $O(V + E)$ 
```

Why?

- every vertex processed at most once
- every edge inspected a constant number of times

Queue space:

TEXT

```
 $O(V)$ 
```

worst case.

~~$O(V)$~~ We'll derive formally in Graph module.

8. APPLICATION 2 — TREE LEVEL ORDER

Tree:

TEXT

```
      1
     / \
    2   3
   / \
  4   5
```

Level-order traversal:

TEXT

1 2 3 4 5

This is essentially:

BFS on a tree

So use queue.

9. LEVEL-ORDER ALGORITHM

TEXT

```
enqueue(root)

while queue not empty:
    node = dequeue()
    print node

    if left child exists:
        enqueue(left)

    if right child exists:
        enqueue(right)
```

~~O(N)~~ Queue preserves left-to-right level order.

10. TREE TRACE

Start:

TEXT

Queue: [1]

Remove 1.

Output:

TEXT

1

Enqueue:

TEXT

2, 3

Queue:

TEXT

[2, 3]

Remove 2:

TEXT

Output: 1 2

enqueue 4,5:

TEXT

[3, 4, 5]

Then process 3,4,5.

Final:

TEXT

1 2 3 4 5

11. BFS VS DFS

Very important:

Traversal	Main Data Structure
BFS	Queue
DFS	Stack / Recursion

Memory trick:

BFS spreads sideways !' Queue DFS dives deep !' Stack

Ø=Must know.

12. APPLICATION 3 — SHORTEST PATH IN UNWEIGHTED GRAPH Ø=Ý%

One extremely important BFS property:

In an unweighted graph, BFS finds shortest path in terms of number of edges.

Suppose:

TEXT

```
A  !'   B  !'   D
 \      /
  !'   C  !'
```

If destination D is first reached at distance 2, BFS guarantees no path using fewer than 2 edges was missed.

Why?

Because BFS processes:

TEXT

```
distance 0
then distance 1
then distance 2
...
```

in order.

13. DISTANCE ARRAY IDEA

Initialize:

TEXT

```
dist[start] = 0
```

When discovering neighbor:

TEXT

```
dist[v] = dist[u] + 1
```

Because BFS visits vertices layer by layer.

Example:

TEXT

```
A
% % %      B
%          % % %    D
% % %      C
```

Distances:

TEXT

```
A = 0
B = 1
C = 1
D = 2
```

14. DON'T USE NORMAL BFS SHORTEST-PATH CLAIM FOR WEIGHTED GRAPHS

If edges have different weights:

TEXT

```
A --100--> B
A --1--> C --1--> B
```

fewest edges is not necessarily minimum total weight.

So normal BFS shortest-path guarantee is for:

TEXT

unweighted graph

or equivalently equal-weight edges.

Ø=No GATE trap.

15. APPLICATION 4 — PRODUCER-CONSUMER

Imagine:

TEXT

P r o d u c e r ! ' B u f f e r ! ' C o n s u m e r

Producer creates items.

Consumer processes them.

A queue acts as the buffer:

TEXT

Producer :
enqueue

TEXT

Consumer :
dequeue

This naturally preserves arrival order.

16. BUFFER VISUALIZATION

Producer creates:

TEXT

A
B
C

Queue:

TEXT

front rear
! " ! "
A B C

Consumer removes:

TEXT

A

first.

Then:

TEXT
B

etc.

17. BOUNDED BUFFER

If buffer capacity is fixed:

TEXT
N

then it can become:

TEXT
FULL

Producer may need to wait.

If buffer is empty:

TEXT
Consumer may need to wait

This connects directly to:

- synchronization
- semaphores
- Operating Systems

Ø=Excellent cross-subject link.

18. WHY CIRCULAR QUEUE IS GOOD FOR BUFFERS

Suppose fixed array buffer.

After consumer removes old values, freed slots should be reused.

Circular queue does exactly this:

TEXT
rear wraps
front wraps

without shifting.

So ring buffers/circular queues are natural for streaming/buffering.

19. APPLICATION 5 — ROUND-ROBIN SCHEDULING

Processes:

TEXT

P1 P2 P3

Each gets a time quantum.

Queue:

TEXT

[P1, P2, P3]

CPU takes:

TEXT

P1

If unfinished, P1 goes back to rear:

TEXT

[P2, P3, P1]

Then P2:

TEXT

[P3, P1, P2]

This repeats.

~~0-1%~~ Circular queue intuition fits round robin perfectly.

20. APPLICATION 6 — REQUEST PROCESSING

Servers may maintain requests:

TEXT

R1

R2

R3

waiting for processing.

A FIFO queue is natural when fairness is based on arrival order.

Examples:

- print jobs
- simple web-job queues
- I/O requests
- message systems
- task dispatching

Actual systems may use priorities, multiple queues, etc., but queue is the base concept.

21. APPLICATION 7 — QUEUE USING TWO STACKS Ø=Ý%Ø=Ý%Ø=Ý%

Classic interview question:

Implement a queue using two stacks.

We need FIFO using only LIFO structures.

Use:

TEXT

```
Stack 1 = input
Stack 2 = output
```

22. CORE IDEA

When enqueueing:

TEXT

```
push into input stack
```

When dequeueing:

- if output stack is not empty, pop from it
- otherwise move all elements from input stack to output stack
- then pop output

Why?

Moving through two stacks reverses order twice conceptually, giving FIFO.

23. EXAMPLE

Enqueue:

TEXT

```
10
20
30
```

Input stack:

TEXT

```
TOP
! "
30
20
10
```


Output stack empty.

Now dequeue.

Move all input !' output.

First pop 30 !' push output.

Then 20.

Then 10.

Output becomes:

```
TEXT
TOP
! "
10
20
30
```

Now pop:

```
TEXT
10
```

Exactly FIFO.

24. NEXT DEQUEUE

Output stack now:

```
TEXT
TOP
! "
20
30
```

Next dequeue returns:

```
TEXT
20
```

No transfer needed.

$O = Y$ is why we don't need to move elements every time.

25. ENQUEUE WHILE OUTPUT HAS ELEMENTS

Suppose after first dequeue:

```
TEXT
Output :
20
30
```

Now enqueue 40.

Just push 40 into input:

```
TEXT
Input :
40
```

Do not disturb output.

Queue logical order:

```
TEXT
20 30 40
```

When output eventually empties, input gets transferred.

26. AMORTIZED COMPLEXITY $O(1)$

With the efficient two-stack method:

Enqueue

```
TEXT
O(1)
```

Dequeue

A particular dequeue may require transferring n elements:

```
TEXT
O(n)
```

But each element is:

- pushed onto input once
- moved to output once
- popped from output once

Across many operations:

```
TEXT
dequeue = O(1) amortized
```

$O(1)$ important interview complexity.

27. DON'T SAY DEQUEUE IS ALWAYS $O(1)$

Worst-case single dequeue:

TEXT

$O(n)$

Amortized over a long sequence:

TEXT

$O(1)$

~~$O=V$~~ Worst-case vs amortized complexity must be distinguished.

28. QUEUE USING TWO STACKS — PSEUDOCODE

Enqueue:

TEXT

`push(in, x)`

Dequeue:

TEXT

```
if out is empty:
    while in not empty:
        push(out, pop(in))
```

```
if out still empty:
    underflow
```

```
return pop(out)
```

29. WHY NOT TRANSFER AFTER EVERY ENQUEUE?

A poor implementation could repeatedly move all elements between stacks just to maintain front at top.

That can make operations:

TEXT

$O(n)$

frequently.

The lazy-transfer method is much better.

30. APPLICATION 8 — STACK USING QUEUES

Reverse challenge:

Implement LIFO stack using FIFO queues.

Possible strategies:

Strategy A

Make push costly.

Strategy B

Make pop costly.

31. ONE-QUEUE METHOD

Suppose queue currently represents stack:

TEXT

```
front !' 30 20 10 !• rear
```

Top should be 30 at front.

To push 40:

1. enqueue 40
2. rotate old elements behind it

After enqueue:

TEXT

```
30 20 10 40
```

Rotate first 3:

TEXT

```
20 10 40 30
10 40 30 20
40 30 20 10
```

Now:

TEXT

```
front = 40
```

So pop can simply dequeue.

32. COMPLEXITY OF THAT METHOD

For stack with n elements:

Push

rotates n elements:

TEXT

```
 $O(n)$ 
```

Pop

TEXT

0(1)

Top

TEXT

0(1)

Alternative designs can make:

TEXT

push 0(1)

pop 0(n)

Ø = Yes de-off question.

33. QUEUE USING STACKS VS STACK USING QUEUES

Problem	Main trick
Queue using stacks	Two reversals create FIFO
Stack using queues	Reorder queue so newest is removable first

Placement questions often test ADT understanding rather than syntax.

34. APPLICATION 9 — SLIDING WINDOW MAXIMUM

We introduced this with deque.

Array:

TEXT

1 3 -1 -3 5 3 6 7

Window size:

TEXT

3

Answer:

TEXT

3 3 5 5 6 7

Optimal data structure:

Deque

35. WHY NORMAL QUEUE IS NOT ENOUGH

For each new element we may need:

TEXT

remove expired index from front

and:

TEXT

remove smaller useless values from rear

Standard queue does not allow rear deletion.

Deque does.

36. MONOTONIC DEQUE

Maintain indices whose corresponding values are:

TEXT

decreasing from front to rear

Therefore:

TEXT

array[deque.front]

is current maximum.

When new value arrives:

TEXT

pop smaller/equal values from rear

Then insert current index.

~~O = S~~ Similar thinking to monotonic stack.

37. SLIDING WINDOW COMPLEXITY

Each index:

TEXT

inserted once
removed at most once

Therefore:

TEXT

Time = $O(n)$
Space = $O(k)$

because deque contains at most roughly window-sized candidates.

38. APPLICATION 10 — FIRST NEGATIVE IN EVERY WINDOW

Another deque/queue pattern:

Given array and window size k , maintain indices of negative numbers.

For each window:

TEXT

front of negative-index queue

is the first negative value.

When an index falls outside the window:

TEXT

remove from front

This can be done in:

TEXT

$O(n)$

39. APPLICATION 11 — MULTI-SOURCE BFS

Suppose grid has several starting points.

Instead of one source, enqueue all starting sources initially.

Example:

TEXT

Sources:
A, B, C

Initial queue:

TEXT

[A, B, C]

Then BFS expands all of them simultaneously level by level.

Used in many placement problems:

- rotten oranges
- nearest zero

- infection spread
- shortest distance to nearest source

0=important pattern.

40. GRID BFS

For a matrix:

TEXT

```
0 0 0
0 S 0
0 0 0
```

Neighbors might be:

TEXT

```
up
down
left
right
```

BFS queue stores cells:

TEXT

```
(row, col)
```

Distance increases by one layer.

This becomes very important during Graphs.

41. QUEUE APPLICATION — TOPOLOGICAL SORT PREVIEW

One method for topological sorting is:

Kahn's Algorithm

It maintains a queue of vertices having:

TEXT

```
indegree = 0
```

Repeatedly:

- dequeue one
- remove its outgoing effect
- enqueue newly zero-indegree vertices

We'll cover this later in Graphs.

42. QUEUE APPLICATION — TREE WIDTH / LEVEL PROCESSING

To process a tree one level at a time:

TEXT

levelSize = current queue size

Then process exactly levelSize nodes.

Children go to the next level in queue.

This enables:

- level sums
- level averages
- maximum width ideas
- zigzag variants

Ø=Placement extension.

43. GATE TRAP — BFS DOES NOT USE PRIORITY QUEUE

Normal BFS on an unweighted graph uses:

TEXT

ordinary FIFO queue

Dijkstra commonly uses:

TEXT

min-priority queue

Don't confuse them.

44. GATE TRAP — BFS VISITING ORDER

BFS order can depend on:

TEXT

neighbor ordering

Example:

If A has neighbors:

TEXT

B, C

then one BFS may be:

TEXT

A B C

If adjacency order is:

TEXT

C, B

then:

TEXT

A C B

can be valid.

Ø=BFS level structure is fixed, exact order inside a level may depend on adjacency order.

45. GATE TRAP — QUEUE FRONT VS REAR IN BFS

When a vertex is discovered:

TEXT

enqueue at rear

When it is processed:

TEXT

dequeue from front

Using stack-like behavior would change BFS into DFS-like exploration.

46. GATE TRAP — VISITED TIMING

Consider graph where two processed vertices both connect to X.

If X is marked visited only when dequeued, X may be inserted more than once.

Safer common BFS practice:

TEXT

mark visited when enqueued

Then it enters queue once.

Ø=important implementation detail.

47. GATE TRAP — SHORTEST PATH

Statement:

BFS always finds minimum-cost path.

False.

Correct:

BFS finds shortest path in number of edges in unweighted/equal-weight graphs.

Weighted graphs may need:

- Dijkstra
- Bellman-Ford
- others

depending on weights.

48. QUEUE IMPLEMENTATION MASTER TABLE

Queue Type	Enqueue	Dequeue	Main Feature
Linear array	$O(1)$	$O(1)$	Wastes freed space
Circular array	$O(1)$	$O(1)$	Reuses slots
Linked queue	$O(1)$	$O(1)$	Dynamic growth
Deque	$O(1)$ ends	$O(1)$ ends	Both ends
Priority queue using heap	$O(\log n)$	$O(\log n)$ extract	Priority-based

49. SPACE COMPARISON

Array queue

Allocated:

TEXT
 $O(\text{capacity})$

whether all slots are currently used or not.

Linked queue

For n nodes:

TEXT
 $O(n)$

plus pointer overhead.

Queue using two stacks

For n logical elements:

TEXT
 $O(n)$

across both stacks.

50. QUEUE PATTERN RECOGNITION

When you see:

- process in arrival order
- level-by-level traversal
- nearest-by-number-of-steps
- producer/consumer
- waiting tasks
- scheduling
- breadth-first search
- first discovered, first processed

think:

QUEUE

51. WHEN TO THINK CIRCULAR QUEUE

When you see:

- fixed buffer
- repeated reuse
- round robin
- streaming
- bounded producer-consumer
- ring buffer

think:

CIRCULAR QUEUE

52. WHEN TO THINK DEQUE

When you see:

- both ends required
- sliding-window maximum/minimum
- remove expired from front
- remove useless candidates from rear

think:

DEQUE

53. WHEN TO THINK PRIORITY QUEUE

When you see:

- highest priority first
- smallest/largest repeatedly
- Dijkstra
- Prim
- scheduling by priority
- event with earliest timestamp
- top-k / repeated best candidate

think:

PRIORITY QUEUE / HEAP

54. MEMORY TRICKS

Queue

First discovered, first handled.

BFS

Breadth = Queue.

Queue using stacks

When OUT is empty, pour IN into OUT.

Sliding window

Front expires; rear cleans.

Round robin

Serve front, unfinished goes rear.

55. NOTEBOOK NOTES — BFS

TEXT

```
enqueue source  
mark source visited
```

```
while queue not empty:  
    u = dequeue
```

```
for each neighbor v:
    if not visited:
        mark visited
        enqueue v
```

Adjacency list:

TEXT

```
Time =  $O(V+E)$ 
Space =  $O(V)$ 
```

56. NOTEBOOK NOTES — QUEUE USING TWO STACKS

TEXT

```
enqueue:
    push input

dequeue:
    if output empty:
        move all input to output

    pop output
```

Complexity:

TEXT

```
enqueue =  $O(1)$ 
dequeue worst case =  $O(n)$ 
dequeue amortized =  $O(1)$ 
```

57. NOTEBOOK NOTES — STACK USING QUEUE

One common design:

TEXT

```
push:
    enqueue new
    rotate previous elements

pop:
    dequeue
```

Complexity:

TEXT

```
push =  $O(n)$ 
pop =  $O(1)$ 
```

Alternative trade-offs exist.

58. QUEUE MODULE FORMULA SHEET

Circular next

TEXT
 $(i+1)\%N$

Circular previous

TEXT
 $(i-1+N)\%N$

Linear non-empty size

TEXT
 $\text{rear} - \text{front} + 1$

Circular size under last-occupied representation

TEXT
 $((\text{rear} - \text{front} + N)\%N) + 1$

Queue logical count

TEXT
initial
+ successful enqueues
- successful dequeues

59. GATE-STYLE QUESTION 1

Which data structure naturally supports shortest path in an unweighted graph?

A. Stack B. Queue C. Heap only D. BST

Answer: B — Queue

through BFS.

60. GATE-STYLE QUESTION 2

A queue is implemented using two stacks using lazy transfer.

What is the amortized complexity of dequeue?

Answer:

TEXT

$O(1)$

although an individual dequeue may be:

TEXT

$O(n)$

61. GATE-STYLE QUESTION 3

Which structure is most suitable for sliding-window maximum?

A. Stack B. Simple queue C. Deque D. BST necessarily

Answer:

C — Deque

because candidates may need removal at both ends.

62. GATE-STYLE QUESTION 4

BFS is run from vertex `s`.

When is a newly discovered vertex normally marked visited?

Answer:

When it is:

TEXT

enqueued/discovered

so duplicate enqueueing is avoided.

63. GATE-STYLE QUESTION 5

Which statement is correct?

A. BFS gives shortest weighted path for arbitrary positive weights B. BFS gives shortest path by edge count in unweighted graph C. DFS always gives shortest path D. Priority queue is mandatory for BFS

Answer:

B

64. INTERVIEW QUESTION

Why is queue used in BFS rather than stack?

Because BFS must process vertices in the order they are discovered so that all vertices at distance d are handled before vertices at distance $d+1$.

FIFO provides this behavior.

65. INTERVIEW QUESTION

How can a queue be implemented using two stacks?

Use one stack for input and one for output. Transfer input to output only when output becomes empty.

This reverses insertion order exactly when needed.

66. INTERVIEW QUESTION

Why is this efficient?

Each element is transferred from input stack to output stack at most once.

So across many operations, total transfer work is linear in the number of elements.

Hence:

TEXT

$O(1)$ amortized

per queue operation.

67. 30-SECOND REVISION

Remember:

1. BFS uses queue.
 2. Level order = BFS on tree.
 3. BFS gives shortest edge-count path in unweighted graph.
 4. Mark visited when enqueueing.
 5. Circular queue is excellent for fixed reusable buffers.
 6. Round robin naturally uses queue/circular behavior.
 7. Queue using two stacks gives $O(1)$ amortized operations.
 8. Deque is ideal for sliding-window max/min.
 9. Priority queue is for best-priority-first processing.
 10. Queue = FIFO.
-

68. ACTIVE RECALL

Q1

Why does BFS use FIFO?

Q2

What is BFS complexity with adjacency list?

Q3

What is the key trick for queue using two stacks?

Q4

Why is sliding-window maximum better suited to deque than normal queue?

Q5

Does ordinary BFS give minimum-weight path for arbitrary weighted graphs?

ANSWERS

A1

Because vertices discovered earlier must be processed earlier to maintain level order.

A2

TEXT

$O(V+E)$

A3

Transfer input stack to output only when output is empty.

A4

Because we need removal of expired indices from front and useless candidates from rear.

A5

No.

69. PRACTICE — BFS

For:

TEXT



Assuming neighbor order left-to-right, find BFS traversal starting at A.

70. PRACTICE — QUEUE USING STACKS

Enqueue:

TEXT

```
10
20
30
```

Then:

TEXT

```
dequeue
enqueue 40
dequeue
dequeue
```

Find returned values and final logical queue.

71. PRACTICE — ROUND ROBIN

Initial:

TEXT

```
[P1, P2, P3]
```

P1 runs but is unfinished.

What is queue after P1's time quantum?

Answer:

TEXT

```
[P2, P3, P1]
```

72. PRACTICE — BFS SHORTEST DISTANCE

Graph:

TEXT

```
A-B-D
| |
C-E
```

Starting from A, find distances to:

TEXT

```
A, B, C, D, E
```

assuming edges shown are unweighted.

73. QUEUE MODULE COMPLETE '

You have now covered:

' FIFO ' Enqueue / Dequeue ' Front / Rear ' Linear Array Queue ' Wasted Space ' Circular Queue ' Modulo Wrap-around ' Linked Queue ' Deque ' Input/Output Restricted Deque ' Priority Queue ' BFS ' Level Order ' Unweighted Shortest Path Intuition ' Producer–Consumer ' Round Robin ' Queue Using Two Stacks ' Stack Using Queues ' Sliding Window Deque Pattern

PROGRESS

MODULE 5 — QUEUE ' COMPLETE

Now we enter one of the biggest and most important sections:

MODULE 6 — TREES Ø<β3

NEXT — LESSON 50

Tree Fundamentals & Terminology — Root, Parent, Child, Sibling, Leaf, Degree, Depth, Level, Height, Ancestor, Descendant & Subtree

We will start from zero and build a precise vocabulary before binary trees.

Example:

TEXT



You will learn exactly:

- root
- node
- edge
- parent / child
- siblings
- leaf / internal node
- degree of node/tree
- path
- ancestor / descendant
- subtree
- depth
- level
- height of node/tree
- forest

- important `nodes !'` properly `edges`
- common GATE terminology traps

After that:

Lesson 51 — Binary Trees.

USER

next

ASSISTANT

MODULE 6 — LESSON 50

Tree Fundamentals & Terminology — Root, Parent, Child, Leaf, Degree, Level, Depth, Height, Ancestor, Descendant & Subtree

We now begin:

TREES 0<B3

Trees are one of the most important data structures in GATE and placements.

They form the foundation for:

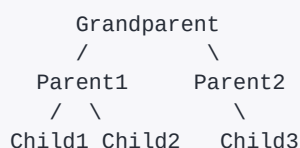
- Binary Trees
- Binary Search Trees
- Heaps
- Expression Trees
- Decision Trees
- File systems
- Hierarchical data
- Traversals
- Recursion
- Graph concepts

GATE importance: 0=Y%0MUSTKNOW **Placement importance:** 0=Y%0MUSTKNOW

1. STORY / INTUITION

Think of a family tree:

TEXT



There is a hierarchy.
Somebody is at the top.
Others branch downward.
This is the basic intuition of a:

Tree

2. BASIC TREE EXAMPLE

Consider:

TEXT



We will use this tree throughout the lesson.

3. WHAT IS A TREE?

A tree is a hierarchical, non-linear data structure consisting of:

- nodes
- edges

with a parent-child relationship.

In a rooted tree:

- one node is designated as root
- every other node has exactly one parent
- there is a unique path from root to every node

One idea:

Tree = hierarchy without cycles

4. NODE

Every item in a tree is called a:

Node

In our tree:

TEXT

A B C D E F

are nodes.

Number of nodes:

TEXT

$n = 6$

5. EDGE

A connection between two nodes is called an:

Edge

Tree:

TEXT

A-B

A-C

B-D

B-E

C-F

So there are:

TEXT

5 edges

6. MOST IMPORTANT TREE PROPERTY $\emptyset = \mathcal{Y}\% \emptyset = \mathcal{Y}\% \emptyset = \mathcal{Y}\%$

For a tree with:

TEXT

n nodes

number of edges is:

FORMULA BOX

TEXT

Edges = $n - 1$

Example:

TEXT

$n = 6$

Then:

TEXT
edges = 5

Matches our tree.

~~0-1%~~ Extremely important GATE property.

7. WHY **n-1** EDGES?

Start with one root:

TEXT
1 node
0 edges

Every new node must connect to the existing tree using exactly one new edge.

So adding:

TEXT
n-1

additional nodes adds:

TEXT
n-1

edges.

8. ROOT

The topmost node is called:

ROOT

In:

TEXT


root:

TEXT
A

~~0-1%~~ Root has:

TEXT

no parent

9. PARENT

If node X directly connects downward to Y, then X is the:

Parent

Example:

TEXT

A

% % %

B

% % %

C

A is parent of:

TEXT

B

C

B is parent of:

TEXT

D

E

C is parent of:

TEXT

F

10. CHILD

If X is parent of Y, then Y is child of X.

Example:

TEXT

Children of B:

D, E

Children of A:

TEXT

B, C

11. SIBLINGS

Nodes having the same parent are called:

Siblings

Example:

TEXT
D and E

have parent B.

So:

TEXT
D, E are siblings

Likewise:

TEXT
B, C

are siblings.

12. LEAF NODE Ø=Ý%

A node with no children is called:

- Leaf
- Terminal node
- External node, in some contexts

In our tree:

TEXT
D
E
F

are leaves.

Their number of children:

TEXT
Ø

13. INTERNAL NODE

A non-leaf node that has at least one child is usually called:

Internal Node

Our internal nodes:

TEXT

A
B
C

depending on convention, root is generally counted as internal if it has children.

Ø=leaf
Read exact wording in questions if they use "internal" or "non-leaf."

14. DEGREE OF A NODE

The degree of a node in a rooted tree usually means:

Number of children of that node.

Example:

TEXT

degree(A) = 2
degree(B) = 2
degree(C) = 1
degree(D) = 0
degree(E) = 0
degree(F) = 0

Ø=leaf
Leaf degree:

TEXT

0

15. DEGREE OF A TREE

The degree of a tree is:

Maximum degree of any node in the tree.

Our tree:

TEXT

max degree = 2

So:

TEXT

Degree(tree) = 2

16. DON'T CONFUSE — GRAPH DEGREE VS TREE DEGREE

In graph theory, degree of a vertex usually counts:

TEXT

number of incident edges

In rooted tree data-structure terminology, degree often means:

TEXT

number of children

Example node B:

TEXT



Graph-degree of B:

TEXT

3

because edges connect B to A,D,E.

Tree-degree as children count:

TEXT

2

Context matters.

17. PATH

A sequence of connected nodes is called a:

Path

Example:

TEXT

A !' B !' E

is a path.

Another:

TEXT

A !' C !' F

18. PATHLENGTH

Path length is commonly measured by:

number of edges

Example:

TEXT

A !' B !' E

contains:

TEXT

2 edges

So path length:

TEXT

2

Do not count nodes unless question explicitly defines length differently.

19. ANCESTOR

A node X is an ancestor of Y if X appears on the path from root to Y.

For node E:

TEXT

A !' B !' E

Ancestors:

TEXT

A

B

Depending on terminology, a node may or may not be considered its own ancestor. GATE questions generally make the intended meaning clear; usually “proper ancestor” excludes the node itself.

20. DESCENDANT

Reverse relation.

If A is ancestor of E, then:

TEXT

E is descendant of A

Descendants of B:

TEXT

D
E

Descendants of A:

TEXT

B
C
D
E
F

21. PROPER ANCESTOR / DESCENDANT

“Proper” generally means excluding the node itself.

So proper ancestors of E:

TEXT

A, B

not E.

Proper descendants of B:

TEXT

D, E

not B.

22. SUBTREE $\emptyset = \mathcal{Y}^{\%}$

A node together with all its descendants forms a:

Subtree

Subtree rooted at B:

TEXT

```
  B
 / \
D   E
```

Subtree rooted at C:

TEXT



Every node defines a subtree rooted at itself.

23. SUBTREE OF A LEAF

A leaf such as D has no descendants.

Its subtree is simply:

TEXT

D

So even a single node is a valid subtree.

24. DEPTH OF A NODE $\text{depth}(v)$

Depth of a node:

Number of edges from the root to that node.

Our tree:

TEXT



Depth:

TEXT

```
depth(A) = 0
depth(B) = 1
depth(C) = 1
depth(D) = 2
depth(E) = 2
depth(F) = 2
```

$\text{depth}(v)$ standard convention:

TEXT

root depth = 0

25. LEVEL OF A NODE — CONVENTION WARNING

Some books define:

Then:

Other books define:

Then:

$\emptyset = \emptyset$ TE-safe rule:

26. LEVEL EXAMPLE

If root is level 1:

Do not mix conventions.

27. HEIGHT OF A NODE

Number of edges on the longest downward path from that node to a leaf.

Consider:

TEXT

B

% % % D

% % % E

Longest path from B to leaf:

TEXT

B ! ' D

or:

TEXT

B ! ' E

one edge.

So:

TEXT

$\text{height}(B) = 1$

28. HEIGHT OF A LEAF

Leaf has no edge below it.

Therefore:

TEXT

$\text{height}(\text{leaf}) = 0$

Under edge-based convention.

So:

TEXT

$\text{height}(D)=0$

$\text{height}(E)=0$

$\text{height}(F)=0$

29. HEIGHT OF ROOT

Longest path from A:

TEXT

A ! ' B ! ' D

or:

TEXT

A !' C !' F

contains:

TEXT

2 edges

Therefore:

TEXT

height(A)=2

Height of root is also:

Height of the tree

So:

TEXT

height(tree)=2

30. HEIGHT VS DEPTH Ø=Ý%Ø=Ý%Ø=Ý%

This is one of the most important distinctions.

Depth

Measured:

TEXT

r o o t !' n o d e

Height

Measured:

TEXT

n o d e !' d e e p e s t l e a f

Memory trick:

****DEPTH looks UP to root.**

HEIGHT looks DOWN to leaves.**

31. EXAMPLE — NODE B

Path root to B:

TEXT

A ! ' B

So:

TEXT

depth(B)=1

Deepest leaf below B:

TEXT

B ! ' D

one edge.

So:

TEXT

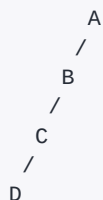
height(B)=1

They happen to be equal here, but that's not always true.

32. BETTER EXAMPLE

Tree:

TEXT



For node C:

TEXT

depth(C)=2

because:

TEXT

A ! ' B ! ' C

two edges.

Height(C):

TEXT

C ! ' D

one edge.

So:

TEXT
 $\text{height}(C)=1$

Clearly different.

33. HEIGHT OF TREE VS NUMBER OF LEVELS

If height is measured in edges:

TEXT
 $\text{number of levels} = \text{height} + 1$

Example:

TEXT
 $\text{height} = 2$

levels:

TEXT
0, 1, 2

so 3 levels.

If level numbering starts at 1:

TEXT
 $\text{levels} = 1, 2, 3$

still 3 total.

FORMULA BOX

TEXT
 $\text{Total levels} = \text{height} + 1$

under edge-based height.

34. EDGE-HEIGHT VS NODE-HEIGHT CONVENTION

Some sources define height as number of nodes on longest path.

Then leaf height may be:

TEXT

1

instead of 0.

But most CS data-structure/GATE contexts commonly use edge-count height.

~~0 = 1~~ Always check definition.

We'll primarily use:

TEXT

leaf height = 0

35. FOREST

A collection of disjoint trees is called a:

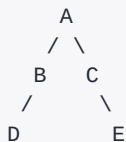
Forest

Interesting property:

If you remove the root of a tree, the remaining child subtrees form a forest.

Example:

TEXT



Remove A:

TEXT

Tree 1: B-D

Tree 2: C-E

Together:

TEXT

Forest

36. ORDERED TREE

An ordered tree is one where the order of children matters.

Example:

TEXT



is considered different from:

TEXT



if child order matters.

Binary trees are ordered structures because left and right child are distinct.

37. UNORDERED TREE

If child order does not matter, swapping siblings may represent the same abstract tree.

This distinction matters more in theoretical contexts.

$\emptyset = \text{Nil}$ know the idea.

38. ROOTED TREE

A tree where one node is designated as:

TEXT

root

and parent-child directions are defined relative to it.

Most data-structure trees are rooted trees.

39. EMPTY TREE

A tree can also be empty:

TEXT

no nodes

Implementation often represents empty binary tree as:

C

```
root = NULL;
```

Useful later.

40. SINGLE-NODE TREE

TEXT

A

Properties:

TEXT

```
nodes = 1
edges = 0
root = A
leaf = A
height = 0
depth(A)=0
```

$\emptyset$ = Root can also be a leaf if it has no children.

41. GATE TRAP — ROOT CAN BE LEAF

Statement:

Root can never be a leaf.

False.

In a one-node tree:

TEXT

A

A is both:

TEXT

root
leaf

$\emptyset$ = Classic conceptual trap.

42. GATE TRAP — LEAF DEGREE

Leaf node degree in rooted-tree terminology:

TEXT

0

because it has no children.

43. GATE TRAP — ROOT PARENT

Root has:

TEXT

no parent

It may have:

TEXT

0 or more children

depending on tree type.

44. GATE TRAP — UNIQUE PATH

In a tree, between any two nodes there is:

Exactly one simple path

Example:

TEXT

D !' B !' A !' C !' F

is the unique path from D to F.

$\emptyset = 70\%$ fundamental tree property.

45. WHY UNIQUE PATH?

If there were:

TEXT

two different paths

between the same two nodes, those paths would form a:

TEXT

cycle

But a tree has no cycles.

46. TREE AS GRAPH PROPERTY $\emptyset = 70\%$

An undirected graph is a tree iff it is:

TEXT

connected
and
acyclic

Equivalent properties include:

- connected with $n-1$ edges
- acyclic with $n-1$ edges
- exactly one simple path between every pair of vertices

Ø=0 This becomes important in Graphs.

47. IMPORTANT EQUIVALENT TREE PROPERTIES

For an undirected graph with n vertices:

Any two of the following imply the others under suitable formulations:

TEXT

connected
acyclic
 $n-1$ edges
unique path between every pair

For GATE, the common theorem:

FORMULA BOX

TEXT

Tree with n vertices has $n-1$ edges

48. NUMBER OF CHILDREN VS DEGREE SUM

Every non-root node is a child of exactly one parent.

So if we sum the number of children over all nodes:

TEXT

sum of child counts = $n - 1$

Why?

Because every edge corresponds to exactly one parent-child relationship.

Ø=0 Not a GATE numerical property.

49. EXAMPLE DEGREE SUM

Our tree:

TEXT

degree(A)=2
degree(B)=2
degree(C)=1
degree(D)=0
degree(E)=0
degree(F)=0

Sum:

TEXT

$$2+2+1 = 5$$

Nodes:

TEXT

$$n=6$$

So:

TEXT

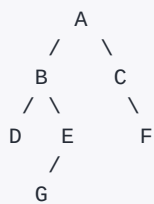
$$\text{sum degrees-as-children} = n-1 = 5$$

Correct.

50. TERMINOLOGY MASTER DIAGRAM

Use:

TEXT



Now:

Root

TEXT

A

Parent of E

TEXT

B

Children of B

TEXT

D, E

Sibling of D

TEXT

E

Leaves

TEXT

D, F, G

Internal nodes

TEXT

A, B, C, E

Ancestors of G

TEXT

A, B, E

Descendants of B

TEXT

D, E, G

Depth of G

TEXT

3

Height of E

TEXT

1

Height of B

TEXT

2

Height of A

TEXT

3

51. DEPTH TABLE

For above tree:

Node	Depth
A	0
B	1
C	1
D	2
E	2
F	2
G	3

52. HEIGHT TABLE

Node	Height
D	0
F	0
G	0
E	1
C	1
B	2
A	3

Height is computed bottom-up.

53. DEPTH = TOP-DOWN

To find depth:

TEXT
start from root
count edges downward

Example G:

TEXT
A! ' B! ' E! ' G

3 edges.

So:

TEXT

depth(G)=3

54. HEIGHT = BOTTOM-UP

To find height:

Look for deepest leaf below node.

For B:

TEXT

B! ' E! ' G

2 edges.

Therefore:

TEXT

height(B)=2

55. IMPLEMENTATION PREVIEW

A general tree node might conceptually need:

TEXT

data
list of children

But GATE focuses heavily on:

Binary Trees

where every node has at most:

TEXT

2 children

So common C node:

C

```
typedef struct Node
{
    int data;
    struct Node *left;
    struct Node *right;
} Node;
```

We'll study this next.

56. WHY TREE IS NON-LINEAR

Array:

```
TEXT
10 20 30 40
```

has one sequential direction.

Linked list:

```
TEXT
1 0 !' 2 0 !' 3 0
```

also follows a chain.

Tree:

```
TEXT
      10
     /  \
    20   30
   /  \
  40  50
```

branches.

Therefore:

Non-linear data structure

57. APPLICATION — FILE SYSTEM

Directory:

```
TEXT
Root
% % %   D o c u m e n t s
%       % % %   R e s u m e
%       % % %   N o t e s
% % %   P i c t u r e s
        % % %   P h o t o
```

This is hierarchical.

Tree is natural.

58. APPLICATION — ORGANIZATION

```
TEXT
CEO
% % %   C T O
%       % % %   E n g i n e e r 1
```

% % % E n g i n e e r 2
% % % C F O

Again hierarchy.

59. APPLICATION — EXPRESSION TREE

Expression:

TEXT
A+B*C

Tree:

TEXT

```
graph TD; Plus["+"] --- A["A"]; Plus --- Star["*"]; Star --- B["B"]; Star --- C["C"];
```

You already used this in stack-expression lessons.

Tree connects many topics together.

60. APPLICATION — SEARCH STRUCTURES

Later:

TEXT
Binary Search Tree

uses tree hierarchy to support efficient search under favorable shape.

TEXT
Binary Heap

uses tree shape for priority queue.

So tree terminology must be crystal clear.

61. GATE-STYLE QUESTION 1

A tree has:

TEXT
50 nodes

How many edges?

Using:

TEXT

edges= $n-1$

Answer:

TEXT

49

62. GATE-STYLE QUESTION 2

A tree has:

TEXT

100 edges

How many nodes?

TEXT

$n-1=100$

Therefore:

TEXT

$n=101$

63. GATE-STYLE QUESTION 3

A rooted tree contains n nodes.

What is the sum of number of children of all nodes?

Answer:

TEXT

$n-1$

because each non-root node contributes exactly once as a child.

64. GATE-STYLE QUESTION 4

Which node has depth 0?

Answer

TEXT

Root

under standard edge-based depth convention.

65. GATE-STYLE QUESTION 5

Which nodes have height 0?

Answer

TEXT

Leaves

under standard edge-height convention.

66. GATE TRAP — HEIGHT VS MAX DEPTH

For a rooted tree with edge-based definitions:

TEXT

$\text{height}(\text{tree})$
=
maximum depth of any node

because both measure the longest root-to-leaf edge count.

$\neq$ useful relationship.

67. GATE TRAP — LEAF VS INTERNAL

A leaf:

TEXT

0 children

An internal/non-leaf node:

TEXT

at least 1 child

But in special tree classes such as full binary trees, internal nodes may have specific child counts.

We study that later.

68. GATE TRAP — TREE DEGREE

If maximum node has 3 children:

TEXT

```
degree(tree)=3
```

It doesn't mean total edges = 3.

0=0% Degree and edge count are completely different quantities.

69. GATE TRAP — LEVEL CONVENTION

Never automatically assume:

TEXT

```
root level=1
```

or:

TEXT

```
root level=0
```

Check the question.

Depth is safer because standard definition usually uses root depth 0.

70. GATE TRAP — HEIGHT OF EMPTY TREE

Different textbooks/programming conventions may define empty-tree height as:

TEXT

```
-1
```

when leaf height = 0,

or sometimes:

TEXT

```
0
```

under node-count conventions.

0=0% If recursion formulas are involved, inspect the definition.

For our standard edge-height convention:

TEXT

```
leaf height = 0
```

and many recursive implementations use:

TEXT

```
height(NULL) = -1
```

so that:

TEXT

$\text{height}(\text{leaf}) = 1 + \max(-1, -1) = 0$

We'll revisit later.

71. MEMORY TRICKS

Root

The one with no parent.

Leaf

The one with no children.

Depth

Root down to me.

Height

Me down to deepest leaf.

Degree

How many children?

Tree edges

One fewer edge than nodes.

72. NOTEBOOK NOTES

Tree

TEXT

Hierarchical non-linear structure

Root

TEXT

No parent

Leaf

TEXT

No children
degree = 0
height = 0

Parent/Child

Direct one-level relationship.

Ancestor/Descendant

Multi-level relationship.

73. FORMULA BOX

For a tree with `n` nodes:

TEXT

`edges = n - 1`

Sum of child counts:

TEXT

`n - 1`

For edge-based height:

TEXT

`levels = height + 1`

Tree height:

TEXT

`max depth`

74. DON'T CONFUSE

Concept	Meaning
Depth	Root !' node
Height	Node !' deepest leaf
Degree of node	Number of children
Degree of tree	Maximum node degree
Path length	Number of edges

Concept	Meaning
Leaf	No children
Root	No parent

Copy this table.

75. 30-SECOND REVISION

Remember:

- 1. Tree is hierarchical and non-linear.
- 2. Root has no parent.
- 3. Leaf has no children.
- 4. Siblings share same parent.
- 5. Degree of node = number of children.
- 6. Tree with n nodes has n-1 edges.
- 7. Depth = edges from root to node.
- 8. Height = longest downward path to leaf.
- 9. Leaf height = 0.
- 10. Tree height = maximum node depth.

76. ACTIVE RECALL

Use:

TEXT

```
graph TD
    A --> B
    A --> C
    B --> D
    B --> E
```

Q1

What is root?

Q2

What are leaves?

Q3

What is degree of B?

Q4

What is depth of E?

Q5

What is height of A?

ANSWERS

A1

TEXT

A

A2

TEXT

C, D, E

A3

TEXT

2

A4

TEXT

2

A5

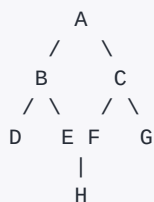
TEXT

2

77. PRACTICE — TERMINOLOGY

For:

TEXT



Find:

1. Root
2. Leaves
3. Parent of H
4. Siblings of F
5. Degree of C
6. Depth of H

7. Height of F
8. Height of C
9. Ancestors of H
10. Descendants of C

78. PRACTICE — NUMERICAL

A

Tree has 72 nodes. Find edges.

B

Tree has 149 edges. Find nodes.

C

Sum of child counts in a rooted tree is 99. Find number of nodes.

79. GATE-STYLE CONCEPT

Which statements are always true for a rooted tree?

1. Root has no parent.
2. Every non-root node has exactly one parent.
3. Every leaf has degree zero.
4. Number of edges = number of nodes.
5. There is exactly one simple path between any two nodes.

Correct:

TEXT

1, 2, 3, 5

Statement 4 is false because:

TEXT

edges = nodes - 1

PROGRESS

MODULE 6 — TREES

' Lesson 50 — Tree Fundamentals & Terminology

NEXT — LESSON 51

Binary Trees — Definition, C Node Structure, Left/Right Child, Array & Linked Representation

Next we study:

TEXT



where every node has at most:

TEXT

2 children

We will cover:

- binary tree definition
- left child vs right child
- why order matters
- C node structure
- linked representation
- array representation
- 0-based and 1-based child/parent formulas
- sparse-tree memory waste
- maximum nodes at level
- maximum nodes for a given height
- minimum possible height for n nodes
- major GATE formulas

Then Lesson 52 will cover:

Full, Complete, Perfect, Balanced & Skewed Binary Trees.

USER

did it complete

ASSISTANT

Not yet. We have completed **up to Lesson 50**, but the official **Programming and Data Structures** syllabus is still not fully complete.

So far:

- ' Programming in C
- ' Recursion
- ' Arrays
- ' Linked Lists
- ' Stacks
- ' Queues
- ~~Ø=Trees~~ — just started
- #~~Ø~~ Binary Search Trees

- ~~#B~~ Binary Heaps
- ~~#G~~ Graphs

We are currently at:

Lesson 50 — Tree Fundamentals & Terminology '

Next is:

Lesson 51 — Binary Trees

After that, the main remaining core lessons are roughly:

- Lessons 51–58 !' Trees
- Lessons 59–64 !' BST
- Lessons 65–71 !' Binary Heaps
- Lessons 72–78 !' Graphs

So the official core syllabus will be essentially complete after **Lesson 78**.

That means:

50 / 78 core lessons completed "164%

About **28 core lessons remain**.

After Lesson 78, we can still do the optional high-value parts:

- Complexity consolidation
- GATE trap revision
- mixed GATE-style tests
- C output questions
- placement coding revision

So for the **official syllabus itself**, we are now around **two-thirds done**.

USER

i need to give complete thing without missing anything

ASSISTANT

Yes. From this point onward, I'll treat **"complete without missing anything" as a hard requirement** for this subject.

I will not stop at only the obvious GATE headings. For every remaining syllabus item, I'll cover the full chain needed to solve GATE questions and placement problems:

- **Trees:** binary-tree representation, all tree types, formulas/properties, array-index formulas, traversals, recursive and iterative intuition, level order, reconstruction from traversals, uniqueness conditions, expression-tree connections, height/node/leaf numericals, and GATE traps.
- **BST:** ordering property, search, insertion, deletion all 3 cases, min/max, predecessor/successor, inorder property, complexity in balanced vs skewed trees, validation, kth smallest/largest, LCA, sorted array !' BST, and GATE numericals.
- **Binary Heaps:** min/max heap, complete-tree requirement, array representation, 0-based and 1-based parent/child formulas, insertion, up-heap, heapify/down-heap, deletion/extract, build-heap, **why build heap is $O(n)$** , heapsort, priority queues, and GATE traces.
- **Graphs:** terminology, directed/undirected, weighted/unweighted, degree/in-degree/out-degree,

paths/cycles/connectivity/components, adjacency matrix/list and complexities, BFS, DFS, traversal-order dependence, connected components, cycle detection, islands/flood fill, and topological-sort intuition relevant to placements.

- **Complexity:** I will explicitly attach time and auxiliary-space complexity to every important operation instead of leaving it implicit.
- **C implementation:** wherever implementation is relevant, I'll show correct C code and pointer/memory behavior.
- **GATE coverage:** formulas, conceptual traps, output traces, numericals, original GATE-style questions, and revision points.
- **Placement coverage:** only after the GATE core is properly covered, I'll add the useful interview extensions.

Most importantly, I'll use a **syllabus-completeness checklist**. Before declaring this subject complete, I will verify every word of your official syllabus:

Programming in C. Recursion. Arrays, stacks, queues, linked lists, trees, binary search trees, binary heaps, graphs.

Current status:

Official syllabus	Status
Programming in C	' Fully covered
Recursion	' Fully covered
Arrays	' Fully covered
Stacks	' Fully covered
Queues	' Fully covered
Linked Lists	' Fully covered
Trees	Ø=ßá In progress
Binary Search Trees	#ó
Binary Heaps	#ó
Graphs	#ó

And I **will not declare the subject finished immediately after Lesson 78**. After the teaching lessons, I'll do a final **COMPLETE SYLLABUS AUDIT** with:

Official syllabus item !' concepts covered !' formulas !' complexities !' C implementations !' GATE traps !' remaining gaps.

If I find even one meaningful gap, we cover it before marking the subject ' complete.

So yes—continue with **NEXT**, and we proceed with **Lesson 51: Binary Trees**, maintaining this complete-coverage standard.

USER

next

TEXT

0 children
1 child
2 children

All are allowed.

Example:

TEXT



This is still a binary tree.

0=0% Common GATE trap.

3. LEFT AND RIGHT ARE DIFFERENT

Consider:

TEXT



versus:

TEXT



These are different binary trees.

Why?

Because:

TEXT

l e f t c h i l d " ` r i g h t c h i l d

Binary trees are **ordered** structures.

0=0% Mid position matters.

4. GENERAL TREE VS BINARY TREE

General tree:

TEXT



A has 3 children.

So this is **not** a binary tree.

Binary tree:

TEXT



maximum children per node:

TEXT

2

5. BINARY TREE NODE IN C

Standard representation:

```
C
typedef struct Node
{
    int data;
    struct Node *left;
    struct Node *right;
} Node;
```

Each node contains:

TEXT

data
left pointer
right pointer

6. MEMORY VISUALIZATION

Suppose node stores 10.

Conceptually:

TEXT

```
+-----+-----+-----+
| data  | left  | right  |
+-----+-----+-----+
|  10   | addr1 | addr2 |
+-----+-----+-----+
```

If no left child:

```
TEXT
left = NULL
```

If no right child:

```
TEXT
right = NULL
```

7. LEAF NODE IN C

Leaf:

```
TEXT
50
```

has no children.

So:

```
C
node->left = NULL;
node->right = NULL;
```

Leaf in linked binary tree:

```
TEXT
left == NULL
AND
right == NULL
```

8. CREATING A NODE

```
C
Node *createNode(int value)
{
    Node *newNode = malloc(sizeof *newNode);

    if (newNode == NULL)
        return NULL;

    newNode->data = value;
    newNode->left = NULL;
    newNode->right = NULL;

    return newNode;
}
```

Time:

TEXT

0(1)

9. BUILDING A SIMPLE BINARY TREE

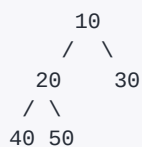
```
C
Node *root = createNode(10);

root->left = createNode(20);
root->right = createNode(30);

root->left->left = createNode(40);
root->left->right = createNode(50);
```

Tree:

TEXT



10. ROOT POINTER

Entire tree is usually accessed through:

```
C
Node *root;
```

Empty tree:

```
C
root = NULL;
```

Non-empty:

TEXT

r o o t ! ' f i r s t / t o p n o d e

If `root` is lost and no other pointer exists, the tree becomes unreachable.

11. LINKED REPRESENTATION

Most general binary-tree implementations use linked nodes:

TEXT



Conceptually:

TEXT

```
A . l e f t   !'   B
A . r i g h t !'   C

B . l e f t   !'   D
B . r i g h t !'   E
```

Advantages:

- dynamic memory
- natural for sparse trees
- no wasted array gaps caused purely by missing children
- easy recursive manipulation

12. NULL LINKS

For tree:

TEXT



B has no children:

TEXT

```
B . l e f t = NULL
B . r i g h t = NULL
```

C has no children:

TEXT

```
C . l e f t = NULL
C . r i g h t = NULL
```

So leaf nodes contain two NULL child links.

13. NUMBER OF CHILD POINTERS

For n nodes in linked binary tree:

Each node has:

TEXT

2 child-pointer fields

Total child-pointer fields:

TEXT

$2n$

But only:

TEXT

$n - 1$

are actual parent-child edges.

Therefore NULL child pointers:

FORMULA BOX

TEXT

NULL child pointers
 $= 2n - (n - 1)$
 $= n + 1$

0 = 100% important GATE property.

14. EXAMPLE OF NULL POINTER COUNT

Suppose binary tree has:

TEXT

$n = 7$ nodes

Total child-pointer slots:

TEXT

$2n = 14$

Actual edges:

TEXT

$n-1 = 6$

NULL links:

TEXT

$14 - 6 = 8$

So:

TEXT

$n+1 = 8$

Correct.

15. WHY $n+1$ NULL LINKS MATTERS

This property is used in:

- threaded binary trees
- tree representation questions
- pointer-count numericals
- GATE conceptual problems

Write this in notes.

16. ARRAY REPRESENTATION OF BINARY TREE

Binary trees can also be stored in arrays.

This is especially useful for:

Complete Binary Trees

and therefore:

Binary Heaps

Example tree:

TEXT



Can be stored as:

TEXT

A B C D E F G

in level order.

17. 1-BASED ARRAY REPRESENTATION

Suppose root stored at index:

TEXT

1

Then for node at index i :

FORMULA BOX

TEXT

Left child = $2i$
Right child = $2i + 1$
Parent = $\text{floor}(i/2)$

for:

TEXT

$i > 1$

~~Do not~~ Must memorize.

18. 1-BASED EXAMPLE

Array:

TEXT

Index: 1 2 3 4 5 6 7
A B C D E F G

For B:

TEXT

$i = 2$

Left child:

TEXT

$2i = 4$!' D

Right:

TEXT

$2i + 1 = 5$!' E

Parent:

TEXT

$\text{floor}(2/2) = 1$!' A

19. ANOTHER 1-BASED EXAMPLE

Node C:

```
TEXT
i = 3
```

Children:

```
TEXT
left  = 6 !' F
right = 7 !' G
```

Parent:

```
TEXT
floor(3 / 2) = 1 !' A
```

20. 0-BASED ARRAY REPRESENTATION

Programming languages normally use index 0.

If root at:

```
TEXT
0
```

then node at index **i** has:

FORMULA BOX

```
TEXT
Left child  = 2i + 1
Right child = 2i + 2
Parent      = floor((i - 1)/2)
```

for:

```
TEXT
i > 0
```

~~Ø=Must~~ Must memorize.

21. 0-BASED EXAMPLE

Array:

```
TEXT
Index: 0 1 2 3 4 5 6
       A B C D E F G
```

For B:

TEXT

i=1

Left:

TEXT

2 (1) + 1 = 3 !' D

Right:

TEXT

2 (1) + 2 = 4 !' E

Parent:

TEXT

f l o o r ((1 - 1) / 2) = 0 !' A

22. 0-BASED EXAMPLE — C

TEXT

i = 2

Left:

TEXT

5 !' F

Right:

TEXT

6 !' G

Parent:

TEXT

floor((2-1)/2)
= floor(1/2)
=0

!' A.

23. DON'T MIX 0-BASED AND 1-BASED FORMULAS

1-based

TEXT

```
left = 2i
right = 2i+1
parent = floor(i/2)
```

0-based

TEXT

```
left = 2i+1
right = 2i+2
parent = floor((i-1)/2)
```

One of the most common heap/tree mistakes.

24. MEMORY TRICK

1-based

Double, double+1

TEXT

```
2i
2i+1
```

0-based

Double+1, double+2

TEXT

```
2i+1
2i+2
```

25. ARRAY REPRESENTATION AND MISSING CHILDREN

Consider:

TEXT

```
  A
 /
B
/
C
```

1-based positions:

TEXT

```
A !' 1
B !' 2
C !' 4
```

Array:

TEXT

Index: 1 2 3 4
A B _ C

Index 3 is unused.

For deeper skewed trees, wasted gaps can become huge.

26. EXTREME ARRAY WASTE

Consider left-skewed tree:

TEXT

A
/
B
/
C
/
D

1-based indices:

TEXT

A = 1
B = 2
C = 4
D = 8

Only 4 nodes, but index 8 is needed.

Most positions:

TEXT

3, 5, 6, 7

are unused.

~~O(N)~~ Array representation is poor for sparse/skewed trees.

27. WHY ARRAY REPRESENTATION IS GREAT FOR HEAPS

Heap must be a:

TEXT

complete binary tree

so nodes fill almost continuously level by level.

That means:

TEXT

very little/no index wastage

Thus heaps are efficiently stored in arrays.

We'll revisit in Module 8.

28. MAXIMUM NODES AT A LEVEL $\emptyset = \text{\'Y\%}\emptyset = \text{\'Y\%}\emptyset = \text{\'Y\%}$

Assume root is at level:

TEXT

0

Maximum nodes at level **L** :

FORMULA BOX

TEXT

2^L

Examples:

TEXT

L e v e l	0	!	1
L e v e l	1	!	2
L e v e l	2	!	4
L e v e l	3	!	8

$\emptyset = \text{\'Y\%}$ Must know.

29. WHY 2^L ?

Each node can produce at most:

TEXT

2 children

So maximum doubles each level:

TEXT

1
2
4
8
16
...

Hence:

TEXT

2^h

30. MAXIMUM NODES FOR HEIGHT h

Assume:

TEXT

root height = h
leaf height = 0

Levels:

TEXT

$0 \dots h$

Maximum nodes:

TEXT

$1 + 2 + 4 + \dots + 2^h$

Geometric series:

FORMULA BOX

TEXT

Maximum nodes
 $= 2^{(h+1)} - 1$

Extremely important GATE formula.

31. EXAMPLE

Height:

TEXT

$h = 3$

Maximum nodes:

TEXT

$2^{(3+1)} - 1$
 $= 16 - 1$
 $= 15$

Levels:

TEXT

$1 + 2 + 4 + 8 = 15$

32. MAXIMUM LEAVES FOR HEIGHT h

At deepest level:

TEXT

h

maximum nodes:

TEXT

2^h

Therefore maximum leaves:

FORMULA BOX

TEXT

2^h

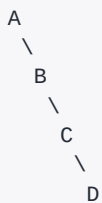
for a binary tree of height h .

33. MINIMUM NODES FOR HEIGHT h

What is the fewest nodes possible while still having height h ?

A skewed chain:

TEXT



If height:

TEXT

h

number of nodes:

FORMULA BOX

TEXT

Minimum nodes = $h + 1$

because longest path of h edges contains $h+1$ nodes.

34. EXAMPLE

Height:

TEXT

$h=4$

Minimum nodes:

TEXT

5

Maximum:

TEXT

$2^5 - 1$
 $= 31$

Therefore any binary tree of height 4 has:

TEXT

5 "d n "d 3 1

Very useful numerical range.

35. HEIGHT BOUNDS FOR n NODES

Given n nodes:

Maximum height

Worst case skewed:

TEXT

$h_{\max} = n - 1$

because chain of n nodes has $n-1$ edges.

36. MINIMUM HEIGHT FOR n NODES

To minimize height, pack tree as densely as possible.

We need:

TEXT

```
n "d 2 ^ ( h + 1 ) - 1
```

Rearrange:

TEXT

```
n + 1 "d 2 ^ ( h + 1 )
```

Taking log base 2:

TEXT

```
l o g 2 ( n + 1 ) "d h + 1
```

Minimum integer height:

FORMULA BOX

TEXT

```
h_min = ceil(log2(n+1)) - 1
```

Equivalent:

TEXT

```
floor(log2 n)
```

for positive integer n .

Ø=important.

37. CHECK EXAMPLE — $n=7$

Perfectly packed:

TEXT

```
      *
     / \
    *   *
   / \ / \
  * * * *
```

Height:

TEXT

```
2
```

Formula:

TEXT

```
ceil(log2(8))-1
=3-1
=2
```

38. CHECK EXAMPLE — $n=8$

Seven nodes fit in height 2.

Need another level.

So:

TEXT

```
h_min = 3
```

Formula:

TEXT

```
ceil(log2(9))-1
"H   c e i l ( 3 . 1 7 ) - 1
=4-1
=3
```

Correct.

39. HEIGHT RANGE FOR N NODES

FORMULA BOX

TEXT

```
ceil(log2(n+1)) - 1
"d   h   "d
n - 1
```

under edge-based height.

$\Theta(\lg n)$ numerical gold.

40. NUMBER OF NODES RANGE FOR HEIGHT h

FORMULA BOX

TEXT

```
h + 1
"d   n   "d
2^(h+1) - 1
```

Again under edge-height convention.

41. DEGREE POSSIBILITIES IN BINARY TREE

Every node's child count is:

TEXT

0
1
2

So if:

- n_0 = number of nodes with 0 children
- n_1 = number of nodes with 1 child
- n_2 = number of nodes with 2 children

then:

TEXT

$$n = n_0 + n_1 + n_2$$

42. EDGE COUNT USING CHILD COUNTS

Each degree-1 node contributes:

TEXT

1 edge to child

Each degree-2 node:

TEXT

2

Leaves:

TEXT

0

So total edges:

TEXT

$$n_1 + 2n_2$$

But tree edges:

TEXT

$$n - 1$$

Therefore:

FORMULA BOX

TEXT

$$n_1 + 2n_2 = n - 1$$

Since:

TEXT
 $n = n_0 + n_1 + n_2$

we can derive a very important result.

43. LEAF / DEGREE-2 RELATION $n_0 = n_2 + 1$

Substitute:

TEXT
 $n_1 = 2n_2$
 $=$
 $n_0 + n_1 + n_2 - 1$

Cancel n_1 :

TEXT
 $2n_2 = n_0 + n_2 - 1$

Therefore:

TEXT
 $n_2 = n_0 - 1$

So:

FORMULA BOX

TEXT
 $n_0 = n_2 + 1$

Meaning:

In any non-empty binary tree, number of leaves equals number of nodes with two children plus 1.

$n_0 = n_2 + 1$ is a very important GATE theorem.

44. IMPORTANT — DOES NOT REQUIRE FULL BINARY TREE

The relation:

TEXT
 $n_0 = n_2 + 1$

holds even if nodes with one child exist.

Example:

TEXT



Counts:

TEXT

$n_0 = \text{leaves} = B, D = 2$
 $n_2 = \text{nodes with 2 children} = A = 1$

Indeed:

TEXT

$2 = 1+1$

Ø=No trap.

45. EXAMPLE NUMERICAL

Binary tree has:

TEXT

20 nodes with two children

How many leaves?

TEXT

$n_0 = n_2 + 1$
 $= 21$

Answer:

21

No need to know how many degree-1 nodes exist.

46. ANOTHER NUMERICAL

Binary tree has:

TEXT

35 leaves

Nodes with exactly 2 children:

TEXT

35-1
=34

47. EXTERNAL NULL LINKS CONNECTION

Recall:

TEXT

n+1 NULL child pointers

This is another binary-tree counting result.

Do not confuse:

TEXT

leaf nodes

with:

TEXT

NULL child links

They are different.

Example leaf has:

TEXT

2 NULL links

while a one-child node has:

TEXT

1 NULL link

48. BINARY TREE HEIGHT RECURSION PREVIEW

For a node:

TEXT

```
height(node)
=
1 + max(height(left), height(right))
```

If we use:

TEXT

height(NULL) = -1

then leaf:

TEXT

```
1+max(-1, -1)
=0
```

which matches edge-based convention.

$O(N)$ recurrence will be used heavily.

49. NUMBER OF NODES RECURSION PREVIEW

For root:

TEXT

```
nodes(root)
=
1
+ nodes(root->left)
+ nodes(root->right)
```

Base:

TEXT

```
nodes(NULL)=0
```

Tree recursion naturally splits into left and right subtrees.

50. COUNT LEAVES RECURSION PREVIEW

Conceptually:

TEXT

```
if node == NULL:
    0

if left == NULL and right == NULL:
    1

otherwise:
    leaves(left) + leaves(right)
```

We will use such patterns during tree coding.

51. WHY RECURSION FITS TREES

A binary tree has recursive structure:

TEXT

```
Tree =
Root
+
Left Binary Tree
+
```

Each subtree is itself a binary tree.

Therefore recursion is extremely natural.

~~Ø=∅~~ This connects directly to your recursion module.

52. RECURSIVE DEFINITION OF BINARY TREE

A binary tree is either:

1. empty, or
2. consists of:
 - a root
 - a left binary subtree
 - a right binary subtree

This is a mathematically clean definition.

53. GATE TRAP — TWO CHILDREN MAXIMUM

If one node has:

TEXT

3 children

the structure is not a binary tree, even if every other node has "d 2 children.

All nodes must satisfy the restriction.

54. GATE TRAP — DEGREE OF BINARY TREE

Maximum node degree in rooted-tree child sense:

TEXT

"d 2

A binary tree may have degree:

TEXT

0, 1, 2

depending on its nodes.

55. GATE TRAP — A BINARY TREE IS ORDERED

These are different:

TEXT

A
/
B

and:

TEXT

A
\
B

Even though the same values exist.

56. GATE TRAP — ARRAY INDEX FORMULA

Question says:

TEXT

root at index 1

Use:

TEXT

left=2i
right=2i+1

Question says:

TEXT

root at index 0

Use:

TEXT

left=2i+1
right=2i+2

Never guess.

57. GATE-STYLE QUESTION — ARRAY INDEX

0-based binary-tree array.

Node index:

TEXT

i=8

Left child:

TEXT

$$2(8)+1=17$$

Right:

TEXT

$$18$$

Parent:

TEXT

$$\begin{aligned} &\text{floor}((8-1)/2) \\ &= \text{floor}(7/2) \\ &= 3 \end{aligned}$$

58. GATE-STYLE QUESTION — 1-BASED

Node index:

TEXT

$$i=11$$

Then:

TEXT

$$\begin{aligned} \text{left} &= 22 \\ \text{right} &= 23 \\ \text{parent} &= \text{floor}(11/2) = 5 \end{aligned}$$

59. GATE-STYLE QUESTION — HEIGHT

Binary tree height:

TEXT

$$h=5$$

Maximum nodes:

TEXT

$$\begin{aligned} &2^6 - 1 \\ &= 63 \end{aligned}$$

Minimum nodes:

TEXT

$$5+1=6$$

So:

TEXT

6 "d n "d 6 3

60. GATE-STYLE QUESTION — MAX LEVEL NODES

Root level = 0.

Maximum number of nodes at level 6:

TEXT

$2^6 = 64$

61. GATE-STYLE QUESTION — MINIMUM HEIGHT

Binary tree has:

TEXT

$n=31$

Minimum height:

TEXT

$\text{ceil}(\log_2(32)) - 1$
 $= 5 - 1$
 $= 4$

Maximum height:

TEXT

30

62. GATE-STYLE QUESTION — $n=32$

Minimum height:

TEXT

$\text{ceil}(\log_2(33)) - 1$

Since:

TEXT

$2^5=32$
 $2^6=64$

TEXT

$\text{ceil}(\log_2 33)=6$

So:

```
TEXT
h_min=5
```

63. LINKED VS ARRAY REPRESENTATION

Property	Linked	Array
Memory allocation	Dynamic	Usually fixed/resizable array
Sparse tree	Efficient	May waste slots
Complete tree	Works	Excellent
Child access	pointers	index formula
Parent access	not automatic unless parent pointer	$O(1)$ by formula
Heap use	possible	preferred

$O(1)$ = important comparison.

64. PARENT ACCESS IN LINKED TREE

With structure:

```
C
struct Node
{
    int data;
    Node *left;
    Node *right;
};
```

a child does not know its parent directly.

To move to parent, we would need:

- extra parent pointer, or
- traversal/search from root, depending on context

Array representation often computes parent instantly by formula.

65. EXTRA PARENT POINTER

Some applications use:

```
C
typedef struct Node
{
    int data;
```

```

    struct Node *left;
    struct Node *right;
    struct Node *parent;
} Node;

```

Then moving upward becomes $O(1)$.

Trade-off:

TEXT
extra pointer memory

$O(1)$ placement/system-design extension.

66. BINARY TREE STORAGE COMPLEXITY

Linked representation with n nodes:

TEXT
 $O(n)$

space.

Array representation capacity depends on allocated array and shape.

For a complete tree with n nodes:

TEXT
 $O(n)$

compact storage.

For sparse tree, highest required index may greatly exceed n .

67. BINARY TREE OPERATION COMPLEXITIES — PREVIEW

There is no general ordering rule like BST yet.

Therefore:

Search arbitrary value

Worst case:

TEXT
 $O(n)$

because every node may need checking.

Traversal

TEXT

$O(n)$

Height computation

TEXT

$O(n)$

Count nodes

TEXT

$O(n)$

Ø=Ø Binary tree " BST.

68. DON'T CONFUSE BINARY TREE WITH BST

Binary tree only says:

TEXT

"d 2 c h i l d r e n

BST additionally says:

TEXT

left keys < node < right keys

under common distinct-key convention.

That ordering gives search benefits.

We'll study BST separately.

Ø=Ø Major GATE trap.

69. DON'T CONFUSE BINARY TREE WITH HEAP

Heap is:

TEXT

complete binary tree
+
heap-order property

Not every binary tree is a heap.

70. DON'T CONFUSE COMPLETE / FULL / PERFECT

These are specific binary-tree categories.

We intentionally study them properly in:

Lesson 52

Do not mix them yet.

71. CORE FORMULA SHEET

Tree edges

TEXT

$n-1$

Binary tree NULL links

TEXT

$n+1$

Maximum nodes at level L

TEXT

2^L

Maximum nodes for height h

TEXT

$2^{(h+1)} - 1$

Minimum nodes for height h

TEXT

$h+1$

Maximum leaves for height h

TEXT

2^h

Height range for n nodes

TEXT

$\text{ceil}(\log_2(n+1)) - 1$ "d h "d n - 1

Leaf/two-child relation

TEXT

$$n_0 = n_2 + 1$$

Copy this entire formula sheet.

72. INDEX FORMULA SHEET

1-Based

TEXT

Left = $2i$
Right = $2i+1$
Parent = $\text{floor}(i/2)$

0-Based

TEXT

Left = $2i+1$
Right = $2i+2$
Parent = $\text{floor}((i-1)/2)$

73. MEMORY TRICKS

Binary

At most TWO branches.

Ordered

Left and Right have identity.

Array 1-based

Double / Double+1.

Array 0-based

Double+1 / Double+2.

Height maximum nodes

Levels sum powers of two.

Leaves

74. 30-SECOND REVISION

Remember:

1. Binary tree has at most two children/node.
2. Left and right child are distinct.
3. C node uses `data, left, right`.
4. Empty tree has `root=NULL`.
5. Linked tree has `n+1` NULL child links.
6. 1-based children: $2i, 2i+1$.
7. 0-based children: $2i+1, 2i+2$.
8. Max nodes level $L = 2^L$.
9. Max nodes height $h = 2^{(h+1)} - 1$.
10. Leaves = nodes with two children + 1.

75. ACTIVE RECALL

Q1

Can a binary-tree node have exactly one child?

Q2

Are left-child and right-child positions interchangeable?

Q3

For 0-based index 5, find left and right child indices.

Q4

Binary tree has 12 nodes with exactly two children. How many leaves?

Q5

Height 4: maximum number of nodes?

ANSWERS

A1

Yes.

A2

No. Binary tree is ordered.

A3

TEXT

```
left = 11  
right = 12
```

A4

TEXT

13 leaves

A5

TEXT

```
2^5 - 1  
=31
```

76. PRACTICE — FORMULAS

A

Binary tree height = 6.

Find:

- minimum nodes
- maximum nodes
- maximum leaves

B

Binary tree contains 50 nodes.

Find:

- maximum possible height
- minimum possible height

C

Binary tree has 27 leaves.

How many nodes have exactly two children?

77. PRACTICE — ARRAY REPRESENTATION

0-based representation.

For node index:

TEXT

i=10

find:

- parent

- left child
- right child

Then solve same for 1-based representation with $i=10$.

78. GATE-STYLE QUESTION

A linked binary tree contains:

TEXT

100 nodes

How many child-pointer fields exist?

TEXT

$2n=200$

How many non-NULL child links?

TEXT

$n-1=99$

How many NULL child links?

TEXT

$200-99$
 $=101$

Answer:

101

79. GATE-STYLE QUESTION

Binary tree has:

TEXT

$n_0 = 18$ leaves

Number of nodes with exactly two children:

TEXT

$n_2 = n_0 - 1$
 $=17$

This holds regardless of the number of one-child nodes.

$\emptyset = \sqrt{0}$ is very important.

80. SYLLABUS CHECKPOINT

For the **Trees** portion, we have now completed:

' General tree terminology ' Binary tree definition ' Binary-tree C node ' Linked representation ' Array representation
' 0-/1-based indexing ' Height/node bounds ' Level-node formulas ' NULL-link formula ' Leaf/two-child relation

Still to come before Trees are marked complete:

- binary-tree types
- all important properties
- preorder
- inorder
- postorder
- level order
- recursive traversal behavior
- traversal reconstruction
- uniqueness conditions
- tree-related GATE traps

So nothing is being skipped.

PROGRESS

MODULE 6 — TREES

' Lesson 50 — Tree Fundamentals & Terminology ' **Lesson 51 — Binary Trees & Representations**

NEXT — LESSON 52

Types of Binary Trees — Full, Complete, Perfect, Balanced, Skewed & Degenerate

This is a **very high-value terminology lesson** because GATE frequently tests the differences between:

TEXT

F u l l " ` C o m p l e t e " ` P e r f e c t " ` B a l a n c e d

Next we will master each with diagrams, formulas, counterexamples, and GATE traps.

USER

next

ASSISTANT

MODULE 6 — LESSON 52

Types of Binary Trees — Full, Complete, Perfect, Balanced, Skewed & Degenerate

This lesson is very important because students often confuse:

Full "Complete" Perfect "Balanced

GATE loves terminology-based traps here.

GATE importance: 0=Ý%04V%0K%0W **Placement importance:** 0=Ý%0P%0TANT

1. WHERE WE ARE

Completed:

' Tree terminology ' Binary tree definition ' Linked representation ' Array representation ' Important node/height formulas

Now:

Types of Binary Trees

We will cover:

1. Full Binary Tree
2. Complete Binary Tree
3. Perfect Binary Tree
4. Balanced Binary Tree
5. Skewed Binary Tree
6. Degenerate Binary Tree
7. Strict / Proper Binary Tree terminology
8. Important relationships
9. GATE formulas and traps

2. FIRST — NORMAL BINARY TREE

Remember:

A binary tree only requires:

```text id="norm-bt" Every node has at most 2 children.

Example:

```
```text id="normal-bt"
  A
 / \
B   C
 \
  D
```

This is a valid binary tree.

It may or may not be:

- full
- complete
- perfect
- balanced

Those are extra properties.

3. FULL BINARY TREE $\emptyset = \hat{Y}\% \emptyset = \hat{Y}\% \emptyset = \hat{Y}\%$

A **Full Binary Tree** is a binary tree in which every node has:

```text id="full-rule" either 0 children or 2 children

No node is allowed to have exactly:

```
```text id="full-no"
1 child
```

4. FULL BINARY TREE EXAMPLE

```text id="full-example" A /\ B C /\ D E

Check:

```
```text id="full-check"
A !' 2 c h i l d r e n
B !' 0 c h i l d r e n
C !' 2 c h i l d r e n
D !' 0
E !' 0
```

So this is:

FULL '

5. NOT FULL

```text id="not-full" A /\ B C / D

Node C has:

```
```text id="not-full-c"
1 child
```

Therefore:

Not Full 'L

6. OTHER NAMES FOR FULL BINARY TREE

Depending on textbook, you may see:

- Full Binary Tree
- Strict Binary Tree
- Proper Binary Tree

These are commonly used to mean:

```text id="strict" every node has either 0 or 2 children

Ø=Ý% G A T E - s a f e n o t e :

> If the question defines the term, follow the question's definition.

---

# 7. FULL BINARY TREE PROPERTY

Let:

```text id="full-counts"

I = number of internal nodes

L = number of leaves

For a non-empty full binary tree:

FORMULA BOX

```text id="full-leaf-formula"  $L = I + 1$

Ø=Ý% V e r y i m p o r t a n t .

---

# 8. WHY  $L = I + 1$ ?

In a full binary tree every internal node has exactly 2 children.

So if internal nodes =  $I$ :

```text id="full-edges"

number of child edges = $2I$

But for any tree:

```text id="tree-edge" edges =  $n - 1$

and:

```text id="full-n"

$n = I + L$

Therefore:

```
```text id="full-proof" 2I = I + L - 1
```

So:

```
```text id="full-proof2"
L = I + 1
```

9. TOTAL NODES IN FULL BINARY TREE

Since:

```
```text id="full-total" n = I + L
```

and:

```
```text id="full-leaf"
L = I + 1
```

then:

```
```text id="full-total2" n = I + I + 1 = 2I + 1
```

So:

# FORMULA BOX

```
```text id="full-node-count"
n = 2I + 1
```

Therefore a non-empty full binary tree always has:

Odd number of nodes

0-0% favorite.

10. LEAVES USING TOTAL NODES

From:

```
```text id="full-node" n = 2I+1
```

we get:

```
```text id="full-i"
I = (n-1)/2
```

and:

```
```text id="full-l" L = I+1 = (n+1)/2
```

So for a full binary tree:

# FORMULA BOX

```
```text id="full-formulas"
Internal nodes = (n-1)/2
Leaves        = (n+1)/2
```

11. EXAMPLE

Full binary tree has:

```
```text id="full-num" n = 31 nodes
```

Internal:

```
```text id="full-num-i"
(31-1)/2 = 15
```

Leaves:

```
```text id="full-num-l" (31+1)/2 = 16
```

Check:

```
```text id="full-check-num"
16 = 15+1
```

Correct.

12. IMPORTANT — FULL DOES NOT MEAN ALL LEVELS FILLED

Example:

```
```text id="full-not-perfect" A/\ B C /\ D E /\ F G
```

Every internal node has exactly 2 children.

So:

```
```text id="full-answer"
F U L L
```

But levels are not completely filled.

Therefore:

```
```text id="full-vs-perfect" Full does NOT mean Perfect.
```

Ø=Ý%   H u g e   t r a p .

```

13. COMPLETE BINARY TREE 0=Ý%0=Ý%0=Ý%

A **Complete Binary Tree** satisfies:

1. Every level except possibly the last is completely filled.
2. Nodes in the last level are filled:

from LEFT to RIGHT

with no gaps in between.

14. COMPLETE EXAMPLE

```text id="complete-example"
      A
     /\
    B  C
   /\ /\
  D E F

```

Levels:

```

```text id="complete-levels" Level 0: A    full Level 1: B C    full Level 2: D E F    filled from left

```

```

Therefore:

* * C o m p l e t e ' * *

15. ANOTHER COMPLETE TREE

```text id="complete2"
      A
     /\
    B  C
   /\
  D

```

Last level contains:

```

```text id="complete-left" D

```

```

at the leftmost available position.

Therefore:

* * C o m p l e t e ' * *

16. NOT COMPLETE – GAP

```text id="not-complete"
      A
     /\
    B  C

```

\
D

At last level:

B has right child D but left position is missing.

That creates a gap before an occupied position.

Therefore:

Not Complete 'L

17. NOT COMPLETE — RIGHT SUBTREE TOO EARLY

```
```text id="not-complete2" A /\ B C / D
```

At the final level, earlier positions under B are empty but D appears later under C.

Therefore:

```
* * N o t C o m p l e t e 'L * *
```

---

```
18. COMPLETE TREE MEMORY TRICK
```

```
> **All upper floors full; last floor fills from LEFT.**
```

```
Ø=Ý% V e r y e a s y m n e m o n i c .
```

---

```
19. WHY COMPLETE TREES ARE IMPORTANT
```

Complete binary trees have very compact array representation.

Example:

```
```text id="complete-array-tree"
```

```
  A
 / \
B   C
/\  /\
D E F
```

0-based array:

```
```text id="complete-array" A B C D E F
```

No internal holes.

That is why:

```
Binary heaps use complete binary trees.
```

```
Ø=Ý% M u s t k n o w .
```

---

## # 20. COMPLETE TREE ARRAY PROPERTY

If a complete binary tree has  $n$  nodes stored 0-based:

```
```text id="complete-index"
valid indices = 0 ... n-1
```

There are no gaps in the occupied array positions.

This becomes extremely important for heaps.

21. PERFECT BINARY TREE

A **Perfect Binary Tree** has:

1. Every internal node has exactly 2 children.
2. All leaves are at the same depth/level.

Equivalently:

Every level is completely filled.

22. PERFECT EXAMPLE

```
```text id="perfect-example" A / \ B C / \ / \ D E F G
```

Every internal node has 2 children.

All leaves:

```
```text id="perfect-leaves"
D E F G
```

are at the same level.

Therefore:

Perfect

23. PERFECT TREE FORMULAS

If height is h , with root height convention based on edges:

Maximum—and exact—number of nodes:

FORMULA BOX

```
```text id="perfect-nodes" n = 2^(h+1) - 1
```

Leaves:

```
```text id="perfect-leaves-form"
L = 2^h
```

Internal nodes:

```
```text id="perfect-internal" I = 2^h - 1
```

---

# 24. EXAMPLE

Perfect tree height:

```
```text id="perfect-h"
h = 4
```

Nodes:

```
```text id="perfect-n" 2^5 - 1 = 31
```

Leaves:

```
```text id="perfect-l"
2^4 = 16
```

Internal:

```
```text id="perfect-i" 16 - 1 = 15
```

Check:

```
```text id="perfect-check"
15 + 16 = 31
```

25. RELATIONSHIP — PERFECT ! $\Rightarrow$ FULL

Every perfect tree has internal nodes with:

```
```text id="perfect-full" exactly 2 children
```

So:

```
* * P e r f e c t ! $\Rightarrow$ F u l l * *
```

---

```
2 6 . P E R F E C T ! $\Rightarrow$ C O M P L E T E
```

Every level is fully filled.

Therefore it automatically satisfies complete-tree rules.



So:

```
* * P e r f e c t ! ò C o m p l e t e * *
```

---

# 27. BUT FULL DOES NOT IMPLY PERFECT

Example:

```
```text id="full-not-perfect2"
```



Full:

```
```text id="yes" YES
```

Perfect:

```
```text id="no"
NO
```

because leaves B,D,E are not all at same depth.

28. COMPLETE DOES NOT IMPLY PERFECT

```
```text id="complete-not-perfect" A /\ B C /\ D E
```

Complete:

```
```text id="yes2"
YES
```

Perfect:

```
```text id="no2" NO
```

because final level is not full.

---

# 29. CAN A COMPLETE TREE BE FULL BUT NOT PERFECT?

Yes.

Example:

```
```text id="complete-full-not-perfect"
```



Check:

A !' 2 children B !' 2 children C !' 0 D,E !' 0

So:

```
""text id=""refill Full L Complete L Perfect L
```

text id= clip Full Complete Perfect L

Ø=Ý% Important relationship example .

```
# 30. BALANCED BINARY TREE 0=Yes0=Yes0=Yes
```

The word **balanced** can depend on the precise tree structure or textbook.

General intuition:

- > The left and right subtree heights are kept reasonably close so the tree does not become

> The left and right subtree heights are kept reasonably close so the tree does not become excessively skewed.

For AVL trees specifically:

```
text id="avl-balance"
| height ( left ) - height ( right ) | "d 1
```

at every node.

But not every use of the word "balanced binary tree" necessarily means AVL.

Ø = 0% TE-safe rule:

Follow the balance definition given in the question.

31. BALANCED INTUITION

Balanced-looking:

```
```text id="balanced" A /\ B  C /  \ D      E
```

Left subtree height and right subtree height are similar

Height is roughly:

```
```text id="balanced-height"
O(log n)
```

for many standard balanced search trees.

32. UNBALANCED TREE

```
```text id="unbalanced" A \ B \ C \ D \ E
```

Height:

```
```text id="unbalanced-height"
n - 1
```

This behaves like a linked list.

33. WHY BALANCE MATTERS

In a BST, operations depend on tree height `h`.

Typical:

```
```text id="bst-h" Search O(h) Insert O(h) Delete O(h)
```

If balanced:

```
```text id="balanced-bst"
h = O(log n)
```

operations can be:

```
```text id="balanced-bst-complexity" O(log n)
```

If skewed:

```
```text id="skew-bst"
h = O(n)
```

operations can degrade to:

```
```text id="skew-bst-complexity" O(n)
```

0=Ý% This will be crucial in BST module .

---

# 34. COMPLETE VS BALANCED

A complete tree is strongly shape-controlled.

It is therefore height-efficient.

A complete binary tree with `n` nodes has height:

```
```text id="complete-height"
floor(log2 n)
```

under edge-height convention.

But "balanced" is a broader concept.

0=00% Don't treat the words as synonyms.

35. SKEWED BINARY TREE Ø=Ý%

A skewed binary tree is one in which each node has only one child, forming a chain.

Two forms:

Left-skewed

Right-skewed

36. LEFT-SKEWED

```
```text id="left-skew" A / B / C / D
```

Every non-leaf has only:

```
```text id="left-child-only"
left child
```

37. RIGHT-SKEWED

```
```text id="right-skew" A \ B \ C \ D
```

Every non-leaf has only:

```
```text id="right-child-only"
right child
```

38. HEIGHT OF SKEWED TREE

If there are:

```
```text id="skew-n" n nodes
```

height:

# FORMULA BOX

```
```text id="skew-height"
h = n - 1
```

Maximum possible height for a binary tree.

39. SKEWED TREE SPACE/SEARCH CONSEQUENCE

In a skewed BST:

```text id="skew-search" search can become  $O(n)$

instead of  $O(\log n)$ .

So tree shape matters enormously.

---

#### # 40. DEGENERATE BINARY TREE

A degenerate tree is commonly one where every internal node has exactly one child.

Thus its shape resembles:

```text id="degenerate"

```
A
 \
  B
 /
C
 \
 D
```

The direction need not always be the same.

It still forms a single chain.

41. SKEWED VS DEGENERATE

A skewed tree is a special form of degenerate tree where all nodes lean consistently:

```text id="skew-direction" all left

or:

```text id="skew-direction2"  
all right

Degenerate tree may alternate directions:

```text id="degenerate-alt" A / B \ C / D

Still each internal node has only one child.

Ø=Ý% Terminology can vary slightly across books, but this

---

#### # 42. PERFECT TREE HEIGHT FOR N NODES

Perfect tree:

```text id="perfect-n-h"

$n = 2^{(h+1)} - 1$

Therefore:

```text id="perfect-h-n"  $n+1 = 2^{(h+1)}$

So:

# FORMULA BOX

```
```text id="perfect-height"
h = log2(n+1) - 1
```

provided `n` is exactly of perfect-tree form.

43. PERFECT TREE NODE COUNTS

Possible total-node counts:

```
```text id="perfect-counts" 1, 3, 7, 15, 31, 63, ...
```

Because:

```
```text id="perfect-series"
2^(h+1)-1
```

0=1% question says "perfect binary tree with 20 nodes", that is impossible.

44. FULL TREE NODE COUNTS

Full tree total nodes are always odd:

```
```text id="full-odd" 1,3,5,7,9,...
```

But not every odd-node full tree is perfect.

Example full tree with 5 nodes:

```
```text id="full-five"
  A
 / \
B   C
 / \
D   E
```

Full but not perfect.

45. COMPLETE TREE HEIGHT

For a complete binary tree with `n` nodes:

FORMULA BOX

```
```text id="complete-height-formula" h = floor(log2 n)
```

under edge-based height.

---

# 46. CHECK COMPLETE HEIGHT

``n = 7``

````text id="complete7"  
floor(log2 7)=2`

Tree can have height 2.

`n = 8`

````text id="complete8" floor(log2 8)=3`

Need level 3.

Correct.

---

# 47. NODE RANGE FOR COMPLETE TREE OF HEIGHT h

For a complete tree of height ``h``:

All levels ``0...h-1`` are full.

That gives:

````text id="complete-min-base"  
2^h - 1`

nodes before final level.

Final level has at least:

````text id="complete-min-last" 1`

node if height truly equals ``h``.

Therefore minimum:

````text id="complete-min"  
2^h`

Maximum:

````text id="complete-max" 2^(h+1)-1`

So:

# FORMULA BOX

````text id="complete-range"  
2 ^ h "d n "d 2 ^ (h + 1) - 1`

for complete tree of height h .

48. PERFECT TREE IS EXTREME COMPLETE TREE

For a complete tree of height h :

```text id="complete-last" last level partially or completely filled

When last level becomes completely full:

```text id="perfect-result"  
tree becomes perfect

Nice conceptual relation.

49. COMPLETE TREE LEAF LOCATIONS

In a complete tree, leaves occur only in:

```text id="complete-leaf-levels" last level and possibly second-last level

Why?

All earlier levels must have their child positions populated to preserve completeness.

$\emptyset = \emptyset$  Useful GATE property.

---

# 50. ARRAY INDEX PROPERTY FOR COMPLETE TREE

For 0-based complete tree with  $n$  nodes:

Nodes with at least one child are approximately among early indices.

A node at index  $i$  has left child if:

```text id="left-exists"  
 $2i+1 < n$

Right child exists if:

```text id="right-exists"  $2i+2 < n$

$\emptyset = \emptyset$  Important for heap implementation.

---

# 51. LEAF INDICES IN COMPLETE TREE / HEAP  $\emptyset = \emptyset$

0-based array with  $n$  nodes:

Last internal node:

```text id="last-internal"



```
floor(n/2) - 1
```

Leaves begin at:

```
```text id="leaf-start" floor(n/2)
```

and continue through:

```
```text id="leaf-end"
n - 1
```

Very important heap formula.

52. WHY?

Node `i` has a left child if:

```
```text id="left-cond" 2i+1 < n
```

Solve:

```
```text id="left-cond2"
i < (n-1)/2
```

So internal indices are:

```
```text id="internal-indices" 0 ... floor(n/2)-1
```

Leaves:

```
```text id="leaves-indices"
floor(n/2) ... n-1
```

53. NUMBER OF LEAVES IN COMPLETE BINARY TREE

Since leaf indices are:

```
```text id="leaf-range" floor(n/2) ... n-1
```

number of leaves:

# FORMULA BOX

```
```text id="complete-leaf-count"
ceil(n/2)
```

Number of internal nodes:

```
```text id="complete-internal-count" floor(n/2)
```

for a non-empty complete binary tree.

Ø=Ý%   V e r y   u s e f u l   f o r   h e a p s .

---

# 54. EXAMPLE

Complete tree:

```
```text id="complete-n10"
n = 10
```

Internal nodes:

```
```text id="complete-i10" floor(10/2)=5
```

Leaves:

```
```text id="complete-l10"
ceil(10/2)=5
```

0-based leaves:

```
```text id="complete-leaf-index10" 5,6,7,8,9
```

---

# 55. EXAMPLE WITH ODD N

```
```text id="complete-n7"
n = 7
```

Internal:

```
```text id="complete-i7" floor(7/2)=3
```

Leaves:

```
```text id="complete-l7"
ceil(7/2)=4
```

Indices:

```
```text id="complete-leaves7" 3,4,5,6
```

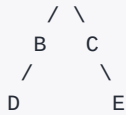
Correct.

---

# 56. FULL VS COMPLETE EXAMPLE

Tree:

```
```text id="full-vs-complete"
A
```



Check Full:

B has exactly 1 child.

Therefore:

```
```text id="full-no2" Not Full
```

Check Complete:

Final level D,E has a gap between positions.

Therefore:

```
```text id="complete-no2"
Not Complete
```

Yet still a valid binary tree.

57. COMPLETE BUT NOT FULL

```
```text id="complete-not-full" A /\ B  C / D
```

Complete:

```
```text id="complete-yes3"
Yes
```

because last level is filled from left.

Full:

```
```text id="full-no3" No
```

because B has exactly one child.

Ø=¥%    S o   c o m p l e t e   d o e s   n o t   i m p l y   f u l l .

---

# 58. FULL BUT NOT COMPLETE

```
```text id="full-not-complete"
```



Full:

```
```text id="full-yes4" Yes
```

Complete:

```
```text id="complete-no4"
No
```

because last-level nodes occur under C while earlier positions under B are empty.

$\emptyset \neq \text{full}$ does not imply complete.

59. PERFECT IS BOTH

A perfect tree is:

```
```text id="perfect-both" Full ' Complete '
```

and all leaves have same depth.

So:

# FORMULA BOX

```
```text id="relation-perfect"
Perfect ! $\emptyset$  Full
Perfect ! $\emptyset$  Complete
```

But reverse implications are false.

60. BALANCED VS PERFECT

Every perfect tree is perfectly height-balanced.

So intuitively:

```
```text id="perfect-balanced" Perfect ! $\emptyset$ Balanced
```

for usual height-balance notions.

But a balanced tree need not be:

- perfect
- complete
- full

Example:

```
```text id="balanced-not-perfect"
      A
     /\
    B  C
   /\
  D
```

Depending on the standard height-balance rule:

- root subtrees differ by 1

- B subtrees differ by 1

so it can be balanced.

But:

```
```text id="balanced-properties" Perfect 'LFull 'L
```

It happens to be complete in this example.

---

# 61. MASTER RELATIONSHIP DIAGRAM

Think:

```
```text id="relation"
  Perfect
    /  \
  Full  Complete
```

Perfect belongs to both categories.

But Full and Complete overlap only partially.

Balanced is a different shape concept that may overlap with them.

Do not imagine a simple single hierarchy for all categories.

62. GATE TRAP — FULL "MAXIMUM NODES

A full tree does not necessarily contain maximum possible nodes for its height.

Example:

```
```text id="full-height3" A /\ B  C /\ D  E /\ F  G
```

Height = 3.

Maximum nodes for height 3:

```
```text id="max-height3"
15
```

Actual nodes:

```
```text id="actual7" 7
```

Still full.

---

# 63. GATE TRAP — COMPLETE TREE CAN HAVE ONE-CHILD NODE

Yes.

Example:

```
```text id="complete-one-child"
```



B has only a left child.

Tree is complete.

In a complete binary tree, at most one internal node can have exactly one child, and if it exists that child must be a **left child** near the end of level-order filling.

$\emptyset = 1$ node-value property.

64. WHY ONLY LEFT CHILD?

If a node had:

```
```text id="right-only" right child but no left child
```

there would be an earlier empty slot before an occupied slot.

That violates completeness.

---

# 65. COMPLETE TREE DEGREE-1 NODE PROPERTY

A complete binary tree can have:

```
```text id="degree1-count"
0 or 1 nodes with exactly one child
```

and that one child, if present, is:

```
```text id="left-child" LEFT
```

$\emptyset = 1$  Good GATE fact .

---

# 66. GATE-STYLE QUESTION – FULL TREE

A full binary tree contains:

```
```text id="full-i20"
20 internal nodes
```

Leaves:

```
```text id="full-l21" 21
```

Total nodes:

```
```text id="full-total41"
```

67. GATE-STYLE QUESTION — FULL NODE COUNT

Can a full binary tree have:

```text id="full-even" 100 nodes?

No.

A non-empty full binary tree has:

```text id="full-odd2"  
odd total nodes

because:

```text id="full-2i1"  $n=2l+1$

---

# 68. GATE-STYLE QUESTION — PERFECT

Perfect tree height:

```text id="perf-h5"  
5

Leaves:

```text id="perf-l32"  $2^5 = 32$

Total:

```text id="perf-n63"  
 $2^6 - 1 = 63$

Internal:

```text id="perf-i31" 31

---

# 69. GATE-STYLE QUESTION — COMPLETE

Complete binary tree:

```text id="complete-n50"  
 $n=50$

Height:

```text id="complete-h50"  $\text{floor}(\log_2 50)$

Since:

```
```text id="powers"
2^5=32
2^6=64
```

height:

```
```text id="answer5" 5
```

---

# 70. GATE-STYLE QUESTION – COMPLETE LEAVES

Complete tree has:

```
```text id="complete-n11"
n=11
```

Leaves:

```
```text id="complete-leaves11" ceil(11/2)=6
```

Internal:

```
```text id="complete-internal11"
floor(11/2)=5
```

71. GATE-STYLE ARRAY QUESTION

Complete tree stored 0-based with:

```
```text id="array-n15" n=15
```

Last internal node:

```
```text id="array-last-int"
floor(15/2)-1
=7-1
=6
```

Leaves:

```
```text id="array-leaves" indices 7 through 14
```

Number:

```
```text id="array-leaf-num"
8
```


72. GATE-STYLE IDENTIFICATION

Tree:

```
```text id="identify" A /\ B C /\ / D E F
```

### Full?

C has one child.

```
```text id="id-full"
No
```

Complete?

Last level:

```
```text id="id-last" D E F
```

filled from left.

```
```text id="id-complete"
Yes
```

Perfect?

Last level not full.

```
```text id="id-perfect" No
```

---

# 73. ANOTHER IDENTIFICATION

```
```text id="identify2"
```

```
  A
 /\ 
B   C
 /\ /\ 
D E F G
```

Full?

```
```text id="all-yes1" Yes
```

Complete?

```
```text id="all-yes2"
Yes
```

Perfect?

```
```text id="all-yes3" Yes
```

Balanced?

```
```text id="all-yes4"
Yes
```

74. ANOTHER IDENTIFICATION

```
```text id="identify3" A \ B \ C
```

Binary?

```
```text id="binaryyes"
Yes
```

Full?

```
```text id="fullno" No
```

Complete?

```
```text id="completeno"
No
```

Perfect?

```
```text id="perfectno" No
```

Skewed?

```
```text id="skewyes"
Yes – right-skewed
```

Degenerate?

```
```text id="degyes" Yes
```

---

# 75. MASTER COMPARISON TABLE

| Type       | Main Rule                                |
|------------|------------------------------------------|
| Binary     | At most 2 children                       |
| Full       | Every node has 0 or 2 children           |
| Complete   | All upper levels full; last filled left! |
| Perfect    | Every level completely full              |
| Balanced   | Subtree heights kept close               |
| Skewed     | Chain leaning left or right              |
| Degenerate | Every internal node has one child        |

Ø=Ý% Copy this table.

---

# 76. FORMULA SHEET

```
Full Binary Tree
```

```
```text id="fs1"  
L = I + 1
```

```
```text id="fs2" n = 2I + 1
```

```
```text id="fs3"  
L = (n+1)/2
```

Perfect Binary Tree

```
```text id="fs4" n = 2^(h+1)-1
```

```
```text id="fs5"  
L = 2^h
```

```
```text id="fs6" I = 2^h-1
```

```
Complete Binary Tree
```

```
```text id="fs7"  
h = floor(log2 n)
```

```
```text id="fs8" internal = floor(n/2)
```

```
```text id="fs9"  
leaves = ceil(n/2)
```

0-based:

```
```text id="fs10" last internal = floor(n/2)-1
```

```
```text id="fs11"  
leaves = floor(n/2) ... n-1
```

77. MEMORY TRICKS

Full

0 or 2 — never 1.

Complete

Fill floors, then left seats first.

Perfect

Nothing missing anywhere.

Balanced

Neither side grows too far away.

Skewed

Tree becomes a stick.

Degenerate

Every parent has only one road forward.

78. DON'T CONFUSE

Full

Concerned with:

```text id="dc1" number of children per node

### Complete

Concerned with:

```text id="dc2"  
position/filling order of nodes

Perfect

Concerned with:

```text id="dc3" all levels completely filled

### Balanced

Concerned mainly with:

```text id="dc4"  
heights/shapes of subtrees

Ø = 1. This distinction solves most terminology questions.

79. 30-SECOND REVISION

Remember:

1. Full !' every node has 0 or 2 children.
 2. Full tree: leaves = internal + 1.
 3. Full tree has odd number of nodes.
 4. Complete !' last level filled left to right.
 5. Perfect !' all levels full.
 6. Perfect is both full and complete.
 7. Complete does not imply full.
 8. Full does not imply complete.
 9. Skewed height = $\lceil \log_2 n \rceil$.
 10. Complete tree height = $\lceil \log_2 n \rceil$.
-

80. ACTIVE RECALL

Q1

Can a full binary tree contain a node with one child?

Q2

Can a complete tree contain a node with one child?

Q3

Does every perfect tree have to be complete?

Q4

Full tree has 14 internal nodes. Leaves?

Q5

Complete tree has 20 nodes. How many leaves?

ANSWERS

A1

No.

A2

Yes, at most one such internal node, with a left child under standard complete-tree shape.

A3

Yes.

A4

15

A5

15

$\text{ceil}(20/2)=10$

81. PRACTICE — IDENTIFY

For each tree determine:

Full / Complete / Perfect / Skewed

Tree A

``text id="prac-a" A /\ B C

Tree B

``text id="prac-b"

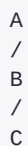


Tree C

``text id="prac-c" A /\ B C /\ D E

Tree D

``text id="prac-d"



82. PRACTICE — NUMERICAL

A

A full binary tree has 51 total nodes.

Find:

- internal nodes
- leaves

B

A perfect binary tree has height 6.

Find:

- nodes
- leaves
- internal nodes

C

A complete tree contains 100 nodes.

Find:

- height
 - number of leaves
 - last internal index using 0-based array
-

83. SYLLABUS CHECKPOINT

Tree module now includes:

' Tree terminology ' Binary tree definition ' C representation ' Linked representation ' Array representation ' Index formulas ' Height/node bounds ' Full binary tree ' Complete binary tree ' Perfect binary tree ' Balanced tree concept ' Skewed tree ' Degenerate tree ' Full/complete/perfect formulas and relationships

Still pending before Trees are marked complete:

- tree properties consolidation
- preorder traversal
- inorder traversal
- postorder traversal
- level-order traversal
- recursion and traversal call behavior
- iterative traversal intuition
- reconstruction from traversals
- uniqueness conditions
- traversal-related GATE questions

Nothing is being skipped.

PROGRESS

MODULE 6 — TREES

' Lesson 50 — Tree Terminology ' Lesson 51 — Binary Trees & Representations ' **Lesson 52 — Binary Tree Types**

NEXT — LESSON 53

Binary Tree Properties & GATE Formula Masterclass

Next we consolidate and derive the high-value numerical properties:

- nodes vs height
 - leaves vs internal nodes
 - degree-count equations
 - NULL pointer count
 - complete/perfect/full formulas
-

- leaf/internal index formulas
- minimum/maximum height
- maximum width
- path/level properties
- mixed GATE numericals

Then we begin traversals:

Lesson 54 — Preorder Traversal.

USER

next

ASSISTANT

MODULE 6 — LESSON 53

Binary Tree Properties & GATE Formula Masterclass

This lesson consolidates the **numerical side of Binary Trees**.

Instead of memorizing disconnected formulas, we will derive them so you can reconstruct them during GATE if you forget.

GATE importance: 0=1% 2=4% 3=10% 4=15% 5=20% 6=25% 7=30% 8=35% 9=40% 10=45% 11=50% 12=55% 13=60% 14=65% 15=70% 16=75% 17=80% 18=85% 19=90% 20=95% 21=100% **Placement importance:** +100%

1. WHY THIS LESSON MATTERS

Binary-tree questions frequently give information such as:

TEXT

```
height = 5
leaves = 20
internal nodes = 19
nodes with one child = 7
```

and ask you to find:

- total nodes
- edges
- leaves
- internal nodes
- maximum/minimum height
- number of NULL pointers
- possible/impossible tree
- array indices
- full/perfect/complete tree properties

This lesson builds the formula sheet needed for all of them.

2. MASTER NOTATION

We will use:

TEXT

n = total number of nodes

n_0 = nodes with 0 children

n_1 = nodes with exactly 1 child

n_2 = nodes with exactly 2 children

h = height of tree

Therefore:

FORMULA BOX

TEXT

$n = n_0 + n_1 + n_2$

~~0-100~~ Keep this notation throughout tree questions.

3. PROPERTY 1 — NUMBER OF EDGES

Any non-empty tree with:

TEXT

n nodes

has:

FORMULA BOX

TEXT

Edges = $n - 1$

Example:

TEXT

$n = 100$

Then:

TEXT

edges = 99

This is true for:

- binary trees
- BSTs

- heaps
- general trees

as long as the structure is a tree.

4. WHY $n-1$?

Every node except the root has exactly:

TEXT

one parent

So the number of parent-child connections is:

TEXT

number of non-root nodes = $n-1$

Each connection is one edge.

5. PROPERTY 2 — CHILD COUNT EQUATION $\sum_{i=1}^n \text{children}(i) = n-1$

Every node with:

TEXT

1 child

contributes one child edge.

Every node with:

TEXT

2 children

contributes two child edges.

Leaves contribute:

TEXT

0

Therefore number of edges is:

TEXT

$n_1 + 2n_2$

But a tree has:

TEXT

$n-1$

edges.

Hence:

FORMULA BOX

TEXT

$$n_1 + 2n_2 = n - 1$$

Extremely useful.

6. DERIVE THE MOST IMPORTANT BINARY-TREE PROPERTY

We know:

TEXT

$$n = n_0 + n_1 + n_2$$

and:

TEXT

$$n_1 + 2n_2 = n - 1$$

Substitute:

TEXT

$$\begin{aligned} n_1 + 2n_2 &= \\ n_0 + n_1 + n_2 - 1 \end{aligned}$$

Cancel n_1 :

TEXT

$$2n_2 = n_0 + n_2 - 1$$

Therefore:

TEXT

$$n_2 = n_0 - 1$$

So:

FORMULA BOX

TEXT

$$n_0 = n_2 + 1$$

Leaves = nodes with exactly 2 children + 1

YOU MUST KNOW

7. IMPORTANT — **n1** DOES NOT MATTER

This relation:

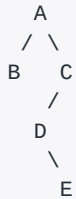
TEXT

$$n_0 = n_2 + 1$$

holds regardless of how many nodes have exactly one child.

Example:

TEXT



Here:

TEXT

Leaves:

$$B, E \quad !' \quad n_0 = 2$$

Two-child nodes:

$$A \quad !' \quad n_2 = 1$$

Indeed:

TEXT

$$2 = 1 + 1$$

even though one-child nodes exist.

8. GATE QUICK QUESTION

Binary tree contains:

TEXT

40 nodes with exactly 2 children

Number of leaves:

TEXT

$$40 + 1 = 41$$

No other information required.

9. REVERSE QUESTION

Binary tree contains:

TEXT

51 leaves

Then nodes with exactly two children:

TEXT

$51 - 1 = 50$

10. FIND TOTAL NODES USING n_0, n_1, n_2

Suppose:

TEXT

$n_1 = 10$

$n_2 = 20$

Then:

TEXT

$n_0 = n_2 + 1$

$= 21$

Total:

TEXT

$n = 21 + 10 + 20$

$= 51$

Answer:

51

11. ANOTHER USEFUL TOTAL-NODE FORMULA

Because:

TEXT

$n_0 = n_2 + 1$

then:

TEXT

$n = n_0 + n_1 + n_2$

becomes:

TEXT

$$n = (n_2 + 1) + n_1 + n_2$$

Therefore:

FORMULA BOX

TEXT

$$n = 2n_2 + n_1 + 1$$

Useful when n_1 and n_2 are given.

12. FIND n_1

Rearrange:

TEXT

$$n = 2n_2 + n_1 + 1$$

So:

FORMULA BOX

TEXT

$$n_1 = n - 2n_2 - 1$$

Example:

TEXT

$$\begin{aligned} n &= 30 \\ n_2 &= 10 \end{aligned}$$

Then:

TEXT

$$\begin{aligned} n_1 &= 30 - 2 \cdot 10 - 1 \\ &= 9 \end{aligned}$$

Leaves:

TEXT

$$n_0 = 11$$

Check:

TEXT

$$11 + 9 + 10 = 30$$

13. PROPERTY 3 — NULL CHILD POINTERS $\emptyset = \text{NULL}$

In linked binary tree:

```
C
struct Node
{
    int data;
    struct Node *left;
    struct Node *right;
};
```

Each node contains:

TEXT
2 child pointers

For n nodes:

TEXT
total child-pointer fields = $2n$

14. NON-NULL CHILD POINTERS

Every edge corresponds to one non-NULL child pointer.

Edges:

TEXT
 $n-1$

Therefore:

TEXT
non-NULL child pointers = $n-1$

15. NULL POINTER FORMULA

Total:

TEXT
 $2n$

Non-NULL:

TEXT
 $n-1$

NULL:

TEXT

$2n - (n - 1)$

Therefore:

FORMULA BOX

TEXT

NULL child pointers = $n + 1$

0 = 0% Classic GATE property.

16. EXAMPLE

Binary tree:

TEXT

$n = 25$

Total child pointers:

TEXT

50

Non-NULL:

TEXT

24

NULL:

TEXT

26

which is:

TEXT

$n + 1$

17. DON'T CONFUSE

For $n = 25$:

TEXT

Leaves = 26

26 is:

TEXT

NULL child pointers

Leaf count depends on shape.

$\emptyset \neq \text{NULL}$ links and leaves are completely different.

18. PROPERTY 4 — MAXIMUM NODES AT LEVEL L

Assume root level:

TEXT

0

Maximum nodes at level L :

FORMULA BOX

TEXT

2^L

Table:

| Level | Maximum nodes |
|-------|---------------|
| 0 | 1 |
| 1 | 2 |
| 2 | 4 |
| 3 | 8 |
| 4 | 16 |
| 5 | 32 |

19. WHY?

Every node has at most two children.

Therefore the maximum doubles each level:

TEXT

1 !' 2 !' 4 !' 8 !' 16 . . .

20. GATE QUESTION

Maximum number of nodes possible at level:

TEXT

$L=10$

is:

TEXT

$2^{10} = 1024$

21. PROPERTY 5 — MAXIMUM NODES FOR HEIGHT h

Using edge-height convention:

TEXT

root at level 0
deepest level = h

Maximum nodes occur when every level is completely filled:

TEXT

$1 + 2 + 4 + \dots + 2^h$

This geometric series equals:

FORMULA BOX

TEXT

Maximum nodes = $2^{(h+1)} - 1$

22. EXAMPLE

Height:

TEXT

$h=4$

Maximum:

TEXT

$2^5 - 1$
 $= 31$

23. MAXIMUM LEAVES FOR HEIGHT h

Maximum leaves occur when deepest level is completely filled.

Deepest level:

TEXT

h

Maximum nodes there:

TEXT

2^h

Therefore:

FORMULA BOX

TEXT

Maximum leaves = 2^h

24. EXAMPLE

Height:

TEXT

$h=6$

Maximum leaves:

TEXT

$2^6=64$

Maximum total nodes:

TEXT

$2^7-1=127$

25. PROPERTY 6 — MINIMUM NODES FOR HEIGHT h

To have height h , we need at least one path containing:

TEXT

h edges

A path of h edges contains:

TEXT

$h+1$ nodes

So:

FORMULA BOX

TEXT

Minimum nodes = $h+1$

achieved by a skewed/degenerate tree.

26. EXAMPLE

Height:

TEXT

$h=7$

Minimum nodes:

TEXT

8

Maximum nodes:

TEXT

$2^8 - 1 = 255$

Thus:

TEXT

8 "d n "d 2 5 5

27. MASTER NODE RANGE FOR GIVEN HEIGHT

FORMULA BOX

For any binary tree of edge-height h :

TEXT

$h + 1$ "d n "d $2^{h+1} - 1$

Very useful.

28. PROPERTY 7 — MAXIMUM HEIGHT FOR n NODES

Make tree completely skewed:

TEXT

A
\



With n nodes:

TEXT

edges on longest path = $n-1$

So:

FORMULA BOX

TEXT

$h_{\max} = n-1$

29. PROPERTY 8 — MINIMUM HEIGHT FOR n NODES

Pack nodes as densely as possible.

Maximum nodes that fit into height h :

TEXT

$2^{h+1} - 1$

Need:

TEXT

$n \leq 2^{h+1} - 1$

Therefore:

FORMULA BOX

TEXT

$h_{\min} = \lceil \log_2(n+1) \rceil - 1$

For positive integer n , equivalent to:

TEXT

$h_{\min} = \lfloor \log_2 n \rfloor$

30. CHECK — $n=15$

TEXT

$\text{floor}(\log_2 15)=3$

Indeed height 3 can hold:

TEXT

15

nodes.

31. CHECK — $n=16$

Height 3 holds maximum:

TEXT

15

So height must become:

TEXT

4

Formula:

TEXT

$\text{floor}(\log_2 16)=4$

Correct.

32. HEIGHT RANGE FOR n NODES

FORMULA BOX

TEXT

$\text{floor}(\log_2 n)$ "d h "d $n - 1$

for n using edge-based height.

Equivalent lower expression:

TEXT

$\text{ceil}(\log_2(n+1))-1$

33. GATE EXAMPLE

Binary tree has:

TEXT

$n=100$

Minimum height:

Since:

TEXT

$2^6=64$

$2^7=128$

TEXT

$\text{floor}(\log_2 100)=6$

Maximum height:

TEXT

99

Therefore:

TEXT

6 "d h "d 9 9

34. WIDTH OF A TREE

Width of a level:

Number of nodes present at that level.

Width of a tree:

Maximum number of nodes on any level.

Example:

TEXT



Widths:

TEXT

Level 0 = 1

Level 1 = 2

Level 2 = 4

Tree width:

TEXT
4

35. MAXIMUM WIDTH FOR HEIGHT h

No level can contain more than:

TEXT
 2^L

nodes.

Deepest possible level h can contain:

TEXT
 2^h

Therefore maximum possible tree width for height h :

FORMULA BOX

TEXT
 2^h

achieved in a perfect tree.

36. FULL BINARY TREE RECAP $\emptyset = \forall$

Full binary tree:

TEXT
every node has 0 or 2 children

Therefore:

TEXT
 $n_1 = 0$

General binary relation:

TEXT
 $n_0 = n_2 + 1$

still holds.

Let:

TEXT

$I = \text{internal nodes} = n_2$
 $L = \text{leaves} = n_0$

Then:

FORMULA BOX

TEXT

$L = I + 1$

37. TOTAL NODES IN FULL TREE

TEXT

$n = I + L$

Since:

TEXT

$L = I + 1$

TEXT

$n = 2I + 1$

Thus:

FORMULA BOX

TEXT

$n = 2I + 1$

and:

TEXT

$I = (n - 1) / 2$
 $L = (n + 1) / 2$

38. FULL TREE ODD-NODE PROPERTY

Since:

TEXT

$n = 2I + 1$

a non-empty full binary tree must have:

Odd total node count

Possible:

TEXT

1, 3, 5, 7, 9, 11, ...

Impossible:

TEXT

2, 4, 6, 8, ...

Quick GATE elimination rule.

39. FULL TREE EXAMPLE

Full binary tree has:

TEXT

$n=101$

Internal:

TEXT

$(101-1)/2 = 50$

Leaves:

TEXT

51

40. MAXIMUM HEIGHT OF A FULL TREE

Interesting extension.

A full tree cannot form a simple one-child chain.

To maximize height, each internal node along the deep path gets:

- one leaf child
- one continuing internal child

Example:

TEXT





If there are I internal nodes:

TEXT

$$h_{\max} = I$$

Since:

TEXT

$$n = 2I + 1$$

then:

FORMULA BOX

TEXT

$$h_{\max}(\text{full tree}) = (n-1)/2$$

Useful advanced property.

41. PERFECT BINARY TREE

Perfect tree:

TEXT

every level completely filled

For height h :

FORMULA BOX

TEXT

$$n = 2^{h+1} - 1$$

TEXT

$$\text{Leaves} = 2^h$$

TEXT

$$\text{Internal nodes} = 2^h - 1$$

42. PERFECT TREE HEIGHT FROM n

Starting:

TEXT

$$n = 2^{(h+1)} - 1$$

Then:

TEXT

$$n+1 = 2^{(h+1)}$$

Therefore:

FORMULA BOX

TEXT

$$h = \log_2(n+1) - 1$$

For a perfect tree, $n+1$ must be a power of 2.

43. POSSIBLE PERFECT-TREENODE COUNTS

TEXT

1
3
7
15
31
63
127
...

Pattern:

TEXT

$$2^k - 1$$

44. GATE TRAP

Can a perfect binary tree have:

TEXT

100 nodes?

No.

Because:

TEXT

$$100+1=101$$

is not a power of 2.

45. COMPLETE BINARY TREE $\lceil \log_2 n \rceil$

A complete tree fills:

- all levels except possibly last completely
- final level from left to right

For $n \geq 1$

FORMULA BOX

TEXT

$\text{height} = \lceil \log_2 n \rceil$

46. COMPLETE TREE NODE RANGE FOR HEIGHT h

Levels before h contain:

TEXT

$1 + 2 + \dots + 2^{(h-1)}$
 $=$
 $2^h - 1$

At least one node must exist on level h .

Minimum:

TEXT

2^h

Maximum:

TEXT

$2^{(h+1)} - 1$

Therefore:

FORMULA BOX

TEXT

$2^h \leq n \leq 2^{(h+1)} - 1$

47. COMPLETE TREE INTERNAL/LEAF COUNT

For a complete binary tree containing n nodes:

FORMULA BOX

TEXT

```
Internal nodes = floor(n/2)
Leaves = ceil(n/2)
```

$\Theta(\log n)$ is very important for heaps.

48. EXAMPLE

TEXT

$n=13$

Internal:

TEXT

$\text{floor}(13/2)=6$

Leaves:

TEXT

$\text{ceil}(13/2)=7$

49. COMPLETE TREE — 0-BASED ARRAY INDICES

With indices:

TEXT

$0 \dots n-1$

Last internal node:

FORMULA BOX

TEXT

$\text{floor}(n/2)-1$

Leaves:

TEXT

$\text{floor}(n/2) \dots n-1$

50. EXAMPLE

TEXT

$n=12$

Last internal index:

TEXT

$\text{floor}(12/2) - 1$
 $= 6 - 1$
 $= 5$

Leaves:

TEXT

6, 7, 8, 9, 10, 11

51. COMPLETE TREE — 1-BASED INDICES

With indices:

TEXT

1 ... n

Internal nodes:

TEXT

1 ... $\text{floor}(n/2)$

Leaves:

FORMULA BOX

TEXT

$\text{floor}(n/2) + 1$... n

52. EXAMPLE

TEXT

n=10

1-based internal indices:

TEXT

1, 2, 3, 4, 5

Leaves:

TEXT

6, 7, 8, 9, 10

53. ARRAY CHILD EXISTENCE — 0 BASED

For node index i :

Left child index:

TEXT

$2i+1$

exists iff:

FORMULA BOX

TEXT

$2i+1 < n$

Right child:

TEXT

$2i+2$

exists iff:

TEXT

$2i+2 < n$

54. EXAMPLE

Complete tree array:

TEXT

$n=10$

Node:

TEXT

$i=4$

Left:

TEXT

$2(4)+1=9$

$9 < 10$!' exists.

Right:

TEXT

10

$10 < 10$!' false.

So node 4 has:

TEXT

only a left child

$\emptyset = Y_1$ illustrates the possible one-child node in a complete tree.

55. HOW MANY ONE-CHILD NODES IN COMPLETE TREE?

A complete binary tree has:

TEXT

at most one node with exactly one child

If such a node exists, that child is:

TEXT

LEFT

When does this happen?

For even n .

56. EXAMPLE — n EVEN

TEXT

$n=10$

The last internal node has one left child.

Therefore:

TEXT

$n_1=1$

57. EXAMPLE — n ODD

TEXT

$n=11$

Last internal node has two children.

Therefore:

TEXT

$n_1=0$

So for complete binary tree:

TEXT

```
n even !' exactly one one-child node
n odd  !' zero one-child nodes
```

for $n > 1$.

$\emptyset = \emptyset$ useful heap property.

58. VERIFY USING COUNTS — $n=10$

Complete tree:

TEXT

$n=10$

Leaves:

TEXT

$n_0=5$

Then:

TEXT

$n_2=n_0-1=4$

So:

TEXT

$n_1=10-5-4$
 $=1$

Correct.

59. VERIFY — $n=11$

TEXT

$n_0=\text{ceil}(11/2)=6$
 $n_2=5$

Then:

TEXT

$n_1=11-6-5$
 $=0$

Correct.

60. SKEWED TREE PROPERTIES

For n nodes:

TEXT

```
n0 = 1
n1 = n-1
n2 = 0
```

Why?

Exactly one leaf at end.

All other nodes have exactly one child.

Check:

TEXT

```
n0=n2+1
1=0+1
```

Correct.

Height:

TEXT

```
n-1
```

61. SINGLE-NODE TREE

For:

TEXT

```
n=1
```

Counts:

TEXT

```
n0=1
n1=0
n2=0
```

Edges:

TEXT

```
0
```

NULL child pointers:

TEXT

```
2
```

Height:

TEXT

0

Root is also:

TEXT

leaf

All formulas still work.

62. NUMBER OF LEAVES BOUNDS

For an arbitrary non-empty binary tree:

Minimum number of leaves:

TEXT

1

achieved by a skewed tree.

Maximum leaves depends on n .

To maximize leaves, minimize one-child nodes and use as many two-child nodes as possible.

For n nodes:

FORMULA BOX

TEXT

Maximum leaves = $\text{ceil}(n/2)$

This maximum is achieved by sufficiently compact/full-like shapes.

63. CHECK

TEXT

$n=10$

Maximum leaves:

TEXT

$\text{ceil}(10/2)=5$

A complete tree with 10 nodes has 5 leaves.

64. CHECK $n=7$

Maximum leaves:

TEXT

$\text{ceil}(7/2)=4$

Perfect tree with 7 nodes has:

TEXT

4 leaves

65. NUMBER OF INTERNAL NODES

If "internal node" means non-leaf node:

TEXT

$\text{Internal} = n - n_0$

For complete tree:

TEXT

$\text{Internal} = \text{floor}(n/2)$

For full tree:

TEXT

$\text{Internal} = (n-1)/2$

Do not apply these special formulas to arbitrary trees.

66. PATHLENGTH REMINDER

Path length usually means:

TEXT

number of edges

Example:

TEXT

A !' B !' C !' D

has:

TEXT

3 edges

so path length:

TEXT

3

not 4.

0 = 1 is important in height/depth questions.

67. DEPTH AND HEIGHT RELATION

Depth of node:

TEXT

root !' node

Height of node:

TEXT

node !' deepest descendant leaf

Tree height:

TEXT

maximum depth of any node

under edge-count definitions.

68. HEIGHT RECURSIVE EQUATION

For non-NULL node:

FORMULA BOX

TEXT

```
height(node)
=
1 + max(height(left), height(right))
```

With convention:

TEXT

height(NULL) = -1

Then leaf:

TEXT

```
1+max(-1, -1)
=0
```

69. NODE COUNT RECURSION

FORMULA BOX

TEXT

```
count(T)
=
1 + count(T.left) + count(T.right)
```

Base:

TEXT

```
count(NULL)=0
```

Time:

TEXT

```
O(n)
```

because every node is visited once.

70. LEAF COUNT RECURSION

TEXT

```
if node == NULL:
    return 0

if node has no children:
    return 1

return leaves(left)+leaves(right)
```

Worst-case time:

TEXT

```
O(n)
```

71. HEIGHT COMPUTATION COMPLEXITY

Standard recursive height function visits every node once.

Therefore:

TEXT

```
Time = O(n)
```

Recursive auxiliary stack space:

TEXT

$O(h)$

where h is tree height.

Balanced tree:

TEXT

$O(\log n)$

stack depth.

Skewed tree:

TEXT

$O(n)$

stack depth.

$O(n)$ Great complexity connection.

72. GATE NUMERICAL 1

Binary tree has:

TEXT

$n_1 = 15$

$n_2 = 25$

Find:

Leaves

TEXT

$n_0 = n_2 + 1$

$= 26$

Total nodes

TEXT

$n = 26 + 15 + 25$

$= 66$

Edges

TEXT

65

NULL child links

TEXT

67

73. GATE NUMERICAL 2

Binary tree has:

TEXT

80 nodes

30 nodes with exactly two children

Find one-child nodes.

Leaves:

TEXT

$n_0 = 30 + 1 = 31$

Therefore:

TEXT

$n_1 = 80 - 31 - 30$

$= 19$

Answer:

19

74. GATE NUMERICAL 3

Binary tree contains:

TEXT

100 NULL child links

Since:

TEXT

$\text{NULL} = n + 1$

then:

TEXT

$n = 99$

Edges:

TEXT

98

$\Theta = \sqrt{N}$ Very fast.

75. GATE NUMERICAL 4

Perfect tree contains:

TEXT

127 nodes

Find height.

TEXT

$127 = 2^7 - 1$

Therefore:

TEXT

$h + 1 = 7$

So:

TEXT

$h = 6$

Leaves:

TEXT

$2^6 = 64$

76. GATE NUMERICAL 5

Full binary tree has:

TEXT

63 nodes

Internal:

TEXT

$(63 - 1) / 2$
 $= 31$

Leaves:

TEXT

32

77. GATE NUMERICAL 6

Complete binary tree contains:

TEXT

$n=100$

Height:

TEXT

$\text{floor}(\log_2 100)=6$

because:

TEXT

6 4 "d 1 0 0 < 1 2 8

Leaves:

TEXT

$\text{ceil}(100/2)=50$

Internal:

TEXT

50

Last internal 0-based index:

TEXT

$50-1=49$

First leaf index:

TEXT

50

78. GATE NUMERICAL 7

A binary tree has height:

TEXT

5

Possible node range:

Minimum:

TEXT
5+1=6

Maximum:

TEXT
 $2^6 - 1 = 63$

Therefore:

TEXT
6 "d n "d 6 3

79. GATE NUMERICAL 8

Binary tree has:

TEXT
 $n=25$

Possible height range:

Minimum:

TEXT
 $\text{floor}(\log_2 25)=4$

because:

TEXT
1 6 "d 2 5 < 3 2

Maximum:

TEXT
24

So:

TEXT
4 "d h "d 2 4

80. GATE TRUE/FALSE TRAPS

Statement 1

Every binary tree with n nodes has $n-1$ edges.

' True.

Statement 2

Every binary tree has leaves = internal nodes + 1.

'LFalse.

That is true for a **full binary tree** if internal means two-child nodes; in an arbitrary binary tree, the universal relation is:

TEXT

$$n_0 = n_2 + 1$$

81. TRAP 3

Every binary tree of height h contains $2^{(h+1)} - 1$ nodes.

'LFalse.

That is the **maximum**.

Exact only for a perfect binary tree.

82. TRAP 4

A binary tree of height h must have at least 2^h nodes.

'LFalse.

A skewed tree needs only:

TEXT

$$h+1$$

nodes.

83. TRAP 5

Every binary tree with n nodes has $n+1$ NULL child links.

' True

for the usual linked representation with two child-pointer fields per node.

84. TRAP 6

Every complete binary tree is full.

'LFalse.

Example:

TEXT



Complete but not full.

85. TRAP 7

Every perfect tree is full and complete.

' True.

86. TRAP 8

A complete tree can have a node having only a right child.

'LFalse.

That creates a left-side gap.

If a complete tree has a one-child node, it must have:

TEXT

left child only

87. TRAP 9

A full tree can have an even number of nodes.

'LFalse for a non-empty full binary tree.

TEXT

$n=2I+1$

always odd.

88. MASTER FORMULA TABLE — GENERAL BINARY TREE

| Property | Formula |
|-------------------------|--------------------------|
| Total nodes | $n = n_0 + n_1 + n_2$ |
| Edges | $n - 1$ |
| Child-edge count | $n_1 + 2n_2 = n - 1$ |
| Leaves relation | $n_0 = n_2 + 1$ |
| Total via degree counts | $n = 2n_2 + n_1 + 1$ |
| NULL child links | $n + 1$ |
| Max nodes level L | 2^L |
| Min nodes height h | $h + 1$ |
| Max nodes height h | $2^{(h+1)} - 1$ |
| Max leaves height h | 2^h |
| Max height n nodes | $n - 1$ |
| Min height n nodes | $\text{floor}(\log_2 n)$ |

0-0 Copy this.

89. FULL TREE FORMULA TABLE

| Property | Formula |
|-------------------|---------------|
| n_1 | 0 |
| Leaves | $L = I + 1$ |
| Total | $n = 2I + 1$ |
| Internal | $(n - 1) / 2$ |
| Leaves from n | $(n + 1) / 2$ |
| Total node parity | Odd |

90. PERFECT TREE FORMULA TABLE

For height h :

| Property | Formula |
|-------------|-----------------|
| Total nodes | $2^{(h+1)} - 1$ |
| Leaves | 2^h |
| Internal | $2^h - 1$ |

| Property | Formula |
|---------------|-----------------|
| Height from n | $\log_2(n+1)-1$ |

91. COMPLETE TREE FORMULA TABLE

For $n \geq 1$

| Property | Formula |
|------------------------|--------------------------|
| Height | $\text{floor}(\log_2 n)$ |
| Internal nodes | $\text{floor}(n/2)$ |
| Leaves | $\text{ceil}(n/2)$ |
| Last internal, 0-based | $\text{floor}(n/2)-1$ |
| First leaf, 0-based | $\text{floor}(n/2)$ |
| Last leaf | $n-1$ |

92. ARRAY FORMULA TABLE

0-Based

TEXT

Parent = $\text{floor}((i-1)/2)$
 Left = $2i+1$
 Right = $2i+2$

1-Based

TEXT

Parent = $\text{floor}(i/2)$
 Left = $2i$
 Right = $2i+1$

0-1s become critical in Binary Heaps.

93. HOW TO ATTACK GATE TREE NUMERICALS

When a question gives degree counts, immediately write:

TEXT

$n = n_0 + n_1 + n_2$

and:

TEXT

$n_0 = n^2 + 1$

If total nodes are involved:

TEXT

Edges = $n - 1$

NULL = $n + 1$

If height is involved:

TEXT

$h + 1 \leq n \leq 2^{(h + 1)} - 1$

If full:

TEXT

$n_1 = 0$

If perfect:

TEXT

$n = 2^{(h + 1)} - 1$

If complete:

TEXT

$h = \text{floor}(\log_2 n)$

$\emptyset = \text{Yes}$ is your exam decision tree.

94. MEMORY TRICKS

Edges

Tree always needs one fewer connection.

TEXT

$n - 1$

NULL pointers

NULL gives one extra.

TEXT

$n + 1$

Binary degrees

Leaves beat two-child nodes by ONE.

TEXT

$n_0 = n_2 + 1$

Height bounds

****Stick = minimum nodes.**

Perfect = maximum nodes.**

TEXT

$h + 1 \leq 2^{(h + 1)} - 1$

95. 30-SECOND REVISION

Remember these first:

TEXT

Edges = $n - 1$

TEXT

NULL child links = $n + 1$

TEXT

$n_0 = n_2 + 1$

TEXT

Max nodes level $L = 2^L$

TEXT

$h + 1 \leq \lceil \log_2 n \rceil \leq 2^{(h + 1)} - 1$

TEXT

$\text{floor}(\log_2 n) \leq h \leq n - 1$

Full:

TEXT

$L = I + 1$

Perfect:

TEXT

$$n=2^{(h+1)}-1$$

Complete:

TEXT

$$h=\text{floor}(\log_2 n)$$

96. ACTIVE RECALL

Q1

Binary tree has 30 nodes with exactly two children. Leaves?

Q2

Binary tree contains 75 nodes. NULL child links?

Q3

Height 6. Maximum number of nodes?

Q4

Full binary tree contains 25 internal nodes. Total nodes?

Q5

Complete tree contains 70 nodes. Height?

ANSWERS

A1

TEXT

31

A2

TEXT

76

A3

TEXT

$$2^7-1 \\ =127$$

A4

TEXT

$$2(25)+1 \\ =51$$

A5

Since:

TEXT

6 4 "d 7 0 < 1 2 8

TEXT

$$\text{floor}(\log_2 70)=6$$

97. PRACTICE — MIXED GATE

Q1

A binary tree has:

TEXT

$$n_1=14 \\ n_2=18$$

Find:

- leaves
- total nodes
- edges
- NULL pointers

Q2

A full tree contains 101 nodes.

Find:

- internal nodes
- leaves

Q3

A perfect tree contains 255 nodes.

Find:

- height
- leaves

Q4

A complete tree contains 34 nodes.

Find:

- height

- internal nodes
- leaves
- last internal 0-based index

Q5

A tree has height 8.

Find minimum and maximum possible node counts.

98. TREE MODULE CHECKPOINT

We have now completed:

' General tree terminology ' Binary-tree definition ' C node structure ' Linked representation ' Array representation ' 0-/1-based indexing ' Full binary tree ' Complete binary tree ' Perfect binary tree ' Balanced/skewed/degenerate concepts ' Edge-count formulas ' Degree-count formulas ' Leaf/two-child theorem ' NULL pointer theorem ' Height/node bounds ' Complete-tree array properties ' Major GATE numericals

Now we begin the second major tree area:

TRAVERSALS

PROGRESS

MODULE 6 — TREES

' Lesson 50 — Tree Fundamentals ' Lesson 51 — Binary Trees & Representation ' Lesson 52 — Binary Tree Types ' Lesson 53 — Binary Tree Properties & Formula Masterclass

NEXT — LESSON 54

Preorder Traversal — Root !' Left !' Right, Recursive Calls, Stack Behavior & GATE Tracing

We will start with:

TEXT



Preorder:

TEXT

A B D E C

and master:

- R o o t !' L e f t !' R i g h t

- recursive C implementation
- exact call-stack tracing
- iterative preorder using stack
- time and space complexity
- preorder properties
- how to reconstruct/use preorder
- GATE traversal traps.

USER

next

ASSISTANT

MODULE 6 — LESSON 54

Preorder Traversal — Root ! ' Left ! ' Right, Recursion, Stack Behavior & GATE Tracing

We now begin the most important operational part of Trees:

TREE TRAVERSALS

Traversal means:

Visit every node of the tree in a systematic order.

The three classical depth-first traversals are:

- Preorder
- Inorder
- Postorder

Today:

PREORDER

GATE importance: Ø=Ý%ØMÚSDKÍOW **Placement importance:** Ø=Ý%ØMÚSDKÍOW

1. CORE RULE

Preorder follows:

ROOT !' LEFT !' RIGHT

Memory trick:

PRE = Root comes PRE/Before the subtrees.

So always think:

TEXT

Visit Root
Traverse Left
Traverse Right

2. SIMPLE EXAMPLE

Tree:

TEXT



Preorder:

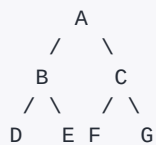
1. Root !' A
2. Left subtree !' B
3. Right subtree !' C

Therefore:

A B C

3. BIGGER EXAMPLE

TEXT



Apply:

TEXT

Root !' A
Left subtree of A
Right subtree of A

Left subtree B:

TEXT

B
D
E

Right subtree C:

TEXT

C
F
G

Therefore preorder:

A B D E C F G

Remember the process, not the answer.

4. RECURSIVE DEFINITION

For any node:

TEXT

Preorder(node):

1. Visit node
2. Preorder(node->left)
3. Preorder(node->right)

That's it.

5. C IMPLEMENTATION

C

```
void preorder(Node *root)
{
    if (root == NULL)
        return;

    printf("%d ", root->data);
    preorder(root->left);
    preorder(root->right);
}
```

This is one of the most important tree functions to know.

6. WHY BASE CASE?

Without:

C

```
if (root == NULL)
    return;
```

eventually recursion reaches a missing child.

Example leaf:

TEXT

D

has:

TEXT

```
D.left = NULL
D.right = NULL
```

So recursive calls become:

TEXT

```
preorder(NULL)
```

and must stop.

7. EXACT RECURSION TRACE

Tree:

TEXT



Call:

TEXT

```
preorder(A)
```

Step 1:

TEXT

```
print A
```

Then:

TEXT

```
preorder(B)
```

8. INSIDE B

Print:

TEXT

B

Then call:

TEXT

```
preorder(D)
```

9. INSIDE D

Print:

TEXT

```
D
```

Then:

TEXT

```
preorder(D->left)
```

which is:

TEXT

```
preorder(NULL)
```

returns.

Then:

TEXT

```
preorder(D->right)
```

also NULL.

Return from D.

Current output:

TEXT

```
A B D
```

10. BACK TO B

Now execute:

TEXT

```
preorder(B->right)
```

which is:

TEXT

```
preorder(E)
```

Print:

TEXT

E

Its children are NULL.

Return.

Current output:

TEXT

A B D E

11. BACK TO A

Left subtree completed.

Now:

TEXT

preorder(A->right)

which is C.

Print:

TEXT

C

Final:

A B D E C

Exact recursive flow matters for GATE output questions.

12. CALL STACK VISUALIZATION

During traversal:

TEXT

```
preorder(A)
  preorder(B)
    preorder(D)
```

Runtime stack conceptually:

TEXT

```
TOP
preorder(D)
preorder(B)
preorder(A)
```

After D finishes:

TEXT

```
preorder(D)
```

frame disappears.

Execution resumes inside:

TEXT

```
preorder(B)
```

at the statement after the recursive left call.

This is your recursion module applied to trees.

13. IMPORTANT EXECUTION ORDER

Code:

```
C
printf("%d ", root->data);
preorder(root->left);
preorder(root->right);
```

The print happens:

before both recursive calls

Therefore:

PREORDER

Print moves between calls, traversal changes.

14. COMPARE CODE POSITIONS

Preorder

```
C
print(root);
traverse(left);
traverse(right);
```

Inorder

```
C
traverse(left);
print(root);
traverse(right);
```

Postorder

C

```
traverse(left);  
traverse(right);  
print(root);
```

This single observation lets you remember all three.

15. MEMORY TRICK

Think about the position of:

TEXT

ROOT

PREorder

TEXT

ROOT L R

Root first.

INorder

TEXT

L ROOT R

Root in the middle.

POSTorder

TEXT

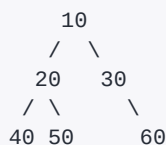
L R ROOT

Root last.

Ø=Extremely important.

16. ANOTHER PREORDER EXAMPLE

TEXT



Preorder:

Root:

TEXT

10

Left subtree:

TEXT

20 40 50

Right subtree:

TEXT

30 60

Answer:

10 20 40 50 30 60

17. HOW TO TRACE QUICKLY IN EXAM

Use this rule:

Whenever you first arrive at a node, write it immediately.

Then go:

TEXT

as far left as possible

while respecting recursive returns.

For preorder:

Write node on first encounter.

This is a powerful visualization trick.

18. "FIRST ENCOUNTER" VIEW

Tree:

TEXT



Walk:

TEXT

```
A !'   w r i t e   A
B !'   w r i t e   B
D !'   w r i t e   D
back
E !'   w r i t e   E
back
C !'   w r i t e   C
```

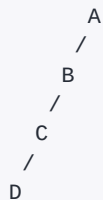
Output:

TEXT

A B D E C

19. PREORDER OF LEFT-SKEWED TREE

TEXT



Preorder:

TEXT

A B C D

20. PREORDER OF RIGHT-SKEWED TREE

TEXT



Preorder:

TEXT

A B C D

Interesting:

Both left-skewed and right-skewed trees can have the same preorder.

$\emptyset \neq \{A\}$ Therefore preorder alone generally does **not** uniquely determine a binary tree.

We'll study uniqueness properly in Lesson 58.

21. SINGLE NODE

TEXT

A

Preorder:

TEXT

A

22. EMPTY TREE

TEXT

root = NULL

Preorder visits:

TEXT

nothing

Time:

TEXT

$O(1)$

for the empty call itself.

23. TIME COMPLEXITY $\Theta = \Upsilon\%$

For a tree with n nodes:

Each node is:

TEXT

visited exactly once

Work per node is constant apart from recursive calls.

Therefore:

FORMULA BOX

TEXT

P r e o r d e r T i m e = $\sim (n)$

Best/worst shape does not change total node visits.

24. AUXILIARY SPACE

Recursive traversal uses call stack.

Maximum simultaneous calls depend on tree height h .

Therefore:

FORMULA BOX

TEXT

Auxiliary Space = $O(h)$

25. BALANCED TREE SPACE

Balanced tree:

TEXT

$h = O(\log n)$

So recursive stack:

TEXT

$O(\log n)$

26. SKEWED TREE SPACE

Skewed tree:

TEXT

$h = n-1$

So recursion stack:

TEXT

$O(n)$

~~0 = Yes~~ The traversal time stays $O(n)$, but recursion space depends strongly on shape.

27. RECURSIVE STACK VS TREE STORAGE

Don't say:

TEXT

preorder uses $O(n)$ extra space always

More accurate:

```
TEXT
recursive auxiliary space = O(h)
```

Worst case:

```
TEXT
O(n)
```

Balanced:

```
TEXT
O(log n)
```

The tree itself already consumes $O(n)$ storage, but that is input storage, not traversal auxiliary space.

28. ITERATIVE PREORDER

Preorder can also be performed without recursion using an explicit:

STACK

Rule:

1. Push root.
2. While stack not empty:
 - pop node
 - visit it
 - push right child
 - push left child

Notice:

RIGHT is pushed before LEFT

because stack is LIFO.

29. WHY PUSH RIGHT FIRST?

We want processing order:

```
TEXT
R o o t ! '   L e f t ! '   R i g h t
```

Suppose node A has:

```
TEXT
left = B
right = C
```

If stack receives:

TEXT
C first
B second

then B is on top.

So B pops first.

Thus logical order remains:

TEXT
Left before Right

0-0% Classic GATE/placement trap.

30. ITERATIVE TRACE

Tree:

TEXT

```
graph TD
  A --> B
  A --> C
  B --> D
  B --> E
```

Start:

TEXT
Stack: [A]

Pop A:

TEXT
Output: A

Push C then B:

TEXT
Stack: [C,B]

with B on top.

31. POP B

Output:

TEXT
A B

Push E then D:

```
TEXT
Stack:
C
E
D !• t o p
```

Pop D:

```
TEXT
A B D
```

D has no children.

Next pop E:

```
TEXT
A B D E
```

Then C:

```
TEXT
A B D E C
```

Correct.

32. ITERATIVE C CODE

```
C
void preorderIterative(Node *root)
{
    if (root == NULL)
        return;

    Node *stack[100];
    int top = -1;

    stack[++top] = root;

    while (top != -1)
    {
        Node *curr = stack[top--];

        printf("%d ", curr->data);

        if (curr->right != NULL)
            stack[++top] = curr->right;

        if (curr->left != NULL)
            stack[++top] = curr->left;
    }
}
```

33. ITERATIVE COMPLEXITY

Every node:

TEXT

pushed once
popped once

Therefore:

TEXT

Time = $O(n)$

Stack can contain up to:

TEXT

$O(h)$

in many shapes, though in general the explicit frontier can reach $O(n)$ for broad trees.

For safe worst-case statement:

FORMULA BOX

TEXT

Iterative Preorder:
Time = $O(n)$
Auxiliary Space = $O(n)$ worst case

Recursive:

TEXT

$O(h)$

call-stack depth.

34. A MORE SPACE-PRECISE NOTE

For a perfect tree, recursive stack depth:

TEXT

$O(\log n)$

The explicit iterative stack during preorder can also stay proportional to height for this standard algorithm, because we defer right children while descending left.

So this particular iterative preorder typically has:

TEXT

$O(h)$

stack space.

For exam safety:

TEXT

```
O(h) standard preorder stack
"d  O ( n )   w o r s t   c a s e
```

State context precisely.

35. GATE TRAP — PUSH ORDER

Wrong:

TEXT

```
push left
push right
```

Because right will be popped first.

That produces:

TEXT

```
R o o t ! '   R i g h t ! '   L e f t
```

instead of preorder.

Correct:

TEXT

```
push right
push left
```

36. GATE TRAP — PREORDER ROOT POSITION

Given traversal sequence:

TEXT

```
A B D E C
```

for a non-empty tree:

The first element:

TEXT

```
A
```

must be:

ROOT

In preorder, root always appears first.

37. PREORDER SUBTREE PROPERTY

For any subtree:

TEXT

Root
Left subtree sequence
Right subtree sequence

appear as contiguous blocks in preorder.

Example:

TEXT



Preorder:

TEXT

A | B D E | C

The entire traversal of B's subtree:

TEXT

B D E

is contiguous.

This becomes important for reconstruction.

38. PREORDER AND BST $\emptyset = \text{YES}$

For a BST, preorder begins with:

TEXT

root

Then values in the left subtree occur before values in right subtree.

Under distinct keys:

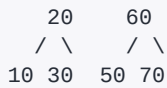
TEXT

left values < root
right values > root

Example BST:

TEXT





Preorder:

TEXT

40 20 10 30 60 50 70

This additional BST ordering can make reconstruction possible.

We'll study later.

39. PREORDER AND EXPRESSION TREES

Expression tree:

TEXT



Preorder:

TEXT

+ A * B C

This corresponds to:

PREFIX EXPRESSION

Ø=Important cross-topic connection.

So:

TEXT

E x p r e s s i o n t r e e p r e o r d e r ! ' P r e f i x

40. MASTER EXPRESSION CONNECTION

For expression tree:

TEXT

P r e o r d e r ! ' P r e f i x
I n o r d e r ! ' I n f i x
P o s t o r d e r ! ' P o s t f i x

You already saw this in Stack lessons.

Ø=This connection is very important.

41. APPLICATIONS OF PREORDER

Preorder is useful when we need to process parent before children.

Examples:

- copying/cloning a tree
 - serialization schemes with NULL markers
 - prefix expression generation
 - directory-like hierarchy printing
 - certain tree construction algorithms
-

42. TREE COPY INTUITION

Suppose we want to create a new copy.

It is natural to:

1. create current/root node
2. recursively copy left
3. recursively copy right

That resembles preorder:

TEXT

R o o t ! ' L e f t ! ' R i g h t

43. SERIALIZATION WARNING

Preorder node values alone do not uniquely encode arbitrary tree structure.

Example two trees:

TEXT

A
/
B

and:

TEXT

A
\
B

both preorder:

TEXT

A B

To serialize uniquely, one common method records NULL markers.

44. PREORDER WITH NULL MARKERS

Tree:

```
TEXT
  A
 /
B
```

Preorder with # for NULL:

```
TEXT
A B # # #
```

Tree:

```
TEXT
A
 \
B
```

becomes:

```
TEXT
A # B # #
```

Now they differ.

Useful placement concept.

45. GATE-STYLE TRACE

Tree:

```
TEXT
      1
     / \
    2   3
   / \ /
  4  5 6
     / \
    7  8
```

Find preorder.

Start root:

```
TEXT
1
```

Left subtree of 1:

TEXT

2

Left of 2:

TEXT

4

Right subtree of 2:

TEXT

5 7 8

Then right subtree of 1:

TEXT

3 6

Answer:

1 2 4 5 7 8 3 6

46. QUICK CHECK USING STRUCTURE

Break into:

TEXT

Root = 1
Left subtree = 2 4 5 7 8
Right subtree = 3 6

Combine:

TEXT

1 2 4 5 7 8 3 6

This decomposition prevents mistakes.

47. GATE OUTPUT QUESTION

C

```
void fun(Node *root)
{
    if (root == NULL)
        return;

    printf("%d ", root->data);
    fun(root->left);
    fun(root->right);
}
```

What traversal?

Because print happens:

TEXT

BEFORE left and right

Answer:

Preorder

48. GATE CODE MODIFICATION

C

```
void fun(Node *root)
{
    if (root == NULL)
        return;

    fun(root->right);
    printf("%d ", root->data);
    fun(root->left);
}
```

This is not standard preorder/inorder/postorder.

Order:

TEXT

R i g h t ! ' R o o t ! ' L e f t

~~0=Always~~ Always inspect actual statement ordering instead of blindly naming traversal.

49. MIRROR PREORDER

If algorithm does:

TEXT

R o o t ! ' R i g h t ! ' L e f t

it is sometimes informally called reverse/mirror preorder.

Do not call it standard preorder.

Standard preorder remains:

TEXT

R o o t ! ' L e f t ! ' R i g h t

50. PREORDER + INORDER PREVIEW

Preorder gives:

```
TEXT
root first
```

Inorder lets us locate:

```
TEXT
left subtree | root | right subtree
```

Therefore:

Preorder + Inorder

can uniquely reconstruct a binary tree when node values are distinct.

We'll prove this in Lesson 58.

51. PREORDER ALONE?

For arbitrary binary tree:

```
TEXT
not enough
```

Example:

```
TEXT
A
/
B
```

and:

```
TEXT
A
 \
  B
```

same preorder.

So:

Preorder alone does not generally determine the tree uniquely.

◊=Imp. Important GATE trap.

52. PREORDER OF FULL TREE — SPECIAL CASE

If additional structural information is known, such as:

- tree is full

- leaves/internal markers are known
- BST ordering exists

then preorder can sometimes determine more.

But never assume uniqueness without stated constraints.

53. NODE VISIT COUNT

For n nodes, number of successful node visits:

TEXT
 n

But recursive calls also occur on NULL child pointers.

How many NULL child pointers?

TEXT
 $n+1$

Therefore a standard recursive traversal conceptually makes:

TEXT
 $n + (n+1)$
 $= 2n+1$

calls if counting both non-NULL and NULL invocations.

0 = ~~Not~~ the GATE-style counting property.

54. VERIFY SMALL TREE

Single node:

TEXT
A

Calls:

TEXT
preorder(A)
preorder(NULL-left)
preorder(NULL-right)

Total:

TEXT
3

Formula:

TEXT

$2n+1 = 3$

Correct.

Advanced useful property.

55. TIME STILL $\sim (n)$

Even though total recursive invocations may be:

TEXT

$2n+1$

that is still:

TEXT

$\sim (n)$

because constants do not change asymptotic complexity.

56. PREORDER ITERATIVE STACK TRACE QUESTION

Tree:

TEXT

```
      10
     /  \
    20   30
   /  \
  40  50
```

Initial:

TEXT

[10]

Pop 10:

TEXT

output 10
push 30
push 20

Stack:

TEXT

30
20 !• top

57. CONTINUE

Pop 20:

TEXT

```
output 20
push 50
push 40
```

Stack:

TEXT

```
30
50
4 0 !• t o p
```

Pop 40:

TEXT

```
output 40
```

Pop 50:

TEXT

```
output 50
```

Pop 30:

TEXT

```
output 30
```

Final:

10 20 40 50 30

58. PREORDER COMPLEXITY TABLE

| Method | Time | Auxiliary Space |
|-----------------|------------|-------------------------------------|
| Recursive | $\sim (n)$ | $O(h)$ |
| Iterative Stack | $\sim (n)$ | $O(h)$ typical / $O(n)$ worst bound |

Where:

TEXT

```
h = tree height
```


59. BALANCED VS SKEWED RECURSION

Balanced

TEXT

```
h "H l o g , n
```

So:

TEXT

```
space = O(log n)
```

Skewed

TEXT

```
h=n-1
```

So:

TEXT

```
space = O(n)
```

~~O=Extremely reusable tree-complexity pattern.~~

60. PREORDER MEMORY TRICK

Think of entering a house:

Meet the parent first, then visit the left family, then the right family.

So:

PARENT !' LEFT !' RIGHT

61. NOTEBOOK NOTES

Preorder

TEXT

```
R o o t !' L e f t !' R i g h t
```

Recursive:

C

```
void preorder(Node *root)
{
    if (root == NULL)
        return;
```

```
visit(root);
preorder(root->left);
preorder(root->right);
}
```

62. ITERATIVE NOTES

TEXT

push root

while stack not empty:

node = pop

visit node

push right

push left

Why right first?

TEXT

Stack is LIFO !' left must be on top

63. FORMULA / COMPLEXITY BOX

TEXT

Time = $\sim (n)$

Recursive stack:

TEXT

$O(h)$

Worst case:

TEXT

$O(n)$

Balanced:

TEXT

$O(\log n)$

64. DON'T CONFUSE

Preorder

TEXT

Root Left Right

Prefix

For expression tree:

TEXT

Preorder = Prefix

Root location

TEXT

First element of preorder

Iterative push order

TEXT

Right first, Left second

65. 30-SECOND REVISION

Remember:

1. Preorder = Root !' Left !' Right.
2. Root is always first.
3. Visit node before recursive children.
4. Recursive time = $\sim (n)$.
5. Recursive space = $O(h)$.
6. Balanced tree stack = $O(\log n)$.
7. Skewed tree stack = $O(n)$.
8. Iterative preorder uses explicit stack.
9. Push right before left.
10. Expression-tree preorder gives prefix.

66. ACTIVE RECALL

Tree:

TEXT



Q1

Preorder?

Q2

Which node appears first?

Q3

Which explicit DS is used for iterative preorder?

Q4

Why push right before left?

Q5

What is recursive auxiliary space?

ANSWERS

A1

TEXT

A B D E C F

A2

TEXT

A

A3

TEXT

Stack

A4

Because stack is LIFO, so left must be pushed later to be processed first.

A5

TEXT

$O(h)$

67. PRACTICE — EASY

Find preorder:

TEXT

```
  1
 / \
2   3
/
```

68. PRACTICE — MODERATE

Find preorder:

TEXT



69. PRACTICE — CODE IDENTIFICATION

What traversal does this perform?

C

```

void f(Node *root)
{
    if (!root)
        return;

    printf("%d ", root->data);
    f(root->right);
    f(root->left);
}
  
```

Answer:

TEXT

R o o t ! ' R i g h t ! ' L e f t

not standard preorder.

70. GATE-STYLE QUESTION

A binary tree has n nodes. A standard recursive preorder routine is called on the root and also recursively called on each NULL child pointer.

How many total function invocations occur?

Non-NULL nodes:

TEXT

n

NULL child links:

TEXT

$n+1$

Total:

$2n + 1$

~~0-6%~~ recursion + tree combined question.

71. SYLLABUS CHECKPOINT

Trees completed so far:

' Terminology ' Binary trees ' Representations ' Tree types ' Formula masterclass ' **Preorder traversal**

Still remaining in Trees:

- Inorder
- Postorder
- Level order
- traversal comparisons
- reconstruction
- uniqueness conditions
- remaining traversal/GATE traps

PROGRESS

MODULE 6 — TREES

' Lesson 50 — Tree Fundamentals ' Lesson 51 — Binary Trees ' Lesson 52 — Binary Tree Types ' Lesson 53 — Properties & Formulas ' **Lesson 54 — Preorder Traversal**

NEXT — LESSON 55

Inorder Traversal — Left !' Root !' Right, BST Sorted Order & GATE Tracing

This is especially important because:

Inorder traversal of a BST gives keys in sorted order

under the standard BST ordering.

We will cover:

- recursive inorder
- exact call-stack trace
- iterative inorder with stack
- expression-tree infix connection
- BST connection

- predecessor/successor intuition
- traversal traps
- complexity.

USER

next

ASSISTANT

MODULE 6 — LESSON 55

Inorder Traversal — Left !' Root !' Right, BST Sorted Order & GATE Tracing

Now we study:

INORDER TRAVERSAL

This traversal becomes especially important when we reach **Binary Search Trees**, because:

Inorder traversal of a BST gives keys in sorted order

for the standard BST ordering.

GATE importance: Ø=Ý%ØMÚ%ØKÍØW **Placement importance:** Ø=Ý%ØMÚ%ØKÍØW

1. CORE RULE

Inorder follows:

LEFT !' ROOT !' RIGHT

Memory trick:

INorder = Root comes IN between Left and Right

So:

TEXT

Traverse Left
Visit Root
Traverse Right

2. SIMPLE EXAMPLE

Tree:

TEXT



Inorder:

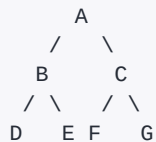
1. Left !' B
2. Root !' A
3. Right !' C

Therefore:

B A C

3. BIGGER EXAMPLE

TEXT



Left subtree of A:

TEXT

D B E

Then:

TEXT

A

Then right subtree:

TEXT

F C G

Therefore:

D B E A F C G

~~0-1%~~ This is one of the standard traversal examples.

4. RECURSIVE DEFINITION

For every node:

TEXT

Inorder(node):

1. Inorder(node->left)
2. Visit node
3. Inorder(node->right)

5. C IMPLEMENTATION

```
C
void inorder(Node *root)
{
    if (root == NULL)
        return;

    inorder(root->left);
    printf("%d ", root->data);
    inorder(root->right);
}
```

The only difference from preorder is where the:

```
C
printf(...)
```

appears.

6. COMPARE WITH PREORDER

Preorder

```
C
printf("%d ", root->data);
preorder(root->left);
preorder(root->right);
```

Order:

TEXT

Root Left Right

Inorder

```
C
inorder(root->left);
printf("%d ", root->data);
inorder(root->right);
```

Order:

TEXT

Left Root Right

Position of `visit(root)` defines the traversal.

7. EXACT RECURSION TRACE

Tree:

TEXT



Call:

TEXT

`inorder(A)`

But A is **not printed immediately**.

First:

TEXT

`inorder(B)`

Again B is not printed.

First:

TEXT

`inorder(D)`

8. INSIDE D

Call:

TEXT

`inorder(D->left)`

NULL != return.

Now print:

TEXT

D

Then:

TEXT

```
inorder(D->right)
```

NULL !' return.

D finishes.

Current output:

TEXT

D

9. BACK TO B

Now B's left subtree is finished.

So print:

TEXT

B

Current:

TEXT

D B

Then traverse:

TEXT

```
inorder(E)
```

E has no children.

So print:

TEXT

E

Current:

TEXT

D B E

Return to A.

10. BACK TO A

A's left subtree is now complete.

So print:

TEXT

A

Current:

TEXT

D B E A

Then traverse C.

C has no children, so:

TEXT

C

Final:

D B E A C

11. CALL-STACK INTUITION

During initial descent:

TEXT

```
inorder(A)
  inorder(B)
    inorder(D)
```

Call stack:

TEXT

TOP
D
B
A

But nothing is printed until D's left call returns.

~~Order~~ Order often means:

Go left as far as possible before printing anything.

12. QUICK EXAM TRICK

For inorder:

Go to extreme LEFT first.

Then:

TEXT

Node
Right subtree

Example:

TEXT



Extreme left:

TEXT

2

Then:

TEXT

4

Then:

TEXT

6

Then:

TEXT

8

Then right subtree:

TEXT

12

Answer:

2 4 6 8 12

13. INORDER OF LEFT-SKEWED TREE

TEXT



D

Inorder:

TEXT
D C B A

Notice:

Preorder was:

TEXT
A B C D

Inorder becomes reverse direction because left subtree is processed first.

14. INORDER OF RIGHT-SKEWED TREE

TEXT
A
 \
 B
 \
 C
 \
 D

Inorder:

TEXT
A B C D

Again, traversal alone does not necessarily reveal exact shape.

15. MOST IMPORTANT BST PROPERTY

For a Binary Search Tree with distinct keys:

TEXT
all keys in left subtree < root
all keys in right subtree > root

Therefore:

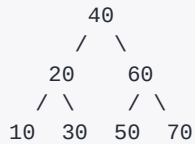
TEXT
L e f t ! ' R o o t ! ' R i g h t

naturally gives:

Sorted ascending order

16. BST EXAMPLE

TEXT



Inorder:

TEXT

10 20 30 40 50 60 70

Exactly sorted.

Ø=Must know.

17. VERY IMPORTANT GATE RESULT

If a binary tree's inorder traversal is:

TEXT

10 20 30 40 50

you **cannot automatically conclude** it is a BST.

Why?

A non-BST tree could accidentally produce sorted inorder values.

The correct direction is:

For a valid BST, inorder is sorted.

Not necessarily:

Any binary tree with sorted inorder must be a BST without checking the actual node ordering/structure assumptions.

For a given labeled tree, sorted inorder is a useful BST validation condition under standard distinct-key rules.

18. DUPLICATE KEYS WARNING

BST duplicate handling varies.

Some implementations:

- disallow duplicates

- put duplicates consistently left
- put duplicates consistently right
- maintain frequency counts

Therefore inorder may be:

TEXT
non-decreasing

rather than strictly increasing.

Read the BST definition used in the problem.

19. EXPRESSION TREE CONNECTION

Expression tree:

TEXT

```

      +
     / \
    A  *
     / \
    B  C
  
```

Inorder:

TEXT
A + B * C

This corresponds to:

INFIX

So:

TEXT

| | | |
|-------------------|---|---------------|
| P r e o r d e r | ! | P r e f i x |
| I n o r d e r | ! | I n f i x |
| P o s t o r d e r | ! | P o s t f i x |

Very important cross-topic connection.

20. PARENTHESES ARE IMPORTANT

Raw inorder:

TEXT
A + B * C

may rely on operator precedence.

To preserve tree structure exactly, use parentheses:

TEXT

$(A + (B * C))$

So expression-tree inorder corresponds structurally to infix, often with parentheses added.

21. TIME COMPLEXITY

Every node is visited exactly once.

Therefore:

FORMULA BOX

TEXT

Inorder Time = $\sim (n)$

22. RECURSIVE SPACE

Maximum recursion depth:

TEXT

tree height h

Therefore:

FORMULA BOX

TEXT

Auxiliary Space = $O(h)$

Balanced:

TEXT

$O(\log n)$

Skewed:

TEXT

$O(n)$

23. ITERATIVE INORDER $\emptyset = \text{Y}\% \emptyset = \text{Y}\% \emptyset = \text{Y}\%$

Inorder can be done using an explicit stack.

The challenge:

We cannot process a node immediately because its left subtree must come first.

So we keep pushing nodes while moving left.

24. MASTER ITERATIVE ALGORITHM

TEXT

```
curr = root

while curr != NULL OR stack not empty:

    while curr != NULL:
        push(curr)
        curr = curr->left

    curr = pop()
    visit(curr)

    curr = curr->right
```

Memorize the idea:

PUSH LEFT CHAIN !' POP/VISIT !' GO RIGHT

25. ITERATIVE TRACE

Tree:

TEXT



Start:

TEXT

```
curr=A
```

Push A.

Move left !' B.

Push B.

Move left !' D.

Push D.

Move left !' NULL.

Stack:

TEXT

A
B
D !• t o p

26. POP D

Pop D.

Visit:

TEXT

D

Move:

TEXT

curr = D->right = NULL

Stack remains:

TEXT

A
B !• t o p

27. POP B

Visit:

TEXT

B

Then:

TEXT

curr = B->right = E

Push E.

Move left !' NULL.

Pop E.

Visit:

TEXT

E

Current output:

TEXT

D B E

28. POP A

Stack:

TEXT

A

Pop and visit:

TEXT

A

Then:

TEXT

curr=C

Push C, then pop/visit.

Final:

D B E A C

29. C IMPLEMENTATION— ITERATIVE

```
C
void inorderIterative(Node *root)
{
    Node *stack[100];
    int top = -1;
    Node *curr = root;

    while (curr != NULL || top != -1)
    {
        while (curr != NULL)
        {
            stack[++top] = curr;
            curr = curr->left;
        }

        curr = stack[top--];

        printf("%d ", curr->data);

        curr = curr->right;
    }
}
```

30. WHY OUTER CONDITION USES OR

```
C
while (curr != NULL || top != -1)
```

Even if:

```
TEXT
curr == NULL
```

there may still be ancestors waiting in stack.

So traversal is not over until:

```
TEXT
curr == NULL
AND
stack empty
```

0-1% important implementation trap.

31. ITERATIVE COMPLEXITY

Every node:

```
TEXT
pushed once
popped once
```

Therefore:

```
TEXT
Time =  $\sim (n)$ 
```

Explicit stack stores at most a root-to-leaf chain:

```
TEXT
 $O(h)$ 
```

Thus:

FORMULA BOX

```
TEXT
Time =  $\sim (n)$ 
Space =  $O(h)$ 
```

Worst case:

TEXT

$O(n)$

32. PREORDER VS INORDER ITERATIVE THINKING

Preorder

Process node immediately.

TEXT

```
pop !' visit !' push children
```

Inorder

Delay processing until left subtree is complete.

TEXT

```
push left chain  
pop !' visit  
go right
```

~~0=1%~~ This conceptual distinction is more useful than memorizing code.

33. ROOT POSITION IN INORDER

Unlike preorder:

TEXT

root is NOT necessarily first

Unlike postorder:

TEXT

root is NOT necessarily last

Root appears between:

TEXT

entire left-subtree sequence

and:

TEXT

entire right-subtree sequence

So:

FORMULA BOX

TEXT

```
Inorder =  
Inorder(Left) + Root + Inorder(Right)
```

34. RECONSTRUCTION CLUE Ø=Ý%Ø=Ý%Ø=Ý%

Suppose:

Preorder:

TEXT

A B D E C F

Inorder:

TEXT

D B E A C F

Preorder tells us:

TEXT

first item = root = A

Find A in inorder:

TEXT

D B E | A | C F

Therefore:

Left subtree nodes:

TEXT

D B E

Right subtree nodes:

TEXT

C F

This is the fundamental reconstruction idea.

We'll master it in Lesson 58.

35. INORDER ALONE IS NOT ENOUGH

Example:

Tree 1:

TEXT

```
A
 \
  B
```

Inorder:

TEXT

A B

Tree 2:

TEXT

```
  B
 /
A
```

Inorder:

TEXT

A B

Same inorder.

Different trees.

Therefore:

Inorder alone does not uniquely determine arbitrary binary tree.

0-0% favorite.

36. PREORDER + INORDER

If all node labels/keys are distinct:

Preorder + Inorder uniquely determine a binary tree.

Why?

- preorder identifies root
- inorder divides left/right subtrees
- recursively repeat

37. POSTORDER + INORDER

Similarly:

Postorder + Inorder uniquely determine a binary tree

with distinct nodes.

Because postorder's:

```
TEXT
last item = root
```

and inorder splits left/right.

38. PREORDER + POSTORDER?

For an arbitrary binary tree:

Not necessarily unique

because neither traversal directly tells whether a single child is left or right.

Example:

```
TEXT
A
/
B
```

and:

```
TEXT
A
 \
  B
```

Preorder both:

```
TEXT
A B
```

Postorder both:

```
TEXT
B A
```

Very important preview.

39. SPECIAL FULL-TREE CASE

For a **full binary tree** with distinct labels:

```
TEXT
Preorder + Postorder
```

can uniquely determine the tree.

Why?

Because no node has exactly one child, removing the main left/right ambiguity.

We'll cover this carefully in reconstruction.

40. BST SEARCH CONNECTION

Because BST inorder is sorted, operations like:

- kth smallest
- kth largest
- validate BST
- ordered output

often use inorder traversal.

Placement importance.

41. KTH SMALLEST INTUITION

BST:

TEXT



Inorder:

TEXT

2 3 4 5 8

3rd smallest:

TEXT

4

So:

Traverse inorder and count visited nodes.

We'll revisit in BST placement lesson.

42. REVERSE INORDER

Standard inorder:

TEXT

L e f t ! ' R o o t ! ' R i g h t

gives ascending BST order.

Reverse inorder:

TEXT

R i g h t ! ' R o o t ! ' L e f t

gives:

Descending order

Example BST:

TEXT

10 20 30 40

reverse inorder:

TEXT

40 30 20 10

Useful for kth largest.

43. INORDER SUCCESSOR INTUITION

In BST inorder sequence:

TEXT

10 20 30 40 50

Successor of 30:

TEXT

40

Predecessor:

TEXT

20

So BST predecessor/successor are closely connected to inorder neighboring values.

We'll study exact algorithms later.

44. GATE-STYLE TREE

TEXT





Find inorder.

Left subtree of 1:

For node 2:

TEXT

4
2

Then subtree 5:

TEXT

7 5 8

So left side:

TEXT

4 2 7 5 8

Then root:

TEXT

1

Right subtree:

TEXT

6 3

because 6 is left child of 3.

Final:

4 2 7 5 8 1 6 3

45. USE SUBTREE BLOCKS

Always decompose:

TEXT

Inorder(left subtree)
Root
Inorder(right subtree)

For previous tree:

TEXT

[4 2 7 5 8] [1] [6 3]

This makes large-tree tracing safer.

46. GATE CODE IDENTIFICATION

C

```
void f(Node *root)
{
    if (root == NULL)
        return;

    f(root->left);
    printf("%d ", root->data);
    f(root->right);
}
```

Traversal:

Inorder

Because visit is:

TEXT

between left and right calls

47. GATE CODE TRAP

C

```
void f(Node *root)
{
    if (root == NULL)
        return;

    f(root->right);
    printf("%d ", root->data);
    f(root->left);
}
```

Order:

TEXT

R i g h t ! ' R o o t ! ' L e f t

This is:

Reverse Inorder

For BST:

TEXT

descending order

48. GATE CODE TRAP — PRINT TWICE

Suppose:

```
c
void f(Node *root)
{
    if (root == NULL)
        return;

    f(root->left);
    printf("%d ", root->data);
    f(root->right);
    printf("%d ", root->data);
}
```

This is not a standard traversal because each node is printed twice.

For output questions, trace the actual code instead of forcing a traversal name.

49. MORRIS INORDER — PLACEMENT EXTENSION

There is an advanced traversal called:

Morris Traversal

It performs inorder using:

TEXT

$O(1)$ auxiliary space

without recursion or explicit stack.

It temporarily uses unused tree links to create threads.

Complexity:

TEXT

Time = $O(n)$
Extra Space = $O(1)$

This is useful for interviews.

For GATE core, understand recursive and stack methods first.

50. THREADED BINARY TREE CONNECTION

Recall binary tree has:

TEXT

n+1 NULL child pointers

Threaded binary trees reuse some NULL links to point to:

- inorder predecessor
- inorder successor

This can make traversal easier without stack/recursion.

Ø=Øbod conceptual extension.

51. INORDER PREDECESSOR / SUCCESSOR

For a node in traversal order:

TEXT

... X NODE Y ...

X is:

TEXT

inorder predecessor

Y is:

TEXT

inorder successor

In a BST, these correspond to the immediately smaller and larger keys when values are distinct.

52. EXPRESSION TREE EXAMPLE

Tree:

TEXT



Inorder:

TEXT

A + B * C - D

But structurally correct parenthesized form:

TEXT

$(A+B) * (C-D)$

Again parentheses are needed to preserve exact tree grouping.

53. NUMBER OF RECURSIVE CALLS

Same as preorder.

For n actual nodes:

TEXT

n

non-NULL calls.

NULL child links:

TEXT

$n+1$

So if function is invoked on every NULL child:

FORMULA BOX

TEXT

Total calls = $2n+1$

Still:

TEXT

$\sim (n)$

time.

54. INORDER PROPERTY OF LEFT SUBTREE

For any node R:

All nodes belonging to its left subtree appear:

TEXT

before R

in inorder.

All nodes belonging to right subtree appear:

TEXT

after R

This holds for any binary tree, regardless of key values.

Ø=§% structural property.

55. BUT "BEFORE ROOT" DOESN'T MEAN SMALLER

For an arbitrary binary tree:

TEXT

left-subtree values

need not be less than root.

Only in BST does:

TEXT

left < root < right

hold.

So don't mix traversal position with numeric ordering.

56. PREORDER VS INORDER ROOT CLUE

Preorder

TEXT

root = first element

Inorder

TEXT

root position unknown unless another traversal/property is given

Postorder

We'll see:

TEXT

root = last element

Ø=§% becomes the heart of reconstruction.

57. ITERATIVE INORDER MEMORY TRICK

LEFT WALL ! ' POP !' RIGHT

Meaning:

1. Push down the left wall.
2. Pop one.
3. Visit it.
4. Enter its right subtree.
5. Again push that right subtree's left wall.

This makes iterative inorder easy to remember.

58. COMPLEXITY TABLE

| Method | Time | Auxiliary Space |
|-----------------|------------|-----------------|
| Recursive | $\sim (n)$ | $O(h)$ |
| Iterative stack | $\sim (n)$ | $O(h)$ |
| Morris | $\sim (n)$ | $O(1)$ |

For skewed tree:

TEXT

$$h = O(n)$$

For balanced tree:

TEXT

$$h = O(\log n)$$

59. GATE-STYLE QUESTION — BST

A BST contains keys:

TEXT

40, 20, 60, 10, 30, 50, 70

in the usual tree shape.

Which traversal produces:

TEXT

10 20 30 40 50 60 70

Answer:

Inorder

60. GATE-STYLE QUESTION — DESCENDING BST

Which traversal gives BST keys in descending order?

Right !' Root !' Left

or:

Reverse Inorder

61. GATE-STYLE UNIQUENESS QUESTION

Which pair uniquely determines an arbitrary binary tree with distinct labels?

A. Preorder only B. Inorder only C. Preorder + Inorder D. Preorder + Postorder always

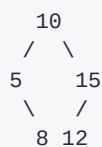
Answer:

C — Preorder + Inorder

62. GATE-STYLE OUTPUT

Tree:

TEXT



Inorder:

Left subtree:

TEXT

5 8

Root:

TEXT

10

Right:

TEXT

12 15

Answer:

5 8 10 12 15

63. NOTEBOOK NOTES

Inorder

TEXT

L E F T ! ' R O O T ! ' R I G H T

Recursive:

```
C
void inorder(Node *root)
{
    if (root == NULL)
        return;

    inorder(root->left);
    visit(root);
    inorder(root->right);
}
```

64. BST SPECIAL PROPERTY

Under standard distinct-key BST rule:

TEXT

Inorder BST = ascending sorted order

Reverse inorder:

TEXT

descending sorted order

~~Do not~~ write this.

65. ITERATIVE TEMPLATE

TEXT

curr=root

while curr != NULL OR stack not empty:

```

while curr != NULL:
    push(curr)
    curr=curr->left

curr=pop()
visit(curr)

curr=curr->right

```

66. DON'T CONFUSE

| Traversal | Order |
|-----------------|-----------------|
| Preorder | Root Left Right |
| Inorder | Left Root Right |
| Reverse Inorder | Right Root Left |

Expression trees:

TEXT

```

P r e o r d e r   ! '   P r e f i x
I n o r d e r     ! '   I n f i x

```

67. MEMORY TRICK

PRE — Root before

IN — Root in middle

POST — Root after

This single mnemonic covers all three DFS traversals.

68. 30-SECOND REVISION

Remember:

1. Inorder = Left !' Root !' Right.
2. Go extreme left first.
3. In BST, inorder gives sorted ascending keys.
4. Reverse inorder gives descending BST order.
5. Time = $\sim (n)$.
6. Recursive/stack space = $O(h)$.
7. Iterative inorder pushes the left chain.

- 8. Root splits inorder into left and right subtree blocks.
- 9. Preorder + inorder uniquely reconstructs distinct-node tree.
- 10. Inorder alone does not uniquely determine arbitrary tree.

69. ACTIVE RECALL

Tree:

TEXT



Q1

Inorder?

Q2

Where does root A appear relative to its subtrees?

Q3

Which traversal of BST gives ascending order?

Q4

What is iterative inorder's main DS?

Q5

What is auxiliary space of recursive inorder?

ANSWERS

A1

TEXT

D B E A C F

A2

After the complete left subtree and before the complete right subtree.

A3

Inorder.

A4

Stack.

A5

TEXT

$O(h)$

70. PRACTICE — EASY

Find inorder:

TEXT



71. PRACTICE — MODERATE

Find inorder:

TEXT



72. PRACTICE — BST

BST:

TEXT



Find:

1. inorder
2. reverse inorder
3. 4th smallest key
4. predecessor of 60
5. successor of 60

You should already be able to infer these from traversal order.

73. SYLLABUS CHECKPOINT

Trees now completed:

' Terminology ' Binary-tree representation ' Types & properties ' Preorder ' **Inorder**

Still remaining:

- Postorder
- Level order
- traversal comparison
- reconstruction
- uniqueness conditions
- remaining tree GATE traps

PROGRESS

MODULE 6 — TREES

' Lesson 50 — Fundamentals ' Lesson 51 — Binary Trees ' Lesson 52 — Binary Tree Types ' Lesson 53 — Formula Masterclass ' Lesson 54 — Preorder ' **Lesson 55 — Inorder**

NEXT — LESSON 56

Postorder Traversal — Left !' Right !' Root, Recursive/Iterative Logic & Deletion Applications

Key rule:

LEFT !' RIGHT !' ROOT

Example:

TEXT



Postorder:

TEXT

B C A

This traversal is extremely useful when children must be processed **before their parent**, such as deleting/freeing an entire tree.

USER

next

Now we study the third classical depth-first traversal:

GATE importance: Ø=Ý%ØÄÝ\$ØKÑW **Placement importance:** Ø=Ý%ØÄÝ\$ØKÑW

The most important idea is:

That makes postorder especially useful for:

- deleting/freeing a tree
- evaluating expression trees
- computing subtree results
- tree DP
- height/size calculations

Postorder follows:

Memory trick:

POST = Root comes after both subtrees.

So:

TEXT

```
Traverse Left
Traverse Right
Visit Root
```

Tree:

TEXT

A

B C

Postorder:

TEXT

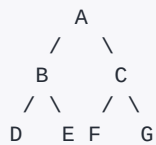
B C A

Why?

1. Left !' B
2. Right !' C
3. Root !' A

3. BIGGER EXAMPLE

TEXT



Postorder of left subtree:

TEXT

D E B

Postorder of right subtree:

TEXT

F G C

Then root:

TEXT

A

Therefore:

D E B F G C A

$\emptyset$ = Root is always last in the postorder traversal of a non-empty tree.

4. RECURSIVE DEFINITION

For every node:

TEXT

Postorder(node):

1. Postorder(node->left)
2. Postorder(node->right)
3. Visit node

5. C IMPLEMENTATION

```
C
void postorder(Node *root)
{
    if (root == NULL)
        return;

    postorder(root->left);
    postorder(root->right);
    printf("%d ", root->data);
}
```

Compare the position of `printf()`.

6. THREE DFS TRAVERSALS — MASTER PATTERN

Preorder

```
C
visit(root);
traverse(left);
traverse(right);
```

TEXT

ROOT LEFT RIGHT

Inorder

```
C
traverse(left);
visit(root);
traverse(right);
```

TEXT

LEFT ROOT RIGHT

Postorder

```
C
traverse(left);
traverse(right);
visit(root);
```

TEXT

LEFT RIGHT ROOT

0-10 is the easiest way to remember all three.

7. EXACT RECURSION TRACE

Tree:

TEXT



Call:

TEXT

postorder(A)

First:

TEXT

postorder(B)

Inside B:

TEXT

postorder(D)

8. INSIDE D

D's left:

TEXT

N U L L ! ' r e t u r n

D's right:

TEXT

N U L L ! ' r e t u r n

Only now print:

TEXT

D

Return to B.

9. BACK TO B

Now:

```
TEXT
postorder(E)
```

E's children are NULL.

So print:

```
TEXT
E
```

After both B subtrees are complete:

```
TEXT
print B
```

Current output:

```
TEXT
D E B
```

10. BACK TO A

Now traverse:

```
TEXT
postorder(C)
```

C has no children.

Print:

```
TEXT
C
```

Finally both subtrees of A are complete.

Print:

```
TEXT
A
```

Final:

D E B C A

11. CALL STACK INTUITION

During initial descent:

TEXT

```
postorder(A)
  postorder(B)
    postorder(D)
```

Stack:

TEXT

```
TOP
D
B
A
```

But D is not processed until both child calls return.

So postorder has the strongest:

"finish children first"

behavior.

12. QUICK EXAM TRICK

For postorder:

Do not write a node when you first reach it. Write it only when you are completely finished with its left and right subtrees.

That is the central intuition.

13. POSTORDER OF LEFT-SKEWED TREE

TEXT

```
      A
     /
    B
   /
  C
 /
D
```

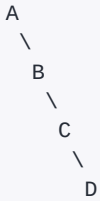
Postorder:

TEXT

```
D C B A
```

14. POSTORDER OF RIGHT-SKEWED TREE

TEXT



Postorder:

TEXT

D C B A

$\emptyset = \text{null}$ can have the same postorder.

Therefore:

Postorder alone does not uniquely determine an arbitrary binary tree.

15. ROOT POSITION $\emptyset = \text{null}$

For a non-empty tree:

Preorder

TEXT

root = first element

Postorder

TEXT

root = last element

Inorder

TEXT

root cannot be identified from inorder alone

This is critical for reconstruction.

16. POSTORDER + INORDER RECONSTRUCTION

Suppose:

Postorder:

TEXT

D E B F C A

Inorder:

TEXT

D B E A F C

Postorder tells us:

TEXT

last = root = A

Find A in inorder:

TEXT

D B E | A | F C

Therefore:

TEXT

Left subtree = D B E
Right subtree = F C

Then repeat recursively.

$\emptyset = \text{nil}$ is why:

Inorder + Postorder uniquely reconstructs a binary tree with distinct labels.

17. POSTORDER ALONE IS NOT ENOUGH

Two trees:

TEXT

A
/
B

and:

TEXT

A
\
B

both have postorder:

TEXT

B A

Different structures, same traversal.

18. EXPRESSION TREE CONNECTION Ø=Ý%Ø=Ý%Ø=Ý%

Expression tree:

TEXT



Postorder:

TEXT

A B C * +

This is:

POSTFIX

Therefore:

TEXT

| | | |
|-------------------|---|---------------|
| P r e o r d e r | ! | P r e f i x |
| I n o r d e r | ! | I n f i x |
| P o s t o r d e r | ! | P o s t f i x |

Ø=Ý%Ø=Ý%Ø=Ý% This unifies Trees with your Stack lessons.

19. WHY POSTORDER IS GOOD FOR EXPRESSION EVALUATION

Consider:

TEXT



Postorder:

TEXT

2 3 4 * +

Evaluate children first:

TEXT

3*4 = 12

then:

TEXT

2+12 = 14

The root operation is naturally evaluated after its operand subtrees.

20. APPLICATION — FREEING A TREE

Suppose nodes are dynamically allocated.

Tree:

TEXT



Can we free A first?

No.

If we do:

C

```
free(root);
```

then trying to access:

C

```
root->left  
root->right
```

afterward is invalid.

So children must be freed first.

That is exactly:

Postorder

21. CORRECT TREE DELETION

C

```
void freeTree(Node *root)  
{  
    if (root == NULL)  
        return;  
  
    freeTree(root->left);  
    freeTree(root->right);  
  
    free(root);  
}
```

Order:

```
TEXT
LEFT
RIGHT
ROOT
```

0-100% Perfect postorder application.

22. WHY PREORDER DELETE IS DANGEROUS

Wrong:

```
C
void wrongDelete(Node *root)
{
    if (root == NULL)
        return;

    free(root);

    wrongDelete(root->left);
    wrongDelete(root->right);
}
```

After:

```
C
free(root);
```

accessing:

```
TEXT
root->left
root->right
```

is use-after-free / undefined behavior.

0-100% Very important C + tree connection.

23. SAFE ALTERNATIVE IF SAVING CHILDREN

Technically you could save:

```
C
Node *left = root->left;
Node *right = root->right;
```

before freeing root.

But standard tree destruction is naturally and cleanly postorder.

So remember:

DELETE TREE !' POSTORDER

24. APPLICATION — COMPUTING HEIGHT

Height of node depends on heights of:

TEXT

```
left subtree  
right subtree
```

So conceptually:

TEXT

```
compute left  
compute right  
combine at root
```

That is postorder-style computation.

Formula:

TEXT

```
height(root)  
=  
1 + max(height(left), height(right))
```

You cannot finalize root height until both children are known.

25. APPLICATION — COUNTING NODES

Likewise:

TEXT

```
count(root)  
=  
1 + count(left) + count(right)
```

The root's result depends on child subtree results.

This is a general:

bottom-up tree computation

pattern.

26. APPLICATION — TREE DP

Many advanced tree problems use:

TEXT

```
solve(left)
solve(right)
combine answers at root
```

Examples:

- subtree size
- tree diameter
- balanced-tree checking
- maximum path sum
- subtree sums
- dynamic programming on trees

This is fundamentally postorder reasoning.

$O(\sqrt{n})$ - Very important placement pattern.

27. TIME COMPLEXITY

Every node visited exactly once.

Therefore:

FORMULA BOX

TEXT

Postorder Time = $\sim(n)$

28. RECURSIVE AUXILIARY SPACE

Call-stack depth depends on tree height.

FORMULA BOX

TEXT

Auxiliary Space = $O(h)$

Balanced tree:

TEXT

$O(\log n)$

Skewed tree:

TEXT

$O(n)$

29. ITERATIVE POSTORDER — WHY HARDER?

Preorder is easy:

TEXT

Root Left Right

because we process node immediately.

Inorder delays root until left completes.

Postorder delays root until:

TEXT

both left AND right

complete.

Therefore iterative postorder requires more careful state tracking.

30. ITERATIVE METHOD 1 — TWO STACKS

This is conceptually simple.

Use:

TEXT

Stack1
Stack2

Algorithm:

TEXT

```
push root into S1

while S1 not empty:
    node = pop S1
    push node into S2

    if node->left:
        push left into S1

    if node->right:
        push right into S1

finally:
    pop everything from S2
```

That output becomes:

TEXT

Left Right Root

31. WHY TWO STACKS WORK

The first phase creates something like:

TEXT

Root Right Left

in reverse arrangement.

Second stack reverses it into:

TEXT

Left Right Root

which is postorder.

32. TWO-STACK TRACE

Tree:

TEXT



Initially:

TEXT

S1 = [A]

S2 = []

Pop A from S1.

Push A to S2.

Push B,C into S1.

One possible convention:

TEXT

S1:

B

C !• t o p

33. CONTINUE

Pop C:

TEXT

S2: A C

C has no children.

Pop B:

TEXT

S2: A C B

Push D,E.

Then E and D are processed.

Eventually S2 stores such that popping it gives:

TEXT

D E B C A

Correct postorder.

34. C IMPLEMENTATION— TWO STACKS

```
C
void postorderIterative(Node *root)
{
    if (root == NULL)
        return;

    Node *s1[100], *s2[100];
    int top1 = -1, top2 = -1;

    s1[++top1] = root;

    while (top1 != -1)
    {
        Node *curr = s1[top1--];

        s2[++top2] = curr;

        if (curr->left != NULL)
            s1[++top1] = curr->left;

        if (curr->right != NULL)
            s1[++top1] = curr->right;
    }

    while (top2 != -1)
        printf("%d ", s2[top2--]->data);
}
```

35. TWO-STACK COMPLEXITY

Each node is pushed/popped a constant number of times.

Therefore:

TEXT

```
T i m e   =   ~ ( n )
```

Auxiliary space:

TEXT

```
O(n)
```

because two explicit stacks may together hold $O(n)$ nodes.

36. ITERATIVE METHOD 2 — ONE STACK

More space-efficient but more subtle.

We must know whether the right subtree has already been processed.

A common approach keeps:

TEXT

```
curr  
lastVisited  
stack
```

37. ONE-STACK IDEA

1. Push left path.
2. Look at stack top.
3. If it has an unvisited right child:
 - move to that right child.
4. Otherwise:
 - visit/pop the node.
 - mark it as last visited.

This prevents processing a parent before its right subtree.

38. ONE-STACK PSEUDOCODE

TEXT

```
curr = root  
last = NULL  
  
while curr != NULL OR stack not empty:  
  
    if curr != NULL:  
        push(curr)  
        curr = curr->left  
  
    else:  
        peek = stack.top
```

```

if peek->right != NULL
    AND last != peek->right:

    curr = peek->right

else:
    visit(peek)
    last = pop()

```

Useful interview technique.

39. WHY lastVisited?

Suppose node A has right child C.

After returning from C, stack top becomes A again.

Without remembering that C was already completed, algorithm might enter C again.

lastVisited tells us:

TEXT
right subtree already done

so A can now be processed.

40. GATE CORE VS PLACEMENT

For GATE, prioritize:

Recursive postorder traversal sequence root-last property expression/postfix relation construction complexity

For placements, also know:

Two-stack postorder One-stack postorder Bottom-up tree DP reasoning

41. GATE-STYLE EXAMPLE

Tree:

TEXT

```

      1
     / \
    2   3
   / \ /
  4  5 6
     / \
    7  8

```

Postorder left subtree of 1:

Node 2:

Left:

TEXT

4

Right subtree rooted 5:

TEXT

7 8 5

Then 2:

TEXT

4 7 8 5 2

42. RIGHT SUBTREE

Node 3 has left child 6.

Postorder:

TEXT

6 3

Then root 1.

Final:

4 7 8 5 2 6 3 1

43. QUICK SUBTREE METHOD

Write:

TEXT

```
Postorder(left subtree)
Postorder(right subtree)
Root
```

For above:

TEXT

[4 7 8 5 2] [6 3] [1]

This is safer than mentally jumping around.

44. GATE CODE IDENTIFICATION

```

C
void f(Node *root)
{
    if (root == NULL)
        return;

    f(root->left);
    f(root->right);
    printf("%d ", root->data);
}

```

Because visit happens after both recursive calls:

Postorder

45. GATE CODE TRAP

```

C
void f(Node *root)
{
    if (root == NULL)
        return;

    f(root->right);
    f(root->left);
    printf("%d ", root->data);
}

```

Order:

```

TEXT
R i g h t  ! '   L e f t  ! '   R o o t

```

This is not standard postorder.

Standard remains:

```

TEXT
L e f t  ! '   R i g h t  ! '   R o o t

```

46. GATE CODE TRAP — OPERATION AFTER LEFT ONLY

```

C
f(root->left);
print(root);
f(root->right);

```

That is:

TEXT

Left Root Right

= inorder.

Always look at `print` location.

47. ROOT-LAST PROPERTY

If a postorder sequence is:

TEXT

D E B F C A

then:

TEXT

A

must be root.

~~0-1%~~ This is one of the fastest reconstruction clues.

48. EXPRESSION TREE EXAMPLE

Tree:

TEXT



Postorder:

TEXT

A B + C D - *

This is postfix for:

TEXT

$(A+B) * (C-D)$

Exactly matches your stack module.

49. POSTORDER EVALUATION CONNECTION

For numeric expression tree:

TEXT



Postorder:

TEXT

2 3 + 8 4 - *

Evaluate:

TEXT

2+3 = 5
8-4 = 4
5*4 = 20

Answer:

TEXT

20

50. DELETE TREE TRACE

Tree:

TEXT



Postorder free order:

TEXT

B
C
A

Correct because:

TEXT

children gone first
parent gone last

51. MORE COMPLEX DELETE ORDER

TEXT



Safe free order:

TEXT

D E B C A

Exactly postorder.

52. NUMBER OF RECURSIVE CALLS

Same structural result as preorder/inorder.

For n nodes:

Non-NULL calls:

TEXT

n

NULL child calls:

TEXT

$n+1$

Total:

FORMULA BOX

TEXT

$2n + 1$

if counting every invocation including NULL calls.

Still:

TEXT

$\sim (n)$

time.

53. POSTORDER AND SUBTREE CONTIGUITY

For any subtree rooted at X, its entire postorder appears as a contiguous block.

And:

TEXT

X appears at the END of that block

Example:

TEXT



Postorder:

TEXT

[D E B] [C] A

B's subtree block:

TEXT

D E B

ends with B.

Useful reconstruction property.

54. PREORDER VS POSTORDER — PARENT RELATION

For any node and its descendants:

Preorder

Parent appears:

TEXT

before descendants

Postorder

Parent appears:

TEXT

after descendants

This is why:

- preorder is good for top-down processing
- postorder is good for bottom-up processing

55. TOP-DOWN VS BOTTOM-UP

Preorder-style

TEXT

```
process parent
then children
```

Useful when parent information flows downward.

Postorder-style

TEXT

```
process children
then parent
```

Useful when child results flow upward.

Excellent placement intuition.

56. EXAMPLE — SUBTREE SIZE

For each node:

TEXT

```
size(node)
=
1 + size(left) + size(right)
```

Need:

TEXT

```
size(left)
size(right)
```

first.

Therefore conceptually postorder.

57. EXAMPLE — BALANCED TREE CHECK

To decide whether node is height-balanced:

Need:

TEXT

```
height(left)
height(right)
```

first.

Then compare:

TEXT

```
abs(leftHeight-rightHeight)
```

Again postorder-style.

58. EXAMPLE — TREE DIAMETER $\emptyset = \text{max}$

Diameter often depends on:

TEXT

```
left height
right height
left diameter
right diameter
```

all child-subtree results.

Then combine at current root.

This is another classic postorder pattern.

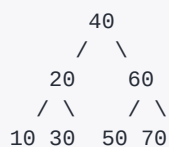
We'll revisit such ideas in placement consolidation.

59. POSTORDER OF BST

Unlike inorder, postorder of BST is **not sorted**.

Example BST:

TEXT



Postorder:

TEXT

```
10 30 20 50 70 60 40
```

Root appears last.

60. BST POSTORDER STRUCTURE

For standard distinct-key BST with root **R**:

Postorder has:

TEXT

```
[left subtree keys]
[right subtree keys]
```

R

Left block values:

TEXT

< R

Right block values:

TEXT

> R

This can be useful for validating/reconstructing BST postorder sequences.

Ø = Placement extension.

61. PREORDER + POSTORDER AMBIGUITY

Again:

Tree 1:

TEXT

A

/

B

Tree 2:

TEXT

A

\

B

Preorder:

TEXT

A B

both.

Postorder:

TEXT

B A

both.

Therefore:

TEXT

Preorder + Postorder

does not uniquely determine an arbitrary binary tree.

$\emptyset = \text{Nil}$ must know.

62. BUT FULL BINARY TREE SPECIAL CASE

In a full binary tree, every internal node has exactly two children.

The single-child ambiguity disappears.

So for distinct labels:

TEXT

Preorder + Postorder

can uniquely determine a full binary tree.

We will formally cover this in Lesson 58.

63. POSTORDER COMPLEXITY TABLE

| Method | Time | Auxiliary Space |
|------------|------------|-------------------------------|
| Recursive | $\sim (n)$ | $O(h)$ |
| Two stacks | $\sim (n)$ | $O(n)$ |
| One stack | $\sim (n)$ | $O(h)$ typical / worst $O(n)$ |

For recursive traversal:

Balanced:

TEXT

$O(\log n)$

stack.

Skewed:

TEXT

$O(n)$

stack.

64. MEMORY TRICK

POST = Parents wait patiently.

Parent waits until:

TEXT

LEFT finished
RIGHT finished

then gets processed.

So:

L R ROOT

65. EXPRESSION MEMORY TRICK

TEXT

P R E ! ' P R E f i x
I N ! ' I N f i x
P O S T ! ' P O S T f i x

Ø=X% most impossible to forget.

66. NOTEBOOK NOTES

Postorder

TEXT

L E F T ! ' R I G H T ! ' R O O T

Recursive:

```
C
void postorder(Node *root)
{
    if (root == NULL)
        return;

    postorder(root->left);
    postorder(root->right);
    visit(root);
}
```

67. IMPORTANT PROPERTIES

TEXT

Root = last element

TEXT

T i m e = ~ (n)

TEXT

Recursive space = $O(h)$

Expression tree:

TEXT

Postorder = Postfix

Deleting tree:

TEXT

Postorder

68. DON'T CONFUSE

| Traversal | Root position |
|-----------|-------------------------|
| Preorder | First |
| Inorder | Middle between subtrees |
| Postorder | Last |

Very important table.

69. 30-SECOND REVISION

Remember:

1. Postorder = Left !' Right !' Root.
2. Root is last.
3. Children finish before parent.
4. Recursive time = $\sim (n)$.
5. Recursive auxiliary space = $O(h)$.
6. Postorder expression-tree traversal gives postfix.
7. Postorder is ideal for freeing/deleting a tree.
8. Two-stack iterative postorder is easy.
9. One-stack version needs state like `lastVisited`.
10. Inorder + postorder can reconstruct a distinct-node tree uniquely.

70. ACTIVE RECALL

Tree:

TEXT



Q1

Postorder?

Q2

Which node appears last?

Q3

Which traversal is best for safely freeing a dynamically allocated tree?

Q4

What notation does expression-tree postorder generate?

Q5

Recursive auxiliary space?

ANSWERS

A1

TEXT

D E B F C A

A2

TEXT

A

A3

Postorder.

A4

Postfix.

A5

TEXT

$O(h)$

71. PRACTICE — EASY

Find postorder:

TEXT



72. PRACTICE — MODERATE

Find postorder:

TEXT



73. PRACTICE — EXPRESSION TREE

TEXT



Find:

1. preorder
2. inorder with appropriate parentheses
3. postorder

Then identify:

- prefix
- infix
- postfix

74. GATE-STYLE QUESTION

A postorder traversal is:

TEXT

2 6 4 10 14 12 8

Which value must be the root?

Answer:

TEXT

8

because root is last in postorder.

75. GATE-STYLE CODE QUESTION

What traversal?

```
c
void f(Node *root)
{
    if (!root)
        return;

    f(root->left);
    f(root->right);
    free(root);
}
```

Answer:

Postorder-style deletion

because parent is freed after both child subtrees.

76. SYLLABUS CHECKPOINT

Trees now completed:

' Terminology ' Binary-tree representation ' Types & formulas ' Preorder ' Inorder ' **Postorder**

Still pending before Trees are marked complete:

- Level-order traversal
- traversal comparison
- iterative/recursive relationships consolidation
- reconstruction from traversals
- uniqueness conditions
- final tree GATE traps

PROGRESS

MODULE 6 — TREES

' Lesson 50 — Tree Fundamentals ' Lesson 51 — Binary Trees ' Lesson 52 — Binary Tree Types ' Lesson 53 — Formula Masterclass ' Lesson 54 — Preorder ' Lesson 55 — Inorder ' **Lesson 56 — Postorder**

NEXT — LESSON 57

Level-Order Traversal — Queue-Based BFS of a Binary Tree

Next we switch from depth-first traversal to:

Breadth First / Level Order

Example:

TEXT



Level order:

TEXT

A B C D E

We will cover:

- queue algorithm
- exact queue traces
- C implementation
- $O(n)$ complexity
- maximum queue size
- level-by-level processing
- tree width
- BFS connection
- GATE traps
- comparison with DFS traversals.

USER

next

ASSISTANT

MODULE 6 — LESSON 57

Level-Order Traversal — Queue-Based BFS of a Binary Tree

So far we completed the three DFS traversals:

```

P r e o r d e r      =   R o o t  ! '   L e f t  ! '   R i g h t
I n o r d e r       =   L e f t  ! '   R o o t  ! '   R i g h t
P o s t o r d e r    =   L e f t  ! '   R i g h t  ! '   R o o t

```

LEVEL-ORDER TRAVERSAL

Level by level, from top to bottom

BFS on a tree

1. BASIC INTUITION

```

      A
     / \
    B   C
   / \
  D   E

```

```
Level 0: A
Level 1: B C
Level 2: D E
```

A B C D E

2. WHY QUEUE?

A

Page 2143 of 2799
www.gpttopdf.in

TEXT

B, C

B is discovered before C, so B should be processed before C.

That requires:

FIFO

Therefore:

QUEUE

$\emptyset = \text{Memory}$:

Level order = BFS = Queue

3. CORE ALGORITHM

For a non-empty tree:

TEXT

```
1. Enqueue root

2. While queue is not empty:
    node = dequeue()

    visit(node)

    if node has left child:
        enqueue(left)

    if node has right child:
        enqueue(right)
```

That is the complete idea.

4. SIMPLE TRACE

Tree:

TEXT

```
  A
 / \
B   C
```

Start:

TEXT

Queue = [A]

Dequeue A.

Output:

```
TEXT
A
```

Enqueue B,C:

```
TEXT
Queue = [B, C]
```

Dequeue B:

```
TEXT
Output = A B
Queue = [C]
```

Dequeue C:

```
TEXT
Output = A B C
Queue = []
```

Traversal complete.

5. BIGGER TRACE Ø=Ý%

Tree:

```
TEXT
      A
     / \
    B   C
   / \ / \
  D  E F  G
```

Start:

```
TEXT
Queue = [A]
Output = -
```

6. PROCESS A

Remove A.

```
TEXT
Output = A
```

Insert its children:

TEXT

Queue = [B, C]

7. PROCESS B

Remove B.

TEXT

Output = A B

Insert:

TEXT

D, E

Queue:

TEXT

[C, D, E]

8. PROCESS C

Remove C.

TEXT

Output = A B C

Insert:

TEXT

F, G

Queue:

TEXT

[D, E, F, G]

9. PROCESS REMAINING

Then:

TEXT

D
E
F
G

Final:

A B C D E F G

Notice that all level-1 nodes were processed before level-2 nodes.

10. C IMPLEMENTATION

Assume:

```
C
typedef struct Node
{
    int data;
    struct Node *left;
    struct Node *right;
} Node;
```

A simple array-based queue of node pointers:

```
C
void levelOrder(Node *root)
{
    if (root == NULL)
        return;

    Node *queue[100];
    int front = 0, rear = 0;

    queue[rear++] = root;

    while (front < rear)
    {
        Node *curr = queue[front++];

        printf("%d ", curr->data);

        if (curr->left != NULL)
            queue[rear++] = curr->left;

        if (curr->right != NULL)
            queue[rear++] = curr->right;
    }
}
```

For actual general-purpose code, the queue should support enough nodes dynamically or use a proper queue implementation, but this is excellent for understanding the algorithm.

11. WHY QUEUE STORES POINTERS

The queue contains:

```
TEXT
Node *
```

not merely node values.

Why?

Because after dequeuing a node, we need:

```
c
curr->left
curr->right
```

to discover its children.

Storing only:

```
TEXT
curr->data
```

would lose the structural links.

Ø = 1/4 Important C implementation point.

12. TIME COMPLEXITY

Every node is:

```
TEXT
enqueued once
dequeued once
visited once
```

Therefore:

FORMULA BOX

```
TEXT
Time = ~ ( n )
```

13. AUXILIARY SPACE

Queue stores nodes waiting to be processed.

Worst-case queue size depends on the maximum number of nodes simultaneously in the frontier.

Therefore:

FORMULA BOX

```
TEXT
Space = O(w)
```

where:

TEXT

w = maximum width of the tree

Since:

TEXT

$w \leq n$

worst-case:

TEXT

$O(n)$

~~$O(n)$~~ More precise than always saying $O(n)$.

14. PERFECT TREE QUEUE SIZE

Consider a perfect binary tree of height h .

Maximum width:

TEXT

2^h

At the deepest level there may be:

TEXT

2^h

nodes waiting.

Since:

TEXT

$n = 2^{(h+1)} - 1$

deepest level has approximately:

TEXT

$(n+1)/2$

nodes.

Thus queue space can be:

TEXT

$\sim (n)$

even though the tree height is only:

TEXT

$\sim (\log n)$

Important comparison with DFS recursion.

15. DFS VS BFS SPACE

For a balanced tree:

Recursive DFS

Uses:

TEXT

$O(h) = O(\log n)$

stack.

Level-order BFS

May use:

TEXT

$O(n)$

queue because a whole wide level can be waiting.

So BFS can require much more memory on a wide tree.

16. SKEWED TREE

TEXT



At any time queue contains roughly:

TEXT

1 node

So level-order auxiliary space:

TEXT

$O(1)$

for this specific shape.

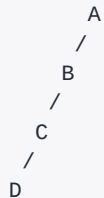
Traversal:

TEXT

A B C D

17. LEVEL ORDER OF LEFT-SKEWED TREE

TEXT



Output:

TEXT

A B C D

Interesting:

For a pure chain, multiple traversals can produce similar-looking sequences depending on direction.

Again, one traversal alone usually does not uniquely identify shape.

18. LEVEL-BY-LEVEL PROCESSING Ø=Ý%Ø=Ý%

Sometimes we do not merely want:

TEXT

A B C D E F

We want to know which nodes belong to each level.

Example:

TEXT



Desired grouping:

TEXT

Level 0: A

Level 1: B C

Level 2: D E F

How?

Use:

current queue size

19. KEY IDEA

At the start of a level:

TEXT

```
levelSize = number of nodes currently in queue
```

Process exactly:

TEXT

```
levelSize
```

nodes.

Their children are inserted for the next level.

Very important placement pattern.

20. LEVEL-BY-LEVEL PSEUDOCODE

TEXT

```
enqueue(root)

while queue not empty:

    levelSize = queue.size()

    repeat levelSize times:
        node = dequeue()
        visit node

    enqueue children
```

After those `levelSize` nodes are processed:

TEXT

```
one complete level is finished
```

21. TRACE

Initial:

TEXT

```
Queue = [A]
```

So:

TEXT

```
levelSize = 1
```

Process A.

During processing, enqueue B,C.

Queue becomes:

TEXT

```
[B,C]
```

Level 0 complete.

22. NEXT LEVEL

Now:

TEXT

```
levelSize = 2
```

Process exactly:

TEXT

```
B  
C
```

Even though their children are being appended.

For example after B:

TEXT

```
[C,D,E]
```

After C:

TEXT

```
[D,E,F]
```

But we stop after 2 nodes because:

TEXT

```
levelSize was 2
```

So B and C alone formed that level.

Do not simply process until queue becomes empty inside the level loop.

23. LEVEL-ORDER WITH DEPTH

We can use this to determine node depth.

Start:

```
TEXT
level = 0
```

For each batch:

```
TEXT
all current nodes have depth = level
```

then:

```
TEXT
level++
```

Example:

```
TEXT
A !' depth 0
B , C !' depth 1
D , E , F !' depth 2
```

24. TREE HEIGHT USING LEVEL ORDER

One way to compute height:

Count the number of levels.

If:

```
TEXT
number of levels = L
```

under edge-based height:

FORMULA BOX

```
TEXT
height = L - 1
```

Example:

```
TEXT
3 levels
```

Height:

```
TEXT
2
```

25. WIDTH OF A TREE Ø=Ý%

Recall:

Width of a level = number of nodes on that level.

Width of a tree = maximum level width.

Using level-order traversal:

TEXT

```
levelSize = queue size at start of level
```

So:

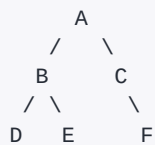
TEXT

```
maxWidth = max(maxWidth, levelSize)
```

Easy.

26. EXAMPLE WIDTH

TEXT



Widths:

TEXT

```
Level 0 = 1  
Level 1 = 2  
Level 2 = 3
```

Therefore:

TEXT

```
Tree width = 3
```

27. LEVEL-ORDER VS PREORDER

Tree:

TEXT



/ \
D E

Preorder

TEXT
A B D E C

Level order

TEXT
A B C D E

Why difference?

Preorder goes:

TEXT
deep into left subtree

Level order finishes:

TEXT
entire current depth

first.

28. LEVEL-ORDER VS DFS

Three DFS traversals:

TEXT
Preorder
Inorder
Postorder

all descend recursively through subtrees.

Level order is:

TEXT
BFS

So:

| Category | Traversals |
|----------|------------------------------|
| DFS | Preorder, Inorder, Postorder |
| BFS | Level Order |

Ø=Must know.

29. DATASTRUCTURE COMPARISON

| Traversal | Natural DS |
|---------------------|----------------|
| Recursive Preorder | Call stack |
| Recursive Inorder | Call stack |
| Recursive Postorder | Call stack |
| Iterative DFS | Explicit stack |
| Level order | Queue |

Memory:

Depth != Stack Breadth != Queue

30. GATE TRAP — STACK FOR LEVEL ORDER?

If you replace the queue with a normal stack:

TEXT
LIFO

you generally get depth-first behavior, not proper level order.

So:

Standard level order requires FIFO queue.

31. GATE TRAP — CHILD INSERTION ORDER

Suppose we require left-to-right level order.

When processing a node:

TEXT
enqueue left
enqueue right

If instead you enqueue:

TEXT
right before left

you get right-to-left ordering within each next level.

Example:

TEXT



Left-first:

TEXT

A B C

Right-first:

TEXT

A C B

~~Queue~~ Queue preserves the order in which children are inserted.

32. EXACT ORDER DEPENDS ON CHILD ORDER

So "level-order traversal" usually assumes:

TEXT

left child before right child

unless stated otherwise.

Always inspect implementation/question.

33. GATE OUTPUT QUESTION

C

```
enqueue(root);

while (!empty())
{
    Node *x = dequeue();

    printf("%d ", x->data);

    if (x->right)
        enqueue(x->right);

    if (x->left)
        enqueue(x->left);
}
```

This is breadth-first, but visits each level:

TEXT

right before left

not standard left-to-right level order.

Place code exactly.

34. LEVEL-ORDER OF EXPRESSION TREE

Expression tree:

TEXT



Level order:

TEXT

+ A * B C

Unlike:

TEXT

```
P r e o r d e r      ! '   P r e f i x
I n o r d e r      ! '   I n f i x
P o s t o r d e r ! '   P o s t f i x
```

level-order does not correspond to the standard prefix/infix/postfix expression notation.

35. APPLICATION — SHORTEST DEPTH

Suppose we need:

Minimum depth from root to any leaf.

BFS is useful because it explores:

TEXT

```
depth 0
depth 1
depth 2
...
```

The first leaf encountered in level order gives minimum depth.

Placement pattern.

36. IMPORTANT LEAF CONDITION

Leaf means:

C

```
node->left == NULL &&  
node->right == NULL
```

So during BFS:

TEXT

first actual leaf encountered

can determine minimum root-to-leaf depth.

Do not confuse a node with only one missing child as a leaf.

37. APPLICATION — RIGHT SIDE VIEW

Tree:

TEXT



Viewed from right:

TEXT

1 3 4

Using level-order:

Take the last node of each level.

38. LEFT SIDE VIEW

Likewise:

Take the first node of each level.

For same tree:

TEXT

1 2 5

depending on structure.

This shows how many placement problems are just variations of level-order traversal.

39. APPLICATION — LEVEL SUM

Suppose:

TEXT

```
      10
     /  \
    20   30
   /  \
  40   50
```

Level sums:

TEXT

```
Level 0 = 10
Level 1 = 50
Level 2 = 90
```

Use:

TEXT

```
levelSize
```

to process each level.

40. APPLICATION — LEVEL AVERAGE

Similarly:

TEXT

```
average = levelSum / levelSize
```

Again BFS by levels.

41. APPLICATION — ZIGZAG LEVEL ORDER

Normal:

TEXT

```
Level 0 !' left to right
Level 1 !' left to right
Level 2 !' left to right
```

Zigzag:

TEXT

```
Level 0 !' left to right
Level 1 !' right to left
Level 2 !' left to right
...
```

Example:

TEXT



Zigzag:

TEXT

```
A
C B
D E F G
```

Can be implemented using:

- queue plus reversed level storage
- deque
- two stacks

Ø = Placement extension.

42. APPLICATION — CONNECTING NODES AT SAME LEVEL

If nodes contain a `next` pointer, level-order traversal helps connect:

TEXT

```
B !' C
```

at the same depth.

This is another BFS pattern.

43. APPLICATION — SERIALIZATION

One tree serialization method stores nodes in level order and records missing children.

Example:

TEXT

```
A
/
B
```

might serialize conceptually as:

TEXT

```
A B # # #
```

depending on format.

This is common in coding platforms.

44. BFS ON TREE DOES NOT NEED VISITED ARRAY

For a normal rooted binary tree:

TEXT

```
p a r e n t ! ' c h i l d
```

and no child points back to parent in our structure.

There are no cycles.

So when traversing only `left` and `right`:

TEXT

```
visited[] is unnecessary
```

Important difference from general graph BFS.

45. WHEN VISITED MAY BE NEEDED

If tree nodes also have:

TEXT

```
parent pointer
```

and we treat parent as another traversable neighbor, then:

TEXT

```
c h i l d ! ' p a r e n t ! ' c h i l d
```

could cause revisits.

In that kind of graph-like traversal:

TEXT

```
visited
```

may be required.

Placement extension.

46. GATE TRAP — TREE VS GRAPH BFS

Tree BFS

Usually:

TEXT

```
Queue only
```

because no cycles through child pointers.

General graph BFS

Usually:

TEXT

```
Queue + visited[]
```

because cycles/repeated paths may exist.

47. LEVEL ORDER AND COMPLETE TREE $\emptyset = \hat{Y}\%$

Complete binary trees are filled in level order:

TEXT

```
top-to-bottom  
left-to-right
```

Therefore level-order numbering corresponds naturally to array indices.

Example:

TEXT

```
      A  
     /\   
    B  C  
   /\  /\   
  D  E F
```

Level order:

TEXT

```
A B C D E F
```

0-based array:

TEXT

```
0 1 2 3 4 5  
A B C D E F
```

This is exactly why binary heaps map beautifully into arrays.

48. HEAP CONNECTION

Heap operations depend on array indexes:

0-based:

TEXT

```
left  = 2i+1  
right = 2i+2
```

The array itself is essentially:

Level-order storage of the complete binary tree

~~O=V%~~ Excellent conceptual connection.

49. LEVEL ORDER C USING LINKED QUEUE IDEA

Instead of fixed array, conceptually:

```
c
enqueue(&q, root);

while (!isEmpty(&q))
{
    Node *curr = dequeue(&q);

    printf("%d ", curr->data);

    if (curr->left)
        enqueue(&q, curr->left);

    if (curr->right)
        enqueue(&q, curr->right);
}
```

All queue operations should be:

TEXT

$O(1)$

for overall:

TEXT

$O(n)$

time.

50. WHAT IF DEQUEUE WERE $O(n)$?

Suppose a bad queue implementation shifts all remaining elements on each dequeue.

Then level-order could become much slower.

Potentially:

TEXT

$O(n^2)$

in worst case.

~~O=V%~~ Data-structure implementation affects algorithm complexity.

Use proper queue operations.

51. MAXIMUM QUEUE SIZE VS HEIGHT

Do not say:

TEXT

queue space = $O(h)$

That is generally false for BFS.

Example perfect tree height 10:

TEXT

height = 10

but last level can contain:

TEXT

$2^{10} = 1024$

nodes.

So queue tracks width, not depth.

FORMULA BOX

TEXT

BFS space = $O(\text{width})$

52. DFS STACK VS BFS QUEUE MEMORY

A useful exam comparison:

DFS

Memory follows:

TEXT

height/depth

BFS

Memory follows:

TEXT

width/frontier

Memory trick:

****DFS remembers the path.**

BFS remembers the frontier.**

Excellent note.

53. GATE-STYLE TRACE

Tree:

TEXT



Level 0:

TEXT

1

Level 1:

TEXT

2 3

Level 2:

TEXT

4 5 6

Level 3:

TEXT

7 8

Therefore:

1 2 3 4 5 6 7 8

54. COMPARE ALL FOUR TRAVERSALS

For:

TEXT



Preorder

TEXT
A B D E C

Inorder

TEXT
D B E A C

Postorder

TEXT
D E B C A

Level order

TEXT
A B C D E

0=00 Copy this example.

55. MASTER TRAVERSAL TABLE

| Traversal | Order | Main DS |
|-------------|----------------|-------------------|
| Preorder | Root L R | Stack / recursion |
| Inorder | L Root R | Stack / recursion |
| Postorder | L R Root | Stack / recursion |
| Level order | Level by level | Queue |

56. ROOT POSITION TABLE

| Traversal | Root position |
|-------------|--------------------------------------|
| Preorder | First |
| Inorder | Between left/right subtree sequences |
| Postorder | Last |
| Level order | First |

Important:

Both preorder and level order begin with root, but their later order differs.

57. GATE TRAP — ROOT FIRST DOES NOT MEAN PREORDER

Suppose sequence starts with root.

Could be:

TEXT

Preorder

or:

TEXT

Level order

or some custom traversal.

So first-element information alone is not enough to identify traversal.

58. GATE-STYLE QUESTION

Which data structure is necessary for standard level-order traversal?

A. Stack B. Queue C. Heap D. BST

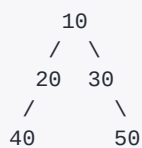
Answer:

B — Queue

59. GATE-STYLE QUESTION

What is level order of:

TEXT



Answer:

TEXT

10 20 30 40 50

60. GATE-STYLE COMPLEXITY QUESTION

A binary tree has n nodes. A proper queue with $O(1)$ enqueue/dequeue is used for level order.

Time:

TEXT

$\sim (n)$

Worst-case auxiliary space:

TEXT

$O(n)$

More precisely:

TEXT

$O(\text{width})$

61. GATE-STYLE WIDTH QUESTION

Tree levels contain:

TEXT

1, 2, 4, 3, 1

nodes.

Tree width:

TEXT

$\max(1, 2, 4, 3, 1)$
 $=4$

Maximum queue usage is $\sim (4)$ for that traversal, ignoring constant implementation details.

62. GATE-STYLE COMPLETE TREE CONNECTION

A complete binary tree has 15 nodes.

Its level order corresponds to array positions:

TEXT

0 ... 14

Because complete tree has no gaps in level-order storage.

Its height:

TEXT

$\text{floor}(\log_2 15) = 3$

Last level maximum width:

TEXT

$2^3=8$

63. GATE TRAP — LEVEL NUMBER CONVENTION

If root level:

TEXT

0

then children are level 1.

If root level:

TEXT

1

children are level 2.

Traversal order is unchanged.

Only numerical level labels change.

Ø = Always check convention.

64. GATE TRAP — MINIMUM DEPTH

Suppose:

TEXT

```
  A
 /
B
 /
C
```

and A has no right child.

Do not say minimum depth is 1 simply because right is NULL.

Minimum depth is root-to-**leaf**, not root-to-missing-pointer.

Leaf:

TEXT

C

So edge-based minimum leaf depth:

TEXT

2

Ø = Very useful tree trap.

65. PLACEMENT PATTERN— MULTI-SOURCE DOESN'T APPLY DIRECTLY TO NORMAL ROOTED TREE

Normal tree level order starts from:

```
TEXT
one root
```

But in graph/grid BFS, several sources can be placed in the queue initially.

That is:

```
TEXT
multi-source BFS
```

Queue idea is the same.

66. LEVEL ORDER USING RECURSION?

It is possible to print nodes level by level using recursion:

```
TEXT
for each level:
    print nodes at that level
```

But naive versions can cost:

```
TEXT
O(nh)
```

and become:

```
TEXT
O(n2)
```

on skewed trees.

Queue-based BFS gives:

```
TEXT
O(n)
```

directly.

~~O=O(n)~~ Queue is the standard efficient method.

67. WHY NAIVE RECURSIVE LEVEL PRINT CAN BE O(n²)

Suppose height:

TEXT

$h=n-1$

For every level, you start again from root.

Repeated traversal causes:

TEXT

$1 + 2 + 3 + \dots + n$

work.

That is:

TEXT

$\sim (n^2)$

So traversal design matters.

68. NOTEBOOK NOTES

Level Order

TEXT

Visit nodes level by level

Natural DS:

TEXT

Queue

Algorithm:

TEXT

enqueue root

while queue not empty:

$x = \text{dequeue}$

 visit x

 enqueue left

 enqueue right

69. COMPLEXITY BOX

FORMULA BOX

TEXT

`T i m e = ~ ( n )`

TEXT

`Auxiliary space = O(width)`

Worst case:

TEXT

`O(n)`

70. LEVEL GROUPING BOX

At beginning of each level:

TEXT

`levelSize = queue.size()`

Process exactly:

TEXT

`levelSize`

nodes.

Applications:

- depth
- level sum
- averages
- views
- zigzag
- maximum width

71. DON'T CONFUSE

DFS

TEXT

Preorder
Inorder
Postorder

uses depth-first behavior.

BFS

TEXT

Level order

uses queue.

BFS space

TEXT

depends on width

DFS recursion space

TEXT

depends on height

Don't copy this distinction.

72. MEMORY TRICKS

Level Order

Finish the floor before taking the stairs down.

BFS

Breadth = Queue.

Width

How crowded is the busiest floor?

Level processing

Freeze queue size before processing the level.

73. 30-SECOND REVISION

Remember:

1. Level order = level-by-level traversal.
2. It is BFS on a tree.
3. Use a queue.
4. Enqueue root first.
5. Dequeue !' visit !' enqueue left !' enqueue right.
6. Time = $\sim (n)$.
7. Space = $O(\text{width})$, worst-case $O(n)$.

8. Queue child insertion order determines left/right order.
9. `levelSize` lets us process one level at a time.
10. Complete-tree arrays correspond naturally to level-order storage.

74. ACTIVE RECALL

Tree:

TEXT



Q1

Level order?

Q2

Which DS is used?

Q3

Time complexity?

Q4

What determines BFS auxiliary space more precisely?

Q5

How do we know how many nodes belong to the current level?

ANSWERS

A1

TEXT

A B C D E F

A2

Queue.

A3

TEXT

$\sim (n)$

A4

Maximum tree width/frontier.

A5

Store the queue size at the start of the level.

75. PRACTICE — EASY

Find level order:

TEXT



76. PRACTICE — MODERATE

Find:

- level order
- width
- number of levels
- height using edge convention

for:

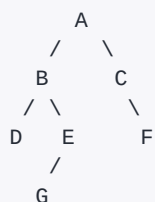
TEXT



77. PRACTICE — ALL TRAVERSALS

For:

TEXT



Find:

1. Preorder
2. Inorder
3. Postorder

4. Level order

This is excellent traversal revision.

78. TRAVERSAL MASTER SUMMARY

For a node x :

Preorder

TEXT

x ! ' L e f t ! ' R i g h t

Parent before children.

Inorder

TEXT

L e f t ! ' x ! ' R i g h t

BST sorting connection.

Postorder

TEXT

L e f t ! ' R i g h t ! ' x

Children before parent.

Level Order

TEXT

D e p t h 0 ! ' D e p t h 1 ! ' D e p t h 2 ! ' . . .

Breadth-first.

79. TREE SYLLABUS CHECKPOINT

Completed:

' Tree terminology ' Binary-tree representation ' Binary-tree types ' Major formulas ' Preorder ' Inorder ' Postorder '

Level order

One major Tree topic remains:

Reconstruction from Traversals

including:

- preorder + inorder
- postorder + inorder
- preorder + postorder
- uniqueness

- full-tree special case
- ambiguity traps

That is our next lesson.

PROGRESS

MODULE 6 — TREES

' Lesson 50 — Tree Fundamentals ' Lesson 51 — Binary Trees ' Lesson 52 — Binary Tree Types ' Lesson 53 — Formula Masterclass ' Lesson 54 — Preorder ' Lesson 55 — Inorder ' Lesson 56 — Postorder ' **Lesson 57 — Level Order**

NEXT — LESSON 58

Binary Tree Reconstruction from Traversals & Uniqueness — GATE Masterclass

This is one of the most important GATE tree lessons.

We will solve questions like:

TEXT

Preorder :
A B D E C F

Inorder :
D B E A C F

and reconstruct the exact tree step-by-step.

We will also master the critical rules:

TEXT

```
P r e o r d e r   +   I n o r d e r       ! '   u n i q u e   '  
P o s t o r d e r   +   I n o r d e r       ! '   u n i q u e   '  
P r e o r d e r   +   P o s t o r d e r ! '   n o t   a l w a y s   u n i q u e   'L
```

for distinct labels, plus the **full binary tree exception** and all common GATE traps.

USER

next

ASSISTANT

MODULE 6 — LESSON 58

Binary Tree Reconstruction from Traversals & Uniqueness — GATE

Masterclass

This is one of the most important Tree lessons for GATE.

You already know:

TEXT

```
Preorder  = Root Left Right
Inorder   = Left Root Right
Postorder = Left Right Root
```

Now we use those traversals in reverse:

Given traversal sequences, can we rebuild the tree?

GATE importance: 0=1% NOT IMPORTANT

1. THE CORE IDEA

To reconstruct a binary tree, we need two things:

1. Identify the root.
2. Determine which nodes belong to the left and right subtrees.

Different traversals provide different clues.

2. ROOT IDENTIFICATION

Preorder

TEXT

Root Left Right

Therefore:

First element = Root

Postorder

TEXT

Left Right Root

Therefore:

Last element = Root

Inorder

TEXT

Left Root Right

Inorder does **not** tell us the root by itself.

But once we know the root from another traversal, inorder tells us:

TEXT

Left subtree | Root | Right subtree

Ø=Yes is why inorder is so powerful for reconstruction.

3. MASTER UNIQUENESS RULE

Assuming all node labels/keys are distinct:

| Traversal Information | Unique Binary Tree? |
|--|---------------------|
| Preorder only | 'L No |
| Inorder only | 'L No |
| Postorder only | 'L No |
| Preorder + Inorder | ' Yes |
| Postorder + Inorder | ' Yes |
| Preorder + Postorder | 'L Not always |
| Preorder + Postorder + tree known to be full | ' Yes |

Ø=Yes Memorize this table.

4. WHY PREORDER + INORDER WORKS

Suppose:

TEXT

Preorder :
A B D E C F

TEXT

Inorder :
D B E A C F

Preorder first element:

TEXT

A

So:

Root = A

Now locate A in inorder:

TEXT

D B E | A | C F

Everything before A belongs to:

TEXT

left subtree

Everything after A belongs to:

TEXT

right subtree

So:

TEXT

Left subtree nodes = D B E

Right subtree nodes = C F

5. SPLIT PREORDER

We know left subtree contains:

TEXT

3 nodes

because inorder left part has:

TEXT

D B E

So after root A in preorder, next 3 elements belong to left subtree:

TEXT

B D E

Remaining:

TEXT

C F

belongs to right subtree.

Thus:

TEXT

Left:
Preorder = B D E
Inorder = D B E

TEXT

Right:
Preorder = C F
Inorder = C F

Then solve recursively.

6. RECONSTRUCT LEFT SUBTREE

Left subtree:

TEXT

Preorder = B D E
Inorder = D B E

First preorder element:

TEXT

B

So root of this subtree:

TEXT

B

Find B in inorder:

TEXT

D | B | E

Therefore:

TEXT

left child subtree = D
right child subtree = E

So:

TEXT

```
  B
 / \
D   E
```

7. RECONSTRUCT RIGHT SUBTREE

Right:

TEXT

Preorder = C F
Inorder = C F

Root:

TEXT

C

Find C in inorder:

TEXT

| C | F

No left subtree.

Right subtree contains:

TEXT

F

So:

TEXT

```
C
 \
  F
```

8. FINAL TREE

Combine:

TEXT

```
  A
 / \
B   C
/ \ \
D  E F
```

Check preorder:

TEXT

A B D E C F

Correct.

Check inorder:

TEXT

D B E A C F

Correct.

Always verify if time permits.

9. PREORDER + INORDER ALGORITHM

Given:

TEXT

Preorder

Inorder

Steps:

1. First preorder element = root.
2. Find root in inorder.
3. Inorder left side = left subtree.
4. Inorder right side = right subtree.
5. Count left-subtree nodes.
6. Split remaining preorder accordingly.
7. Repeat recursively.

10. MEMORY TRICK

PRE gives ROOT. IN gives SPLIT.

That's the entire reconstruction idea.

11. SOLVED EXAMPLE 2

Given:

TEXT

Preorder :

1 2 4 5 3 6 7

TEXT

Inorder :

4 2 5 1 6 3 7

First preorder:

TEXT

1

Root = 1.

Inorder:

TEXT

4 2 5 | 1 | 6 3 7

So:

TEXT

Left subtree = 4 2 5

Right subtree = 6 3 7

12. SPLIT PREORDER

Left subtree has 3 nodes.

After root 1:

TEXT

2 4 5

go left.

Remaining:

TEXT

3 6 7

go right.

13. LEFT SUBTREE

TEXT

Preorder = 2 4 5

Inorder = 4 2 5

Root = 2.

Split inorder:

TEXT

4 | 2 | 5

So:

TEXT



14. RIGHT SUBTREE

TEXT

Preorder = 3 6 7
Inorder = 6 3 7

Root = 3.

Split:

TEXT

6 | 3 | 7

So:

TEXT



15. FINAL TREE

TEXT



Perfect binary tree.

16. POSTORDER + INORDER

Now suppose:

TEXT

Postorder:
D E B F C A

TEXT

Inorder:
D B E A F C

Postorder last element:

TEXT
A

Therefore:

Root = A

Find A in inorder:

TEXT
D B E | A | F C

So:

TEXT
Left subtree nodes = D B E
Right subtree nodes = F C

17. SPLIT POSTORDER

Left subtree has:

TEXT
3 nodes

Postorder before root:

TEXT
D E B F C

First 3:

TEXT
D E B

belong to left.

Remaining:

TEXT
F C

belong to right.

So:

TEXT
Left:
Postorder = D E B

Inorder = D B E

TEXT

Right:

Postorder = F C

Inorder = F C

18. LEFT SUBTREE

Postorder last:

TEXT

B

So B is subtree root.

Inorder:

TEXT

D | B | E

Therefore:

TEXT

```
  B
 / \
D   E
```

19. RIGHT SUBTREE

Postorder:

TEXT

F C

Root:

TEXT

C

Inorder:

TEXT

F | C |

So C has:

TEXT

left child = F

No right child.

TEXT

```
  C
 /
F
```

20. FINAL TREE

TEXT

```
  A
 / \
B   C
/ \ /
D E F
```

Check:

Postorder:

TEXT

D E B F C A

Inorder:

TEXT

D B E A F C

Correct.

21. POSTORDER + INORDER ALGORITHM

Steps:

1. Last postorder element = root.
2. Find root in inorder.
3. Split inorder into left/right subtrees.
4. Count left/right subtree sizes.
5. Split postorder accordingly.
6. Recurse.

Memory trick:

POST gives ROOT from the end. IN gives SPLIT.

22. IMPORTANT DIFFERENCE IN RECURSIVE IMPLEMENTATION

For preorder + inorder:

TEXT

root consumed from beginning of preorder

For postorder + inorder:

TEXT

root consumed from end of postorder

If implementing postorder reconstruction recursively from the end, often build:

TEXT

RIGHT subtree first
then LEFT subtree

because when scanning postorder backward, order is:

TEXT

R o o t ! ' R i g h t ! ' L e f t

Placement implementation detail.

23. C-STYLE PREORDER + INORDER RECONSTRUCTION

Conceptually:

```
c
Node *build(int preorder[], int inorder[],
            int inStart, int inEnd,
            int *preIndex)
{
    if (inStart > inEnd)
        return NULL;

    int value = preorder[*preIndex];
    (*preIndex)++;

    Node *root = createNode(value);

    if (inStart == inEnd)
        return root;

    int pos = search(inorder, inStart, inEnd, value);

    root->left =
        build(preorder, inorder, inStart, pos - 1, preIndex);

    root->right =
        build(preorder, inorder, pos + 1, inEnd, preIndex);

    return root;
}
```

Core logic matters more than memorizing code.

24. COMPLEXITY OF NAIVE RECONSTRUCTION

If for every root we linearly search inorder:

```
TEXT
search = O(n)
```

and do this for many nodes.

Worst-case:

$O(n^2)$

Example skewed tree can trigger repeated long searches.

25. OPTIMIZED RECONSTRUCTION

If inorder value $\rightarrow$ index mapping is available:

```
TEXT
root location lookup = O(1)
```

Then each node is created once.

Total:

```
TEXT
Time = O(n)
```

Auxiliary mapping:

```
TEXT
O(n)
```

plus recursion stack:

```
TEXT
O(h)
```

~~O(n)~~ Placement extension.

26. WHY ONE TRAVERSAL IS NOT ENOUGH

Consider preorder:

TEXT

A B

Possible Tree 1:

TEXT

A
/
B

Possible Tree 2:

TEXT

A
\
B

Both preorder:

TEXT

A B

Therefore preorder alone:

Not unique

27. INORDER ALONE NOT UNIQUE

Sequence:

TEXT

A B

Possible:

TEXT

A
\
B

Inorder:

TEXT

A B

Also:

TEXT

B
/
A

Inorder:

TEXT
A B

Therefore not unique.

28. POSTORDER ALONE NOT UNIQUE

Sequence:

TEXT
B A

Could come from:

TEXT
A
/
B

or:

TEXT
A
\
B

Both:

TEXT
B A

So postorder alone not unique.

29. WHY PREORDER + POSTORDER IS NOT ALWAYS UNIQUE Ø=Ý%Ø=Ý%Ø=Ý%

Consider:

Tree 1:

TEXT
A
/
B

Tree 2:

TEXT
A
\

B

For both:

Preorder:

TEXT

A B

Postorder:

TEXT

B A

So even with both traversals:

TEXT

Preorder + Postorder

we cannot determine whether B is left or right child.

Therefore:

Not unique for arbitrary binary trees

Ø-Extremely common GATE question.

30. THE SOURCE OF AMBIGUITY

The ambiguity happens when a node has:

TEXT

exactly one child

Preorder tells us:

TEXT

parent before child

Postorder tells us:

TEXT

child before parent

But neither tells whether that child is:

TEXT

LEFT

or:

TEXT
RIGHT

Inorder would tell us.

31. FULL BINARY TREE EXCEPTION $\emptyset = Y\% \emptyset = Y\% \emptyset = Y\%$

Recall full binary tree:

TEXT
every internal node has exactly 2 children

There are no one-child nodes.

Therefore the left/right ambiguity responsible for preorder+postorder non-uniqueness disappears.

For distinct labels:

Preorder + Postorder uniquely determine a full binary tree.

$\emptyset = \text{Must know.}$

32. EXAMPLE — FULL TREE

Preorder:

TEXT
A B D E C F G

Postorder:

TEXT
D E B F G C A

Root from preorder:

TEXT
A

Root from postorder:

TEXT
A

Next preorder element:

TEXT
B

must be root of A's left subtree.

Find B in postorder:

TEXT

D E B | F G C | A

So:

TEXT

left subtree = D E B

contains 3 nodes.

Then corresponding preorder left block:

TEXT

B D E

The rest:

TEXT

C F G

belongs to right.

Continue recursively.

33. RECONSTRUCTED FULL TREE

TEXT



Unique because each internal node must have two children.

34. IMPORTANT ASSUMPTION — DISTINCT LABELS

Our uniqueness statements assume nodes can be identified uniquely.

Suppose duplicate labels exist:

TEXT

A A B

Finding "A" in inorder may be ambiguous.

Therefore standard reconstruction questions usually assume:

TEXT

distinct node values

or otherwise provide extra information.

Read question carefully.

35. PREORDER + INORDER WITH DUPLICATES

If values repeat:

TEXT

Preorder = A A

Inorder = A A

Which occurrence of A is root in inorder?

Could be first or second depending on actual structure.

Therefore uniqueness can fail without additional rules.

36. POSTORDER + INORDER WITH DUPLICATES

Same problem.

Root value is known, but the corresponding occurrence in inorder may not be uniquely identifiable.

So:

Distinct-label assumption matters.

37. CAN INORDER + LEVEL ORDER RECONSTRUCT?

With distinct labels:

TEXT

Inorder + Level order

can also determine a unique binary tree.

Why?

Level order's first element is root.

Then inorder splits left/right node sets.

Filter remaining level-order elements into those two sets.

Then recurse.

Less commonly tested than preorder/postorder combinations, but valid and useful.

38. LEVEL ORDER ALONE?

Not unique.

Example:

```
TEXT
A B
```

could represent:

```
TEXT
  A
 /
B
```

or:

```
TEXT
A
 \
  B
```

Both level order:

```
TEXT
A B
```

So:

Level order alone not unique

39. PREORDER + LEVEL ORDER?

For an arbitrary binary tree, not generally enough to guarantee unique reconstruction without extra structure.

The major standard GATE rules to memorize remain:

```
TEXT
P r e   +   I n       '
P o s t   +   I n       '
P r e   +   P o s t   'L   g e n e r a l l y
```

40. GATE MASTER TABLE

Assuming distinct node labels:

| Information | Unique? |
|-------------|---------|
| Preorder | 'L |

| Information | Unique? |
|----------------------------------|--------------|
| Inorder | 'L |
| Postorder | 'L |
| Level order | 'L |
| Preorder + Inorder | ' |
| Postorder + Inorder | ' |
| Preorder + Postorder | 'L generally |
| Preorder + Postorder for full BT | ' |
| Inorder + Level order | ' |

Ø=Yes table is worth memorizing.

41. HOW TO FIND MISSING TRAVERSAL

Very common GATE problem:

Given:

TEXT

Preorder
Inorder

Find:

TEXT

Postorder

Strategy:

1. Reconstruct tree.
2. Perform postorder.

You don't necessarily have to draw the entire tree beautifully. Recursive partitioning is enough.

42. SOLVED EXAMPLE — FIND POSTORDER

Given:

TEXT

Preorder :
A B D E C F

TEXT

Inorder :
D B E A C F

We reconstructed:

TEXT



Now postorder:

TEXT

D E B F C A

Answer:

D E B F C A

43. FIND PREORDER FROM INORDER + POSTORDER

Given:

TEXT

Inorder:
D B E A F C

TEXT

Postorder:
D E B F C A

Root:

TEXT

A

Left subtree:

TEXT

D B E

Right:

TEXT

F C

Reconstruct:

TEXT



/ \ /
D E F

Preorder:

A B D E C F

44. LARGE GATE EXAMPLE

Given:

TEXT

Preorder :

10 5 2 7 15 12 20

TEXT

Inorder :

2 5 7 10 12 15 20

Root:

TEXT

10

Split inorder:

TEXT

2 5 7 | 10 | 12 15 20

Left subtree size = 3.

Preorder split:

TEXT

Left = 5 2 7

Right = 15 12 20

45. LEFT SUBTREE

TEXT

Pre = 5 2 7

In = 2 5 7

Root:

TEXT

5

So:

TEXT



46. RIGHT SUBTREE

TEXT

Pre = 15 12 20
In = 12 15 20

Root:

TEXT

15

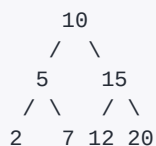
So:

TEXT



47. FINAL

TEXT



Postorder:

TEXT

2 7 5 12 20 15 10

48. NOTICE — THIS ALSO LOOKS LIKE BST

Inorder:

TEXT

2 5 7 10 12 15 20

is sorted.

Therefore this reconstructed tree is consistent with BST ordering.

But reconstruction itself relied on:

TEXT

```
preorder + inorder
```

not merely on BST properties.

49. BST SPECIAL RECONSTRUCTION

For a BST with distinct keys:

A preorder traversal alone can uniquely reconstruct the BST.

Why?

Because BST property tells us:

TEXT

```
v a l u e s < r o o t ! ' l e f t s u b t r e e
v a l u e s > r o o t ! ' r i g h t s u b t r e e
```

Example preorder:

TEXT

```
40 20 10 30 60 50 70
```

Root:

TEXT

```
40
```

Values after root less than 40:

TEXT

```
20 10 30
```

form left subtree.

Values greater:

TEXT

```
60 50 70
```

form right subtree.

Then recurse.

Ø = Important BST extension.

50. BST POSTORDER ALONE

Likewise, for a BST with distinct keys, postorder alone can also reconstruct the BST uniquely.

Last value is root.

Then partition remaining sequence according to BST ordering.

We'll revisit in BST module.

51. IMPORTANT — THIS DOES NOT APPLY TO ORDINARY BINARY TREE

For arbitrary binary tree:

TEXT

```
p r e o r d e r   a l o n e ' L
```

For BST:

TEXT

```
p r e o r d e r   a l o n e ' 
```

because BST gives additional ordering information.

~~O=Major~~ or conceptual distinction.

52. GATE TRAP — SORTED INORDER

For any BST:

TEXT

```
inorder = sorted
```

So if you know only a set of keys and inorder of a BST, you essentially know the sorted order but not necessarily its exact shape.

Example keys:

TEXT

```
10 20 30
```

BST 1:

TEXT

```
    20
   /  \
  10   30
```

BST 2:

TEXT

```
10
 \
  20
 \
  30
```

Both inorder:

```
TEXT
10 20 30
```

So BST inorder alone does not determine shape.

53. GATE TRAP — PREORDER ROOT

Given:

```
TEXT
Preorder :
8 4 2 6 12 10 14
```

Root:

```
TEXT
8
```

Always.

Do not try to find the largest/smallest value unless BST-specific reasoning is required.

54. GATE TRAP — POSTORDER ROOT

Given:

```
TEXT
Postorder :
2 6 4 10 14 12 8
```

Root:

```
TEXT
8
```

because:

```
TEXT
last element
```

No further reasoning needed.

55. GATE TRAP — INORDER ROOT

Given only:

TEXT

Inorder:
D B E A C

Can we say A is root?

No.

Without another traversal or structural information, root is not known.

$\emptyset = \text{Very}$ important.

56. GATE TRAP — SAME SET "VALID PAIR"

Suppose preorder and inorder have different node sets.

Example:

TEXT

Preorder:
A B C

TEXT

Inorder:
A B D

They cannot belong to the same tree because:

TEXT

C " " D

Node multisets must match.

57. VALIDITY CHECK

Even if both traversals contain the same nodes, not every arbitrary pair necessarily represents a valid tree under all constraints.

During recursive reconstruction, partitions must remain consistent.

If at some point root from preorder/postorder cannot be found in the corresponding inorder segment:

TEXT

invalid pair

58. COUNT OF LEFT SUBTREE NODES

Suppose inorder:

TEXT

P Q R X S T

Root:

TEXT

X

Then:

TEXT

Left count = 3

because:

TEXT

P Q R

occur before X.

This count tells exactly how many nodes to take for the left subtree from preorder/postorder.

Ø = Critical exam technique.

59. PREORDER PARTITION FORMULA

Let:

TEXT

preorder = [root][left subtree][right subtree]

If left subtree has L nodes:

TEXT

root = pre[0]
left preorder = next L elements
right preorder = remaining elements

60. POSTORDER PARTITION FORMULA

TEXT

postorder = [left subtree][right subtree][root]

If left subtree has L nodes and right has R:

TEXT

left postorder = first L
right postorder = next R
root = final element

61. RECURSION COMPLEXITY — NAIVE

Without a fast index map:

At each subtree, search root in inorder.

Worst-case recurrence:

TEXT

$$T(n) = T(n-1) + O(n)$$

Therefore:

$$O(n^2)$$

62. RECURSION COMPLEXITY — OPTIMIZED

With root-position lookup:

TEXT

$$O(1)$$

per node.

Then:

TEXT

$$T(n) = T(\text{left}) + T(\text{right}) + O(1)$$

Total:

$$O(n)$$

Space:

TEXT

$$O(n)$$

for index mapping plus $O(h)$ recursion.

63. GATE-STYLE QUESTION 1

Given distinct labels, which traversal pair always uniquely determines an arbitrary binary tree?

A. Preorder + Postorder B. Preorder + Inorder C. Preorder only D. Level order only

Answer:

B — Preorder + Inorder

64. GATE-STYLE QUESTION 2

Which traversal pair may fail to determine whether a single child is left or right?

A. Preorder + Inorder B. Postorder + Inorder C. Preorder + Postorder D. Inorder + Level order

Answer:

C — Preorder + Postorder

65. GATE-STYLE QUESTION 3

If a binary tree is known to be full and all labels are distinct, which pair can reconstruct it uniquely?

TEXT

Preorder + Postorder

Answer:

' Yes.

66. GATE-STYLE QUESTION 4

Preorder:

TEXT

A B C

Postorder:

TEXT

C B A

How many shapes may be possible if each internal node could have one child?

At minimum, ambiguity exists because B and C can appear as left/right single-child chains in multiple orientations.

So:

Not uniquely determined

Do not force a single tree.

67. GATE-STYLE QUESTION 5

Given:

TEXT

Preorder :
A B D E C

TEXT

Inorder :
D B E A C

Root:

TEXT

A

Left subtree nodes:

TEXT

D B E

Right subtree:

TEXT

C

Left-subtree root:

TEXT

B

Tree:

TEXT



Postorder:

TEXT

D E B C A

68. GATE-STYLE QUESTION 6

Given:

TEXT

Inorder :
4 2 5 1 3

TEXT

Postorder:
4 5 2 3 1

Root:

TEXT

1

Inorder split:

TEXT

4 2 5 | 1 | 3

Left postorder:

TEXT

4 5 2

Right:

TEXT

3

Left root:

TEXT

2

So tree:

TEXT



Preorder:

1 2 4 5 3

69. RECONSTRUCTION WITHOUT DRAWING FULL TREE

During GATE, if only missing traversal is required, you can recursively generate it directly.

For preorder + inorder !' postorder:

1. root = preorder first.
2. split inorder.
3. recursively produce left postorder.

4. recursively produce right postorder.

5. append root.

So:

TEXT

Postorder = LeftPost + RightPost + Root

This can save drawing time.

70. EXAMPLE DIRECT METHOD

Pre:

TEXT

A B D E C

In:

TEXT

D B E A C

Root:

TEXT

A

Left part:

TEXT

Pre = B D E

In = D B E

Its postorder:

TEXT

D E B

Right:

TEXT

C

Postorder:

TEXT

C

Then root:

TEXT

A

Final:

TEXT

D E B C A

71. NUMBER OF POSSIBLE TREES FROM PRE + POST — ADVANCED $\emptyset = \text{pre}$

When a node has exactly one child, preorder and postorder do not tell whether that child is left or right.

If there are k such ambiguous one-child parent situations, there can be multiple possible trees, often a factor related to:

TEXT

2^k

depending on the exact sequence and independence of ambiguities.

For GATE core, the key conclusion is simply:

Not unique unless extra restrictions remove ambiguity.

72. FULL TREE REMOVES SINGLE-CHILD AMBIGUITY

Full tree requires:

TEXT

0 or 2 children

So no choice like:

TEXT

left only vs right only

exists.

That's why pre+post becomes enough.

73. COMPLETE TREE + PREORDER/POSTORDER?

If additional tree-shape information is given, such as the tree being complete, reconstruction can sometimes become unique with traversal information that would otherwise be insufficient.

But do not memorize unsupported blanket rules.

$\emptyset = \text{GATE}$ -safe principle:

Use exactly the structural constraints provided.

74. PERFECT TREE

If tree is perfect, shape is already fixed by node count/height.

Then traversal labels can assign values to that fixed shape.

This is much more constrained than arbitrary binary trees.

75. RECONSTRUCTION AND TREE TYPES

Extra properties can give extra information:

Binary Tree

Only:

TEXT

```
"d 2   c h i l d r e n"
```

Full

TEXT

```
0 or 2
```

removes one-child ambiguity.

Complete

Shape follows level-order filling.

Perfect

Every level filled.

BST

Numeric/key ordering identifies left/right partitions.

$\emptyset = \text{null}$ structural constraints matter.

76. NOTEBOOK NOTES — PRE + IN

TEXT

```
Preorder first = root
```

```
Find root in inorder
```

```
Inorder:
```

```
LEFT | ROOT | RIGHT
```

```
Use left-size to split preorder
```

```
Recurse
```

Memory:

PRE ROOT + IN SPLIT

77. NOTEBOOK NOTES — POST + IN

TEXT

Postorder last = root

Find root in inorder

Inorder:

LEFT | ROOT | RIGHT

Use subtree sizes to split postorder

Recurse

Memory:

POST ROOT + IN SPLIT

78. UNIQUENESS NOTES

For distinct labels:

TEXT

```
P r e   +   I n       !'   U n i q u e
P o s t   +   I n     !'   U n i q u e
P r e   +   P o s t !'   N o t   u n i q u e   g e n e r a l l y
```

Special:

TEXT

```
F u l l   b i n a r y   t r e e   +   P r e   +   P o s t !'   U n i q u e
```

0-100% exactly.

79. DON'T CONFUSE

Preorder

Root:

TEXT

FIRST

Postorder

Root:

TEXT

LAST

Inorder

Root:

TEXT

unknown until another clue identifies it

Inorder's main role

TEXT

separates left and right subtree

80. MEMORY TRICK

INORDER IS THE KNIFE

It cuts the traversal around the root:

TEXT

LEFT | ROOT | RIGHT

Preorder/postorder tells you **which node is root**.

Inorder tells you **where to cut**.

81. 30-SECOND REVISION

Remember:

1. Preorder first = root.
2. Postorder last = root.
3. Inorder splits left/right subtrees.
4. Pre + In gives unique tree for distinct labels.
5. Post + In gives unique tree for distinct labels.
6. Pre + Post is not unique for arbitrary binary tree.
7. Ambiguity comes from one-child nodes.
8. Full tree removes that ambiguity.
9. Full tree + Pre + Post is unique for distinct labels.
10. BST ordering can allow reconstruction from preorder alone.

82. ACTIVE RECALL

Q1

Which traversal identifies root from its first element?

Q2

Which identifies root from its last element?

Q3

Why is inorder important?

Q4

Does preorder + postorder always uniquely determine arbitrary binary tree?

Q5

When does preorder + postorder become uniquely sufficient in the standard special case?

ANSWERS

A1

Preorder.

A2

Postorder.

A3

It separates the left and right subtree around the known root.

A4

No.

A5

When the binary tree is known to be full and node labels are distinct.

83. PRACTICE 1

Given:

TEXT

Preorder :

A B D E C F G

TEXT

Inorder :

D B E A F C G

Find:

1. root
2. left subtree
3. right subtree

4. complete tree
5. postorder

84. PRACTICE 2

Given:

TEXT

Inorder:
H D I B E A F C G

TEXT

Postorder:
H I D E B F G C A

Find:

1. root
2. left subtree root
3. right subtree root
4. preorder

85. PRACTICE 3 — UNIQUENESS

For each, say whether enough to uniquely determine an arbitrary binary tree with distinct labels:

1. preorder
2. inorder
3. postorder
4. preorder + inorder
5. postorder + inorder
6. preorder + postorder
7. preorder + postorder, tree known full

Answers should be:

TEXT

' L
' L
' L
'
'
' L
'

86. TREE MODULE COMPLETE '

We have now covered the official **Trees** portion comprehensively:

' Tree terminology ' Nodes, edges, root, parent/child ' Degree, level, depth, height ' Ancestors/descendants/subtrees

' Binary-tree definition ' Linked representation in C ' Array representation ' 0-based & 1-based index formulas ' Full binary tree ' Complete binary tree ' Perfect binary tree ' Balanced/skewed/degenerate trees ' Edge/node/height formulas ' $n_0 = n_2 + 1$ ' $n+1$ NULL child-link theorem ' Preorder ' Inorder ' Postorder ' Level order/BFS ' Recursive traversal stack behavior ' Iterative traversal intuition ' Expression-tree traversal connections ' Reconstruction from traversals ' Uniqueness conditions ' Full-tree preorder/postorder exception ' BST reconstruction preview ' Complexity and GATE traps

MODULE 6 — TREES ' COMPLETE

WHERE WE ARE IN THE OFFICIAL SYLLABUS

Official syllabus:

Programming in C. Recursion. Arrays, stacks, queues, linked lists, trees, binary search trees, binary heaps, graphs.

Status:

| Topic | Status |
|---------------------|---------|
| Programming in C | ' |
| Recursion | ' |
| Arrays | ' |
| Stacks | ' |
| Queues | ' |
| Linked Lists | ' |
| Trees | ' |
| Binary Search Trees | #ó NEXT |
| Binary Heaps | #ó |
| Graphs | #ó |

We are now moving into:

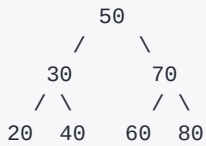
MODULE 7 — BINARY SEARCH TREES Ø<ß3Ø=Ý

NEXT — LESSON 59

Binary Search Tree Fundamentals — BST Property, Search Intuition, Inorder Sorting & Complexity

We will start with:

TEXT



and master:

- exact BST definition
- left/right subtree property
- local vs global BST rule
- duplicate-key conventions
- recursive structure
- why inorder is sorted
- search intuition
- balanced vs skewed BST
- BST vs ordinary binary tree
- BST vs binary heap
- GATE validity questions
- minimum/maximum intuition
- time complexity $O(h)$
- common traps

Then:

Lesson 60 — BST Search & Insertion.

USER

next

ASSISTANT

MODULE 7 — LESSON 59

Binary Search Tree Fundamentals — BST Property, Search Intuition, Inorder Sorting & Complexity

We now begin:

BINARY SEARCH TREES — BST

This is one of the most important tree topics for both:

- GATE
- placements
- coding interviews

It adds an **ordering rule** that makes searching much more efficient when the tree has a good shape.

1. START WITH A NORMAL BINARY TREE

TEXT

Example:

TEXT



Binary Tree

Why?

2. BST DEFINITION Ø=Ý%Ø=Ý%Ø=Ý%

For every node x :

TEXT

All keys in LEFT subtree $< X$

All keys in RIGHT subtree $> X$

under the standard distinct-key convention.

Example:

TEXT



TEXT

Left subtree:
20,30,40 < 50

Right subtree:
60,70,80 > 50

Correct.

3. VERY IMPORTANT — GLOBAL PROPERTY

Many students make this mistake.

They check only:

TEXT

left child < parent
right child > parent

That is not enough.

The property must hold for the:

ENTIRE SUBTREE

4. INVALID BST EXAMPLE Ø=Ý%

Consider:

TEXT



Local checks:

TEXT

```
3 0 < 5 0 '  
7 0 > 5 0 '  
6 0 > 3 0 '
```

Looks okay locally.

But node 60 lies inside:

TEXT

LEFT subtree of 50

Therefore it must satisfy:

TEXT

60 < 50

which is false.

So:

NOT A BST 'L

0=Major GATE trap.

5. BST PROPERTY IS RECURSIVE

A tree is a BST if:

1. Left subtree is itself a BST.
2. Right subtree is itself a BST.
3. Every left-subtree key satisfies root ordering.
4. Every right-subtree key satisfies root ordering.

So BST has a recursive structure.

6. RANGE VIEW — BEST WAY TO UNDERSTAND VALIDITY

For root 50:

TEXT

Allowed range:
(-" , +")

Left child 30 receives:

TEXT

(-" , 5 0)

Right child 70 receives:

TEXT

(5 0 , +")

Now node 30's right subtree must satisfy both:

TEXT

> 30
AND
< 50

So allowed range:

TEXT

(30,50)

A value 60 there is invalid.

0-1% range method is the safest BST-validation logic.

7. VALID EXAMPLE

TEXT



Check:

40 belongs to left subtree of 50:

TEXT

40 < 50

and right subtree of 30:

TEXT

40 > 30

So:

TEXT

30 < 40 < 50

Correct.

Likewise:

TEXT

50 < 60 < 70

Correct.

8. BINARY TREE VS BST

| Property | Binary Tree | BST |
|-------------------|-------------|--------------|
| Max children | 2 | 2 |
| Ordering required | 'L' | '<' |
| Left subtree | arbitrary | smaller keys |

| Property | Binary Tree | BST |
|------------------|-----------------|-------------|
| Right subtree | arbitrary | larger keys |
| Inorder sorted | not necessarily | yes |
| Search potential | $O(n)$ | $O(h)$ |

Ø= Every BST is a binary tree.

But:

Not every binary tree is a BST.

9. BST NODE STRUCTURE IN C

The node structure is the same as ordinary binary tree:

```
c
typedef struct Node
{
    int data;
    struct Node *left;
    struct Node *right;
} Node;
```

What makes it a BST is not the struct.

It is:

how nodes are arranged

Ø= Important conceptual point.

10. WHY BST IS CALLED "SEARCH" TREE

Suppose we need to search:

TEXT
60

in:

TEXT

```

      50
     /  \
    30   70
   /  \  /  \
  20  40 60  80
```

Start 50.

Since:

TEXT

60 > 50

we know:

60 cannot be anywhere in the left subtree.

So immediately discard half-ish of the search area.

Move right.

At 70:

TEXT

60 < 70

move left.

Reach:

TEXT

60

Found.

11. SEARCH DECISION RULE Ø=Ý%

At node `root`:

TEXT

```
key == root->data
  FOUND
```

```
key < root->data
  search LEFT
```

```
key > root->data
  search RIGHT
```

Only one path is followed.

That is the key advantage.

12. NORMAL BINARY TREE SEARCH

Without ordering:

TEXT

```
    50
   /  \
  30   70
 ...
```

If searching 60, you may need to inspect:

TEXT

left subtree
and/or
right subtree

Worst case:

TEXT

$O(n)$

13. BST SEARCH

BST lets us follow only:

TEXT

one root-to-node path

So complexity depends on:

TREE HEIGHT

If height = h :

FORMULA BOX

TEXT

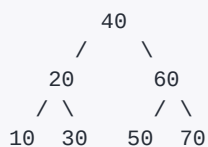
Search = $O(h)$

~~$O(n)$~~ This idea is critical.

14. BALANCED BST

Example:

TEXT



Height:

TEXT

$\sim (\log n)$

So search:

TEXT

$O(\log n)$

15. SKEWED BST

Insert sorted values:

TEXT

10, 20, 30, 40, 50

using normal BST insertion.

Result:

TEXT



Height:

TEXT

$n-1$

Search:

TEXT

$O(n)$

~~$O(\log n)$~~ does **not** automatically guarantee $O(\log n)$.

16. VERY IMPORTANT GATE TRAP

Statement:

Searching in a BST takes $O(\log n)$.

Not always.

Correct:

TEXT

Search = $O(h)$

Balanced:

TEXT

$O(\log n)$

Worst-case skewed:

TEXT

$O(n)$

$O(1)$ vs $O(n)$ distinction is fundamental.

17. BEST CASE SEARCH

If searched value is root:

TEXT

one comparison

So:

TEXT

Best case = $O(1)$

18. WORST CASE SEARCH

Search may go to deepest node or fail after reaching NULL.

So:

TEXT

Worst case = $O(h)$

and since:

TEXT

$h \geq \log_2 n - 1$

worst possible BST:

TEXT

$O(n)$

19. INORDER OF BST

Recall inorder:

TEXT

L E F T ! ' R O O T ! ' R I G H T

BST ordering:

TEXT

LEFT < ROOT < RIGHT

Therefore:

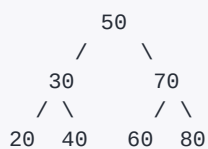
Inorder traversal of BST gives sorted ascending keys

under distinct-key rule.

20. EXAMPLE

BST:

TEXT



Inorder:

TEXT

20 30 40 50 60 70 80

Sorted.

One of the most important BST properties.

21. WHY INORDER SORTS

Consider any node X.

Inorder first prints:

TEXT

all left-subtree keys

which are all:

TEXT

< X

Then prints:

TEXT

X

Then:

TEXT

all right-subtree keys

which are all:

TEXT

> X

Recursively each subtree itself is sorted.

So complete traversal becomes sorted.

22. REVERSE INORDER

Reverse inorder:

TEXT

R I G H T ! ' R O O T ! ' L E F T

For BST gives:

Descending order

Example:

TEXT

80 70 60 50 40 30 20

23. MINIMUM ELEMENT Ø=Ý%

Where is minimum key in BST?

Since smaller values are always on the:

TEXT

LEFT

Keep moving left until:

TEXT

left == NULL

That node contains minimum.

Example:

TEXT

```
      50
     /
    30
   /
  20
```

Minimum:

TEXT

20

24. MAXIMUM ELEMENT Ø=Ý%

Keep moving:

TEXT

RIGHT

until:

TEXT

right == NULL

That node contains maximum.

25. MIN/MAX COMPLEXITY

Both follow one root-to-node path.

Therefore:

TEXT

Minimum = $O(h)$

Maximum = $O(h)$

Balanced:

TEXT

$O(\log n)$

Worst skewed:

TEXT

$O(n)$

26. C CODE — FIND MINIMUM

```

c
Node *findMin(Node *root)
{
    if (root == NULL)
        return NULL;

    while (root->left != NULL)
        root = root->left;

    return root;
}

```

27. C CODE — FIND MAXIMUM

```

c
Node *findMax(Node *root)
{
    if (root == NULL)
        return NULL;

    while (root->right != NULL)
        root = root->right;

    return root;
}

```

28. BST SEARCH C CODE — ITERATIVE

```

c
Node *search(Node *root, int key)
{
    while (root != NULL)
    {
        if (key == root->data)
            return root;

        if (key < root->data)
            root = root->left;
        else
            root = root->right;
    }

    return NULL;
}

```

Time:

TEXT
O(h)

Space:

TEXT

$O(1)$

for iterative search.

29. RECURSIVE SEARCH

C

```
Node *search(Node *root, int key)
{
    if (root == NULL || root->data == key)
        return root;

    if (key < root->data)
        return search(root->left, key);

    return search(root->right, key);
}
```

Time:

TEXT

$O(h)$

Auxiliary recursion stack:

TEXT

$O(h)$

30. ITERATIVE VS RECURSIVE SEARCH

| Version | Time | Extra Space |
|-----------|--------|-------------|
| Iterative | $O(h)$ | $O(1)$ |
| Recursive | $O(h)$ | $O(h)$ |

~~$O(h)$~~ Good placement comparison.

31. SEARCH TRACE

BST:

TEXT



Search:

```
TEXT
40
```

Start:

```
TEXT
4 0 < 5 0 !' L E F T
```

At 30:

```
TEXT
4 0 > 3 0 !' R I G H T
```

At 40:

```
TEXT
FOUND
```

Path:

```
TEXT
5 0 !' 3 0 !' 4 0
```

32. UNSUCCESSFUL SEARCH

Search:

```
TEXT
35
```

Path:

```
TEXT
50
!" l e f t
30
!" r i g h t
40
!" l e f t
NULL
```

So key absent.

Notice:

```
TEXT
5 0 !' 3 0 !' 4 0 !' N U L L
```

Only one path is examined.

33. DUPLICATES IN BST — IMPORTANT WARNING

The standard mathematical BST often assumes:

```
TEXT
all keys distinct
```

But implementations may allow duplicates.

Possible conventions:

```
TEXT
duplicates always left
```

or:

```
TEXT
duplicates always right
```

or:

```
TEXT
store count/frequency in node
```

Example node:

```
C
struct Node
{
    int key;
    int count;
    struct Node *left, *right;
};
```

~~Never~~ Never assume duplicate policy unless specified.

34. IF DUPLICATES ARE DISALLOWED

Standard property:

```
TEXT
left < root < right
```

Inorder:

```
TEXT
strictly increasing
```

35. IF DUPLICATES ARE ALLOWED

Depending on policy, inorder is usually:

TEXT

non-decreasing

rather than strictly increasing.

Example:

TEXT

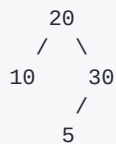
10 10 20 30

could be valid under an allowed duplicate convention.

36. BST VALIDITY — LOCAL CHILD CHECK IS WRONG $\emptyset = Y\% \emptyset = Y\% \emptyset = Y\%$

Consider:

TEXT



Immediate checks:

TEXT

```
1 0 < 2 0 '  
3 0 > 2 0 '  
5 < 3 0 '
```

Yet:

TEXT

5

is in right subtree of 20.

So it must satisfy:

TEXT

5 > 20

False.

Therefore:

Invalid BST

37. CORRECT BST VALIDATION— RANGE METHOD

Start:

```
TEXT
root allowed:
( -" , +" )
```

For left subtree:

```
TEXT
(min, root)
```

For right:

```
TEXT
(root, max)
```

Recursively update allowed range.

38. C-STYLE VALIDATION

Conceptually:

```
C
int isBST(Node *root, long long low, long long high)
{
    if (root == NULL)
        return 1;

    if (root->data <= low || root->data >= high)
        return 0;

    return isBST(root->left, low, root->data) &&
        isBST(root->right, root->data, high);
}
```

For distinct-key strict BST.

$\emptyset \neq \{x\}$ checks the **global** property.

39. ALTERNATIVE VALIDATION— INORDER

Since valid distinct-key BST inorder must be strictly increasing:

Keep previous visited value.

If at any point:

```
TEXT
current <= previous
```

then invalid.

This is also a common validation technique.

40. RANGE VS INORDER VALIDATION

Range method

Checks each node against allowed interval.

Time:

TEXT

$O(n)$

Recursive space:

TEXT

$O(h)$

Inorder method

Checks traversal is strictly increasing.

Time:

TEXT

$O(n)$

Space:

TEXT

$O(h)$

if recursive.

Both are valid.

41. DON'T USE ONLY MIN/MAX CHILD CHECK

Wrong idea:

TEXT

```
root->left->data < root->data  
root->right->data > root->data
```

This checks only direct children.

It misses deeper violations.

~~0-6%~~ OATE trap.

42. BST HEIGHT

BST height depends entirely on insertion order/shape.

Same set of keys:

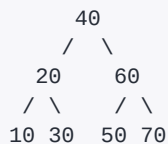
TEXT

10, 20, 30, 40, 50, 60, 70

can produce different BSTs.

Balanced possible:

TEXT



Height:

TEXT

2

43. SAME KEYS, BAD ORDER

Insert:

TEXT

10, 20, 30, 40, 50, 60, 70

Result:

TEXT



Height:

TEXT

6

Same values.

Huge performance difference.

$O(h)$ Insertion order matters.

44. MINIMUM POSSIBLE HEIGHT OF BST

BST is still a binary tree.

For n nodes, minimum height:

TEXT
 $\text{floor}(\log_2 n)$

under edge-height convention.

Maximum:

TEXT
 $n - 1$

So:

TEXT
 $\text{floor}(\log_2 n) \leq h \leq n - 1$

same binary-tree height bounds.

45. BST SEARCH COMPLEXITY FORMULA

FORMULA BOX

For search, insertion, and deletion:

TEXT
 $O(h)$

where:

TEXT
 $h = \text{height}$

Balanced:

TEXT
 $O(\log n)$

Skewed:

TEXT
 $O(n)$

$O(h)$ will use this repeatedly.

46. BST VS BINARY HEAP Ø=Ý%

Students often confuse them.

BST

Ordering:

TEXT

Left subtree < Root < Right subtree

Main purpose:

TEXT

search / ordered operations

Min-Heap

Ordering:

TEXT

P a r e n t "d c h i l d r e n

but no complete ordering between entire left and right subtrees.

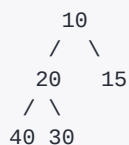
Main purpose:

TEXT

fast minimum / priority queue

47. EXAMPLE MIN-HEAP

TEXT



This can be a min-heap because parents are "dchildren.

But it is not necessarily a BST because:

TEXT

right child 15 > 10

is okay for BST root, but deeper/global ordering doesn't follow BST's complete left/right separation generally.

Also BST does not require completeness.

Heap does.

48. BST VS HEAP TABLE

| Property | BST | Binary Heap |
|----------------------|-----------------------|-----------------------------|
| Shape | Any binary-tree shape | Complete binary tree |
| Ordering | Left < root < right | Parent relation only |
| Search arbitrary key | O(h) | O(n) generally |
| Min/max | O(h) ordinary BST | root gives one extreme O(1) |
| Inorder sorted | ' | 'L |
| Priority queue | not standard choice | ' |

Keep this distinction clear.

49. BST DOES NOT HAVE TO BE COMPLETE

Example:

TEXT

```
50
 \
  80
 /
60
```

Check:

TEXT

```
80 > 50
60 > 50
60 < 80
```

Valid BST.

But not complete.

So:

BST shape and complete-tree shape are separate ideas.

50. BST DOES NOT HAVE TO BE BALANCED

Same:

TEXT

```
10
 \
  20
  \
```


valid BST.

But badly unbalanced.

Balanced BST variants such as:

TEXT

AVL tree
Red-Black tree

actively control height.

These are useful extensions, though not in your stated Programming/Data Structures syllabus unless separately needed.

51. BST DOES NOT HAVE TO BE FULL

Example:

TEXT

```

      20
     /
    10
  
```

valid BST.

Node 20 has one child.

So not full.

52. BST DOES NOT HAVE TO BE PERFECT

Obviously:

TEXT

```

      20
     /  \
    10   30
   /
  5
  
```

can be valid BST but not perfect.

53. ROOT IS NOT NECESSARILY MEDIAN

Consider BST:

TEXT

```

  10
   \
  
```

20
 \
 30

Keys:

TEXT
10, 20, 30

Median:

TEXT
20

Root:

TEXT
10

So root need not be median.

Only a balanced construction might deliberately choose medians.

Useful misconception correction.

54. INORDER IS SORTED, BUT PREORDER IS NOT

Example:

TEXT
 40
 /< \
20 60

Preorder:

TEXT
40 20 60

not sorted.

Inorder:

TEXT
20 40 60

sorted.

Postorder:

TEXT
20 60 40

not sorted.

55. PREORDER OF BST HAS STRUCTURE

For root R :

```
TEXT
R
[left-subtree preorder]
[right-subtree preorder]
```

For distinct keys:

all keys in left block:

```
TEXT
< R
```

all keys in right block:

```
TEXT
> R
```

This helps reconstruct BST from preorder.

56. POSTORDER OF BST HAS STRUCTURE

```
TEXT
[left-subtree postorder]
[right-subtree postorder]
R
```

Again:

left block:

```
TEXT
< R
```

right block:

```
TEXT
> R
```

This helps validate/reconstruct BST sequences.

57. BST FROM SORTED ARRAY — PREVIEW

Given sorted:

TEXT

10 20 30 40 50 60 70

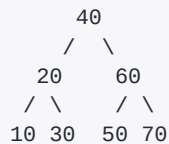
If we repeatedly choose middle:

TEXT

40

as root, then:

TEXT



we get a height-efficient BST.

~~$O(n)$~~ We will cover this properly in placement extensions.

58. SEARCH COMPARISON COUNT

Suppose balanced BST with:

TEXT

$n = 7$

height:

TEXT

2

Searching deepest node visits at most roughly:

TEXT

$h+1 = 3$ nodes

because height counts edges.

~~$O(n)$~~ Remember:

TEXT

height h
!' root - t o - d e e p e s t p a t h c o n t a i n s $h + 1$ n o d e s

59. FAILED SEARCH PATH

An unsuccessful search may end at:

TEXT

NULL

just below a leaf.

So number of comparisons to actual nodes is still bounded by root-to-leaf path length.

Complexity:

TEXT

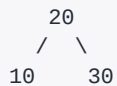
$O(h)$

60. GATE-STYLE VALIDITY QUESTION 1

Which is a BST?

Tree A

TEXT



Valid:

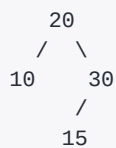
TEXT

$10 < 20 < 30$

' BST.

61. VALIDITY QUESTION 2

TEXT



15 is in right subtree of 20.

Must be:

TEXT

>20

but:

TEXT

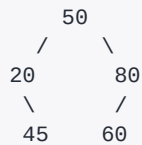
$15 < 20$

Therefore:

Invalid BST

62. VALIDITY QUESTION 3

TEXT



Check:

TEXT

20 < 45 < 50

correct.

TEXT

50 < 60 < 80

correct.

Therefore:

Valid BST

63. GATE-STYLE INORDER QUESTION

BST has keys:

TEXT

7, 2, 9, 1, 5, 8, 10

Regardless of its valid BST shape, inorder output must be:

TEXT

1 2 5 7 8 9 10

assuming these are exactly its keys and distinct.

Important: inorder result depends only on sorted key set, not BST shape.

64. GATE-STYLE HEIGHT QUESTION

A BST has:

TEXT

100 nodes

What is worst-case height?

Using edge convention:

TEXT

99

Worst-case search:

TEXT

$O(100) = O(n)$

65. GATE-STYLE BALANCED QUESTION

A height-efficient BST with 1023 nodes can be perfect.

Since:

TEXT

$1023 = 2^{10} - 1$

height:

TEXT

9

So search can take:

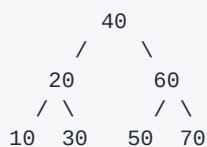
TEXT

$O(\log n)$

66. GATE-STYLE SEARCH TRACE

BST:

TEXT



Search 55:

At 40:

TEXT

5 5 > 4 0 !' r i g h t

At 60:

TEXT

5 5 < 6 0 !' l e f t

At 50:

TEXT

5 5 > 5 0 !' r i g h t

NULL.

Therefore:

TEXT

55 absent

Visited:

TEXT

40, 60, 50

67. GATE TRAP — SEARCH BOTH SUBTREES?

No.

At each node, the BST property determines exactly one direction unless equality occurs.

If code searches:

TEXT

both left and right

for an ordinary key search, it is ignoring the BST advantage.

68. GATE TRAP — INORDER OF ANY BINARY TREE IS SORTED?

False.

Example:

TEXT

```
  10
 /  \
50   1
```

Inorder:

TEXT

50 10 1

Not sorted.

Sorted inorder is a BST property.

69. GATE TRAP — HEAP ROOT VS BST ROOT

Min-Heap root

Guaranteed:

TEXT

minimum

Max-Heap root

Guaranteed:

TEXT

maximum

BST root

Not necessarily minimum, maximum, or median.

Not important.

70. MINIMUM IN BST IS NOT NECESSARILY LEAF?

Interesting subtlety.

The minimum node:

TEXT

has no left child

but may have a:

TEXT

right child

Example:

TEXT

```
  20
 /
10
 \
 15
```

Minimum:

TEXT

10

10 is not a leaf because it has right child 15.

$\emptyset = \$0\%$

Minimum BST node has no left child, but need not be a leaf.

Similarly, maximum has no right child but may have a left child.

71. WHY MIN CANNOT HAVE LEFT CHILD

If minimum node had a left child:

TEXT

left child < minimum

contradiction.

So no left child.

72. WHY MAX CANNOT HAVE RIGHT CHILD

If maximum had a right child:

TEXT

right child > maximum

contradiction.

Therefore no right child.

73. BST SUCCESSOR/PREDECESSOR PREVIEW

In sorted inorder:

TEXT

10 20 30 40 50

For 30:

TEXT

predecessor = 20
successor = 40

So:

BST predecessor/successor are inorder neighbors

under distinct keys.

We'll study exact algorithms in Lesson 62.

74. INSERTION INTUITION PREVIEW

To insert key:

TEXT

65

into:

TEXT

```
50
 \
  70
 /
60
```

Compare:

TEXT

```
6 5 > 5 0 !' r i g h t
6 5 < 7 0 !' l e f t
6 5 > 6 0 !' r i g h t
```

First NULL position becomes insertion point.

Result:

TEXT

```
50
 \
  70
 /
60
 \
  65
```

Lesson 60 will cover this deeply.

75. DELETION PREVIEW

Deletion is more complex because node can have:

TEXT

```
0 children
1 child
2 children
```

Three cases.

We'll cover fully in:

Lesson 61

76. BST OPERATIONS MASTER PREVIEW

| Operation | Complexity |
|-----------|------------|
| Search | $O(h)$ |
| Insert | $O(h)$ |
| Delete | $O(h)$ |
| Minimum | $O(h)$ |
| Maximum | $O(h)$ |
| Traversal | $O(n)$ |

Balanced:

TEXT

$h = O(\log n)$

Skewed:

TEXT

$h = O(n)$

77. BST ADT KEY IDEA

Why use BST?

Because it maintains:

TEXT

dynamic ordered data

while supporting:

- search
- insert
- delete
- min/max
- predecessor/successor
- sorted traversal

Unlike sorted array, insertion/deletion need not require shifting $O(n)$ elements.

But balance matters.

78. BST VS SORTED ARRAY — INTUITION

Sorted Array

Search:

TEXT
 $O(\log n)$

with binary search.

Insert:

TEXT
 $O(n)$

because shifting may be needed.

Balanced BST

Search:

TEXT
 $O(\log n)$

Insert:

TEXT
 $O(\log n)$

Delete:

TEXT
 $O(\log n)$

conceptually.

This is one reason balanced search trees are useful.

79. BUT ORDINARY BST MAY DEGRADE

Without self-balancing:

TEXT
 $O(\log n)$

is not guaranteed.

Worst case:

TEXT
 $O(n)$

So whenever GATE says:

```
TEXT
ordinary BST
```

be careful before assuming logarithmic complexity.

80. NOTEBOOK NOTES — BST DEFINITION

For every node X:

```
TEXT
left subtree keys < X
right subtree keys > X
```

under distinct keys.

Most important:

The rule applies to entire subtrees, not only immediate children.

81. NOTEBOOK NOTES — SEARCH

```
TEXT
key == root !' found
key < root !' left
key > root !' right
```

Complexity:

```
TEXT
O(h)
```

Balanced:

```
TEXT
O(log n)
```

Skewed:

```
TEXT
O(n)
```

82. NOTEBOOK NOTES — INORDER

TEXT

BST inorder = sorted ascending

Reverse inorder:

TEXT

sorted descending

83. NOTEBOOK NOTES — EXTREMES

Minimum:

TEXT

keep moving LEFT

Maximum:

TEXT

keep moving RIGHT

Both:

TEXT

$O(h)$

84. DON'T CONFUSE

Binary Tree

TEXT

only "d 2 children

BST

TEXT

"d 2 children + global ordering

Heap

TEXT

complete shape + parent/child heap ordering

Balanced BST

TEXT

BST + height control

Ø=70% different ideas.

85. MEMORY TRICKS

BST

Smaller Left, Larger Right.

Validity

Don't check child only — check the RANGE.

Inorder

BST + LNR = sorted.

Search

Compare once, choose one road.

Complexity

BST speed follows HEIGHT.

86. 30-SECOND REVISION

Remember:

1. BST is a binary tree with ordering.
2. Entire left subtree < root.
3. Entire right subtree > root.
4. Rule is global, not just parent-child.
5. Search follows one path.
6. Search/insert/delete = $O(h)$.
7. Balanced BST !' $O(\log n)$.
8. Skewed BST !' $O(n)$.
9. Inorder gives sorted ascending keys.
10. Minimum is leftmost; maximum is rightmost.

87. ACTIVE RECALL

Q1

Is every binary tree a BST?

Q2

What is the safest conceptual way to validate a BST?

Q3

What traversal gives BST keys in ascending order?

Q4

What is search complexity in terms of height?

Q5

Does ordinary BST guarantee $O(\log n)$ search?

ANSWERS

A1

No.

A2

Maintain valid lower/upper ranges for every node.

A3

Inorder.

A4

TEXT

$O(h)$

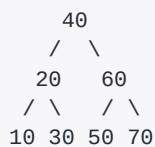
A5

No. A skewed BST can require $O(n)$.

88. PRACTICE — VALID OR INVALID

Tree A

TEXT



BST?

Tree B

TEXT

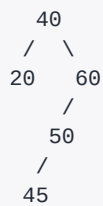


BST?

Careful: 45 is in left subtree of 40.

Tree C

TEXT

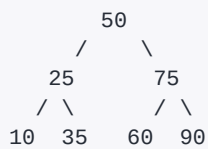


BST?

89. PRACTICE — SEARCH TRACE

For:

TEXT



Trace searches for:

1. 35
2. 65
3. 90

Write only the visited-node path.

90. PRACTICE — COMPLEXITY

For a BST containing n nodes, state:

1. best-case search
2. worst-case search
3. balanced search
4. traversal complexity
5. iterative-search auxiliary space

91. BST MODULE ROADMAP CHECKPOINT

We now begin Module 7:

' Lesson 59 — BST Fundamentals

Remaining:

- Lesson 60 — Search & Insertion
- Lesson 61 — Deletion
- Lesson 62 — Min, Max, Predecessor & Successor
- Lesson 63 — Complexity, Height & GATE Properties
- Lesson 64 — Placement BST Patterns

Nothing is being skipped.

NEXT — LESSON 60

BST Search & Insertion — Exact C Logic, Pointer Updates, Tracing & Complexity

Next we will master:

TEXT

Search:

```
c o m p a r e  ! '   c h o o s e   l e f t / r i g h t
```

and insertion:

TEXT

follow search path

```
! '   r e a c h   N U L L
```

```
! '   a t t a c h   n e w   l e a f
```

with:

- recursive search
 - iterative search
 - recursive insertion
 - iterative insertion
 - Node ** pointer approach
 - duplicate handling
 - exact insertion traces
 - how insertion order changes tree shape
 - GATE sequence questions
 - balanced vs skewed complexity.
-

USER

next

ASSISTANT

MODULE 7 — LESSON 60

BST Search & Insertion — Exact Logic, C Code, Pointer Updates, Tracing & Complexity

Now we move from BST theory to actual operations.

Today:

SEARCH + INSERTION

These two are closely related because insertion first follows the **same path as search**.

GATE importance: Ø=Ý%ØMÚ%ØKØW **Placement importance:** Ø=Ý%ØMÚ%ØKØW

1. CORE BST RULE

For every node:

TEXT

```
left subtree < node
right subtree > node
```

under distinct-key convention.

So for a target `key`:

TEXT

```
key == root !' found
key < root !' go left
key > root !' go right
```

That single rule drives both search and insertion.

2. SEARCH INTUITION

BST:

TEXT



Search:

```
TEXT
60
```

At 50:

```
TEXT
60 > 50
```

go right.

At 70:

```
TEXT
60 < 70
```

go left.

At 60:

```
TEXT
found
```

Path:

50 !' 70 !' 60

3. SEARCH — UNSUCCESSFUL CASE

Search:

```
TEXT
65
```

Trace:

```
TEXT
50 !'   r i g h t
70 !'   l e f t
60 !'   r i g h t
NULL
```

So:

```
TEXT
65 is absent
```

~~Ø=Nil~~ Unsuccessful BST search ends when the required child pointer is `NULL`.

4. ITERATIVE SEARCH IN C

```
C
Node *search(Node *root, int key)
{
    while (root != NULL)
    {
        if (key == root->data)
            return root;

        if (key < root->data)
            root = root->left;
        else
            root = root->right;
    }

    return NULL;
}
```

5. SEARCH COMPLEXITY

Search follows one path of length at most proportional to height.

FORMULA BOX

TEXT

Time = $O(h)$

Iterative extra space:

TEXT

$O(1)$

Recursive version:

TEXT

$O(h)$

stack space.

6. RECURSIVE SEARCH

```
C
Node *search(Node *root, int key)
{
    if (root == NULL || root->data == key)
        return root;

    if (key < root->data)
        return search(root->left, key);

    return search(root->right, key);
}
```

```
}
```

7. WHY ONLY ONE RECURSIVE CALL?

Normal binary-tree search might explore:

```
TEXT
left AND right
```

But BST ordering lets us discard one subtree.

Example:

```
TEXT
key < root
```

means every right-subtree key is too large relative to that root's ordering region.

So only:

```
TEXT
left
```

needs searching.

0-100% is the entire BST advantage.

8. NOW INSERTION

Suppose BST:

```
TEXT
      50
     /  \
    30   70
```

Insert:

```
TEXT
60
```

We must preserve BST ordering.

Compare:

```
TEXT
60 > 50 !' right
60 < 70 !' left
```

Left child of 70 is NULL.

So insert 60 there:

TEXT



9. INSERTION MASTER RULE

Search until the required child position becomes NULL.

Then attach the new node there.

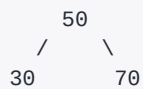
~~0-New~~ New node is initially inserted as a:

LEAF

10. INSERT 40

Start:

TEXT



Insert 40:

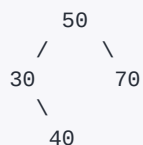
TEXT

```
40 < 50 !' l e f t
40 > 30 !' r i g h t
```

NULL.

So:

TEXT



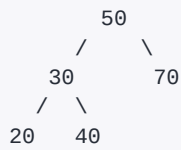
11. INSERT 20

TEXT

```
20 < 50 !' l e f t
20 < 30 !' l e f t
```


Result:

TEXT



12. RECURSIVE INSERTION — CORE IDEA

For subtree rooted at `root`:

TEXT

```
if root == NULL:
    create and return new node

if key < root:
    insert into left subtree

if key > root:
    insert into right subtree

return root
```

13. C IMPLEMENTATION— RECURSIVE

C

```
Node *insert(Node *root, int key)
{
    if (root == NULL)
        return createNode(key);

    if (key < root->data)
        root->left = insert(root->left, key);
    else if (key > root->data)
        root->right = insert(root->right, key);

    return root;
}
```

This version ignores duplicates.

0-1% Extremely important code pattern.

14. WHY DO WE RETURN `root`?

Suppose:

C

```
root->left = insert(root->left, key);
```

The recursive call may:

- return the existing subtree root
- or return a newly created node if subtree was NULL

So we reconnect the returned subtree pointer.

Then ultimately:

```
c
return root;
```

preserves the entire tree.

15. EXACT POINTER TRACE

Insert 20 into:

```
TEXT
30
```

Call:

```
TEXT
insert(30,20)
```

Since:

```
TEXT
20 < 30
```

execute:

```
c
root->left = insert(root->left, 20);
```

Current:

```
TEXT
root->left = NULL
```

So recursive call:

```
TEXT
insert(NULL,20)
```

returns new node:

```
TEXT
20
```

Then:

```
C
root->left = new20;
```

Result:

```
TEXT
    30
    /
    20
```

Then function returns pointer to 30.

0-1% This is why assignment to `root->left` or `root->right` matters.

16. COMMON WRONG CODE

Wrong:

```
C
if (key < root->data)
    insert(root->left, key);
```

Why wrong?

If `root->left == NULL`, recursive call creates a node and returns its pointer.

But nobody stores that returned pointer.

So the new node is lost from the tree.

Correct:

```
C
root->left = insert(root->left, key);
```

0-1% Important pointer-update trap.

17. INSERTING INTO EMPTY TREE

Initially:

```
TEXT
root = NULL
```

Call:

```
C
root = insert(root, 50);
```

`insert(NULL, 50)` returns:

TEXT

```
new node 50
```

So:

TEXT

```
root = 50
```

If you write only:

C

```
insert(root, 50);
```

without assigning returned root, original caller's `root` remains NULL.

Very important.

18. WHY ROOT MAY CHANGE?

For ordinary insertion into a non-empty BST, the root value stays same.

But if tree is empty:

TEXT

```
NULL
```

new node becomes root.

So safest call:

C

```
root = insert(root, key);
```

This pattern also becomes essential for deletion, where subtree roots can definitely change.

19. NODE CREATION FUNCTION

C

```
Node *createNode(int key)
{
    Node *newNode = malloc(sizeof *newNode);

    if (newNode == NULL)
        return NULL;

    newNode->data = key;
    newNode->left = NULL;
    newNode->right = NULL;

    return newNode;
}
```

Time:

TEXT

0(1)

20. INSERTION TRACE — FULL EXAMPLE

Insert sequence:

TEXT

50, 30, 70, 20, 40, 60, 80

Start empty.

21. INSERT 50

TEXT

50

50 becomes root.

22. INSERT 30

TEXT

30 < 50

go left:

TEXT

```
  50
 /
30
```

23. INSERT 70

TEXT

70 > 50

go right:

TEXT

```
  50
 /  \
30    70
```

24. INSERT 20

TEXT

20 < 50
20 < 30

Result:

TEXT

```
      50
     /  \
    30    70
   /
  20
```

25. INSERT 40

TEXT

40 < 50
40 > 30

Result:

TEXT

```
      50
     /  \
    30    70
   /  \
  20   40
```

26. INSERT 60

TEXT

60 > 50
60 < 70

Result:

TEXT

```
      50
     /  \
    30    70
   /  \  /
  20  40 60
```

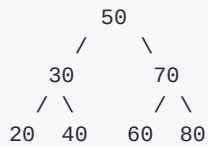
27. INSERT 80

TEXT

80 > 50
80 > 70

Final:

TEXT



This happens to be balanced/perfect.

28. INORDER CHECK

Inorder:

TEXT

20 30 40 50 60 70 80

Sorted.

This is a quick verification that insertion preserved BST ordering.

29. INSERTION ORDER MATTERS Ø=Ý%Ø=Ý%Ø=Ý%

Same key set:

TEXT

10, 20, 30, 40, 50

Insert in ascending order.

30. TRACE ASCENDING INSERTION

10:

TEXT

10

20:

TEXT

10
 \
 20

30:

TEXT

```
10
 \
  20
   \
    30
```

Continue:

TEXT

```
10
 \
  20
   \
    30
     \
      40
       \
        50
```

Right-skewed.

31. COMPLEXITY CONSEQUENCE

Height:

TEXT

$h = n - 1$

So insertion/search:

TEXT

$O(n)$

Worst case.

~~$O(n^2)$~~ Inserted insertion into ordinary BST can produce worst-case shape.

32. BETTER INSERTION ORDER

Same values:

TEXT

10, 20, 30, 40, 50, 60, 70

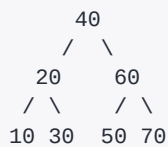
Insert:

TEXT

40, 20, 60, 10, 30, 50, 70

Result:

TEXT



Height:

TEXT

2

Operations:

TEXT

$O(\log n)$

on this shape.

33. GATE TRAP — SAME SET, DIFFERENT BST

A set of keys does **not** uniquely determine a BST shape.

Insertion order matters.

Example keys:

TEXT

10, 20, 30

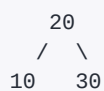
Sequence:

TEXT

20, 10, 30

produces:

TEXT



But:

TEXT

10, 20, 30

produces:

TEXT



```
20
 \
 30
```

Same inorder.

Different shape.

34. ITERATIVE INSERTION

We can avoid recursion.

Need pointers:

```
TEXT
curr
parent
```

`curr` walks down.

`parent` remembers where to attach new node.

35. ITERATIVE LOGIC

```
TEXT
curr = root
parent = NULL

while curr != NULL:
    parent = curr

    if key < curr->data:
        curr = curr->left
    else:
        curr = curr->right

create new node

attach to parent on correct side
```

36. C IMPLEMENTATION— ITERATIVE

```
C
Node *insertIterative(Node *root, int key)
{
    Node *newNode = createNode(key);

    if (newNode == NULL)
        return root;

    if (root == NULL)
        return newNode;

    Node *curr = root;
```

```

Node *parent = NULL;

while (curr != NULL)
{
    parent = curr;

    if (key < curr->data)
        curr = curr->left;
    else if (key > curr->data)
        curr = curr->right;
    else
    {
        free(newNode);
        return root;
    }
}

if (key < parent->data)
    parent->left = newNode;
else
    parent->right = newNode;

return root;
}

```

37. WHY `parent` IS NEEDED

At the end:

```

TEXT
curr = NULL

```

We know where insertion should happen, but NULL itself doesn't tell us whose child pointer must change.

`parent` remembers the previous node.

Example:

```

TEXT
5 0  !'   3 0  !'   N U L L

```

At termination:

```

TEXT
curr = NULL
parent = 30

```

Then determine:

```

TEXT
key < 30 ?

```

Attach left or right.

38. ITERATIVE COMPLEXITY

Walking down one path:

TEXT

$O(h)$

Extra space:

TEXT

$O(1)$

So:

| Version | Time | Auxiliary Space |
|------------------|--------|-----------------|
| Recursive insert | $O(h)$ | $O(h)$ |
| Iterative insert | $O(h)$ | $O(1)$ |

39. DUPLICATES — WHAT SHOULD INSERT DO?

Suppose tree has:

TEXT

50

and we insert another 50.

Possible policies:

Policy 1 — Reject

TEXT

duplicate not inserted

Policy 2 — Always left

TEXT

key <= root

left.

Policy 3 — Always right

TEXT

key >= root

right.

Policy 4 — Frequency count

TEXT

key=50
count=2

NOTE questions usually specify or assume distinct keys.

40. DUPLICATE POLICY MUST BE CONSISTENT

Bad approach:

Sometimes equal values left, sometimes right.

That makes search/validation ambiguous.

If duplicates are allowed, use a deterministic convention.

41. Node ** INSERTION — POINTER-TO-POINTER

You can modify the caller's pointer directly using:

```
C
Node **
```

Example:

```
C
void insert(Node **root, int key)
{
    if (*root == NULL)
    {
        *root = createNode(key);
        return;
    }

    if (key < (*root)->data)
        insert(&(*root)->left, key);
    else if (key > (*root)->data)
        insert(&(*root)->right, key);
}
```

Then call:

```
C
insert(&root, 50);
```

42. WHY Node ** WORKS

Suppose:

TEXT

```
root = NULL
```

`&root` is the address of the root pointer.

Inside function:

TEXT

```
*root
```

refers to caller's actual pointer variable.

So:

C

```
*root = createNode(key);
```

directly changes caller's root.

This connects to your pointer-to-pointer lessons.

Good placement/C understanding.

43. RECURSIVE RETURN STYLE VS Node **

Both are valid.

Return-new-root style

C

```
root = insert(root, key);
```

Pointer-to-pointer style

C

```
insert(&root, key);
```

For GATE/DSA explanations, return-new-root style is often simpler.

44. SEARCH VS INSERT

Search:

TEXT

```
follow comparisons
```

until:

TEXT

```
found  
or NULL
```

Insertion:

TEXT
follow comparisons

until:

TEXT
NULL

then:

TEXT
create/attach node

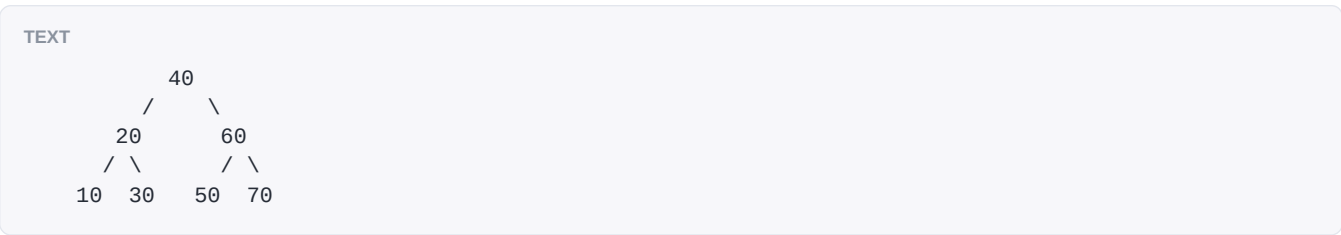
$\emptyset = \text{link}$:

Insertion = unsuccessful search + attach

Excellent memory trick.

45. SEARCH TRACE TABLE

BST:



Search 50:

| Current | Compare | Action |
|---------|---------|--------|
| 40 | 50 > 40 | Right |
| 60 | 50 < 60 | Left |
| 50 | equal | Found |

46. INSERT TRACE TABLE

Insert:

TEXT
55

| Current | Compare | Action |
|---------|-----------|--------|
| 40 | $55 > 40$ | Right |
| 60 | $55 < 60$ | Left |
| 50 | $55 > 50$ | Right |
| NULL | — | Insert |

Result:

TEXT

```
graph TD; 40 --> 20; 40 --> 60; 60 --> 50; 60 --> 70; 50 --> 55;
```

47. GATE-STYLE INSERTION SEQUENCE

Insert into empty BST:

TEXT

50, 30, 20, 40, 70, 60, 80

Result:

TEXT

```
graph TD; 50 --> 30; 50 --> 70; 30 --> 20; 30 --> 40; 70 --> 60; 70 --> 80;
```

Inorder:

TEXT

20 30 40 50 60 70 80

48. GATE QUESTION — ROOT FROM INSERTION SEQUENCE

Insert into empty BST:

TEXT

40, 20, 60, 10, 30

Which key is root?

Answer:

TEXT

40

Because first inserted key becomes root and ordinary BST insertion never rotates/rebalances.

Unless self-balancing behavior is explicitly specified.

49. IMPORTANT — ORDINARY BST DOES NOT REARRANGE EXISTING NODES

When inserting:

TEXT

new key

we simply attach it at a NULL position.

Ordinary BST does not automatically:

- rotate
- rebalance
- choose new median root

Those are features of self-balancing BST variants.

50. GATE TRAP — FIRST KEY

For an ordinary BST built by sequential insertion:

First key becomes root

Example:

TEXT

8, 3, 10, 1, 6

Root:

TEXT

8

Simple but frequently useful.

51. GATE TRAP — LAST INSERTED NODE

A newly inserted node is initially:

Leaf

because its child pointers start NULL.

But later insertions may make it an internal node.

Example:

Insert 30.

Initially leaf.

Later insert 20 and 40 below it.

Then 30 becomes internal.

52. GATE TRAP — INSERTION COST IS NOT ALWAYS $\log n$

Again:

TEXT

Insertion = $O(h)$

not automatically:

TEXT

$O(\log n)$

Worst ordinary BST:

TEXT

$O(n)$

53. TOTAL COST OF SORTED INSERTIONS

Insert:

TEXT

$1, 2, 3, \dots, n$

into ordinary empty BST.

Costs approximately:

TEXT

$1 + 2 + 3 + \dots + (n-1)$

because each new value traverses deeper.

Total:

FORMULA BOX

TEXT

$\sim (n^2)$

0=important GATE/algorithm analysis result.

54. EXAMPLE

Insert:

TEXT

10, 20, 30, 40, 50

comparisons/path lengths grow:

TEXT

```
1 0 !' r o o t
2 0 !' 1 c o m p a r i s o n
3 0 !' 2
4 0 !' 3
5 0 !' 4
```

Total proportional to:

TEXT

1+2+3+4

So quadratic trend.

55. BALANCED INSERTION SEQUENCE TOTAL COST

If insertion maintains a balanced-ish tree of height:

TEXT

$O(\log n)$

each insertion costs approximately:

TEXT

$O(\log n)$

for n insertions:

TEXT

$O(n \log n)$

This is one reason balancing matters.

56. BST SORT USING INSERTION

Conceptually:

1. Insert all elements into BST.
2. Perform inorder traversal.

If tree balanced:

```
TEXT
insertions  O(n log n)
inorder     O(n)
```

Total:

```
TEXT
O(n log n)
```

But worst-case ordinary BST:

```
TEXT
O(n2)
```

because tree may skew.

Ø=Useful algorithmic connection.

57. FAILED SEARCH AND INSERTION POSITION

If searching for a missing key:

```
TEXT
35
```

gives path:

```
TEXT
5 0 !'  3 0 !'  4 0 !'  N U L L - l e f t
```

then if we now insert 35, it goes exactly at that NULL:

```
TEXT
      40
     /
    35
```

Ø=Search failure path determines insertion position.

58. PARENT/CHILD CONSTRAINT AFTER INSERT

Suppose key 35 is inserted into:

TEXT

```
50
/
30
 \
  40
```

It becomes:

TEXT

```
50
/
30
 \
  40
/
35
```

Check all ancestors:

TEXT

```
35 < 50
35 > 30
35 < 40
```

All constraints satisfied automatically because search path encoded them.

59. WHY INSERTION PRESERVES GLOBAL BST PROPERTY

At every comparison we narrow valid range.

Example:

TEXT

```
5 0 !' l e f t
3 0 !' r i g h t
4 0 !' l e f t
```

means new value must satisfy:

TEXT

```
>30
<40
<50
```

By following those decisions, the final NULL location corresponds to the correct valid range.

So local attachment after a correct search path preserves the global property.

60. GATE-STYLE SHAPE QUESTION

Insert:

TEXT

4, 2, 6, 1, 3, 5, 7

Result:

TEXT



Height:

TEXT

2

Perfect tree.

61. GATE-STYLE SHAPE QUESTION 2

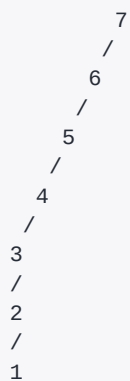
Insert:

TEXT

7, 6, 5, 4, 3, 2, 1

Result:

TEXT



Left-skewed.

Height:

TEXT

6

Search worst case:

TEXT

$O(7)=O(n)$

62. GATE-STYLE QUESTION — INORDER AFTER INSERTIONS

Insert any set of distinct keys into a BST.

Question asks inorder traversal.

You often don't need to construct the tree.

Just:

Sort the keys ascending

Example insertion:

TEXT

50, 20, 80, 10, 30, 70, 90

Inorder:

TEXT

10 20 30 50 70 80 90

Ø = Null
Use exam shortcut.

63. BUT FOR PREORDER/POSTORDER YOU NEED SHAPE

Given insertion order:

TEXT

50, 20, 80, 10, 30

To find preorder/postorder, tree shape matters.

Preorder:

TEXT

50 20 10 30 80

Postorder:

TEXT

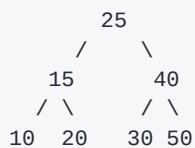
10 30 20 80 50

Do not just sort.

64. GATE-STYLE SEARCH PATH QUESTION

BST:

TEXT



Could the search path for 35 be:

TEXT

2 5 !' 4 0 !' 3 0

Yes.

After 30:

TEXT

35 > 30

would continue right.

Could the path be:

TEXT

2 5 !' 1 5 !' 2 0 !' 3 5

No.

Once key $35 > 25$, search must go right immediately.

~~0-50~~ Search paths must respect comparison constraints.

65. SEARCH PATHRANGE TRICK

Suppose path begins:

TEXT

5 0 !' 2 0 !' 4 0

Target must satisfy:

At 50, went left:

TEXT

target < 50

At 20, went right:

TEXT

target > 20

So after reaching 40, target range is:

TEXT

$20 < \text{target} < 50$

Useful in GATE path-validity questions.

66. EXAMPLE INVALID SEARCH PATH

Search key 65.

Proposed path:

TEXT

5 0 !' 3 0 !' 7 0 !' 6 5

Impossible.

At 50:

TEXT

$65 > 50$

must go right.

Cannot go to 30.

So proposed search path is invalid.

67. GATE-STYLE COMPARISON COUNT

Balanced BST of height h .

Worst number of actual nodes examined during successful search:

TEXT

$h+1$

because height counts edges.

Asymptotically:

TEXT

$O(h)$

68. UNSUCCESSFUL SEARCH COMPARISONS

May examine at most:

TEXT

$h+1$

actual nodes before following one NULL pointer.

Again:

```
TEXT
o(h)
```

69. BEST CASE INSERTION

If tree empty:

```
TEXT
o(1)
```

new root created immediately.

If non-empty, insertion generally follows some path.

Complexity still expressed:

```
TEXT
o(h)
```

70. MEMORY MANAGEMENT TRAP

When `malloc` fails:

```
C
createNode()
```

may return NULL.

Robust code should not blindly attach it as though allocation succeeded.

For exam pseudocode, allocation failure is often ignored, but in real C it matters.

71. DO NOT FREE EXISTING TREE DURING INSERT

Insertion allocates:

```
TEXT
one new node
```

and modifies:

```
TEXT
one NULL child link
```

It does not free or relocate existing nodes in an ordinary BST.

72. POINTER UPDATE VISUAL

Before inserting 35:

```
TEXT
      40
     /
    NULL
```

Pointer:

```
TEXT
40 . l e f t ! '   N U L L
```

Create:

```
TEXT
n e w ! '   [ 3 5   |   N U L L   |   N U L L ]
```

Update:

```
TEXT
40 . l e f t = n e w
```

After:

```
TEXT
      40
     /
    35
```

Think of insertion as replacing one **NULL** child pointer with a new-node address.

73. HOW MANY NULL LINKS CHANGE?

Before insertion in a non-empty BST with **n** nodes:

```
TEXT
NULL links = n+1
```

When adding one node:

- one old NULL link becomes non-NULL
- new node contributes two NULL links

Net increase:

```
TEXT
-1 + 2 = +1
```

New tree has:

TEXT

$(n+1)+1 = n+2$

NULL links.

Matches formula for $n+1$ nodes:

TEXT

$(n+1)+1$

0-Max GATE consistency result.

74. INSERTION EDGE COUNT CHANGE

Tree with n nodes:

TEXT

edges = $n-1$

Insert one new node.

New nodes:

TEXT

$n+1$

New edges:

TEXT

n

Exactly one new edge was added.

Makes sense because inserted node attaches to one parent.

75. BST INSERTION DOES NOT CHANGE OLD INORDER RELATIVE ORDER

Suppose old inorder:

TEXT

10 20 30 40

Insert 25.

New inorder:

TEXT

10 20 25 30 40

Existing keys remain in sorted relative order.

The new key appears in its proper sorted position.

76. PLACEMENT PATTERN— INSERT INTO BST

Typical interview request:

Given BST root and integer key, return root after insertion.

Standard solution:

```
c
Node *insert(Node *root, int key)
{
    if (!root)
        return createNode(key);

    if (key < root->data)
        root->left = insert(root->left, key);
    else if (key > root->data)
        root->right = insert(root->right, key);

    return root;
}
```

This concise pattern is worth mastering.

77. PLACEMENT PATTERN— SEARCH BST

```
c
Node *search(Node *root, int key)
{
    while (root)
    {
        if (key == root->data)
            return root;

        root = key < root->data
            ? root->left
            : root->right;
    }

    return NULL;
}
```

For your exam notes, avoid the ternary if you prefer clearer branching.

78. SEARCH/INSERT MASTER COMPLEXITY

FORMULA BOX

TEXT

Search = $O(h)$
Insert = $O(h)$

Balanced:

TEXT

$O(\log n)$

Skewed:

TEXT

$O(n)$

Iterative extra space:

TEXT

$O(1)$

Recursive extra space:

TEXT

$O(h)$

79. DON'T CONFUSE

Search

TEXT

Stops at FOUND or NULL

Insertion

TEXT

Stops at NULL and attaches new node

New node

TEXT

initially a leaf

Insertion order

TEXT

determines shape

Ordinary BST

TEXT

does not self-balance

80. MEMORY TRICKS

Search

Compare and choose one road.

Insert

Search until NULL, then plant the node.

Recursive insert

Return the subtree root and reconnect it.

Complexity

BST operation speed = tree height.

81. NOTEBOOK NOTES

Search

TEXT

```
key == node !' found
key < node !' left
key > node !' right
```

TEXT

Time = $O(h)$

Insert

TEXT

Follow search path.
At first NULL position,
create and attach new node.

TEXT

Time = $O(h)$

82. RECURSIVE INSERT TEMPLATE

```

c
Node *insert(Node *root, int key)
{
    if (root == NULL)
        return createNode(key);

    if (key < root->data)
        root->left = insert(root->left, key);
    else if (key > root->data)
        root->right = insert(root->right, key);

    return root;
}

```

Copy this.

83. 30-SECOND REVISION

Remember:

1. Search compares and chooses one subtree.
2. Search ends at key or NULL.
3. Insertion follows the same search path.
4. New key is attached at first valid NULL.
5. New node starts as leaf.
6. Recursive insertion must reconnect returned child pointer.
7. Call `root = insert(root, key)`.
8. Search/insert = $O(h)$.
9. Balanced = $O(\log n)$.
10. Sorted insertion can create $O(n)$ -height skew.

84. ACTIVE RECALL

Q1

What is insertion conceptually?

Q2

Why do we write:

```

c
root->left = insert(root->left, key);

```

instead of merely calling `insert(...)`?

Q3

What is insertion time complexity?

Q4

What happens when sorted data is inserted into an ordinary BST?

Q5

What is iterative insertion auxiliary space?

ANSWERS

A1

An unsuccessful BST search followed by attaching a new node at the NULL position.

A2

Because the recursive call may return a newly created subtree root that must be connected to the parent.

A3

TEXT

$O(h)$

A4

It can become skewed with height $n-1$.

A5

TEXT

$O(1)$

85. PRACTICE — BUILD THE BST

Insert:

TEXT

45, 25, 75, 15, 35, 60, 90, 30, 40

Find:

1. final tree
2. inorder
3. preorder
4. postorder
5. height
6. path to search 30
7. path to search 65

86. PRACTICE — INSERTION ORDER

Build two BSTs using the same keys:

TEXT

10, 20, 30, 40, 50, 60, 70

Sequence A

TEXT

10, 20, 30, 40, 50, 60, 70

Sequence B

TEXT

40, 20, 60, 10, 30, 50, 70

Compare their:

- height
- shape
- worst search cost

This captures the entire BST performance idea.

87. BST MODULE PROGRESS

' Lesson 59 — BST Fundamentals ' **Lesson 60 — Search & Insertion**

Remaining:

- Lesson 61 — BST Deletion
- Lesson 62 — Min, Max, Predecessor, Successor
- Lesson 63 — Complexity & GATE Properties
- Lesson 64 — Placement BST Patterns

NEXT — LESSON 61

BST Deletion — Leaf, One Child, Two Children, Inorder Successor/Predecessor & Exact Pointer Updates

This is the most important BST operation lesson because deletion has **three separate cases**:

TEXT

Case 1: 0 children
Case 2: 1 child
Case 3: 2 children

We will cover each using:

BEFORE !' POINTER CHANGES !' AFTER

plus recursive C code, why successor/predecessor replacement works, complexity, and GATE deletion traces.

next

MODULE 7 — LESSON 61

BST deletion is more difficult than search or insertion because removing a node can break the tree structure.

Delete the key while preserving the BST property

GATE importance: 0=Ý% ÐÁÚÓÐÆÐÞ Placement importance: 0=Ý% ÐÁÚÓÐÆÐÞ

For the node we want to delete:

```
C a s e   1   !'   0   c h i l d r e n
C a s e   2   !'   1   c h i l d
C a s e   3   !'   2   c h i l d r e n
```

0 !' Remove

2 !' Replace by inorder neighbor

Ø=Memorize this before anything else.

Deletion begins exactly like BST search.

```
key < root !' go left
key > root !' go right
key == root !' delete this node
```

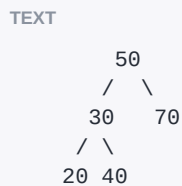
So reaching the node costs:

TEXT
 $O(h)$

where h is tree height.

3. CASE 1 — DELETE A LEAF $O = Y\%$

Consider:



Delete:

TEXT
20

20 has:

TEXT
0 children

So it is a leaf.

4. BEFORE



Relevant pointer:

TEXT
30 . left !' 20

5. POINTER CHANGES

Free node 20:

```
C
free(node20);
```

and make parent's pointer:

```
TEXT
30 . left !' NULL
```

6. AFTER

```
TEXT
      50
     /  \
    30   70
     \
     40
```

Simple.

Leaf deletion = parent link becomes NULL

7. CASE 1 RECURSIVE VIEW

If recursive deletion reaches:

```
TEXT
root = node 20
```

and both children are NULL:

```
C
root->left == NULL
root->right == NULL
```

we can:

```
C
free(root);
return NULL;
```

The returned NULL gets assigned into the parent's child pointer.

That is why deletion functions commonly return the new subtree root.

8. CASE 2 — NODE WITH ONE CHILD

Consider:

TEXT



Delete:

TEXT

70

70 has only one child:

TEXT

60

We cannot simply set:

TEXT

`50.right = NULL`

because then subtree containing 60 would be lost.

Instead:

Connect parent directly to the child

9. BEFORE

TEXT



Pointers:

TEXT

```
50 . right !' 70
70 . left !' 60
```

10. POINTER CHANGES

We want:

TEXT

```
50 . right !' 60
```

Then free:

TEXT
70

11. AFTER

TEXT

```
  50
 /  \
30    60
```

BST remains valid because:

TEXT
 $60 > 50$

12. ONE-CHILD RULE

If deleted node has only left child:

TEXT
return left child

If only right child:

TEXT
return right child

The parent receives that child as the new root of the affected subtree.

13. CASE 2 EXAMPLE — RIGHT CHILD ONLY

Before:

TEXT

```
  30
   \
   40
```

Delete 30.

30 has one child:

TEXT
40

After:

TEXT

40

If 30 was root of the entire tree:

TEXT

root changes from 30 to 40

Important: deletion can change the root.

14. WHY WE WRITE

C

```
root = deleteNode(root, key);
```

Because the root may change.

Examples:

- deleting the only node
- deleting root with one child
- recursive subtree root replacement

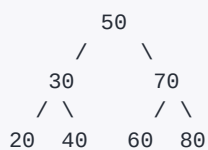
So always preserve returned pointer.

15. CASE 3 — NODE WITH TWO CHILDREN

This is the most important case.

Consider:

TEXT



Delete:

TEXT

50

50 has two children.

We cannot simply connect one child to the parent because both subtrees must be preserved.

16. WHAT VALUE CAN REPLACE 50?

We need a replacement that keeps:

TEXT

```
all left values < replacement
all right values > replacement
```

Two standard choices:

Option 1

Inorder Successor

Smallest key in the right subtree.

Option 2

Inorder Predecessor

Largest key in the left subtree.

Both are valid.

17. SUCCESSOR METHOD

Right subtree:

TEXT

```
  70
 /  \
60   80
```

Smallest value there:

TEXT

60

So inorder successor of 50 is:

60

Replace 50's value with 60.

18. BEFORE

TEXT

```
      50
     /  \
    30   70
   / \  / \
  20 40 60 80
```

19. STEP 1 — COPY SUCCESSOR VALUE

Copy:

TEXT

60

into node currently containing 50.

Temporary logical state:

TEXT



Now there are temporarily two 60s.

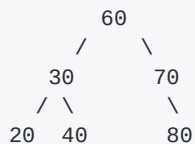
20. STEP 2 — DELETE ORIGINAL SUCCESSOR NODE

Original 60 lies in right subtree.

Delete it.

Result:

TEXT



Correct BST.

21. WHY SUCCESSOR IS EASY TO DELETE

The inorder successor is:

TEXT

minimum node in right subtree

Therefore it cannot have:

TEXT

a left child

Otherwise there would be an even smaller node.

So successor has:

TEXT

0 or 1 child

specifically possibly a right child.

Ø=1% This converts difficult Case 3 into simpler Case 1 or Case 2.

22. PREDECESSOR METHOD

Instead choose:

TEXT

largest value in left subtree

For:

TEXT

```
  30
 /  \
20   40
```

largest is:

TEXT

40

Replace 50 with 40, then delete original 40.

Result:

TEXT

```
    40
   /  \
  30   70
 /  \  /  \
20  60 80
```

Also valid.

23. GATE-SAFE RULE

When deleting a node with two children:

Use either

- inorder successor = minimum of right subtree
- inorder predecessor = maximum of left subtree

unless the question explicitly specifies one.

Different choices can produce different valid BST shapes.

Ø=0% Do not assume the final tree is unique unless deletion method is specified.

24. DON'T CONFUSE SUCCESSOR

Successor is not simply:

TEXT
right child

It is:

leftmost node of right subtree

Example:

TEXT

```
graph TD; 50 --> 80; 80 --> 70; 70 --> 60;
```

Successor of 50:

TEXT
60

not 80.

25. DON'T CONFUSE PREDECESSOR

Predecessor is:

rightmost node of left subtree

Example:

TEXT

```
graph TD; 50 --> 20; 20 --> 30; 30 --> 40;
```

Predecessor of 50:

TEXT
40

not 20.

26. FULL RECURSIVE C CODE Ø=Ý%Ø=Ý%Ø=Ý%

```
c
Node *deleteNode(Node *root, int key)
{
    if (root == NULL)
        return NULL;

    if (key < root->data)
    {
        root->left = deleteNode(root->left, key);
    }
    else if (key > root->data)
    {
        root->right = deleteNode(root->right, key);
    }
    else
    {
        // Case 1 and Case 2
        if (root->left == NULL)
        {
            Node *temp = root->right;
            free(root);
            return temp;
        }
        else if (root->right == NULL)
        {
            Node *temp = root->left;
            free(root);
            return temp;
        }

        // Case 3: two children
        Node *temp = findMin(root->right);

        root->data = temp->data;

        root->right =
            deleteNode(root->right, temp->data);
    }

    return root;
}
```

Ø=Ý% is the standard BST deletion pattern.

27. WAIT — WHERE IS THE LEAF CASE?

Notice:

```
c
if (root->left == NULL)
{
    Node *temp = root->right;
    free(root);
    return temp;
}
```

For a leaf:

TEXT

```
left  = NULL
right = NULL
```

So:

TEXT

```
temp = NULL
```

free root.

Return NULL.

Therefore the same branch handles:

TEXT

```
0 children
```

and:

TEXT

```
right-child-only
```

cases.

28. SECOND BRANCH

C

```
else if (root->right == NULL)
```

means left exists but right does not.

So this handles:

TEXT

```
left-child-only
```

case.

29. findMin() HELPER

C

```
Node *findMin(Node *root)
{
    while (root != NULL && root->left != NULL)
        root = root->left;

    return root;
}
```

For two-child deletion:

```
C
Node *temp = findMin(root->right);
```

gets inorder successor.

30. WHY COPY DATA INSTEAD OF MOVING WHOLE NODE?

The simplest implementation does:

```
C
root->data = temp->data;
```

Then recursively deletes the successor node.

This avoids complicated pointer rewiring of two entire subtrees.

Conceptually:

```
TEXT
copy replacement key
then remove duplicate
```

31. EXACT TRACE — DELETE LEAF

BST:

```
TEXT
      50
     /  \
    30   70
   /
  20
```

Call:

```
TEXT
deleteNode(50, 20)
```

$20 < 50$:

```
TEXT
root->left = deleteNode(30, 20)
```

$20 < 30$:

```
TEXT
30.left = deleteNode(20, 20)
```

At 20:

TEXT

```
key == data
```

left NULL.

So:

TEXT

```
temp = right = NULL
free(20)
return NULL
```

Then:

TEXT

```
30.left = NULL
```

Correct.

32. EXACT TRACE — DELETE ONE CHILD

Tree:

TEXT

```
    50
   /
  30
   \
   40
```

Delete 30.

At 30:

TEXT

```
left = NULL
right = 40
```

Execute:

TEXT

```
temp = root->right
```

So:

TEXT

```
temp != 40
```

Then:


```
c
free(root);
return temp;
```

Parent receives:

```
TEXT
50.left = 40
```

33. BEFORE !' POINTER CHANGES !' AFTER

BEFORE

```
TEXT
5 0 . l e f t !'   3 0
3 0 . r i g h t !'  4 0
```

POINTER CHANGES

```
TEXT
t e m p !'   4 0
free(30)
return temp
5 0 . l e f t !'   4 0
```

AFTER

```
TEXT
      50
     /
    40
```

0=1% This pointer sequence is important.

34. EXACT TRACE — TWO CHILDREN

Tree:

```
TEXT
      50
     /  \
    30   70
       /  \
      60  80
```

Delete:

```
TEXT
50
```

At 50:

TEXT
two children

Find:

TEXT
minimum of right subtree

which is:

TEXT
60

35. REPLACEMENT STEP

C
root->data = temp->data;

Now root key:

TEXT
60

Then:

C
root->right =
deleteNode(root->right, 60);

This searches right subtree:

TEXT
7 0 ! ' l e f t ! ' 6 0

and removes original successor.

36. AFTER

TEXT

```
graph TD; 60 --> 30; 60 --> 70; 70 --> 80;
```

BST property preserved.

37. TWO-CHILD DELETION WITH SUCCESSOR HAVING CHILD

Important case.

Before:

TEXT



Delete 50.

Successor:

TEXT

60

But 60 has right child:

TEXT

70

That's okay.

38. STEP

Copy 60 into root:

TEXT



Now delete original 60.

Original 60 has one child:

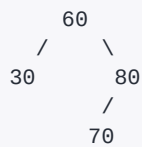
TEXT

70

So replace it by 70.

Final:

TEXT



$\emptyset \neq \gamma$ This proves successor need not be a leaf; it may have a right child.

39. SUCCESSOR CANNOT HAVE LEFT CHILD

Because successor is:

TEXT

minimum of right subtree

If it had a left child L:

TEXT

$L < \text{successor}$

Then successor wasn't minimum.

Contradiction.

So:

TEXT

`successor.left = NULL`

always.

40. PREDECESSOR CANNOT HAVE RIGHT CHILD

Similarly:

TEXT

`predecessor = maximum of left subtree`

Therefore:

TEXT

`predecessor.right = NULL`

It may have a left child.

41. DELETE ROOT — LEAF

Tree:

TEXT

50

Delete 50.

Result:

TEXT

EMPTY

Pointer:

TEXT

root = NULL

So call must be:

C

```
root = deleteNode(root, 50);
```

42. DELETE ROOT — ONE CHILD

Before:

TEXT

```
50
 \
  70
```

Delete 50.

Result:

TEXT

70

New root:

TEXT

70

Again shows why deletion returns subtree root.

43. DELETE ROOT — TWO CHILDREN

Before:

TEXT

```
50
 / \
```

30 70

Using successor:

TEXT

70

because right subtree only contains 70.

Final:

TEXT

70
/
30

Using predecessor instead:

TEXT

30
\
70

Both valid.

Final shape depends on chosen replacement method.

44. DELETION COMPLEXITY

We first search down one path:

TEXT

$O(h)$

For two-child case we additionally search for successor/predecessor inside a subtree.

That also traverses at most:

TEXT

$O(h)$

So total:

FORMULA BOX

TEXT

Delete = $O(h)$

not:

TEXT

$O(h^2)$

because:

TEXT

$O(h) + O(h) = O(h)$

45. BALANCED BST

If:

TEXT

$h = O(\log n)$

then:

TEXT

Delete = $O(\log n)$

46. SKEWED BST

If:

TEXT

$h = O(n)$

then:

TEXT

Delete = $O(n)$

~~Ordinary~~ Binary BST does not guarantee logarithmic deletion.

47. RECURSIVE SPACE

Recursive deletion:

TEXT

$O(h)$

auxiliary call-stack space.

Balanced:

TEXT

$O(\log n)$

Skewed:

TEXT

$O(n)$

48. DOES DELETION PRESERVE INORDER SORTING?

Yes.

If deletion is done correctly, remaining BST inorder is still sorted.

Before:

TEXT

20 30 40 50 60 70 80

Delete 50:

TEXT

20 30 40 60 70 80

Still sorted.

49. EASY GATE SHORTCUT

If question only asks:

What is inorder sequence after deleting key X from BST?

and no duplicate complications:

Simply remove X from sorted inorder sequence.

You may not need to reconstruct exact post-deletion shape.

Example:

Before inorder:

TEXT

10 20 30 40 50

Delete 30:

TEXT

10 20 40 50

Shape irrelevant for inorder output.

50. BUT PREORDER/POSTORDER DEPEND ON DELETION METHOD

Suppose:

```
TEXT
  20
 /  \
10   30
```

Delete 20.

Successor method:

```
TEXT
  30
 /
10
```

Preorder:

```
TEXT
30 10
```

Predecessor method:

```
TEXT
10
 \
 30
```

Preorder:

```
TEXT
10 30
```

So if a question asks exact resulting tree or preorder:

Check which deletion convention is specified.

51. COMMON ERROR — DELETE TWO-CHILD NODE BY JUST USING RIGHT CHILD

Wrong:

```
TEXT
parent != root -> right
```

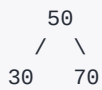
when root has two children.

Why?

Left subtree can become disconnected/lost.

Example:

TEXT



Replacing 50 directly with 70 without carefully attaching 30 may still be possible with more rewiring, but the simple one-child rule does not directly apply.

Use successor/predecessor method.

52. COMMON ERROR — FREE BEFORE SAVING CHILD

Wrong:

```
C
free(root);
return root->left;
```

After `free(root)`:

TEXT

```
root->left
```

must not be accessed.

That's use-after-free / undefined behavior.

Correct:

```
C
Node *temp = root->left;
free(root);
return temp;
```

✓ 100% memory safety.

53. COMMON ERROR — FORGETTING RETURN ASSIGNMENT

Wrong:

```
C
deleteNode(root->left, key);
```

Correct:

```
C
root->left = deleteNode(root->left, key);
```

Why?

Deletion may change the root of that subtree.
Same fundamental pointer principle as insertion.

54. EXAMPLE

Before:

```
TEXT
    50
   /
  30
 /
20
```

Delete 30.

30 has one child 20.

Deletion returns:

```
TEXT
20
```

If parent doesn't assign:

```
c
root->left = ...
```

50's left pointer may still point to freed 30.

0 = dangerous dangling pointer.

55. CASE IDENTIFICATION

When key found:

```
TEXT
left NULL, right NULL
!' l e a f
```

```
TEXT
exactly one non-NULL
!' o n e   c h i l d
```

```
TEXT
both non-NULL
!' t w o   c h i l d r e n
```

Simple classification.

56. MEMORY TRICK

0 CHILD

Cut it.

1 CHILD

Promote the child.

2 CHILDREN

Swap/copy with inorder neighbor, then delete simpler node.

57. GATE-STYLE DELETION 1

BST:

TEXT

```
      40
     /  \
    20   60
   /  \
  10   30
```

Delete 10.

10 is:

TEXT

leaf

Result:

TEXT

```
      40
     /  \
    20   60
     \
      30
```

58. GATE-STYLE DELETION 2

TEXT

```
      40
     /  \
```

```
20 60
  /
 50
```

Delete 60.

60 has:

TEXT
one left child

Promote 50:

TEXT
40
/ \
20 50

59. GATE-STYLE DELETION 3

TEXT
40
/ \
20 60
 / \
 50 70

Delete 60 using successor.

Successor:

TEXT
70

Final:

TEXT
40
/ \
20 70
 /
 50

60. DELETION 4 — SUCCESSOR DEEPER

TEXT
40
/ \
20 80
 / \
 60 90

/
50

Delete 40.

Successor = minimum right:

TEXT

50

not 80 and not 60.

Replace root by 50.

Delete original 50.

Final:

TEXT

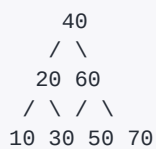


61. DELETION SEQUENCE MATTERS

Like insertion, deletion order can change the tree shape.

Starting tree:

TEXT



Delete 20 then 60 may create a different intermediate structure than deleting 60 then 20 depending on successor/predecessor convention.

So in sequence questions:

Perform deletions one by one on the updated tree.

62. DON'T DELETE FROM ORIGINAL TREE EACH TIME

If sequence is:

TEXT

delete 20
delete 30
delete 60

the second operation acts on:

TEXT

tree after deleting 20

not original tree.

0-5% simple but common exam mistake.

63. BST SUCCESSOR METHOD AND INORDER

Take inorder:

TEXT

10 20 30 40 50 60 70

Delete 40.

Its inorder successor is simply:

TEXT

next value in sorted inorder = 50

So successor concept is easy to visualize.

64. PREDECESSOR

Same inorder:

TEXT

10 20 30 40 50 60 70

Predecessor of 40:

TEXT

30

Exactly previous value.

65. WHY SUCCESSOR REPLACEMENT PRESERVES LEFT SIDE

Suppose deleting X.

Successor S is:

TEXT

smallest value greater than X

All values in X's left subtree:

TEXT

< X < S

So they remain valid under S.

For right subtree, S is the minimum of it, so all remaining right-subtree values are:

TEXT

> S

after removing original S.

Thus BST property remains intact.

Ø=0% Conceptual proof.

66. WHY PREDECESSOR ALSO WORKS

Predecessor P is:

TEXT

largest value smaller than X

All left subtree values other than P are:

TEXT

< P

All right subtree values:

TEXT

> X > P

So P can safely replace X.

67. ITERATIVE DELETION

It can be implemented iteratively by maintaining:

TEXT

curr
parent

and doing explicit rewiring.

But because two-child deletion is pointer-heavy, recursive code is usually easier to understand and less error-prone.

Ø=5% For placements, know the concept even if recursive implementation is preferred.

68. ROOT REPLACEMENT REASONING

Every recursive call:

```
c
root->left = deleteNode(root->left, key);
```

means:

"Delete from my left subtree and tell me what its new root is."

Similarly:

```
c
root->right = deleteNode(root->right, key);
```

This is one of the most important recursive-tree patterns.

69. GENERAL TREE FUNCTION PATTERN

You'll see this repeatedly:

```
c
root->left = operation(root->left, ...);
root->right = operation(root->right, ...);
return root;
```

Why?

Because an operation may modify the subtree root.

Insertion and deletion are classic examples.

70. GATE TRAP — SUCCESSOR OF MAXIMUM

Maximum node has:

```
TEXT
no greater key
```

So it has no inorder successor inside that BST.

Similarly, minimum has no predecessor.

We'll cover this properly next lesson.

71. GATE TRAP — SUCCESSOR MAY BE ANCESTOR

For arbitrary BST node, successor is not always found inside its right subtree.

If the node has no right subtree, successor may be an ancestor.

But in **two-child deletion**, the deleted node definitely has a right subtree.

Therefore successor is safely:

TEXT
minimum of right subtree

Ø=important distinction.

72. DELETION CASE TABLE

| Deleted node | Action |
|------------------|--|
| 0 children | Free it, return NULL |
| Only right child | Free node, return right child |
| Only left child | Free node, return left child |
| 2 children | Copy successor/predecessor, then delete replacement node |

73. COMPLEXITY TABLE

| BST Shape | Delete Time | Recursive Space |
|-----------|-------------|-----------------|
| Balanced | $O(\log n)$ | $O(\log n)$ |
| Skewed | $O(n)$ | $O(n)$ |
| General | $O(h)$ | $O(h)$ |

74. GATE TRUE/FALSE

Statement 1

Deleting a leaf can change its parent's pointer to NULL.

' True.

Statement 2

A node with one child can be replaced directly by that child.

' True.

Statement 3

For a two-child node, its inorder successor is the largest value in its right subtree.

'L False.

It is the:

TEXT

smallest value in right subtree

75. STATEMENT4

The inorder predecessor of a node is the minimum key in its left subtree.

'LFalse.

It is:

TEXT

maximum key in left subtree

76. STATEMENT5

An inorder successor chosen from the right subtree may have a left child.

'LFalse.

It may have a right child, but cannot have a left child.

77. STATEMENT6

Deletion in an ordinary BST is always $O(\log n)$.

'LFalse.

Correct:

TEXT

$O(h)$

Worst case:

TEXT

$O(n)$

78. STATEMENT7

Deleting a two-child node using successor and predecessor must produce the same final tree.

'LFalse.

Both preserve BST validity but can produce different shapes.

79. NOTEBOOK NOTES

BST DELETE

TEXT

Search for key.

If found:

TEXT

```
0 child:
    delete node

1 child:
    replace node by child

2 children:
    successor = min(right)
    OR predecessor = max(left)

    copy replacement value
    delete original replacement node
```

80. CORE C TEMPLATE

C

```
Node *deleteNode(Node *root, int key)
{
    if (root == NULL)
        return NULL;

    if (key < root->data)
        root->left = deleteNode(root->left, key);

    else if (key > root->data)
        root->right = deleteNode(root->right, key);

    else
    {
        if (root->left == NULL)
        {
            Node *temp = root->right;
            free(root);
            return temp;
        }

        if (root->right == NULL)
        {
            Node *temp = root->left;
            free(root);
            return temp;
        }

        Node *temp = findMin(root->right);

        root->data = temp->data;

        root->right =
            deleteNode(root->right, temp->data);
    }
}
```

```
    return root;
}
```

Copy and understand this.

81. MEMORY TRICK

BST DELETE = 0-1-2 RULE

TEXT

```
0 !' N U L L
1 !' C H I L D
2 !' N E I G H B O R
```

Where neighbor means:

TEXT

successor or predecessor

82. 30-SECOND REVISION

Remember:

1. Deletion first searches for the key.
2. Leaf !' remove and return NULL.
3. One child !' return that child.
4. Two children !' use successor or predecessor.
5. Successor = minimum of right subtree.
6. Predecessor = maximum of left subtree.
7. Successor has no left child.
8. `root = deleteNode(root, key)` because root may change.
9. Deletion time = $O(h)$.
10. Balanced $O(\log n)$, skewed $O(n)$.

83. ACTIVE RECALL

Q1

What are the three deletion cases?

Q2

What replaces a one-child node?

Q3

What is the inorder successor used in two-child deletion?

Q4

Why is successor deletion easier afterward?

Q5

What is deletion complexity?

ANSWERS

A1

TEXT

0 child, 1 child, 2 children

A2

Its only child.

A3

Minimum key in the right subtree.

A4

It has no left child, so it has at most one child.

A5

TEXT

$O(h)$

84. PRACTICE — COMPLETE DELETION TRACE

Start:

TEXT



Perform, in order:

TEXT

delete 20
delete 60
delete 50

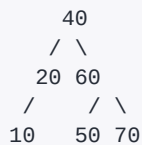
Use the **inorder-successor method** whenever two children exist.

After every deletion, draw the updated tree.

85. PRACTICE — IDENTIFY CASE

For each deletion in:

TEXT



identify whether it is initially:

- Case 0
- Case 1
- Case 2

for deleting:

TEXT

10
20
60

86. PRACTICE — FIND REPLACEMENT

For:

TEXT



If deleting 50:

Using successor:

Find replacement.

Using predecessor:

Find replacement.

Then draw both possible resulting trees.

87. BST MODULE PROGRESS

' Lesson 59 — BST Fundamentals ' Lesson 60 — Search & Insertion ' **Lesson 61 — BST Deletion**

Remaining:

- Lesson 62 — Minimum, Maximum, Predecessor & Successor
- Lesson 63 — BST Complexity & GATE Properties
- Lesson 64 — Placement BST Patterns

NEXT — LESSON 62

BST Minimum, Maximum, Inorder Predecessor & Successor

This next lesson will completely resolve:

TEXT

minimum
maximum
successor
predecessor

including both cases:

- node has relevant subtree
- node has no relevant subtree, so answer comes from an ancestor

with exact examples, C logic, and GATE traps.

USER

next

ASSISTANT

MODULE 7 — LESSON 62

BST Minimum, Maximum, Inorder Predecessor & Successor

This lesson is extremely important because **successor and predecessor** appear directly in BST deletion and GATE theory questions.

We will cover both situations:

1. Required subtree exists.
2. Required subtree does not exist, so the answer comes from an ancestor.

GATE importance: Ø=Ý%ØMÚSDKÍØW **Placement importance:** Ø=Ý%ØMÚSDKÍØW

1. WHERE WE ARE

Completed:

' BST definition ' Global BST property ' Search ' Insertion ' Deletion

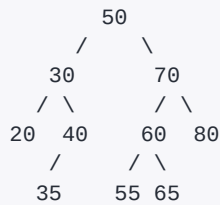
Now:

Minimum, Maximum, Predecessor, Successor

2. EXAMPLE BST

Use this tree:

TEXT



Its inorder traversal is:

TEXT

20 30 35 40 50 55 60 65 70 80

Because inorder of BST gives sorted order.

This sorted sequence is the easiest way to understand predecessor and successor.

3. MINIMUM IN BST Ø=Ý%

Minimum key is the:

Leftmost node

Why?

BST rule:

TEXT

s m a l l e r v a l u e s ! ' l e f t

Therefore repeatedly move:

TEXT

L E F T ! ' L E F T ! ' L E F T . . .

until:

TEXT

left == NULL

4. EXAMPLE — MINIMUM

From root 50:

TEXT

```
50
!" l e f t
30
!" l e f t
20
```

20 has no left child.

Therefore:

Minimum = 20

5. C CODE — MINIMUM

```
C
Node *findMin(Node *root)
{
    if (root == NULL)
        return NULL;

    while (root->left != NULL)
        root = root->left;

    return root;
}
```

6. MINIMUM COMPLEXITY

We follow one downward path.

Therefore:

TEXT

Time = $O(h)$

Balanced:

TEXT

$O(\log n)$

Skewed:

TEXT

$O(n)$

Iterative extra space:

TEXT

$O(1)$

7. IMPORTANT TRAP — MINIMUM NEED NOT BE A LEAF

Consider:

```
TEXT
  30
 /
10
 \
 20
```

Minimum is:

```
TEXT
10
```

But 10 has right child 20.

So:

Minimum node has no left child, but may have a right child.

Ø=Important.

8. MAXIMUM IN BST Ø=Ý%

Maximum key is the:

Rightmost node

Move repeatedly:

```
TEXT
R I G H T ! '   R I G H T ! '   R I G H T . . .
```

until:

```
TEXT
right == NULL
```

9. EXAMPLE — MAXIMUM

From:

```
TEXT
5 0 ! '   7 0 ! '   8 0
```

80 has no right child.

Therefore:

Maximum = 80

10. C CODE — MAXIMUM

```
c
Node *findMax(Node *root)
{
    if (root == NULL)
        return NULL;

    while (root->right != NULL)
        root = root->right;

    return root;
}
```

Complexity:

TEXT
 $O(h)$

11. MAXIMUM NEED NOT BE A LEAF

Example:

TEXT

```
30
 \
  50
 /
40
```

Maximum:

TEXT
50

but 50 has a left child.

So:

Maximum has no right child, but may have a left child.

12. INORDER PREDECESSOR — DEFINITION Ø=Ý%Ø=Ý%Ø=Ý%

For node X:

Inorder predecessor = node immediately before X in inorder traversal

Since inorder of BST is sorted:

predecessor = immediately smaller key

Example inorder:

TEXT

20 30 35 40 50 55 60 65 70 80

Predecessor of:

TEXT

50

is:

TEXT

40

Predecessor of:

TEXT

60

is:

TEXT

55

13. INORDER SUCCESSOR — DEFINITION Ø=Ý%Ø=Ý%Ø=Ý%

Inorder successor = node immediately after X in inorder traversal

For BST:

successor = immediately larger key

Example:

TEXT

20 30 35 40 50 55 60 65 70 80

Successor of 50:

TEXT

55

Successor of 60:

```
TEXT
65
```

14. MEMORY TRICK

Think of sorted inorder:

```
TEXT
. . . predecessor !• X !' successor . . .
```

So:

```
TEXT
Predecessor = previous
Successor   = next
```

Very easy.

15. SUCCESSOR — TWO CASES

To find successor of node X , there are two major cases.

Case 1

X has a right subtree.

Case 2

X has no right subtree.

These require different logic.

16. SUCCESSOR CASE 1 — RIGHT SUBTREE EXISTS

If X has a right subtree:

Successor = minimum node in X 's right subtree

Why?

Successor must be:

```
TEXT
> X
```

The right subtree contains larger values.

We want the **smallest** among them.

Therefore:

TEXT

successor = leftmost node of right subtree

17. EXAMPLE — SUCCESSOR OF 50

Tree:

TEXT



50 has right subtree.

Go:

TEXT

right once !' 7 0
then left as far as possible
7 0 !' 6 0 !' 5 5

Therefore:

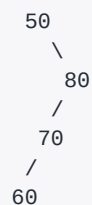
Successor(50) = 55

Not 70.

18. SUCCESSOR IS NOT ALWAYS RIGHT CHILD

This is a huge trap.

TEXT



Successor of 50:

TEXT

60

not:

TEXT

80

Rule:

Right once, then all the way left.

19. SUCCESSOR CASE 2 — NO RIGHT SUBTREE $\emptyset = Y\% \emptyset = Y\% \emptyset = Y\%$

Consider:

TEXT



Find successor of 40.

40 has:

TEXT

no right subtree

So Case 1 cannot be used.

Look at inorder:

TEXT

30 40 50 70

Successor is:

TEXT

50

But 50 is an:

ancestor

20. ANCESTOR LOGIC FOR SUCCESSOR

When X has no right subtree:

Move upward until you find the first ancestor for which X lies in that ancestor's left subtree.

That ancestor is the successor.

For 40:

TEXT

40
!' p a r e n t 3 0

40 is right child/subtree of 30.

So 30 is smaller, not successor.

Continue upward:

TEXT

30
!' 5 0

40 lies inside left subtree of 50.

Therefore:

Successor = 50

21. SUCCESSOR MEMORY RULE

Successor:

Right exists:

Right once !' then all left.

No right:

Go up until you come from the left.

Ø=Memorize exactly.

22. ANOTHER SUCCESSOR EXAMPLE

Tree:

TEXT

```
      50
     /
    30
     \
     40
     /
    35
```

Find successor of 35.

35 has no right subtree.

Go upward.

35 is left child of 40.

Therefore first ancestor reached from left:

TEXT

40

So:

Successor(35) = 40

23. SUCCESSOR OF 40

Same tree:

TEXT

```
      50
     /
    30
     \
     40
```

40 has no right.

Move up:

40 is right child of 30 !' continue.

30 is left child of 50 !' stop.

Therefore:

Successor(40) = 50

24. SUCCESSOR OF MAXIMUM

Suppose maximum node:

TEXT

80

Since maximum has no greater key:

Successor does not exist

Return conceptually:

TEXT

NULL

∅ = Null. Maximum has no inorder successor.

25. PREDECESSOR — TWO CASES

Symmetric logic.

Case 1

X has a left subtree.

Case 2

X has no left subtree.

26. PREDECESSOR CASE 1 — LEFT SUBTREE EXISTS $\emptyset \neq Y\%$

If X has a left subtree:

Predecessor = maximum node in left subtree

Because left subtree contains keys:

TEXT

< X

and predecessor must be the:

TEXT

largest among them

So:

Left once !' then all right

27. EXAMPLE — PREDECESSOR OF 50

Tree:

TEXT

```
      50
     /
    30
     \
     40
```

50 has left subtree.

Maximum in left subtree:

TEXT

40

Therefore:

Predecessor(50) = 40

28. PREDECESSOR NOT ALWAYS LEFT CHILD

Example:

TEXT



Predecessor of 50:

TEXT

40

not 20.

Rule:

Left once !' then all right

29. PREDECESSOR CASE 2 — NO LEFT SUBTREE Ø=Ý%Ø=Ý%Ø=Ý%

Example:

TEXT



Find predecessor of 60.

60 has no left subtree.

Look at inorder:

TEXT

30 50 60 70

Predecessor:

TEXT

50

50 is an ancestor.

30. ANCESTOR LOGIC FOR PREDECESSOR

When X has no left subtree:

Move upward until you find the first ancestor for which X lies in that ancestor's right subtree.

That ancestor is predecessor.

For 60:

TEXT

60
! ' 7 0

60 lies in left subtree of 70.

70 is greater, so continue.

TEXT

70
! ' 5 0

60 lies in right subtree of 50.

Therefore:

Predecessor = 50

31. PREDECESSOR MEMORY RULE

Predecessor:

Left exists:

Left once !' then all right.

No left:

Go up until you come from the right.

Ø = Exact mirror of successor.

32. MASTER TABLE

| Operation | If relevant subtree exists | Otherwise |
|-----------|----------------------------|---|
| Successor | Min of right subtree | Lowest ancestor where node lies in left subtree |

| Operation | If relevant subtree exists | Otherwise |
|-------------|----------------------------|--|
| Predecessor | Max of left subtree | Lowest ancestor where node lies in right subtree |

Copy this table.

33. SUCCESSOR/PREDECESSOR USING PARENT POINTERS

If node structure contains:

```
c
struct Node
{
    int data;
    struct Node *left;
    struct Node *right;
    struct Node *parent;
};
```

then ancestor case can simply move upward using:

```
TEXT
parent
```

34. SUCCESSOR WITH PARENT POINTER — LOGIC

Suppose node `x`.

If:

```
TEXT
x->right != NULL
```

return:

```
TEXT
findMin(x->right)
```

Otherwise:

```
TEXT
p = x->parent

while p != NULL AND x == p->right:
    x = p
    p = p->parent

return p
```

35. WHY LOOP WHILE WE COME FROM RIGHT?

If X is in an ancestor's right subtree, that ancestor is smaller than X.

Therefore it cannot be successor.

Keep climbing.

The first time X/subtree is on ancestor's left side:

TEXT

ancestor > X

and it is the nearest such larger ancestor.

That's the successor.

36. C CODE — SUCCESSOR WITH PARENT POINTER

```
C
Node *successor(Node *x)
{
    if (x == NULL)
        return NULL;

    if (x->right != NULL)
        return findMin(x->right);

    Node *p = x->parent;

    while (p != NULL && x == p->right)
    {
        x = p;
        p = p->parent;
    }

    return p;
}
```

37. PREDECESSOR WITH PARENT POINTER

Mirror version:

```
C
Node *predecessor(Node *x)
{
    if (x == NULL)
        return NULL;

    if (x->left != NULL)
        return findMax(x->left);

    Node *p = x->parent;

    while (p != NULL && x == p->left)
    {
        x = p;
        p = p->parent;
    }

    return p;
}
```

```
    return p;
}
```

38. WHAT IF THERE IS NO PARENT POINTER?

Common BST node:

```
c
typedef struct Node
{
    int data;
    struct Node *left;
    struct Node *right;
} Node;
```

Then to find successor/predecessor, we can start from the root and remember the best ancestor candidate.

Ø=Important placement approach.

39. SUCCESSOR FROM ROOT — IDEA

Suppose searching successor of key `x`.

Start:

```
TEXT
succ = NULL
curr = root
```

If:

```
TEXT
x < curr->data
```

then current node is a possible successor because it is larger.

Store:

```
TEXT
succ = curr
```

Then move:

```
TEXT
left
```

to search for a closer larger value.

40. IF X > CURRENT

If:

TEXT

```
x > curr->data
```

current is too small to be successor.

Move:

TEXT

```
right
```

without updating successor.

41. IF X IS FOUND

If node has right subtree:

TEXT

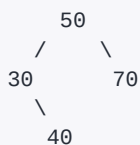
```
successor = minimum(right subtree)
```

Otherwise use the best ancestor candidate accumulated during search.

42. EXAMPLE — SUCCESSOR USING ROOT SEARCH

BST:

TEXT



Find successor of 40.

Start:

TEXT

```
succ=NULL  
curr=50
```

40 < 50:

TEXT

```
succ=50  
go left
```

curr=30.

40 > 30:

TEXT

go right

curr=40.

No right subtree.

Current best candidate:

TEXT

50

Therefore successor:

50

43. SUCCESSOR C CODE WITHOUT PARENT POINTER

```
c
Node *findSuccessor(Node *root, int key)
{
    Node *succ = NULL;
    Node *curr = root;

    while (curr != NULL)
    {
        if (key < curr->data)
        {
            succ = curr;
            curr = curr->left;
        }
        else if (key > curr->data)
        {
            curr = curr->right;
        }
        else
        {
            if (curr->right != NULL)
                return findMin(curr->right);

            return succ;
        }
    }

    return NULL;
}
```

Assumes distinct keys and key should normally exist for "node successor" queries.

44. PREDECESSOR FROM ROOT — IDEA

Maintain:

TEXT

```
pred = NULL
```

If:

TEXT

```
key > curr->data
```

then current is smaller and could be predecessor.

Store:

TEXT

```
pred = curr
```

then go right searching for something larger but still below key.

If:

TEXT

```
key < curr->data
```

current is too large.

Go left.

45. PREDECESSOR C CODE

```
c
Node *findPredecessor(Node *root, int key)
{
    Node *pred = NULL;
    Node *curr = root;

    while (curr != NULL)
    {
        if (key > curr->data)
        {
            pred = curr;
            curr = curr->right;
        }
        else if (key < curr->data)
        {
            curr = curr->left;
        }
        else
        {
            if (curr->left != NULL)
                return findMax(curr->left);

            return pred;
        }
    }

    return NULL;
}
```

46. COMPLEXITY

All these operations traverse at most a root-to-leaf/root-to-ancestor path.

Therefore:

FORMULA BOX

TEXT

Minimum = $O(h)$
Maximum = $O(h)$
Successor = $O(h)$
Predecessor = $O(h)$

Balanced:

TEXT

$O(\log n)$

Skewed:

TEXT

$O(n)$

47. SUCCESSOR IN TWO-CHILD DELETION

Recall Lesson 61.

When deleting node X with two children:

TEXT

successor = minimum(X->right)

Why is the ancestor case irrelevant there?

Because X definitely has:

TEXT

a right subtree

So successor must be found inside it.

Ø = Important distinction.

48. GENERAL SUCCESSOR VS DELETION SUCCESSOR

General successor

May come from:

TEXT

right subtree
OR ancestor

Two-child deletion successor

Always:

TEXT

minimum of right subtree

because right subtree exists.

49. GENERAL PREDECESSOR VS DELETION PREDECESSOR

Same idea.

General predecessor may come from:

TEXT

left subtree
OR ancestor

For deleting a two-child node:

TEXT

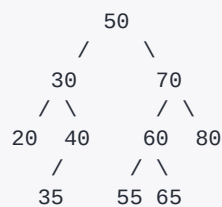
maximum of left subtree

because left subtree definitely exists.

50. FULL EXAMPLE — FIND ALL FOUR

BST:

TEXT



Inorder:

TEXT

20 30 35 40 50 55 60 65 70 80

Find for node 60.

51. MIN/MAX OF WHOLE TREE

Minimum:

TEXT

20

Maximum:

TEXT

80

52. PREDECESSOR OF 60

60 has left subtree:

TEXT

55

Maximum of left subtree:

TEXT

55

So:

Pred(60)=55

53. SUCCESSOR OF 60

60 has right subtree:

TEXT

65

Minimum of right subtree:

TEXT

65

So:

Succ(60)=65

54. FIND FOR NODE 40

40 has left subtree rooted at 35.

Therefore:

TEXT

Pred(40)=35

40 has no right subtree.

So climb/find ancestor candidate.

In inorder:

TEXT

...35 40 50...

Therefore:

TEXT

Succ(40)=50

55. FIND FOR NODE 30

30 has left subtree 20.

Therefore:

TEXT

Pred(30)=20

30 has right subtree rooted at 40, whose minimum is 35.

Therefore:

TEXT

Succ(30)=35

∅ = None
Min successor is not necessarily right child 40.

56. FIND FOR NODE 20

20 has no left subtree.

And it is minimum of tree.

Therefore:

TEXT

Pred(20)=NULL

20 has no right subtree.

Go upward until first ancestor where 20 lies on left.

Parent 30 qualifies.

So:

TEXT

$\text{Succ}(20)=30$

57. FIND FOR NODE 80

80 is maximum.

Therefore:

TEXT

$\text{Succ}(80)=\text{NULL}$

80 has no left subtree in this example.

Climb upward.

80 is right child of 70, so predecessor may be 70.

Indeed:

TEXT

$\text{Pred}(80)=70$

58. GATE TRAP — PREDECESSOR IS NOT ALWAYS PARENT

Example:

TEXT

```
  50
 /
20
 \
  40
 /
 30
```

Predecessor of 50:

TEXT

40

not parent because 50 may have no parent/root.

Predecessor is based on inorder ordering, not family relationship.

59. GATE TRAP — SUCCESSOR IS NOT ALWAYS CHILD

Example:

TEXT



Successor of 40:

TEXT

50

which is grandparent/ancestor.

So:

Successor/predecessor describe traversal order, not direct structural relation.

60. GATE TRAP — SUCCESSOR OF NODE WITH RIGHT SUBTREE

If right subtree exists, do **not** climb ancestors first.

Immediately:

TEXT

minimum of right subtree

That will always be closer in sorted order than any ancestor larger than the node.

61. WHY?

Suppose X has right subtree.

Every right-subtree key:

TEXT

> X

The smallest among them is the first larger key.

Any larger ancestor cannot come before that smaller right-subtree value in inorder.

Thus successor must be:

TEXT

min(right)

62. SYMMETRIC PREDECESSOR PROOF

If X has left subtree:

Every left-subtree key:

TEXT
< X

Largest among them is immediately before X.

Therefore:

TEXT
pred = max(left)

63. GATE-STYLE QUESTION 1

BST inorder:

TEXT
10 20 25 30 40 50

Predecessor of 30:

TEXT
25

Successor:

TEXT
40

No tree drawing needed if inorder is explicitly given.

Ø = \$ shortcut.

64. GATE-STYLE QUESTION 2

In BST, node X has a non-empty right subtree.

Its inorder successor is:

A. Root of right subtree always B. Maximum of right subtree C. Minimum of right subtree D. Parent always

Answer:

C — Minimum of right subtree

65. GATE-STYLE QUESTION 3

Node X has a non-empty left subtree.

Its inorder predecessor is:

Maximum of left subtree

66. GATE-STYLE QUESTION 4

What is successor of maximum element?

Does not exist / NULL

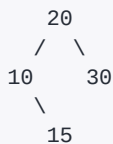
What is predecessor of minimum?

Does not exist / NULL

67. GATE-STYLE QUESTION 5

BST:

TEXT



Successor of 15?

15 has no right subtree.

Go up:

15 is right of 10 !' continue.

10 is left of 20 !' stop.

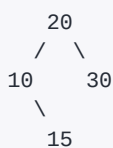
Therefore:

20

68. GATE-STYLE QUESTION 6

Same tree:

TEXT



Predecessor of 15?

15 has no left subtree.

Go upward.

15 is right child of 10.

Therefore:

10

69. GATE-STYLE QUESTION 7

TEXT



Find successor of 40.

40 no right.

40 is left child of 45.

Therefore:

45

Find predecessor of 40.

No left.

40 is left of 45 !' climb.

45 is right of 30.

Therefore:

30

Ø = Excellent ancestor-case example.

70. ANCESTOR RULE IN RANGE LANGUAGE

Suppose searching successor for key 40.

During root-to-node search, every time we move:

TEXT

LEFT from a larger node

that larger node becomes a successor candidate.

We keep replacing candidate with a closer one.

Similarly for predecessor:

Every time we move:

TEXT

RIGHT from a smaller node

that node becomes a predecessor candidate.

This is why root-search implementation works.

71. SUCCESSOR CANDIDATE EXAMPLE

BST:

TEXT



Find successor of 70.

Search:

At 100:

TEXT

```
70 < 100
succ = 100
go left
```

At 50:

TEXT

```
70 > 50
go right
```

At 80:

TEXT

```
70 < 80
succ = 80
go left
```

Reach 70.

No right subtree.

Latest candidate:

TEXT

80

Therefore successor:

72. PREDECESSOR CANDIDATE EXAMPLE

Same tree.

Find predecessor of 70.

At 100:

TEXT

```
70 < 100
go left
```

At 50:

TEXT

```
70 > 50
pred=50
go right
```

At 80:

TEXT

```
70 < 80
go left
```

At 70.

No left subtree.

Candidate:

TEXT

```
50
```

Therefore predecessor:

73. KTH SMALLEST CONNECTION Ø=PE

Because BST inorder is sorted:

TEXT

```
1 s t   v i s i t e d   ! '   m i n i m u m
2 n d   v i s i t e d   ! '   s e c o n d   s m a l l e s t
...
k t h   v i s i t e d   ! '   k t h   s m a l l e s t
```

Example inorder:

TEXT

10 20 30 40 50

3rd smallest:

TEXT

30

This is a direct extension of successor ordering.

74. KTH LARGEST

Reverse inorder:

TEXT

R I G H T ! ' R O O T ! ' L E F T

visits:

TEXT

l a r g e s t ! ' s m a l l e r ! ' s m a l l e r . . .

So kth visited gives kth largest.

Ø=Placement pattern.

75. SUCCESSIVE SUCCESSORS

If you repeatedly compute successor starting from minimum:

TEXT

minimum
!' s u c c e s s o r
!' s u c c e s s o r
!' . . .

you generate the entire BST in sorted ascending order.

Conceptually equivalent to inorder traversal.

76. SUCCESSIVE PREDECESSORS

Start from maximum and repeatedly take predecessor:

TEXT

maximum
!' p r e d e c e s s o r
!' . . .

gives descending order.

77. COMPLEXITY OF REPEATED SUCCESSOR — NUANCE $\emptyset = \neq$

Calling a fresh $O(h)$ successor search n times could give up to:

TEXT
 $O(nh)$

But an ordinary inorder traversal generates all keys in:

TEXT
 $O(n)$

time.

So for listing all sorted keys:

Use inorder traversal, not repeated independent root searches.

78. RELATION TO BST DELETION

For node with two children:

TEXT
successor = next inorder key

or:

TEXT
predecessor = previous inorder key

Using one preserves ordering.

This is exactly why these operations were introduced before/within deletion.

79. NOTEBOOK NOTES

Minimum

TEXT
Leftmost node

Maximum

TEXT
Rightmost node

Successor

TEXT

If right exists:
 `min(right)`

Else:
 first ancestor where node lies in left subtree

Predecessor

TEXT

If left exists:
 `max(left)`

Else:
 first ancestor where node lies in right subtree

80. MASTER MEMORY TRICK

SUCCESSOR

R-L-L-L

Meaning:

TEXT

Right once, then Left all the way

if right subtree exists.

Otherwise:

Climb until you come from LEFT.

81. PREDECESSOR

L-R-R-R

Meaning:

TEXT

Left once, then Right all the way

if left subtree exists.

Otherwise:

Climb until you come from RIGHT.

0-1% is the easiest exam memory method.

82. COMPLEXITY SUMMARY

| Operation | General | Balanced | Skewed |
|-------------|---------|-------------|--------|
| Minimum | $O(h)$ | $O(\log n)$ | $O(n)$ |
| Maximum | $O(h)$ | $O(\log n)$ | $O(n)$ |
| Successor | $O(h)$ | $O(\log n)$ | $O(n)$ |
| Predecessor | $O(h)$ | $O(\log n)$ | $O(n)$ |

83. DON'T CONFUSE

Minimum

TEXT

leftmost of whole BST

Maximum

TEXT

rightmost of whole BST

Successor

TEXT

next node in inorder

Predecessor

TEXT

previous node in inorder

84. 30-SECOND REVISION

Remember:

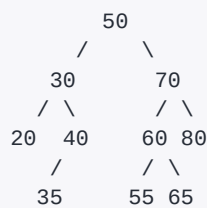
1. Minimum = leftmost.
2. Maximum = rightmost.
3. Successor = next inorder node.
4. Predecessor = previous inorder node.

5. If right exists !' successor = min(right).
6. If no right !' climb until coming from left.
7. If left exists !' predecessor = max(left).
8. If no left !' climb until coming from right.
9. Maximum has no successor.
10. Minimum has no predecessor.

85. ACTIVE RECALL

For:

TEXT



Q1

Minimum?

Q2

Maximum?

Q3

Successor of 30?

Q4

Predecessor of 60?

Q5

Successor of 40?

Q6

Predecessor of 20?

ANSWERS

A1

TEXT

20

A2

TEXT

80

A3

30 has right subtree rooted at 40.

Minimum there is 35.

TEXT

Successor(30)=35

A4

60 has left child/subtree 55.

TEXT

Predecessor(60)=55

A5

40 has no right subtree.

Its next larger ancestor is:

TEXT

50

A6

20 is minimum.

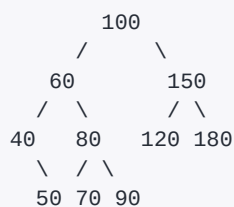
TEXT

NULL

86. PRACTICE — FIND ALL

BST:

TEXT



Find:

1. minimum
2. maximum
3. successor of 50

4. predecessor of 50
5. successor of 90
6. predecessor of 120
7. successor of 180
8. predecessor of 40

87. PRACTICE — ANCESTOR CASES

BST:

TEXT



Find predecessor and successor of:

TEXT

20
30
35
40
50

Try using both:

- inorder sequence
- structural rules

88. BST MODULE PROGRESS

' Lesson 59 — BST Fundamentals ' Lesson 60 — Search & Insertion ' Lesson 61 — BST Deletion ' **Lesson 62 — Min, Max, Predecessor & Successor**

Remaining:

- Lesson 63 — BST Complexity, Height, Structural Properties & GATE Traps
- Lesson 64 — BST Placement Patterns & Advanced Problem Solving

NEXT — LESSON 63

BST Complexity & GATE Properties — Height, Comparisons, Construction Cost, Valid Search Paths & Counting

Next we consolidate the entire BST topic with:

- $O(h)$ master rule

- balanced vs skewed cases
- best/average/worst-case intuition
- sorted insertion worst case
- total BST-construction cost
- number of comparisons
- successful/unsuccessful search paths
- validity of proposed search sequences
- inorder properties
- same-key-set/different-shape questions
- BST counting with Catalan numbers
- distinct BSTs from n ordered keys
- major GATE traps

This will be the **GATE consolidation lesson for BSTs** before placement extensions.

USER

next

ASSISTANT

MODULE 7 — LESSON 63

BST Complexity & GATE Properties — Height, Comparisons, Construction Cost, Search Paths & Counting

This lesson consolidates the **entire GATE side of Binary Search Trees**.

We now connect:

- height
- search
- insertion
- deletion
- comparisons
- construction cost
- valid/invalid search paths
- inorder properties
- number of possible BSTs
- Catalan numbers

GATE importance: Ø=Ý%ØMÚ%ØKÍØW

1. THE MASTER BST COMPLEXITY RULE

Almost every basic BST operation depends on:

HEIGHT `h`

For an ordinary BST:

TEXT

Search = $O(h)$
Insertion = $O(h)$
Deletion = $O(h)$
Minimum = $O(h)$
Maximum = $O(h)$
Successor = $O(h)$
Predecessor = $O(h)$

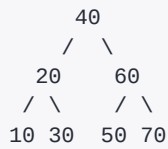
~~$O(n)$~~ If you remember only one complexity rule:

BST speed follows its height.

2. WHY HEIGHT?

Example:

TEXT



Searching 70 follows:

TEXT

40 ! 60 ! 70

It does not visit every node.

It follows one root-to-node path.

So number of comparisons is proportional to path length, hence:

TEXT

$O(h)$

3. BEST, BALANCED & WORST CASE

For n nodes:

Best possible shape

Tree is height-efficient:

TEXT

$$h = \sim (\log n)$$

Operations:

TEXT

$$\sim (\log n)$$

Worst possible shape

Tree becomes skewed:

TEXT

$$h = n-1$$

Operations:

TEXT

$$\sim (n)$$

4. MASTER COMPLEXITY TABLE

| Operation | Best | Balanced / Expected-good shape | Worst |
|-------------------|-------------------------------|--------------------------------|------------|
| Search | $O(1)$ if root | $O(\log n)$ | $O(n)$ |
| Insert | $O(1)$ empty tree | $O(\log n)$ | $O(n)$ |
| Delete | can be $O(1)$ in trivial case | $O(\log n)$ | $O(n)$ |
| Min/Max | $O(1)$ possible | $O(\log n)$ | $O(n)$ |
| Inorder traversal | $\sim (n)$ | $\sim (n)$ | $\sim (n)$ |

~~Yes~~ Inorder traversal always visits every node, so balancing does not reduce traversal from $\sim (n)$

5. IMPORTANT GATE TRAP

Statement:

Search in a BST takes $O(\log n)$.

'L Not always.

Correct general statement:

TEXT

Search = $O(h)$

Only if:

TEXT

$h = O(\log n)$

do we get:

TEXT

$O(\log n)$

6. HEIGHT BOUNDS OF BST

A BST is still a binary tree.

For n nodes:

TEXT

$\text{floor}(\log_2 n) \leq h \leq n - 1$

using edge-based height.

Minimum height occurs in a highly compact tree.

Maximum occurs in a chain.

7. EXAMPLE — $n = 15$

Minimum possible height:

TEXT

$\text{floor}(\log_2 15) = 3$

Maximum:

TEXT

$15 - 1 = 14$

Therefore:

TEXT

$3 \leq h \leq 14$

Search complexity can range accordingly.

8. BALANCED BST EXAMPLE

TEXT



n = 7

Height:

TEXT

h = 2

A deepest search examines:

TEXT

h+1 = 3 nodes

Example:

TEXT

8 !' 1 2 !' 1 4

9. SKEWED BST EXAMPLE

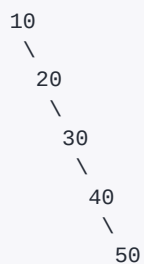
Insert:

TEXT

10, 20, 30, 40, 50

Tree:

TEXT



n = 5

Height:

TEXT

4

Search 50:

TEXT

1 0 !' 2 0 !' 3 0 !' 4 0 !' 5 0

5 node comparisons.

Thus:

TEXT

$\sim (n)$

10. NUMBER OF COMPARISONS

If target node is at depth d :

Successful search generally compares with:

FORMULA BOX

TEXT

$d + 1$ nodes

because:

TEXT

root depth = 0

Example:

Node at depth 3:

TEXT

4 comparisons

11. HEIGHT vs NUMBER OF NODES VISITED

Height counts:

TEXT

edges

A root-to-deepest path of height h contains:

TEXT

$h+1$ nodes

So worst successful search can make approximately:

TEXT

$h+1$ comparisons

Asymptotically:

TEXT

$\sim (h)$

$\emptyset \neq \emptyset$ not confuse edges with nodes.

12. UNSUCCESSFUL SEARCH

Suppose:

TEXT



Search 55:

TEXT

4 0 ! ' 6 0 ! ' 5 0 ! ' N U L L

Actual key comparisons occur with:

TEXT

40, 60, 50

Then one NULL pointer tells us search failed.

Still:

TEXT

$O(h)$

13. SEARCH PATH MUST FOLLOW BST DECISIONS $\emptyset \neq \emptyset \neq \emptyset$

Suppose searching:

TEXT

65

and root is:

TEXT

50

Since:

TEXT
65 > 50

the next node **must** lie in the right subtree.

Therefore a proposed path:

TEXT
50 !' 30 !' . . .

is impossible.

This is a common GATE pattern.

14. RANGE METHOD FOR SEARCH PATHS

Suppose search path:

TEXT
50 !' 20 !' 40

What range must the target satisfy?

At 50 we moved left:

TEXT
target < 50

At 20 we moved right:

TEXT
target > 20

Therefore:

TEXT
20 < target < 50

When examining later nodes, keep narrowing this interval.

~~0-70~~ This is the safest way to solve proposed-search-path questions.

15. EXAMPLE — VALID SEARCH PATH

BST search for:

TEXT
37

Proposed path:

TEXT

5 0 !' 2 0 !' 4 0 !' 3 0 !' 3 5

Check constraints.

At 50:

TEXT

37 < 50

' move left.

At 20:

TEXT

37 > 20

' move right.

At 40:

TEXT

37 < 40

' move left.

At 30:

TEXT

37 > 30

' move right.

At 35:

TEXT

37 > 35

possible.

So this path can be valid.

16. EXAMPLE — INVALID SEARCH PATH

Search for:

TEXT

37

Proposed:

TEXT

5 0 !' 2 0 !' 4 0 !' 4 5

At 40:

TEXT

37 < 40

so search must go left.

But 45 is:

TEXT

>40

and would lie on right side.

Therefore:

Impossible path

17. EVEN STRONGER RANGE CHECK

Suppose path:

TEXT

5 0 !' 2 0 !' 4 0

At 40, valid key range for all nodes there is:

TEXT

20 < key < 50

Suppose next proposed node is:

TEXT

60

Impossible even before knowing target, because 60 violates the subtree's allowable range.

Global BST rule again.

18. INSERTION PATH= UNSUCCESSFUL SEARCH PATH

To insert x :

1. Search for x .
2. Follow comparisons.
3. Reach a NULL pointer.
4. Put new node there.

So:

Insertion is unsuccessful search + attachment

Therefore insertion also costs:

TEXT

$O(h)$

19. FIRST INSERTED KEY PROPERTY

When building an ordinary BST by sequential insertion into an empty tree:

First inserted key becomes root

Example:

TEXT

Insertion sequence:
50, 20, 80, 10, 30

Root:

TEXT

50

Ordinary BST insertion does not rotate automatically.

20. INSERTION ORDER DETERMINES SHAPE

Keys:

TEXT

1, 2, 3, 4, 5, 6, 7

Sequence A:

TEXT

1, 2, 3, 4, 5, 6, 7

produces a chain.

Sequence B:

TEXT

4, 2, 6, 1, 3, 5, 7

produces:

TEXT



Same set.

Completely different performance.

21. INORDER DOES NOT DETERMINE BST SHAPE

Any BST containing distinct keys:

TEXT

1, 2, 3, 4, 5

has inorder:

TEXT

1 2 3 4 5

regardless of shape.

Therefore:

BST inorder determines sorted key order, not tree shape.

Ø=important.

22. PREORDER CAN DETERMINE A BST

For an ordinary arbitrary binary tree:

TEXT

preorder alone

is insufficient.

But for BST with distinct keys:

Preorder alone uniquely determines the BST

because BST ordering tells us where the left block ends and the right block begins.

23. EXAMPLE

BST preorder:

TEXT

40 20 10 30 60 50 70

Root:

TEXT

40

Following values smaller than 40:

TEXT

20 10 30

belong to left subtree.

Then values greater:

TEXT

60 50 70

belong to right subtree.

Repeat recursively.

Tree:

TEXT

```
      40
     /  \
    20   60
   / \  / \
  10 30 50 70
```

24. POSTORDER ALONE ALSO DETERMINES DISTINCT-KEY BST

Postorder:

TEXT

10 30 20 50 70 60 40

Last:

TEXT

40

is root.

Values before root split into:

TEXT

```
< 40  !'   l e f t   s u b t r e e
> 40  !'   r i g h t   s u b t r e e
```

Then recurse.

Special power comes from the BST ordering property.

25. BST INORDER ALONE?

Still not enough.

Example keys:

TEXT

10, 20, 30

BST 1:

TEXT

```
  20
 /  \
10    30
```

BST 2:

TEXT

```
10
 \
  20
   \
    30
```

Both inorder:

TEXT

10 20 30

Different shapes.

26. SORTED INSERTION WORST CASE $\Theta(n^2)$

Insert:

TEXT

1, 2, 3, ..., n

into an ordinary empty BST.

Each value is larger than all previous values.

So tree becomes right-skewed.

The cost of insertions grows like:

TEXT

$$0 + 1 + 2 + \dots + (n-1)$$

27. SUM

Using:

TEXT

$$\begin{aligned} &1+2+\dots+(n-1) \\ &= \\ &n(n-1)/2 \end{aligned}$$

Therefore total construction work:

FORMULA BOX

TEXT

$$\sim (n^2)$$

~~O = E~~ Extremely important.

28. REVERSE SORTED INSERTION

Insert:

TEXT

$$n, n-1, n-2, \dots, 1$$

Tree becomes left-skewed.

Again:

TEXT

$$\sim (n^2)$$

construction time.

29. GOOD SHAPE CONSTRUCTION COST

If tree remains height:

TEXT

$$O(\log n)$$

each insertion:

TEXT

$O(\log n)$

For n insertions:

TEXT

$O(n \log n)$

This is why balanced trees are desirable.

30. ORDINARY BST AVERAGE INTUITION

If insertion order is suitably random rather than deliberately sorted, an ordinary BST has expected height on the order of:

TEXT

$O(\log n)$

and expected search/insertion cost is also:

TEXT

$O(\log n)$

But worst case remains:

TEXT

$O(n)$

~~$O(n)$~~ NOTE distinction:

Expected/average behavior does not remove the worst-case skew possibility.

31. BST SORT

Algorithm:

1. Insert all n elements into BST.
2. Perform inorder traversal.

Inorder costs:

TEXT

$\sim (n)$

Construction:

Balanced/good case:

TEXT

$O(n \log n)$

Worst skewed case:

TEXT

$O(n^2)$

So BST sort:

TEXT

Average/good: $O(n \log n)$

Worst: $O(n^2)$

for an ordinary unbalanced BST.

32. SEARCH OPERATION DOES NOT MODIFY TREE

Search:

TEXT

read-only operation

Shape stays unchanged.

Insertion and deletion modify structure.

Traversal also normally doesn't modify the tree.

Simple, but useful in operation-sequence questions.

33. DELETE COMPLEXITY

Deletion involves:

1. Search for node.
2. Possibly find successor/predecessor.
3. Rewire pointers.

Search:

TEXT

$O(h)$

Successor search:

TEXT

$O(h)$

So:

TEXT

$O(h) + O(h) = O(h)$

Thus:

Deletion = $O(h)$

not $O(h^2)$.

34. MIN/MAX COMPLEXITY

Minimum:

TEXT

keep going left

Maximum:

TEXT

keep going right

So both:

TEXT

$O(h)$

But specific trees can make one of them $O(1)$.

Example:

TEXT

```
10
 \
  20
   \
    30
```

Minimum:

TEXT

10

at root !' $O(1)$.

Maximum:

TEXT

30

requires full chain !' $O(n)$.

35. SUCCESSOR/PREDECESSOR COMPLEXITY

Using root or parent pointers:

TEXT

$O(h)$

general worst case.

For a balanced BST:

TEXT

$O(\log n)$

For skewed:

TEXT

$O(n)$

36. TRAVERSAL COMPLEXITY DOES NOT BECOME LOGARITHMIC

Even a perfect BST with one million nodes:

Inorder must print:

TEXT

one million nodes

Therefore:

TEXT

$\sim (n)$

not $\sim (\log n)$

~~$O = \Theta$~~ Search and traversal are fundamentally different tasks.

37. INORDER VALIDATION PROPERTY

For distinct-key BST:

TEXT

inorder must be strictly increasing

Example:

TEXT

10 20 30 40 50

If inorder of a given labeled tree is:

TEXT

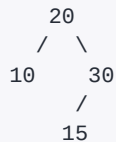
10 30 20 40

it cannot be a valid strict BST.

38. GATE TRAP — LOCAL RULE

Tree:

TEXT



Every direct parent-child comparison:

TEXT

10 < 20
30 > 20
15 < 30

looks fine.

But 15 is in right subtree of 20.

It must satisfy:

TEXT

15 > 20

It does not.

Invalid BST.

39. VALID BST RANGE PROPERTY

Every node has an allowable interval.

Example:

TEXT



X must satisfy:

TEXT

$30 < X < 50$

not merely:

TEXT

$X > 30$

Range constraints are global.

40. ROOT DOES NOT HAVE TO BE MEDIAN

Valid BST:

TEXT

```
10
 \
  20
   \
    30
```

Root:

TEXT

10

Median:

TEXT

20

No rule says root must be median.

Choosing median is a construction strategy for balanced trees, not a BST requirement.

41. MINIMUM AND MAXIMUM ROOT POSSIBILITY

Can root be minimum?

Yes.

Example right-skewed:

TEXT

```
10
 \
  20
   \
    30
```

Root 10 is minimum.

Can root be maximum?

Yes.

TEXT

30
/
20
/
10

Root 30 is maximum.

Root has no fixed rank in an ordinary BST.

42. ROOT RANK AND SUBTREE SIZES

Suppose BST contains keys:

TEXT

1, 2, 3, 4, 5, 6, 7

and root is 4.

Then exactly:

TEXT

3 keys smaller !' left subtree
3 keys larger !' right subtree

If root is 2:

TEXT

1 key smaller !' left
5 keys larger !' right

This property is useful in counting BSTs.

43. NUMBER OF DISTINCT BSTs

Suppose we have:

TEXT

n distinct ordered keys

How many structurally different BSTs can be formed using all of them?

Answer:

Catalan Number

FORMULA BOX

TEXT

$$C_n = 1/(n+1) \times \text{binomial}(2n, n)$$

Equivalent:

TEXT

$$C_n = (2n)! / [n!(n+1)!]$$

Very important counting formula.

44. FIRST CATALAN NUMBERS

For $n = 0, 1, 2, \dots$

TEXT

$C_0 = 1$
 $C_1 = 1$
 $C_2 = 2$
 $C_3 = 5$
 $C_4 = 14$
 $C_5 = 42$
 $C_6 = 132$
 $C_7 = 429$

Memory sequence:

1, 1, 2, 5, 14, 42, 132, 429...

45. $n = 1$

One key:

TEXT

10

Only:

TEXT

1 BST

So:

TEXT

$C_1 = 1$

46. $n = 2$

Keys:

TEXT

10, 20

Possible:

TEXT

10
 \
 20

or:

TEXT

20
 /
 10

So:

TEXT

C2 = 2

47. $n = 3$

Keys:

TEXT

1, 2, 3

Number of distinct BSTs:

TEXT

C3 = 5

Let's derive rather than memorize.

48. CATALAN RECURRENCE $C_n = \sum_{i=0}^{n-1} C_i C_{n-1-i}$

Choose each key as root.

If root has:

TEXT

i keys on left

then:

TEXT

n-1-i keys on right

Number of possibilities for that root:

TEXT

$$C_i \times C_{(n-1-i)}$$

Therefore:

FORMULA BOX

TEXT

$$C_n = \sum_{i=0}^{n-1} C_i \times C_{(n-1-i)}$$

49. DERIVE C3

For 3 keys:

Root = smallest

Left keys:

TEXT

$$0$$

Right:

TEXT

$$2$$

Ways:

TEXT

$$C_0 \times C_2 = 1 \times 2 = 2$$

Root = middle

TEXT

$$C_1 \times C_1 = 1$$

Root = largest

TEXT

$$C_2 \times C_0 = 2$$

Total:

TEXT

$$2+1+2 = 5$$

Therefore:

$$C_3 = 5$$

50. DERIVE C4

TEXT

$$C_4 = C_0C_3 + C_1C_2 + C_2C_1 + C_3C_0$$

TEXT

$$= 1 \times 5 + 1 \times 2 + 2 \times 1 + 5 \times 1$$

TEXT

$$= 14$$

So 4 distinct ordered keys can form:

14 distinct BSTs

51. GATE SHORTCUT

If question asks:

Number of structurally distinct BSTs possible with 5 distinct keys?

Immediately:

TEXT

$$C_5 = 42$$

52. IMPORTANT — KEY VALUES DON'T MATTER

Keys:

TEXT

1, 2, 3, 4

or:

TEXT

10, 50, 100, 900

Both have four distinct ordered keys.

Number of BSTs:

TEXT

C4 = 14

Only relative ordering matters.

53. DISTINCT BSTs VS BINARY TREES

Catalan numbers also count several related ordered tree structures, including structurally distinct binary trees with n nodes under the usual ordered-left/right interpretation.

For BSTs with n distinct ordered keys:

TEXT

Number of BSTs = C_n

because once a shape is chosen, inorder labeling is forced by sorted key order.

Important reasoning.

54. WHY BST LABELING IS FORCED

Suppose BST shape:

TEXT

```
  ?
 / \
?   ?
```

and keys:

TEXT

10, 20, 30

Inorder must be:

TEXT

10 20 30

Therefore:

TEXT

```
  20
 /  \
10    30
```

You cannot arbitrarily permute labels while preserving BST property.

So each BST shape corresponds to one valid labeling by the ordered key set.

55. ARBITRARY LABELED BINARY TREE IS DIFFERENT

If we were counting arbitrary binary trees with distinct labels and no BST ordering, labels could be permuted across a shape.

That is a different counting question.

0-1 ways identify whether problem says:

- binary tree
- BST
- labeled/unlabeled
- ordered/unordered

before applying Catalan formulas.

56. NUMBER OF INSERTION ORDERS "NUMBER OF BSTs

Another major trap.

For n distinct keys:

TEXT

number of insertion permutations = $n!$

But many different insertion orders can produce the same BST.

Therefore:

TEXT

number of BSTs = Catalan C_n

is not equal to:

TEXT

$n!$

57. EXAMPLE

Keys:

TEXT

1, 2, 3

There are:

TEXT

$3! = 6$

insertion sequences.

But distinct BST shapes:

TEXT

C3 = 5

So at least two insertion sequences produce the same tree.

58. WHICH SEQUENCES PRODUCE SAME BST?

Tree:

TEXT

```
  2
 / \
1   3
```

Insertion sequences:

TEXT

2, 1, 3

and:

TEXT

2, 3, 1

both produce the same tree.

Because root must be inserted first, but left/right subtree insertions can interleave.

$\emptyset = \emptyset$ at GATE concept.

59. COUNT INSERTION SEQUENCES FOR A GIVEN BST $\emptyset = \emptyset$

Suppose root has:

TEXT

L nodes in left subtree
R nodes in right subtree

After root is inserted, left-subtree insertion order and right-subtree insertion order may be interleaved while preserving internal relative orders.

If:

TEXT

waysLeft
waysRight

are numbers of sequences producing those subtrees, then:

FORMULA BOX

TEXT

$$\text{Ways}(T) = C(L+R, L) \times \text{Ways}(\text{left}) \times \text{Ways}(\text{right})$$

Here $C(a, b)$ means binomial coefficient.

0=Advanced GATE/placement counting concept.

60. SIMPLE EXAMPLE

Tree:

TEXT

```
  2
 / \
1   3
```

Root 2 must be first.

Then remaining:

TEXT

1 and 3

can appear in either order:

TEXT

2, 1, 3
2, 3, 1

Formula:

TEXT

L=1
R=1

TEXT

$\text{binomial}(2, 1) = 2$

Correct.

61. ANOTHER EXAMPLE

Tree:

TEXT

```
    4
   / \
  2   6
 / \
1   3
```

Left subtree size:

TEXT
3

Right subtree size:

TEXT
1

Root 4 must be first.

Ways to interleave left/right subtree insertions:

TEXT
 $\text{binomial}(4, 3) = 4$

Left subtree rooted at 2 can be produced by:

TEXT
2, 1, 3
or
2, 3, 1

SO:

TEXT
 $\text{ways}(\text{left}) = 2$

Right:

TEXT
 $\text{ways}(\text{right}) = 1$

Total:

TEXT
 $4 \times 2 \times 1 = 8$

Ø=0 Useful advanced counting.

62. ROOT OF BST FROM PREORDER

Preorder:

TEXT
first element = root

Always true for any binary tree.

For BST, additionally:

After root **R**, left-subtree preorder contains keys:

TEXT

<R

before right-subtree block:

TEXT

>R

under distinct keys.

63. VALID BST PREORDER SEQUENCE — BASIC TEST

Suppose:

TEXT

40 20 10 30 60 50 70

Root 40.

Left block:

TEXT

20 10 30

all <40 .

Right block:

TEXT

60 50 70

all >40 .

Then recursively verify each block.

Valid.

64. INVALID PREORDER EXAMPLE

Suppose proposed BST preorder:

TEXT

40 20 10 60 30

Root 40.

After encountering:

TEXT

60 > 40

we have entered root's right-subtree block.

Later:

```
TEXT
30 < 40
```

appears.

That would require returning to root's left subtree after starting its right subtree.

Impossible in preorder.

Therefore:

Invalid BST preorder

Ø=important GATE reasoning.

65. POSTORDER ANALOGUE

BST postorder:

```
TEXT
[left block][right block][root]
```

Once root is known from final element, the earlier sequence must divide consistently into:

```
TEXT
values < root
then
values > root
```

with recursive validity.

66. SEARCH PATHVALIDITY — ANOTHER GATE PATTERN

Suppose a BST search for key `X=25` visits:

```
TEXT
40, 10, 30, 20, 25
```

Check:

At 40:

```
TEXT
25 < 40 !' l e f t
```

10 can be there.

At 10:

TEXT

2 5 > 1 0 !' r i g h t

30 can be there.

At 30:

TEXT

2 5 < 3 0 !' l e f t

20 can be there.

At 20:

TEXT

2 5 > 2 0 !' r i g h t

25 found.

So valid.

67. RANGE VIEW OF SAME PATH

Start:

TEXT

(-" , +")

After 40 left:

TEXT

(-" , 4 0)

After 10 right:

TEXT

(10,40)

After 30 left:

TEXT

(10,30)

After 20 right:

TEXT

(20,30)

25 satisfies:

TEXT

$20 < 25 < 30$

Valid.

$\emptyset = \text{Excellent universal technique.}$

68. NUMBER OF NULL POINTERS IN BST

BST is still a binary tree.

For n nodes with two child-pointer fields each:

TEXT

NULL child pointers = $n+1$

No special BST change.

69. DEGREE RELATION IN BST

For any non-empty BST:

TEXT

$n_0 = n_2 + 1$

where:

TEXT

n_0 = leaves

n_2 = nodes with two children

Again, this is a binary-tree structural identity, independent of ordering.

70. NUMBER OF EDGES

Any BST with n nodes:

TEXT

edges = $n-1$

Simple but sometimes combined with BST questions.

71. LEAST POSSIBLE COMPARISONS

For successful search:

Best case target is root.

Comparisons:

TEXT

1

So:

TEXT

$\sim (1)$

72. GREATEST POSSIBLE COMPARISONS

Worst skewed BST with n nodes.

Search deepest node:

TEXT

n comparisons

So:

TEXT

$\sim (n)$

73. BALANCED TREE COMPARISON COUNT

For perfect BST height h :

Maximum successful search comparisons:

TEXT

$h+1$

And:

TEXT

$n = 2^{(h+1)} - 1$

Thus:

TEXT

$h+1 = \log_2(n+1)$

So deepest successful search uses:

TEXT

$\log_2(n+1)$

node comparisons.

~~$O(n)$~~ is the exact relation for perfect BST.

74. EXAMPLE — PERFECT BST

`n = 31`

TEXT

$31 = 2^5 - 1$

So:

TEXT

$h = 4$

Maximum successful search comparisons:

TEXT

5

75. SEARCH TREE vs BINARY SEARCH ALGORITHM

Do not confuse:

Binary Search

Operates usually on:

TEXT

sorted array

and repeatedly chooses middle index.

Binary Search Tree

Is:

TEXT

dynamic linked tree structure

where shape depends on insertion/deletion.

Both can achieve logarithmic search under favorable structure, but they are not the same algorithm/data structure.

76. SORTED ARRAY VS BALANCED BST

| Operation | Sorted Array | Balanced BST |
|-----------|---------------------|--------------|
| Search | $O(\log n)$ | $O(\log n)$ |
| Insert | $O(n)$ due shifting | $O(\log n)$ |
| Delete | $O(n)$ due shifting | $O(\log n)$ |

| Operation | Sorted Array | Balanced BST |
|------------------|--------------|--------------|
| Sorted traversal | $O(n)$ | $O(n)$ |

This explains the dynamic ordered-set advantage of balanced BSTs.

77. ORDINARY BST DOES NOT GUARANTEE BALANCE

If GATE says merely:

TEXT
BST

do not silently assume AVL/Red-Black behavior.

Worst-case operation:

TEXT
 $O(n)$

unless balance/height information is supplied.

$\emptyset = \emptyset$ critical.

78. WHAT SELF-BALANCING BSTs DO

Structures such as:

- AVL tree
- Red-Black tree

use rotations/rebalancing to keep height:

TEXT
 $O(\log n)$

Therefore operations are guaranteed logarithmic.

$\emptyset = \emptyset$ These are outside the explicit syllabus line you gave, so we do not need to build a separate deep module unless later required for placements.

79. GATE TRUE/FALSE — 1

Inorder traversal of every binary tree is sorted.

'L False.

Only BST ordering gives sorted inorder.

80. GATE TRUE/FALSE — 2

Inorder traversal of a distinct-key BST is strictly increasing.

' True.

81. GATE TRUE/FALSE — 3

An ordinary BST with n nodes always has height $\sim (\log n)$

'LFalse.

Can have:

TEXT

$n-1$

height.

82. GATE TRUE/FALSE — 4

Search in BST always examines one subtree at each decision.

' Under standard key search, yes.

83. GATE TRUE/FALSE — 5

The root of a BST must be the median key.

'LFalse.

84. GATE TRUE/FALSE — 6

First inserted key becomes root of an ordinary BST built from an empty tree.

' True, assuming no balancing rotations.

85. GATE TRUE/FALSE — 7

Sorted insertion order can cause worst-case BST height.

' True.

86. GATE TRUE/FALSE — 8

Number of BSTs possible with n distinct ordered keys is $n!$.

'L False.

It is:

TEXT

Catalan C_n

87. GATE TRUE/FALSE — 9

Different insertion permutations can produce the same BST.

' True.

88. GATE TRUE/FALSE — 10

A BST preorder alone can uniquely determine a BST when keys are distinct.

' True.

The BST ordering property supplies the missing structure.

89. CATALAN FORMULA SHEET Ø=Ý%

FORMULA BOX

TEXT

$C_n = \frac{1}{(n+1)} \times (2n \text{ choose } n)$

or:

TEXT

$C_n = \frac{(2n)!}{(n!(n+1)!)}$

Recurrence:

TEXT

$C_0 = 1$

TEXT

$C_n = \sum_{i=0}^{n-1} C_i C_{n-1-i}$

for:

TEXT

$i = 0 \dots n-1$

90. CATALAN MEMORY VALUES

TEXT

| | | | | | | | | |
|-----|---|---|---|---|----|----|-----|-----|
| n : | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
| Cn: | 1 | 1 | 2 | 5 | 14 | 42 | 132 | 429 |

~~0-10%~~ minimum memorize:

TEXT

1, 2, 5, 14, 42

for $n=1 \dots 5$.

91. SOLVED GATE NUMERICAL

How many BSTs can be made with:

TEXT

$n=4$

distinct keys?

TEXT

$C_4 = 14$

Answer:

14

92. SOLVED GATE NUMERICAL 2

For:

TEXT

$n=5$

TEXT
C5=42

Answer:

42

93. SOLVED GATE NUMERICAL 3

Number of BSTs using 3 distinct keys where the **middle key must be root**?

Middle key leaves:

TEXT
1 smaller key
1 larger key

Left BST possibilities:

TEXT
C1=1

Right:

TEXT
C1=1

Total:

TEXT
 $1 \times 1 = 1$

Only one.

94. ROOT-CONSTRAINED COUNTING Ø=Ý%

Suppose n sorted keys and the root has:

TEXT
L smaller keys
R larger keys

Then number of BSTs with that fixed root is:

FORMULA BOX

TEXT
 $C_L \times C_R$

because left and right subtrees can independently take any valid BST shape.

95. EXAMPLE

Keys:

TEXT

1, 2, 3, 4, 5

Root fixed as:

TEXT

3

Left has:

TEXT

2 keys

Right:

TEXT

2 keys

Ways:

TEXT

$C_2 \times C_2$

$= 2 \times 2$

$= 4$

So 4 BSTs have 3 as root.

$O = \sqrt{N}$ is a very GATE-friendly counting idea.

96. ANOTHER ROOT COUNT

Same 5 keys.

Root fixed as smallest key 1.

Left:

TEXT

0

Right:

TEXT

4

Ways:

TEXT

$C_0 \times C_4$
 $= 1 \times 14$
 $= 14$

So 14 BSTs have minimum key as root.

97. WHY THAT MAKES SENSE

If minimum is root:

TEXT

left subtree = empty

All remaining $n-1$ keys must form any possible BST on the right.

So count:

TEXT

C_{n-1}

Similarly if maximum is root.

98. GATE EXAMPLE — HEIGHT AFTER SORTED INSERTION

Insert:

TEXT

2, 4, 6, 8, 10, 12

into empty ordinary BST.

Result is right-skewed.

$n=6$

Height:

TEXT

5

Worst successful search:

TEXT

6 comparisons

Construction:

TEXT

$\sim (n^2)$

99. GATE EXAMPLE — GOOD INSERTION

Insert:

TEXT

8, 4, 12, 2, 6, 10, 14

Tree:

TEXT



Height:

TEXT

2

Maximum successful search:

TEXT

3 comparisons

100. MASTER GATE FORMULA SHEET

General BST operations

TEXT

Search/Insert/Delete = $O(h)$

Height bounds

TEXT

$\text{floor}(\log_2 n) \leq h \leq n - 1$

Balanced shape

TEXT

$h = O(\log n)$

Worst skew

TEXT

$h = n - 1$

Sorted/reverse-sorted construction

TEXT

$\sim (n^2)$

Inorder

TEXT

sorted ascending

Distinct BST count

TEXT

$C_n = (2n)! / [n!(n+1)!]$

101. MASTER GATE TRAPS

Trap 1

BST search = $O(\log n)$ always.

'L

Use:

TEXT

$O(h)$

Trap 2

Checking only direct children proves BST validity.

'L

Need global subtree/range property.

Trap 3

BST root must be median.

'L

Trap 4

BST must be balanced.

'L

Trap 5

BST must be complete.

'L

Trap 6

Inorder determines exact BST structure.

'L

It only gives sorted key order.

Trap 7

Number of BSTs = $n!$.

'L

TEXT

Catalan Cn

Trap 8

All insertion orders create different BSTs.

'L

Different insertion orders can create the same tree.

102. MEMORY TRICKS

Complexity

Everything follows H.

TEXT

Search / Insert / Delete !' $O(h)$

Worst case

Sorted input makes a stick.

Inorder

BST + LNR = Sorted.

Search path

Every move narrows the range.

Counting

BST shapes = Catalan.

103. 30-SECOND REVISION

Remember:

1. BST operations depend on height.
2. Search/insert/delete = $O(h)$.
3. Balanced != $O(\log n)$.
4. Skewed != $O(n)$.
5. Sorted insertion can make height $n-1$.
6. Building skewed BST from sorted input costs $\sim (n^2)$.
7. Inorder of distinct-key BST is sorted.
8. Preorder alone can reconstruct a distinct-key BST.
9. Number of BSTs from n ordered distinct keys = Catalan C_n .
10. $C_1, C_2, C_3, C_4, C_5 = 1, 2, 5, 14, 42$.

104. ACTIVE RECALL

Q1

General BST search complexity?

Q2

Worst BST height with n nodes?

Q3

Total time to insert sorted n keys into ordinary BST?

Q4

How many distinct BSTs from 4 distinct keys?

Q5

Can different insertion sequences produce the same BST?

Q6

Does inorder alone uniquely determine a BST's shape?

ANSWERS

A1

TEXT

$O(h)$

A2

TEXT

$n-1$

A3

TEXT

$\sim (n^2)$

A4

TEXT

$C4 = 14$

A5

Yes.

A6

No.

105. PRACTICE — GATE MIX

Q1

A BST has 127 nodes and happens to be perfect.

Find:

- height
- maximum successful-search comparisons

Q2

Insert into an empty ordinary BST:

TEXT

5, 10, 15, 20, 25, 30

Find:

- final height
- worst search complexity
- total construction complexity

Q3

How many BSTs can be constructed from 6 distinct ordered keys?

Use:

TEXT

C6

Q4

With keys:

TEXT

1, 2, 3, 4, 5

how many BSTs have key 2 as root?

Hint:

TEXT

Left keys = 1

Right keys = 3

Use:

TEXT

C1 × C3

Q5

Can this be a BST search path while searching for 27?

TEXT

5 0 !' 2 0 !' 4 0 !' 3 0 !' 2 5 !' 2 7

Track the allowable range at every step.

106. BST GATE FOUNDATION — COMPLETE '

We have now covered the core BST material:

' BST definition ' Global ordering rule ' BST validation ' Search ' Recursive & iterative search ' Insertion ' Recursive & iterative insertion ' Pointer updates ' Deletion — all 3 cases ' Inorder successor ' Inorder predecessor ' Minimum / maximum ' Search-path validity ' Height and complexity ' Balanced vs skewed performance ' Sorted insertion worst case ' Construction cost ' BST traversal properties ' Reconstruction properties ' Catalan counting ' Root-constrained BST counting ' GATE traps

BST MODULE STATUS

' Lesson 59 — Fundamentals ' Lesson 60 — Search & Insertion ' Lesson 61 — Deletion ' Lesson 62 —

One BST lesson remains before we mark the whole BST module complete:

Lesson 64 — Placement BST Patterns & Advanced Problem Solving

It will cover practical problems such as:

- validate BST
- kth smallest
- kth largest
- lowest common ancestor in BST
- range search
- floor and ceil
- sorted array != balanced BST
- BST != sorted sequence
- two-sum in BST
- recover/search patterns
- complexity choices
- interview traps

Then we move to:

MODULE 8 — BINARY HEAPS

starting with:

Lesson 65 — Binary Heap Fundamentals: Complete Tree + Heap Property, Min Heap vs Max Heap.

USER

next

ASSISTANT

MODULE 7 — LESSON 64

BST Placement Patterns & Advanced Problem Solving — Validate, Kth Smallest, LCA, Floor/Ceil, Range Search & More

This lesson completes the **placement/interview extension** of Binary Search Trees.

GATE mainly tests BST properties, operations, complexity, and traversal logic. Placements go one step further and ask you to combine those ideas into problem-solving patterns.

GATE importance: +IMPORTANT **Placement importance:** 0=Ý%04Ú%0K10W

1. WHERE WE ARE

Already completed:

' BST definition ' Search ' Insertion ' Deletion ' Min/Max ' Predecessor/Successor ' Complexity ' Search-path questions ' Catalan counting

Now we apply those concepts to common coding/interview problems.

We will cover:

1. Validate BST
2. Kth smallest
3. Kth largest
4. Lowest Common Ancestor
5. Floor
6. Ceil
7. Range search
8. Sorted array != balanced BST
9. BST != sorted array
10. Two Sum in BST
11. Closest value
12. BST iterator intuition
13. Common interview traps

2. PATTERN1 — VALIDATEBST Ø=Ý%Ø=Ý%Ø=Ý%

Problem:

Given a binary tree, determine whether it satisfies BST rules.

Wrong approach:

TEXT

```
left child < parent
right child > parent
```

That checks only local relationships.

We need global constraints.

3. INVALID EXAMPLE

TEXT



Local:

TEXT

```
3 0 < 5 0 '  
7 0 > 5 0 '  
6 0 > 3 0 '
```

But 60 belongs to the left subtree of 50.

Therefore it must be:

TEXT

```
60 < 50
```

False.

So:

Not a BST

4. BEST VALIDATION PATTERN— RANGE METHOD

Each node receives an allowable range.

Initially:

TEXT

```
( -" , +" )
```

For node X:

Left subtree gets:

TEXT

```
(low, X)
```

Right subtree gets:

TEXT

```
(X, high)
```

5. C IMPLEMENTATION

```
C  
int isBST(Node *root, long long low, long long high)  
{  
    if (root == NULL)  
        return 1;  
  
    if (root->data <= low || root->data >= high)  
        return 0;  
  
    return isBST(root->left, low, root->data) &&  
        isBST(root->right, root->data, high);  
}
```

```
}
```

Assuming distinct keys.

Time:

```
TEXT  
O(n)
```

Space:

```
TEXT  
O(h)
```

6. ALTERNATIVE VALIDATION — INORDER

Because BST inorder is sorted, we can check that every current key is greater than the previously visited key.

Concept:

```
TEXT  
previous < current
```

for every inorder step.

If not:

```
TEXT  
invalid BST
```

Time:

```
TEXT  
O(n)
```

Space:

```
TEXT  
O(h)
```

recursive.

7. WHICH VALIDATION METHOD IS BETTER?

Both are standard.

For understanding:

Range method is safest.

It directly represents the BST definition.

For coding:

Both are valid.

8. PATTERN2 — KTH SMALLEST $\emptyset = \text{Y}\% \emptyset = \text{Y}\% \emptyset = \text{Y}\%$

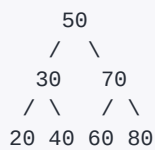
BST inorder gives:

TEXT

ascending sorted order

Example:

TEXT



Inorder:

TEXT

20 30 40 50 60 70 80

3rd smallest:

TEXT

40

So:

Perform inorder traversal and count nodes.

9. KTH SMALLEST RECURSIVE IDEA

Maintain:

TEXT

count

Every time a node is visited in inorder:

TEXT

count++

When:

TEXT

```
count == k
```

that node is answer.

10. SIMPLE C-STYLE LOGIC

```
C
void kthSmallest(Node *root, int k, int *count, int *ans)
{
    if (root == NULL)
        return;

    kthSmallest(root->left, k, count, ans);

    (*count)++;

    if (*count == k)
        *ans = root->data;

    kthSmallest(root->right, k, count, ans);
}
```

Basic version shown for concept.

11. COMPLEXITY

Worst case:

TEXT

$O(n)$

Time.

Recursion:

TEXT

$O(h)$

space.

With subtree-size augmentation, kth smallest can be improved to approximately:

TEXT

$O(h)$

but that requires storing subtree sizes.

~~$O(n)$~~ Advanced extension.

12. PATTERN3 — KTH LARGEST

Use:

Reverse inorder

TEXT

R I G H T ! ' R O O T ! ' L E F T

This generates descending order.

Example:

TEXT

80 70 60 50 40 30 20

2nd largest:

TEXT

70

Memory trick:

****Smallest !' inorder.**

Largest !' reverse inorder.**

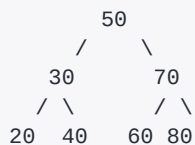
13. PATTERN4 — LOWEST COMMON ANCESTOR IN BST Ø=Ý%Ø=Ý%Ø=Ý%

LCA of two nodes:

Lowest/deepest node that is an ancestor of both.

Example:

TEXT



Find LCA of:

TEXT

20 and 40

Both lie below 30.

Answer:

30

14. BST MAKES LCA EASY

At current node `root` :

If both keys are smaller:

```
TEXT
go LEFT
```

If both are larger:

```
TEXT
go RIGHT
```

Otherwise they split around root, or root equals one key.

Then:

root is LCA

15. EXAMPLE — LCA(20,40)

At 50:

```
TEXT
20 < 50
40 < 50
```

Both left.

Move to 30.

At 30:

```
TEXT
20 < 30
40 > 30
```

They split.

Therefore:

LCA = 30

16. LCA(20,80)

At root 50:

TEXT

20 < 50
80 > 50

Split immediately.

Therefore:

LCA = 50

17. C IMPLEMENTATION

```
C
Node *lca(Node *root, int a, int b)
{
    while (root != NULL)
    {
        if (a < root->data && b < root->data)
            root = root->left;

        else if (a > root->data && b > root->data)
            root = root->right;

        else
            return root;
    }

    return NULL;
}
```

Time:

TEXT

O(h)

Space:

TEXT

O(1)

iterative.

18. IMPORTANT LCA ASSUMPTION

Many versions assume both keys exist in the BST.

If they may be absent, existence may need to be checked separately.

Read the question.

19. PATTERN5 — FLOOR IN BST

For a value x :

Floor = greatest BST key " $\leq x$ "

Example keys:

```
TEXT
10 20 30 40 50
```

Floor of:

```
TEXT
36
```

is:

30

Floor of 40:

```
TEXT
40
```

20. FLOOR SEARCH IDEA

At node:

If node == x

```
TEXT
floor = x
```

finished.

If node > x

Node is too large.

Go:

```
TEXT
LEFT
```

If node < x

Node is a possible floor.

Store it.

Then move:

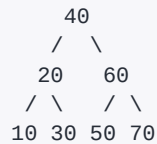
TEXT
RIGHT

looking for a closer/larger value still "dx."

21. FLOOR TRACE

BST:

TEXT



Find floor of 55.

At 40:

TEXT

```
40 < 55
floor = 40
go right
```

At 60:

TEXT

```
60 > 55
go left
```

At 50:

TEXT

```
50 < 55
floor = 50
go right
```

NULL.

Answer:

50

22. C CODE — FLOOR

C

```
Node *floorBST(Node *root, int x)
{
    Node *ans = NULL;

    while (root != NULL)
```

```

{
    if (root->data == x)
        return root;

    if (root->data > x)
    {
        root = root->left;
    }
    else
    {
        ans = root;
        root = root->right;
    }
}

return ans;
}

```

Time:

TEXT

$O(h)$

23. PATTERN6 — CEIL IN BST

Ceil = smallest BST key $\geq x$

For keys:

TEXT

10 20 30 40 50

Ceil of 36:

TEXT

40

24. CEIL LOGIC

At node:

node == x

Answer node.

node < x

Too small.

Go:

TEXT
RIGHT

node > x

Possible ceil.

Store it.

Then move:

TEXT
LEFT

for a smaller valid answer.

25. CEIL TRACE

Find ceil of 55 in:

TEXT



At 40:

TEXT
4 0 < 5 5 !' r i g h t

At 60:

TEXT
60 > 55
ceil=60
go left

At 50:

TEXT
5 0 < 5 5 !' r i g h t

NULL.

Answer:

60

26. FLOOR VS PREDECESSOR

Do not confuse them.

Predecessor of a node:

TEXT

largest key strictly smaller than that node

Floor of arbitrary x:

TEXT

largest key "d" x

Example keys:

TEXT

10 20 30

For $x=20$:

Floor:

TEXT

20

Predecessor of node 20:

TEXT

10

Important distinction.

27. CEIL VS SUCCESSOR

Similarly:

Ceil of 20:

TEXT

20

Successor of node 20:

TEXT

30

assuming distinct keys.

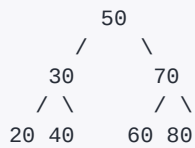
28. PATTERN7 — RANGE SEARCH Ø=Ý%

Problem:

Print all BST keys in interval $[L, R]$.

Example BST:

TEXT



Range:

TEXT

35 to 70

Output:

TEXT

40 50 60 70

29. NAIVE METHOD

Traverse every node:

TEXT

$O(n)$

and print values inside range.

Works, but BST lets us prune unnecessary branches.

30. BST PRUNING

At node X:

If:

TEXT

$X > L$

left subtree may contain useful values.

If:

TEXT

L "d X "d R

print X.

If:

TEXT

$X < R$

right subtree may contain useful values.

31. RANGE SEARCH C LOGIC

```
C
void rangePrint(Node *root, int low, int high)
{
    if (root == NULL)
        return;

    if (root->data > low)
        rangePrint(root->left, low, high);

    if (root->data >= low && root->data <= high)
        printf("%d ", root->data);

    if (root->data < high)
        rangePrint(root->right, low, high);
}
```

Because traversal order is inorder, output is sorted.

32. RANGE SEARCH COMPLEXITY

If k matching keys are reported, in a balanced BST a typical bound is:

TEXT

$O(h + k)$

or:

TEXT

$O(\log n + k)$

for a balanced tree.

Worst ordinary BST:

TEXT

$O(n)$

$\Theta = \Omega$ important output-sensitive idea.

33. PATTERN8 — SORTED ARRAY! BALANCED BST $\Theta = \Omega$ $\Theta = \Omega$ $\Theta = \Omega$

Input:

TEXT

10 20 30 40 50 60 70

If inserted sequentially into ordinary BST:

TEXT

```
10
 \
  20
   \
    ...
```

bad.

Instead, choose middle element as root.

34. STEP 1

Middle:

TEXT

40

Root:

TEXT

40

Left sorted half:

TEXT

10 20 30

Right:

TEXT

50 60 70

35. RECURSE

Left middle:

TEXT

20

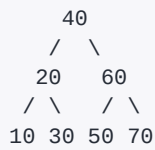
Right middle:

TEXT

60

Then:

TEXT



Balanced.

36. C IMPLEMENTATION

C

```
Node *sortedToBST(int a[], int left, int right)
{
    if (left > right)
        return NULL;

    int mid = left + (right - left) / 2;

    Node *root = createNode(a[mid]);

    root->left =
        sortedToBST(a, left, mid - 1);

    root->right =
        sortedToBST(a, mid + 1, right);

    return root;
}
```

Time:

TEXT

$O(n)$

because each element becomes exactly one node.

Recursion space:

TEXT

$O(\log n)$

for the balanced construction.

37. WHY NOT $O(n \log n)$?

We are not repeatedly inserting/searching.

We directly build each node from an array segment.

Each array element is processed once.

Therefore:

O(n)

$O(1)$ placement complexity trap.

38. EVEN NUMBER OF ELEMENTS

Example:

```
TEXT
10 20 30 40
```

There are two middle choices depending on convention:

```
TEXT
20
```

or:

```
TEXT
30
```

Both can produce valid height-balanced BSTs.

So exact output tree may depend on chosen midpoint rule.

39. PATTERN9 — BST !' SORTED ARRAY

Very simple.

Do:

Inorder traversal

and append each key.

BST:

```
TEXT
      30
     /  \
    20   50
   /  \  /
  10  40
```

Inorder:

```
TEXT
10 20 30 40 50
```

Time:

TEXT

$O(n)$

Space:

TEXT

$O(n)$

for output plus traversal auxiliary space.

40. PATTERN10 — TWO SUM IN BST

Problem:

Are there two BST keys whose sum is K ?

Example keys:

TEXT

10 20 30 40 50

Target:

TEXT

70

Possible:

TEXT

20 + 50

or:

TEXT

30 + 40

Answer:

TEXT

YES

41. SIMPLE METHOD

1. Inorder BST into sorted array.
2. Use two pointers:

TEXT

```
i = start  
j = end
```

If:

TEXT

```
a[i] + a[j] == K
```

found.

If sum too small:

TEXT

```
i++
```

If too large:

TEXT

```
j--
```

42. COMPLEXITY

Inorder:

TEXT

```
O(n)
```

Two-pointer:

TEXT

```
O(n)
```

Total:

TEXT

```
O(n)
```

Extra array:

TEXT

```
O(n)
```

43. ADVANCED TWO-SUM METHOD

Using two BST iterators:

- one ascending
- one descending

can reduce additional storage to:

```
TEXT
O(h)
```

instead of $O(n)$.

~~0-16~~ Advanced placement concept.

Core GATE preparation does not require it.

44. PATTERN11 — CLOSEST VALUE IN BST

Problem:

Find BST key closest to target x .

At each node:

1. Compare current distance:

```
TEXT
abs(node->data - x)
```

with current best.

2. Use BST comparison to choose one subtree.

Because if x is smaller:

```
TEXT
go left
```

if larger:

```
TEXT
go right
```

Time:

```
TEXT
O(h)
```

45. EXAMPLE

BST:

```
TEXT
      40
     /  \
    20   60
     /
    /
```

50

Target:

TEXT

53

Path:

TEXT

4 0 !' 6 0 !' 5 0

Distances:

TEXT

|40-53| = 13

|60-53| = 7

|50-53| = 3

Closest:

50

46. PATTERN12 — BST ITERATOR

A BST iterator is designed to repeatedly give:

TEXT

next smallest element

without storing the whole inorder result.

Use a stack.

Initially push:

TEXT

root
and all its left descendants

47. NEXT() IDEA

1. Pop stack top !' next smallest.
2. If node has right subtree:
 - move right once
 - push its complete left chain.

This is essentially:

lazy iterative inorder traversal

48. COMPLEXITY

Each node is pushed and popped once across all operations.

So:

TEXT

`next() = O(1) amortized`

Storage:

TEXT

`O(h)`

~~O=h)~~ Strong interview pattern.

49. PATTERN13 — FIND IF TWO BSTs HAVE SAME KEYS

Simplest:

1. Inorder first BST.
2. Inorder second BST.
3. Compare sorted outputs.

Time:

TEXT

`O(n1+n2)`

assuming outputs stored.

If both contain same distinct key set, inorder sequences match.

Shape may still differ.

50. SAME INORDER DOES NOT MEAN SAME TREE

Example:

Tree A:

TEXT

```
  20
 /  \
10   30
```

Tree B:

TEXT

```
10
 \
  20
   \
    30
```

Both inorder:

TEXT

10 20 30

but structures differ.

Ø=Imp important interview/GATE trap.

51. PATTERN14 — RECOVER BST WITH TWO SWAPPED KEYS Ø=pe

Suppose two nodes' values are accidentally swapped.

A correct BST inorder should be sorted.

Example correct:

TEXT

10 20 30 40 50

After swap:

TEXT

10 40 30 20 50

We detect inversions where:

TEXT

previous > current

Using inorder, identify misplaced nodes and swap values back.

Ø=Common interview problem.

52. WHY INORDER IS SO POWERFUL

BST's entire ordered nature is exposed by:

TEXT

L E F T ! ' R O O T ! ' R I G H T

Therefore many problems reduce to:

Use inorder or reverse inorder

Examples:

- validation
- kth smallest
- kth largest
- sorted output
- predecessor/successor intuition
- recover swapped BST
- two sum

$O(n)$ Major pattern.

53. PATTERN15 — MERGE TWO BSTs

Simple method:

1. Inorder BST1 !' sorted array A.
2. Inorder BST2 !' sorted array B.
3. Merge sorted arrays.
4. Optionally create balanced BST from merged array.

Complexity:

TEXT

$O(n + m)$

for traversal+merge.

Additional storage:

TEXT

$O(n+m)$

54. PATTERN16 — COUNT KEYS IN RANGE

Instead of printing keys `[L, R]` , maintain:

TEXT

count

and use same pruning logic.

Naive:

TEXT

$O(n)$

Balanced BST with good pruning can often do much less, though exact bound depends on tree/output.

If subtree sizes and augmented metadata are maintained, even stronger operations become possible.

55. PATTERN17 — FIND MINIMUM DIFFERENCE BETWEEN BST KEYS

Since inorder is sorted:

TEXT

$a_1 < a_2 < a_3 < \dots$

minimum difference must occur between:

adjacent inorder values

So during inorder:

TEXT

$\text{difference} = \text{current} - \text{previous}$

keep minimum.

Time:

TEXT

$O(n)$

Space:

TEXT

$O(h)$

recursive.

~~0-Placement~~ pattern.

56. WHY ONLY ADJACENT?

Suppose:

TEXT

$a < b < c$

Then:

TEXT

$c - a$

is larger than at least one of:

TEXT

$b - a$

$c - b$

So closest pair in sorted sequence must be adjacent.

This is a sorted-order property.

57. PATTERN18 — BST TO GREATER-SUM TREE Ø=pe

A common interview pattern:

Replace every node value by sum of keys greater than/equal to it.

Normal inorder:

TEXT

s m a l l ! ' l a r g e

Instead use:

Reverse inorder

TEXT

l a r g e ! ' s m a l l

Maintain running sum.

This is another example of exploiting BST ordering.

58. PLACEMENT MASTER PATTERNMAP

If problem asks...

Sorted ascending

TEXT

Inorder

Sorted descending

TEXT

Reverse inorder

Kth smallest

TEXT

Inorder + count

Kth largest

TEXT

Reverse inorder + count

Validation

TEXT

Range or inorder

LCA

TEXT

Compare both keys with current root

Floor/Ceil

TEXT

Search while keeping best candidate

Range

TEXT

BST pruning

Balanced construction from sorted data

TEXT

Choose middle recursively

Ø=7% This table is the heart of BST interviews.

59. COMMON INTERVIEW TRAP — USING GENERAL TREE SOLUTION

Suppose LCA is asked in a BST.

A general binary-tree LCA algorithm may work in:

TEXT

$O(n)$

But BST ordering gives:

TEXT

$O(h)$

Use the structure you are given.

Ø=1% Always ask:

"What property of this data structure can prune work?"

60. COMMON TRAP — KTH SMALLEST BY BFS

Level order has nothing to do with numeric sorted order.

Example:

TEXT

```
    50
   /  \
  20  80
```

Level order:

TEXT

50 20 80

not sorted.

For kth smallest:

Use inorder

61. COMMON TRAP — MINIMUM BY SCANNING ALL NODES

You can scan all nodes in:

TEXT

$O(n)$

but BST property gives:

TEXT

leftmost

in:

TEXT

$O(h)$

Use ordering.

62. COMMON TRAP — FLOOR/CEIL BY FULL INORDER

Full inorder works:

TEXT

$O(n)$

but direct BST search finds floor/ceil in:

TEXT

$O(h)$

Better.

63. COMMON TRAP — SORTED ARRAY INSERTION

If you simply insert sorted array keys one by one:

TEXT

1, 2, 3, 4, 5...

ordinary BST becomes skewed.

To deliberately build balanced BST:

Choose midpoint recursively

64. COMMON TRAP — $O(\log n)$ ASSUMPTION

Every placement solution using ordinary BST should still be described as:

TEXT

$O(h)$

Then specify:

TEXT

balanced: $O(\log n)$

worst: $O(n)$

unless the data structure guarantees balance.

~~0-1%~~ Interviewers often check this nuance.

65. VALIDATE BST COMPLEXITY

You cannot validate an arbitrary binary tree by following only one path.

Every node may contain a violation.

Therefore:

TEXT

Time = $O(n)$

even though ordinary BST search is $O(h)$.

~~0-5%~~ Search and validation are different problems.

66. KTH SMALLEST COMPLEXITY NUANCE

A simple inorder can stop after kth item.

It may visit approximately:

```
TEXT
O(h+k)
```

nodes in favorable implementations/analysis rather than always scanning all n.

Safe worst case:

```
TEXT
O(n)
```

Without subtree-size augmentation.

67. AUGMENTED BST $O(\log n)$

A BST node can store extra metadata:

```
C
struct Node
{
    int key;
    int size;
    struct Node *left;
    struct Node *right;
};
```

where:

```
TEXT
size = number of nodes in subtree
```

Then we know the rank of root:

```
TEXT
leftSize + 1
```

68. KTH SMALLEST USING SUBTREE SIZE

Suppose:

```
TEXT
leftSize = L
```

Then root is:

TEXT

$(L+1)$ th smallest

If:

TEXT

$k == L+1$

answer root.

If:

TEXT

$k <= L$

go left.

Otherwise:

TEXT

$k = k - (L+1)$

go right.

Time:

TEXT

$O(h)$

$O = \log N$ is called an order-statistic style augmentation.

69. WHY AUGMENTATION MATTERS

Basic BST stores:

TEXT

key
left
right

Augmented BST may additionally store:

- subtree size
- subtree sum
- height
- min/max
- frequency

This can speed up specialized queries.

Not part of your GATE core syllabus, but useful placement knowledge.

70. GATE CONNECTION — INORDER STILL CENTRAL

Even with all these placement patterns, remember the GATE core:

TEXT

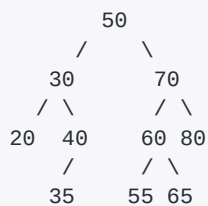
BST inorder = sorted

Most advanced patterns are simply exploiting this fact without always generating the whole inorder sequence.

71. SOLVED PLACEMENT EXAMPLE

BST:

TEXT



Find:

1. 4th smallest

Inorder:

TEXT

20 30 35 40 50 55 60 65 70 80

Answer:

TEXT

40

72. 3RD LARGEST

Reverse:

TEXT

80 70 65 60 55 50 40 35 30 20

3rd:

TEXT

65

73. LCA OF 55 AND 65

At 50:

TEXT

both > 50

go right.

At 70:

TEXT

both < 70

go left.

At 60:

TEXT

55 < 60 < 65

split.

Answer:

60

74. FLOOR OF 58

Sorted:

TEXT

...50 55 60...

Greatest "d58:

TEXT

55

75. CEIL OF 58

Smallest "e58:

TEXT

60

76. RANGE [35,65]

Output:

TEXT

35 40 50 55 60 65

sorted using pruned inorder.

77. CLOSEST TO 63

Nearby keys:

TEXT

60
65

Distances:

TEXT

3
2

Answer:

TEXT

65

78. GATE + PLACEMENT MASTER COMPLEXITY TABLE

| Problem | Time |
|------------------------------|--------------|
| Search | $O(h)$ |
| Insert | $O(h)$ |
| Delete | $O(h)$ |
| Min/Max | $O(h)$ |
| Pred/Succ | $O(h)$ |
| LCA in BST | $O(h)$ |
| Floor/Ceil | $O(h)$ |
| Validate BST | $O(n)$ |
| Inorder | $O(n)$ |
| Kth smallest basic | $O(n)$ worst |
| Sorted array !' balanced BST | $O(n)$ |
| BST !' sorted array | $O(n)$ |
| Two sum using array | $O(n)$ |

79. NOTEBOOK REVISION — 8 KEY PATTERNS

TEXT

| | | |
|-------------------------|----|---|
| V a l i d a t e | !' | R a n g e |
| K t h s m a l l e s t | !' | I n o r d e r |
| K t h l a r g e s t | !' | R e v e r s e i n o r d e r |
| L C A | !' | B o t h l e f t / b o t h r i g h t / s p l i t |
| F l o o r | !' | b e s t s m a l l e r - o r - e q u a l |
| C e i l | !' | b e s t l a r g e r - o r - e q u a l |
| R a n g e | !' | P r u n e d i n o r d e r |
| S o r t e d ! ' B S T | !' | M i d d l e r e c u r s i v e l y |

Write this as your placement cheat sheet.

80. 30-SECOND REVISION

Remember:

1. Validate BST with global ranges.
2. kth smallest = inorder.
3. kth largest = reverse inorder.
4. BST LCA uses ordering.
5. Floor = greatest "d" . x
6. Ceil = smallest "e" . x
7. Range queries prune impossible subtrees.
8. Sorted array != balanced BST by recursive midpoint.
9. BST != sorted array using inorder.
10. Always state complexity in terms of height first.

81. ACTIVE RECALL

Q1

Which traversal gives kth smallest?

Q2

Which traversal gives kth largest?

Q3

If both LCA keys are smaller than current root, where do we go?

Q4

What is floor of x?

Q5

What is ceil of x?

Q6

How do you directly construct a balanced BST from sorted array?

ANSWERS

A1

Inorder.

A2

Reverse inorder.

A3

Left subtree.

A4

Largest key "d" . x

A5

Smallest key "e" . x

A6

Choose the middle element as root and recursively build the two halves.

82. PRACTICE SET

Use:

TEXT



Find:

1. Is the tree a valid BST?
2. 5th smallest
3. 2nd largest
4. LCA of 50 and 65
5. LCA of 25 and 90
6. floor of 55
7. ceil of 55
8. keys in [25, 70]
9. predecessor of 60
10. successor of 65

83. BST MODULE — COMPLETE '

We have now completed **Binary Search Trees** from GATE foundation through placement extensions:

' BST definition ' Global ordering property ' Search ' Insertion ' Deletion ' Min/max ' Predecessor/successor ' Height and complexity ' Valid search paths ' BST reconstruction ' Catalan counting ' Validation ' kth smallest/largest ' LCA ' floor/ceil ' range search ' balanced construction ' BST-to-sorted conversion ' two-sum pattern ' iterator intuition ' key placement/interview traps

MODULE 7 — BINARY SEARCH TREES ' COMPLETE

OFFICIAL SYLLABUS PROGRESS

Programming in C. Recursion. Arrays, stacks, queues, linked lists, trees, binary search trees, binary heaps, graphs.

| Topic | Status |
|---------------------|------------|
| Programming in C | ' Complete |
| Recursion | ' Complete |
| Arrays | ' Complete |
| Stacks | ' Complete |
| Queues | ' Complete |
| Linked Lists | ' Complete |
| Trees | ' Complete |
| Binary Search Trees | ' Complete |
| Binary Heaps | #ó NEXT |
| Graphs | #ó |

NEXT — MODULE 8, LESSON 65

Binary Heap Fundamentals — Complete Binary Tree + Heap Property, Min-Heap vs Max-Heap

We now begin **Binary Heaps**, where two ideas must be combined:

TEXT

1. COMPLETE BINARY TREE SHAPE
2. HEAP ORDER PROPERTY

We will cover:

- what a binary heap really is
- min-heap
- max-heap
- why heap is complete
- array representation
- parent/child formulas
- heap vs BST
- heap vs priority queue
- root properties
- what is and is not sorted in a heap
- height $\sim (\log n)$
- GATE traps and identification questions.

USER

next

ASSISTANT

MODULE 8 — LESSON 65

Binary Heap Fundamentals — Complete Binary Tree + Heap Property, Min-Heap vs Max-Heap

We now begin:

BINARY HEAPS

A binary heap combines exactly **two ideas**:

1. Complete Binary Tree Shape

2. Heap Order Property

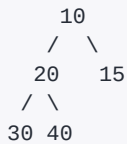
If even one of these is missing, it is **not a binary heap**.

GATE importance: 0=Ý%0450KNO Placement importance: 0=Ý%0450KNO

1. START WITH THE BASIC IDEA

Consider:

TEXT



This is a:

Min-Heap

because:

1. It is a **complete binary tree**.
2. Every parent is less than or equal to its children.

2. DEFINITION OF BINARY HEAP

A binary heap is a binary tree satisfying:

Shape Property

It must be:

Complete Binary Tree

Meaning:

- every level except possibly the last is completely filled
- last level is filled from left to right

Heap Property

Depending on heap type:

Min-Heap

TEXT

P a r e n t "d C h i l d r e n

Max-Heap

TEXT

P a r e n t "e C h i l d r e n

0-100 conditions are mandatory.

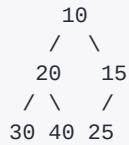
3. MIN-HEAP

In a min-heap:

Every parent key is less than or equal to its children

Example:

TEXT



Check:

At 10:

TEXT

```
1 0 "d 2 0
1 0 "d 1 5
```

At 20:

TEXT

```
2 0 "d 3 0
2 0 "d 4 0
```

At 15:

TEXT

```
1 5 "d 2 5
```

So heap property holds.

Shape is complete.

Therefore:

Valid Min-Heap '

4. ROOT OF MIN-HEAP Ø=Ý%Ø=Ý%Ø=Ý%

Since every parent is smaller than its descendants through repeated parent-child ordering:

Minimum element is always at the root

Therefore:

TEXT

Find minimum = 0(1)

in a min-heap.

Ø=Ý%Ø=Ý%Ø=Ý%

5. IS SECOND SMALLEST ALWAYS A CHILD OF ROOT?

For distinct keys:

Yes.

Why?

Any node deeper than level 1 has a parent above it.

That parent must be smaller than that node.

So the second-smallest element must be among the root's children.

Not the GATE property.

But determining exactly which child requires comparing the children.

6. MAX-HEAP

In max-heap:

Every parent $\geq$ its children

Example:

TEXT

```
      90
     /  \
    70   80
   /  \  /
  20 60 50
```

Check:

TEXT

```
90 "e 70 , 80
70 "e 20 , 60
80 "e 50
```

Valid complete shape.

Therefore:

Valid Max-Heap

7. ROOT OF MAX-HEAP

Maximum element is always root

So:

TEXT

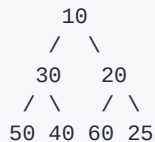
Find maximum = $O(1)$

in max-heap.

8. VERY IMPORTANT — HEAP IS NOT SORTED

Consider min-heap:

TEXT



Is this valid?

Check:

TEXT

```
1 0 "d 3 0 , 2 0
3 0 "d 5 0 , 4 0
2 0 "d 6 0 , 2 5
```

Yes.

But level order:

TEXT

```
10 30 20 50 40 60 25
```

is clearly not sorted.

~~Min-heap~~ only guarantees:

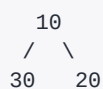
Parent-child ordering

not full ordering between siblings/subtrees.

9. SIBLINGS HAVE NO REQUIRED ORDER

In a min-heap:

TEXT



This is valid even though:

TEXT

30 > 20

No rule says:

TEXT

left child < right child

or vice versa.

Ø=Min trap.

10. LEFT SUBTREE VS RIGHT SUBTREE

In BST:

TEXT

all left keys < root < all right keys

In heap:

No such relationship exists between left and right subtrees.

Example min-heap:

TEXT

```
      5
     /\
    20 10
   /\
  30
```

Valid.

There is no rule requiring:

TEXT

left subtree values < right subtree values

11. HEAP VS BST Ø=Ý%Ø=Ý%Ø=Ý%

BST

TEXT

Left subtree < Root < Right subtree

Main advantage:

TEXT

search arbitrary key

Min-Heap

TEXT

P a r e n t "d c h i l d r e n

Main advantage:

TEXT

minimum quickly

Max-Heap

TEXT

P a r e n t "e c h i l d r e n

Main advantage:

TEXT

maximum quickly

12. MASTER COMPARISON

| Property | BST | Binary Heap |
|-----------------------|-----------------------|-----------------|
| Shape | any binary-tree shape | complete |
| Ordering | global left/right | parent-child |
| Root | arbitrary rank | min or max |
| Arbitrary search | $O(h)$ | $O(n)$ |
| Find min | $O(h)$ BST | $O(1)$ min-heap |
| Find max | $O(h)$ BST | $O(1)$ max-heap |
| Inorder sorted | yes for BST | no |
| Natural array storage | not always | yes |

0-0% Copy this.

13. WHY MUST A BINARY HEAP BE COMPLETE?

Completeness gives:

- small height
- no unnecessary gaps
- efficient array storage
- simple parent/child formulas
- height $\sim (\log n)$

Example:

TEXT



Array:

TEXT

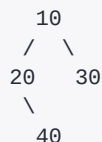
A B C D E F

No gaps.

This is exactly what heaps need.

14. INVALID SHAPE EXAMPLE

TEXT



Heap-order may satisfy:

TEXT

```

1 0"d 2 0
1 0"d 3 0
2 0"d 4 0

```

but shape is not complete because:

- node 20 has a right child
- left-child position is missing

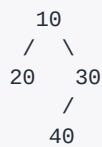
Therefore:

Not a binary heap 'L

Heap order alone is insufficient.

15. ANOTHER INVALID SHAPE

TEXT



Is it complete?

Level 2 positions should fill:

TEXT

20.left
20.right
30.left
30.right

from left to right.

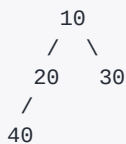
But 30.left exists while earlier positions under 20 are missing.

Therefore:

Not complete !' not heap

16. VALID COMPLETE EXAMPLE

TEXT



Last level starts from far left.

So complete.

If parent ordering holds:

Valid heap

17. PERFECT VS COMPLETE

Heap needs:

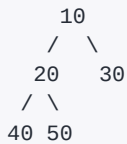
Complete

not necessarily:

Perfect

Example:

TEXT



This is complete but not perfect.

Still valid heap.

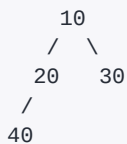
Do not say heaps must be perfect.

18. FULL VS HEAP

Heap also does not need to be full.

Example:

TEXT



Node 20 has only one child.

Still a valid complete binary tree.

So:

Heap need not be full

19. HEAP HEIGHT $\Theta(\log n)$

Because heap is complete:

For n nodes:

FORMULA BOX

TEXT

$$h = \text{floor}(\log_2 n)$$

under edge-based height.

This gives:

TEXT

$$h = \sim (\log n)$$

Unlike ordinary BST, heap height cannot degrade to $n-1$.

Completeness guarantees logarithmic height.

20. EXAMPLE

Heap with:

```
TEXT
n = 15
```

height:

```
TEXT
floor(log2 15)=3
```

Heap with:

```
TEXT
n = 100
```

height:

```
TEXT
floor(log2 100)=6
```

because:

```
TEXT
2 ^ 6 <= 100 < 2 ^ 7
```

21. WHY ARRAY REPRESENTATION IS IDEAL

Complete tree:

```
TEXT
      A
     / \
    B   C
   / \ / \
  D  E F  G
```

Store level order:

```
TEXT
A B C D E F G
```

No explicit pointers needed.

This saves memory and gives instant index relationships.

0-1 n-ary heaps are almost always implemented using arrays.

22. 0-BASED ARRAY REPRESENTATION

Suppose:

TEXT

Index: 0 1 2 3 4 5 6
Value: A B C D E F G

For node index i :

FORMULA BOX

TEXT

Left child = $2i + 1$
Right child = $2i + 2$
Parent = $\text{floor}((i-1)/2)$

for $i > 0$.

Ø=NOT KNOWN.

23. EXAMPLE — 0-BASED

At:

TEXT

$i = 2$

node C.

Left:

TEXT

$2(2)+1 = 5$

! F

Right:

TEXT

$2(2)+2 = 6$

! G

Parent:

TEXT

$\text{floor}((2-1)/2)=0$

! A

Correct.

24. 1-BASED REPRESENTATION

If array indexes start at 1:

TEXT

Index: 1 2 3 4 5 6 7
Value: A B C D E F G

Then:

FORMULA BOX

TEXT

Left child = $2i$
Right child = $2i + 1$
Parent = $\text{floor}(i/2)$

~~0=100%~~ Do not mix 0-based and 1-based formulas.

25. MEMORY TRICK

1-based

TEXT

$L = 2i$
 $R = 2i+1$

Very clean.

0-based

Everything shifts by 1:

TEXT

$L = 2i+1$
 $R = 2i+2$

26. LEAF NODES IN 0-BASED HEAP

For n elements:

First leaf index:

FORMULA BOX

TEXT

$\text{floor}(n/2)$

Last leaf:

TEXT

$n-1$

Therefore internal nodes:

TEXT

$0 \dots \text{floor}(n/2)-1$

Leaves:

TEXT

$\text{floor}(n/2) \dots n-1$

$O(n)$ important for heapify/build-heap.

27. EXAMPLE $n=10$

0-based indices:

TEXT

0 1 2 3 4 5 6 7 8 9

First leaf:

TEXT

$\text{floor}(10/2)=5$

Leaves:

TEXT

5, 6, 7, 8, 9

Last internal node:

TEXT

4

28. 1-BASED VERSION

For n heap elements:

Internal nodes:

TEXT

1 ... floor(n/2)

Leaves:

TEXT

floor(n/2)+1 ... n

Again, convention matters.

29. NUMBER OF LEAVES

For complete binary tree/heap with n nodes:

FORMULA BOX

TEXT

Leaves = ceil(n/2)

Internal nodes:

TEXT

floor(n/2)

30. HEAP PROPERTY IN ARRAY

Min-heap 0-based:

For every valid child:

TEXT

A[parent] <= A[child]

Max-heap:

TEXT

A[parent] >= A[child]

No full sorting requirement.

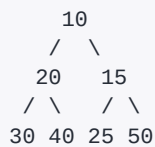
31. MIN-HEAP ARRAY EXAMPLE

TEXT

Index: 0 1 2 3 4 5 6
Value: 10 20 15 30 40 25 50

Tree:

TEXT



Check every parent-child relation.

Valid min-heap.

32. IS THIS MIN-HEAP?

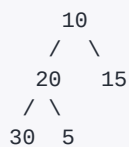
Array:

TEXT

10 20 15 30 5

Tree:

TEXT



At 20:

TEXT

20 "d 5

false.

Therefore:

Not a min-heap

One violation is enough.

33. IS THIS MAX-HEAP?

Array:

TEXT

100 80 90 40 70 60 50

Tree:

TEXT





All parents "echildren.

Complete.

Therefore:

Valid max-heap

34. HOW TO VERIFY HEAP QUICKLY

Because completeness is implied by a compact heap array representation, for an array representing a heap candidate:

For each internal node:

TEXT

```
i = 0 ... floor(n/2)-1
```

check child relations.

For min-heap:

TEXT

```
A [ i ] "d A [ l e f t ]
A [ i ] "d A [ r i g h t ]
```

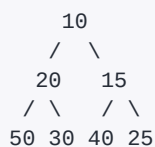
if children exist.

~~O = N~~ need to compare every pair of elements.

35. SEARCHING ARBITRARY KEY IN HEAP

Suppose min-heap:

TEXT



Search for:

TEXT

40

Can we use binary-search-like decisions?

No.

At root 10:

TEXT

40 > 10

But 40 could be:

- left subtree
- right subtree

because both subtrees can contain larger values.

Worst case:

$O(n)$

Heap is not designed for arbitrary search.

36. WHY HEAP IS GOOD

Heap's strength is not arbitrary lookup.

Its strength is quickly maintaining an extreme element:

Min-Heap

TEXT

minimum at root

Max-Heap

TEXT

maximum at root

This is perfect for:

Priority Queues

37. HEAP VS PRIORITY QUEUE

Important distinction:

Priority Queue = ADT

It describes operations such as:

- insert
- peek highest/lowest priority
- extract highest/lowest priority

Binary Heap = Data Structure

It is one efficient way to implement a priority queue.

~~0=0%~~ not say heap and priority queue are exactly the same thing.

38. MIN-PRIORITY QUEUE

If smaller value means higher priority:

Use:

Min-Heap

Root contains smallest key.

Operations eventually:

TEXT

```
peek-min    0(1)
insert      0(log n)
extract-min 0(log n)
```

We'll derive these in upcoming lessons.

39. MAX-PRIORITY QUEUE

If larger value means higher priority:

Use:

Max-Heap

Root contains largest key.

40. APPLICATIONS OF HEAPS

~~0=0%~~ Common:

- priority queues
- CPU/job scheduling
- Dijkstra implementations
- Prim's algorithm implementations
- heapsort
- kth largest/smallest
- top-k elements
- merge k sorted lists
- event simulation
- streaming median with two heaps

Some graph algorithms come later.

41. HEAP IS A TREE LOGICALLY, ARRAY PHYSICALLY

Conceptually:

TEXT
binary tree

Implementation usually:

TEXT
array

This distinction is important.

Example:

TEXT
Array:
10 20 15 30 40

represents:

TEXT

```
      10
     /  \
    20   15
   /  \
  30  40
```

42. DOES ARRAY NEED POINTERS?

No.

Because index formulas determine:

TEXT
parent
left child
right child

So heap nodes typically do not need:

TEXT
left pointer
right pointer

This is one major memory advantage.

43. INSERTION LOCATION IS FIXED BY SHAPE

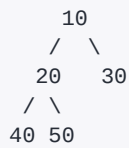
Because heap must remain complete, when inserting a new node we cannot choose arbitrary location.

The new node must occupy:

next free position in level order

Example:

TEXT



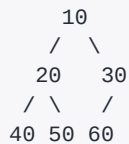
Next node must go:

TEXT

right subtree's left position

giving:

TEXT



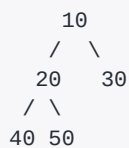
Not anywhere else.

$\emptyset$ = Shape determines insertion position.

44. BUT NEW VALUE MAY VIOLATE HEAP PROPERTY

Suppose min-heap:

TEXT



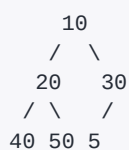
Insert:

TEXT

5

Shape says put at next slot:

TEXT



But:

TEXT

3 0 "d 5

false.

So we must repair heap order.

This repair is:

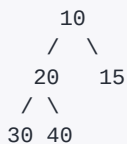
Bubble Up / Percolate Up / Heapify Up

Lesson 67.

45. DELETION/EXTRACT ROOT PREVIEW

Suppose min-heap:

TEXT



Remove root 10.

We cannot leave a gap at root.

Typical idea:

1. Move last element to root.
2. Reduce heap size.
3. Restore heap property downward.

This repair is:

Heapify Down / Sift Down

Lesson 68.

46. HEAP HEIGHT MAKES REPAIR FAST

Since heap height:

TEXT

$\sim (\log n)$

moving a node:

TEXT

up one root-to-leaf path

or:

TEXT

down one path

takes:

TEXT

$O(\log n)$

That's why:

TEXT

insert = $O(\log n)$
extract = $O(\log n)$

47. GATE TRAP — MIN-HEAP ROOT

Which is definitely true?

A. Root is smallest. B. Last leaf is largest. C. Left child < right child. D. Inorder is sorted.

Answer:

A

The others are not guaranteed.

48. IS THE LARGEST ELEMENT AT A LEAF IN MIN-HEAP?

For a min-heap, the maximum element must occur at a leaf.

Why?

If a maximum-value node were an internal node with children, heap property says:

TEXT

parent > child

If keys are distinct, each child must be greater than the parent, contradicting parent being maximum.

With equal keys allowed, a maximum value may appear internally and also below it.

So the clean GATE statement under **distinct keys**:

Maximum of a min-heap lies at a leaf

Similarly:

Minimum of a distinct-key max-heap lies at a leaf

Good property with duplicate caveat.

49. CAN WE FIND MAXIMUM IN MIN-HEAP IN $O(1)$?

No.

We know maximum lies among leaves.

Number of leaves:

```
TEXT
ceil(n/2)
```

So scanning leaves:

```
TEXT
~ ( n )
```

Thus:

```
TEXT
Find max in min - heap = ~ ( n )
```

in the standard array without extra metadata.

$\Theta(n)$ important.

50. FIND MINIMUM IN MAX-HEAP

Symmetrically:

```
TEXT
Find min in max - heap = ~ ( n )
```

because inspect leaves in worst case.

51. EXTREME-OPERATION TABLE

| Operation | Min-Heap | Max-Heap |
|--------------|------------|------------|
| Find minimum | $O(1)$ | $\sim (n)$ |
| Find maximum | $\sim (n)$ | $O(1)$ |

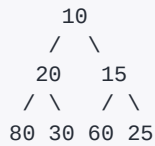
Assuming standard binary heap and no extra metadata.

52. ARE LEAVES SORTED?

No.

Min-heap:

TEXT



Leaves:

TEXT

80, 30, 60, 25

No sorted relation.

Heap guarantees only ancestor direction.

53. ANCESTOR PROPERTY

In min-heap:

For any ancestor **A** and descendant **D**:

TEXT

A "d D

because parent-child relation repeats down the path.

In max-heap:

TEXT

A "e D

$\emptyset \neq \forall$ This stronger ancestor-descendant consequence is valid.

54. DOES **A [i] "d HOLD IN MIN-HEAP?**

No.

Example:

TEXT

10 30 20

Valid min-heap:

TEXT

10
/ \
30 20

But:

TEXT

3 0 "d 2 0

false.

So heap array is not sorted.

55. LEVEL ORDER OF HEAP

Heap array corresponds to:

level-order traversal

Therefore:

TEXT

A[0]

root,

then level 1,

then level 2,

etc.

This is because completeness makes all positions contiguous.

56. PARENT OF LAST NODE

0-based last index:

TEXT

n-1

Parent:

TEXT

$\text{floor}((n-2)/2)$

Useful in insertion reasoning.

Example n=7:

last index 6.

Parent:

TEXT

$\text{floor}((6-1)/2)=2$

or equivalently:

TEXT

$\text{floor}((7-2)/2)=2$

57. LAST INTERNAL NODE $\emptyset = \lceil Y\% \rceil$

0-based:

TEXT

$\text{floor}(n/2) - 1$

This becomes crucial in build-heap.

Why?

Every index after that is a leaf and already trivially satisfies heap property by itself.

58. EXAMPLE $n=9$

Indices:

TEXT

0 1 2 3 4 5 6 7 8

First leaf:

TEXT

$\text{floor}(9/2)=4$

Last internal:

TEXT

3

So build-heap can start heapifying from index:

TEXT

3

and move backward.

$\emptyset = \lceil Y\% \rceil$ review of Lesson 69.

59. DUPLICATES IN HEAP

Heaps can usually allow equal keys.

Min-heap condition:

TEXT

p a r e n t "d c h i l d r e n

not necessarily:

TEXT

parent < children

Example:

TEXT

```
  10
 /  \
10   20
```

valid min-heap.

Max-heap similarly uses:

TEXT

"e

60. HEAP ORDER IS PARTIAL ORDER

A heap does not fully sort all elements.

It maintains enough order to guarantee an extreme at the root.

This can be viewed as:

Partial ordering

Example:

TEXT

10

is known smaller than descendants in min-heap.

But between:

TEXT

20 and 15

both children, either may be smaller.

No total ordering is guaranteed.

61. WHY THIS IS USEFUL

Fully sorting after every insertion would be expensive.

Heap maintains only the ordering needed for priority operations.

So it gives a strong tradeoff:

TEXT

```
Peek extreme =  $O(1)$ 
Insert       =  $O(\log n)$ 
Extract      =  $O(\log n)$ 
```

without keeping every element globally sorted.

62. GATE-STYLE VALIDITY QUESTION

Which tree is a min-heap?

A

TEXT



Complete and parent $\geq$ child.

' Valid.

B

TEXT



Not complete.

' Invalid.

63. GATE-STYLE VALIDITY QUESTION 2

TEXT



Check $10 \geq 9$:

TEXT

1 0 "d 9 'L

Therefore:

Not a min-heap

Even though shape is complete.

64. GATE-STYLE QUESTION — INDEX

0-based heap array.

Node at index:

TEXT

i=7

Parent:

TEXT

$\text{floor}((7-1)/2)$
=
3

Left child:

TEXT

$2(7)+1 = 15$

Right:

TEXT

16

If those indexes exceed heap size, children don't exist.

65. GATE-STYLE QUESTION — 1-BASED

Node at:

TEXT

i=5

Parent:

TEXT

$\text{floor}(5/2)=2$

Left:

TEXT

10

Right:

TEXT

11

66. GATE-STYLE HEIGHT QUESTION

A binary heap contains:

TEXT

$n = 1000$

Height:

TEXT

$\text{floor}(\log_2 1000)$

Since:

TEXT

$2^9=512$

$2^{10}=1024$

therefore:

$h = 9$

67. GATE-STYLE LEAF QUESTION

Heap has:

TEXT

$n=15$

Leaves:

TEXT

$\text{ceil}(15/2)=8$

Internal:

TEXT

$\text{floor}(15/2)=7$

0-based first leaf:

TEXT

$\text{floor}(15/2)=7$

Indices:

TEXT

7...14

68. GATE-STYLE HEAP ARRAY CHECK

Is:

TEXT

[2, 5, 3, 9, 8, 7, 6]

a min-heap?

Check:

Index 0 = 2:

TEXT

2"d 5

2"d 3

Index 1 = 5:

TEXT

5"d 9

5"d 8

Index 2 = 3:

TEXT

3"d 7

3"d 6

All true.

Yes

Notice array itself:

TEXT

2, 5, 3, 9, 8, 7, 6

is not sorted.

Still valid heap.

69. GATE-STYLE COUNTEREXAMPLE

Array:

TEXT

[2, 5, 3, 9, 1, 7, 6]

At index 1:

TEXT

5

child index 4:

TEXT

1

For min-heap:

TEXT

5" d 1

false.

So invalid.

70. HEAP PROPERTY CHECK COMPLEXITY

To verify array is a heap:

Only internal nodes need checking.

There are:

TEXT

$\text{floor}(n/2)$

internal nodes.

Each has at most 2 children.

Therefore:

$\sim(n)$

time.

Checking heap property is not $O(\log n)$.

71. HEAP CONSTRUCTION PREVIEW

Suppose arbitrary array:

TEXT

4 10 3 5 1

It may not be heap.

Two broad construction methods:

1. Insert elements one-by-one

! $O(n \log n)$

2. Bottom-up Build Heap

! $O(n)$ $\Theta(n)$

This surprising result will be Lesson 69.

72. WHY $O(n)$ BUILD HEAP IS SURPRISING

There are many nodes, each heapify may look $O(\log n)$.

Naively:

TEXT

$n \times \log n$

seems expected.

But most nodes are near leaves and require very little work.

The total sums to:

$O(n)$

We'll derive it carefully later.

73. HEAPSORT PREVIEW

Using max-heap:

1. Build max-heap.
2. Maximum at root.
3. Swap root with last active element.
4. Reduce heap size.
5. Heapify root.
6. Repeat.

Result:

TEXT

ascending sorted array

Time:

TEXT

$O(n \log n)$

Lesson 70.

74. PRIORITY QUEUE PREVIEW

Heap supports:

Peek

TEXT

$O(1)$

Insert

TEXT

$O(\log n)$

Extract extreme

TEXT

$O(\log n)$

Build heap

TEXT

$O(n)$

This combination makes binary heaps one of the standard priority-queue implementations.

75. BINARY HEAP VS BINOMIAL/FIBONACCI HEAP

For your current official syllabus:

Binary Heap

is what we focus on.

Other heap families exist:

- binomial heap
- Fibonacci heap
- pairing heap

but they are not necessary for this syllabus unless a separate topic explicitly requires them.

76. MIN-HEAP INORDER?

Example:

TEXT

```
  10
 /  \
20  15
```

Inorder:

TEXT

20 10 15

Not sorted.

Therefore:

~~Do not~~ Do not transfer the BST inorder rule to heaps.

77. PREORDER/POSTORDER OF HEAP?

They are just tree traversals.

They do not have a special sorted meaning.

For heap algorithms, array indexing and level-order representation are usually more relevant.

78. CAN A TREE BE BOTH BST AND MIN-HEAP?

Interesting question.

With distinct keys and ordinary BST/min-heap rules:

If root is min-heap root, it must be global minimum.

In BST, if root is global minimum, it cannot have a left subtree.

Completeness then severely restricts shape.

For multiple distinct nodes, a complete BST that is also min-heap can exist only in very restricted shapes; e.g. with 2 nodes:

TEXT

```
1
 \
 2
```

would satisfy BST and min-heap ordering but is **not complete**, because a single child in a complete binary tree must be left.

With 2 nodes, complete shape is:

TEXT

```
  1
 /
2
```

which violates BST.

Thus for distinct keys under standard definitions, nontrivial simultaneous BST + min-heap is generally impossible under complete shape.

Ø=0 This is more conceptual than required, but it highlights the conflict between their ordering rules.

79. CAN A TREE BE BOTH BST AND MAX-HEAP?

Similar conflict.

BST wants smaller left and larger right.

Max-heap wants root maximum.

A nontrivial complete structure with distinct keys conflicts with standard BST ordering.

Again, these are fundamentally different data structures.

80. NOTEBOOK NOTES — DEFINITION

Binary Heap

TEXT

Complete Binary Tree
+
Heap Order Property

Min-heap:

TEXT

P a r e n t "d c h i l d r e n

Max-heap:

TEXT

P a r e n t "e c h i l d r e n

81. NOTEBOOK NOTES — ROOT

Min-heap:

TEXT

Root = Minimum

Max-heap:

TEXT

Root = Maximum

Opposite extreme:

TEXT

$\sim (n)$

to find in standard heap.

82. NOTEBOOK NOTES — ARRAY FORMULAS

0-based

TEXT

Left = $2i+1$
Right = $2i+2$
Parent = $\text{floor}((i-1)/2)$

1-based

TEXT

Left = $2i$
Right = $2i+1$
Parent = $\text{floor}(i/2)$

Copy exactly.

83. NOTEBOOK NOTES — STRUCTURE

For n nodes:

TEXT

Height = $\text{floor}(\log_2 n)$

0-based:

TEXT

First leaf = $\text{floor}(n/2)$
Last internal = $\text{floor}(n/2)-1$

Leaves:

TEXT

$\text{ceil}(n/2)$

Internal:

TEXT

$\text{floor}(n/2)$

84. DON'T CONFUSE

BST

TEXT

global left/right ordering

Heap

TEXT

parent-child ordering

Complete Tree

TEXT

shape property

Priority Queue

TEXT

ADT

Binary Heap

TEXT

data structure often used to implement PQ

Very important distinctions.

85. MEMORY TRICKS

Heap Definition

Complete + Compare Parent

Min-Heap

Minimum sits on the throne.

Max-Heap

Maximum sits on the throne.

Array

Heap has no gaps, so array fits perfectly.

Search

86. 30-SECOND REVISION

Remember:

1. Binary heap = complete binary tree + heap property.
2. Min-heap parent $\leq$ children.
3. Max-heap parent $\geq$ children.
4. Min-heap root = minimum.
5. Max-heap root = maximum.
6. Heap is not fully sorted.
7. Siblings have no required ordering.
8. Heap height = $\lfloor \log_2 n \rfloor$.
9. Heap uses array naturally.
10. Arbitrary-key search can be $O(n)$.

87. ACTIVE RECALL

Q1

What two properties define a binary heap?

Q2

What is guaranteed at root of min-heap?

Q3

Does min-heap require left child $\leq$ right child?

Q4

Why can heap be stored efficiently in array?

Q5

What is arbitrary-key search worst case?

Q6

What is heap height with n nodes?

ANSWERS

A1

Complete binary-tree shape + heap-order property.

A2

Minimum key.

A3

No.

A4

Because complete-tree level-order positions have no gaps.

A5

TEXT

$O(n)$

A6

TEXT

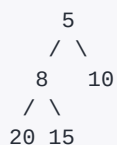
$\text{floor}(\log_2 n)$

under edge-based height.

88. PRACTICE — VALID OR INVALID MIN-HEAP

A

TEXT



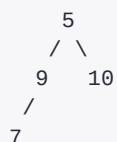
B

TEXT



C

TEXT



For each, check:

1. complete shape
2. heap property

89. PRACTICE — ARRAY FORMULAS

Using 0-based heap array of size 20:

For index:

TEXT

i = 6

find:

1. parent
2. left child
3. right child

Then determine whether child indexes exist.

90. PRACTICE — HEAP STRUCTURE

Heap has:

TEXT

n = 31

Find:

1. height
2. number of leaves
3. number of internal nodes
4. first leaf index in 0-based representation
5. last internal index

91. MODULE 8 PROGRESS

' Lesson 65 — Binary Heap Fundamentals

Remaining:

- Lesson 66 — Array Representation & Index Mastery
- Lesson 67 — Heap Insertion / Bubble Up
- Lesson 68 — Heapify Down, Root Deletion & Extract-Min/Max
- Lesson 69 — Build Heap and Why It Is $O(n)$
- Lesson 70 — Heap Sort
- Lesson 71 — Priority Queue & Heap Applications

NEXT — LESSON 66

Binary Heap Array Representation — Parent/Child Indices, Leaf/Internal Nodes & GATE Numericals

Next we will go deeper into the formulas:

TEXT

0-based:

$L = 2i+1$

$R = 2i+2$

$P = \text{floor}((i-1)/2)$

and solve many GATE-style questions on:

- finding parent/children
- locating leaf nodes
- last internal node
- heap height
- index-to-level relation
- level-to-index ranges
- complete-tree structure
- array validity
- why heaps need no pointers.

USER

next

ASSISTANT

MODULE 8 — LESSON 66

Binary Heap Array Representation — Parent/Child Indices, Leaf/Internal Nodes, Levels & GATE Numericals

In Lesson 65, we learned:

Binary Heap = Complete Binary Tree + Heap Property

Now we go deeper into the reason heaps are so convenient to implement:

A complete binary tree can be stored directly inside an array without explicit child pointers.

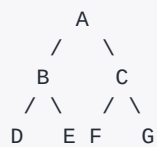
This lesson is heavily formula-oriented and very useful for GATE numericals.

GATE importance: 0=Ý%040%0KNO Placement importance: 0=Ý%040%0KNO

1. CORE IDEA

Consider:

TEXT



Store nodes level-by-level:

TEXT

A B C D E F G

Using 0-based indexing:

TEXT

Index: 0 1 2 3 4 5 6
Value: A B C D E F G

Because the binary tree is **complete**, there are no missing positions before the last occupied position.

That is why an array works perfectly.

2. WHY ORDINARY BINARY TREES CAN WASTE ARRAY SPACE

Consider:

TEXT



Using positional array representation:

TEXT

Index 0 = A
Index 1 = B
Index 3 = C

Index 2 is unused.

For larger skewed trees, huge gaps appear.

But a heap is complete, so occupied positions remain contiguous:

TEXT

0, 1, 2, 3, ..., n-1

Therefore:

Heap completeness eliminates array gaps.

3. 0-BASED INDEX FORMULAS $\emptyset = \text{floor}(\frac{i}{2})$ $\emptyset = \text{floor}(\frac{i}{2})$ $\emptyset = \text{floor}(\frac{i}{2})$

For a node at index i :

FORMULA BOX

TEXT

```
Left child  = 2i + 1
Right child = 2i + 2
Parent      = floor((i - 1)/2)
```

for $i > 0$.

Root:

TEXT

```
i = 0
```

has no parent.

4. MEMORY TRICK — 0 BASED

Think:

TEXT

```
Left  = double + 1
Right = double + 2
```

So:

$$L = 2i+1$$

$$R = 2i+2$$

Parent reverses that relation:

TEXT

```
P = floor((i-1)/2)
```

5. EXAMPLE — INDEX 3

Heap array:

TEXT

```
Index: 0 1 2 3 4 5 6 7 8
```

Take:

TEXT

$i = 3$

Left child:

TEXT

$2(3)+1 = 7$

Right child:

TEXT

$2(3)+2 = 8$

Parent:

TEXT

$\text{floor}((3-1)/2)$
 $= \text{floor}(2/2)$
 $= 1$

So:

TEXT

Parent = 1
Left = 7
Right = 8

6. EXAMPLE — INDEX 4

TEXT

$i = 4$

Parent:

TEXT

$\text{floor}((4-1)/2)$
 $=$
 $\text{floor}(3/2)$
 $=$
 1

Notice indices 3 and 4 have the same parent:

TEXT

1

because they are its left and right children.

7. HOW SIBLING INDICES WORK

For 0-based heap:

If a node is a **left child**, its index is odd:

TEXT

$2p+1$

If a node is a **right child**, index is even and nonzero:

TEXT

$2p+2$

Therefore:

Odd index

TEXT

left child

Even nonzero index

TEXT

right child

Useful GATE shortcut.

8. EXAMPLE

Index:

TEXT

7

Odd.

Therefore it is a:

TEXT

left child

Parent:

TEXT

$\text{floor}((7-1)/2)=3$

Its sibling should be:

TEXT

8

9. INDEX 8

Even and nonzero.

Therefore:

```
TEXT
right child
```

Parent:

```
TEXT
floor((8-1)/2)
=
floor(7/2)
=
3
```

Sibling:

```
TEXT
7
```

10. SIBLING FORMULA — 0 BASED

If index i is odd:

```
TEXT
sibling = i+1
```

If index i is even and $i > 0$:

```
TEXT
sibling = i-1
```

provided that sibling exists inside heap size.

Nice shortcut, but parent formulas are more fundamental.

11. 1-BASED INDEX FORMULAS $\emptyset = \checkmark\%$

If heap array uses:

```
TEXT
1, 2, 3, \dots, n
```

then for index i :

FORMULA BOX

TEXT

```
Left child  = 2i
Right child = 2i+1
Parent      = floor(i/2)
```

This is often algebraically cleaner.

12. EXAMPLE — 1-BASED INDEX 4

TEXT

$i = 4$

Parent:

TEXT

$\text{floor}(4/2)=2$

Left:

TEXT

8

Right:

TEXT

9

13. DON'T MIX INDEXING CONVENTIONS $\emptyset=\text{Y}\%\emptyset=\text{Y}\%\emptyset=\text{Y}\%$

A common mistake:

TEXT

0-based array

but using:

TEXT

$\text{left} = 2i$

That is wrong.

Memorize:

| Indexing | Left | Right | Parent |
|----------|--------|--------|-------------------------|
| 0-based | $2i+1$ | $2i+2$ | $\text{floor}((i-1)/2)$ |
| 1-based | $2i$ | $2i+1$ | $\text{floor}(i/2)$ |

Copy this table.

14. COMPLETE TREE LEVELS

Using root at level 0:

TEXT

```
L e v e l    0    !'    1    p o s s i b l e    n o d e
L e v e l    1    !'    2
L e v e l    2    !'    4
L e v e l    3    !'    8
...
```

Maximum nodes at level L :

FORMULA BOX

TEXT

2^L

Same binary-tree formula applies to heaps.

15. 0-BASED INDEX RANGE OF EACH FULL LEVEL 0-100%

Consider perfect tree positions:

TEXT

```
L e v e l    0    !'    i n d e x    0
L e v e l    1    !'    i n d i c e s    1 . . 2
L e v e l    2    !'    i n d i c e s    3 . . 6
L e v e l    3    !'    i n d i c e s    7 . . 14
```

Pattern:

FORMULA BOX

For level L :

TEXT

```
First index =  $2^L - 1$ 
Last index  =  $2^{(L+1)} - 2$ 
```

in 0-based representation.

Very useful numerical formula.

16. VERIFY LEVEL 3

First:

TEXT

$$2^3 - 1 = 7$$

Last:

TEXT

$$\begin{aligned} 2^4 - 2 \\ = 16 - 2 \\ = 14 \end{aligned}$$

So level 3 indices:

TEXT

7, 8, 9, 10, 11, 12, 13, 14

Exactly 8 nodes.

17. 1-BASED LEVEL INDEX RANGE

For level L with root at level 0:

TEXT

$$\begin{aligned} \text{First index} &= 2^L \\ \text{Last index} &= 2^{(L+1)} - 1 \end{aligned}$$

Example level 3:

TEXT

8 ... 15

Much cleaner in 1-based indexing.

18. FIND LEVEL FROM INDEX — 0 BASED Ø=Ý%

For node at 0-based index i :

FORMULA BOX

TEXT

$$\text{Level} = \text{floor}(\log_2(i+1))$$

Example:

TEXT

$$i = 10$$

Then:

TEXT

```
floor(log2(11))
```

Since:

TEXT

```
8 "d 1 1 < 1 6
```

level:

TEXT

```
3
```

19. WHY `i+1` ?

Because converting 0-based index `i` to equivalent 1-based position gives:

TEXT

```
i+1
```

And in 1-based heap:

TEXT

```
level = floor(log2 i)
```

So 0-based becomes:

TEXT

```
floor(log2(i+1))
```

20. EXAMPLE — INDEX 6

TEXT

```
i=6
```

Level:

TEXT

```
floor(log2 7)
```

Since:

TEXT

```
4 "d 7 < 8
```

answer:

TEXT

2

Check level 2 indices:

TEXT

3, 4, 5, 6

Correct.

21. HEAP HEIGHT $\emptyset = \text{floor}(\log_2 n)$

For n nodes:

FORMULA BOX

TEXT

$h = \text{floor}(\log_2 n)$

because heap is complete.

This uses edge-based height/root level 0.

22. EXAMPLE — $n = 31$

TEXT

$\text{floor}(\log_2 31)$

Since:

TEXT

$16 < 31 < 32$

height:

TEXT

4

And indeed:

TEXT

$31 = 2^5 - 1$

is a perfect tree of height 4.

23. EXAMPLE — $n = 32$

TEXT

$\text{floor}(\log_2 32)=5$

The first node has appeared on level 5.

Therefore height becomes:

TEXT

5

24. NODE RANGE FOR HEAP HEIGHT h

Because complete tree levels before the last must be full:

For heap height h :

FORMULA BOX

TEXT

$2^h \leq n < 2^{h+1}$

Same complete-tree formula.

25. EXAMPLE — HEIGHT 5

Minimum nodes:

TEXT

$2^5 = 32$

Maximum:

TEXT

$2^6 - 1$
 $= 63$

So a heap of height 5 can have:

TEXT

$32 \leq n < 64$

26. NUMBER OF NODES BEFORE LAST LEVEL

If heap height is h , all levels:

TEXT

$0 \dots h-1$

are full.

Total nodes there:

TEXT

$1+2+4+\dots+2^{(h-1)}$

Using geometric sum:

FORMULA BOX

TEXT

$2^h - 1$

Therefore nodes on last level:

FORMULA BOX

TEXT

$n - (2^h - 1)$

27. EXAMPLE

Heap has:

TEXT

$n=20$

Height:

TEXT

$\text{floor}(\log_2 20)=4$

Nodes before level 4:

TEXT

$2^4 - 1 = 15$

Nodes on final level:

TEXT

$20 - 15 = 5$

So last level contains:

5 nodes

28. FIRST LEAF INDEX — 0 BASED $\emptyset = \text{floor}(n/2)$

For heap with n nodes:

FORMULA BOX

TEXT

First leaf index = $\text{floor}(n/2)$

Leaves occupy:

TEXT

$\text{floor}(n/2), \dots, n-1$

29. LAST INTERNAL NODE — 0 BASED

FORMULA BOX

TEXT

Last internal index = $\text{floor}(n/2)-1$

provided $n \neq 2$

This index becomes extremely important for:

Build Heap

because all later nodes are already leaves.

30. WHY FIRST LEAF = $\text{floor}(n/2)$?

A node at index i is internal if it has at least a left child.

Left child exists when:

TEXT

$2i+1 < n$

Therefore:

TEXT

$2i < n-1$

and internal indices end at approximately:

TEXT

$\text{floor}(n/2) - 1$

Everything after that has no children.

31. EXAMPLE — $n = 10$

Indices:

TEXT

0 1 2 3 4 5 6 7 8 9

First leaf:

TEXT

$\text{floor}(10/2) = 5$

Leaves:

TEXT

5, 6, 7, 8, 9

Last internal:

TEXT

4

32. CHECK INDEX 4

Left:

TEXT

$2(4) + 1 = 9$

exists.

Right:

TEXT

10

does not exist.

So index 4 is indeed the final internal node.

33. CHECK INDEX 5

Left:

TEXT

$2(5)+1=11$

outside array size 10.

Therefore index 5 is leaf.

Correct.

34. ODD n CASE

Let:

TEXT

$n=9$

First leaf:

TEXT

$\text{floor}(9/2)=4$

Leaves:

TEXT

4, 5, 6, 7, 8

Last internal:

TEXT

3

Index 3:

TEXT

left = 7
right = 8

both exist.

35. EVEN n SPECIAL PROPERTY Ø=Ý%

For a complete binary tree with even n:

There is exactly one node with one child.

In 0-based representation:

TEXT

last internal node = $n/2 - 1$

and it has:

TEXT

left child only

because final node occupies the left slot.

Example $n=10$:

Last internal index:

TEXT

4

Left child:

TEXT

9

Right:

TEXT

10

absent.

36. ODD n PROPERTY

For odd $n > 1$, every internal node has:

TEXT

2 children

in a complete binary tree.

Example:

TEXT

$n=9$

last internal index 3:

TEXT

children 7 and 8

both exist.

$\lceil \frac{n}{2} \rceil$ is GATE property.

37. NUMBER OF LEAVES $\lceil \frac{n}{2} \rceil$

For heap with n nodes:

FORMULA BOX

TEXT

$\text{Leaves} = \text{ceil}(n/2)$

Internal nodes:

TEXT

$\text{floor}(n/2)$

38. EXAMPLE — $n=10$

Leaves:

TEXT

$\text{ceil}(10/2)=5$

Internal:

TEXT

$\text{floor}(10/2)=5$

39. EXAMPLE — $n=11$

Leaves:

TEXT

$\text{ceil}(11/2)=6$

Internal:

TEXT

$\text{floor}(11/2)=5$

Total:

TEXT

11

40. 1-BASED LEAF RANGE

If heap uses indices:

TEXT

$1 \dots n$

then:

Internal nodes:

TEXT
 $1 \dots \text{floor}(n/2)$

Leaves:

TEXT
 $\text{floor}(n/2)+1 \dots n$

First leaf:

TEXT
 $\text{floor}(n/2)+1$

41. COMPLETE HEAP ARRAY CONTAINS CONTIGUOUS INDICES

If heap has n elements:

0-based valid indexes are exactly:

TEXT
 $0 \dots n-1$

No gap like:

TEXT
 $0, 1, 2, 4$

is allowed in structural representation.

A missing positional node before $n-1$ would violate completeness.

42. HEAP ARRAY VS HEAP PROPERTY

Important:

An array automatically gives a **complete-tree shape** if interpreted through consecutive indices.

But it is only a heap if values also satisfy:

Min-heap

TEXT
 $A[\text{parent}] \geq A[\text{child}]$

Max-heap

TEXT

A [p a r e n t] " e A [c h i l d]

So:

Contiguous array !' complete shape

Value comparisons !' heap order

43. CHECKING MIN-HEAP ARRAY

Array:

TEXT

[4, 8, 7, 15, 10, 20, 9]

Indices:

TEXT

0 1 2 3 4 5 6

Only internal nodes need checking.

n=7

Last internal:

TEXT

$\text{floor}(7/2)-1$
 $=3-1$
 $=2$

So inspect indices:

TEXT

0, 1, 2

44. CHECK INDEX 0

Value:

TEXT

4

Children:

TEXT

8, 7

Check:

TEXT

4"d 8 ' '
4"d 7 ' '

45. CHECK INDEX 1

Value:

TEXT

8

Children:

TEXT

15,10

Valid.

46. CHECK INDEX 2

Value:

TEXT

7

Children:

TEXT

20,9

Valid.

Therefore:

Valid min-heap

47. HEAP CHECK ALGORITHM

For min-heap:

```
c
int isMinHeap(int a[], int n)
{
    for (int i = 0; i < n / 2; i++)
    {
        int left = 2 * i + 1;
        int right = 2 * i + 2;

        if (left < n && a[i] > a[left])
```

```

        return 0;

    if (right < n && a[i] > a[right])
        return 0;
}

return 1;
}

```

Time:

TEXT
 $\sim (n)$

Extra space:

TEXT
 $O(1)$

48. WHY CHECK ONLY INTERNAL NODES?

Leaf has no children.

A single leaf by itself automatically satisfies heap property.

Therefore only:

TEXT
 $\text{floor}(n/2)$

internal nodes need checking.

Still:

TEXT
 $\sim (n)$

asymptotically.

49. GATE INDEX NUMERICAL 1

0-based heap.

Node index:

TEXT
 $i=12$

Find parent.

TEXT

```
floor((12-1)/2)
=
floor(11/2)
=
5
```

So parent index:

5

50. GATE INDEX NUMERICAL 2

At $i=12$:

Left child:

TEXT

25

Right child:

TEXT

26

If heap size is only:

TEXT

$n=20$

neither exists.

Therefore node 12 is a:

TEXT

leaf

51. GATE INDEX NUMERICAL 3

Heap size:

TEXT

$n=20$

First leaf:

TEXT

```
floor(20/2)
=10
```

Therefore indices:

```
TEXT
10...19
```

are leaves.

Internal:

```
TEXT
0...9
```

52. LAST INTERNAL INDEX

For $n=20$:

```
TEXT
floor(20/2)-1
=
10-1
=
9
```

Index 9 has:

```
TEXT
left = 19
right = 20
```

Only left exists.

Correct because n is even.

53. GATE LEVEL NUMERICAL

Find level of 0-based index 25.

Formula:

```
TEXT
floor(log2(25+1))
=
floor(log2 26)
```

Since:

```
TEXT
1 6 "d 2 6 < 3 2
```

level:

54. VERIFY USING INDEX RANGE

Level 4 indices:

TEXT

$2^4 - 1$
to
 $2^5 - 2$

TEXT

15 ... 30

Index 25 lies there.

Correct.

55. GATE QUESTION — HOW MANY NODES ON LAST LEVEL?

Heap:

TEXT

$n = 50$

Height:

TEXT

$\text{floor}(\log_2 50) = 5$

Nodes before last level:

TEXT

$2^5 - 1$
 $= 31$

Last-level nodes:

TEXT

$50 - 31$
 $= 19$

Answer = 19

56. LAST LEVEL CAPACITY

At height 5, maximum nodes at level 5:

TEXT

$2^5=32$

Currently:

TEXT

19

occupied.

Therefore remaining free positions before height increases:

TEXT

$32-19$

$=13$

57. WHEN DOES HEAP HEIGHT INCREASE?

Current height h .

Maximum nodes at this height:

TEXT

$2^{(h+1)} - 1$

The next insertion after reaching this number creates a new level.

Example height 3:

Maximum:

TEXT

$2^4 - 1 = 15$

At $n=15$:

TEXT

$h=3$

Insert 16th node:

TEXT

$h=4$

$O(\log n)$ Useful.

58. LOCATION OF NEXT INSERTION

If current heap size is n , using 0-based indexing:

New element is initially placed at:

TEXT

$\text{index} = n$

because existing elements occupy:

TEXT

$0 \dots n-1$

Its parent:

FORMULA BOX

TEXT

$\text{floor}((n-1)/2)$

because new node index is n .

59. EXAMPLE

Current heap has:

TEXT

$n=10$

New element enters index:

TEXT

10

Parent:

TEXT

$\text{floor}((10-1)/2)$
=
4

This will matter in **bubble-up** insertion.

60. NEXT INSERTION POSITION IS FIXED

Suppose:

TEXT



Array:

TEXT

A B C D E F

Current $n=6$.

New node goes to:

TEXT

index 6

which is:

TEXT

right child of C

because:

TEXT

parent = floor((6-1)/2)=2

61. PARENT/CHILD EXISTENCE TESTS — 0 BASED

For node i and heap size n :

Left exists if:

TEXT

$2i+1 < n$

Right exists if:

TEXT

$2i+2 < n$

Parent exists if:

TEXT

$i > 0$

~~0=Very~~ useful in code.

62. LEAF TEST — 0 BASED

Node i is leaf if:

TEXT

$2i+1 \geq n$

because if no left child exists, no child can exist in a complete binary tree.

Equivalent:

TEXT

```
i >= floor(n/2)
```

63. INTERNAL NODE TEST

Node is internal if:

TEXT

```
i < floor(n/2)
```

for 0-based heap.

Simple.

64. EXAMPLE — $n=12$

TEXT

```
floor(12/2)=6
```

Internal indices:

TEXT

```
0, 1, 2, 3, 4, 5
```

Leaves:

TEXT

```
6, 7, 8, 9, 10, 11
```

65. HOW MANY ANCESTORS CAN A NODE HAVE?

A node at level L has:

TEXT

```
L
```

proper ancestors.

Because root level 0.

So index i has level:

TEXT

```
floor(log2(i+1))
```

Therefore number of proper ancestors:

TEXT

$\text{floor}(\log_2(i+1))$

$\emptyset = 0$ formula.

66. EXAMPLE

Index:

TEXT

14

Level:

TEXT

$\text{floor}(\log_2 15)=3$

So it has:

TEXT

3

proper ancestors.

Path length from root:

TEXT

3 edges

67. FIND ANCESTORS USING PARENT FORMULA

Index 14:

TEXT

$\text{parent} = \text{floor}(13/2)=6$

Parent of 6:

TEXT

$\text{floor}(5/2)=2$

Parent of 2:

TEXT

$\text{floor}(1/2)=0$

So path:

TEXT

0 !' 2 !' 6 !' 1 4

Three edges.

68. RELATION TO BINARY REPRESENTATION

1-based heap indices have an elegant property.

Suppose index:

TEXT

13

Binary:

TEXT

1101

Ignore leading 1.

Remaining bits:

TEXT

101

Interpret:

TEXT

0 !' l e f t
1 !' r i g h t

Starting from root.

Thus path:

TEXT

R i g h t !' L e f t !' R i g h t

This is an advanced complete-tree indexing insight.

Not essential for basic GATE, but useful.

69. WHY IT WORKS

For 1-based indexing:

TEXT

left child = $2i$

which appends binary bit:

TEXT

0

and:

TEXT

right child = $2i+1$

which appends:

TEXT

1

So binary index itself encodes the root-to-node path.

Ø = Elegant connection.

70. ARRAY MEMORY REQUIREMENT

For n heap elements stored in an array:

Ignoring resizing overhead:

TEXT

$\sim (n)$

memory.

No:

TEXT

$2n$ pointer fields

as in linked binary-tree nodes.

So heap arrays are compact.

71. STATIC VS DYNAMIC ARRAY

Simple C teaching implementation may use:

C

```
int heap[100];  
int size;
```

This has fixed capacity.

Real implementation may use:

TEXT

dynamic array

that resizes when full.

The heap logic remains unchanged.

72. HEAP'S LOGICAL SIZE VS ARRAY CAPACITY

Important distinction:

TEXT

```
capacity = allocated slots  
size      = number of active heap elements
```

Example:

TEXT

```
capacity = 100  
size     = 17
```

Heap indices only:

TEXT

```
0...16
```

are currently meaningful.

Do not apply child existence based on capacity.

Use:

heap size

Ø = implementation trap.

73. EXAMPLE

Capacity:

TEXT

```
20
```

Heap size:

TEXT

```
7
```

Node index 3:

Left:

TEXT

```
7
```

Although index 7 is inside allocated capacity, it is **not** an existing heap child because:

TEXT

```
7 >= size
```

So child does not exist.

Do not compare against `size`, not array capacity.

74. GATE TRAP — ARRAY SORTING

Min-heap array:

TEXT

```
[1, 4, 2, 9, 7, 8, 3]
```

Can contain:

TEXT

```
4 > 2
```

between neighboring array positions.

Still valid.

Do not require:

TEXT

```
A [ i ] "d A [ i + 1 ]
```

75. GATE TRAP — PARENT VS ALL DESCENDANTS

In min-heap, parent is "dits immediate children.

By transitivity, it is also "dall descendants.

But:

A node has no guaranteed ordering against nodes outside its ancestor/descendant chain.

Example siblings/subtrees may be in any order.

76. GATE QUESTION — MIN-HEAP ARRAY

Given:

TEXT

```
[10, 30, 20, 40, 50, 25, 35]
```

Valid?

Internal indices for $n=7$:

TEXT

0, 1, 2

Index 0:

TEXT

1 0"d 3 0 , 2 0 ' '

Index 1:

TEXT

3 0"d 4 0 , 5 0 ' '

Index 2:

TEXT

2 0"d 2 5 , 3 5 ' '

Therefore:

Valid Min-Heap

77. GATE QUESTION — INVALID ARRAY

TEXT

[10, 30, 20, 40, 15, 25, 35]

Index 1:

TEXT

30

right child index 4:

TEXT

15

But:

TEXT

3 0 "d 1 5 ' 'L

Therefore invalid.

78. GATE NUMERICAL — $n=100$

Find:

Height

TEXT

$\text{floor}(\log_2 100)=6$

Leaves

TEXT

$\text{ceil}(100/2)=50$

Internal

TEXT

$\text{floor}(100/2)=50$

First leaf 0-based

TEXT

50

Last internal

TEXT

49

Last index

TEXT

99

Ø=Excellent formula practice.

79. GATE NUMERICAL — n=101

Height:

TEXT

$\text{floor}(\log_2 101)=6$

Leaves:

TEXT

$\text{ceil}(101/2)=51$

Internal:

TEXT

50

First leaf:

TEXT

50

Last internal:

TEXT

49

Notice first leaf index stays 50.

80. INTERNAL NODE COUNT RELATION

In a complete binary tree:

TEXT

$\text{internal} = \text{floor}(n/2)$

This includes nodes with:

- 2 children
- possibly the one left-only child when n even

Leaves:

TEXT

$\text{ceil}(n/2)$

81. NUMBER OF TWO-CHILD NODES

Useful derived property.

If n is odd

No one-child node.

So:

TEXT

$n_1 = 0$

Internal nodes all have 2 children:

TEXT

$n_2 = \text{floor}(n/2)$

If n is even

Exactly one one-child node:

TEXT

$$n_1 = 1$$

Then:

TEXT

$$n_2 = \text{floor}(n/2) - 1$$

0-100% advanced but GATE-useful.

82. CHECK n=10

Internal:

TEXT

5

One-child nodes:

TEXT

1

Two-child:

TEXT

4

Leaves:

TEXT

5

Check universal relation:

TEXT

$$n_0 = n_2 + 1$$

TEXT

$$5 = 4 + 1$$

Correct.

83. CHECK n=9

Internal:

TEXT

4

No one-child node.

Two-child:

TEXT

4

Leaves:

TEXT

5

Again:

TEXT

5 = 4+1

Correct.

84. HEAP INDEX FORMULA MASTER SHEET

0-based

TEXT

Parent(i) = floor((i-1)/2)
Left(i) = 2i+1
Right(i) = 2i+2

TEXT

First leaf = floor(n/2)
Last internal = floor(n/2)-1

TEXT

Level(i) = floor(log2(i+1))

Level L range:

TEXT

$2^L - 1 \dots 2^{L+1} - 2$

85. 1-BASED MASTER SHEET

TEXT

```
Parent(i) = floor(i/2)
Left(i)   = 2i
Right(i)  = 2i+1
```

Internal:

TEXT

```
1 ... floor(n/2)
```

Leaves:

TEXT

```
floor(n/2)+1 ... n
```

Level L :

TEXT

```
 $2^L \dots 2^{(L+1)}-1$ 
```

86. STRUCTURAL FORMULA MASTER SHEET

For heap with n elements

TEXT

```
height = floor(log2 n)
```

TEXT

```
leaves = ceil(n/2)
```

TEXT

```
internal = floor(n/2)
```

For height h :

TEXT

```
 $2^h \leq n < 2^{h+1}$ 
```

Last-level occupied nodes:

TEXT

```
 $n - (2^h - 1)$ 
```

Copy this entire box.

87. COMPLEXITY CONNECTION

Why insertion and extraction can be $O(\log n)$:

Heap height:

TEXT

$\text{floor}(\log_2 n)$

An element can move at most:

TEXT

one root-to-leaf path

So:

TEXT

$O(\log n)$

We'll use this directly in the next lessons.

88. WHY HEAP INSERTION STARTS AT END

Completeness says new node must occupy:

TEXT

next level-order slot

Array representation says that slot is simply:

TEXT

$\text{heap}[\text{size}]$

Then we only need to repair ordering.

This is much simpler than searching for a structural insertion location.

~~$O(\log n)$~~ Very important heap intuition.

89. WHY EXTRACT ROOT USES LAST ELEMENT

When root is removed, putting the last node at root:

- removes the last occupied position
- keeps array indices contiguous
- preserves complete-tree shape

Then only heap-order property needs repair.

This is the foundation of extract-min/max.

90. 30-SECOND REVISION

Remember:

1. Heap array is level-order storage.

2. 0-based: $L=2i+1$, $R=2i+2$, $P=\text{floor}((i-1)/2)$.
 3. 1-based: $L=2i$, $R=2i+1$, $P=\text{floor}(i/2)$.
 4. Heap height = $\text{floor}(\log_2 n)$.
 5. First leaf 0-based = $\text{floor}(n/2)$.
 6. Last internal = $\text{floor}(n/2)-1$.
 7. Leaves = $\text{ceil}(n/2)$.
 8. Internal nodes = $\text{floor}(n/2)$.
 9. New insertion starts at index n .
 10. Always compare child indices against heap **size**, not capacity.
-

91. ACTIVE RECALL

For a 0-based heap with:

TEXT

$n=18$

Q1

What is its height?

Q2

First leaf index?

Q3

Last internal index?

Q4

How many leaves?

Q5

For index 5, parent?

Q6

For index 5, left and right children?

ANSWERS

A1

TEXT

$\text{floor}(\log_2 18)=4$

A2

TEXT

$\text{floor}(18/2)=9$

A3

TEXT

8

A4

TEXT

$\text{ceil}(18/2)=9$

A5

TEXT

$\text{floor}((5-1)/2)=2$

A6

TEXT

left = 11
right = 12

Both exist because:

TEXT

11, 12 < 18

92. PRACTICE — GATE NUMERICAL 1

A 0-based heap contains:

TEXT

$n = 37$

Find:

1. height
2. first leaf index
3. last internal index
4. number of leaves
5. number of internal nodes
6. number of nodes on last level

93. PRACTICE — GATE NUMERICAL 2

For 0-based index:

TEXT

i = 19

find:

1. parent
2. left child
3. right child
4. level

If heap size is 35, identify which children exist.

94. PRACTICE — GATE NUMERICAL 3

A heap has height:

TEXT

h=6

Find:

1. minimum possible number of nodes
2. maximum possible number of nodes
3. maximum possible nodes on final level
4. 0-based index range corresponding to level 6 in a perfect positional array

95. PRACTICE — VALIDATE ARRAYS

Which are valid min-heaps?

A

TEXT

[2, 5, 4, 9, 8, 7, 6]

B

TEXT

[2, 5, 4, 9, 1, 7, 6]

C

TEXT

[1, 3, 2, 7, 6, 5, 4]

Check only necessary internal nodes.

96. MODULE 8 PROGRESS

Remaining:

- Lesson 67 — Heap Insertion / Bubble Up
- Lesson 68 — Heapify Down, Deletion & Extract-Min/Max
- Lesson 69 — Build Heap and Why It Is $O(n)$
- Lesson 70 — Heap Sort
- Lesson 71 — Priority Queue & Applications

NEXT — LESSON 67

Heap Insertion — Add at End, Bubble Up / Sift Up, Min-Heap & Max-Heap Tracing

Next we will insert values like:

TEXT

Min-heap:
[10, 20, 15, 30, 40, 25]

then insert:

TEXT

5

and trace:

TEXT

```
Place at end
!' compare with parent
!' swap
!' compare again
!' reach correct position
```

We will cover recursive/iterative C logic, exact index calculations, number of swaps/comparisons, $O(\log n)$ proof, and all common GATE insertion-sequence questions.

USER

next

ASSISTANT

MODULE 8 — LESSON 67

Heap Insertion — Add at End, Bubble Up / Sift Up, Min-Heap & Max-Heap Tracing

Now we

next

MODULE 8 — LESSON 67

We now study one of the most important heap operations:

The central idea is:

2. Repair heap order second

For a binary heap, insertion always happens in two stages:

Swap upward if heap property is violated

This upward repair is called:

- **Bubble Up**
- **Sift Up**
- **Percolate Up**
- **Heapify Up**

All mean essentially the same thing.

GATE importance: $\emptyset = \gamma\%$ MUST KNOW Placement importance: $\emptyset = \gamma\%$ MUST KNOW

Suppose min-heap:

$$\begin{array}{cc} & 10 \\ & / \quad \backslash \\ 20 & & 15 \\ / \quad \backslash \end{array}$$

30 40

Array:

TEXT

[10, 20, 15, 30, 40]

If we insert another value, the complete-tree shape dictates exactly where the new node must initially go.

Next free position:

TEXT

```
      10
     /  \
    20   15
   / \  /
  30 40 ?
```

So in the array:

TEXT

new index = current size

Current size:

TEXT

5

New element goes to:

TEXT

index 5

~~0-4~~ We do **not** search for a convenient tree location.

Completeness fixes it automatically.

2. CORE INSERTION ALGORITHM — MIN-HEAP

Suppose we insert `x`.

Steps:

TEXT

1. Put `x` at end of heap array.
2. Let `i` = index of `x`.
3. While `i > 0`:
 `parent = floor((i-1)/2)`

 if `heap[parent] <= heap[i]`:
 stop

```
swap heap[parent], heap[i]

i = parent
```

This restores:

```
TEXT
P a r e n t   " d   C h i l d
```

for min-heap.

3. MEMORY TRICK

ADD AT END !' CLIMB TO FRIENDLY PARENT

For a min-heap:

If new value is smaller than parent, it climbs.

For a max-heap:

If new value is larger than parent, it climbs.

4. SIMPLE MIN-HEAP INSERTION

Heap:

```
TEXT
      10
     /  \
    20   15
   /  \
  30  40
```

Array:

```
TEXT
[10, 20, 15, 30, 40]
```

Insert:

```
TEXT
25
```

5. STEP 1 — PLACE AT END

New index:

TEXT

5

Array:

TEXT

[10, 20, 15, 30, 40, 25]

Tree:

TEXT

```
      10
     /  \
    20   15
   / \  /
  30 40 25
```

Parent of index 5:

TEXT

```
floor((5-1)/2)
=
2
```

Parent value:

TEXT

15

Check min-heap:

TEXT

```
1 5 "d 2 5
```

True.

So:

STOP

Final heap:

TEXT

[10, 20, 15, 30, 40, 25]

No swap needed.

6. INSERTION WITH ONE SWAP

Start:

TEXT

[10, 20, 15, 30, 40, 25]

Insert:

TEXT

12

New index:

TEXT

6

Array temporarily:

TEXT

[10, 20, 15, 30, 40, 25, 12]

Tree:

TEXT

```
      10
     /  \
    20   15
   / \  / \
  30 40 25 12
```

7. CHECK PARENT

Index:

TEXT

i=6

Parent:

TEXT

```
floor((6-1)/2)
=
2
```

Parent value:

TEXT

15

But:

TEXT

1 5 "d 1 2

False.

So swap:

```
TEXT
1 5  !"  1 2
```

Array becomes:

```
TEXT
[10, 20, 12, 30, 40, 25, 15]
```

Now new element 12 is at index:

```
TEXT
2
```

8. CHECK AGAIN

Parent of index 2:

```
TEXT
floor((2-1)/2)
=
0
```

Parent:

```
TEXT
10
```

Check:

```
TEXT
1 0  "d 1 2
```

True.

Stop.

Final min-heap:

```
TEXT
[10, 20, 12, 30, 40, 25, 15]
```

9. INSERTION WITH MULTIPLE SWAPS Ø=Ý%Ø=Ý%Ø=Ý%

Start:

TEXT

[10, 20, 15, 30, 40, 25]

Insert:

TEXT

5

Place at index:

TEXT

6

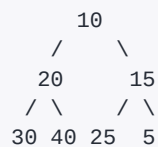
Temporary:

TEXT

[10, 20, 15, 30, 40, 25, 5]

Tree:

TEXT



10. FIRST COMPARISON

New value:

TEXT

5

Parent index:

TEXT

$\text{floor}((6-1)/2)=2$

Parent value:

TEXT

15

Violation:

TEXT

15 > 5

Swap.

Array:

TEXT

[10, 20, 5, 30, 40, 25, 15]

11. SECOND COMPARISON

Now 5 is at index:

TEXT

2

Parent:

TEXT

0

Value:

TEXT

10

Violation:

TEXT

10 > 5

Swap.

Array:

TEXT

[5, 20, 10, 30, 40, 25, 15]

Now 5 is root.

Stop.

Final tree:

TEXT

```
      5
     / \
    20  10
   / \ / \
  30 40 25 15
```

Valid min-heap.

12. IMPORTANT OBSERVATION

During insertion, only one path can violate heap order:

The path from the newly inserted node to the root

Why?

Before insertion:

TEXT

old heap was already valid

The only new relationship introduced is between:

TEXT

new node
and its ancestors

So no need to check every node.

∅=Y% This is why insertion is efficient.

13. WHY SWAPPING DOESN'T BREAK LOWER SUBTREE

Suppose smaller value x swaps with its parent p in a min-heap.

Before:

TEXT

p
/
x

with:

TEXT

$x < p$

After:

TEXT

x
/
p

What about p 's new children?

The subtree below was already a valid heap before.

Because p was "at those descendants, putting p lower still preserves those relevant comparisons.

The only potential violation moves upward with x .

∅=Y% This is the invariant behind bubble-up.

14. MIN-HEAP INSERTION RULE

For min-heap:

While:

TEXT

```
new value < parent value
```

swap upward.

Stop when:

TEXT

```
parent "d" new value
```

or when new value reaches root.

15. C IMPLEMENTATION— MIN-HEAP INSERT

```
c
void insertMinHeap(int heap[], int *size, int value)
{
    int i = *size;

    heap[i] = value;
    (*size)++;

    while (i > 0)
    {
        int parent = (i - 1) / 2;

        if (heap[parent] <= heap[i])
            break;

        int temp = heap[parent];
        heap[parent] = heap[i];
        heap[i] = temp;

        i = parent;
    }
}
```

Very important implementation.

16. TRACE THE CODE

Suppose:

TEXT

```
heap = [10, 20, 15, 30, 40, 25]
size = 6
```

Insert 5.

Initially:

```
C
i = *size;
```

SO:

```
TEXT
i=6
```

Then:

```
C
heap[6]=5;
```

and:

```
TEXT
size becomes 7
```

Bubble-up begins.

17. WHY $i > 0$?

Root index:

```
TEXT
0
```

has no parent.

So if:

```
TEXT
i==0
```

we stop.

Without this check, expression:

```
C
(i-1)/2
```

would not represent a valid parent conceptually.

18. MAX-HEAP INSERTION $\emptyset = \text{YES}$

Same structural process.

Difference:

Max-heap requires:

TEXT

P a r e n t " e c h i l d r e n

So new element bubbles upward if:

TEXT

new value > parent

19. MAX-HEAP EXAMPLE

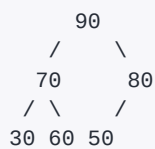
Start:

TEXT

[90, 70, 80, 30, 60, 50]

Tree:

TEXT



Insert:

TEXT

85

At end:

TEXT

[90, 70, 80, 30, 60, 50, 85]

20. FIRST COMPARISON

New index:

TEXT

6

Parent:

TEXT

2

Parent value:

TEXT

80

Since:

TEXT

85 >80

swap:

TEXT

[90, 70, 85, 30, 60, 50, 80]

Now 85 index:

TEXT

2

Parent:

TEXT

0

Value 90.

Check:

TEXT

85 >90

false.

Stop.

Final:

TEXT

[90, 70, 85, 30, 60, 50, 80]

21. MAX-HEAP C CODE

```
C
void insertMaxHeap(int heap[], int *size, int value)
{
    int i = *size;

    heap[i] = value;
    (*size)++;

    while (i > 0)
    {
        int parent = (i - 1) / 2;
```

```

        if (heap[parent] >= heap[i])
            break;

        int temp = heap[parent];
        heap[parent] = heap[i];
        heap[i] = temp;

        i = parent;
    }
}

```

Only comparison direction changes.

22. INSERTION COMPLEXITY $O(\log n)$

Initially inserted node goes to last level.

In worst case, it may climb all the way to root.

Maximum number of levels:

```

TEXT
h = floor(log2 n)

```

Therefore:

FORMULA BOX

```

TEXT
Heap insertion =  $O(\log n)$ 

```

23. BEST CASE

If new element already satisfies heap property with parent:

No swaps.

Example min-heap:

```

TEXT
parent=20
new=50

```

Then:

```

TEXT
2 0" d 5 0

```

Stop immediately.

Best case:

$O(1)$

for bubble-up work.

24. WORST CASE

For min-heap, inserting a new smallest element:

TEXT

new < every ancestor

causes it to reach root.

Number of swaps at most:

TEXT

height of new node

which is:

TEXT

$O(\log n)$

25. EXACT MAXIMUM SWAPS

Suppose after insertion heap size becomes:

TEXT

N

New node index:

TEXT

N-1

Its level:

TEXT

$\text{floor}(\log_2 N)$

So maximum number of upward parent transitions/swaps is:

FORMULA BOX

TEXT

$\text{floor}(\log_2 N)$

if the new item climbs all the way to root.

26. EXAMPLE

Heap has 15 elements.

Insert 16th element.

New node goes to index:

TEXT

15

Level:

TEXT

$\text{floor}(\log_2(15+1))$

=

4

Maximum upward swaps:

TEXT

4

27. INSERTION ARRAY SIZE BEFORE VS AFTER

Be precise.

If old heap has:

TEXT

n

elements:

New index:

TEXT

n

After insertion new size:

TEXT

n+1

Maximum height after insertion:

TEXT

$\text{floor}(\log_2(n+1))$

28. GATE TRAP — START AT ROOT?

Heap insertion does **not** start searching from root for a position.

Structural position is already known:

TEXT

next array slot

Then heap order is repaired upward.

Contrast with BST insertion.

29. BST INSERTION VS HEAP INSERTION

BST

Structural location is found by comparing keys:

TEXT

search downward

Shape depends on key order.

Heap

Structural location is fixed by completeness:

TEXT

append at end

Then key moves upward as needed.

Excellent distinction.

30. HEAP INSERTION DOES NOT FULLY SORT ARRAY

Start min-heap:

TEXT

[10, 20, 15, 30, 40, 25]

Insert 12.

Final:

TEXT

[10, 20, 12, 30, 40, 25, 15]

This is not sorted:

TEXT

20 >12

But valid heap.

Remember:

Bubble-up restores heap order, not total sorting.

31. GATE-STYLE INSERTION SEQUENCE

Insert into empty min-heap:

TEXT

20, 15, 30, 5, 10

Let's build it.

32. INSERT 20

TEXT

[20]

33. INSERT 15

Append:

TEXT

[20, 15]

Compare 15 with parent 20.

Swap:

TEXT

[15, 20]

34. INSERT 30

Append:

TEXT

[15, 20, 30]

Parent 15.

TEXT

1 5"d 3 0

No swap.

35. INSERT 5

Append:

TEXT

[15, 20, 30, 5]

Index 3.

Parent index 1:

TEXT

20

Swap:

TEXT

[15, 5, 30, 20]

Now index 1.

Parent 0:

TEXT

15

Swap:

TEXT

[5, 15, 30, 20]

36. INSERT 10

Append:

TEXT

[5, 15, 30, 20, 10]

Parent index 1:

TEXT

15

Swap:

TEXT

[5,10,30,20,15]

Parent of index 1 is root 5.

TEXT

5"d 1 0

Stop.

Final:

[5,10,30,20,15]

37. TREE REPRESENTATION

TEXT

```
      5
     / \
    10  30
   /  \
  20  15
```

Check:

TEXT

```
5"d 1 0 , 3 0
1 0"d 2 0 , 1 5
```

Valid.

38. IMPORTANT GATE POINT — RESULT DEPENDS ON INSERTION ORDER

Same values:

TEXT

5,10,15,20,30

can produce different valid heaps depending on insertion sequence.

Heap property does not uniquely determine arrangement.

Not like many key sets can produce multiple BSTs/heaps.

39. EXAMPLE

Sequence A:

TEXT

20, 15, 30, 5, 10

gave:

TEXT

[5, 10, 30, 20, 15]

But inserting:

TEXT

5, 10, 15, 20, 30

gives:

TEXT

[5, 10, 15, 20, 30]

Both valid min-heaps.

40. HOW MANY PARENTS ARE CHECKED?

At each iteration:

TEXT

```
i != parent(i)
```

approximately halves index.

For 0-based:

TEXT

```
parent "H i / 2"
```

So index decreases geometrically:

TEXT

```
n
n/2
n/4
n/8
...
1
```

Number of steps:

TEXT

```
~ (log n)
```

Another proof of insertion complexity.

41. RECURSIVE BUBBLE-UP

It can also be written recursively.

```
C
void siftUp(int heap[], int i)
{
    if (i == 0)
        return;

    int parent = (i - 1) / 2;

    if (heap[parent] > heap[i])
    {
        int temp = heap[parent];
        heap[parent] = heap[i];
        heap[i] = temp;

        siftUp(heap, parent);
    }
}
```

For min-heap.

42. RECURSIVE COMPLEXITY

Time:

```
TEXT
O(log n)
```

Auxiliary recursion stack:

```
TEXT
O(log n)
```

Iterative version:

```
TEXT
O(1)
```

auxiliary space.

For heaps, iterative bubble-up is usually preferable.

43. IN-PLACE PROPERTY

Heap insertion inside an array uses:

```
TEXT
O(1)
```

extra algorithmic memory, assuming array already has capacity.

However, a dynamic array implementation may occasionally resize its underlying storage.

That is an array-management issue, separate from heap-order repair.

44. DYNAMIC ARRAY RESIZING — PLACEMENT EXTENSION $\Theta(n)$

If heap uses a dynamic array:

When capacity is full:

1. Allocate larger array.
2. Copy old values.
3. Append new value.
4. Bubble up.

A resize itself can cost:

TEXT

$O(n)$

occasionally.

But with geometric growth, e.g. doubling capacity, append cost is:

TEXT

$O(1)$ amortized

So heap insertion remains commonly described as:

TEXT

$O(\log n)$

because bubble-up dominates amortized append.

45. GATE TRAP — HEAPIFY UP DIRECTION

Insertion repairs:

UPWARD

Why?

New element is added at bottom.

Its children do not exist initially, so it cannot violate with children.

Only parent relation may be wrong.

Therefore:

TEXT

check ancestors

$\Theta(n)$ is different from root extraction, which repairs downward.

46. INSERT VS EXTRACT REPAIR DIRECTION

Insert

TEXT

```
new node at bottom  
!' s i f t   U P
```

Delete/extract root

TEXT

```
last node moved to root  
!' s i f t   D O W N
```

Memory:

Insert climbs. Extract falls.

Ø=Excellent shortcut.

47. MIN-HEAP INSERT — WHEN DOES IT STOP?

Bubble-up stops if any of these:

1. New element reaches root.
2. Parent "dnew element.
3. Under duplicate values, parent == new value is already valid.

No need to continue upward because heap property above parent was already valid.

48. PROOF OF EARLY STOP

Suppose min-heap and:

TEXT

```
p a r e n t "d   x
```

Old heap guarantees:

TEXT

```
g r a n d p a r e n t "d   p a r e n t
```

Therefore:

TEXT

```
g r a n d p a r e n t "d   p a r e n t "d   x
```

So x cannot violate any higher ancestor.

Thus stop is safe.

$\emptyset = \text{Nil}$ proof.

49. MAX-HEAP EARLY STOP

If:

TEXT

```
p a r e n t "e x
```

and old heap guarantees:

TEXT

```
g r a n d p a r e n t "e p a r e n t
```

then:

TEXT

```
g r a n d p a r e n t "e p a r e n t "e x
```

so no higher violation.

Stop.

50. DUPLICATES

Min-heap allows:

TEXT

```
p a r e n t "d c h i l d
```

So equal values require no swap.

Example:

TEXT

```
    10
   /
  10
```

valid.

If insertion code swaps only when:

TEXT

```
heap[parent] > heap[i]
```

duplicates stay without unnecessary movement.

51. STABILITY?

Heaps are generally **not stable** data structures with respect to equal keys.

Two equal-priority items may change relative order through swaps unless extra sequence metadata is stored.

Ø=Useful priority-queue extension.

52. GATE QUESTION — INSERT 2

Min-heap:

TEXT

[4, 7, 6, 10, 8, 9]

Insert:

TEXT

2

Append:

TEXT

[4, 7, 6, 10, 8, 9, 2]

New index:

TEXT

6

Parent:

TEXT

2

Value:

TEXT

6

Swap:

TEXT

[4, 7, 2, 10, 8, 9, 6]

Now index 2.

Parent index 0:

TEXT

4

Swap:

TEXT

[2, 7, 4, 10, 8, 9, 6]

Final.

53. CHECK FINAL HEAP

TEXT

```
      2
     /\
    7  4
   /\ /\
 10 8 9 6
```

Check:

TEXT

```
2"d 7 , 4
7"d 10 , 8
4"d 9 , 6
```

Valid.

54. GATE QUESTION — NUMBER OF SWAPS

For previous insertion:

TEXT

2

started at level:

TEXT

2

and became root.

Swaps:

TEXT

2

Exactly equal to number of levels climbed.

55. GATE QUESTION — INSERT WITHOUT SWAP

Min-heap:

TEXT

[3, 8, 5, 12, 10, 9]

Insert:

TEXT

20

New index 6.

Parent index 2:

TEXT

5

Since:

TEXT

5 > 2 0

no swap.

Final:

TEXT

[3, 8, 5, 12, 10, 9, 20]

56. GATE QUESTION — MAX-HEAP

Max-heap:

TEXT

[90, 70, 80, 40, 60, 50]

Insert:

TEXT

100

Append:

TEXT

[90, 70, 80, 40, 60, 50, 100]

Index 6 parent 2 != 80.

Swap:

TEXT

[90, 70, 100, 40, 60, 50, 80]

Index 2 parent 0 != 90.

Swap:

TEXT

```
[100, 70, 90, 40, 60, 50, 80]
```

Final max-heap.

57. INSERTION IN 1-BASED HEAP

If using index `1...n`:

New node enters:

TEXT

```
index = size + 1
```

Parent:

TEXT

```
floor(i/2)
```

Bubble-up becomes particularly simple.

Pseudocode:

TEXT

```
i = ++size
heap[i] = x

while i > 1 and heap[i] < heap[i/2]:
    swap
    i = i/2
```

for min-heap.

58. 0-BASED VS 1-BASED INSERTION

0-based parent:

TEXT

```
(i-1)/2
```

1-based:

TEXT

```
i/2
```

Concept identical.

~~0-based~~ Always identify indexing convention before tracing.

59. BUILDING HEAP BY REPEATED INSERTION

If we start empty and insert n elements one at a time:

Each insertion worst:

TEXT
 $O(\log n)$

So easy upper bound:

TEXT
 $O(n \log n)$

This is one valid way to construct a heap.

But later we will learn:

Bottom-up build heap = $O(n)$

Very important difference.

60. DON'T CONFUSE REPEATED INSERTION WITH BUILD-HEAP

Repeated insertion

Start:

TEXT
empty heap

insert items individually.

Complexity:

TEXT
 $O(n \log n)$

Bottom-up Build Heap

Start:

TEXT
all n elements already in array

heapify internal nodes downward.

Complexity:

TEXT
 $O(n)$

61. HEAP INSERTION AND ROOT

For min-heap:

If inserted value is smaller than current root, it will eventually become:

```
TEXT
new root
```

because it keeps violating each ancestor.

For max-heap:

If inserted value is larger than current root, it can become new root.

62. EXAMPLE

Min-heap:

```
TEXT
root=10
```

Insert:

```
TEXT
1
```

Since 1 is smaller than every ancestor on its path, bubble-up eventually puts:

```
TEXT
1
```

at root.

Thus after insertion:

```
TEXT
root = minimum of all elements
```

as required.

63. CAN INSERTION MOVE OTHER NODES SIDEWAYS?

No.

Bubble-up swaps only along one:

TEXT

ancestor path

Other subtrees remain untouched.

This helps both correctness and efficiency.

64. INVERSION VIEW

For min-heap, insertion introduces potentially one chain of violations:

TEXT

parent > child

Each swap moves the smaller child upward until no violation remains.

This is analogous to restoring a local order condition.

65. INSERTION AND COMPLETE SHAPE

Important:

Swapping **values** inside array positions does not alter tree shape.

Positions remain:

TEXT

0...size-1

So completeness stays valid automatically.

Heap insertion separates:

TEXT

shape maintenance

from:

TEXT

value-order maintenance

One heap design idea.

66. POINTER-BASED HEAP?

A binary heap could theoretically be stored as linked nodes, but then locating the next complete-tree insertion position becomes more cumbersome.

Array representation makes it trivial:

TEXT

append at end

This is why heaps are normally arrays.

67. GATE TRUE/FALSE

Statement 1

A new heap element is initially inserted at the root.

'LFalse.

It is appended at the next complete-tree position.

Statement 2

Insertion into binary heap may require $O(\log n)$ swaps.

' True.

Statement 3

Insertion can make the heap non-complete temporarily.

'LFalse.

Appending at next slot preserves completeness immediately.

Only heap order may temporarily fail.

Statement 4

Min-heap insertion bubbles an element upward when it is smaller than its parent.

' True.

Statement 5

Every insertion requires at least one swap.

'LFalse.

Best case requires zero swaps.

68. GATE TRUE/FALSE — COMPLEXITY

Inserting into a binary heap takes $O(n)$.

'LAs a standard heap operation:

TEXT

$O(\log n)$

worst case.

69. GATE TRAP — FINDING INSERTION LOCATION COST

One might think:

We need BFS to find next free node !' $O(n)$.

Not in array representation.

Next slot is directly:

TEXT

$\text{index} = \text{size}$

So structural placement:

TEXT

$O(1)$

Then bubble-up:

TEXT

$O(\log n)$

Total:

TEXT

$O(\log n)$

$O = \log$ important.

70. COMPARISON COUNT — APPROXIMATE

If node climbs k levels:

It requires approximately:

TEXT

k

successful violation comparisons/swaps, plus possibly one final comparison where it stops.

Asymptotically:

TEXT

$O(\log n)$

Exact comparison counts depend on implementation and whether reaching root ends without an extra parent comparison.

71. GATE-STYLE TRACE — MIN HEAP

Initial:

TEXT

[2, 5, 4, 9, 8, 7, 6]

Insert:

TEXT

1

Append index 7:

TEXT

[2, 5, 4, 9, 8, 7, 6, 1]

Parent:

TEXT

$\text{floor}((7-1)/2)=3$

value 9.

Swap:

TEXT

[2, 5, 4, 1, 8, 7, 6, 9]

72. CONTINUE

New index:

TEXT

3

Parent:

TEXT

1

value:

TEXT

5

Swap:

TEXT

[2, 1, 4, 5, 8, 7, 6, 9]

Now index 1.

Parent 0:

TEXT

2

Swap:

TEXT

[1, 2, 4, 5, 8, 7, 6, 9]

Final.

Swaps:

TEXT

3

73. WHY 3 SWAPS?

New node at index 7 is level:

TEXT

$\text{floor}(\log_2(8))=3$

It climbed all the way to root.

Thus maximum:

TEXT

3

swaps.

74. GATE SEQUENCE QUESTION

Start empty min-heap.

Insert:

TEXT

9, 4, 7, 1, 5

Insert 9

TEXT

[9]

Insert 4

TEXT

[9, 4]

! ' [4 , 9]

Insert 7

TEXT

[4, 9, 7]

75. INSERT 1

Append:

TEXT

[4, 9, 7, 1]

Swap with 9:

TEXT

[4, 1, 7, 9]

Swap with 4:

TEXT

[1, 4, 7, 9]

76. INSERT 5

Append:

TEXT

[1, 4, 7, 9, 5]

Parent 4.

TEXT

4"d 5

Stop.

Final:

[1,4,7,9,5]

77. EXAM SHORTCUT

When manually inserting into a heap:

Step A

Append value.

Step B

Write its index.

Step C

Calculate parent.

Step D

Swap if violated.

Step E

Repeat with parent index.

This avoids mistakes.

78. COMMON MANUAL-TRACE ERROR

Suppose value swaps upward.

Students sometimes continue comparing using its **old index**.

Wrong.

After swap:

TEXT

i = parent

Then calculate the new parent from the new index.

~~Keep~~ updating the index.

79. COMMON ERROR — SWAP WITH SMALLER CHILD DURING INSERT

That's a downward-heapify idea.

During insertion, compare the new node with:

its parent

not with sibling/children.

New node starts at leaf, so no children.

80. COMMON ERROR — REBUILD ENTIRE HEAP

After every insertion, you do **not** need to call bottom-up build-heap on the whole array.

That would cost unnecessarily more.

Use:

```
TEXT
sift-up
```

only.

81. COMMON ERROR — USE BST RULE

For min-heap, do not choose:

```
TEXT
left if smaller
right if larger
```

That's BST insertion.

Heap location is determined by completeness, not comparison.

~~0-0%~~ ~~One~~ of the biggest DS distinctions.

82. MIN-HEAP INSERTION PSEUDOCODE

```
TEXT
INSERT(x)

heap[size] = x
i = size
size++

while i > 0:

    p = floor((i-1)/2)
```

```

    if heap[p] <= heap[i]:
        break

    swap(heap[p], heap[i])

    i = p

```

83. MAX-HEAP PSEUDOCODE

TEXT

INSERT(x)

```

heap[size] = x
i = size
size++

while i > 0:

    p = floor((i-1)/2)

    if heap[p] >= heap[i]:
        break

    swap(heap[p], heap[i])

    i = p

```

Only inequality changes.

84. MASTER COMPLEXITY TABLE

| Aspect | Complexity |
|----------------------------|-------------|
| Initial append | $O(1)$ |
| One parent comparison/swap | $O(1)$ |
| Maximum levels climbed | $O(\log n)$ |
| Worst insertion | $O(\log n)$ |
| Best insertion | $O(1)$ |
| Iterative auxiliary space | $O(1)$ |
| Recursive auxiliary space | $O(\log n)$ |

85. CONNECTION TO PRIORITY QUEUE

Heap-backed priority queue insertion:

TEXT

```
enqueue/insert priority item
```

is essentially:

TEXT

```
append + sift-up
```

Therefore:

Priority queue insertion using binary heap = $O(\log n)$

Root remains highest-priority item after repair.

86. NOTEBOOK NOTES

Heap Insertion

TEXT

1. Insert at end
2. Compare with parent
3. Swap upward while violated

Min-heap:

TEXT

```
if child < parent !' swap
```

Max-heap:

TEXT

```
if child > parent !' swap
```

Complexity:

TEXT

```
Best =  $O(1)$   
Worst =  $O(\log n)$ 
```

87. MEMORY TRICKS

Structural step

Heap insertion always joins the queue at the end.

Min-heap

Small values climb.

Max-heap

Big values climb.

Direction

INSERT = UP

Complexity

Climb at most the heap height !' $O(\log n)$.

88. 30-SECOND REVISION

Remember:

1. New heap element goes to next complete-tree position.
2. In array, append at index `size`.
3. Min-heap violation: child < parent.
4. Max-heap violation: child > parent.
5. Repair using bubble-up/sift-up.
6. Parent 0-based = $(i-1)/2$.
7. Only ancestor path can be affected.
8. Best insertion $O(1)$.
9. Worst insertion $O(\log n)$.
10. Iterative insertion uses $O(1)$ extra space.

89. ACTIVE RECALL

Q1

Where is a new heap element initially placed?

Q2

Which direction does insertion repair?

Q3

In min-heap, when do we swap?

Q4

In max-heap, when do we swap?

Q5

Worst-case insertion time?

Q6

Why doesn't insertion need to examine both subtrees?

ANSWERS

A1

At the next complete-tree position / end of heap array.

A2

Upward.

A3

When child value is smaller than parent.

A4

When child value is larger than parent.

A5

TEXT

$O(\log n)$

A6

Only the newly created ancestor path can contain a new heap-order violation.

90. PRACTICE — MIN-HEAP

Start:

TEXT

[3, 8, 5, 12, 10, 9, 7]

Insert:

TEXT

2

Find:

1. initial new index
 2. every parent index
 3. every intermediate array
 4. number of swaps
 5. final heap
-

91. PRACTICE — MAX-HEAP

Start:

TEXT

[100, 80, 90, 50, 70, 60, 40]

Insert:

TEXT

95

Trace bubble-up completely.

92. PRACTICE — BUILD BY INSERTION

Construct a min-heap by inserting:

TEXT

25, 15, 30, 10, 20, 5, 35

one by one.

After every insertion, record the array.

Then find:

- final height
- total number of swaps
- root
- leaves

93. MODULE 8 PROGRESS

' Lesson 65 — Heap Fundamentals ' Lesson 66 — Array Representation ' **Lesson 67 — Heap Insertion / Bubble Up**

Remaining:

- Lesson 68 — Heapify Down, Root Deletion & Extract-Min/Max
- Lesson 69 — Build Heap and Why It Is $O(n)$
- Lesson 70 — Heap Sort
- Lesson 71 — Priority Queue & Heap Applications

NEXT — LESSON 68

Heapify Down / Sift Down — Extract-Min, Extract-Max & Root Deletion

Next we will study the opposite operation:

REMOVE ROOT !' MOVE LAST ELEMENT TO ROOT !' SIFT DOWN

For example:

TEXT

Min-heap:
[5, 10, 15, 20, 30, 25]

Extract minimum 5:

TEXT

Move last 25 to root
!' compare with children
!' swap with smaller child
!' repeat

We will cover why we choose the **smaller child in a min-heap** and the **larger child in a max-heap**, exact C implementations, $O(\log n)$ proof, arbitrary heap deletion, and GATE tracing problems.

USER

next

ASSISTANT

MODULE 8 — LESSON 68

Heapify Down / Sift Down — Extract-Min, Extract-Max, Root Deletion & Arbitrary Deletion

Now we study the opposite of heap insertion.

Insertion was:

ADD AT END !' SIFT UP

Deletion of the root is:

REMOVE ROOT !' MOVE LAST ELEMENT TO ROOT !' SIFT DOWN

This lesson is essential because extracting the highest-priority item from a heap is one of the main reasons heaps are useful.

GATE importance: $\emptyset = \text{Y\%} \emptyset \text{M\%} \emptyset \text{K\%} \emptyset \text{W}$ **Placement importance:** $\emptyset = \text{Y\%} \emptyset \text{M\%} \emptyset \text{K\%} \emptyset \text{W}$

1. RECALL — WHY ROOT IS SPECIAL

In a min-heap:

TEXT

Root = Minimum

In a max-heap:

TEXT

Root = Maximum

So the most important delete operation is usually:

Min-Heap

TEXT

Extract-Min

Max-Heap

TEXT

Extract-Max

2. WHY CAN'T WE JUST DELETE ROOT?

Suppose min-heap:

TEXT

```
    5
   / \
  10 15
 / \ /
20 30 25
```

Array:

TEXT

[5, 10, 15, 20, 30, 25]

If we simply remove 5:

TEXT

```
    ?
   / \
  10 15
 / \ /
20 30 25
```

root position becomes empty.

That destroys the complete-tree representation.

So we need to preserve the shape.

3. MASTER EXTRACTION IDEA

To remove root:

TEXT

1. Save root value.
2. Move last heap element to root.
3. Reduce heap size by 1.
4. Sift the new root downward.

Why last element?

Because deleting the last occupied array position preserves completeness.

Shape first, order second.

4. MIN-HEAP EXAMPLE

Initial heap:

TEXT

[5, 10, 15, 20, 30, 25]

Tree:

TEXT

```
      5
     /\
    10 15
   /\ /\
  20 30 25
```

Extract minimum:

TEXT

5

5. STEP 1 — SAVE ROOT

TEXT

min = 5

6. STEP 2 — MOVE LAST ELEMENT TO ROOT

Last element:

TEXT

25

Move it to index 0.

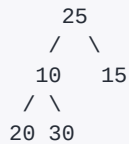
Temporary array:

TEXT

[25, 10, 15, 20, 30]

Tree:

TEXT



Complete shape is preserved.

But min-heap property is broken.

7. WHAT VIOLATION EXISTS?

At root:

TEXT

25 > 10
25 > 15

So 25 must move down.

This is:

Sift Down / Bubble Down / Percolate Down / Heapify Down

8. WHICH CHILD SHOULD WE SWAP WITH? Ø=Ý%Ø=Ý%Ø=Ý%

In a min-heap:

Swap with the smaller child

Here children:

TEXT

10 and 15

Smaller:

TEXT

10

So swap:

TEXT

2 5 ! " 1 0

Array becomes:

TEXT

[10, 25, 15, 20, 30]

Tree:

TEXT

```
      10
     /  \
    25   15
   /  \
  20  30
```

Still not valid because:

TEXT

25 > 20

9. CONTINUE DOWN

Node 25 has children:

TEXT

20 and 30

Smaller child:

TEXT

20

Since:

TEXT

25 > 20

swap:

TEXT

[10, 20, 15, 25, 30]

Final:

TEXT

```
      10
     /  \
    20   15
   /  \
```

25 30

Valid min-heap.

Extracted value:

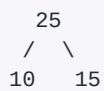
5

10. WHY SWAP WITH SMALLER CHILD?

This is extremely important.

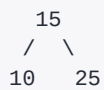
Suppose:

TEXT



If we swap 25 with larger child 15:

TEXT



Now:

TEXT

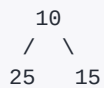
1 5 "d 1 0

is false.

Heap is still invalid at the new parent.

But if we swap with smaller child 10:

TEXT



Then:

TEXT

1 0"d 2 5
1 0"d 1 5

root relation becomes valid.

Therefore:

Min-heap !' choose smaller child

11. MAX-HEAP RULE

For max-heap, mirror logic:

Choose larger child

because parent must be:

TEXT

```
"e  b o t h  c h i l d r e n"
```

12. MIN-HEAP SIFT-DOWN ALGORITHM

At index `i`:

TEXT

```
left  = 2i+1
right = 2i+2
```

Find smallest among:

TEXT

```
i
left
right
```

If smallest is already `i`:

TEXT

```
stop
```

Otherwise:

TEXT

```
swap i with smallest child
i = smallest child index
continue
```

13. C IMPLEMENTATION— MIN-HEAPIFY DOWN

C

```
void minHeapifyDown(int heap[], int size, int i)
{
    while (1)
    {
        int left = 2 * i + 1;
        int right = 2 * i + 2;
        int smallest = i;
```



```

        if (left < size && heap[left] < heap[smallest])
            smallest = left;

        if (right < size && heap[right] < heap[smallest])
            smallest = right;

        if (smallest == i)
            break;

        int temp = heap[i];
        heap[i] = heap[smallest];
        heap[smallest] = temp;

        i = smallest;
    }
}

```

Standard heapify-down pattern.

14. EXTRACT-MIN C CODE

```

c
int extractMin(int heap[], int *size)
{
    if (*size <= 0)
        return -1;

    int minValue = heap[0];

    heap[0] = heap[*size - 1];
    (*size)--;

    minHeapifyDown(heap, *size, 0);

    return minValue;
}

```

Conceptually correct.

In real code, error handling may be done differently than returning `-1`.

15. EDGE CASE — ONE ELEMENT

Heap:

TEXT
[5]

Extract min.

Save:

TEXT
5

Move last to root — same element.

Decrease size:

TEXT
0

No heapify needed.

Result:

TEXT
empty heap

Time:

TEXT
 $O(1)$

16. EDGE CASE — TWO ELEMENTS

Min-heap:

TEXT
[5, 10]

Extract 5.

Move last 10 to root:

TEXT
[10]

Size 1.

No children.

Done.

17. MAX-HEAP EXAMPLE

Initial:

TEXT
[100, 80, 90, 40, 70, 60, 50]

Tree:

TEXT

```
      100
     /  \
    80   90
   / \  / \
  40 70 60 50
```

Extract max:

TEXT
100

Move last:

TEXT
50

to root:

TEXT
[50, 80, 90, 40, 70, 60]

18. SIFT DOWN IN MAX-HEAP

Children of 50:

TEXT
80 and 90

Choose larger:

TEXT
90

Swap:

TEXT
[90, 80, 50, 40, 70, 60]

Now 50 at index 2.

Its child:

TEXT
60

Since:

TEXT
60 > 50

swap:

TEXT
[90, 80, 60, 40, 70, 50]

Final max-heap.

Extracted:

100

19. MAX-HEAPIFY DOWN CODE

```
c
void maxHeapifyDown(int heap[], int size, int i)
{
    while (1)
    {
        int left = 2 * i + 1;
        int right = 2 * i + 2;
        int largest = i;

        if (left < size && heap[left] > heap[largest])
            largest = left;

        if (right < size && heap[right] > heap[largest])
            largest = right;

        if (largest == i)
            break;

        int temp = heap[i];
        heap[i] = heap[largest];
        heap[largest] = temp;

        i = largest;
    }
}
```

Only comparison direction changes.

20. EXTRACTION COMPLEXITY $O(\log n)$

The replacement node may move:

```
TEXT
root != leaf
```

at most heap height:

```
TEXT
floor(log2 n)
```

Therefore:

FORMULA BOX

```
TEXT
Extract-Min =  $O(\log n)$ 
Extract-Max =  $O(\log n)$ 
```

21. BEST CASE

After moving last element to root, it may already satisfy heap property.

Then no swaps.

Example min-heap after replacement:

TEXT

```
  10
 /  \
20  15
```

valid immediately.

Repair:

TEXT

$O(1)$

Best case.

Worst:

TEXT

$O(\log n)$

22. INSERT VS EXTRACT

| Operation | Initial structural action | Repair direction |
|--------------|---------------------------|------------------|
| Insert | Add at last position | Up |
| Extract root | Move last to root | Down |

Memory:

INSERT CLIMBS

EXTRACT FALLS

$O(\log n)$ to memorize.

23. WHY ONLY ONE DOWNWARD PATH?

Before extraction, all heap relations were valid.

After moving last value to root, only relations involving that moved value may be violated.

As it swaps downward, the possible violation follows that same path.

No need to rebuild whole heap.

Thus:

```
TEXT
0(log n)
```

instead of $O(n)$.

24. HEAPIFY DOWN WITH ONLY LEFT CHILD

In a complete tree, an internal node may have only:

```
TEXT
left child
```

never only right child.

Example:

```
TEXT
      10
     /  \
    20  30
   /
  40
```

So code must handle:

```
TEXT
right >= size
```

safely.

Our implementation checks existence first.

25. MIN-HEAP EXAMPLE WITH ONE CHILD

Suppose after replacement:

```
TEXT
[5, 10, 15, 20]
```

Imagine heapify at index 1:

Value:

```
TEXT
10
```

Left child:

TEXT

20

Right does not exist.

Check only left.

Since:

TEXT

1 0"d 2 0

stop.

26. GATE TRAP — CHOOSE ANY CHILD?

No.

For min-heap:

TEXT

smaller child

For max-heap:

TEXT

larger child

Choosing the wrong child can leave a violation immediately.

$\emptyset = \sqrt{n}$ very common.

27. GATE TRAP — MOVE FIRST CHILD TO ROOT?

No.

To preserve complete shape, replacement must normally be:

last element

not arbitrarily left/right child.

Why?

Removing last occupied node leaves no gap.

28. GATE TRAP — DELETE ROOT AND SHIFT ARRAY LEFT?

You could physically shift every array element one position, but:

- costs $O(n)$
- changes structural positions unnecessarily

- does not automatically preserve heap property

Standard heap extraction uses:

TEXT

l a s t ! ' r o o t

then $O(\log n)$ repair.

29. GATE-STYLE MIN-HEAP TRACE

Initial:

TEXT

[2, 5, 4, 9, 8, 7, 6]

Extract min.

Root:

TEXT

2

Last:

TEXT

6

Move 6 to root:

TEXT

[6, 5, 4, 9, 8, 7]

Children:

TEXT

5 and 4

Choose:

TEXT

4

Swap:

TEXT

[4, 5, 6, 9, 8, 7]

Now 6 at index 2.

Child:

TEXT

7

Since:

TEXT

6"d 7

stop.

Final:

[4,5,6,9,8,7]

Extracted:

2

30. GATE TRACE — ANOTHER EXAMPLE

Initial min-heap:

TEXT

[1, 3, 2, 7, 6, 5, 4]

Extract 1.

Move 4:

TEXT

[4, 3, 2, 7, 6, 5]

Children:

TEXT

3, 2

Choose 2.

Swap:

TEXT

[2, 3, 4, 7, 6, 5]

At index 2, child 5.

TEXT

4"d 5

Stop.

Final:

TEXT

[2, 3, 4, 7, 6, 5]

31. WHY FINAL ARRAY NEED NOT BE SORTED

After extraction:

TEXT

[2, 3, 4, 7, 6, 5]

Notice:

TEXT

7 > 6

neighboring indices are not sorted.

Still valid min-heap.

Same principle as before.

32. ROOT DELETION IN A HEAP

Usually:

Min-heap

TEXT

delete root = extract-min

Max-heap

TEXT

delete root = extract-max

So root deletion complexity:

TEXT

$O(\log n)$

worst case.

33. WHAT ABOUT DELETING AN ARBITRARY HEAP ELEMENT? $\Theta(\log n)$

Suppose we know its array index:

TEXT

i

Standard idea:

1. Replace `heap[i]` with last element.
2. Reduce size.
3. Decide whether replacement should move:
 - up
 - or down.

Why both possibilities?

Because unlike root extraction, arbitrary position has both:

TEXT

parent
children

34. ARBITRARY DELETION EXAMPLE

Min-heap:

TEXT

[5, 10, 15, 20, 30, 25, 40]

Delete element at index 1:

TEXT

10

Move last 40 into index 1:

TEXT

[5, 40, 15, 20, 30, 25]

Now 40 compared with children:

TEXT

20, 30

Too large.

So sift down:

TEXT

[5, 20, 15, 40, 30, 25]

Done.

35. CASE WHERE REPLACEMENT MUST SIFT UP

Min-heap:

Suppose after replacing an arbitrary node with last element, the replacement becomes smaller than its parent.

Then it violates upward.

Example conceptual state:

TEXT



If 8 replaces some node below 20:

TEXT

8 < 20

so it should sift up.

Thus arbitrary deletion may require either direction.

36. HOW TO DECIDE DIRECTION

For min-heap after replacement at index `i`:

If:

TEXT

```
i > 0
and heap[i] < heap[parent(i)]
```

then:

TEXT

sift up

Otherwise:

TEXT

sift down

For max-heap reverse the inequality.

37. ARBITRARY DELETION COMPLEXITY IF INDEX IS KNOWN

Replacement + repair follows at most one path.

Therefore:

$O(\log n)$

if the index is already known.

But...

38. WHAT IF ONLY THE VALUE IS GIVEN?

Suppose:

Delete key 37 from heap.

Heap does not support efficient arbitrary search.

Finding 37 may require:

TEXT

$O(n)$

Then deletion repair:

TEXT

$O(\log n)$

Total:

TEXT

$O(n)$

because:

TEXT

$O(n) + O(\log n) = O(n)$

$O(n)$ is important GATE/placement distinction.

39. DECREASE-KEY / INCREASE-KEY CONNECTION $\emptyset = \text{p€}$

Heaps often support changing a key.

Min-heap

If a key decreases:

TEXT

it may violate with parent
!' s i f t u p

If key increases:

TEXT

```
it may violate with children  
!' s i f t d o w n
```

Max-heap

Opposite direction.

This is important in graph algorithms and priority queues.

40. MIN-HEAP DECREASE-KEY

Example:

TEXT

```
[5, 10, 15, 20, 30]
```

Change 20 to:

TEXT

```
2
```

Array:

TEXT

```
[5, 10, 15, 2, 30]
```

2 at index 3.

Parent:

TEXT

```
10
```

Swap:

TEXT

```
[5, 2, 15, 10, 30]
```

Then compare with root 5:

TEXT

```
[2, 5, 15, 10, 30]
```

Done.

So decrease-key uses:

Sift Up

41. MIN-HEAP INCREASE-KEY

Change root 5 to 25:

TEXT

[25, 10, 15, 20, 30]

Now choose smaller child:

TEXT

10

Swap:

TEXT

[10, 25, 15, 20, 30]

Then 25 has children 20,30.

Swap with 20:

TEXT

[10, 20, 15, 25, 30]

So increase-key uses:

Sift Down

42. MAX-HEAP DIRECTION TABLE

| Key change | Max-Heap Repair |
|--------------|-----------------|
| Increase key | Sift up |
| Decrease key | Sift down |

Min-heap:

| Key change | Min-Heap Repair |
|--------------|-----------------|
| Decrease key | Sift up |
| Increase key | Sift down |

Useful.

43. HEAPIFY-DOWN RECURSIVE VERSION

Min-heap:

```

c
void minHeapify(int heap[], int size, int i)
{
    int left = 2 * i + 1;
    int right = 2 * i + 2;
    int smallest = i;

    if (left < size && heap[left] < heap[smallest])
        smallest = left;

    if (right < size && heap[right] < heap[smallest])
        smallest = right;

    if (smallest != i)
    {
        int temp = heap[i];
        heap[i] = heap[smallest];
        heap[smallest] = temp;

        minHeapify(heap, size, smallest);
    }
}

```

Time:

TEXT

$O(\log n)$

recursive auxiliary space:

TEXT

$O(\log n)$

Iterative uses:

TEXT

$O(1)$

extra space.

44. WHAT PRECONDITION DOES HEAPIFY DOWN ASSUME? Ø=Ý%

This is subtle and important.

heapifyDown(i) assumes:

The left and right subtrees rooted below i are already valid heaps.

Only node i itself may violate heap order.

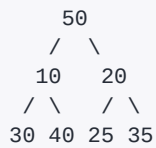
Then heapify pushes that value downward until the whole subtree becomes valid.

This assumption becomes crucial in **Build Heap** next lesson.

45. EXAMPLE OF PRECONDITION

Suppose:

TEXT



Subtree rooted at 10:

TEXT

1 0 "d 3 0 , 4 0

valid.

Subtree rooted at 20:

TEXT

2 0 "d 2 5 , 3 5

valid.

Only root 50 violates.

Heapify-down at root can fix entire tree.

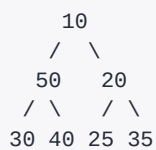
46. SIFT-DOWN TRACE

Children 10,20.

Choose 10.

Swap:

TEXT

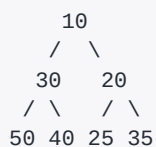


Now 50 compares with 30,40.

Choose 30.

Swap:

TEXT



Valid.

Only one downward path changed.

47. WHAT IF CHILD SUBTREES ARE NOT HEAPS?

A single call:

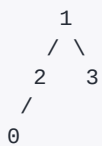
TEXT

```
heapifyDown(root)
```

is not guaranteed to magically fix arbitrary disorder everywhere.

Example:

TEXT



Root 1 is fine relative to children.

Heapify at root would stop.

But:

TEXT

```
2 > 0
```

violates min-heap lower down.

So child subtrees must already be heaps.

$\Theta(n)$ is key for understanding build-heap order.

48. INSERT HEAPIFY-UP PRECONDITION

Analogously, sift-up assumes:

Heap was valid before appending/changing this one node.

Only the ancestor path may be invalid.

Again, local repair works because everything else is already valid.

49. EXTRACTION MASTER COMPLEXITY

Suppose heap size before extraction is n .

Structural operations:

TEXT

```
save root = O(1)
move last = O(1)
decrease size = O(1)
```

Sift down:

TEXT

$O(\log n)$

Total:

$O(\log n)$

50. PEEK VS EXTRACT

Do not confuse:

Peek-Min

Return:

TEXT

heap[0]

without removing.

TEXT

$O(1)$

Extract-Min

Return and remove root, then repair.

TEXT

$O(\log n)$

Same for max-heap.

Very common priority-queue distinction.

51. GATE OPERATION TABLE

| Heap Operation | Time |
|-------------------|-------------|
| Peek min/max root | $O(1)$ |
| Insert | $O(\log n)$ |
| Extract root | $O(\log n)$ |

| Heap Operation | Time |
|----------------------|-------------|
| Heapify one node | $O(\log n)$ |
| Search arbitrary key | $O(n)$ |
| Delete known index | $O(\log n)$ |
| Delete unknown value | $O(n)$ |

52. GATE QUESTION — EXTRACT MIN

Initial:

TEXT

[3, 8, 5, 12, 10, 9, 7]

Extract 3.

Move 7:

TEXT

[7, 8, 5, 12, 10, 9]

Children 8,5.

Choose 5.

Swap:

TEXT

[5, 8, 7, 12, 10, 9]

Now 7 has child 9.

TEXT

7" d 9

stop.

Final:

[5,8,7,12,10,9]

53. GATE QUESTION — EXTRACT MAX

Initial max-heap:

TEXT

[100, 90, 80, 70, 60, 50, 40]

Extract 100.

Move 40:

TEXT

[40, 90, 80, 70, 60, 50]

Choose larger child:

TEXT

90

Swap:

TEXT

[90, 40, 80, 70, 60, 50]

Now 40 children:

TEXT

70, 60

choose 70.

Final:

TEXT

[90, 70, 80, 40, 60, 50]

54. GATE TRAP — CHILD COMPARISON ORDER

Suppose min-heap current node:

TEXT

20

children:

TEXT

5 and 7

You must choose:

TEXT

5

not just "first child that violates".

If code sees 7 first and swaps with 7:

TEXT

7
/ \

still invalid.

Therefore:

Find best child first, then swap.

55. WHY CODE COMPARES BOTH CHILDREN

This is why standard heapify code does:

TEXT

```
smallest = i
compare left
compare right
```

Then performs exactly one swap with the best candidate.

Not:

TEXT

```
swap immediately with left
then maybe right
```

That can complicate or break logic.

56. ARRAY BOUNDS TRAP

When computing:

TEXT

```
left = 2i+1
right = 2i+2
```

always check:

TEXT

```
left < size
right < size
```

before accessing those array cells.

Especially last internal node may have only left child.

57. HEAP SIZE AFTER EXTRACT

If old size:

TEXT

n

new size:

TEXT

n-1

The old final array position is now outside active heap.

Even if its old value remains physically in memory, it is not part of the heap.

Ø=10 Logical size matters.

58. EXAMPLE

Before:

TEXT

```
heap = [5,10,15,20]
size = 4
```

After moving 20 to root and decrementing:

TEXT

```
heap memory might look like:
[20,10,15,20]
```

but active heap size is:

TEXT

3

So active part:

TEXT

```
[20,10,15]
```

Then heapify:

TEXT

```
[10,20,15]
```

Index 3 is irrelevant.

59. HEAPIFY DOWN AND HEIGHT

Starting at index **i**, exact cost depends on:

TEXT

height of subtree rooted at i

not necessarily whole heap height.

Root:

TEXT

$O(\log n)$

Near leaves:

TEXT

$O(1)$

~~$O=1$~~ This observation is the key to proving build-heap is $O(n)$.

60. EXAMPLE

In heap of height 10:

A node one level above leaves has subtree height:

TEXT

1

so heapifying it costs at most:

TEXT

$O(1)$

A leaf:

TEXT

Θ work

Most nodes are near the bottom.

We'll exploit that in Lesson 69.

61. ROOT EXTRACTION AND HEAPSORT CONNECTION

In a max-heap:

TEXT

root = maximum

If we repeatedly:

1. swap root with last active element
2. reduce active heap size

3. heapify-down root

we place largest elements at the array end one by one.

This is exactly:

Heap Sort

Lesson 70.

62. ROOT EXTRACTION AND PRIORITY QUEUE

A priority queue needs:

TEXT

remove highest-priority item

In heap:

- root stores highest priority
- extract root
- restore heap

Thus:

TEXT

extract priority = $O(\log n)$

This is why heaps are so useful.

63. NOTEBOOK NOTES — MIN-HEAP EXTRACT

TEXT

1. Save root
2. root = last
3. size--
4. Compare with children
5. Swap with SMALLER child
6. Repeat downward

64. NOTEBOOK NOTES — MAX-HEAP EXTRACT

TEXT

1. Save root
2. root = last
3. size--
4. Compare with children
5. Swap with LARGER child
6. Repeat downward

65. SIFT-UP VS SIFT-DOWN

| Feature | Sift Up | Sift Down |
|------------------|--------------------|-----------------|
| Common use | Insertion | Root extraction |
| Direction | leaf !' root | root !' leaf |
| Compare with | parent | children |
| Min-heap chooses | smaller node rises | smaller child |
| Max-heap chooses | larger node rises | larger child |
| Worst time | $O(\log n)$ | $O(\log n)$ |

0=Copy this table.

66. MEMORY TRICKS

Extraction

Root leaves, last fills the seat.

Min-Heap

When falling, follow the smaller child.

Max-Heap

When falling, follow the larger child.

Direction

Insert climbs; extract falls.

67. 30-SECOND REVISION

Remember:

1. Extract-min removes min-heap root.
2. Extract-max removes max-heap root.
3. Move last element to root.
4. Decrease heap size.
5. Min-heap sift-down chooses smaller child.
6. Max-heap chooses larger child.
7. Root extraction = $O(\log n)$.

- 8. Peek root = $O(1)$.
 - 9. Arbitrary deletion by known index = $O(\log n)$.
 - 10. Heapify-down assumes child subtrees are already heaps.
-

68. ACTIVE RECALL

Q1

Why do we move the last element to root?

Q2

Which child does min-heap sift-down choose?

Q3

Which child does max-heap choose?

Q4

Worst-case extract-root complexity?

Q5

If arbitrary value's index is unknown, what can deletion cost?

Q6

What is heapify-down's key precondition?

ANSWERS

A1

To preserve complete-tree shape without leaving an internal gap.

A2

The smaller child.

A3

The larger child.

A4

TEXT

$O(\log n)$

A5

TEXT

$O(n)$

because locating the arbitrary key may require linear search.

A6

The child subtrees must already satisfy heap property; only the current root of that subtree may violate it.

69. PRACTICE — MIN-HEAP

Start:

TEXT

[1, 4, 3, 8, 6, 5, 7, 10, 9]

Extract minimum.

Write:

1. removed value
2. replacement value
3. every intermediate array
4. every chosen child
5. total swaps
6. final heap

70. PRACTICE — MAX-HEAP

Start:

TEXT

[90, 70, 80, 50, 60, 40, 30, 20]

Extract maximum and trace completely.

71. PRACTICE — ARBITRARY DELETE

Min-heap:

TEXT

[5, 10, 15, 20, 30, 25, 40]

Delete element at index:

TEXT

1

Trace:

- replacement
- direction
- final heap

Then ask what changes if only the value 10 is provided but its index is unknown.

72. MODULE 8 PROGRESS

' Lesson 65 — Heap Fundamentals ' Lesson 66 — Array Representation ' Lesson 67 — Heap Insertion / Sift Up '
Lesson 68 — Heapify Down / Extract-Min/Max

Remaining:

- Lesson 69 — Build Heap and Why It Is $O(n)$
- Lesson 70 — Heap Sort
- Lesson 71 — Priority Queue & Heap Applications

NEXT — LESSON 69

Build Heap — Bottom-Up Heap Construction & Why It Is $O(n)$, Not $O(n \log n)$

This is one of the classic GATE heap questions.

We will compare:

TEXT

Method 1:
Insert n elements one-by-one
!' $O(n \log n)$

versus:

TEXT

Method 2:
Treat array as complete tree
!' start from last internal node
!' heapify downward to root
!' $O(n)$

and derive mathematically why the second method is **linear time**, even though one heapify operation can cost $O(\log n)$.

USER

next

ASSISTANT

MODULE 8 — LESSON 69

Build Heap — Bottom-Up Construction & Why It Is $O(n)$, Not $O(n \log n)$

This is one of the most important and most frequently confused heap concepts in GATE.

The key question is:

If one heapify operation can take $O(\log n)$, and we heapify many nodes, why is Build Heap only $O(n)$?

We will answer that completely.

GATE importance: $\emptyset = \text{Yes} \%$ **Placement importance:** $\emptyset = \text{Yes} \%$

1. WHAT IS BUILD HEAP?

Suppose we are given an arbitrary array:

TEXT

[4, 10, 3, 5, 1]

This is not necessarily a heap.

Our goal:

Rearrange the same elements so they satisfy min-heap or max-heap property.

There are two major approaches.

2. METHOD 1 — REPEATED INSERTION

Start with an empty heap.

Insert:

TEXT

4
10
3
5
1

one by one.

Each insertion can take:

TEXT

$O(\log n)$

For n elements:

$O(n \log n)$

This works.

But it is not the optimal Build Heap algorithm.

3. METHOD 2 — BOTTOM-UP BUILD HEAP $\emptyset = \text{Yes} \%$

Take the whole array immediately as a complete binary tree.

For:

TEXT
[4, 10, 3, 5, 1]

tree:

TEXT

```
      4
     / \
    10  3
   / \
  5   1
```

Shape is already complete because array indices are contiguous.

Only heap-order property may be wrong.

So:

Fix internal nodes from bottom to top.

4. WHY START FROM INTERNAL NODES?

Leaves already satisfy heap property automatically.

A leaf has:

TEXT
0 children

So there is nothing to violate.

Therefore we do not heapify leaves.

For 0-based array:

TEXT
First leaf = $\text{floor}(n/2)$

Hence:

TEXT
Last internal node = $\text{floor}(n/2) - 1$

0-based Heap starts there.

5. BUILD MIN-HEAP ALGORITHM

For 0-based array:

TEXT

```
for i = floor(n/2)-1 down to 0:  
    minHeapifyDown(a,n,i)
```

That is the entire algorithm.

6. C CODE — BUILD MIN-HEAP

C

```
void buildMinHeap(int heap[], int n)  
{  
    for (int i = n / 2 - 1; i >= 0; i--)  
        minHeapifyDown(heap, n, i);  
}
```

For max-heap:

C

```
void buildMaxHeap(int heap[], int n)  
{  
    for (int i = n / 2 - 1; i >= 0; i--)  
        maxHeapifyDown(heap, n, i);  
}
```

Very important.

7. WHY DO WE MOVE FROM BOTTOM TO TOP?

Recall heapify-down's precondition:

The left and right child subtrees must already be heaps.

Suppose we process an internal node near the bottom.

Its children are leaves.

Leaves are already heaps.

So heapify-down works.

Then move to its parent.

By that time:

TEXT

its child subtrees have already been fixed

Eventually root is processed last.

This is the logic behind bottom-up order.

8. WHY NOT ROOT TO BOTTOM?

Suppose arbitrary tree:

TEXT



If we heapify root first for min-heap:

TEXT

```
1 "d 5
1 "d 3
```

Root looks fine.

Heapify stops.

But lower subtree:

TEXT

```
5
/
2
```

violates:

TEXT

```
5 "d 2 'L
```

So processing root before fixing children does not guarantee final heap.

Bottom-up solves this.

9. FULL BUILD-MIN-HEAP EXAMPLE

Given:

TEXT

```
[4,10,3,5,1]
```

n = 5

Last internal node:

TEXT

```
floor(5/2)-1
=
2-1
=
1
```

So process:

TEXT

```
i = 1
then
i = 0
```

10. HEAPIFY INDEX 1

Current:

TEXT

```
[4,10,3,5,1]
```

Index 1 value:

TEXT

```
10
```

Children:

TEXT

```
l e f t   =   3   ! '   v a l u e   5
r i g h t  =   4   ! '   v a l u e   1
```

For min-heap choose smaller child:

TEXT

```
1
```

Swap:

TEXT

```
1 0   ! "   1
```

Array becomes:

TEXT

```
[4,1,3,5,10]
```

Tree:

TEXT

```
      4
     / \
    1   3
   / \
  5  10
```

Subtree rooted at index 1 is now a min-heap.

11. HEAPIFY INDEX 0

Root:

TEXT

4

Children:

TEXT

1 and 3

Choose smaller:

TEXT

1

Swap:

TEXT

[1, 4, 3, 5, 10]

Now 4 is at index 1.

Children:

TEXT

5, 10

Check:

TEXT

4 "d 5

4 "d 1 0

Stop.

Final:

[1,4,3,5,10]

12. FINAL MIN-HEAP

TEXT

```
      1
     /\
    4  3
   /\
  5 10
```

Check:

TEXT

```
1"d 4 , 3
4"d 5 , 1 0
```

Valid.

13. ANOTHER BUILD HEAP EXAMPLE Ø=Ý%

Build max-heap from:

TEXT

```
[4,10,3,5,1]
```

Again:

TEXT

```
last internal = 1
```

Process index 1.

Value:

TEXT

```
10
```

Children:

TEXT

```
5,1
```

For max-heap:

TEXT

```
1 0 "e 5
1 0 "e 1
```

Already valid.

Array unchanged:

TEXT

```
[4,10,3,5,1]
```

14. PROCESS ROOT

Root:

TEXT

```
4
```

Children:

TEXT
10, 3

Choose larger:

TEXT
10

Swap:

TEXT
[10, 4, 3, 5, 1]

Now 4 index 1.

Children:

TEXT
5, 1

Choose 5.

Swap:

TEXT
[10, 5, 3, 4, 1]

Final max-heap:

TEXT

```
      10
     /  \
    5    3
   /  \
  4    1
```

15. BUILD HEAP COMPLEXITY — THE BIG QUESTION Ø=Ý%Ø=Ý%Ø=Ý%

A single heapify-down can take:

TEXT
 $O(\log n)$

There are approximately:

TEXT
 $n/2$

internal nodes.

A rough multiplication gives:

TEXT

$(n/2) \times \log n$
=
 $O(n \log n)$

But this is only a loose upper bound.

It is **not tight**.

Actual Build Heap complexity:

$\sim (n)$

Why?

Because:

Most nodes are very close to the leaves and therefore require almost no heapify work.

16. CORE INTUITION

Consider a large complete tree.

Roughly:

TEXT

$n/2$ nodes are leaves

Their heapify cost:

TEXT

0

About:

TEXT

$n/4$ nodes

are one level above leaves.

Their maximum heapify distance:

TEXT

1

About:

TEXT

$n/8$ nodes

are two levels above leaves.

Their cost:

TEXT

2

And so on.

Only:

TEXT

1 root

can travel approximately:

TEXT

$\log n$

levels.

$O(\sqrt{n})$ Very few nodes pay the expensive cost.

17. COST DISTRIBUTION

Approximately:

| Nodes | Maximum heapify work |
|--------|----------------------|
| $n/2$ | 0 |
| $n/4$ | 1 |
| $n/8$ | 2 |
| $n/16$ | 3 |
| ... | ... |

So total work is approximately:

TEXT

$(n/4)(1)$
+
 $(n/8)(2)$
+
 $(n/16)(3)$
+ ...

Leaves contribute zero.

18. WRITE THE SERIES

Factor out n :

TEXT

```
T(n)
"H
n[
  1/4
+ 2/8
+ 3/16
+ 4/32
+ ...
]
```

Equivalent:

TEXT

```
T(n)
"H
n  £ ( k / 2 ^ ( k + 1 ) )
```

The infinite series converges to a constant.

Therefore:

$T(n) = \sim(n)$

$\Theta(n)$ is the rigorous reason.

19. IMPORTANT SERIES

Known result:

TEXT

```
£ k / 2 ^ k
f r o m k = 1 t o "
=
2
```

Therefore:

TEXT

```
£ k / 2 ^ ( k + 1 )
=
1
```

So total work is bounded by a constant multiple of n .

Hence:

Build Heap = $O(n)$

And because every input element must at least be represented/considered as part of the construction process:

Build Heap = $\sim(n)$

20. ANOTHER WAY TO SEE IT

Suppose heap height is h .

Number of nodes at height k from bottom is at most approximately:

TEXT

$$n / 2^{(k+1)}$$

Each such node costs:

TEXT

$$O(k)$$

Therefore:

TEXT

$$\begin{aligned} T(n) \\ = \\ \sum [n / 2^{(k+1)}] O(k) \end{aligned}$$

Factor n :

TEXT

$$\begin{aligned} T(n) \\ = \\ O(n) \sum k / 2^{(k+1)} \end{aligned}$$

Series is constant.

Thus:

TEXT

$$T(n) = O(n)$$

~~$O(n \log n)$~~ is a standard GATE derivation.

21. WHY $n \times \log n$ IS TOO PESSIMISTIC

$O(\log n)$ heapify is only true as a maximum for a node near root.

It assumes every internal node might move $\log n$ levels.

But that's impossible.

A node near the leaves only has:

TEXT

1 or 2 levels below it

So its heapify cannot cost $\log n$.

The per-node bounds differ.

That's why simply multiplying:

TEXT

number of nodes $\times$ worst per-node cost

gives a valid but non-tight upper bound.

22. IMPORTANT GATE DISTINCTION

These are both correct statements:

Statement A

Each individual heapify is at most:

TEXT

$O(\log n)$

Statement B

Bottom-up Build Heap over all nodes is:

TEXT

$O(n)$

There is no contradiction.

Because most heapify calls cost much less than $\log n$.

23. REPEATED INSERTION VS BOTTOM-UP

| Method | Technique | Time |
|----------------------|---------------------------------|---------------------|
| Repeated insertion | Add each item, sift up | $O(n \log n)$ worst |
| Bottom-up Build Heap | Heapify internal nodes downward | $\sim (n)$ |

$\emptyset = \text{Must know.}$

24. DOES REPEATED INSERTION ALWAYS TAKE $\sim (n \log n)$?

Be precise.

Worst-case upper bound:

TEXT

$O(n \log n)$

and there are inputs producing:

TEXT

$\sim (n \log n)$

work.

But specific insertion sequences can require much less.

For example, inserting already suitable values into a min-heap may cause few swaps.

So standard comparison:

TEXT

Repeated insertion !' $O(n \log n)$
Bottom-up build !' $\sim (n)$

25. BUILD HEAP DOES NOT PRODUCE UNIQUE HEAP

Given the same set of values, there may be many valid heaps.

The particular heap produced depends on:

- original array order
- heapify procedure
- tie handling

Example values:

TEXT

1, 2, 3, 4, 5

Both may be valid min-heaps:

TEXT

[1, 2, 3, 4, 5]

and:

TEXT

[1, 3, 2, 4, 5]

So heap property does not define a unique arrangement.

26. DOES BUILD HEAP SORT THE ARRAY?

No.

Example:

TEXT

[4, 10, 3, 5, 1]

became min-heap:

TEXT

[1, 4, 3, 5, 10]

Not sorted because:

TEXT

4 > 3

Build Heap only creates:

TEXT

partial heap ordering

not total ordering.

27. BUILD HEAP IN PLACE

Given an array:

TEXT

a[]

the algorithm rearranges elements inside the same array.

Iterative heapify can use:

TEXT

$O(1)$

auxiliary memory.

So:

TEXT

Build Heap auxiliary space = $O(1)$

for an iterative in-place implementation.

Recursive heapify may use call stack:

TEXT

$O(\log n)$

worst case.

28. START INDEX — GATE FAVORITE

For n elements using 0-based indexing:

TEXT

last internal index = $\text{floor}(n/2)-1$

Example:

TEXT

$n = 20$

Start:

TEXT

$20/2 - 1$

$=$

9

Process:

TEXT

9, 8, 7, ..., 0

Indices:

TEXT

10...19

are leaves.

No need to heapify them.

29. 1-BASED VERSION

For indices:

TEXT

1...n

Last internal node:

TEXT

$\text{floor}(n/2)$

Build Heap loops:

TEXT

$i = \text{floor}(n/2)$ down to 1

Again, indexing convention matters.

30. GATE NUMERICAL — START INDEX

Array has:

```
TEXT
n=15
```

0-based.

Last internal:

```
TEXT
floor(15/2)-1
=
7-1
=
6
```

So heapify order:

```
TEXT
6, 5, 4, 3, 2, 1, 0
```

Leaves:

```
TEXT
7...14
```

31. GATE NUMERICAL — n=16

Last internal:

```
TEXT
floor(16/2)-1
=
8-1
=
7
```

Index 7 has only a left child:

```
TEXT
left = 15
right = 16 absent
```

because even-sized complete tree has one internal node with only left child.

32. FULL BUILD-MIN-HEAP TRACE

Given:

TEXT

[9, 4, 7, 1, 5, 3]

n=6

Last internal:

TEXT

6/2 -1

=

2

Process:

TEXT

2, 1, 0

33. INDEX 2

Value:

TEXT

7

Children:

TEXT

index 5 = 3

right child does not exist.

Since:

TEXT

7 >3

swap:

TEXT

[9, 4, 3, 1, 5, 7]

34. INDEX 1

Value:

TEXT

4

Children:

TEXT

1 and 5

Actually child values:

TEXT

index3 = 1
index4 = 5

Choose smaller:

TEXT

1

Swap:

TEXT

[9, 1, 3, 4, 5, 7]

35. INDEX 0

Value:

TEXT

9

Children:

TEXT

1 and 3

Choose 1.

Swap:

TEXT

[1, 9, 3, 4, 5, 7]

Now 9 at index 1.

Children:

TEXT

4 and 5

Choose 4.

Swap:

TEXT

[1, 4, 3, 9, 5, 7]

Final:

[1,4,3,9,5,7]

36. CHECK FINAL HEAP

Tree:

TEXT

```
      1
     /\
    4  3
   /\ /\
  9 5 7
```

Check:

TEXT

```
1"d 4 , 3
4"d 9 , 5
3"d 7
```

Valid.

37. NOTICE SOMETHING IMPORTANT

When heapifying index 0, the node moved through:

TEXT

```
0 !' 1 !' 3
```

Only one path.

It did not disturb the already-heapified subtree rooted at index 2.

This is why bottom-up processing works cleanly.

38. BUILD MAX-HEAP TRACE

Given:

TEXT

[3, 5, 1, 10, 2, 7]

n=6

Start index:

TEXT

2

39. HEAPIFY $i=2$

Value:

TEXT

1

Child:

TEXT

$\text{index}5 = 7$

For max-heap:

TEXT

$7 > 1$

swap:

TEXT

$[3, 5, 7, 10, 2, 1]$

40. HEAPIFY $i=1$

Value:

TEXT

5

Children:

TEXT

$10, 2$

Choose 10.

Swap:

TEXT

$[3, 10, 7, 5, 2, 1]$

41. HEAPIFY $i=0$

Root 3.

Children:

TEXT

10, 7

Choose 10.

Swap:

TEXT

[10, 3, 7, 5, 2, 1]

Now 3 children:

TEXT

5, 2

Choose 5.

Swap:

TEXT

[10, 5, 7, 3, 2, 1]

Final max-heap.

42. GATE TRAP — HEAPIFY ALL N NODES?

You may write a loop over all nodes, but heapifying leaves is unnecessary.

Correct efficient starting point:

TEXT

$\text{floor}(n/2) - 1$

and move backward.

Leaves already satisfy heap property.

43. GATE TRAP — START FROM INDEX 0 AND MOVE FORWARD?

Not standard bottom-up build.

Reason:

Heapify-down assumes children are heaps.

Moving from root forward may process parent before child subtrees are repaired.

Correct order:

Bottom !' Top

44. GATE TRAP — BUILD HEAP = $O(n \log n)$?

'L False for bottom-up Build Heap.

Correct:

$\sim(n)$

Repeated insertion is the method with $O(n \log n)$ worst-case bound.

$\emptyset = \text{Extremely likely GATE confusion.}$

45. GATE TRAP — WHY NOT $O(n/2 \times \log n)$?

Because not all $n/2$ internal nodes have height $\log n$.

Most have subtree height:

TEXT

1

or:

TEXT

2

Only a tiny number are near root.

46. PERFECT TREE COST INTUITION

Suppose perfect heap has height h .

At depth:

TEXT

$h-1$

there are:

TEXT

$2^{(h-1)}$

nodes.

Each has subtree height:

TEXT

1

At depth:

TEXT

$h-2$

there are:

TEXT

$2^{(h-2)}$

nodes.

Each can move at most:

TEXT

2

levels.

And so on.

Total remains linear in number of nodes.

47. ALTERNATIVE EXACT-STYLE BOUND

For a heap of n nodes, total sift-down distance in bottom-up heap construction is bounded by:

TEXT

$< n$

up to constant-factor conventions for swaps/comparisons in the classic analysis.

Therefore:

TEXT

$\sim (n)$

This is why Floyd's bottom-up heap construction is linear.

0=0 The exact operation count depends on implementation, but asymptotic result is fixed.

48. FLOYD'S BUILD-HEAP

Bottom-up heap construction is often associated with:

Floyd's heap construction algorithm

Core:

TEXT

for each internal node from bottom to root:
sift down

Useful terminology for interviews and textbooks.

49. BUILD HEAP VS HEAPIFY

Do not confuse the terms.

Heapify-down

Repairs:

TEXT

one subtree/root position

Worst time:

TEXT

$O(\log n)$

Build Heap

Calls heapify-down across the necessary internal nodes.

Total:

TEXT

$\sim (n)$

50. BUILD HEAP VS INSERT

Insert

Assumes:

TEXT

existing array is already heap

Adds one new element.

Repair:

TEXT

up

Build Heap

Assumes:

TEXT

arbitrary array

Repairs all internal nodes:

TEXT

bottom-up/downward

$O(n)$ different operations.

51. PRIORITY QUEUE INITIALIZATION

Suppose you already have n jobs and want to initialize a heap-based priority queue.

Do not necessarily insert each one individually.

If all values are already available:

TEXT

Build Heap

can initialize priority structure in:

$\sim(n)$

This is often faster than:

TEXT

$O(n \log n)$

repeated insertion.

52. HEAPSORT CONNECTION

Heap Sort begins by:

TEXT

Build Max-Heap

This first phase costs:

TEXT

$\sim(n)$

Then repeatedly extracts/moves maximum:

TEXT

$n \text{ times} \times O(\log n)$

Therefore total Heap Sort:

TEXT

$\sim(n \log n)$

Build Heap does not dominate.

0=7% is important for Lesson 70.

53. IF BUILD HEAP IS $O(n)$, WHY HEAPSORT ISN'T $O(n)$?

Because after building the heap, we still repeatedly perform:

TEXT

extract/root swap + heapify

roughly n times.

Each can cost:

TEXT

$O(\log n)$

So:

TEXT

$O(n) + O(n \log n)$
=
 $O(n \log n)$

54. BUILD HEAP OUTPUT MAY DIFFER BY METHOD

Repeated insertion and bottom-up Build Heap on the same input order can produce different valid heaps.

Example:

TEXT

input = [4, 10, 3, 5, 1]

Bottom-up min-heap:

TEXT

[1, 4, 3, 5, 10]

Repeated insertion may produce a different valid arrangement.

That is okay.

Heap validity does not require unique shape/value arrangement beyond complete shape and parent-child ordering.

55. GATE-STYLE QUESTION 1

Bottom-up build-heap on an array of n elements takes:

A. $\sim (10)$ B. $\sim (6)$ C. $\sim (n \log n)$ D. $\sim (n^2)$

Answer:

56. GATE-STYLE QUESTION 2

Last internal node of a 0-based heap array of size n :

Answer:

TEXT

$\text{floor}(n/2) - 1$

57. GATE-STYLE QUESTION 3

Why are leaves skipped during build heap?

Because:

TEXT

a single node is already a valid heap

No children means no parent-child violation.

58. GATE-STYLE QUESTION 4

Suppose:

TEXT

$n=100$

How many nodes are leaves?

TEXT

50

Those 50 nodes require no heapify work.

This already hints why average per-node Build Heap work is small.

59. GATE-STYLE QUESTION 5

Which is faster asymptotically for constructing a heap from an existing array?

Repeated insertion

TEXT

$O(n \log n)$

Bottom-up Build Heap

TEXT

~ (n)

Answer:

Bottom-up Build Heap

60. GATE-STYLE TRACE

Build min-heap from:

TEXT

[8, 7, 6, 5, 4, 3, 2, 1]

n=8

Last internal:

TEXT

$8/2 - 1 = 3$

Process:

TEXT

3, 2, 1, 0

Try manually.

Important:

After each heapify, use the **updated array** for the next index.

~~0=0%~~ Do not restart from the original array.

61. BUILD HEAP AND COMPARISON DIRECTION

Min-heap:

TEXT

choose smaller child

Max-heap:

TEXT

choose larger child

Exactly the same heapify-down rule as extraction.

62. BUILD HEAP SUMMARY ALGORITHM

For min-heap:

TEXT

BUILD-MIN-HEAP(A, n)

for $i = \text{floor}(n/2) - 1$ down to 0 :
 MIN-HEAPIFY-DOWN(A, n, i)

For max-heap:

TEXT

BUILD-MAX-HEAP(A, n)

for $i = \text{floor}(n/2) - 1$ down to 0 :
 MAX-HEAPIFY-DOWN(A, n, i)

63. COMPLEXITY SUMMARY

One heapify-down

TEXT

$O(\log n)$

Repeated insertion heap construction

TEXT

$O(n \log n)$

Bottom-up Build Heap

$\sim (n)$

Iterative Build Heap auxiliary space

TEXT

$O(1)$

64. MEMORY TRICK

BUILD HEAP = START WHERE CHILDREN START TO EXIST

That is:

TEXT

last internal node

Then:

BOTTOM !' ROOT

Why?

Children must be heaps before parent is heapified.

65. MEMORY TRICK FOR COMPLEXITY

****Half the nodes do zero work.**

A quarter do one step. An eighth do two steps.**

Therefore:

Total = linear

$O(n)$ is the easiest conceptual memory.

66. NOTEBOOK NOTES

Bottom-Up Build Heap

TEXT

Start:

$i = \text{floor}(n/2) - 1$

Go:

$i \neq 0$

At each:

heapify down

Complexity:

TEXT

$\sim (n)$

Reason:

TEXT

most nodes are near leaves

67. DON'T CONFUSE

Heapify one root

TEXT

$O(\log n)$

Build entire heap bottom-up

TEXT

$\sim (n)$

Build by n insertions

TEXT

$O(n \log n)$

0-1 heap separate facts.

68. 30-SECOND REVISION

Remember:

1. Arbitrary array already represents a complete tree.
2. Only heap order must be repaired.
3. Leaves need no heapify.
4. Start from $\text{floor}(n/2) - 1$ in 0-based array.
5. Move backward to root.
6. Use sift-down.
7. Child subtrees are fixed before parent.
8. One heapify can be $O(\log n)$.
9. Bottom-up Build Heap is $\sim (n)$.
10. Repeated insertion is $O(n \log n)$ worst case.

69. ACTIVE RECALL

Q1

Where does Build Heap start in 0-based indexing?

Q2

Why not heapify leaves?

Q3

Why process bottom-up?

Q4

Time for one heapify-down?

Q5

Time for full bottom-up Build Heap?

Q6

Why isn't Build Heap $n \log n$?

ANSWERS

A1

TEXT

$\text{floor}(n/2) - 1$

A2

A leaf has no children, so it is already a valid heap.

A3

Heapify-down requires child subtrees to already be heaps.

A4

TEXT

$O(\log n)$

A5

TEXT

$\sim (n)$

A6

Most nodes are near leaves and have only constant or zero heapify distance; only very few nodes can move $\sim (\log n)$ levels.

70. PRACTICE — BUILD MIN-HEAP

Build a min-heap from:

TEXT

[12, 7, 9, 3, 10, 5, 8, 1]

Write:

1. last internal index
2. heapify order
3. array after each heapify
4. final min-heap
5. final tree

71. PRACTICE — BUILD MAX-HEAP

Build max-heap from:

TEXT

[2, 8, 5, 3, 9, 1, 7, 6]

Trace every bottom-up heapify.

72. PRACTICE — COMPLEXITY

Explain in your own words why these are different:

TEXT

```
H e a p i f y   o n e   n o d e   ! '   0 ( l o g   n )
B u i l d   H e a p           ! '   ~ ( n )
n   i n s e r t i o n s       ! '   0 ( n   l o g   n )
```

If you can explain this clearly, your heap complexity foundation is strong.

73. MODULE 8 PROGRESS

' Lesson 65 — Heap Fundamentals ' Lesson 66 — Array Representation ' Lesson 67 — Heap Insertion ' Lesson 68 — Sift Down / Extraction ' **Lesson 69 — Build Heap & $\sim(n \log n)$ proof**

Remaining:

- Lesson 70 — Heap Sort
- Lesson 71 — Priority Queue & Heap Applications

NEXT — LESSON 70

Heap Sort — Build Max-Heap, Repeated Root Swap, Heapify & $\sim(n \log n)$

Next we will trace an entire sorting process:

TEXT

```
Array
! '   B u i l d   m a x - h e a p
! '   m a x i m u m   r e a c h e s   r o o t
! '   s w a p   r o o t   w i t h   l a s t
! '   s h r i n k   a c t i v e   h e a p
! '   h e a p i f y   r o o t
! '   r e p e a t
```

and master:

- ascending vs descending heapsort
- why max-heap gives ascending result
- exact array trace
- $\sim(n \log n)$ analysis
- in-place property
- why heapsort is generally not stable

- best/average/worst complexity
- comparison with merge sort and quicksort
- GATE traps.

USER

gate

ASSISTANT

Understood — from here, I'll keep this subject **strictly GATE-focused first** and avoid unnecessary placement/interview extensions unless they directly help a GATE concept.

For the remaining Binary Heap lessons, I'll prioritize:

- ~~0-8%~~ exact GATE concepts and formulas
- GATE-style numericals and traps
- operation traces
- time/space complexities
- PYQ-type reasoning
- short memory tricks
- only essential C/pseudocode

So the next lesson will be:

MODULE 8 — LESSON 70

Heap Sort — GATE Masterclass

We'll cover **Build Max-Heap !' root-last swap !' reduce heap size !' heapify**, exact array traces, why max-heap gives ascending order, `~ ( n - 1 )` proof, implce/stability properties, and the important GATE comparison points.

Send **NEXT**.

USER

next

ASSISTANT

MODULE 8 — LESSON 70

Heap Sort — GATE Masterclass

We now study **Heap Sort**, one of the standard comparison-based sorting algorithms in GATE.

The full idea is:

Build a Heap !' Repeatedly remove the root extreme

For **ascending order**, we normally use a:

Max-Heap

because the maximum element is at the root.

GATE importance: 0=100% MUST KNOW

1. CORE IDEA OF HEAP SORT

Suppose array:

TEXT

[4, 10, 3, 5, 1]

Goal:

TEXT

[1, 3, 4, 5, 10]

Heap Sort does:

TEXT

Step 1:

Build MAX-HEAP

Step 2:

Largest element is at root

Step 3:

Swap root with last active element

Step 4:

Reduce heap size by 1

Step 5:

Heapify root

Step 6:

Repeat

2. WHY MAX-HEAP FOR ASCENDING ORDER?

In a max-heap:

TEXT

```
root = maximum
```

We swap maximum with the last element.

So the largest element goes permanently to the:

TEXT

rightmost end

Then the second-largest goes just before it.

Then third-largest.

So final array becomes:

TEXT

s m a l l ! ' l a r g e

Hence:

Max-Heap !' Ascending Heap Sort

Ø=Memory trick:

MAX goes to END !' ascending order

3. WHAT ABOUT DESCENDING ORDER?

For descending order, use:

Min-Heap

because the smallest element repeatedly moves to the end.

Final arrangement becomes:

TEXT

l a r g e ! ' s m a l l

So:

| Heap Used | Final Order |
|-----------|-------------|
| Max-Heap | Ascending |
| Min-Heap | Descending |

Ø=Common GATE question.

4. COMPLETE HEAP SORT EXAMPLE

Sort:

TEXT

[4, 10, 3, 5, 1]

ascending.

First:

Build Max-Heap

From Lesson 69, max-heap becomes:

TEXT

[10, 5, 3, 4, 1]

Tree:

TEXT



Now sorting begins.

5. PASS 1

Active heap:

TEXT

[10, 5, 3, 4, 1]

Maximum:

TEXT

10

Swap root with last active element:

TEXT

[1, 5, 3, 4, 10]

Now:

TEXT

10

is in its final sorted position.

Reduce heap size:

TEXT

4

Active heap:

TEXT

[1, 5, 3, 4]

Sorted region:

TEXT

[10]

6. HEAPIFY ACTIVE PART

Current active tree:

TEXT



For max-heap:

children:

TEXT

5, 3

choose larger:

TEXT

5

Swap:

TEXT

[5, 1, 3, 4, 10]

Now 1 has child:

TEXT

4

Swap:

TEXT

[5, 4, 3, 1, 10]

Active max-heap restored.

7. PASS 2

Current:

TEXT

[5, 4, 3, 1, 10]

Active heap size:

TEXT

4

Root maximum:

TEXT

5

Swap root with last active element, index 3:

TEXT

[1, 4, 3, 5, 10]

Now sorted suffix:

TEXT

[5, 10]

Reduce active size to:

TEXT

3

8. HEAPIFY

Active:

TEXT

[1, 4, 3]

Root children:

TEXT

4, 3

choose:

TEXT

4

Swap:

TEXT

[4, 1, 3, 5, 10]

Heap restored.

9. PASS 3

Active heap:

TEXT

[4, 1, 3]

Swap root 4 with last active element 3:

TEXT

[3, 1, 4, 5, 10]

Sorted suffix:

TEXT

[4, 5, 10]

Active size:

TEXT

2

Active heap:

TEXT

[3, 1]

Already max-heap.

10. PASS 4

Swap root 3 with last active element 1:

TEXT

[1, 3, 4, 5, 10]

Done.

Final:

[1,3,4,5,10]

11. MASTER TRACE TABLE

| Stage | Array |
|----------------|--------------|
| Original | [4,10,3,5,1] |
| Build max-heap | [10,5,3,4,1] |
| Put 10 at end | [5,4,3,1,10] |
| Put 5 at end | [4,1,3,5,10] |
| Put 4 at end | [3,1,4,5,10] |
| Put 3 at end | [1,3,4,5,10] |

Ø=10 is the exact GATE logic.

12. IMPORTANT IDEA — TWO REGIONS

During Heap Sort, think of the array as:

TEXT

[ACTIVE HEAP | SORTED REGION]

Example:

TEXT

[5, 4, 3, 1 | 10]

Later:

TEXT

[4, 1, 3 | 5, 10]

Later:

TEXT

[3, 1 | 4, 5, 10]

The sorted region grows from:

right to left

for ascending max-heap Heap Sort.

13. HEAP SIZE VS ARRAY SIZE

Very important.

Suppose:

TEXT

[5, 4, 3, 1, 10]

Array size is still:

TEXT

5

but active heap size is:

TEXT

4

The element:

TEXT

10

must not participate in further heapify operations.

0-6% Common implementation/GATE tracing mistake.

14. HEAP SORT PSEUDOCODE

For ascending order:

TEXT

BUILD-MAX-HEAP(A)

for end = n-1 down to 1:

 swap(A[0], A[end])

 heapSize--

 MAX-HEAPIFY(A, 0, heapSize)

15. SIMPLE C LOGIC

C

```
void heapSort(int a[], int n)
{
    buildMaxHeap(a, n);

    for (int end = n - 1; end > 0; end--)
    {
        int temp = a[0];
        a[0] = a[end];
        a[end] = temp;

        maxHeapifyDown(a, end, 0);
    }
}
```


Notice:

```
TEXT
end
```

also acts as current heap size.

Why?

After swapping with `a[end]`, valid active indices are:

```
TEXT
0 ... end-1
```

So active count:

```
TEXT
end
```

16. TIME COMPLEXITY — PHASE 1

Build Max-Heap:

$\sim(n)$

from Lesson 69.

17. TIME COMPLEXITY — PHASE 2

We perform approximately:

```
TEXT
n-1
```

root removals/swaps.

Each heapify can cost:

```
TEXT
O(log n)
```

So second phase:

```
TEXT
O(n log n)
```

Therefore:

FORMULA BOX

TEXT

Heap Sort

$$\begin{aligned} &= \sim(n) + \sim(n \log n) \\ &= \sim(n \log n) \end{aligned}$$

18. BEST-CASE TIME $\Theta = \%$

Heap Sort still performs the repeated extraction/heapify structure.

Standard complexity:

$$\sim(n \log n)$$

best case.

19. AVERAGE-CASE TIME

$$\sim(n \log n)$$

20. WORST-CASE TIME

$$\sim(n \log n)$$

$\Theta = \%$ Important advantage:

Heap Sort has guaranteed:

TEXT

$$\sim(n \log n)$$

worst-case time.

21. MASTER COMPLEXITY TABLE

| Case | Heap Sort Time |
|---------|------------------|
| Best | $\sim(n \log n)$ |
| Average | $\sim(n \log n)$ |
| Worst | $\sim(n \log n)$ |

This is very important for GATE comparison questions.

22. AUXILIARY SPACE $O(1)$

Heap Sort can be performed directly inside the original array.

It does not require another array proportional to n .

With iterative heapify:

Auxiliary Space = $O(1)$

Thus Heap Sort is:

In-place

under standard implementation.

23. WHAT DOES IN-PLACE MEAN?

It means sorting uses only constant extra memory, apart from the input array itself.

Heap Sort:

TEXT

$O(1)$ auxiliary space

Merge Sort on arrays normally:

TEXT

$O(n)$ auxiliary space

This distinction is often tested.

24. IS HEAP SORT STABLE? $O(1)$

Generally:

NO

Heap Sort is **not stable**.

25. WHAT IS STABILITY?

Suppose records:

TEXT

(5, A)

(3, X)

(5, B)

Two keys are equal:

TEXT

5A
5B

A stable sort preserves their original relative order:

TEXT

5A before 5B

Heap Sort uses long-distance swaps.

Equal elements may change relative order.

Therefore:

Heap Sort is not stable

26. MEMORY TRICK

Heap Sort:

TEXT

T i m e ! ' N L O G N a l w a y s
S p a c e ! ' c o n s t a n t
S t a b l e ! ' N o

Mnemonic:

Heap = predictable speed, tiny space, no stability.

27. WHY BUILD HEAP DOESN'T DOMINATE

First phase:

TEXT

$\sim (n)$

Second:

TEXT

$\sim (n \log n)$

Therefore:

TEXT

$\sim (n \log n)$

dominates.

So total:

```
TEXT
~ ( n   l o g   n )
```

28. EXACT SORTING INTUITION

For max-heap:

```
TEXT
largest
```

is always root.

After first swap:

```
TEXT
largest !'   f i n a l   p o s i t i o n   n - 1
```

After restoring heap:

```
TEXT
second - largest !'   r o o t
```

Next swap:

```
TEXT
second - largest !'   n - 2
```

and so on.

Hence sorted suffix is produced one element at a time.

29. GATE TRAP — MAX-HEAP ARRAY ALREADY SORTED?

No.

Example max-heap:

```
TEXT
[10,8,9,2,5,6,7]
```

Valid.

But:

```
TEXT
8 < 9
```

between adjacent positions.

Heap only guarantees:

TEXT

p a r e n t " e c h i l d r e n

Heap Sort needs repeated root extraction to obtain total order.

30. GATE TRAP — BUILD HEAP ALONE IS SORTING?

'LNo.

Build Heap:

TEXT

~ (n)

only creates heap property.

Heap Sort requires:

TEXT

repeated extraction + heapify

afterward.

31. GATE TRAP — MIN-HEAP FOR ASCENDING?

If using the normal in-place root-to-end Heap Sort procedure:

- max-heap !' ascending
- min-heap !' descending

~~$O(n \log n)$~~ not reverse them.

32. WHY?

Suppose min-heap:

TEXT

root = smallest

Swap smallest to end.

Then next-smallest also goes near end.

So the array's right side fills as:

TEXT

smallest,
then second-smallest to its left,
...

leading to descending order overall.

33. EXAMPLE — MIN-HEAP DESCENDING

Suppose values:

```
TEXT
1, 2, 3, 4
```

Min-heap root 1.

Move 1 to last:

```
TEXT
_ _ _ 1
```

Then 2 to previous:

```
TEXT
_ _ 2 1
```

Then 3:

```
TEXT
_ 3 2 1
```

Final:

```
TEXT
4 3 2 1
```

descending.

34. HEAP SORT LOOP COUNT

For `n` elements, after building heap:

We need:

```
TEXT
n-1
```

root-last swaps.

Why not `n`?

When only one active element remains:

```
TEXT
it is already in correct position
```

No work needed.

35. GATE-STYLE TRACE

Max-heap:

TEXT

[20, 15, 18, 8, 10, 5]

This is already max-heap.

Heap Sort ascending.

First swap:

TEXT

20 15 18 8 10 5

Temporary:

TEXT

[5, 15, 18, 8, 10, 20]

Active size 5.

36. HEAPIFY

Root 5 children:

TEXT

15, 18

choose 18.

TEXT

[18, 15, 5, 8, 10, 20]

Node 5 has no active child requiring swap.

Now:

TEXT

[18, 15, 5, 8, 10 | 20]

37. SECOND PASS

Swap:

TEXT

18 15 5 8 10 20

TEXT

[10, 15, 5, 8, 18, 20]

Active size 4.

Root children:

TEXT

15, 5

choose 15.

TEXT

[15, 10, 5, 8, 18, 20]

Now active:

TEXT

[15, 10, 5, 8]

38. THIRD PASS

Swap:

TEXT

1 5 ! " 8

TEXT

[8, 10, 5, 15, 18, 20]

Active size 3.

Root children:

TEXT

10, 5

swap with 10:

TEXT

[10, 8, 5, 15, 18, 20]

39. FOURTH PASS

Swap 10 with 5:

TEXT

[5, 8, 10, 15, 18, 20]

Active size 2.

Heapify:

TEXT

[8, 5, 10, 15, 18, 20]

40. FIFTH PASS

Swap:

TEXT

[5, 8, 10, 15, 18, 20]

Final ascending array.

41. GATE PROPERTY — COMPARISON SORT

Heap Sort is a:

Comparison-based sorting algorithm

because ordering decisions depend on comparisons such as:

TEXT

parent vs child

Therefore the general comparison-sorting lower bound:

TEXT

$\Theta(n \log n)$

applies in the worst-case decision-tree model.

Heap Sort achieves:

TEXT

$\sim (n \log n)$

42. HEAP SORT VS SELECTION SORT — INTUITION

Selection Sort repeatedly finds maximum/minimum by scanning:

TEXT

$O(n)$

each time.

Heap Sort also repeatedly selects an extreme, but heap maintains extreme efficiently.

Selection Sort:

TEXT

$\sim (n^2)$

Heap Sort:

TEXT

$\sim (n \log n)$

Think of Heap Sort as:

Efficient repeated selection using a heap.

Good conceptual memory.

43. HEAP SORT VS MERGE SORT — GATE

| Property | Heap Sort | Merge Sort |
|-----------------------|-------------------------------|------------------------------|
| Best | $\sim (n \log n)$ | $\sim (n \log n)$ |
| Average | $\sim (n \log n)$ | $\sim (n \log n)$ |
| Worst | $\sim (n \log n)$ | $\sim (n \log n)$ |
| Auxiliary array space | $O(1)$ iterative heap version | $O(n)$ typical array version |
| Stable | No | Yes, standard implementation |
| In-place | Yes | Usually no for arrays |

Important GATE comparison.

44. HEAP SORT VS QUICK SORT — STANDARD GATE VIEW

| Property | Heap Sort | Quick Sort |
|----------|-------------------|--|
| Average | $\sim (n \log n)$ | $\sim (n \log n)$ |
| Worst | $\sim (n \log n)$ | $\sim (n^2)$ |
| In-place | Yes | Usually considered in-place apart from recursion stack |
| Stable | No | No, standard version |

Quick Sort often performs very well practically, but GATE complexity questions focus on the formal bounds.

45. HEAP SORT VS INSERTION SORT

| Property | Heap Sort | Insertion Sort |
|----------|-------------------|----------------|
| Best | $\sim (n \log n)$ | $\sim (n)$ |
| Average | $\sim (n \log n)$ | $\sim (n^2)$ |
| Worst | $\sim (n \log n)$ | $\sim (n^2)$ |
| Stable | No | Yes |
| In-place | Yes | Yes |

This helps identify algorithm properties in MCQs.

46. IMPORTANT GATE TRAP — BEST CASE

Some students think if input already sorted:

TEXT

Heap Sort = $O(n)$

'LNo.

Standard Heap Sort still performs:

TEXT

heap construction
+
repeated swaps/heapify

Its asymptotic best-case complexity is generally stated:

$\sim (n \log n)$

47. HEAPIFY SPACE

If `maxHeapifyDown()` is iterative:

TEXT

$O(1)$

auxiliary space.

If recursive:

TEXT

$O(\log n)$

stack worst case.

Thus when GATE says standard in-place Heap Sort, typically:

TEXT

$O(1)$ auxiliary

assuming iterative heapify.

48. IS HEAP SORT ADAPTIVE?

Generally:

No

An adaptive sorting algorithm benefits significantly from pre-existing sortedness.

Heap Sort does not become linear merely because input is nearly sorted.

Ø=Useful property.

49. EXTERNAL SORTING?

Heap Sort is generally not the standard choice for external sorting.

Merge Sort is more suitable for large data stored externally/sequentially.

Ø=Lower-priority GATE knowledge.

50. ROOT SWAP DOES TWO THINGS

Suppose active max-heap:

TEXT

[20, 15, 18, 8, 10]

Swapping root with last:

TEXT

[10, 15, 18, 8, 20]

does:

1. puts 20 in final sorted position.
2. gives us a complete active tree on first 4 positions.

Then heapify restores only order.

Again:

Shape preserved automatically by active-array shrinkage.

51. WHY HEAPIFY ONLY ROOT AFTER SWAP?

Before root-last swap:

TEXT

active portion was a valid heap

After swapping last element into root, the only possible violation begins at root.

The child subtrees remain valid heaps.

Therefore:

TEXT

heapifyDown(root)

is enough.

Same precondition from Lesson 68.

52. DOES SORTED SUFFIX EVER CHANGE AGAIN?

No.

Once maximum is moved to index:

TEXT

end

we reduce active heap size so that index is excluded.

Thus it stays permanently fixed.

This is the loop invariant of Heap Sort.

53. LOOP INVARIANT — GATE REASONING

Before each Heap Sort iteration:

1. $A[0 \dots \text{end}]$ is a max-heap.
2. $A[\text{end}+1 \dots n-1]$ is already sorted.
3. Every item in sorted suffix is "every item in active heap."

Then root is maximum of active part.

Swap it to `end`.

Invariant continues after heapify.

Useful algorithm-correctness understanding.

54. GATE QUESTION

What is the complexity of building the initial heap in Heap Sort?

- A. $\sim (\log n)$ B. $\sim (n \log n)$ C. $\sim (n)$ D. $\sim (n^2)$

Answer:

B — $\sim (n)$

55. GATE QUESTION

What dominates Heap Sort complexity?

Answer:

TEXT

Repeated heapify after root-last swaps

approximately:

TEXT

$n \times \log n$

giving:

TEXT

$\sim (n \log n)$

56. GATE QUESTION

Which property is false for standard Heap Sort?

- A. Comparison based B. In-place C. Worst case $O(n \log n)$ D. Stable

Answer:

D — Stable

Heap Sort is generally unstable.

57. GATE QUESTION

Which heap should be used for standard ascending in-place Heap Sort?

Max-Heap

58. GATE QUESTION

If max-heap contains n elements, where is maximum?

TEXT

$A[0]$

So finding maximum:

TEXT

$O(1)$

but removing it and restoring heap:

TEXT

$O(\log n)$

59. GATE QUESTION — ACTIVE HEAP

Suppose:

TEXT

$[50, 30, 40, 10, 20, 35, 25]$

is a max-heap.

After first root-last swap:

TEXT

$[25, 30, 40, 10, 20, 35, 50]$

Which part should be heapified?

Only:

TEXT

indices $0 \dots 5$

because:

TEXT

50

is already sorted.

$O = \frac{1}{2}n$ Exact tracing concept.

60. HEAP SORT FORMULA SUMMARY

BUILD

TEXT

$\sim (n)$

REPEATED EXTRACTION

TEXT

$(n-1) \times O(\log n)$

TOTAL

TEXT

$\sim (n \log n)$

EXTRA SPACE

TEXT

$O(1)$

STABLE?

TEXT

No

61. NOTEBOOK NOTES

Heap Sort Ascending

TEXT

1. Build max-heap
2. for end = n-1 down to 1:
 swap root with A[end]
 reduce heap size
 max-heapify root

Complexity:

TEXT

Build Heap = $\sim (n)$

Sorting phase = $\sim (n \log n)$

Total = $\sim (n \log n)$

Properties:

TEXT

In-place = YES
Stable = NO

62. MEMORY TRICK

B-S-H-R

TEXT

B = Build max heap
S = Swap root-last
H = Heap size minus one
R = Restore heap

Repeat.

63. ANOTHER MEMORY TRICK

For ascending Heap Sort:

MAX goes BACK.

Every maximum goes to the back of the array.

64. 30-SECOND REVISION

Remember:

1. Ascending Heap Sort uses max-heap.
2. Descending standard Heap Sort uses min-heap.
3. Build Heap costs $\sim (n)$
4. Root is swapped with last active element.
5. Reduce heap size.
6. Heapify root.
7. Repeat $n-1$ times.
8. Best/average/worst = $\sim (n \log n)$
9. Heap Sort is in-place.
10. Heap Sort is not stable.

65. ACTIVE RECALL

Q1

Why does max-heap produce ascending order?

Q2

Build Heap complexity?

Q3

Total Heap Sort complexity?

Q4

Is Heap Sort stable?

Q5

Is Heap Sort in-place?

Q6

What region grows during ascending Heap Sort?

ANSWERS

A1

Because each maximum root is moved to the rightmost remaining position.

A2

TEXT

$\sim (n)$

A3

TEXT

$\sim (n \log n)$

A4

No.

A5

Yes, standard iterative implementation uses $O(1)$ auxiliary space.

A6

The sorted suffix on the right.

66. GATE PRACTICE 1

Heap Sort ascending on:

TEXT

[7, 2, 9, 4, 1, 5]

Do:

1. Build max-heap.
2. Write array after every root-last swap.
3. Write array after every heapify.
4. Give final array.

67. GATE PRACTICE 2

Given max-heap:

TEXT

[20, 18, 15, 13, 12, 10, 8]

Perform only the **first two Heap Sort iterations**.

Find the array after each heapify.

68. GATE PRACTICE 3

State True/False:

1. Build Heap is $\sim (n \log n)$
2. Heap Sort worst case is $\sim (n \log n)$
3. Heap Sort is stable.
4. Heap Sort is comparison based.
5. Max-heap produces ascending order in standard in-place Heap Sort.
6. The sorted portion grows from the right.

Answers:

TEXT

1. False
2. True
3. False
4. True
5. True
6. True

69. GATE COMPARISON CHEAT SHEET

| Sort | Best | Average | Worst | Stable | Typical Aux Space |
|-----------|-------------------|-------------------|-------------------|--------|-------------------|
| Insertion | $\sim (n)$ | $\sim (n^2)$ | $\sim (n^2)$ | Yes | $O(1)$ |
| Merge | $\sim (n \log n)$ | $\sim (n \log n)$ | $\sim (n \log n)$ | Yes | $O(n)$ |
| Quick | $\sim (n \log n)$ | $\sim (n \log n)$ | $\sim (n^2)$ | No | recursion stack |
| Heap | $\sim (n \log n)$ | $\sim (n \log n)$ | $\sim (n \log n)$ | No | $O(1)$ |

Keep this for sorting questions.

70. MODULE 8 PROGRESS

' Lesson 65 — Heap Fundamentals ' Lesson 66 — Array Representation ' Lesson 67 — Heap Insertion ' Lesson 68 — Heapify Down / Extraction ' Lesson 69 — Build Heap ~ (' Lesson 70 — Heap Sort

One final GATE-focused Binary Heap lesson remains:

NEXT — LESSON 71

Priority Queue Using Binary Heap + Complete Heap GATE Consolidation

We will cover only the GATE-relevant parts:

- priority queue vs heap
- min-priority and max-priority queues
- insert / peek / extract complexities
- heap operation master table
- kth smallest/largest logic where relevant
- heap vs BST final comparison
- Build Heap vs Insert vs Heapify vs Heap Sort
- high-frequency GATE traps
- formula sheet
- mixed GATE-style questions

After Lesson 71, **Binary Heaps will be fully completed**, and we will move to:

MODULE 9 — GRAPHS

USER

next

ASSISTANT

MODULE 8 — LESSON 71

Priority Queue Using Binary Heap + Complete GATE Consolidation

This is the final **Binary Heap** lesson.

We will finish:

- Priority Queue
- Min-Priority Queue
- Max-Priority Queue
- Heap operation complexities
- Heap vs BST

- Build Heap vs Heapify vs Insert vs Heap Sort
- High-frequency GATE traps
- Final formulas
- Mixed GATE questions

GATE importance: 0=Ý%0MÚ%0KÍ%0W

1. PRIORITY QUEUE — BASIC IDEA

A normal queue follows:

FIFO

TEXT

F i r s t I n ! ' F i r s t O u t

A priority queue does **not** necessarily remove the oldest item.

Instead:

Highest-priority item is removed first

Example:

TEXT

T a s k A ! ' p r i o r i t y 3
T a s k B ! ' p r i o r i t y 1 0
T a s k C ! ' p r i o r i t y 5

If larger number means higher priority:

TEXT

B is processed first

even if B entered later.

2. PRIORITY QUEUE IS AN ADT 0=Ý%

This distinction is very important.

Priority Queue = ADT

ADT = **Abstract Data Type**

It defines:

- what operations are required
- what behavior they should have

It does not specify exactly how memory must be organized.

Common operations:

TEXT

```
insert(x)
peek()
extract()
```

3. BINARY HEAP = IMPLEMENTATION

A Binary Heap is a **data structure** commonly used to implement a priority queue.

So:

TEXT

```
P r i o r i t y   Q u e u e   " `   B i n a r y   H e a p
```

More precisely:

TEXT

```
P r i o r i t y   Q u e u e   ! '   c o n c e p t / i n t e r f a c e
B i n a r y   H e a p           ! '   o n e   i m p l e m e n t a t i o n
```

Ø = 6% TE trap.

4. MAX-PRIORITY QUEUE

Suppose larger key = higher priority.

Use:

Max-Heap

Why?

Max-heap guarantees:

TEXT

```
largest key = root
```

So highest-priority element can be accessed instantly.

5. MIN-PRIORITY QUEUE

Suppose smaller key = higher priority.

Use:

Min-Heap

Example:

Lower execution time or smaller distance has higher priority.

Root:

```
TEXT
minimum key
```

6. PRIORITY QUEUE OPERATIONS

Using a binary heap:

Insert

```
TEXT
append at end
!' s i f t u p
```

Time:

$O(\log n)$

Peek

Just access root:

```
TEXT
heap[0]
```

Time:

$O(1)$

Extract highest priority

```
TEXT
save root
l a s t !' r o o t
size--
sift down
```

Time:

$O(\log n)$

7. MASTER PRIORITY QUEUE TABLE

| Operation | Binary Heap |
|--------------|-------------|
| Peek min/max | $O(1)$ |

| Operation | Binary Heap |
|-----------------------|-------------|
| Insert | $O(\log n)$ |
| Extract min/max | $O(\log n)$ |
| Build from n items | $\sim (n)$ |
| Search arbitrary item | $O(n)$ |

Memorize this.

8. WHY NOT USE AN UNSORTED ARRAY?

Suppose priority queue implemented with an unsorted array.

Insert

Just append:

TEXT

$O(1)$

But extracting maximum/minimum requires scanning:

TEXT

$O(n)$

9. WHY NOT USE A SORTED ARRAY?

If array kept sorted:

Peek extreme

TEXT

$O(1)$

Extract from convenient end

TEXT

$O(1)$

But insertion may require shifting:

TEXT

$O(n)$

10. WHY HEAP IS A GOOD COMPROMISE

Heap gives:

TEXT

```
Insert    !'  0 ( log  n )
Extract   !'  0 ( log  n )
Peek      !'  0 ( 1 )
```

This balanced performance is why heaps are widely used for priority queues.

11. IMPLEMENTATION COMPARISON

| Structure | Insert | Peek Extreme | Extract Extreme |
|----------------|----------|--------------|-----------------|
| Unsorted Array | O(1) | O(n) | O(n) |
| Sorted Array | O(n) | O(1) | often O(1) |
| Binary Heap | O(log n) | O(1) | O(log n) |

0-6% Good GATE reasoning table.

12. HEAP INSERT — RECAP

Min-heap insertion:

TEXT

```
append
!"
if child < parent
swap upward
```

Max-heap:

TEXT

```
append
!"
if child > parent
swap upward
```

Memory:

INSERT = UP

13. EXTRACT — RECAP

Min-heap:

TEXT

```
remove root
last !'  root
```

swap downward with smaller child

Max-heap:

TEXT

```
remove root
last !' root
swap downward with larger child
```

Memory:

EXTRACT = DOWN

14. HEAPIFY VS BUILD HEAP Ø=Ý%Ø=Ý%Ø=Ý%

Do not confuse these.

Heapify Down

Repairs **one subtree**.

Worst time:

TEXT

$O(\log n)$

Build Heap

Heapifies all required internal nodes bottom-up.

Time:

$\sim (n)$

15. WHY BUILD HEAP IS NOT $O(n \log n)$

Because most nodes are near leaves.

Approximately:

TEXT

```
n / 2  nodes !' 0 downward work
n / 4  nodes !' at most 1 level
n / 8  nodes !' at most 2 levels
...
```

Total:

TEXT

$\sim (n)$

0-0% Classic GATE question.

16. FOUR OPERATIONS YOU MUST SEPARATE

Insert one element

TEXT

$O(\log n)$

Heapify one node

TEXT

$O(\log n)$

Build Heap

TEXT

$\sim (n)$

Heap Sort

TEXT

$\sim (n \log n)$

0-0% Memorize these four together.

17. HEAP SORT RECAP

Ascending:

Use Max-Heap

Steps:

TEXT

```
Build max-heap
swap root with last
shrink heap
heapify root
repeat
```

Complexity:

TEXT

$\sim (n \log n)$

Auxiliary space:

TEXT
O(1)

Stable?

TEXT
NO

18. HEAP HEIGHT

Binary heap is complete.

Therefore for n ≥ 1

FORMULA BOX

TEXT
 $h = \text{floor}(\log_2 n)$

Hence:

TEXT
 $h = \sim (\log n)$

This logarithmic height guarantees fast up/down operations.

19. HEAP ARRAY FORMULAS

0-based indexing

TEXT
Parent = $\text{floor}((i-1)/2)$
Left = $2i+1$
Right = $2i+2$

1-based indexing

TEXT
Parent = $\text{floor}(i/2)$
Left = $2i$
Right = $2i+1$

0-1 MUST KNOW.

20. LEAF/INTERNAL FORMULAS

For 0-based heap with n elements:

TEXT

First leaf = $\text{floor}(n/2)$

Leaves:

TEXT

$\text{floor}(n/2) \dots n-1$

Last internal:

TEXT

$\text{floor}(n/2)-1$

Number of leaves:

TEXT

$\text{ceil}(n/2)$

Internal nodes:

TEXT

$\text{floor}(n/2)$

21. ROOT PROPERTY

Min-Heap

TEXT

root = minimum

Max-Heap

TEXT

root = maximum

But heap is **not sorted**.

22. VERY IMPORTANT GATE TRAP

In a min-heap:

TEXT

parent "d" children

Does that imply:

TEXT

l e f t c h i l d " d r i g h t c h i l d ?

NO.

Example:

TEXT

```
  5
 / \
20 10
```

Valid min-heap.

No sibling ordering exists.

23. ANCESTOR PROPERTY

In min-heap:

TEXT

a n c e s t o r " d d e s c e n d a n t

because inequalities repeat along the path.

In max-heap:

TEXT

a n c e s t o r " e d e s c e n d a n t

But unrelated nodes need not be ordered.

24. ARBITRARY SEARCH IN HEAP

Suppose min-heap:

TEXT

```
  5
 / \
10 8
 / \ / \
30 20 9 15
```

Search for:

TEXT

20

At root:

TEXT

20 > 5

Could 20 be left or right?

Either.

Heap does not give BST-like directional search.

Worst case:

$O(n)$

~~$O=0\%$~~ Extremely important.

25. HEAP VS BST — MASTER TABLE

| Property | BST | Binary Heap |
|------------------------|-----------------------|----------------------|
| Shape | arbitrary binary tree | complete binary tree |
| Ordering | global left/right | parent-child |
| Search arbitrary key | $O(h)$ | $O(n)$ |
| Root | arbitrary rank | extreme value |
| Min in min-heap | — | $O(1)$ |
| Max in max-heap | — | $O(1)$ |
| Inorder sorted? | Yes | No |
| Natural storage | linked or array | array |
| Height guaranteed log? | No | Yes |

~~$O=0\%$~~ One of the most important comparisons.

26. BST ROOT VS HEAP ROOT

BST:

TEXT

root may be any rank

Heap:

TEXT

root must be extreme

Min-heap:

TEXT

global minimum

Max-heap:

TEXT

global maximum

27. BST SEARCH VS HEAP SEARCH

BST uses:

TEXT

```
key < root !' left
key > root !' right
```

Heap cannot do this.

Therefore:

TEXT

```
BST !' designed for searching ordered keys
Heap !' designed for accessing extreme priority
```

28. PRIORITY QUEUE APPLICATIONS — GATE RELEVANT

Heaps are important for algorithms requiring repeated minimum/maximum extraction.

Examples:

- Dijkstra's shortest-path implementation
- Prim's MST implementation
- scheduling
- event handling
- top-k problems

For GATE, most important connection later will be:

Dijkstra / Prim + Min-Priority Queue

when we reach graph algorithms.

29. KTH LARGEST USING HEAP — BASIC IDEA

Suppose we want kth largest.

One approach:

Build max-heap:

TEXT

$\sim (n)$

Extract max k times:

TEXT

$O(k \log n)$

Total:

TEXT

$O(n + k \log n)$

Useful connection.

30. ALTERNATIVE TOP-K IDEA

For k th largest, maintain a min-heap of size k .

Process all values.

Heap keeps the largest k seen so far.

Root becomes k th largest.

Typical complexity:

TEXT

$O(n \log k)$

Useful but secondary to core GATE heap syllabus.

31. GATE TRAP — PRIORITY QUEUE FIFO?

Priority queue is not ordinary FIFO.

If priorities differ:

TEXT

higher-priority element leaves first

Tie behavior depends on implementation.

Priority alone defines extraction order.

32. GATE TRAP — HEAP = PRIORITY QUEUE?

Not exactly.

Correct:

Binary heap is one common implementation of a priority queue.

Other implementations are possible.

33. GATE TRAP — HEAP IS ALWAYS MIN-HEAP?

'L

Binary heap can be:

TEXT

min-heap
or
max-heap

"Heap" alone does not automatically mean min-heap unless context specifies it.

34. GATE TRAP — COMPLETE MEANS PERFECT?

'L

A heap must be complete.

Example:

TEXT

```
      5
     /\
    8  10
   /\
  20
```

Complete but not perfect.

Still can be a valid heap.

35. GATE TRAP — FULL?

Heap need not be full.

A complete tree with one left child at the last internal node is allowed.

36. GATE TRAP — HEAP HEIGHT

For n nodes:

TEXT

$h = \text{floor}(\log_2 n)$

not:

TEXT

$n-1$

Completeness prevents skewing.

37. GATE TRAP — FIND MAX IN MIN-HEAP

Root gives minimum.

But maximum is not known directly.

For distinct keys, maximum lies among leaves.

Number of leaves:

TEXT

$\sim (n)$

So:

Find max in min-heap = $\sim (n)$

Similarly:

Find min in max-heap = $\sim (n)$

38. WHY MAXIMUM OF DISTINCT MIN-HEAP IS AT A LEAF

Suppose an internal node were maximum.

It has at least one child.

Min-heap requires:

TEXT

$\text{parent} < \text{child}$

for distinct keys.

Then child would be larger.

Contradiction.

Therefore maximum must be at a leaf.

Ø = Not a GATE property.

39. DUPLICATE CAVEAT

If equal keys exist, maximum value might also occur at an internal node.

Example:

TEXT



valid min-heap.

So statements involving strict leaf location are safest under distinct-key assumptions.

40. MIN-HEAP SUCCESSIVE EXTRACTS

If we repeatedly perform:

TEXT

extract-min

we get elements in:

TEXT

ascending order

But repeatedly extracting from a separate min-priority queue is different from standard in-place Heap Sort mechanics.

Do not mix the two ideas.

41. GATE MASTER OPERATION TABLE Ø=Ý%Ø=Ý%Ø=Ý%

| Operation | Binary Heap Complexity |
|----------------------|------------------------|
| Get root min/max | $O(1)$ |
| Insert | $O(\log n)$ |
| Sift up | $O(\log n)$ |
| Sift down | $O(\log n)$ |
| Extract root | $O(\log n)$ |
| Delete known index | $O(\log n)$ |
| Search arbitrary key | $O(n)$ |
| Build heap | $\sim (n)$ |
| Heap sort | $\sim (n \log n)$ |
| Validate heap array | $\sim (n)$ |

Copy this table.

42. GATE QUESTION 1

A binary min-heap contains n elements.

Time to find minimum?

Answer

TEXT

$O(1)$

because minimum is root.

43. GATE QUESTION 2

Time to delete minimum?

Answer

TEXT

$O(\log n)$

because after root removal, sift-down may traverse heap height.

44. GATE QUESTION 3

Time to find arbitrary key?

Answer

TEXT

$O(n)$

worst case.

45. GATE QUESTION 4

Time to build a heap from an existing array using bottom-up algorithm?

Answer

TEXT

$\sim (n)$

46. GATE QUESTION 5

Time to build a heap through n repeated insertions?

Answer

TEXT

$O(n \log n)$

worst case.

47. GATE QUESTION 6

Heap Sort worst case?

Answer

TEXT

$\sim (n \log n)$

48. GATE QUESTION 7

Heap Sort stable?

Answer

TEXT

No

49. GATE QUESTION 8

Heap Sort in-place?

Answer

TEXT

Yes

with iterative heapify:

TEXT

$O(1)$ auxiliary space

50. GATE QUESTION 9

For 0-based heap:

TEXT

$n = 20$

First leaf?

TEXT

$\text{floor}(20/2)$
=
10

Last internal?

TEXT

9

51. GATE QUESTION 10

$n=31$.

Height:

TEXT

$\text{floor}(\log_2 31)=4$

Leaves:

TEXT

$\text{ceil}(31/2)=16$

Internal:

TEXT

15

52. GATE QUESTION — VALID MIN-HEAP?

Array:

TEXT

[2, 5, 4, 9, 8, 7, 6]

Check internal nodes.

Index 0:

TEXT

2"d 5 , 4 ' '

Index 1:

TEXT

5"d 9 , 8 ' '

Index 2:

TEXT

4"d 7 , 6 ' '

Therefore:

Valid min-heap

53. INVALID EXAMPLE

TEXT

[2, 5, 4, 9, 3, 7, 6]

At index 1:

TEXT

5

child:

TEXT

3

Violation:

TEXT

5"d 3 ' 'L

Not min-heap.

54. GATE QUESTION — INSERTION

Min-heap:

TEXT

[4, 7, 6, 10, 8, 9]

Insert:

TEXT

2

Append:

TEXT

[4, 7, 6, 10, 8, 9, 2]

2 parent = index 2 != 6.

Swap:

TEXT

[4, 7, 2, 10, 8, 9, 6]

2 parent = root 4.

Swap:

TEXT

[2, 7, 4, 10, 8, 9, 6]

Final.

55. GATE QUESTION — EXTRACT

Min-heap:

TEXT

[2, 5, 4, 9, 8, 7, 6]

Extract 2.

Last 6 != root:

TEXT

[6, 5, 4, 9, 8, 7]

Smaller child:

TEXT

4

swap:

TEXT

[4, 5, 6, 9, 8, 7]

Done.

56. GATE QUESTION — BUILD HEAP START INDEX

$n=15$, 0-based.

TEXT

```
floor(15/2) - 1  
=  
6
```

Heapify:

TEXT

```
6, 5, 4, 3, 2, 1, 0
```

Leaves:

TEXT

```
7...14
```

57. GATE QUESTION — HEAPSORT DIRECTION

Which heap for ascending standard Heap Sort?

Max-Heap

Which for descending?

Min-Heap

58. HEAP FORMULA SHEET

Definition

TEXT

```
Complete Binary Tree  
+  
Heap Order
```

Min:

TEXT

```
P a r e n t "d   c h i l d r e n
```

Max:

TEXT

```
P a r e n t "e   c h i l d r e n
```

59. ARRAY FORMULAS

0-based

TEXT

```
P = floor((i-1)/2)
L = 2i+1
R = 2i+2
```

1-based

TEXT

```
P = floor(i/2)
L = 2i
R = 2i+1
```

60. STRUCTURAL FORMULAS

TEXT

```
h = floor(log2 n)
```

TEXT

```
Leaves = ceil(n/2)
```

TEXT

```
Internal = floor(n/2)
```

0-based:

TEXT

```
First leaf = floor(n/2)
Last internal = floor(n/2)-1
```

61. COMPLEXITY FORMULAS

TEXT

```
Insert           = O(log n)
Extract root     = O(log n)
Peek root        = O(1)
Search arbitrary = O(n)
Heapify one node = O(log n)
Build heap       = ~ ( n )
Heap sort        = ~ ( n log n )
```

Ø = Final heap complexity box.

62. MEMORY MAP

HEAP = C-P-E

TEXT

C = Complete
P = Parent-child order
E = Extreme at root

63. OPERATION MEMORY

TEXT

I n s e r t ! ' U P
E x t r a c t ! ' D O W N
B u i l d ! ' B O T T O M - U P
S o r t ! ' R O O T T O E N D

Ø = Extremely useful.

64. COMPLEXITY MEMORY

TEXT

O N E f i x ! ' l o g n
A L L b u i l d ! ' n
A L L s o r t ! ' n l o g n

Meaning:

TEXT

one sift $O(\log n)$
b u i l d w h o l e $\sim (n)$
s o r t w h o l e $\sim (n \log n)$

65. 30-SECOND BINARY HEAP REVISION

Remember:

1. Heap = complete binary tree + heap property.
2. Min-heap root = minimum.
3. Max-heap root = maximum.
4. Heap array is not sorted.
5. Arbitrary search = $O(n)$.
6. Insert = append + sift up = $O(\log n)$.
7. Extract = last-to-root + sift down = $O(\log n)$.
8. Build Heap = $\sim (n)$.
9. Heap Sort = $\sim (n \log n)$.

10. Priority Queue is an ADT; binary heap is a common implementation.

66. ACTIVE RECALL

Q1

Priority Queue and Binary Heap — same thing?

Q2

Min-priority queue usually uses which heap?

Q3

Peek complexity?

Q4

Insert complexity?

Q5

Extract complexity?

Q6

Build Heap complexity?

Q7

Heap Sort complexity?

Q8

Arbitrary search complexity?

ANSWERS

A1

No.

Priority Queue = ADT.

Binary Heap = one implementation.

A2

Min-heap.

A3

TEXT

$O(1)$

A4

TEXT

$O(\log n)$

A5

TEXT

$O(\log n)$

A6

TEXT

$\sim (n)$

A7

TEXT

$\sim (n \log n)$

A8

TEXT

$O(n)$

67. MIXED GATE PRACTICE

Q1

A min-heap has 100 nodes.

Find:

- height
- number of leaves
- first leaf index in 0-based representation
- last internal index

Q2

Is this a valid max-heap?

TEXT

[50, 40, 45, 20, 30, 35, 10]

Q3

Build a min-heap by inserting:

TEXT

10, 4, 15, 2, 8, 6

Trace the final array.

Q4

Given:

TEXT

[2, 5, 4, 9, 8, 7, 6]

perform one extract-min.

Q5

Why is:

TEXT

Heapify one node = $O(\log n)$

but:

TEXT

Build Heap = $\sim (n)$

Explain in one sentence.

Q6

State:

TEXT

Heap Sort:
Best =
Average =
Worst =
Auxiliary space =
Stable =

68. BINARY HEAP MODULE — COMPLETE '

We have fully covered the GATE-relevant Binary Heap syllabus:

' Complete binary-tree foundation ' Min-Heap ' Max-Heap ' Heap property ' Array representation ' Parent/child formulas ' Heap height ' Leaves/internal nodes ' Heap vs BST ' Insertion / sift-up ' Extraction / sift-down ' Arbitrary deletion concept ' Heapify ' Bottom-up Build Heap ' $\sim (n)$ Build Heap proof ' Heap Sort ' Priority Queue ' All major complexities ' Major GATE traps

MODULE 8 — BINARY HEAPS ' COMPLETE

OFFICIAL SYLLABUS PROGRESS

Programming in C. Recursion. Arrays, stacks, queues, linked lists, trees, binary search trees, binary heaps, graphs.

| Topic | Status |
|---------------------|---------|
| Programming in C | ' |
| Recursion | ' |
| Arrays | ' |
| Stacks | ' |
| Queues | ' |
| Linked Lists | ' |
| Trees | ' |
| Binary Search Trees | ' |
| Binary Heaps | ' |
| Graphs | #Ó NEXT |

NEXT — MODULE 9, LESSON 72

Graph Fundamentals — Vertices, Edges, Degree, Paths, Cycles, Connectivity & Graph Types

We will start Graphs from absolute basics and build toward:

- directed vs undirected
- weighted vs unweighted
- simple graphs
- complete graphs
- degree / indegree / outdegree
- paths, walks, trails, cycles
- connected components
- handshaking lemma
- maximum edge formulas
- trees as special graphs
- GATE traps

Then we will move to graph representation, BFS and DFS.

next

MODULE 9 — LESSON 72

We now begin the final official syllabus block:

Graphs are one of the most important areas in GATE because they connect directly to:

- GATE importance: 0=Ý%04Ú\$0KÍ0W**

A graph is written as:

where:

V = set of vertices
E = set of edges

nodes

Edges represent:

TEXT
connections

Example:

TEXT
A ----- B
| |
| |
C ----- D

Vertices:

TEXT
 $V = \{A, B, C, D\}$

Edges:

TEXT
 $E = \{(A,B), (A,C), (B,D), (C,D)\}$

So:

TEXT
 $|V| = 4$
 $|E| = 4$

2. MEMORY TRICK

Graph = Things + Connections

TEXT
V e r t i c e s ! ' T h i n g s
E d g e s ! ' C o n n e c t i o n s

Examples:

TEXT
C i t i e s ! ' v e r t i c e s
R o a d s ! ' e d g e s

TEXT
P e o p l e ! ' v e r t i c e s
F r i e n d s h i p s ! ' e d g e s

3. UNDIRECTED GRAPH Ø=Ý%

An undirected edge has no direction.

Example:

TEXT
A ----- B

means:

TEXT
A connected to B
AND
B connected to A

Edge can be written:

TEXT
{A, B}

or informally:

TEXT
(A, B)

In an undirected graph:

TEXT
(A, B) = (B, A)

4. DIRECTED GRAPH Ø=Ý%

A directed graph is also called:

Digraph

Example:

TEXT
A ----> B

means:

TEXT
A points to B

not necessarily:

TEXT
B points to A

Directed edge:

TEXT

(A, B)

is different from:

TEXT

(B, A)

$\emptyset$ = Not important.

5. UNDIRECTED VS DIRECTED

| Property | Undirected | Directed |
|-----------------|--------------|----------------------|
| Edge | no direction | direction exists |
| (u,v) and (v,u) | same edge | different edges |
| Degree | one degree | indegree + outdegree |
| Example | friendship | Twitter follow |

6. WEIGHTED GRAPH

An edge may have a numerical value called:

Weight

Example:

TEXT

A --5-- B

The weight may represent:

- distance
- cost
- time
- capacity
- probability

Example:

TEXT

Hyderabad ---- 600 km ---- Bangalore

7. UNWEIGHTED GRAPH

If edges have no explicit cost:

TEXT

A ----- B

we call it unweighted.

For many algorithms, an unweighted graph can conceptually be treated as all edges having equal weight:

TEXT

1

8. SIMPLE GRAPH $\emptyset = \%$

A simple graph usually means:

1. No self-loops.
2. No multiple parallel edges between the same pair of vertices.

Example:

TEXT

A ----- B

Only one edge between A and B.

9. SELF LOOP

An edge from a vertex to itself:

TEXT

A !°

Formally:

TEXT

(A, A)

This is called a:

Self-loop

Simple graphs do not allow self-loops.

10. PARALLEL EDGES

If multiple distinct edges connect the same two vertices:

TEXT

A ===== B

that is a:

Multigraph-style situation

A simple graph does not allow parallel edges.

11. DEGREE OF A VERTEX — UNDIRECTED

Degree of vertex v :

Number of incident edge-ends at v

Usually for a simple undirected graph:

TEXT

$\text{degree}(v)$ = number of neighboring vertices

Example:

TEXT

```
  B
  |
A---C---D
```

Degree:

TEXT

$\text{deg}(A)=1$
 $\text{deg}(B)=1$
 $\text{deg}(C)=3$
 $\text{deg}(D)=1$

12. SELF-LOOP DEGREE CONTRIBUTION

In an undirected graph:

A self-loop contributes 2 to degree

Why?

Both ends of the edge touch the same vertex.

NOTE trap.

13. HANDSHAKING LEMMA

For any undirected graph:

FORMULA BOX

TEXT

$$\sum \deg(v) = 2|E|$$

Why?

Every edge contributes:

TEXT

1 degree to one endpoint
+
1 degree to the other endpoint

So total contribution per edge:

TEXT

2

14. EXAMPLE

Degrees:

TEXT

2, 3, 1, 2

Sum:

TEXT

8

Therefore:

TEXT

$$2|E| = 8$$

So:

TEXT

$$|E| = 4$$

15. HUGE CONSEQUENCE $\sum \deg(v) = 2|E|$

Since:

TEXT

$$\sum \deg(v) = 2|E|$$

the sum of all degrees is always:

EVEN

Therefore:

The number of odd-degree vertices in an undirected graph must be even.

Very common GATE theorem.

16. WHY ODD-DEGREE VERTICES ARE EVEN IN NUMBER

Suppose total degree sum is even.

Even-degree vertices contribute an even total.

Therefore the sum contributed by odd-degree vertices must also be even.

A sum of an odd number of odd integers is odd.

Hence number of odd-degree vertices must be:

Even

17. GATE EXAMPLE

Can an undirected graph have exactly:

TEXT

5 odd-degree vertices?

No.

Number of odd-degree vertices must be even.

Possible:

TEXT

0, 2, 4, 6, ...

18. DIRECTED GRAPH — INDEGREE & OUTDEGREE

For directed graphs:

Indegree

Number of incoming edges.

TEXT

$\deg^-(v)$

Outdegree

Number of outgoing edges.

TEXT

$\deg^+(v)$

19. EXAMPLE

TEXT

```
A ---> B ---> C
      ^
      |
      D
```

For B:

Incoming:

TEXT

A! ' B
D! ' B

So:

TEXT

$\text{indegree}(B)=2$

Outgoing:

TEXT

B! ' C

So:

TEXT

$\text{outdegree}(B)=1$

20. DIRECTED DEGREE FORMULAS Ø=Ý%

For any directed graph:

FORMULA BOX

TEXT

$\sum \text{indegree}(v) = |E|$

and:

TEXT

$\sum \text{outdegree}(v) = |E|$

Therefore:

TEXT

$\sum \text{indegree}$
 $=$
 $\sum \text{outdegree}$
 $=$
 $|E|$

Why?

Every directed edge:

TEXT

$u \rightarrow v$

contributes:

TEXT

1 outgoing from u
1 incoming to v

21. DIRECTED GRAPH EXAMPLE

Suppose a digraph has:

TEXT

12 edges

Then total indegree:

TEXT

12

Total outdegree:

TEXT

12

Combined:

TEXT

24

22. ADJACENT VERTICES

Two vertices are adjacent if an edge directly connects them.

Example:

TEXT

A ----- B ----- C

A and B:

TEXT

adjacent

A and C:

TEXT

not adjacent

even though a path exists.

$\emptyset \neq \%$ adjacent means direct edge.

23. NEIGHBORHOOD

Neighbors of a vertex are vertices directly adjacent to it.

If:

TEXT

A connected to B,C,D

then:

TEXT

$N(A) = \{B, C, D\}$

and:

TEXT

$\deg(A)=3$

in a simple undirected graph.

24. WALK $\emptyset \neq \%$

A walk is a sequence:

TEXT

$v_0, v_1, v_2, \dots, v_k$

where consecutive vertices are connected by edges.

Vertices and edges may repeat.

Example:

TEXT

A !' B !' C !' B !' A

can be a walk.

Repetition allowed.

25. LENGTH OF A WALK/PATH

Length is usually:

number of edges traversed

Example:

TEXT

A !' B !' C !' D

contains:

TEXT

3 edges

So length:

TEXT

3

$\emptyset = \text{No}$ 4.

26. TRAIL

A trail is a walk where:

No edge repeats

Vertices may repeat.

Memory:

TEXT

T r a i l !' e d g e s d o n ' t r e p e a t

27. PATH $\emptyset = \text{Yes}$ $\emptyset = \text{Yes}$ $\emptyset = \text{Yes}$

A simple path normally means:

No vertex repeats

Example:

TEXT

A !' B !' C !' D

valid path.

But:

TEXT

A !' B !' C !' B !' D

is not a simple path because B repeats.

28. WALK vs TRAIL vs PATH

| Object | Repeated vertices? | Repeated edges? |
|--------|--------------------|-----------------|
| Walk | allowed | allowed |
| Trail | allowed | not allowed |
| Path | not allowed | not allowed |

Ø=Ý%Ø=Ý%Ø=Ý%
Very useful GATE table.

29. CYCLE Ø=Ý%Ø=Ý%Ø=Ý%

A cycle is essentially a closed path:

TEXT

start = end

with no repeated internal vertices under the simple-cycle definition.

Example:

TEXT

A
|\
|\
C--B

Cycle:

TEXT

A !' B !' C !' A

Length:

TEXT

3

30. ACYCLIC GRAPH

A graph with no cycle is:

Acyclic

Examples:

- Trees
- Forests
- DAGs in directed graphs

31. DAG Ø=Ý%

Directed Acyclic Graph

A directed graph containing no directed cycle.

Example:

TEXT

```
A  !'  B  !'  C
 \      ^
  !'    D  -  -  |
```

If no route brings us back to start following edge directions:

TEXT

DAG

DAGs are important for:

- dependencies
- scheduling
- topological ordering

We'll revisit later.

32. CONNECTED GRAPH — UNDIRECTED Ø=Ý%Ø=Ý%Ø=Ý%

An undirected graph is connected if:

Every pair of vertices has a path between them.

Example:

TEXT

```
A -- B -- C
    |
    D
```

Connected.

You can reach every vertex from every other vertex.

33. DISCONNECTED GRAPH

Example:

TEXT

```
A -- B
C -- D
```

No path between A and C.

Therefore graph is:

TEXT

disconnected

34. CONNECTED COMPONENT

A:

Connected component

is a maximal connected portion of an undirected graph.

Example:

TEXT

```
A -- B -- C      D -- E      F
```

Components:

TEXT

```
{A, B, C}
{D, E}
{F}
```

Number of connected components:

3

$\emptyset = \{x\}$ isolated vertex itself forms a component.

35. ISOLATED VERTEX

A vertex with degree:

TEXT

0

is isolated.

Example:

TEXT

A

with no edges.

In an undirected graph:

TEXT

$\deg(A)=0$

36. PENDANT / LEAF VERTEX

A vertex of degree:

TEXT

1

is often called:

- pendant vertex
- leaf vertex in a tree

Example:

TEXT

A --- B

If A has only this edge:

TEXT

$\deg(A)=1$

37. STRONGLY CONNECTED — DIRECTED GRAPH $\emptyset \neq V$

A directed graph is strongly connected if:

For every pair u, v :

TEXT

u can reach v
AND
v can reach u

following edge directions.

Example:

TEXT

A !' B
!' !"
C !•

if all vertices mutually reachable:

TEXT

strongly connected

38. WEAKLY CONNECTED — DIRECTED

A directed graph is weakly connected if:

When edge directions are ignored, the graph becomes connected.

Useful distinction.

39. COMPLETE GRAPH K_n

A complete undirected simple graph:

Every pair of distinct vertices has an edge.

Notation:

TEXT

K_n

Example:

TEXT

K_3

is a triangle.

40. MAXIMUM EDGES IN SIMPLE UNDIRECTED GRAPH K_n

Every edge corresponds to a pair of distinct vertices.

Number of pairs:

FORMULA BOX

TEXT

$$C(n, 2) = \frac{n(n-1)}{2}$$

Therefore maximum edges:

$$n(n-1)/2$$

41. EXAMPLE

Simple undirected graph:

TEXT

$$n=5$$

Maximum edges:

TEXT

$$5 \times 4 / 2 = 10$$

So:

TEXT

K5 has 10 edges

42. DEGREE IN COMPLETE GRAPH

In:

TEXT

K_n

each vertex connects to every other vertex.

So:

FORMULA BOX

TEXT

$$\deg(v) = n-1$$

for every vertex.

43. HANDSHAKING CHECK FOR K_n

Total degree:

TEXT
 $n(n-1)$

Handshaking:

TEXT
 $2E = n(n-1)$

Therefore:

TEXT
 $E = n(n-1)/2$

Same result.

44. MAXIMUM EDGES — SIMPLE DIRECTED GRAPH

If:

- no self-loops
- directed edges allowed both ways

For every ordered pair of distinct vertices:

TEXT
 $u \neq v$

Number:

FORMULA BOX

TEXT
 $n(n-1)$

because:

TEXT
 (u, v)

and:

TEXT
 (v, u)

are distinct.

Directed max is double the undirected max.

45. IF SELF-LOOPS ARE ALLOWED

Simple directed case without loops:

TEXT
 $n(n-1)$

If one self-loop per vertex is also allowed:

TEXT
 n^2

possible directed edges.

Always read assumptions.

46. REGULAR GRAPH

A graph where every vertex has the same degree is:

Regular Graph

If every vertex has degree r :

TEXT
 r -regular graph

47. REGULAR GRAPH FORMULA $\sum \deg v = nr$

For an r -regular undirected graph with n vertices:

TEXT
 $\sum \deg v = nr$

Using handshaking:

TEXT
 $nr = 2E$

So:

FORMULA BOX

TEXT
 $E = nr/2$

48. IMPORTANT CONSEQUENCE

For an r -regular graph to exist:

TEXT

nr

must be even.

Example:

TEXT

$n=5$

$r=3$

Then:

TEXT

$nr=15$

odd.

Impossible.

$\emptyset = \text{No GATE numerical.}$

49. SUBGRAPH

A subgraph uses:

- some vertices of original graph
- some edges of original graph

without inventing new ones.

If:

TEXT

$G=(V, E)$

then subgraph:

TEXT

$H=(V', E')$

where:

TEXT

$V' \subseteq V$
 $E' \subseteq E$

with edges compatible with chosen vertices.

50. SPANNING SUBGRAPH

A spanning subgraph contains:

all original vertices

but possibly fewer edges.

So:

TEXT

$$V' = V$$

This concept becomes very important for:

Spanning Trees

in Algorithms.

51. TREE AS A GRAPH $\emptyset = \dot{Y}\% \emptyset = \dot{Y}\% \emptyset = \dot{Y}\%$

An undirected tree is:

Connected + Acyclic

For a tree with n vertices:

FORMULA BOX

TEXT

$$E = n - 1$$

This relation is extremely important.

52. WHY TREE HAS $n - 1$ EDGES

Start with one vertex:

TEXT

0 edges

Every time we add a new vertex while maintaining connected acyclic structure:

TEXT

add exactly one edge

After adding $n - 1$ more vertices:

TEXT

$E = n - 1$

53. TREE EQUIVALENT PROPERTIES Ø=Ý%

For an undirected simple graph with n vertices, the following are closely related:

If connected and:

TEXT

$E = n - 1$

then it is a tree.

If acyclic and:

TEXT

$E = n - 1$

then it is a tree.

A tree:

- connected
- acyclic
- exactly one simple path between every pair
- $n - 1$ edges

Ø=Ý% Very useful GATE equivalences.

54. FOREST

A forest is an:

Acyclic undirected graph

It may be disconnected.

Each connected component is a tree.

55. FOREST EDGE FORMULA Ø=Ý%Ø=Ý%

Suppose a forest has:

TEXT

n vertices

c connected components

Then:

FORMULA BOX

TEXT

$$E = n - c$$

Why?

Each component i is a tree:

TEXT

$$E_i = n_i - 1$$

Summing:

TEXT

$$\begin{aligned} E &= \sum n_i - c \\ &= n - c \end{aligned}$$

Excellent GATE formula.

56. EXAMPLE

Forest:

TEXT

$$\begin{aligned} n &= 10 \\ c &= 3 \end{aligned}$$

Edges:

TEXT

$$\begin{aligned} 10 - 3 \\ = \\ 7 \end{aligned}$$

57. MINIMUM EDGES FOR CONNECTED UNDIRECTED GRAPH

To connect n vertices:

minimum edges:

$$n - 1$$

because a tree is the sparsest connected graph.

So:

TEXT

$$\text{Connected graph! } E = n - 1$$

58. MAXIMUM EDGES IN CONNECTED SIMPLE GRAPH

Maximum remains:

TEXT

$$n(n-1)/2$$

So connected simple undirected graph:

FORMULA BOX

TEXT

$$n - 1 \leq E \leq n(n-1)/2$$

for $n \geq 1$

59. DISCONNECTED GUARANTEE

If simple undirected graph has:

TEXT

$$E < n-1$$

then it **cannot** be connected.

~~Ø=5%~~ Long quick test.

But:

TEXT

$$E \geq n - 1$$

does not automatically guarantee connectedness.

60. EXAMPLE

$n=5$, $E=3$.

Since:

TEXT

$$3 < 4$$

graph definitely disconnected.

But $n=5$, $E=4$ could be:

- a tree !' connected
- triangle + separate edge !' disconnected

So edge count alone may not guarantee connectedness.

61. CYCLE AND EDGE COUNT

A connected undirected graph with:

TEXT

$E = n - 1$

is a tree !' no cycles.

If connected graph has:

TEXT

$E \geq n$

then it must contain at least one cycle.

Ø=important.

Reason:

Tree is maximum acyclic connected structure with $n - 1$ edges.

Any additional edge creates a cycle.

62. ACYCLIC UNDIRECTED GRAPH EDGE BOUND

A forest with n vertices has at most:

TEXT

$n - 1$

edges.

Equality occurs when:

TEXT

$c = 1$

meaning the forest is a tree.

63. BIPARTITE GRAPH — BASIC GATE FOUNDATION Ø=Ý%

A graph is bipartite if its vertices can be divided into two disjoint sets:

TEXT

U
 V

such that every edge connects:

TEXT

one vertex in U
to

one vertex in V

No edge connects two vertices within the same side.

64. EXAMPLE

TEXT

$U = \{A, B\}$

$V = \{C, D\}$

Possible edges:

TEXT

A-C

A-D

B-C

No:

TEXT

A-B

and no:

TEXT

C-D

65. COMPLETE BIPARTITE GRAPH

Notation:

$K_{\{m,n\}}$

Every vertex in first set connects to every vertex in second set.

Number of edges:

FORMULA BOX

TEXT

$m \times n$

Example:

TEXT

$K_{2,3}$

Edges:

$$2 \times 3 = 6$$

An undirected graph is bipartite iff:

Equivalent BFS/DFS coloring interpretation:

2-colorable

We will use this later with BFS/DFS.

A cycle of length:

4

Example square:

$$\begin{array}{ccc} A & - & - & - & B \\ | & & & & | \\ D & - & - & - & C \end{array}$$
$$\begin{array}{l} \{A, C\} \\ \{B, D\} \end{array}$$

Triangle:

$$\begin{array}{c} A - - - B \\ \quad \backslash \quad / \\ \quad \quad C \end{array}$$

Length:

TEXT

3

Cannot be bipartite.

$\Delta = 3$ angle immediately proves non-bipartite.

69. DEGREE SEQUENCE

A degree sequence lists degrees of vertices.

Example:

TEXT

3, 2, 2, 1

First quick GATE test:

TEXT

sum degrees

must be even.

If sum is odd:

Impossible for any undirected graph

70. EXAMPLE

Can:

TEXT

3, 3, 2, 1

be degree sequence?

Sum:

TEXT

9

odd.

Therefore:

Impossible

No need for deeper analysis.

71. BUT EVEN SUM DOES NOT GUARANTEE GRAPHICALITY

Example degree sequence may have even total but still be impossible as a simple graph.

There are deeper tests like Havel-Hakimi.

Ø=0 For current syllabus, main immediate fact is handshaking parity unless a specific question requires more.

72. SPARSE VS DENSE GRAPH

Informal but useful.

Sparse

Number of edges relatively small:

TEXT

$E \text{ closer to } V$

Dense

Number of edges near maximum:

TEXT

$\sim (V^2)$

This becomes important when choosing:

- adjacency list
- adjacency matrix

next lesson.

73. GRAPH ORDER AND SIZE

Some textbooks say:

TEXT

Order of graph = number of vertices

Size of graph = number of edges

So:

TEXT

order = $|V|$

size = $|E|$

Ø=0 Useful terminology.

74. GATE TRAP — PATHLENGTH

Path:

TEXT

A-B-C-D-E

Vertices:

TEXT

5

Edges:

TEXT

4

Path length:

4

not 5.

75. GATE TRAP — DEGREE SUM

Suppose undirected graph has:

TEXT

15 edges

Total degree:

TEXT

$2 \times 15 = 30$

not 15.

76. GATE TRAP — DIRECTED TOTAL DEGREE

Digraph has 15 edges.

Then:

TEXT

total indegree = 15
total outdegree = 15

Not 30 individually.

Combined:

TEXT

30

77. GATE TRAP — MAXIMUM EDGES

Simple undirected:

TEXT
 $n(n-1)/2$

Simple directed, no self-loops:

TEXT
 $n(n-1)$

Do not mix them.

78. GATE TRAP — TREE DEFINITION

Tree is not merely:

TEXT
acyclic

An acyclic disconnected graph is:

TEXT
forest

Tree requires:

TEXT
connected + acyclic

79. GATE TRAP — CONNECTED + n EDGES

For connected undirected graph with n vertices and n edges:

It cannot be a tree.

It contains at least:

TEXT
one cycle

80. GATE TRAP — ODD DEGREE COUNT

Can graph have:

TEXT

exactly 1 odd-degree vertex?

No.

Can it have exactly:

TEXT

2?

Yes.

Can it have:

TEXT

7?

No.

Odd-degree vertex count must be even.

81. SOLVED GATE EXAMPLE 1

An undirected graph has:

TEXT

10 vertices

Each degree:

TEXT

3

How many edges?

Use:

TEXT

$nr = 2E$

TEXT

$10 \times 3 = 2E$

TEXT

$30 = 2E$

TEXT

$E = 15$

Answer: 15

82. SOLVED GATE EXAMPLE 2

Can a 3-regular graph have 7 vertices?

TEXT

$$nr = 7 \times 3 = 21$$

But:

TEXT

$$2E$$

must be even.

21 odd.

Therefore:

Impossible

83. SOLVED GATE EXAMPLE 3

A forest has:

TEXT

20 vertices
5 components

Edges:

TEXT

$$E = n - c$$

TEXT

$$20 - 5 = 15$$

Answer: 15

84. SOLVED GATE EXAMPLE 4

A connected simple undirected graph has:

TEXT

8 vertices

Minimum edges:

TEXT
 $8 - 1 = 7$

Maximum:

TEXT
 $8 \times 7 / 2 = 28$

So:

TEXT
7 "d E "d 2 8

85. SOLVED GATE EXAMPLE 5

Complete bipartite graph:

TEXT
 $K_4, 7$

Edges:

TEXT
 $4 \times 7 = 28$

Degree:

Each vertex on size-4 side:

TEXT
7

Each vertex on size-7 side:

TEXT
4

Total degree:

TEXT
 $4 \times 7 + 7 \times 4 = 56$

And:

TEXT
 $2E = 56$

Correct.

86. SOLVED GATE EXAMPLE 6

Can a bipartite graph contain cycle length 5?

No.

Because 5 is odd.

Bipartite graphs contain no odd cycles.

87. SOLVED GATE EXAMPLE 7

Graph has:

TEXT

n=6
E=4

Can it be connected?

Minimum required:

TEXT

n-1=5

But:

TEXT

4<5

Therefore:

No

88. MASTER FORMULA SHEET $\emptyset = \text{Yes} \emptyset = \text{Yes} \emptyset = \text{Yes}$

Undirected Handshaking

TEXT

$\sum \text{deg}(v) = 2E$

Directed

TEXT

$\sum \text{indegree} = E$
 $\sum \text{outdegree} = E$

Complete undirected graph

TEXT

$$E = n(n-1)/2$$

Complete directed graph without self-loops

TEXT

$$E = n(n-1)$$

r-regular graph

TEXT

$$E = nr/2$$

Tree

TEXT

$$E = n - 1$$

Forest with c components

TEXT

$$E = n - c$$

Complete bipartite graph $K(m,n)$

TEXT

$$E = mn$$

Copy this entire box.

89. MASTER CONCEPT TABLE

| Concept | Meaning |
|---------|---------------------|
| Vertex | node |
| Edge | connection |
| Degree | incident edge count |
| Walk | repetitions allowed |
| Trail | no repeated edge |
| Path | no repeated vertex |

| Concept | Meaning |
|-----------|----------------------------|
| Cycle | closed path |
| Connected | path between every pair |
| Component | maximal connected part |
| Tree | connected + acyclic |
| Forest | acyclic graph |
| DAG | directed acyclic graph |
| Bipartite | vertices split into 2 sets |

90. MEMORY TRICKS

Handshaking

Every edge shakes two hands

SO:

TEXT

$\text{sum degree} = 2E$

Directed

Every arrow leaves once and enters once

SO:

TEXT

$\text{outdegree total} = \text{indegree total} = E$

Tree

N nodes need N-1 links

Forest

Each extra component removes one required edge

TEXT

$E = n - c$

Bipartite

91. 30-SECOND REVISION

Remember:

1. Graph $G=(V, E)$.
2. Undirected edge has no direction.
3. Directed graph has indegree/outdegree.
4. $\sum \text{degree} = 2E$
5. Number of odd-degree vertices is even.
6. Simple undirected max edges = $n(n-1)/2$
7. Tree = connected + acyclic = $n-1$ edges.
8. Forest with c components has $n-c$ edges.
9. K_m, n has mn edges.
10. Bipartite graph has no odd cycle.

92. ACTIVE RECALL

Q1

An undirected graph has 18 edges. Sum of degrees?

Q2

Can an undirected graph have 9 odd-degree vertices?

Q3

Maximum edges in simple graph with 8 vertices?

Q4

A forest has 12 vertices and 4 components. Edges?

Q5

What is a DAG?

Q6

What cycle property characterizes bipartite graphs?

ANSWERS

A1

TEXT

36

A2

No.

A3

TEXT

$$8 \times 7 / 2 = 28$$

A4

TEXT

$$12 - 4 = 8$$

A5

Directed Acyclic Graph.

A6

No odd-length cycle.

93. GATE PRACTICE

Q1

A simple undirected graph has 10 vertices, each of degree 6.

Find number of edges.

Q2

Is a 5-regular graph on 9 vertices possible?

Use handshaking.

Q3

A connected graph has:

TEXT

$$n=12$$

$$E=11$$

What can you conclude?

Q4

A connected graph has:

TEXT

$$n=12$$

$$E=12$$

What must be true regarding cycles?

Q5

Complete bipartite graph:

TEXT

$K_{5,8}$

Find:

- vertices
- edges
- degree of each vertex in first partition
- degree of each vertex in second partition

94. MODULE 9 PROGRESS

' Lesson 72 — Graph Fundamentals

Next:

LESSON 73 — Graph Representation

Adjacency Matrix vs Adjacency List — Space, Edge Lookup, Traversal & GATE Complexity

We will cover:

TEXT

Adjacency Matrix !' $\sim (V^2)$ space
Adjacency List !' $\sim (V + E)$ space

plus:

- undirected/directed storage
- weighted graph representation
- degree calculation
- edge insertion/deletion
- edge lookup complexities
- sparse vs dense graphs
- why BFS/DFS become $O(V+E)$ with adjacency lists
- major GATE representation traps.

USER

next

MODULE 9 — LESSON 73

Graph Representation — Adjacency Matrix vs Adjacency List

Now we study how a graph is actually stored in memory.

This is very important for GATE because representation directly affects:

- space complexity
- edge lookup
- degree calculation
- BFS/DFS complexity
- sparse vs dense graph choice

GATE importance: 0% to 100% (0% to 100%)

1. WHY DO WE NEED GRAPH REPRESENTATION?

Consider:

TEXT

```
A ----- B
|         |
|         |
C ----- D
```

Vertices:

TEXT

```
V = {A, B, C, D}
```

Edges:

TEXT

```
E = {(A, B), (A, C), (B, D), (C, D)}
```

We need some data structure to store:

Which vertices are connected to which?

Two main GATE representations:

1. Adjacency Matrix

2. Adjacency List

2. ADJACENCY MATRIX $\emptyset = \mathcal{V} \times \mathcal{V}$

For $\mathcal{V}$ vertices, create a:

$\mathcal{V} \times \mathcal{V}$ matrix

Let:

TEXT

$M[i][j] = 1$

if edge exists between vertex i and vertex j .

Otherwise:

TEXT

$M[i][j] = 0$

3. EXAMPLE

Graph:

TEXT

```
A  -----  B
|           |
C  -----  D
```

Edges:

TEXT

A-B
A-C
B-D
C-D

Matrix:

TEXT

| | A | B | C | D |
|---|---|---|---|---|
| A | 0 | 1 | 1 | 0 |
| B | 1 | 0 | 0 | 1 |
| C | 1 | 0 | 0 | 1 |
| D | 0 | 1 | 1 | 0 |

4. UNDIRECTED GRAPH MATRIX PROPERTY $\emptyset = \mathcal{V} \times \mathcal{V}$

In an undirected graph:

TEXT

$\text{edge}(i, j) = \text{edge}(j, i)$

Therefore:

Adjacency matrix is symmetric

TEXT

$M[i][j] = M[j][i]$

✓
Very common GATE property.

5. DIAGONAL OF SIMPLE GRAPH

For a simple graph:

TEXT

no self loops

Therefore:

TEXT

$M[i][i] = 0$

So main diagonal contains:

TEXT

0

6. MATRIX FOR DIRECTED GRAPH

Suppose:

TEXT

| | | | |
|---|---|---|---|
| A | ! | ' | B |
| B | ! | ' | C |
| C | ! | ' | A |

Matrix:

TEXT

| | A | B | C |
|---|---|---|---|
| A | 0 | 1 | 0 |
| B | 0 | 0 | 1 |
| C | 1 | 0 | 0 |

Here matrix need not be symmetric.

Because:

TEXT

$A \not\Rightarrow B$

does not imply:

TEXT

$B \not\Rightarrow A$

7. ROW MEANING — DIRECTED GRAPH $\emptyset \Rightarrow \hat{Y}\%$

If:

TEXT

$M[i][j] = 1$

means:

TEXT

$i \rightarrow j$

then:

Row i = outgoing edges from i

Therefore:

TEXT

sum of row i = $\text{outdegree}(i)$

8. COLUMN MEANING

Similarly:

TEXT

sum of column i = $\text{indegree}(i)$

$\emptyset \Rightarrow \hat{Y}\%$ Important GATE shortcut.

9. UNDIRECTED DEGREE FROM MATRIX

For simple undirected graph:

TEXT

$\text{degree}(i) = \text{sum of row } i$

because each **1** in the row represents one neighbor.

With self-loops, conventions need care because an undirected self-loop contributes 2 to degree.

For standard simple-graph questions, row sum is enough.

10. ADJACENCY MATRIX SPACE $\mathcal{O}(V^2)$

For V vertices:

Matrix has:

TEXT

$V \times V$

entries.

Therefore:

FORMULA BOX

TEXT

Space $\sim (V^2)$

This does not depend on how many edges actually exist.

$\mathcal{O}(V^2)$ is very important.

11. SPARSE GRAPH PROBLEM

Suppose:

TEXT

$V = 10000$
 $E = 10000$

Graph is sparse.

Matrix still needs:

TEXT

10000^2
 $=$
 $100,000,000$

entries.

A lot of entries are:

TEXT

0

So matrix wastes space for sparse graphs.

12. DENSE GRAPH

If graph has nearly:

TEXT

$\sim (V^2)$

edges, matrix is more reasonable.

So general intuition:

TEXT

Dense graph !' adjacency matrix can be good
Sparse graph !' adjacency list usually better

13. EDGE LOOKUP IN MATRIX $O=V^2$

Question:

Does edge (u, v) exist?

Simply inspect:

TEXT

$M[u][v]$

Therefore:

$O(1)$

$O=V^2$ Major advantage of adjacency matrix.

14. EDGE INSERTION — MATRIX

Set:

TEXT

$M[u][v] = 1$

For undirected:

TEXT

$M[v][u] = 1$

Time:

$O(1)$

15. EDGE DELETION — MATRIX

Set:

TEXT

```
M[u][v] = 0
```

and for undirected:

TEXT

```
M[v][u] = 0
```

Time:

$O(1)$

16. FIND ALL NEIGHBORS — MATRIX

To find neighbors of vertex `u`, scan entire row:

TEXT

```
M[u][0 ... V-1]
```

Therefore:

$\sim(V)$

even if the vertex has only 2 neighbors.

$O(V)$ is one disadvantage for sparse graphs.

17. ADJACENCY LIST

Instead of storing all possible pairs, store only actual neighbors.

Example graph:

TEXT

```
A ----- B
|         |
C ----- D
```

Adjacency list:

TEXT

```
A  !'   B  !'   C
B  !'   A  !'   D
C  !'   A  !'   D
D  !'   B  !'   C
```

Each vertex stores a list of its neighbors.

18. UNDIRECTED GRAPH EDGE STORAGE

In an undirected graph:

```
TEXT
A-B
```

appears in:

```
TEXT
A ' s   l i s t   ! '   B
B ' s   l i s t   ! '   A
```

So every undirected edge is stored:

Twice

19. TOTAL LIST ENTRIES — UNDIRECTED $\emptyset = \hat{Y}\%$

For undirected graph:

```
TEXT
total adjacency entries = 2E
```

because of handshaking:

```
TEXT
£ d e g r e e   =   2 E
```

20. DIRECTED ADJACENCY LIST

For directed graph:

```
TEXT
A   ! '   B
```

normally store B only in:

```
TEXT
A's outgoing adjacency list
```

So total entries:

```
TEXT
E
```

if storing outgoing edges only.

21. ADJACENCY LIST SPACE $\mathcal{O}(V + E)$

Need:

- one list/head for each vertex $\mathcal{O}(V)$
- adjacency entries for edges $\mathcal{O}(E)$

Therefore:

FORMULA BOX

TEXT

$$\text{Space} = \mathcal{O}(V + E)$$

$\mathcal{O}(V)$ MUST KNOW.

22. WHY THIS IS GREAT FOR SPARSE GRAPHS

Suppose:

TEXT

$$\begin{aligned} V &= 10000 \\ E &= 10000 \end{aligned}$$

List storage roughly:

TEXT

$$\begin{array}{c} V + E \\ \text{"H"} \quad 2 \ 0 \ 0 \ 0 \ 0 \end{array}$$

rather than:

TEXT

$$V^2 = 100,000,000$$

matrix entries.

Huge improvement.

23. FIND NEIGHBORS — ADJACENCY LIST

To list all neighbors of vertex u :

Only traverse:

TEXT

$\text{Adj}[u]$

Length:

TEXT

$\text{degree}(u)$

So:

$\sim (\text{deg}(u))$

for undirected graph.

This is much better than $\sim (V)$ when degree is small.

24. EDGE LOOKUP IN ADJACENCY LIST

Question:

Does edge (u, v) exist?

Search through u 's adjacency list.

Worst time:

TEXT

$O(\text{deg}(u))$

which can become:

TEXT

$O(V)$

in worst case.

So matrix has faster edge existence queries.

25. MASTER COMPARISON

| Operation | Matrix | List |
|-----------------------|--------------|-------------------------------------|
| Space | $\sim (V^2)$ | $\sim (V + E)$ |
| Test edge (u, v) | $O(1)$ | $O(\text{deg}(u))$ |
| List neighbors of u | $\sim (V)$ | $\sim (\text{deg}(u))$ |
| Insert edge | $O(1)$ | often $O(1)$ if insert at list head |
| Delete edge | $O(1)$ | $O(\text{deg}(u))$ |
| Good for | dense | sparse |

Copy this table.

26. IMPORTANT IMPLEMENTATION NUANCE

Adjacency-list insertion complexity depends on list representation.

If using linked list and inserting at head:

TEXT

$O(1)$

If you must search to avoid duplicates:

TEXT

$O(\deg(u))$

For GATE, when discussing basic insertion into an existing list without duplicate checking:

TEXT

$O(1)$

is commonly assumed.

27. WEIGHTED GRAPH — MATRIX

Suppose:

TEXT

A --5-- B
A --2-- C

Instead of storing 1, matrix can store weight:

TEXT

| | A | B | C |
|---|---|---|---|
| A | 0 | 5 | 2 |
| B | | | 5 |
| C | | | 2 |

Depending on convention:

TEXT

"

or another sentinel means no edge.

28. WEIGHTED GRAPH — ADJACENCY LIST

Store neighbor plus weight:

TEXT

A !' (B , 5) !' (C , 2)

So each entry contains:

```
TEXT
(destination, weight)
```

This is commonly used by:

- Dijkstra
- Prim
- weighted graph algorithms

29. MATRIX SUM PROPERTY — UNDIRECTED

For simple undirected graph matrix containing 0/1:

Every edge produces two 1s:

```
TEXT
M[u][v] = 1
M[v][u] = 1
```

Therefore:

FORMULA BOX

```
TEXT
Sum of all matrix entries = 2E
```

$\emptyset = \text{None}$ GATE numerical.

30. DIRECTED MATRIX SUM

Each directed edge produces exactly one 1.

Therefore:

```
TEXT
Sum of all entries = E
```

for a 0/1 directed adjacency matrix.

31. EXAMPLE

Undirected adjacency matrix has total number of 1s:

```
TEXT
20
```

Then:

TEXT

$$2E = 20$$

So:

TEXT

$$E = 10$$

32. MATRIX ROW SUM — UNDIRECTED

Example:

TEXT

| | A | B | C | D |
|---|---|---|---|---|
| A | 0 | 1 | 1 | 0 |

Row A sum:

TEXT

2

Therefore:

TEXT

$$\deg(A)=2$$

33. DIRECTED EXAMPLE

Matrix:

TEXT

| | A | B | C |
|---|---|---|---|
| A | 0 | 1 | 1 |
| B | 0 | 0 | 1 |
| C | 1 | 0 | 0 |

Row A sum:

TEXT

2

So:

TEXT

$$\text{outdegree}(A)=2$$

Column A sum:

TEXT

1

So:

TEXT

$\text{indegree}(A)=1$

34. GRAPH TRAVERSAL WITH MATRIX $\mathcal{O}(V^2)$

Suppose BFS/DFS visits one vertex u .

To find all neighbors:

TEXT

scan V matrix entries

If potentially all V vertices are processed:

TEXT

$V \times V$

Therefore:

BFS/DFS with adjacency matrix = $\sim(V^2)$

in standard implementation.

$\mathcal{O}(V^2)$ important.

35. GRAPH TRAVERSAL WITH LIST $\mathcal{O}(V + E)$

Using adjacency list:

Across all vertices:

- each vertex processed once !' $\sim(V)$
- every adjacency entry scanned

For directed graph:

TEXT

E

entries.

For undirected:

TEXT

$2E$

entries.

Both asymptotically:

TEXT

$\sim (E)$

Therefore:

FORMULA BOX

TEXT

$BFS / DFS = \sim (V + E)$

with adjacency list.

~~$O(V^2)$~~ One of the most important graph complexity results.

36. WHY NOT $\sim (VE)$?

Because adjacency lists are not scanned fully for every vertex independently.

Across the entire traversal, the total size of all lists is:

TEXT

$2E$

undirected.

So total neighbor scanning is only:

TEXT

$\sim (E)$

37. EXAMPLE — UNDIRECTED

Graph:

TEXT

```
A -- B
|    |
C -- D
```

Adjacency entries:

TEXT

```
A: B, C
B: A, D
C: A, D
D: B, C
```

Total:

TEXT

$2+2+2+2=8$

Edges:

TEXT

$E=4$

Indeed:

TEXT

$2E=8$

38. BFS/DFS COMPLEXITY FOR CONNECTED GRAPH

If graph is connected and represented by adjacency list:

TEXT

$\sim (V + E)$

If graph is disconnected but algorithm traverses **all components**:

still:

TEXT

$\sim (V + E)$

because each vertex and edge is still processed a constant number of times.

39. IF BFS STARTS FROM ONLY ONE VERTEX

If graph is disconnected, BFS from one source only reaches its connected/reachable component.

Then actual work depends on:

TEXT

vertices and edges reachable from source

But standard full-graph complexity is stated:

TEXT

$O(V+E)$

40. DEGREE CALCULATION COMPARISON

For undirected graph:

Matrix

Degree of u :

TEXT
scan row

Time:

TEXT
 $\sim (V)$

List

If list length is not stored separately:

TEXT
 $\sim (\deg(u))$

If degree counter is maintained:

TEXT
 $O(1)$

to return stored degree.

For theoretical representation comparison, list traversal is usually $\sim (\deg(u))$

41. COMPLETE GRAPH SPACE

For complete graph:

TEXT
 $E = \sim (V^2)$

Then adjacency list space:

TEXT
 $\sim (V + E)$
 $=$
 $\sim (V^2)$

Matrix:

TEXT
 $\sim (V^2)$

So asymptotically they use the same order of space for dense graphs.

42. SPARSE GRAPH

A common sparse-graph condition:

TEXT

$$E = O(V)$$

Then adjacency list:

TEXT

$$\begin{aligned} &\sim (V + E) \\ &= \\ &\sim (V) \end{aligned}$$

while matrix:

TEXT

$$\sim (V^2)$$

Huge asymptotic difference.

$O=V^2$ important.

43. DENSE GRAPH

For dense graph:

TEXT

$$E = \sim (V^2)$$

List:

TEXT

$$\sim (V^2)$$

Matrix:

TEXT

$$\sim (V^2)$$

So matrix can be attractive because edge lookup is $O(1)$ and representation is simple.

44. UNDIRECTED MATRIX STORAGE OPTIMIZATION

Since adjacency matrix is symmetric, theoretically only one triangle needs to be stored:

TEXT

upper triangular
or
lower triangular

This can roughly halve constants.

But asymptotic space remains:

TEXT

$$\sim (V^2)$$

~~Q=6%~~ NOTE may ask asymptotics, so factor 1/2 does not change big-Theta.

45. MAXIMUM MATRIX ENTRIES

Even if an undirected simple graph has at most:

TEXT

$$V(V-1)/2$$

edges, matrix contains:

TEXT

$$V^2$$

positions.

That is why matrix space is:

TEXT

$$\sim (V^2)$$

not:

TEXT

$$\sim (E)$$

46. ADJACENCY LIST DOES NOT REQUIRE SORTED NEIGHBORS

Example:

TEXT

A !' C !' B !' D

could represent the same adjacency as:

TEXT

A !' B !' D !' C

Neighbor ordering depends on insertion/implementation.

~~Q=7%~~ This matters because BFS/DFS traversal order can depend on adjacency-list ordering.

47. VERY IMPORTANT — TRAVERSAL ORDER MAY NOT BE UNIQUE

Suppose:

TEXT

```
  B
 /
A
 \
  C
```

From A, if adjacency list is:

TEXT

A: B, C

BFS may visit:

TEXT

A, B, C

If:

TEXT

A: C, B

BFS may visit:

TEXT

A, C, B

Both valid.

◊=1% GATE asks exact traversal order, inspect neighbor-order assumptions carefully.

48. MATRIX NEIGHBOR ORDER

With adjacency matrix and vertices numbered:

TEXT

0, 1, ..., V-1

if implementation scans row left-to-right, neighbor processing naturally follows vertex index order.

But that is an implementation choice, not a graph property.

49. ADJACENCY LIST C-STYLE STRUCTURE

Conceptual linked-list version:

```
c
struct Node
{
    int vertex;
    struct Node *next;
```

```
};
```

Graph:

```
c
struct Graph
{
    int V;
    struct Node **adj;
};
```

Here:

```
TEXT
adj[i]
```

points to list of neighbors of vertex i.

Ø=Û` Enough C knowledge for GATE; exact implementation details are less important than complexity.

50. ARRAY-OF-LISTS VIEW

Think:

```
TEXT
a d j [ 0 ] !' n e i g h b o r s o f 0
a d j [ 1 ] !' n e i g h b o r s o f 1
a d j [ 2 ] !' n e i g h b o r s o f 2
...
```

This explains space:

```
TEXT
V list heads
+
E/2E adjacency records
```

51. EDGE LIST — EXTRA REPRESENTATIONØ=Û`

Another graph representation is simply:

```
TEXT
(u1,v1)
(u2,v2)
...
```

called an:

Edge List

Space:

TEXT

$\sim (E)$

Useful in algorithms such as Kruskal.

But for BFS/DFS, it is usually inefficient for finding all neighbors because you may need to scan all edges repeatedly.

Your current core focus remains:

- adjacency matrix
- adjacency list

52. INCIDENCE MATRIX — KNOW

Another representation:

Rows may represent vertices and columns represent edges.

For undirected graph:

TEXT

entry = 1

if vertex is incident on edge.

This representation is less common in programming implementations but can appear in graph theory.

Don't know the name, but adjacency matrix/list are much more important for GATE algorithms.

53. GATE TRAP — ADJACENCY MATRIX SPACE FOR SPARSE GRAPH

Even if:

TEXT

$E=V$

matrix space is still:

TEXT

$\sim (V^2)$

It does not shrink with E .

54. GATE TRAP — LIST SPACE UNDIRECTED

Because every edge is stored twice, exact adjacency entries are:

TEXT

$2E$

But:

TEXT

$$\begin{aligned} &\sim (V + 2E) \\ &= \\ &\sim (V + E) \end{aligned}$$

So standard asymptotic space remains:

$$\sim (V + E)$$

55. GATE TRAP — BFS MATRIX COMPLEXITY

Even if graph has only $E=V$ edges, matrix BFS can still take:

TEXT

$$\sim (V^2)$$

because finding neighbors requires scanning rows of length V .

56. GATE TRAP — EDGE LOOKUP LIST

Adjacency list does not usually provide guaranteed:

TEXT

$$O(1)$$

edge lookup.

You may need to search neighbor list:

TEXT

$$O(\deg(u))$$

unless an additional hash/set structure is used.

For standard adjacency list, use degree-based search.

57. GATE TRAP — UNDIRECTED EDGE COUNT FROM LIST

Suppose total adjacency-list entries:

TEXT

$$40$$

for simple undirected graph.

Then:

TEXT
2E=40

So:

TEXT
E=20

$\frac{0}{2} = 0\%$ common numerical.

58. DIRECTED EDGE COUNT FROM LIST

If outgoing adjacency lists contain a total of:

TEXT
40

entries:

TEXT
E=40

because each directed edge appears once.

59. MATRIX DEGREE NUMERICAL

Undirected matrix row:

TEXT
0 1 0 1 1 0

Degree:

TEXT
3

because there are three incident neighbor entries.

60. DIRECTED NUMERICAL

Matrix:

TEXT

| | 0 | 1 | 2 | 3 |
|---|---|---|---|---|
| 0 | 0 | 1 | 1 | 0 |
| 1 | 0 | 0 | 1 | 1 |
| 2 | 1 | 0 | 0 | 0 |
| 3 | 0 | 0 | 1 | 0 |

For vertex 2:

Row 2:

TEXT
1 0 0 0

Outdegree:

TEXT
1

Column 2:

TEXT
1
1
0
1

Indegree:

TEXT
3

Easy GATE-style question.

61. MATRIX SYMMETRY TEST

If a 0/1 adjacency matrix for a claimed simple undirected graph is not symmetric:

TEXT
 $M[i][j] \neq M[j][i]$

then it cannot represent that undirected graph under standard adjacency-matrix definition.

Quick test.

62. DIAGONAL TEST

For simple graph:

TEXT
 $M[i][i] = 0$

If diagonal contains 1:

TEXT
self-loop exists

so graph is not simple.

63. COMPLETE GRAPH MATRIX

For K_n :

Adjacency matrix:

- diagonal = 0
- every off-diagonal entry = 1

Total 1s:

TEXT
 $n(n-1)$

Since each undirected edge counted twice:

TEXT
 $E = n(n-1)/2$

64. COMPLETE DIRECTED GRAPH WITHOUT LOOPS

Adjacency matrix again has:

TEXT
0 on diagonal
1 everywhere else

But now each 1 is a distinct directed edge.

So:

TEXT
 $E = n(n-1)$

Same matrix pattern can represent different edge interpretation depending on directed/undirected context.

Important nuance.

65. GRAPH DENSITY AND REPRESENTATION CHOICE

Exam shortcut:

If:

TEXT
 $E \ll V^2$

choose:

TEXT

Adjacency List

If:

TEXT

$E \gg V^2$

matrix may be reasonable.

But if question prioritizes constant-time edge lookup, matrix can still be chosen even before considering density.

66. WHICH REPRESENTATION FOR BFS/DFS?

Normally:

Adjacency List

because traversal becomes:

TEXT

$\sim (V + E)$

This is optimal for inspecting graph connectivity in sparse graphs.

67. WHICH FOR $O(1)$ EDGE TEST?

Adjacency Matrix

because:

TEXT

$M[u][v]$

is direct array access.

68. WHICH FOR VERY DENSE GRAPH?

Often:

Adjacency Matrix

because its $\sim V^2$ storage is no worse asymptotically than list when $E = \sim V^2$ (and edge testing is faster).

69. WHICH FOR VERY SPARSE GRAPH?

Adjacency List

because:

TEXT

$\sim (V + E)$

can be nearly linear.

70. MASTER GATE FORMULA BOX

Adjacency Matrix

TEXT

Space = $\sim (V^2)$

TEXT

Edge lookup = $O(1)$

TEXT

Neighbors of u = $\sim (V)$

TEXT

BFS / DFS = $\sim (V^2)$

Adjacency List

TEXT

Space = $\sim (V + E)$

TEXT

Edge lookup = $O(\deg(u))$

TEXT

Neighbors of u = $\sim (\deg(u))$

TEXT

BFS / DFS = $\sim (V + E)$

Copy exactly.

71. MEMORY TRICK

MATRIX

Pay for every possible connection.

So:

TEXT

V^2 space

Benefit:

TEXT

edge check instantly

72. LIST

Pay only for connections that exist.

So:

TEXT

$V+E$ space

Benefit:

TEXT

efficient neighbor traversal

73. 30-SECOND REVISION

Remember:

1. Matrix = $V \times V$.
2. Matrix space = $\sim (V^2)$
3. Undirected matrix is symmetric.
4. Matrix edge lookup = $O(1)$.
5. Finding neighbors from matrix = $\sim (V)$
6. List space = $\sim (V + E)$
7. Undirected adjacency list stores each edge twice.
8. List neighbor traversal = $\sim (deg(v))$
9. BFS/DFS list = $\sim (V + E)$
10. BFS/DFS matrix = $\sim (V^2)$

74. ACTIVE RECALL

Q1

Which representation uses $\sim (V^2)$ space?

Q2

Which uses $\sim (V + E)$ space?

Q3

In an undirected list, how many total adjacency entries?

Q4

Matrix edge lookup complexity?

Q5

List edge lookup complexity?

Q6

BFS with adjacency list?

ANSWERS

A1

Adjacency matrix.

A2

Adjacency list.

A3

TEXT

$2E$

A4

TEXT

$O(1)$

A5

TEXT

$O(\deg(u))$

worst $O(V)$.

A6

TEXT

$\sim (V + E)$

75. GATE PRACTICE 1

An undirected graph has:

TEXT

V=1000
E=2000

Compare asymptotic storage for:

1. adjacency matrix
2. adjacency list

Which is more suitable?

76. GATE PRACTICE 2

An undirected adjacency matrix has:

TEXT

36 ones

How many edges?

Answer:

TEXT

18

77. GATE PRACTICE 3

A directed adjacency matrix has:

TEXT

36 ones

How many directed edges?

Answer:

TEXT

36

assuming 0/1 entries and each directed edge represented once.

78. GATE PRACTICE 4

For a directed graph, row v contains 5 ones and column v contains 3 ones.

Find:

TEXT

outdegree(v)
indegree(v)

Answer:

TEXT

outdegree = 5
indegree = 3

79. GATE PRACTICE 5

Why does BFS on an adjacency matrix take $\sim (V^2)$ for a sparse graph?

Answer:

Because for each visited vertex, its row of V possible neighbors may need to be scanned.

80. MODULE 9 PROGRESS

' Lesson 72 — Graph Fundamentals ' Lesson 73 — Graph Representation

Next:

LESSON 74 — Breadth-First Search (BFS)

Queue, Level-by-Level Traversal, Shortest Paths in Unweighted Graphs & GATE Complexity

We will cover:

- BFS intuition
- queue logic
- visited array
- exact traversal trace
- BFS tree
- distances/levels
- why BFS gives shortest path in unweighted graphs
- disconnected graphs
- $\sim (V^2)$ vs $\sim (V)$
- BFS applications
- major GATE traps.

USER

still how many were there

ASSISTANT

For the **Graphs module**, we originally planned **7 lessons total (72–78)**.

You have completed:

- ' 72 — Graph Fundamentals
- ' 73 — Graph Representation

So **5 Graph lessons are still remaining**:

1. 74 — BFS
2. 75 — DFS
3. 76 — BFS vs DFS + traversal properties
4. 77 — Connected components, cycle detection, bipartite checking and related GATE problems
5. 78 — Topological Sort / DAG fundamentals

After Graphs, we still have the **final GATE consolidation and syllabus audit**, because you specifically asked me **not to miss anything**. I'll use that stage to check every official syllabus item and fill any remaining gaps before saying the subject is complete.

So right now: **5 graph lessons remaining**.